IC Compiler™ II Implementation User Guide

Version X-2025.06-SP1, August 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

New in This Release 28
Related Products, Publications, and Trademarks 28
Conventions 28
Customer Support 29
Statement on Inclusivity and Diversity 30

1. **Working With the IC Compiler II Tool** 31
Place and Route Design Flow Overview 32
IC Compiler II Concepts 34
Power Intent Concepts 34
UPF Flows 36
Multiple-Patterning Concepts 37
Mask Constraints 40
Working With the 3DIC Compiler User Interfaces 40
Starting the CLI 41
Exiting the IC Compiler II Tool 42
Entering icc2_shell Commands 42
Interrupting or Terminating Command Processing 43
Getting Information About Commands 43
Displaying Command Help 44
Using Application Options 44
Using Variables 45
Viewing Man Pages 45
Using Tcl Scripts 46
Adding Changes to a Script With Checkpoints 47
Defining Checkpoints 49
Configuring Checkpoints 51
Defining Checkpoint Behaviors 51
Associating Checkpoints and Checkpoint Behaviors 53
Querying Checkpoints and Checkpoint Behaviors 57
Viewing Your Checkpoint History 58
Using Setup Files 59

Using the Command Log File 60
Enabling Multicore Processing 60
Configuring Multithreading 61
Configuring Distributed Processing 62
Reporting Multicore Configurations 63
Removing Multicore Configurations 63
Running Tasks in Parallel 64
Running Commands in Parallel on Your Local Host 64
Running Commands in the Background 64
Running Commands in Parallel 66

2. Preparing the Design 67
Defining the Search Path 68
Setting Up Libraries 68
Working With Design Libraries 68
Setting Up Reference Libraries 70
Library Configuration 71
Restricting Library Cell Usage 72
Restricting the Target Libraries Used 73
Using Pin Access Checker Utility 75
Working With Designs 76
Importing the Floorplan Information 77
Reading DEF Files 78
Fixing Site Name Mismatches 80
Validating DEF Files 80
Physical Constraints Extracted From the DEF File 80
Setting Up Multivoltage Designs 84
Applying the Multivoltage Power Intent 84
Loading and Applying UPF Information 85
Specifying UPF Constraints for Physical-Only Cells 86
Saving UPF Information 86
Preparing the Power Network 87
Creating Logical Power and Ground Connections 87
Creating Floating Logical Supply Nets 89
Defining Voltage Areas 90
Merging Voltage Area Shapes 91
Resolving Overlapping Voltage Areas 92
Modifying the Stacking Order 94

Defining Guard Bands 94
Defining Gas Stations 96
Querying Voltage Areas 96
Modifying Voltage Areas 97
Controlling Physical-Feedthrough Nets in Voltage Areas 97
Removing Voltage Areas 100
Inserting Multivoltage Cells 101
Inserting Level Shifters 101
Inserting Isolation Cells 101
Associating Power Strategies With Existing Multivoltage Cells 102
Controlling the Placement of Multivoltage Cells 102
Enabling Improved Buffering for Multivoltage Nets 102
Analyzing Multivoltage Information 102
Specifying Timing Constraints and Settings 103
Specifying Logical Design Rule Constraints 104
Controlling Clock-Gate Latencies 105
Integrated Clock-Gate Latency Estimation 106
User-Specified Clock-Gate Latency 107
Specifying Physical Constraints for Placement and Legalization 111
Defining Keepout Margins 111
Defining an Outer Keepout Margin 112
Defining an Inner Keepout Margin 112
Defining Area-Based Placement Blockages 113
Defining a Hard Placement Blockage 114
Defining a Hard Macro Placement Blockage 114
Defining a Soft Placement Blockage 115
Defining a Partial Placement Blockage 115
Define a Blockage of a Predefined Category 116
Defining Blockages That Exclude Registers 117
Defining Blockages That Exclude Relative Placement Groups 117
Defining Blockages That Allow Relative Placement Cells Only 117
Defining Blockages That Allow Buffers Only 118
Querying Placement Blockages 118
Removing Placement Blockages 118
Defining Placement Bounds 118
Defining Move Bounds 120
Defining Group Bounds 121
Querying Placement Bounds 122
Removing Placement Bounds 122
Defining Placement Attractions 122
Defining Cell Spacing Constraints for Legalization 124

Reporting Cell Spacing Constraints 126
Removing Cell Spacing Constraints 126
Specifying Placement Settings 126
Performing Placement With Inaccurate Constraints at Early Stages 127
Generating Automatic Group Bounds for Clock Gating Cells 127
Controlling the Placement Density 127
Controlling Congestion-Driven Restructuring During Placement 129
Reducing Congestion 129
Considering Wide Cell Density During Placement 130
Considering the Effects of Cell Pins During Placement 130
Considering the Congestion Effects Due to the Nondefault Routing Rules of Clock Nets 131
Considering the Effects of Clock Gating Cells of Sequential Arrays During Placement 131
Considering Legalization Effects During Placement 131
Considering DFT Connections During Placement 131
Considering the Dynamic Power QoR During Placement 132
Performing IR-Drop-Aware Placement 132
Controlling IR-Drop-Aware Placement 133
Spreading Repeater Cells During Placement 135
Specifying Legalization Settings 135
Minimizing Large Displacements During Legalization 136
Optimizing Pin Access During Legalization 136
Enabling Concurrent Legalize and Detail Rroute 136
Enabling Advanced PG Net Checks 136
Enabling Advanced Legalization Algorithms 137
Setting Up for Variant-Aware Legalization 138
Defining Equivalent Cell Groups 140
Enabling Variant-Aware Legalization 140
Checking if Library Cells Are Legally Placeable 141
Controlling the Optimization of Cells, Nets, Pins, and Ports 142
Preserving Cells and Nets During Optimization 142
Restricting Optimization to Cell Sizing Only 144
Preserving Networks During Optimization 144
Marking the Clock Networks 145
Disabling Design Rule Checking (DRC) 145
Preserving Pin Names During Sizing 147
Preserving Ports of Existing Hierarchies 148
Isolating Input and Output Ports 148

Fixing Multiple-Port Nets 149
Controlling the Addition of New Cells to Modules, Hierarchical Cells, and Voltage Areas 150
Specifying a Cell Name Prefix for Optimization 151
Specifying Settings for Preroute Optimization 151
Controlling DFT Optimization for the place_opt Command 152
Specifying Parasitic Estimation Settings for the Preroute Optimization 153
Enabling Global-Route-Layer-Based (GRLB) Preroute Optimization 153
Enabling Route-Driven Estimation (RDE) for Preroute Optimization 154
Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization 154
Specifying Via Ladder Candidates for Library Pins 155
Enabling Area Recovery in Regions of High Utilization 156
Enabling Advanced Logic Restructuring 156
Setting Up for Power-Related Features 157
Annotating the Switching Activity 158
Using RTL Switching Activity With a Name-Mapping File 158
Scaling the Switching Activity 160
Specifying Switching Probability for Supply Nets 161
Enabling Power Optimization for the place_opt and clock_opt Commands . . . 162
Performing Conventional Leakage-Power Optimization 163
Performing Dynamic-Power Optimization 163
Performing Total-Power Optimization 163
Improving Yield By Limiting the Percentage of Low-Threshold-Voltage (LVT) Cells 164
Updating Activity for Improved Power Optimization 165
Enabling the Power Integrity Features 166
Setting Up for Dynamic Power Shaping 167
Setting Up for Voltage-Drop-Aware Placement 168
Manually Enabling Dynamic Power Shaping and Voltage-Drop-Aware Placement 169
Specifying the Routing Resources 169
Specifying the Global Layer Constraints 170
Reporting Global Layer Constraints 171
Removing Global Layer Constraints 171
Specifying Net-Specific Layer Constraints 172
Removing Net-Specific Routing Layer Constraints 173
Specifying Clock-Tree Layer Constraints 173
Enabling Level-Based Clock Nondefault Routing Rule 175
Setting the Preferred Routing Direction for Layers 177

Handling Design Data Using the Early Data Check Manager 177
Applying Mega-Switch Command Settings 179
Applying Required Settings for Advanced Technology Nodes 179
Applying Required Settings for High Performance Cores 180
Applying Required Settings for Improving Specific QoR Metrics 180

3. Placement and Optimization 182
Performing Placement and Optimization 182
Performing Standalone Placement and Legalization 183
Performing Placement and Optimization With the place_opt Command 184
Creating a Temporary Clock Tree for Placement and Optimization 185
Optimizing Clock-Gating Cells 185
Using Accurate Latencies for Clock Gates 186
Enabling Global Route Based High-Fanout Synthesis 187
Enabling the Rebuilding of Buffer Trees 187
Changing the Congestion Effort 187
Using of Nondefault Routing Rules for Critical Nets 188
Enabling Path Optimization 188
Performing IR-Drop-Aware Placement During the place_opt Command . . 188
Performing Concurrent Clock and Data Optimization During the place_opt Command 189
Using Physical Guidance From the Design Compiler Tool 190
Performing Multibit Register Optimization 192
Performing Integrated Multibit Register Optimization 193
Performing Multibit Register Optimization Using Discrete Commands . . . 194
Banking Multibit Retention Registers 197
Performing Magnet Placement 197
Refining Placement 199
Performing Placement and Optimization on Multivoltage Blocks 200
Rebuilding Buffer Trees 200
Adding and Removing Tie Cells 201
Identifying Issues That Cannot Be Fixed During Optimization 201
Analyzing the Bufferability of Nets 202
Analyzing Violations That Cannot Be Fixed 202
Analyzing the Placement 203
Reporting Utilization 204
Reporting the Placement QoR 204
Querying and Changing the Placement Status 206
Analyzing the Placement in the GUI 207

Analyzing Timing 208
Analyzing Power 208
Creating Power Groups for Reporting 209
Reporting Pin-Based Clock Network Power 210
Comparing QoR Data 211
Setting Your Baseline Run 213
Changing the QoR Display Style 215
Sorting and Filtering the Data 216
Sorting the Data 217
Filtering Metrics 218
Filtering Runs 218
Example Analysis 219
Exploring the Detailed Comparison Data 221
Filtering the Detailed Data 221

4. Clock Tree Synthesis 223
Prerequisites for Clock Tree Synthesis 223
Defining the Clock Trees 224
Deriving the Clock Trees 225
Identifying the Clock Roots 225
Identifying the Clock Endpoints 227
Defining Clock Tree Exceptions 230
Defining Sink Pins 230
Defining Insertion Delay Requirements 231
Defining Ignore Pins 232
Ensuring Clock Tree Exceptions are Valid 233
Restricting Optimization on the Clock Network 234
Setting Don’t Touch Settings 234
Setting Size-Only Settings 235
Copying Clock Tree Exceptions Across Modes 235
Deriving Clock Tree Exceptions From Ideal Clock Latencies 236
Handling Endpoints With Balancing Conflicts 237
Enabling Solver-Augmented Skew Optimization 239
Verifying the Clock Trees 240
Setting Clock Tree Design Rule Constraints 242
Specifying the Clock Tree Synthesis Settings 243
Specifying the Clock Tree References 244
Deriving Clock Tree References for Preexisting Gates 245

Restricting the Target Libraries Used 245
Setting Skew and Latency Targets 246
Enabling Local Skew Optimization 246
Specifying the Primary Corner for Clock Tree Synthesis 247
Preventing Specific Clocks From Being Synthesized 247
Preserving Preexisting Clock Trees 247
Enabling Clock Tree Power Reduction Techniques 247
Enabling Wire Length Aware Power-Driven Relocation 248
Reducing Electromigration 248
Handling Inaccurate Constraints During Clock Tree Synthesis 249
Defining Clock Cell Spacing Rules 250
Creating Skew Groups 251
Defining a Name Prefix for Clock Cells 251
Using the Global Router During Initial Clock Tree Synthesis 252
Specifying Constraints for Clock Nets 252
Constructing Faster Clock Tree to Specific Endpoints 252
Improving Latency-Driven Clock-Gate Splitting 254
Improving the Repeater Levels in Clock Tree 254
Enabling Latency-Driven Clock-Gate Collapsing Levels 254
Reducing Signal Integrity Effects on Clock Nets 255
Specifying Settings for Clock Latency Adjustments 256
Timing Driven Sink Transition Target 256
Reporting the Clock Tree Settings 256
Pre-CTS Latency Bottleneck Reporting 257
Reporting Auto Exceptions and Summary Tables 258
Dynamic Clock Power Improvement for Structural Multisource Clock Tree Synthesis 259
Enabling Target Latency for Skew Groups and ICDB Improvement 259
Debugging and Usability Improvement Using the check_clock_trees Command and CTS Log File 260
Clock Routing Improvement Using Intelligent Minimum Layer Handling of Clock Nets 261
Implementing Clock Trees and Performing Post-CTS Optimization 262
Performing Standalone Clock Trees Synthesis 262
Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command 263
Considering Voltage Drop Information During Clock Tree Synthesis 264
Using Nondefault Routing Rules for Critical Nets During Optimization . . . 265
Performing Concurrent Clock and Data Optimization During the clock_opt Command 265

Controlling Multibit Optimization Performed During the clock_opt Command . . . 266
Performing Power or Area Recovery on the Clock Network . . . 266
Performing IR-Drop-Aware Placement During the clock_opt Command . . 267
Clock Power Recovery . . . 268
Controlling Concurrent Clock and Data Optimization . . . 268
Limiting the Latency Adjustment Values . . . 269
Excluding Boundary Paths . . . 269
Excluding Specific Path Groups . . . 270
Excluding Specific Scenarios . . . 270
Excluding Specific Sinks . . . 271
Controlling Timing Optimization Effort . . . 271
Controlling Hold Time Optimization Effort . . . 271
Controlling the Adjustment of I/O Clock Latencies . . . 272
Performing Dynamic-Voltage-Drop-Driven Concurrent Clock and Data Optimization During the route_opt Command . . . 272
Specifying Optimization Targets at the Preroute Stage . . . 273
Specifying Optimization Targets at the Postroute Stage . . . 274
Enabling Buffer Removal at the Postroute Stage . . . 275
Reporting Concurrent Clock and Data Timing . . . 276
Scaling CCD Offsets to New Scenarios for Reporting Timing . . . 278
Skewing Latch and Discrete Clock Gates . . . 278
Optimizing Power Using Data Path Aware CUS . . . 278
Optimizing Timing Using Data Path Aware Clock Useful Skew . . . 279
Incremental Clock-Data Optimization . . . 280
Timer-Based Predictive Concurrent Clock-Data Optimization . . . 280
Splitting Clock Cells . . . 281
Balancing Skew Between Different Clock Trees . . . 281
Defining the Interclock Delay Balancing Constraints . . . 282
Generating Interclock Delay Balancing Constraints Automatically . . . 283
Running Interclock Delay Balancing . . . 284
Performing Global-Route-Based Optimization Using Machine Learning Data . 284
Routing Clock Trees . . . 286
Inserting Via Ladders During Clock Tree Synthesis, Optimization, and Clock Routing . . . 287
Marking Clocks as Propagated After Clock Tree Synthesis . . . 287
Performing Postroute Clock Tree Optimization . . . 288
Performing Voltage Optimization . . . 288
Marking Clock Trees as Synthesized . . . 290
Removing Clock Trees . . . 290
Optimizing Nworst Skew Framework for Multithreaded Clock Tree . . . 292
Implementing Multisource Clock Trees . . . 292

Introduction to Multisource Clock Trees Structures 293
Implementing a Regular Multisource Clock Tree 295
Implementing a Regular Multisource Clock Tree Using Integrated Tap Assignment 296
Implementing a Regular Multisource Clock Tree With an H-Tree-Only Global Clock Tree Structure 298
Implementing MSCTS Multiple H-Tree 300
Analyzing the H-Tree Quality of Results 301
Enabling Timing-Aware Latency Implementation in Structural Multisource Clock Tree Synthesis 302
Implementing Power Aware Flexible H-Tree 303
Implementing a Structural Multisource Clock Tree 303
H-Tree Support in Physical Hierarchy Flat-First Section Designs 305
Implementing a Structural Multisource Clock Tree Using Integrated Subtree Synthesis 307
Improving the Correlation Between the SMSCTS and CCD 309
Early CCD-iSMSCTS Flow for Structural Multisource Clock Tree Synthesis . . 310
Configuring the Compile initial_opto Stage 311
Configuring the Compile final_place Stage 311
Configuring the Compile final_opto Stage 312
Configuring the clock_opt build_clock Stage 312
Inserting Clock Drivers 312
Inserting Clock Drivers for Designs With Multiple Levels of Physical Hierarchy 315
Synthesizing the Global Clock Trees 316
Inserting Clock Drivers for Designs With Multiple Levels of Physical Hierarchy 317
Creating Clock Straps 318
Routing to Clock Straps 321
Analyzing the Clock Mesh 325
Performing Automated Tap Insertion and H-Tree Synthesis 327
Specifying Tap Assignment Options and Settings 329
Building the Local Clock Subtree Structures 331
Performing Irregular Global Tree Synthesis 334
Analyzing the Clock Tree Results 335
Generating Clock Tree QoR Reports 335
Reporting Clock Tree Power 337
Creating Collections of Clock Network Pins 338
Analyzing Clock Timing 339
Analyzing Clock Trees in the GUI 339

5. Routing and Postroute Optimization 340

Introduction to Zroute 341

Basic Zroute Flow 343

Prerequisites for Routing 344

Defining Vias 345

Reading Via Definitions from a LEF File 346

Creating a Via Definition 346

Defining Simple Vias 346

Defining Custom Vias 347

Inserting Via Ladders 348

Defining Via Ladder Rules 349

Generating Via Ladder Rules for Electromigration Via Ladders 351

Generating Via Ladder Rules for Performance Via Ladders 353

Via Ladder Rule Files 355

Via Ladder Association File 355

Constraint-Based Via Ladder Insertion 356

Defining Via Ladder Constraints 357

Defining Global Via Ladder Constraints 357

Defining Instance-Specific Via Ladder Constraints 358

Inserting Via Ladders 359

Protecting Via Ladders 361

Verifying Via Ladders 362

Updating Via Ladders 363

Manual Via Ladder Insertion 365

Querying Via Ladders 366

Removing Via Ladders 366

Checking Routability 366

Routing Constraints 372

Defining Routing Blockages 374

Reserving Space for Top-Level Routing 376

Querying Routing Blockages 376

Removing Routing Blockages 376

Defining Routing Guides 377

Using Routing Guides to Control the Routing Direction 378

Using Routing Guides to Limit Edges in the Nonpreferred Direction 379

Using Routing Guides to Control the Routing Density 380

Using Routing Guides to Prioritize Routing Regions 380

Using Routing Guides to Encourage River Routing 381

Querying Routing Guides 382
Removing Routing Guides 382
Deriving Routing Guides 382
Deriving Pin Access Routing Guides 383
Deriving Metal Cut Routing Guides 384
Controlling Routing Around the Block Boundary 384
Inserting Metal Shapes in the Preferred Direction 385
Inserting Routing Guides Along the Nonpreferred-Direction Edges 389
Inserting Routing Blockages Along the Boundary Edges 391
Removing Perimeter Constraint Objects 393
Routing Nets Within a Specific Region 393
Defining Routing Corridors 394
Assigning Nets to a Routing Corridor 395
Verifying Routing Corridors 396
Modifying Routing Corridors 396
Reporting Routing Corridors 397
Removing Routing Corridors 397
Using Nondefault Routing Rules 397
Defining Nondefault Routing Rules 398
Assigning Nondefault Routing Rules to Nets 409
Controlling Off-Grid Routing 413
Preventing Off-Grid Routing 413
Discouraging Off-Grid Routing for Vias 415
Routing Must-Join Pins 415
Controlling Pin Connections 417
Controlling Pin Tapering 420
Specifying the Tapering Method 421
Controlling the Tapering Width 422
Controlling Via Ladder Connections 422
Setting the Rerouting Mode 423
Routing Application Options 423
Routing Clock Nets 424
Routing Critical Nets 425
Routing Secondary Power and Ground Pins 426
Verifying the Secondary Power and Ground Pin Attributes 426
Setting the Routing Constraints 427
Routing the Secondary Power and Ground Pins 428
Routing Signal Nets 429
Global Routing 430
Global Routing During Design Planning 433

Timing-Driven Global Routing 433
Crosstalk-Driven Global Routing 435
Incremental Global Routing 435
Track Assignment 435
Detail Routing 437
Routing Signal Nets by Using Automatic Routing 441
Shielding Nets 443
Defining the Shielding Rules 444
Performing Preroute Shielding 445
Soft Shielding Rules During Signal Routing 448
Performing Postroute Shielding 449
Shielding Example 449
Performing Incremental Shielding 450
Reporting Shielding Information 450
Querying Shield Shapes 450
Reporting Shielding Statistics 451
Performing Shielding Checks 452
Performing Postroute Optimization 452
Performing Postroute Logic Optimization 453
Performing Postroute Optimization Using the hyper_route_opt Command . . . 455
Fixing DRC Violations Caused by Pin Access Issues 456
Analyzing and Fixing Signal Electromigration Violations 458
Loading the Signal Electromigration Constraints 459
Analyzing Signal Electromigration 459
Fixing Signal Electromigration Violations 461
Performing ECO Routing 462
Routing Nets in the GUI 464
Modifying Routed Nets 465
Cleaning Up Routed Nets 467
Analyzing the Routing Results 468
Generating a Congestion Report 468
Generating a Congestion Map 470
Performing Design Rule Checking Using Zroute 473
Performing Signoff Design Rule Checking 476
Performing Design Rule Checking in an External Tool 477
Performing Layout-Versus-Schematic Checking 477
Reporting the Routing Results 480
Reporting the Wire Length 480

Using the DRC Query Commands 481
Saving Route Information 482
Deriving Mask Colors 482
Inserting and Removing Cut Metal Shapes 483

6. Chip Finishing and Design for Manufacturing 485

Inserting Tap Cells 485
Using the create_tap_cells Command 486
Using the create_cell_array Command 490
Inserting Exterior Tap Walls 492
Inserting Interior Tap Walls 493
Inserting Tap Meshes 495
Inserting Dense Tap Arrays 496
Performing Boundary Cell Insertion 497
Specifying the Boundary Cell Insertion Requirements 497
Specifying the Library Cells for Boundary Cell Insertion 498
Specifying Boundary Cell Placement Rules 500
Specifying the Naming Convention for Boundary Cells 501
Creating Routing Guides During Boundary Cell Insertion 501
Creating Placement Blockages During Boundary Cell Insertion 502
Reporting the Boundary Cell Insertion Requirements 502
Removing Boundary Cell Insertion Requirements 503
Inserting Boundary Cells 503
Verifying the Boundary Cell Placement 503
Finding and Fixing Antenna Violations 504
Defining Antenna Rules 505
Calculating the Maximum Antenna Ratio 505
Calculating the Antenna Ratio for a Pin 517
Specifying Antenna Properties 520
Analyzing and Fixing Antenna Violations 522
Inserting Diodes During Detail Routing 523
Inserting Diodes After Detail Routing 525
Inserting Redundant Vias 525
Inserting Redundant Vias on Clock Nets 526
Inserting Redundant Vias on Signal Nets 527
Viewing the Default Via Mapping Table 528
Defining a Customized Via Mapping Table 529
Postroute Redundant Via Insertion 530

Concurrent Soft-Rule-Based Redundant Via Insertion 531
Near 100 Percent Redundant Via Insertion 533
Preserving Timing During Redundant Via Insertion 534
Reporting Redundant Via Rates 534
Optimizing Wire Length and Via Count 536
Reducing Critical Areas 536
Performing Wire Spreading 537
Performing Wire Widening 538
Inserting Metal-Insulator-Metal Capacitors 539
Inserting Filler Cells 540
Standard Filler Cell Insertion 541
Controlling Standard Filler Cell Insertion 543
Checking for Filler Cell DRC Violations 546
Fixing Remaining Mask Design Rule Violations 547
Abutting Standard Cells With Specific Filler Cells 559
Threshold-Voltage-Based Filler Cell Insertion 561
Controlling Threshold-Voltage-Based Filler Cell Insertion 562
Removing the Threshold-Voltage Filler Cell Information 563
Removing Filler Cells 563
Inserting Metal Fill 563

7. IC Validator In-Design 564
Preparing to Run IC Validator In-Design Commands 565
Setting Up the IC Validator Environment 565
Enabling IC Validator Multicore Processing 565
Running IC Validator on Specific Hosts 566
Running IC Validator Using a Job Scheduler 567
Running IC Validator Using Hybrid Multicore Processing 568
Defining the Layer Mapping for IC Validator In-Design Commands 568
Performing Signoff Design Rule Checking 569
Running the signoff_check_drc Command 569
Setting Options for Signoff Design Rule Checking 571
Reading Blocks for Signoff Design Rule Checking 573
Signoff Design Rule Checking 574
Signoff DRC Results Files 576
Viewing the Violations in an ICV Heat Map 577
Configuring an ICV Heat Map 580
Highlighting Violations From the Error Browser Onto an ICV Heat Map . . 586
Automatically Fixing Signoff DRC Violations 588

Creating an Autofix Configuration File 588
Setting Options for Signoff DRC Fixing 589
Running the signoff_fix_drc Command 591
Automatically Fixing Double-Patterning Odd-Cycle Violations 595
Summary Report for Automatic Design Rule Fixing 595
Checking Signoff Design Rules Interactively in the GUI 597
Displaying Objects for Design Rule Checking 598
DRC Toolbar 599
Setting Options for Interactive Design Rule Checking 602
Improving Instance Voltage Drop by Augmenting the Power Grid 604
Standard Power Grid Augmentation 605
Setting Options for Power Grid Augmentation 607
Timing-Driven Power Grid Augmentation 608
Guided Power Grid Augmentation 610
Removing PG Augmentation Shapes 611
ECO-Aware Removal of PG Augmentation Shapes 611
Performing Timing-Aware Trim PG Augmentation 612
Trim Approach for Optimal QoR and Timing Recovery 613
PG Augmentation Advanced User Mode for Improved Ease of Use 616
Automating Transition Metal and Via Width Using Power Grid Augmentation . 617
Inserting Metal Fill With IC Validator In-Design 618
Setting Options for Signoff Metal Fill Insertion 619
Performing Metal Fill Insertion 622
Pattern-Based Metal Fill Insertion 622
Track-Based Metal Fill Insertion 627
Using an IC Validator Parameter File 630
Typical Critical Dimension Metal Fill Insertion 635
Timing-Driven Metal Fill Insertion 635
Incremental Metal Fill Insertion 640
Foundry Fill and Track Fill Combined for Different Flows of Foundry 642
Signoff Metal Fill Result Files 643
Querying Metal Fill 643
Viewing Metal Fill in the GUI 644
Reporting the Metal Density 645
Viewing Density Heat Maps in the GUI 648
Removing Metal Fill 650
Removing Metal Fill With the IC Validator Tool 651
ECO-Aware Removal of Track Based Metal Fill Shapes 652
Modifying Metal Fill 652
Performing Real Metal Fill Extraction 652

Automatically Fixing Isolated Vias 653
Setting Options for Fixing Isolated Vias 653
Running the signoff_fix_isolated_via Command 654
Checking for Isolated Vias 654
Checking and Fixing Isolated Vias 655

8. Routing Using Custom Router 657

Using Custom Router in the IC Compiler II Tool 658

Before Using Custom Router 659

Defining Routing Constraints 660
Defining the Bus Routing Style 661
Creating Differential Groups 663
Inserting Shields on the Nets 664
Defining the Net Priority 666
Defining Minimum Wire Lengths 666
Defining Matching Wire Lengths 669

Managing Constraint Groups 674

Using Custom Routing Application Options 675
Bus Routing Options 677
Corner Type 677
Intra-shield Placement 678
Pin-Trunk Offset 679
Trunk Splitting 679
Tapoffs 680
Track Adherence Options 680
Differential-Pair Options 681
Shielding Options 681
Single-Loop Matching 682
Balance Mode Option 683
Voltage Area Aware Routing in Custom Router 684

Routing With the Custom Router 685

Shielding the Nets 686

Checking the Routing Results 686

Using a Hybrid Routing Flow 686

Using a DDR Routing Flow 687

9. Physical Datapath With Relative Placement 692

Introduction to Physical Datapath With Relative Placement 692

Benefits of Relative Placement 693

Relative Placement Flow 694

Creating Relative Placement Groups 694

Adding Objects to a Group 696

Adding Leaf Cells 696

Specifying Orientations for Leaf Cells 697

Adding Hard Macro Cells 697

Adding Relative Placement Groups 697

Creating Hierarchical Relative Placement Groups 698

Using Hierarchical Relative Placement for Straddling 699

Using Hierarchical Relative Placement for Compression 701

Adding Blockages 702

Adding Cells Within a Predefined Relative Placement Area 703

Specifying Options for Relative Placement Groups 703

Anchoring Relative Placement Groups 704

Aligning Leaf Cells Within a Column 707

Aligning by the Left Edges 707

Aligning by the Right Edges 707

Aligning by Pin Location 708

Overriding the Alignment When Adding Objects 710

Controlling the Tiling Within Relative Placement Groups 711

Applying Compression to Groups With Straddling Leaf Cells 712

Specifying the Orientation of Relative Placement Groups 713

Specifying a Keepout Margin 715

Handling Fixed Cells During Relative Placement 715

Allowing Nonrelative Placement Cells 716

Controlling the Optimization of Relative Placement Cells 716

Controlling Movement When Legalizing Relative Placement Groups 717

Handling Tap Columns and Relative Placement Group Overlaps 717

Changing the Structures of Relative Placement Groups 718

Generating Relative Placement Groups for Clock Sinks 719

Performing Placement and Legalization of Relative Placement Groups 720

Relative Placement in a Design Containing Obstructions 720

Legalizing Relative Placement Groups in a Placed Design 721

Creating New Relative Placement Groups in a Placed Design 722

Analyzing Relative Placement Groups 723
Checking Relative Placement Groups Before Placement 723
Analyzing the Placeability of a Relative Placement Group 724
Reporting Relative Placement Constraint Violations 724
Querying Relative Placement Groups 725
Analyzing Relative Placement in the GUI 725
Saving Relative Placement Information 726
Summary of Relative Placement Commands 726

10. Hierarchical Implementation 728
Overview of Abstract Views 729
Creating Abstract Views 730
Creating Abstracts With Power Information 732
Creating Abstracts for Signal Electromigration Analysis 732
Handling Multiple Levels of Physical Hierarchy 733
Creating Abstracts for Minimum Pulse Width Analysis 733
Reporting Abstract Inclusion Reasons 734
Making Changes to a Block After Creating an Abstract 735
Shrinking Abstract Views While Maintaining Timing 735
Recreating the Power and Ground (PG) Structure 737
Creating a Frame View 737
Linking to Abstract Views at the Top-Level 738
Changing the Block From Abstract to Design View 738
Linking to Subblocks With Multiple Labels 739
Specifying the Editability of Blocks From the Top-Level 740
Preparing for Top-Level Closure With Abstracts 741
Checking Designs With Abstracts for Top-Level-Closure Issues 742
Handling Design Data Using the Early Data Check Manager 745
Prerequisites for Handling Early Design Data 746
Early Data Checks, Policies, and Strategies 746
Setting the Policy for Early Data Checks 750
Reporting Early Data Check Records 751
Generating a Report of Early Data Check Records 753
Performing Top-Level Closure With Abstract Views 754
Creating ETMs and ETM Cell Libraries 755

Creating ETMs and ETM Cell Libraries in the IC Compiler II Tool 755
Creating ETMs in the PrimeTime Tool 756
Creating ETM Cell Libraries in the Library Manager Tool 758
Linking to ETMs at the Top Level 759
Performing Top-Level Closure With ETMs 760
Transparent Hierarchy Optimization 760
Accessing Libraries Before Enabling Transparent Hierarchy Optimization 762
Setting up Transparent Hierarchy Optimization 763
Performing THO During place_opt final_opto Stage 764
Rebudgeting After place_opt 764
Performing THO During route_opt or hyper_route_opt 765

11. Thermal Analysis Using Kelvin 766
Supporting Flows in Kelvin 766
Basic Flow 766
Design Block With Macro Model 767
Compact Model Flow 769
Multicorner-Multimode Flow 770
Displaying Maps in GUI 772
Reporting Thermal Results 773

12. RedHawk and RedHawk-SC Fusion 775
Running Rail Analysis Using RedHawk-SC Fusion 776
An Overview for RedHawk Fusion and RedHawk-SC Fusion 778
RedHawk Fusion and RedHawk-SC Fusion Data Flow 780
RedHawk/RedHawk-SC Fusion Analysis Flow 781
Running RedHawk Fusion Commands in the Background 782
Setting Up the Executables 783
Specifying RedHawk and RedHawk-SC Working Directories 783
Preparing Design and Input Data for Rail Analysis 785
Generating Rail Analysis Script Files 788
Supporting RedHawk-SC Customized Python Script Files 789
Specifying Ideal Voltage Sources as Taps 794
Validating the Taps and Finding Invalid Taps 796
Tap Attributes 797

Finding Invalid Taps 797
Finding Substitute Locations for Invalid Taps 798
Removing Taps 799
Missing Via and Unconnected Pin Checking 799
Setting Checking Options 800
Checking Missing Vias and Unconnected Pins 801
Viewing Error Data 802
Fixing Missing Vias 803
Running Rail Analysis with Multiple Rail Scenarios 804
Specifying a Design Scenario for Rail Analysis 805
Creating and Specifying Rail Scenarios for Rail Analysis 805
Performing Voltage Drop Analysis 808
Viewing Voltage Drop Analysis Results 810
Performing Sigma Analysis 812
Performing PG Electromigration Analysis 814
Viewing PG Electromigration Analysis Results 815
Displaying the PG Electromigration Map 815
Checking PG Electromigration Violations 816
Performing Minimum Path Resistance Analysis 819
Viewing Minimum Path Resistance Analysis Results 820
Tracing Minimum Path Resistance Using the Mouse Tool 822
Performing Effective Resistance Analysis 825
Performing Distributed RedHawk Fusion Rail Analysis 826
Voltage Driven Power Switch Cell Sizing 829
Working With Macro Models 831
Generating Macro Models 831
Creating Block Contexts 833
Performing Signoff Analysis 835
Writing Analysis and Checking Reports 836
Displaying Block-Level Rail Results 840
Generating Instance-Based Analysis Reports 840
Generating Geometry-Based Analysis Reports 842
Performing Effective Voltage Drop (Eff-VD) and Node-Based IR-Drop Flows 843
Enabling Eff-VD and Node-Based IR-Drop Flows 843
Displaying Maps in GUI 846
Displaying Maps in the GUI 855

Displaying ECO Shapes in the GUI 860
Voltage Hotspot Analysis 861
Generating Hotspots 862
Reporting Hotspots 863
Voltage Hotspot Analysis Examples 865
Querying Attributes 867

13. ECO Flow 869
Generic ECO Flow for Timing or Functional Changes 870
Freeze Silicon ECO Flow 871
Signoff ECO Flow 872
Incremental Signoff ECO Flow 873
ECO Fusion Flow 875
ECO Fusion Power Integrity Flow 876
Resolving RTL Mismatches in Power and Signal Elevation 877
Manually Instantiating Spare Cells 878
Automatically Adding Spare Cells 880
Adding Programmable Spare Cells 882
Making ECO Changes Using the eco_netlist Command 883
Making ECO Changes Using Netlist Editing Commands 884
Using ECO Scripts for Netlist Editing 884
Resizing Cells 884
Reverting Changes Made During Resizing 885
Adding Buffers on Nets 885
Adding Buffers on Routed Nets 886
Specifying the Net Names, Buffers Types, and Their Locations 886
Adding Buffers in a Specified Configuration 887
Adding Buffers at Specified Locations 887
Adding Buffers at a Specified Interval 888
Adding Buffers at an Interval That is a Ratio of the Net Length 889
Adding Buffers on a Bus in a Specified Pattern 889
Controlling How Buffers are Added 892
Specifying Settings for Multivoltage Designs 892
Specifying Settings for the Freeze Silicon ECO Flow 893
Optimizing the Fanout of a Net 893

Reporting Available Sites for Placing ECO Cells 894
Identifying and Reverting Nonoptimal ECO Changes 894
Placing ECO Cells 895
Controlling Placement When Using the place_eco_cells Command 895
Controlling Legalization When Using the place_eco_cells Command 897
Placing ECO Cells With Minimal Physical Impact (MPI) 899
Optimizing ECO Cell Legalization 899
Placing and Mapping ECO Cells to Spare Cells 900
Specifying Mapping Rules for Programmable Spare Cells 901
Mapping ECO Cells to Specific Spare Cells 901
Mapping ECO Cells to Logically Equivalent Spare Cells 902
Updating Supply Nets for ECO Cells 903
Recording the Changes Made to a Layout 903
Performing Prerequisite Check for Group Repeater Insertion and Placement 904
Adding a Group of Repeaters 905
Defining a group of repeaters 906
Grouping a list of repeaters 906
Setting Constraints for a Group of Repeaters 906
Adding Voltage Area Aware Group Repeaters 907
Reporting the Constraints Assigned to a Group of a Repeaters 907
Removing Constraints for a Group of Repeaters 908
Placing Group Repeaters Before Routing 908
Performing On Route Placement of Repeaters 908
Placing Group Repeaters For Multibit Registers 909
Specifying Locations for Repeater Groups 909
Allowing Repeater Groups Over Macros 910
Specifying Cut Space and Cut Distance for Repeater Groups 910
Specifying Horizontal and Vertical Spacing for Repeater Groups 910
Specifying Library Cells as Repeaters 911
Avoiding Overlapping Repeaters With Existing Tap Cells 911
Avoiding Crosstalk During Group Repeater Insertion 911
Previewing Repeater Groups 912
Unplacing the Repeaters 912
Removing Repeater Groups 912
Querying Group Repeater 912
Performing Auto Grouping Flow 913
Performing Manual Grouping Flow 914

Cell Input Mode 915
Swapping Variant Cell 915
Setting Constraints of Variant Cell 916
Setting Application Option 916
Grouping Variant Cell 917
Running and Placing Group Repeaters 923
Troubleshooting 923
Fixing Multivoltage Violations 930
Fixing Buffers 931
Fixing Diodes 932

Glossary 934

About This User Guide

The Synopsys IC Compiler II tool provides a complete netlist-to-GDSII or netlist-to-OASIS® design solution, which combines proprietary design planning, physical synthesis, clock tree synthesis, and routing for logical and physical design implementation throughout the design flow.

This guide describes the IC Compiler II implementation and integration flow. For more information about the IC Compiler II tool, see the following companion volumes:

- *Library Manager User Guide*
- *IC Compiler II Design Planning User Guide*
- *IC Compiler II Data Model User Guide*
- *IC Compiler II Timing Analysis User Guide*
- *IC Compiler II Graphical User Interface User Guide*
- *IC Compiler II Multivoltage User Guide*
- *IC Compiler II Power Analysis User Guide*

This user guide is for design engineers who use the IC Compiler II tool to implement designs.

To use the IC Compiler II tool, you need to be skilled in physical design and synthesis and be familiar with the following:

- Physical design principles
- The Linux operating system
- The tool command language (Tcl)

This preface includes the following sections:

- New in This Release
- Related Products, Publications, and Trademarks
- Conventions
- Customer Support
- Statement on Inclusivity and Diversity

Feedback

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the IC Compiler II Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the IC Compiler II tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

https://solvnetplus.synopsys.com

You might also want to see the documentation for the following related Synopsys products:

- Design Compiler®
- IC Validator
- PrimeTime® Suite
- StarRC™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
`Courier`	Indicates syntax, such as `write_file`
`Courier italic`	Indicates a user-defined value in syntax, such as `write_file` *`design_list`*
`Courier bold`	Indicates user input—text you type verbatim—in examples, such as `prompt> write_file top`
`Purple`	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as `include_enclosing = true | false`
[ ]	Denotes optional arguments in syntax, such as `write_file [-format fmt]`

Feedback

Convention	Description
...	Indicates that arguments can be repeated as many times as needed, such as *pin1 pin2 ... pinN*.
|	Indicates a choice among alternatives, such as `low | medium | high`
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the **Edit** menu and choosing **Copy**.
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Working With the IC Compiler II Tool

The IC Compiler II tool supports the following functionality:

- Extraction and timing analysis
- Non-physical synthesis
- Placement and optimization, including relative placement
- Clock tree synthesis
- Routing
- Chip finishing
- Top-level closure for hierarchical designs
- Engineering change orders (ECO)
- Reporting
- ASCII output interfaces

It takes as input a Verilog netlist, timing constraints, logic and physical libraries, and foundry-process data. It generates as output an OASIS- or GDSII-format file of the layout or a Design Exchange Format (DEF) file of placed netlist data ready for a third-party router. The IC Compiler II tool can also output the design at any time as a binary design database or as ASCII files (Verilog, DEF, and timing constraints).

The following topics describe how to use the IC Compiler II tool:

- Place and Route Design Flow Overview
- IC Compiler II Concepts
- Working With the 3DIC Compiler User Interfaces
- Entering icc2_shell Commands
- Using Application Options
- Using Variables
- Viewing Man Pages

Feedback

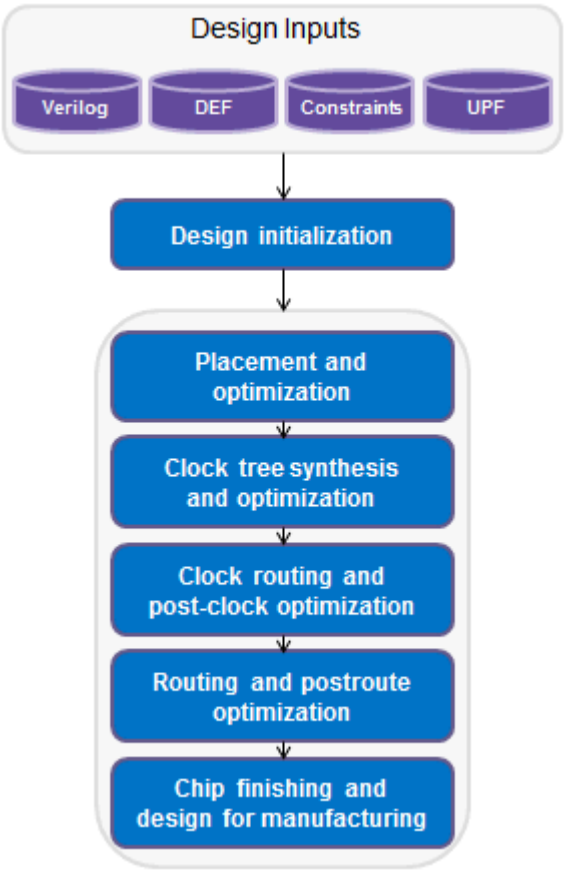
- Using Tcl Scripts
- Adding Changes to a Script With Checkpoints
- Using Setup Files
- Using the Command Log File
- Enabling Multicore Processing

For information about working with design data in the IC Compiler II tool, see the *IC Compiler II Data Model User Guide*.

Place and Route Design Flow Overview

Figure 1 shows the basic place and route design flow using the IC Compiler II tool.

Figure 1 IC Compiler II Place and Route Flow



Feedback

To run the IC Compiler II place and route flow,

1. Set up the libraries and prepare the design data, as described in Preparing the Design.
2. Perform design planning and power planning.

   When you perform design planning and power planning, you create a floorplan to determine the size of the design, create the boundary and core area, create site rows for the placement of standard cells, set up the I/O pads, and create a power plan.

   For more information about design planning and power planning, see the *IC Compiler II Design Planning User Guide*.
3. Perform placement and optimization.

   To perform placement and optimization, use the `place_opt` command.

   The `place_opt` command addresses and resolves timing closure for your design. This iterative process uses enhanced placement and synthesis technologies to generate legalized placement for leaf cells and an optimized design. You can supplement this functionality by optimizing for power, recovering area for placement, minimizing congestion, and minimizing timing and design rule violations.

   For more information about placement and optimization, see Placement and Optimization.
4. Perform clock tree synthesis and optimization.

   To perform clock tree synthesis and optimization, use the `clock_opt` command.

   IC Compiler II clock tree synthesis and embedded optimization solve complicated clock tree synthesis problems, such as blockage avoidance and the correlation between preroute and postroute data. Clock tree optimization improves both clock skew and clock insertion delay by performing buffer sizing, buffer relocation, gate sizing, gate relocation, level adjustment, reconfiguration, delay insertion, dummy load insertion, and balancing of interclock delays.

   For more information about clock tree synthesis and optimization, see Clock Tree Synthesis.
5. Perform routing and postroute optimization, as described in Routing and Postroute Optimization.

   The IC Compiler II tool uses Zroute to perform global routing, track assignment, detail routing, topological optimization, and engineering change order (ECO) routing. To perform postroute optimization, use the `route_opt` command. For most designs, the default postroute optimization setup produces optimal results. If necessary, you can supplement this functionality by optimizing routing patterns and reducing crosstalk or by customizing the routing and postroute optimization functions for special needs.

6. Perform chip finishing and design for manufacturing tasks, as described in Chip Finishing and Design for Manufacturing.

   The IC Compiler II tool provides chip finishing and design for manufacturing and design for yield capabilities that you can apply throughout the various stages of the design flow to address process design issues encountered during chip manufacturing.

7. Save the design.

IC Compiler II Concepts

This topic introduces the following concepts used in the IC Compiler II tool:

- Power Intent Concepts
- UPF Flows
- Multiple-Patterning Concepts

Power Intent Concepts

The UPF language establishes a set of commands to specify the low-power design intent for electronic systems. Using UPF commands, you can specify the supply network, switches, isolation, retention, and other aspects relevant to power management of a chip design. The same set of low-power design specification commands is to be used throughout the design, analysis, verification, and implementation flow. Synopsys tools are designed to follow the official IEEE 1801 UPF standard.

The UPF language provides a way to specify the power requirements of a design, but without specifying explicitly how those requirements are implemented. The language specifies how to create a power supply network for each design element, the behavior of supply nets with respect to each other, and how the logic functionality is extended to support dynamic power switching to design elements. It does not contain any placement or routing information.

In the UPF language, a *power domain* is a group of elements in the design that share a common set of power supply needs. By default, all logic elements in a power domain use the same primary supply and primary ground. Other power supplies can be defined for a power domain as well. A power domain is typically implemented as a contiguous *voltage area* in the physical chip layout, although this is not a requirement of the language.

Each power domain has a *scope* and an *extent*. The *scope* is the level of logic hierarchy designated as the root of the domain. The *extent* is the set of logic elements that belong to the power domain and share the same power supply needs. The scope is the hierarchical level at which the domain is defined and is an ancestor of the elements belonging to the

power domain, whereas the extent is the actual set of elements belonging to the power domain.

Each scope in the design has *supply nets* and *supply ports* at the defined hierarchical level of the scope. A *supply net* is a conductor that carries a supply voltage or ground throughout a given power domain. A supply net that spans more than one power domain is said to be “reused” in multiple domains. A *supply port* is a power supply connection point between two adjacent levels of the design hierarchy, between the parent and child blocks of the hierarchy. A supply net that crosses from one level of the design hierarchy to the next passes through a supply port.

A *supply set* is an abstract collection of supply nets, consisting of two supply functions, power and ground. A supply set is *domain-independent*, which means that the power and ground in the supply set are available to be used by any power domain defined within the scope where the supply set was created. However, each power domain can be restricted to limit its usage of supply sets within that power domain.

You can use supply sets to define power intent at the RTL level, so you can synthesize a design even before you know the names of the actual supply nets. A supply set is an abstraction of the supply nets and supply ports needed to power a design. Before such a design can physically implemented (placed and routed), its supply sets must be *refined*, or associated with actual supply nets.

A *supply set handle* is an abstract supply set created for a power domain. By default, a power domain has supply set handles for the domain’s primary supply set, a default isolation supply set, and a default retention supply set. These supply set handles let you synthesize a design even before you create any supply sets, supply nets, and supply ports for the power domain. Before such a design can be physically implemented, its supply set handles must be *refined*, or associated with actual supply sets; and those supply sets must be refined so that they are associated with actual supply nets.

A *power switch* (or simply *switch*) is a device that turns on and turns off power for a supply net. A switch has an input supply net, an output supply net that can be switched on or off, and at least one input signal to control switching. The switch can optionally have multiple input control signals and one or more output acknowledge signals. A *power state table* lists the allowed combinations of voltage values and states of the power switches for all power domains in the design.

A *level shifter* must be present where a logic signal leaves one power domain and enters another at a substantially different supply voltage. The level shifter converts a signal from the voltage swing of the first domain to that of the second domain.

An *isolation* cell must be present where a logic signal leaves a switchable power domain and enters a different power domain. The level shifter generates a known logic value during shutdown of the domain. If the voltage levels of the two domains are substantially different, the interface cell must be able to perform both level shifting (when the domain

Feedback

is powered up) and isolation (when the domain is powered down). A cell that can perform both functions is called an *enable level shifter*.

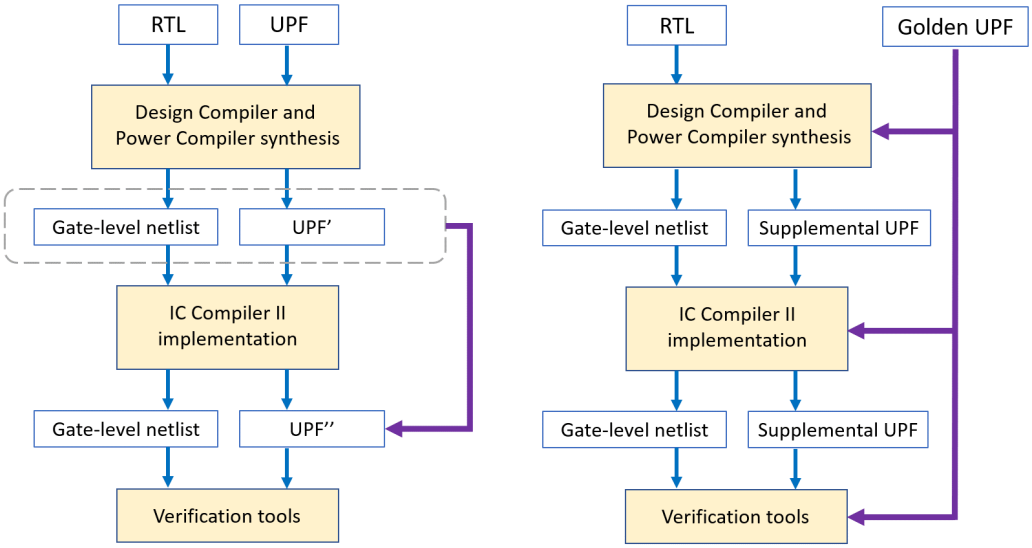
In a power domain that has power switching, any registers that must retain data during shutdown must be implemented as *retention registers*. A retention register has a separate, always-on supply net, sometimes called the backup supply, which keeps the data stable in the retention register while the primary supply of the domain is shut down.

UPF Flows

The IC Compiler II tool supports both the traditional UPF flow and the golden UPF flow. The golden UPF flow is an optional method of maintaining the UPF multivoltage power intent of the design. It uses the original “golden” UPF file throughout the synthesis, physical implementation, and verification steps, along with supplemental UPF files generated by the Design Compiler and IC Compiler II tools.

Figure 2 compares the traditional UPF flow with the golden UPF flow.

Figure 2 UPF-Prime (Traditional) and Golden UPF Flows



The golden UPF flow maintains and uses the same, original “golden” UPF file throughout the flow. The Design Compiler and IC Compiler II tools write power intent changes into a separate “supplemental” UPF file. Downstream tools and verification tools use a combination of the golden UPF file and the supplemental UPF file, instead of a single UPF’ or UPF’’ file.

Feedback

The golden UPF flow offers the following advantages:

- The golden UPF file remains unchanged throughout the flow, which keeps the form, structure, comment lines, and wildcard naming used in the UPF file as originally written.
- You can use tool-specific conditional statements to perform different tasks in different tools. Such statements are lost in the traditional UPF-prime flow.
- Changes to the power intent are easily tracked in the supplemental UPF file.
- You can optionally use the Verilog netlist to store all PG connectivity information, making `connect_supply_net` commands unnecessary in the UPF files. This can significantly simplify and reduce the overall size of the UPF files.

To enable the golden UPF flow, use the following application option setting before you load the UPF:

```
icc2_shell> set_app_options -as_user_default  \
   -list {mv.upf.enable_golden_upf true}
```

To load supplemental UPF files, use the `-supplemental` option with the `load_upf` command.

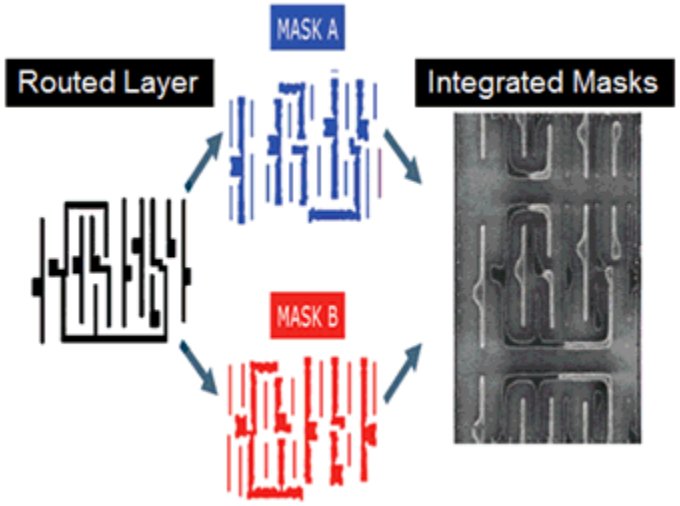
For more information about using the golden UPF flow, see SolvNet article 1412864, "Golden UPF Flow Application Note."

Multiple-Patterning Concepts

At the 20-nm process node and below, printing the required geometries is extremely difficult with the existing photolithography tools. To address this issue, a new technique, *multiple patterning*, is used to partition the layout mask into two or more separate masks, each of which has an increased manufacturing pitch to enable higher resolution and better printability. Figure 3 shows an example of double-patterning, where the layout mask is partitioned into two separate masks, MASK A and MASK B.

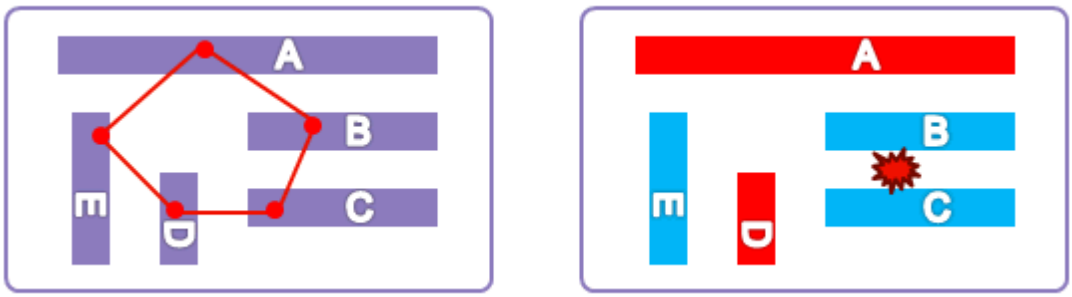
Feedback

Figure 3 Double-Patterning Example



To use multiple patterning, you must be able to decompose the layout into two or more masks, each of which meets the multiple-patterning spacing requirements. A multiple-patterning violation occurs if your layout contains a region with an odd number of neighboring shapes where the distance between each pair of shapes is smaller than the multiple-patterning minimum spacing. This type of violation, which is called an *odd cycle*, is shown in Figure 4.

Figure 4 Odd-Cycle Violation



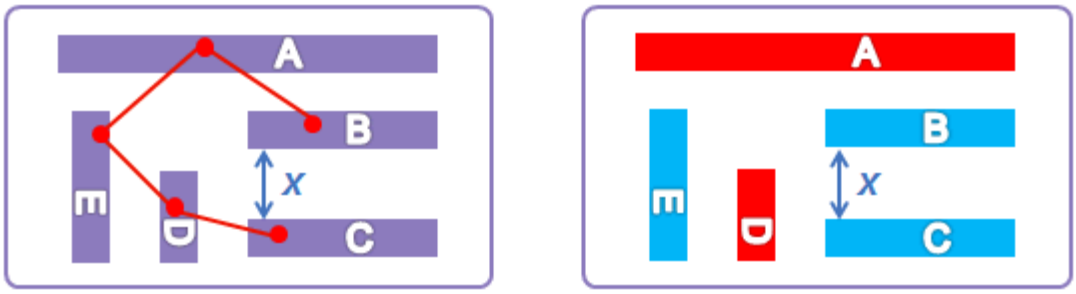
Layout with an odd cycle

Spacing violation prevents decomposition

If the spacing between any pair in the loop is greater than the multiple-patterning minimum spacing, no violation occurs and the layout can be decomposed. For example, in Figure 5, if the spacing, *x*, between segments B and C is greater than the multiple-patterning minimum spacing, there is no odd cycle and the layout can be decomposed.

Feedback

Figure 5 No Odd-Cycle Violation



Layout with no odd cycle

Layout can be decomposed

The IC Compiler II tool ensures that the generated layout is conducive to double patterning by considering the multiple-patterning spacing requirements during placement and routing and preventing odd cycles.

In general, double patterning is performed only on the bottom (lowest) metal layers, which are referred to as *multiple-patterning layers*. The metal shapes on the multiple-patterning layers must meet the multiple-patterning spacing requirements, whether they are routing shapes or metal within the standard cells and macros. The metal shapes on other layers do not need to meet the stricter multiple-patterning spacing requirements.

Multiple-patterning considerations affect all parts of the place and route flow. Depending on your standard cell library, you follow either an uncolored or precolored multiple-patterning flow.

- You use the *uncolored* flow if the cells in your standard cell library have sufficient spacing to the cell boundaries to ensure that multiple-patterning violations do not occur during placement. This type of library is referred to as a *correct-by-construction library*; most multiple-patterning libraries are correct-by-construction libraries.

  In the uncolored flow, the tool determines the appropriate mask settings for the pins and net shapes.

- You use the *precolored* flow if the cells in your standard cell library have assigned masks on the metal shapes inside the cells. These assigned masks are often referred to as *colors* and this type of library is referred to as a *precolored library*.

  In the precolored flow, the tool must consider these mask assignments to ensure that multiple-patterning violations do not occur during placement. The tool also uses the mask assignments to determine the appropriate mask settings for the pins and net shapes.

  The mask assignments are represented as *mask constraints* in the IC Compiler II tool. You must ensure that the mask constraints are properly set before starting place and route. For information about the mask constraints, see Mask Constraints.

Mask Constraints

Mask constraints indicate the mask requirements for the metal shapes of the physical pins and nets in a block that uses multiple-patterning technology. These mask requirements drive placement and routing to ensure that the resulting layout is multiple-patterning compliant.

Note:

Mask constraints are used only for the precolored flow; they are not necessary in the uncolored flow.

You can set mask constraints on timing-critical nets (net shapes, routing rules, and routing blockages) and vias. For nets, the mask constraint is defined in the `mask_constraint` attribute. For vias, the mask constraints are defined in the `lower_mask`, `upper_mask`, and `cut_mask` attributes. Table 1 shows the supported values for these attributes.

Table 1 Mask Constraint Values

Attribute value	Description
`same_mask`	This constraint means that the mask color is not yet been determined. Shapes with this attribute must be at least the multiple-patterning minimum spacing distance from any other colored metal shape.
`mask_one`	This constraint means that the shape has the mask1 color. Shapes with this attribute must be at least the multiple-patterning minimum spacing distance from other mask1-colored metal shapes.
`mask_two`	This constraint means that the shape has the mask2 color. Shapes with this attribute must be at least the multiple-patterning minimum spacing distance from other mask2-colored metal shapes.
`mask_three`	This constraint means that the shape has the mask3 color. Shapes with this attribute must be at least the multiple-patterning minimum spacing distance from other mask3-colored metal shapes.
`any_mask`	This constraint means that the shape is not colored. Shapes with this attribute must be at least the standard minimum spacing distance from other metal shapes; the multiple-patterning minimum spacing rules do not apply to these shapes.

Working With the 3DIC Compiler User Interfaces

The IC Compiler II tool operates in the X Window environment on Linux. The tool provides a flexible working environment with both a shell command-line interface and a GUI. The CLI is always available during a IC Compiler II session. You can start or exit a session in either the shell or the GUI, and you can open or close the GUI during a session.

Feedback

The tool uses Tcl, which is used in many applications in the EDA industry. Using Tcl, you can extend the icc2_shell command language by writing reusable procedures and scripts (see the *Using Tcl With Synopsys Tools* manual).

The following topics describe how to start and exit the tool using the command-line interface.

- Starting the CLI
- Exiting the IC Compiler II Tool

For information about using the GUI, see the *IC Compiler II Graphical User Interface User Guide*.

The following topics describe how to use the user interfaces of the IC Compiler II tool:

- Starting the GUI
- Closing the GUI

Starting the CLI

The CLI interface is a text-only environment in which you enter commands at the command-line prompt. It is typically used for scripts, batch mode, and push-button operations.

Before you start the command-line interface, ensure that the path to the bin directory is included in your $PATH variable.

To start icc2_shell,

► Enter the `icc2_shell` command in a Linux shell.

```
% icc2_shell
```

You can include other options on the command line when you start the shell. For example, you can use

- `-file` *`script_file_name`* to execute a script
- `-x` *`command`* to execute a command
- `-output_log_file` *`file_name`* to create a log file of your session
- `-help` to display a list of the available options (without starting the shell)

At startup, the tool :

1. Creates a command log file.
2. Reads and executes the setup files.

3. Executes any script files or commands specified by the `-file` and `-x` options, respectively, on the command line.
4. Displays the program header and icc2_shell> prompt in the shell.

See Also

- Using the Command Log File
- Using Setup Files
- Using Tcl Scripts

Exiting the IC Compiler II Tool

You can end the session and exit the IC Compiler II tool at any time. To exit the tool, use the `exit` or `quit` command.

Note:

When you exit the tool from the command line, the tool exits without saving the open blocks.

Entering icc2_shell Commands

You interact with the IC Compiler II tool by using icc2_shell commands, which are based on the Tool Command Language (Tcl) and include certain command extensions needed to implement specific IC Compiler II functionality. The IC Compiler II command language provides capabilities similar to Linux command shells, including variables, conditional execution of commands, and control flow commands. You can

- Enter individual commands interactively at the icc2_shell> prompt in icc2_shell
- Enter individual commands interactively on the console command line in the GUI
- Run one or more Tcl scripts, which are text files that contain icc2_shell commands (see Using Tcl Scripts)

When entering a command, an option, or a file name, you can minimize your typing by pressing the Tab key when you have typed enough characters to specify a unique name; the IC Compiler II tool completes the remaining characters. If the characters you typed could be used for more than one name, the IC Compiler II tool lists the qualifying names, from which you can select by using the arrow keys and the Enter key.

If you need to reuse a command from the output for a command-line interface, you can copy and paste the portion by selecting it, moving the pointer to the icc2_shell command line, and clicking with the middle mouse button.

Feedback

When you run a command, the IC Compiler II tool echoes the command output (including processing messages and any warnings or error messages) in icc2_shell and, if the GUI is open, in the console log view. By default, the tool does not use page mode, so the output might scroll. To enable page mode, set the `sh_enable_page_mode` variable to `true`.

Interrupting or Terminating Command Processing

If you enter the wrong options for a command or enter the wrong command, you can interrupt command processing and remain in icc2_shell. To interrupt or terminate a command, press Ctrl+C.

Some commands and processes cannot be interrupted. To stop these commands or processes, you must terminate icc2_shell at the system level. When you terminate a process or the shell, no data is saved.

When you use Ctrl+C, keep the following points in mind:

- If a script file is being processed and you interrupt one of its commands, the script processing is interrupted and no further script commands are processed.
- If you press Ctrl+C three times before a command responds to your interrupt, icc2_shell is interrupted and exits with this message:

  ```
  Information: Process terminated by interrupt.
  ```

Getting Information About Commands

The following online information resources are available while you are using the IC Compiler II tool:

- Command help, which is information about an IC Compiler II command
- Man pages

See Also

- Viewing Man Pages

Feedback

Displaying Command Help

Command help consists of either a brief description of an IC Compiler II command or a list of the options and arguments supported by an IC Compiler II command.

- To display a brief description of an IC Compiler II command, enter the `help` command followed by the command name. For example, to display a brief description of the `report_timing` command, use the following command:

  ```
  icc2_shell> help report_timing
  ```

- To display the options supported by an IC Compiler II command, enter the command name with the `-help` option on the command line. For example, to see the options supported by the `report_timing` command, use the following command:

  ```
  icc2_shell> report_timing -help
  ```

Using Application Options

The IC Compiler II tool uses application options to control the tool behavior. Application options use the following naming convention:

category[.*subcategory*].*option_name*

where *category* is the name of the engine affected by the application option. Some application option categories have subcategories to further refine the area affected by the application option.

Application options have either a global scope or a block scope.

- Block-scoped application options apply only to the block on which they are set. They are saved in the design library and are persistent across tool sessions.
- Global-scoped application options apply to all blocks, but only within the current session. They are not saved in the design library; you must specify them in each icc2_shell session. You might want to consider adding the global settings to your .synopsys_icc2.setup file.

To get a list of available application options, use the `get_app_options` command. By default, this command lists all application options. To restrict the reported application options, provide a pattern string as an argument to the command.

For example, to list all available application options, use the following command:

```
icc2_shell> get_app_options
```

To list all available timer application options, use the following command:

```
icc2_shell> get_app_options timer.*
```

Feedback

To generate a report of application options, use the `report_app_options` command.

For detailed information about application options, see Application Options in the *IC Compiler II Data Model User Guide*.

See Also

- Using Setup Files

Using Variables

In general, the IC Compiler II tool modifies default behavior by using application options rather than application variables; however it does support user-defined Tcl variables, as well as a minimal number of application variables, such as the `search_path` variable.

To list the variables and their values, use the `printvar` command. For example, to list all variables defined in the current session, use the following command:

```
icc2_shell> printvar *
```

To print the value of the search_path variable, use the following command:

```
icc2_shell> printvar search_path
```

See Also

- Defining the Search Path

Viewing Man Pages

To display the man page for an IC Compiler II command or application option, enter the `man` command followed by the command or application option name. For example, to see the man page for the `report_timing` command, use the following command:

```
icc2_shell> man report_timing
```

Feedback

To display the man page for an IC Compiler II application option, enter the `man` command followed by the option name. You can also view the following types of summary pages for application options:

- Category summaries

  To view a man page that summarizes all of the application options for a specific category, enter the `man` command followed by *category*_options. For example, to see the man page that summarizes all timer application options, use the following command:

  ```
  icc2_shell> man timer_options
  ```

- Subcategory summaries

  To view a man page the summarizes all of the application options for a specific subcategory, enter the `man` command followed by *category*.*subcategory*_options. For example, to see the man page that summarizes all common route application options, use the following command:

  ```
  icc2_shell> man route.common_options
  ```

- Command summaries

  To view a man page the summarizes all of the application options for a specific command, enter the `man` command followed by *command*_options. For example, to see the man page that summarizes all application options that affect the `report_timing` command, use the following command:

  ```
  icc2_shell> man report_timing_options
  ```

If you enter the `man` command on the icc2_shell command line, the man page is displayed in the IC Compiler II shell and in the console log view if the GUI is open. If you enter this command on the console command line in the GUI, the man page is displayed in the GUI man page viewer.

Using Tcl Scripts

You can use Tcl scripts to accomplish routine, repetitive, or complex tasks. You create a command script file by placing a sequence of IC Compiler II commands in a text file. Any IC Compiler II command can be executed within a script file.

In Tcl, a pound sign (#) at the beginning of a line denotes a comment. For example,

```
# This is a comment
```

For more information about writing scripts and script files, see the *Using Tcl With Synopsys Tools* manual.

Feedback

Use one of the following methods to run a Tcl script:

- Use the `-file` option with the `icc2_shell` command when you start the IC Compiler II tool.
- Use the `source` command from the icc2_shell command line.
- Choose File > Execute Script in the GUI.

If an error occurs when running a command, the IC Compiler II tool raises the TCL_ERROR condition, which immediately stops the script execution. To tolerate errors and allow the script to continue executing, either

- Check for TCL_ERROR error conditions with the Tcl `catch` command on the commands that might generate errors.
- Set the `sh_continue_on_error` variable to `true` in the script file.

See Also

- Starting the CLI
- Adding Changes to a Script With Checkpoints

Adding Changes to a Script With Checkpoints

When running experiments on a block, you might might modify your golden flow scripts in two ways:

- By making flow changes, such as setting application options, modifying constraints, or making other changes to improve the quality of results
- By generating reports (either additional reports or reports at new places in the flow) to help debug an issue

In most flows, you apply changes through a small number of static, modifiable files. Here is a typical example to run the place_opt flow, where you source a pre_place_opt_settings.tcl file to add flow changes and a generate_reports.tcl file to run all your reporting at the end of the script:

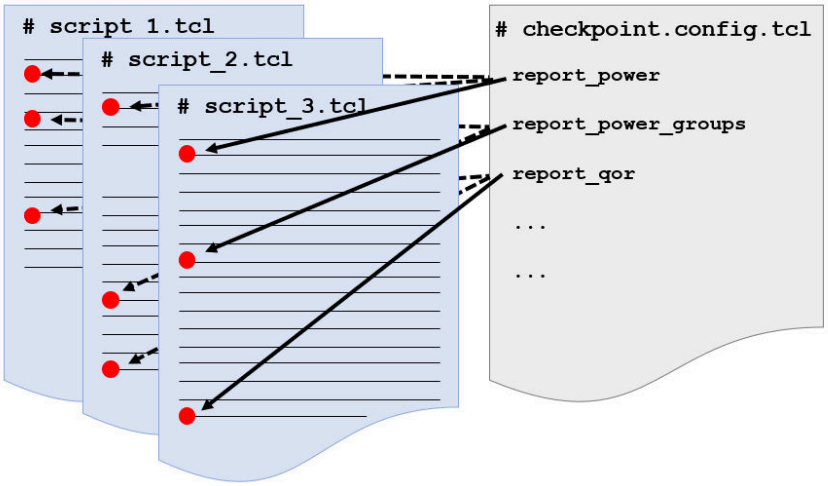
```
open_block ./design:init_design
source ./scripts/pre_place_opt_settings.tcl
remove_buffer_trees -all
place_opt -from initial_place -to initial_drc
create_placement -incremental -timing_driven -congestion
place_opt -from initial_drc -to -initial_opto
place_opt -from final_place -to final_opto
source ./scripts/generate_reports.tcl
```

Feedback

In some cases, complex changes require modifying your golden flow script directly, which might be discouraged or difficult to do in many design environments. These changes can be time consuming to implement, especially if you are implementing them at multiple stages in the design flow, across multiple blocks, or for several different flow experiments.

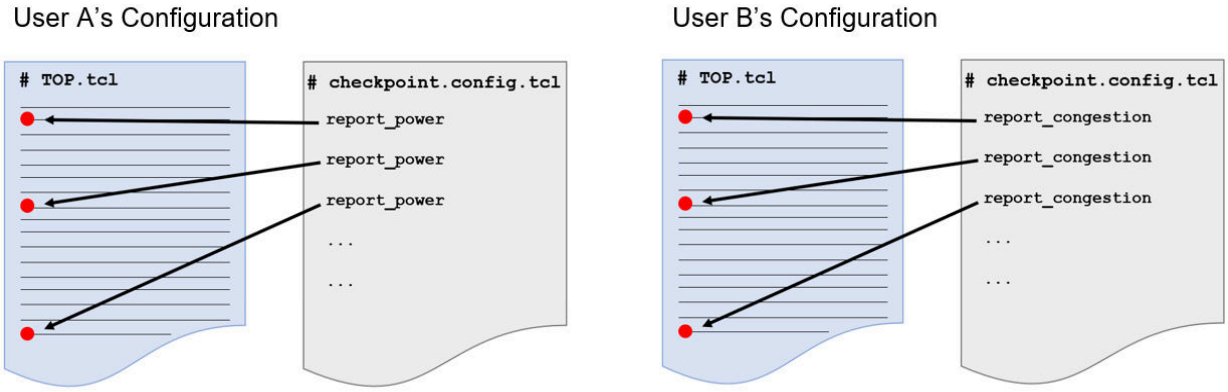
The checkpoint system streamlines this process and allows you to run experiments without modifying your golden flow scripts. When you use the checkpoint system, you define checkpoints for important steps in your golden flow. These checkpoints enwrap the commands associated with the given steps. By default, the checkpoints run only the code they enwrap. That is, simply inserting checkpoints in your script does not change your flow. However, you can associate these checkpoints with flow changes or reports that you want to run before, after, or in place of the code the checkpoints enwrap. By defining these associations in a portable configuration file, you isolate your golden flow scripts from the changes you want to apply for a particular run.

The following figure shows how you can use checkpoints to instantly apply a set of flow changes or reports to multiple designs, without modifying each design's golden flow script. In this case, the configuration file has been copied to each design's run directory.



Feedback

In addition, when multiple users are working on a project, each user can have an individual checkpoint.config.tcl file with his or her preferred settings:



The checkpoint system also allows you to

- Insert flow changes and reports with more precision than you could using a typical non-checkpointed flow script
- Reference a single source of truth where you can find all changes that have been applied to your default golden flow
- Report a history of the checkpoints that were run, including the flow changes or reports that were run at those checkpoints
- Report runtime and memory information for each checkpoint that was run
- Generate unique and descriptive report names

The general process for using the checkpoint system is as follows:

1. Insert checkpoints in your script.
2. Configure your checkpoints by
   - Defining the flow changes or reports you want to associate with the checkpoints
   - Associating the flow changes or reports you defined with your checkpoints
3. Run your script.

Defining Checkpoints

To define a checkpoint, insert the `eval_checkpoint` command in your script. Specify a unique name for the checkpoint and wrap the checkpoint around a block of code.

Feedback

Note the following:

- If you define a checkpoint with the same name as an existing checkpoint, the tool automatically adds a unique suffix to the name of the newly-defined checkpoint when it encounters the checkpoint in a run. For example, if you duplicate a checkpoint named placement, the tool renames the duplicate to placement2 when it encounters the checkpoint in a run.
- Do not nest checkpoints within other checkpoints. Although the tool does not prevent you from defining nested checkpoints, the tool ignores nested checkpoints during a run, executing only the outermost checkpoints (note that the Tcl code inside nested checkpoints still runs, but runs as though the checkpoints do not exist).

Suppose you have a simple place_opt script:

```
open_block ./design:init_design
source ./scripts/pre_place_opt_settings.tcl
remove_buffer_trees -all
place_opt -from initial_place -to initial_drc
create_placement -incremental -timing_driven -congestion
place_opt -from initial_drc -to -initial_opto
place_opt -from final_place -to final_opto
```

The following example wraps a checkpoint around each major step in your golden flow. The checkpoints are named remove_buffers, place_opt_to_initial_drc, incr_placement, place_opt_to_initial_opto, and place_opt_to_final_opto.

```
open_block ./design.nlib:init_design
source ./scripts/pre_place_opt_settings.tcl

eval_checkpoint remove_buffers {
    remove_buffer_trees -all
}

eval_checkpoint place_opt_to_initial_drc {
    place_opt -from initial_place -to initial_drc
}

eval_checkpoint incr_placement {
    create_placement -incremental -timing_driven -congestion
}

eval_checkpoint place_opt_to_initial_opto {
    place_opt -from initial_drc -to initial_opto
}

eval_checkpoint place_opt_to_final_opto {
    place_opt -from final_place -to final_opto
}
```

Feedback

By default, the checkpoints run the Tcl code they enwrap. That is, simply inserting checkpoints with the `eval_checkpoint` command does not change your flow. However, you can associate these checkpoints with flow changes or reports that you want to run before, after, or in place of the code the checkpoints enwrap, as described in Configuring Checkpoints.

See Also

- Configuring Checkpoints
- Querying Checkpoints and Checkpoint Behaviors

Configuring Checkpoints

For a checkpoint to affect your flow, you must associate it with a flow change or report that you want to run before, after, or in place of the code the checkpoint enwraps.

The process of defining flow changes and reports and associating them with individual checkpoints is described in the following topics:

- Defining Checkpoint Behaviors
- Associating Checkpoints and Checkpoint Behaviors

Defining Checkpoint Behaviors

You can define two types of checkpoint behaviors: reports and actions. Define a checkpoint report to generate one or more reports supported by the tool or custom reports you create yourself. Define a checkpoint action to execute any kind of flow change before, after, or in place of the code block enwrapped by a checkpoint. For example, you might set an application option to a specific value before a particular checkpoint.

Define these behaviors in your checkpoint.config.tcl file, which is automatically sourced by the checkpoint system.

Defining Checkpoint Reports

To define a checkpoint report, use the `create_checkpoint_report` command in your checkpoint.config.tcl file. Specify a unique name for the report and define the Tcl commands to generate the report.

The following example defines a checkpoint report named timing, which writes the output of the `report_qor` and `report_timing` commands to disk:

```
create_checkpoint_report timing {
   set name [get_current_checkpoint -name]
   set pos [get_current_checkpoint -position]

   report_qor -nosplit > ./checkpoint/$name.$pos.qor.rpt
```

Feedback

```
   report_qor -summary -nosplit >> ./checkpoint/$name.$pos.qor.rpt

   report_timing -nosplit -max_paths 10 \
      > ./checkpoint/$name.$pos.path.rpt
}
```

Notice the use of the `get_current_checkpoint` command with the `-name` and `-position` options to give the generated reports a meaningful name. When these reports are generated, their names reflect the checkpoint that triggered them (`get_current_checkpoint -name`), as well as whether they were generated before or after the checkpoint (`get_current_checkpoint -position`). For example, if this checkpoint report is executed after a checkpoint named checkpoint_A, the generated reports are saved to the checkpoint directory, and the name of the QoR report is checkpoint_A.after.qor.rpt.

The next example defines a checkpoint report named app_options, which writes all your non-default application options to disk:

```
create_checkpoint_report app_options {
   set name [get_current_checkpoint -name]
   set pos [get_current_checkpoint -position]

   report_app_options -non_default \
      > ./checkpoint/$name.$pos.app_options.rpt
}
```

Defining Checkpoint Actions

To define a checkpoint action, use the `create_checkpoint_action` command in your checkpoint.config.tcl file. Specify a unique name for the action and define the Tcl commands that constitute the action.

The following example defines a checkpoint action named gropto, which enables global-route based buffering:

```
create_checkpoint_action gropto {
   set_app_options -name place_opt.initial_drc.global_route_based \
      -value true
}
```

The next example defines a checkpoint action named placer_high_effort_cong, which runs high-effort congestion reduction:

```
create_checkpoint_action placer_high_effort_cong {
      set placer_command [get_current_checkpoint -script]
      set cong_option "-congestion_effort high"
      eval $placer_command $cong_option
}
```

Notice the use of the `get_current_checkpoint -script` command and option, which retrieves the command originally enwrapped by the checkpoint and sets its congestion

Feedback

effort to high (`-congestion_effort high`). The `-script` option is typically used to define actions that replace the contents of a command. In this example, the action modifies the `-congestion_effort` option of the command that is enwrapped by the checkpoint associated with this action.

Associating Checkpoints and Checkpoint Behaviors

After defining checkpoints with the `eval_checkpoint` command and checkpoint behaviors with the `create_checkpoint_*` command, you must associate the two so that the tool executes the behaviors when it encounters the checkpoints in a run, as described in the following topics:

- Associating Checkpoints With Checkpoint Reports
- Associating Checkpoints With Checkpoint Actions

See Also

- Defining Checkpoints
- Defining Checkpoint Behaviors

Associating Checkpoints With Checkpoint Reports

To associate a checkpoint report with one or more checkpoints, use the `associate_checkpoint_report` command in your checkpoint.config.tcl file.

- Specify the checkpoint report name with the `-enable` option.
- To enable the report to run before one or more checkpoints, specify the checkpoint names with the `-before` option.
- To enable the report to run after one or more checkpoints, specify the checkpoint names with the `-after` option.

When associating a report with multiple checkpoints, you can list the checkpoint names or specify the asterisk wildcard character.

Assume you have defined three checkpoints in your script named remove_buffers, place_opt_to_initial_opto, and place_opt_to_final_opto using the `eval_checkpoint` command, and two checkpoint reports named timing and app_options using the `create_checkpoint_report` command.

To enable your timing report to run after the place_opt_to_initial_opto and place_opt_to_final_opto checkpoints, use the following command:

```
associate_checkpoint_report -enable timing \
   -after { place_opt_to_initial_opto place_opt_to_final_opto }
```

Feedback

To enable your app_options report to run before all the checkpoints in your script, use the following command:

```
associate_checkpoint_report -enable app_options -before *
```

You can also add the command like this, which enables the report to run before the remove_buffers checkpoint and any checkpoints beginning with the phrase place_opt:

```
associate_checkpoint_report -enable app_options -before remove_buffers
 place_opt*
```

The following is a portion of your checkpointed script showing what happens when the script runs. Note that the highlighted code is not actually in your script, but shows the checkpoint behaviors that are executed when the tool encounters the checkpoints in the run.

- The code that is commented out identifies the original script body
- The blue code identifies the checkpointed Tcl commands in your golden flow
- The green code identifies the app_options report, configured to run before each checkpoint
- The purple code identifies the timing report, configured to run after the place_opt_to_final_opto checkpoint

```
# eval_checkpoint remove_buffers {
#     remove_buffer_trees -all
# }
report_app_options -non_default > ./checkpoint/remove_buffers.before.app_options.rpt
remove_buffer_trees -all
...
# eval_checkpoint place_opt_to_final_opto {
#     place_opt -from final_place -to final_opto
# }
report_app_options -non_default > ./checkpoint/place_opt_to_final_opto.before.app_options.rpt
place_opt -from final_place -to final_opto
report_qor -nosplit > ./checkpoint/place_opt_to_final_opto.after.qor.rpt
report_qor -summary -nosplit >> ./checkpoint/place_opt_to_final_opto.after.qor.rpt
report_timing -nosplit -max_paths 10 > ./checkpoint/place_opt_to_final_opto.after.path.rpt
```

Associating Checkpoints With Checkpoint Actions

To associate a checkpoint action with one or more checkpoints, use the `associate_checkpoint_action` command in your checkpoint.config.tcl file.

Feedback

- Specify the checkpoint action name with the `-enable` option.
- To enable the action to run before one or more checkpoints, specify the checkpoint names with the `-before` option.
- To enable the action to run after one or more checkpoints, specify the checkpoint names with the `-after` option.
- To enable the action to run instead of the code block enwrapped by the checkpoint, specify the checkpoint names with the `-replace` option.

When associating an action with multiple checkpoints, you can list the checkpoint names or specify the asterisk wildcard character.

Assume you have defined two checkpoints in your script named place_opt_to_initial_drc and incr_placement with the `eval_checkpoint` command, and two checkpoint actions named gropto and placer_high_effort_cong with the `create_checkpoint_action` command.

To enable your gropto action to run before the place_opt_to_initial_drc checkpoint, use the following command:

```
associate_checkpoint_action -enable gropto \
   -before place_opt_to_initial_drc
```

To enable your placer_high_effort_cong action to run instead of the code block enwrapped by the incr_placement checkpoint, use the following command:

```
associate_checkpoint_action -enable placer_high_effort_cong \
   -replace incr_placement
```

The following is a portion of your checkpointed script showing what happens when the script runs. Note that the highlighted code is not actually in your script, but shows the checkpoint behaviors that are executed when the tool encounters the checkpoints in a run.

- The code that is commented out identifies the original script body
- The blue code identifies the checkpointed Tcl commands in your golden flow
- The green code identifies the gropto action, configured to run before the place_opt_to_initial_drc checkpoint
- The purple code identifies the placer_high_effort_cong action, configured to run in place of the code block enwrapped by the incr_placement checkpoint

Feedback

```
# eval_checkpoint remove_buffers {
#     remove_buffer_trees -all
# }
remove_buffer_trees -all

# eval_checkpoint place_opt_to_initial_drc {
#     place_opt -from initial_place -to initial_drc
# }
set_app_options -name place_opt.initial_drc.global_route_based -value true
place_opt -from initial_place -to initial_drc

# eval_checkpoint incr_placement {
#     create_placement -incremental -timing_driven -congestion
# }
## Runs 'create_placement -incremental -timing_driven -congestion -congestion_effort high'
set placer_command [get_current_checkpoint -script]
set cong_option "-congestion_effort high"
eval $placer_command $cong_option
create_placement -incremental -timing_driven -congestion
```

Associating a Checkpoint With Multiple Behaviors

When you associate a checkpoint with multiple behaviors, the tool executes those behaviors in the following order:

- Any reports configured to run before the checkpoint
- Any actions configured to run before the checkpoint
- The checkpointed code block, or any actions configured to replace it
- Any actions configured to run after the checkpoint
- Any reports configured to run after the checkpoint

If multiple reports or multiple actions are configured to run at the same position (`-before`, `-after`, or `-replace`) in the same checkpoint, the tool executes them in the order in which they were associated with the checkpoint with the `associate_checkpoint_*` commands. For example, suppose you define a timing report before an area report. If you associate the area report to run after a checkpoint before you associate the timing report to run after the same checkpoint, the tool runs the area report first when it encounters the checkpoint.

Feedback

See Also

- Defining Checkpoints
- Defining Checkpoint Behaviors
- Querying Checkpoints and Checkpoint Behaviors

Querying Checkpoints and Checkpoint Behaviors

To return a list of all the checkpoints or checkpoint behaviors in the current session, or to return detailed information for any checkpoint or checkpoint behavior, use the `get_checkpoint_data` command.

- To return a list of all the checkpoints that have been executed, specify the `-list_names` option:

```
icc2_shell> get_checkpoint_data -list_names
remove_buffers place_opt_to_initial_drc
incr_placement place_opt_to_initial_opto
place_opt_to_final_opto
```

- To return detailed information for a checkpoint, specify the checkpoint with the `-name` option. The tool returns the output as a Tcl dictionary:

```
icc2_shell> get_checkpoint_data -name place_opt_to_initial_drc
memory 173.85 start_time 2.34 end_time 2.34 before_runtime 2.34
 before_report_runtime 0.00 after_report_runtime 0.00 self_runtime
 0.00 before_reports {} after_reports {app_options timing}
 before_actions {gropto} after_actions {} replace_actions {}
```

- To return a list of all the checkpoint behaviors defined in your configuration, specify the `-list_reports` or `-list_actions` option:

```
icc2_shell> get_checkpoint_data -list_reports
timing app_options

icc2_shell> get_checkpoint_data -list_actions
gropto placer_high_effort_congestion
```

- To return the contents and associations of a checkpoint report or action, specify the report or action with the `-report` or `-action` option:

```
icc2_shell> get_checkpoint_data -report app_options
contents {
   set name [get_current_checkpoint -name]
   set pos [get_current_checkpoint -position]

   report_app_options -non_default \
      > ./checkpoint/$name.$pos.app_options.rpt
} before_patterns {*} after_patterns {}
```

Feedback

See Also

- Viewing Your Checkpoint History

Viewing Your Checkpoint History

To view your checkpoint history, navigate to the checkpoint directory. This directory contains a checkpoint_history.rpt file that captures the history of the checkpoints the tool encounters during each run, along with the runtime and memory usage for each checkpoint. The checkpoint directory also contains any checkpoint reports you have written to it.

In some cases, you might want to clear the full or partial contents of your checkpoint history. A typical example is when a checkpointed run fails and you want to clear some or all of the data associated with that run, depending on whether you plan to rerun a portion of the flow or the entire flow.

- To clear your full checkpoint history, use the `reset_checkpoints` command.

  ```
  icc2_shell> reset_checkpoints
  ```

  The `reset_checkpoints` command clears the full contents of the checkpoint directory, including any reports you have written to it. However, before clearing the checkpoint directory, the `reset_checkpoints` command saves a timestamped copy of the directory in your run directory.

- To clear a portion of the data in your checkpoint_history.rpt file, use the `-from` option to specify the name of a checkpoint.

  This option removes all checkpoints including and following the specified checkpoint from your checkpoint_history.rpt file; it does not remove any checkpoint reports you have written to the checkpoint directory.

  For example, suppose your checkpoint_history.rpt file contains the following data after your run a full checkpointed flow:

  ```
  Checkpoints                Memory     StartTime            ...
  remove_buffers             32290.86   2020-03-23_14:34:17  ...
  place_opt_to_initial_drc   36389.15   2020-03-23_17:21:56  ...
  incr_placement             37002.09   2020-03-23_19:13:02  ...
  place_opt_to_initial_opto  45655.59   2020-03-23_23:44:36  ...
  place_opt_to_final_opto    46322.34   2020-03-23_30:12:55  ...
  ```

  Suppose you want to rerun your flow beginning with incremental placement. To clear the last three checkpoints from your checkpoint_history.rpt file before rerunning that portion of your flow, use the following command:

  ```
  icc2_shell> reset_checkpoints -from incr_placement
  ```

Feedback

After clearing the checkpoints, your checkpoint_history.rpt file lists only the first two checkpoints in your flow:

```
Checkpoints                 Memory     StartTime            ...
remove_buffers              32290.86   2020-03-23_14:34:17  ...
place_opt_to_initial_drc    36389.15   2020-03-23_17:21:56  ...
```

Similarly, the `get_checkpoint_data -list_names` command returns only the remove_buffers and place_opt_to_initial_drc checkpoints:

```
icc2_shell> get_checkpoint_data -list_names
remove_buffers place_opt_to_initial_drc
```

Using Setup Files

When you start the IC Compiler II tool, it automatically executes the commands in the .synopsys_icc2.setup file.

The tool looks for this file both in your home directory and in the project directory (the current working directory in which you start the IC Compiler II tool). The file is read in the following order:

1. The .synopsys_icc2.setup file in your home directory
2. The .synopsys_icc2.setup file in the project directory

The setup files can contain commands that perform basic tasks, such as initializing application options and setting GUI options. You can add commands and Tcl procedures to the setup files in your home and project directories. For example,

- To set application options that define your IC Compiler II working environment, create setup files in your home directory.
- To set project- or block-specific application options that affect the processing of a block, create a setup file in the design directory.

See Also

- Working With the 3DIC Compiler User Interfaces
- Using Application Options
- Using Variables
- Using Tcl Scripts

Using the Command Log File

The command log file records the commands processed by the IC Compiler II tool, including setup file commands and application option settings. By default, the IC Compiler II tool writes the command log to a file named icc2_command.log in the directory from which you invoked icc2_shell.

You can change the name of the command log file by setting the `sh_command_log_file` variable in your .synopsys_icc2.setup file. You should make any changes to this variable before you start the IC Compiler II tool. If your user-defined or project-specific setup file does not contain this variable, the IC Compiler II tool automatically creates the icc2_command.log file.

Each IC Compiler II session overwrites the command log file. To save a command log file, move it or rename it. You can use the command log file to

- Produce a script for a particular implementation strategy
- Record the physical implementation process
- Document any problems you are having

Enabling Multicore Processing

Several functions in the IC Compiler II tool support multicore processing, whether through multithreading, distributed processing, or parallel command execution. Multicore processing improves turnaround time by performing tasks in parallel much more quickly than if they were run sequentially on a single core.

Note:

A single machine has one or more CPUs and each CPU has one or more cores. The total number of cores available for processing on a machine is the number of CPUs multiplied by the number of cores in each CPU.

When using multicore processing, you need one IC Compiler II license for every eight parallel tasks. For example, to run 16 parallel tasks, you need 2 IC Compiler II licenses.

In most cases, you configure multicore processing by using the `set_host_options` command. The following topics describe how to use the `set_host_options` command to configure multicore processing:

- Configuring Multithreading
- Configuring Distributed Processing

Feedback

- Reporting Multicore Configurations
- Removing Multicore Configurations

The following topic describes how to use distributed processing to run the same task on several blocks in a hierarchical design:

- Running Tasks in Parallel

The following topic describes how to use parallel command execution for checking and reporting commands:

- Running Commands in Parallel on Your Local Host

Configuring Multithreading

Multithreading performs tasks in parallel by using multiple cores on the same machine, using a single process memory image. When using multithreading, each parallel task is called a thread. For the best performance during multithreading, you should limit the number of threads to the number of available cores, which is the number of CPUs in the machine times the number of cores per CPU.

The following commands support multithreading configured by the `set_host_options` command:

- `place_opt`
- `clock_opt`
- `check_legality`, when advanced legalization is enabled by setting the `place.legalize.enable_advanced_legalizer` application option to `true`.
- `insert_via_ladders`
- `route_auto`, `route_global`, `route_track`, `route_detail`, and `route_eco`

  Note:

  When you run routing with a single thread, the result is deterministic; if you start with the same block, you always get the same result. However, if you use multiple threads, the routing results are not deterministic; the final routing is slightly different between runs due to the varying division of tasks between threads. Global routing supports a deterministic mode for multicore routing. To enable this mode, set the `route.global.deterministic` application option to `on`.

- `route_opt`
- `signoff_check_drc`

Feedback

- signoff_fix_drc
- signoff_create_metal_fill
- signoff_fix_isolated_via
- write_def

By default, all commands use a single thread. To enable multithreading for those commands that support it, set the -max_cores option of the set_host_options command to a value greater than one and less than or equal to the number of cores available on your machine, which is the number of CPUs in the machine times the number of cores per CPU. The number of cores specified by the -max_cores option applies to all commands that support multithreading.

When you enable multithreading, multithreaded commands create and use the specified number of threads, even if the number is more than the number of available cores. You must set an appropriate number of threads, so that the command does not try to use more resources than it has. Overthreading can reduce performance because the extra threads compete for resources. For best performance, do not run more than one thread per available core.

For example, if your machine has two CPUs and each CPU has three cores, specify six as the maximum number of threads:

```
icc2_shell> set_host_options -max_cores 6
```

Configuring Distributed Processing

Distributed processing performs tasks in parallel by using multiple machines; each process uses its own process memory image. When using distributed processing, each parallel task is called a process. For the best performance during distributed processing, you should limit the number of processes to the number of available cores, which is the sum of the number CPUs times the number of cores per CPU for each host machine.

The following commands support distributed processing configured by the set_host_options command:

- analyze_rail
- create_placement -floorplan
- signoff_check_drc
- signoff_fix_drc
- signoff_create_metal_fill
- signoff_fix_isolated_via

Feedback

When you configure distributed processing, you can specify one or more of the following settings:

- The job submission command (the `-submit_command` option)

  If you do not specify this option, the tool uses the `rsh` command to submit the parallel processes.

- The list of host machines (the *`host_names`* argument)
- The maximum number of processes (the `-num_processes` option)

By default, the tool assigns a name to each configuration you define with the `set_host_options` command. To specify the configuration name, use the `-name` option. You use the configuration name to select the configuration to use for specific commands and to remove a configuration.

For example, to specify a distributed processing configuration that uses the qsub command to submit the parallel processes, use the following command:

```
icc2_shell> set_host_options -name dp_config \
   -submit_command [list qsub -P bnormal -cwd]
```

Reporting Multicore Configurations

To report the values set by the `set_host_options` command, use the `report_host_options` command. The generated report has the settings for multi-threaded and distributed processing created by the `set_host_options` command.

Removing Multicore Configurations

To remove the multicore configurations defined by the `set_host_options` command, use the `remove_host_options` command. To remove all multicore configurations, use the `-all` option. To remove a specific multicore configuration, specify the configuration name with the `-name` option.

Feedback

Running Tasks in Parallel

To efficiently run the same task on several blocks in your design, you can use the `run_block_script` command to enable distributed processing and perform the tasks in parallel. The `run_block_script` command accepts a Tcl script and applies the commands in the script to the blocks you specify.

To control the order in which blocks are processed, use the `-run_order` option with the `top_down` or `bottom-up` argument. When you specify the `-run_order top-down` option, the tool delays processing for a child block until all parent blocks for the child block are processed. When you specify the `-run_order bottom-up` option, the tool begins by processing the child blocks, then processes the parent blocks. By default, blocks are processed in a bottom-up order.

Running Commands in Parallel on Your Local Host

You can improve runtime by running checking and reporting commands in parallel on your local host. You can run the commands either in the background (with the `redirect -bg` command) or in the foreground (with the `parallel_execute` command). These techniques do not require the use of additional licenses beyond the licenses required for the parent run. They also do not require you to use or configure distributed processing.

When you use these techniques, consider the following guidelines:

- To reduce runtime and memory usage, run the `update_timing` command before running the commands in parallel; otherwise, each command that requires updated timing runs the `update_timing` command independently.
- To pass variables from a child process to the parent process, you must write the contents of the variables to a file during the child process, and then read that file in the parent process.

See Also

- Running Commands in the Background
- Running Commands in Parallel

Running Commands in the Background

To improve runtime, you can run checking and reporting commands in the background while you run other commands in the foreground. This is useful in interactive sessions when you want to continue your work while the tool generates a report.

To run commands in the background, use the `redirect` command with the `-bg` option. When you use this command, icc2_shell returns immediately to execute the next

Feedback

command. If you issue an `exit` command in the parent process, the tool waits for all `redirect -bg` commands to complete before exiting.

To list the commands supported by the `redirect -bg` command, use the `list_commands -bg` command. You can run either a single command or source a Tcl script that contains only supported commands. If the command or script includes a `redirect -bg` command, the `-bg` option is ignored.

You can run at most two jobs in the background. If you specify more than two background jobs, they are queued.

To specify the maximum number of cores to use for the background jobs, use the `-max_cores` option with the `redirect` command. The number of cores available for the parent process (as specified by the `-max_cores` option of the `set_host_options` command) is reduced by the number of cores allocated for background jobs.

The following example redirects a Tcl script to run in the background:

```
icc2_shell> set_host_options -max_cores 8
icc2_shell> redirect -bg -max_cores 3 -file bg_log.out \
   {source bg_script.tcl}
Information: redirect -bg with max_cores 3 started. The maximum number of
 cores available in parent is reduced to 5. (BGE-004)
```

Reporting Background Jobs

To report the background jobs submitted with the `redirect -bg` command, use the `report_background_jobs` command. This command reports both the completed jobs and the jobs currently running in the background, as shown in the following example:

```
icc2_shell> report_background_jobs
JOB 'redirect -bg  -file {background.log} source bg_script.tcl
-max_cores 4 ' completed
JOB(pid:13010) 'redirect -bg  -file {background_1.log}
source bg_script_1.tcl  -max_cores 3 ' is running
```

To omit the completed jobs, use the `-reset` option with the `report_background_jobs` command.

Feedback

Running Commands in Parallel

To improve runtime, you can run checking and reporting commands in parallel and return to the parent process after the longest running command in the parallel execution list completes. This is useful in batch scripts.

To run commands in parallel, use the `parallel_execute` command, as shown in the following example:

```
icc2_shell> update_timing
icc2_shell> parallel_execute {
   report_cmd1 log_file1
   report_cmd2 log_file2
   report_cmd3 log_file3
   ...
}
```

To list the supported commands, use the `-list_allowed_commands` option with the `parallel_execute` command.

To specify the maximum number of cores to use when running the `parallel_execute` command, use the `-max_cores` option. If you do not use this option, the tool uses the value of the `-max_cores` option from the `set_host_options` command. If you do not specify the maximum number of cores with either command, the tool runs the commands sequentially instead of in parallel.

To run commands in parallel as a background job, use the `redirect -bg` command to run the `parallel_execute` command, as shown in the following example:

```
icc2_shell> redirect -bg -max_cores 3 -file bg_log.out {
   parallel_execute {
      report_cmd1 log_file1
      report_cmd2 log_file2
   }
}
```

2

Preparing the Design

The IC Compiler II tool uses a design library to store your design and its associated library information. This topic describes how to create a design library and how to prepare and save your design.

These steps are explained in the following topics:

- Defining the Search Path
- Setting Up Libraries
- Using Pin Access Checker Utility
- Working With Designs
- Importing the Floorplan Information
- Setting Up Multivoltage Designs
- Specifying Timing Constraints and Settings
- Specifying Logical Design Rule Constraints
- Controlling Clock-Gate Latencies
- Specifying Physical Constraints for Placement and Legalization
- Specifying Placement Settings
- Specifying Legalization Settings
- Controlling the Optimization of Cells, Nets, Pins, and Ports
- Specifying Settings for Preroute Optimization
- Setting Up for Power-Related Features
- Specifying the Routing Resources
- Handling Design Data Using the Early Data Check Manager
- Applying Mega-Switch Command Settings

Feedback

Defining the Search Path

The IC Compiler II tool uses a search path to look for files that are specified with a relative path or with no path.

To specify the search path, use the `set_app_var` command to set the `search_path` application variable to the list of directories, in order, in which to look for files. When the tool looks for a file, it starts searching in the leftmost directory specified in the `search_path` variable and uses the first matching file it finds.

You can also use the Tcl `lappend` command to add your directories to the default search path, which is the directory from which you invoked the tool. For example,

```
icc2_shell> lappend search_path ./mylibdir
```

Setting Up Libraries

A *block* is a container for physical and functional design data. A *design library* is a collection of related blocks, together with technology data that applies to the block collection. A *chip design* consists of one or more blocks, often stored in different design libraries. A design library uses instances of blocks defined in lower-level libraries, called *reference libraries*. A design library can serve as a reference library for another design library.

To learn about setting up libraries, see the following topics:

- Working With Design Libraries
- Setting Up Reference Libraries
- Library Configuration
- Restricting Library Cell Usage
- Restricting the Target Libraries Used

Working With Design Libraries

You can create, open, query, save, or close a design library, using an absolute path, a relative path, or no path, by using the following commands:

- `create_lib`

  This command creates the library in memory and sets it as the current library. When you run this command to create a new design library, you must specify the library

Feedback

name. Slash (/) and colon (:) characters are not allowed in library names. The following command creates the my_libA library using a relative path:

```
icc2_shell> create_lib ../my_lib_dir/my_libA
{my_libA}
```

- `open_lib`

  This command opens the specified library, makes that library the current library, and opens all its associated reference libraries. Opening a library means loading it into memory and making its blocks accessible. The following example opens the my_libA library saved on disk:

```
icc2_shell> open_lib my_libA
Information: Loading library file '/usr/lib/StdCells.ndm' (FILE-007)
Information: Loading library file '/usr/lib/RAMs.ndm' (FILE-007)
Information: Loading library file
 '/usr/lib/PhysicalOnly.ndm' (FILE-007)
{my_libA}
```

- `current_lib`

  By default, the library most recently opened is the current library. You can explicitly set any open library to be the current library by using the `current_lib` command. For example,

```
icc2_shell> current_lib my_libA
{my_libA}
```

- `save_lib`

  When you create or change a library, the changes are stored in memory only. To save a library to disk, use this command. For example,

```
icc2_shell> save_lib lib_A
Saving library 'lib_A'
1
```

- `close_lib`

  When you no longer need access to data in a library, you can close it by using the `close_lib` command. Be sure to save the changes in the library before you close it. For example,

```
icc2_shell> close_lib
Closing library 'lib_A'
1
```

In addition, you can use the `current_lib`, `get_libs`, and `report_lib` commands to query design libraries.

Feedback

For more information, see the Design Libraries topic in the *IC Compiler II Data Model User Guide*.

Setting Up Reference Libraries

You can specify a reference library list for a design library when you create the design library by using the `-ref_libs` option of the `create_lib` command. You can also change the reference library list at any time by using the `set_ref_libs` command.

Use the following commands to specify, rebind, and report reference libraries:

- `create_lib -ref_libs`

  You can specify a relationship between a new design library and its lower-level reference libraries by using the `create_lib` command. For example,

```
icc2_shell> create_lib lib_B \
   -ref_libs {../LIBS/lib_c ../STND/stdhvt.ndm} ...
{lib_B}
```

- `set_ref_libs -ref_libs`

  For an existing design library, open the library and then use the `set_ref_libs` command to specify the reference libraries. For example,

```
icc2_shell> current_lib
{lib_B}
icc2_shell> set_ref_libs \
   -ref_libs {../LIBS/lib_C ../STND/stdhvt.ndm}
../LIBS/lib_C ../STND/stdhvt.ndm
```

- `report_ref_libs`

  To report the reference libraries of a design library, use the `report_ref_libs` command.

  For example,

```
icc2_shell> create_lib lib_A -ref_libs \
   {../libs/SCLL.ndm ../libs/SCHH.ndm ../BLOCKS/MACROS}
{lib_A}
icc2_shell> report_ref_libs...
Name Path Location
-----------------------------------------------------------------
*+ SCLL ../libs/SCLL.ndm /remote/project/libs/SCLL.ndm
*+ SCHH ../libs/SCHH.ndm /remote/project/libs/SCHH.ndm
* MACROS ../BLOCKS/MACROS /remote/project/BLOCKS/MACROS
"*" = Library currently open
"+" = Library has technology information
```

- `set_ref_libs -rebind`

Feedback

When you make a change that invalidates the reference library list, such as moving a reference library to a new location, you need to rebind the reference libraries. To do so, use the `-rebind` option, which rebinds each reference library path specified by the `search_path` variable to libraries that are currently loaded in memory. For example,

```
icc2_shell> current_lib
{lib_A}
icc2_shell> set_app_var search_path {. ../REFLIBS ../CLIBS}
. ../LIBS ../BLOCKs
icc2_shell> set_ref_libs -rebind
../REFLIBS/lib_C ../REFLIBS/lib_D ../CLIBS/stdhvt.ndm}
```

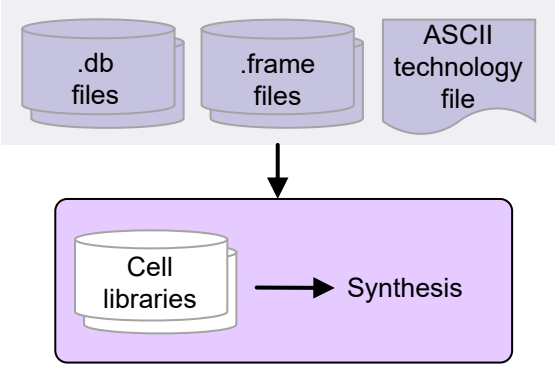
Rebinding a library does not affect the bindings of blocks already existing in the design library. To rebind these blocks using an updated reference library list, use the `-rebind` option with the `link_block` command.

Library Configuration

Library configuration allows you to specify which vendor libraries to use as reference libraries for the current design. You specify the technology file, physical libraries, and logic libraries by using the `search_path` and `link_library` variables, and then you use the `create_lib` or `set_ref_libs` command to assemble the cell libraries.

During library configuration,

- The IC Compiler II tool automatically calls the Library Manager tool without user intervention to generate cell libraries, as shown in the following figure:



- The tool saves the generated cell libraries to disk and adds them to the reference library list of the design library.
- These cell libraries are the same as when the cell libraries are created during library preparation in the Library Manager tool.

Feedback

For more information, see the Configuring Cell Libraries topic in the *IC Compiler II Data Model User Guide*.

Restricting Library Cell Usage

By default, the IC Compiler II tool can use all of the library cells available in the cell libraries when performing optimization or clock tree synthesis on the block. To restrict the cell usage, use the `set_lib_cell_purpose` command after the block is in memory. The command has a block scope and is persistent across tool sessions. The specified settings are restored when the block is reopened.

To specify how the tool can or cannot use library cells, use the `-include` and `-exclude` options with the `set_lib_cell_purpose` command. Both of these options accept one or more of the following values: `all`, `cts`, `hold`, `optimization`, `none`.

- The `-include` option sets the `included_purposes` attribute on the specified library cells.
- The `-exclude` option sets the `excluded_purposes` attribute on the specified library cells.

Note:

If a library cell has a `dont_use` attribute, it is excluded from all uses, which is the same as if you specified `set_lib_cell_purpose -include none` for that cell.

When the tool performs optimization, which includes setup, hold, and logical DRC fixing, it can use library cells that have an included purpose of `optimization`, `hold`, or both. When the tool performs clock tree synthesis, it can use library cells that have an included purpose of `cts`.

For example, to disallow the use of a set of library cells for all uses, use the following command:

```
icc2_shell> set_lib_cell_purpose -include none lib_cells
```

To allow a set of library cells to be used only for clock tree synthesis, use the following commands:

```
icc2_shell> set_lib_cell_purpose -include none lib_cells
icc2_shell> set_lib_cell_purpose -include cts lib_cells
```

To allow a set of library cells to be used for all uses except clock tree synthesis, use the following command:

```
icc2_shell> set_lib_cell_purpose -exclude cts lib_cells
```

Feedback

Restricting the Target Libraries Used

By default, the tool can select any library cell from the target library during optimization. In some cases, you might want to restrict the library cells used for clock tree synthesis and optimization. For example, you might want to exclude specific double-height cells from the target library for some design blocks during optimization.

To restrict the library cells used for clock tree synthesis and optimization,

1. Specify the library subset by using the `set_target_library_subset` command.

   By default, the library subset restriction applies to

   - The top block and all its subblocks

     To set it for a specific subblock, use the `-objects` option.

   - Both clock and data paths.

     To set it for only the clock or data paths, use the `-clock` or `-data` option.

   You can further restrict the target library subset setting as follows:

   - Specify a list of cells from the libraries that should not be used by using the `-dont_use` option.
   - Specify that these libraries cannot be used for any other objects, other than the specified objects, by using the `-only_here` option.

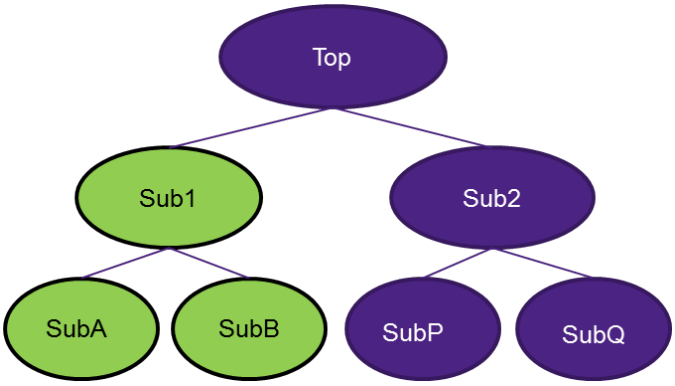
2. Enable the use of the target library subset by setting the `opt.common.enable_target_library_subset_opt` application option to 1.

When you set target library subsets, remember the following points:

- The subset restriction applies to hierarchical cells but not to leaf cells.
- The command enforces the subset restriction on the specified blocks and their subdesigns in the hierarchy, except those subdesigns where a different subset restriction is set.
- A subset specified at a lower level supersedes any subset specified at a higher level.

Feedback

Figure 6 Logic Hierarchy of Design



For example, assume your design has a logic hierarchy as shown in Figure 6 and you want to implement the following library restrictions during optimization and clock tree synthesis:

- Use only the cells from the library named LVT_lib for the Sub1 block and its subblocks, SubA and SubB.
- Do not use the cells from this library anywhere else in the design.

To do so, use the following settings:

```
icc2_shell> set_target_library_subset -objects {top/Sub1} \
   -only_here [get_lib_cells LVT_lib/*] [get_libs LVT_lib]
icc2_shell> set_app_options \
   -name opt.common.enable_target_library_subset_opt -value 1
```

In addition to these settings, assume you specify the following setting:

```
icc2_shell> set_lib_cell_purpose -include cts \
   {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
```

Then, when adding buffers on the clock network during clock tree synthesis, the tool uses

- The buf1 and buf2 cells from the LVT_lib library for the block named Sub1 and its subblocks
- The buf1 and buf2 cells from the HVT_lib library for the rest of the design

Reporting Target Library Subsets

To find out which target library subsets have been defined for a top block or hierarchical cell, use the `report_target_library_subset` command.

Reports that are generated by reporting commands, such as `report_cells` and `report_timing`, show the `td` attribute attached to the cells that are specified by the `-dont_use` or `-only_here` option.

Feedback

Removing Target Library Subsets

To remove a target library subset restriction from a top block or hierarchical cell, use the `remove_target_library_subset` command.

Using Pin Access Checker Utility

The pin access checker (PAC) utility helps you to identify the pin accessibility issues in the library development stage. This helps you to enhance the cell layout as you cannot modify the cell layout later, after the library is finalized and released to the Place and Route group.

The routing DRCs identify the pin access issues after routing. When the cells are abutted or placed too close, two or multiple adjacent pins compete for common routing resources and causes routing DRCs.

The benefits of PAC utility are listed as follows:

- Produces a place and route database with various permutations of abutted cell placement.
- Generates a place and route database and assesses the ability to place and pin accessibility of the library created based on the reports.

The PAC performs various checks listed as follows:

- Placement of cells
- Number of accessible tracks for each library cell's pins
- Legalization check after placement
- Design rule check after routing
- All the earlier listed checks with PG routes

You can use the `create_pin_check_lib` command to create or open a new library based on the input technology and reference libraries. PAC creates and stores blocks in this library. You must provide minimum three inputs by using the following application options with the `create_pin_check_lib` command:

- Technology file: Use the `-technology` option to provide the technology file information.
- Reference library: Use the `-ref_libs` option to provide the reference library information.
- Routing direction for metal layers: Use the `set_attribute [get_layers Mx] routing_direction` **`horizontal/vertical`** command to define routing direction in Tcl script. You must specify the name of the script using the `pin_check.place.preplace_option_file` application option.

Feedback

To enable the pin access check utility, you can use the `check_libcell_pin_access` command. You can use this command and the `pin_check.*` application option to enable the following pin access checks:

- Legalization
- Router
- Automatic derivation of cell spacing rule

Use the `-mode` option with the `check_libcell_pin_access` command to perform multiple checks by specifying different modes.

```
create_pin_check_lib myTest.nlib -ref_libs ../lib/refLib.ndm
 -technology ../tf/refTech.tf

set_app_options -as_user_default -name
 pin_check.place.preplace_option_file -value

check_libcell_pin_access -mode analyze_lib_cell

check_libcell_pin_access -mode analyze_lib_pin

check_libcell_pin_access -mode design_under_test
```

If you have been using the earlier Tcl based version, you can translate your old scripts into new IC Compiler II commands by using the `translate_pin_check_ui` command.

```
translate_pin_check_ui -from old_script.tcl -to new_script.tcl
```

To enable reporting pin access by router, set the `pin_check.analyze.analyze_lib_pin_check_routability` application option to `true`. When set to `false`, it generates report of pin access by the `report_cell_pin_access` command.

Working With Designs

In most cases, when you work with a design in the IC Compiler II tool, you use the design view of one of its versions (blocks) from the design library. In this manual, the term *block* is used to refer to a specific design version in the design library, whether it is a top-level design or a subdesign, and the term *design* is used generically to refer to the design being processed. In both cases, unless otherwise specified, the term refers to the design view.

Feedback

The IC Compiler II tool reads designs in Verilog format. The Verilog netlist file is a structural or gate-level design in one file. To read a design into the IC Compiler II tool,

1. Create or open the design library associated with the design.
   - If the design library does not yet exist, use the `create_lib` command to create it.
   - If the design library already exists, use the `open_lib` command to open it.
2. Read the Verilog netlist files for the design by using the `read_verilog` command.

   By default, when the tool reads the Verilog netlist files, it creates a block in the current design library and increments its open count. The tool determines the top-level module of the block by identifying the module that is not instantiated by any other modules in the specified Verilog files and uses the top-level module name as the block name.

   The tool also creates a default power domain for the block. For multivoltage designs, this default power domain is replaced when you specify the power intent, as described in Loading and Applying UPF Information.

Use the following commands to work with blocks:

- `create_block`: Creates a block
- `open_block`: Opens an existing block
- `current_block`: Sets or reports the current block
- `save_block`: Saves a block
- `close_blocks`: Closes a block

See Also

- Blocks
- Working With Design Libraries

Importing the Floorplan Information

A floorplan contains physical constraints such as the core area and shape, port locations, macro locations and orientations, and so on, which are required for performing physical synthesis and optimization.

Feedback

If you have a floorplan for your block, read in the floorplan as a DEF file, as described in Reading DEF Files.

If you do not have a floorplan for your block, you can perform design planning and generate a floorplan as described in the *IC Compiler II Design Planning User Guide*.

Reading DEF Files

To read the floorplan information from a DEF file, use the `read_def` command.

```
icc2_shell> read_def block.def
```

Note:

When possible, use DEF v5.8 or later, as this version supports more types of physical objects and obstructions than previous versions.

By default, the `read_def` command

- Annotates the floorplan information onto the current block

  To annotate the information onto a different block, use the `-design` option to specify the block name.

- Preserves the existing floorplan information

  In incremental mode,

  - The placement area is imported based on the current core area and site rows in the DEF files
  - Physical constraints that can have only one value are overwritten by the value from the latest DEF file; for example, port location and macro location are overwritten.
  - Physical constraints that can have accumulated values are recomputed; that is, core area can be recomputed based on the existing value and the site row definitions in the latest DEF file. Placement keepouts from different DEF files are accumulated and the final keepout geometry is computed internally during synthesis.

  To remove the existing floorplan information before annotating the floorplan information from the DEF file, use the `-no_incremental` option. In this mode, the placement area is imported based on the site rows in the DEF files.

- Uses rule-based name matching for macros and ports

Feedback

Rule-based name matching automatically resolves name differences by using the tool's intelligent name matching capability. By default, when rule-based name matching is enabled, the following characters are considered equivalent:

- Hierarchical separators { / _ . }

  For example, a cell named a.b_c/d_e is automatically matched with the string a/b_c.d/e in the DEF file.

- Bus notations { [ ] __ ( ) }

  For example, a cell named a [4] [5] is automatically matched with the string a_4__5_ in the DEF file.

To disable rule-based name matching and require exact name matches between the DEF file and the block, set the `file.def.rule_based_name_matching` application option to `false`.

For more information, see "Rule-Based Name Matching" in the *IC Compiler II Data Model User Guide*.

- Ignores any objects in the DEF file that do not exist in the block, except for PG objects

  To allow new non-PG objects to be created from the DEF file, use the `-add_def_only_objects` option to specify the types of objects to create. Specify one or more of the following keywords:

  - `cells`

    The tool creates the cells that exist only in the DEF file and connects their power and ground pins as defined in the DEF file; it does not connect the signal, clock, or tie pins even if these connections are defined in the DEF file. The tool also does not create new hierarchical cells; any hierarchy specified in the DEF file must already exist in the block.

  - `nets`

    The tool creates the signal, clock, and tie nets that exist only in the DEF file and connects them to the ports specified in the DEF PINS section; it does not connect the nets to any other ports or pins in the netlist even if these connections

Feedback

are defined in the DEF file. The tool does not create new hierarchical nets; any hierarchy specified in the DEF file must already exist in the block.

- ports

  The tool creates the signal, clock, and tie ports that exist only in the DEF file and connects them to the nets specified in the DEF PINS section.

- all

  The tool creates the non-PG cells, nets, and ports that exist only in the DEF file, as if you had specified cells, nets, and ports.

Fixing Site Name Mismatches

If the site names used in the DEF file do not match the site names defined in the technology file, use the -convert_sites option to specify the site name mapping. For example, if the DEF file uses a site named CORE, but the technology file defines only a site named unit, use the following command to convert the site names when reading the DEF file:

```
icc2_shell> read_def -convert_sites { {CORE unit} } block.def
```

Validating DEF Files

To analyze the input DEF files before annotating the floorplan information on the block, enable check-only mode by using the -syntax_only option. The check-only mode provides diagnostic information about the correctness and integrity of the DEF file. The check-only mode does not annotate any floorplan information onto the block.

```
icc2_shell> read_def -syntax_only block.def

icc2_shell> check_duplicates -remove
```

The check_duplicates -remove command ensures that no duplicate shapes and vias are present in the block due to the execution of the read_def command.

Physical Constraints Extracted From the DEF File

The read_def command extracts physical constraint information from DEF files and annotates it on the block. However, only the following physical constraints are extracted and annotated:

- Placement Area
- Port Locations
- Cell Locations
- Placement Blockages

Feedback

- Site Rows
- Routing Tracks
- Placement Bounds
- Routing Blockages
- Preroutes

To visually inspect the extracted physical constraints, use the layout view in the GUI. All physical constraints extracted from the DEF file are automatically added to the layout view.

Placement Area

Placement area is computed based on the site array information.

Port Locations

For each port with the location specified in the DEF file, the tool sets the location on the corresponding port in the block.

Note:

If the DEF file does not contain port-location information, the tool inherits the port locations from the locations of the pad cells, as described in .

Example 1 DEF Port Location Information

```
PINS 2 ;
    -Out1 + NET Out1 + DIRECTION OUTPUT + USE SIGNAL +
        LAYER M3 (0 0) (4200 200) + PLACED (80875 0) N;
    -Sel0 + NET Sel0 + DIRECTION INPUT + USE SIGNAL +
        LAYER M4 (0 0)(200 200) + PLACED (135920 42475) N;
END PINS
```

Ports with changed names and multiple layers are supported.

Example 2 DEF Port Locations With Changed Names and Multiple Layers

```
PINS 2 ;
    - sys_addr\[23\].extra2 + NET sys_addr[23] + DIRECTION INPUT +USE
 SIGNAL
    + LAYER METAL4 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
    - sys_addr[23] + NET sys_addr[23] + DIRECTION INPUT + USE SIGNAL +
 LAYER
    METAL3 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
END PINS
```

Example 3 DEF Port Orientation Information

```
PINS 1;
    - OUT + NET OUT + DIRECTION INPUT + USE SIGNAL
        + LAYER m4 ( -120 0 ) ( 120 240 )
```

Feedback

```
        + FIXED ( 4557120 1726080 ) S ;
END PINS
```

Cell Locations

For each cell with a location and the FIXED attribute specified in the DEF file, the tool sets the location on the corresponding cell in the block. Example 4 shows DEF macro location and orientation information, where the letters E and W denote east rotation and west rotation respectively.

Example 4 DEF Cell Location Information

```
COMPONENTS 2 ;
    - macro_cell_abx2 + FIXED ( 4350720 8160 ) E ;
    - macro_cell_cdy1 + FIXED ( 4800 8160 ) W ;
END COMPONENTS
```

Placement Blockages

The `read_def` command imports hard, soft, and partial placement blockages defined in the DEF file.

Note:

DEF versions before version 5.7 did not support partial blockages. In addition, if your floorplanning tool creates a DEF file with DEF version 5.6, you need to manually add the `#SNPS_SOFT_BLOCKAGE` pragma to specify a soft blockage, as shown in Example 7.

Example 5 DEF Hard Placement Blockage Information

```
BLOCKAGES 50 ;
...
    - PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 )
...
END BLOCKAGES
```

Example 6 DEF Version 5.7 Soft Placement Blockage Information

```
BLOCKAGES 50 ;
...
- PLACEMENT + SOFT RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

Example 7 DEF Version 5.6 Soft Placement Blockage Information

```
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 ) ; #SNPS_SOFT_BLOCKAGE
...
END BLOCKAGES
```

Feedback

Example 8 DEF Partial Placement Blockage Information

```
BLOCKAGES 50 ;
...
- PLACEMENT + PARTIAL 80 RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

Site Rows

Site row information in the DEF file defines the placement area.

Example 9 DEF Site Row Information

```
ROW ROW_0 core 0 0 N DO 838 BY 1 STEP 560 0;
```

Routing Tracks

The track information in the DEF file defines the routing grid for designs based on standard cells. This information can be used during routing, and track support can enhance congestion evaluation and reporting to make it match more closely with the routing results.

Example 10 DEF Routing Track Information

```
TRACKS X 330 DO 457 STEP 660 LAYER METAL1 ;
TRACKS Y 280 DO 540 STEP 560 LAYER METAL1 ;
```

Placement Bounds

If REGIONS defining bounds exist in the DEF file, the `read_def` command imports those placement bounds. Also, if any cells in the related GROUP are attached to the region, fuzzy cell matching occurs between these cells and the ones in the block.matched cells are attached to the bounds in the following ways:

- If there are regions in the block with the same name as in the DEF, the cells in the related group are attached to the region by the `add_to_bound` command in incremental mode.
- If the region does not exist in the block, it is created with the same name as in the DEF file by applying the `create_bound` command; matched cells in the related group are also attached.

Example 11 DEF Placement Bound Information

```
REGIONS 1 ;
- c20_group ( 201970 40040 ) ( 237914 75984 ) + TYPE FENCE ;
END REGIONS
GROUPS 1 ;
- c20_group
     cell_abc1
     cell_sm1
     cell_sm2
```

Feedback

```
    + SOFT
    + REGION c20_group ;
END GROUPS
```

Routing Blockages

Routing blockages are extracted similar to the way that placement blockages are extracted.

Preroutes

The tool extracts preroutes that are defined in the DEF file.

Example 12 DEF Preroute Information

```
SPECIALNETS 2 ;
- vdd
 + ROUTED METAL3 10000 + SHAPE STRIPE ( 10000 150000 ) ( 50000 * )
 + USE POWER ;
...
END SPECIALNETS
```

Setting Up Multivoltage Designs

The following topics describe the tasks you need to perform when setting up multivoltage designs:

- Applying the Multivoltage Power Intent
- Preparing the Power Network
- Defining Voltage Areas
- Inserting Multivoltage Cells
- Controlling the Placement of Multivoltage Cells
- Enabling Improved Buffering for Multivoltage Nets
- Analyzing Multivoltage Information

Applying the Multivoltage Power Intent

To learn about applying the multivoltage power intent, see

- Loading and Applying UPF Information
- Specifying UPF Constraints for Physical-Only Cells
- Saving UPF Information

Feedback

Loading and Applying UPF Information

To load the power intent and apply it to a multivoltage design,

1. Read the UPF file by using the `load_upf` command.

   ```
   icc2_shell> load_upf block.upf
   ```

2. If you are using the golden UPF flow and have a name-mapping file from the Design Compiler tool, read the file by using the `read_name_map` command.

   ```
   icc2_shell> read_name_map block.nmf
   ```

3. (Optional) Verify the UPF consistency and identify any PG conflicts among the netlist, floorplan, and power intent.

   To verify the UPF consistency and identify PG conflicts, use the `resolve_pg_nets -check_only` command. This command identifies any issues and reports the changes that are made to resolve these issues when you commit the power intent. If you prefer to resolve the issues differently, you can use manual editing commands to resolve the issues before running the `commit_upf` command.

4. Commit the power intent by using the `commit_upf` command.

   ```
   icc2_shell> commit_upf
   ```

   The `commit_upf` command performs global checks for UPF consistency; resolves PG conflicts among the netlist, floorplan, and UPF specification; and associates power strategies with existing multivoltage cells. For more information about associating power strategies with existing multivoltage cells, see Associating Power Strategies With Existing Multivoltage Cells.

5. Report the associations made for the multivoltage cells by using the `report_mv_path` command.

   If the tool failed to associate any multivoltage cells, the command reports the causes for these failures. You must successfully commit the power intent before you continue with the design flow.

Note:

After successfully running the `commit_upf` command, the tool issues an error message if you try to use additional UPF commands, except for the `set_related_supply_net`, `connect_supply_net`, `set_design_attributes`, `set_port_attributes`, `find_objects`, and `set_scope` commands. To modify the power intent after running the `commit_upf` command, you must remove the existing UPF specification by using the `reset_upf` command and then reload the power intent.

Feedback

Specifying UPF Constraints for Physical-Only Cells

The UPF constraints for physical-only cells can cause issues when the UPF file is read into other tools that do not support physical-only cells, such as verification tools. Therefore, surround the UPF constraints for physical-only cells with the syntax shown in the following example:

```
if {[info exists snps_handle_physical_only] && \
                            $snps_handle_physical_only} {

  /* Supply net connections for filler cells */
   connect_supply_net VDDS -ports [get_pins FILLER*/VDD]

}
```

By default, the `snps_handle_physical_only` variable is set to `true` in the IC Compiler II tool. Therefore, when you load and commit the UPF file, the constraints are applied to the physical-only cells.

When you save a UPF file with the `save_upf` command, the tool uses the same syntax for the user-specified and tool-derived UPF constraints for physical-only cells. Therefore, these UPF constraints are ignored by any tool that does not have the `snps_handle_physical_only` variable set to `true`.

To create a UPF file that contains commands only for specific types of cells, use the `-include` or `-exclude` option with the `save_upf` command. These options accept arguments such as `diode_cells`, `pad_cells`, and `physical_only_cells`. See the `save_upf` command man page for the complete list of arguments.

Command filtering for specified cell types applies to the `connect_supply_net`, `set_related_supply_net`, `create_power_domain`, and `set_isolation` commands. You can apply command filtering to both block-level and full-chip UPF files.

Saving UPF Information

During physical implementation, the tool updates the power intent of the design. To generate an updated UPF file, use the `save_upf` command. You can use this UPF file in the subsequent steps of the design flow.

The `save_upf` command separates the UPF commands based on the input UPF file, and adds a comment indicating the input UPF file and the time it was loaded.

Assume you load, commit, and save UPF information as shown in the following example script:

```
load_upf top_1.upf
load_upf top_2.upf
load_upf top_3.upf
commit_upf
save_upf top.upf
```

Feedback

The resulting top.upf file contains the following information:

```
## Start - load_upf top_1.upf on Wed Oct 26 17:03:18 2016
…
…
## End - load_upf
## Start - load_upf top_2.upf on Wed Oct 26 17:03:49 2016
…
…
## End - load_upf
## Start - load_upf top_3.upf on Wed Oct 26 17:04:22 2016
…
…
## End - load_upf
```

Preparing the Power Network

To learn about preparing the power network for physical implementation, see

- Creating Logical Power and Ground Connections
- Creating Floating Logical Supply Nets

Creating Logical Power and Ground Connections

After you read in the design, you must ensure that there are logical connections between the power and ground nets and the power, ground, and tie-off pins on the cells in your design. If your design does not already have these connections, use the `connect_pg_net` command to create them. This command creates the logical power and ground connections for leaf cells, hierarchical cells, and physical-only cells in both single-voltage and multivoltage designs.

Before creating the logical power and ground connections, you must resolve any PG conflicts among the netlist, floorplan, and UPF specification.

- For multivoltage designs, the conflicts are resolved when you commit the power intent, as described in Loading and Applying UPF Information.
- For single-voltage designs, you must run the `resolve_pg_netx` command to resolve the conflicts. Note that the UPF specification for a single-voltage design is the default power domain generated by the `read_verilog` command.

If your design contains unmapped instances, the tool issues an information message to indicate that only mapped instances are connected.

Feedback

The `connect_pg_net` command operates in two modes:

- Automatic

  In automatic mode, the command derives all power and ground nets, power and ground pins, tie-off pins, and connections from the UPF specification. If the supply nets do not exist, the tool creates them.

  To create the logical power and ground connections in automatic mode, use the `-automatic` option with the `connect_pg_net` command.

- Manual

  In manual mode, the command makes the connections that you specify. If a specified pin or port has an existing connection, the tool removes the existing connection and then creates the specified connection. The tool verifies that the connections match the power intent. If it finds a mismatch, the tool issues a warning but still creates the connection.

  To create logical power and ground connections in manual mode, use the `-net` option to specify the power or ground net and specify the pins and ports to be connected to that net as an argument to the command. For example, to connect the VDD power net to all pins named vdd and the VSS ground net to all pins named vss, use the following commands:

```
icc2_shell> connect_pg_net -net VDD [get_pins */vdd]
icc2_shell> connect_pg_net -net VSS [get_pins */vss]
```

  To connect PG nets to power and ground pins only, use the `-pg` option. To connect PG nets to tie-off pins only, use the `-tie` option. By default, if neither option is specified, the tool makes connections to all power, ground, and tie-off pins. These options cannot be used with an object list or with the `-create_nets_only` and `-net` options.

  Regardless of the command options used, the tool always creates connections required to build a complete PG netlist structure, such as top-level PG ports and intermediate hierarchical PG pins.

Feedback

Creating Floating Logical Supply Nets

A floating logical supply net is a power or ground net that is not connected to any logical pin or port in the block. For example, a power feedthrough net created during power planning is a floating supply net.

To create a floating supply net with the `connect_pg_net` command, use the following steps:

1. Specify the supply net type in the UPF file by using the `supply_net_pg_type` attribute, as shown in the following UPF file example:

```
…
create_supply_net VDD
…
…
set_design_attributes -elements VDD \
   -attribute supply_net_pg_type power
…
```

2. Enable floating supply nets by setting the `mv.pg.create_floating_pg` application option to `true` before you create supply nets with the `connect_pg_net` command, as shown in the following script example:

```
…
…
# Apply the UPF
load_upf  $upf_input_file_name
commit_upf
…
…
# create logical supply nets
set_app_options  -as_user_default \
    -name mv.pg.create_floating_pg -value true
connect_pg_net -automatic
…
check_mv_design
```

The tool creates the logical supply nets in the topmost hierarchy of the current UPF scope for domain-independent supply nets. For domain-dependent supply nets, the tool creates the logical supply nets in the topmost hierarchy of the domain.

The tool does not create a logical supply net in the following situations:

- There is a conflict between the connection and the supply net type specified with the `supply_net_pg_type` attribute.
- The supply net is connected to a PG pin of an instance.
- The `connect_pg_net` command is not specified in the current or parent physical hierarchy.

Feedback

Defining Voltage Areas

A voltage area is a physical placement area for the cells associated with a power domain. For multivoltage designs, the power domains are defined in the UPF specification. For single-voltage designs, the tool creates a default power domain when you read the Verilog netlist and associates it with a default voltage area, which comprises the core area of the block.

The placer treats a voltage area the same as an exclusive move bound; it must place the cells in a voltage area within a specified region and it must place all other cells outside of the voltage area. Voltage areas can be rectangular or rectilinear. In addition, they can be disjoint, nested, or overlapping. For overlapping voltage areas, the effective shape of each voltage area is determined by the stacking order of the voltage area shapes.

To define a voltage area, use the `create_voltage_area` command. When you define a voltage area, at a minimum, you must specify the power domains associated with the voltage area. To specify the power domains, use the `-power_domains` option. You can specify one or more power domains; however, all specified power domains must have the same primary supply net. If you specify a single power domain, the name of the voltage area is derived from the name of the power domain. If you specify multiple power domains, you must specify a name for the voltage area by using the `-name` option.

A voltage area consists of one or more rectangular or rectilinear shapes, which can be abutted, disjoint, or overlapping. To define the boundaries of these shapes, use the `-region` option with the `create_voltage_area` command (if you are creating a new voltage area) or the `create_voltage_area_shape` command (if you are adding shapes to an existing voltage area). Note that you can specify one or more shapes when using the `create_voltage_area` command, but only a single shape in each `create_voltage_area_shape` command.

- To specify the boundary of a rectangle shape, use the following format to specify its lower-left and upper-right corners:

  ```
  {{llx lly} {urx ury}}
  ```

- To specify the boundary of a rectilinear shape, use the following format to specify the coordinates of its vertices:

  ```
  {{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
  ```

If a voltage area consists of multiple abutting or overlapping shapes, you can merge the shapes into a minimum set of disjoint shapes based on the stacking order of the shapes. For information about how to merge the voltage area shapes, see Merging Voltage Area Shapes.

The tool also uses the stacking order to resolve overlapping shapes from different voltage areas. For information about resolving overlapping voltage areas, see Resolving Overlapping Voltage Areas.

Feedback

To ensure that no shorts occur at the boundaries of the voltage areas, you can define guard bands for the voltage areas, which act as hard keepout margins surrounding the voltage areas. If you define guard bands for a voltage area shape, the guard bands are included in the effective boundary of the shape; however, they are not included in the effective placement area of the voltage area. For information about defining guard bands, see Defining Guard Bands.

To modify an existing voltage area, use the `set_voltage_area` command, as described in Modifying Voltage Areas.

Multivoltage designs typically have power domains that are shut down and powered up during the operation of the chip while other power domains remain powered up. When dealing with shutdown domains, there can be some situations in which certain cells in the shutdown portion need to continuously stay active, such as for implementing retention registers, isolation cells, retention control paths, and isolation enable paths. These cells are referred to as always-on cells. To define a special placement area for always-on cells (an always-on well) within a voltage area, define an exclusive move bound within the boundary of the voltage area. For information about defining exclusive move bounds, see Defining Move Bounds.

After creating the voltage areas, run the `check_mv_design` command to verify that the design does not have any multivoltage violations.

Merging Voltage Area Shapes

To merge the voltage area shapes into a minimum set of disjoint shapes, use the `-merge_regions` option with the `create_voltage_area`, `create_voltage_area_shape`, or `set_voltage_area` command.

- When you use the `create_voltage_area -merge_regions` command, the merges the shapes specified with the `-region` option.
- When you use the `create_voltage_area_shape -merge_regions` command, the merges the shape specified with the `-region` option and the existing shapes of the specified voltage area.
- When you use the `set_voltage_area -merge_regions` command, the tool merges all existing shapes of the specified voltage area.

The tool merges the voltage area shapes based on their stacking order. By default, the stacking order is the order in which you define the shapes, with the last shape defined on top. The merged shape replaces the top shape of a set of abutting or overlapping shapes; the other shapes in the set are removed and are no longer associated with the voltage area.

Feedback

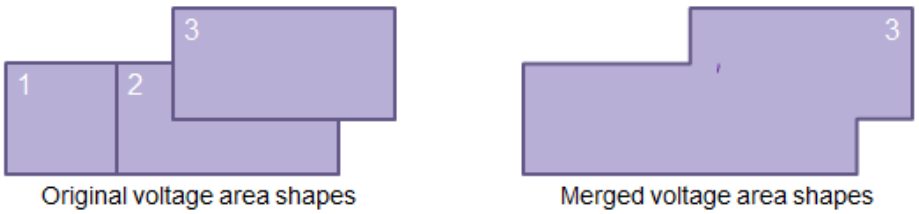
For example, assume that you use the following command to create a voltage area comprising three rectangle shapes, as shown on the left side of Figure 7.

```
icc2_shell> create_voltage_area -power_domains {PD1} \
   -region { {{0 0} {10 10}} {{10 0} {30 10}} {{15 5} {35 15}} }
{PD1}
icc2_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_1 VOLTAGE_AREA_SHAPE_2 VOLTAGE_AREA_SHAPE_3}
```

After you use the `-merge_regions` option to merge these shapes, the voltage area consists of a single rectilinear shape, as shown on the right side of Figure 7. The merged voltage area shape is named VOLTAGE_AREA_SHAPE_3, which was the last voltage area shape defined when the voltage area was created.

```
icc2_shell> set_voltage_area PD1 -merge_regions
Information: Merging abutted and overlapping shapes in voltage_area
 'PD1'. (NDMUI-154)
1
icc2_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_3}
```

Figure 7 Merging Voltage Area Shapes



To report the stacking order of the voltage area shapes, use the `report_voltage_areas -verbose` command.

To modify the stacking order of the voltage area shapes, use the `set_voltage_area_shape` command, as described in Modifying the Stacking Order.

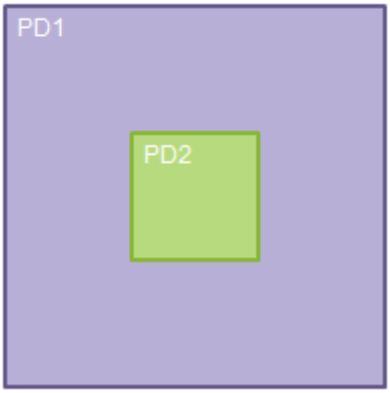
Resolving Overlapping Voltage Areas

If voltage area shapes from two or more voltage areas overlap, either completely or partially, the tool uses the stacking order of the shapes to determine the effective shapes of the voltage areas. By default, the stacking order is the order in which you define the shapes, with the last shape defined on top. The tool assigns the overlapped region to the voltage area associated with the top shape. Unlike merging shapes within a voltage area, when the tool resolves overlapping shapes from different voltage areas, it does not remove any shapes; only the interpretation of the shapes changes.

Feedback

For example, assume you want to create nested voltage areas, as shown in Figure 8.

Figure 8 Nested Voltage Areas



To generate these effective voltage areas, you must specify the outer voltage area first, followed by the inner voltage area, so that the voltage area shape for the inner voltage area is on top:

```
icc2_shell> create_voltage_area -power_domains PD1 \
   -region { {0 0} {30 30} }
{PD1}
icc2_shell> create_voltage_area -power_domains PD2 \
   -region { {10 10} {20 20} }
{PD2}
icc2_shell> get_attribute -objects [get_voltage_areas PD2] \
   -name effective_shapes
{{10.0000 10.0000} {20.0000 20.0000}}
icc2_shell> get_attribute -objects [get_voltage_areas PD1] \
   -name effective_shapes
{{0.0000 0.0000} {10.0000 0.0000} {10.0000 20.0000} {20.0000 20.0000}
 {20.0000 10.0000} {10.0000 10.0000} {10.0000 0.0000} {30.0000 0.0000}
 {30.0000 30.0000} {0.0000 30.0000}}
```

If you specify the inner voltage area first, the shape for the outer voltage area is on top and it masks the inner voltage area, so it is ignored by the tool, as shown in the following example:

```
icc2_shell> create_voltage_area -power_domains PD2 \
   -region { {10 10} {20 20} }
{PD2}
icc2_shell> create_voltage_area -power_domains PD1 \
   -region { {0 0} {30 30} }
{PD1}
icc2_shell> get_attribute -objects [get_voltage_areas PD2] \
   -name effective_shapes
icc2_shell> get_attribute -objects [get_voltage_areas PD1] \
```

Feedback

```
   -name effective_shapes
{{0.0000 0.0000} {30.0000 30.0000}}
```

To report the stacking order of the voltage area shapes, use the `report_voltage_areas -verbose` command.

To modify the stacking order of the voltage area shapes, use the `set_voltage_area_shape` command, as described in Modifying the Stacking Order.

Modifying the Stacking Order

You can modify the stacking order of the voltage area shapes by using the `set_voltage_area_shape` command.

- To raise a voltage area shape one position, use the `-raise` option.
- To lower a voltage area shape one position, use the `-lower` option.
- To move a voltage area shape to the top, use the `-top` option.
- To move a voltage area shape to the bottom, use the `-bottom` option.
- To move a voltage area shape directly above another shape, use the `-above` option.
- To move a voltage area shape directly below another shape, use the -`below` option.

Defining Guard Bands

Guard bands define hard keepout margins surrounding the voltage areas in which no cells, including level shifters and isolation cells, can be placed. The guard bands guarantee that the cells in different voltage areas are separated so that power planning does not introduce shorts.

By default, voltage areas do not have guard bands. To define guard bands, use the `-guard_band` option with the `create_voltage_area` or `create_voltage_area_shape` command to specify the horizontal and vertical guard band width for each shape specified in the `-region` option. The horizontal guard band width applies to all vertical edges of the voltage area, while the vertical guard band width applies to all horizontal edges of the voltage area.

Note:

If you also use the `-merge_regions` option, you must specify the guard band widths for each disjoint shape after merging. You would typically use this option only when all the shapes to be merged are abutted or overlapping and therefore merge into a single shape.

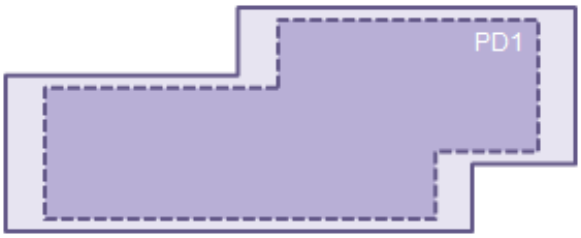
The effective boundary of a voltage area shape includes its guard band; however, the effective placeable area of the shape does not.

Feedback

For example, Figure 9 shows the guard band around the PD1 voltage area that is defined by the following command:

```
icc2_shell> create_voltage_area -power_domains PD1 \
  -region {{0 0} {30 0} {30 10} {40 10} {40 30} {20 30} {20 25} {0 25}} \
  -guard_band { {3 1} }
```

Figure 9 Voltage Area Guard Band



To determine the effective guard bands for abutting or overlapping shapes associated with the same voltage area, the tool merges the shapes as described in Merging Voltage Area Shapes and then applies the guard bands defined for the top shape to the merged shape.

For example, assume that you use the following command to define guard bands for the voltage area shapes shown on the right side of Figure 7:

```
icc2_shell> create_voltage_area -power_domains {PD1} \
   -region { {{0 0} {10 10}} {{10 0} {30 10}} {{15 5} {35 15}} } \
   -guard_band { {1 1} {3 3} {3 1} }
{PD1}
```

In this case, the effective guard band is the same as the guard band shown in Figure 9, which is the guard band defined for the merged shape.

To determine the effective placement areas and guard bands for overlapping shapes associated with different voltage areas, the tool uses the effective boundaries to resolve the shapes as described in Resolving Overlapping Voltage Areas. The top shape retains it placement area and guard band; the effective placement area and guard bands of lower shapes does not include the overlapping region. Note that if abutting shapes have guard bands, they are no longer abutting, but overlapping, due to the effective boundary that includes the guard bands.

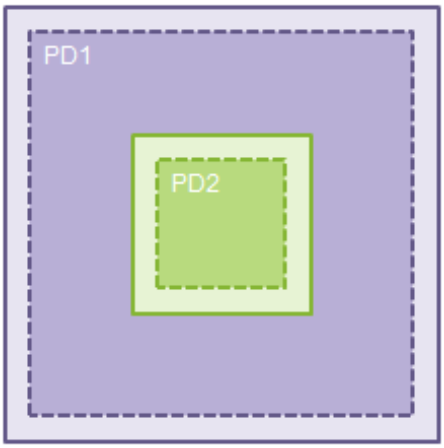
For example, assume that you use the following commands to define guard bands for the voltage areas shown in Figure 8:

```
icc2_shell> create_voltage_area -power_domains PD1 \
   -region {{0 0} {30 30}} -guard_band { {2 2} }
{PD1}
icc2_shell> create_voltage_area -power_domains PD2 \
   -region {{10 10} {20 20}} -guard_band { {2 2} }
{PD2}
```

Feedback

In this case, the effective placement area of PD1 is reduced by the effective boundary of PD2, which includes its guard band, as shown in Figure 10.

Figure 10 Effective Boundaries of Overlapping Voltage Areas



Defining Gas Stations

Gas stations are small voltage areas that are created for the purpose of minimizing the use of dual-rail buffers on physical feedthrough paths. Only single-rail repeaters are allowed in gas stations, and only optimization steps can add cells to gas stations.

You create gas stations by using the `create_voltage_area` command during the design planning stage. The tool recognizes gas stations automatically and uses them by trading off the cost of a routing detour with the cost of using a dual-rail cell.

Use the `-nwell` and `-pwell` options of the `create_voltage_area` command to specify n-well and p-well supply nets for a voltage area. Defining well supplies for gas station voltage areas provides flexibility for using gas stations for different design styles. If you do not use these options for a voltage area, the well bias values are assumed to be the same as the domain supply values.

The `report_voltage_areas` command lists the n-well and p-well supply nets regardless of whether they are explicitly set.

Querying Voltage Areas

You can query the following information about voltage areas:

- The voltage areas in the current block

  To create a collection of voltage areas in the current block, including the default voltage area, use the `get_voltage_areas` command.

- Detailed information about the voltage areas

Feedback

To display detailed information about voltage areas, including the default voltage area, use the `report_voltage_areas` command.

To include information about the voltage area shapes that comprise each voltage area, use the `-verbose` option with the `report_voltage_areas` command.

- Effective placement area of a voltage area

  To display the effective placement area of a voltage area, query the `effective_shapes` attribute of the voltage area.

- Effective guard bands of a voltage area

  To display the effective guard bands of a voltage area, query the `effective_guard_band_boundaries` attribute of the voltage area. Note that you can also query this attribute for individual voltage area shapes.

Modifying Voltage Areas

After you have created voltage areas, you can make the following modifications to a voltage area:

- Change the power domains associated with the voltage area

  To change the power domains associated with a voltage area, use the `-add_power_domains` and `-remove_power_domains` options with the `set_voltage_area` command.

- Change the voltage area name

  To change the name of the voltage area, use the `-name` option with the `set_voltage_area` command.

- Change the voltage area region

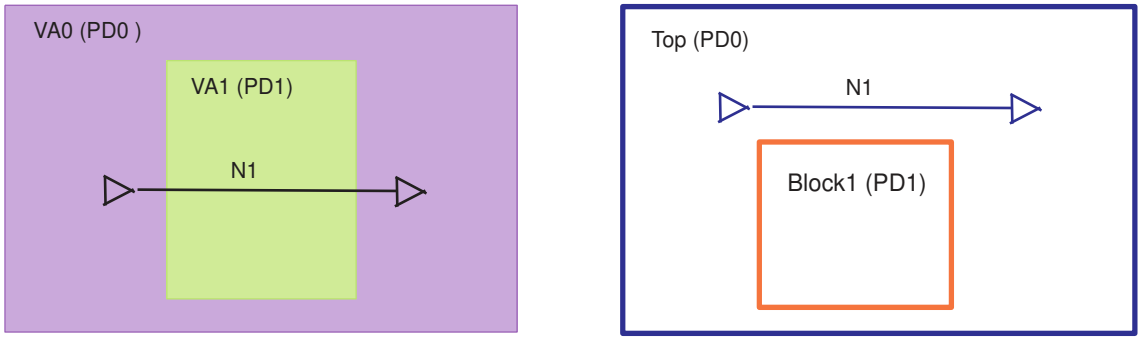
  To add shapes to a voltage area, use the `create_voltage_area_shape` command. To remove shapes from a voltage area, use the `remove_voltage_area_shapes` command.

Controlling Physical-Feedthrough Nets in Voltage Areas

A net is considered to be native to a voltage area if one or more segments of that net are in a logical hierarchy of that voltage area. If a net must physically route over a nonnative voltage area, then it is a physical feedthrough net of that voltage area, as shown in the following figure.

Feedback

Figure 11 Physical-Feedthrough Nets of Voltage Areas

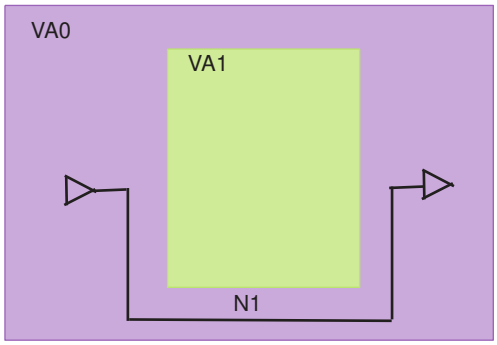


By default, physical-feedthrough nets are allowed in voltage areas. To prevent physical-feedthrough nets in a voltage area, define a voltage area rule using the `create_voltage_area_rule -allow_pass_through false` command, as shown in the following example:

```
icc2_shell> create_voltage_area_rule -allow_pass_through false \
   -name VA1_rule -voltage_areas VA1
```

With this rule, the N1 net detours around the VA1 voltage area, as shown in Figure 12.

Figure 12 Physical-Feedthrough Nets Disabled for a Voltage Area



By default, optimization does not insert buffers on physical-feedthrough nets of voltage areas. Inserting a buffer on a physical-feedthrough net can cause a mismatch between the

Feedback

logical and physical view of the buffer. The tool resolves such mismatches by supporting the following types of optimization for such nets:

- Physical-feedthrough buffering

  During physical-feedthrough buffering the tool performs the following:

  1. Inserts buffers in the logical hierarchy of its drivers or loads and applies the power domain of the nonnative voltage area on the buffer using power-domain-on-instance (PDOI) constraints
  2. Places the buffers within the nonnative voltage area

  To allow physical-feedthrough buffering, use the `create_voltage_area_rule -allow_physical_feedthrough true` command, as shown in the following example:

```
icc2_shell> create_voltage_area_rule \
   -allow_physical_feedthrough true \
   -name VA1_rule -voltage_areas VA1
```

- Logical-feedthrough buffering

  During logical-feedthrough the tool performs the following:

  1. Adds buffers in the logical hierarchy corresponding to the nonnative voltage area
  2. Places the buffers within the nonnative voltage area

  To allow logical-feedthrough buffering, use the `create_voltage_area_rule -allow_logical_feedthrough true` command. To specify logical hierarchies in which feedthrough buffers are allowed or not allowed, use the `-include_logical_feedthrough_hierarchy` or `-exclude_logical_feedthrough_hierarchy` option, respectively.

  The following example enables logical-feedthrough buffering for the VA1 voltage area and limits the feedthrough buffers to only the U1 and U2 hierarchical cells:

```
icc2_shell> create_voltage_area_rule \
   -allow_logical_feedthrough true \
   -include_logical_feedthrough_hierarchy {U1 U2}\
   -name VA1_rule -voltage_areas VA1
```

  The following example enables logical-feedthrough buffering for the VA2 voltage area and excludes the U3 hierarchical cell from feedthrough buffering:

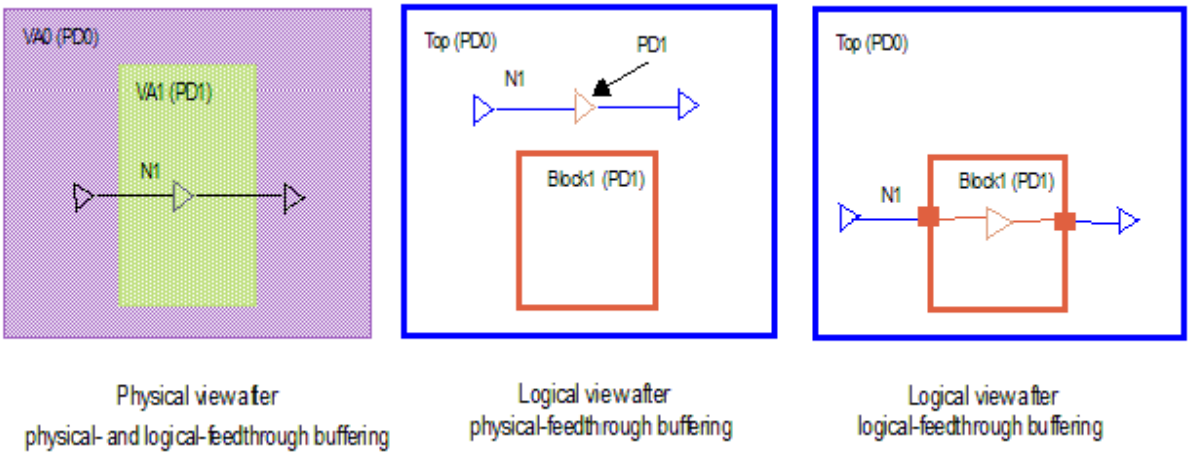
```
icc2_shell> create_voltage_area_rule \
   -allow_logical_feedthrough true \
   -exclude_logical_feedthrough_hierarchy {U3}\
   -name VA2_rule -voltage_areas VA2
```

Feedback

Note:

To add a logical-feedthrough buffer to a lower-level block, the tool has to add new boundary ports to that block. Therefore, freezing the boundary of a block by using the `set_freeze_ports` command prevents the tool from adding a logical-feedthrough buffer to that block.

Figure 13 Difference in the Logical View After Physical- and Logical-Feedthrough Buffering



To create a default rule that applies to all voltage areas that do not have a specific rule, use the `create_voltage_area_rule` command with the `-default_rule` option, as shown in the following example:

```
icc2_shell> create_voltage_area_rule -default_rule \
   -allow_physical_feedthrough true
```

If a voltage area does not have a specific rule, and there is no default rule, feedthrough buffering for that voltage area is controlled by the `opt.common.allow_physical_feedthrough` application option setting.

To report voltage area rules, use the `report_voltage_area_rules` command. To remove voltage area rules, use the `remove_voltage_area_rules` command.

Removing Voltage Areas

To remove voltage areas from a block, use the `remove_voltage_areas` command. To remove all voltage areas from a block, use the `-all` option. To remove specific voltage areas from a block, specify the voltage area names.

Feedback

Inserting Multivoltage Cells

A multivoltage design requires special multivoltage cells, such as level shifters and isolation cells, at the interface between power domains. Level shifters are required between power domains that operate at different voltage levels, while isolation cells are required between power domains that are in different states (powered-down versus always-on or powered-up). Typically, multivoltage cells are inserted during logic synthesis; however, you can also insert them with the IC Compiler II tool.

Inserting Level Shifters

Level-shifter cells function as the interface between power domains that operate at different voltage levels. These cells ensure that the output transition of a driver can cause the receiving cell to switch even though the receiver is operating at a different voltage level.

Note:

If the block was synthesized using Design Compiler, automatic level-shifter insertion is done as part of the `compile_ultra` command.

To insert level shifters in the current block, use the `create_mv_cells` command. The tool inserts the level shifters using the cell mapping and strategy defined in the UPF specification. When the tool inserts a level shifter, it sets a `size_only` attribute on the level shifter and a `dont_touch` attribute on the port-to-level-shifter net.

By default, the command inserts cells only from the generic library. If you use the `-mapped` option with the `create_mv_cells` command, the command inserts cells only from the user-provided logic library and issues an error if a suitable cell is not available.

If the tool does not insert a level-shifter cell, you can use the `analyze_mv_design -level_shifter` command to obtain more information. If you specify a net or pin with the `-through` option, the report lists the power domains and related supplies for the driver and load sides of the net or pin, along with error messages that indicate why a level-shifter cell was not inserted. The report includes errors about the specific insertion point as well as errors that are applicable to the entire path through the net or pin specified with the `-through` option.

Inserting Isolation Cells

Isolation cells are used to selectively shut off the input side of the voltage interface of a power domain; they do not shift the voltage. Isolation cells should be instantiated at the RTL level to prevent formal verification errors. However, you can also insert them using the IC Compiler II tool.

To insert isolation cells in the current block, use the `create_mv_cells` command. The tool inserts the isolation shifters using the cell mapping and strategy defined in the UPF

Feedback

specification. When the tool inserts an isolation cell, it sets a `size_only` attribute on the level shifter and a `dont_touch` attribute on the port-to-level-shifter net.

By default, the command inserts cells only from the generic library. If you use the `-mapped` option with the `create_mv_cells` command, the command inserts cells only from the user-provided logic library and issues an error message if a suitable cell is not available.

Associating Power Strategies With Existing Multivoltage Cells

The IC Compiler II tool automatically associates power strategies with existing multivoltage cells when you run the `associate_mv_cells` or `commit_upf` command. If this automatic association is not correct, you can manually modify the associations by using the `set_power_strategy_attribute` command. To determine the power strategies for a power domain, use the `get_power_strategies` command.

Controlling the Placement of Multivoltage Cells

A net that connects a multivoltage cell, such as a level-shifter cell or an isolation cell, to at least one cell in another voltage area is referred to as a multivoltage net. To reduce the length of multivoltage nets, enable advanced multivoltage cell placement by setting the `place.coarse.enable_advanced_mv_cell_placement` application option to `true`.

When you enable this feature, the tool reduces the length of the multivoltage nets by placing the multivoltage cells closer to the voltage area boundaries. To optimize multivoltage nets, the tool uses dual-rail buffers. By reducing the length of multivoltage nets, the tool can reduce the number of dual-rail buffers used during optimization and prevent unoptimized multivoltage nets due to the unavailability of dual-rail buffers.

Enabling Improved Buffering for Multivoltage Nets

Multivoltage nets are nets that logically or physically cross over more than one voltage area. You can enable the use of improved buffering techniques for fixing logical DRC violations on such nets during preroute optimization by setting the `opt.buffering.enable_hierarchy_mapping` application option to `true`.

This application option affects the `place_opt` and `clock_opt` commands. Enabling this feature reduces the number of buffers used to fix logical DRC violations. However, it can slightly increase the total negative slack or number of hold violations.

Analyzing Multivoltage Information

To ensure that the design does not have any multivoltage design violations, use the `check_mv_design` command.

Feedback

You can restrict the types of rules to check with the `check_mv_design` command. For example, the `-isolation` option specifies to check isolation strategy and isolation cell rules, while the `-pg_pin` option specifies to check rules associated with PG pins.

To report multivoltage information for your design, use the commands shown in the following table.

Table 2 Commands for Reporting Multivoltage Information

To do this	Use this command
Report a summary of multivoltage information	`report_mv_design`
Report information about the multivoltage cells	`report_mv_cells`
Report information about the multivoltage library cells	`report_mv_lib_cells`
Report paths with multivoltage constraints and the associated multivoltage cells	`report_mv_path`
Report the power domains	`report_power_domains`
Check whether the specified power domains are equivalent	`check_equivalent_power_domains`
Display a list of equivalent power domains	`get_equivalent_power_domains`
Report the voltage areas	`report_voltage_areas`

Specifying Timing Constraints and Settings

Timing constraints describe the timing requirements of a design. An essential part of timing constraints are accurately specified clocks and clock effects, such as latency and uncertainty. When you specify clocks, the tool automatically constrains the paths between the registers driven by these clocks. However, you can change the default behavior of these timing paths by specifying timing exceptions such as paths that should not be analyzed (false paths), paths that require multiple clock cycles (multicycle paths), and so on. In addition, you can constrain the boundary timing paths by specifying input and output delays for input and output ports.

The IC Compiler II tool uses on-chip variation (OCV) mode to perform timing analysis, which models the effects of variation in operating conditions across the chip. This mode performs a conservative timing analysis that allows both minimum and maximum delays to apply to different paths at the same time. For a setup check, it uses maximum delays for the launch clock path and data path and minimum delays for the capture clock path. For a hold check, it uses minimum delays for the launch clock path and data path and maximum delays for the capture clock path.

Feedback

A block might operate under several different conditions, such as different temperatures and voltages, and might operate in several different functional modes. For timing analysis, each set of conditions is represented by a *corner* and each functional mode is represented by a *mode*. A *scenario* is a combination of a corner and mode used to perform timing analysis and optimization. Before you start working with a block, you must define the modes, corners, and scenarios that are used for the block, as well as the delay calculation model and routing layers to use. The routing layer information you specify is used for RC estimation during timing analysis.

For detailed information about specifying

- Clock and clock effects, see the “Defining Clocks” topic in the *IC Compiler II Timing Analysis User Guide*.
- Exceptions for timing paths and constraints for boundary paths, see the “Constraining Timing Paths” topic in the *IC Compiler II Timing Analysis User Guide*.
- Modes, corners, and scenarios, see the “Defining Modes, Corners, and Scenarios” topic in the *IC Compiler II Timing Analysis User Guide*.
- Operating conditions and on-chip variation (OCV) related settings, see the “Specifying Operating Conditions” topic in the *IC Compiler II Timing Analysis User Guide*.
- Parasitic information for RC estimation and extraction, see the “Performing Parasitic Extraction” topic in the *IC Compiler II Timing Analysis User Guide*.

Specifying Logical Design Rule Constraints

Minimum capacitance, maximum capacitance, and maximum transition are logical design rule constraints that your design must meet to function as intended. They are technology-specific restrictions that are specified in the logic libraries. However, you can specify more restrictive design rule constraints by using the constraint commands given in Table 3.

During optimization, the IC Compiler II tries to meet the design rule constraints, even if it means violating optimization constraints such as timing, power, and area goals; these design rule constraints have a higher priority. After optimization, you can use the reporting commands given in Table 3 to identify design rule constraint violations in a block.

Table 3 Design Rule Commands

To do this	Use this command
Specify the minimum allowed capacitance for input ports, library cell pins, leaf cell pins, clocks, or blocks	`set_min_capacitance`
Specify the maximum allowed capacitance for input ports, library cell pins, leaf cell pins, clocks, or blocks	`set_max_capacitance`

Feedback

Table 3 Design Rule Commands (Continued)

To do this	Use this command
Specify the maximum allowed signal transition time for input ports, library cell pins, leaf cell pins, clocks, or blocks	`set_max_transition`
Remove a user-specified minimum capacitance constraint	`remove_min_capacitance`
Remove a user-specified maximum capacitance constraint	`remove_max_capacitance`
Remove a user-specified maximum transition constraint	`remove_max_transition`
Report minimum capacitance constraint violations	`report_constraints -min_capacitance`
Report maximum capacitance constraint violations	`report_constraints -max_capacitance`
Report maximum transition constraint violations	`report_constraints -max_transition`

Note:

The maximum fanout design rule constraint is not honored by the IC Compiler II tool. However, you can specify a maximum fanout for the data paths in a block by using the `opt.common.max_fanout` application option. This is a soft optimization constraint. During optimization the tool tries to ensure that data path cells do not drive more than the specified maximum fanout.

Controlling Clock-Gate Latencies

During synthesis, the IC Compiler II tool assumes that clocks are ideal. An ideal clock incurs no delay through the clock network. This assumption is made because real clock-network delays are not known until after clock tree synthesis. In reality, clocks are not ideal and there is a nonzero delay through the clock network. For designs with clock gating, the clock-network delay at the registers is different from the clock-network delay at the clock-gating cell. This difference in the clock-network delay at the registers and at the clock-gating cell results in tighter constraints for the setup condition at the enable input of the clock-gating cell.

The tool can obtain clock-gate latencies in the following ways:

- Integrated latency estimation

  By default, the tool estimates and updates clock-gate latencies throughout the flow. Estimated clock latencies are more accurate than user-specified clock latencies.

Feedback

- User-specified latency

  You can disable integrated latency estimation for specific instances and specify latency values manually.

Integrated Clock-Gate Latency Estimation

By default, the tool estimates and updates clock-gate latencies throughout the `place_opt` command flow. This ensures that the tool uses accurate clock-gate latencies during datapath optimization and useful-skew optimization performed before clock tree synthesis, when clocks are ideal.

If you plan to perform structural multisource clock tree synthesis for your design, ensure that the tool estimates clock-gate latencies that are appropriate for structural multisource clock subtrees by setting the `opt.clock_latency_estimation.estimate_smscts_subtrees` application option to `true`.

To improve the accuracy of the estimated clock-gate latencies, specify clock tree synthesis settings before you run the `place_opt`command. Such settings include the clock tree buffer library cell list, the clock tree inverter library cell list, and clock nondefault routing rules.

Estimated clock latencies are more accurate than user-specified clock latencies. In addition, the tool updates the estimated clock latencies after steps such as placement and multibit banking, while user-specified latencies are static values.

To prevent the tool from estimating the latency for a specific clock gate, set the `dont_estimate_clock_latency` attribute on its clock pin by using the `set_attribute` command, as shown in the following example:

```
icc2_shell> set_attribute \
   [get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

For best results, do not use the following features with integrated clock-gate latency estimation:

- Trial clock tree synthesis

  To disable this feature, set the `place_opt.flow.trial_clock_tree` application option to `false`.

- Clock-gate optimization

  To disable this feature, set the `place_opt.flow.optimize_icgs` application option to `false`.

Feedback

- User-specified latencies

  User-specified latencies are set by the `set_clock_latency`, `set_clock_gate_latency`, `set_clock_gating_check -setup`, or `set_path_margin -setup` commands.

The tool does not modify any `set_clock_latency` constraints that you specify on clock-gating cells. Tool-estimated clock-gate latency values are stored as offsets from the user-specified values. You can see the offset values by using the `write_script -format icc2` command and checking for the `set_clock_latency -offset` commands in the generated output. If you do not set any latency values, the reported offsets are equal to the estimated latency values.

The estimated latency offset values are not captured in an SDC file generated by the `write_sdc` command.

Latency estimates are most accurate if the relative locations of the clock gates and their gated registers do not change during clock tree synthesis. To preserve the placement of integrated clock gates, enable the automatic relocation of clock network cells by setting the `cts.compile.enable_cell_relocation` application option to `auto` before clock tree synthesis.

User-Specified Clock-Gate Latency

You cannot disable general integrated clock-gate latency estimation. However, you can disable the feature for specific clock gates by setting a `dont_estimate_clock_latency` attribute on their clock pins. This topic describes how to manually specify latency values on those clock gates.

Specify the clock network latency by using either the `set_clock_latency` or `set_clock_gate_latency` command. The `set_clock_gate_latency` command can be used for both gate-level and RTL designs.

The clock latency specified using the `set_clock_gate_latency` command is annotated on the registers during the `compile_fusion` command when the clock-gating cells are inserted. However, if you modify the latency values on the clock gates after the compilation, you must manually apply the latency values on the existing clock-gating cells using the `apply_clock_gate_latency` command.

Note:

After you modify the clock-gate latency using the `set_clock_gate_latency` command, if you compile your design using the `compile_fusion` command, it is not necessary to use the `apply_clock_gate_latency` command to apply the latency values. The tool annotates the specified value during compilation.

To remove clock latency information previously set on clock-gating cells with the `set_clock_gate_latency` or `apply_clock_gate_latency` commands, use the

Feedback

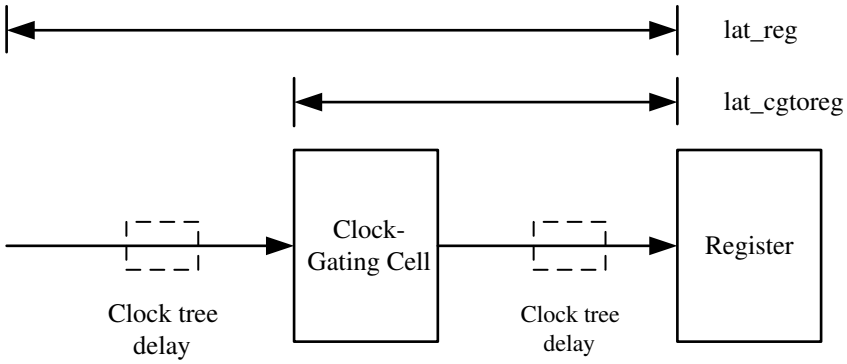
`reset_clock_gate_latency` command. This command removes the clock latency values on the specified clocks. If you do not specify the clock, the clock latency values on all the clock-gating cells are removed.

The set_clock_latency Command

Use the `set_clock_latency` command to specify clock network latency for specific clock-gating cells.

In Figure 14, `lat_cgtoreg` is the estimated delay from the clock pin of the clock-gating cell to the clock pin of the gated register and `lat_reg` is the estimated clock-network latency to the clock pins of the registers without clock gating.

Figure 14 Clock Latency With Clock-Gating Design



For all clock pins of registers (gated or ungated) in the design that are driven by a specific clock, use the `lat_reg` value for the `set_clock_latency` command. For clock pins of all the clock-gating cells, use the difference between the `lat_reg` and `lat_cgtoreg` values for the `set_clock_latency` command. Because the purpose of setting the latency values is to account for the different clock-network delays between the registers and the clock-gating cell, it is important to get a reasonably accurate value of the difference (`lat_cgtoreg`). The absolute values used are less important unless you are using these values to account for clock-network delay issues not related to clock gating.

The set_clock_gate_latency Command

When you use the `compile_fusion` command, clock gates are inserted during the compilation process. To specify the clock network latency before the clock-gating cells are inserted by the tool, use the `set_clock_gate_latency` command. This command lets you specify the clock network latency for the clock-gating cells as a function of the clock domain, clock-gating stage, and the fanout of the clock-gating cell. The latency that you specify is annotated on the clock-gating cells when they are inserted by the `compile_fusion` command. You can manually annotate the latency values on the existing clock-gating cells in your design using the `apply_clock_gate_latency` command.

Feedback

The `set_clock_gate_latency` command takes the following arguments:

- `-clock` *`clock_list`*
- `-stage` *`clock_gate_stage`*
- `-fanout_latency` *`fanout_list`*

The `fanout_list` is a list of tuples specifying a fanout range and a delay decrement. In the following example, three fanout ranges are created: 1 to 5 with a latency value of 0.5, 6-15 with a latency value of 0.8, and 16 to infinity with a latency value of 0.6.

```
set_clock_gate_latency -stage 1 \
      -fanout_latency {{1-5 0.5} {6-15 0.8} {16-inf 0.6}}
```

To specify a clock latency value for clock-gated registers, use the `-stage` option with a value of 0. Because you are specifying the latency value for the clock-gated registers, the value for the `-fanout_latency` option should be `1-inf` (1 to infinity) and the latency is the absolute clock latency value for the registers, as shown in the following example:

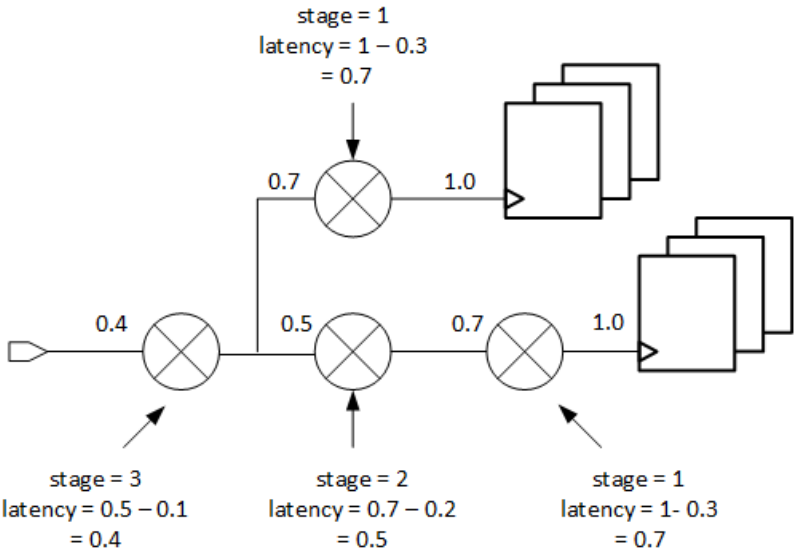
```
set_clock_gate_latency -clock CLK -stage 0\
      -fanout_latency {1-inf 1.0}
```

If the `-clock` option is not specified, the setting applies to all clocks in the design.

Clock latencies using the `set_clock_latency` command have higher precedence and are not overwritten.

Figure 15 shows an example of clock-gate stages and fanout.

Figure 15 Clock-Gating Stages and Latency Calculations



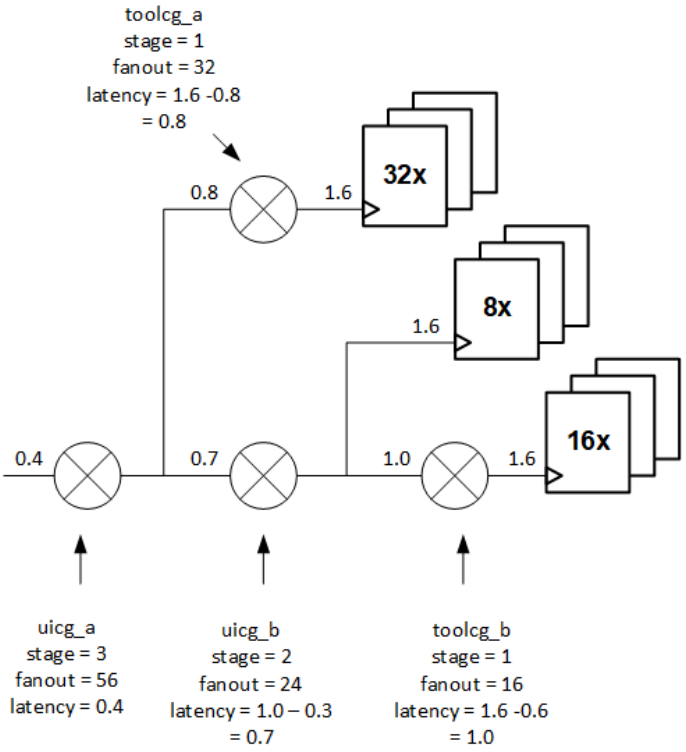
Feedback

The following commands set the latency values for Figure 15:

```
set_clock_gate_latency -stage 0 -fanout_latency {{1-inf 1.0}}
set_clock_gate_latency -stage 1 -fanout_latency {{1-inf 0.3}}
set_clock_gate_latency -stage 2 -fanout_latency {{1-inf 0.2}}
set_clock_gate_latency -stage 3 -fanout_latency {{1-inf 0.1}}
```

Figure 16 shows another example of fanout and latency calculations.

Figure 16 Latency Calculations With Varying Fanout



The following commands specify the latency and fanout for Figure 16.

```
set_clock_gate_latency -stage 0 \
   -fanout_latency {{1-inf 1.6}}
set_clock_gate_latency -stage 1 \
   -fanout_latency {{1-20 0.6} {21-inf 0.8}}
set_clock_gate_latency -stage 2 \
   -fanout_latency {{1-20 0.2} {21-inf 0.3}}
set_clock_gate_latency -stage 3 \
   -fanout_latency {{1-30 0.1} {31-65 0.8} {66-inf 0.18}}
set_clock_latency "0.4" {uicg_a/CK}
```

Note that for the clock gate uicg_a, the latency value of 0.4 is assigned using the `set_clock_latency` command, which cannot be overwritten.

Feedback

Specifying Physical Constraints for Placement and Legalization

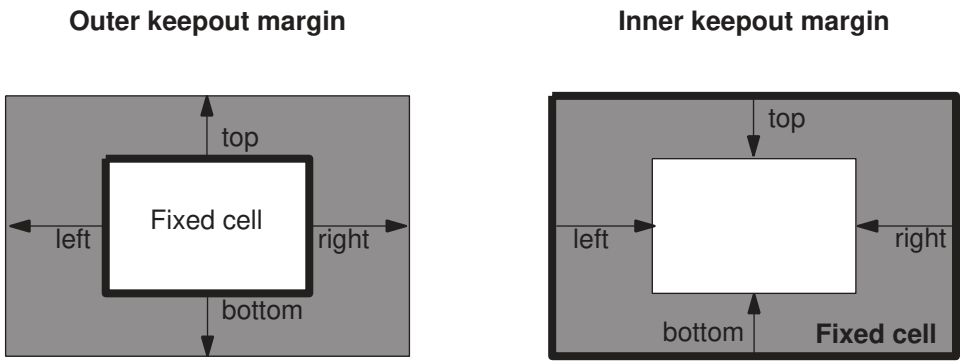
During placement and legalization, the floorplan information dictates where cells are placed. The following topics describe how to specify additional physical constraints that affect placement and legalization:

- Defining Keepout Margins
- Defining Area-Based Placement Blockages
- Defining Placement Bounds
- Defining Placement Attractions
- Defining Cell Spacing Constraints for Legalization

Defining Keepout Margins

A keepout margin is a region (the shaded portions in Figure 17) around the boundary of fixed cells in a block in which no other cells are placed.

Figure 17 Placement Keepout Margins



An outer keepout margin is a region outside the cell boundary, while an inner keepout margin is a region inside the cell boundary. The width of the keepout margin on each side of the fixed cell can be the same or different, depending on how you define the keepout margin. In addition, keepout margins can be defined as hard or soft. Keeping the placement of cells out of such regions avoids congestion and net detouring and produces better QoR.

To define a keepout margin, use the `create_keepout_margin` command. By default, the command creates a hard keepout margin. To create a soft keepout margin, use the `-type soft` option.

Feedback

Defining an Outer Keepout Margin

You can define an outer keepout margin on a hard macro, a hierarchical cell, or a leaf cell. When you define an outer keepout margin, you can either specify the keepout distance explicitly or, for hard macros, you can have the tool derive the keepout distance based on the macro pin count.

To explicitly specify an outer keepout margin, use the `-outer` option to specify the margin distance for each side. You specify the left, bottom, right, and top margins using the following format: `{lx by rx ty}`. A value of 0 results in no keepout margin for that side.

For example, to create a hard outer keepout margin with a margin of 10 on each side for a macro named my_macro, use the following command:

```
icc2_shell> create_keepout_margin -outer {10 10 10 10} my_macro
```

To have the tool derive the outer keepout distance for a hard macro based on its pin count, use the `-tracks_per_macro_pin` option to specify the track-to-pin ratio. When you use this option, the tool calculates the keepout margin from the track width, the number of macro pins, and the specified track-to-pin ratio, which is typically set to a value near 0.5. A larger value results in larger keepout margins. The derived keepout margin is always hard; the `-type` setting is ignored. The derived margins are subject the minimum and maximum values specified by the `-min_padding_per_macro` and `-max_padding_per_macro` options.

For example, to have the tool derive the outer keepout margin for a macro named my_macro by using a track-to-pin ratio of 0.6 with a minimum keepout distance of 0.1 and a maximum keepout distance of 0.2, use the following command:

```
icc2_shell> create_keepout_margin -tracks_per_macro_pin 0.6 \
   -min_padding_per_macro 0.1 -max_padding_per_macro 0.2 my_macro
```

Defining an Inner Keepout Margin

You can define an inner keepout margin on a hierarchical cell, but not on a hard macro or a leaf cell. When you define an inner keepout margin, you must specify the keepout distance explicitly.

To explicitly specify an inner keepout margin, use the `-inner` option to specify the margin distance for each side. You specify the left, bottom, right, and top margins using the following format: `{lx by rx ty}`. A value of 0 results in no keepout margin for that side.

For example, to create a hard inner keepout margin with a margin of 10 on each side for a hierarchical cell named my_hcell, use the following command:

```
icc2_shell> create_keepout_margin -inner {10 10 10 10} my_hcell
```

Feedback

Defining Area-Based Placement Blockages

An area-based placement blockage is a rectangular region in which cells cannot be placed or in which the types or number of cells is limited. The IC Compiler II tool supports the following types of area-based placement blockages:

- Hard

  A hard blockage prevents the placement of standard cells and hard macros within the specified area during coarse placement, optimization, and legalization.

- Hard macro

  A hard macro blockage prevents the placement of hard macros within the specified area during coarse placement, optimization, and legalization.

- Soft

  A soft blockage prevents the placement of standard cells and hard macros within the specified area during coarse placement, but allows optimization and legalization to place cells within the specified area.

- Partial

  A partial blockage limits the cell density in the specified area during coarse placement, but has no effect during optimization and legalization.

To define placement blockages, use the `create_placement_blockage` command. At a minimum, you must specify the coordinates of the placement blockage. To create a rectangular placement blockage, use the `-boundary` option to specify the lower-left and upper-right coordinates of the rectangle. To create a rectilinear placement blockage, use the `-boundary` option to specify the coordinates of the polygon. You can also create multiple placement blockages by specifying the boundary polygon as a geo_mask or a collection of physical objects. If the resulting area resolves to multiple, noncontiguous polygons, the command creates multiple placement blockages, one corresponding to each polygon.

By default, the `create_placement_blockage` command creates a hard placement blockage. To create another type of placement blockage, use the `-type` option to specify the blockage type. A single `create_placement_blockage` command can create just one type of placement blockage.

You can optionally assign a name to a placement blockage by using the `-name` option. You can then reference that blockage by name to query or remove the placement blockage.

Feedback

Defining a Hard Placement Blockage

To define a hard placement blockage, specify the boundary and optionally a name for the placement blockage.

For example, to create a hard placement blockage in the area enclosed by a rectangle with corners at (10, 20) and (100, 200), use the following command:

```
create_placement_blockage -boundary {10 20 100 200}
```

Note:

Hard placement blockages are honored during placement, legalization, optimization, and clock tree synthesis.

Hard placement blockages can also be defined in the DEF as shown in Example 13.

Example 13 Placement Blockages in DEF

```
BLOCKAGES 2 ;
 - PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
 - PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
END BLOCKAGES
1
```

Defining a Hard Macro Placement Blockage

To define a hard macro blockage, specify the boundary, type (`-type hard_macro` option), and optionally the name for the placement blockage.

For example, to define a hard macro blockage enclosed by a rectangle with corners at (120, 75) and (230, 200), use the following command:

```
create_placement_blockage -boundary {120 75 230 200} \
   -type hard_macro
```

Note:

Hard macro placement blockages are honored during placement, legalization, and optimization. This is the only type of placement blockage that is honored by hard macro placement.

Feedback

Defining a Soft Placement Blockage

To define a soft blockage, specify the boundary, type (`-type soft` option), and optionally the name for the placement blockage.

For example, to define a soft blockage enclosed by a rectangle with corners at (120, 75) and (230, 200), use the following command:

```
create_placement_blockage -boundary {120 75 230 200} \
   -type soft
```

A soft blockage prevents the initial placement from placing cells within the specified area, but allows legalization, optimization, and clock tree synthesis to do so. However, after placement and optimization, during subsequent incremental placement, the tool can move the cells added during legalization, optimization, and clock tree synthesis out of the soft blockage area. To prevent this, use the following application option setting:

```
set_app_options -name place.coarse.enable_enhanced_soft_blockages \
   -value true
```

Note:

Soft placement blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Defining a Partial Placement Blockage

To define a partial blockage, specify the boundary, type (`-type partial` option), blockage percentage (`-blocked_percentage` option), and optionally the name for the placement blockage.

For example, to define a partial blockage with a maximum allowed cell density of 60 percent (a blocked percentage of 40), enclosed by the rectangle with corners at (10, 20) and (100, 200), use the following command:

```
create_placement_blockage -boundary {10 20 100 200} \
   -type partial -blocked_percentage 40
```

To allow unlimited usage of a partial blockage area, specify a blockage percentage of zero (`-blocked_percentage 0` option).

Note:

Partial placement blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Feedback

Define a Blockage of a Predefined Category

A category blockage is a special type of partial blockage that controls the placement of a predefined category of cells within the partial blockage.

To create a category blockage,

1. Create the cell category by defining a user attribute for the category by using the `define_user_attribute` command and applying it to the affected cell references or instances by using the `set_attribute` command.

   To prevent the cell from being placed in the category blockage, set the attribute to `true`. To allow the cell in the category blockage, set the attribute to `false`.

   When defining and applying attributes, observe the following rules:

   - A blockage can only be controlled by a single attribute.
   - Multiple blockages can be controlled by the same attribute.
   - A cell can have multiple attributes, which could impact its placement in multiple category blockages.
   - The same attribute can be applied to a reference and an instance.
   - If a cell instance and its reference have different attributes, the attribute on the cell instance takes precedence.

2. Create the category blockage by using the `create_placement_blockage` command with the `-type category`, `-blocked_percentage`, and `-category` options. The argument to the `-category` option is the name of the user attribute.

The following example defines a blockage that prevents a certain category of cells from being placed within it:

```
define_user_attribute -type boolean \
   -classes lib_cell dr_cells
set_attribute [get_lib_cells mylib/dr*] dr_cells true
create_placement_blockage -boundary {{4300 2400} {4500 2700}} \
   -type category -blocked_percentage 45 -category dr_cell
```

The following example defines a blockage that allows only a certain category of cells to be placed within it:

```
define_user_attribute -type boolean \
  -classes lib_cell non_iso
set_attribute [get_lib_cells mylib/*] non_iso true
set_attribute [get_lib_cells mylib/iso*] non_iso false
create_placement_blockage -boundary {{4300 2400}{4500 2700}} \
   -type category -blocked_percentage 30 -category non_iso
```

Feedback

Note:

Category blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Defining Blockages That Exclude Registers

To define partial blockages that exclude registers, specify the boundary, type (`-type register` option), blockage percentage (`-blocked_percentage` option), and optionally the name for the placement blockage.

For example, to define a partial blockage that excludes registers, but allows a cell density of 50 percent for other cells, use the following commands:

```
create_placement_blockage -boundary {{10 20} {100 200}} \
   -type register -blocked_percentage 50
```

Note:

Register blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Defining Blockages That Exclude Relative Placement Groups

To define partial blockages that exclude relative placement groups, specify the boundary, type (`-type rp_group` option), blockage percentage (`-blocked_percentage` option), and optionally the name for the placement blockage.

For example, to define a partial blockage that excludes relative placement groups, but allows a cell density of 100 percent for all other cells, use the following commands:

```
create_placement_blockage -boundary {{10 20} {100 200}} \
   -type rp_group -blocked_percentage 0
```

Note:

Relative placement group blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Defining Blockages That Allow Relative Placement Cells Only

To define partial blockages that allow relative placement cells only, specify the boundary, type (`-type allow_rp_only` option), blockage percentage (`-blocked_percentage` option), and optionally the name for the placement blockage.

For example, to defines a partial blockage that allows relative placement cells only, with a maximum allowed cell density of 80 percent (a blocked percentage of 20), and enclosed by the rectangle with corners at (10, 20) and (100,200), use the following command:

```
create_placement_blockage -name rp_only  -type allow_rp_only \
   -boundary {10 20 100 200}  -blocked_percentage 20
```

Feedback

These blockages allows the tool to place relative placement groups, which are usually large structures, without disturbing the placement of other cells. After the tool places and legalizes the relative placement groups, you can remove these blockages and allow the tool to place other cells in them.

Note:

Relative-placement-cells-only blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Defining Blockages That Allow Buffers Only

To define a partial blockage that allows buffers only, specify the boundary, type (`-type allow_buffer_only` option), blockage percentage (`-blocked_percentage` option), and optionally the name for the placement blockage.

For example, to define a partial blockage that allows only the placement of buffers and inverters, with a cell density of 30 percent, use the following commands:

```
create_placement_blockage -boundary {{10 20} {100 200}} \
   -type allow_buffer_only -blocked_percentage 70
```

Note:

Buffer-only blockages are honored during placement, but not during legalization, optimization, or clock tree synthesis.

Querying Placement Blockages

To return a collection of placement blockages in the current block that match certain criteria, use the `get_placement_blockages` command.

Removing Placement Blockages

To remove placement blockages from a block, use the `remove_placement_blockages` command. To remove all placement blockages from a block, use the `-all` option. To remove specific placement blockages from a block, specify the placement blockage names.

Defining Placement Bounds

A placement bound is a constraint that controls the placement of groups of cells. It allows you to group cells to minimize wire length and place the cells at the most appropriate locations. For example, you might want to define a bound for clock-gating cells or extremely timing-critical groups of cells that you want to guarantee will not be disrupted for placement by other logic.

Table 4 lists the types of placement bounds supported by the IC Compiler II tool.

Feedback

Table 4 Types of Placement Bounds

Bound type	Description
Soft move bound	The tool tries to place the cells in the move bound within a specified region, during coarse placement. However, there is no guarantee that the cells are placed inside the bounds. Soft move bounds are honored during coarse placement, but not legalization.
Hard move bound	The tool must place the cells in the move bound within a specified region. Hard move bounds are honored during both coarse placement,and legalization. However, they are not honored during clock tree synthesis.
Exclusive move bound	The tool must place the specified cells within the specified region and place all other cells outside of the exclusive move bounds, during both coarse placement and legalization. The placement of newly added buffers in exclusive move bounds is controlled by the `opt.buffering.exclusive_hard_bound_buffering_mode` application option. By default, during buffering, the tool can add new cells to an exclusive hard move bound if it improves the QoR. If you set this application option to `0`, the tool honors the exclusive hard move bound.
Soft group bound	The tool tries to place the cells in the group bound within a floating region; however, there is no guarantee that the cells are placed inside the bounds.
Hard group bound	The tool must place the cells in the group bound within a floating region, whose actual coordinates are determined by the tool.
Dimensionless group bound	The tool determines the shape and location of the group bound based on the effort used to bring cells closer inside the group bound.

To define a placement bound, use the `create_bound` command. When you define a bound, you must use the `-name` option to specify its name.

In general, you also specify the cells and ports to be included in the bound. If a hierarchical cell is included, all cells in the subdesign belong to the bound. However, you can create an empty bound and specify the contents of the bound later by using the `add_to_bound` command. You can remove objects from a bound by using the `remove_from_bound` command.

You must also specify the options required for the specific type of bound you want to create. The following topics describe how to create the various types of move bounds:

- Defining Move Bounds
- Defining Group Bounds

Feedback

- Querying Placement Bounds
- Removing Placement Bounds

Defining Move Bounds

A move bound is a fixed region within which to place a set of cells. It comprises one or more rectangular or rectilinear shapes, which can be abutted, disjoint, or overlapping. To define the boundaries of these shapes, use the `-boundary` option with the `create_bound` command (if you are creating a new move bound) or the `create_bound_shape` command (if you are adding shapes to an existing move bound). Note that you can specify one or more shapes when using the `create_bound` command, but only a single shape in each `create_bound_shape` command.

- To specify the boundary of a rectangle shape, use the following format to specify its lower-left and upper-right corners:

  ```
  {{llx lly} {urx ury}}
  ```

- To specify the boundary of a rectilinear shape, use the following format to specify the coordinates of its vertices:

  ```
  {{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
  ```

Move bounds can be hard, soft, or exclusive.

- To define a soft move bound, use the following syntax:

  ```
  create_bound -name name [-type soft] [-effort effort_level] \
     -boundary {coordinates} [bound_objects]
  ```

  The default effort is `ultra`; you can also specify `low`, `medium`, or `high`.

  For example, to define a rectangular soft move bound for the INST_1 cell instance with its lower-left corner at (100, 100) and its upper-right corner at (200, 200), use the following command:

  ```
  icc2_shell> create_bound -name b1 -boundary {100 100 200 200} INST_1
  ```

- To define a hard move bound, use the following syntax:

  ```
  create_bound -name name -type hard -boundary {coordinates}\
     [bound_objects]
  ```

  For example, to define a rectangular hard move bound for the INST_1 cell instance with its lower-left corner at (100, 100) and its upper-right corner at (200, 200), use the following command:

  ```
  icc2_shell> create_bound -name b2 -type hard \
     -boundary {100 100 200 200} INST_1
  ```

Feedback

- To define an exclusive move bound, use the following syntax:

```
create_bound -name name -exclusive -boundary {coordinates} \
   [bound_objects]
```

  For example, to define a rectangular exclusive move bound for the INST_1 cell instance with its lower-left corner at (100, 100) and its upper-right corner at (200, 200), use the following command:

```
icc2_shell> create_bound -name b3 -exclusive \
   -boundary {100 100 200 200} INST_1
```

To add shapes to an existing move bound, use the `create_bound_shape` command. To remove shapes from an existing move bound, use the `remove_bound_shapes` command.

Defining Group Bounds

A group bound is a floating region within which to place a set of cells. Group bounds can be hard, soft, or dimensionless.

- To define a soft group bound, use the following syntax:

```
create_bound -name name [-type soft] [-effort effort_level] \
   [-dimensions {width height} bound_objects]
```

  The default effort is `ultra`; you can also specify `low`, `medium`, or `high`.

  For example, to define a soft group bound for the INST_1 and INST_2 cell instances with a width of 100 and a height of 100, use the following command:

```
icc2_shell> create_bound -name b4 -dimensions {100 100} \
    {INST_1 INST_2}
```

- To define a hard group bound, use the following syntax:

```
create_bound -name name -type hard -dimensions {width height} \
   [bound_objects]
```

  For example, to define a hard group bound for the INST_1 and INST_2 cell instances with a width of 100 and a height of 100, use the following command:

```
icc2_shell> create_bound -name b5 -type hard -dimensions {100 100} \
   {INST_1 INST_2}
```

- To define a dimensionless group bound, use the following syntax:

```
create_bound -name name [-effort effort_level] \
   [bound_objects]
```

Feedback

The default effort is `medium`; you can also specify `low`, `high`, or `ultra`.

For example, to define a dimensionless group bound for the INST_1 and INST_2 cell instances in which the tool uses a high level of effort to place the cells closer within the group bound, use the following command:

```
icc2_shell> create_bound -name b6 -effort high {INST_1 INST_2}
```

Querying Placement Bounds

To report the placement bounds in a block, use the `report_bounds` command.

To return a collection of placement bounds in the current block that match certain criteria, use the `get_bounds` command.

To return a collection of bound shapes associated with one or more move bounds, use the `get_bound_shapes` command.

Removing Placement Bounds

To remove placement bounds from a block, use the `remove_bounds` command. To remove all placement bounds from a block, use the `-all` option. To remove specific placement bounds from a block, specify the placement bound names.

Defining Placement Attractions

A placement attraction is a constraint that you can use to specify that a large group of cells be placed together in the same vicinity of the placement area. It is a soft constraint that the tool considers during initial placement. However, it has less effect during subsequent incremental placement stages. To specify a more restrictive placement constraint for a smaller group of cells, create a move bound or group bound by using the `create_bound` command.

To define a placement attraction, use the `create_placement_attraction` command. For example, to specify that the cells from the subblocks named add1 and mult1 be placed in the same vicinity, use the following command:

```
icc2_shell> set add1_cells [get_flat_cells add1/*]
icc2_shell> set mult1_cells [get_flat_cells mult1/*]
icc2_shell> create_placement_attraction -name add1_mult1 \
   "$add1_cells $mult1_cells"
```

Feedback

You can use the `-region` option to specify

- A region in which to place the cells

  To do so, specify the lower-left and upper-right coordinates of the region, as shown in the following example:

```
icc2_shell> set U1_cells [get_flat_cells U1/*]
icc2_shell> create_placement_attraction -name U1 \
   -region {{0 0} {2000 1500}} $U1_cells
```

- A straight line along which to place the cells, such as an edge of a macro cell

  To do so, specify the coordinates of the two ends of the line, as shown in the following example:

```
icc2_shell> set U2_cells [get_flat_cells U2/*]
icc2_shell> create_placement_attraction -name U2 \
   -region {{1000 200} {1000 800}} $U2_cells
```

- A location around which to place the cells, such as a port location

  To do so, specify the coordinates of the location, as shown in the following example:

```
icc2_shell> set U3_cells [get_flat_cells U3/*]
icc2_shell> create_placement_attraction -name U3 \
   -region {{0 700}} $U3_cells
```

The following table shows additional commands available for changing, reporting, and removing placement attractions. For more information, see the command man pages.

Table 5 Commands Related to Placement Attractions

To do this	Use this command
Define a placement attraction	`create_placement_attraction`
Add cells to an existing placement attraction	`add_to_placement_attraction`
Remove cells from an existing placement attraction	`remove_from_placement_attraction`
Report placement attractions	`report_placement_attractions`
Find existing placement attractions	`get_placement_attractions`
Remove existing placement attractions	`remove_placement_attractions`

See Also

- Defining Move Bounds

Feedback

Defining Cell Spacing Constraints for Legalization

Cell spacing constraints control the spacing between a standard cell and another standard cell or a boundary (the chip boundary, a hard macro, a hard macro keepout margin, a hard placement blockage, or a voltage area guard band). You assign library cells to groups (the boundaries are in a predefined group named SNPS_BOUNDARY), and then define the required spacing between cells in these groups.

By default, there are no spacing constraints between standard cells during legalization. To enhance yield, you can define the valid spacing between standard cells or between a standard cell and a boundary.

Note:

The support for spacing constraints between standard cells and power or ground nets depends on how the net is represented in the block. If the power or ground net is defined as a complete blockage, the legalizer and the `check_legality` command ignore spacing rule violations between standard cells and the power or ground net. If the power or ground net is defined as a partial blockage, the legalizer and the `check_legality` command check for spacing rule violations between standard cells and the power or ground net.

Cell spacing constraints are implemented by attaching labels, which are similar to attributes, to the left and right sides of library cells, assuming that the cell is in its north orientation, and specifying the invalid spacings between these labels.

To define cell spacing constraints,

1. Add labels to the library cells that have spacing constraints by using the `set_placement_spacing_label` command.

   You must specify the following information for each label:

   - The label name (the `-name` option)
   - The library cells to which to apply the label (the `-lib_cells` option)
   - The sides of the library cells to which to apply the label (the `-side` option, which can take a value of `right`, `left`, or `both`)

   The label definitions are additive; you can specify the same label to be used on the right side of some cells and the left side of other cells. For example, to assign a label named X to the right side of the cellA library cell and the left side of the cellB and cellC library cells, use the following commands:

```
icc2_shell> set_placement_spacing_label -name X \
   -lib_cells {cellA} -side right
icc2_shell> set_placement_spacing_label -name X \
   -lib_cells {cellB cellC} -side left
```

Feedback

You can assign multiple labels to a side of a library cell. For example, to assign labels named Y and Z to the right side of the cellB library cell, use the following commands:

```
icc2_shell> set_placement_spacing_label -name Y \
   -lib_cells {cellB} -side right
icc2_shell> set_placement_spacing_label -name Z \
   -lib_cells {cellB} -side right
```

2. Define the spacing requirements between the labels by using the `set_placement_spacing_rule` command.

   You must specify the following information for each rule:

   - The labels being constrained (the `-labels` option)

     You must specify exactly two labels in each `set_placement_spacing_rule` command. You can specify any of the labels defined by the `set_placement_spacing_label` command or the predefined SNPS_BOUNDARY label, which includes the chip boundary, hard macro boundaries, a hard macro keepout margins, hard placement blockages, and voltage area guard bands. The two labels can be the same or different.

   - The range of invalid spacings, in number of unit tiles

   For example, to specify that there must be at least one unit tile between labels X and Y (they cannot abut), use the following command:

   ```
   icc2_shell> set_placement_spacing_rule -labels {X Y} {0 0}
   ```

   To specify that there must be at least one unit tile between X labels and any boundary, use the following command:

   ```
   icc2_shell> set_placement_spacing_rule \
        -labels {X SNPS_BOUNDARY} {0 0}
   ```

   To specify that two X labels cannot have a spacing of two unit tiles, use the following command:

   ```
   icc2_shell> set_placement_spacing_rule -labels {X X} {2 2}
   ```

   To specify that labels X and Z must have a spacing of less than two unit tiles or more than four unit tiles, use the following command:

   ```
   icc2_shell> set_placement_spacing_rule -labels {X Z} {2 4}
   ```

Caution:

The cell spacing constraints are not saved with the block; they apply only to the current session and must be redefined in each session.

Feedback

Reporting Cell Spacing Constraints

To report the cell spacing rules, use the `report_placement_spacing_rules` command. This command reports the cell spacing labels defined in the current session, the library cells (and their sides) to which they apply, and the rules defined for them.

Removing Cell Spacing Constraints

To remove spacing rules, use the `remove_placement_spacing_rules` command. You must specify which rules to remove.

- To remove all spacing rules, use the `-all` option.
- To remove a specific label and all rules associated with that label, use the `-label` option to specify the label.
- To remove a rule between labels, use the `-rule` option to specify two labels associated with the rule.

Specifying Placement Settings

The tool performs coarse placement at various points during the preroute stage of the design flow. The following topics describe settings for controlling coarse placement:

- Performing Placement With Inaccurate Constraints at Early Stages
- Generating Automatic Group Bounds for Clock Gating Cells
- Controlling the Placement Density
- Controlling Congestion-Driven Restructuring During Placement
- Reducing Congestion
- Considering Wide Cell Density During Placement
- Considering the Effects of Cell Pins During Placement
- Considering the Congestion Effects Due to the Nondefault Routing Rules of Clock Nets
- Considering the Effects of Clock Gating Cells of Sequential Arrays During Placement
- Considering Legalization Effects During Placement
- Considering DFT Connections During Placement
- Considering the Dynamic Power QoR During Placement
- Performing IR-Drop-Aware Placement
- Spreading Repeater Cells During Placement

Feedback

Performing Placement With Inaccurate Constraints at Early Stages

During the early stages of a design cycle, placement constraints can be inaccurate causing the tool to exit during coarse placement. To continue with placement when a region is overutilized due to it being too small or being covered by blockages, set the `place.coarse.handle_early_data` application option to `true`. When you do so, the tool issues warning messages as shown in the following example output:

```
Warning: Utilization of move_bound_0 is 109%. (PLACE-089)
Warning: Placement continues with over utilized regions in the design.
 (PLACE-083)
Warning: Overutilized regions for move_bound_0 are modified by removal of
 partial and full blockages. (PLACE-084)
```

When you enable this feature, the tool prints a warning message if the utilization of a region is more that 90 percent. To change this threshold, use the `place.coarse.utilization_warning_threshold` application option.

Generating Automatic Group Bounds for Clock Gating Cells

The tool can automatically generate group bounds for integrated clock gating cells and the sequential cells they drive. To do so, use the following application option setting:

```
icc2_shell> set_app_options -name place.coarse.icg_auto_bound  \
 -value true
```

When you enable this feature, the tool creates the automatic group bounds at the beginning of placement and removes them at the end of placement. The tool does not include cells that already belong to another group bound in the automatic group bounds.

To limit the maximum number of fanouts that can be included in an automatic bound, use the `icg_auto_bound_fanout_limit` application option setting, as shown in the following example:

```
icc2_shell> set_app_options \
   -name place.coarse.icg_auto_bound_fanout_limit -value 30
```

The default fanout limit is 40.

Controlling the Placement Density

You can control the placement density of a block as described in the following table.

Feedback

Table 6 Application Options for Controlling Placement Density

To do this	Use this application option
Specify a maximum density that controls how densely the tool can place cells during non-congestion-driven placement	`place.coarse.auto_density_control` (Default is `-enhanced`, and the tool follows a preset schedule during the flow if `place.coarse.auto_density_control` is not set to `false`. Disabling `place.coarse.auto_density_control` or manually setting `place.coarse.max_density` to `0` causes uniform spreading of the cells in the core area)
Specify a maximum utilization that controls how densely the tool can place cells in less congested areas that surround highly congested areas, so that the cells in the congested areas can be spread out to reduce the congestion	`place.coarse.congestion_driven_max_util` (Default is `0.93`)

When specifying the maximum density or maximum utilization, choose a value between 1 and the overall utilization of the block. For example, if the utilization of a block is 40 percent, you can choose values between 1 and 0.4, as shown in the following example:

```
icc2_shell> set_app_options -name place.coarse.max_density -value 0.6
icc2_shell> set_app_options \
   -name place.coarse.congestion_driven_max_util -value 0.8
```

If you do not specify a value for the maximum density or maximum utilization, the tool automatically derives a value for the maximum density and maximum utilization based on the stage of the design flow, considering the `place.coarse.auto_density_control` application option is not set to `false`.

By default, the `place.coarse_auto_density_control` application option is enabled and set to `enhanced`. This improves the total power and wire length for the design.

Note:

It is not necessary to disable the `place.coarse.auto_density_control` application option to override the `max_density` and `congestion_driven_max_util` values. User settings always take precedence over the tool derived values.

Throughout placement, the tool prints PLACE-027 information messages indicating the values it uses for these settings.

```
Information: Automatic density control has selected the following
 settings: max_density 0.60, congestion_driven_max_util 0.77. (PLACE-027)
```

Feedback

The tool applies the maximum density and congestion-driven maximum utilization settings independently to each placeable area such as a voltage area or exclusive move bound, rather than taking an average over the entire block.

Controlling Congestion-Driven Restructuring During Placement

By default, the tool restructures nets to reduce congestion during

- The `create_placement` command, except when you use the `-timing_driven` option
- The `initial_place` stage of the `place_opt` command

You can control the congestion-driven restructuring as follows:

- Specify an effort level by setting the `place.coarse.cong_restruct_effort` application option to `low`, `medium` (default), `high`, or `ultra`.
- Prevent the tool from increasing the path depth by more than three levels of logic by setting the `place.coarse.cong_restruct_depth_aware` application option to `true`.

Reducing Congestion

To reduce the congestion of your block, use the following settings:

- Consider the congestion of each layer separately and improve the accuracy of congestion reduction in coarse placement by setting the `place.coarse.congestion_layer_aware` application option to `true`.

  By default, the tool combines the congestion information of all layers during placement. By considering the congestion of each layer separately, the tool can identify and fix areas with high pin densities that have insufficient resources on the lower layers for making pin connections.

- Expand the virtual area of cells during placement based on the local routing resource needs by setting the `place.coarse.increased_cell_expansion` application option to `true`.

  By increasing the virtual area of cells in areas where there is a shortage of routing resources, the tool can minimize the cell density and congestion.

  By default, the tool expands the cells in the horizontal direction, which helps reduce congestion in the vertical routing layers. For designs with congestion mainly in the horizontal routing layers, set the `place.coarse.congestion_expansion_direction` application option to `both`. The default is `horizontal`.

  By default, the tool uses the uses global route congestion map to identify highly congested areas that need cell expansion. However, the tool does not consider the

Feedback

congestion due to soft routing rules. To consider the congestion effects of soft routing rules, set the `route.global.export_soft_congestion_maps` application option to `true`.

Considering Wide Cell Density During Placement

If a cell cannot straddle the vertical power straps and the width of the cell is more than half the pitch of the power straps, only one such cell can be placed between the power straps. Therefore, during placement, the tool must minimize the density of such wide cells to ensure that they can be placed and legalized without large displacements.

For advanced technology nodes with wide cells, enable wide-cell modeling during placement by setting the `place.coarse.wide_cell_use_model` application option to `true`.

Considering the Effects of Cell Pins During Placement

When you specify a technology node setting of `7`, `7+`, `5`, `s5`, or `s4` by using the `set_technology -node` command, you can specify that the tool uses a technology-specific pin-cost model during placement by setting the `place.coarse.pin_cost_aware` application option to `true`. The default is `false`. When you enable this feature, during placement, the tool tries to improve pin accessibility by using a technology-specific model to predict the routing resources required to access each pin.

For all other technology nodes, you can control the maximum local pin density during placement by setting the `place.coarse.pin_density_aware` application option to `true`. The default is `false`.

Table 7 Settings for Controlling the Effects of Pins During Placement

set_technology -node	**place.coarse. pin_cost_aware**	**place.coarse. pin_density_aware**	**The tool does this**
`7`, `7+`, `5`, `s5`, or `s4`	`true`	`true` or `false`	Performs technology-specific pin-cost-aware placement
`7`, `7+`, `5`, `s5`, or `s4`	`false`	`true`	Performs pin-density-aware placement
`7`, `7+`, `5`, `s5`, or `s4`	`false`	`false`	Does not consider the effects of cell pins during placement
Any other setting	`true` or `false`	`true`	Performs pin-density-aware placement
Any other setting	`true` or `false`	`false`	Does not consider the effects of cell pins during placement

Feedback

Considering the Congestion Effects Due to the Nondefault Routing Rules of Clock Nets

To improve timing and reduce crosstalk, nondefault routing rules are used extensively for routing clock nets. However, these nondefault routing rules require additional space, which can increase the congestion after clock routing.

To consider the effects of the nondefault routing rules of the clock nets during placement, set the `place.coarse.ndr_area_aware` application option to `true`.

Considering the Effects of Clock Gating Cells of Sequential Arrays During Placement

A sequential array, which is commonly used in storage devices, is a group of registers that are arranged as a two-dimensional array. The clock pins of the registers along a row of a sequential array are usually driven by a single clock-gating cell.

To consider the clock-gating cells of a sequential array during placement, set the `place.coarse.seq_array_icg_aware` application option to `true`. Doing so can reduce the congestion caused by the clock nets.

Considering Legalization Effects During Placement

To minimize many cells from being displaced during legalization, the tool should be aware of legalization limits and restrictions during coarse placement. For example, the tool should be aware that it cannot place a wide cell in a specific location due to the position of the power straps at that location.

To consider legalization effects during placement, set the `place.coarse.enhanced_legalizer_driven_placement` application option to `true`. Use this feature for blocks that have many cells with small to medium displacements during legalization and use the RMS displacement values reported during legalization to see if it reduces the displacement.

Considering DFT Connections During Placement

To improve the DFT logic placement by enhancing the tool's awareness of DFT connectivity, while still optimizing for functional placement, set the `place.coarse.enable_dft_modeling` application option to `auto`. The default is `false`.

Feedback

Considering the Dynamic Power QoR During Placement

To consider the dynamic power QoR during placement, perform the following steps:

1. Annotate switching activity on the design, as described in Annotating the Switching Activity.
2. Ensure that at least one scenario is enabled for dynamic-power optimization by using the `set_scenario_status -dynamic_power true` command.
3. Enable power-driven placement for the `create_placement`, `refine_placement`, `place_opt`, or `clock_opt` command by using one of the following methods:
   - Enable dynamic-power-driven placement by setting the `place.coarse.enhanced_low_power_effort` application option to `none`, `low`, `medium`, or `high`. The default is `low`.

     During dynamic-power-driven placement, the tool tries to improve both the timing and power of timing-critical nets and the power of the other nets. This improves the power QoR without affecting the timing QoR.

Performing IR-Drop-Aware Placement

During placement, the tool can use the voltage (IR) drop values of cells to identify areas of high power density and spread the cells with high voltage drop values, which reduces the power density of such areas.

To perform IR-drop-aware placement, use the following steps:

1. Place and legalize the block.
2. Set up for RedHawk Fusion and perform static or dynamic voltage drop analysis by using the `analyze_rail -voltage_drop` command as shown in the following example:

   ```
   icc2_shell> source redhawk_setup.tcl
   icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
   ```

   For more information, see Performing Voltage Drop Analysis.
3. Enable IR-drop-aware placement by setting the `place.coarse.ir_drop_aware` application option to `true`.
4. (Optional) Specify additional settings for IR-drop-aware placement, as described in Controlling IR-Drop-Aware Placement.
5. Rerun placement.

Feedback

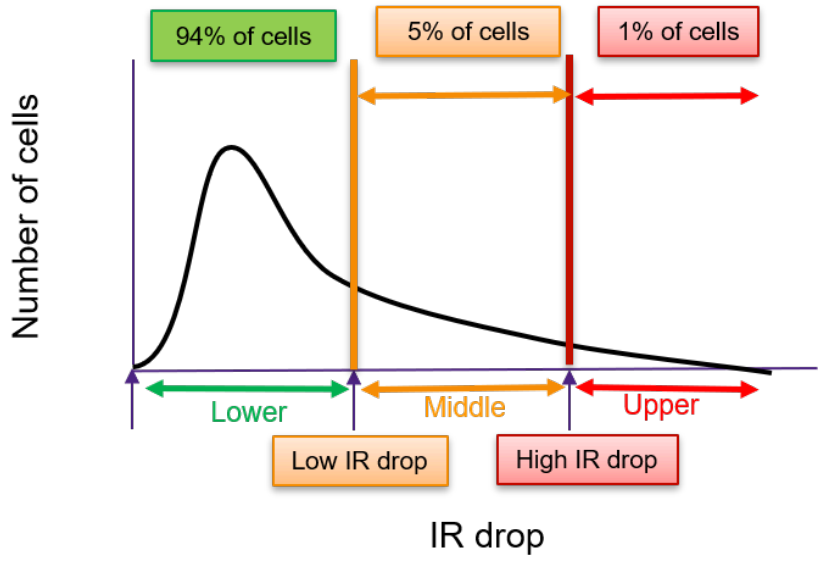
Note:

To perform IR-drop-aware placement, you must have Digital-AF and SNPS_INDESIGN_RH_RAIL license keys.

Controlling IR-Drop-Aware Placement

When you enable IR-drop-aware placement by setting the `place.coarse.ir_drop_aware` application option to `true`, the tool creates three categories of cells based on the total number of cells.

Figure 18 Default Cell Categories for IR-Drop-Aware Placement



By default, the tool selects cells for the three categories by using the criteria shown in Figure 18.

- The upper category consists of the top one percent of the total number of cells with the highest voltage drop values. During placement, the tool spreads these cells the most.
- The middle category consists of the next five percent of the cells. During placement, the tool spreads these cells less than those in the upper category.
- The lower category consists of the rest of the cells. The tool does not spread these cells.

You can change the percentage of cells in the upper and middle categories by using the `place.coarse.ir_drop_default_target_high_percentage` and `place.coarse.ir_drop_default_target_low_percentage` application options.

Feedback

The following example puts the top 2 percent of cells in the upper category and the next 6 percent of cells in the middle category:

```
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_high_percentage \
   -value 2.0
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_low_percentage \
   -value 6.0
```

Instead of using a percentage of the total number of cells to define the cell categories, you can use the cell voltage drop as a percentage of supply voltage by setting the `place.coarse.ir_drop_default_target_high_population` and `place.coarse.ir_drop_default_target_low_population` application options to `false`. They are `true` by default.

The following example puts cells with a voltage drop larger than 8 percent of the supply voltage into the upper category and cells with a voltage drop between 4 to 8 percent of the supply voltage into the middle category:

```
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_high_population \
   -value false
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_high_percentage \
   -value 8.0
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_low_population \
   -value false
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_low_percentage \
   -value 4.0
```

You can specify a percentage of the total number of cells to define the upper category and cell voltage drop as a percentage of supply voltage to define the middle category, or vice versa.

The following example puts the top 2 percent of the total cells in the upper category, and the next cells that have a voltage drop larger than 4 percent of supply voltage in the middle category:

```
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_high_population \
   -value true
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_high_percentage \
   -value 2.0
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_low_population \
   -value false
```

Feedback

```
icc2_shell> set_app_options \
   -name place.coarse.ir_drop_default_target_low_percentage \
   -value 4.0
```

You can set constraints for specific voltage areas by using the `set_placement_ir_drop_target` command. The following example puts the top 1.5 percent of the cells in the VA1 voltage area into the upper category and the next 4.5 percent of the cells into the middle category:

```
icc2_shell> set_placement_ir_drop_target VA1 high 1.5
icc2_shell> set_placement_ir_drop_target VA1 low 4.5
```

The following example puts the cells with a voltage drop larger than 10 percent of the supply voltage in the VA2 voltage area into the upper category and the cells with a voltage drop between 5 and 10 percent of the supply voltage into the middle category:

```
icc2_shell> set_placement_ir_drop_target VA2 high 10 -irdrop
icc2_shell> set_placement_ir_drop_target VA2 low 5 -irdrop
```

To get, report, and reset the voltage-area-based constraints, use the `get_placement_ir_drop_target`, `report_placement_ir_drop_target`, and `reset_placement_ir_drop_target` commands.

Spreading Repeater Cells During Placement

If a block has many chains of repeater cells (buffers, inverters, or pipelined registers) along the edges or corners of macro cells or blockages, the tool might clump the cells along the edges or corners, increasing congestion. You can reduce this congestion by setting the `place.coarse.spread_repeater_paths` application option to `true` before you run the `create_placement`, `place_opt`, or `clock_opt` command. Then, the tool reduces congestion along the edges and corners by spreading the repeater cells in an orthogonal direction.

Specifying Legalization Settings

The tool performs legalization at various points during the design flow. The following topics describe settings for controlling legalization:

- Minimizing Large Displacements During Legalization
- Optimizing Pin Access During Legalization
- Enabling Concurrent Legalize and Detail Rroute
- Enabling Advanced PG Net Checks
- Enabling Advanced Legalization Algorithms

Feedback

- Setting Up for Variant-Aware Legalization
- Checking if Library Cells Are Legally Placeable

Minimizing Large Displacements During Legalization

To minimize large displacements of cells during legalization, enable

- Orientation optimization by setting the `place.legalize.optimize_orientations` application option to `true`.

  When you do so, the tool considers flipping the orientations of cells to reduce the displacements during legalization.

- Stream placement by setting the `place.legalize.stream_place` application option to `true`.

  When you do so, the tool moves many cells a small distance, to avoid moving a single cell a large distance during legalization. You can further control stream placement by using the `place.legalize.stream_effort` and `place.legalize.stream_effort_limit` application options.

Optimizing Pin Access During Legalization

For advanced technology nodes, to improve the routability of areas with high pin densities by redistributing the cells, enable pin optimization during legalization by setting the `place.legalize.optimize_pin_access_using_cell_spacing` application option to `true`.

Enabling Concurrent Legalize and Detail Rroute

Concurrent Legalize and Detail Rroute (CLDR) is a DRC convergence application to fix pin access DRC. To enable the CLDR integration to improve DRC convergence after RO or HRO, set the `place.legalize.enable_cldr` application option to `true`.

Example:

```
set_app_options -name place.legalize.enable_cldr -value true
```

Enabling Advanced PG Net Checks

For advanced technology nodes, such as 12 nanometer, 7 nanometer, or smaller technology nodes, you can enable advanced physical DRC checks between cells and prerouted PG nets by setting the `place.legalize.enable_advanced_prerouted_net_check` application option to `true`.

Feedback

When you enable this feature, you can further control the accessibility checks performed for standard cell pins by using the `place.legalize.advanced_layer_access_check` and `place.legalize.advanced_libpin_access_check` application options. However, the default behavior of this feature is suitable for most designs.

Enabling Advanced Legalization Algorithms

To enable advanced legalization algorithms for 2D rule checking and cell interaction, which can reduce legalization runtime, set the `place.legalize.enable_advanced_legalizer` application option to `true`.

To specify additional advanced legalization rules that are not automatically detected, set the `place.legalize.enable_advanced_legalizer_rules` application option.

When you enable the advanced legalization algorithms, by default, the tool aligns vertical pin shapes during pin access optimization to improve routability. However, you can specify additional strategies for pin access optimization by setting the `place.legalize.optimize_pin_access_strategies` application option as shown in the following table.

Table 8 Settings for the place.legalize.optimize_pin_access_strategies Application Option

To do this	Use this setting
Align horizontal pin shapes	`horizontal_align`
Move cells away from areas of high pin density	`avoid_high_pin_density`
Align horizontal pin shapes and move cells away from areas of high pin density	`horizontal_align avoid_high_pin_density`

During advanced legalization and legality checking, the tool can simultaneously check for DRC violations between multiple cells and PG net shapes. To enable this feature, set the `place.legalize.enable_multi_cell_pnet_checks` application option to `true`.

To enable multi cell checks at the last run of `route_opt` command or after the `route_auto` command, set the following application options to `true`. This helps to improve the

Feedback

runtime for QoR considerations. To enable the following application options, set the `place.legalize.enable_multi_cell_pnet_checks` application option to `true`:

- `place.legalize.enable_multi_cell_access_check`: considers impact on a cell's pin access due to presence of abutting neighbor cells and any prerouted net PG net shapes in the vicinity. Default is `false`.
- `place.legalize.enable_multi_cell_track_capacity_check`: analyzes the track capacity of the cell's pins and abutting neighbor cells. Track capacity estimates the sufficiency of routing tracks in the area to access all the specified pins. To enable this application option, set the `place.legalize.enable_multi_cell_access_check` application option to `true`. Default is `false`.

To enable multithreaded advanced legalization, perform the following steps:

1. Enable multithreaded advanced legalization algorithms by setting the `place.legalize.enable_threaded_advanced_legalizer` application option to `true`.
2. Configure for multithreading as described in Configuring Multithreading.

To perform multithreaded advanced legality checking,

1. Enable advanced legalization algorithms by setting the `place.legalize.enable_advanced_legalizer` application option to `true`.
2. Configure for multithreading as described in Configuring Multithreading.
3. Perform legality checking using the `check_legality` command.

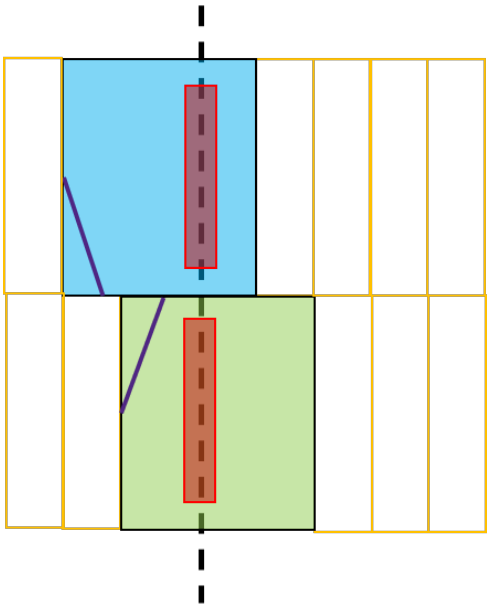
Setting Up for Variant-Aware Legalization

Some libraries provide sets of functionally equivalent cells that can be legalized in different locations. This allows the tool to legalize a cell by replacing it with a variant, instead of moving the cell.

The following figure shows functionally equivalent cells with varying pin locations. With variant-aware legalization, if a cell pin does not align with a track, the tool can try using its variant to align the cell pin to the track.

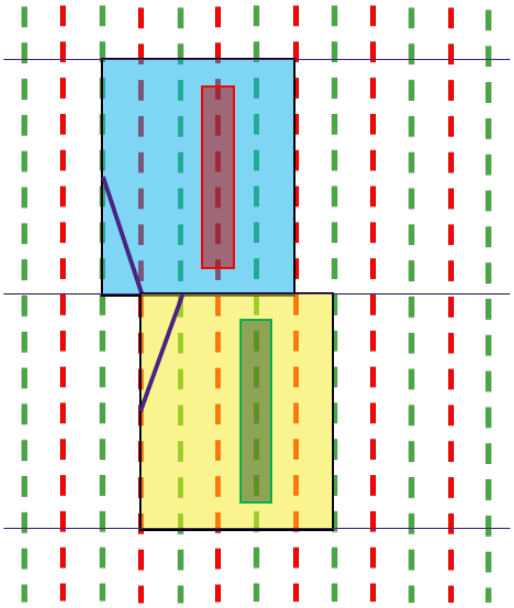
Feedback

Figure 19 Equivalent Cells With Different Pin Locations



The following figure shows functionally equivalent cells with different pin colors, which need to be aligned to a track of the same color. With variant-aware legalization, if a cell pin does not align with the corresponding colored track, the tool can try using its variant with a different colored pin and try aligning it to the appropriately colored track.

Figure 20 Equivalent Cells With Pins of Different Colors



Feedback

The following topics describe the tasks you need to perform to set up the reference libraries and design for variant-aware legalization:

- Defining Equivalent Cell Groups
- Enabling Variant-Aware Legalization

Defining Equivalent Cell Groups

The Library Compiler tool supports creating variant-ready libraries from the Liberty source files. The `physical_variant_cells` attribute in the Liberty source defines the variants of a cell. When the `physical_variant_cells` attribute exists in the logic libraries, the compiled library is variant ready.

If you logic library is not variant ready, you can define the equivalent cell variants during the library preparation stage in the Library Manager tool. To do so, create equivalent cell groups by using the `create_cell_groups` command. The following example adds equivalent cell groups to an existing reference library in the Library Manager tool:

```
lm_shell> create_workspace -flow edit LIB.ndm
lm_shell> create_cell_group -name A [get_lib_cells MY_LIB/A_*]
lm_shell> create_cell_group -name B [get_lib_cells MY_LIB/B_*]
lm_shell> check_workspace
lm_shell> commit_workspace
```

You can report the equivalent cell groups in a library by using the `report_cell_groups` command in either the Library Manager tool or the IC Compiler II tool. This command reports equivalent cell groups that are defined in Liberty source files of variant-ready logic libraries as well as those defined by using the `create_cell_groups` command in the Library Manager tool.

Enabling Variant-Aware Legalization

To enable variant-aware legalization, set the `place.legalize.enable_variant_aware` application option to `true`.

When variant-aware legalization is enabled, the tool tries to fix legalization DRC violations by swapping the violating cell with a variant, rather than moving it. If the cell has no variants, it is moved to a legal location.

For pin-track and pin-color alignment during variant-aware legalization, set the following application options to `true`:

- `place.legalize.enable_prerouted_net_check`
- `place.legalize.enable_pin_color_alignment_check`

In addition, you must specify the layers to be aligned by using the `place.legalize.pin_color_alignment_layers` application option.

Feedback

Checking if Library Cells Are Legally Placeable

After you specify all your placement and legalization constraints and settings, you can check if specific library cells can be legally placed in the block by using the `analyze_lib_cell_placement -lib_cells` command.

- To limit the analysis to a specific region of the core area, use the `-region` option. By default, the command searches the entire core area.
- To limit the analysis to a specific number of sites, use the `-trials` option. By default, the command analyzes 1000 random sites to see if the specified library cells can be placed. If you specify a value of 0, the command analyzes all free sites, which can increase runtime
- To ignore physical design constraints or advanced design rules during placement analysis, use the `-no_pdc` or `-no_adv` option.
- To report only the relative placement groups that do not meet a specific threshold, use the `-threshold` option. The tool reports a cell only if the percentage of sites the cell can be placed, relative to the total number of sites analyzed, is less than the specified threshold.
- To limit the report to a maximum number of cells, use the `-max_cells` option. The default is 100, starting with the library cell with the worst pass rate.

If your block contains library cells that have a very low pass rate, they will cause large displacements and increased runtime during legalization. In such cases, review the power plan, cell layout, and the legalization settings to improve the pass rate.

The following example analyzes all the library cells that have instances in the current block:

```
icc2_shell> analyze_lib_cell_placement -lib_cells [add_to_collection \
 -unique "" [get_attribute [get_cells -physical_context] ref_phys_block]]

Analyzing 215 lib cell(s).
Warning: Routing direction of metal layer PO is neither "horizontal" nor
 "vertical".  PDC checks will not be performed on this layer. (PDC-003)

PDC app_options settings =========
        place.legalize.enable_prerouted_net_check: 1
        place.legalize.num_tracks_for_access_check: 1
        place.legalize.use_eol_spacing_for_access_check: 0
        place.legalize.allow_touch_track_for_access_check: 1
        place.legalize.reduce_conservatism_in_eol_check: 0
        place.legalize.preroute_shape_merge_distance: 0.0

Layer M1: cached 0 shapes out of 869 total shapes.
Layer M2: cached 773 shapes out of 773 total shapes.
Cached 41757 vias out of 99466 total vias.
```

```
0.0%...2.0%...4.0%...6.0%...8.0%...10.0%........

  Lib Cell                                  Pass Rate
  ----------------------------------------  ---------
  saed32_lvt_lsup:LSUPX1_LVT.frame          0.4450
  saed32_lvt_lsup:LSUPX8_LVT.frame          0.4780
  saed32_lvt_lsup:LSUPX2_LVT.frame          0.4810
  saed32_lvt_lsup:LSUPX4_LVT.frame          0.5040
  ..............
  ..............
  ..............
  saed32_rvt_std:HADDX1_RVT.frame           0.8600
  saed32_lvt_std:OA21X1_LVT.frame           0.8610
```

Controlling the Optimization of Cells, Nets, Pins, and Ports

The following topics describe how you can control the optimization of cells, nets, pins, and ports:

- Preserving Cells and Nets During Optimization
- Restricting Optimization to Cell Sizing Only
- Preserving Networks During Optimization
- Marking the Clock Networks
- Disabling Design Rule Checking (DRC)
- Preserving Pin Names During Sizing
- Preserving Ports of Existing Hierarchies
- Isolating Input and Output Ports
- Fixing Multiple-Port Nets
- Controlling the Addition of New Cells to Modules, Hierarchical Cells, and Voltage Areas
- Specifying a Cell Name Prefix for Optimization

Preserving Cells and Nets During Optimization

To preserve design objects during optimization, use the `set_dont_touch` command. The command places the `dont_touch` attribute on cells, nets, references, and subdesigns in the current design to prevent these objects from being modified or replaced during optimization.

This table shows how the tool preserves the design objects during optimization.

Feedback

Design objects	Tool behavior
Cells	Excludes the cells from optimization.
Nets	Preserves the connectivity of all pins to the nets.
Modules, designs, library cells	Prevents optimization of all instances that reference them.
Hierarchical cells or modules	Excludes the cells and all child cells from optimization.

When you use the `set_dont_touch` command, keep the following in mind:

- The command prevents the ungrouping of hierarchy.
- A child cell must inherit the same `dont_touch` attribute from its parent module. For example, you cannot remove the `dont_touch` attribute from a child cell when its parent module is marked with the `dont_touch` attribute.
- Use the command on fully mapped logic only, including cells, modules, and designs, as well as nets with fully mapped surrounding logic.
- The `dont_touch` attribute has higher precedence over the `boundary_optimization` and the `size_only` attributes.

To report design objects marked with the `dont_touch` attribute, use the `report_dont_touch` command.

This command sets the `dont_touch` attribute on cell A.

```
icc2_shell> set_dont_touch [get_cells A] true
```

To remove the `dont_touch` attribute, use the `remove_attributes` command or the `set_dont_touch` command set to `false`, as shown in the following two commands:

```
icc2_shell> remove_attributes [get_cells A] dont_touch
icc2_shell> set_dont_touch [get_cells A] false
```

The following table summaries the commands that you can use to set, remove, or report the `dont_touch` attribute on design objects, including modules, cells, nets, and library cells. Yes denotes the command operation of the `dont_touch` attribute is supported, while No denotes the command operation is not supported.

Table 9 Command Support for the dont_touch Attribute on Design Objects

Command	Module	Cell (Instance)	Net	Library cell
`set_dont_touch`	Yes	Yes	Yes	Yes

Feedback

Table 9 Command Support for the dont_touch Attribute on Design Objects (Continued)

Command	Module	Cell (Instance)	Net	Library cell
`set_attribute`	Yes	Yes	Yes	Yes
`define_user_attribute`	No	No	No	No
`get_attribute`	Yes	Yes	Yes	Yes
`remove_attributes`	Yes	Yes	Yes	Yes
`report_dont_touch`	Yes	Yes	Yes	No
`set_dont_touch` *`unmapped_object`*	No	No	Yes	NA

Restricting Optimization to Cell Sizing Only

To allow only sizing on cells or instances during optimization, use the `set_size_only` command. The command places the `size_only` attribute on specified cells or instances to prevent them from being modified or replaced except cell sizing.

To query the `size_only` attribute, use the `report_attributes`, `get_attribute`, or `report_cells` command. To remove the attribute, use the `remove_attributes` command or the `set_size_only` command set to `false`.

This example sets the `size_only` attribute on a specific cell.

```
icc2_shell> set_size_only [get_cells cell_name] true
```

This example queries the `size_only` attribute on the cell_name cell.

```
icc2_shell> get_attribute [get_cells cell_name] size_only
```

This example reports all the `size_only` information on design objects in the current design.

```
icc2_shell> report_size_only -all
```

Preserving Networks During Optimization

You can preserve networks, such as clock trees, during optimization by using the `set_dont_touch_network` command. The command places the `dont_touch` attribute on the clocks, pins, or ports to prevent the cells and nets in the transitive fanout of the network from being modified or replaced.

Feedback

By default, the command preserves both clock paths and clock as data paths and propagates the dont_touch attribute throughout the hierarchy of the network. To preserve clock paths only, specify the -clock_only option. To remove the dont_touch attribute from the network, specify the -clear option.

This example sets the dont_touch attribute on the CLK clock network.

```
icc2_shell> set_dont_touch_network [get_clocks CLK]
```

This example sets the dont_touch attribute on the CLK clock paths only.

```
icc2_shell> set_dont_touch_network [get_clocks CLK] -clock_only
```

This example removes the dont_touch attribute from the CLK clock network.

```
icc2_shell> set_dont_touch_network [get_clocks CLK] -clear
```

Marking the Clock Networks

If a block does not contain clock trees, you should perform optimization using ideal clocks.

To mark all clock networks as ideal, use the following command:

```
foreach_in_collection mode [all_modes] {
 current_mode $mode
 set_ideal_network [all_fanout -flat -clock_tree]
}
```

To model the clock tree effects for placement, you should also define the uncertainty, latency, and transition constraints for each clock by using the set_clock_uncertainty, set_clock_latency, and set_clock_transition commands.

Before performing clock tree synthesis, you must use the remove_ideal_network command to remove the ideal setting on the fanout of the clock trees.

Disabling Design Rule Checking (DRC)

You can disable design rule checking (DRC) on clock, constant, scan enable, and scan clock nets by using the set_auto_disable_drc_nets command.

To control design rule checking on specific nets in the current design, use the options with the set_auto_disable_drc_nets command as shown in Table 10.

Table 10 Options for the set_auto_disable_drc_nets Command

Use this option	To do this
-none	Enable DRC for all nets.

Feedback

Table 10 Options for the set_auto_disable_drc_nets Command (Continued)

Use this option	To do this
`-all`	Disable DRC on all clock, constant, scan enable, and scan clock nets.
`-constant true | false`	Disable (set to `true`) or enable (set to `false`) DRC on constant nets.
`-on_clock_network true | false`	Disable (set to `true`) or enable (set to `false`) DRC on clock networks.
`-scan true | false`	Disable (set to `true`) or enable (set to `false`) DRC on scan enable and scan clock nets.

For example, to enable design rule checking for all clock, constant, scan enable, and scan clock nets, specify the `-none` option:

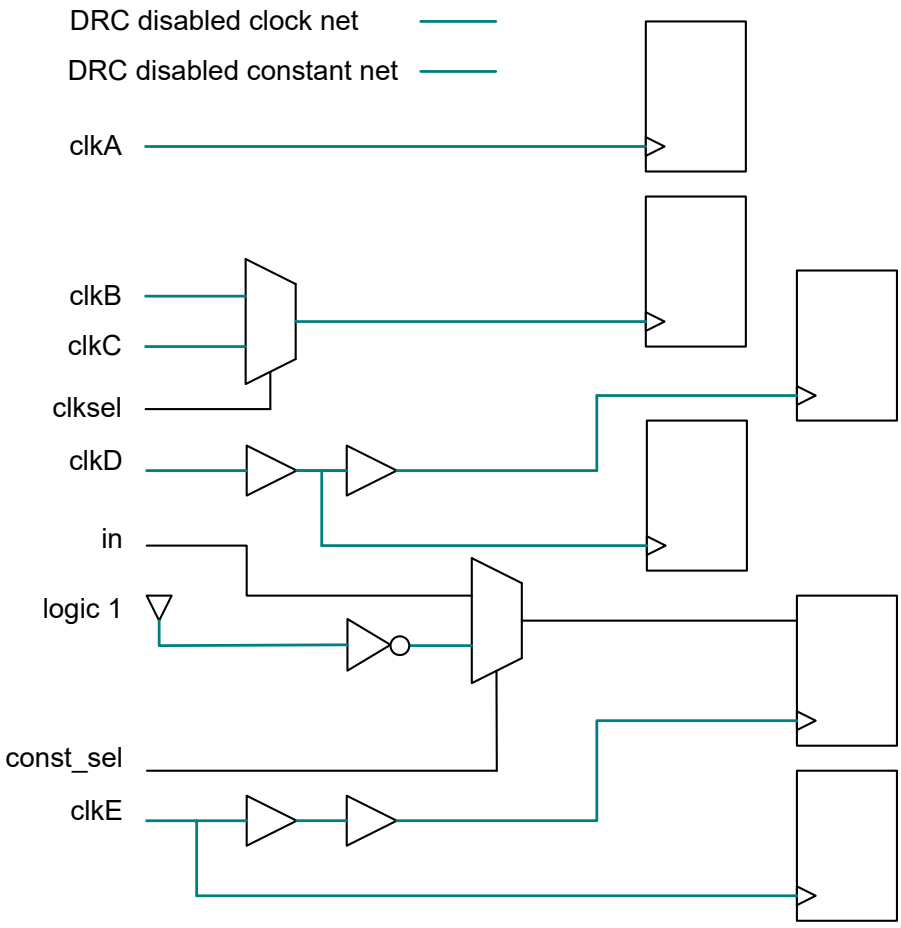
```
icc2_shell> set_auto_disable_drc_nets -none
```

To disable design rule checking on nets connecting to clocks and constants, set the `-on_clock_network` and `-constant` options to `true`, as shown in the following example and figure:

```
icc2_shell> set_auto_disable_drc_nets \
   -on_clock_network true -constant true
```

Feedback

Figure 21 DRC Disabled Clock and Constant Nets Highlighted



Preserving Pin Names During Sizing

By default, optimization can change the pin names of leaf cells during sizing if the functionality of the resulting circuit is equivalent. The IC Compiler II tool automatically updates its version of the internal constraints to reflect the new pin name if the pin is part of an exception constraint. If this occurs, it means that the original constraints used to constrain the block in the IC Compiler II tool cannot be used for signoff purposes; instead, you must write out the constraints from the IC Compiler II tool and include these constraints with the resulting block.

This behavior provides the optimization engine with the most flexibility to select cells and improve the cost functions. You can restrict this sizing capability so that the constraints remain unchanged through optimization by setting the `opt.common.preserve_pin_names` application option. This application option defaults to the setting of `never`, and accepts values of either `never` or `always`.

Feedback

To restrict sizing to pin-name equivalent cells, use the following command:

```
icc2_shell> set_app_options \
   -name opt.common.preserve_pin_names -value always
```

To restore the default behavior, use the following command:

```
icc2_shell> set_app_options \
   -name opt.common.preserve_pin_names -value never
```

Preserving Ports of Existing Hierarchies

During optimization and clock tree synthesis, the tool can create new ports on existing hierarchical blocks. To prevent the tool from doing so, use the `set_freeze_ports` command and set the `freeze_clock_port` attribute to `true` on the corresponding cell instance.

You can prevent the tool from adding clock ports, data ports, or both by using the `-clock`, `-data`, or `-all` option. For example, to prevent the tool from creating additional clock ports on the MBX22 cell instance, use the following command:

```
icc2_shell> set_freeze_ports -clock [get_cells MBX22] true
```

To report the freeze-port settings, use the `report_freeze_ports` command.

Isolating Input and Output Ports

You can isolate input and output ports to improve the accuracy of timing models. To insert isolation logic at specified input or output ports, use the `set_isolate_ports` command.

Isolation logic can be either a buffer or a pair of inverters; by default, the command places buffers to isolate

- An input port from its fanout networks
- An output port from its driver

When you specify the `-force` option with the `set_isolate_ports` command, the tool ensures that the library cells specified by the `-driver` option are not sized.

Note that the tool does not place isolation logic on the following ports:

- Bidirectional ports
- Ports defined as clock sources or power pins
- Ports connected to nets that are marked with the `dont_touch` attribute

Feedback

Example

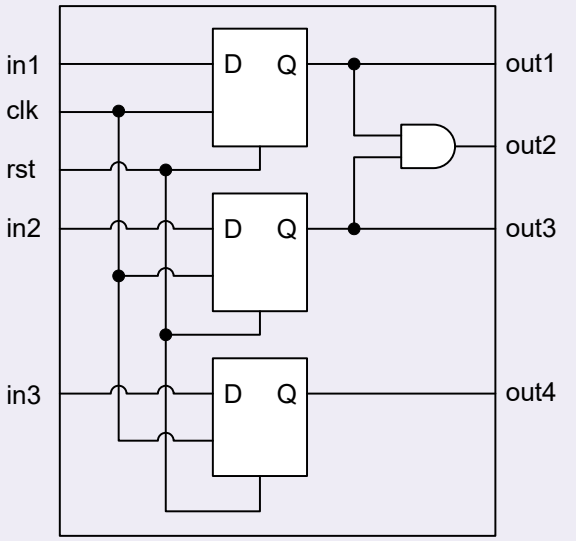
The following commands insert isolation logic to

- Input port in3 with a buffer
- Output port out1 with a buffer
- Output port out3 with the LIBCELL_BUFF library cell
- Output port out4 with a pair of inverters

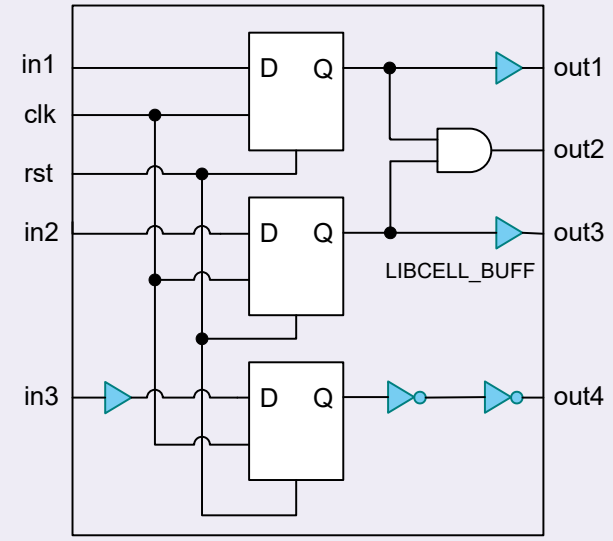
```
icc2_shell> set_isolate_ports {in3 out1}
icc2_shell> set_isolate_ports -driver LIBCELL_BUFF out3 -force
icc2_shell> set_isolate_ports -type inverter out4
```

The following figures show the design without and with the isolation logic inserted by the `set_isolate_ports` commands, including buffers and an inverter pair:

Without port isolation



After running the port isolation commands

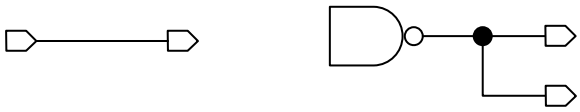


Fixing Multiple-Port Nets

Multiple-port nets include

- Feedthrough nets, where an input port feeds into an output port
- Nets connected to multiple output ports, logically equivalent outputs

Feedback



Feedthrough net Logically equivalent outputs

Multiple-port nets, which are represented with the `assign` statement in the gate-level netlist, might cause design rule violations further in the design flow. By default, the tool does not fix multiple-port nets during optimization.

To fix multiple-port nets, set the `opt.port.eliminate_verilog_assign` application option to `true`. This application option is `false` by default.

During multiple-port-net fixing, the tool performs the following tasks in sequence:

1. Rewires the connections to ensure that each net is connected to one hierarchical port, if possible.

   The tool first performs rewiring to avoid unnecessary buffering. The tool also rewires across the hierarchy because boundary optimization is enabled by default.

2. Inserts buffers or inverters to the nets that are not fixed by rewiring.

Controlling the Addition of New Cells to Modules, Hierarchical Cells, and Voltage Areas

To control the addition of new cells to

- Modules or hierarchical cells, use the `set_allow_new_cells` command
- Voltage areas, use the `-allow_new_cells` option with the `create_voltage_area_rule` command

You can use these settings to prevent new cells from being added to the top-level hierarchies and voltage areas of abutted designs. These settings are saved in the design library and honored by the tool throughout the implementation flow.

The following example prevents new cells from being added to the top-level module:

```
icc2_shell> set_allow_new_cells \
   [get_attribute [current_block] top_module] false
```

The following example prevents new cells from being added to the M1 module:

```
icc2_shell> set_allow_new_cells [get_modules M1] false
```

Feedback

The following example prevents new cells from being added to the U22 hierarchical cell:

```
icc2_shell> set_allow_new_cells [get_cells U22] false
```

The following example prevents new cells from being added to the VA1 voltage area:

```
icc2_shell> create_voltage_area_rule -name VA1_rule \
   -allow_new_cells false -voltage_areas VA1
```

Specifying a Cell Name Prefix for Optimization

You can specify a name prefix for the cells added on the data nets during optimization by using the `opt.common.user_instance_name_prefix` application option.

The following example specifies a name prefix of

- PO_ for the cells added on the data nets during the `place_opt` command
- CO_ for the cells added on the data nets during the `clock_opt` command
- RO_ for the cells added on the data nets during the `route_opt` command

```
icc2_shell> set_app_options \
   -name opt.common.user_instance_name_prefix -value "PO_"
icc2_shell> place_opt
icc2_shell> set_app_options \
   -name opt.common.user_instance_name_prefix -value "CO_"
icc2_shell> clock_opt
icc2_shell> set_app_options \
   -name opt.common.user_instance_name_prefix -value "PO_"
icc2_shell> route_opt
```

To specify a name prefix for the cells added on the clock network during clock tree synthesis, use the `cts.common.user_instance_name_prefix` application option, as described in Defining a Name Prefix for Clock Cells.

Specifying Settings for Preroute Optimization

Optimizing the design for performance, power, and area (PPA) is one of the primary goals of the tool. The following topics describe how to specify settings for performance, power, and area optimization at the preroute stage:

- Controlling DFT Optimization for the place_opt Command
- Specifying Parasitic Estimation Settings for the Preroute Optimization
- Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

Feedback

- Enabling Area Recovery in Regions of High Utilization
- Enabling Advanced Logic Restructuring

Controlling DFT Optimization for the place_opt Command

If your block contains scan chains, by default, the `create_placement`, `place_opt`, and `clock_opt` commands perform physical DFT optimization. During initial placement, the tool focuses on the QoR for the functional nets by ignoring the scan chains. After initial placement, the tool further improves the QoR by repartitioning and reordering the scan chains based on the initial placement.

Using the physical information to repartition and reorder the scan chains

- Reduces scan chain wire length
- Minimizes congestion and improves routability

To run the `create_placement`, `place_opt` or `clock_opt` command on blocks with scan chains, the block must contain SCANDEF information. To annotate SCANDEF information, use the `read_def` command.

```
icc2_shell> read_def ./myscan.def
```

Note:

If you have previously annotated the scan chain information on the block and the SCANDEF data contains a chain with the same name as an existing scan chain, the existing chain definition is preserved.

The SCANDEF file specifies the scan chain components and their pins used for the scan path, along with reordering constraints. For best results, you should use TestMAX DFT for DFT insertion and Design Compiler topographical mode to generate the SCANDEF file. For more information about DFT insertion, see the *TestMAX DFT User Guide*. For more information about generating the SCANDEF file, see the *Design Compiler User Guide*.

The tool uses the SCANDEF information during DFT optimization. You can control DFT optimization as described in the following table.

Table 11 Settings for Controlling DFT Optimization

To do this	Use this setting
Run the `create_placement`, `place_opt` or `clock_opt` command for blocks with no SCANDEF information	Set the `place.coarse.continue_on_missing_scandef` application option to `true`
Disable DFT optimization	Set the `opt.dft.optimize_scan_chain` application option to `false`

Feedback

Table 11 Settings for Controlling DFT Optimization (Continued)

To do this	Use this setting
Disable scan chain repartitioning	Set the `opt.dft.do_repartition` application option to `false`
Disable scan chain reordering based on timing	Set the `opt.dft.timing_friendly_reorder` application option to `false`
Consider hold violations along scan paths during reordering Use this feature after clock tree synthesis to reduce hold violations on scan paths	Set the `opt.dft.clock_aware_scan_reorder` application option to `true`
Consider hold violations along scan paths during repartitioning and reordering Use this feature after clock tree synthesis to reduce hold violations on scan paths	Set the `opt.dft.clock_aware_scan_repartition_reorder` application option to `false`
Preserve hierarchical ports during DFT optimization When you enable this feature, the tool does not perform timing-friendly or clock-aware DFT optimization	Set the `opt.dft.hier_preservation` application option to `true`

To report the application option settings specified for DFT optimization, use the `report_app_options` command.

To perform standalone DFT optimization on a placed block, use the `optimize_dft` command.

Specifying Parasitic Estimation Settings for the Preroute Optimization

The following topics describe how you can control parasitic estimation during the preroute stage of the design flow:

- Enabling Global-Route-Layer-Based (GRLB) Preroute Optimization
- Enabling Route-Driven Estimation (RDE) for Preroute Optimization

Enabling Global-Route-Layer-Based (GRLB) Preroute Optimization

To improve correlation with the postroute stage of the design flow, you can enable global-route-layer-based RC estimation during the `place_opt` and `clock_opt` commands. With this feature, the tool uses global routes for all nets and identifies the layers with the most

Feedback

appropriate per-unit resistance values. The nets are then constrained to minimum and maximum layers for preroute RC estimation.

To enable this feature, use the `opt.common.use_route_aware_estimation` application option. If you set it to

- `auto`, the feature is enabled only when there is a variation in the per-unit resistance value of the different routing layers
- `true`, the feature is enabled irrespective of the variation in the per-unit resistance value of the different routing layers

If you enable this feature, use the `remove_route_aware_estimation` command to remove all global route based estimation before you perform routing.

Enabling Route-Driven Estimation (RDE) for Preroute Optimization

To improve correlation with the postroute stage of the design flow, the tool can perform global routing, perform extraction based on the global routes, and use this parasitic information when performing optimization.

If the tool detects that the technology used by the design is an advanced technology node that is less than 16 nm, the tool enables this feature by default. To enable it for any technology, set the `opt.common.enable_rde` application option to `true`.

When enabled, the tool performs route-driven parasitic estimation during the `final_opt` stage of the `place_opt` and `clock_opt` commands. The parasitic information is stored in the design library and used during subsequent optimization steps. If you enable this feature, you should enable it for all subsequent preroute optimization steps in the design flow.

During route-driven estimation, the tool honors the capacitance and resistance scaling factors specified with the `-early_cap_scale`, `-late_cap_scale`, `-early_res_scale`, and `-late_res_scale` options of the `set_extraction_options` command.

When route-driven estimation is enabled, the tool ignores the setting of the `opt.common.use_route_aware_estimation` application option, which enables global-route-layer-based (GRLB) RC estimation.

Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

A via ladder is a stacked via that starts from the pin layer and extends into the upper routing layers. Using via ladders during optimization improves the performance and electromigration robustness of a design. The tool can automatically insert via ladders for cell pins on timing-critical paths during the `place_opt` and `clock_opt` commands.

Feedback

To perform via ladder insertion during preroute optimization,

1. Ensure that the via ladder rules are defined as described in Defining Via Ladder Rules.
2. Specify the via ladders that can be used for specific library pins by using the `set_via_ladder_candidate` command, as described in Specifying Via Ladder Candidates for Library Pins.
3. (Optional) Enable high-performance and electromigration via ladder insertion for critical paths by setting the `opt.common.enable_via_ladder_insertion` application option to `true`.
4. (Optional) Enable the insertion of global-route-based via ladders on pins with via ladder constraints by setting the `route.global.insert_gr_via_ladders` application option to `true`.
5. Perform optimization using the `place_opt` or `clock_opt` command.

Specifying Via Ladder Candidates for Library Pins

To specify the valid via ladders that can be used for a given library pin during preroute optimization, use the `set_via_ladder_candidate` command. You can specify only one via ladder each time you use the `set_via_ladder_candidate` command. To specify multiple via ladder candidates for a library pin, use the command multiple times, as shown in the following example:

```
icc2_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
   -ladder_name "VP4"
icc2_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
   -ladder_name "VP2"
```

The order in which you specify multiple via ladders for the same library pin indicates the priority to use when selecting a via ladder to insert for that pin. Specify the via ladder candidates starting with the highest priority. In the previous example, the tool gives the VP4 via ladder priority over the VP2 via ladder.

Note:

If both the `pattern_must_join` attribute and a via ladder candidate apply to a pin, the via ladder has a higher priority than the pattern-based must-join connection when both methods have the same estimated delay.

To specify that a library pin requires an electromigration via ladder, set the `is_em_via_ladder_required` pin attribute to `true`, as shown in the following example:

```
icc2_shell> set_attribute [get_lib_pins lib1/INV4/A] \
   is_em_via_ladder_required true
```

Feedback

If you set the `is_em_via_ladder_required` pin attribute to `true` for a specific library pin, you must specify an electromigration via ladder as a candidate with the `set_via_ladder_candidate` command.

A via ladder can be identified as an electromigration via ladder by using one of the following methods:

- By using the `forElectromigration=1` construct when defining the via rule in the technology file, as shown in the following example:

```
ViaRule "EMVP1" {
   ...
   ...
   ...
   forElectromigration=1
}
```

- By setting the `for_electro_migration` attribute to `true`, as shown in the following example:

```
icc2_shell> set_attribute [get_via_rules EMVP1] \
   for_electro_migration true
```

See Also

- Defining Via Ladder Rules

Enabling Area Recovery in Regions of High Utilization

For designs that cannot be legalized due to areas of high utilization, you can specify that the tool perform area recovery in regions where the utilization is high by setting the `opt.common.small_region_area_recovery` application option to `true`.

When you enable this feature, the tool performs the area recovery during the `place_opt` and `clock_opt` commands. However, doing so can slightly degrade the timing QoR.

Enabling Advanced Logic Restructuring

To improve the area, timing, and power QoR, you can enable advanced logic restructuring. To enable this feature for the `final_opto` stage of the `place_opt` and `clock_opt` commands, use the `opt.common.advanced_logic_restructuring_mode` application option, as shown in the following table.

Feedback

Table 12 Settings for the opt.common.advanced_logic_restructuring_mode Application Option

To do this	Use this setting
Perform no restructuring	`none` (default)
Perform restructuring to improve the area QoR	`area`
Perform restructuring to improve the timing QoR	`timing`
Perform restructuring to improve the power QoR	`power`
Perform restructuring to improve the area and timing QoR	`area_timing`
Perform restructuring to improve the timing and power QoR	`timing_power`
Perform restructuring to improve the area and power QoR	`area_power`
Perform restructuring to improve the area, timing, and power QoR	`area_timing_power`

When restructuring, the tool does not

- Restructure across logical hierarchy
- Consider cells with `dont_touch`, `size_only`, or `fixed` attributes
- Accept the results if it increases the wire length

  However, if your design does not have congestion or routing issues, you can specify that the tool accepts the restructuring results even if it increases the wire length by setting the `opt.common.advanced_logic_restructuring_wirelength_costing` application option to `medium` or `none`. The default is `high`.

Note:

This feature requires a Digital-AF license key

Setting Up for Power-Related Features

The following topics describe the tasks you need to perform for setting up a design for power-related features:

- Annotating the Switching Activity
- Enabling Power Optimization for the place_opt and clock_opt Commands
- Improving Yield By Limiting the Percentage of Low-Threshold-Voltage (LVT) Cells

Feedback

- Updating Activity for Improved Power Optimization
- Enabling the Power Integrity Features

Annotating the Switching Activity

When performing low-power placement and dynamic-power optimization, you obtain better power savings if you annotate switching activity on the design. You can annotate the switching activity in the following ways:

- Read a switching activity file (SAIF) by using the `read_saif` command.

  The switching activity is scenario specific. So, when you use this command, ensure that the current scenario is enabled for dynamic power optimization.

- Set the switching activity by using the `set_switching_activity` command.

  When you use this command, you can set the switching activity for a specific mode, corner, or scenario by using the `-mode`, `-corner`, or `-scenario` options. When doing so, ensure that the scenarios you specify or the scenarios corresponding to the modes and corners you specify are enabled for dynamic power optimization.

If you do not specify the switching activity, the tool applies the default toggle rate to the primary inputs and black box outputs and then propagates it throughout the design.

To report the switching activity of a block, use the `report_activity` command. To remove switching activity of specific nets, pins, ports, or the entire block, use the `reset_switching_activity` command.

Using RTL Switching Activity With a Name-Mapping File

To use an RTL SAIF file in the IC Compiler II tool, you must first perform the following steps in the Design Compiler tool:

1. Read in the RTL SAIF file by using the `read_saif` command.
2. Create a name-mapping database that tracks the changes to object names during optimization by using the `saif_map -start` command before optimization.
3. Perform optimization and other design changes.
4. Generate a name-mapping file that maps RTL object names to gate-level objects by using the `saif_map -write_map` command.

Feedback

Then, in the IC Compiler II tool, use the following steps:

1. Create a name-mapping database in the IC Compiler II tool and start tracking changes to object names by reading in the Design Compiler name-mapping file using the `saif_map -read_map` command.

   If you do not want to read in a name-mapping file, but you want to track the netlist changes that occur within the IC Compiler II tool, use the `saif_map -start` command to create a name-mapping database and begin tracking the name changes.

2. (Optional) Manually change the name mapping as described in Controlling the Name Mapping.

3. Read in the same RTL SAIF file by using the `read_saif` command. Reading an RTL SAIF file does not affect the name-mapping database.

4. Perform optimization and other design changes.

Controlling the Name Mapping

In the SAIF-map flow, the name-mapping database tracks the changes to pin, port, cell, and net names that occur during optimization. It contains the original name for each object, so that the original SAIF can be applied even after optimization.

You can manually control the name mapping in the database by using the `saif_map` command as shown in the following table.

Table 13 Options for Manually Controlling the Name Mapping

To do this	Use this option of the saif_map command
Specify a name mapping for an object and overwrite the name mapping that might exist in the database	`-set_name`
Add a name mapping to the existing mapping in the database	`-add_name`
Apply the name mapping specified with the `-set_name` or `-add_name` option to the logically inverted object	`-inverted`, with `-set_name` or `-add_name`
Change an object name using a name rule, when reading in the SAIF file	`-change_name`
Report all manual name-mapping settings specified	`-report`
Remove the name mapping settings for specific objects	`-remove_name`
Reset the name-mapping settings by clearing all manually created name mappings and name rules	`-reset`

Feedback

Table 13 Options for Manually Controlling the Name Mapping (Continued)

To do this	Use this option of the saif_map command
Retrieve name-mapping settings from the database	`-get_saif_names`, `-get_object_names`

Scaling the Switching Activity

If the clock frequency used in the SAIF file is different from the clock frequency used in the IC Compiler II tool, you can scale the switching activity by performing the following steps:

1. Read in the SAIF file by using the `read_saif` command.
2. Enable scaling by setting the `power.use_generated_clock_scaling_factor` application option to `on`.
3. Scale the switching activity by using the `set_power_clock_scaling` command.

   When you use this command, you must specify the following:

   - The clock objects associated with the switching activity you want to scale
   - The clock period used in the SAIF file by using the `-period` option or the ratio between the clock period used in the SAIF file and the clock period used in the IC Compiler II tool by using the `-ratio` option

   In addition, you can specify the scenario for which to apply the scaling by using the `-scenario` option.

The following example reads in a SAIF file, enables scaling, scales the switching activity associated with clocks named CLK1 and CLK2 by a ratio of five, and scales the switching activity associated with clock named CLK3 by a ratio of two:

```
icc2_shell> read_saif top.saif
icc2_shell> set_app_options -list \
  {power.use_generated_clock_scaling_factor true}
icc2_shell> set_power_clock_scaling -ratio 5 {CLK1 CLK2}
icc2_shell> set_power_clock_scaling -ratio 2 {CLK3}
```

If you run the `set_power_clock_scaling` command again for the same clock, the tool scales the already scaled switching activity.

Feedback

When you use the `set_power_clock_scaling` command, the tool scales only the switching activity applied with the `read_saif` command. The tool does not scale the following:

- Switching activity applied with the `set_switching_activity` command
- Switching activity within block abstracts

The scaled switching activity is persistent in the design. You can write it out by using the `write_saif` command and use it in the subsequent steps of the design flow.

Specifying Switching Probability for Supply Nets

The leakage power of a block is scaled based on the switching probability of its supply nets, which represents the fraction of time a supply net is in the *on* state. You can specify the switching probability for one or more supply nets by using the `set_supply_net_probability -static_probability` command. By default, the switching probability is applied to the current scenario and the corresponding mode and corner. To specify modes, corners, or scenarios in which to apply the setting, use the `-modes`, `-corners`, or `-scenarios` option. To get the switching probability of a supply net, use the `get_supply_net_probability` command, and to reset the value, use the `reset_supply_net_probability` command.

By default, the tool propagates supply net activity through power switches and determines the static probability of the switched supply net based on the UPF power switch constraint. For example, consider the following UPF power switch constraint:

```
create_power_switch my_switch \
   -output_supply_port {vout VDDS} \
   -input_supply_port {vin VDD} \
   -control_port {ms_sel ctrl1} \
   -control_port {ms_ctrl ctrl2} \
   -on_state {on vin {ms_ctrl && !ms_sel}}
```

The tool derives the static probability of the supply net named VDDS, which is connected to the output of the power switch, based on the probability of the power switch being *on*. This is derived based on the following:

- The Boolean function specified with the `-on_state` option, which is ms_ctrl && ! ms_sel, and the switching activity (static probability) of the nets connected to the corresponding control ports, which are nets named ctrl1 and ctrl2.
- The switching probability of the supply net connected to the input supply port specified with the `-on_state` option, which is the supply net named VDD.

Feedback

The following application options control whether dynamic and leakage power are scaled based on the supply switching activity:

- The `power.scale_dynamic_power_at_power_off` option controls whether dynamic power is scaled. The default is `false` (no scaling).
- The `power.scale_leakage_power_at_power_off` option controls whether leakage power is scaled. The default is `true` (scaling is performed).

Enabling Power Optimization for the place_opt and clock_opt Commands

The IC Compiler II tool can optimize both dynamic and static (leakage) power.

- Dynamic power

  This is the energy dissipated due to the voltage or logic transitions in the design objects, such as cells, pins, and nets. The dynamic power consumption is directly proportional to the number and frequency of transitions in the design.

- Static (leakage) power

  This is the energy dissipated even when there are no transitions in the circuit. This is also known as leakage power and depends on the device characteristics. The main contributor to leakage power is the sub-threshold-voltage leakage in the device. At lower technology nodes, leakage power consumption contributes significantly to the total power consumption of the circuit.

The following topics describe how to enable the different types of power optimization the tool performs:

- Performing Conventional Leakage-Power Optimization
- Performing Dynamic-Power Optimization
- Performing Total-Power Optimization

Feedback

Performing Conventional Leakage-Power Optimization

To set up for conventional leakage-power optimization during the preroute optimization stage, perform the following:

1. Ensure that at least one scenario is enabled for leakage-power optimization by using the `set_scenario_status -leakage_power true` command and the `set_qor_strategy -metric leakage_power | total_power` command for deciding the target of power flow.
2. (Optional) Change the effort level for power optimization by setting the `set_qor_strategy -mode` command to `balanced` or `extreme_power`. The default is `balanced`. This effort control for the power flow.

When you use these settings, the tool performs conventional leakage-power optimization during the `place_opt` and `clock_opt` commands. However, it does not perform any dynamic-power optimization.

Performing Dynamic-Power Optimization

To set up for dynamic-power optimization during the preroute optimization stage, perform the following:

1. Annotate switching activity on the design, as described in Annotating the Switching Activity.
2. Ensure that at least one scenario is enabled for dynamic-power optimization by using the `set_scenario_status -dynamic_power true` command.

When you use these settings, the tool performs dynamic-power optimization during the `place_opt` and `clock_opt` commands. However, it does not perform any leakage-power optimization.

Performing Total-Power Optimization

Total power optimization considers the combined leakage- and dynamic-power cost during optimization. To setup for total-power optimization,

1. Setup for leakage-power optimization by performing the following steps:
   a. Ensure that at least one scenario is enabled for leakage-power optimization by using the `set_scenario_status -leakage_power true` command.
2. Setup for dynamic-power optimization by performing the following steps:
   a. Annotate switching activity on the design, as described in Annotating the Switching Activity.
   b. Ensure that at least one scenario is enabled for dynamic-power optimization by using the `set_scenario_status -dynamic_power true` command.

Feedback

When you use this setting, the tool performs total-power optimization during the `place_opt` and `clock_opt` commands.

Improving Yield By Limiting the Percentage of Low-Threshold-Voltage (LVT) Cells

The IC Compiler II tool can perform percentage-LVT-based optimization. By limiting the percentage of LVT cells in a block, you can improve yield of your design. You can also use this feature during the early design phases to get an idea of the quality of the RTL.

To set up for percentage-LVT-based optimization,

1. Specify the LVT library cell group as shown in the following example:

```
icc2_shell> remove_attributes -quiet  \
 [get_lib_cells -quiet */*] threshold_voltage_group
icc2_shell> set_attribute [get_lib_cells -quiet *lvt*/*] \
   threshold_voltage_group LVT
icc2_shell> set_threshold_voltage_group_type \
   -type low_vt LVT
```

   Ensure that the LVT library cells you specify

   - Have a valid purpose defined by using the `set_lib_cell_purpose` command and are included in the appropriate target-library subsets defined by using the `set_target_library_subset` command
   - Do not have a `dont_touch` or `dont_use` attribute setting

2. Specify the percentage limit for LVT cells by using the `set_multi_vth_constraint -low_vt_percentage` command.

   To specify the limit as a percentage of the

   - Cell count, use the `-cost cell_count` option
   - Cell area, use the `-cost area` option

   To reset this constraint, use the `reset_multi_vth_constraint` command, and to report it, use the `report_multi_vth_constraint` command.

After you set up percentage-LVT-based optimization, when you run the `place_opt`, `clock_opt`, or `route_opt` command, the tool uses the specified LVT cell percentage constraint for the data path cells in the block. However, to minimize QoR disturbance, the tool limits the percentage-LVT-based optimization performed during the `route_opt` command. Therefore, it is important to enable this feature earlier in the design flow.

Feedback

Note:

LVT cells have a smaller cell delay, but higher leakage power dissipation. Therefore, limiting the number of LVT cells reduces leakage-power dissipation. However, to further reduce leakage power, enable leakage power optimization for the different stages of the design flow.

Updating Activity for Improved Power Optimization

The PrimePower In-Design flow in the IC Compiler II tool uses input waveform information from the RTL FSDB file and performs time-based analysis to update the activity for enhanced power estimation accuracy.

Use the `set_indesign_primepower_options` command to set PrimePower analysis options for the In-Design flow. You must specify the RTL activity file with the `-fsdb` option and the path to the PrimePower executable with the `-pwr_shell` option. Other options allow you to specify details about the FSDB file, settings for distributed or concurrent processing, settings for scenarios and libraries, and output files.

Use the `update_indesign_activity` command to perform the analysis, which includes the following tasks:

- Invokes the PrimePower tool and reads the design data which includes the netlist, libraries, PVT settings, constraints, parasitics, essential name mapping data, UPF, and so on. Library details come from the design database stored in the IC Compiler II tool memory.
- Reads the RTL FSDB file into the PrimePower tool and performs time-based analysis to generate a gate-level SAIF.
- Reads the gate-level SAIF back into the IC Compiler II tool.

Set the power scenario as the current scenario before starting analysis with the `update_indesign_activity` command. Activity is refreshed for the current scenario.

If you specify multiple scenarios using the `-scenarios` option of the `set_indesign_primepower_options` command, the refreshed gate-level SAIF from the PrimePower tool is read back into the IC Compiler II tool for all of the specified scenarios.

The PrimePower In-Design UI allows you to pass input waveforms using the native ZeBu database (ZTDB). The `-ztdb` option of the `set_indesign_primepower_options` command sets up PrimePower In-Design to create a script using the `update_indesign_activity` command. This script uses the PrimePower `read_ztdb` command to perform time-based analysis and issues a gate-level SAIF file that is automatically back-annotated into the IC Compiler II tool.

Feedback

Specifying Distributed Analysis

For faster runtime, use distributed analysis, which is disabled by default in the PrimePower tool.

To enable distributed analysis in the PrimePower tool, specify the following options with the `set_indesign_primepower_options` command:

- `-num_processes`: Use this option to specify the number of hosts to launch. The value must be greater than 1.
- `-submit_command` or `-host_names`: Use either of these options to specify the configuration of host machines.
- `-max_cores`: Use this option to specify the maximum number of cores. The default is 4.

The tool prints the following message if valid options are specified:

```
Information: PrimePower is being called in distributed mode
```

Enabling the Power Integrity Features

The IC Compiler II tool provides the following techniques for ensuring power integrity of the design:

- Dynamic power shaping (DPS), which reduces transient power by utilizing concurrent clock and data optimization (useful skew) techniques.
- Voltage-drop-aware placement, which use the voltage (IR) drop values of cells to identify areas of high power density and spread the cells with high voltage drop values to reduce the power density of such areas.

To set up for the recommended power integrity flow, perform the following steps:

1. Enable the power integrity flow by setting the `opt.common.power_integrity` application option to `true`.

   When you do so, the IC Compiler II uses the following techniques during the different stages of the design flow:

   a. Dynamic power shaping (DPS) during the `final_opto` stage of the `place_opt` command

   b. Voltage-drop-aware placement during the `clock_opt` command

2. (Optional) Specify that the tool should not reduce the voltage drop at the expense of the timing QoR by reducing the power integrity effort level by setting the `opt.common.power_integrity_effort` application option to `low`.

Feedback

The default is `high`, and by default the tool tries to reduce the voltage drop to less than eight percent of the supply voltage at the expense of the timing QoR.

3. (Optional) Specify a maximum voltage drop threshold by using the `opt.common.ir_drop_threshold` application option.

   The default is eight percent of the supply voltage. If you change the default effort level by setting the `opt.common.power.integrity_effort` application option to `low`, the tool ignores the threshold specified with the `opt.common.ir_drop_threshold` application option.

4. Specify settings for dynamic power shaping as described in Setting Up for Dynamic Power Shaping.

5. Specify settings for voltage-drop-aware placement as described in Setting Up for Voltage-Drop-Aware Placement.

6. (Optional) Enable IR-driven sizing, which uses the RedHawk dynamic voltage drop analysis results to identify cells involved in voltage drop violations, and then tries to replace those cells with cells having smaller leakage current. To enable this feature, set the `clock_opt.flow.enable_irdrivenopt` application option to `true`, in addition to setting the `opt.common.power_integrity` application option to `true`.

Instead of using the recommended power integrity flow setting, you can manually enable dynamic power shaping or voltage-drop-aware placement for the `place_opt` or `clock_opt` commands as described in Manually Enabling Dynamic Power Shaping and Voltage-Drop-Aware Placement

Setting Up for Dynamic Power Shaping

To set up for dynamic power shaping, perform the following steps:

1. Ensure UPF settings are specified for multiple-power nets.

2. Ensure both setup and hold scenarios are created and are active.

3. (Optional) Control the power analysis and optimization performed during dynamic power shaping as follows:

   - Specify the scenarios to focus on by setting the `ccd.dps.focus_power_scenario` application option.

     By default, the tool focuses on all active scenarios. When the design has many active scenarios, specifying one or two scenarios, as shown in this example, can reduce the runtime.

     ```
     icc2_shell> set_app_options -name \
      -name ccd.dps.focus_power_scenario -value {{S1 S2}}
     ```

Feedback

- Control the type of internal power analysis performed by setting the `ccd.dps.use_case_type` application option.

  By default, this application option is set to `autovector`. With this setting, the tool runs internal power analysis with a 20 percent probability of toggling a flip-flop and all flip-flops are effectively clocked every cycle. However, the clock gating is not modeled.

  If you set this application option to `activity`, the tool chooses probability of toggling a flip-flop and the clock-gating model based on the activity.

- Define the overall dynamic power optimization goal by setting the `ccd.dps.optimize_power_target_modes` application option.

  - To reduce the global peak current, where the optimization goal is to reduce the total peak current, set this application option to `global_peak`.
  - To reduce the local peak current in windows stepped across the design, where the optimization goal is to reduce voltage drop violations, set this application option to `stepped_psgs`, which is the default.

- Specify the tradeoff between power and setup timing during dynamic power shaping by setting the `ccd.dps.optimize_setup_tradeoff_level` application option to `low`, `medium` (default), or `high`.

- Specify the tradeoff between power and hold timing during dynamic power shaping by setting the `ccd.dps.optimize_hold_tradeoff_level` application option to `low`, `medium` (default), or `high`.

You can further control the power analysis and optimization performed during dynamic power shaping by using application options such as `ccd.dps.use_cases`, `ccd.dps.auto_targets`, and `ccd.dps.auto_target_constraint_parameters`, which are more complex to use. For more information on these application options, see the corresponding man pages.

Setting Up for Voltage-Drop-Aware Placement

To set up for voltage-drop-aware placement, perform the following steps:

1. Specify the voltage-drop-analysis type by using the `rail.analysis_type` application option.

   The valid values are `static`, `dynamic` (default), `dynamic_vcd`, and `dynamic_vectorless`.

2. Specify settings required to perform RedHawk voltage drop analysis, as described in Performing Voltage Drop Analysis.

Feedback

Manually Enabling Dynamic Power Shaping and Voltage-Drop-Aware Placement

To manually enable

- Dynamic power shaping for the `place_opt` command, use the following settings:

```
icc2_shell> set_app_option  -name opt.common.power_integrity \
  -value false
icc2_shell> set_app_option  -name place_opt.flow.enable_dps \
  -value true
```

- Voltage-drop-aware placement for the `place_opt` command, use the following settings:

```
icc2_shell> set_app_option  -name opt.common.power_integrity \
  -value false
icc2_shell> set_app_option  -name place_opt.flow.enable_irap \
  -value true
```

- Voltage-drop-aware placement for the `clock_opt` command, use the following settings:

```
icc2_shell> set_app_option  -name opt.common.power_integrity \
  -value false
icc2_shell> set_app_option  -name clock_opt.flow.enable_irap \
  -value true
```

Specifying the Routing Resources

You can specify the minimum and maximum routing layers both for the block (global layer constraints), for specific nets (net-specific layer constraints), and for unsynthesized clock nets (clock-tree layer constraints). If you specify both global layer constraints and net-specific layer constraints, the net-specific constraints override the global constraints.
In addition to constraining the routing layers, you can also specify a preferred routing direction for each layer. The routing layer constraints are used by RC estimation, congestion analysis, and routing. Because these constraints affect RC estimation and congestion analysis as well as routing, you should set these constraints before performing placement on the block.

The following topics describe how to specify routing layer constraints, preferred routing direction, and via ladders :

- Specifying the Global Layer Constraints
- Specifying Net-Specific Layer Constraints
- Specifying Clock-Tree Layer Constraints

Feedback

- Enabling Level-Based Clock Nondefault Routing Rule
- Setting the Preferred Routing Direction for Layers

Specifying the Global Layer Constraints

To specify the global layer constraints, use the `set_ignored_layers` command. By default, the global layer constraints are used for RC estimation, congestion analysis, and as soft constraints for routing. Use the following options to set the constraints and change the default behavior.

- To specify the minimum and maximum routing layers, use the `-min_routing_layer` and `-max_routing_layer` options. Specify the routing layers by using the layer names from the technology file.

  For example, to use layers M2 through M7 for routing, RC estimation, and congestion analysis, use the following command:

  ```
  icc2_shell> set_ignored_layers \
     -min_routing_layer M2 -max_routing_layer M7
  ```

- To allow the use of layers beyond the minimum or maximum routing layer only for pin connections, set the `route.common.global_min_layer_mode` and `route.common.global_max_layer_mode` application options to `allow_pin_connection`.
- To change the constraints to hard constraints, set the `route.common.global_min_layer_mode` and `route.common.global_max_layer_mode` application options to `hard`.
- To specify additional layers to be ignored for RC estimation and congestion analysis, use the `-rc_congestion_ignored_layers` option.

  The specified layers must be between the minimum and maximum routing layers. For example, to use layers M2 through M7 for routing and layers M3 through M7 for RC estimation and congestion analysis, use the following command:

  ```
  icc2_shell> set_ignored_layers \
     -min_routing_layer M2 -max_routing_layer M7 \
     -rc_congestion_ignored_layers {M2}
  ```

Feedback

- To change an existing layer constraint setting, simply reset that option.

  When you reset an option, it overrides the existing value of only that option; the other option settings remain unchanged.

  For example, assume that you used the previous command to set the minimum routing layer to M2 and the maximum routing layer to M7. To change the maximum routing layer from M7 to M8, but keep the other settings, use the following command:

  ```
  icc2_shell> set_ignored_layers -max_routing_layer M8
  ```

Reporting Global Layer Constraints

To report the ignored layers, use the `report_ignored_layers` command. For example,

```
icc2_shell> report_ignored_layers
****************************************
Report : Ignored Layers
Design : my_design
Version: J-2014.12
Date   : Wed Oct 22 15:58:23 2014
****************************************
Layer Attribute                 Value
--------------------------------------------------------------------------------
Min Routing Layer               M2
Max Routing Layer               M7
RC Estimation Ignored Layers    PO M1 M2 M8 M9 MRDL
1
```

Removing Global Layer Constraints

To remove the global layer constraints, use the `remove_ignored_layers` command. You must specify one or more of the following options:

- `-min_routing_layer`

  This option removes the minimum routing layer setting. When you remove the minimum routing layer setting, it also removes the ignored layers for RC estimation and congestion analysis that are below the minimum routing layer.

- `-max_routing_layer`

  This option removes the maximum routing layer setting. When you remove the maximum routing layer setting, it also removes the ignored layers for RC estimation and congestion analysis that are above the maximum routing layer.

- `-all`

  This option removes the ignored layers for RC estimation and congestion analysis that are between the minimum and maximum routing layers.

Feedback

- `-rc_congestion_ignored_layers` *`layer_list`*

  This option removes the specified ignored layers for RC estimation and congestion analysis. The specified layers must be between the minimum and maximum routing layers.

Specifying Net-Specific Layer Constraints

To specify net-specific layer constraints, use the `set_routing_rule` command. To specify the minimum and maximum routing layers, use the `-min_routing_layer` and `-max_routing_layer` options. Specify the routing layers by using the layer names from the technology file.

For example, to use layers M2 through M7 when routing the n1 net, use the following command:

```
icc2_shell> set_routing_rule [get_nets n1] \
   -min_routing_layer M2 -max_routing_layer M7
```

By default, net-specific minimum layer constraints are soft constraints, while net-specific maximum layer constraints are hard constraints. You can change the default behavior as follows:

- To change the default constraint strength for all net-specific layer constraints, set the `route.common.net_min_layer_mode` and `route.common.net_max_layer_mode` application options.

  Set these application options to `soft` to specify a soft constraint, `allow_pin_connection` to allow the use of lower layers only for pin connections, or `hard` to specify a hard constraint.

- To change the constraint strength for a single net-specific layer constraint, use the `-min_layer_mode` and `-max_layer_mode` options when you define the constraint with the `set_routing_rule` command.

  Set these options to `soft` to specify a soft constraint, `allow_pin_connection` to allow the use of lower layers only for pin connections, or `hard` to specify a hard constraint.

- To set the cost of violating soft constraints for all net-specific layer constraints, set the `route.common.net_min_layer_mode_soft_cost` and `route.common.net_max_layer_mode_soft_cost` application options.

  Set these application options to `low`, `medium` (the default), or `high`. The cost setting controls the effort expended by the router to avoid violations. The `high` setting can reduce violations at a cost of increased runtime. The `low` setting can reduce runtime at a cost of increased violations.

Feedback

- To set the cost of violating soft constraints for a single net-specific layer constraint, use the `-min_layer_mode_soft_cost` and `-max_layer_mode_soft_cost` options when you define the constraint with the `set_routing_rule` command.

  Set these options to `low`, `medium` (the default), or `high`. The cost setting controls the effort expended by the router to avoid violations. The `high` setting can reduce violations at a cost of increased runtime. The `low` setting can reduce runtime at a cost of increased violations.

Removing Net-Specific Routing Layer Constraints

To remove net-specific routing layer constraints, use the `set_routing_rule -clear` command. Note that when you use this command, it also removes any nondefault routing rules assigned to the specified nets.

Specifying Clock-Tree Layer Constraints

To specify clock-tree layer constraints, use the `set_clock_routing_rules` command. To specify the minimum and maximum routing layers, use the `-min_routing_layer` and `-max_routing_layer` options. Specify the routing layers by using the layer names from the technology file.

By default, the `set_clock_routing_rules` command assigns the specified layer constraints to all clock trees. Table 14 shows the options used to restrict the layer constraints.

Table 14 Restricting Clock-Tree Layer Constraints

To assign a layer constraint to	Use this option
Specific clock trees	`-clocks` *clocks*
Nets connected to the clock root[1]	`-net_type root`
Nets connected to one or more clock sinks[1]	`-net_type sink`
Internal nets in the clock tree (all nets except the root and sink nets)[1]	`-net_type internal`
Specific clock nets	`-nets` *nets*

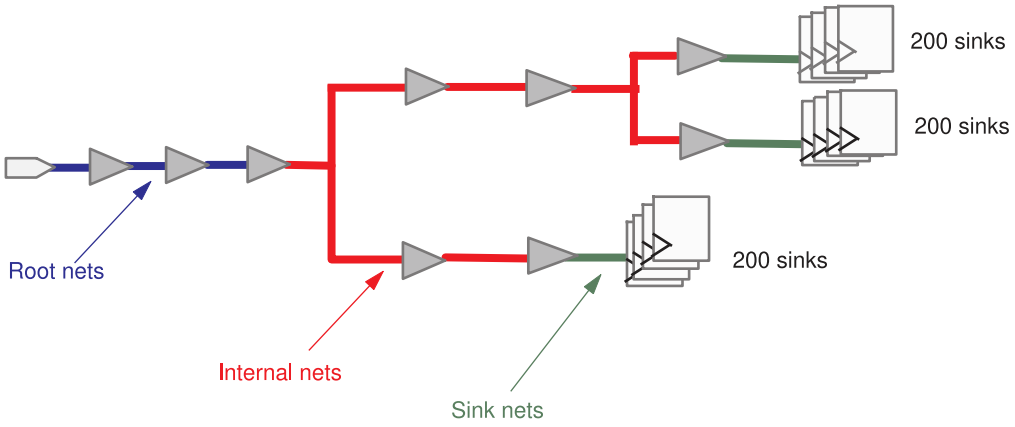
Figure 22 shows the root, internal, and sink nets of a clock tree after clock tree synthesis. By default, the root-net layer constraints are applied to all the single-fanout clock nets starting from the clock root up to the point where the clock tree branches out to a fanout

1. *You can use this option with the `-clocks` option to further restrict the assignment. This option is not valid with the `-nets` option.*

Feedback

of more than one. Internal-net layer constraints are applied to the nets from this point until the sink nets.

Figure 22 Root, Internal, and Sink Clock Net Types



For example, to use layers M4 through M7 when routing the root nets of the CLK1 clock, use the following command:

```
icc2_shell> set_clock_routing_rules -clocks CLK1 -net_type root \
    -min_routing_layer M4 -max_routing_layer M7
```

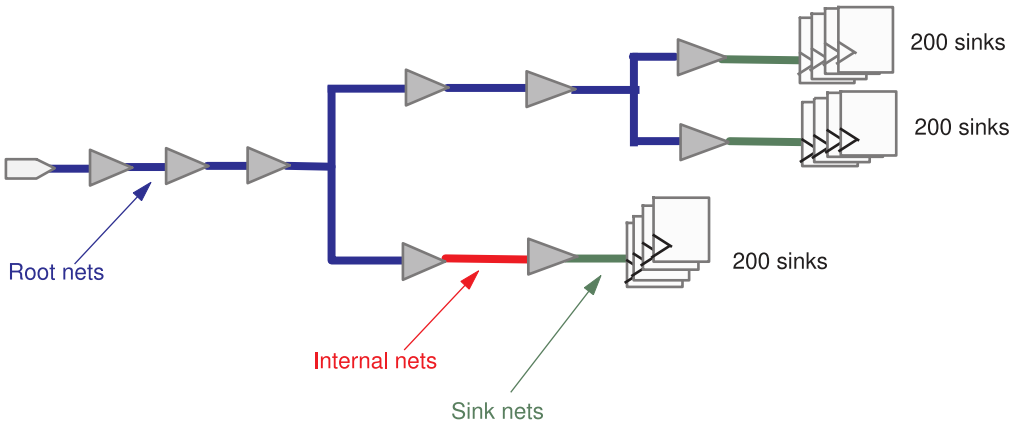
To specify a transitive fanout limit to use when identifying root nets, use the `set_clock_tree_options -root_ndr_fanout_limit` command. For example, to specify that any clock net with a transitive fanout of more than 300 sinks be considered as a root net, use the following command:

```
icc2_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

Figure 23 shows the root, internal, and sink nets of the same clock tree when a transitive fanout limit of 300 is specified for identifying the clock root nets.

Feedback

Figure 23 Using a Fanout Limit for Selecting Root Nets



When calculating the transitive fanout of clock nets for the purpose of identifying root nets, the tool includes only the valid clock sinks; It does not include the ignore pins. If a net identified as a root net is less than 10 microns, the tool uses internal-net layer constraints for that net.

Note:

Specifying a smaller value with the `set_clock_tree_options -root_ndr_fanout_limit` command increases the number of clock nets that are assigned the root-net layer constraints, which can increase routing congestion.

To remove the transitive fanout limit specified with the `set_clock_tree_options -root_ndr_fanout_limit` command, use the `remove_clock_tree_options -root_ndr_fanout_limit` command.

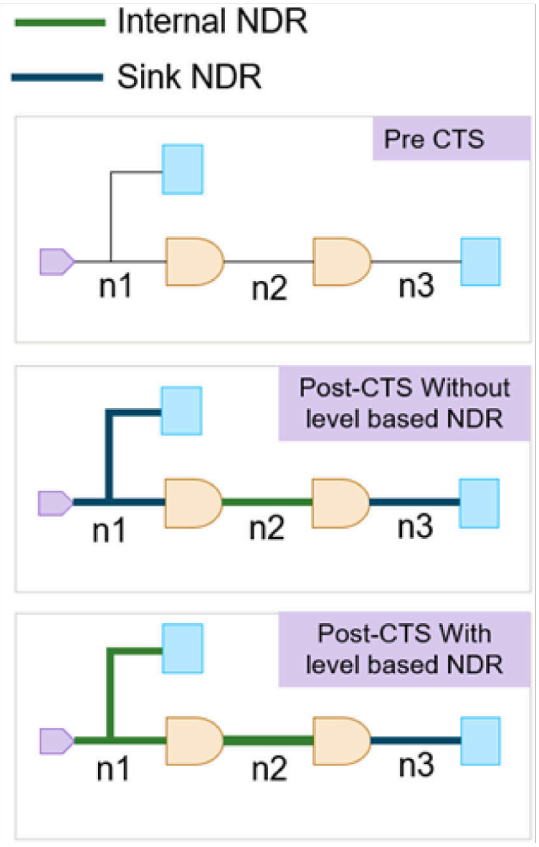
Enabling Level-Based Clock Nondefault Routing Rule

During clock tree synthesis, you can assign continuous nondefault routing rule to nets based on different levels of the clock tree. The tool implements the changes only during the initial clock tree synthesis and assigns nondefault routing rule in the following ways:

- Nets connected to pure sink carry sink nondefault routing rule
- Nets connected to both sink and intermediate cells (repeaters or clock gates) must receive internal nondefault routing rule
- Nets at root level until the branching point downstream must get root nondefault routing rule

Feedback

Figure 24 Level-Based Clock Nondefault Routing Rule



As shown in the figure, after clock tree synthesis:

- Without the level-based clock nondefault routing rule: The n1 and n3 nets must get sink nondefault routing rule after compiling the clock tree because they are connected to at least one sink. These nets leave the discontinuous nondefault routing rules applied at different levels.
- With the level-based clock nondefault routing rule: All the nondefault routing rules in a clock tree path must be in a topologically sorted order from clock root to sinks, that is, Root nondefault routing rule > Internal nondefault routing rule > Sink nondefault routing rule. Therefore, n1 and n2 nets are assigned with internal nondefault routing rule while net n3 is assigned to a sink nondefault routing rule.

To enable the level-based clock nondefault routing rule application, use the following application option:

```
icc2_shell> set_app_options -list {cts.compile.topological_ndr true}
```

Feedback

Setting the Preferred Routing Direction for Layers

The IC Compiler II tool requires that the preferred routing direction is specified for the routing layers defined in the technology file. Typically this information is defined in the cell library. To set or change the preferred routing direction for a layer, use the following syntax to set its `routing_direction` attribute:

```
set_attribute -objects layers
   -name routing_direction
   -value vertical | horizontal
```

Specify the routing layers by using the layer names from the technology file.

The layer direction set with this attribute applies only to the current block.

For example, to set the preferred routing direction to vertical for the M5 and M7 layers, use the following command:

```
icc2_shell> set_attribute -objects [get_layers {M5 M7}] \
   -name routing_direction -value vertical
```

Note:

Settings made with the `create_routing_guide -switch_preferred_direction` command, which changes the preferred direction within the area that is covered by the routing guide, override the `routing_direction` attribute settings.

To report the user-defined preferred routing direction for one or more routing layers, use the `get_attribute` command. To remove the user-defined preferred routing direction for one or more routing layers, use the `remove_attributes` command.

See Also

- Preparing Routing Layers

Handling Design Data Using the Early Data Check Manager

During the early iterations of a design cycles, the design data might be incomplete or incorrect. This can prevent you from implementing the design for exploration purposes and so on. The IC Compiler II Early Data Check Manager allows you to specify how the tool should handle the data checks it performs during the following implementation tasks:

- Design planning
- Multivoltage design implementation
- Optimization

Feedback

- Placement
- Legalization
- Hierarchical implementation of designs with physical hierarchy

The following table shows the Early Data Check Manager commands available for controlling how the tool handles the data checks and for generating related information.

Table 15 Early Data Check Manager Commands

To do this	Use this command
Specify the policy for handling one or more data checks	`set_early_data_check_policy`
Obtain a report about data check violations	`report_early_data_checks`
Get a Tcl collection of violations that can be used in other commands	`get_early_data_check_records`
Save the settings in a file for future use	`write_early_data_check_config`
Clear the data check settings in preparation for another iteration	`remove_early_data_check_records`

You should specify the policies for handling the data checks before you begin implementation.

The following example specifies that the tool should be strict when checking for the SCANDEF information. Therefore, the tool does not proceed with the `create_placement` command because the SCANDEF information is missing.

```
icc2_shell> set_early_data_check_policy -policy strict \
   -check place.coarse.missing_scan_def
icc2_shell> create_placement
Information: Policy for early data check 'place.coarse.missing_scan_def'
 is 'error'. (EDC-001)
Error: No valid scan def found. (PLACE-042)
Information: Ending 'create_placement' (FLW-8001)
icc2_shell> report_early_data_checks -check place.coarse.missing_scan_def
Check                          Policy  Strategy  Fail Count
-------------------------------------------------------------
place.coarse.missing_scan_def   error             1
-------------------------------------------------------------
```

The following example specifies that the tool should be lenient with regards to all data checks related to coarse placement. Therefore, the tool proceeds with the `create_placement` command without the SCANDEF information.

Feedback

```
icc2_shell> set_early_data_check_policy -policy lenient \
   -check place.coarse.*
icc2_shell> create_placement
Information: Policy for early data check 'place.coarse.missing_scan_def'
 is 'tolerate'. (EDC-001)
Continuing without valid scan def.
Start transferring placement data.
Creating placement from scratch.
...
...
```

For more information about the Early Data Check Manager, see the *IC Compiler II Data Model User Guide*.

Applying Mega-Switch Command Settings

You can use mega-switch commands to apply setting for different stages of the tool flow that are applicable for specific situations with the use of a single command.

- Applying Required Settings for Advanced Technology Nodes
- Applying Required Settings for High Performance Cores
- Applying Required Settings for Improving Specific QoR Metrics

Applying Required Settings for Advanced Technology Nodes

To apply placement, legalization, routing, and extraction setting specific to 12 or 7 nanometer technology nodes, use the `set_technology` command with the `-node 12` or `-node 7` option.

When using this command, perform the following steps before placement, optimization, and routing:

1. Load and link the design.
2. Apply the settings required for the technology nodes by using the `set_technology -node` command, as shown in the following example:

   ```
   icc2_shell> set_technology -node 7
   ```

3. (Optional) Report the technology-node settings by using the `set_technology -report_only` command.
4. Apply your design-specific application option or command settings to override the generic setting of the `set_technology` command.

When you save the design library with the `save_lib` command, the technology-node setting is saved.

Feedback

Note:

After you specify a technology node with the `set_technology` command, you cannot change it.

For more information about which ASIC technologies are supported by this command, contact Synopsys Support.

Applying Required Settings for High Performance Cores

You can apply tool settings required for achieving the best QoR for high performance cores by using the `set_hpc_options` command. The applied settings affect placement, legalization, optimization, clock tree synthesis, routing, extraction, and timing analysis.

To

- List the supported high performance core types and implementation stages, use the `-list` option
- Specify the type of high performance core, use the `-core` option
- Specify the stage of the implementation flow, use the `-stage` option

If your design uses a 12 or 7 nanometer technology node, apply the node-specific tool settings by using the `set_technology` command before you run the `set_hpc_options` command, as shown in the following example script:

```
set_technology -node 7
set_hpc_options -core A72 -stage clock_opt_cts
...
clock_opt -from build_clock -to route_clock
...
set_hpc_options -core A72 -stage clock_opt_opto
...
clock_opt -from final_opto
...
set_hpc_options -core A72 -stage route_auto
...
route_auto
...
set_hpc_options -core A72 -stage route_opt
...
route_opt
...
```

Applying Required Settings for Improving Specific QoR Metrics

To apply tool settings required for improving specific QoR metrics, use the `set_qor_strategy -stage pnr` command.

Feedback

To specify the QoR metrics to improve, use the `-metric` option. To optimize the block for the best

- Timing, use the `timing` setting
- Leakage power and timing, use the `leakage_power` setting
- Total power and timing, use the `total_power` setting.

  To further improve total power, at the expense of runtime, you can use the `-mode extreme_power` option, with the `-metric total_power` option.

To specify a target mode for the design flow, use the `-mode` option. The supported values for this option are:

- `balanced`, which is the default mode that optimizes the design for the target metrics specified with the `-metric` option.
- `early_design`, which is suitable for the early stages of the design flow, when the emphasis is on short turnaround time for prototyping purposes.

  With this mode, you can reduce runtime at the expense of QoR.
- `extreme_power`, which is recommended for further improvement of total power at the expense of runtime.

  This mode should only be specified along with the `-metric leakage_power` or `-metric total_power` option. If you specify this mode with the `-metric timing` option, the tool ignores the specified mode setting and uses the `balanced` mode.

When you use this command, the tool applies application option settings that affect optimization, placement, legalization, clock tree synthesis, routing, extraction, and timing analysis.

To review the required setting, use one of the following methods:

- Generate a Tcl script that contains the settings by using the `-output` option
- Report the settings by using the `-report_only` option
- Report only the settings that are not set to their required values by using the `-diff_only` option

3

Placement and Optimization

To learn how to perform placement and optimization in the IC Compiler II tool, see

- Performing Placement and Optimization
- Identifying Issues That Cannot Be Fixed During Optimization
- Analyzing the Placement
- Analyzing Timing
- Analyzing Power
- Comparing QoR Data

Performing Placement and Optimization

After you have finished design planning and power planning, you can perform placement, optimization, and legalization on your design. The following topics describe how to perform placement, optimization, and legalization:

- Performing Standalone Placement and Legalization
- Performing Placement and Optimization With the place_opt Command
- Using Physical Guidance From the Design Compiler Tool
- Performing Multibit Register Optimization
- Performing Magnet Placement
- Refining Placement
- Performing Placement and Optimization on Multivoltage Blocks
- Rebuilding Buffer Trees
- Adding and Removing Tie Cells

Feedback

Performing Standalone Placement and Legalization

To perform coarse placement, use the `create_placement` command. If a block contains scan chains that were annotated on the block by reading a SCANDEF file, the coarse placer also performs scan chain optimization.

You can control the tradeoff between runtime and QoR by setting the `-effort` option.

To enable

- Timing-driven placement, use the `-timing_driven` option
- Timing-driven placement that considers high-fanout nets and long nets and accounts for the buffers that are subsequently added on these nets during optimization, use the `-buffering_aware_timing_driven` option
- Congestion-driven placement, use the `-congestion` option

  To control the CPU effort used for congestion removal, use the `-congestion_effort` option
- Congestion driven restructuring, use the `-congestion_driven_restructuring` option

  When you use this option, the tool performs several iterations of placement and restructuring of the logic to help reduce congestion.

Note:

Before you run placement using the `create_placement` command, you should merge the clock-gating logic by running the `merge_clock_gates` command.

To perform legalization, use the `legalize_placement` command. For more information about legalization constraints and settings that you can specify, see Specifying Legalization Settings.

Note:

The `-cells` option of the `legalize_placement` command is not supported when you enable the advanced legalizer as described in Enabling Advanced Legalization Algorithms.

Feedback

Performing Placement and Optimization With the place_opt Command

To perform coarse placement, physical optimization, and legalization with a single command, use the `place_opt` command.

This command supports multithreading and uses the number of threads specified by the `set_host_options -max_cores` command.

The `place_opt` command consists of the following stages:

1. Initial placement (`initial_place`)

   During this stage, the tool merges the clock-gating logic and performs coarse placement. If a block contains scan chains that are annotated by reading a SCANDEF file, the tool also performs scan chain optimization.

2. Initial DRC violation fixing (`initial_drc`)

   During this stage, the tool removes existing buffer trees and performs high-fanout-net synthesis and electrical DRC violation fixing.

3. Initial optimization (`initial_opto`)

   During this stage, the tool performs timing, area, congestion, and leakage-power optimization.

4. Final placement (`final_place`)

   During this stage, the tool performs incremental placement to improve timing and congestion, and legalizes the design.

5. Final optimization (`final_opto`)

   During this stage, the tool performs further optimization and legalization to improve timing and congestion.

When you run the `place_opt` command, by default, the tool runs all stages of placement and optimization.

To limit the `place_opt` command to run only one or more of these stages, use the `-from` option to specify the stage from which you want to begin and the `-to` option to specify the stage after which you want to end.

If you do not specify the `-from` option, the tool begins from the `initial_place` stage. Similarly, if you do not specify the `-to` option, the tool continues until the `final_opto` stage is completed.

Feedback

For example, to skip initial placement, but run all the other stages of optimization and placement on a design that has already completed initial placement, use the following command:

```
icc2_shell> place_opt -from initial_drc
```

Creating a Temporary Clock Tree for Placement and Optimization

By default, the `place_opt` command uses ideal clock information during placement and optimization. However, you can specify that the tool builds a temporary clock tree and use propagated clocks during the `place_opt` command by setting the `place_opt.flow.trial_clock_tree` application option to `true`.

After placement and optimization, when you perform clock tree synthesis by using the `clock_opt` or `synthesize_clock_trees` command, the tool removes the temporary clock tree and builds the actual clock tree. To ensure that the temporary clock tree correlates to the actual clock tree, specify your clock tree synthesis settings before you run the `place_opt` command.

Optimizing Clock-Gating Cells

By default, the tool merges clock-gating cells at the beginning of the

- `initial_place` stage of the `place_opt` command when the Synopsys physical guidance flow is not enabled.
- `initial_opt` stage of the `place_opt` command when the Synopsys physical guidance (SPG) flow is enabled by setting the `place_opt.flow.do_spg` application option to `true`.

However, if you run the `merge_clock_gates` command before you run the `place_opt` command, the tool does not merge clock gates during the `place_opt` command.

You can control the merging of clock-gating cells as follows:

- To merge only the clock-gating cells that are at the same level of the clock tree, set the `cts.icg.merge_cross_level` application option to `false`.

  By default, the tool can merge clock-gating cells that are at different levels of the clock tree.
- To disable the merging of clock-gating cells, set the `place_opt.flow.merge_clock_gates` application option to `false`.

The `place_opt` command can further optimize clock-gating cells if the enable pin of the clock gate is in a critical timing path. To enable this feature, set the `place_opt.flow.optimize_icgs` application option to `true`.

Feedback

During the clock-gating cell optimization, the tool

- Performs clock-aware placement, irrespective of the setting of the `place_opt.flow.clock_aware_placement` application option, and places critical clock-gating cells and their fanouts at more optimal locations
- Builds a temporary clock tree, irrespective of the setting of the `place_opt.flow.trial_clock_tree` application option, and uses the clock tree to identify timing-critical clock-gating cells
- Splits clock-gating cells if the enable pin of the clock gate is in a critical timing path

  You can control the extent of splitting by specifying a critical range using the `place_opt.flow.optimize_icgs_critical_range` application option.

Using Accurate Latencies for Clock Gates

By default, the tool estimates and updates the clock-gate latencies throughout the `place_opt` command flow. This ensures that the tool uses more accurate clock-gate latencies during datapath optimization and useful-skew optimization performed before clock tree synthesis, when clocks are ideal.

To improve the accuracy of the estimated clock-gate latencies, specify clock tree synthesis settings such as the clock tree buffers and inverters, clock nondefault routing rules, and so on before you run the `place_opt` command.

Note:

If you enabled trial clock tree synthesis as described in Creating a Temporary Clock Tree for Placement and Optimization or clock-gate optimization as described in Optimizing Clock-Gating Cells, which use more accurate methods of modeling the clock tree effects prior to clock tree synthesis, the tool does not perform clock-gate latency estimation and clock-gate-latency-aware placement.

The tool does not modify any `set_clock_latency` constraints specified by you on clock-gating cells. The tool-estimated clock-gate latency values are stored by using the `set_clock_latency -offset` command. You can see these estimated values by using the `write_script -format icc2` command and checking for the `set_clock_latency -offset` commands in the generated output. The estimated latency offset values are not captured in an SDC file generated by the `write_sdc` command.

To prevent the tool from estimating the latency for a specific clock-gate, set a `dont_estimate_clock_latency` attribute on its clock pin by using the `set_attribute` command, as shown in the following example:

```
icc2_shell> set_attribute \
   [get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

Feedback

Enabling Global Route Based High-Fanout Synthesis

When the tool performs high-fanout synthesis and electrical DRC violation fixing, by default, it uses virtual routes.

To use global routes during high-fanout synthesis and electrical DRC violation fixing, set the `place_opt.initial_drc.global_route_based` application option before you run the `place_opt` command:

```
icc2_shell> set_app_options \
   -name place_opt.initial_drc.global_route_based -value 1
```

To enable an integrated two-pass initial placement flow, which includes a trial high-fanout synthesis, set the following application options before you run the `place_opt` command:

```
icc2_shell> set_app_options \
   -name place_opt.initial_drc.global_route_based -value 1
icc2_shell> set_app_options \
   -name place_opt.initial_place.two_pass -value true
```

The integrated two-pass initial placement flow consist of wire-length-driven placement, trial high-fanout synthesis, and timing-driven placement. The tool discards the trial high-fanout synthesis results, but retains the timing-driven placement and proceeds to global route based high-fanout synthesis.

For designs with fragmented floorplans, enabling the integrated two-pass initial placement flow and global route based high-fanout synthesis can help improve timing and reduce congestion.

Enabling the Rebuilding of Buffer Trees

During the `initial_drc` stage of the `place_opt` command, by default, the tool performs high-fanout net synthesis, which builds buffers trees for high-fanout nets. After the initial_drc stage, the tool can remove and rebuild entire buffers trees or portions of buffer trees if doing so improves the setup timing, logical DRC violations, wire length, or area.

To enable rebuilding of buffer trees during

- The initial_opto stage for multivoltage nets, set the `opt.buffering.enable_mv_rebuffering` application option to `true`
- The final_opto stage for any net, set the `opt.buffering.enable_rebuffering` application option to `true`

Changing the Congestion Effort

By default, the `place_opt` command performs congestion optimization to improve the routability of the design. You can disable or control the effort used for this optimization by using the `place_opt.place.congestion_effort` application option setting, which defaults to `medium`.

Feedback

Using of Nondefault Routing Rules for Critical Nets

To improve timing QoR, the tool can use nondefault routing rules on timing critical nets during preroute optimization. In addition, the tool can guide the router to honor these nondefault rule assignments as soft constraints.

To enable this capability for the `place_opt` command, set the `place_opt.flow.optimize_ndr` application option to `true`.

```
icc2_shell> set_app_options -name place_opt.flow.optimize_ndr \
    -value true
```

Enabling Path Optimization

Path optimization is an incremental optimization capability that focus on improving the QoR of timing paths. You can also enable it during the `place_opt` command by setting the `place_opt.flow.do_path_opt` application option to `true`.

```
icc2_shell> set_app_options -name place_opt.flow.do_path_opt -value true
```

Performing IR-Drop-Aware Placement During the place_opt Command

During placement, the tool can use the voltage (IR) drop values of cells to identify areas of high power density and spread the cells with high voltage drop values, which reduces the power density of such areas.

To perform IR-drop-aware placement during the `place_opt` command, use the following steps:

1. Run the `place_opt` command through the final-placement stage by using the `place_opt -to final_place` command.
2. Set up for RedHawk Fusion and perform static or dynamic voltage drop analysis by using the `analyze_rail -voltage_drop` command as shown in the following example:

   ```
   icc2_shell> source redhawk_setup.tcl
   icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
   ```

   For more information, see Performing Voltage Drop Analysis.
3. Enable IR-drop-aware placement by setting the `place.coarse.ir_drop_aware` application option to `true`.
4. (Optional) Specify additional settings for IR-drop-aware placement, as described in Controlling IR-Drop-Aware Placement.
5. Rerun the final-placement stage by using the `place_opt -from final_place` command.

Feedback

Performing Concurrent Clock and Data Optimization During the place_opt Command

Applying useful skew techniques during datapath optimization to improve the timing QoR by taking advantage of the positive slack and adjusting the clock arrival times of registers is referred to as concurrent clock and data (CCD) optimization.

By default, the tool performs concurrent clock and data optimization during the final optimization stage of the `place_opt` command. You can disable concurrent clock and data optimization for the `place_opt` command by setting the `place_opt.flow.enable_ccd` application option to `false`.

The clock latency adjustments derived during the `place_opt` command are stored as clock-latency offsets by using the `set_clock_latency -offset` command and clock-balance-point-delay offsets by using the `set_clock_balance_point -offset` command. You can see these clock-latency offset values by using the `write_script -format icc2` command and checking for the `set_clock_latency -offset` commands in the generated output. The clock-latency offset values are not captured in an SDC file generated by the `write_sdc` command. You can report the clock-balance-point-delay offsets by using the `report_clock_balance_point` command.

To set up the block for concurrent clock and data optimization during the `place_opt` command, perform the following steps:

1. Ensure that the same scenarios that you set as active for the `clock_opt` command are also active for the `place_opt` command.
2. Specify that the tool builds temporary clock trees and uses propagated clocks during the `place_opt` command by using one of the following two settings:

   - ```
     icc2_shell> set_app_options -name place_opt.flow.trial_clock_tree \
       -value true
     ```

   - ```
     icc2_shell> set_app_options -name place_opt.flow.optimize_icgs \
       -value true
     ```

   Alternatively, the tool can use ideal clocks for useful skew computation. However, you must specify accurate ideal clock latencies for the clock pins of all clock gating cells and sequential cells, including macro cells.

You can change the default behavior of concurrent clock and data optimization during the `place_opt` command by performing any of the following optional steps:

1. (Optional) Limit the latency adjustment values for concurrent clock and data optimization as described in Limiting the Latency Adjustment Values.
2. (Optional) Control the latency adjustment of boundary registers for concurrent clock and data optimization as described in Excluding Boundary Paths.

3. (Optional) Ignore specific path groups during concurrent clock and data optimization as described in Excluding Specific Path Groups.
4. (Optional) Ignore specific scenarios during concurrent clock and data optimization as described in Excluding Specific Scenarios.
5. (Optional) Ignore specific sinks during concurrent clock and data optimization as described in Excluding Specific Sinks.
6. (Optional) Analyze the effects of concurrent clock and data optimization as described in Reporting Concurrent Clock and Data Timing.
7. (Optional) Derive automatic zero balance point on macros with internal delays as described in .
8. (Optional) Skew latch and discrete clock together by the same offset as described in Skewing Latch and Discrete Clock Gates.

Using Physical Guidance From the Design Compiler Tool

When you use the `place_opt` command to perform placement and optimization, you can use the Synopsys physical guidance information from the Design Compiler Graphical tool as a starting point.

Using Design Compiler Graphical placement provides the following benefits:

- Reduces the runtime of the placement step
- Achieves better correlation between the Design Compiler Graphical and IC Compiler II tools

To transfer the design from the Design Compiler Graphical tool, use a gate-level netlist in the Verilog format, along with the constraint information.

To preserve consistency and improve correlation, you must

- Use the same logical constraints, including timing and UPF information, in both tools.

  However, you must convert the Design Compiler scenario constraints into the mode, corner, and scenario format supported by the IC Compiler II tool.
- Use the Design Compiler Graphical placement in the IC Compiler II tool and the same floorplan and physical constraints in both tools.

  However, you can provide additional floorplan information that is not currently supported by the Design Compiler Graphical tool, but is needed in the IC Compiler II tool for physical implementation.

Feedback

To transfer the physical information (placement, floorplan, and physical constraints) from the Design Compiler Graphical tool, perform the following tasks:

- Use the `write_def` command in the Design Compiler Graphical tool to generate a DEF file and read it into the IC Compiler II tool by using the `read_def -add_def_only_objects {cells} -convert_sites` command.

  The DEF file generated from the Design Compiler Graphical tool contains the following physical information:

  - Die area, site rows, and tracks
  - Locations and shapes of ports
  - Locations of macros, standard cells and physical only cells
  - Route guides and prerouted nets
  - Placement blockages

- Create a Tcl file containing physical constraints not captured in the DEF file and apply those constraints to the design in the IC Compiler II tool.

  Physical constraints not captured in DEF include

  - Voltage area definitions
  - Layer constraints
  - Special route guides with utilization
  - Special blockages with blocked layers

To use of Synopsys physical guidance information in the IC Compiler II tool, set the `place_opt.flow.do_spg` application option before you run the `place_opt` command:

```
icc2_shell> set_app_options -name place_opt.flow.do_spg -value true
```

The following example script shows how to use the Synopsys physical guidance with the `place_opt` command:

```
# Read in the netlist
read_verilog design_spg.v

# Apply the timing constraints
source timing_const.tcl

# Apply the UPF constraints
load_upf design.upf

# Apply the DEF file generated from the Design Compiler Graphical tool
read_def -add_def_only_objects {cells} -convert_sites design_dcg.def
```

Feedback

```
# Apply the placement constraints not supported by DEF
source placement_const.tcl

# Enable the Synopsys placement guidance flow
set_app_options -name place_opt.flow.do_spg -value true

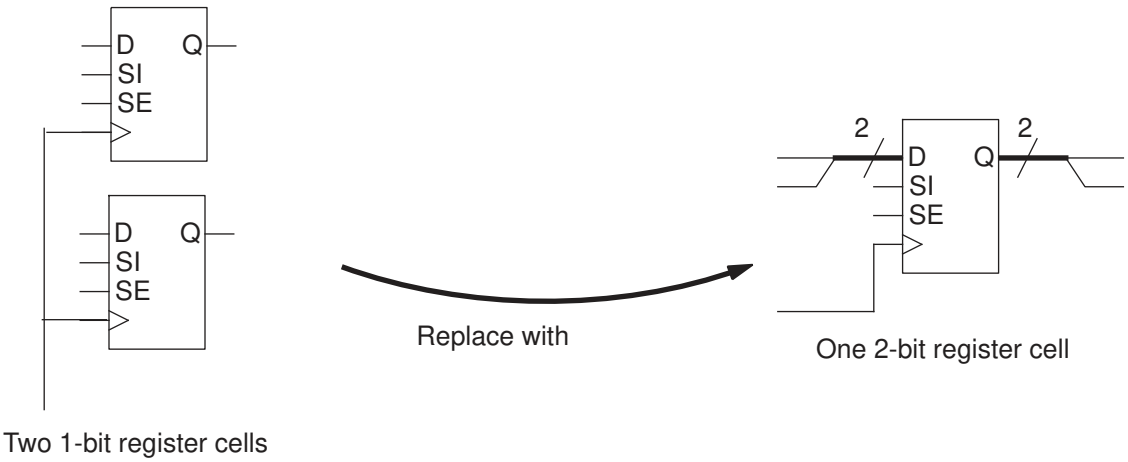
# Perform placement and optimization
place_opt
```

For more information, see the Synopsys physical guidance information in the *Design Compiler User Guide*.

Performing Multibit Register Optimization

The IC Compiler II tool can combine (bank) single-bit registers or smaller multibit registers and replace them with equivalent larger multibit registers. For example, the tool can combine eight 1-bit registers or four 2-bit register banks and replace them with one 8-bit register bank or two 4-bit register banks. The tool merges single-bit registers only if they have the same timing constraints, and it copies the timing constraints to the resulting multibit register.

Figure 25 Replacing Multiple Single-Bit Register Cells With a Multibit Register Cell



Replacing single-bit registers with multibit registers reduces

- Area due to shared transistors and optimized transistor-level layout
- The total clock tree net length
- The number of clock tree buffers and clock tree power

Feedback

The tool can also split (debank) large multibit registers into smaller multibit registers or single-bit registers, if it improves the total negative slack.

Some logic libraries have mixed-drive-strength multibit registers where some bits have a higher drive strength than others. For designs that use such cells, if a violating path goes through a lower-drive-strength bit, the tool can rewire the mixed-drive-strength multibit cell such that the violating path goes through a higher-drive-strength bit.

The following topics provide information about how to perform multibit register optimization:

- Performing Integrated Multibit Register Optimization
- Performing Multibit Register Optimization Using Discrete Commands
- Banking Multibit Retention Registers

Performing Integrated Multibit Register Optimization

To perform multibit register optimization during the `place_opt` command,

1. Enable multibit register banking by setting the `place_opt.flow.enable_multibit_banking` application option to `true`.
2. (Optional) Enable multibit register debanking by setting the `place_opt.flow.enable_multibit_debanking` application option to `true`.
3. (Optional) Enable banking of registers driven by equivalent integrated-clock-gating (ICG) cells by setting the `multibit.banking.across_equivalent_icg` application option to `true`.

   By default, the tool only banks registers that are driven by the same clock net. When you enable this feature, the tool banks registers driven by equivalent clock-gating cells and merges the clock-gating cells. However, when doing so, it might lose nondefault routing rules applied to the nets connected to the clock-gating cells. Therefore, check and reapply these nondefault routing rules after optimization.
4. (Optional) Enable multibit register rewiring by setting the `place_opt.flow.enable_multibit_rewiring` application option to `true`.
5. (Optional) Specify settings for multibit register banking by using the `set_multibit_options` command.

   For example, the following command excludes the reg[1] cell from multibit optimization:

   ```
   icc2_shell> set_multibit_options -exclude [get_cells "reg[1]"]
   ```
6. (Optional) Control multibit register banking and debanking as shown in the following table.

Feedback

Table 16 Application Options for Controlling Multibit Register Banking and Debanking

To do this	Set this application option to true
Ignore scan flip-flops that are not included in the SCANDEF information during banking	`multibit.banking.enable_strict_scan_check-`
Consider multibit register library cells with multiple scan-in pins and internal scan-in pins during banking	`multibit.banking.lcs_consider_multi_si_cells`
Ignore user-specified size-only cells during banking and debanking	`multibit.common.ignore_sizeonly_cells`

7. Perform placement and optimization by using the `place_opt` command.
8. Report multibit components by using the `report_multibit` command.

Performing Multibit Register Optimization Using Discrete Commands

You can perform multibit banking using the following discrete steps:

1. Perform initial placement by using the `create_placement` command.
2. (Optional) Specify settings for multibit register banking by using the `set_multibit_options` command.
3. Identify the groups of cells that can be replaced by multibit cells by using the `identify_multibit` command, as described in Identifying Multibit Banks.
4. Optimize the design by using the `place_opt -from initial_drc` command.
5. (Optional) Report multibit banking information by using the `report_multibit` command.
6. (Optional) If multibit banking does not improve QoR, split specific multibit banks into individual registers or smaller multibit banks by using the `split_multibit` command, as described in Splitting Multibit Banks.

Feedback

Identifying Multibit Banks

To identify groups of registers, isolation cells, or level-shifter cells that can be replaced by multibit cells, use the `identify_multibit` command.

When you use the `identify_multibit` command, you must specify:

- How to apply the multibit banking information by using one of the following methods:
  - To have the tool apply the banking information to the design, use the `-apply` option.
  - To manually apply the banking information, generate an output file containing `create_multibit` commands by using the `-output_file` option.

    You can view the output, edit it if necessary, and apply it to the design by using the `source` command.
- The type of cells you want to bank by using one of the following methods:
  - To identify groups of registers that can be banked, use the `-register` option.
  - To identify groups of isolation or level-shifter cells that can be banked, use the `-mv_cell` option.

When banking registers with the `-register` option, optionally, you can

- Control the mapping of single bits to multibit banks by specifying an input map file using the `-input_map_file` option. This file contains mapping information about which multibit register bank replaces which single-bit registers, as shown in the following example:

```
reg_group_1 {REGX1 REGX2 REGX4}
2 {1 MREG2}
4 {1 MREG4}
6 {1 MREG2}{1 MREG4}
```

  This map file contains a functional group named reg_group_1, which specifies how single-bit registers of reference cell types REGX1, REGX2, and REGX4 can be grouped to form multibit registers as follows:
  - The first line specifies that two single-bit registers can be combined to form a register bank consisting of one MREG2 reference cell.
  - The second line specifies that four single-bit registers can be combined to form a register bank consisting of one MREG4 reference cell.
  - The third line specifies that six single-bit registers can be combined to form a register bank consisting of one MREG2 and one MREG4 reference cells.

- Exclude registers based on their slack by specifying a slack threshold with the `-slack_threshold` option

  Registers with a slack less than the specified threshold are ignored. By default, the tool considers all registers for banking.

In addition, you can optionally control the cells that are banked by specifying:

- A list of instance to consider for banking by using the `-cells` option.

  When you use this option, the tool does not consider any other cells for banking.

- A list of instances to exclude by using the `-exclude_instance` option.
- A list of library cells to exclude all instances by using the `-exclude_library_cells` option.

By default, the `identify_multibit` command performs DFT optimization, which repartitions scan chains such that registers driven by the same clock driver or gating cell are put in the same scan chain. This increases the number of single-bit registers that can be combined to form multibit register banks. If you do not want the tool to repartition the scan chains, use the `-no_dft_opt` option with the `identify_multibit` command.

Splitting Multibit Banks

To improve local congestion or path slack, you can split a multibit bank into smaller multibit banks or single-bit cells by using the `split_multibit` command. By default, the command splits all multibit cells on timing paths with negative slack.

To split

- A specific cell, specify the cell instance name.

  You can specify the reference names of the smaller cells to use after splitting by using the `-lib_cells` option.

  The following example splits the multibit cell instance named reg_gr0, which is a four-bit register bank, into two two-bit multibit banks:

  ```
  icc2_shell> split_multibit reg_gr0 -lib_cells {lib1/MREG2 lib1/MREG2}
  ```

- Multibit cells on paths with slack less than a specific value, use the `-slack_threshold` option.

  The following example splits the multibit cells on paths with a slack worse than -1:

  ```
  icc2_shell> split_multibit -slack_threshold -1
  ```

- Multibit cells in a specific timing path group, use the `-path_groups` option.

Feedback

When you use this option, the tool splits the multibit cells on the pats with negative slack in the specified path group. You can specify a slack threshold limit for the path group by using the `-slack_threshold` option with the `-path_groups` option.

The following example splits the multibit cells on paths with a slack worse than -0.5 in the timing path group CLKA:

```
icc2_shell> split_multibit -path_groups CLKA \
   -slack_threshold -0.5
```

You can further control splitting as follows:

- Exclude specific cells by using the `-exclude_instance` option
- Consider only specific cells by using the `-cells` option

The `split_multibit` command only splits multibit cells that were created by the `create_multibit` or `identify_multibit` command.

Banking Multibit Retention Registers

The tool can combine single-bit retention registers and create a multibit retention register if

- The single-bit retention registers are associated with the same retention strategy
- The retention pins of the single-bit retention registers have the same driver
- The cell library has a matching multibit retention register
- The library cell for the multibit retention register is specified as a library cell for the retention strategy by using the `map_retention_cell -lib_cells` command

The tool can split a multibit retention register if

- The cell library has matching single-bit retention registers
- The library cells for the single-bit retention registers are specified as library cells to use for the retention strategy by using the `map_retention_cell -lib_cells` command

Note:

If you are using the `identify_multibit` command, to identify multibit retention registers, use the `-register` option, and not the `-mv_cell` option.

Performing Magnet Placement

To improve congestion for a complex floorplan or to improve timing for the design, you can use magnet placement to specify a fixed object as a magnet and have the tool place all the standard cells connected to the magnet object close to it. You can specify fixed macro cells, pins of a fixed macro cells, or I/O ports as the magnet object.

Feedback

For best results, perform magnet placement before standard cell placement.

To perform magnet placement, use the `magnet_placement` command with a specification of the magnets and options for any special functions you need to perform.

Table 17 shows the `magnet_placement` options.

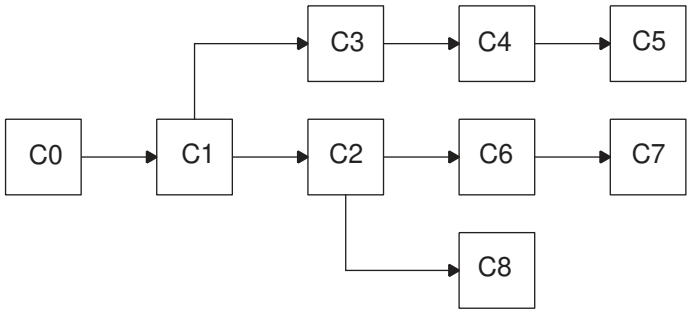
Table 17 The magnet_placement Command Options

To do this	Use this option
Specify the cells to pull toward the magnet object.	`-cells` *`cell_list`*
Enable hierarchical cells to be specified with the `-cells` option and specify how these hierarchical cells should be handled	`-hierarchy_mode default | block | all`
Enable the movement of cells marked as fixed or legalize-only.	`-move_fixed` `-move_legalize_only`
Mark the moved cells as fixed or legalize-only after magnet placement.	`-mark_fixed` `-mark_legalize_only`
Specify the number of logic levels from the magnet that are considered for magnet placement.	`-logical_levels` *`number`*
Exclude buffers and inverters when determining the level of logic.	`-exclude_buffers`
Prevent the tracing of paths at inputs of sequential cells when finding the cells to pull.	`-stop_by_sequential_cells`
Prevent the tracing of paths at the outputs of sequential cells when finding the cells to pull.	`-stop_on_sequential_cells`
Prevent the tracing of paths beyond the specified pins, ports, or cells when finding the cells to pull.	`-stop_points`
Prevent placing cells on soft blockages. By default, the command only avoids hard blockages.	`-avoid_soft_blockage`
Specify how to place cells when the magnet objects are multiple ports or long ports	`-multiple_long_port_mode auto|nearby`
Report the cells that will be moved toward the magnet object, but not pull them	`-only_report_magnet_cells`
Create a collection of cells that are being pulled toward the magnet object.	`-get_collection`

Feedback

The following examples use the data paths shown in Figure 26.

Figure 26 Example of Pulling Cells in a Contiguous Data Path



The following command pulls all cells, C1 through C8, toward the C0 magnet object:

```
icc2_shell> magnet_placement C0
```

The following command pulls the C6, C7, and C8 cells toward the C2 magnet object:

```
icc2_shell> magnet_placement C2 -cells {C6 C7 C8}
```

Although the C3, C4, and C5 cells form a contiguous datapath, they are isolated from the C0 cell, so the following command pulls no cells toward the C0 cell.

```
icc2_shell> magnet_placement C0 -cells {C3 C4 C5}
```

The following command pulls no cells toward the C0 magnet object because the C3, C6, and C8 cells do not form a contiguous data path.

```
icc2_shell> magnet_placement C0 -cells {C3 C6 C8}
```

Refining Placement

To refine the placement and minimize congestion, use the `refine_placement` command. By default, this command performs incremental placement on the entire design.

- To refine the placement of a specific region, use the `-coordinates` option and specify the coordinates of the region.
- To change the effort used for placement, use the `-effort` option and specify either `low`, `medium`, or `high`. The default is `medium`.

- To change the effort used for minimizing congestion, use the `-congestion_effort` option and specify either `low`, `medium`, or `high`. The default is `medium`.
- To control the perturbation of the existing placement, use the `-perturbation_level` option and specify either `min`, `medium`, `high`, or `max`. The default is `medium`.

Performing Placement and Optimization on Multivoltage Blocks

Placement and optimization automatically consider multivoltage constraints, such as voltage areas.

During placement, the tool

- Places cells within their associated voltage area or voltage area shape
- Places level shifters and isolation cells close to the voltage area boundaries

During optimization, when the tool inserts buffers, it

1. Considers the related supplies of the driver and loads of the net and does a dynamic analysis of the power states using one of the following:
   - The UPF power state table you created using the `create_pst` command
   - The UPF power state groups you created using `add_power_state` command
2. Chooses a buffering supply that does not introduce new multivoltage violations, based on the power states and the available supplies in the voltage area being buffered.

   Single-rail buffers are preferred to dual-rail buffers to save power and area, and improve routability.

To understand how or whether specific nets are buffered, use the `check_bufferability` command, as described in Analyzing the Bufferability of Nets.

Rebuilding Buffer Trees

During the `initial_drc` stage of the `place_opt` command, the tool removes existing buffer trees and performs high-fanout synthesis. After you run the `place_opt` command, to rebuild specific buffers trees to meet specific requirements, use the `create_buffer_trees` command.

By default, the `create_buffer_trees` command

- Rebuilds the buffer trees for all high-fanout nets

  To rebuild a specific high-fanout net, use the `-from` option.
- Removes the existing buffer trees before rebuilding

Feedback

To keep the existing buffers and incrementally build the buffer trees, use the `-incremental` option.

- Builds the buffer trees using virtual route topologies and extraction

  To build the buffer trees based on global routes, use the `-global_route_based` option.

- Does not legalize the newly added cells

  To legalize the added cells, run the `legalize_placement` command after you run the `create_buffer_trees` command.

The following example rebuilds the buffer tree for the n33 net using only buf4x and buf8x type buffers:

```
icc2_shell> set_lib_cell_purpose -include none {tech_lib/*}
icc2_shell> set_lib_cell_purpose \
   -include optimization {tech_lib/buf4x tech_lib/buf8x}
icc2_shell> create_buffer_trees -from n33 -global_route_based
```

Note:

The `create_buffer_trees` command should only be used at the preroute stage of the design flow.

Adding and Removing Tie Cells

When you run the `place_opt` or `clock_opt` command, the tool can add tie cells during optimization. In addition, you can add tie cells for specific pins or ports by using the `add_tie_cells` command. The first time tie cells are added to a block by the `place_opt`, `clock_opt`, or `add_tie_cells` command, the tool optimizes the tie cell connections by removing all existing tie cells and adding tie cells which are optimally clustered.

To remove specific tie cells, use the `remove_tie_cells` command. By default, the tool removes all tie cells in the current block. To remove specific instances or all instances of a specific library cell, use the `-objects` option.

Identifying Issues That Cannot Be Fixed During Optimization

The IC Compiler II tool provides the following capabilities for identifying issues that cannot be fixed during optimization:

- Analyzing the Bufferability of Nets
- Analyzing Violations That Cannot Be Fixed

Feedback

Analyzing the Bufferability of Nets

To analyze whether a net in the current block can be buffered or to report why the net cannot be buffered, use the `check_bufferability` command. This command checks if there are

- Supply nets available for buffering the net in the specified voltage area
- Suitable library cells available for buffering the net
- Settings on the net that prevent buffering, such as ideal-network or don't-touch constraint settings

With this command, you must specify the following:

- The net to analyze by using one of the following methods:
  - Specify the net name using the `-nets` option.
  - Specify the driver and loads by using the `-driver` and `-loads` options, and the logical hierarchy in which to perform the analysis by using the `-hierarchy` option.
- The voltage area in which to analyze the bufferability by using the `-voltage_area` option.

For example, to analyze if buffers can be added on the net named n34 in the PD1 power domain, use the following command:

```
icc2_shell> check_bufferability -nets n34 -voltage_area PD1
Information: Can insert dual rail buffers (8 lib_cells) and inverters (6
 lib_cells) with supply nets (power: VDDSW, ground: VSS). (MV-454)
```

To analyze whether buffers can be added between the top_reg_1_/Q driver pin and its top_U1_9/B load pin in the top hierarchy in the PD_TOP voltage area, use the following command:

```
icc2_shell> check_bufferability -driver top_reg_1_/Q \
   -loads top_U1_9/B -hierarchy / -voltage_area PD_TOP
Information: Cannot insert buffers or inverters, because required supply
 nets are not available. (MV-459)
```

For a more information about the bufferability of a net, use the `-verbose` option.

Analyzing Violations That Cannot Be Fixed

To analyze a block and generate a report that contains information about the violations that cannot be fixed, use the `analyze_design_violations` command. The report contains a summary of the analysis, followed by a detailed list of the violations, in

descending order. The violations are categorized based on the reason why they cannot be fixed.

When you use this command, you must specify the type of violation you want to analyze by specifying one of the following setting with the `-type` option:

- `max_trans` to analyze maximum-transition violations
- `setup` to analyze setup violations
- `hold` to analyze hold violations
- `static_noise` to analyze static noise violations
- `only_lib` to analyze library settings that affect optimization

In addition, you can specify the following:

- A threshold for identifying high-fanout nets by using the `-fanout` option.

  By default, the tool considers nets with a fanout larger than 30 to be high-fanout nets.
- A slack threshold for identifying small timing violations by using the `-slack` option.

  By default, the tool considers any slack violation less than 5 picoseconds as a small violation.
- Nets for maximum-transition analysis by using the `-drc_nets`.

  By default, analyzes all nets for maximum-transition violations.
- Endpoints for setup and hold analysis by using the `-endpoints` option.

  By default, analyzes all endpoints for setup and hold violations.
- An output file name by using the `-output` option.

  The tool adds a .txt extension to the output file name you specify. By default, the tool generates a file named analyze_design_violations.txt.

Analyzing the Placement

The IC Compiler II tool provides the following GUI- and report-based capabilities for analyzing the placement:

- Reporting Utilization
- Reporting the Placement QoR
- Querying and Changing the Placement Status
- Analyzing the Placement in the GUI

Feedback

Reporting Utilization

The utilization of a block is calculated by using the following formula, where demand is the total area occupied by objects and capacity is the total available area:

Utilization = Demand / Capacity

To report the utilization, use the following steps:

1. (Optional) Create a configuration for reporting the utilization by using the `create_utilization_configuration` command.

   The following example creates a utilization configuration for the current block named config1 that uses the core area for calculating the capacity and excludes hard and soft placement blockages:

   ```
   icc2_shell> create_utilization_configuration -scope block config1 \
      -capacity core_area -exclude {hard_blockages soft_blockages}
   ```

   To remove a utilization configuration, use the `remove_utilization_configurations` command.

2. Report the utilization by using the `report_utilization` command.

   To specify a utilization configuration and where it is stored, use the `-config` and `-scope` options. The valid values for the `-scope` option are `tech`, `lib`, and `block`. If you do not specify the `-scope` option with the `-config` option, the tool first searches the current block, then the design library, and finally the logic library for the specified configuration.

   To report the utilization for specific objects, use the `-of_objects` option and specify one or more blocks, voltage areas, site arrays, or bounds. All objects must be of the same type.

Reporting the Placement QoR

The `report_placement` command displays information about the placement QoR for the current block. By default, the command reports the total half-perimeter boundary box wire length for all nets in the block, as well as any placement violations, as shown in the following example:

```
icc2_shell> report_placement
...
...

  Wire length report (all)
  ==================
  wire length in design leon3mp: 12386597.894 microns.
```

Feedback

```
  Physical hierarchy violations report
  ====================================
  Violations in design leon3mp:
     0 cells have placement violation.

1
```

To report on the subblocks, as well as the current block, use the `-hierarchical` option.

To restrict the wire length calculation to specific types of nets, use the `-wirelength` option with the appropriate keyword.

- To include only nets that cross a physical boundary, use `-wirelength interface`.
- To include only nets connected to hard macros, use `-wirelength hard_macro`.
- To include only nets connected to I/O pins or pads, use `-wirelength IO`.
- To suppress the wire length report, use `-wirelength none`.

To restrict the types of placement violations reported, use the `-physical_hierarchy_violations` option with the appropriate keyword.

- To report only the cells placed outside of the placement area of the current block, use `-physical_hierarchy_violations internal`.
- To report only the cells that overlap a subblock, use `-physical_hierarchy_violations external`.
- To suppress the placement violation report, use `-physical_hierarchy_violations none`.

You can also report the following placement metrics:

- Swimming pool area

  A *swimming pool* is a region that is enclosed by subblocks, hard macros, and placement blockages. Swimming pools can make it difficult to successfully place or route the block and should either be covered with a placement blockage or removed.

  - To report the total swimming pool area in the current block, use the `-swimming_pool_area` option.
  - To report swimming pool regions with an area smaller than a specific threshold, use the `-maximum_swimming_pool_area` option.

- Thin channel area

  A *thin channel* is a narrow region between subblocks of the current block, whether they are hard macros or physical hierarchies. Thin channels can make it difficult to successfully place or route the block and should either be covered with a placement blockage or widened.

Feedback

  - To report the total thin channel area in the current block, use the `-thin_channel_area` option.
  - To report thin channels with a width or height smaller than a specific threshold, use the `-maximum_thin_channel_width` or `-maximum_thin_channel_height` option.

- Wire length of nets routed over hard macros

  To report the wire length of nets routed over hard macros in the current block, use the `-hard_macro_route_over` option. This option reports the horizontal and vertical wire length for each hard macro that has over-macro routing, as well as the total wire length for over-macro routing.

- The number of hard macro overlaps

  To report the total number of overlapping hard macros in the current block, use the `-hard_macro_overlap` option.

Querying and Changing the Placement Status

The `status` attribute of a cell instance indicates the current status of its placement. Table 18 shows the possible values of the `status` attribute.

Table 18 Placement Status Keywords

Placement status	Description	DEF syntax
`unplaced`	The cell instance has not yet been placed. **Note:** To revert all placed cells to the unplaced state, use the `reset_placement` command. This command modifies the placement status only for cells with a placement status of `placed`; it does not modify the placement status for cells with a placement status of `fixed` or `locked`.	+UNPLACED
`placed`	The cell instance has been placed, but can be moved by subsequent operations.	+PLACED
`fixed`	The cell instance has been placed and can be sized but not moved by subsequent operations. **Note:** To prevent these cells from being sized, you must set a `dont_touch` attribute on these cells, as described in Preserving Cells and Nets During Optimization.	+FIXED

Feedback

Table 18 Placement Status Keywords (Continued)

Placement status	Description	DEF syntax
`legalize_only`	The cell instance has been placed and can be moved only by the legalizer.	N/A
`locked`	The cell instance has been placed and cannot be moved either manually or by the tool.	+COVER

To get the placement status of a cell instance, query its `status` attribute by using the `get_attribute` command. For example, to get the placement status of the cell instance named INST1, use the following command:

```
icc2_shell> get_attribute -objects [get_cells INST1] -name status
```

To change the placement status of a cell instance, use the `set_placement_status` command. For example, to set the placement status of the cell instance named INST1 to `fixed`, so that it is not moved during subsequent optimization and placement, use the following command:

```
icc2_shell> set_placement_status fixed [get_cells INST1]
```

Analyzing the Placement in the GUI

The IC Compiler II graphical user interface (GUI) provides the following tools for analyzing and visualizing the quality of results after placement:

- Global Route Congestion Map

  You can use the global route congestion map to visualize the quality of placement with respect to the avoidance of routing congestion.

- Cell Density Map

  You can use the cell density map to identify areas of high cell density in the block or in a rectangular area of the block.

- Pin Density Map

  You can use the pin density map to identify areas of high pin density in the block or in a rectangular area of the block.

Feedback

For more information about using these analysis tools in the GUI, see the *Using Map and Visual Modes* topic in the *IC Compiler II Graphical User Interface User Guide.*

Analyzing Timing

The IC Compiler II tool automatically updates timing and delay estimation results when needed. However, you can explicitly update the timing by using the `update_timing` command.

You can use the following commands to generate timing reports for a block:

- `report_timing`

  The `report_timing` command reports the worst-case timing paths for the current block.

- `report_qor`

  The `report_qor` command displays QoR information and statistics for the current block.

- `report_constraints`

  The `report_constraints` command reports the logical DRC violations for the current block.

For more information about generating these reports, see the *Generating Reports* topic in the *IC Compiler II Timing Analysis User Guide*.

Analyzing Power

To calculate and report the power for a block, use the `report_power` command.

The `report_power` command supports the following methods for calculating the leakage power:

- Average

  When using this method, the calculation is based on equal weighted probabilities for all states.

- Unconditional

  When using this method, the calculation is based on the `default_cell_leakage_power` library attribute or `cell_leakage_power` library cell attribute.

Feedback

- State-dependent

  When using this method, the calculation is based on the weighted sum of each state condition of the cell. The calculation depends on the state-dependent leakage library model characterization and static probability. If the state-depended leakage information is not available, the power calculation uses the unconditional method.

To select the leakage-power calculation method for the `report_power` command, set the `power.leakage_mode` application option to `average`, `unconditional`, or `state`. To not report leakage power, set this application option to `off`. The default is `state`.

```
icc2_shell> set_app_options -name power.leakage_mode -value average
```

To use composite current source (CCS) receiver model capacitance during power analysis and reporting, set the `power.use_ccs_rcv_cap` application option to `true`.

Example 14 shows a default power report.

Example 14 Default Power Report

```
...
...

Mode: my_mode
Corner: bc_corner
Scenario: bc_scenario
Voltage: 1.160000
Temperature: 125.000000

Voltage unit: 1.00V
Temperature unit: 1.00C
Power unit: 1.00pW

Cell Leakage Power    = 5125495.58 uW

Power Group         Leakage Power           Total Power            (    %  )
------------------------------------------------------------------------------
---
io_pad                    0.00e+00              0.00e+00           (  0.00%)
memory                    0.00e+00              0.00e+00           (  0.00%)
black_box                 0.00e+00              0.00e+00           (  0.00%)
clock_network             3.32e+10              3.32e+01           (  0.65%)
register                  1.62e+12              1.62e+03           ( 31.67%)
sequential                1.16e+09              1.16e+00           (  0.02%)
combinational             3.47e+12              3.47e+03           ( 67.66%)
------------------------------------------------------------------------------
---
Total                     5.13e+12 pW           5.13e+03 mW
1
```

Creating Power Groups for Reporting

When you run the `report_power` command without specifying a list of cells, the tool reports the power information for the default power groups. You can create power groups consisting of cell instances or library cells by using the `set_power_group` command. The

Feedback

`report_power` command reports the power information for both the default and user-specified power groups.

If a cell belongs to multiple power groups, the power group it belongs to for reporting purposes is determined by the following precedence:

1. The power group specified for the cell instance with the `set_power_group` command.
2. The power group specified for its library cell with the `set_power_group` command.
3. The default power group of the cell.

To get the power group of a specific cell, use the `get_power_group` command.

- Use the `-default` option to get the default power group it belongs to.
- Use the `-user` option to get the user-defined power groups it might belong to.
- Use no option to get the power group it belongs to for reporting purposes.

To report all the power groups in the block, use the `report_power_groups` command. To get a collection of cells in a specific power group, use the `get_power_group_objects` command.

To remove a power group or remove a cell from a specific power group, use the `reset_power_group` command.

Reporting Pin-Based Clock Network Power

The `report_power` command reports the power information for the clock network as part of the default power group named clock_network.

By default, the clock network power that was reported is cell-based. If an output of a cell is on the clock network, the power of that cell, including the power of all its inputs and outputs, is considered as part of the clock network. However, you can limit the clock network power reported by the `report_power` command to include only the pins that are on the clock network by using the following application option setting:

```
icc2_shell> set_app_options -list {power.clock_network pin_based}
```

The default is `cell_based`.

The power information that is reported by the `report_clock_qor -type power` command is pin-based.

Feedback

Comparing QoR Data

You can generate a web-based report to view and compare your QoR data with a QORsum report by performing the following steps:

1. (Optional) Configure the QoR data, which is captured and displayed in the subsequent steps, by using the `set_qor_data_options` command.
   - To specify the most critical power scenarios in your design, use the `-leakage_scenario` and `-dynamic_scenario` options.

     The tool uses the power scenarios you specify to generate the high-level summary of the power QoR in the QORsum report. If you do not specify these options, it uses the active power scenario with the highest total power for both the leakage and dynamic scenario for the power QoR summary.

     These settings are only used for the power QoR summary. The tool uses the power information of all active power scenarios to capture and report the detailed power information in the QORsum report.
   - To specify the most critical clock name and clock scenario, use the `-clock_name` and `-clock_scenario` options.

     The tool uses the clock name and scenario you specify to generate the high-level summary of the clock QoR in the QORsum report. If you do not specify these options, the tool identifies the most critical clock and uses it for the clock QoR summary.

     These settings are only used for the clock QoR summary. The tool uses all clocks to generate the detailed clock QoR information in the QORsum report.
   - To specify a name to identify the run in the QORsum report, use the `-run_name` option.

     By default, the tool names each run with a number, such as Run1, Run2, and so on. You can use this option to give a more meaningful name to each run. You can also specify the run name by using the `-run_names` option when you generate the QORsum report by using the `compare_qor_data` command in step 3. If you do so, the tool ignores the run name specify by the `set_qor_data_options -run_name` command.

   The following example specifies the leakage-power scenario, dynamic-power scenario, clock scenario, and the clock to use for the corresponding summary in the QORsum report:

```
icc2_shell> set_qor_data_options \
   -leakage_scenario S1 -dynamic_scenario S2 \
   -clock_scenario S3 -clock_name sys_clk
```

Feedback

2. Collect the QoR data for your report by using the `write_qor_data` command. The `write_qor_data` command captures QoR data related to timing, power, runtime, and other metrics.

   You can run the `write_qor_data` command multiple times, at each stage of the design flow at which you want to capture QoR data. Use the `-label` and specify a value to indicate at which stage of the flow you are capturing the data.

   By default, the `write_qor_data` command captures data relevant to a placed and optimized design. You can specify the `-report_group` option with any of the five supported value (`unmapped`, `mapped`, `placed`, `cts`, `routed`) to tune the set of reports being captured, based on the state of the flow where you are capturing the data. For finer-grain control, use the `-report_list` option to explicitly specify each report you want to generate.

   The `write_qor_data` command captures a standard set of QoR data to disk from most of the common tool reports like the `report_qor` and `report_power`. If there is additional custom data to capture that is not part of the command collection, use the `capture_qor_data` command to capture fully custom QoR data or GUI layout images to include in the web-based QORsum report. This can be used to supplement the standard QoR data captured by the `write_qor_data` command, or to create fully custom QORsum reports. Related commands include `define_qor_data_panel`, that lets you create a new panel in the QORsum report for viewing QoR data, and `set_qor_data_metric_properties`, that lets you customize the properties of any metric column in any QORsum report panel, so you can control things like the style of coloring, or the thresholds for coloring when comparing data.

3. Generate the QORsum report by using the `compare_qor_data` command.

   This command takes the data captured by one or more runs of the `write_qor_data` command and creates a web-based report for viewing and comparing those results.

   You must specify the location of the output of each of the `write_qor_data` runs from the previous step by using `-run_locations` option. Each path location corresponds to the run result that you want to compare.

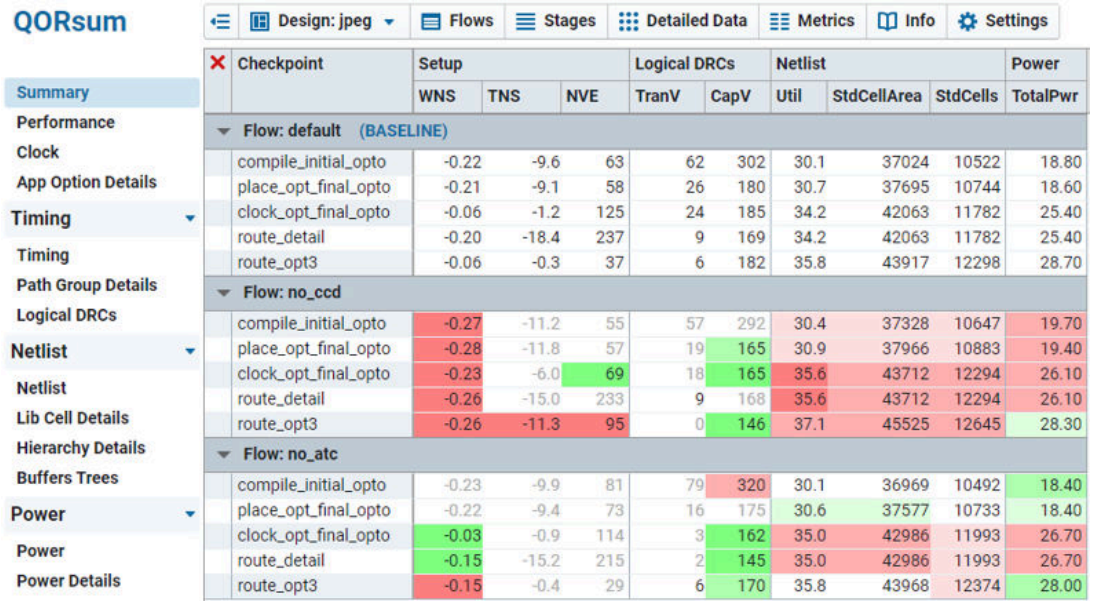
   You can assign a specific name to identify the run in the QORsum report by using the `-run_names` option. If you do so, the tool ignores the run name specify by using the `set_qor_data_options -run_name` command. By default, the tool names each run by a number, such as run1, run2, and so on.

   You can specify the output directory by using the `-output` option. By default, the tool writes the report to a directory named compare_qor_data. To overwrite exiting output data, use the `-force` option. By default, the tool does not overwrite existing output data.

Feedback

4. View the generated QORsum report by using the `view_qor_data` command.

Figure 27 QORsum Report



Checkpoint	Setup			Logical DRCs		Netlist			Power
	WNS	TNS	NVE	TranV	CapV	Util	StdCellArea	StdCells	TotalPwr
Flow: default (BASELINE)									
compile_initial_opto	-0.22	-9.6	63	62	302	30.1	37024	10522	18.80
place_opt_final_opto	-0.21	-9.1	58	26	180	30.7	37695	10744	18.60
clock_opt_final_opto	-0.06	-1.2	125	24	185	34.2	42063	11782	25.40
route_detail	-0.20	-18.4	237	9	169	34.2	42063	11782	25.40
route_opt3	-0.06	-0.3	37	6	182	35.8	43917	12298	28.70
Flow: no_ccd									
compile_initial_opto	-0.27	-11.2	55	57	292	30.4	37328	10647	19.70
place_opt_final_opto	-0.28	-11.8	57	19	165	30.9	37966	10883	19.40
clock_opt_final_opto	-0.23	-6.0	69	18	165	35.6	43712	12294	26.10
route_detail	-0.26	-15.0	233	9	168	35.6	43712	12294	26.10
route_opt3	-0.26	-11.3	95	0	146	37.1	45525	12645	28.30
Flow: no_atc									
compile_initial_opto	-0.23	-9.9	81	79	320	30.1	36969	10492	18.40
place_opt_final_opto	-0.22	-9.4	73	16	175	30.6	37577	10733	18.40
clock_opt_final_opto	-0.03	-0.9	114	3	162	35.0	42986	11993	26.70
route_detail	-0.15	-15.2	215	2	145	35.0	42986	11993	26.70
route_opt3	-0.15	-0.4	29	6	170	35.8	43968	12374	28.00

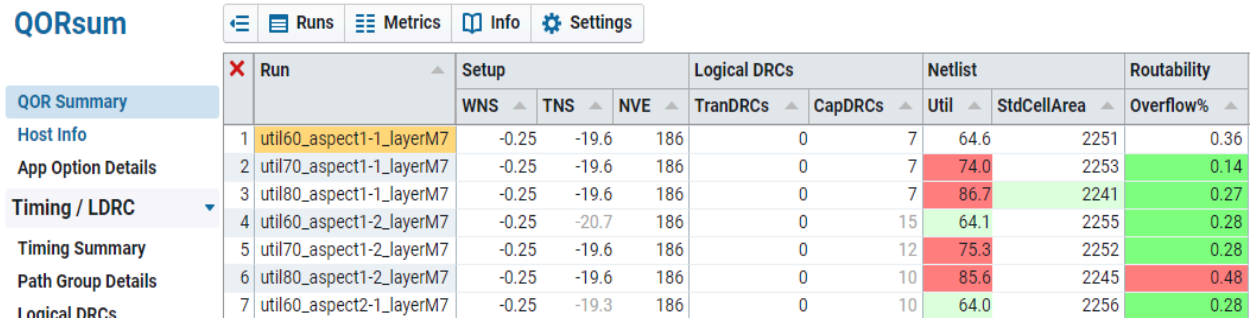
For more information about exploring the comparison data, see the following topics:

- Setting Your Baseline Run
- Changing the QoR Display Style
- Sorting and Filtering the Data
- Exploring the Detailed Comparison Data

Setting Your Baseline Run

By default, the first row of data is your baseline run against which all other runs are compared. The name of the baseline run is highlighted in gold to mark it as the “golden,” or baseline, result.

Feedback

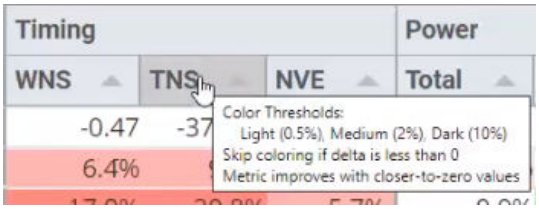


	Run	Setup			Logical DRCs		Netlist		Routability
		WNS	TNS	NVE	TranDRCs	CapDRCs	Util	StdCellArea	Overflow%
1	util60_aspect1-1_layerM7	-0.25	-19.6	186	0	7	64.6	2251	0.36
2	util70_aspect1-1_layerM7	-0.25	-19.6	186	0	7	74.0	2253	0.14
3	util80_aspect1-1_layerM7	-0.25	-19.6	186	0	7	86.7	2241	0.27
4	util60_aspect1-2_layerM7	-0.25	-20.7	186	0	15	64.1	2255	0.28
5	util70_aspect1-2_layerM7	-0.25	-19.6	186	0	12	75.3	2252	0.28
6	util80_aspect1-2_layerM7	-0.25	-19.6	186	0	10	85.6	2245	0.48
7	util60_aspect2-1_layerM7	-0.25	-19.3	186	0	10	64.0	2256	0.28

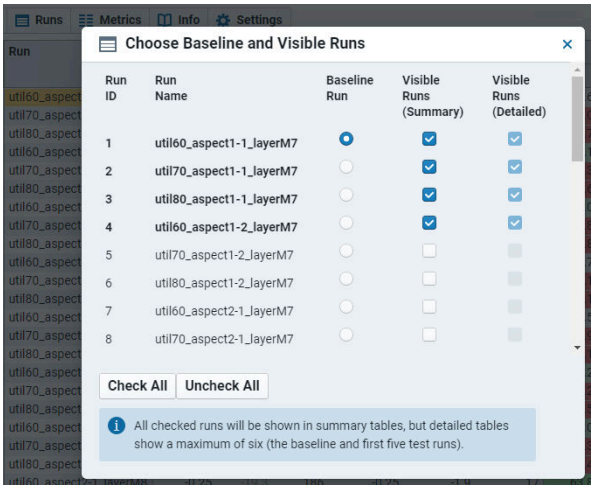
The shading of non-baseline cells indicates the direction and degree by which the data differs from the baseline:

- Red indicates a degradation compared to the baseline; green indicates an improvement compared to the baseline
- Lighter shading represents a smaller difference compared to the baseline; darker shading represents a larger difference compared to the baseline

To view the delta thresholds corresponding to each shade, hover the cursor over the column headers:



To change your baseline run, click the Runs button and select a new run as the baseline from the Baseline Run column, as shown in the following figure:



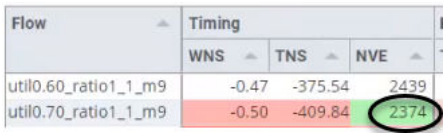
Feedback

See Also

- Changing the QoR Display Style
- Sorting and Filtering the Data
- Exploring the Detailed Comparison Data

Changing the QoR Display Style

By default, the comparison tables show the QoR values for all runs. For example, the NVE number for the following run is 2374:



Flow	Timing		
	WNS	TNS	NVE
util0.60_ratio1_1_m9	-0.47	-375.54	2439
util0.70_ratio1_1_m9	-0.50	-409.84	2374

To cycle through different display styles of the data, right-click anywhere in the table. You can view the data

- as a percentage delta against the baseline
- as an absolute delta against the baseline (shown in *italic*)

For example, as a percentage delta, the NVE number shows 2.7% fewer failing endpoints than the baseline:



Flow	Timing		
	WNS	TNS	NVE
util0.60_ratio1_1_m9	-0.47	-375.54	2439
util0.70_ratio1_1_m9	6.4%	9.1%	-2.7%

As an absolute delta, the NVE number shows 65 fewer failing endpoints than the baseline:



Flow	Timing		
	WNS	TNS	NVE
util0.60_ratio1_1_m9	-0.47	-375.54	2439
util0.70_ratio1_1_m9	*-0.03*	*-34.30*	*-65*

See Also

- Sorting and Filtering the Data
- Exploring the Detailed Comparison Data

Sorting and Filtering the Data

You can sort and filter the run data to reveal patterns in the results and determine the parameters you want to explore further.

To sort and filter the data, see the following topics:

- Sorting the Data
- Filtering Metrics
- Filtering Runs
- Example Analysis

See Also

- Exploring the Detailed Comparison Data

Feedback

Sorting the Data

Click any column header to sort the data using that metric. The first click performs an ascending sort, while the second click performs a descending sort.

For example, to show the worst TNS numbers at the top of the table and the best TNS numbers at the bottom, click the TNS column header:



Timing		
WNS	TNS	NVE
-0.76	-1106.58	7227
-0.76	-1066.94	6386
-0.76	-1057.45	6500
-0.76	-1018.50	6402
-0.76	-999.51	6459

Click the TNS column header again to show the best TNS numbers at the top and the worst at the bottom:



Timing		
WNS	TNS	NVE
-0.44	-328.70	2222
-0.44	-338.62	2405
-0.38	-347.69	2233
-0.40	-352.97	2209
-0.44	-354.87	2175
-0.47	-368.99	2548
-0.48	-371.02	2240
-0.47	-371.04	2593
-0.46	-378.48	2705
-0.49	-391.04	2610
-0.48	-407.98	2659
-0.45	-419.23	2655
-0.46	-420.94	2758
-0.51	-428.26	2719

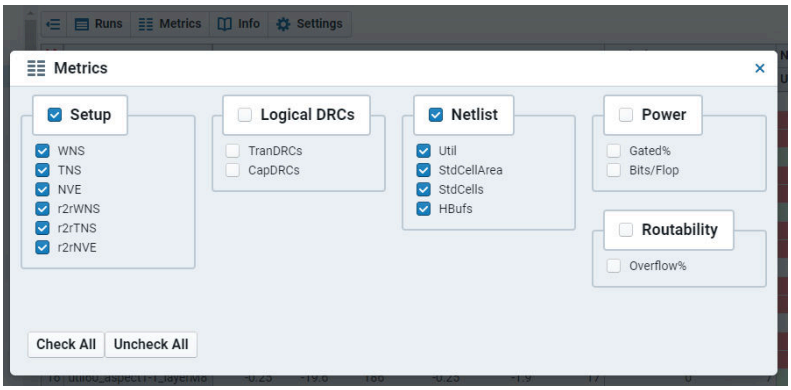
Feedback

Filtering Metrics

To control which metrics are displayed in the table, click the Metrics button and select or deselect the metrics from the Metrics dialog box accordingly.

For example, to show only the setup timing, netlist area, and cell counts, select the metrics as shown in Figure 28.

Figure 28 Metrics Dialog Box

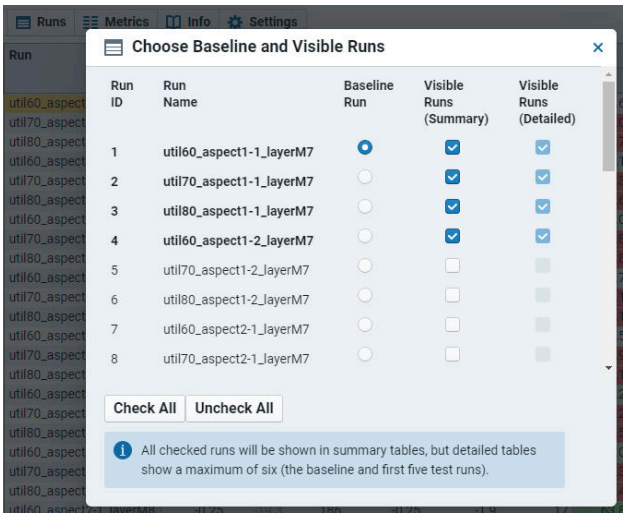


Filtering Runs

To control which runs are displayed in the table, click the Runs button and select or deselect the exploration runs accordingly from the Visible Runs (Summary) column.

For example, to display only the first four runs in a table, select the runs as shown in Figure 29.

Figure 29 Choose Baseline and Visible Runs Dialog Box



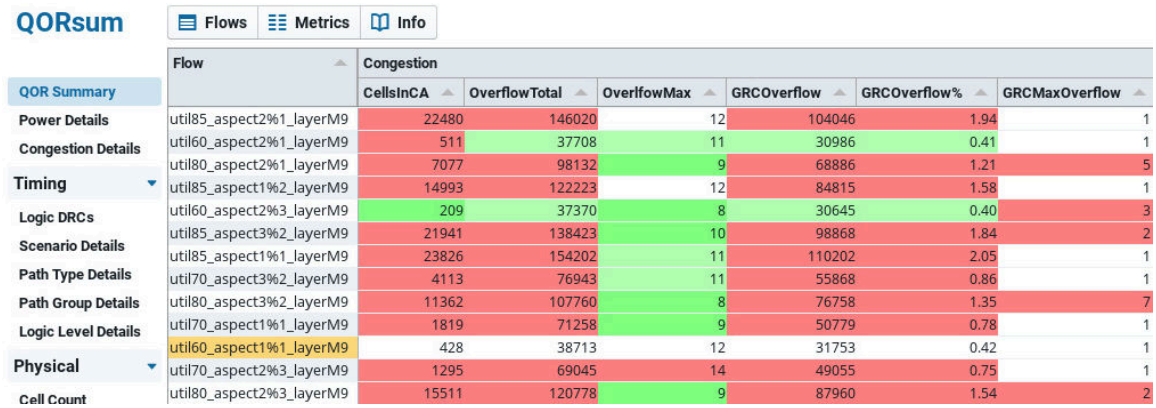
Feedback

Example Analysis

The following example demonstrates how you might sort and filter your data to narrow down the runs you would like to explore further.

Suppose you open the comparison report shown in Figure 30 and set the util60_aspect1%1_layerM9 run as the base run.

Figure 30 Comparison Report



Flow	Congestion					
	CellsInCA	OverflowTotal	OverIfowMax	GRCOverflow	GRCOverflow%	GRCMaxOverflow
util85_aspect2%1_layerM9	22480	146020	12	104046	1.94	1
util60_aspect2%1_layerM9	511	37708	11	30986	0.41	1
util80_aspect2%1_layerM9	7077	98132	9	68886	1.21	5
util85_aspect1%2_layerM9	14993	122223	12	84815	1.58	1
util60_aspect2%3_layerM9	209	37370	8	30645	0.40	3
util85_aspect3%2_layerM9	21941	138423	10	98868	1.84	2
util85_aspect1%1_layerM9	23826	154202	11	110202	2.05	1
util70_aspect3%2_layerM9	4113	76943	11	55868	0.86	1
util80_aspect3%2_layerM9	11362	107760	8	76758	1.35	7
util70_aspect1%1_layerM9	1819	71258	9	50779	0.78	1
util60_aspect1%1_layerM9	428	38713	12	31753	0.42	1
util70_aspect2%3_layerM9	1295	69045	14	49055	0.75	1
util80_aspect2%3_layerM9	15511	120778	9	87960	1.54	2

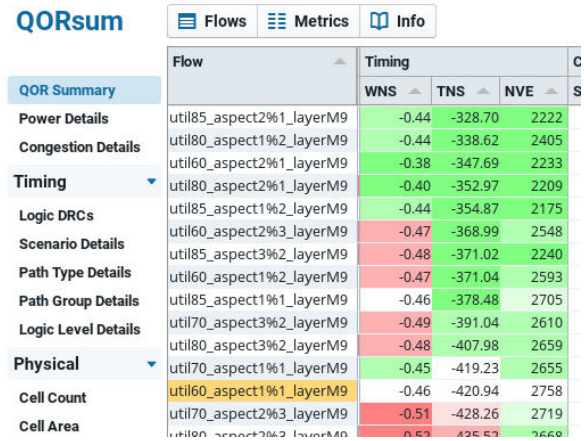
You could look at your TNS numbers first and sort the data from best TNS to worst TNS, as shown in the following figure:

Flow	Timing		
	WNS	TNS	NVE
util85_aspect2%1_layerM9	-0.44	-328.00	2222
util60_aspect2%1_layerM9	-0.38	-347.69	2233
util80_aspect2%1_layerM9	-0.40	-352.97	2209
util85_aspect1%2_layerM9	-0.44	-354.87	2175
util60_aspect2%3_layerM9	-0.47	-368.99	2548
util85_aspect3%2_layerM9	-0.48	-371.02	2240
util85_aspect1%1_layerM9	-0.46	-378.48	2705
util70_aspect3%2_layerM9	-0.49	-391.04	2610
util80_aspect3%2_layerM9	-0.48	-407.98	2659
util70_aspect1%1_layerM9	-0.45	-419.23	2655
util60_aspect1%1_layerM9	-0.46	-420.94	2758
util70_aspect2%3_layerM9	-0.51	-428.26	2719
util80_aspect2%3_layerM9	-0.52	-435.52	2668
util85_aspect2%3_layerM9	-0.51	-445.29	2879
util60_aspect3%2_layerM9	-0.52	-458.61	2985
util70_aspect2%1_layerM9	-0.47	-458.86	3146
util80_aspect1%1_layerM9	-0.62	-514.59	3243

Notice that the best TNS runs have the M9 top layer, and the worst have the M7 top layer. This suggests that restricting the metal layers significantly impacts timing.

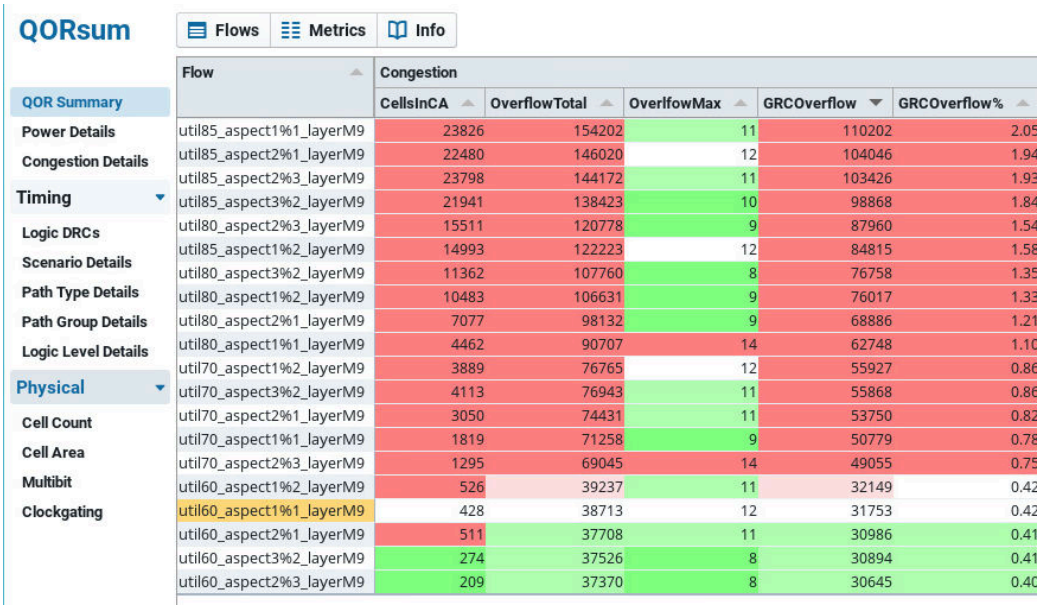
Feedback

You could then restrict your analysis to your M9 runs by turning off the visibility of your M7 runs, as shown in the following figure:



Flow	Timing		
	WNS	TNS	NVE
util85_aspect2%1_layerM9	-0.44	-328.70	2222
util80_aspect1%2_layerM9	-0.44	-338.62	2405
util60_aspect2%1_layerM9	-0.38	-347.69	2233
util80_aspect2%1_layerM9	-0.40	-352.97	2209
util85_aspect1%2_layerM9	-0.44	-354.87	2175
util60_aspect2%3_layerM9	-0.47	-368.99	2548
util85_aspect3%2_layerM9	-0.48	-371.02	2240
util60_aspect1%2_layerM9	-0.47	-371.04	2593
util85_aspect1%1_layerM9	-0.46	-378.48	2705
util70_aspect3%2_layerM9	-0.49	-391.04	2610
util80_aspect3%2_layerM9	-0.48	-407.98	2659
util70_aspect1%1_layerM9	-0.45	-419.23	2655
util60_aspect1%1_layerM9	-0.46	-420.94	2758
util70_aspect2%3_layerM9	-0.51	-428.26	2719

Now that you have your best TNS runs, you could compare their congestion by sorting the GRCOverflow column to show the worst overflow at the top, as shown in the following figure:



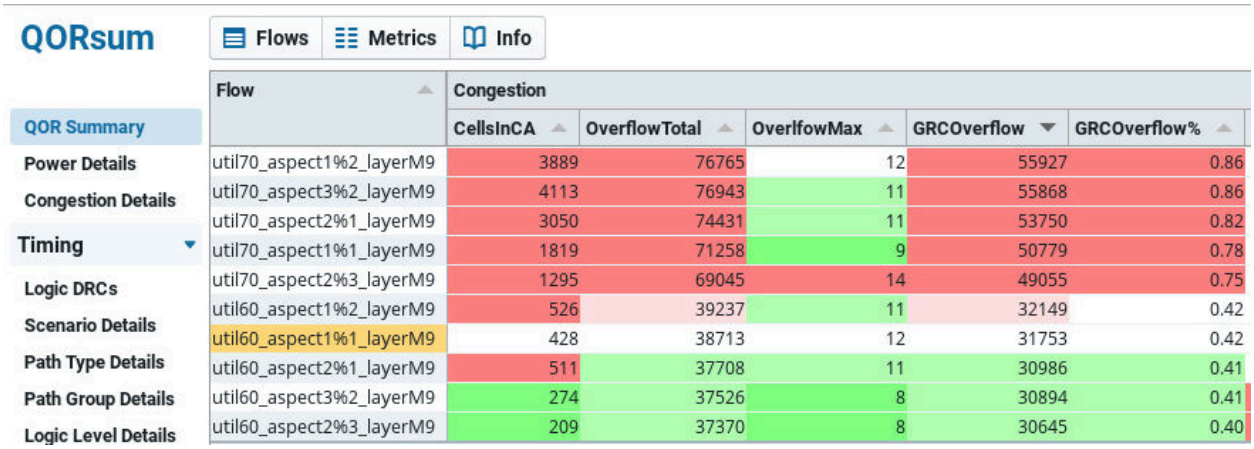
Flow	Congestion				
	CellsInCA	OverflowTotal	OverIfowMax	GRCOverflow	GRCOverflow%
util85_aspect1%1_layerM9	23826	154202	11	110202	2.05
util85_aspect2%1_layerM9	22480	146020	12	104046	1.94
util85_aspect2%3_layerM9	23798	144172	11	103426	1.93
util85_aspect3%2_layerM9	21941	138423	10	98868	1.84
util80_aspect2%3_layerM9	15511	120778	9	87960	1.54
util85_aspect1%2_layerM9	14993	122223	12	84815	1.58
util80_aspect3%2_layerM9	11362	107760	8	76758	1.35
util80_aspect1%2_layerM9	10483	106631	9	76017	1.33
util80_aspect2%1_layerM9	7077	98132	9	68886	1.21
util80_aspect1%1_layerM9	4462	90707	14	62748	1.10
util70_aspect1%2_layerM9	3889	76765	12	55927	0.86
util70_aspect3%2_layerM9	4113	76943	11	55868	0.86
util70_aspect2%1_layerM9	3050	74431	11	53750	0.82
util70_aspect1%1_layerM9	1819	71258	9	50779	0.78
util70_aspect2%3_layerM9	1295	69045	14	49055	0.75
util60_aspect1%2_layerM9	526	39237	11	32149	0.42
util60_aspect1%1_layerM9	428	38713	12	31753	0.42
util60_aspect2%1_layerM9	511	37708	11	30986	0.41
util60_aspect3%2_layerM9	274	37526	8	30894	0.41
util60_aspect2%3_layerM9	209	37370	8	30645	0.40

Notice that your higher-utilization runs have more congestion than your lower-utilization runs. You could restrict your analysis to your lower-utilization runs by turning off the

Feedback

visibility of your higher-utilization runs as shown in Figure 31, leaving you with a manageable subset of exploration runs that better meet your timing and congestion goals.

Figure 31 Displaying Lower-Utilization Runs



Flow	Congestion				
	CellsInCA	OverflowTotal	OverlfowMax	GRCOverflow	GRCOverflow%
util70_aspect1%2_layerM9	3889	76765	12	55927	0.86
util70_aspect3%2_layerM9	4113	76943	11	55868	0.86
util70_aspect2%1_layerM9	3050	74431	11	53750	0.82
util70_aspect1%1_layerM9	1819	71258	9	50779	0.78
util70_aspect2%3_layerM9	1295	69045	14	49055	0.75
util60_aspect1%2_layerM9	526	39237	11	32149	0.42
util60_aspect1%1_layerM9	428	38713	12	31753	0.42
util60_aspect2%1_layerM9	511	37708	11	30986	0.41
util60_aspect3%2_layerM9	274	37526	8	30894	0.41
util60_aspect2%3_layerM9	209	37370	8	30645	0.40

Exploring the Detailed Comparison Data

When you launch the QORsum report, the application opens the QOR Summary table, which summarizes high-level timing, power, and congestion metrics for each of your runs. This and other summary views help you sort and filter the data to pinpoint the runs you want to explore further.

Filtering the Detailed Data

Detailed views offer filters to focus on specific data. Some views have default filters that are shown automatically. For example, the following detailed views have default scenario and path group filters: Path Type Details, Path Group Details, and Logic Level Details.

You can modify the default filters by

- Removing the filter by clicking the X symbol before the filter name



- Changing the filter value



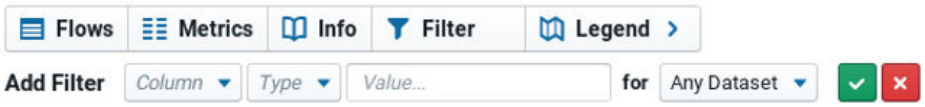
- Enable and disable the filter by clicking anywhere on the filter but the X symbol or value field



Feedback

You can also apply custom filters to the detailed data. To create a custom filter,

1. Click the Filter button to display the Add Filter fields, which are shown in the following figure:



2. Define the filter by defining the filter criteria and selecting the datasets to include in the results.

   To define the filter, select the column and comparison type, and then specify the comparison value.

   - To perform a numeric comparison, select one of the following comparison types: `=`, `!=`, `<`, `<=`, `>`, or `>=` and specify a numeric value in the Value field.
   - To perform a string comparison, select the `contains` comparison type and specify a string value in the Value field.
   - To perform a regular expression comparison, select the `regexp` comparison type and specify a regular expression in the Value field.
   - To filter based on the available values of the selected column, select the `enum` comparison type, which populates the Value field with a drop-down menu that contains the available values, and then enable one or more of the displayed values. When a value is enabled, a check mark is displayed before the value.

     To enable or disable a value, highlight the value by clicking it or navigating to it by using the Up and Down arrows, and then press Enter, which toggles the value status. You can also type a string in the Value field to filter the available values in the drop-down menu. To dismiss selected values, click the X symbol.

3. Apply the filter by clicking the green check mark.

4

Clock Tree Synthesis

To learn how to perform clock tree synthesis in the IC Compiler II tool, see the following topics:

- Prerequisites for Clock Tree Synthesis
- Defining the Clock Trees
- Verifying the Clock Trees
- Setting Clock Tree Design Rule Constraints
- Specifying the Clock Tree Synthesis Settings
- Implementing Clock Trees and Performing Post-CTS Optimization
- Implementing Multisource Clock Trees
- Analyzing the Clock Tree Results

Prerequisites for Clock Tree Synthesis

This topic details the prerequisites required before running the clock tree synthesis.

Before you run clock tree synthesis on a block, it should meet the following requirements:

- The clock sources are identified with the `create_clock` or `create_generated_clock` commands.
- The block is placed and optimized.

  Use the `check_legality -verbose` command to verify that the placement is legal. Running clock tree synthesis on a block that does not have a legal placement might result in long runtimes and reduced QoR.

  The estimated QoR for the block should meet your requirements before you start clock tree synthesis. This includes acceptable results for

  - Congestion

    If congestion issues are not resolved before clock tree synthesis, the addition of clock trees can increase congestion. If the block is congested, you can rerun the

Feedback

`create_placement` command with the `-congestion` and `-congestion_effort high` options, but the runtime can be long.

  - Timing
  - Maximum capacitance
  - Maximum transition time

- The power and ground nets are prerouted.
- High-fanout nets, such as scan enables, are synthesized with buffers.
- The active scenarios are defined.

  By default, the IC Compiler II tool synthesizes and optimizes all clocks in all active scenarios that are enabled for setup or hold analysis.

Defining the Clock Trees

This section details the information required to analyze and define each clock tree before running clock tree synthesis.

Before you run clock tree synthesis, analyze each clock tree in the block and determine

- What the clock root is
- What the required clock sinks and clock tree exceptions are
- Whether the clock tree contains preexisting cells, such as clock-gating cells
- Whether the clock tree converges, either with itself (a convergent clock path) or with another clock tree (an overlapping clock path)
- Whether the clock tree has timing relationships with other clock trees in the block, such as interclock skew requirements

Use this information to define the clock trees and validate that the tool has the correct clock tree definitions.

The IC Compiler II tool derives the clock trees based on the clocks defined in the block. If the derived clock trees do not meet your requirements, you can define clock trees as described in the following topics:

- Deriving the Clock Trees
- Defining Clock Tree Exceptions
- Restricting Optimization on the Clock Network
- Copying Clock Tree Exceptions Across Modes

Feedback

- Deriving Clock Tree Exceptions From Ideal Clock Latencies
- Handling Endpoints With Balancing Conflicts
- Enabling Solver-Augmented Skew Optimization

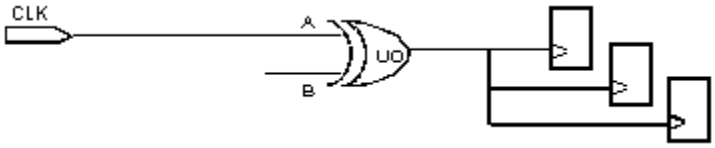
Deriving the Clock Trees

The IC Compiler II tool derives the clock trees by tracing through the transitive fanout from the clock roots to the clock endpoints. In general, the tracing terminates when it finds a clock pin of a sequential cell or macro; however, the tool traces through sequential cells if they are integrated clock-gating (ICG) cells or their fanout drives a generated clock.

If the clock-gating logic uses a non-unate cell, such as an XOR or XNOR gate, the tool uses both the positive-unate timing arc and the negative-unate timing arc when tracing the clock path. If this does not correspond to the functional mode of the block, use the `set_case_analysis` command to hold all nonclock inputs of the cell at a constant value, which forces the cell into the required functional mode for clock tree synthesis.

For example, suppose a block has the gating logic shown in Figure 32.

Figure 32 Non-Unate Gated Clock



To force the XOR gate (U0) into functional mode for timing analysis, use the following command:

```
icc2_shell> set_case_analysis 0 U0/B
```

Identifying the Clock Roots

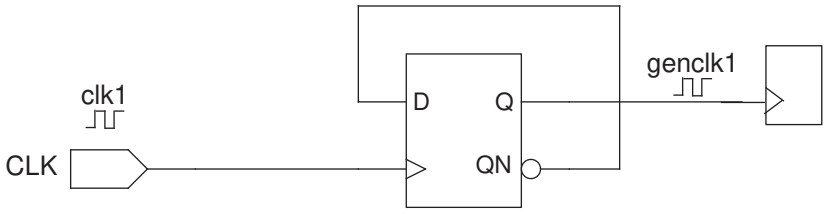
The IC Compiler II tool uses the clock sources defined by the `create_clock` command, which can be either input ports or internal hierarchical pins, as the clock roots.

For nested clock trees with generated clocks, which are defined by the `create_generated_clock` command, the tool considers the master-clock source to be the clock root, and the clock endpoints of the nested clock tree are considered endpoints of the master-clock source.

For example, for the netlist shown in Figure 33, the tool considers the CLK port to be the clock root for the genclk1 generated clock.

Feedback

Figure 33 Nested Clock Tree With a Generated Clock



If the block contains generated clocks, ensure that the master-clock sources are correctly defined, as incorrect definitions can result in poor skew and timing QoR. In particular,

- If the tool cannot trace back to the master-clock source, it cannot balance the sink pins of the generated clock with the sink pins of its source.
- If the master-clock source is not a clock source defined by the `create_clock` or `create_generated_clock` command, the tool cannot synthesize a clock tree for the generated clock or its source.

Use the `check_clock_trees` command to verify that your master-clock sources are correctly defined, as described in Verifying the Clock Trees.

Specifying the Clock Root Timing Characteristics

To get realistic clock tree synthesis results, you must ensure that the timing characteristics of the clock roots are correctly modeled.

- If the clock root is an input port without an I/O pad cell, you must accurately specify the driving cell of the input port.

  If you specify a weak driving cell, the tool might insert extra buffers to try to meet the clock tree design rule constraints, such as maximum transition time and maximum capacitance.

  If you do not specify a driving cell (or drive strength), the tool assumes that the port has infinite drive strength.

  For example, if the CLK1 port is the root of the CLK1 clock tree, use the following command to set its driving cell as the CLKBUF cell in the mylib cell library:

  ```
  icc2_shell> set_driving_cell -lib_cell mylib/CLKBUF [get_ports CLK1]
  ```

- If the clock root is an input port with an I/O pad cell, you must accurately specify the input transition time of the input port.

Feedback

For example, if the CLK1 port is the root of the CLK1 clock tree and the I/O pad cell has already been inserted, use the following commands to set its input transition time to 0.3 for rising delays and 0.2 for falling delays:

```
icc2_shell> set_input_transition -rise 0.3 [get_ports CLK1]
icc2_shell> set_input_transition -fall 0.2 [get_ports CLK1]
```

Identifying the Clock Endpoints

When expanding the clock trees, the tool identifies two types of clock nodes:

- Sink pins

  Sink pins are the clock endpoints that are used for delay balancing. The tool assumes an insertion delay of zero to all sink pins and uses this delay during delay balancing.

  During clock tree synthesis, the tool uses the clock network to the sink pins in calculations and optimizations for both design rule constraints and clock tree timing (skew and insertion delay).

  Sink pins are also referred to as balancing pins.

- Ignore pins

  Ignore pins are clock nodes that are excluded from clock tree timing calculations and optimizations for skew and insertion delay. The tool uses ignore pins only in calculations and optimizations for design rule constraints.

  During clock tree synthesis, the tool isolates ignore pins from the clock tree by inserting a guide buffer before the ignore pin. Beyond the ignore pin, the tool never performs skew or insertion delay optimization, but does perform design rule fixing.

The tool identifies the following clock endpoints as sink pins:

- A clock pin on a sequential cell (a latch or flip-flop), unless that cell drives a generated clock
- A clock pin on a macro cell

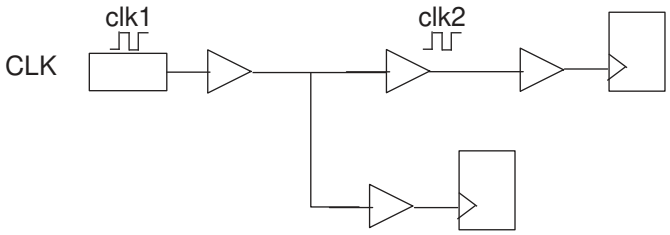
Feedback

The tool identifies the following clock endpoints as ignore pins:

- Source pins of clock trees in the fanout of another clock

  For example, in Figure 34, the source pin of the driven clock (clk2) is an ignore pin of the driving clock (clk1). Sinks of the driven clock are not considered sinks of the driving clock.

Figure 34 Cascaded Clock With Two Source Clocks



- Nonclock input pins of sequential cells
- Three-state enable pins
- Output ports
- Incorrectly defined clock pins (for example, the clock pin does not have trigger edge information or does not have a timing arc to the output pin)
- Buffer or inverter input pins that are held constant by using the `set_case_analysis` command

  Note:

  The tool does not synthesize a clock tree if its source is held constant by using the `set_case_analysis` command.

- Input pins of combinational cells or integrated clock-gating cells that do not have any fanout or that do not have any enabled timing arcs

To verify that the tool has correctly identified the sink pins and ignore pins, examine the clock trees in the GUI, as described in Analyzing Clock Trees in the GUI.

If the default sink and ignore pins are correct, you are done with the clock tree definition. Otherwise, first identify any timing settings, such as disabled timing arcs and case analysis settings, that affect the clock tree traversal.

Feedback

- To identify disabled timing arcs in the block, use the `report_disable_timing` command
- To identify case analysis settings in the block, use the `report_case_analysis` command

Note:
Remove any timing settings that cause an incorrect clock tree definition.

If necessary, you can override the default balancing and ignore pin settings by using the `set_clock_balance_points` command, as described in Defining Clock Tree Exceptions.

The implicit ignore on an intermediate pin can be inferred by any of the following reasons:

- All the register pins in the fanout are EXPLICIT IGNORE
- Disabled path on a MUX input is caused by the `set_case_analysis` command
- Disabled timing arc on a downstream pin is set by the `set_disable_timing`, `set_false_path` commands, disable timing due to conditional arcs.
- `set_sense -stop_propogation` constraint on a downstream pin
- Dangling pin in the downstream path
- Downstream path goes to a data pin of a register or a subblock
- Downstream path has an unconstrained output port without any `set_clock_balance_point -consider_for_balancing true` setting
- Downstream path has a constrained `set_output_delay` output port
- Pins on the path to cascaded `create_clock` definition

Note:

This implicit or explicit ignore does not essentially mean that the clock endpoints are completely excluded from clock tree calculations. Explicit ignore means, the endpoints are not considered for skew balancing with other endpoints of the clock, but the portion is considered for DRC fixing in clock tree.

Feedback

Defining Clock Tree Exceptions

Clock tree exceptions are user-defined changes to the default endpoints (balancing and ignore pins) derived by the tool for a specific clock.

The IC Compiler II tool supports the following types of clock tree exceptions:

- User-defined sink pins

  These exceptions define sink pins in addition to those derived by the tool for a clock tree. For example, you might define a tool-derived ignore pin as a clock sink.

  For information about defining sink pins, see Defining Sink Pins.

- User-defined insertion delay requirements

  These exceptions specify special insertion delay requirements for a sink pin (either derived or user-specified).

  For information about defining insertion delay requirements, see Defining Insertion Delay Requirements.

- User-defined ignore pins

  These exceptions exclude clock endpoints that were derived as sink pins by the tool. For example, you might define an ignore pin to exclude all branches of the clock tree that fan out from some combinational logic or to exclude a tool-derived sink pin.

  For information about defining ignore pins, see Defining Ignore Pins.

Defining Sink Pins

To define one or more pins as sink pins, use the following syntax:

```
set_clock_balance_points
   -clock clock
   -consider_for_balancing true
   -balance_points pins
   [-corners corners]
```

The `-clock` option is not required. If you do not use it, the sink pin applies to all clocks.

By default, the insertion delay for the user-specified sink pins is 0. For information about overriding the default insertion delay, see Defining Insertion Delay Requirements.

When you define sink pins, by default, they apply to all corners. To apply the definition to specific corners, use the `-corners` option.

Feedback

For example, to specify pin U2/A as a sink pin for the CLK clock in the current corner, use the following command:

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
   -consider_for_balancing true -balance_points [get_pins U2/A]
```

To report the user-defined sink pins, use the `report_clock_balance_points` command. This command reports both the user-defined sink pins and the user-defined ignore pins.

To remove the sink pin definition from a pin, use the `remove_clock_balance_points` command.

For example, to remove the sink pin definition from the U2/A pin for the CLK clock, use the following command:

```
icc2_shell> remove_clock_balance_points -clock [get_clocks CLK] \
   -balance_points [get_pins U2/A]
```

Defining Insertion Delay Requirements

To override the default phase delay of zero for sink pins (derived or user-specified), use the `-delay` option with the `set_clock_balance_points` command to specify the insertion delay requirements. The tool adds the specified delay (positive or negative) to the calculated insertion delay up to the specified sink pin.

The syntax to define the insertion delay requirement for one or more sink pins is

```
set_clock_balance_points
   [-clock clock]
   -delay delay
   [-rise] [-fall] [-early] [-late]
   -balance_points pins
   [-corners corners]
```

The `-clock` option is not required. If you do not use it, the insertion delay requirement applies to all clocks.

Clock tree synthesis uses the specified delay value for the path delay calculations used to build the clock tree. By default, the specified delay value is used for both longest-path and shortest-path calculations for both rising-edge and falling-edge paths. To control when the specified delay value is used, use the `-rise`, `-fall`, `-early`, and `-late` options.

When you define insertion delay requirements, by default, they apply to all corners. To apply the definition to specific corners, use the `-corners` option.

For example, to specify that clock tree synthesis should use an insertion delay of 2.0 ns for rising-edge shortest-path calculations for the U2/CLK pin for the CLK clock in the current corner, use the following command:

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
   -rise -early -delay 2.0 -balance_points [get_pins U2/CLK]
```

Feedback

To report the user-defined insertion delay for sink pins, use the `report_clock_balance_points` command. This command reports the user-defined sink pins, the derived sink pins with user-specified insertion delay, and the user-defined ignore pins.

Defining Ignore Pins

To define one or more pins as ignore pins, use the following syntax:

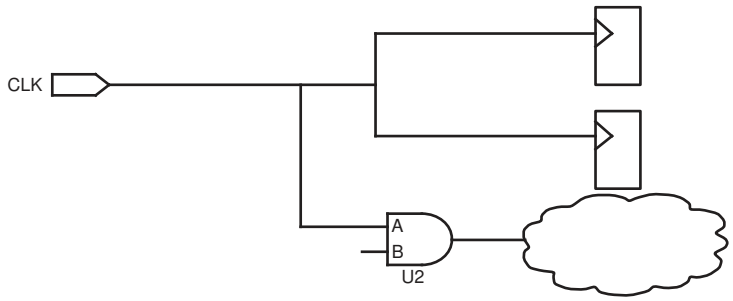
```
set_clock_balance_points
   [-clock clock]
   -consider_for_balancing false
   -balance_points pins
```

The `-clock` option is not required. If you do not use it, all user-defined ignore pins apply to all clocks.

For the CLK clock tree shown in Figure 35, assume that you want to ignore all branches of the clock tree beyond the U2 cell. To do so, you would use the following command:

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
   -consider_for_balancing false -balance_points [get_pins U2/A]
```

Figure 35 User-Defined Ignore Pin



To report the user-defined ignore pins, use the `report_clock_balance_points` command. This command reports both the user-defined sink pins and the user-defined ignore pins.

During clock tree synthesis, the tool adds guide buffers and isolates ignore pins from the rest of the clock network. During subsequent data path optimization, the tool fixes any existing DRC violations beyond the guide buffers.

To remove the ignore pin definition from a pin, use the `remove_clock_balance_points` command.

Feedback

For example, to remove the ignore pin definition from pin U2/CLK for the CLK clock, use the following command:

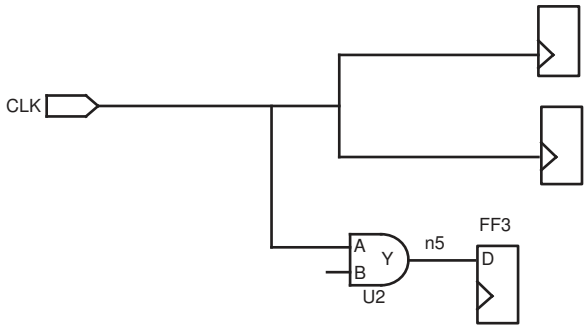
```
icc2_shell> remove_clock_balance_points -clock [get_clocks CLK] \
   -balance_points [get_pins U2/CLK]
```

Ensuring Clock Tree Exceptions are Valid

To identify pins on the clock network, the tool checks if the `is_clock_is_used_as_clock` pin attribute is `true` during clock tree synthesis. If you use the `set_clock_balance_points` command to specify a clock tree exception on a pin for which the `is_clock_is_used_as_clock` attribute is `false`, the tool accepts the exception, but ignores it during clock tree synthesis.

Consider the clock network in the following figure.

Figure 36 Specifying Exceptions on Pins Beyond the Clock Network



By default, the tool sets

- The `is_clock_is_used_as_clock` attribute of the U2/A, U2/Y, and FF3/D pins to `false` and considers the network beyond the U2/A pin as part of the data network
- The U2/A pin as an ignore pin and excludes the network beyond this from clock tree synthesis

During clock tree synthesis, the tool adds a guide buffer and isolates the U2/A pin from the rest of the clock network. During subsequent data path optimization, the tool can fix any existing DRC violations beyond the guide buffer.

Assume you want clock tree synthesis to fix DRC violations up to the FF3/D pin using clock tree constraints. To do so, you must

1. Specify the FF3/D pin as an explicit ignore pin as follows:

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
    -consider_for_balancing false -balance_points [get_pins FF3/D]
```

Feedback

2. Force clock tree synthesis to consider the FF3/D pin as a pin on the clock network by using the `set_sense -clock_leaf` command as follows:

   ```
   icc2_shell> set_sense -clock_leaf [get_pins FF3/D]
   ```

   The `set_sense -clock_leaf` command sets

   - The `is_clock_used_as_clock` attribute to `true` for all the pins of the clock branches that fanin to the pin specified as the clock leaf
   - The `is_clock_used_as_clock` attribute to `false` for all the pins in the fanout from the pin specified as the clock leaf, and the tool does not propagate the clock beyond this pin

   To check the value of the `is_clock_used_as_clock` pin attribute, use the `get_attribute` command, as shown in the following example:

   ```
   icc2_shell> get_attribute [get_pins FF3/D] is_clock_used_as_clock
   ```

Restricting Optimization on the Clock Network

You can restrict the optimization on portions of the clock network by specifying

- Don’t touch settings

  These settings identify parts of the clock tree on which clock tree synthesis and optimization is not performed. You can use don’t touch settings to prevent the modification of a clock network beyond a specific pin, the buffering of specific nets, or the sizing of specific cells.

  For information about defining don’t touch settings, see Setting Don’t Touch Settings.

- Size-only settings

  These settings identify clock tree cells for which the tool can perform only cell sizing. Note that these cells can be moved, unless they have a placement status of fixed.

  For information about defining size-only settings, see Setting Size-Only Settings.

Setting Don’t Touch Settings

To set a don't touch setting on a clock tree, use the `set_dont_touch_network` command.

```
icc2_shell> set_dont_touch_network -clock_only [get_pins pin_name]
```

The `-clock_only` option sets the don't touch setting only on the clock network. If you do not specify the `-clock_only` option, the don't touch setting is set on both the data path and the clock path.

Feedback

To remove the don't touch setting from a clock tree, use the `-clear` option with the `set_dont_touch_network` command.

```
icc2_shell> set_dont_touch_network -clock_only [get_pins pin_name] -clear
```

You can also set don't touch settings on specific objects in the clock network by using the `set_dont_touch` command.

The following commands set a don't touch setting on a cell and a net:

```
icc2_shell> set_dont_touch [get_cells cell_name] true
icc2_shell> set_dont_touch [get_nets -segments net_name] true
```

To report all the don't touch settings set on the current block, use the `report_dont_touch` command.

```
icc2_shell> report_dont_touch -all
```

Note:

If a cell has a placement status of fixed, it is treated like a don’t touch cell during clock tree synthesis.

Setting Size-Only Settings

To set a size-only setting on a cell, use the `set_size_only` command.

The following command sets a size-only setting on a cell:

```
icc2_shell> set_size_only [get_cells cell_name] true
```

To report all the size-only settings set on the current block, use the `report_size_only` command.

```
icc2_shell> report_size_only -all
```

Copying Clock Tree Exceptions Across Modes

You can specify that the tool copies clock tree exceptions from one mode to one or more equivalent modes by using the `set_clock_tree_options` command with the `-copy_exceptions_across_modes`, `-from_mode`, and `-to_mode` options.

The following example specifies that during clock tree synthesis the clock tree exceptions in the mode name M1 should be copied to the modes name M2 and M3:

```
icc2_shell> set_clock_tree_options -copy_exceptions_across_modes \
   -from_mode M1 -to_mode {M2 M3}
```

Feedback

Deriving Clock Tree Exceptions From Ideal Clock Latencies

If you have set ideal clock latencies for specific sinks in your design, you can use the `derive_clock_balance_points` command to convert these ideal latency settings to clock tree exceptions.

The `derive_clock_balance_points` command converts ideal latencies specified with the `set_clock_latency` command on clock sink pins to `set_clock_balance_points` commands. Use the `derive_clock_balance_points` command before clock tree synthesis, when the design has ideal clocks.

By default, the `derive_clock_balance_points` command

- Generates balance point constraints for all ideal primary clocks of all active scenarios that are enabled for setup analysis, hold analysis, or both. It does not generate balance points for generated clocks.

  To generate the balance points for

  - Specific primary clocks, use the `-clocks` option.
  - All active scenarios of specific corners, use the `-corners` option.

- Calculates a reference latency for each primary ideal clock, which is the sum of the ideal source and network latency specified on the clock with the `set_clock_latency` command.

  To specify a different reference latency value, use the `-reference_latency` option.

- Applies the derived clock balance point constraints to the design.

  To generate an output file containing `set_clock_balance_points` commands, instead applying it to the design, use the `-output` option.

Balance points that the tool derives are clock and corner specific. Ensure that appropriate `set_clock_latency` constraints are set for all clocks for all modes used for clock tree synthesis in at least the worst corner.

The tool uses the following formula to derive the delay values for the corresponding `set_clock_balance_points` commands:

(Clock balance point delay value at the sink) =

(Reference latency of the clock) - (Total ideal clock latency at the sink)

The total ideal clock latency at the sink is the sum of the source latency of the clock and the ideal network latency for the sink.

Feedback

For example, assume you have the following timing constraint settings:

```
icc2_shell> set_clock_latency -source 0.5 [get_clocks clk]
icc2_shell> set_clock_latency 1.0 [get_clocks clk]
icc2_shell> set_clock_latency 1.5 [get_pins reg1/CK]
icc2_shell> set_clock_latency 0.5 [get_pins reg2/CK]
```

If you run the `derive_clock_balance_points` command, the tool derives the following clock balance point constraints:

```
icc2_shell> set_clock_balance_points -delay -0.5 \
   -balance_points [get_pins reg1/CK] -clock clk -corners worst
icc2_shell> set_clock_balance_points -delay 0.5 \
   -balance_points [get_pins reg2/CK] -clock clk -corners worst
```

To avoid conflicts, do not manually apply any clock balance points for a clock that you derive balance points with the `derive_clock_balance_points` command.

Handling Endpoints With Balancing Conflicts

In some blocks, the clocks have endpoints that are structurally impossible to balance. The IC Compiler II tool can automatically detect endpoints with balancing conflicts and derive an ignore pin to resolve the conflict. This capability is enabled by default for the `synthesize_clock_trees` and `clock_opt` commands. To disable this capability, set the `cts.common.enable_auto_exceptions` application option to `false`.

Feedback

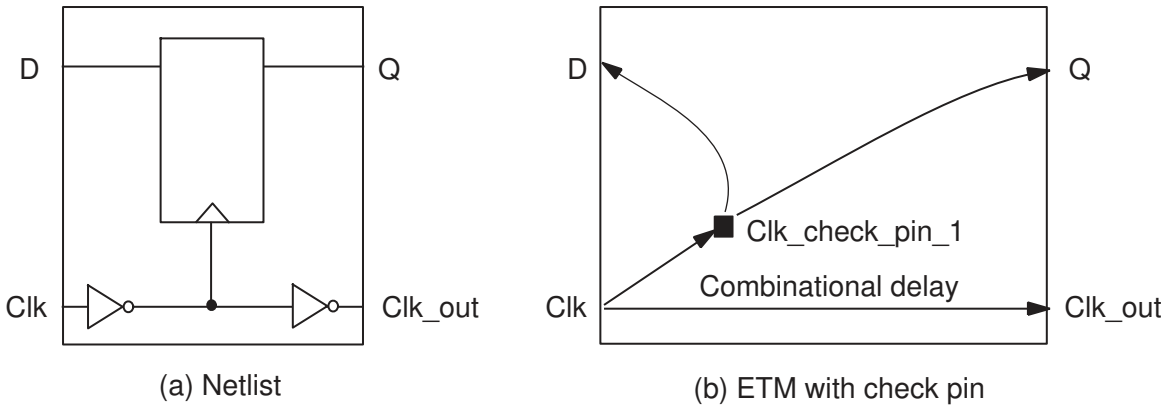
With this capability, the tool detects and fixes the following types of balancing conflicts:

- Internal sink pins of modules that have a combinational clock path

  If a module contains both internal sink pins and combinational clock paths, it is impossible to balance the internal sink pins. To resolve the balancing conflict, the tool defines the internal sink pins as ignore pins.

  For example, assume that you have a module that contains a combinational path in the clock network, as shown in figure (a) in Figure 37, which is represented by the extracted timing model (ETM) shown in figure (b).

Figure 37 Module With Combinational Clock Path



In this case, the internal check pin inside the ETM has delays coming from the ETM timing library; these delays are considered by clock tree synthesis as sinks and are balanced with other sinks at the top level. If the path to the check pin is the shortest path, top-level clock tree synthesis cannot insert buffers on the shortest path and therefore leaves a large skew at the top level. Setting the internal check pin as an ignore pin resolves the skew issue coming from the ETM.

- Sink pins that cannot be balanced simultaneously

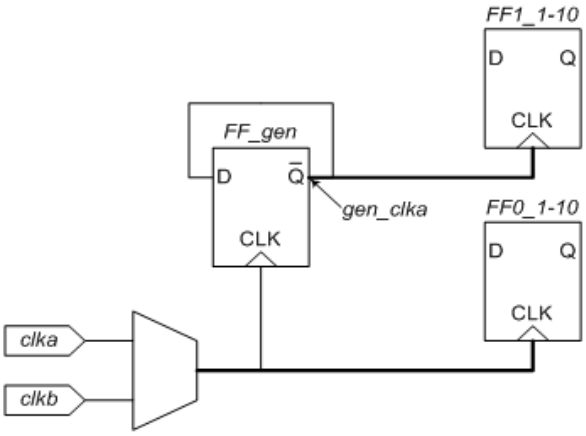
  If a block contains two clocks that drive common sinks, one of which is a clock pin that is the parent of another sink, due to a generated clock, it is impossible to simultaneously balance the parent and child clock pins with another common sink. To resolve the balancing conflict, the tool defines the parent clock pin as an ignore pin.

  For example, in Figure 38, FF_gen/CLK and FF0_1-10/CLK are sinks of clkb, FF1_1-10/CLK and FF0_1-10/CLK are sinks of clka, and FF_gen/CLK is the parent of

Feedback

FF1_1-10/CLK. In this case, the tool defines FF_gen/CLK as an ignore pin for clkb to resolve the balancing conflict.

Figure 38 Clock Pins That Cannot Be Balanced Simultaneously



Enabling Solver-Augmented Skew Optimization

Solver-augmented skew optimization in clock tree synthesis (CTS) improves the accuracy and efficiency of skew minimization for complex clock trees. The engine analyzes and abstracts the clock structures using a linear programming model. It then generates a solution with budgeted delays on the clock tree to guide skew balancing. This approach optimizes the skew of all paths, rather than focusing only on the longest and shortest paths.

The following are the benefits of solver-augmented skew optimization in clock tree synthesis:

- Enhances the ability to meet stringent skew requirements in complex designs.
- Provides better stability of skew and latency optimization.

To enable solver-augmented skew optimization, set the `cts.optimize.enable_solver_augmented_skew_optimization` application option to `true`.

```
icc2_shell> set_app_options -name
 cts.optimize.enable_solver_augmented_skew_optimization -value true
```

Feedback

Verifying the Clock Trees

Before you synthesize the clock trees, use the `check_clock_trees` command to verify that the clock trees are properly defined. For example, to verify that the CLK clock tree is properly defined, use the following command:

```
icc2_shell> check_clock_trees -clocks [get_clocks CLK]
```

If you do not specify the `-clock` option, the tool checks all clocks in the current block.

The `check_clock_trees` command checks for the following issues:

- Clock (master or generated) with no sinks
- Loops in the clock network
- Multiple clocks reach the same register because of overlapping clocks, but multiple-clocks-per-register propagation is not enabled
- Ignored clock tree exceptions
- Stop pin or float pin defined on an output pin
- Buffers with multiple timing arcs used in clock tree references
- Situations that cause an empty buffer list
- Generated clock without a valid master clock source

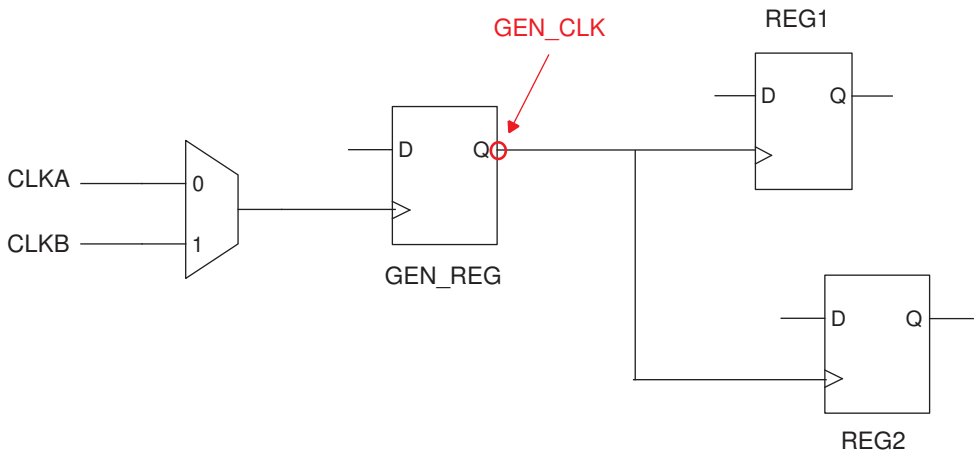
  A generated clock does not have a valid master-clock source in the following situations:

  - The master clock specified in `create_generated_clock` does not exist
  - The master clock specified in `create_generated_clock` does not drive the source pin of the generated clock
  - The source pin of the generated clock is driven by multiple clocks, and some of the master clocks are not specified with `create_generated_clock`.

Feedback

For example, in Figure 39, the GEN_REG/Q pin is driven by both CLKA and CLKB. If only CLKA is specified as a master clock in a `create_generated_clock` command, GEN_CLK does not have a valid master clock source.

Figure 39 Generated Clock With Invalid Master Clock Source



For multicorner-multimode designs, the `check_clock_trees` command checks all active scenarios for the following issues:

- Conflicting per-clock exception settings
- Conflicting balancing settings

Before you implement the clock trees, you should manually fix the reported issues. Each message generated by the `check_clock_trees` command has a detailed man page that describes how to fix the identified issue. You can improve the clock tree and timing QoR by fixing all the issues identified by the `check_clock_trees` command.

Feedback

Setting Clock Tree Design Rule Constraints

The IC Compiler II tool supports the following design rule constraints for clock tree synthesis:

- Maximum capacitance

  To specify maximum capacitance constraints for clock tree synthesis, use the `-clock_path` option with the `set_max_capacitance` command.

  If you do not specify this constraint, the clock tree synthesis default is 0.6 pF.

- Maximum transition time

  To specify maximum transition time constraints for clock tree synthesis, use the `-clock_path` option with the `set_max_transition` command.

  If you do not specify this constraint, the clock tree synthesis default is 0.5 ns.

By default, these constraints apply to all corners associated with the current mode. To set the constraint for a specific mode, use the `-mode` option with the `get_clocks` command to identify the clocks. To set the constraint for specific corners associated with the specified mode, use the `-corners` option with the constraint command. Be careful to apply the correct constraints across all the modes and their associated corners.

For example, to set a maximum transition time of 0.20 ns on all pins in the CLK clock path for all corners of the current mode, use the following command:

```
icc2_shell> set_max_transition 0.20 -clock_path [get_clocks CLK]
```

To set different maximum transition time constraints for different corners associated with a specific mode, use the `-mode` option with the `get_clocks` command to specify the mode and use the `-corners` option to specify the corner.

For example, to set the maximum transition time to 0.15 ns for corner1 in mode1 and 0.10 ns for corner2 in mode1, use the following commands:

```
icc2_shell> set_max_transition 0.15 -corners corner1 \
   -clock_path [get_clocks -mode mode1]
icc2_shell> set_max_transition 0.10 -corners corner2 \
   -clock_path [get_clocks -mode mode1]
```

To set the same maximum capacitance constraint for different corners associated with a specific mode, use the following command:

```
icc2_shell> set_max_capacitance 0.6 \
    -clock_path [get_clocks -mode mode2] \
   -corners [get_corner \
      [get_attribute [get_modes mode2] associated_corners]]
```

Specifying the Clock Tree Synthesis Settings

The clock tree synthesis options guide the implementation of the clock trees. The following topics describe how to set these options:

- Specifying the Clock Tree References
- Setting Skew and Latency Targets
- Enabling Local Skew Optimization
- Specifying the Primary Corner for Clock Tree Synthesis
- Preventing Specific Clocks From Being Synthesized
- Preserving Preexisting Clock Trees
- Enabling Clock Tree Power Reduction Techniques
- Enabling Wire Length Aware Power-Driven Relocation
- Reducing Electromigration
- Handling Inaccurate Constraints During Clock Tree Synthesis
- Defining Clock Cell Spacing Rules
- Creating Skew Groups
- Defining a Name Prefix for Clock Cells
- Using the Global Router During Initial Clock Tree Synthesis
- Specifying Constraints for Clock Nets
- Constructing Faster Clock Tree to Specific Endpoints
- Improving Latency-Driven Clock-Gate Splitting
- Improving the Repeater Levels in Clock Tree
- Enabling Latency-Driven Clock-Gate Collapsing Levels
- Reducing Signal Integrity Effects on Clock Nets
- Specifying Settings for Clock Latency Adjustments
- Timing Driven Sink Transition Target
- Reporting the Clock Tree Settings
- Pre-CTS Latency Bottleneck Reporting

Feedback

- Reporting Auto Exceptions and Summary Tables
- Dynamic Clock Power Improvement for Structural Multisource Clock Tree Synthesis
- Enabling Target Latency for Skew Groups and ICDB Improvement
- Debugging and Usability Improvement Using the check_clock_trees Command and CTS Log File
- Clock Routing Improvement Using Intelligent Minimum Layer Handling of Clock Nets

Specifying the Clock Tree References

The buffer and inverter cells that can be used to build a clock tree and the reference cells of the preexisting gates of the clock tree are referred to as *clock tree references*. To specify the clock tree references, use the `set_lib_cell_purpose -include cts` command.

Ensure that the list of cells you specify as clock tree references meets the following criteria:

- The list contains at least one buffer or one inverter
- The list contains the library cells of the preexisting gates

  If they are not included in the list, the tool is unable to resize these preexisting gates during clock tree synthesis and optimization. You can automatically derive equivalent cells for all the preexisting clock tree cells that are not buffers or inverters and specify them as clock references by using the `derive_clock_cell_references` command, as described in Deriving Clock Tree References for Preexisting Gates.
- For multivoltage designs with always-on buffering requirements, the list contains always-on cells
- The library cells in the reference list do not have a `dont_touch` attribute

  If library cells have the `dont_touch` attribute set on them, they are not used by clock tree synthesis even if you specify them as clock tree references.

To ensure that clock tree synthesis uses only the specified set of cells and that these cells are not used by any optimization step other than clock tree synthesis, run the following commands:

```
icc2_shell> set cts_cells list_of_cells
icc2_shell> set_lib_cell_purpose -exclude cts [get_lib_cells]
icc2_shell> set_lib_cell_purpose -include none [get_lib_cells $cts_cells]
icc2_shell> set_lib_cell_purpose -include cts [get_lib_cells $cts_cells]
```

Feedback

Deriving Clock Tree References for Preexisting Gates

To ensure that the tool resize preexisting gates in the clock network during clock tree synthesis and optimization, you must specify equivalent cells as clock references. Not specifying a complete list of equivalent cells for preexisting gates can affect clock tree QoR.

You can automatically derive equivalent cells for all the preexisting clock tree cells and specify them as clock references by using the `derive_clock_cell_references` command.

When you run this command, the tool sets the `valid_purpose` attribute to `true` on equivalent library cells of the preexisting clock tree cells that are not buffers or inverters. This command does not derive equivalent library cells for preexisting buffers and inverters, Therefore, you must manually specify the buffers and inverters to use for clock tree synthesis, as shown in the following example:

```
icc2_shell> set_lib_cell_purpose -include cts \
   {tech_lib/clk_buf* tech_lib/clk_inv*}
icc2_shell> derive_clock_cell_references
icc2_shell> synthesize_clock_trees
```

Instead of automatically specifying the equivalent library cells of the preexisting clock tree cells as clock tree references, you can generate a Tcl script that specifies the equivalent library cells of the preexisting clock tree cells as clock tree references. To do so, use the `-output` option.

```
icc2_shell> derive_clock_cell_references -output cts_leq_cells.tcl
```

You can edit the output file, source it, and then run clock tree synthesis, as shown in the following example:

```
icc2_shell> set_lib_cell_purpose -include cts \
   {tech_lib/clk_buf* tech_lib/clk_inv*}
icc2_shell> source cts_leq_cells.tcl
icc2_shell> synthesize_clock_trees
```

Restricting the Target Libraries Used

You can restrict the libraries used during clock tree synthesis for the top level or a lower level of the logical hierarchy of a design by using the `set_target_library_subset -clock` command. To enable the use of the target library subset, you must set the `opt.common.enable_target_library_subset_opt` application option to `1`.

The following example specifies the buf1 and buf2 cells from the HVT_lib and LVT_lib libraries as clock tree references. However, it restricts the lower-level block named TSK_BLK to use only the cells from the LVT_lib library for its clock nets. Therefore, only buf1 and buf2 cells from the LVT_lib library are used as clock references for that block.

Feedback

```
icc2_shell> set_lib_cell_purpose -include cts \
   {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
icc2_shell> set_target_library_subset -clock {LVT_lib} \
       -objects [TOP/TSK_BLK]
icc2_shell> set_app_options \
  -name opt.common.enable_target_library_subset_opt -value 1
```

Setting Skew and Latency Targets

By default, clock tree synthesis tries to achieve the best skew and latency for all clocks. However, this can lead to area, power, and runtime overhead for low frequency clocks, which have relaxed skew and latency targets.

- To specify a skew target, use the `-target_skew` option with the `set_clock_tree_options` command.
- To specify a latency target, use the `-target_latency` option with the `set_clock_tree_options` command.

By default, when you define skew and latency targets, they apply to all clocks in all corners. To define targets for specific clocks, use the `-clocks` option. To define targets for specific corners, use the `-corners` option.

To report the user-defined skew and latency targets, use the `report_clock_tree_options` command.

To remove user-defined skew or latency targets, use the `remove_clock_tree_options` command. To remove all skew and latency targets, use the `-all` option; otherwise, use the appropriate `-target_skew`, `-target_latency`, `-clocks`, and `-corners` options to remove the specific targets.

Enabling Local Skew Optimization

During clock tree synthesis, by default, the tool tries to minimize the global skew, which is the difference between the longest and shortest clock paths. If there is no timing path between the registers with the longest and shortest clock paths, optimizing global skew does not improve the timing QoR of the design. However, by optimizing the local skew, which is the worst skew between launch and capture registers of timing paths, you can improve the timing QoR of the design. During local skew optimization, the tool works on improving local skew of all violating setup and hold timing paths.

To enable local skew optimization during the clock tree synthesis and clock tree optimization stages of the `synthesize_clock_trees` and `clock_opt` commands, set the `cts.compile.enable_local_skew` and `cts.optimize.enable_local_skew` application options to `true`.

Feedback

```
icc2_shell> set_app_options -list {cts.compile.enable_local_skew true}
icc2_shell> set_app_options -list {cts.optimize.enable_local_skew true}
```

When you enable local skew optimization using the previous settings, by default, the tool derives skew targets that help improve the timing QoR, and ignores the target skew you specify with the `set_clock_tree_options -target_skew` command. To prevent the tool from deriving skew targets, use the following application option setting:

```
icc2_shell> set_app_options \
 -name cts.common.enable_auto_skew_target_for_local_skew -value false
```

Specifying the Primary Corner for Clock Tree Synthesis

During initial clock tree synthesis, by default, the tool identifies the corner with the worst clock delays and inserts buffers to balance the clock delays in all modes of this corner. To identify a specific corner as the primary corner for initial clock tree synthesis, use the `cts.compile.primary_corner` application option.

Preventing Specific Clocks From Being Synthesized

By default, the tool synthesizes all clocks defined with the `create_clock` and `create_generated_clock` commands. To prevent the tool from synthesizing a specific clock, specify it by using the `cts.common.skip_cts_clocks` application option.

```
icc2_shell> set_app_options \
   -name cts.common.skip_cts_clocks -value CK_B
```

Preserving Preexisting Clock Trees

When you perform clock tree synthesis, by default, the tool removes all preexisting clock buffers and inverters. You can prevent this by setting the `cts.compile.remove_existing_clock_trees` application option to `false`.

To preserve specific preexisting clock buffers or inverters, apply a don’t touch or size-only exception on them.

Enabling Clock Tree Power Reduction Techniques

You can enable clock tree power reduction techniques by using the `cts.compile.power_opt_mode` application option as shown in the following table.

Feedback

Table 19 Settings for the cts.compile.power_opt_mode Application Option

To do this	Use this setting
Relocate clock-gating cells closer to their drivers to reduce the length of input nets of the clock-gating cells, which have high switching activity	`gate_relocation`
Use clustering techniques to improve power	`low_power_targets`
Use both clock-gate relocation and clustering techniques to improve power	`all`

Enabling Wire Length Aware Power-Driven Relocation

The IC Compiler II tool supports improved activity driven clock gate relocation by controlling the amount of wire length degradation and considering the power impact before performing the clock gate relocation. To enable activity driven clock gate relocation, set the `cts.compile.power_opt_mode` application option to `all` or `gate_relocation`.

The clock gates are not relocated or moved if:

- Relocating an integrated clock gate (ICG) has a minor power benefit or degrades power while degrading wire length.
- Moving the clock gate has a significant wire length impact for a less power benefit.

Reducing Electromigration

Clock cells consume more power than cells that are not in the clock network. Clock cells that are clustered together in a small area increase the current densities for the power and ground rails, which increases the potential for electromigration problems. One way to avoid the problem is to set spacing requirements between clock cells to prevent local clumping of the clock cells along a standard cell power rail between the perpendicular straps.

To use this method to reduce electromigration in the block,

1. Define clock cell spacing rules for the inverters, buffers, and integrated clock-gating cells in the clock network by using the `set_clock_cell_spacing` command, as described in Defining Clock Cell Spacing Rules.
2. Perform clock tree synthesis by using the `synthesize_clock_trees` command, as described in Performing Standalone Clock Trees Synthesis, or the `clock_opt` command, as described in Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command.

Feedback

Note:

The clock cell spacing rules are also honored by the `balance_clock_groups` command. For information about using this command, see Balancing Skew Between Different Clock Trees.

3. Check clock cell spacing rule violations by using the `check_legality -verbose` command.

   You should not see any violations if you set the appropriate clock cell spacing constraints.

Handling Inaccurate Constraints During Clock Tree Synthesis

In the early stages of the implementation of a design, the constraints can be inaccurate or too restrictive. Running clock tree synthesis on such a design can lead to long runtime or poor clock tree QoR.

You can run clock tree synthesis on such a design, but still get usable results in a reasonable runtime. To do so, set the following application variable before you begin clock tree synthesis:

```
icc2_shell> set_app_options \
   -name cts.common.enable_dirty_design_mode -value true
```

When you use this application option setting, the tool

- Ignores `dont_touch` and `dont_touch_network` attribute settings on clock nets.
- Removes maximum transition and maximum capacitance constraints if they are too tight.
- Removes the maximum net length constraint if it is less than 50 microns.
- Ignores clock cell spacing rules if the horizontal spacing requirement is more than three times the site height and the vertical spacing requirement is more than twenty times the site height.
- Ignores nondefault routing rules if the width plus the spacing requirement of the routing rule is more than ten times the default width plus the default spacing.
- Reports balance point delay values that are large enough to cause an increase in clock insertion delay.

Feedback

Defining Clock Cell Spacing Rules

To define clock cell spacing rules, use the `set_clock_cell_spacing` command. At a minimum, you must specify the minimum spacing in the x-direction or the y-direction; typically you would specify the minimum spacing in both directions.

To specify the minimum spacing in microns in the

- X-direction, set the `-x_spacing` option to a nonzero value

  This value is usually dependent on the library cell's drive strength and clock frequency.

- Y-direction, set the `-y_spacing` option to a nonzero value

  This value is usually less than half a row height.

Note:

Using a large spacing value increases the skew because the cells are spaced further away from the intended clusters of sinks.

By default, the specified cell spacing requirements apply to all clock cells. To restrict the clock cell spacing rules to specific

- Library cells, use the `-lib_cells` option
- Clocks, use the `-clocks` option

If adjoining clock cells both have cell spacing rules for a given direction, the sum of the spacing values applies. For example, if two adjoining clock cells both have a minimum cell spacing of 5 microns in the x-direction, they are placed at least 10 microns apart in the x-direction. You can relax the adjoining cell spacing rule by setting the `cts.placement.cell_spacing_rule_style` application option to `maximum`, in which case the tool uses the larger of the spacing settings between two adjoining cells instead of the sum of the spacing settings. For example, the clock cells in the previous example are placed at least 5 microns apart in the x-direction, instead of 10 microns.

The clock cell spacing rules defined by the `set_clock_cell_spacing` command are honored by the `synthesize_clock_trees`, `balance_clock_groups`, and `clock_opt` commands.

To report clock cell spacing rules, use the `report_clock_cell_spacings` command.

To remove clock cell spacing rules, use the `remove_clock_cell_spacings` command. By default, this command removes all clock cell spacing rules. To remove specific clock cell spacing rules, use the `-lib_cells` option.

Feedback

Creating Skew Groups

During clock tree synthesis, you might want to balance a group of sinks only among each other, and not the rest of the sinks. To do so, define a skew group by using the `create_clock_skew_group` command.

When you use this command, specify the sinks you want to group, by using the `-objects` option.

Optionally, you can specify the following:

- A clock or generated clock for which to create the skew group by using the `-clock` option.
- A mode for which to create the skew group by using the `-mode` option.

  By default, the tool creates the skew group for the current mode.
- A name for the skew group by using the `-name` option.

The following example creates a skew group named sg1 consisting of sinks reg1/CP, reg2/CP, and reg3/CP:

```
icc2_shell> create_clock_skew_group -name sg1 \
   -objects {reg1/CP reg2/CP reg3/CP}
```

To report skew groups, use the `report_clock_skew_groups` command. To remove skew groups, use the `remove_clock_skew_groups` command.

Defining a Name Prefix for Clock Cells

You can specify a name prefix for the cells added on the clock network during clock tree synthesis by using the `cts.common.user_instance_name_prefix` application option.

The following example specifies CTS_ as the name prefix for the cells added on the clock network during clock tree synthesis:

```
icc2_shell> set_app_options \
   -name cts.common.user_instance_name_prefix -value "CTS_"
```

To specify a name prefix for the cells added on the data nets during optimization, including during the `clock_opt` command, use the `opt.common.user_instance_name_prefix` application option, as described in Specifying a Cell Name Prefix for Optimization.

Feedback

Using the Global Router During Initial Clock Tree Synthesis

By default, initial clock tree synthesis uses the virtual router to build the clock trees. For designs with complex floorplans, this can introduce congestion hotspots and degrade clock QoR after clock routing.

The tool can use the global router during the initial clock tree synthesis stage of the `synthesize_clock_trees` and `clock_opt` commands. To enable this feature, set the `cts.compile.global_route_aware_buffering` application option to `true`.

Specifying Constraints for Clock Nets

By default, the IC Compiler II tool uses the default routing rule and any available routing layers to route the clock trees.

You can specify the minimum and maximum routing layers for clock nets as described in Specifying the Routing Resources. These routing layer constraints are used during RC estimation, congestion analysis, and routing.

To reduce the wire delays in the clock trees, you can use wide wires and higher metal layers instead. Wide wires are represented by nondefault routing rules. To use nondefault routing rules on the clock nets, you must first define the routing rules and assign them to the clock nets as described in Using Nondefault Routing Rules.

Constructing Faster Clock Tree to Specific Endpoints

The IC Compiler II tool enables you to construct a faster clock tree to specified endpoints without any additional detouring or delay balancing. This improves clock latency to the specified endpoints. To achieve a faster clock tree to specific endpoints, you must create a user skew group for the required endpoints. To create a skew group for a specific endpoint, use the following command before running the `synthesize_clock_tree` or the `build_clock` stage of the `clock_opt` command:

```
icc2_shell> create_clock_skew_group -objects <endpoints>
```

Note:

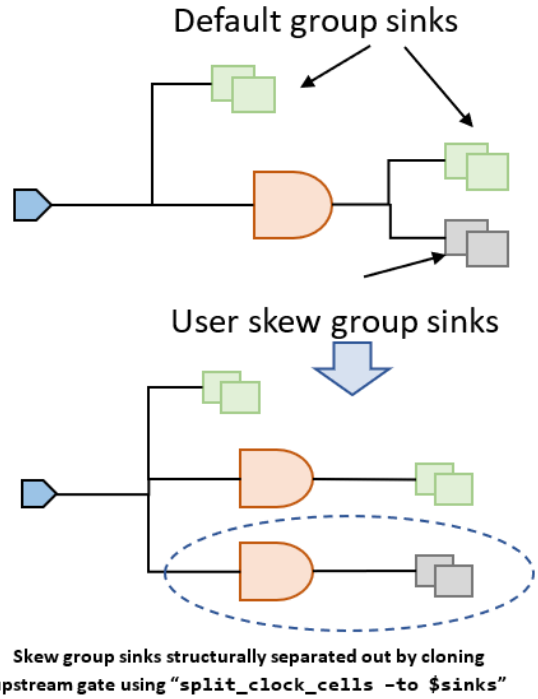
- If there is a balancing requirement between the fast endpoints, you must put them in the same skew group.
- If there is no balancing requirement between the endpoints, you must create separate skew groups for each endpoint.

The upstream clock logic of the required skew group endpoints is shared with the endpoints of the default group or other user skew groups. This leads to

Feedback

detouring for balancing among the endpoints of the default group or other user skew group.

Figure 40 Faster Clock Tree to Specific Endpoints



If a gate drives both skew group and non-skew group endpoints, the skew group endpoints might be delayed due to the balancing requirements of the non-skew group endpoints. In such cases, in addition to creating skew groups, you must also completely isolate the skew group endpoints from the rest of the clock tree by cloning the gates along the path to these endpoints. To create a separate clock path from the clock source to the required endpoints, use the `-to` option with the `split_clock_cells` command before clock tree synthesis, that is, before using the `synthesize_clock_trees` or the `build_clock` stage of the `clock_opt` command.

```
icc2_shell> split_clock_cells -to $endpoints
```

The IC Compiler II tool enables you to construct a faster clock tree by identifying the optimal cloning solution to achieve the best latency for the group of specified sinks. To achieve a fast clock tree to certain endpoints, use the `split_clock_cells -to < $sinks> -latency_driven` command. The endpoints must also be created as part of the user skew group using the `create_clock_skew_group` command.

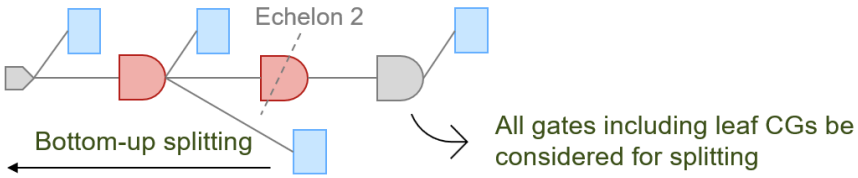
Feedback

Improving Latency-Driven Clock-Gate Splitting

The latency-driven clock-gate (CG) splitting now automatically identifies and evaluates different CG cloning moves. This improves clock insertion delay to meet the maximum level constraint.

To enable level driven CG splitting, set the `cts.icg.latency_driven_cloning` application option to `true`.

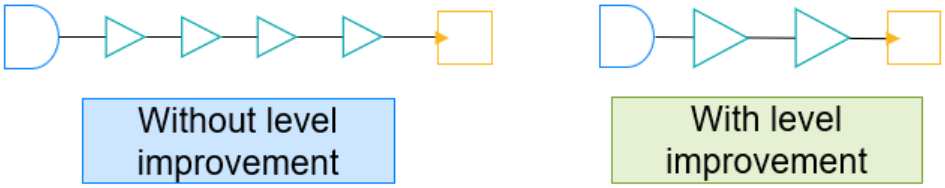
Figure 41 Latency Driven Clock-Gate Splitting



Improving the Repeater Levels in Clock Tree

The clock level improvement ensures that the tool considers level improvement as a metric while building the clock trees. This also considers buffering solutions that helps to improve the repeater levels. With lesser buffer trees chosen the level improvement takes place locally, which further helps to improve the metric for overall clock tree.

Figure 42 Improving the Repeater Levels in Clock Tree



To enable clock level improvement, set the `cts.buffering.reduce_clock_level_effort` application option to `high` before the `build_clock` stage of the `clock_opt` command.

Enabling Latency-Driven Clock-Gate Collapsing Levels

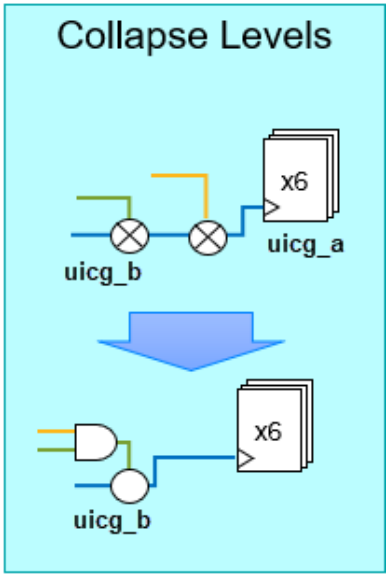
The latency-driven clock-gate (CG) collapsing level feature provides a restructuring step, during the final_place stage of the `compile_fusion` command of the design flow. By enabling this feature, the tool collapses clock-gates that are available on latency critical paths, which helps reduce the maximum clock latency.

Feedback

To enable latency-driven clock-gate collapsing level, set the `cts.icg.latency_driven_collapsing` application option to `true` as follows:

```
icc2_shell> set_app_options -name  cts.icg.latency_driven_collapsing
 -value true
```

Figure 43 Latency Driven Clock-Gate Collapsing Levels



Reducing Signal Integrity Effects on Clock Nets

The timing of a design can be improved by reducing signal integrity issues on clock nets. To do so, the tool can derive nondefault routing rules with larger spacing requirements and assign them to the clock nets in regions without routing congestion.

To enable this feature, use the following application option setting:

```
icc2_shell> set_app_options \
   -name cts.optimize.enable_congestion_aware_ndr_promotion -value true
```

When the tool derives a new nondefault routing rule for a clock net, the new spacing requirement is dependent on the spacing requirement of the existing routing rule of that net. If the existing routing rule is

- A nondefault routing rule with a spacing requirement of less than or equal to two times the default spacing, the spacing requirement is doubled

  The name of this new nondefault rule is the original name with an _ext_spacing postfix.

- A nondefault routing rule with a spacing requirement of more than two times the default spacing, the spacing requirement is increased by one default spacing

Feedback

The name of this new nondefault rule is the original name with an _ext_spacing postfix.

- The default routing rule, the spacing requirement is doubled

  The name of this new nondefault rule is default_rule_equivalent_ndr_double_spacing.

Specifying Settings for Clock Latency Adjustments

I/O ports are usually constrained using real or virtual clocks. After clock tree synthesis, the latencies of the clocks constraining the I/O ports need to be updated to ensure that the boundary constraints are accurate. When you run clock tree synthesis with the `clock_opt` command, the tool automatically updates the latencies of the clocks constraining the I/O ports. You can control these latency adjustments by using the `set_latency_adjustment_options` command.

The following example specifies that the clock latencies of the VCLK1 and VCLK2 clocks should be updated based on the latency of the PC_CLK clock, and that the latency of the VCLK3 clock should not be updated:

```
icc2_shell> set_latency_adjustment_options \
   -reference_clock PC_CLK -clocks_to_update {VCLK1 VCLK2} \
   -exclude_clocks VCLK3
```

If you perform clock tree synthesis with the `synthesize_clock_trees` command, the tool does not automatically update the clock latencies. To do so, use the `compute_clock_latency` command after clock tree synthesis. This command honors the settings you specify with the `set_latency_adjustment_options` command.

Timing Driven Sink Transition Target

The tool supports the setup and hold timing on critical endpoints by achieving a sharper transition on the critical launch and capture clock pins. The tool populates the ideal timing information from the `place_opt` and uses it to identify the critical endpoints. A transition target tighter than the specified `max_transition` is applied for the clock pins of timing critical endpoints and the clock tree is built considering the tightened transition which helps to improve the timing.

To enable timing driven sink transition target, set the `cts.compile.timing_driven_sink_transition` application option to `true` before the `place_opt` stage.

Reporting the Clock Tree Settings

To report the settings that are used by clock tree synthesis, use the `report_clock_settings` command. By default, the commands reports the clock tree

Feedback

configuration, including the design rule constraints, target skew, and target latency; the clock tree references; the clock cell spacing rules; and the nondefault routing rules for all clocks in the current block.

To report the settings for specific clocks, use the `-clock` option to specify the clocks of interest. To report specific information, use the `-type` option with the appropriate keyword.

- To report the clock tree design rule constraints, target skew, and target latency, use `-type configurations`.
- To report the clock tree references, use `-type references`.
- To report the clock cell spacing rules, use `-type spacing_rules`.
- To report the nondefault routing rule used for the clock nets, use `-type routing_rules`.

Pre-CTS Latency Bottleneck Reporting

After executing clock tree synthesis (CTS), the longest critical paths can only be identified using the `report_clock_qor` command or `cts.compile.report_latency_bottleneck` application option. It is challenging to identify the latency critical paths before running CTS.

To identify the longest latency bottleneck paths even before running clock tree synthesis, run the `report_estimated_clock_latency` command along with the following switches.

Table 20 Report Estimated Clock Latency Switches and Their Descriptions

Switches	Description
[-clocks *clock_name*]	Reports the longest path only for the specified clock.
[-corner *corner_name*]	Reports longest paths for the specified corner. By default, it is reported for the primary corner.
[-longest *longest*]	Specifies the number of longest paths to be reported per clock.
[-through *through_pin_name*]	Reports the longest path passing through the specified pin or driver.
[-unique_gate_levels *unique_gate_levels*]	Specifies the number of driver levels that must be unique to each reported sink. Usually used with `-longest`.

Feedback

Note:

The `report_estimated_clock_latency` is a standalone command and can be used independently of the `synthesize_clock_trees` command.

By default, pre-CTS latency bottleneck analysis reports one longest path per clock for the primary corner.

The effect of integrated clock gating (ICG) relocation on latency during the `synthesize_clock_trees` command is not considered since this is pre-CTS reporting.

Reporting Auto Exceptions and Summary Tables

Currently, the auto exceptions are handled in clock tree synthesis during the clock tree initialization step. The `check_clock_trees` command is enhanced to report the auto exceptions beforehand, which are derived during clock tree synthesis.

Note:

A file with the naming convention `check_clock_trees_clock_auto_exceptions*.tcl` is dumped in the current working directory. This feature does not make changes to the current database.

To print summary table before and after each step in multithreaded clock tree optimization (MTCTO), set the `cts.optimize.print_qor_summary_table` application option to `true`. This enables you to have a clear view of clock QoR.

```
icc2_shell> set_app_options -name cts.optimize.print_qor_summary_table
 -value true
```

Note:

The QoR summary tables are printed for each clock, corner, and mode.

To report all the H-tree drivers, use the `is_mscts_global_driver_object` attribute.

```
icc2_shell> get_clock_tree_pins -clock$clock -filter
 is_mscts_global_driver_object
```

Note:

This feature works for multitap driver setup as well.

To report all the concurrent clock and data (CCD) boundary pins in the design, use the `is_on_boundary_register` attribute.

```
icc2_shell> get_clock_tree_pins -filter is_on_boundary_register
```

Feedback

Dynamic Clock Power Improvement for Structural Multisource Clock Tree Synthesis

Switching activity interchange format (SAIF) awareness enhances the structural multisource clock tree synthesis (SMSCTS) flow to improve clock dynamic power. This approach reduces the wire length of high toggle rate nets by relocating cells and clusters of cells having high input toggle rate towards their driver. This improves the overall clock dynamic power. It provides an accept or reject mechanism after each power aware relocation so that the IC Compiler II tool only accepts moves where dynamic power is improved.

To enable SAIF awareness in both standalone and integrated SMSCTS flows, set the `cts.multisource.enable_activity_aware_relocation` application option to `true`.

```
icc2_shell> set_app_options -name
 cts.multisource.enable_activity_aware_relocation -value true
```

Enabling Target Latency for Skew Groups and ICDB Improvement

By default, the IC Compiler II tool aims to achieve the best possible latency for all clocks during clock tree synthesis (CTS). You can apply a target latency constraint to achieve a latency greater than the one the tool has already achieved. So, during the target delay insertion step, the tool aims to achieve the user-defined latency target.

You can set the target latency on skew groups during the target latency insertion step in the `build_clock` stage of the `clock_opt` stage. To specify target latency for a specific skew group, you can use the `-target_latency` option along with the `-skew_groups` option with the `set_clock_tree_options` command as follows:

```
icc2_shell> set_clock_tree_options -skew_groups $SG -target_latency
 $value
```

Currently, the IC Compiler II tool does not add repeaters to nets that have a derived `size_only` constraint and searches for an alternative valid driver. Recent enhancements to the target latency insertion and inter clock delay balancing (ICDB) steps improve the implementation of intended delays. These enhancements include:

- Improved handling of nets with the derived `size_only` constraints.
- An enhanced driver search algorithm to search for valid drivers.

To enable the handling of nets with the derived `size_only` constraint, set the `cts.optimize.delay_insertion_enhancements` application option to `true` as follows:

```
icc2_shell> set_app_options -name
 cts.optimize.delay_insertion_enhancements -value true [Default : false]
```

Feedback

Note:

When this application option is enabled, the tool adds repeaters to the net regardless of the derived `size_only` constraint on the nets.

Additionally, since the target latency insertion and ICDB steps use the same engine, the tool uses multithreading to speed up runtime when adding buffer chains. During ICDB, the tool balances two clock domains against each other.

Debugging and Usability Improvement Using the check_clock_trees Command and CTS Log File

The `check_clock_trees` command and the clock tree synthesis (CTS) log file provide the following usability and debugging improvements:

- Printing warnings indicating restrictions on logical hierarchical pins: The `check_clock_trees` command does not provide information about restrictions on logical hierarchical pins, and you need to manually check the log file for the corresponding gate-levels to identify these restrictions. With this enhancement, the `check_clock_trees` command prints the exact reasons for restrictions on logical hierarchical pins, along with a CTS-076 warning message. This eliminates the need for manual log inspection.
- Printing progress heartbeat messages during all the multithreaded clock tree optimization (MTCTO) steps in the CTS log file: The IC Compiler II tool prints progress heartbeat messages during all MTCTO steps in the CTS log file. This helps to track the progress of a run, in cases of high runtime issues. The heartbeat messages include:
  - Initial design rule check (DRC) fixing
  - Path-based global latency and skew optimization
  - Area recovery
  - Final DRC fixing

Note:

Heartbeat messages are printed every 20 minutes or after processing more than 1000 batches, whichever occurs first.

- Automatic handling of output ports during CTS: In some designs, feedthrough clock nets exist from one input to one output without any sinks. The tool reports a CTS-905 warning message for clocks with no sinks. These feedthrough clock nets are considered part of the design rule check (DRC) fixing beyond exceptions and are not optimized for skew and latency. To address this, the tool automatically identifies output ports driven by a clock as clock sinks. This eliminates the need to manually find these ports and set them as balance points using the `set_clock_balance_point -consider_for_balancing` command.

Feedback

Auto exception derivation during CTS automatically identifies the output ports and sets a balance point constraint to `true`. To automatically identify the output ports, set the `cts.common.auto_exception_output_ports_balance_point` application option to `true` as follows:

```
icc2_shell> set_app_options -list
 {cts.common.auto_exception_output_ports_balance_point true}
```

Note:

This information is captured in the auto exception derivation file, the `check_clock_trees_clock_auto_exceptions*.tcl` file which gets created in the current working directory. This does not make any changes to the current database.

Clock Routing Improvement Using Intelligent Minimum Layer Handling of Clock Nets

The existing clock routing behavior provides only extreme cases of control with different `min_layer_mode` values:

- `soft`: The net uses too many lower layers, impacting delays.
- `allow_pin_connection`: The net tries hard to stay on the higher layers, causing wirelength to increase.

Currently, when higher layers are used for routing a net, the tool does not differentiate if the parts or segments are close to the driver or close to the sinks.

With the intelligent minimum layer handling enhancement, the tool routes the portions of the net close to the driver on higher layers while routing the portions of the net close to the sinks on the lower layers. Since resistance on the driver side has a greater impact on net delay, this helps to maintain delay while improving wirelength closer to the sinks, especially for high fanout nets.

You can achieve better clock routing quality with improved clock wirelength, convergence, and neutral or improved latency using intelligent minimum layer handling of clock nets. To enable the intelligent minimum layer handling, set the `route.global.enable_clock_clustering` application option to `true` as follows:

```
icc2_shell> set_app_options -name route.global.enable_clock_clustering
 -value true
```



This video explains how intelligent minimum layer handling improves clock routing quality, reduces delays, and enhances overall design performance.

Feedback

Implementing Clock Trees and Performing Post-CTS Optimization

Before you perform clock tree synthesis, you should save the block. This allows you to refine the clock tree synthesis goals and rerun clock tree synthesis with the same starting point, if necessary.

The following topics describe the different methods available for implementing clock trees:

- Performing Standalone Clock Trees Synthesis
- Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command
- Controlling Concurrent Clock and Data Optimization
- Splitting Clock Cells
- Balancing Skew Between Different Clock Trees
- Performing Global-Route-Based Optimization Using Machine Learning Data
- Routing Clock Trees
- Inserting Via Ladders During Clock Tree Synthesis, Optimization, and Clock Routing
- Marking Clocks as Propagated After Clock Tree Synthesis
- Performing Postroute Clock Tree Optimization
- Performing Voltage Optimization
- Marking Clock Trees as Synthesized
- Removing Clock Trees
- Optimizing Nworst Skew Framework for Multithreaded Clock Tree

Performing Standalone Clock Trees Synthesis

To perform standalone clock tree synthesis, use the `synthesize_clock_trees` command, as shown in the following example:

```
icc2_shell> synthesize_clock_trees
```

Feedback

When you use this command, the tool performs the following tasks:

1. Clock tree synthesis

   Before clock tree synthesis, the tool performs virtual routing of the clock nets and uses RC estimation to determine the clock net timing.

2. Clock tree optimization

   Before clock tree optimization, the tool performs global routing on the clock nets and uses RC extraction to determine the clock net timing. During clock tree optimization, the tool performs incremental global routing on clock nets modified during optimization.

Note:

This command does not detail route the clock trees.

By default, this command works on all clocks in all active scenarios that are enabled for setup or hold analysis. You can specify the clocks to be built by using the `-clock` option. For example, to build only the CLK clock tree, use the following command:

```
icc2_shell> synthesize_clock_trees -clocks [get_clocks CLK]
```

After clock tree synthesis, the tool sets the clocks in all modes of all active scenarios as propagated.

Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command

To synthesize the clock trees, route the clock nets, and further optimize the design by using a single command, use the `clock_opt` command.

The `clock_opt` command consist of the following stages:

1. The `build_clock` stage, during which the tool synthesizes and optimizes the clock trees for all clocks in all modes of all active scenarios. After clock tree synthesis, the tool sets the synthesized clocks as propagated.
2. The `route_clock` stage, during which the tool detail routes the synthesized clock nets.
3. The `final_opto` stage, during which the tool performs further optimization, timing-driven placement, and legalization. It then global routes the block and performs extensive global-route-based optimization, which includes incremental legalization and route patching.

You can limit the `clock_opt` command to one or more contiguous stages by using the `-from` option to specify the stage from which you want to begin and the `-to` option to specify the stage after which you want to end. If you do not specify the `-from` option, the

Feedback

tool begins from the `build_clock` stage. Similarly, if you do not specify the `-to` option, the tool continues until the `final_opto` stage is completed.

For example, to synthesize and optimize the clock trees and detail route the clock nets only, limit the execution to the `build_clock` and `route_clock` stages by use the following command:

```
icc2_shell> clock_opt -to route_clock
```

Because the tool performs global routing during the `final_opto` stage, you should specify all router related settings, such as routing rules, route guides, application option settings required for the technology node, and so on, before you run the `clock_opt` command.

You should run the `final_opto` only one time for each block. After this stage is completed, all the signal routes in the block are global routed. Therefore, when subsequently run any routing command, such as the `route_auto`,`route_global`, or `route_group` command, the tool skips global routing. To avoid errors during subsequent track assignment and detail routing, do not change any global route shapes after you run the `clock_opt` command.



This video explains the two Clock Tree Synthesis (CTS) flows in : the classic Clock Tree Synthesis flow and the Concurrent Clock and Data Optimization (CCD) or Useful Skew flow. It highlights their differences, stages, and goals.

Considering Voltage Drop Information During Clock Tree Synthesis

By default, the `clock_opt` command does not consider voltage drop information when inserting clock buffers. Therefore, the tool might place clock buffers in locations with high PG resistance, which increases the voltage drop. To prevent this, you can use the RedHawk Fusion feature to analyze the PG network and use this information with the `clock_opt` command, as shown in the following flow:

1. Use the RedHawk Fusion feature to analyze the PG network by using the `analyze_rail -min_path_resistance` command.
2. Load the RedHawk Fusion results by using the `open_rail_result` command.
3. Enable voltage-drop-aware clock tree synthesis by setting the `clock_opt.flow.enable_voltage_drop_aware` application option to `true`.
4. Run the `clock_opt` command.

Feedback

Using Nondefault Routing Rules for Critical Nets During Optimization

To improve timing QoR, the tool can use nondefault routing rules on timing critical nets during preroute optimization. In addition, the tool can guide the router to honor these nondefault rule assignments as soft constraints.

To enable this capability for the `clock_opt` command, set the `clock_opt.flow.optimize_ndr` application option to `true`.

```
icc2_shell> set_app_options -name clock_opt.flow.optimize_ndr \
    -value true
```

Performing Concurrent Clock and Data Optimization During the clock_opt Command

Applying useful skew techniques during datapath optimization to improve the timing QoR by taking advantage of the positive slack and adjusting the clock arrival times of registers is referred to as concurrent clock and data (CCD) optimization.

By default, the tool performs concurrent clock and data optimization during the `clock_opt` command.

To disable concurrent clock and data optimization for the `clock_opt` command, set the `clock_opt.flow.enable_ccd` application option to `false`.

You can change the default behavior of concurrent clock and data optimization during the `clock_opt` command by performing any of the following optional steps:

1. (Optional) Limit the latency adjustment values for concurrent clock and data optimization as described in Limiting the Latency Adjustment Values.
2. (Optional) Control the latency adjustment of boundary registers for concurrent clock and data optimization as described in Excluding Boundary Paths.
3. (Optional) Ignore specific path groups during concurrent clock and data optimization as described in Excluding Specific Path Groups.
4. (Optional) Ignore specific scenarios during concurrent clock and data optimization as described in Excluding Specific Scenarios.
5. (Optional) Ignore specific sinks during concurrent clock and data optimization as described in Excluding Specific Sinks.
6. (Optional) Control the effort of timing optimization performed during concurrent clock and data optimization as described in Controlling Timing Optimization Effort.
7. (Optional) Control the effort of hold timing optimization performed during concurrent clock and data optimization as described in Controlling Hold Time Optimization Effort.

Feedback

8. (Optional) Control the adjustment of I/O clock latencies performed during concurrent clock and data optimization as described in Controlling the Adjustment of I/O Clock Latencies.
9. (Optional) Analyze the effects of concurrent clock and data optimization as described in Reporting Concurrent Clock and Data Timing.

Controlling Multibit Optimization Performed During the clock_opt Command

The following topics describe how you can control the multibit optimization performed during the `clock_opt` command:

- Enabling the Rewiring of Mixed-Drive-Strength Multibit Cells
- Enabling Post-Clock-Tree-Synthesis Multibit Debanking

Enabling the Rewiring of Mixed-Drive-Strength Multibit Cells

Some logic libraries have mixed-drive-strength multibit cells where some bits have a higher drive strength than others. For designs that use such cells, if a violating path goes through a lower-drive-strength bit, the tool can rewire the mixed-drive-strength multibit cell such that the violating path goes through a higher-drive-strength bit. To enable this feature for the `clock_opt` command, set the `clock_opt.flow.enable_multibit_rewiring` application option to `true`.

Enabling Post-Clock-Tree-Synthesis Multibit Debanking

To improve the timing QoR, the `clock_opt` command can debank multibit registers during the `final_opt` stage, if doing so does not introduce hold timing violations. To enable this feature, set the `clock_opt.flow.enable_multibit_debanking` application option to `true` before you run the `final_opt` stage of the `clock_opt` command.

Performing Power or Area Recovery on the Clock Network

If you enable concurrent clock and data optimization for the `clock_opt` command, the tool performs clock power recovery on clock cells and registers during the `final_opt` stage. If you do not enable concurrent clock and data optimization, you can enable clock power recovery by using the following application option setting:

```
icc2_shell> set_app_options \
   -name clock_opt.flow.enable_clock_power_recovery \
   -value power
```

Feedback

Before you perform power recovery using this technique, you must

1. Enable scenarios for dynamic, leakage, or total power optimization by using the `-dynamic_power` and `-leakage_power` options of the `set_scenario_status` command
2. (Optional) Provide a switching activity by using the `read_saif` command

   If you do not provide switching activity, the tool uses default switching activity for dynamic power recovery.

Instead of power recovery, you can enable area recovery on the clock cells and registers during the `final_opt` stage `clock_opt` command by using the following application option setting:

```
icc2_shell> set_app_options \
   -name clock_opt.flow.enable_clock_power_recovery \
   -value area
```

Performing IR-Drop-Aware Placement During the clock_opt Command

During placement, the tool can use the voltage (IR) drop values of cells to identify areas of high power density and spread the cells with high voltage drop values, which reduces the power density of such areas.

To perform IR-drop-aware placement during the `clock_opt` command, use the following steps:

1. Run the `clock_opt` command through the route-clock stage by using the `clock_opt -to route_clock` command.
2. Set up for RedHawk Fusion and perform static or dynamic voltage drop analysis by using the `analyze_rail -voltage_drop` command as shown in the following example:

   ```
   icc2_shell> source redhawk_setup.tcl
   icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
   ```

   For more information, see Performing Voltage Drop Analysis.
3. Enable IR-drop-aware placement by setting the `place.coarse.ir_drop_aware` application option to `true`.
4. (Optional) Specify additional settings for IR-drop-aware placement, as described in Controlling IR-Drop-Aware Placement.
5. Run the final optimization stage of the `clock_opt` command by using the `clock_opt -from final_opto` command.

Feedback

Clock Power Recovery

In multithreaded clock tree optimization (MTCTO), metrics such as design rule check (DRC), latency, global skew, and area are used to improve quality of results (QoR) of the clock tree, but power is never used as a metric for QoR improvements.

In the Fusion Compiler tool, QoR improvements are achieved using area recovery, followed by an additional clock tree synthesis step called Power Optimization. Power optimization happens after area recovery and before the final DRC fixing step to have improved clock power. It uses buffer removal and sizing to reduce power without degrading latency and global skew.

To control the power optimization, set the `cts.optimize.enable_cto_power_optimization` application option to `true` .

```
icc2_shell> set_app_options -list
 {cts.optimize.enable_cto_power_optimization true}
```

Note:

If the power optimization option is off by default, return to the current behavior for area recovery.

Controlling Concurrent Clock and Data Optimization

Applying useful skew techniques during datapath optimization to improve the timing QoR by taking advantage of the positive slack and adjusting the clock arrival times of registers is referred to as concurrent clock and data (CCD) optimization.

Concurrent clock and data optimization is enabled by default for the `place_opt` and `clock_opt` commands. To enabled it for the `route_opt` command, set the `route_opt.flow.enable_ccd` application option to `true`.

You can change the default behavior of concurrent clock and data optimization by

- Limiting the Latency Adjustment Values
- Excluding Boundary Paths
- Excluding Specific Path Groups
- Excluding Specific Scenarios
- Excluding Specific Sinks
- Controlling Timing Optimization Effort
- Controlling Hold Time Optimization Effort
- Controlling the Adjustment of I/O Clock Latencies

Feedback

- Performing Dynamic-Voltage-Drop-Driven Concurrent Clock and Data Optimization During the route_opt Command
- Specifying Optimization Targets at the Preroute Stage
- Specifying Optimization Targets at the Postroute Stage
- Enabling Buffer Removal at the Postroute Stage
- Reporting Concurrent Clock and Data Timing
- Scaling CCD Offsets to New Scenarios for Reporting Timing
- Skewing Latch and Discrete Clock Gates
- Optimizing Power Using Data Path Aware CUS
- Optimizing Timing Using Data Path Aware Clock Useful Skew
- Incremental Clock-Data Optimization
- Timer-Based Predictive Concurrent Clock-Data Optimization

Limiting the Latency Adjustment Values

You can limit the latency adjustment values for concurrent clock and data optimization performed during the `place_opt`, `clock_opt`, and `route_opt` commands as follows:

- Limit the amount that the clock latencies are advanced by using the `ccd.max_prepone` application option.
- Limit the amount that the clock latencies are delayed by using the `ccd.max_postpone` application option.

There is no default for these application options. Specify these values in the library timing units. The following example sets a limit of 0.2 for advancing and 0.1 for delaying clock latencies:

```
icc2_shell> set_app_options -list {ccd.max_prepone 0.2}
icc2_shell> set_app_options -list {ccd.max_postpone 0.1}
```

Excluding Boundary Paths

By default, the tool performs concurrent clock and data optimization on all paths in a block. However, you might want to exclude paths that are connected to boundary registers, which are registers in the transitive fanout of input ports and transitive fanin of output ports. To exclude boundary paths from concurrent clock and data optimization, set the `ccd.optimize_boundary_timing` application option to `false`.

When you do so, you can selectively ignore the boundary paths of some ports during boundary path exclusion by using the

Feedback

ccd.ignore_ports_for_boundary_identification application option. The following example disables concurrent clock and data optimization for all boundary paths except for those connected to the ports named IN_A and OUT_A:

```
icc2_shell> set_app_options -name ccd.optimize_boundary_timing \
   -value false
icc2_shell> set_app_options \
   -name ccd.ignore_ports_for_boundary_identification \
   -value {IN_A OUT_A}
```

Even when you disable concurrent clock and data optimization on boundary registers, the tool can still optimize the clock tree fanin cone of these boundary registers, when doing so improves the timing QoR of other internal registers that share the same clock paths. To prevent the clock tree fanin of boundary registers from changing, set the ccd.optimize_boundary_timing_upstream application option to false. However, doing so heavily restricts the scope of concurrent clock and data optimization.

These application option settings affect concurrent clock and data optimization performed during the place_opt, clock_opt, and route_opt commands.

Excluding Specific Path Groups

To exclude a specific path group from concurrent clock and data optimization, use the ccd.skip_path_groups application option and specify the name of the path group you want ignored.

You can ignore a path group for all scenarios or a specific scenario. For example, to ignore the path group named CLK1 for all scenarios and CLK2 for the scenario named scnA, use the following command:

```
icc2_shell> set_app_options -name ccd.skip_path_groups \
   -value {CLK1 {CLK2 scnA}}
```

This setting affects concurrent clock and data optimization performed during the place_opt, clock_opt, and route_opt commands.

Excluding Specific Scenarios

To exclude a specific scenarios from concurrent clock and data optimization, use the ccd.ignore_scenarios application option and specify the name of the scenarios you want to ignore.

For example, to ignore the scenario named scnB, use the following command:

```
icc2_shell> set_app_options -name ccd.ignore_scenarios \
   -value {scnB}}
```

This setting affects concurrent clock and data optimization performed during the place_opt, clock_opt, and route_opt commands.

Feedback

Excluding Specific Sinks

To exclude a specific sink from concurrent clock and data optimization,

1. Apply a `cts_fixed_balance_pin` attribute on the sink pin by using the `set_attribute` command as shown in the following example:

```
icc2_shell> set_attribute -objects reg21/CK \
   -name cts_fixed_balance_pin -value true
```

2. Set the `ccd.respect_cts_fixed_balance_pins` application option to `true`.

   These setting prevents the tool from adjusting the latencies of the sinks that have a `cts_fixed_balance_pin` attribute set to `true`. However, changes in the clock tree due to concurrent clock and data optimization performed on other sinks can change the latencies of the sinks that have a `cts_fixed_balance_pin` attribute set to `true`. To prevent the tool from making any changes to the clock paths between the sinks that have a `cts_fixed_balance_pin` attribute set to `true` and the clock root, set the `ccd.respect_cts_fixed_balance_pins` application option to `upstream` instead of `true`.

   This application option setting affects concurrent clock and data optimization performed during the `place_opt`, `clock_opt`, and `route_opt` commands.

Controlling Timing Optimization Effort

By default, the tool focuses on improving the timing and power QoR during concurrent clock and data optimization. You can change the effort level for timing optimization by setting the `ccd.timing_effort` application option to `low` or `high`. The default is `medium`.

This setting affects concurrent clock and data optimization performed during the `final_opt` stage of the `clock_opt` command and the `route_opt` command.



This video explains the two Clock Tree Synthesis (CTS) flows in : the classic Clock Tree Synthesis flow and the Concurrent Clock and Data Optimization (CCD) or Useful Skew flow. It highlights their differences, stages, and goals.

Controlling Hold Time Optimization Effort

By default, the tool focuses on fixing setup violations during concurrent clock and data optimization. However, if your design has hold violations that are difficult to fix, you can increase the effort for hold fixing by setting the `ccd.hold_control_effort` application option to `medium`, `high`, or `ultra`. The default is `low`.

Feedback

Increasing the priority of hold violations reduces the number of setup violations that are fixed. Therefore, the hold priority should be increased only if the hold timing is critical. This setting affects concurrent clock and data optimization performed during the `final_opt` stage of the `clock_opt` command and the `route_opt` command.

Controlling the Adjustment of I/O Clock Latencies

By default, the tool adjusts the latencies of the I/O clocks, which are the clocks that constrain the boundary ports, when it performs concurrent clock and data optimization during the `clock_opt` command. The tool also uses concurrent clock and data optimization techniques to adjust the I/O clock latencies when you subsequently run the `compute_clock_latency` command.

The tool does not update the I/O clock latencies

- For the entire block when you disable this feature by setting the `ccd.adjust_io_clock_latency` to `false` or when you disable concurrent clock and data optimization for boundary paths by setting the `ccd.optimize_boundary_timing` application option to `false`
- For specific I/O paths when you exclude the corresponding path groups from concurrent clock and data optimization by using the `ccd.skip_path_groups` application option
- For specific I/O clocks when you disable latency adjustment those I/O clocks by using the `set_latency_adjustment_options -exclude_clocks` command

Performing Dynamic-Voltage-Drop-Driven Concurrent Clock and Data Optimization During the route_opt Command

The concurrent clock and data optimization performed during the `route_opt` command can reduce the peak dynamic voltage drop without affecting the timing QoR. To enable this feature, set the `route_opt.flow.enable_voltage_drop_opt_ccd` application option to `true`.

When you enable this feature, during the `route_opt` command, the tool uses the RedHawk Fusion feature to identify dynamic-voltage-drop hotspots and cells rows where the cells from the hotspots can be relocated. Then, during optimization, the tool relocates or downsizes the cells in the hotspots to reduce the peak dynamic voltage drop without hurting the timing QoR.

You can specify one of the following thresholds for selecting cells for optimization:

- A threshold for the cell voltage drop, as a percentage of the supply voltage, by using the `ccd.voltage_drop_voltage_threshold` application option.

  Any cell with a voltage drop that exceeds this threshold is selected for optimization.

Feedback

- A threshold for the number of cells, as a percentage of the total number of cells, by using the `ccd.voltage_drop_population_threshold` application option.

  Cells are selected starting with the worst violator.

During postroute optimization, the tool can perform RedHawk dynamic voltage drop analysis to identify cells involved in voltage drop violations, and then use optimization techniques on those cells to improve the dynamic voltage drop.

You can further improve the dynamic voltage drop by enabling IR-driven sizing, which uses the RedHawk dynamic voltage drop analysis results to identify cells involved in voltage drop violations, and then tries to replace those cells with cells having smaller leakage current. To enable this feature, set the `route_opt.flow.enable_irdrivenopt` application option to `true`, in addition to setting the `route_opt.flow.enable_voltage_drop_opt_ccd` application option to `true`.

Specifying Optimization Targets at the Preroute Stage

When performing concurrent clock and data optimization using the `place_opt` or `clock_opt` command, you can give a higher priority to the WNS optimization of

- Certain path groups by specifying them by using the `ccd.targeted_ccd_path_groups` application option, as shown in the following example:

  ```
  icc2_shell> set_app_options -name ccd.targeted_ccd_path_groups \
     -value {PG1 PG2}
  ```

  If a path group you specify with this application option is also specified as a path group to skip with the `ccd.skip_path_groups` application option, the path group is skipped during concurrent clock and data optimization.

- Certain endpoints by specifying a file containing the endpoints by using the `ccd.targeted_ccd_end_points_file` application option, as shown in the following example:

  ```
  icc2_shell> set_app_options -name ccd.targeted_ccd_end_points_file \
     -value endpoint_targets.tcl
  ```

  If you specify both path groups and endpoints, the tool optimizes only the specified endpoints that are in the specified paths groups.

- The worst 300 timing paths by setting the `ccd.enable_top_wns_optimization` application option to `true`

Feedback

Specifying Optimization Targets at the Postroute Stage

At the final stages of your design flow, you can use the `route_opt` command to perform concurrent clock and data optimization on specific critical path groups or endpoints by using the following steps:

1. Enable targeted concurrent clock and data optimization for the `route_opt` command by setting the `route_opt.flow.enable_targeted_ccd_wns_optimization` application option to `true`.
2. Specify the optimization target by using one or both of the following settings:
   - Specify the path groups to optimize by using the `ccd.targeted_ccd_path_groups` application option

     Note:

     If a path group you specify with this application option is also specified as a path group to skip by using the `ccd.skip_path_groups` application option, the path group is skipped during targeted concurrent clock and data optimization.
   - Specify the endpoints to optimize by using the `ccd.targeted_ccd_end_points_file` application option

   You must specify at least one of these settings. If you specify both, the tool optimizes the specified endpoints of the specified paths groups.
3. (Optional) Specify the type of optimization to perform by using the `ccd.targeted_ccd_select_optimization_moves` application option.

   The valid values are
   - `auto`, the default, which enables all optimization types, including buffering
   - `size_only`, which enables sizing only
   - `equal_or_smaller_sizing`, which enables sizing only to cells that are the same size or smaller
   - `footprint_sizing`, which enables sizing only to cells with the same footprint
4. (Optional) Specify an optimization effort by using the `ccd.targeted_ccd_wns_optimization_effort` application option.

   The valid values are `high` (default), `medium`, and `low`. The higher the effort, the more timing QoR improvement you will see in the targeted paths, but at the expense of the timing QoR of other paths.

Feedback

5. (Optional) Specify a threshold beyond which the slack for the path groups that are not targeted can be degraded by using the `ccd.targeted_ccd_threshold_for_nontargeted_path_groups` application option.

   By default, if the path groups that are not targeted have positive slack, the tool can degrade this slack until it reaches zero.

For the `route_opt` command, you can enable both regular concurrent clock and data optimization by using the `route_opt.flow.enable_ccd` application option and targeted concurrent clock and data optimization by using the `route_opt.flow.enable_targeted_ccd_wns_optimization` application option. If you enable both, the tool performs regular concurrent clock and data optimization first, followed by targeted concurrent clock and data optimization.

The following script performs targeted concurrent clock and data optimization on a block that has already undergone routing and postroute optimization:

```
open_lib design1
open_block blk.post_route

#Disable regular CCD optimization, which was previously performed
set_app_options \
  -name route_opt.flow.enable_ccd -value false

#Enable targeted CCD optimization
set_app_options \
  -name route_opt.flow.enable_targeted_ccd_wns_optimization \
  -value true

#Specify the path group and endpoints to target
set_app_options -name ccd.targeted_ccd_path_groups -value clkA
set_app_options -name ccd.targeted_ccd_end_points_file \
   -value clkA_ep.txt

#Specify the optimization type and effort
set_app_options \
   -name ccd.targeted_ccd_select_optimization_moves -value size_only
set_app_options -name ccd.targeted_ccd_wns_optimization_effort \
   -value medium

# Perform the targeted CCD optimization
route_opt
```

Enabling Buffer Removal at the Postroute Stage

To improve QoR by removing buffers or pairs of inverters when concurrent clock and data optimization is performed during `route_opt` command, set the

Feedback

`ccd.post_route_buffer_removal` application option to `true`, as shown in the following example:

```
icc2_shell> set_app_options -name route_opt.flow.enable_ccd \
   -value true
icc2_shell> set_app_options -name ccd.post_route_buffer_removal \
   -value true
icc2_shell> route_opt
```

Reporting Concurrent Clock and Data Timing

You analyze the timing effects of concurrent clock and data optimization by using the `report_ccd_timing` command. By default, it reports the setup and hold slack of the worst capture (D-slack) and launch (Q-slack) paths of the five most critical endpoint registers in the block, as shown in the following example report:

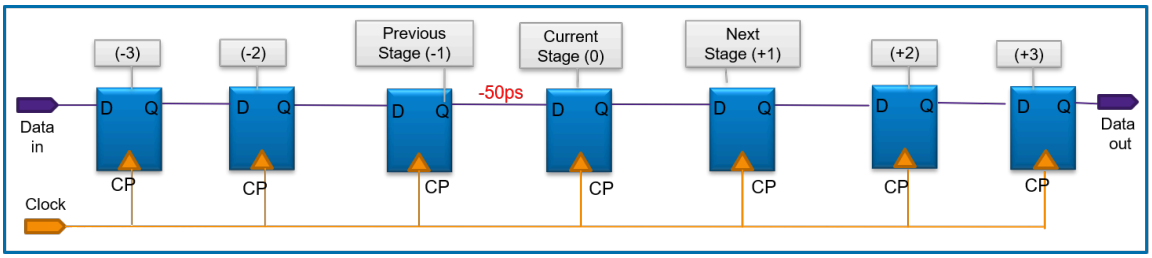
```
icc2_shell> report_ccd_timing
...
...
------------------------------------------------------------------
   Setup                      Hold
------------------------------------------------------------------
 D-slack    Q-slack       D-slack    Q-slack     Pin
------------------------------------------------------------------
-0.567710  -0.173974      0.524202   0.167434    sink2/CP
-0.175268   0.125309      0.168582   0.077625    sink3e/CP
-0.173974   0.126836      0.167434   0.076277    sink3a/CP
-0.033271  -0.175268      0.068856   0.168582    sink3d/CP
 0.017364  -0.033271     -0.022138   0.068856    sink3c/CP
```

To control the type of report generated, use the `-type` option as follows:

- To report information for the paths in the fanin or fanout of the most critical endpoints, use the `-type fanin` or `-type fanout` option.
- To report information for the previous, current, and next path stages of the most critical endpoints, use the `-type stage` option.
- To report information for all the previous, current, and next path stages of the most critical endpoints, use the `-type chain` option. Figure 44 shows the previous, current, and next stages for an endpoint register.

Feedback

Figure 44 Previous, Current, and Next Path Stages



By default, the command reports setup slacks at D and Q pins of registers. To report the hold slacks at these pins, use the `-hold` option.

By default, the command analyzes timing paths across all active scenarios and reports the worst D and Q slacks across scenarios. To analyze and report the slack for specific scenarios use the `-scenarios` option.

To report the information for specific path endpoints, specify the corresponding clock pins by using the `-pins` option.

You can analyze the effects latency adjustments performed during concurrent clock and data optimization by increasing or decreasing the clock arrival of a specific endpoint using the `-prepone` or `-postpone` option with the `-pins` option as shown in the following example:

```
icc2_shell> report_ccd_timing -pins Reg11/clk -postpone 0.02
```

You can also see the effects of changing specific cell and net delay values by using the `-annotate_cell_delay` and `-annotate_net_delay` options, as shown in the following example:

```
icc2_shell> report_ccd_timing -annotate_cell_delay {{U1/a U1/y 0.2}} \
   -pin I2/d -type stage
```

The delay values you specify with these options are used only for analyzing and reporting the concurrent clock and data timing. They are not saved and used during subsequent concurrent clock and data optimization.

Debugging Concurrent Clock and Data QoR

Debugging concurrent clock and data (CCD) QoR for timing improvement is always challenging. Individual timing paths are analyzed to check if CCD has used its full potential. Debugging CCD QoR for timing improvement becomes difficult because this is a manual process that makes it error-prone.

The `report_ccd_timing` command only reports the launch flip-flop on the most critical path. Reporting the most critical path does not ensure if the endpoint violation can be fixed. This helps to prepone the debuggability.

Feedback

- Reporting the subcritical path: To report the subcritical path with a smaller D-slack margin in the previous stage, which impacts preponing, use the `-consider_subcritical_registers` option with the `report_ccd_timing` command.

  ```
  icc2_shell> report_ccd_timing -consider_subcritical_registers
  ```

- Reporting clock arrival and skew information: To report the clock arrival and skew information, use the `-clock_timing` option with the `report_ccd_timing` command.

  ```
  icc2_shell> report_ccd_timing -consider_subcritical_registers
   -clock_timing
  ```

Note:

The `-clock_timing` option simplifies the process of correlating clock timings with slacks.

Scaling CCD Offsets to New Scenarios for Reporting Timing

To copy concurrent clock and data (CCD) offsets to the new scenarios for consistent timing or QoR reports, use the `copy_useful_skew` command. During CCD, when a pin's CTS balance point offsets are derived in only a subset of the active scenarios, use this command to copy the offsets to the in-active scenarios and active scenarios that do not have the CCD offsets.

The `copy_useful_skew` command also ensures that the corresponding constraints of the `set_clock_latency` command are set on all the pins and scenarios, where a new offset value is computed. The average value of the offset is computed across all scenarios, where a balance point offset is defined. The average value is then scaled back in each active scenario using the scaling factor of the scenario corner.

Skewing Latch and Discrete Clock Gates

To skew latches and discrete clock gates together, set the `ccd.skew_opt_merge_dcg_cells_by_driver` application option to `true` before `compile_fusion` or `place_opt final_opto`. By default, the application option is set to `false`. Setting the `ccd.skew_opt_merge_dcg_cells_by_driver` application option to `true` enhances the concurrent clock and data optimization to skew latches and gating element by the same offset. This ensures the latch and gate are at the same level and improves hold timing of discrete clock gates.

```
icc2_shell> set_attribute
   -name ccd.skew_opt_merge_dcg_cells_by_driver -value true
```

Optimizing Power Using Data Path Aware CUS

The IC Compiler II tool uses CUS for power optimization to increase positive slack at startpoints and endpoints. These startpoints and endpoints are driven by logic cones having power potential.

Feedback

Compute useful skew (CUS) changes the launch and capture clock arrival times to move slack from pipeline stages having zero power potential to the stages that have some power potential. As a result, the stages with power potential are now also available with some timing potential and can be viewed and optimized by further data optimization.

To control the CUS behavior, set the `place_opt.flow.enable_dpa_ccd_power` application option to `true` .

For example

```
icc2_shell> set_app_options -list {place_opt.flow.enable_dpa_ccd_power
 true}
```

Optimizing Timing Using Data Path Aware Clock Useful Skew

The current pre-CTS concurrent clock and data optimization (CCD) follows sequential calls of CCD and data optimization for achieving better timing quality of results (QoR). It derives offset by considering the scope of prepone or postpone available and implementation. However, the offsets derived by CCD in the `place_opt` stage are not data path aware (DPA).

When the timing is optimized on a timing path and zero slack is achieved, the path cannot be disturbed further for slack borrowing for its fanin or fanout paths, even if the other paths are critical for timing. This leaves the extra timing potential for zero slack path, which limits the design frequency.

Delay potential of an endpoint is computed dynamically based on sizing potential of the cells.

After computing the delay potential value of the endpoint, a negative path margin proportionate to the delay potential is applied.

- No path margins are applied for endpoints with delay potential less than `0.4`.
- For the rest of the endpoints, the applied path margin is multiplication of the delay potential of that endpoint by the `max_path_margin` (maximum of design worse negative slack (WNS) and `-50` ps) value.

  Note:

  The path margin is applied at the final call of CUS and removed while exiting the CUS call. The path margin is not applied to all the endpoints but applied only to the endpoints having paths with available delay potential.

  To enable the data path aware (DPA) clock useful skew (CUS) along with dynamic delay potential computation in the `place_opt` stage, set the `place_opt.flow.enable_dpa_ccd_timing_with_delay_potential` application option to `true`.

  For example

Feedback

```
icc2_shell> set_app_options -list
 {place_opt.flow.enable_dpa_ccd_timing_with_delay_potential true}
```

DPA CUS for timing feature improves the design frequency by over optimizing the paths having positive slack. It provides extra borrowable slack for paths with limiting frequency.

Note:

This feature impacts final calls of CUS in the `place_opt final_opto` stage.

Incremental Clock-Data Optimization

Pre-CTS concurrent clock and data optimization (CCD) contributes to approximately 10% of overall `place_opt` stage runtime. This runtime primarily comes from the medium and high-effort calls of compute useful skew (CUS) in the and the stage.

With incremental clock data (ICD) optimization, one call of the global CUS call is reduced in the `final_opto` stage of the `place_opt` stage. Instead, it introduces two local CUS calls, which focus on optimizing only certain critical endpoints for CUS. These CUS calls are not runtime intensive and result in overall runtime improvement.

To enable ICD in the `place_opt` flow, set the `place_opt.flow.enable_fast_cus_in_final_opto` application option to `true`.

For example

```
icc2_shell> set_app_options -name
 place_opt.flow.enable_fast_cus_in_final_opto -value true [default :
 false]
```

Timer-Based Predictive Concurrent Clock-Data Optimization

The post-CTS concurrent clock and data (CCD) engine follows sequential calls of CCD and data optimization for achieving better timing quality of results (QoR). This leads to cases where the paths with available timing potential are not used for fixing the maximum frequency of the design during CCD.

The IC Compiler II tool uses a timer-based approach for predictive CCD to improve the maximum frequency of the design. The tool creates easily fixable violations on neighboring paths and fix the critical paths by borrowing the extra generated slack.

Predictive CCD, also known as Useful Skew, enhances the post-CTS stage by leveraging timer-based potential to derive more realistic margins. It achieves this by:

- Providing more accurate estimations.
- Improving the correlation between the estimation of a cell arc delay and the delta delay from the timer.

Feedback

This feature helps map the predictability to the sizing potential of the path.

To enable predictive CCD for timing in the `clock_opt final_opto` flow, set the `ccd.enable_dpa_ccd -value` application option to `1`.

```
icc2_shell> set_app_options -name ccd.enable_dpa_ccd -value 1
```

Note:

This timer-based approach for predictive CCD impacts CCD (CG-CCD & WNS-CCD stage specifically) during the `clock_opt final_opto` stage. It impacts multi-vector CCD engine if multi-vector CCD is enabled.

Splitting Clock Cells

You can manually split clock cells that have DRC violations by using the `split_clock_cells -cells` command, as shown in the following example:

```
icc2_shell> split_clock_cells -cells [get_cells U1/ICG*]
```

The tool does not split the specified cells if

- They do not have DRC violations
- They have don't-touch, size-only, or fixed-placement attribute settings
- It is necessary to punch ports on the boundaries of power domains or blocks that have been identified with the `set_freeze_ports` command

After splitting a cell, the tool

- Names the new cells using the <original_cell_name>_split_<integer> naming convention
- Copies all the settings and constraints from the original cell to the newly created cells

Instead of specifying the cells to split, you can specify one or more collection of loads that are driven by the same driver by using the `-loads` option, as shown in the following example:

```
icc2_shell> set loads1 [get_pins I1/reg*/CK]
icc2_shell> set loads2 [get_pins I2/reg*/CK]
icc2_shell> split_clock_cells -loads [list $load1 $load2]
```

After splitting, each set of loads is driven by a newly created driver.

Balancing Skew Between Different Clock Trees

The IC Compiler II tool can automatically balance the skew between a group of clocks. The set of clocks considered during delay balancing is referred to as a *clock balance*

Feedback

group. You can define multiple clock balance groups. For each clock balance group, you can define a delay offset between the clocks. Together, the clock balance groups and their delay offset settings are referred to as *interclock delay balancing constraints*.

Note:

The tool cannot balance skew between a generated clock and other clocks.

To balance multiple clocks, perform the following steps:

1. Generate the interclock delay balancing constraints either by

   Manually defining the clock balance groups and their delay offsets, as described in Defining the Interclock Delay Balancing Constraints

   Having the tool generate them as described in Generating Interclock Delay Balancing Constraints Automatically

2. Balance the interclock delays as described in Running Interclock Delay Balancing.

Defining the Interclock Delay Balancing Constraints

To define interclock delay balancing constraints, use the `create_clock_balance_group` command. At a minimum, you must specify a name for the clock balance group and the clocks in the group.

- To specify a name for the clock balance group, use the `-name` option.
- To specify the clock trees in the group, use the `-objects` option with the `get_clocks` command. By default, the clock balance group is defined for the clocks in the current mode. To define a clock balance group for the clocks of a specific mode, use the `-objects` option with the `get_clocks -mode` command.

For example, to define a clock balance group named group1 that contains the clocks named clk1 and clk2 in the current mode, use the following command:

```
icc2_shell> create_clock_balance_group -name group1 \
   -objects [get_clocks {clk1 clk2}]
```

To define a clock balance group named group2 that contains all the clocks in the mode named m2, use the following command:

```
icc2_shell> create_clock_balance_group -name group2 \
   -objects [get_clocks -mode m2]
```

By default, the tool has a goal of zero delay offset between clocks. All clocks are balanced with the same insertion delay which usually is the longest insertion delay among the clocks. To specify a delay offset between clocks in the group, use the `-offset_latencies` option. By default, the delay offset applies to the current corner. To apply the delay offset to a specific corner, use the `-corner` option.

Feedback

For example, assume the design has three clocks named clk1, clk2, and clk3. If you want the insertion delay of clk1 and clk2 to be the same and the insertion delay of clk3 to be 100 less than that of clk1 and clk2, use the following command:

```
icc2_shell> create_clock_balance_group -name group3 \
   -objects [get_clocks {clk1 clk2 clk3}] \
   -offset_latencies {0 0 -100}
```

Reporting Clock Balance Groups

To report clock balance groups, use the `report_clock_balance_groups` command. The report lists the clock balance groups for all modes in all active scenarios.

Removing Clock Balance Groups

To remove clock balance groups, use the `remove_clock_balance_groups` command. You can either remove specific clock balance groups by specifying the clock balance groups or all clock balance groups by using the `-all` option. If you specify the clock balance groups to remove, they are removed from the current mode. However, if a specified group does not exist in the current mode, but does exist in another mode, it is removed from that mode. If you use the `-all` option, the clock balance groups are removed from all modes.

For example, to remove a previously defined clock balance group named group1 from the current mode, use the following command:

```
icc2_shell> remove_clock_balance_groups group1
```

To remove all clock balance groups from all modes, use the following command:

```
icc2_shell> remove_clock_balance_groups -all
```

Generating Interclock Delay Balancing Constraints Automatically

The IC Compiler II tool can automatically generate the interclock delay balancing constraints based on the timing relationships between the clocks. To automatically generate the interclock delay balancing constraints, use the following command:

```
icc2_shell> derive_clock_balance_constraints
```

This command identifies clocks that have interclock timing paths and places them in the same balance group. After running this command, use the `report_clock_balance_groups` command to report the generated interclock delay balancing constraints, as described in Reporting Clock Balance Groups. If necessary, you can modify the clock balance groups as described in Defining the Interclock Delay Balancing Constraints.

By default, the tool considers all timing paths when identifying the timing relationships between the clocks. To consider only those timing paths with slack less than a specified value, use the `-slack_less_than` option with the `derive_clock_balance_constraints` command.

Feedback

For example, to generate interclock balancing constraints only for paths with slack less than -0.2 ns, use the following command:

```
icc2_shell> derive_clock_balance_constraints -slack_less_than -0.2
```

Assume you run this command on a block for which clock A has a timing relationship only with clock B and the worst negative slack (WNS) of this group of timing paths is -0.1 ns and clock C has a timing relationship only to clock D and the WNS of this group of timing paths is -0.3 ns. The command considers only those timing paths with slack less than -0.2 ns, so it defines a single balance group that contains clocks C and D. Clocks A and B are not constrained because the timing paths between them have slack greater than -0.2 ns.

Running Interclock Delay Balancing

Before you perform interclock delay balancing, you must generate the interclock delay balancing constraints as described in Defining the Interclock Delay Balancing Constraints.

To perform interclock delay balancing, use the `balance_clock_groups` command. For multicorner-multimode designs, the tool performs interclock delay balancing on all active scenarios.

```
icc2_shell> balance_clock_groups
```

Performing Global-Route-Based Optimization Using Machine Learning Data

For designs with poor timing correlation between the preroute and postroute stages, you can improve the correlation by performing optimization after clock tree synthesis using machine learning (ML) data derived after detail routing.

The machine learning concept uses the following terminology:

- Labels, which are the outputs you are trying to predict
- Features, which are the inputs that affect the outputs
- Model, which is the relationship between the features (inputs) and the labels (outputs)
- Training, which is the process of learning the relationship between the features (inputs) and the labels (outputs), based on the data collected

Feedback

To perform global-route-based optimization using machine learning (ML) data derived from the postroute stage,

1. Collect machine learning features and labels, and train the model by performing the following steps in a specific iteration, iteration N:

   a. Perform clock tree synthesis and clock routing by completing the `build_clock` and `route_clock` stages of the `clock_opt` command.

   b. Specify the output directory for the machine learning data by using the new `est_delay.ml_delay_opto_dir` application option.

   ```
   icc2_shell> set_application_option \
      -name est_delay.ml_delay_opto_dir -value ./ML
   ```

   c. Enable integrated feature extraction by setting the new `est_delay.ml_delay_gre_mode` application option to `feature`.

   ```
   icc2_shell> set_application_option \
      -name est_delay.ml_delay_gre_mode -value feature
   ```

   d. Perform global-route-based optimization by using the `clock_opt -from final_opt` command.

   e. Perform detail routing.

   f. Collect labels associated with the features that were previously collected during the `clock_opt -from final_opt` command by using the `estimate_delay -training_labels` command, as shown in the following example:

   ```
   icc2_shell> estimate_delay -training_labels "label_1" \
      -output_dir ./ML
   ```

   The labels must be collected after detail routing, but before postroute optimization.

   g. Train the machine learning model for the collected features and label by using the `estimate_delay -train_model` command as shown in the following example:

   ```
   icc2_shell> estimate_delay -train_model "features_1 label_1" \
      -output_dir ./ML
   ```

   h. Continue with postroute optimization and the rest of the steps in the implementation flow.

Feedback

2. Use the machine learning data to improve correlation by performing the following steps in the subsequent iteration, iteration N+1:

   a. Perform clock tree synthesis and clock routing by completing the `build_clock` and `route_clock` stages of the `clock_opt` command.

   b. Specify the output directory for the machine learning data by using the new `est_delay.ml_delay_opto_dir` application option.

   ```
   icc2_shell> set_application_option \
      -name est_delay.ml_delay_opto_dir -value ./ML
   ```

   c. Enable the machine learning model, which was derived in the previous iteration, by setting the new `est_delay.ml_delay_gre_mode` application option to `enable`.

   ```
   icc2_shell> set_application_option \
      -name est_delay.ml_delay_gre_mode -value enable
   ```

   d. Perform global-route-based optimization with the machine learning data by using the `clock_opt -from final_opto` command.

   e. Perform detail routing.

   f. Disable the machine learning model after you complete all the detail routing steps by using the `estimate_delay -disable_model` command, as shown in the following example:

   ```
   icc2_shell> estimate_delay -disable_model -output_dir ./ML
   ```

   g. Continue with postroute optimization and the rest of the steps in the implementation flow.

When using this feature, ensure that

- The same scenarios are active at each step and each iteration of the flow
- The machine learning model is re-created if there are changes to the design, its environment, or the flow

Routing Clock Trees

After synthesizing and optimizing the clocks, you can detail route the clock nets by using the `route_group` command, as shown in the following example:

```
icc2_shell> route_group -all_clock_nets -reuse_existing_global_route true
```

When you set the `-reuse_existing_global_route` option of the `route_group` command to `true`, the detail router uses the existing clock global routes, which ensures better correlation.

Feedback

Alternatively, you can detail route the clock nets by using the `clock_opt` command, as shown in the following example:

```
icc2_shell> clock_opt -from route_clock -to route_clock
```

Inserting Via Ladders During Clock Tree Synthesis, Optimization, and Clock Routing

A via ladder is a stacked via that starts from the pin layer and extends into an upper layer where the router connects to it. Via ladders reduce the via resistance, which can improve performance and electromigration robustness.

The via ladder insertion flow during clock tree synthesis, optimization, and clock routing consists of the following steps:

1. Ensure that the via ladder rules are defined as described in Defining Via Ladder Rules.
2. Specify the via ladders that can be used for specific library pins by using the `set_via_ladder_candidate` command, as described in Specifying Via Ladder Candidates for Library Pins.
3. Define the via ladder constraints by using the `set_via_ladder_rules` command, as described in Defining Via Ladder Constraints.
4. Enable high-performance and electromigration via ladder insertion for critical paths during the `clock_opt` command by setting the `opt.common.enable_via_ladder_insertion` application option to `true`.
5. Perform clock tree synthesis and optimization by using the `clock_opt -to build_clock` command.
6. Insert the via ladders by using the `insert_via_ladders` command, as described in Inserting Via Ladders.
7. Route the clock nets by using the `clock_opt -from route_clock` command.

Marking Clocks as Propagated After Clock Tree Synthesis

If you run the `synthesize_clock_trees` or `clock_opt` command, after clock tree synthesis, the tool removes the ideal setting from all the synthesized clocks of all active scenarios and sets all the corresponding register clock pins as propagated.

If your block has scenarios that were not set as active before clock tree synthesis, set these scenarios as active by using the `set_scenario_status -active true`

Feedback

command and mark all clocks as propagated by using the `synthesize_clock_trees -propagate_only` command, as shown in the following example:

```
icc2_shell> set_scenario_status -active true [all_scenarios]
icc2_shell> synthesize_clock_trees -propagate_only
```

The `synthesize_clock_trees -propagate_only` command removes the ideal setting from the clocks; it does not perform clock tree synthesis.

Performing Postroute Clock Tree Optimization

When you detail route the clock nets of a block, its clock tree QoR can degrade due to the differences between the clock global routes used during clock tree synthesis and the clock detail routes. Clock tree QoR can further degrade when you detail route the signal nets due to coupling capacitance and crosstalk effects.

You can perform clock tree optimization on a postroute design by using the `synthesize_clock_trees -postroute` command. When you do so, you should specify the type of routing performed on the design by using the `-routed_clock_stage` option.

For example, to perform clock tree optimization on a design that has completed clock routing, use the following command:

```
icc2_shell> synthesize_clock_trees -postroute \
   -routed_clock_stage detail
```

To perform clock tree optimization on a design that has completed both clock and signal routing, use the following command:

```
icc2_shell> synthesize_clock_trees -postroute \
   -routed_clock_stage detail_with_signal_routes
```

Note:

When you enable concurrent clock and data optimization, including power or area recovery that uses this technique, the tool does not optimize the clock latency or skew during postroute clock tree optimization; it only fixes the logical DRC violations on the clock network.

Performing Voltage Optimization

The voltage level of a design is determined by the technology node (at device level) and the performance requirement. The IC Compiler II tool can perform voltage optimization, which tries to achieve better performance, power, and area (PPA) at a lower voltage level.

To perform voltage optimization, you need

- Scaled library panes

  Voltage optimization works on the fundamental principle of scaling the voltage, Therefore, scaled library panes with different voltages are required.

- A signoff methodology with a scaled library setup

The voltage optimization flow consists of the following high-level steps:

1. Perform placement and optimization at the original design voltage using the `place_opt` command.
2. Perform clock tree synthesis and optimization at the original design voltage by using the `clock_opt` command.
3. Associate a set of related library panes to form a scaling group by using the `define_scaling_lib_group` command.
4. Specify voltage ranges for specific corners by using the `set_vopt_range` command, as shown in the following example:

```
icc2_shell> set_vopt_range -corner C1 \
            -low_voltage 0.81 -high_voltage 1.0
icc2_shell> set_vopt_range -corner C2 \
            -low_voltage 0.81 -high_voltage 1.0
```

5. Specify a total negative slack (TNS) and power target and tolerance for voltage optimization by using the `set_vopt_target` command.

   The following example specifies a total negative slack target for voltage optimization and specifies that I/O paths should be excluded from the total negative slack costing:

```
icc2_shell> set_vopt_target -tns -1.0 -excludeIO
```

6. Perform voltage optimization by using the `voltage_opt` command.
7. Perform routing at the optimized voltage by using the `route_auto` command.
8. Perform postroute optimization at the optimized voltage by using the `route_opt` command.

Feedback

Marking Clock Trees as Synthesized

To prevent the IC Compiler II tool from modifying them, you can mark existing clock trees in your design as synthesized clock tree by using the `mark_clock_trees` command as shown in Table 21.

Table 21 Using the mark_clock_trees Command

To do this	Use this option
Mark only the clock trees of specific clocks By default, the command marks all clocks defined by the `create_clock` and `create_generated_clock` commands in all modes of all active scenarios	`-clocks` *`clock_list`*
Mark the clock tree as synthesized This is the default behavior	`-synthesized`
Remove the synthesized attribute settings from the clock trees	`-clear`
Apply the `dont_touch` attribute setting on the clock trees	`-dont_touch`
Remove the `dont_touch` attribute settings from the clock trees	`-clear -dont_touch`
Propagate the nondefault clock routing rules specified by the `set_clock_routing_rules` command	`-routing_rules`
Propagate the clock cell spacing rules specified by the `set_clock_cell_spacing` command	`-clock_cell_spacing`
Mark the clock sinks as fixed	`-fix_sinks`
Freeze the routing of the clock nets	`-freeze_routing`

The tool traverses the clock trees and marks the clock trees as specified. Clock tree traversal continues until it finds an exception pin or a default sink pin.

Removing Clock Trees

To remove the buffers and inverters on a clock tree, use the `remove_clock_trees` command. The `remove_clock_trees` commands traverses the clock tree from its root to its sinks and removes all buffers and inverters, except those with `dont_touch` or `size_only` attributes. Cells beyond clock tree exceptions are considered part of the clock tree, but cells beyond a clock-to-data pin are considered part of the data path and are

Feedback

not removed. In addition to removing the buffers and inverters, the `remove_clock_trees` command resets the attributes related to clock tree synthesis.

Note:

The `remove_clock_trees` command does not support clock mesh nets.

By default, this command removes the buffers and inverters from all clock trees in all modes. To remove the buffers and inverters from specific clock trees, use the `-clocks` option to specify the clock trees. By default, when you use the `-clocks` option, the clocks are selected from the current mode. To select clocks from a specific mode, use the `get_clocks -mode` command to select the clocks.

For example, to remove only the clock tree in the current mode named my_clk, use the following command:

```
icc2_shell> remove_clock_trees -clocks [get_clocks my_clk]
```

Table 22 shows how clock tree removal is affected by the structure of the clock tree.

Table 22 Clock Tree Removal Behavior

Object	Impact on clock tree removal
Boundary cell	Removed.
Cells on don’t touch net	Preserved.
Don’t touch cell	Preserved.
Fixed cell	Preserved.
Generated clock	Preserved, if generated clock is defined on buffer/inverter pin. Traversal and clock tree removal continue past the generated clock.
Guide buffer	Removed.
Integrated clock-gating (ICG) cell	Preserved. Traversal (and clock tree removal) continues past the integrated clock-gating cell.
Block abstraction model	Preserved. Traversal (and clock tree removal) continues past the block abstraction model.
Inverter	Removed in pairs only. If a clock tree contains a single inverter, it is not removed.
Isolation cell	Preserved.
Level shifter	Preserved.

Feedback

Table 22 Clock Tree Removal Behavior (Continued)

Object	Impact on clock tree removal
Three-state buffer	Preserved. Traversal (and clock tree removal) stops at the three-state buffer.
Buffer or inverter added beyond exception	Removed.

By default, the `remove_clock_trees` command removes the detail route shapes of the clock nets it removes. However, you can preserve the route shapes of the clock nets by setting the `shape_use` attribute of the clock nets to `user_route`.

Optimizing Nworst Skew Framework for Multithreaded Clock Tree

The multithreaded clock tree optimization (MTCTO) engine optimizes skew by minimizing the delay difference between the longest and shortest paths (global skew). If the optimization gets stuck on the shortest path, the remaining paths remain under-optimized. To avoid early bail out during skew optimization, the tool optimizes the skew of the nworst paths instead of focusing solely on the shortest path.

To enable the nworst skew optimization, set the `cts.optimize.enable_nworst_skew_optimization` application option to `true`:

```
icc2_shell> set_app_options -name
 cts.optimize.enable_nworst_skew_optimization -value true
```

Implementing Multisource Clock Trees

The following topics introduce the different types of multisource clock trees and provides detailed information about the different multisource clock tree implementation flows and steps:

- Introduction to Multisource Clock Trees Structures
- Implementing a Regular Multisource Clock Tree
- Implementing a Regular Multisource Clock Tree Using Integrated Tap Assignment
- Implementing a Regular Multisource Clock Tree With an H-Tree-Only Global Clock Tree Structure
- Implementing MSCTS Multiple H-Tree
- Analyzing the H-Tree Quality of Results

Feedback

- Enabling Timing-Aware Latency Implementation in Structural Multisource Clock Tree Synthesis
- Implementing Power Aware Flexible H-Tree
- Implementing a Structural Multisource Clock Tree
- H-Tree Support in Physical Hierarchy Flat-First Section Designs
- Implementing a Structural Multisource Clock Tree Using Integrated Subtree Synthesis
- Improving the Correlation Between the SMSCTS and CCD
- Early CCD-iSMSCTS Flow for Structural Multisource Clock Tree Synthesis
- Inserting Clock Drivers
- Synthesizing the Global Clock Trees
- Creating Clock Straps
- Routing to Clock Straps
- Analyzing the Clock Mesh
- Performing Automated Tap Insertion and H-Tree Synthesis
- Specifying Tap Assignment Options and Settings
- Building the Local Clock Subtree Structures
- Performing Irregular Global Tree Synthesis

Introduction to Multisource Clock Trees Structures

A multisource clock tree is a custom clock structure that has more tolerance to on-chip variation and has better performance across corners than traditional clock tree structures.

These custom clock trees consist of

- A global clock structure, which includes
  - The clock root
  - A global clock tree, which is usually an H-tree structure
  - Clock mesh drivers
  - A clock mesh
- Local subtrees that can be driven

Feedback

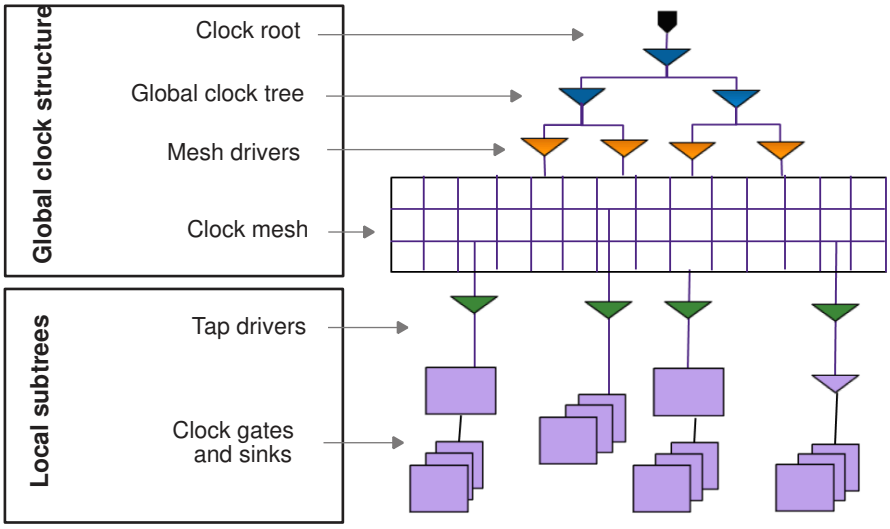
- By a predefined set of drivers, called tap drivers, that are connected to the clock mesh, as shown in Figure 45.

  Because the subtrees are built using regular clock tree synthesis commands such as the `synthesize_clock_trees` or `clock_opt` command, such a structure is called a regular multisource clock tree.

- Directly from multiple points of the clock mesh, as shown in Figure 46.

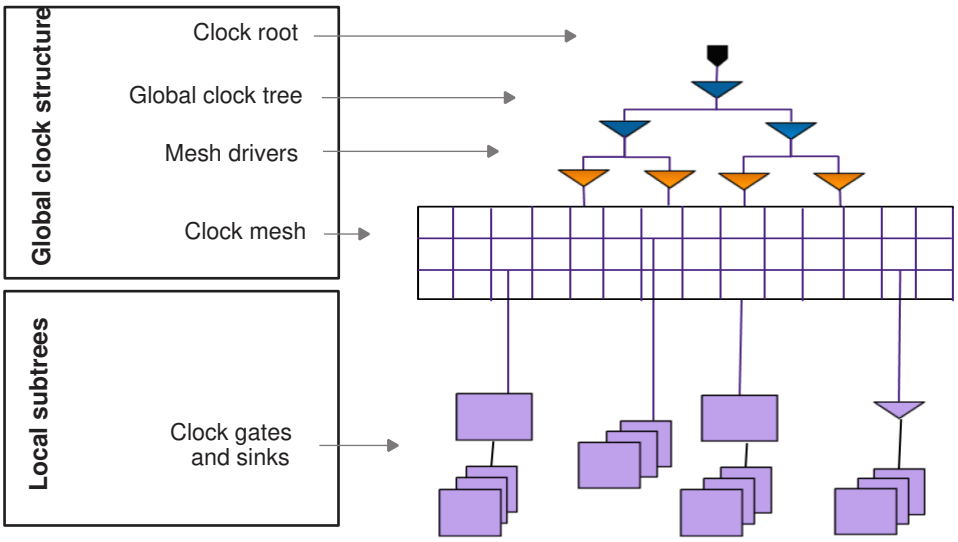
  Because the subtrees are built preserving the user-defined structure and optimized by merging and splitting the clock cells, such a structure is called a structural multisource clock tree.

Figure 45 Regular Multisource Clock Tree



Feedback

Figure 46 Structural Multisource Clock Tree



Implementing a Regular Multisource Clock Tree

To implement a regular multisource clock tree, use the following steps:

1. Specify your clock tree constraints and settings.
2. Perform the following steps for multivoltage designs:
   a. Enable multivoltage support by setting the `cts.multisource.enable_full_mv_support` application option to `true`.
   b. Enable the addition of physical feedthrough cells, which belong to a voltage area physically but not logically, by setting the `opt.common.allow_physical_feedthrough` application option to `true`.
   c. Run the `check_mv_design` command and fix all multivoltage issues.
3. Enable cell electromigration fixing by setting the `cts.multisource.cell_em_aware` application option to `true`.
4. Insert the tap drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
5. Create the clock mesh by using the `create_clock_straps` command, as described in Creating Clock Straps.

Feedback

6. Insert the mesh drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
7. Build the global clock tree structure by using the `synthesize_multisource_global_clock_trees` command, as described in Synthesizing the Global Clock Trees.
8. Route the connections between the clock mesh and the tap drivers by using the `route_clock_straps` command, as described in Routing to Clock Straps.
9. Analyze the clock mesh by using the `analyze_subcircuit` command, as described in Analyzing the Clock Mesh.
10. Specify options and settings for tap assignment as described in Specifying Tap Assignment Options and Settings.
11. Perform tap assignment by using the `synthesize_multisource_clock_taps` command, which
    - Merges equivalent clock cells to remove any artificial boundaries between clusters of sinks
    - Assigns endpoints to the closest tap driver and split cells along the path honoring any sink groups defined
    - Copies the UPF and SDC constraints and the user-specified attributes onto the newly created cells across the active scenarios
12. Synthesize the entire clock tree, from the clock root, by using the following command:

    ```
    icc2_shell> clock_opt -from build_clock -to route_clock
    ```

    During this step, the tool builds the local subtrees that are driven by the tap drivers. It also fixes DRC violations beyond ignore exceptions of the multisource clock tree, if applicable. You can also synthesize other clocks in the design that are not synthesized.

    For more information about the `clock_opt` command, see Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command.
13. Analyze the clock tree results as described in Analyzing the Clock Tree Results.

Implementing a Regular Multisource Clock Tree Using Integrated Tap Assignment

A regular multisource clock tree has tap drivers that are driven by a clock mesh and drive clock-gating cells and sinks. If you insert these tap drivers before you run the `place_opt` command, the tool can assign sinks to these drivers such that it improves the overall timing QoR of the block. It also allows the tool to improve the timing of clock-gating paths.

Feedback

To perform regular multisource clock tree synthesis integrated with placement and optimization, use the following steps:

1. Specify the placement and optimization constraints and settings.
2. Specify the clock tree constraints and settings.
3. Perform the following steps for multivoltage designs:
   a. Enable multivoltage support by setting the `cts.multisource.enable_full_mv_support` application option to `true`.
   b. Enable the addition of physical feedthrough cells, which belong to a voltage area physically but not logically, by setting the `opt.common.allow_physical_feedthrough` application option to `true`.
   c. Run the `check_mv_design` command and fix all multivoltage issues.
4. Enable cell electromigration fixing by setting the `cts.multisource.cell_em_aware` application option to `true`.
5. Insert the tap drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
6. Create the clock mesh by using the `create_clock_straps` command, as described in Creating Clock Straps.
7. Insert the mesh drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
8. Build the global clock tree structure by using the `synthesize_multisource_global_clock_trees` command, as described in Synthesizing the Global Clock Trees.
9. Route the connections between the clock mesh and the tap drivers by using the `route_clock_straps` command, as described in Routing to Clock Straps.
10. Analyze the clock mesh by using the `analyze_subcircuit` command, as described in Analyzing the Clock Mesh.
11. Specify options and settings for tap assignment as described in Specifying Tap Assignment Options and Settings.
12. Enable integrated tap assignment by setting the `place_opt.flow.enable_multisource_clock_trees` application option to `true` and run the `place_opt` command.

```
icc2_shell> set_app_options \
   -name place_opt.flow.enable_multisource_clock_trees -value true
icc2_shell> place_opt
```

Feedback

13. Synthesize the entire clock tree, from the clock root, by using the following command:

    ```
    icc2_shell> clock_opt -from build_clock -to route_clock
    ```

    During this step, the tool builds the local subtrees that are driven by the tap drivers. It also fixes DRC violations beyond ignore exceptions of the multisource clock tree, if applicable. You can also synthesize other clocks in the design that are not synthesized.

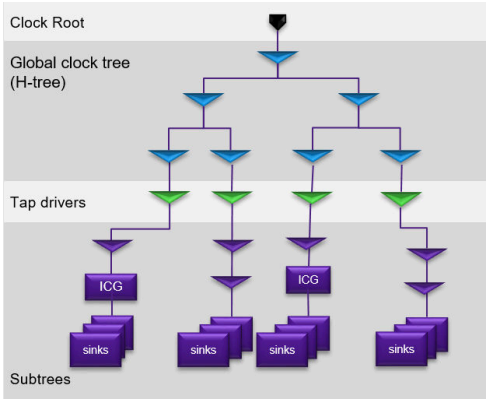
    For more information about the `clock_opt` command, see Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command.

14. Analyze the clock tree results as described in Analyzing the Clock Tree Results.

Implementing a Regular Multisource Clock Tree With an H-Tree-Only Global Clock Tree Structure

The following figure shows a regular multisource clock where the global clock structure does not have a clock mesh. The global clock structure consists only of an H-tree, which directly drives the tap drivers.

Figure 47 Regular Multisource Clock Tree With an H-Tree-Only Global Clock Structure



Feedback

To implement a regular multisource clock tree structure with an H-tree-only global clock tree structure,

1. Specify your clock tree constraints and settings.
2. Perform the following steps for multivoltage designs:
   a. Enable multivoltage support by setting the `cts.multisource.enable_full_mv_support` application option to `true`.
   b. Enable the addition of physical feedthrough cells, which belong to a voltage area physically but not logically, by setting the `opt.common.allow_physical_feedthrough` application option to `true`.
   c. Run the `check_mv_design` command and fix all multivoltage issues.
3. Enable cell electromigration fixing by setting the `cts.multisource.cell_em_aware` application option to `true`.
4. Specify the required settings by using the `set_regular_multisource_clock_tree_options` command and perform tap insertion and global clock tree (H-tree) synthesis by using the `synthesize_regular_multisource_clock_trees` command, as described in Performing Automated Tap Insertion and H-Tree Synthesis.
5. Specify options and settings for tap assignment, as described in Specifying Tap Assignment Options and Settings.
6. Perform tap assignment by using the `synthesize_multisource_clock_taps` command, which
   - Merges equivalent clock cells to remove any artificial boundaries between clusters of sinks
   - Assigns endpoints to the closest tap driver and splits cells along the path honoring any sink groups defined
   - Copies the UPF and SDC constraints and the user-specified attributes onto the newly created cells across the active scenarios
7. Synthesize the entire clock tree, from the clock root, by using the following command:

   ```
   icc2_shell> clock_opt -from build_clock -to route_clock
   ```

   During this step, the tool builds the local subtrees that are driven by the tap drivers. It also fixes DRC violations beyond ignore exceptions of the multisource clock tree, if applicable. You can also synthesize other clocks in the design that are not synthesized.

Feedback

For more information about the `clock_opt` command, see Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command.

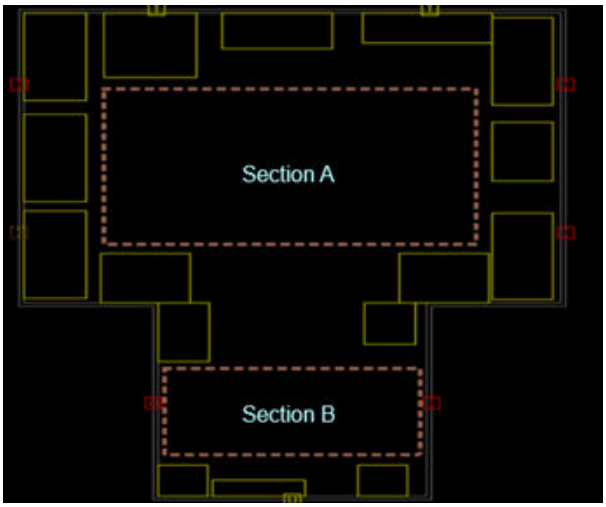
8. Analyze the clock tree results, as described in Analyzing the Clock Tree Results.

Implementing MSCTS Multiple H-Tree

To improve clock QoR, latency, and skew, you can build multiple global trees in different sections of the floorplan by using the `-htree_sections` option with the `set_regular_multisource_clock_tree_options` command.

The following figure shows a regular multisource clock where the global clock structure does not have a clock mesh. The global clock structure consists only of an H-tree, which directly drives the tap drivers.

Figure 48 MSCTS Multiple H-Tree

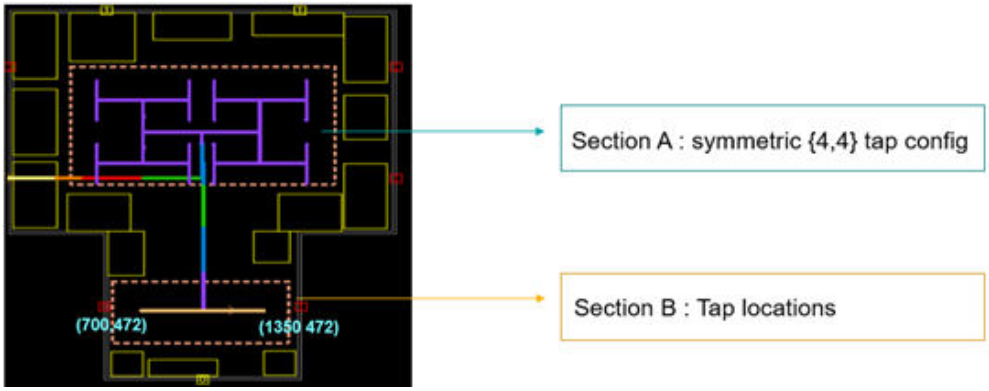


You can define the different sections in the floorplan by using a symmetric configuration of tap drivers or only tap locations by coordinates, as shown in the following example:

```
set_regular_multisource_clock_tree_options - htree_sections [ list \
[list -section_name "sectionA_name" -tap_boundary {{315.8800 1275.8000}
 {1750.0400 2187.1000}} -prefix "sectionA" -tap_boxes {4 4}] \
[list -section_name "sectionB_name" -tap_locations {{700472} {1350 472}}
 -prefix "section"]]
```

Feedback

Figure 49 MSCTS Multiple H-Tree



Analyzing the H-Tree Quality of Results

The `report_global_clock_tree_qor` command analyzes the Quality of Results (QoR) for H-tree style clock trees. You can generate the QoR data right after the H-tree is created. By providing detailed, level-based reporting, this command helps you identify potential issues and optimize their clock tree design.

The following are the benefits of analyzing the H-Tree QoR:

- Enables detailed analysis of H-tree quality immediately after its creation.
- Provides multi-metric reporting for latency and levelized metrics.
- Lets you focus on specific levels of H-tree for targeted analysis.
- Supports enhanced decision-making for clock tree optimization.

To enable the H-Tree quality analysis, use the following steps:

1. To generate a latency report for the H-tree, use the `report_global_clock_tree_qor` command with the `-type latency` switch.

   Example:

   ```
   icc2_shell> report_global_clock_tree_qor -clock -net <global tree root
    net> -type latency
   ```

2. To produce a detailed level-based report for the H-tree, use the `-type levelized` switch.

   Example:

   ```
   icc2_shell> report_global_clock_tree_qor -clock -net <global tree root
    net> -type levelized
   ```

Feedback

3. To generate QoR data for a specific level of the H-tree, use the `-level` switch. Valid values range from `0` (root driver level) to the tap input pins level.

   Example:

   ```
   icc2_shell> report_global_clock_tree_qor -clock -net <global tree root
    net> -type levelized -level 2
   ```

4. To refine the report output, add optional switches such as `-modes`, `-corners`, `-scenarios`, `-largest`, `-smallest`, `-all`, and `-significant_digits`.

   Example:

   ```
   icc2_shell> report_global_clock_tree_qor -clock -net <global tree root
    net> -type levelized -level 1 -modes modeA -corners corner
   ```

The `report_global_clock_tree_qor` command generates detailed reports based on the specified type and level. These reports include metrics such as latency, level-based QoR data, and other relevant details.

Enabling Timing-Aware Latency Implementation in Structural Multisource Clock Tree Synthesis

The Structural Multisource Clock Tree Synthesis (SMSCTS) with timing awareness feature uses timing slack information from the post `compile_fusion` or `place_opt` stage to reduce area in clock paths that do not impact critical timing startpoints.

During this process, the tool assigns each sink within the structural subtree, the worst timing slack (Q-slack) from all timing paths it runs. It then identifies sinks with positive slack. For these non-timing-critical sinks, the tool relaxes latency optimization, allowing user-defined network delay targets to be exceeded if necessary. This approach reduces clock area and power consumption while maintaining timing neutrality.

The following are the benefits of enabling timing-aware latency in SMSCTS:

- Reduces aggressive latency implementation at specific sinks and improves clock area.
- Maintains neutral or improved timing while optimizing for clock power.
- Improves overall clock tree quality by balancing power and timing trade-offs.

To enable timing awareness in SMSCTS:

1. Set the `cts.multisource.subtree_synthesis_timing_aware_flow` application option before running the `compile_fusion` or `place_opt final_place` step.

   ```
   icc2_shell> set_app_options -name
    cts.multisource.subtree_synthesis_timing_aware_flow true
   ```

Feedback

Note:

The `cts.multisource.subtree_synthesis_timing_aware_flow` application option must be retained throughout the `clock_opt build_clock` stage.

2. Continue with your standard flow, ensuring you have set the application option before the relevant steps (`compile_fusion` or `place_opt final_place` and `clock_opt build_clock`).

Implementing Power Aware Flexible H-Tree

The tool derives tap driver configurations by adding power awareness in the flexible H-Tree synthesis flow, which is optimal in terms of total clock power and overall clock latency.

This requires at least one of the dynamic power scenarios to be defined and active during H-tree Synthesis. In case of multiple scenarios, the worst active scenario is considered for power costing.

The initial tap configurations are generated based on floorplan shape, size, sink distribution, placement, routing constraints, and or each tap configuration. This helps to estimate the latency and power for each subtree, and then tabulates and compares the results of different tap configurations to determine the best solution for power aware flexible H-Tree synthesis.

To enable power aware flexible H-tree synthesis, set the `cts.multisource.power_aware_flexible_htree_synthesis` application option to `true`.

Implementing a Structural Multisource Clock Tree

In a structural multisource clock tree, the subtrees are driven directly by the clock mesh or through predefined drivers. These drivers are in turn driven by global clock distribution structure such as a clock mesh and H-tree. The local subtrees of a structural multisource clock tree are optimized by clock cell merging, splitting, sizing, and relocation, while preserving the levels of the user-defined clock tree structure.

Feedback

To implement a structural multisource clock tree, perform the following steps:

1. Specify your clock tree constraints and settings.
2. For multivoltage designs,
   - Enable multivoltage support by using the following application option setting:

     ```
     icc2_shell> set_app_options \
        -name cts.multisource.enable_full_mv_support -value true
     ```

   - Enable the addition of physical feedthrough cells, which belong to a voltage area physically but not logically, by using the following application option setting:

     ```
     icc2_shell> set_app_options \
        -name opt.common.allow_physical_feedthrough -value true
     ```

   - Run the `check_mv_design` command and fix all multivoltage issues
3. Enable cell electromigration fixing by setting the `cts.multisource.cell_em_aware` application option to `true`.
4. Create the clock mesh by using the `create_clock_straps` command, as described in Creating Clock Straps.
5. Insert the mesh drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
6. Build the global clock tree structure by using the `synthesize_multisource_global_clock_trees` command, as described in Synthesizing the Global Clock Trees.
7. Model the delays of the mesh nets using `set_annotated_delay` and `set_annotated_transition` commands.

   The loads of the clock mesh are not finalized until the local subtrees are synthesized in step 6 and routed to the mesh in step 7. Therefore the clock mesh can only be analyzed after you complete those steps. However, to prevent the tool from seeing a very large delay through the clock mesh when synthesizing the local subtrees, annotate a realistic delay and transition value on the clock mesh net.
8. Build the local subtrees by using the `synthesize_multisource_clock_subtrees` command, as described in Building the Local Clock Subtree Structures.
9. Route the connections between the clock mesh and the local subtrees by using the `route_clock_straps` command, as described in Routing to Clock Straps.
10. Analyze the clock mesh by using the `analyze_subcircuit` command, as described in Analyzing the Clock Mesh.

Feedback

11. Synthesize the entire clock tree, from the clock root, by using the `clock_opt -from build_clock -to route_clock` command.

    During this step, the tool fixes DRC violations beyond ignore exceptions of the multisource clock tree, if applicable. You can also synthesize other clocks in the design that are not synthesized.

12. Analyze the clock tree results as described in Analyzing the Clock Tree Results.

H-Tree Support in Physical Hierarchy Flat-First Section Designs

To support H-tree in the flat first physical hierarchy flow, use the `-uncommitted_hierarchy_boundary` option with the `set_regular_multisource_clock_tree_options` command. You can define H-tree sections spanning over multiple uncommitted physical hierarchies.

For example:

```
icc2_shell> set_regular_multisource_clock_tree_options -clock clk -net
 [get_nets clk] \
-tap_boxes { 4 4} \
-uncommitted_hierarchy_boundary {{"BlockA_LogHierName " {{1500 500} {1500
 3000} {3000 3000} {3000 500}}} {"BlockB_LogHierName" {{3100 1000}{3100
 4200}{4100 4200}{4100 1000}}} } \
-tap_lib_cells [get_lib_cells abc/libcell1] \
-htree_lib_cells [get_lib_cells def/libcell2] \
-topology {htree_only} \
-htree_sections [list \
[list -section_name A -prefix "SA" -tap_boundary $A_bndry -tap_boxes{2
 2} ] \
[list -section_name B -prefix "SB" -tap_boundary $B_bndry -tap_boxes{2
 2} ]

icc2_shell> synthesize_regular_multisource_clock_trees
```

In flat first H-tree flow, the `set_regular_multisource_clock_tree_options` command uses this information to insert clock drivers in the correct logical hierarchies corresponding to physical hierarchies while the design remains flat. The tool identifies the overlap of tap cells and H-tree repeaters with their respective physical blocks, which are introduced later in the flow based on the specified coordinates of uncommitted blocks. The tool inserts the cells and nets of the global tree in logical hierarchies based on the physical boundaries of the top-level design. Cells that do not overlap with any physical hierarchy boundary are inserted in the top-level hierarchy.

Ensure that the provided H-tree sections are reasonable in terms of physical hierarchy, connectivity, clock, and timing quality of results (QoR). Unreasonable input might lead to non-optimal tap locations. The tool does not optimize tap locations across multiple sections for better clock or timing QoR.

Feedback

By default, the tool constructs H-trees by generating solutions for each level, selecting a repeater that is sufficient to meet the design rule check (DRC). It then builds the entire solution tree in a bottom-up manner. Among these solutions, the tool selects the one with the lowest latency. However, this approach does not guarantee the solution with the lowest delay.

Additionally, to achieve targeted H-tree QoR improvements, you can enable the `cts.multisource.htree_explore_all_repeater_solutions` and `cts.multisource.htree_single_repeater_at_node` application options, which are independent of each other:

- H-tree improvement using all repeater solution: To evaluate all repeaters for each node and generate a solution if the repeater meets the DRC constraints, set the `cts.multisource.htree_explore_all_repeater_solutions` application option to `true` as follows:

  ```
  icc2_shell> set_app_options -name
   cts.multisource.htree_explore_all_repeater_solutions -value true
  ```

  This ensures that all possible solutions with all repeaters are available at the top-level. Using this application option, the tool selects the solution with the least H-tree latency at the top-level, thereby ensuring latency-wise best solution.

  Note:
  Runtime degradation is expected when a larger number of library cells are specified in the H-tree library cell collection.

- H-tree improvement using a single repeater solution: By default, the tool constructs H-trees considering logical DRC, latency metrics, and the number of levels in the H-tree. The repeaters per level depend on these metrics.

  During H-tree synthesis, the tool computes all viable DRC-compliant solutions for building the H-tree to the tap drivers and selects the one with minimal latency. As a result, the H-tree can be synthesized with levels that include two repeaters at one steiner node, placed adjacently, each driving loads in separate directions of the downstream tree.

  To avoid routing issues at the reference cell pins and avoid placing two H-tree repeaters at the same node, set the `cts.multisource.htree_single_repeater_at_node` application option to `true` as follows:

  ```
  icc2_shell> set_app_options -name
   cts.multisource.htree_single_repeater_at_node -value true
  ```

  Note:
  Placing a single repeater at each node may increase the number of levels in the H-tree to ensure DRC, which may affect the H-tree latency.

Feedback

Implementing a Structural Multisource Clock Tree Using Integrated Subtree Synthesis

In a structural multisource clock tree, the subtrees are driven directly by the clock mesh or through predefined drivers. These drivers are, in turn, driven by a global clock distribution structure such as a clock mesh or an H-tree. By using the `place_opt` command to synthesize and optimize the local subtrees of a structural multisource clock tree, you can improve the timing QoR of the design.

To implement a structural multisource clock tree using the integrated subtree synthesis capabilities, perform the following steps:

1. Specify your clock tree constraints and settings.
2. For multivoltage designs,
   - Enable multivoltage support by using the following application option setting:

     ```
     icc2_shell> set_app_options \
        -name cts.multisource.enable_full_mv_support -value true
     ```

   - Enable the addition of physical feedthrough cells, which belong to a voltage area physically but not logically, by using the following application option setting:

     ```
     icc2_shell> set_app_options \
        -name opt.common.allow_physical_feedthrough -value true
     ```

   - Run the `check_mv_design` command and fix all multivoltage issues
3. Enable multicorner optimization by setting the `cts.multisource.enable_multi_corner_support` application option to `true`. The default is `false`.

   By default, the tool uses the worst corner associated with the mode in which the subtree options are defined. When you enable this feature, the tool considers all corners associated with the mode in which the subtree options are defined.
4. Enable cell electromigration fixing by setting the `cts.multisource.cell_em_aware` application option to `true`.
5. Create the clock mesh by using the `create_clock_straps` command, as described in Creating Clock Straps.
6. Insert the mesh drivers by using the `create_clock_drivers` command, as described in Inserting Clock Drivers.
7. Build the global clock tree structure by using the `synthesize_multisource_global_clock_trees` command, as described in Synthesizing the Global Clock Trees.

Feedback

8. Model the delays of the mesh nets by using the `set_annotated_delay` and `set_annotated_transition` commands.
9. Specify settings for local subtree synthesis by using the `set_multisource_clock_subtree_options` command.

   You must specify

   - The clock for synthesizing the subtrees by using the `-clock` option
   - The drivers of the subtrees to be synthesized by using the `-driver_objects` option

   Optionally you can

   - Prevent the tool from merging specific clock tree cells by using the `-dont_merge_cells` option
   - Balance the levels of the local subtree by using the `-balance_levels true` option and specify a target number of levels, which applies to all sinks of the local subtree, by using the `-target_level` option

     Optionally, you can apply a different target number of levels to specific sink pins and clock balance points by using the `set_multisource_clock_subtree_constraints` command with the `-pins` and `-target_level` options.
   - Specify a maximum total wire delay from any subtree driver to any of its sinks by using the `-max_total_wire_delay` option and the corner it applies to by using the `-corner` option
   - Enable the reordering of clock-gating cells by using the `-enable_icg_reordering true` option

     When you enable this feature, the tool swaps the position of a clock-gating cell with the buffer that is driving the clock-gating cell. Then, the buffer is gated by the clock-gating cell, thereby reducing the dynamic power consumption.

     To perform reordering, you must

     - Enable the scenarios used for structural multisource clock tree synthesis
     - Annotate the switching activity for the enable pins of the clock-gating cells

     You can prevent specific clock-gating cells from being reordered by using the `set_multisource_clock_subtree_constraints` command with the `-cells` and `-ignore_for_icg_reordering` options.

   To report the settings you specified, use the `report_multisource_clock_subtree_options` command. To remove the settings you specified, use the `remove_multisource_clock_subtree_options` command.

Feedback

10. Enable integrated structural multisource clock tree synthesis by setting the `place_opt.flow.enable_multisource_clock_trees` application option to `true`, and build the local subtree by running the `place_opt` command.

```
icc2_shell> set_app_options \
   -name place_opt.flow.enable_multisource_clock_trees -value true
icc2_shell> place_opt
```

11. Route the connections between the clock mesh and the local subtrees by using the `route_clock_straps` command, as described in Routing to Clock Straps.

12. Analyze the clock mesh by using the `analyze_subcircuit` command, as described in Analyzing the Clock Mesh.

13. Enable integrated structural multisource clock tree synthesis by setting the `clock_opt.flow.enable_multisource_clock_trees` application option to `true`, and synthesize the entire clock tree from the clock root by using the `clock_opt -from build_clock -to route_clock` command.

```
icc2_shell> set_app_options \
   -name clock_opt.flow.enable_multisource_clock_trees -value true
icc2_shell> clock_opt -from build_clock -to route_clock
```

    During this step, the tool incrementally optimizes the existing subtrees.

14. Analyze the clock tree results as described in Analyzing the Clock Tree Results.

Improving the Correlation Between the SMSCTS and CCD

To improve the correlation between the structural multisource clock tree synthesis (SMSCTS) and concurrent clock and data (CCD), the IC Compiler II tool supports the enhanced CCD engine with more SMSCTS specific offsets. This improves the CCD balance point implementation and timing after the SMSCTS step in the `clock_opt` command.

To enable the improved correlation between the SMSCTS and CCD in the `place_opt` command and `build_clock` stage of the `clock_opt` command, set the `cts.multisource.enable_subtree_synthesis_aware_ccd` application option to `true`.

The improved correlation between the SMSCTS and CCD:

- Improves the overall latency implementation after SMSCTS in the `clock_opt` command.
- Reduces long and short path violations.
- Improves timing after the `clock_opt` command with CCD offsets.

Feedback

Early CCD-iSMSCTS Flow for Structural Multisource Clock Tree Synthesis

Early concurrent clock and data–integrated structural multisource clock tree synthesis (CCD-iSMSCTS) lets you use concurrent clock and data optimization (also known as Useful Skew or CCD) during the `initial_opto` stage of `compile_fusion` or `place_opt on` clocks synthesized with integrated structural multisource clock tree synthesis (iSMSCTS).

This feature lets CCD generate SMSCTS–aware and level-friendly useful skew offsets for timing endpoints during the `initial_opto` stage. It introduces a new step, called Buffer Adjust, in the integrated structural multisource clock tree synthesis (iSMSCTS) flow during the `compile_fusion` or `place_opt final_place` stage. The Buffer Adjust step occurs after the pre-process step and before the merge step during subtree synthesis. This step is performed before placement in the `final_place` stage. During this step, the tool adds or removes repeaters to modify the clock subtree for each sink, ensuring that the number of levels match the useful skew offsets generated during `initial_opto` CCD.

The Buffer Adjust (`buffer_adjust`) step can also make the SMSCTS subtree deeper and split clock-gate at lower levels thereby saving power.

By default, the buffer delay value for each timing endpoint is determined using CCD. This value helps the tool decide the number of buffer levels to insert. You can modify the buffer delay by setting the `cts.multisource.subtree_buf_delay_for_cus_prepone_postpone` application option.

After the Buffer Adjust and subsequent merge steps, the flow performs a design rule check (DRC) fixing step to resolve any issues within the subtrees before moving to the placement stages. When placement is complete, the flow runs a second round of iSMSCTS that includes DRC fixing and an offset guidance handshake with the `compile_fusion` or `place_opt final_opto` step.

Ensure that CCD is enabled during the `compile_fusion` and `place_opt` stages by setting the `compile.flow.enable_ccd` and `place_opt.flow.enable_ccd` application options. These options are enabled (set to `true`) by default.

To enable the integrated structural MSCTS flow, set the `compile.flow.enable_multisource_clock_trees` and `place_opt.flow.enable_multisource_clock_trees` application options to `true`.

```
icc2_shell> set_app_options -name
 compile.flow.enable_multisource_clock_trees -value true

icc2_shell> set_app_options -name
 place_opt.flow.enable_multisource_clock_trees -value true
```

Feedback

To enable the early CCD-SMSCTS flow, set the `cts.multisource.integrated_subtree_synthesis_early_ccd_support` application option to `true`.

```
icc2_shell> set_app_options -name
 cts.multisource.integrated_subtree_synthesis_early_ccd_support -value
 true
```

Note:

You must enable the `cts.multisource.integrated_subtree_synthesis_early_ccd_support` application option throughout the flow from `compile_fusion initial_opto` until the `clock_opt build_clock` stage.

Configuring the Compile initial_opto Stage

The Early CCD-iSMSCTS flow enables the CCD engine during the `initial_opto` stage.

Before running the `initial_opto` stage, set the CCD resolution and specify the maximum prepone and postpone limits. This ensures that the CCD engine can derive the required offsets.

To enable CCD, set the `compile.flow.enable_ccd` and `place_opt.flow.enable_ccd` application options to `true`.

```
icc2_shell> set_app_options -list {compile.flow.enable_ccd true}
```

```
icc2_shell> set_app_options -list {place_opt.flow.enable_ccd true}
```

In this flow, the tool determines prepones and postpones according to the clock structure. The CCD engine can generate skew offsets at intermediate points within the clock tree, not only at the leaf nodes.

The tool applies these offsets in a repeater-based manner—postpones are implemented by inserting repeaters, and prepones are implemented by removing or reparenting existing clock repeaters.

Configuring the Compile final_place Stage

If the CCD engine is enabled during the `initial_opto` stage, the tool applies CCD offsets at the start of the `final_place` stage. During buffer adjustment optimization, the tool implements these prepone and postpone offsets.

To achieve postpones, the tool inserts additional repeater levels. To implement prepones, the tool removes existing repeaters.

By default, the buffer delay value for each timing endpoint is determined using Concurrent Clock and Data (CCD) optimization, also known as useful skew resolution. This value helps the tool decide how many buffer levels to insert. You can modify the buffer delay

Feedback

by setting the `cts.multisource.subtree_buf_delay_for_cus_prepone_postpone` application option.

The tool also ensures that balance points and latency offsets are updated to reflect the new clock structure. This approach maintains the clock latency at the sinks.

To use inverters as repeaters, set the `cts.multisource.subtree_preprocess_prefer_inverters` application option to `true`.

```
icc2_shell> set_app_options -list
 {cts.multisource.subtree_preprocess_prefer_inverters true}
```

Configuring the Compile final_opto Stage

The behavior of the `compile_fusion final_opto` stage is aligned with the default iSMSCTS.

During the `final_opto` stage, the tool runs another round of the CCD engine based on the updated clock structure and the offset limits derived from SMSCTS-CCD handshake at `compile_fusion final_place`.

Structural Multisource Clock Tree Synthesis (SMSCTS) then determines prepones and postpones that are compatible with the SMSCTS flow. Here, CCD can derive finer offset values that are implemented through structural moves such as cell sizing. As a result, in the iSMSCTS flow, running CCD optimization during the `final_opto` stage lets you use a finer CCD resolution (`ccd.resolution`) than in the `initial_opto` stage.

Configuring the clock_opt build_clock Stage

With the new Early CCD iSMSCTS flow, the tool preserves the locations set by placement during the `compile_fusion` stage. As a result, during the `build_clock` stage, the tool does not relocate the entire clock tree to the centroid in the Structural MSCTS step. For this, set the `cts.multisource.integrated_subtree_synthesis_early_ccd_support` application option to `true`.

Inserting Clock Drivers

Before you insert clock drivers,

- Specify all clock tree synthesis settings, including clock routing rules
- Specify that the inserted clock drivers should be aligned with the clock straps of the mesh structure by setting the `cts.multisource.enable_clock_driver_snapping` application option to `true`

Feedback

During multisource clock tree synthesis, you can use the `create_clock_drivers` command for the following:

- Insert mesh drivers for both regular or structural multisource clock trees.
- Tap drivers for regular multisource clock trees.

When you insert mesh or tap drivers,

- Specify the loads to drive by using the `-loads` option. The load can be a net or a set of pins or ports connected to the same net.
- Specify where to add the clock drivers as exact locations by using the `-location` option.

  Alternatively, you can add the clock drivers in an X by Y grid pattern over the entire core area by using the `-boxes` option. You can limit the X by Y grid pattern to a specific area by using the `-boundary` option with the `-boxes` option.
- Specify a list of cells that can be used as clock drivers by using the `-lib_cells` option. You can specify both single-rail and dual-rail library cells.

  Alternatively, you can select the existing driver of the load net as a template for the clock drivers by using the `-template` option. If the template cell you specify is a buffer or inverter, you can also specify different versions of this buffer or inverter as clock drivers by using the `-lib_cells` option with the `-template` option.

When you insert mesh drivers, to specify that the outputs of all the clock drivers must be connected together to drive the mesh net, use the `-short_outputs` option.

The following example creates a grid of eight by eight mesh drivers, places them in a regular pattern that covers the full core area, shorts the outputs of the clock drivers and connects to the net named clk_mesh, and transfers preexisting routing shapes and vias from the net named clk to this net:

```
icc2_shell> create_clock_drivers -loads [get_nets clk1_mesh] \
    -boxes {8 8} -lib_cells [get_lib_cells my_lib/CKBUF8X] \
    -short_outputs -output_net_name clk_mesh \
    -transfer_wires_from [get_nets clk]
```

The following example creates a grid of five by five tap drivers that are placed within the rectangle bounded at the lower-left by (200, 200) and upper right by (1000, 1000).

```
icc2_shell> create_clock_drivers -loads [get_nets clkA] \
    -lib_cells [get_lib_cells my_lib/CKBUF8X] \
    -boxes {5 5} -boundary [list {{200 200} {1000 1000}} ]
```

In this example, one of the tap drivers drives all the loads of the original clock net named clkA. To distribute the loads among all the clock drivers, you must subsequently perform automated multisource tap assignment by using the

Feedback

`synthesize_multisource_clock_taps` command, as described in Specifying Tap Assignment Options and Settings.

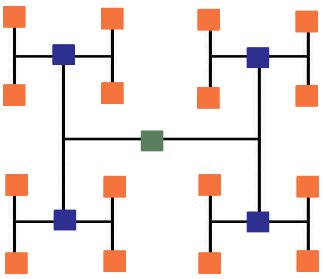
You can use the `create_clock_drivers` to add multiple levels of clock drivers by using the `-configuration` option and specifying the configuration of drivers at each level.

The following example inserts three levels of clock drivers. The first level consists of one buffer, the second level consists of four inverters in a 2x2 grid, and the third level consists of 16 inverters in a 4x4 grid.

```
icc2_shell> create_clock_drivers -loads [get_nets clk] \
   -configuration [list \
                  [list -level 1  -boxes {1 1} -lib_cells buf32x ] \
                  [list -level 2  -boxes {2 2} -lib_cells inv16x ] \
                  [list -level 3  -boxes {4 4} -lib_cells inv8x ]]
```

For this example, because there is no bounding box specified at any level, the drivers at each level are evenly distributed in the core area. The buffer in the first level is placed at the center of the core area, the four buffers in the next level are placed at the center of the four quadrants of the core area, and so on, resulting in an evenly placed clock drivers that can be routed to form an H-tree structure as shown in Figure 50

Figure 50 Clock Drivers Placed in an H-Tree Structure



After it inserts the clock drivers, the tool marks the clock drivers that it inserts as fixed and don't touched. The tool avoids overlapping the clock drivers with other fixed cells, blockages, and macros. However, the clock drivers can overlap with cells that are not fixed. The `create_clock_drivers` command does not legalize the design. To do so, run the `legalize_placement` command. If you run the `create_clock_drivers` command multiple times, to reduce runtime, run the `legalize_placement` command one time, after you complete all the `create_clock_drivers` command runs.

When routing the H-trees, the tool uses the highest routing layers available, based on the routing rules specified for the clock nets. These same layers can contain prerouted nets such as power and ground nets. To prevent placing clock drivers under these preroutes, which can cause pin accessibility issues, the `create_clock_drivers` command creates temporary placement blockages for the preroutes on the same layers it uses for routing.

Feedback

After it completes routing, it removes these temporary placement blockages, as shown in the following example output.

```
...
Net: clk; Rule: htree_ndr; Min Layer: M8; Max Layer: M9
...
Information: Using routing H/V layer pair 'M9'/'M8' for net 'clk'.
 (CTS-659)
...
Converting metal shapes in horizontal layer M9 and vertical layer M8 into
 placement blockages.
In total 703 placement blockages created.
...
Successfully deleted all temporary placement blockages and the cell map.
```

If you specify multirow-height cells with the `-lib_cell` option of the `create_clock_drivers` command, the tool might not be able to place them due to the temporary placement blockages it creates for the preroutes. If so, you can prevent the tool from creating the temporary placement blockages by setting the `cts.multisource.enable_pin_accessibility_for_global_clock_trees` application option to `false`.

To remove clock drivers inserted by the `create_clock_drivers` command, use the `remove_clock_drivers` command.

Inserting Clock Drivers for Designs With Multiple Levels of Physical Hierarchy

To insert clock drivers from the top level of a design with multiple levels of physical hierarchy,

1. Enable the lower-level physical blocks for editing by using the `set_editability` command.
2. Enable clock driver insertion for multiple levels of physical hierarchy by setting the `cts.multisource.enable_mlph_flow` application option to `true`.
3. Insert clock drivers by using the `create_clock_drivers` command.
4. Route the inserted clock drivers by using the `synthesize_multisource_global_clock_trees -roots -leaves -use_zroute_for_pin_connections` command.

When inserting clock drivers to lower-level blocks, the tool reuses existing clock ports. If there are no existing clock ports, the tool creates new ports, if it is allowed. You can allow new ports to be added to a block by using use the `set_freeze_ports` command.

Feedback

Synthesizing the Global Clock Trees

Before you insert clock drivers, ensure that all clock tree synthesis settings, including clock routing rules, are specified.

To perform clock tree synthesis and detail routing to build an H-tree style global clock tree, use the `synthesize_multisource_global_clock_trees` command. When building the H-tree, the tool tries to minimize the skew between the endpoints, which is essential for multisource clock tree synthesis.

When you use this command to synthesize and detail route a global clock tree, you must specify

- The clock net to synthesize by using the `-nets` option.
- The library cells to use by using the `-lib_cells` option. You can specify both single-rail and dual-rail library cells.

By default, the tool uses the Custom Router to route the H-tree structure and connect to the pins of the clock tree cells. If the Custom Router is unable to resolve all routing DRC violation when making pin connections, use the Zroute to make the pin connections by using the `-use_zroute_for_pin_connections` options. To stop the routes at the highest available metal layer close to the pin shape, use the `-skip_pin_connections` option.

The following example synthesizes and detail routes a global clock tree for the clock net named clkA.

```
icc2_shell> synthesize_multisource_global_clock_trees \
    -nets [get_nets clkA] -lib_cells [get_lib_cells my_lib/CKBUF8X] \
    -use_zroute_for_pin_connections
```

You can also use the `synthesize_multisource_global_clock_trees` command to only perform clock detail routing for an H-tree style clock structure that has already been synthesized.

When doing so, you must specify

- The startpoint of the global clock structure, by using the `-roots` option.
- The endpoints of the global clock structure by using the `-leaves` option.

The following example detail routes an existing H-tree style clock structure. The startpoint is the port named clk1 and the endpoints are the inputs of a group of mesh drivers.

```
icc2_shell> synthesize_multisource_global_clock_trees \
    -roots [get_ports clk1] -leaves [get_pins mesh_buf*/A] \
    -use_zroute_for_pin_connections
```

When routing the H-trees, the tool uses the highest routing layers available, based on the routing rules specified for the clock nets. These same layers can

Feedback

contain prerouted nets such as power and ground nets. To prevent placing clock drivers under these preroutes, which can cause pin accessibility issues, the `synthesize_multisource_global_clock_trees` command creates temporary placement blockages for the preroutes on the same layers it uses for routing. After it completes routing, it removes these temporary placement blockages, as shown in the following example output.

```
...
Net: clk; Rule: htree_ndr; Min Layer: M8; Max Layer: M9
...
Information: Using routing H/V layer pair 'M9'/'M8' for net 'clk'.
 (CTS-659)
...
Converting metal shapes in horizontal layer M9 and vertical layer M8 into
 placement blockages.
In total 703 placement blockages created.
...
Successfully deleted all temporary placement blockages and the cell map.
```

If you specify multirow-height cells with the `-lib_cell` option of the `synthesize_multisource_global_clock_trees` command, the tool might not be able to place them due to the temporary placement blockages it creates for the preroutes. If so, you can prevent the tool from creating the temporary placement blockages by setting the `cts.multisource.enable_pin_accessibility_for_global_clock_trees` application option to `false`.

To remove a clock structure created by the `synthesize_multisource_global_clock_trees` command, use the `remove_multisource_global_clock_trees` command.

Inserting Clock Drivers for Designs With Multiple Levels of Physical Hierarchy

To insert clock drivers from the top level of a design with multiple levels of physical hierarchy,

1. Enable the lower-level physical blocks for editing by using the `set_editability` command.
2. Enable clock driver insertion for multiple levels of physical hierarchy by setting the `cts.multisource.enable_mlph_flow` application option to `true`.
3. Insert clock drivers by using the `create_clock_drivers` command.

When inserting clock drivers to lower-level blocks, the tool reuses existing clock ports. If there are no existing clock ports, the tool creates new ports, if it is allowed. You can allow new ports to be added to a block by using use the `set_freeze_ports` command.

Feedback

Creating Clock Straps

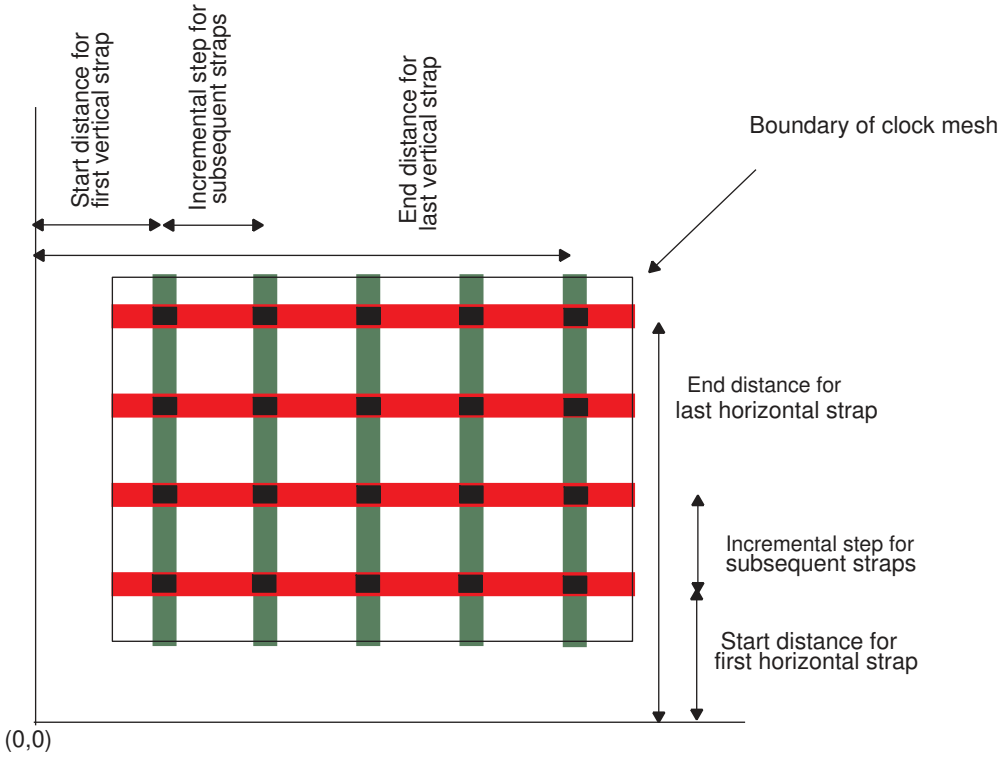
You can create clock straps, which are straight metal shapes in a single routing layer, by using the `create_clock_straps` command.

You can use this command to implement

- A clock mesh, which is a two-dimensional grid in a horizontal and a vertical layer, where the straps are connected by vias at the intersection points, as shown in Figure 51.
- A clock spine, which can be either a one- or two-dimensional structures.

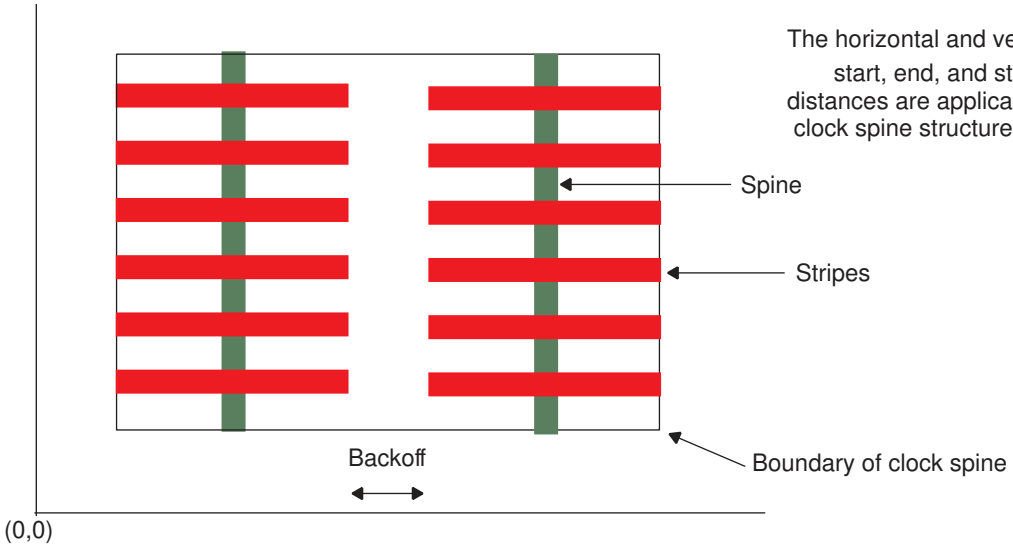
  One-dimensional spines are straps in a single direction. Two-dimensional spines consists of one-dimensional spines connected to multiple stripes in the orthogonal direction. Stripes connected to one spine do not connect to stripes of a different spine and the minimum distance between the stripes of different spines is called the backoff, as shown in Figure 52

Figure 51 Clock Mesh Structure



Feedback

Figure 52 Clock Spine Structure



When you use the `create_clock_straps` command,

- To specify the clock net for which to create the structure, use the `-net` option.
- To specify the bounding box to confine the structure, use the `-boundary` option.
- To specify keepouts, use the `-keepouts` option.

  If part of a strap is over a keepout, by default, the tool splits the strap and creates the portions that are outsides the keepouts. However, you can disable splitting and specify that the tool not create a strap if it is over a keepout by using the `-allow_splitting false` option.
- To specify the layers on which create the straps, use the `-layers` option.

  The tool determines the direction of the straps based on the metal layer you specify. To create straps in both directions, specify a layer for each direction.
- To specify the width of the straps, use the `-width` option.

Feedback

- To specify where to create the straps, use the `-grid` option as follows:
  - To create a single horizontal or vertical strap, specify the distance from the x- or y-axis.
  - To create a multiple straps in a single direction, specify an iterator list consisting of the start distance to the first strap, the end distance to the last strap, and the incremental distance between straps.
  - To create a multiple straps in a both the horizontal and vertical directions, specify an iterator list for each direction, starting with the horizontal direction.

  Use the `-grid` option to create multiple straps for both the clock mesh and clock spine structures.
- To specify a margin within which the tool can move a strap from the position it derives based on the `-grid` option settings, use the `-margins` option.

  If the tool cannot create a strap within the specified margin, due to obstructions or keepouts, it does not create the strap. By default, the tool uses a margin of zero and only creates the strap if it can do so at the exact position it derives.
- To create straps on the boundary specified with the -boundary option, use the `-create_ends` option.
- To allow straps that are unconnected to orthogonal straps, use the `-allow_floating true` option.

  By default, the tool does not allow orthogonal straps that are unconnected.
- To specify the type of straps to create, use the `-type` option and specify `user_route`, `stripe`, or `detect`.

  When you create straps in both direction, specify a list of two values starting with the type for the horizontal direction.

  You can specify `detect` as the type only when you are creating a clock spine structure and want the tool to detect and use existing spines. When you do so, you must specify a list consisting of the type for the orthogonal stripes and `detect`, with the type for the horizontal direction specified first in the list. In addition, when the tool detects existing spine, you can specify a minimum length for the spine, by using the `-detect_length` option.
- To specify the maximum length of the orthogonal stripes, when creating a clock spine structure, use the `-length` option.
- To specify the backoff distance between stripes of different spines, when creating a clock spine structure, use the `-backoff` option.

Feedback

- To specify the direction of the spines, when creating a clock spine structure and the tool is not detecting and using existing spines, use the `-spine_direction` option.
- To shield the straps with power and ground nets, use the `-bias` option.
- To shield the straps with specific nets, use the `-bias_to_nets` with the list of nets.
- To specify the distance from the shielding nets, use the `-bias_margins` option.
- To remove the straps of a specific clock net, use the `-clear` option.

For example, the following command creates a clock mesh for the net named clk1_mesh that is bounded by coordinates (0,0) and (1200, 980). The straps are on layers M7 and M8 with a width of 2.4 units and of type stripe. The horizontal straps on layer M7 start at a distance of 20 units from the x-axis and repeat every 100 units, until they reach 1200 units from the x-axis. The vertical straps on layer M8 start at a distance of 60 units from the y-axis and repeat every 150 units, until they reach 980 units from the x-axis.

```
icc2_shell> create_clock_straps -nets [get_nets clk1_mesh] \
   -layers {M7 M8} -widths {2.4 2.4} -types {stripe stripe} \
   -grids {{20 1200 100} {60 980 150}}-boundary {{0 0} {1200 980}}
```

The following example creates a two-dimensional spine structure that has spines on the vertical layer M8 with a width of 3.6 units and of type `user_route` and stripes on the horizontal layer M7 with a width of 2.4 units and of type `stripe`. The vertical spines on layer M8 start at a distance of 60 units from the y-axis and repeat every 150 units, until they reach 980 units from the x-axis. The horizontal stripes on layer M7 have a length of 120 units and they start at a distance of 20 units from the x-axis and repeat every 100 units, until they reach 1200 units from the x-axis. The minimum distance (backoff) between stripes of different spines is 5 units.

```
icc2_shell> create_clock_straps -nets [get_nets clk1_mesh] \
   -layers {M7 M8} -widths {2.4 3.6} -types {stripe user_route} \
   -grids {{20 1200 100} {60 980 150}} -length 120 -backoff 5
```

Routing to Clock Straps

After you create clock straps for a clock net by using the `create_clock_straps` command, you can route the drivers and loads of the clock net to the clock straps by using the `route_clock_straps` command and specify the clock net name by using the `-nets` option. The tool connects the drivers and the loads only to clock straps with the `shape_use` attribute setting of `stripe`. It does not connect to clock straps with the `shape_use` attribute setting of `user_route`.

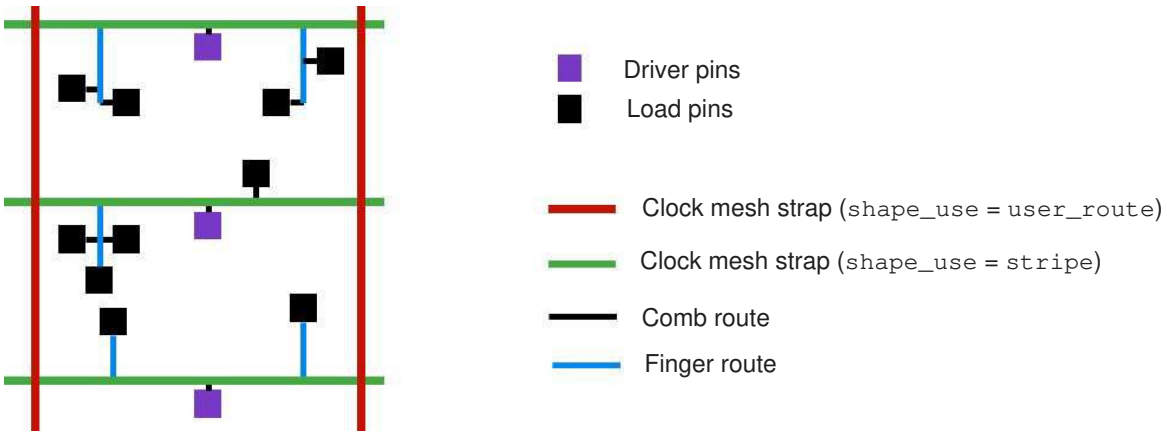
Feedback

To specify a topology for connecting the clock drivers and sinks to the clock straps, use the `-topology` option as follows:

- To create a fishbone routing topology, the default topology, use the `-topology fishbone` option.

  In a fishbone topology, each driver pin is individually connected to the nearest stripe. To minimize wire length, multiple load pins are connected using comb routing to a single finger, which is connected to the stripe, as shown in Figure 53.

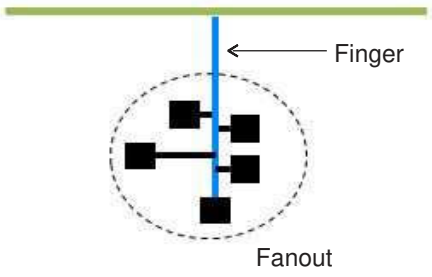
Figure 53 Fishbone Topology



If you are using a fishbone topology, you can specify

  - A maximum fanout for the loads of a finger, as shown in Figure 54, by using `-fishbone_fanout` option.

Figure 54 Fanout of a Finger of the Fishbone Topology

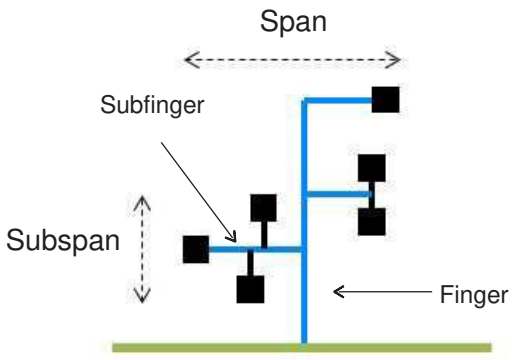


  - A maximum span between any two loads connected to a finger, as shown in Figure 55, by using `-fishbone_span` option.

Feedback

The span is measured orthogonal to the direction of the finger.

Figure 55 Span and Subspan of the Fishbone Topology



  - A maximum subspan between any two loads connected to a subfinger, as shown in Figure 55, by using `-fishbone_sub_span` option.

    The subspan is measured orthogonal to the direction of the subfinger.

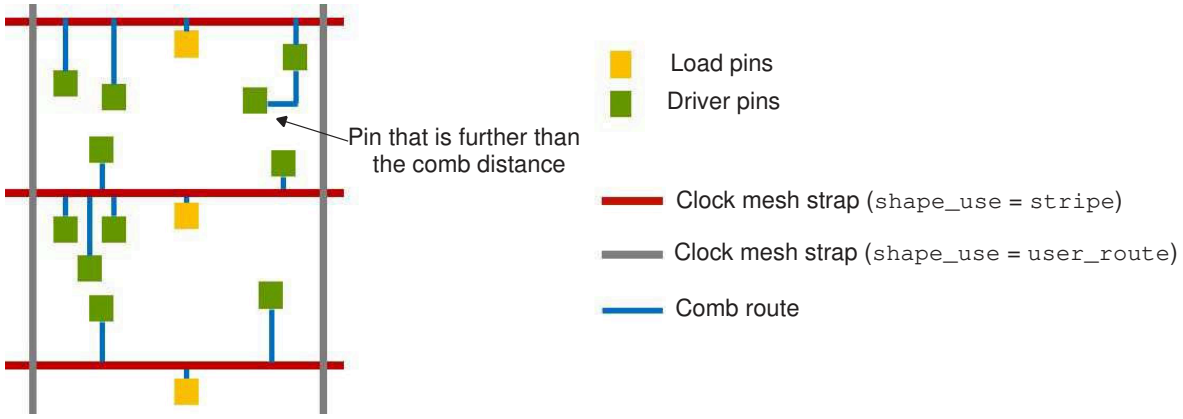
  - The layers to use for routing the fingers and subfingers by using the `-fishbone_layers` option.

- To create a comb routing topology, use the `-topology comb` option.

  In a comb topology, each driver and load pin is directly routed to the nearest stripe. However, if the Manhattan distance from a pin to the nearest stripe is more than the comb distance, the tool routes the pin to the nearest net shape using a Steiner topology, as shown in Figure 56. The default comb distance is two global routing cells. To change the comb distance, use the `route.common.comb_distance` application option.

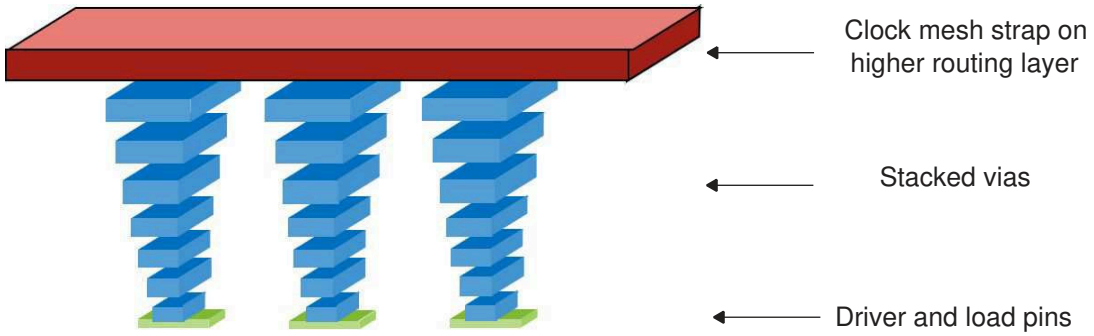
Feedback

Figure 56 Comb Topology



Comb routing is suitable when there are a large number of driver and load pins directly under the clock-mesh stripes. The tool uses stacked vias to connect the pins to the stripes, which are usually in the higher routing layers. However, this can contribute to physical DRC violations due to many adjacent stacked vias, as shown in Figure 57.

Figure 57 Vias in the Comb Topology

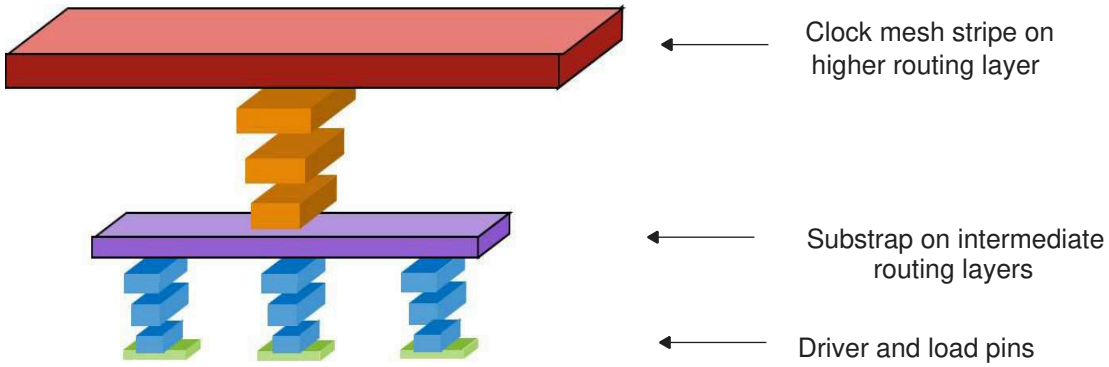


- To create a substrap routing topology, use the `-topology sub_strap` option.

  In a substrap topology, additional straps that are parallel to the stripes are created on intermediate routing layers. This reduces the number of stacked vias, as compared to the comb topology.

Feedback

Figure 58 Vias in the Substrap Topology



For the substrap topology, you

- Must specify the layers in which to create the substrap by using the `-sub_strap_layers` option
- Can specify the maximum RC delay allowed for the substrap by using the `-sub_strap_max_delay` option

  The default is 2 ps. This delay constrains the length and the number of loads connected to the substrap.

Analyzing the Clock Mesh

To reduce skew variation, clock mesh structures require higher timing accuracy than traditional clock structures. Therefore, to analyze a clock mesh structures, use the `analyze_subcircuit` command, which performs transistor level circuit simulation for the clock mesh and back-annotates accurate timing information.

Before you run the `analyze_subcircuit` command, you must

- Detail route the clock mesh net.
- Have a circuit-level model for each of the gates in your clock tree and a transistor model for each of the transistors in the circuit-level models.
- Have access to a SPICE simulator such, as NanoSim, FineSim, or HSPICE.

Feedback

For the `analyze_subcircuit` command, you must specify

- The clock mesh net you want to simulate by using the `-net` option.

  Alternatively, you can specify the sinks of the mesh net by using the `-to` option.

  If multiple clocks reach the net or the sinks you specify, use the `-clock` option to distinguish the clock you want to analyze.

- A name by using the `-name` option.

  This name is used to construct the circuit elements. It is also used in the name of the output files and the directory where the output is stored.

Optionally you can specify an annotated transition on the clock port or the start point of your clock mesh. If you do so, the clock must be specified as propagated by using the `set_propagated_clock` command.

When you run the `analyze_subcircuit` command, the tool performs the following steps:

1. Performs RC extraction and generates parasitic files.

   You can run extraction as a standalone step by using the `-extraction` option.

2. Generates SPICE files for simulating the clock mesh, using the parasitic files from the previous step as input.

   You can generate the SPICE files as a standalone step by using the `-create_spice_deck` option. When you do so, you can use parasitic files generated by a different extraction tool. If these parasitic files have a different naming convention, you can specify the appropriate file suffix by using the `-spef_input_file_suffix` and `-rc_include_file_suffix` options.

3. Runs SPICE simulation, using the SPICE files generated in the previous step as input.

   By default, the tool uses the NanoSim simulator. You can use the FineSim or HSPICE simulators by using the `finesim` or `hspice` setting with the `-simulator` option.

   You must specify the location of the circuit-level and transistor-level models by using the `-driver_subckt_files` and `-spice_header_files` options. You customize these settings by specifying different files for the maximum and minimum conditions within each scenario by using the `-configuration` option.

   You can run simulation as a standalone step by using the `-run_simulation` option.

4. Generates timing annotation files containing `set_disable_timing`, `set_annotated_delay`, and `set_annotated_transition` commands, using the simulation results as input.

Feedback

For clock mesh nets, which have multiple drivers, the `set_disable_timing` command is used to disable all except one of the drivers, which is called the anchor driver. The annotated net delay arcs are defined from the anchor driver.

You can generate the timing annotation files as a standalone step by using the `-write_annotation` option.

5. Applies the annotation files generated in the previous step.

   You can apply the annotation files as a standalone step by using the `-apply_annotation` option.

If you want to run some of the steps of the mesh analysis flow, such as extraction or SPICE simulation, using other tools, you can do so and run the rest of the steps of the flow using the standalone options of the `analyze_subcircuit` command, following the same sequence.

The following example analyzes the clock mesh net named clk_mesh using the HSPICE simulator. It customizes the simulation by using different files for the minimum and maximum conditions of each scenario.

```
icc2_shell> analyze_subcircuit -net clk_mesh \
   -driver_subckt_files max_spice_model \
   -spice_header_files header_file \
   -configuration { \
      {-scenario_name scenario1 \
         -max_driver_subckt_files max_file1 \
         -max_spice_header_files header_max1 \
         -min_driver_subckt_files min_file1 \
         -min_spice_header_files header_min1} \
      {-scenario_name scenario2 \
         -max_driver_subckt_files max_file2 \
         -max_spice_header_files header_max2 \
         -min_driver_subckt_files min_file2 \
         -min_spice_header_files header_min2}}
   -simulator hspice \
   -name clk_mesh_analysis
```

Performing Automated Tap Insertion and H-Tree Synthesis

To perform this task, you must

1. Specify the required settings by using the `set_regular_multisource_clock_tree_options` command as follows:
   - The clock name by using the `-clock` option
   - The topology for the global clock tree by using the `-topology htree_only` option

Feedback

- The number of rows and columns of tap cells to insert by using the `-tap_boxes` option
- A list of library cells to use as tap drivers by using the `-tap_lib_cells` option

  The library cells you specify must be enabled for clock tree synthesis by using the `set_lib_cell_purpose -include cts` command.
- A list of library cells to use for synthesizing the global clock tree (H-tree) by using the `-htree_lib_cells` option

  The library cells you specify must be enabled for clock tree synthesis by using the `set_lib_cell_purpose -include cts` command.

By default, the tool uses

- The net connected to the clock root to insert the tap drivers and build the clock structure

  You can specify a different net of the clock network by using the `-net` option.
- The locations of the sinks to determine the boundary within which to insert the tap cells

  You can control this boundary by using the `-tap_boundary` option. You can also specify keepout areas within which tap cells should not be inserted by using the `-keepouts` option.
- The layers and routing rules specified by the clock tree synthesis settings to route the H-tree

  You can specify the layers and routing rules for the H-tree by using the `-htree_layers` and `-htree_routing_rule` options.

You can report the options you specified by using the `report_regular_multisource_clock_tree_options` command, and remove them by using the `remove_regular_multisource_clock_tree_options` command.

2. (Optional) Enable flexible H-tree synthesis by setting the `cts.multisource.flexible_htree_synthesis` application option to `true`.

   When you enable this feature, the tool identifies the best count, configuration, and placement of tap drivers for minimum clock tree insertion delay.

   To specify that the tool derives a symmetric configuration of tap drivers, set the `cts.multisource.tap_selection` application option to `symmetric`. The default is `user`, and by default the tool uses the configuration specified by the `-tap_boxes` option of the `set_regular_multisource_clock_tree_options` command for the tap drivers.

Feedback

3. Insert the tap drivers and build the H-tree by using the `synthesize_regular_multisource_clock_trees` command, which consists of the following two stages:

   - `tap_synthesis`, during which the tool inserts the tap drivers
   - `htree_synthesis`, during which the tool builds the H-tree that drives the tap drivers

   By default, the tool performs both stages. You can perform only one of these stages by using the `-to` or `-from` option.

The following example script inserts tap drivers in four columns and two rows and builds the H-tree:

```
## Include only tap driver and H-tree library cell for CTS
set cts_references [get_lib_cells -filter valid_purposes=~*cts*]
set_lib_cell_purpose -exclude cts [get_lib_cells $cts_references]
set_lib_cell_purpose -include cts { LIB1/BUF1 LIB1/BUF2}

## Set up and insert tap drivers and build H-tree
set_regular_multisource_clock_tree_options \
   -clock clk -topology htree_only -tap_boxes {4 2} \
   -tap_lib_cells [get_lib_cells */CKBUF*] \
   -htree_lib_cells [get_lib_cells */CKINV*] \
   -htree_layers "m9 m10" -htree_routing_rule "htree_ndr"
synthesize_regular_multisource_clock_trees

## Reapply CTS library cell list as required for the subsequent steps
set_lib_cell_purpose -include cts [get_lib_cells $cts_references]
```

Specifying Tap Assignment Options and Settings

Before you can perform automated tap assignment by using the `synthesize_multisource_clock_taps` command, you must perform the following steps:

1. Remove `dont_touch`, `size_only`, `fixed`, and `locked` attributes setting on the clock cells that are not required. These settings prevent the tool from merging or splitting clock cells during tap assignment, if necessary.
2. Specify settings for automated tap assignment by using the `set_multisource_clock_tap_options` command.

   You must specify

   - The clock for tap assignment by using the `-clock` option.

     All the sinks reachable from the given clock are considered for redistribution.
   - The tap drivers among which the sinks are redistributed by using the `-driver_objects` option.

Feedback

- The number of taps that the sinks are redistributed among by using the `-num_taps` option.

  Currently, the number of taps specified with this option must be equal to the number of drivers specified with the `-driver_objects` option.

To prevent the tool from merging specific clock tree cells, specify the instance names using the `-dont_merge_cells` option.

To report the settings you specify for automated tap assignment, use the `report_multisource_clock_tap_options` command. To remove the settings you specify for automated tap assignment, use the `remove_multisource_clock_tap_options` command.

3. (Optional) Create multisource sink groups for tap assignment by using the `create_multisource_clock_sink_group` command.

   Skew groups defined for the clock can be distributed among different taps. To retain the objects of a skew group under the same tap, create a corresponding multisource sink group.

   Use the following commands to remove, report, or manipulate multisource sink groups:

   - `remove_multisource_clock_sink_groups`
   - `report_multisource_clock_sink_groups`
   - `get_multisource_clock_sink_groups`
   - `add_to_multisource_clock_sink_group`
   - `remove_from_multisource_clock_sink_group`

4. (Optional) Specify name prefixes and suffixes to use when merging and splitting clock cells during tap assignment by using the following application options:

   - `cts.multisource.subtree_merge_cell_name_prefix`
   - `cts.multisource.subtree_merge_cell_name_suffix`
   - `cts.multisource.subtree_split_cell_name_prefix`
   - `cts.multisource.subtree_split_cell_name_suffix`
   - `cts.multisource.subtree_split_net_name_prefix`
   - `cts.multisource.subtree_split_net_name_suffix`

Feedback

Building the Local Clock Subtree Structures

In a structural multisource clock tree, the local subtrees are directly driven by the clock mesh. You can synthesize these local subtrees by using the `synthesize_multisource_clock_subtrees` command.

Before you synthesize the local subtrees, you must

1. Build the global clock distribution structure.
2. Model the clock mesh net with a realistic annotated delay and transition values using the `set_annotated_delay` and `set_annotated_transition` commands.
3. Remove `dont_touch`, `size_only`, `fixed`, and `locked` attributes setting on the clock cells that are not required. These settings prevent the `synthesize_multisource_clock_subtrees` command from merging or splitting clock cells, if necessary.
4. Specify settings for local subtree synthesis by using the `set_multisource_clock_subtree_options` command.

   You must specify

   - The clock for synthesizing the subtrees by using the `-clock` option.
   - The drivers of the subtrees to be synthesized by using the `-driver_objects` option.

     The clock specified with the `-clock` option should pass through each drivers specified with the `-driver_objects` option.

   Optionally you can

   - Prevent the tool from merging specific clock tree cells by using the `-dont_merge_cells` option.
   - Balance the levels of the local subtree by using the `-balance_levels true` option and specify a target number of levels, which applies to all sinks of the local subtree, by using the `-target_level` option.

     Optionally, you can apply a different target number of levels to specific sink pins and clock balance points by using the `set_multisource_clock_subtree_constraints` command with the `-pins` and `-target_level` options.
   - Specify a maximum total wire delay from any subtree driver to any of its sinks by using the `-max_total_wire_delay` option and the corner it applies to by using the `-corner` option.

Feedback

- Enable the reordering of clock-gating cells by using the `-enable_icg_reordering true` option.

  When you enable this feature, the tool swaps the position of a clock-gating cell with the buffer that is driving the clock-gating cell. Then, the buffer is gated by the clock-gating cell, thereby reducing the dynamic power consumption.

  To perform reordering, you must

  - Enable scenarios used for structural multisource clock tree synthesis for dynamic power optimization
  - Annotate switching activity for the enable pins of the clock-gating cells

  You can prevent specific clock-gating cells from being reordered by using the `set_multisource_clock_subtree_constraints` command with the `-cells` and `-ignore_for_icg_reordering` options.

To report the settings you specified, use the `report_multisource_clock_subtree_options` command. To remove the settings you specified, use the `remove_multisource_clock_subtree_options` command.

5. (Optional) Enable multicorner optimization by setting the `cts.multisource.enable_multi_corner_support` application option to `true`. The default is `false`.

   By default, the tool uses the worst corner associated with the mode in which the subtree options are defined. When you enable this feature, the tool considers all corners associated with the mode in which the subtree options are defined.

6. (Optional) Ignore the maximum capacitance, maximum fanout, or both constraints of the clock drivers by using the `cts.multisource.ignore_drc_on_subtree_driver` application option.

   When you synthesize multisource clock subtrees, by default, the tool considers the maximum transition, capacitance, and fanout constraints of the clock drivers you specify. However, the clock drivers typically have relaxed or no maximum capacitance or maximum fanout constraint, which can affect the multisource clock tree QoR.

   The following example ignores both the maximum capacitance and maximum fanout constraints of the clock drivers:

   ```
   icc2_shell> set_app_options \
     -name cts.multisource.ignore_drc_on_subtree_driver \
     -value "max_fanout max_capacitance"
   ```

7. (Optional) Enable fishbone routing for the subtree clock nets by using the `cts.multisource.subtree_routing_mode` application option.

Feedback

```
icc2_shell> set_app_options \
   -name cts.multisource.subtree_routing_mode -value fishbone
```

8. (Optional) Specify name prefixes and suffixes to use when merging and splitting clock cells during local subtree synthesis by using the following application options:
   - cts.multisource.subtree_merge_cell_name_prefix
   - cts.multisource.subtree_merge_cell_name_suffix
   - cts.multisource.subtree_split_cell_name_prefix
   - cts.multisource.subtree_split_cell_name_suffix
   - cts.multisource.subtree_split_net_name_prefix
   - cts.multisource.subtree_split_net_name_suffix

When you run the synthesize_multisource_clock_subtrees command the tool performs the following steps:

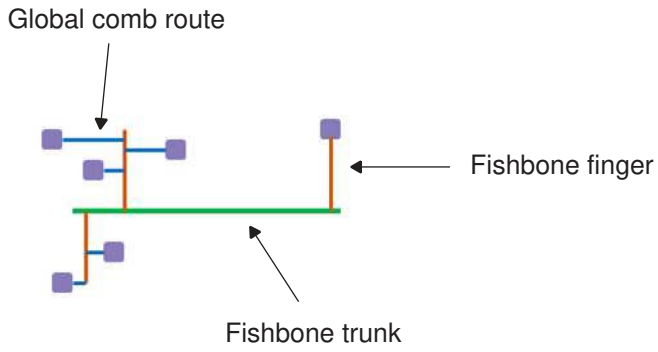
1. Merges equivalent clock cells to remove any artificial boundaries between clusters of sinks.
2. Optimizes the subtree by splitting, sizing, and relocating the clock cells, and snaps the first level of clock cells to the clock straps.
3. Routes the clock nets.

   By default, it global routes the nets. However, if you enable fishbone routing by using the cts.multisource.subtree_routing_mode application option, the tool creates a mix of detail routed fishbone trunks and fingers, and global comb routes to connect the net loads to the fishbone fingers, as shown in Figure 59.

   You can control the fishbone routing by using the following application options:
   - cts.routing.fishbone_max_tie_distance
   - cts.routing.fishbone_max_sub_tie_distance
   - cts.routing.fishbone_bias_threshold
   - cts.routing.fishbone_bias_window
   - cts.routing.fishbone_horizontal_bias_spacing
   - cts.routing.fishbone_vertical_bias_spacing
4. Refines the clock trees by sizing and relocating cells based on the routing.

Feedback

Figure 59 Detail Fishbone Routes and Global Comb Routes



You can run specific steps of the `synthesize_multisource_clock_subtrees` command by using the `-from` and `-to` options and specifying `merge`, `optimize`, `route_clock`, or `refine`.

Structural subtrees that you build by using the `synthesize_multisource_clock_subtrees` command are not changed or removed when you subsequently run any clock tree synthesis command.

Performing Irregular Global Tree Synthesis

Automated global trees can only be created for symmetrically inserted tap drivers, distributed evenly across the layout. This results in floorplan and global tree complexities that require manual global tree insertion.

Irregular Global Tree Synthesis automatically constructs multilevel design rule check (DRC) clean global trees to non-symmetric, unevenly inserted tap drivers without tight on-chip variation (OCV) issues or common path requirements.

In irregular global tree synthesis, global trees are created using multilevel buffer trees in a non-H-tree style, ensuring minimum latency for all tap drivers. You can use the global tree style to construct a global tree for irregularly placed tap drivers at the block or even to build a global distribution of clock to block inputs in top-level channels.

Feedback

Analyzing the Clock Tree Results

After synthesizing the clock trees, analyze the clock tree results to verify that they meet your requirements. Typically, the analysis process consists of the following tasks:

- Analyzing the clock tree quality of results (QoR) (as described in Generating Clock Tree QoR Reports)
- Analyzing the clock tree timing (as described in Analyzing Clock Timing)
- Verifying the placement of the clock instances, using the GUI (as described in Analyzing Clock Trees in the GUI)

Generating Clock Tree QoR Reports

To generate clock tree QoR reports, use the `report_clock_qor` command.

The `report_clock_qor` command can generate the following types of reports:

- Summary

  By default, this command reports a summary of the clock tree QoR, which includes the latency, skew, DRC violations, area, and buffer count.

- Latency

  To report the longest and shortest path for each clock, use the `-type latency` option.

- DRC violators

  To report the maximum transition and capacitance constraint violators, use the `-type drc_violators` option.

- Robustness

  To report the robustness of each sink, use the `-type robustness` option. The robustness of a sink is the ratio between its latency for the corner for which the report is being generated and its latency for the corner specified by the `-robustness_corner` option. The mode use for both latency values is the mode for which the report is being generated. When you use the`-type robustness` option, you must specify a corresponding robustness corner by using the `-robustness_corner` option.

- Clock-balance groups QoR

  To report a clock QoR summary for each clock-balance group, use the `-type balance_groups` option. It also reports the clock latency offset values specified with the `-offset_latencies` option of the `create_clock_balance_group` command and the actual latency offset values.

Feedback

- Local skew

  To report the QoR summary and the worst local skew for each clock or skew group, use the `-type local_skew` option. It also reports the five largest and five smallest local skew values and the corresponding endpoints.

- Clock tree power

  To report a summary of the leakage, internal, sink, net switching, dynamic, and total power per clock per scenario, use the `-type power` option. If the `power.clock_network_include_clock_sink_pin_power` application option is set to `off`, the sink power is not reported.

When you use the `-type` option, you can generate the output as a comma separated values (CSV) file by using the `-csv` option and specifying the output file name using the `-output` option.

The `report_clock_qor` command can also generate the following types of histograms:

- Latency histogram

  To report the latency of each sink in a histogram format, use the `-histogram_type latency` option.

- Transition histogram

  To report the transition time of each sink in a histogram format, use the `-histogram_type transition` option.

- Capacitance histogram

  To report the capacitance of each sink in a histogram format, use the `-histogram_type capacitance` option.

- Local skew histogram

  To report the local skew in a histogram, use the `-histogram_type local_skew` option.

- Robustness histogram

  To report the robustness of each sink, with respect to a robustness corner specified by using the `-robustness_corner` option, in a histogram format, use the `-histogram_type robustness` option.

- Wire delay fraction histogram

  To report the ratio between the wire delay and the total delay for each logic stage of the clock tree in a histogram format, use the `-histogram_type wire_delay_fraction` option.

Feedback

By default, the tool generates a report for all clock trees in all active modes and corners in all active scenarios. You can limit the report to specific

- Portions of a clock network by using the `-from`, `-to`, and `-through` options
- Clock trees by using the `-clock` option
- Skew groups by using the `-skew_group` option
- Modes by using the `-mode` option
- Corners by using the `-corner` option
- Scenarios by using the `-scenario` option

  If you use the `-scenario` option, you cannot use the `-mode` and `-corner` options.

Reporting Clock Tree Power

To report the power of a clock tree, use the `report_clock_power` command. By default, it reports the power for all clock trees in all modes and corners of all active scenarios. To report the power

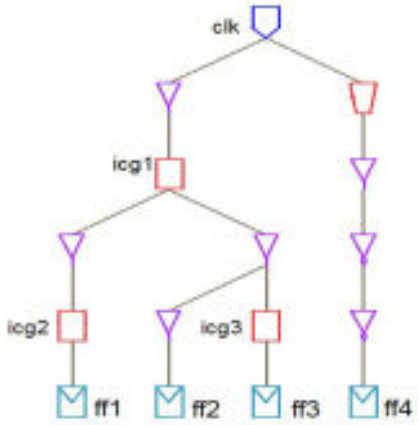
- Only for specific clocks, use the `-clocks` option
- Only for specific modes, use the `-modes` option
- Only for specific corners, use the `-corners` option
- Only for specific scenarios, use the `-scenario` option
- On a per-segment or per-subtree basis, use the `-type per_segment` or `-type per_subtree` option
  - A segment is the clock buffer tree from one integrated-clock-gating (ICG) cell to the next ICG cells or sinks in its fanout

    For example, in the following figure, the buffer tree from the output of the ICG1 cell to the inputs of the ICG2, ICG3, and ff2 cells is considered a segment.
  - A subtree is a clock tree from an ICG cell all the way to the sinks in its fanout.

    For example, in the following figure, the clock tree from the output of the ICG1 cell to the inputs of the ff1, ff2, and ff3 cells is considered a subtree.

Feedback

Figure 60 Segments and Subtrees of a Clock Tree



Creating Collections of Clock Network Pins

You can create a collection of the pins on a clock network by using the `get_clock_tree_pins` command. By default, this command returns all the pins on all the clock networks of all scenarios.

You can limit the selection by using one of the following methods

- Limit it to specific clock by using the `-clocks` option.
- Limit it to specific scenario, scenarios of modes, or scenarios of corners by using the `-scenarios`, `-modes`, or `-corners` option.

You can further limit the selection by

- Considering only the clock paths that go from, through, and to specific pins by using the `-from`, `-through`, and `-to` options.
- Filtering the pins based on their attributes by using the `-filter` option.

For more information about all the available options and all the supported pin attributes, see the man page for the `get_clock_tree_pins` command.

The following example creates a collection named clk1_icg_pins, which consists of the pins of integrated-clock-gating cells that are on the clock network of the clock named clk1:

```
icc2_shell> set clk1_icg_pin \
   [get_clock_tree_pins -filter is_on_ICG -clocks clk1]
```

Feedback

Analyzing Clock Timing

The timing characteristics of the clock network are important in any high-performance design. To obtain detailed information about the clock networks in the current block, use the `report_clock_timing` command.

You must use the `-type` option to specify the type of report to generate. The `report_clock_timing` command can generate the following types of reports:

- Single-clock local skew

  To generate a single-clock local skew report, use the `-type skew` option.

- Interclock skew

  To generate an interclock skew report, use the `-type interclock_skew` option.

- Latency

  To generate a latency report, use the `-type latency` option.

- Transition

  To generate a transition time report, use the `-type transition` option.

Analyzing Clock Trees in the GUI

The IC Compiler II GUI provides the Clock Tree Visual Mode to help you visualize and analyze the clock trees in your design. In this mode, you can overlay the clock tree information to the layout view, or view the clock tree structure in the schematic view.

For more information about analyzing clock tree information in the GUI, see the *Using Map and Visual Modes* topic in the *IC Compiler II Graphical User Interface User Guide*.

5

Routing and Postroute Optimization

This topic describes the routing capabilities of Zroute, which is the router for the IC Compiler II tool. Zroute is architected for multicore hardware and efficiently handles advanced design rules for 45 nm and below technologies and design-for-manufacturing (DFM) tasks. It also describes the postroute optimization features supported by the IC Compiler II tool.

To learn about routing and postroute optimization, see the following topics:

- Introduction to Zroute
- Basic Zroute Flow
- Prerequisites for Routing
- Defining Vias
- Inserting Via Ladders
- Checking Routability
- Routing Constraints
- Routing Application Options
- Routing Clock Nets
- Routing Critical Nets
- Routing Secondary Power and Ground Pins
- Routing Signal Nets
- Shielding Nets
- Performing Postroute Optimization
- Analyzing and Fixing Signal Electromigration Violations
- Performing ECO Routing
- Routing Nets in the GUI
- Cleaning Up Routed Nets

Feedback

- Analyzing the Routing Results
- Saving Route Information
- Deriving Mask Colors
- Inserting and Removing Cut Metal Shapes

Introduction to Zroute

Zroute has five routing engines: global routing, track assignment, detail routing, ECO routing, and routing verification. You can invoke global routing, track assignment, and detail routing by using task-specific commands or by using an automatic routing command. You invoke ECO routing and route verification by using task-specific commands.

Zroute includes the following main features:

- Multithreading on multicore hardware for all routing steps, including global routing, track assignment, and detail routing
- A realistic connectivity model where Zroute recognizes electrical connectivity if the rectangles touch; it does not require the center lines of wires to connect
- A dynamic maze grid that permits Zroute to go off-grid to connect pins, while retaining the speed advantages of gridded routers
- A polygon manager, which allows Zroute to recognize polygons and to understand that design rule checks (DRCs) are aimed at polygons
- Concurrent optimization of design rules, antenna rules, wire optimization, and via optimization during detail routing
- Concurrent redundant via insertion during detail routing
- Support for soft rules built into global routing, track assignment, and detail routing
- Timing- and crosstalk-driven global routing, track assignment, detail routing, and ECO routing
- Intelligent design rule handling, including merging of redundant design rule violations and intelligent convergence
- Net group routing with layer constraints and nondefault routing rules
- Clock routing
- Route verification

Feedback

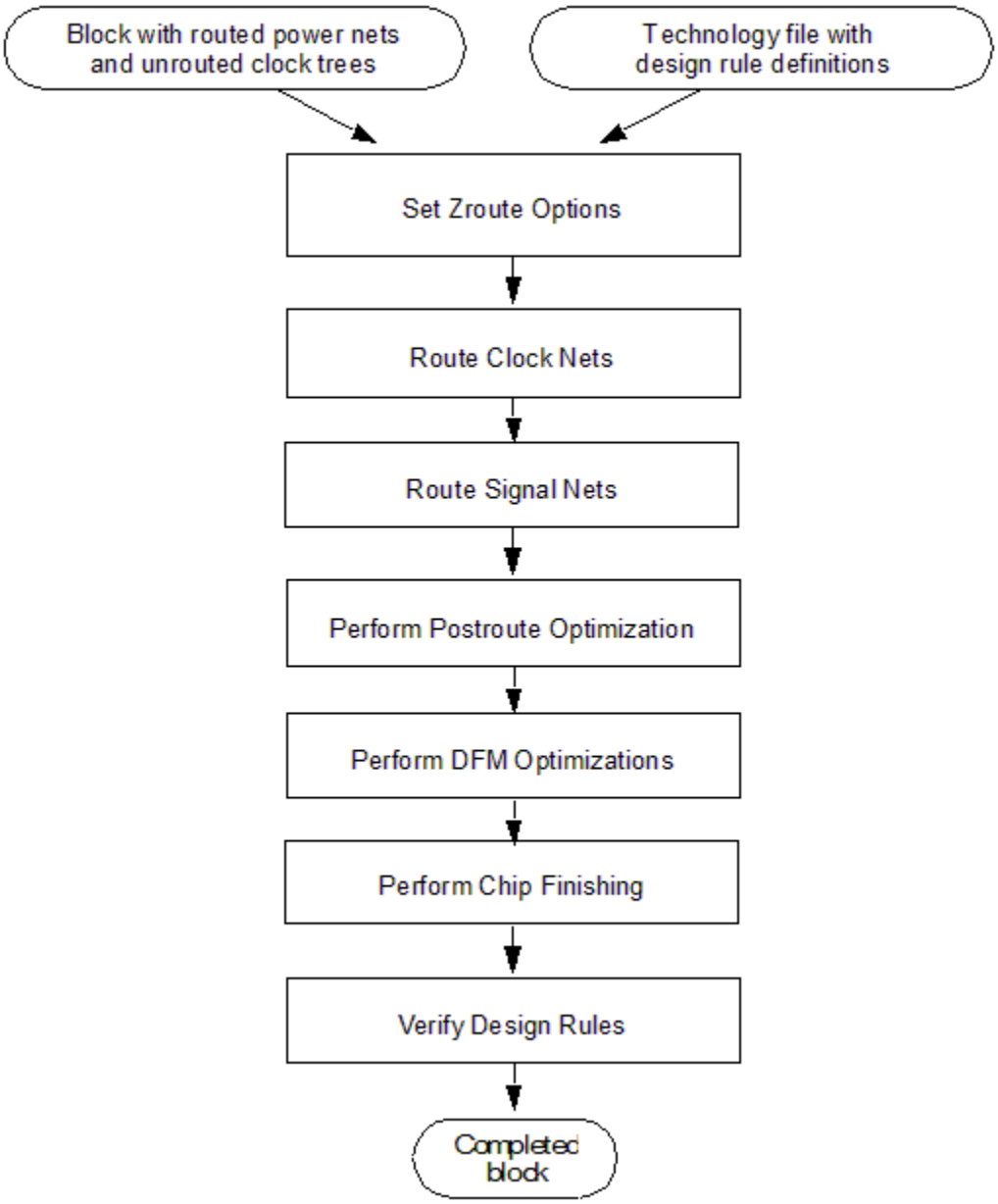
- Optimization for DFM and design-for-yield (DFY) using a soft rule approach
- Support for advanced design rules, such as multiple patterning

Feedback

Basic Zroute Flow

Figure 61 shows the basic Zroute flow, which includes clock routing, signal routing, DFM optimizations, and route verification.

Figure 61 Basic Zroute Flow



Feedback

Prerequisites for Routing

Before you can run Zroute, you must ensure that the block and physical library meet the following requirements:

- Library requirements

  Zroute gets all of the design rule information from the technology file; therefore, you must ensure that all design rules are defined in the technology file before you start routing.

  For more information about the technology file and defining routing design rules, see the *Synopsys Technology File and Routing Rules Reference Manual*.

- Block requirements

  Before you perform routing, your block must meet the following conditions:

  - Power and ground nets have been routed after design planning and before placement.

    For more information, see the *IC Compiler II Design Planning User Guide*.

  - Clock tree synthesis and optimization have been performed.

    For more information, see Clock Tree Synthesis.

  - Estimated congestion is acceptable.
  - Estimated timing is acceptable (about 0 ns of slack).
  - Estimated maximum capacitance and transition have no violations.

  To verify that your block meets the last three prerequisites, you can check the routability of its placement as explained in Checking Routability.

Feedback

Defining Vias

The router supports the following types of via definitions:

- Simple vias and simple via arrays

  A simple via is a single-cut via. It is specified by a cut layer and the height and width of the rectangular shapes on its cut and metal layers. A simple via array is a multiple-cut simple via. Simple vias and simple via arrays can be used for the following purposes:

  - Clock or signal routing using nondefault routing rules
  - Redundant via insertion
  - Power and ground routing with advanced via rules

- Custom vias

  A custom via is a multiple-cut, odd-shaped via that is created from an arbitrary collection of Manhattan polygons. Custom vias can be used only for redundant via insertion.

Via definitions can come from the following sources:

- The `ContactCode` sections of the technology file associated with the design library

  The technology file can contain definitions for simple vias, simple via arrays, and custom vias. For information about defining vias in the technology file, see the *Synopsys Technology File and Routing Rules Reference Manual*.

- Via rule `GENERATE` statements in a LEF file

  The via rule GENERATE statements define simple via arrays. For information about defining vias in a LEF file, see Reading Via Definitions from a LEF File.

- User-defined via definitions

  You can define simple vias, simple via arrays, and custom vias. For information about creating user-defined via definitions, see Creating a Via Definition.

If you want Zroute to use a via definition for signal routing, ensure that is has the following attribute settings:

- `is_default` attribute is `true`
- `is_excluded_for_signal_routing` attribute is `false`

In addition, Zroute uses nondefault vias that are explicitly specified in a fat via table in the technology file or in a nondefault routing rule defined by the `create_routing_rule` command.

Feedback

Reading Via Definitions from a LEF File

To read via definitions specified by via rule `GENERATE` statements in a LEF file, use the `read_tech_lef` command. This command supports the following LEF syntax:

```
via_rule_name GENERATE [DEFAULT]
   LAYER lower_layer_name
      ENCLOSURE lower_overhang1 lower_overhang2
   LAYER upper_layer_name
      ENCLOSURE upper_overhang1 upper_overhang2
   LAYER cut_layer_name
      RECT llx lly urx ury
      SPACING x_spacing BY y_spacing
```

The WIDTH and RESISTANCE statements are not supported.

Creating a Via Definition

To create a via definition, use the `create_via_def` command. After it is created, the via definition can be used anywhere in the design, similar to a `ContactCode` definition in the technology file. The user-specified via definition is stored in the design library, so it can be used only in that design.

The `create_via_def` command can define simple vias, simple via arrays, and custom vias. The following topics describe how to define these types of vias.

- Defining Simple Vias
- Defining Custom Vias

Defining Simple Vias

To define a simple via or via array, use the following `create_via_def` syntax:

```
create_via_def
   -cut_layer layer
   -cut_size {width height}
   -upper_enclosure {width height}
   -lower_enclosure {width height}
   [-min_rows number_of_rows]
   [-min_columns number_of_columns]
   [-min_cut_spacing distance]
   [-cut_pattern cut_pattern]
   [-is_default]
   [-force]
   via_def_name
```

You do not need to specify the enclosure layers; the tool determines them from the technology file by getting the metal layers adjacent to the specified via layer.

Feedback

By default, when you define a simple via, it is a single-cut via. To define a via array, specify the minimum number of rows and columns for the array (`-min_rows` and `-min_columns` options), as well as the minimum cut spacing (`-min_cut_spacing` option). By default, the cut pattern is a full array of cuts. To modify the cut pattern, use the `-cut_pattern` option.

To overwrite an existing via definition, use the `-force` option; otherwise, the command fails if the specified via definition already exists.

For example, to create a single-cut via definition named design_via1_HV for the VIA12 via layer, use the following command:

```
icc2_shell> create_via_def design_via1_HV \
  -cut_layer VIA12 -cut_size {0.05 0.05} \
  -lower_enclosure {0.02 0.0} -upper_enclosure {0.0 0.2} \
```

To create a via definition with an alternating 2x2 cut pattern in which the lower-left cut is omitted, use the following command:

```
icc2_shell> create_via_def design_via12 -cut_pattern "01 10"
```

To report information about the user-defined via definitions, use the `report_via_defs` command.

Defining Custom Vias

To define a custom via, use the following `create_via_def` syntax:

```
create_via_def
   -shapes { {layer {coordinates} [mask_constraint]} ... }
   [-lower_mask_pattern alternating | uniform]
   [-upper_mask_pattern alternating | uniform]
   [-force]
   via_def_name
```

In the `-shapes` option, you must specify shapes for one via layer and two metal layers, which are the enclosure layers for the via. You can specify multiple shapes per layer.

For example, to create a custom via definition, use a command similar to the following:

```
icc2_shell> create_via_def design_H_shape_via \
   -shapes { {VIA12 {0.035 -0.035} {0.100 0.035}}
             {VIA12 {-0.100 -0.035} {-0.035 0.035}}
             {METAL1 {-0.130 -0.035} {0.130 0.035}}
             {METAL2 {-0.100 -0.065} {-0.035 0.065}}
             {METAL2 {0.035 -0.065} {0.100 0.065}}
             {METAL2 {-0.100 -0.035} {0.100 0.035}} }
```

To overwrite an existing via definition, use the `-force` option; otherwise, the command fails if the specified via definition already exists.

Feedback

If you are using double-patterning technology, you can assign mask constraints to the shapes or the enclosure layers, but not both.

- To assign mask constraints to the shapes, use the `mask_one`, `mask_two`, or `mask_three` keywords for the *mask_constraint* arguments when specifying the `-shapes` option. You would use this method if the mask constraints follow an arbitrary pattern. When you create a via with the `create_via` command, the via shapes inherit the mask constraints specified in the via definition. If the mask constraint you specify for an enclosure layer when you create a via differs from the mask constraint specified for the first shape in the via definition, the specified mask constraint determines the mask-shift used for all shapes on that layer.

  For example,

```
icc2_shell> create_via_def VIA12 \
   -shapes { {M1 {-0.037 -0.010} {-0.017 0.01} mask_two}
             {M1 {0.017 -0.010} {0.037 0.01} mask_one}
             {VIA1 {-0.037 -0.01} {0.037 0.01} mask_two}
             {M2 {-0.044 -0.010} {0.044 0.010} mask_one} }
icc2_shell> create_via -net n1 -via_def VIA12 \
   -origin {100.100 200.320}
```

- To assign mask constraints to the enclosure layers, use the `-lower_mask_pattern` and `-upper_mask_pattern` options. You can specify either `uniform` or `alternating` as the mask pattern. In either case, you specify the mask constraint for the first shape when you create a via with the `create_via` command. If you specify `uniform`, all shapes on the layer use the specified mask constraint. If you specify `alternating`, the colors alternate from shape to shape after the first shape.

  For example,

```
icc2_shell> create_via_def VIA12 \
   -shapes { {M1 {-0.010 -0.030} {0.027 0.050}}
             {M1 {0.047 -0.030} {0.084 0.050}}
             {VIA1 {0.000 0.000} {0.074 0.020}}
             {M2 {-0.030 0.000} {0.104 0.020}} } \
   -lower_mask_pattern uniform
icc2_shell> create_via -via_def VIA12 -origin {000.100 200.320}
   -lower_mask_constraint mask_two
```

To report information about the via definitions, use the `report_via_defs` command. This command reports the shapes that comprise the via definition but does not report the upper and lower mask patterns.

Inserting Via Ladders

A *via ladder*, which is also referred to as a *via pillar*, is a stacked via that starts from the pin layer and extends into an upper layer where the router connects to it. Via

Feedback

ladders reduce the via resistance, which can improve performance and electromigration robustness.

To use via ladders,

- Via ladder rules must be defined in the technology data for the design

  For information about defining via ladder rules, see Defining Via Ladder Rules.

- You must insert the via ladders before you perform global routing

  The IC Compiler II tool provides the following methods for inserting via ladders:

  - Automatic insertion during preroute optimization, as described in Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization
  - Constraint-based insertion by using the `insert_via_ladders` command, as described in Constraint-Based Via Ladder Insertion
  - Manual insertion by using the `create_via_ladder` command, as described in Manual Via Ladder Insertion

See Also

- Querying Via Ladders
- Removing Via Ladders
- Controlling Via Ladder Connections

Defining Via Ladder Rules

A via ladder rule defines the number of rows and the number of cuts in each row for each layer in a via ladder. Via ladder rules can be defined in the technology file or by using the `create_via_rule` command.

For example, each of the following methods creates an identical via ladder rule:

- Define the rule in the technology file.

```
ViaRule "VL1" {
   cutLayerNameTblSize     = 2
   cutLayerNameTbl         = (VIA1, VIA2)    # via layer names
   cutNameTbl              = (CUT1A, CUT2A) # cut names
   numCutRowsTbl           = (2, 2)          # rows in ladder
   numCutsPerRowTbl        = (2, 2)          # columns in ladder
   upperMetalMinLengthTbl  = (L1, L2)        # minimum length rule
   cutXMinSpacingTbl       = (X1, X2)        # X-direction spacing
   cutYMinSpacingTbl       = (Y1, Y2)        # Y-direction spacing
   maxNumStaggerTracksTbl  = (S1, S2)        # maximum staggering
   forHighPerformance  = 0
```

Feedback

```
   forElectromigration = 1
}
```

For details about defining via ladder rules in the technology file, see the "Via Ladder Rule" topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- Define the rule on the command line by using the `create_via_rule` command with a detailed specification and then specifying additional details by setting its attributes.

```
icc2_shell> create_via_rule -name VL1 -cut_layer_names {VIA1 VIA2} \
   -cut_names {CUT1A CUT2A} -cut_rows {2 2} -cuts_per_row {2 2}
icc2_shell> set_attribute [get_via_rules VL1] \
   upper_metal_min_length_table {L1 L2}
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_x_min_spacing_table {X1 X2}
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_y_min_spacing_table {Y1 Y2}
icc2_shell> set_attribute [get_via_rules VL1] \
   max_num_stagger_tracks_table {S1 S2}
icc2_shell> set_attribute [get_via_rules VL1] \
   for_high_performance false
icc2_shell> set_attribute [get_via_rules VL1] \
   for_electro_migration true
```

- Define the rule on the command line by creating an empty via rule with the `create_via_rule` command and then specifying the details by setting its attributes.

```
icc2_shell> create_via_rule -name VL1
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_layer_name_table_size 2
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_layer_name_table {via1 via2}
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_name_table {CUT1A CUT2A}
icc2_shell> set_attribute [get_via_rules VL1] \
   num_cuts_per_row_table {2 2}
icc2_shell> set_attribute [get_via_rules VL1] num_cut_rows_table {2 2}
icc2_shell> set_attribute [get_via_rules VL1] \
   upper_metal_min_length_table {L1 L2}
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_x_min_spacing_table {X1 X2}
icc2_shell> set_attribute [get_via_rules VL1] \
   cut_y_min_spacing_table {Y1 Y2}
icc2_shell> set_attribute [get_via_rules VL1] \
   max_num_stagger_tracks_table {S1 S2}
icc2_shell> set_attribute [get_via_rules VL1] \
   for_high_performance false
icc2_shell> set_attribute [get_via_rules VL1] \
   for_electro_migration true
```

Feedback

The tool treats the via rules the same, regardless of the method used to create them. You can perform the following tasks for via rules:

- Create a collection of via rules by using the `get_via_rules` command
- Remove via rules by using the `remove_via_rules` command
- Display detailed information about the via rules by using the `report_via_rules` command
- Set or query via rule attributes

  To see the attributes supported on via rule objects, use the `list_attributes -application -class via_rule` command.
- Save the via rules in a technology file by using the `write_tech_file` command

See Also

- Generating Via Ladder Rules for Electromigration Via Ladders

Generating Via Ladder Rules for Electromigration Via Ladders

Instead of explicitly defining via ladder rules for electromigration via ladders, you can use the `generate_via_ladder_template` command to generate via ladder rules and constraints based on a configuration file. The configuration file is an XML file that provides information about the via ladder rules, as well as the association between the via ladder rules and specific pins. You must specify the name of the configuration file by using the `-config_file` option. The command generates two script files:

- A script file that defines the via ladder rules, which is called the template script file

  This script file contains a `create_via_rule` command for each template defined in the configuration file, as well as `set_attribute` commands to set the appropriate via rule attributes.

  By default, the file is named auto_gen_via_ladder_template.tcl. To specify a different file name, use the `-template_file` option.
- A script file that defines the via ladder constraints, which is called the association script file

  This script file contains `set_via_ladder_candidate` commands for the pins specified in the configuration file, as well as `set_attribute` commands to set the `is_em_via_ladder_required` attribute for the pins to `true`.

  By default, the file is named auto_gen_via_ladder_association.tcl. To specify a different file name, use the `-association_file` option.

Feedback

The configuration file has the following syntax:

```
<EmRule>
   <!-- template definition -->
   <Template name="rule_name">
      <Layer name="layer_name" row_number="cuts_per_row"
         [max_stagger_tracks="count"]
         [upper_cut_x_min_spacing="x_spacing"]
         [upper_cut_y_min_spacing="y_spacing"] />
      ... more layers
   </Template>
   … more templates

   <!-- Pin to template association -->
   <Pin name="pin_name">
      <Template name="rule_name"/>
   </Pin>
   ... more associations
</EmRule>
```

Each `Template` section specifies the template for a via ladder rule. You can specify one or more `Template` sections. Within a template section, you specify the via ladder structure by specifying the number of cuts per row for each metal layer involved in the via ladder. The command uses the information in the configuration file and associated information from the technology file to generate the via ladder rules. The command derives the following information from the technology file

- The cut layers that connect the specified metal layers
- The cut names
- The minimum required length for each metal layer

Note:

You must open the block before running the `generate_via_ladder_template` command to ensure that the command has access to the technology data for the block.

Each `Pin` section specifies the template associated with a pin, where the pin name can include the asterisk wildcard character (*). You can specify one or more `Pin` sections. The template specified in the `Pin` section's `Template` attribute must be one of the templates specified in the `Template` sections.

For example, assume you have a configuration file named vl_config that has the following contents:

```
<EmRule>
   <Template name="template_1_3231">
      <Layer name="M1" row_number="3"/>
      <Layer name="M2" row_number="2"/>
      <Layer name="M3" row_number="3"/>
```

Feedback

```
      <Layer name="M4" row_number="1"/>
   </Template>
   <Pin name="*/BCELLD5A11*/Z">
      <Template name="template_1_3231"/>
   </Pin>
</EmRule>
```

To generate the script files to define the via ladder rules and constraints, use the following command:

```
icc2_shell> generate_via_ladder_template -config_file vl_config
```

Example 15 shows the template script file generated by this command. Example 16 shows the association script file generated by this command.

Example 15 Template Script File

```
create_via_rule -name template_1_3231 \
   -cut_layer_names {VIA1 VIA2 VIA3} -cut_names {V1S V2S V3S} \
   -cut_rows {2 3 1} -cuts_per_row {3 2 3}
set viaRule [get_via_rules template_1_3231]
set_attribute $viaRule upper_metal_min_length_table {0.3 0.2 0.4}
set_attribute $viaRule for_electro_migration true
```

Example 16 Association Script File

```
foreach_in_collection pin [get_lib_pins -quiet */BCELLD5A11*/Z] {
   set_attribute -quiet $pin is_em_via_ladder_required true
   set_via_ladder_candidate $pin -ladder_name "template_1_3231"
}
```

See Also

- Defining Via Ladder Rules
- Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

Generating Via Ladder Rules for Performance Via Ladders

Instead of explicitly defining via ladder rules for performance via ladders, you can use the `setup_performance_via_ladder` command to generate the via ladder rules and associations.

By default, this command

- Generates via ladder rules for all layers up to the M5 layer

  To specify a different maximum layer, use the `-max_layer` attribute.

Feedback

The command uses information from the technology file to generate the via ladder rules. The command derives the following information from the technology file:

- The cut layers that connect the specified metal layers
- The cut names
- The minimum required length for each metal layer

Note:

You must open the block before running the `setup_performance_via_ladder` command to ensure that the command has access to the technology data for the block.

The command outputs an XML file that provides information about the via ladder rules and a script file that defines the via ladder rules. For details about these files, see Via Ladder Rule Files.

You can also generate these files by using the `generate_via_rules_for_performance` command.

- Associates via ladder rules with the pins of all library cells that do not have a `dont_use` attribute

  To include the pins of library cells that have a `dont_use` attribute, use the `-dont_use` option.

  To associate via ladder rules only with specific pins, use the `-lib_pins` option.

  The command analyzes the information about each library cell pin and its shapes to determine the via ladder rules to associate with that pin and then outputs a script file that defines the associations. For details about this file, see Via Ladder Association File.

  You can also generate this file by using the `associate_performance_via_ladder` command.

- Runs the generated script files

See Also

- Defining Via Ladder Rules
- Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

Feedback

Via Ladder Rule Files

The command generates two via ladder rule files:

- An XML file that provides information about the via ladder rules

  The XML file has the following syntax:

```
<EmRule>
   <!-- template definition -->
   <Template name="rule_name" for_electro_migration="true"
      for_high_performance="true">
      <Layer name="layer_name" row_number="cuts_per_row"
      ... more layers
   </Template>
   … more templates

</EmRule>
```

  Each `Template` section specifies the template for a via ladder rule. Within a template section, the via ladder structure is specified by specifying the number of cuts per row for each metal layer involved in the via ladder.

  By default, the generated XML file is named auto_perf_via_ladder_rule.xml. To specify a different name, use the `-xml_file` option.

- A script file that defines the via ladder rules

  The script file contains a `create_via_rule` command for each template defined in the XML file, as well as `set_attribute` commands to set the appropriate via rule attributes.

  By default, the generated script file is named auto_perf_via_ladder_rule.tcl. To specify a different file name, use the `-rule_file` option.

Via Ladder Association File

The command generates a script file that defines the association between the library cell pins and the via ladder rules.

This script file contains `set_via_ladder_candidate` commands for the library cell pins.

By default, the generated script file is named auto_perf_via_ladder_association.tcl. To specify a different file name, use the `-association_file` option.

Feedback

Constraint-Based Via Ladder Insertion

To use the constraint-based method to insert via ladders,

1. Ensure that the via ladder rules are defined, as described in Defining Via Ladder Rules.
2. Define the via ladder constraints by using the `set_via_ladder_rules` command, as described in Defining Via Ladder Constraints.
3. Insert the via ladders by using the `insert_via_ladders` command, as described in Inserting Via Ladders.
4. Verify the via ladders by using the `verify_via_ladders` command, as described in Verifying Via Ladders.

Feedback

Defining Via Ladder Constraints

The via ladder constraints define the pins on which to insert via ladders and the via ladder templates that can be used for those pins. You can define both global and instance-specific via ladder constraints; if the constraints conflict, the instance-specific constraints override the global constraints.

- To define global via ladder constraints, which are referred to as via ladder rules, use the `set_via_ladder_rules` command, as described in Defining Global Via Ladder Constraints.
- To define instance-specific via ladder constraints, use the `set_via_ladder_constraints` command, as described in Defining Instance-Specific Via Ladder Constraints.

In addition, you can specify via ladder candidates for specific pins by using the `set_via_ladder_candidate` command. This command is used by preroute optimization to insert via ladders on timing-critical paths, as described in Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization.

To report the via ladder constraints, use the `report_via_ladder_constraints` command.

See Also

- Inserting Via Ladders

Defining Global Via Ladder Constraints

To define global via ladder constraints, which are referred to as via ladder rules, use the `set_via_ladder_rules` command to set via ladder constraints on library cell pins. You must specify the mapping between library cell pins and via ladder templates, as well as the pins to which to apply the mapping.

Use the following options to specify the mapping:

- `-master_pin_map` or `-master_pin_map_file`

  These options specify the mappings for specific library pins using the following format:

  `{ {`*`lib_cell/pin`* `{`*`via_ladder_list`*`}} ...}`

Feedback

To specify the mapping on the command line, use the `-master_pin_map` option. To specify the mapping in an external file, use the `-master_pin_map_file` option to specify the mapping file name.

- `-default_ladders`

  This option specifies the via ladder templates to use for all library pins not explicitly specified in the mapping.

Use the following options to specify the pins to which to apply the mapping:

- `-all_instances_of`

  This option explicitly specifies library cell pins to which to apply the mapping.

- `-all_clock_outputs`

  Set this option to `true` to apply the mapping to all clock output pins.

- `-all_clock_inputs`

  Set this option to `true` to apply the mapping to all clock input pins.

- `-all_pins_driving`

  This option applies the mapping to all pins that drive one of the specified ports.

To report the via ladder rules, use the `report_via_ladder_rules` command. To remove via ladder rules, use the `remove_via_ladder_rules` command.

Defining Instance-Specific Via Ladder Constraints

To define instance-specific via ladder constraints, use the `set_via_ladder_constraints` command. You must specify the instance pins and the via ladder templates that can be used for those pins.

For example, to enable the use of the VL1, VL2, and VL3 via ladder templates for the u1/i1 and u2/i2 pins, use the following command:

```
icc2_shell> set_via_ladder_constraints -pins {u1/i1 u2/i2} \
   {VL1 VL2 VL3}
```

To report the via ladder constraints, use the `report_via_ladder_constraints` command. To remove via ladder constraints, use the `remove_via_ladder_constraints` command.

Feedback

Inserting Via Ladders

To insert via ladders, use the `insert_via_ladders` command. By default, this command

- Does not remove existing via ladders in the block

  To remove the existing via ladders before via ladder insertion, use the `-clean true` option.

- Targets all pins specified in the via ladder constraints, whether on signal nets or clock nets

  To restrict via ladder insertion to specific nets, use the `-nets` option.

- Inserts a single via ladder on each target pin

  If the constraints specify more than one via ladder template for a pin, the command inserts the via ladder using the first template that does not cause a DRC violation.

  - By default, if all of the via ladder templates cause a DRC violation, the command inserts the via ladder with the lowest DRC cost.
  - To ignore detail routing shapes when checking for DRC violations, use the `-ignore_routing_shape_drcs true` option.
  - To ignore the line-end cut enclosure rule, use the `-relax_line_end_via_enclosure_rule true` option.
  - To ignore metal spacing rules on the pin layer when the via ladder shape is fully enclosed within the pin, use the `-relax_pin_layer_metal_spacing_rules true` option.
  - To prevent the insertion of via ladders that cause DRC violations, use the `-allow_drcs false` option.

  Note:

  If a pin is too small to accommodate any of the via ladders specified for it, the `insert_via_ladders` command does not insert a via ladder for that pin.

- Uses via cuts other than those specified in the template if they improve compliance with fat via rules

  To require the command to use only those via cuts specified in the template, use the `-strictly_honor_cut_table true` option.

- Centers the via ladder cuts at the intersection of routing tracks between adjacent layers

  To shift the via ladder cuts on transition layers off the routing tracks for improved DRC compliance, use the `-shift_vias_on_transition_layers true` option.

Feedback

- Honors nondefault width rules for the wires on all layers of the via ladder

  To use the default width for wires on lower layers of the via ladder and honor the nondefault width only for the top layer, use the `-ndr_on_top_layer_only true` option.

- Does not require the via inserted above the pin layer to be contained within the pin shape

  To prevent changes to the pin shape boundary, you can require that the via inserted above the pin layer be fully contained within the pin shape boundary, which includes the pin shape and its extensions. To specify this requirement, use one or both of the following options:

  - `-connect_within_metal true`

    This option controls via enclosures for all types of via ladders: performance, electromigration, and pattern-must-join.

  - `-connect_within_metal_for_via_ladder true`

    This option controls the via enclosures only for performance and electromigration via ladders. By default, the tool uses the global setting specified by the `-connect_within_metal` option. To explicitly specify the behavior for performance and electromigration via ladders, set this option to `true` or `false`.

- Does not extend the via enclosures to meet the minimum length rule for the layer

  To enable patching of the via enclosures to meet the minimum length rule and allow additional stacking of the via ladder, use the `-allow_patching true` option.

- Uses staggering only when it is defined for the via ladder rule

  Via ladder staggering extends the row metal of a via ladder level in the preferred direction of the upper layer for the level to avoid obstructions above or near the pin, which can increase the success rate of via ladder insertion. In many cases, the via ladder rule specifies the maximum number of stagger tracks. To allow staggering if the maximum number of stagger tracks is not defined for the via ladder rule, use the `-auto_stagger true` option.

- Does not report on the insertion status

  To report the via ladder insertion status for each target pin, use the `-verbose true` option.

Feedback

To output detailed information about insertion failures, use the `-user_debug true` option. The detailed information reports the following types of violations:

- Internal DRC violations

  These are DRC violations internal to a via ladder. These violations are never allowed.

- Hard DRC violations

  These are DRC violations between a via ladder and a fixed shape, such as a pin, preroute, or obstruction. These violations are never allowed.

- Soft DRC violations

  These are DRC violations between a via ladder and a detail routing shape, which can be rerouted. These violations are allowed if one or more of the following options are `true`: `-allow_drcs`, `-ignore_rippable_shapes`, or `-ignore_routing_shape_drcs`.

After inserting the via ladders, the command reports statistics on the inserted ladders and any DRC violations caused by the insertion.

See Also

- Defining Via Ladder Constraints
- Verifying Via Ladders
- Updating Via Ladders
- Querying Via Ladders
- Removing Via Ladders

Protecting Via Ladders

To prevent via ladders from being edited, set the `design.enable_via_ladder_protection` application option to `true`.

```
icc2_shell> set_app_options -name design.enable_via_ladder_protection \
   -value true
```

When the `design.enable_via_ladder_protection` application option is `true`, the following commands process only objects that do not belong to via ladders:

- `remove_shapes`
- `remove_vias`

Feedback

- remove_objects
- set_attribute
- set_via_def
- add_to_edit_group

If an object belongs to a via ladder, these commands issue a warning message and do not process the object.

The following example sets the design.enable_via_ladder_protection application option to true and attempts to remove a collection of shapes:

```
icc2_shell> set_app_options -name design.enable_via_ladder_protection \
   -value true
...
icc2_shell> sizeof_collection [get_shapes -of net1]
77

icc2_shell> remove_shapes [get_shapes -of net1] -verbose
Warning: Cannot edit PATH_32_7403 because it is part of via_ladder.
 (NDM-160)
76
```

Verifying Via Ladders

To verify that the via ladders in the block match the via ladder constraints and are properly connected to pins, use the verify_via_ladders command. By default, this command checks all via ladders. To restrict the checking to specific nets, use the -nets option.

Note:

If you use the -shift_vias_on_transition_layers true option when you insert the via ladders, you must also use this option when you verify the via ladders; otherwise, the command reports false violations.

A via ladder matches the via ladder constraints if the pattern of connected vias and wires for the via ladder structure satisfies any of the via ladder templates assigned to the connected pin. The command flags a violation in the following situations:

- A via ladder is connected to a pin that does not have a via ladder constraint
- A via ladder is connected to a pin with a via ladder constraint, but none of the templates correctly describe the current via ladder structure
- A pin has a via ladder constraint, but does not have a via ladder connected to it

Feedback

A via ladder is properly connected to a pin if all the via enclosures at the base of the via ladder touch the same pin and that pin is logically connected to the same net as the via ladder. The command flags a violation in the following situations:

- An enclosure of the via ladder base touches a different pin, unless the pins are must-join pins that belong to the same must-join set
- An enclosure of the via ladder base does not touch any pin
- An enclosure of the via ladder touches a pin that is not of the same net as the via ladder

By default, the `verify_via_ladders` command reports the via ladder insertion status for each target pin, which can result in a very large report. To limit the number of via ladders reported for each category to 40, use the `-report_all_via_ladders false` option with the `verify_via_ladders` command.

Updating Via Ladders

You can use the following methods to update existing via ladders in a design:

- Use the `refresh_via_ladders` command
- Automatic updates during the `route_group`, `route_auto`, and `route_eco` commands is enabled by default by setting the `route.auto_via_ladder.update_during_route` application option to `true`

Both of these methods perform the following tasks:

- Verifies the existing via ladders based on the current via ladder constraints
- Removes invalid via ladders
- Inserts via ladders where needed based on the settings of the `route.auto_via_ladder` application options

  By default, the tool

  - Updates all pins with via ladder constraints, whether on signal nets or clock nets

    By default, during the `route_group` command, the via ladder updates is restricted to only those nets specified in the `route_group` command by setting the `route.auto_via_ladder.update_on_route_group_nets_only` application option to `true`.

  - Does not require the via inserted above the pin layer to be contained within the pin shape

    To prevent changes to the pin shape boundary, you can require that the via inserted about the pin layer be fully contained within the pin shape boundary, which includes

Feedback

the pin shape and its extensions. To specify this requirement, set the following application options:

- `route.auto_via_ladder.connect_within_metal`

  This application option controls via enclosures for all types of via ladders: performance, electromigration, and pattern-must-join. To prevent changes to the pin shape boundary, set this application option to `true`.

- `route.auto_via_ladder.connect_within_metal_for_via_ladder`

  This application option controls the via enclosures only for performance and electromigration via ladders. By default, the tool uses the global setting specified by the `route.auto_via_ladder.connect_within_metal` application option. To explicitly specify the behavior for performance and electromigration via ladders, set this application option to `true` or `false`.

◦ Uses staggering only when it is defined for the via ladder rule

Via ladder staggering extends the row metal of a via ladder level in the preferred direction of the upper layer for the level to avoid obstructions above or near the pin, which can increase the success rate of via ladder insertion. In many cases, the via ladder rule specifies the maximum number of stagger tracks. To allow staggering if the maximum number of stagger tracks is not defined for the via ladder rule, the `route.auto_via_ladder.auto_stagger` application option is set to `true` by default.

◦ Reports the via ladder insertion status for each target pin

To limit the number of via ladders reported for each category to 40, set the `route.auto_via_ladder.report_all_via_ladders` application option to `false`.

See Also

- Inserting Via Ladders
- Verifying Via Ladders
- Querying Via Ladders
- Removing Via Ladders

Feedback

Manual Via Ladder Insertion

To manually insert a via ladder,

1. Create the shapes that compose the via ladder by using the `create_shape` and `create_via` commands.
2. Create the via ladder by using the `create_via_ladder` command.

   At a minimum, you must specify the shapes that compose the via ladder by using the `-shapes` option.

   The command creates a via ladder named VIA_LADDER_*n*, where *n* is a unique integer.

Use the following options with the `create_via_ladder` command to specify attributes of the created via ladder:

- `-via_rule`

  This option specifies the via ladder rule associated with the created via ladder. It sets the `via_rule_name` attribute of the created via ladder.

- `-electromigration`

  This option specifies that the via ladder is an electromigration via ladder. It sets the `is_electromigration` attribute of the created via ladder to `true`.

- `-high_performance`

  This option specifies that the via ladder is a performance via ladder. It sets the `is_high_performance` attribute of the created via ladder to `true`.

- `-pattern_must_join`

  This option specifies that the via ladder is a pattern-must-join via ladder. It sets the `is_pattern_must_join` attribute of the created via ladder to `true`.

- `-pin`

  This option specifies the physical pin connected to the created via ladder. It sets the `pin` attribute of the created via ladder.

See Also

- Querying Via Ladders
- Removing Via Ladders

Feedback

Querying Via Ladders

To query the via ladders in a block, use the `get_via_ladders` command. To report on the via ladders in a block, use the `report_via_ladders` command.

Removing Via Ladders

To remove via ladders and their associated shapes, use the `remove_via_ladders` command. You must specify the via ladders to remove. To remove all via ladders from the block, use the asterisk (*) wildcard character, as shown in the following example:

```
icc2_shell> remove_via_ladders *
```

To remove via ladders only from specific nets, use the `-nets` option.

Checking Routability

After placement is completed, you can use the `check_routability` command to check whether your block is ready for detail routing.

By default, this command checks for

- Blocked standard cell ports

  A standard cell port is considered blocked if none of its physical pins is accessible.

  A standard cell pin is considered accessible if the pin contains a via that extends to a neighboring layer, there is a path on the pin layer that is at least as long as the search range distance, or there is a shorter path on the pin layer that ends at a via to a neighboring layer. By default, the search range distance is two times the layer pitch. To change this distance, use the `-standard_cell_search_range` option to specify a different pitch multiplier, up to a maximum of 10. If you specify a value larger than 10, the command sets the value to 10.

  By default, the tool does not check whether via connections are fully inside the standard cell pins. To enable this check, use the `-connect_standard_cells_within_pins true` option.

  To disable the checking of blocked standard cell ports, use the `-check_standard_cell_blocked_ports false` option.

Feedback

- Blocked top-level or macro cell ports

  A top-level or macro cell port is considered blocked if none of its physical pins is accessible.

  A top-level or macro pin is considered accessible if a legal path can be extended from the pin to a certain distance around it. This path can be just on the pin layer or can extend to a neighboring layer by a single via. By default, the distance is 10 times the layer pitch. To change this distance for same-layer paths, use the `-blocked_range` option to specify a different pitch multiplier, up to a maximum of 40. To change this distance for neighboring-layer paths through a via, use the `-blocked_range_via_side` option to specify a different pitch multiplier, up to a maximum of 40. If you specify a value larger than 40 for either of these options, the command sets its value to 40.

  By default, the command checks whether a pin is accessible in either the horizontal or vertical direction. To require that the pin is accessible in the preferred direction for its layer, use the `-obey_direction_preference true` option.

  To disable the checking of blocked top-level or macro cell ports, use the `-check_non_standard_cell_blocked_ports false` option.

- Out-of-boundary pins

  This check verifies that all pins are within the block boundary.

  To disable the checking of out-of-boundary pins, use the `-check_out_of_boundary false` option.

- Minimum grid violations

  This check verifies that all pins, including those within library cells, are on the minimum grid, as defined by the `gridResolution` attribute in the technology file.

  To disable the checking of minimum grid violations, use the `-check_min_grid false` option.

- Incorrect via definitions

  This check verifies that

  - Uncolored via arrays do not have more than 20 rows or columns
  - Custom or asymmetric simple via definitions do not have more than 1000 cuts
  - The design does not contain more than 65535 via definitions

  You cannot disable these checks.

Feedback

- Invalid real metal via cut blockages

  This check verifies that all real metal via cut blockages match a legal cut size, as defined by the `cutWidthTbl`, `cutHeightTbl`, and `minWidth` attributes in the technology file.

  To disable this check, use the `-check_via_cut_blockage false` option.

- Minimum width settings

  This check verifies that the nondefault minimum width and shield width settings are no larger than the maximum width defined in the technology file.

  You cannot disable these checks.

The `check_routability` command also supports the following optional checks:

- Blocked power or ground ports

  To enable this check, use the `-check_pg_blocked_ports true` option.

- Redundant power or ground shapes

  To enable this check, use the `-check_redundant_pg_shapes true` option.

- Staggered power and ground vias that block the routing tracks

  To enable this check, use the `-check_routing_track_space true` option.

- Blocked ports on frozen nets

  To enable this check, use the `-check_frozen_net_blocked_ports true` option.

- Blocked unconnected pins

  To enable this check, use the `-check_no_net_pins true` option.

- Real metal blockages that overlap library cell pins

  To enable this check, use the `-check_real_metal_blockage_overlap_pin true` option.

- Invalid real metal via cut blockages in the library cells

  To enable this check, use the `-check_lib_via_cut_blockage true` option.

Feedback

- Shielding checks

  These checks detect possible issues with shielding by checking for the following conditions:

  - Signal net shapes with a `shape_use` attribute of `shield_route`.
  - PG net shapes, which might be a PG strap or rail, but have a `shape_use` attribute of `detail_route`.
  - Signal, clock, or PG nets that have a shielding nondefault rule but no associated shield shapes, which might be caused by inappropriate `shape_use` attributes.

  To enable these checks, use the `-check_shield true` option.

- Via ladder checks

  These checks detect possible issues with via ladder insertion by checking for the following conditions:

  - Library cell pins whose top-layer terminals have multiple pin shapes but do not have a `pattern_must_join` attribute

    This check applies only to signal and secondary PG pins.
  - Missing performance or electromigration via ladder on a pin that has the `is_em_via_ladder_required` attribute
  - Via ladder application options that do not have the required settings

  To enable these checks, use the `-via_ladder true` option.

The `check_routability` command supports the following additional pin connection controls that apply to all pin access checks:

- Pin access edges

  During frame view extraction, the tool annotates the frame views with information about the pin access edges. By default, the `check_routability` command ignores the pin access edges and allows pin connections at any point. You can restrict pin connections to the defined access edges by setting the `-obey_access_edges` option to `true`. You can further restrict pin connections to the access edge mark, which can be a narrow rectangle, a short line, or even a single point, by setting the `-access_edge_whole_side` option to `true`. In addition, the command can report unconnected pins that do not have an access edge defined. To enable this check, use the `-report_no_access_edge true` option.

- Via rotation

  By default, pin connections can use rotated vias. To disallow pin connections that use rotated vias, set the `-allow_via_rotation` option to `false`.

Feedback

By default, this command considers the following routing layer constraints when checking for blocked ports:

- The global minimum and maximum routing layer constraints set by the `-min_routing_layer` and `-max_routing_layer` options of the `set_ignored_layers` command

  These constraints can cause blocked ports only if they are defined as hard constraints. By default, these constraints are soft constraints. They are hard constraints only if the `route.common.global_min_layer_mode` and `route.common.global_max_layer_mode` application options are set to `hard`.

- The net-specific minimum and maximum routing layer constraints set by the `-min_routing_layer` and `-max_routing_layer` options of the `set_routing_rule` command

  These constraints can cause blocked ports only if they are defined as hard constraints. By default, the net-specific minimum layer constraint is a soft constraint and the net-specific maximum layer constraint is a hard constraint. They are hard constraints only if the `route.common.net_min_layer_mode` and `route.common.net_max_layer_mode` application options are set to `hard`.

- The clock minimum and maximum routing layer constraints set by the `-min_routing_layer` and `-max_routing_layer` options of the `set_clock_routing_rules` command

  These constraints can cause blocked ports only if they are defined as hard constraints. By default, the clock minimum layer constraint is a soft constraint and the clock maximum layer constraint is a hard constraint. They are hard constraints only if the `route.common.net_min_layer_mode` and `route.common.net_max_layer_mode` application options are set to `hard`.

- The freeze layer constraints set by the `route.common.freeze_layer_by_layer_name` and `route.common.freeze_via_to_frozen_layer_by_layer_name` application options

To ignore the routing layer constraints during the blocked port checks, use the `-honor_layer_constraints false` option.

To specify the maximum number of DRC violations that the tool reports, use the `-max_error` option. If the `-max_error` is set to `20`, the toll reports the first 20 DRC violations.

You can use the error browser to examine the errors detected by the `check_routability` command. By default, the error data generated by the `check_routability` command is named check_routability.err; to specify a different name for the error data, use the `-error_data` option. The error data is saved in the design library when you save the

Feedback

block. For information about using the error browser, see the *IC Compiler II Graphical User Interface User Guide*.

Feedback

Routing Constraints

Routing constraints provide guidance during routing. Table 23 describes the types of guidance provided by the routing constraints.

Table 23 Routing Constraints

Guidance	Description
Controlling the layers used for routing	For information about specifying the routing layers, see Specifying the Routing Resources.
Setting stricter minimum width and spacing rules	To set stricter minimum width and spacing rules, define a nondefault routing rule and apply it to the affected nets. For information about defining and applying nondefault routing rules, see Using Nondefault Routing Rules.
Controlling routing through voltage areas	By default, routing can pass through any voltage areas. To restrict the voltage-area pass-throughs, define a voltage area routing rule, as described in Controlling Physical-Feedthrough Nets in Voltage Areas.
Controlling the routing direction	You can specify the preferred routing direction for specific layers or for specific regions. • For information about specifying the preferred routing direction for specific layers, see Setting the Preferred Routing Direction for Layers. • To control the routing direction in a specific region, use either a preferred-direction-only routing guide or a switch-preferred-direction routing guide. For information about routing guides, see Defining Routing Guides.
Limiting the number of edges in the nonpreferred routing direction	To limit the number of edges in the nonpreferred routing direction, use a maximum-number-of-pattern routing guide. For information about routing guides, see Defining Routing Guides.
Controlling off-grid routing	By default, off-grid routing is allowed for wires or vias unless the `onWireTrack` or `onGrid` attribute is set to 1 for the layer in the technology file. You can prevent or discourage off-grid routing, as described in Controlling Off-Grid Routing.
Preventing routing of specific nets	To prevent routing of specific nets, set the `physical_status` attribute of each net to `locked`, as described in Setting the Rerouting Mode.
Preventing routing in specific region	To prevent routing in a specific region, define a routing blockage, as described in Defining Routing Blockages.
Reserving space for top-level routing	To reserve space for top-level routing, create a corridor routing blockage, as described in Reserving Space for Top-Level Routing.
Restricting routing to specific regions	To restrict routing to a specific region, use a routing corridor, as described in Routing Nets Within a Specific Region.

Feedback

Table 23 Routing Constraints (Continued)

Guidance	Description
Prioritizing routing regions	To prioritize routing regions, use an access preference routing guide. For information about routing guides, see Defining Routing Guides.
Controlling the routing density	To control the routing density for specific layers, use a utilization routing guide. For information about routing guides, see Defining Routing Guides.
Encouraging river routing	To encourage river routing on specific layers, use a single-layer routing guide. For information about routing guides, see Defining Routing Guides.
Improving routability of hard macro pins	To improve the routability of hard macro pins, use pin access routing guides, as described in Deriving Pin Access Routing Guides.
Controlling routing along the perimeter of a block	To control detail routing around the perimeter of a block, insert constraint objects, as described in Controlling Routing Around the Block Boundary.
Creating routing guides for the metal cut allowed and forbidden preferred grid extension rules	You can use the following methods to create these routing guides: • Automatic creation during boundary cell insertion, as described in Creating Routing Guides During Boundary Cell Insertion • Manual creations by using the `derive_metal_cut_route_guides` command, as described in
Controlling the pin connections	You can control both the allowed types of pin connections and the pin tapering. • For information about must-join pins, see Routing Must-Join Pins. • For information about controlling the allowed types of pin connections, see Controlling Pin Connections. • For information about controlling pin tapering, see Controlling Pin Tapering.
Controlling via ladder connections	For information about controlling the via ladder connections, see Controlling Via Ladder Connections.
Restricting the extent of rerouting	For information about restricting the extent of rerouting for specific nets, see Setting the Rerouting Mode.

Feedback

Defining Routing Blockages

A routing blockage defines a region where routing is not allowed on specific layers. Zroute considers routing blockages to be hard constraints.

To define a routing blockage, use the `create_routing_blockage` command. At a minimum, you must define the boundary of the routing blockage and the affected layers.

- To create a rectangular routing blockage, use the `-boundary` option to specify the lower-left and upper-right corners of the rectangle using the following syntax: `{ {llx lly} {urx ury} }`.
- To create a rectilinear routing blockage, use the `-boundary` option to specify the coordinates of the polygon using the following syntax: `{ {x1 y1} {x2 y2} ... }`.

  You can also create a rectilinear routing blockage by specifying the polygon as the combined area of a heterogeneous collection of objects with physical geometry, such as poly_rects, geo_masks, shapes, layers, and other physical objects. If you specify a layer, the resulting area includes the area of every shape on the layer. For all other objects, the resulting area includes the area of each object. You can create multiple routing blockages by specifying the boundary polygon as a geo_mask or a collection of physical objects. If the resulting area resolves to multiple, noncontiguous polygons, the command creates multiple routing blockages, one corresponding to each polygon.
- To specify the affected layers, use the `-layers` option.

  Specify the routing layers by using the layer names from the technology file. The layers can be metal, via, or poly layers.

By default, the tool creates a routing blockage named RB_*objId* in the current block that prevents routing of all nets within the routing blockage boundary. The routing blockage is considered as real metal for RC extraction and DRC checking and the router must meet the minimum spacing requirements between the routing blockage boundary and the net shapes. Use the following options to change the default behavior:

- `-name` *`blockage_name`*

  Specifies a name for the routing blockage.
- `-cell` *`cell`*

  Creates the routing blockage in a different physical cell.

  When you use this option, the tool creates the routing blockage in the cell's reference block using the coordinate system of the cell's top-level block.
- `-zero_spacing`

Feedback

Disables the minimum spacing rule between the routing blockage boundary and the net shapes (zero minimum spacing). This option also prevents the routing blockage from being treated as real metal during extraction and DRC checking.

When you use this option, the net shapes can touch, but not overlap, the routing blockage boundary.

Note:

When you create a routing blockage to prevent via insertion, you must use the `-zero_spacing` option; otherwise, frame view extraction does not use the route guide to trim the via region.

- `-net_types` *`list_of_types`*

  Applies the routing blockage only to the specified net types.

  Specify one or more of the following net types: `analog_ground`, `analog_power`, `analog_signal`, `clock`, `deep_nwell`, `deep_pwell`, `ground`, `nwell`, `power`, `pwell`, `reset`, `scan`, `signal`, `tie_high`, and `tie_low`. To remove the net type settings and prevent routing of all nets in the routing blockage, use the `-net_types unset` setting.

To prevent signal routing on the M1 layer within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_blockage \
   -boundary { {0.0 0.0} {100.0 100.0} } \
   -net_types signal -layers M1
```

To prevent vias on the V1 layer within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_blockage \
   -boundary { {0.0 0.0} {100.0 100.0} } \
   -net_types signal -layers V1 -zero_spacing
```

To prevent PG routing on the M2 layer within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_blockage \
   -boundary { {0.0 0.0} {100.0 100.0} } \
   -net_types {power ground} -layers M2
```

Feedback

Reserving Space for Top-Level Routing

To reserve space for top-level routing, create a corridor routing blockage by using the `-reserve_for_top_level_routing` option when you create the routing blockage.

During block-level implementation, a corridor routing blockage acts as a regular blockage to prevent routing in the blockage area. During frame view extraction, the tool removes the corridor routing blockage to allow top-level routing in this area.

Querying Routing Blockages

To find routing blockages, use the `get_routing_blockages` command. For example, to get all the routing blockages in a block, use the following command:

```
icc2_shell> get_routing_blockages *
```

To find the routing blockages for specific nets, use the `-of_objects` option to specify the nets of interest. For example, to find the routing blockages for the n1 net, use the following command:

```
icc2_shell> get_routing_blockages -of_objects [get_nets n1]
```

To find the routing blockages in a specific location, use the `get_objects_by_location -classes routing_blockage` command.

Removing Routing Blockages

To remove routing blockages from the current block, use the `remove_routing_blockages` command.

- To remove specific routing blockages, specify the routing blockages, either as a list or collection, such as that returned by the `get_routing_blockages` command.
- To remove all routing corridors, specify the `-all` option.

Feedback

Defining Routing Guides

Routing guides provide routing directives for specific areas of a block. Some types of routing guides are user-defined; others are derived from the block data. This topic describes how to create user-defined routing guides.

Note:

User-defined routing guides are honored by Zroute; however, they are not honored by the Advanced Route tool.

To define a routing guide, use the `create_routing_guide` command. When you define a routing guide, you must specify

- Its rectangular boundary

  To specify the boundary, use the `-boundary` option to specify the lower-left and upper-right corners of the rectangle using the following syntax:

  `{{llx lly} {urx ury}}`

- Information specific to the purpose of the routing guide

  The following table describes the user-defined routing guides.

Table 24 User-Defined Routing Guides

Purpose	Option
Route all nets in the preferred direction within the routing guide boundary. For details, see Using Routing Guides to Control the Routing Direction.	`-preferred_direction_only`
Switch the preferred routing direction within the routing guide boundary. For details, see Using Routing Guides to Control the Routing Direction.	`-switch_preferred_direction`
Limit the number of occurrences of a specific routing pattern within the routing guide boundary. For details, see Using Routing Guides to Limit Edges in the Nonpreferred Direction.	`-max_patterns`
Control the routing density within the routing guide boundary. For details, see Using Routing Guides to Control the Routing Density.	`-horizontal_track_utilization` `-vertical_track_utilization`

Feedback

Purpose	Option
Prioritize regions within the routing guide boundary for routing. For details, see Using Routing Guides to Prioritize Routing Regions.	`-access_preference`
Encourage river routing within the routing guide boundary. For details, see Using Routing Guides to Encourage River Routing.	`-river_routing`

By default, the tool creates a routing guide named RD#*n* in the current block, where *n* is a unique integer. Use the following options to change the default behavior:

- To specify a name for the routing guide, use the `-name` option.
- To specify the routing layers affected by the routing guide, use the `-layers` option.

  Specify the routing layers by using the layer names from the technology file.
- To create the routing guide in a different physical cell, use the `-cell` option.

  When you use this option, the tool creates the routing guide in the cell's reference block using the coordinate system of the cell's top-level block.

See Also

- Deriving Routing Guides
- Querying Routing Guides
- Removing Routing Guides

Using Routing Guides to Control the Routing Direction

You can use a routing guide to control the routing direction within the routing guide boundary, either by requiring routes to be in the preferred direction within the routing guide boundary or by switching the preferred direction within the routing guide boundary.

- To force the router to route all nets in the preferred direction within the routing guide boundary, use the `-preferred_direction_only` option with the `create_routing_guide` command.

  You can use this type of routing guide to prevent wrong-way jog wires on specific layers.

  By default, this routing guide applies to all layers within the routing guide boundary. To require preferred direction routing only for specific layers, use the `-layers` option to specify the affected layers.

Feedback

For example, to force the router to use only the preferred direction on the M4 layer within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
   -layers {M4} -preferred_direction_only
```

- To switch the preferred routing direction within the routing guide boundary, use the `-switch_preferred_direction` option with the `create_routing_guide` command.

  You can use this type of routing guide to allow routing over macros, which might reduce congestion for a block that contains much detour routing.

  By default, this routing guide applies to all layers within the routing guide boundary. To switch the preferred routing direction only for specific layers, use the `-layers` option to specify the affected layers.

  For example, to switch the preferred routing direction for the M1 and M2 layers within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

  ```
  icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
     -layers {M1 M2} -switch_preferred_direction
  ```

Note:

If a switch-preferred-direction routing guide overlaps with a preferred-direction-only routing guide, the switch-preferred-direction routing guide takes precedence.

Using Routing Guides to Limit Edges in the Nonpreferred Direction

You can use a routing guide to limit the number of occurrences of a specific routing pattern within the routing guide boundary on specific layers. Zroute tries to route the nets within the routing guide to meet this constraint; however, if the number of edges in the nonpreferred direction exceeds the specified threshold, Zroute reports a violation.

To create this type of routing guide, use the `-max_patterns` option with the `create_routing_guide` command. Use the following syntax to specify the routing pattern and its threshold for each affected layer:

```
{ {layer pattern limit} ... }
```

In addition to using the `-max_patterns` option, you must also use the `-layers` option when you create this type of routing guide. Specify the same layers in the `-layers` option as those specified in the `-max_patterns` option.

Feedback

If you do not specify a pattern threshold for a layer, the routing pattern has no limit for that layer.

Currently, the only supported pattern is `non_pref_dir_edge`, which represents the edges in the nonpreferred direction. For example, to limit the number of edges in the nonpreferred direction to two on M2 and to three on M4, with no limits on other layers, use the following command:

```
icc2_shell> create_routing_guide -boundary {{50 50} {200 200}} \
   -max_patterns {{M2 non_pref_dir_edge 2} {M4 non_pref_dir_edge 3}} \
   -layers {M2 M4}
```

Using Routing Guides to Control the Routing Density

By default, the maximum track utilization is 100 percent. You can use a routing guide to control the routing density within the routing guide boundary.

- To set the maximum track utilization for layers with a horizontal preferred direction, use the `-horizontal_track_utilization` option with the `create_routing_guide` command.
- To set the maximum track utilization for layers with a vertical preferred direction, use the `-vertical_track_utilization` option with the `create_routing_guide` command.

By default, when you create these routing guides, they apply to all layers within the routing guide boundary. To set the routing density only for specific layers, use the `-layers` option to specify the affected layers.

For example, to set a maximum track utilization of 50 percent for all layers with a horizontal preferred direction and a maximum track utilization of 30 percent for all layers with a vertical preferred direction within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
   -horizontal_track_utilization 50 -vertical_track_utilization 30
```

Using Routing Guides to Prioritize Routing Regions

You can use a routing guide to prioritize regions within the routing guide boundary for routing. The regions are called access preference areas. You prioritize the access preference areas by assigning strengths to them. Access preference areas with higher strengths are preferred for routing. You can define access preference areas for both wires and vias. When you define a via access-preference area, you are defining a preference

Feedback

area for the via surrounds on the specified metal layer for vias coming from both above and below that metal layer.

To create this type of routing guide, use the `-access_preference` option with the `create_routing_guide` command. Use the following syntax to specify the access preference areas and their strengths:

```
{layer [{wire_access_preference wire_rect wire_strength}]
       [{via_access_preference via_rect via_strength}] ...}
```

You can define multiple access preference areas. To define the relative preference of the access preference areas, use strength values between 0 and 1. If access preference areas overlap, the stronger access preference area takes precedence over the weaker access preference area. To require routing in a specific access preference area, use a strength value of 1. When you define access preference areas with a strength of 1, all access preference areas with a strength less than 1 are ignored and Zroute treats the access preference routing guide as a hard constraint.

The following example creates an access preference routing guide whose boundary is a rectangle with its lower-left corner at (0, 0) and its upper-right corner at (300, 300). It contains one wire access-preference area with coordinates of (0, 0) and (2, 1) and two via access-preference areas, one with coordinates of (0, 0) and (5, 5) and one with coordinates of (40, 40) and (45, 45). The wire access-preference area is slightly preferred over areas outside of the access preference area because it has a strength of 0.2. The via access-preference area with coordinates at (40, 40) and (45, 45) is ignored, because routing is required in the via access-preference area with coordinates at (0, 0) and (5, 5), which has a strength of 1.

```
icc2_shell> create_routing_guide -boundary {{0 0} {300 300}} \
   -access_preference {M1 {wire_access_preference {{0 0} {2 1}} 0.2}
                          {via_access_preference {{0 0} {5 5}} 1.0}
                          {via_access_preference {{40 40} {45 45}} 0.5}}
```

To report information about the access preference routing guides defined for your block, use the `report_routing_guides` command.

Using Routing Guides to Encourage River Routing

River routing is a special routing topology that tries to minimize the space taken by the router by compressing the routes to follow a particular contour. River routing is useful if there are tight routing channels or you need to minimize the space taken by routing. You can use a routing guide to encourage river routing within the routing guide boundary.

To create this type of routing guide, use the `-river_routing` option with the `create_routing_guide` command. You must also use the `-layers` option to specify the affected layers.

Feedback

For example, to encourage river routing on the M2 layer within the rectangle with its lower-left corner at (0, 0) and its upper-right corner at (100, 100), use the following command:

```
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
   -river_routing -layers {M2}
```

Querying Routing Guides

To find routing guides, use the `get_routing_guides` command. For example, to get all the routing guides in your block, use the following command:

```
icc2_shell> get_routing_guides *
```

To find the routing guides in a specific location, use the `get_objects_by_location -classes routing_guide` command.

Removing Routing Guides

To remove routing guides from the current block, use the `remove_routing_guides` command.

- To remove specific routing guides, specify the routing rules, either as a list or collection, such as that returned by the `get_routing_guides` command.
- To remove all routing guides, specify the `-all` option.

For example, to remove the new_width_rule routing rule, use the following command:

```
icc2_shell> remove_routing_rules new_width_rule
```

Deriving Routing Guides

Routing guides provides routing directives for specific areas of a block. Some types of routing guides are user-defined; others are derived from the block data. This topic described how to derive the following types of routing guides:

- Deriving Pin Access Routing Guides
- Deriving Metal Cut Routing Guides

See Also

- Defining Routing Guides

Feedback

Deriving Pin Access Routing Guides

When you reuse hard macros, such as IP blocks, at a smaller technology node, it can cause routability issues to the macro pins. You can use routing guides and blockages to enable access to the hard macro pins.

To create these routing guides, which are referred to as *pin access routing guides*, and the associated blockages, use the `derive_pin_access_routing_guides` command. You must specify the following information:

- The hard macros around which to create the routing guides and routing blockages (the `-cells` option)
- The metal layers for which to create the routing guides and routing blockages (the `-layers` option)
- The width of the routing guides and blockages in the x- and y-directions (the `-x_width` and `-y_width` options)

For each specified hard macro, the command creates routing guides to enable routing to the pins on the specified layers. In addition, the command creates metal blockages that surround the macro on the specified layers, with cutouts to allow pin access, and via blockages that cover the macro on the via layers adjacent to the specified metal layers. Vertical routing guides and blockages have the specified x-width. Horizontal routing guides and blockages have the specified y-width. By default, the command creates pin cutouts only in the preferred routing direction. To also create pin cutouts in the nonpreferred direction, use the `-nonpreferred_direction` option. To extend the pin cutouts beyond the metal blockages, use the `-pin_extension` option to specify the extension distance in microns.

To enable proper routing to pins of different sizes, the command sets the `is_rectangle_only_rule_waived` attribute on the macro pins to `true` to waive the rectangle-only rule.

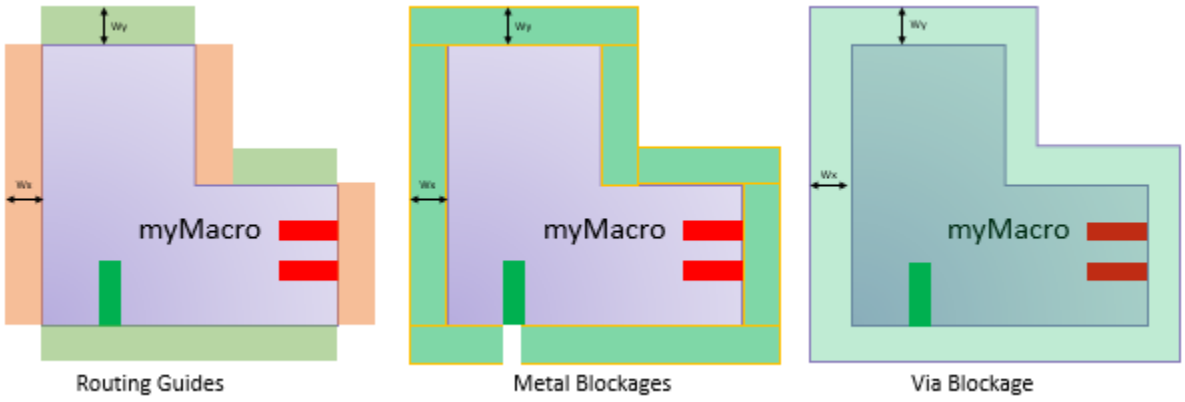
For example, the following command creates the routing guides and blockages shown in Figure 62.

```
icc2_shell> derive_pin_access_routing_guides -cells myMacro \
   -layers {M2 M3} -x_width 0.6 -y_width 0.7
```

In the figure, the two red pins are on M2 and the green pin is on M3. The figure shows the routing guides created on both the M2 and M3 layers, the metal blockages created on the M3 layer, and the via blockage created on the V2 and V3 layers. The tool also creates metal blockages on the M2 layer, which have cutouts for the M2 pins, and via blockages on the V1 and V2 layers.

Feedback

Figure 62 Pin Access Routing Guides and Blockages



If a hard macro already has pin access routing guides and their associated blockages, the command removes these existing objects and creates new routing guides and blockages.

If you move a hard macro after deriving the pin access routing guides, you must regenerate the pin access routing guides. You cannot manually modify pin access routing guides.

Deriving Metal Cut Routing Guides

If your technology file defines metal cut allowed and forbidden preferred grid extension rules and your block already contains boundary cells, you can create routing guides for these rules by using the `-add_metal_cut_allowed` option with the `derive_metal_cut_routing_guides` command. When you use this option, the `derive_metal_cut_routing_guides` command creates the following routing guides:

- Metal cut allowed routing guides, which cover the area taken up by all the placeable site rows reduced by the vertical shrink factor, which is 50 percent of the smallest site row height
- Forbidden preferred grid extension routing guides, which cover the remaining area up to the block boundary

To check the inserted routing guides, use the `-check_only` option with the `derive_metal_cut_routing_guides` command. By default, the checking results are stored in an error data file named *block_name*.err. To specify a different name for the error data file, use the `-error_view` option.

Controlling Routing Around the Block Boundary

To control routing around the block boundary, use the `derive_perimeter_constraint_objects` command to place rectangular metal shapes,

Feedback

routing guides, and routing blockages along the boundary edges, as described in the following topics:

- Inserting Metal Shapes in the Preferred Direction
- Inserting Routing Guides Along the Nonpreferred-Direction Edges
- Inserting Routing Blockages Along the Boundary Edges
- Removing Perimeter Constraint Objects

If you change the block boundary after creating the constraint objects, you must regenerate the objects. You cannot manually modify the constraint objects.

To check the perimeter constraint objects, use the `-check_only` option with the `derive_perimeter_constraint_objects` command. By default, the checking results are stored in an error data file named *block_name*.err. To specify a different name for the error data file, use the `-error_view` option

See Also

- Removing Perimeter Constraint Objects

Inserting Metal Shapes in the Preferred Direction

To insert rectangular metal shapes along the preferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects metal_preferred` option.

When you specify this object type, the command creates one or more rectangular metal shapes parallel to each preferred-direction boundary edge. The metal shapes are floating; they are not connected to any logical nets.

- If your technology does not specify a maximum area for these floating shapes, you can insert continuous metal shapes along the preferred-direction boundary edges, as described in Inserting Continuous Metal Shapes Parallel to Preferred-Direction Edges.
- If your technology specifies a maximum area for these floating shapes or you need more flexibility in placing the metal shapes, insert multiple metal shapes, called metal stubs, along the preferred-direction boundary edges, as described in Inserting Metal Stubs Parallel to Preferred-Direction Edges.

In addition to inserting metal shapes along the preferred-direction edges, you can insert short metal shapes perpendicular to the nonpreferred-direction edges, as described in Inserting Short Metal Shapes Perpendicular to Nonpreferred-Direction Edges.

Feedback

Inserting Continuous Metal Shapes Parallel to Preferred-Direction Edges

To insert continuous rectangular metal shapes along the preferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects metal_preferred` option.

When you specify this option, the command creates a single rectangular metal shape parallel to each preferred-direction boundary edge. By default,

- The metal shapes are inserted only in the top-level block

  To insert metal shapes in all soft macros in a hierarchical block, use the `-hierarchical` option.

- The metal shapes are inserted on all routing layers

  To insert the metal shapes only on specific routing layers, use the `-layers` option.

- The metal shapes have the default metal width specified for the routing layer in the technology file

  For information about specifying the default metal width for a layer, see the “Default Width Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- The metal shapes abut the nonpreferred-direction edges

  To specify an offset from the nonpreferred-direction edges, use the `-spacing_from_boundary` option. Use the following format to specify the offset for each layer:

  `{{`*`layer_name offset_value`*`} ...}`

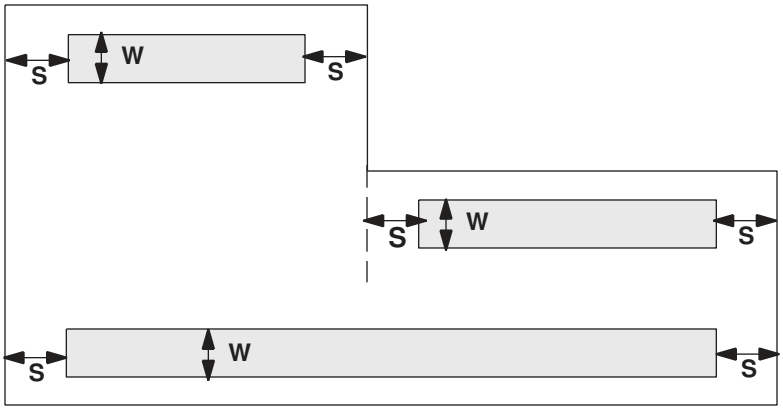
  The offset value must be a positive value.

- The metal shapes are placed on the closest wire track to the preferred-direction edge

Figure 63 shows the metal shapes inserted for an L-shaped block on a layer whose preferred direction is horizontal. Each metal shape is placed on the closest track to the edge. W is the default width for the layer and S is the spacing specified for the layer by the `-spacing_from_boundary` option.

Feedback

Figure 63 Continuous Metal Shape Perimeter Constraint Objects



Inserting Metal Stubs Parallel to Preferred-Direction Edges

To insert metal stubs along the preferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects metal_preferred` and `-stub` options.

When you specify both of these options, the command creates rectangular metal shapes parallel to each preferred-direction boundary edge based on the parameters you specify with the `-stub` option. Use the following format to specify the shape parameters in microns:

`{`*`offset_value start_value length_value spacing_value`*`}`

By default,

- The metal shapes are offset from the preferred-direction edge by the value specified in the *`offset_value`* argument

  The offset value is measured from the block edge to the center of the metal shape. It can be a positive value, zero, or a negative value. The absolute value of a negative value must be greater than the stub width. A positive value must be less than half of the smaller of the width or height of the block boundary.

- The first metal shape is offset from the nonpreferred-direction edge by the value specified in the *`start_value`* argument

  The start value must be greater than or equal to 0 and less than the smaller of the width or height of the block boundary. For horizontal shapes, the metal shapes start at the left edge. For vertical shapes, the metal shapes start at the bottom edge.

  Note:

  When you use the `-stub` option, the command uses the specified *`start_value`* and ignores the `-spacing_from_boundary` option.

Feedback

- The metal shapes have the length specified by the *length_value* argument

  The length value must be greater than 0 and less than or equal to the smaller of the width or height of the block boundary.

- The metal shapes have the default metal width specified for the routing layer in the technology file

  For information about specifying the default metal width for a layer, see the “Default Width Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- The metal shapes are separated the space specified by the *spacing_value* argument

  The spacing value must be greater than or equal to 0.

- The metal shapes are inserted only in the top-level block

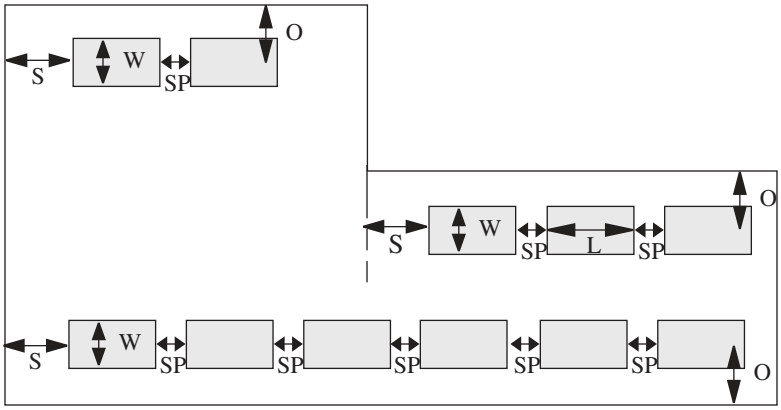
  To insert metal shapes in all soft macros in a hierarchical block, use the `-hierarchical` option.

- The metal shapes are inserted on all routing layers

  To insert the metal shapes only on specific routing layers, use the `-layers` option.

Figure 64 shows the metal stubs inserted for an L-shaped block on a layer whose preferred direction is horizontal. Each metal shape is placed at the specified offset, O, from the horizontal (preferred-direction) edge, where O is measured from the boundary to the center of the shape. The first metal stub is placed at the specified offset, S, from the vertical (nonpreferred-direction) edge. Each metal shape has a width of W, the default width for the layer and the specified length, L. The spacing between metal stubs is the specified spacing, SP.

Figure 64 Metal Stub Perimeter Constraint Objects



Feedback

Inserting Short Metal Shapes Perpendicular to Nonpreferred-Direction Edges

To insert additional short metal shapes in the preferred direction perpendicular to the nonpreferred-direction boundary edges,

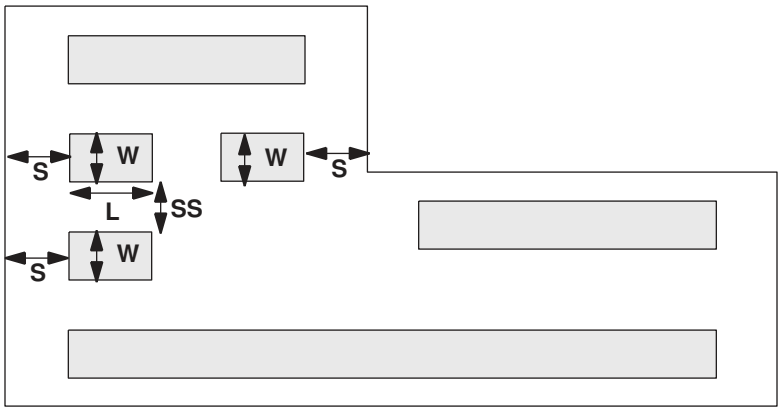
- Specify the length of the metal shapes by using the `-short_metal_length` option
- Specify the distance between the metal shapes by using the `-metal_spacing` option

For both of these options, you use the following format to specify the value for each layer:

```
{{layer_name value} ...}
```

Figure 65 shows the short metal shapes inserted for an L-shaped block on a layer whose preferred direction is horizontal. Each metal shape has a width of W, which is the default width for the layer, and a length of L, which is the length specified for the layer by the `-short_metal_length` option. The offset from the nonpreferred-direction edge is S, which is the spacing specified for the layer by the `-spacing_from_boundary` option. The distance between short metal shapes is SS, which is specified for the layer by the `-metal_spacing` option.

Figure 65 Metal Shape Perimeter Constraint Objects With Additional Short Shapes



Inserting Routing Guides Along the Nonpreferred-Direction Edges

To insert maximum pattern routing guides along the nonpreferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects mprg_nonpreferred` option.

When you specify this object type, the command creates maximum pattern routing guides parallel to each nonpreferred-direction boundary edge. The maximum edge threshold for these routing guides is zero.

Feedback

By default,

- The routing guides are inserted only in the top-level block

  To insert routing guides in all soft macros in a hierarchical block, use the `-hierarchical` option.

- The routing guides are inserted on all routing layers

  To insert the routing guides only on specific routing layers, use the `-layers` option.

- The routing guides have the default metal width specified for the routing layer in the technology file

  To specify a different width in microns, use the `-width_nonpreferred` option.

  For information about specifying the default metal width for a layer, see the “Default Width Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- The routing guides abut the preferred-direction edges

- The routing guides are offset from the nonpreferred-direction edges by the minimum spacing specified for the routing layer in the technology file

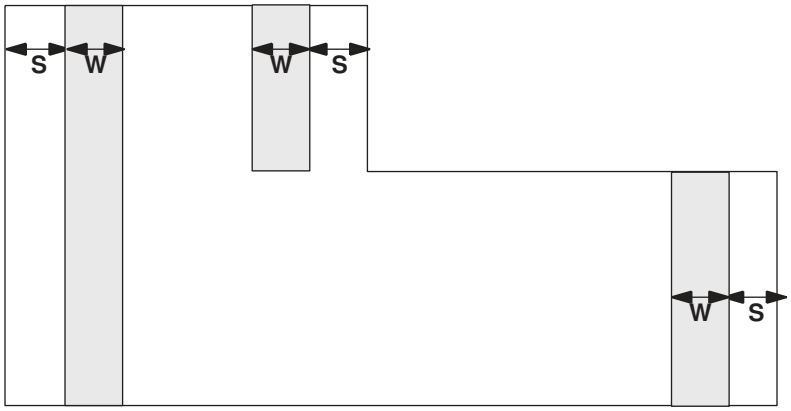
  To specify a different offset in microns, use the `-spacing_nonpreferred` option.

  For information about specifying the minimum spacing for a layer, see the “Minimum Spacing Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

Figure 66 shows the routing guides inserted for an L-shaped block on a layer whose preferred direction is horizontal. S is the spacing specified by the `-spacing_nonpreferred` option. W is the width specified by the `-width_nonpreferred` option.

Feedback

Figure 66 Routing Guide Perimeter Constraint Objects



See Also

- Using Routing Guides to Limit Edges in the Nonpreferred Direction

Inserting Routing Blockages Along the Boundary Edges

You can insert routing blockages along the preferred-direction edges, the nonpreferred-direction edges, or all edges.

- To insert routing blockages along the preferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects route_blockage_preferred` option.
- To insert routing blockages along the preferred-direction edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects route_blockage_nonpreferred` option.
- To insert routing blockages along all edges of the current block, use the `derive_perimeter_constraint_objects` command with the `-perimeter_objects {route_blockage_preferred route_blockage_nonpreferred}` option.

When you specify this object type, the command creates routing blockages parallel to the boundary edges in the specified directions.

Feedback

By default,

- The routing blockages are inserted only in the top-level block

  To insert routing blockages in all soft macros in a hierarchical block, use the `-hierarchical` option.

- The routing blockages are inserted on all routing layers

  To insert the routing blockages only on specific routing layers, use the `-layers` option.

- The routing blockages have the default metal width specified for the routing layer in the technology file

  To specify a different width in microns for preferred-direction routing blockages, use the `-width_preferred` option. To specify a different width in microns for nonpreferred-direction routing blockages, use the `-width_nonpreferred` option.

  For information about specifying the default metal width for a layer, see the “Default Width Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- The routing blockages are offset from the boundary edges by the minimum spacing specified for the routing layer in the technology file

  To specify a different offset in microns tor preferred-direction routing blockages, use the `-spacing_preferred` option. To specify a different offset in microns tor nonpreferred-direction routing blockages, use the `-spacing_nonpreferred` option.

  For information about specifying the minimum spacing for a layer, see the “Minimum Spacing Rule” topic in the *Synopsys Technology File and Routing Rules Reference Manual*.

- The routing blockages do not have pin cutouts

  To generate pin cutouts, use the `-pin_cutout` option.

- Cut layer routing blockages are not created

  To create via blockages, specify the via layers in the `-layers` option.

  By default, the width of the cut layer routing blockages is the maximum width for the preferred-direction and nonpreferred-direction routing blockages.

  - To specify the width for cut layer routing blockages along the horizontal edges, use the `-width_cut_layer_horizontal` option.
  - To specify the width for cut layer routing blockages along the vertical edges, use the `-width_cut_layer_vertical` option.

Feedback

By default, the cut layer routing blockages are offset from the boundary edges by the is the maximum spacing for the preferred-direction and nonpreferred-direction routing blockages

- To specify the spacing for cut layer routing blockages along the horizontal edges, use the `-spacing_cut_layer_horizontal` option.
- To specify the spacing for cut layer routing blockages along the vertical edges, use the `-spacing_cut_layer_vertical` option.

Removing Perimeter Constraint Objects

The `derive_perimeter_constraint_objects` command places the inserted constraint objects in an edit group named PC_Edit_Group_*blockname_n*. Each time you run the `derive_perimeter_constraint_objects` command, it creates a unique edit group.

To remove a group of constraint objects, use the following command:

```
icc2_shell> remove_objects [get_attribute \
   [get_edit_groups PC__Edit_Group_blockname_n] objects]
```

Routing Nets Within a Specific Region

To route one or more nets within a specific region,

1. Define a routing corridor by using the `create_routing_corridor` command, as described in Defining Routing Corridors.
2. Assign the nets and supernets to the routing corridor by using the `add_to_routing_corridor` command, as described in Assigning Nets to a Routing Corridor.
3. Verify that the routing corridors are valid by using the `check_routing_corridors` command, as described in Verifying Routing Corridors.

   If necessary, modify the routing corridors, as described in Modifying Routing Corridors.
4. Route the nets by using the route_group command, as described in Routing Critical Nets.
5. Remove the routing corridors, as described in Removing Routing Corridors.

See Also

- Reporting Routing Corridors

Feedback

Defining Routing Corridors

A routing corridor restricts Zroute global routing for specific nets to the region defined by a set of connected rectangles. In addition to specifying the region in which the routing occurs, you can also specify the minimum and maximum routing layers for each of the rectangles that comprise the routing corridor.

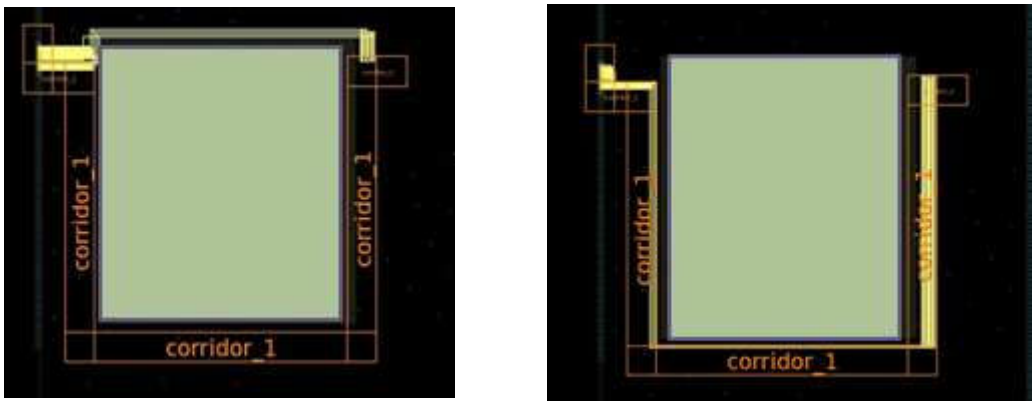
Routing corridors are intended to be used to route critical nets before signal routing. Zroute global routing considers routing corridors as a hard constraint, while track assignment and detail routing consider routing corridors as a soft constraint and might route nets slightly outside of the routing corridor to fix DRC violations.

Note:

If a routing guide overlaps with a routing corridor and its attributes conflict with the routing corridor, the routing corridor takes precedence.

For example, Figure 67 shows a routing corridor named corridor_1, which is made up of six rectangles. This routing corridor is associated with the nets shown in yellow. The figure on the left shows the nets before routing, while the figure on the right shows the nets routed within the routing corridor.

Figure 67 Using a Routing Corridor



Before routing After routing

To define a routing corridor, use the `create_routing_corridor` command. At a minimum, you must define the boundary of the routing corridor.

- To create a rectangular routing corridor, use the `-boundary` option to specify the lower-left and upper-right corners of the rectangle using the following syntax: `{ {llx lly} {urx ury} }`.
- To create a rectilinear routing corridor, use the `-boundary` option to specify the coordinates of the polygon using the following syntax: `{ {x1 y1} {x2 y2} ... }`.

Feedback

- To create a path-based routing corridor, use the `-path` option to specify the path and the `-width` option to specify the width in microns. Specify the path using the following syntax: `{ {x1 y1} {x2 y2} ... }`.

  By default, path-based routing corridors have flush end caps at the start and end of the path. To change the end cap style, use the `-start_endcap` and `-end_endcap` options. Valid styles are `flush`, `full_width`, and `half_width`.

By default, the tool creates a routing corridor named CORRIDOR_*objId* in the current block that honors the existing minimum and maximum routing layer constraints (for information about specifying routing layer constraints, see Specifying the Routing Resources). Use the following options to change the default behavior.

- To specify a name for the routing corridor, use the `-name` option.
- To create the routing guide in a different physical cell, use the `-cell` option.

  When you use this option, the tool creates the routing guide in the cell's reference block using the coordinate system of the cell's top-level block.
- To specify the minimum and maximum routing layers for the routing corridor, use the `-min_layer_name` and `-max_layer_name` options.

  Specify the routing layers by using the layer names from the technology file.

Assigning Nets to a Routing Corridor

To assign nets and supernets to a routing corridor, use the `add_to_routing_corridor` command. You must specify the routing corridor and the nets or supernets to add to it. You can assign a net or supernet to only one routing corridor and that routing corridor must cover all pins connected to the associated nets. In addition, the supernets must belong to the same block as the routing corridor.

For example, to define a routing corridor named corridor_a and assign the nets named n1 and n2 to this routing corridor, use the following commands:

```
icc2_shell> create_routing_corridor -name corridor_a \
   -boundary { {10 10} {20 35} } \
   -min_layer_name M2 -max_layer_name M4
icc2_shell> create_routing_corridor_shape -routing_corridor corridor_a \
   -boundary { {20 25} {40 35} } \
   -min_layer_name M2 -max_layer_name M4
icc2_shell> create_routing_corridor_shape -routing_corridor corridor_a \
   -boundary { {40 10} {50 35} } \
   -min_layer_name M2 -max_layer_name M4
icc2_shell> add_to_routing_corridor corridor_a [get_nets {n1 n2}]
```

Feedback

Note:

You can also assign nets or supernets to the routing corridor by using the `-object` option when you use the `create_routing_corridor` command to create the routing corridor.

Verifying Routing Corridors

To successfully route a net within a routing corridor, the routing corridor must meet the following requirements:

- It must be a contiguous region; all regions that comprise the routing corridor must be connected.
- It must contain the pins that connect to the nets and supernets associated with the routing corridor.

To verify that a routing corridor meets these requirements, use the `check_routing_corridors` command.

```
icc2_shell> check_routing_corridors RC_0
```

You can view the errors detected by the `check_routing_corridors` command in the message browser.

Modifying Routing Corridors

You can make the following modifications to an existing routing corridor:

- Add new shapes to the routing corridor.

  To add new shapes, use the `create_routing_corridor_shape` command. Use the `-routing_corridor` option to specify the routing corridor that you want to update. You must specify the boundary of the routing corridor by using the `-boundary` or `-path` option (for details about these options, see Defining Routing Corridors). You can also specify the minimum and maximum routing layers for the shape by using the `-min_layer_name` and `-max_layer_name` options.

- Remove rectangles from the routing corridor.

  To remove shapes, use the `remove_routing_corridor_shapes` command.

- Change the nets and supernets associated with the routing corridor.

  To add nets or supernets to a routing corridor, use the `add_to_routing_corridor` command. To remove nets or supernets from a routing corridor, use the `remove_from_routing_corridor` command.

You can also modify routing corridors in the GUI by using the Create Route Corridor tool, the Move/Resize tool, or the Delete tool, or by editing the attributes in the Properties dialog box.

Feedback

Reporting Routing Corridors

To report the routing corridors in your block, use the `report_routing_corridors` command. By default, the command reports all routing corridors; to report specific routing corridors, specify the routing corridors to report.

By default, the command reports the following information for each routing corridor: its name; the shapes associated with the routing corridor, including their names, minimum routing layer, maximum routing layer, and boundary; the connectivity of the routing corridor shapes; and the nets and supernets associated with the routing corridor. To output a Tcl script that re-creates the routing corridors, use the `-output` option.

To report the routing corridor for a specific net or supernet, use the `get_routing_corridors` command. When you run this command, you must use the `-of_objects` option to specify the nets of interest.

Removing Routing Corridors

To remove routing corridors from the current block, use the `remove_routing_corridors` command.

- To remove specific routing corridors, specify the routing corridors, either as a list or collection, such as that returned by the `get_routing_corridors` command.
- To remove all routing corridors, specify the `-all` option.

You can also remove routing corridors in the GUI by using the Delete tool.

Using Nondefault Routing Rules

Zroute supports the use of nondefault routing rules, both for routing and for shielding.

- For routing, you can use nondefault routing rules to define stricter wire width and spacing rules, to define the pin tapering distance, to specify the vias used when routing nets with nondefault routing rules, and to specify multiple-patterning mask constraints.
- For shielding, you can use nondefault routing rules to define the minimum width and spacing rules.

For information about working with nondefault routing rules, see the following topics:

- Defining Nondefault Routing Rules
- Reporting Nondefault Routing Rule Definitions
- Removing Nondefault Routing Rules
- Modifying Nondefault Routing Rules

- Assigning Nondefault Routing Rules to Nets
- Reporting Nondefault Routing Rule Assignments

Defining Nondefault Routing Rules

To define a nondefault routing rule, use the `create_routing_rule` command. When you define a nondefault routing rule, you must specify a name for the nondefault routing rule. You use the name to assign the nondefault routing rule to nets or clocks.

The following topics describe how to create nondefault routing rules for various purposes:

- Defining Minimum Wire Width Rules
- Defining Minimum Wire Spacing Rules
- Defining Minimum Via Spacing Rules
- Specifying Nondefault Vias
- Specifying Mask Constraints
- Defining Shielding Rules
- Reporting Nondefault Routing Rule Definitions
- Removing Nondefault Routing Rules
- Modifying Nondefault Routing Rules

You can create a single routing rule that serves multiple purposes. In addition, you can assign multiple nondefault routing rules to a net.

Defining Minimum Wire Width Rules

You can define nondefault minimum wire width rules that are stricter than the minimum width rules defined in the technology file. Nondefault minimum width rules are hard constraints, which must be met during routing.

Note:

If you specify a nondefault width that violates the `signalRouteMaxWidth` setting in the technology file, the tool ignores the nondefault width.

Feedback

The minimum width defined in a nondefault routing rule applies to all metal segments, including via enclosures. To avoid DRC violations, ensure that the enclosures for nondefault vias meet the minimum width rule.

To define a minimum wire width rule, use the `create_routing_rule` command. You can specify the minimum width by specifying a multiplier that is applied to the default width for each layer, by specifying the minimum width in microns for each layer, or both.

- To use a multiplier to specify the minimum width, use the following syntax:

  ```
  create_routing_rule rule_name
     [-default_reference_rule | -reference_rule_name ref_rule]
     -multiplier_width multiplier
  ```

  The multiplier value must be between 0.001 and 2000. The default width for each layer is determined from the reference rule, which is either the default routing rule or the reference rule specified in the `-reference_rule_name` option.

  For example, to define a nondefault routing rule named new_width_rule that uses the default routing rule as the reference rule and defines the nondefault width as two times the default width, use the following command:

  ```
  icc2_shell> create_routing_rule new_width_rule -multiplier_width 2.0
  ```

- To specify the minimum width values for each layer, use the following syntax:

  ```
  create_routing_rule rule_name
     [-default_reference_rule | -reference_rule_name ref_rule]
     -widths {layer1 width1 layer2 width2 … layern widthn}
  ```

  Specify the routing layers by using the layer names from the technology file. You can specify a single width value per layer. Zroute uses the wire width from the reference rule, which is either the default routing rule or the reference rule specified in the `-reference_rule_name` option, for any layers not specified in the `-widths` option.

  For example, to define a nondefault routing rule named new_width_rule2 that uses the default routing rule as the reference rule and defines nondefault width rules for the M1 and M4 layers, use the following command:

  ```
  icc2_shell> create_routing_rule new_width_rule2 \
     -widths {M1 0.8 M4 0.9}
  ```

- If you specify both with `-multiplier_width` option and the `-widths` option, the tool uses the `-widths` option to determine the base width, and then applies the multiplier to that value to determine the minimum width requirement.

  For example, to define a nondefault routing rule named new_width_rule3 that uses the default routing rule as the reference rule and defines the nondefault width as 0.8 for the

Feedback

M1 layer, 0.9 for the M4 layer, and two times the default width for all other layers, use the following command:

```
icc2_shell> create_routing_rule new_width_rule3 \
   -multiplier_width 2.0 -widths {M1 0.4 M4 0.45}
```

Defining Minimum Wire Spacing Rules

You can define nondefault minimum wire spacing rules that are stricter than the rules defined in the technology file. Nondefault wire spacing rules can be defined as hard constraints, which must be met, or as soft constraints, which Zroute tries to meet.

Note:

The spacing rules defined in the technology file are always considered hard constraints.

By default, Zroute checks the nondefault spacing rules between signal nets and other signal nets, PG nets, and blockages, but not between shapes of the same signal net or between signal nets and shield wires for PG nets. For information about modifying these checks, see Configuring Nondefault Spacing Checks.

To define a minimum wire spacing rule, use the `create_routing_rule` command. You can specify the minimum spacing by specifying a multiplier that is applied to the default spacing for each layer, by specifying the minimum spacings in microns for each layer, or both.

- To use a multiplier to specify the minimum spacing, use the following syntax:

  ```
  create_routing_rule rule_name
     [-default_reference_rule | -reference_rule_name ref_rule]
     -multiplier_spacing multiplier
  ```

  The multiplier value must be between 0.001 and 2000. The default wire spacing for each layer is determined from the reference rule, which is either the default routing rule or the reference rule specified in the `-reference_rule_name` option.

  For example, to define a nondefault routing rule named new_spacing_rule that uses the default routing rule as the reference rule and defines the nondefault spacing as two times the default spacing, use the following command:

  ```
  icc2_shell> create_routing_rule new_spacing_rule \
     -multiplier_spacing 2.0
  ```

- To specify the minimum spacing values for each layer, use the following syntax:

  ```
  create_routing_rule rule_name
     -spacings { layer1 {spacing11 spacing12 ... spacing1n}
                 layer2 {spacing21 spacing22 ... spacing2n}
                 ...
                 layern {spacingn1 spacingn2 ...spacingnn} }
     -spacing_weight_levels { layer1 {weight11 weight12 ... weight1n}
  ```

Feedback

layer$_2$ {*weight*$_{21}$ *weight*$_{22}$ ... *weight*$_{2n}$}
...
layer$_n$ {*weight*$_{n1}$ *weight*$_{n2}$ ... *weight*$_{nn}$} }

Specify the routing layers by using the layer names from the technology file. You can define multiple spacing values per layer. Zroute uses the spacing values from the reference rule, which is either the default routing rule or the reference rule specified in the `-reference_rule_name` option, for any layers not specified in the `-spacings` option.

If you specify more than one spacing value per layer, you must assign a weight to each spacing value by using the `-spacing_weight_levels` option. The valid weight values are `low`, `medium`, `high`, and `hard`. When you assign a weight level other than hard, the spacing rule is a soft spacing rule. By default, Zroute does not fix soft routing rule violations. To fix soft routing rules, you must map the weight levels to routing effort levels, as described in Specifying the Routing Effort for Soft Spacing Violations.

For example, to define a nondefault routing rule named new_spacing_rule2 that uses the default routing rule as the reference rule and defines nondefault spacing rules for the M1 and M4 layers, use the following command:

```
icc2_shell> create_routing_rule new_spacing_rule2 \
   -spacings { M1 {0.12 0.24} M4 {0.14 0.28} } \
   -spacing_weight_levels { M1 {hard medium} M4 {hard medium} }
```

- If you specify both with `-multiplier_spacing` option and the `-spacings` option, the tool uses the `-spacings` option to determine the base spacing, and then applies the multiplier to that value to determine the minimum wire spacing requirement.

To limit spacing to one side of the net, use the `-single_side_spacing` option with the `create_routing_rule` command.

For example, the following command creates single-side spacing rules on the M2, M3, and M4 metal layers METAL2, METAL3, and METAL4:

```
icc2_shell> create_routing_rule new_spacing_rule3 \
   -spacings {M2 1.300 M3 1.400 M4 1.500 } \
   -single_side_spacing  \
   -widths {M1 0.230 M2 0.280 M3 0.280 M4 0.280 M5 0.280 M6 0.440}
```

Feedback

Configuring Nondefault Spacing Checks

You can configure the checking of nondefault spacing rules by enabling or disabling checks between signal nets and other objects. In addition, you can ignore violations of nondefault spacing rules for short parallel distances.

- To honor nondefault spacing rules between shapes of the same signal net, set the `route.detail.var_spacing_to_same_net` application option to `true`.
- To honor nondefault spacing rules between signal nets and shield wires for PG nets
  - For all signal nets, set the `route.common.ignore_var_spacing_to_shield` application option to `false`
  - For specific signal nets, use the `-ignore_spacing_to_shield false` option when you define the nondefault spacing rule with the `create_routing_rule` command
- To ignore nondefault spacing rules between signal nets and PG nets
  - For all signal nets, set the `route.common.ignore_var_spacing_to_pg` application option to `true`
  - For specific signal nets, use the `-ignore_spacing_to_pg true` option when you define the nondefault spacing rule with the `create_routing_rule` command

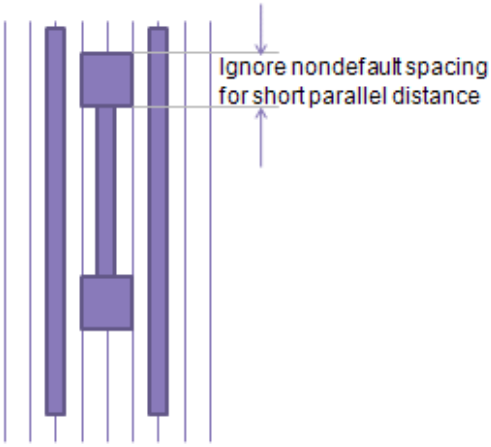
  Note:

  When you ignore nondefault spacing rules between signal nets and PG nets, the rules are also ignored between the signal nets and the shield wires regardless of the setting of the `route.common.ignore_var_spacing_to_shield` and `create_routing_rule -ignore_spacing_to_shield` options.
- To ignore nondefault spacing rules between signal nets and blockages
  - For all signal nets, set the `route.common.ignore_var_spacing_to_blockage` application option to `true`
  - For specific signal nets, use the `-ignore_spacing_to_blockage true` option when you define the nondefault spacing rule with the `create_routing_rule` command

Feedback

You can relax the nondefault spacing checks for specific nondefault routing rules by ignoring violations for short parallel distances, as shown in Figure 68. This technique increases the flexibility of DRC convergence without adversely affecting crosstalk.

Figure 68 Ignoring Nondefault Spacing Rule Violations



To define the length thresholds within which to ignore the nondefault spacing violations, use the `-spacing_length_thresholds` option when you create the nondefault routing rule with the `create_routing_rule` command. The length threshold values are in microns and must have a one-to-one correspondence with the spacing entries specified in the `-spacings` option.

If the `-spacing_length_thresholds` is not specified, the `route.detail.ndr_spacing_length_threshold_factor` application option derives the length threshold of the routing rule. By default, it is set to `0`.

For example,

```
icc2_shell> create_routing_rule new_rule \
   -spacings { M1 {0.09 0.15 0.2}
               M2 {0.09 0.15 0.2}
               M3 {0.09 0.15 0.2} } \
   -spacing_weight_levels { M1 {hard medium low}
               M2 {hard medium low}
               M3 {hard medium low} } \
   -spacing_length_thresholds { M1 {0.01 0 0}
               M2 {0.01 0 0}
               M3 {0.01 0 0} }
```

Feedback

Specifying the Routing Effort for Soft Spacing Violations

By default, Zroute does not fix soft spacing violations. To fix soft spacing violations, you must assign a routing effort to each of the soft weight levels. These assignments apply to all soft spacing rules.

To assign a routing effort for each weight level, use the following syntax to set the `route.common.soft_rule_weight_to_effort_level_map` application option:

```
set_app_options
   -name route.common.soft_rule_weight_to_effort_level_map
   -value { {weight effort} ... }
```

Each *weight* argument can be one of `low`, `medium`, or `high`. You should specify each weight level only one time; if you specify a weight level multiple times, the router uses the last specification. Each *effort* argument can be one of the following values:

- `off` (the default)

  Zroute does not fix the soft spacing rule violations.

- `low`

  Zroute uses a small number of rip-up and reroute passes to resolve soft spacing rule violations.

- `medium`

  Zroute uses a medium number of rip-up and reroute passes to resolve soft spacing rule violations. Note that you cannot specify this effort level for the `low` weight level.

- `high`

  Zroute treats soft spacing rule violations the same as regular design rule violations during rip up and reroute. Note that you cannot specify this effort level for the `low` weight level.

For example, to assign low routing effort to low-weight soft spacing rules, medium routing effort to medium-weight soft spacing rules, and high routing effort to high-weight soft spacing, use the following command:

```
icc2_shell> set_app_options \
   -name route.common.soft_rule_weight_to_effort_level_map \
   -value { {low low} {medium medium} {high high} }
```

Feedback

Defining Minimum Via Spacing Rules

You can define nondefault minimum via spacing rules that are stricter than the rules defined in the technology file. Nondefault via spacing rules are hard constraints that must be met.

To define a minimum via spacing rule, use the `-via_spacings` option with the `create_routing_rule` command using the following syntax:

```
create_routing_rule rule_name
   -via_spacings { {layer1 layer2 spacing} ... }
```

Specify the routing layers by using the layer names from the technology file. You can define a single width value per layer pair.

The minimum via spacing for vias between any unspecified layer combinations is determined from the technology file. For vias on the same layer, the router uses the minimum spacing defined in the `Layer` section for the via layer. For vias on different layers, the router uses the minimum spacing defined in the `DesignRule` section for the via layer combination.

For example, to define a nondefault routing rule named via_spacing_rule that defines nondefault spacing rules between vias on the V1 layer and between vias on the V1 and V2 layers, and uses the minimum spacing rules defined in the technology file between vias on all other layer combinations, use the following command:

```
icc2_shell> create_routing_rule via_spacing_rule \
   -via_spacings {{V1 V1 2.3} {V1 V2 3.2}}
```

Specifying Nondefault Vias

By default, when routing nets with nondefault routing rules, Zroute selects vias based on the design rules.

To specify the vias to use when routing nets with nondefault routing rules, use the `create_routing_rule` command. You can define the nondefault vias by using either the `-cuts` option or the `-vias` option.

- When you use the `-cuts` option, the tool determines the suitable via definitions from the technology file based on the specification in the `-cuts` option and the rules defined in the technology file.
- When you use the `-vias` option, you explicitly specify the nondefault via definitions, including the allowed cut numbers and rotation for each via definition.

Feedback

Specifying Nondefault Vias Using the -cuts Option

The syntax for specifying nondefault vias using the `-cuts` option is

```
create_routing_rule rule_name
   -cuts { {cut_layer1 {cut_name1, ncuts} {cut_name2, ncuts} …}
           {cut_layer2 {cut_name1, ncuts} {cut_name2, ncuts} …}
           ...
           {cut_layern {cut_name1, ncuts} {cut_name2, ncuts} …}
         }
```

The *cut_layer* arguments refer to the via layer names in the technology file and the *cut_name* arguments refer to the cut names defined in the `cutNameTbl` attribute in the associated `Layer` section in the technology file. You can specify multiple cut names per layer. For each cut name, you must specify the minimum number of cuts, which must be an integer between 1 and 255.

The tool searches for the vias defined in the `ContactCode` section of the technology file that meet the rules defined in the technology file for the specified cut name, such as the `cutWidthTbl`, `cutHeightTbl`, and `fatTblFatContactNumber` rules. In addition, the width of the via enclosure must meet the nondefault width and the `xLegalWidthTbl` and `yLegalWidthTbl` rules defined in the technology file for the adjacent metal layers. If the fat metal contact rule is not defined for a via layer, the tool searches for the default vias that meet the cut width, cut height, and via enclosure width requirements.

The minimum number of cuts required is the larger of the *ncuts* value in the `-cuts` option and the value defined in the `fatTblFatContactMinCuts` attribute. For the selected vias, the tool always allows both the rotated and unrotated orientations for the via.

For example, assume the following information is defined in the technology file:

```
Layer "VIA1" {
   fatTblThreshold         = ( 0, 0.181, 0.411 )
   fatTblFatContactNumber  = ( "2,3,4,5,6 ","5,6,20", "5,6,20" )
   fatTblFatContactMinCuts = ( "1,1,1,1,1", "1,1,1",  "2,2,2" )

   cutNameTbl    = (  Vsq,  Vrect )
   cutWidthTbl   = (  0.05,  0.05  )
   cutHeightTbl  = (  0.05,  0.13  )
   …
}

ContactCode "VIA12_LH" {
   contactCodeNumber = 5
   cutWidth          = 0.13
   cutHeight         = 0.05
   …
}
ContactCode "VIA12_LV" {
   contactCodeNumber = 6
   cutWidth          = 0.05
```

Feedback

```
   cutHeight          = 0.13
   …
}
ContactCode "VIA12_P" {
   contactCodeNumber = 20
   cutWidth          = 0.05
   cutHeight         = 0.05
   …
}
```

If you use the following command,

```
icc2_shell> create_routing_rule cut_rule -cuts {VIA1 {Vrect 1}}
```

The tool selects the following vias: {VIA12_LH 1x1 R}, {VIA12_LH 1x1 NR}, {VIA12_LV 1x1 R}, {VIA12_LV 1x1 NR}, {VIA12_LH 1x2 R}, {VIA12_LH 1x2 NR}, {VIA12_LH 2x1 R}, {VIA12_LH 2x1 NR}, {VIA12_LV 1x2 R}, {VIA12_LV 1x2 NR}, {VIA12_LV 2x1 R}, and {VIA12_LV 2x1 NR}.

Specifying Nondefault Vias Using the -vias Option

The syntax for specifying nondefault vias using the `-vias` option is

```
create_routing_rule rule_name
   -vias { {via_type1 cut_number1 orientation1}
           {via_type2 cut_number2 orientation2}
           ...
           {via_typen cut_numbern orientationn}
         }
```

You can specify multiple via types per layer; each via type must be a via definition defined in the technology file or a via definition created by the `create_via_def` command. For each via type, you must explicitly specify the allowed cut numbers and orientation. To specify the orientation, use `NR` to indicate that the via is not rotated or `R` to indicate that the via is rotated. The order of via specification is not important; during routing, Zroute selects the lowest cost nondefault via.

For example, to specify the vias selected by the `-cuts` option in the previous example, use the following command:

```
icc2_shell> create_routing_rule via_rule \
   -vias { {VIA12_LH 1x1 R} {VIA12_LH 1x1 NR} {VIA12_LV 1x1 R}
           {VIA12_LV 1x1 NR} {VIA12_LH 1x2 R} {VIA12_LH  1x2 NR}
           {VIA12_LH 2x1 R} {VIA12_LH  2x1 NR} {VIA12_LV 1x2 R}
           {VIA12_LV 1x2 NR} {VIA12_LV 2x1 R} {VIA12_LV 2x1 NR} }
```

Specifying Mask Constraints

If you are using the precolored design flow for a design that uses multiple-patterning technology, you can set mask constraints on timing-critical nets, such as clock nets,

Feedback

by defining a precoloring rule and applying it to the nets. For details about the mask constraints, see Mask Constraints.

To define a precoloring rule, use the `create_routing_rule` command using the following syntax:

```
create_routing_rule rule_name
   -mask_constraints
 {layer1 constraint1 layer2 constraint2 ... layern constraintn}
```

where *constraint* is one of `same_mask`, `mask1_soft`, or `mask2_soft`.

For example, to define a precoloring routing rule that sets `mask_one` constraints on the M4 and M5 layers, use the following command:

```
icc2_shell> create_routing_rule clock_mask1 \
   -mask_constraints {M4 mask_one M5 mask_one}
```

Note:

To ensure DRC convergence, you should set double-patterning mask constraints only on a very few timing-critical nets.

Defining Shielding Rules

To define shielding rules, use the `create_routing_rule` command using the following syntax:

```
create_routing_rule rule_name
   -shield_widths {layer1 width1 layer2 width2 … layern widthn}
   -shield_spacings {layer1 spacing1 layer2 spacing2 … layern spacingn}
   [-snap_to_track]
```

Specify the routing layers by using the layer names from the technology file. You can define a single width and spacing value per layer. Zroute uses the default wire width for any layers not specified in the `-shield_widths` option and the default spacing for any layers not specified in the `-shield_spacings` option.

By default, shielding wires are not snapped to the routing tracks. To snap shielding wires to the routing tracks, use the `-snap_to_track` option when you define the nondefault routing rule.

For example, to specify a shielding rule that uses spacing of 0.1 microns and a width of 0.1 microns for M1 through M5 and spacing of 0.3 microns and a width of 0.3 microns for M6, use the following command:

```
icc2_shell> create_routing_rule shield_rule \
   -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
   -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
```

Feedback

Reporting Nondefault Routing Rule Definitions

To report the nondefault routing rules defined by the `create_routing_rule` command, use the `report_routing_rules -verbose` command. By default, this command reports all of the nondefault routing rules for the current block. To limit the report to specific nondefault routing rules, specify the rule names as an argument to the command.

```
icc2_shell> report_routing_rules {rule_names}
```

To output a Tcl script that contains the `create_routing_rule` commands used to define the specified nondefault routing rules, use the `-output` option when you run the `report_routing_rules` command.

Removing Nondefault Routing Rules

To remove nondefault routing rules from the current block, use the `remove_routing_rules` command. When you remove a nondefault routing rule, the rule is removed from all nets to which it is applied and the rule definition is removed from the design library.

- To remove specific routing rules, specify the routing rules.
- To remove all routing rules, specify the `-all` option.

For example, to remove the new_width_rule routing rule, use the following command:

```
icc2_shell> remove_routing_rules new_width_rule
```

Modifying Nondefault Routing Rules

To change the definition for an existing rule, you must use the `remove_routing_rules` command to remove the rule and then use the `create_routing_rule` command to redefine the rule. The tool issues an error message if you try to redefine an existing routing rule.

Assigning Nondefault Routing Rules to Nets

The IC Compiler II tool provides two commands for assigning nondefault routing rules to nets:

- `set_clock_routing_rules`

  This command assigns nondefault routing rules to clock nets before clock tree synthesis. During clock tree synthesis and optimization, the tool propagates the nondefault routing rules to the newly created clock nets.

- `set_routing_rule`

  This command assigns nondefault routing rules to signal nets and to clock nets after clock tree synthesis.

Feedback

The following topics describe how to assign nondefault routing rules to nets:

- Assigning Nondefault Routing Rules to Clock Nets
- Assigning Nondefault Routing Rules to Signal Nets
- Reporting Nondefault Routing Rule Assignments

Assigning Nondefault Routing Rules to Clock Nets

To assign a clock routing rule to a clock net, use the `-rule` option with the `set_clock_routing_rules` command. The specified rule must be a rule that you previously defined with the `create_routing_rule` command.

To reset a clock routing rule to the default routing rule, use the `-default_rule` option. To change a clock routing rule, first reset it to the default routing rule and then use the `-rule` option after resetting the assignment to the default routing rule.

By default, the `set_clock_routing_rules` command assigns the specified clock routing rule to all clock trees in the block. Table 25 shows the options used to restrict the routing rule assignment.

Table 25 Restricting Clock Routing Rule Assignments

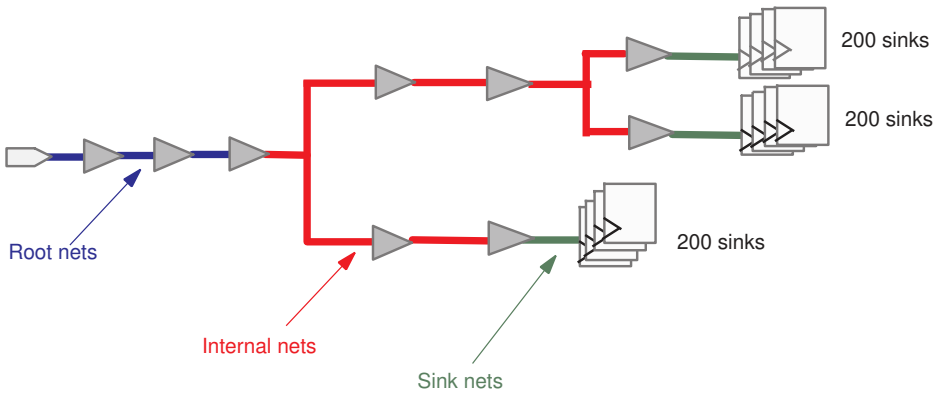
To assign a nondefault routing rule to	Use this option
Specific clock trees	`-clocks` *clocks*
Nets connected to the clock root[2]	`-net_type root`
Nets connected to one or more clock sinks[2]	`-net_type sink`
Internal nets in the clock tree (all nets except the root and sink nets)[2]	`-net_type internal`
Specific clock nets	`-nets` *nets*

Figure 69 shows the root, internal, and sink nets of a clock tree after clock tree synthesis. By default, the root-net routing rule is applied to all the single-fanout clock nets starting from the clock root up to the point where the clock tree branches out to a fanout of more than one. Internal-net routing rules are applied to the nets from this point until the sink nets.

2. *You can use this option with the `-clocks` option to further restrict the assignment. This option is not valid with the `-nets` option.*

Feedback

Figure 69 Root, Internal, and Sink Clock Net Types



The following example specifies routing rules for the root, internal, and sink nets:

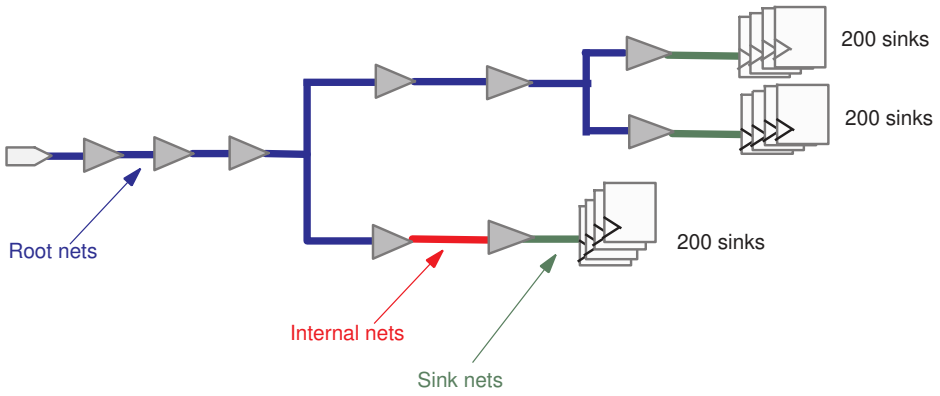
```
icc2_shell> set_clock_routing_rules -rules NDR1 -net_type root
icc2_shell> set_clock_routing_rules -rules NDR2 -net_type internal
icc2_shell> set_clock_routing_rules -rules NDR3 -net_type sink
```

To specify a transitive fanout limit to use when identifying root nets, use the `set_clock_tree_options -root_ndr_fanout_limit` command. For example, to specify that any clock net with a transitive fanout of more than 300 be considered as a root net, use the following command:

```
icc2_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

Figure 70 shows the root, internal, and sink nets of the same clock tree when a transitive fanout limit of 300 is used for identifying the clock root nets.

Figure 70 Using a Fanout Limit for Selecting Root Nets



Feedback

When calculating the transitive fanout of clock nets for the purpose of identifying root nets, the tool includes only the valid clock sinks; It does not include the ignore pins. If a net identified as a root net is less than 10 microns, the tool uses internal-net routing rules for that net.

Note:

Specifying a smaller value with the `set_clock_tree_options -root_ndr_fanout_limit` command increases the number of clock nets that are assigned the root-net routing rule, which can increase routing congestion.

During clock tree synthesis and optimization, the tool also honors nondefault routing rules set by using the `set_routing_rule` command. However, the tool does not propagate these routing rules to any new clock nets it creates.

If a net is assigned more than one nondefault routing rule, the tool uses the following priority to determine the effective routing rule:

1. Nondefault routing rule set by the `set_routing_rule` command
2. Net-specific clock routing rule set by the `set_clock_routing_rules -nets` command
3. Clock-specific clock routing rule set by the `set_clock_routing_rules -clocks` command
4. Global clock routing rule set by the `set_clock_routing_rules` command

Assigning Nondefault Routing Rules to Signal Nets

To assign a nondefault routing rule to a net, use the `-rule` option with the `set_routing_rule` command. The specified rule must be a rule that you previously defined with the `create_routing_rule` command. You must specify the nets to which to assign the nondefault routing rule. You can assign multiple nondefault routing rules to a net.

To change the routing rule assignment for one or more nets,

- Use the `-default_rule` option to reset the nets to the default routing rule.

  To assign different nondefault routing rules to the nets, use the `-rule` option after resetting the nets to the default routing rule.
- Use the `-no_rule` option to remove all routing rules from the nets and allow the tool to automatically assign a routing rule to them.
- Use the `-clear` option to remove all routing rules and net-specific layer constraints from the nets.

Feedback

For example, to assign a nondefault routing rule called WideMetal to the CLK net, use the following command:

```
icc2_shell> set_routing_rule -rule WideMetal [get_nets CLK]
```

To reset the routing rule for the CLK net to the default routing rule, use the following command:

```
icc2_shell> set_routing_rule -default_rule [get_nets CLK]
```

Reporting Nondefault Routing Rule Assignments

The IC Compiler II tool provides commands to report the nondefault routing rules assigned by the `set_routing_rule` command and the clock routing rules assigned by the `set_clock_routing_rules` command.

- To report the nondefault routing rules assigned by the `set_routing_rule` command, use the `-of_objects` option with the `report_routing_rules` command.

  ```
  icc2_shell> report_routing_rules -of_objects [get_nets *]
  ```

- To report the clock routing rule assignments, use the `report_clock_routing_rules` command. This command reports only the routing rules assigned by the `set_clock_routing_rules` command.

Controlling Off-Grid Routing

You can prevent off-grid routing of wires or vias by requiring them to be aligned to the wire track or discourage off-grid routing of vias by increasing the cost associated with off-grid routing.

Preventing Off-Grid Routing

By default, wires and vias need to be aligned to the wire track grid for a metal layer only if the `onWireTrack` or `onGrid` attribute is set to 1 in its `Layer` section in the technology file.

To override the technology file settings, set the following application options:

- `route.common.wire_on_grid_by_layer_name`

  This option controls off-grid routing for metal layers.

- `route.common.via_on_grid_by_layer_name`

  This option controls off-grid routing for via layers.

Use the following syntax to set these options:

```
{ {layer true|false} ... }
```

Feedback

Specify the layers by using the layer names from the technology file. Specify `true` to forbid off-grid routing and `false` to allow off-grid routing.

If you use either of these options, the tool ignores all settings for the `onWireTrack` and `onGrid` attributes in the technology file and uses only the settings specified by these options. If you do not specify a layer in these options, off-grid routing is allowed on that layer, regardless of the setting in the technology file.

For example, to prevent off-grid routing for wires on the M2 and M3 metal layers and for vias on the V2 via layer, regardless of the settings in the technology file, use the following commands:

```
icc2_shell> set_app_options \
   -name route.common.wire_on_grid_by_layer_name \
   -value {{M2 true} {M3 true}}
icc2_shell> set_app_options \
   -name route.common.via_on_grid_by_layer_name \
   -value {{V2 true}}
```

Feedback

Discouraging Off-Grid Routing for Vias

To discourage off-grid routing for vias, you can increase the cost of routing the via enclosure metal shapes off the wire track grid. To specify the extra cost multiplier for the metal layers on which the via enclosures are routed, set the `route.common.extra_via_off_grid_cost_multiplier_by_layer_name` application option.

Use the following syntax to set this option:

```
{ {layer multiplier} ... }
```

Specify the layers by using the layer names from the technology file. The cost multiplier must be a value between 0.0 and 20.0.

When you specify this option, the effective cost is the base cost times (1+multiplier). For example, assume that the technology file defines the VIA12 layer between the M1 and M2 metal layers and the VIA23 via layer between the M2 and M3 metal layers. To set the extra cost multiplier for the via enclosures on the M2 metal layer (and therefore the vias on the VIA12 and VIA23 via layers) to 0.5 (for an effective via cost of 1.5 times the base cost), use the following command:

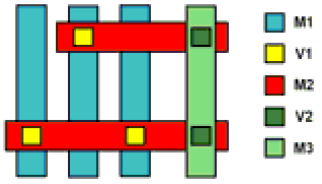
```
icc2_shell> set_app_options \
   -name route.common.extra_via_off_grid_cost_multiplier_by_layer_name \
   -value {{M2 0.5}}
```

Routing Must-Join Pins

When a pin is defined as a must-join pin, the router connects all terminals of the pin as a single net. The terminals of a must-join pin are specified with the `must_join_port` library pin attribute.

By default, Zroute connects must-join pins using a random structure, as shown in the following figure:

Figure 71 Default Must-Join Pin Connection

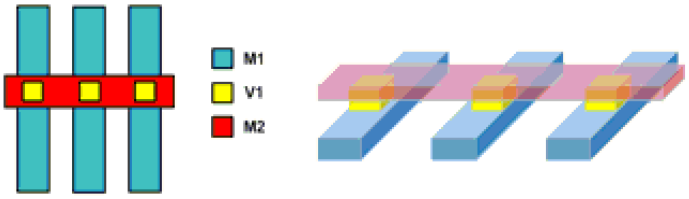


To achieve better electromagnetic results, Zroute can use the via ladder insertion capabilities to connect the must-join pins with a simplified structure, which is referred to as a pattern-based must-join connection.

Feedback

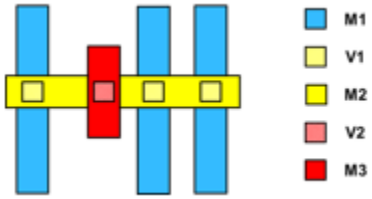
The following figure shows the simplified structure:

Figure 72 Single-Level Pattern-Based Must-Join Pin Connection



The single-level structure shown in Figure 72 is the default structure used for a pattern-based must-join connection. To use a multi-level structure, which is shown in Figure 73, set the `route.auto_via_ladder.pattern_must_join_over_pin_layer` application option to 2.

Figure 73 Multi-Level Pattern-Based Must-Join Pin Connection



Zroute automatically uses a pattern-based must-join pin connection when a library cell pin has a `pattern_must_join` attribute and you use one of the following commands to perform the routing: `route_auto`, `route_group`, or `route_eco`.

To query cells with the `pattern_must_join` attribute, use the following command:

```
icc2_shell> get_attribute \
   [get_lib_pins -all -of_objects */*/frame ] pattern_must_join
```

To get a list of pins that are marked with the `pattern_must_join` attribute, use the following command:

```
icc2_shell> get_lib_pins -all \
   -of_objects */*/frame -filter "pattern_must_join==true"
```

By default, a via ladder is not inserted if there are fixed shapes on higher layers that block the insertion. To increase the insertion rate, you can allow staggering of the vias on each level of the via ladder to avoid the fixed shapes. To specify the maximum number of tracks allowed for staggering on each level, set the `route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks` application option. You can specify a value between 0 and 9 for each level; the default is 0, which disables staggering.

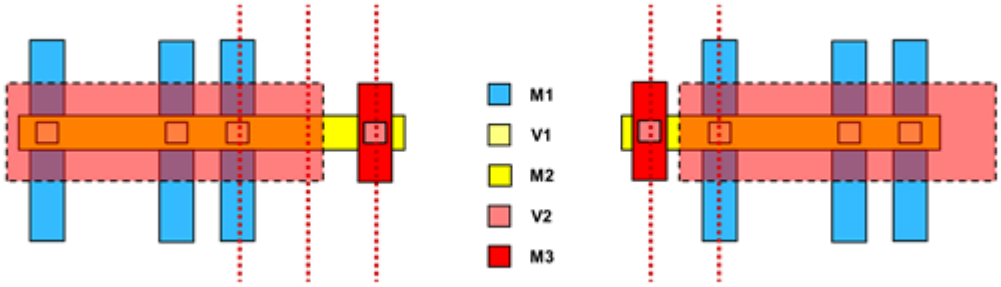
Feedback

For example, to allow staggering of up to three tracks on the first level, use the following command:

```
icc2_shell> set_app_options -name \
   route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks \
   -value {3 0}
```

Figure 74 shows the possible solutions to avoid a blockage on the M3 layer when you allow staggering of up to three tracks.

Figure 74 Pattern-Based Must-Join Pin Connection With Staggering



By default, the pattern-based must-join connections are updated each time you run the `route_auto`, `route_group`, or `route_eco` command.

- During the `route_group` command, the updates are restricted to only those nets specified in the `route_group` command by setting the `route.auto_via_ladder.update_on_route_group_nets_only` application option to `true`.
- To disable all updates during each of these routing commands, set the `route.auto_via_ladder.update_pattern_must_join_during_route` application option to `false`.

If Zroute cannot create a pattern-based must-join connection, it reports a “Needs pattern must join pin connection” DRC violation.

See Also

- Inserting Via Ladders

Controlling Pin Connections

By default, Zroute connects a signal route to a pin by using wires or vias anywhere on the pin. To restrict the allowed types of pin connections on a per-layer basis, use the following

Feedback

syntax to set the `route.common.connect_within_pins_by_layer_name` application option:

```
set_app_options
   -name route.common.connect_within_pins_by_layer_name
   -value { {layer mode} ... }
```

Valid values for the *mode* argument are

- `off` (the default)

  There are no restrictions on pin connections.

- `via_standard_cell_pins`

  Only the connections to standard cell pins by using a via are restricted. When using a via connection, the via's metal enclosure must be contained within the pin shape.

  There are no restrictions on signal routes connected to macro cell and pad cell pins by using a via or to any pins by using wires.

- `via_wire_standard_cell_pins`

  The connections to standard cell pins by using a via or a wire are restricted. When using a via connection, the via's metal enclosure must be contained within the pin shape. When using a wire, the wire must be contained within the pin shape.

- `via_all_pins`

  The connections to any pins (standard cell, macro cell, or pad cell) by using a via are restricted. When using a via connection, the via's metal enclosure must be contained within the pin shape.

  There are no restrictions on signal routes connected to any pins by using wires.

- `via_wire_all_pins`

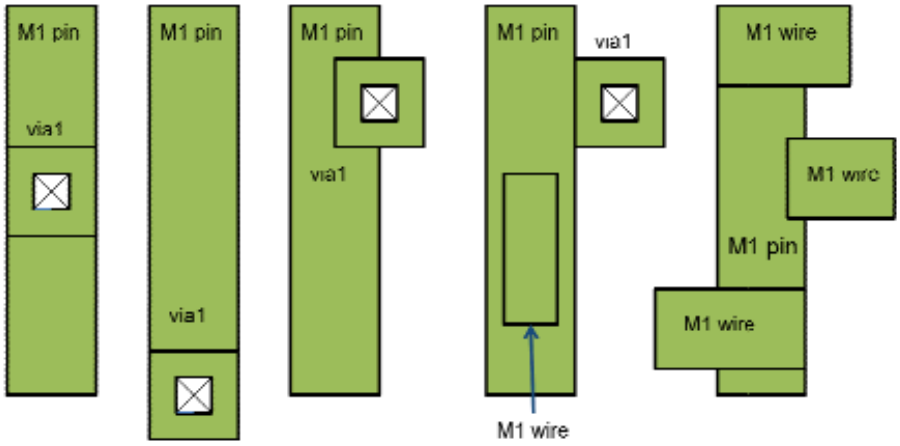
  The connections to any pins by using a via or a wire are restricted. When using a via connection, the via's metal enclosure must be contained within the pin shape. When using a wire, the wire must be contained within the pin shape.

For example, if you use the following command (or use the default settings), all of the connections shown in Figure 75 are valid and no DRC violations are reported:

```
icc2_shell> set_app_options \
   -name route.common.connect_within_pins_by_layer_name \
   -value {{M1 off}}
```

Feedback

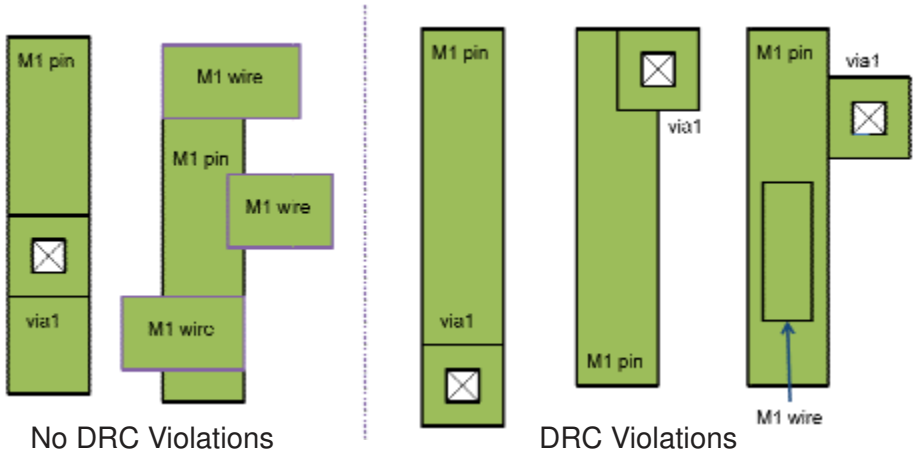
Figure 75 Unrestricted Pin Connections



If you set the mode for M1 to `via_all_pins`, as shown in the following example, the via enclosures must be inside the pin shape. The connections shown on the left side of Figure 76 are valid; however, the connections on the right side of the figure cause DRC violations.

```
icc2_shell> set_app_options \
   -name route.common.connect_within_pins_by_layer_name \
   -value {{M1 via_all_pins}}
```

Figure 76 Restricted Via-to-Pin Connections



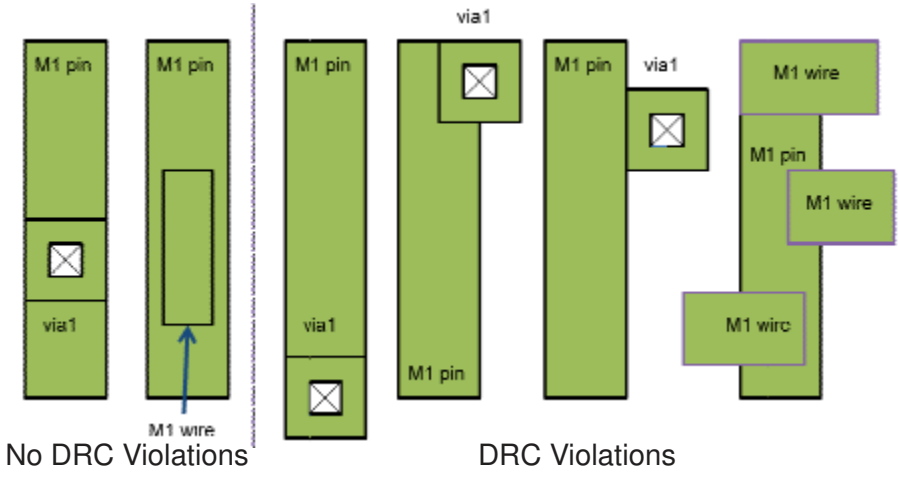
If you set the mode for M1 to `via_wire_standard_cell_pins`, as shown in the following example, both the via enclosures and wires must be inside the pin shape. The connections

Feedback

shown on the left side of Figure 77 are valid; however the connections on the right side of the figure cause DRC violations.

```
icc2_shell> set_app_options \
   -name route.common.connect_within_pins_by_layer_name \
   -value {{M1 via_wire_standard_cell_pins}}
```

Figure 77 Restricted Via-to-Pin and Wire-to-Pin Connections



Controlling Pin Tapering

Pin tapering is the method used to connect wires with nondefault routing rules to pins. Zroute supports pin tapering for both hard and soft nondefault routing rules; the tapering implementation is the same for both types of nondefault routing rules.

Note:

By default, if a net has both a nondefault routing rule defined by the `create_routing_rule` command and voltage-based spacing rules defined in the technology file, Zroute performs pin tapering on that net based on the nondefault routing rule. To disable pin tapering on these nets, set the `route.detail.enable_ndr_tapering_on_voltage_rule` application option to `false`.

You can specify the method used for pin tapering and control the tapering width, as described in the following topics:

- Specifying the Tapering Method
- Controlling the Tapering Width

Feedback

Specifying the Tapering Method

You specify the tapering method for a nondefault routing rule when you define the rule with the `create_routing_rule` command. By default, Zroute uses distance-based pin tapering; it uses the default routing rule within the tapering distance from the pin and uses the nondefault routing rule beyond the tapering distance. However, advanced process nodes are very sensitive to jogs and fat metal, and sometimes the tapering distance is not sufficient to fix the routing DRC violations on nets with nondefault routing rules. In these cases, you can use layer-based tapering, which targets DRC violations on nets with nondefault routing rules.

Note:

Distance-based tapering and layer-based tapering are mutually exclusive. If you define a routing rule that uses both a distance-based tapering option and a layer-based tapering option, the tool uses the layer-based tapering settings and ignores the distance-based tapering settings.

When Zroute performs distance-based pin tapering, it

- Determines the tapering distance, which is about 10 times the mean number of tracks for all routing layers.

  To explicitly specify the tapering distance, use the `-taper_distance` option when you create a nondefault routing rule with the `create_routing_rule` command.

- Uses the same tapering distance for all pins.

  To specify a different tapering distance for driver pins, use the `-driver_taper_distance` option when you create a nondefault routing rule with the `create_routing_rule` command.

To perform layer-based pin tapering, use the `-taper_over_pin_layers` or `-taper_under_pin_layers` option when you create a nondefault routing rule with the `create_routing_rule` command. To specify a different layer-based tapering distance for driver pins, use the `-driver_taper_over_pin_layers` or `-driver_taper_under_pin_layers` option.

- For pins on the M1 or M2 layers, use the `-taper_over_pin_layers` option (or the `-driver_taper_over_pin_layers` option for driver pins) to specify the number of layers on or above the pin layer available for tapering. A value of 1 enables pin tapering only on the pin layer; a larger value enables pin tapering on additional layers above the pin layer.
- For pins on upper layers, use the `-taper_under_pin_layers` option (or the `-driver_taper_under_pin_layers` option for driver pins)to specify the number of layers on or below the pin layer available for tapering. A value of 1 enables pin tapering only on the pin layer; a larger value enables pin tapering on additional layers below the pin layer.

Feedback

Controlling the Tapering Width

By default, the wire is tapered to the default routing width, which is the routing width that is defined for the metal layer in the technology file.

- To taper the wire to the pin width rather than the default routing width, change the `route.detail.pin_taper_mode` application option to `pin_width` from its default of `default_width` before you perform detail routing.
- To enable pin tapering only when required to avoid DRC violations, set the `route.detail.use_wide_wire_effort_level` application option to either `low` or `high`.

  This setting improves the nondefault via rate at a cost of longer runtime. You should use this option only when the majority of pins on the nets being routed are accessible with nondefault vias.
- To disable tapering for certain types of pins, set one or more of the following application options to `true`: `route.detail.use_wide_wire_to_input_pin`, `route.detail.use_wide_wire_to_output_pin`, `route.detail.use_wide_wire_to_macro_pin`, `route.detail.use_wide_wire_to_pad_pin`, and `route.detail.use_wide_wire_to_port`.
- To disable tapering for all pins, set the `-taper_distance` option to 0 when you create the nondefault routing rule with the `create_routing_rule` command.

Controlling Via Ladder Connections

If a pin has a via ladder, by default, Zroute connects to the topmost layer of a via ladder and not directly to the existing pin geometry. To modify the connection constraints, set the following application options:

- `route.detail.via_ladder_upper_layer_via_connection_mode`

  To allow vias to connect to the top level of a via ladder from below, set this application option to `any`.
- `route.detail.allow_default_rule_nets_via_ladder_lower_layer_connection`

  To allow nets with default routing rules to connect directly to the pin or to any layer of the via ladder, set this application option to `true`.

Feedback

Setting the Rerouting Mode

By default, Zroute can reroute nets as needed. You can prevent rerouting or limit rerouting to minor changes by setting the `physical_status` attribute on the nets.

- To freeze the net and prevent rerouting, set the attribute to `locked`.
- To limit rerouting to minor changes, set the attribute to `minor_change`.
- To allow Zroute to reroute the nets as needed, set the attribute to `unrestricted`.

For example, to prevent rerouting of the net1 net, which uses the default routing rule, use the following command:

```
icc2_shell> set_attribute -objects [get_nets net1] \
   -name physical_status -value locked
```

Routing Application Options

The IC Compiler II tool provides application options that affect the individual routing engines (global routing, track assignment, and detail routing), as well as application options that affect all three routing engines.

- For information about the application options that affect all three routing engines, see the `route.common_options` man page.
- For information about the application options that affect global routing, see the `route.global_options` man page.
- For information about the application options that affect track assignment, see the `route.track_options` man page.
- For information about the application options that affect detail routing, see the `route.detail_options` man page.

Zroute uses these option settings whenever you perform routing functions. When you run a routing command, Zroute writes the settings for any routing options that you have set (or that the tool has set for you) in the routing log. To display the settings for all routing options, not only those that have been set, set the `route.common.verbose_level` application option to 1.

```
icc2_shell> set_app_options \
   -name route.common.verbose_level -value 1
```

Feedback

Routing Clock Nets

To route clock nets before routing the rest of the nets in the block, use the `route_group` command with the appropriate option to select the nets.

- To route all clock nets, use the `-all_clock_nets` option.
- To route specific clock nets, use the `-nets` option.

For example, to route all clock nets, use the following command:

```
icc2_shell> route_group -all_clock_nets
```

The `route_group` command runs global routing, track assignment, and detail routing on the clock nets.

- When routing clock nets, the `route_group` command uses A-tree routing to minimize the distance between each pin and driver.
- When routing clock mesh nets, by default, the `route_group` command uses Steiner routing, which minimizes the total wire length. To use comb routing, set the `route.common.clock_topology` application option to `comb` before routing the clock mesh nets.
- By default, if the block contains existing global routes, the `route_group` command ignores them during global routing. To perform incremental global routing by reusing existing global routes, use the `-reuse_existing_global_route true` option.
- If the block contains existing detail routes for the clock nets, the `route_group` command performs incremental detail routing.
- By default, the detail router performs a maximum of 40 search and repair iterations. To modify the maximum number of detail routing iterations, use the `-max_detail_route_iterations` option.

  Note:

  Zroute stops before completing the maximum number of iterations if it determines that all violations have been fixed or that it cannot fix the remaining violations.

Feedback

Routing Critical Nets

To route a group of critical nets before routing the rest of the nets in the block, use the `route_group` command. To specify the nets to route, use one of the following options:

- `-nets`

  Use this option to specify the nets on the command line:

  ```
  icc2_shell> route_group -nets collection_of_critical_nets
  ```

- `-from_file`

  Use this option to specify the nets in a file:

  ```
  icc2_shell> route_group -from_file file_name
  ```

If the specified nets are associated with a routing corridor, the nets are routed within the defined region. Note that global routing considers routing corridors as a hard constraint, while track assignment and detail routing consider routing corridors as a soft constraint and might route nets slightly outside of the routing corridor to fix DRC violations.

By default, the `route_group` command ignores existing global routes, reuses dangling wires and existing detail routes on the specified nets, and runs global routing, track assignment, and detail routing on the specified nets.

- To perform incremental global routing, set the `-reuse_existing_global_route` option to `true`.

  Note:

  You cannot use the `-reuse_existing_global_route true` option when routing the nets in a routing corridor. If you use this option, the global router ignores the routing corridors.

- To disable the reuse of dangling wires, set the `-utilize_dangling_wires` option to `false`.
- To stop after global routing, set the `-stop_after_global_route` option to `true`.

By default, the detail router performs a maximum of 40 iterations search and repair iterations. If Zroute determines before then that all violations have been fixed or that it cannot fix the remaining violations, it stops. To change the maximum number of detail routing iterations, use the `-max_detail_route_iterations` option.

By default, the `route_group` command does not fix soft DRC violations, such as bridge rule violations. To enable the fixing of soft DRC violations after the final detail routing

Feedback

iteration, set the `route.common.post_group_route_fix_soft_violations` application option to `true`.

```
icc2_shell> set_app_options \
   -name route.common.post_group_route_fix_soft_violations \
   -value true
```

Routing Secondary Power and Ground Pins

To use Zroute to perform secondary power and ground pin routing,

1. Verify that the secondary power and ground pins have the appropriate attributes in the standard cell frame views.
2. Set the routing constraints for secondary power and ground pin routing.
3. Perform secondary power and ground pin routing by using the `route_group` command.

The following topics describe these steps.

Verifying the Secondary Power and Ground Pin Attributes

Before you use Zroute to perform secondary power and ground pin routing, you must verify that the cell libraries have the correct attributes on the secondary power and ground pins of the standard cell frame views.

The following attributes are required for secondary power and ground pins:

- `is_secondary_pg`

  This attribute must have a setting of `true`.

- `port_type`

  This attribute must have a setting of `power` or `ground`.

To verify that these attributes are correctly set on the library pins in the standard cell frame view, use the following command:

```
icc2_shell> report_attributes -application \
   [get_lib_pins -of_objects reflib/cell/frame -all \
      -filter "name == attr_name"]
```

where *reflib* is the cell library name, *cell* is the name of the library cell you want to check (or * if you want to check all library cells), and *attr_name* is either `is_secondary_pg` or `port_type`.

Feedback

See Also

- Identifying Secondary PG Pins

Setting the Routing Constraints

You can set routing constraints for secondary power and ground pin routing in the same way as for regular signal routing. For example, you can set constraints by

- Defining the minimum and maximum routing layers by using the `set_routing_rule` command

  For more information about using the `set_routing_rule` command, see Specifying Net-Specific Layer Constraints.

- Specifying the preferred pin connections by setting the `route.common.single_connection_to_pins` and `route.common.connect_within_pins_by_layer_name` application options

  For example, to require a single connection to the secondary power and ground pins and require that the M1 connections use vias contained within the pin shapes, use the following command:

```
icc2_shell> set_app_options \
   -name route.common.single_connection_to_pins \
   -value standard_cell_pins
icc2_shell> set_app_options \
   -name route.common.connect_within_pins_by_layer_name \
   -value {{M1 via_standard_cell_pins}}
```

- Defining the maximum number of power or ground pins in a cluster by setting the `route.common.number_of_secondary_pg_pin_connections` application option

  A cluster is a set of connected secondary power or ground pins that has one connection to a PG strap or ring. By default, the value of this option is 0, which means that there is no limit on the number of secondary power or ground pins in a cluster.

  For example, to connect all secondary power and ground pins directly to a PG strap or ring, use the following command:

```
icc2_shell> set_app_options \
   -name route.common.number_of_secondary_pg_pin_connections \
   -value 1
```

Feedback

- Defining a nondefault routing rule for secondary power and ground pin routing

  For example, to define a nondefault routing rule named wideSVDD for wide M1 and M2 and set the nondefault routing rule on the VDD2 net, to which the secondary power and ground pins are connected, use the following commands:

```
icc2_shell> create_routing_rule wideSVDD -widths { M1 0.3 M2 0.3 }
icc2_shell> set_routing_rule -rule wideSVDD {VDD2}
```

  For more information about using nondefault routing rules, see Using Nondefault Routing Rules.

By default, the constraints apply to both the secondary power and ground connections and the tie-off connections. To separate these connections so that you can set constraints only for the secondary power and ground connections, set the `route.common.separate_tie_off_from_secondary_pg` application option to `true`.

Routing the Secondary Power and Ground Pins

If the secondary power and ground pins have the appropriate attributes in the frame view, you can use Zroute to route the secondary power and ground pins.

By default, Zroute performs four topology ECO iterations to fix secondary PG cluster fanout violations.The topology ECO iterations can be time consuming, so you can control the effort level, and therefore the runtime, by setting the `route.detail.topology_eco_effort_level` application option to one of the following values:

- `off`, which disables topology ECO; this is the default
- `low`, which peforms one topology ECO iteration
- `medium`, which peforms four topology ECO iterations
- `high`, which peforms eight topology ECO iterations

For example, to connect the secondary power pins to the VDD1 net, run the following command:

```
icc2_shell> route_group -nets VDD1
```

The following example add new cells, which have secondary power and ground pins, to a block; logically connects the power and ground pins; and then connects the secondary power pins to the VDD2 net.

```
icc2_shell> add_buffer {TOP/U1001/Z} {libA/BUFHVT} \
   -new_cell_names mynewHVT
icc2_shell> add_buffer {TOP/U1002/Z} {libA/BUFHVT} \
   -new_cell_names mynewHVT
icc2_shell> legalize_placement
```

Feedback

```
icc2_shell> connect_pg_net -automatic
icc2_shell> route_group -nets {VDD2}
```

Routing Signal Nets

Before you route the signal nets, all clock nets must be routed without violations.

You can route the signal nets by using one of the following methods:

- Use the task-specific commands to perform the standalone routing tasks.
  - To perform global routing, use the `route_global` command.

    For details see, Global Routing.
  - To perform track assignment, use the `route_track` command.

    For details see, Track Assignment.
  - To perform detail routing, use the `route_detail` command.

    For details see, Detail Routing.

  When you run a standalone routing command, such as `route_global` or `route_detail`, Zroute reads in the block at the beginning of each routing command and updates the block at the end of each command. The router does not check the input data. For example, if the track assignment step is skipped and you run detail routing directly, Zroute might generate bad routing results.

  If you need to customize your routing flow or you need to run a large block step-by-step, you might want to use the standalone routing commands instead of automatic routing.

- Use automatic routing (the `route_auto` command).

  The `route_auto` basic command performs global routing, track assignment, and detail routing. For details, see Routing Signal Nets by Using Automatic Routing.

  When you run `route_auto`, Zroute reads the block before starting routing and updates the block when all routing steps are done. If you stop automatic routing before it performs detail routing, Zroute checks the input data when you restart routing with this command.

  Use the `route_auto` command when you run routing to verify convergence, congestion, and design rule quality-of-results (QoR). You also might want to use `route_auto` if congestion QoR is your main goal, rather than timing QoR.

Zroute can insert redundant vias during signal routing. For information about this capability, see Inserting Redundant Vias on Signal Nets.

Feedback

Global Routing

Before you run global routing,

- Define the common routing application options

  For information about the common routing application options, see the `route.common_options` man page.

- Define the global routing application options

  For information about the global routing application options, see the `route.global_options` man page.

To perform standalone global routing, use the `route_global` command. By default, global routing is not timing-driven. For information about enabling timing-driven global routing, see Timing-Driven Global Routing.

The global router divides a block into global routing cells. By default, the width of a global routing cell is the same as the height of a standard cell and is aligned with the standard cell rows.

For each global routing cell, the routing capacity is calculated according to the blockages, pins, and routing tracks inside the cell. Although the nets are not assigned to the actual wire tracks during global routing, the number of nets assigned to each global routing cell is noted. The tool calculates the demand for wire tracks in each global routing cell and reports the overflows, which are the number of wire tracks that are still needed after the tool assigns nets to the available wire tracks in a global routing cell.

For advanced node designs, via density can be a concern. To enable via density modeling, set the `route.global.via_cut_modeling` application option to `true`. The tool then calculates the number of vias in each global routing cell and reports via overflows.

Global routing is done in two phases:

- The initial routing phase (phase 0), in which the tool routes the unconnected nets and calculates the overflow for each global routing cell
- The rerouting phases, in which the tool tries to reduce congestion by ripping up and rerouting nets around global routing cells with overflows

  The tool might perform several rerouting phases. At the end of each rerouting phase, the tool recalculates the overflows. You should see a reduction in the total number of global routing cells with overflow and in the total overflow numbers. The global router stops and exits from the rerouting phase when the congestion is solved or cannot be solved further or after the maximum number of phases has occurred, as defined by the `-effort_level` option. You can force the global router to perform the maximum number of phases based on the specified effort level by setting the `route.global.force_full_effort` application option to `true`. By default, the tool

Feedback

uses medium effort and performs a maximum of three rerouting phases. You can perform up to six rerouting phases by specifying ultra effort.

There are five global routing effort levels: `minimum`, `low`, `medium`, `high`, and `ultra`.

- Minimum (`-effort_level minimum`)

  The minimum effort level uses two times larger global routing cells relative to the other effort levels. It also has a much lower congestion cost and runs only one rerouting phase. It should only be used for prototype routing or for an initial congestion evaluation, not for detail routing.

- Low (`-effort_level low`)

  Low effort runs a maximum of two rerouting phases with very similar congestion cost. It is faster in comparison to medium effort and has reasonable QoR. If your block is not very congested, you can use the low effort level.

- Medium (`-effort_level medium`)

  Medium effort is the default effort level and runs a maximum of three rerouting phases. Global routing stops after the third phase or when the overflow is resolved, whichever occurs first.

- High (`-effort_level high`)

  High effort runs up to four rerouting phases. If your block is congested, use the high effort level.

- Ultra (`-effort_level ultra`)

  Ultra effort runs up to six rerouting phases. If your block is very congested, use the ultra effort level.

At the end of global routing, the following information is stored in the design library:

- The g-links and g-vias on each routed net

  This information is used for the next routing steps. After Zroute performs track assignment and detail routing, it removes these g-links and g-vias from the design library.

- The congestion data

  This information is used to generate a congestion map. By default, only the hard congestion data is saved in the design library. To also save the soft congestion data, set the `route.global.export_soft_congestion_maps` application option to `true` before performing global routing. The soft congestion data includes demand from soft nondefault spacing rules, as well as tool-generated soft rules.

Feedback

The global router reports block statistics and congestion data after the initial routing phase and after each rerouting phase. When global routing is complete, the global router reports a summary of the wire length and via count.

Example 17 shows a global routing report. In the congestion report, the Overflow value is the total number of wires in the block that do not have a corresponding track available. The Max value corresponds to the highest number of overutilized wires in a single global routing cell. The GRCs value is the total number of overcongested global routing cells in the block.

Example 17 Global Routing Report

```
Start Global Route ...
...
Design statistics:
Design Bounding Box (0.00,0.00,3180.00,1154.00)
Number of routing layers = 10
layer M1, dir Hor, min width = 0.05, min space = 0.05 pitch = 0.15
layer M2, dir Ver, min width = 0.06, min space = 0.06 pitch = 0.15
...
Net statistics:
Total number of nets     = 255165
Number of nets to route  = 248716
Number of single or zero port nets = 1721
4728 nets are fully connected,
 of which 4728 are detail routed and 0 are global routed.
1648 nets have non-default rule clock_spacing
...
phase3. Routing result:
phase3. Both Dirs: Overflow =  3320 Max = 3 GRCs =  4405 (0.08%)
phase3. H routing: Overflow =  1759 Max = 2 (GRCs =  1) GRCs =  2756 (0.10%)
phase3. V routing: Overflow =  1560 Max = 3 (GRCs = 20) GRCs =  1649 (0.06%)
phase3. M1         Overflow =  1475 Max = 2 (GRCs =  1) GRCs =  2426 (0.09%)
phase3. M2         Overflow =  1265 Max = 3 (GRCs = 20) GRCs =  1343 (0.05%)
...
Overflow over macro areas

phase3. Both Dirs: Overflow =   293 Max =  2 GRCs =   300 (0.04%)
phase3. H routing: Overflow =   133 Max =  1 (GRCs = 129) GRCs =   139 (0.03%)
phase3. V routing: Overflow =   160 Max =  2 (GRCs =  2) GRCs =   161 (0.04%)
phase3. M1         Overflow =     0 Max =  0 (GRCs =  0) GRCs =     0 (0.00%)
phase3. M2         Overflow =     0 Max =  0 (GRCs =  0) GRCs =     0 (0.00%)
...
Density distribution:
Layer    0.0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1.0  1.1  1.2  >
 1.2
M1       86.6 10.5 0.37 1.46 0.02 0.44 0.18 0.14 0.07 0.03 0.10 0.00 0.00 0.02
M2       68.2 14.4 7.71 4.54 2.07 1.55 0.66 0.28 0.13 0.03 0.33 0.00 0.00 0.02
...
phase3. Total Wire Length = 21154552.81
phase3. Layer M1 wire length = 947324.93
phase3. Layer M2 wire length = 3959478.25
...
phase3. Total Number of Contacts = 2530044
phase3. Via VIA12SQ_C count = 1050582
phase3. Via VIA23SQ_C count = 856311
...
phase3. completed.
```

Feedback

Before proceeding to detail routing, display the congestion map in the GUI and check the overflow distribution. The congestion report and map help you to identify congested areas. For more information about the congestion report, see Generating a Congestion Report. For more information about the congestion map, see Generating a Congestion Map.

Global Routing During Design Planning

During design planning you can perform exploration-mode global routing by using the `-floorplan true` option with the `route_global` command.

```
icc2_shell> route_global -floorplan true
```

If you are using the hierarchical flow, enable virtual-flat global routing by using the `-virtual_flat` option with the `route_global` command. When you set the `-virtual_flat` option to `all_routing`, Zroute routes all the nets in the block and preserves the hierarchy and pin constraints. You can increase the virtual-flat global routing speed by routing only the top-level nets. To do this, set the `-virtual_flat` option to `top_and_interface_routing_only`. When the `-virtual_flat` option is set to `off`, which is the default, Zroute ignores the physical hierarchy and routes the block as flat.

```
icc2_shell> route_global -floorplan true -virtual_flat all_routing
```

Timing-Driven Global Routing

By default, the `route_global` command is not timing-driven. To enable timing-driven global routing, set the `route.global.timing_driven` application option before you run the `route_global` command. During timing-driven global routing, the global router considers the impact of layer resistance by minimizing the usage of pin access layers, even if this increases wire length.

By default, when you enable timing-driven global routing, the tool

1. Calculates the net delays of the block

   If the design library contains global route information, the tool uses the global route information to calculate the net delays; otherwise, it uses virtual routing to calculate the net delays.

   The global routing results can vary depending on whether the initial net delays were calculated by using global route information or virtual routing. To remove existing global route information from the signal nets in the block, use the `-global_route` and `-net_types signal` options with the `remove_routes` command, as shown in the following example:

   ```
   icc2_shell> remove_routes -global_route -net_types signal
   ```

2. Performs an initial routing phase (phase 0), in which the tool routes the unconnected nets and calculates the overflow for each global routing cell

Feedback

3. Performs two rerouting phases, in which the tool tries to reduce congestion by ripping up and rerouting nets around global routing cells with overflows
4. Updates the design timing based on the first three global routing phases
5. Identifies newly critical nets based on the timing update
6. Performs three rerouting phases, in which the tool rips up and reroutes the newly critical nets

To increase the number of rerouting phases after the timing update, set the global routing effort level to `high` or `ultra` by using the `-effort_level` option with the `route_global` command.

To reduce runtime, at a possible cost to the timing QoR, set the `route.global.advance_node_timing_driven_effort` application option.

- When you set this application option to `medium`, the tool rips up and reroutes the newly critical nets only if doing so does not increase congestion. In addition, the number of rerouting phases is reduced for medium-, high-, and ultra-effort global routing.
- When you set this application option to `low`, the tool does not perform the timing update and the number of rerouting phases is reduced for medium, high-, and ultra-effort global routing.

The following table summarizes the timing-driven global routing behavior based on the effort level settings. The shaded cell shows the default behavior.

Table 26 Number of Rerouting Phases Based on the Effort Level Settings

route_global -effort_level setting	**route.global.advance_node_timing_driven_effort setting**		
	`low`	**`medium`**	**`high` (default)**
`low`	2	2	2
`medium` (default)	3	3[3,4]	5[3,5]
`high`	4	4[3,4]	6[3,4]
`ultra`	6	6[3,4]	8[3,4]

To control the tradeoff between timing QoR and DRC convergence, set the `route.global.timing_driven_effort_level` application option. By default, this option has a setting of `high`, which favors timing QoR over DRC convergence.

3. *Timing update performed after the second rerouting phase.*
4. *Newly critical nets are ripped up and rerouted only if it does not increase congestion.*
5. *Newly critical nets are always ripped up and rerouted.*

Feedback

Crosstalk-Driven Global Routing

By default, the `route_global` command is not crosstalk-driven. To enable crosstalk-driven global routing, set the `route.global.crosstalk_driven` and `time.si_enable_analysis` application options to `true` before you run the `route_global` command.

When you enable crosstalk-driven global routing, the tool calculates the net delays before invoking the global router. If the design library contains global route information, the tool uses the global route information to calculate the net delays; otherwise, it uses virtual routing to calculate the net delays. The global routing results can vary depending on whether the initial net delays were calculated by using global route information or virtual routing. To remove existing global route information from the signal nets in the block, use the `-global_route` and `-net_types signal` options with the `remove_routes` command, as shown in the following example:

```
icc2_shell> remove_routes -global_route -net_types signal
```

Incremental Global Routing

By default, the global router ignores existing global routes. To perform incremental global routing by reusing the existing global routes, use the `-reuse_existing_global_route true` option when you run global routing. Note that this option affects only the global router and not the net delay calculation that occurs before timing-driven or crosstalk-driven global routing.

Track Assignment

Before you run track assignment,

- Define the common routing application options

  For information about the common routing application options, see the `route.common_options` man page.

- Define the global routing application options

  For information about the track assignment application options, see the `route.track_options` man page.

- Complete global routing

To perform standalone track assignment, run the `route_track` command.

Feedback

The main task of track assignment is to assign routing tracks for each global route. During track assignment, Zroute performs the following tasks:

- Assigns tracks in horizontal partitions.
- Assigns tracks in vertical partitions.
- Reroutes overlapping wires.

After track assignment finishes, all nets are routed but not very carefully. There are many violations, particularly where the routing connects to pins. Detail routing works to correct those violations.

Note:

Because track assignment replaces the global routes with actual metal shapes, the block no longer contains global routes after track assignment completes.

By default, the `route_track` command is not timing-driven or crosstalk-driven.

- To enable timing-driven mode, set the `route.track.timing_driven` application option.
- To enable crosstalk-driven mode, set the `route.track.crosstalk_driven` and `time.si_enable_analysis` application options to `true`.

At the end of track assignment, Zroute reports a summary of the wire length and via count. Example 18 shows a track assignment report.

Example 18 Track Assignment Report

```
Wire length and via report:
---------------------------
Number of M1 wires: 215327                 : 0
Number of M2 wires: 1124740               VIA12SQ_C: 1067462
...
Total number of wires: 2508734            vias: 2769482

Total M1 wire length: 924480.9
Total M2 wire length: 4147032.0
...
Total wire length: 21281278.0

Longest M1 wire length: 1541.7
Longest M2 wire length: 926.0
...
```

Feedback

Detail Routing

Before you run detail routing,

- Define the common routing application options

  For information about the common routing application options, see the `route.common_options` man page.

- Define the detail routing application options

  For information about the detail routing application options, see the `route.detail_options` man page.

- Complete global routing and track assignment

The detail router uses the general pathways suggested by global routing and track assignment to route the nets, and then it divides the block into partitions and looks for DRC violations in each partition. When the detail router finds a violation, it rips up the wire and reroutes it to fix the violation. During detail routing, Zroute concurrently addresses routing design rules and antenna rules and optimizes via count and wire length. For more information about antenna rules, see Finding and Fixing Antenna Violations.

To perform standalone detail routing, run the `route_detail` command.

By default, the `route_detail` command

- Performs detail routing on the whole block

  You can restrict the routing to a specific area of the block by using the `-coordinates` option (or by specifying or selecting the bounding box in the GUI).

- Uses one uniform partition for the first iteration and adjusts the partitioning for subsequent iterations

  Zroute uses the single uniform partition for the first iteration to generate all DRC violations for the chip at the same time. At the beginning of each subsequent iteration, the router checks the distribution of the DRC violations. If the DRC violations are evenly distributed, the detail router uses a uniform partition. If the DRC violations are located in some local areas, the detail router uses nonuniform partitions.

  In some cases, such as when a design contains standard cells with long pins, you can improve the detail routing QoR and reduce the runtime by increasing the partition size. To enable this feature, set the `route.detail.optimize_partition_size_for_drc` application option to `true`.

Feedback

- Performs iterations until one of the following conditions exists:
  - All of the violations have been fixed
  - The maximum number of iterations has been reached

    By default, the maximum number of iterations is 40. You can change this limit by setting the `-max_number_iterations` option.

    ```
    icc2_shell> route_detail -max_number_iterations 20
    ```

  - It cannot fix any of the remaining violations

    You can change the effort that the detail router uses for fixing the remaining violations before it gives up by setting the `route.detail.drc_convergence_effort_level` application option.

    ```
    icc2_shell> set_app_options \
       -name route.detail.drc_convergence_effort_level -value high
    ```

    You can force the detail router to complete the maximum number of iterations, regardless of the DRC convergence status, by setting the `route.detail.force_max_number_iterations` application option to `true`.

    ```
    icc2_shell> set_app_options \
       -name route.detail.force_max_number_iterations -value true
    ```

- Is not timing-driven

  To enable timing-driven detail routing, set the `route.detail.timing_driven` application option to `true`.

  ```
  icc2_shell> set_app_options \
     -name route.detail.timing_driven -value true
  ```

  By default, when you enable timing-driving detail routing, Zroute uses medium effort to assign timing-critical nets to low-resistance metal layers. To change the extent to which timing-driven detail routing prefers lower-resistance metal layers when routing timing-critical nets, set the `route.common.rc_driven_setup_effort_level` application option. To increase the effort level to use more low-resistance metal layers for routing, set the option to `high`. To reduce the effort level, set the option to `low`. To disable resistance-based routing layer preferences, set the option to `off`.

- Does not fix shorted nets over macro cells

  If default detail routing leaves shorted nets over macro cells, analyze the block to determine if the shorts are caused by the availability of only a single layer for routing over the macro cells. If so, use routing guides to encourage river routing over the macros with shorted nets and rerun detail routing. For details, see Using Routing Guides to Encourage River Routing.

Feedback

If shorted nets remain after using river routing, enable the fixing of shorted nets over macro cells by automatically ripping up and rerouting the shorted nets by setting the `route.detail.repair_shorts_over_macros_effort_level` application option to `low`, `medium`, or `high` and running incremental detail routing. The higher the effort level, the more ECO routing iterations are performed, which can reduce the number of DRC violations at the expense of runtime.

```
icc2_shell> set_app_options \
   -name route.detail.repair_shorts_over_macros_effort_level \
   -value high
```

- Does not fix soft DRC violations, such as bridge rule violations

  To enable the fixing of soft DRC violations after the final detail routing iteration, set the `route.common.post_detail_route_fix_soft_violations` application option to `true`.

  ```
  icc2_shell> set_app_options \
     -name route.common.post_detail_route_fix_soft_violations \
     -value true
  ```

You can run additional detail routing iterations on a routed block by running incremental detail routing (the `-incremental` option). Be sure to use the `-incremental` option; otherwise, Zroute restarts at iteration 0 with a fixed-size partition.

```
icc2_shell> route_detail -incremental true
```

By default, incremental detail routing does not fix soft DRC violations, such as bridge rule violations. To enable the fixing of soft DRC violations after the final incremental detail routing iteration, set the `route.common.post_incremental_detail_route_fix_soft_violations` application option to `true`.

```
icc2_shell> set_app_options \
   -name route.common.post_incremental_detail_route_fix_soft_violations \
   -value true
```

Note:

Incremental detail routing does not fix open nets. To fix open nets, you must run ECO routing. For information about ECO routing, see Performing ECO Routing.

If you want to override certain runtime-intensive application options, you can run detail routing in early mode. To turn on the early mode in detail routing, use the `set_qor_strategy` command with the `-mode` option set as `early`.

```
icc2_shell> set_qor_strategy -mode early -stage pnr
```

Feedback

Early mode detail routing automatically creates a new DRC type called skipped regions that can detect dirty regions with pin overflow. To identify the dirty regions, open the error browser and select **Skipped region**.

If you want to view the DRC violations before postroute optimization, you can save the block after a specified number of iterations by setting the `route.detail.save_after_iterations` application option. The saved block is called DR_itr*n*, where n is the specified iteration. You can use a string other than DR as the prefix by setting the `route.detail.save_cell_prefix` application option.

Zroute generates a DRC violations summary at the end of each iteration. After completing detail routing, Zroute outputs a final summary report. This report includes all violations detected by Zroute, as well as information about the redundant via conversion rates. If you want an additional report that excludes violations that are not of interest to you, specify the rules to exclude by setting the `route.detail.report_ignore_drc` application option. The syntax to set this option is

```
set_app_options -name route.detail.report_ignore_drc -value list_of_drcs
```

The values used in the *list_of_drcs* argument are the DRC names used in the summary report. If the DRC name includes a space, you must enclose the name in double quotation marks. For a complete list of the supported DRC names, see the man page.

Example 19 shows a detail routing report.

Example 19 Detail Routing Report

```
Start DR iteration 0: uniform partition
Routed  1/27405 Partitions, Violations =        0
Routed  137/27405 Partitions, Violations =      264
…

DR finished with 7398 violations

DRC-SUMMARY:
        @@@@@@@ TOTAL VIOLATIONS =      7398
        Diff net spacing : 194
        Same net spacing : 4
        Diff net via-cut spacing : 2328
        Same net via-cut spacing : 1
        Less than minimum width : 5
        Less than minimum area : 36
        Short : 87
        End of line enclosure : 4742
        Less than NDR width : 1

Total Wire Length =                    23928849 micron
Total Number of Contacts =             2932706
Total Number of Wires =                2878293
Total Number of PtConns =              88536
Total Number of Routed Wires =        2878293
Total Routed Wire Length =            23920011 micron
Total Number of Routed Contacts =       2932706
        Layer                 M1 :     962827 micron
        Layer                 M2 :    4233755 micron
```

Feedback

```
…
        Via            VIA78SQ_C :       1742
        Via       VIA78SQ_C(rot) :          9
…

Redundant via conversion report:
--------------------------------

  Total optimized via conversion rate = 98.96% (2902130 / 2932706 vias)

    Layer VIA1       = 97.81% (1091973/ 1116367 vias)
        Weight 1     = 97.81% (1091973 vias)
        Un-optimized =  2.19% (24394   vias)
…
  Total double via conversion rate    = 98.96% (2902130 / 2932706 vias)

    Layer VIA1       = 97.81% (1091973/ 1116367 vias)
    Layer VIA2       = 99.97% (1071650/ 1071978 vias)
…
  The optimized via conversion rate based on total routed via count =
98.96% (2902130 / 2932706 vias)

    Layer VIA1       = 97.81% (1091973/ 1116367 vias)
        Weight 1     = 97.81% (1091973 vias)
        Un-optimized =  2.19% (24394   vias)
…
Total number of nets = 255165
0 open nets, of which 0 are frozen
Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0
 nets
                                   0 ports without pins of 0 cells connected to
 0 nets
                                   0 ports of 0 cover cells connected to 0
 non-pg nets
Total number of DRCs = 7398
Total number of antenna violations = antenna checking not active
Information: Routes in non-preferred voltage areas = 8170 (ZRT-559)

Topology ECO iteration 1 ended with 0 qualifying violations.
```

Routing Signal Nets by Using Automatic Routing

Before you run automatic routing,

- Set the common routing application options

  For information about the common routing application options, see the `route.common_options` man page.

- Set the global routing application options

  For information about the detail routing application options, see the `route.global_options` man page.

Feedback

- Set the track assignment application options

  For information about the detail routing application options, see the `route.track_options` man page.

- Set the detail routing application options

  For information about the detail routing application options, see the `route.detail_options` man page.

To run automatic routing, use the `route_auto` command. By default, the `route_auto` command ignores existing global routes and sequentially invokes global routing, track assignment, and detail routing. The tool does not save the block between routing steps. Use the following options to change the default behavior:

- To perform incremental global routing, use the `-reuse_existing_global_route true` option.
- To stop after track assignment, use the `-stop_after_track_assignment true` option.
- To change the number of search and repair iterations during detail routing from the default of 40, use the `-max_detail_route_iterations` option.
- To save the block after each routing step, set the `-save_after_global_route`, `-save_after_track_assignment`, and `-save_after_detail_route` options to `true`.

  The tool uses *auto* as the default prefix for the saved block names. To specify the prefix, use the `-save_cell_prefix` option.

By default, the `route_auto` command is not timing-driven or crosstalk-driven and does not fix soft DRC violations, such as bridge rule violations.

- To enable timing-driven mode, set the `route.global.timing_driven`, `route.track.timing_driven`, and `route.detail.timing_driven` application options to `true`.

  By default, when you enable timing-driving routing, Zroute uses medium effort to assign timing-critical nets to low-resistance metal layers. To change the extent to which timing-driven routing prefers lower-resistance metal layers when routing timing-critical nets, set the `route.common.rc_driven_setup_effort_level` application option. To increase the effort level to use more low-resistance metal layers for routing, set the option to `high`. To reduce the effort level, set the option to `low`. To disable resistance-based routing layer preferences, set the option to `off`.

Feedback

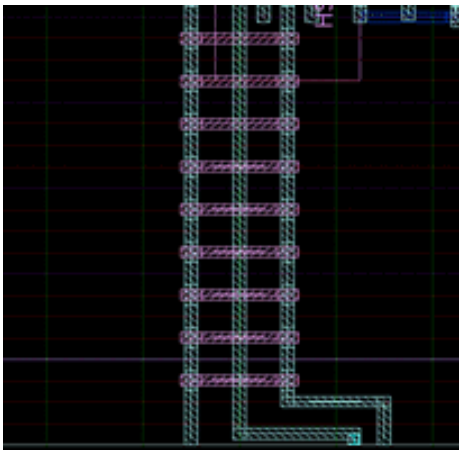
- To enable crosstalk-driven mode, set the `route.global.crosstalk_driven`, `route.track.crosstalk_driven`, and `time.si_enable_analysis` application options to `true`.
- To enable the fixing of soft DRC violations after the final detail routing iteration, set the `route.common.post_detail_route_fix_soft_violations` application option to `true`.

```
icc2_shell> set_app_options \
   -name route.common.post_detail_route_fix_soft_violations \
   -value true
```

Shielding Nets

Zroute shields routed nets by generating shielding wires that are based on the shielding widths and spacing defined in the shielding rules. In addition to shielding nets on the same layer, you also have the option to shield one layer above and one layer below. Shielding above or below the layer is called *coaxial shielding*. Figure 78 shows an example of coaxial shielding. Coaxial shielding provides even better signal isolation than same-layer shielding, but it uses more routing resources.

Figure 78 Coaxial Shielding

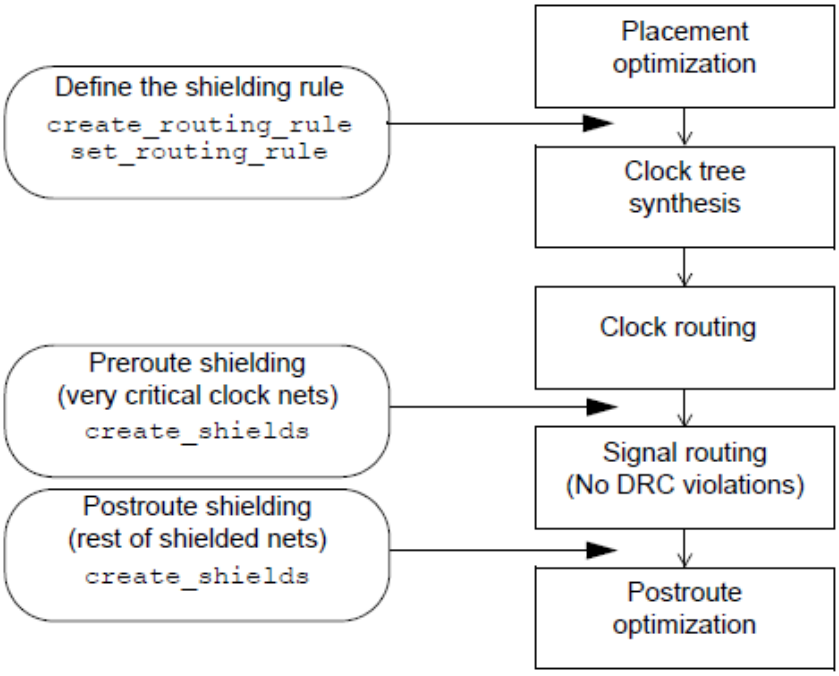


You can perform shielding either before or after signal routing. Shielding before signal routing, which is referred to as preroute shielding, provides better shielding coverage but can result in congestion issues during signal routing. Preroute shielding is typically used to shield critical clock nets. Shielding after signal routing, which is referred to as postroute shielding, has a very minimal impact on routability, but provides less protection to the shielded nets.

Feedback

Figure 79 shows the Zroute shielding flow, which is described in the topics that follow.

Figure 79 Zroute Shielding Flow



Defining the Shielding Rules

Before you perform shielding, you must

1. Define the shielding rules.

   To define shielding rules, use the `-shield_spacings` and `-shield_widths` options of the `create_routing_rule` command. For example, to specify a shielding rule that uses spacing of 0.1 microns and width of 0.1 microns for metal1 through metal5 and spacing of 0.3 microns and width of 0.3 microns for metal6, use the following command:

   ```
   icc2_shell> create_routing_rule shield_rule \
      -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
      -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
   ```

   For more information about the `create_routing_rule` command, see Using Nondefault Routing Rules.

Feedback

2. Assign shielding rules to the nets to be shielded.

   To avoid congestion issues and achieve the best balance of DRC convergence and timing closure, you should apply shielding rules only to high-frequency or critical clock nets and apply double-spacing rules to the lower-frequency clock nets.

   - To assign shielding rules to clock nets, use the `set_clock_routing_rules` command.

     Note:

     You can use the `set_clock_routing_rules` command only before clock tree synthesis.

   - To assign shielding rules to signal nets, use the `set_routing_rule` command.

   For more information about these commands, see Assigning Nondefault Routing Rules to Nets.

Performing Preroute Shielding

To provide the most protection for critical clock nets, perform shielding on those nets after clock tree routing but before signal net routing.

To add shielding to the routed clock nets based on the assigned shielding rules, use the `create_shields` command.

By default, the `create_shields` command

- Performs shielding on all nets that have predefined shielding rules, except those marked as frozen

  To explicitly specify the nets on which to perform shielding, use the `-nets` option.

- Does not perform shielding on wires that are less than four pitches long

  To specify the minimum wire length in microns on which to perform shielding for each layer, set the `route.common.min_shield_length_by_layer_name` application option. Use the following syntax to set this option:

  ```
  { {layer wire_length} ... }
  ```

  Specify the routing layers by using the layer names from the technology file. If you do not specify a value for a layer, Zroute uses the default minimum length of four pitch lengths for that layer.

  Note:

  Wires that are less than the minimum length are not considered when computing the shielding ratio.

Feedback

- Ties the shielding wires to the ground net

  If the block contains multiple ground nets or you want to tie the shielding wires to the power net, use the `-with_ground` option to specify the power or ground net to which to tie the shielding wires.

  If the shielding wires can be tied to multiple power or ground nets, specify the nets by setting the `route.common.shielding_nets` application option before running the `create_shields` command.

  Note:

  If you specify both the `create_shields -with_ground` option and the `route.common.shielding_nets` option, the tool issues a warning message and uses the `route.common.shielding_nets` setting.

- Performs same-layer shielding

  To perform coaxial shielding, use the `-coaxial_above` and `-coaxial_below` options. By default, the `create_shields` command leaves one routing track open between each used track. To change the default behavior, use one of the following methods:

  - Specify the number of open tracks between coaxial shielding segments by using the following options:

    - `-coaxial_above_skip_tracks`

      This option specifies the number of open tracks between used tracks for coaxial shielding above the shielded net segment layer (`-coaxial_above true`).

    - `-coaxial_below_skip_tracks`

      This option specifies the number of open tracks between used tracks for coaxial shielding below the shielded net segment layer (`-coaxial_below true`).

    For either of these options, you can specify an integer between zero and seven.

  - Specify the number of open tracks between coaxial shielding segments on a per-layer basis by using the `-coaxial_skip_tracks_on_layers` option

    To disable shielding on a specific layer, set the value to -1; otherwise, specify an integer between 0 and 7 to specify the number of tracks to skip.

  - Specify the spacing between coaxial shielding segments by using the following options:

    - `-coaxial_above_user_spacing`

      This option specifies the spacing in microns between shielding segments for coaxial shielding above the shielded net segment layer (`-coaxial_above true`).

Feedback

- `-coaxial_below_user_spacing`

  This option specifies the spacing in microns between shielding segments for coaxial shielding below the shielded net segment layer (`-coaxial_below true`).

You cannot use this method when performing incremental shielding.

Note:

You cannot mix these methods of specifying coaxial shielding.

If the generated coaxial shielding wires violate minimum area or minimum length rules, Zroute automatically patches the wires to satisfy these design rules.

- Connects the shielding wires to the standard-cell power or ground pins and the standard-cell rails

  To prevent connections to the standard-cell power and ground pins, set the `-ignore_shielding_net_pins` option to `true`. To prevent connections to the standard-cell rails, set the `-ignore_shielding_net_rails` option to `true`.

- Creates the shielding wires such that they surround the shielded routing shape

  To trim the shielding wires so that they align with the shielded routing shape ends, set the `-align_to_shape_end` option to `true`. To force Zroute to create shielding wires only in the preferred direction, set the `-preferred_direction_only` option to `true`. Note that the `-preferred_direction_only` option does not honor route guides to change the preferred routing direction. When you set either of these options to `true`, extra effort is required to connect the shielding wires to the power and ground network and any shielding wires that are not connected to the power and ground network are deleted.

For example, to perform coaxial shielding below the shielded net segment layer on the clock nets, prevent signal routing below the shielded net segment layer, and tie the shielding wires to VSS, use the following command:

```
icc2_shell> create_shields -nets $clock_nets \
   -coaxial_below true -coaxial_below_skip_tracks 0 \
   -with_ground VSS
```

When you run the `create_shields` command, it reports both the net-based and length-based average shielding ratios, as shown in the following example:

```
Shielded 82% side-wall of (reset)
Shielded 96% side-wall of (clk)
Shielded 2 nets with average ratio as follows.
    1) 89.00%          (total shield ratio/number of shielded nets)
    2) 87.92%          (total shield length/total shielded net length)
```

Feedback

Note:

In some cases there is a slight difference in the shielding ratios reported by the `create_shields` and `report_shields` commands. This is due to graph connectivity differences between the two commands. When the reported values differ, use the values reported by the `report_shields` command. The difference in reported values is typically less than three percent, but can be up to five percent.

If you use the `-preferred_direction_only true` option when running the `create_shields` command, but Zroute must use some nonpreferred direction wires for shielding, the shielding ratio report specifies the percentage of nonpreferred direction wires.

By default, the power and ground structure is not included in the shielding ratio calculation. To include the power and ground structure within a threshold distance in the shielding ratio calculation, set the `route.common.pg_shield_distance_threshold` application option, which specifies the distance threshold in microns.

```
icc2_shell> set_app_options \
   -name route.common.pg_shield_distance_threshold -value distance
```

Note that this option affects only the shielding ratio calculation and does not change the routing behavior.

Soft Shielding Rules During Signal Routing

Zroute considers shielding rules as soft rules during signal routing. If a net has a shielding rule and is not shielded before signal routing, by default, Zroute reserves shielding space during the whole routing process: global routing, track assignment, and detail routing. At the end of detail routing, it reports the shielding space violations and the locations where shielding wires cannot be established. The following log file example shows shielding soft spacing violations, which are highlighted in bold text:

```
DRC-SUMMARY:
          @@@@@@@ TOTAL VIOLATIONS = 13
          Diff net var rule spacing : 2
          Same net spacing : 2
          Less than minimum area : 4
          Short : 1
          Soft spacing (shielding) : 2
```

Signal routing only reserves space for the shielding; it does not actually insert it. You must run `create_shields` after signal routing to physically place the shielding wires. The reported soft rule violations help you to understand the shielding rate. Note that the router reserves space only for same-layer shielding and not for coaxial shielding; therefore, postroute coaxial shielding can produce a very low shielding rate.

Feedback

If you want to use power and ground nets for shielding, and do not want Zroute to reserve space for the shielding when power and grounds nets are available, set the `route.common.allow_pg_as_shield` application option to `true` before running signal routing.

Performing Postroute Shielding

To perform postroute shielding, you use the same command, `create_shields`, that is used for preroute shielding.

If you want to use the default spacings and widths from the technology file for postroute shielding, you do not need to define and assign nondefault routing rules. If you specify the nets to be shielded by using the `-nets` option, the `create_shields` command shields these nets with the default spacing and widths.

Postroute shielding should introduce few to no DRC violations. If DRC violations are created during shielding, the `create_shields` command triggers five detail routing iterations to fix them. If this does not fix the DRC violations, you can fix the remaining violations by running incremental detail routing with the `route_detail -incremental true` command. It is possible that postroute shielding might break some tie-off connections during the shield trimming process. In this case, use the `route_eco` command instead of the `route_detail` command to rebuild the tie-off connections and to fix the DRC violations.

Shielding Example

Example 20 provides an example of shielding clock nets using preroute shielding and shielding critical nets using postroute shielding.

Example 20 Shielding Flow Example

```
# Define shielding rule
create_routing_rule shield_rule \
  -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
  -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}

# Assign shielding rule to clock nets
set_clock_routing_rules -clocks CLK \
  -rules shield_rule

# Perform clock tree synthesis
synthesize_clock_trees

# Route the clock nets, reusing the global routing result
route_group -all_clock_nets -reuse_existing_global_route true

# Perform preroute shielding for the clock nets
create_shields -nets $clock_nets -with_ground VSS
```

Feedback

```
# Assign shielding rule to critical nets
set_routing_rule -rule shield_rule $critical_nets

# Route signal nets using shielding soft rules
route_auto

# Perform postroute shielding for critical nets
create_shields -nets $critical_nets -with_ground VSS
```

Performing Incremental Shielding

By default, Zroute does not perform incremental shielding on nets that are modified after they were shielded. However, you can enable this capability for nets that were initially shielded with the `create_shields` command by changing the `route.common.reshield_modified_nets` application option from its default of `off`.

When you enable incremental shielding, Zroute performs incremental shielding during detail routing by removing the existing shielding from the modified nets and optionally reshielding these nets based on the new topology.

- To remove the existing shielding only, set the `route.common.reshield_modified_nets` option to `unshield`.
- To remove the existing shielding and reshield the modified nets, set the `route.common.reshield_modified_nets` option to `reshield`.

Note:

Zroute automatically detects the nets modified within the IC Compiler II tool; however, nets modified externally and input by reading a DEF file are not supported by incremental shielding.

Reporting Shielding Information

After you run the `create_shields` command, you can

- Query the shield shapes associated with a shielded net by using the `get_shapes -shield_only` command, as described in Querying Shield Shapes
- Report the shielding statistics by using the `report_shields` command, as described in Reporting Shielding Statistics

Querying Shield Shapes

To query the shield shapes associated with a shielded net, use the `get_shapes` command with the `-shield_only` option, as shown in the following example:

```
icc2_shell> get_shapes -shield_only -of_objects [get_nets myclk]
{PATH_19_4317 PATH_19_4318 PATH_19_4319 PATH_19_4320 PATH_19_4322
```

Feedback

```
PATH_17_11590 PATH_17_11591 PATH_17_11592 PATH_17_11593 PATH_17_11594
PATH_17_11595 PATH_17_11596 PATH_17_11597 PATH_17_11599 PATH_17_11600
PATH_17_11601 PATH_17_11602 PATH_17_11604 PATH_17_11605 PATH_17_11606
PATH_17_11607 PATH_17_11608 PATH_17_11609 PATH_17_11610 PATH_17_11611
PATH_17_11612 PATH_17_11613 PATH_17_11614 PATH_17_11615 PATH_17_11616
PATH_17_11617 PATH_17_11618 PATH_17_11619 PATH_17_11620 PATH_17_11621
PATH_17_11622 PATH_17_11623 PATH_17_11624 PATH_17_11625 PATH_17_11626}
```

Reporting Shielding Statistics

To report the shielding statistics, use the `report_shields` command.

Note:

In some cases there is a slight difference in the shielding ratios reported by the `create_shields` and `report_shields` commands. This is due to graph connectivity differences between the two commands. When the reported values differ, use the values reported by the `report_shields` command. The difference in reported values is typically less than three percent, but can be up to five percent.

By default, the power and ground structure is not included in the shielding ratio calculation. To include the power and ground structure within a threshold distance in the shielding ratio calculation, set the `route.common.pg_shield_distance_threshold` application option, which specifies the distance threshold in microns.

```
icc2_shell> set_app_options \
   -name route.common.pg_shield_distance_threshold -value distance
```

The default report generated by the `report_shields` command provides overall statistics, as shown in Reporting Shielding Statistics.

Example 21 Default Shielding Report

```
icc2_shell> report_shields
...
Shielded 82% side-wall of (reset)
Shielded 96% side-wall of (clk)
Shielded 2 nets with average ratio as follows.
    1) 89.00%          (total shield ratio/number of shielded nets)
    2) 87.92%          (total shield length/total shielded net length)
```

You can output the statistics for each layer by using the `-per_layer true` option with the `report_shields` command, as shown in Example 22.

Example 22 Layer-Based Shielding Report

```
icc2_shell> report_shields -per_layer true
...
Shielded 82% side-wall of (reset)
 Layer: M1  ratio: 0%
 Layer: M2  ratio: 40%
```

Feedback

```
 Layer: M3  ratio: 85%
 Layer: M4  ratio: 80%
Shielded 96% side-wall of (clk)
 Layer: M3  ratio: 0%
 Layer: M4  ratio: 96%
 Layer: M5  ratio: 100%
Shielded 2 nets with average ratio as follows.
    1) 89.00%          (total shield ratio/number of shielded nets)
    2) 87.92%          (total shield length/total shielded net length)
```

Performing Shielding Checks

During routing, Zroute can detect possible issues with shielding by checking for the following conditions:

- Signal net shapes with a `shape_use` attribute of `shield_route`.
- PG net shapes, which might be a PG strap or rail, but have a `shape_use` attribute of `detail_route`.
- Signal, clock, or PG nets that have a shielding nondefault rule but no associated shield shapes, which might be caused by inappropriate `shape_use` attributes.

To enable these checks, set the `route.common.check_shield` application option to `true`.

Performing Postroute Optimization

The IC Compiler II tool can perform two types of postroute optimization:

- Logic optimization

  This optimization improves the timing, area, and power QoR and fixes logical DRC violations and performs legalization and ECO routing. To perform these optimizations, use the `route_opt` command, as described in Performing Postroute Logic Optimization.

- Routability optimization

  This optimization increases the spacing between cells to fix routing DRC violations caused by pin access issues. To perform this optimization, use the `optimize_routability` command, as described in Fixing DRC Violations Caused by Pin Access Issues.

If you run both logic optimization and routability optimization, you should first perform the logic optimization and then the routability optimization.

Feedback

Performing Postroute Logic Optimization

Before performing postroute optimization, ensure that the block is fully routed and legalized and does not have excessive logical or routing DRC violations.

- To verify the legality of the design, use the `check_legality` command.
- To verify the routing, use the `check_routes` command.

To perform postroute logic optimization,

1. Update the clock latency by using the `compute_clock_latency` command.
2. Run the `route_opt` command two times.

   The legalization and ECO routing performed by the first `route_opt` run might impact the block timing, which is then optimized by the second `route_opt` run. As the number of changes during postroute logic optimization decreases, the timing improvement from subsequent `route_opt` runs decreases.

The `route_opt` command performs the following tasks:

1. Performs extraction and updates the timing

   By default, the `route_opt` command uses the PrimeTime delay calculation engine, which requires that you use cell libraries generated by O-2018.06 or later versions of the Library Manager tool. Before using PrimeTime delay calculation, use the `check_consistency_settings` command to verify that the IC Compiler II and PrimeTime settings are consistent, as described in Checking for Consistency in Timing Analysis and Extraction Settings in the *IC Compiler II Timing Analysis User Guide*.

   The `route_opt` command supports both graph-based analysis (GBA), the default, and path-based analysis (PBA). To enable path-based optimization, use the `time.pba_optimization_mode` application option.

   - To use path-based timing for the worst path of each endpoint, which is identified by using graph-based analysis, set this application option to `path`.

     This is the recommended setting because it provides the best tradeoff between runtime and QoR.

   - To use exhaustive path-based search algorithms for each endpoint, set this application option to `exhaustive`.

     Using this setting increases the runtime.

Feedback

2. Performs the enabled optimizations

   By default, the `route_opt` command optimizes for setup, hold, area, and logical DRC violations for the data paths in the block. To enable additional optimizations, set the application options shown in Table 27 before running the `route_opt` command:

Table 27 Application Options to Enable route_opt Optimizations

Optimization	Application option settings
Concurrent clock and data optimization[6] and timing-driven clock logical DRC fixing	`route_opt.flow.enable_ccd` = `true`
Clock logical DRC fixing[7]	`route_opt.flow.enable_cto` = `true`
Clock tree area recovery	`route_opt.flow.enable_clock_power_recovery` = `area`
Clock tree power recovery	`route_opt.flow.enable_clock_power_recovery` = `power`
Power optimization[8]	`route_opt.flow.enable_power` = `true`
Path-based optimization with machine learning	`route_opt.flow.enable_ml_opto` = `true` `time.pba_optimization_mode` = `path`
IR-driven optimization with machine learning	For IRDOPT, set the following application options:`set_app_options -name route_opt.flow.enable_irdrivenopt -value true` `set_app_options -name opt.common.ir_drop_threshold -value 10`For IRDCCD, set: `set_app_options-name opt.common.power_integrity-value true` To turn on the machine learning flow, set the following application option: `set_app_options -name ccd.ccd_opt_voltage_drop_enable_ml_predictor -value true`

Note:

The `route_opt` command honors only the `route_opt` application options; it does not honor the settings of the `opt` application options, except the `opt.common.allow_physical_feedthrough` application option.

3. Legalizes the block
4. Performs ECO routing

6. *For information about setting application options to control concurrent clock and data optimization, see Controlling Concurrent Clock and Data Optimization.*
7. *This optimization is supported only when concurrent clock and data optimization is not enabled.*
8. *The type of power optimization performed depends on scenario configuration. Total power optimization is performed if there is an active scenario with both leakage power and dynamic power enabled. Leakage power optimization is performed if there is an active scenario with only leakage power enabled.*

Feedback

By default, the `route_opt` command performs five detail routing iterations during the ECO routing phase. To change the number of iterations, use the `route.detail.eco_max_number_of_iterations` application option. To perform track assignment instead, set the `route_opt.eco_route.mode` application option to `track`.

If the PrimeTime tool reports setup or hold violations during signoff timing analysis, you can use the following process to fix these violations:

1. Specify the target endpoints and their PrimeTime timing information by using the `set_route_opt_target_endpoints` command with the `-setup_timing` and `-hold_timing` options.
2. Optimize the endpoints by using the `route_opt` command.
3. Remove the adjusted timing information by using the `set_route_opt_target_endpoints -reset` command.

You can also use this process to perform incremental optimization on specific endpoints. In this case, use the `-setup_endpoints`, `-hold_endpoints`, and `-ldrc_objects` options to specify the endpoints.

To close the final setup, hold, or logical DRC violations with minimal disturbance to the block, use size-only mode when you run the `route_opt` command. To enable size-only mode, set the `route_opt.flow.size_only_mode` application option to one of the following values:

- `true_footprint`, which allows resizing a cell only to a library cell that is an exact physical match, as determined by the tool after analysis

  This ensures that legalization or ECO routing is not required after resizing
- `footprint`, which allows resizing a cell only to a library cell that has the same `footprint` attribute in the library
- `equal`, which allows resizing a cell only to a library cell that is equal in size
- `equal_or_smaller`, which allows resizing a cell only to a library cell that is equal or smaller in size

To disable size-only mode, set the `route_opt.flow.size_only_mode` application option to `none` or `""`, which is the default.

Performing Postroute Optimization Using the hyper_route_opt Command

The recommended flow for performing postroute logic optimization consists of multiple iterations of the `route_opt` command with different settings. However, you can reduce

Feedback

the number of postroute optimization iterations by using a single iteration of the `hyper_route_opt` command.

The postroute optimization flow using the `hyper_route_opt` command consists of the following steps:

1. Enable different settings for postroute optimization using the same settings as for the `route_opt` command.

   For example, you can

   - Specify the path-based analysis (PBA) mode by using the `time.pba_optimization_mode` application option
   - Enable concurrent clock and data optimization by using the `route_opt.flow.enable_ccd` application option
   - Enable power optimization by using the `route_opt.flow.enable_power` application option

2. (Optional) Specify settings for metal fill insertion, PG augmentation, and so on that should to be performed during the different stages of optimization and ECO routing within the `hyper_route_opt` command by using the `snps_hyper_route_opt_post_eco` Tcl-callback procedure.
3. (Optional) To improve the timing QoR, the `hyper_route_opt` command can debank sequential multibit registers. To enable this feature, set the `route_opt.flow.enable_multibit_debanking` application option to `true`. By default, the `route_opt.flow.enable_multibit_debanking` application option is set to `false`.
4. Run postroute optimization using the `hyper_route_opt` command.
5. (Optional) Specify endpoints to target by using the `set_route_opt_target_endpoints` command and optimize these endpoints by using the `route_opt` command if there are any violating endpoints remaining after the `hyper_route_opt` command.
6. (Optional) Fix any remaining route DRC violations by using the `route_detail -incremental` command.

Fixing DRC Violations Caused by Pin Access Issues

In advanced process nodes, the distance between pins decreases, which can result in DRC violations caused by pin access issues. These types of DRC violations can be fixed by using keepout margins to increase the spacing between cells.

Feedback

If your postroute design has DRC violations, analyze the violations to determine if they are caused by pin access issues. If so, run the `optimize_routability` command to increase the spacing between cells where the DRC violations occur.

The `optimize_routability` command performs the following tasks:

- Analyzes the cells with DRC violations to find violations where cells abut

  By default, the command considers all DRC violations. To consider only specific DRC violations, use the `-drc_rules` option. To consider only violations on specific layers, use the `-layer_rules` option.

  To preview the number of DRC violations and affected cells, use the `-check_drc_rules` option. When you use this option, the command generates a report but does not move any cells.

- Sets keepout margins on these cells on the side of the cell where the error occurs

  By default, the command uses the site width as the keepout width. To specify a keepout width in microns, use the `-keepout_width` option.

  Note:

  These keepout margins are in addition to any existing keepout margins on the cells; the command does not modify the existing keepout margins.

  To try to fix the DRC violations by flipping the cells instead of adding keepout margins, use the `-flip` option.

- Legalizes the affected cells

You must use one of the following methods to complete the routes for the cells moved by the optimization:

- Run ECO routing by using the `route_eco` command.
- Run ECO routing by using the `-route` option with the `optimize_routability` command.

  To fix the DRC in the areas that have changes, set the `route.common.eco_fix_drc_in_changed_area_only` application option to `on`.

  When you use this option, the command runs the `route_eco` and `check_routes` commands after completing the optimization.

After moving the cells and performing ECO routing, remove the keepout margins from the cells by using the `optimize_routability -remove_keepouts` command.

Feedback

Analyzing and Fixing Signal Electromigration Violations

Signal electromigration problems result from an increase in current density caused by the use of smaller line widths and higher operational speeds in IC designs. Electromigration can lead to shorts or opens due to metal ion displacement caused by the flow of electrons. The more frequently a net switches, the more susceptible it is to electromigration.

To analyze and fix signal electromigration violations in a detail routed block,

1. Apply the signal electromigration constraints by using the `read_signal_em_constraints` command.

   For more information, see Loading the Signal Electromigration Constraints.

2. Apply switching activity by either reading in a SAIF file with the `read_saif` command or annotating the switching activity information on the nets with the `set_switching_activity` command.

   For more information, see Annotating the Switching Activity.

3. Report the signal electromigration information by using the `report_signal_em` command.

   For more information, see Analyzing Signal Electromigration.

4. Fix any signal electromigration violations by using the `fix_signal_em` command.

   For more information, see Fixing Signal Electromigration Violations.

The following example script shows the signal electromigration flow.

```
# Open the block and apply the signal electromigration constraints
open_block block1_routed
read_signal_em_constraints em.itf

# Load switching information
read_saif block1.saif

# Perform signal electromigration analysis
report_signal_em -violated -verbose > block1.signal_em.rpt

# Fix signal electromigration violations
fix_signal_em
```

Feedback

Loading the Signal Electromigration Constraints

The IC Compiler II tool supports the following formats for specifying electromigration constraints:

- Interconnect Technology File (ITF)

  An ITF file can contain the following types of electromigration constraints: delta temperature, duration and duty ratio, via size, via direction, metal length, and via-to-via spacing relaxation.

  The IC Compiler II tool supports both unencrypted and encrypted ITF files.

- Advanced Library Format (ALF)

  An ALF file can contain the following types of electromigration constraints: temperature, metal width, and contact area.

The electromigration constaints are stored in the design library as a lookup table. The tool determines the limit values by looking up the values in the lookup table and then using linear interpolation or extrapolation.

To load the signal electromigration constaints into the design library, use the `read_signal_em_constraints` command. By default, the command reads an unencrypted ITF file. To read an encrypted ITF file, use the `-encrypted` option. To read an ALF file, use the `-format ALF` option. The constraints loaded by the `read_signal_em_constraints` command overwrite any existing signal electromigration constraints in the design library.

For example, to read an unencrypted ITF file, use the following command:

```
icc2_shell> read_signal_em_constraints em.itf
```

For example, to read an encrypted ITF file, use the following command:

```
icc2_shell> read_signal_em_constraints -encrypted em.itf.enc
```

To read an ALF file, use the following command:

```
icc2_shell> read_signal_em_constraints -format ALF em.alf
```

Analyzing Signal Electromigration

The `report_signal_em` command performs signal electromigration analysis for each net by calculating the current on every edge and comparing this data with the constraints set by the `read_signal_em_constraints` command. For example,

```
icc2_shell> report_signal_em -violated -verbose
```

Feedback

The `report_signal_em` command performs a timing update, if needed, and analyzes all nets for electromigration. The `-verbose` option causes the report to show detailed electromigration analysis information, which usually produces a very large report when used by itself. To get a report with a reasonable size, use the `-violated` option as well, which limits the report to only the nets with electromigration violations. To limit the analysis to specific nets, use the `-nets` option.

You can specify settings for electromigration analysis by setting the application options shown in the following table.

Table 28 Application Options for Signal Electromigration Analysis

Application option	Default	Description
`em.net_delta_temperature`	5.0	Specifies the delta temperature used for RMS limit lookup.
`em.net_duration_condition_for_peak`	0.0	Specifies the duration value for peak current checking.
`em.net_global_rms_relaxation_factor`	1.0	Specifies the global relaxation factor for the RMS limit specified in the constraints file.
`em.net_metal_line_number`	0	Specifies the metal line number used to look up the relaxation factor in the RMS constraint factor table provided in the constraints file.
`em.net_min_duty_ratio`	0	Specifies the minimum duty ratio for computing the peak current.
`em.net_use_waveform_duration`	`false`	When `false`, the command uses current integral based duration. When `true`, the command uses current waveform based duration.
`em.net_violation_rule_types`	""	Specifies the type of constraint rules to use.

When you perform signal electromigration analysis, the tool computes three current values:

- Average
- Root mean square
- Peak

The default report generated by the `report_signal_em` command lists the number of nets with electromigration violations and the types of constraints violated (mean, absolute

Feedback

average, RMS, or peak). If you use the `-verbose` option, the report displays detailed information about the violations, as shown in Example 23.

Example 23 Verbose Electromigration Report

```
icc2_shell> report_signal_em -violated -verbose
...
Net Name: clks/trimch_macroout_cb_group_gpaf0lts1_gpu_c_0_s_0[3]
Violations: RMS PEAK

Flag Segment         Layer BBox Coordinates                          Width/Cut Required
         AVERAGE              RMS                              PEAK
-------------------------------------------------------------------------------------
v    PATH_36_392230  M6D   (1450.000, 995.612 -> 1452.715, 995.652)   0.040     0.085
         1.514e-02 (-)         9.650e-01 (5.231e-01)          1.365e+00 (1.349e+00)
     VIA_S_11051278  V5D   (1449.969, 995.582 -> 1450.071, 995.682)   1         -
         1.465e-02 (-)         7.380e-01 (-)                  1.044e+00 (-)
v    PATH_35_765189  M5A   (1450.001, 994.973 -> 1450.039, 995.651)   0.038     0.047
         1.417e-02 (-)         6.088e-01 (4.981e-01)          8.609e-01 (1.274e+00)
     VIA_S_11051279  V5D    (1449.969, 994.942 -> 1450.071, 995.042)      1          -
         1.369e-02 (-)         5.249e-01 (-)                  7.423e-01 (-)
     PATH_36_392231  M6D   (1407.370, 994.972 -> 1450.040, 995.012)   0.040     -
         1.311e-02 (-)         2.047e-01 (5.231e-01)          2.894e-01 (9.443e-01)
     VIA_S_11051280  V5D   (1407.339, 994.942 -> 1407.441, 995.042)   1         -
         1.252e-02 (-)         1.926e-01 (-)                  2.723e-01 (-)
...
```

The report shows the segment name (Segment column), layer name (Layer column), location coordinates (Bbox Coordinates column), metal width or via cut number (Width/Cut column), the constraint value (Required column), the average, RMS, and peak current in mA for the wire segment or via at that location. The letter “v” in the Flag column indicates a violation at that location. A hyphen in the Required column means that there is no electromigration constraint for that metal or via layer. In the AVERAGE, RMS, and PEAK columns, the first value is the calculated value and the value in parenthesis is the limit imposed by the electromigration constraints, where a hyphen indicates that there is no electromigration constraint.

Fixing Signal Electromigration Violations

Before fixing signal electromigration violations, make sure that the block is detail routed, the signal electromigration constraints are defined in the design library, the switching activity is defined for all boundary nets, crosstalk analysis is enabled, and there are no timing or DRC violations. You can optionally use the `report_signal_em` command first to report electromigration violations before attempting to fix them.

Feedback

To fix signal electromigration violations, use the `fix_signal_em` command, which performs the following tasks:

- Runs signal electromigration analysis for nets by calculating the current on every edge of a net and comparing it with the constraints set by the `read_signal_em_constraints` command.
- Fixes violations by applying nondefault routing rules to widen the metal shapes and then performing ECO routing.

  The `fix_signal_em` command runs a single fixing iteration; if violations remain after running this command, you must rerun it to address the remaining violations.

By default, the command fixes signal electromigration violations on all nets. To fix the violations only on specific nets, use the `-nets` option.

See Also

- Analyzing Signal Electromigration
- Performing ECO Routing

Performing ECO Routing

Whenever you modify the nets in your block, you need to run engineering change order (ECO) routing to reconnect the routing.

To run ECO routing, use the `route_eco` command. The `route_eco` command sequentially performs global routing, track assignment, and detail routing to reconnect the routing.

By default, the `route_eco` command

- Considers timing and crosstalk during routing, which means that the tool performs extraction and updates the timing before performing the routing.

  The extraction and timing update can be time consuming and might not be necessary for your block. To prevent the extraction and timing update, use the following commands to disable the timing-driven and crosstalk-driven modes:

```
icc2_shell> set_app_options \
   -name route.global.crosstalk_driven -value false
icc2_shell> set_app_options \
   -name route.global.timing_driven -value false
icc2_shell> set_app_options \
   -name route.track.crosstalk_driven -value false
icc2_shell> set_app_options \
   -name route.track.timing_driven -value false
icc2_shell> set_app_options \
   -name route.detail.timing_driven -value false
```

Feedback

- Works on all open nets in the block

  To perform ECO routing only on specific nets, use the `-nets` option to specify the nets.

- Ignores existing global routes

  To honor the existing global routes and perform incremental global routing, set the `-reuse_existing_global_route` option to `true`.

- Relieves congestion by reusing dangling wires instead of rerouting detail routed wires

  To disable the reuse of dangling wires, set the `-utilize_dangling_wires` option to `false`.

- Performs a maximum of 40 detail routing iterations

  To change the maximum number of detail routing iterations, use the `-max_detail_route_iterations` option.

- Fixes hard DRC violations on the entire block by rerouting any wires, whether modified or not

  - To fix DRC violations only in the neighborhood of the open nets, set the `-open_net_driven` option to `true`.

    Note:

    When you use the `-nets` option to perform ECO routing on specific nets, Zroute fixes DRC violations only within the bounding box of the specified nets. In this case, Zroute ignores the setting of the `-open_net_driven` option.

  - To change the scope of rerouting performed by the ECO router, use the `-reroute` option.

    - To limit rerouting to modified wires, set this option to `modified_nets_only`.
    - To first attempt to fix DRC violations by rerouting modified nets and then reroute other nets if necessary, set this option to `modified_nets_first_then_others`.

      A common use for this option is to route the clock nets affected by an ECO in a fully routed block.

  - To enable the fixing of soft DRC violations, such as bridge rule violations, after the final detail routing iteration, set the `route.common.post_eco_route_fix_soft_violations` application option to `true`.

Zroute generates a DRC violations summary at the end of each detail routing iteration. Before reporting the final DRC violations, Zroute merges redundant violations. For more

Feedback

information about the DRC violations reported by Zroute, see Performing Design Rule Checking Using Zroute.

Zroute also reports the nets changed during ECO routing. By default, it reports the first 100 changed nets. You can use the `-max_reported_nets` option to set a different limit on the reported nets. To report all changed nets, set the `-max_reported_nets` option to -1.

See Also

- Routing Nets in the GUI

Routing Nets in the GUI

To route nets interactively in the active layout view, draw the routes with the Create Route tool. To activate the Create Route tool, click the  button on the Edit toolbar or choose Create > Route.

To draw route segments, you click points in the layout view. The tool displays flylines and target port and pin locations to guide you in drawing the route segments. It can also check for routing design rule violations as you draw the route segments. You can set options to adjust the routing, control the tool operation, and enable or disable routing aids.

By default, the Create Route tool

- Ignores routing blockages

  To force the tool to honor these blockages, change the setting in the Mouse Tool Options panel.

  Note:

  The Create Route tool does not honor routing guides defined by the `create_routing_guide` command.

- Uses the metal width and metal spacing requirements defined in the technology file and ignores nondefault routing rules

  To force the tool to honor nondefault routing rules, change the setting in the Mouse Tool Options panel. When you enable this feature, the Create Route tool honors the metal width and metal spacing requirements from the nondefault routing rule and uses these settings to determine the width and pitch of the routes.

  Note:

  The shielding width and shield spacing defined in the nondefault routing rule are not used by the Create Route tool.

Feedback

If you route on grid and know that there no obstacles, you can reduce runtime by skipping the spacing checks during automatic welding and automatic alignment. To force the tool to ignore the spacing requirements during these tasks, change the settings in the Mouse Tool Options panel.

You can reverse and reapply Create Route tool operations by using the GUI undo and redo capabilities.

For detailed information about using the Create Route tool, click  on the Mouse Tool Options panel after you activate the Create Route tool. This opens a Help page in the man page viewer.

Modifying Routed Nets

You can modify routed nets in the GUI by using the following tools:

- Area Push tool

  To activate the Area Push tool, click the  button on the Edit toolbar or choose Edit > Area Push. You can use the Area Push tool to move unfixed objects away from a rectangular area on a layer while maintaining their physical connections. You select the layer and control whether the tool complies with nondefault routing rules. The tool supports both interactive and batch push operations.

- Spread Wire tool

  To activate the Spread Wires tool, click the  button on the Edit toolbar or choose Edit > Spread Wires. You can use the Spread Wires tool to move selected, unfixed wires evenly between two points on a layer while maintaining their physical connections. You control whether the tool spreads the wires by layer and whether the tool complies with nondefault routing rules.

- Stretch Connected tool

  To activate the Stretch Connected tool, click the  button or choose Edit > Stretch Connected. You can use the Stretch Connected tool to move and stretch unfixed wire shapes while optionally maintaining their physical connections.

- Quick Connect tool

  To activate the Quick Connect tool, click the  button or choose Edit > Route Utilities > Quick Connect. You can use the Quick Connect tool to quickly connect wires to pin shapes, port shapes, terminals, or other wires.

Feedback

If you need assistance while using these tools, click  to open a Help page in the man page viewer.

Feedback

Cleaning Up Routed Nets

After routing is complete, you can clean up the routed nets by running the `remove_redundant_shapes` command.

```
icc2_shell> remove_redundant_shapes
```

By default, this command reads the DRC information stored in the design view of the block and then removes dangling and floating net shapes from all nets in the block based on this information. To run the `check_routes` command to get the DRC information instead of using the information stored in the design view, use the `-initial_drc_from_input false` option.

You can restrict the removal to

- Specific nets by using the `-nets` option
- Specific layers by using the `-layers` option
- Fixed or unfixed route types by using the `-route_types` option

When removing dangling net shapes, the tool does not change topologies or connections and does not touch terminals. In addition, no changes are made to open nets or nets with DRC violations.

You can disable the removal of dangling net shapes by using the `-remove_dangling_shapes false` option. You can disable the removal of floating net shapes by using the `-remove_floating_shapes false` option.

In addition to removing dangling and floating net shapes, this command can also remove loops in the specified nets. To remove loops, use the `-remove_loop_shapes true` option.

By default, the `remove_redundant_shapes` does not report the changes it makes. To report the changes, use the `-report_changed_nets true` option.

For example, to remove dangling net shapes, floating net shapes, and loops from the net named my_net, use the following command:

```
icc2_shell> remove_redundant_shapes -nets my_net \
   -remove_loop_shapes true
```

After cleaning up the routed nets, reverify the routing, as described in Performing Design Rule Checking Using Zroute.

Feedback

Analyzing the Routing Results

You can analyze the routing results by reporting on the cell placement and routing statistics. The following topics describe how to perform these tasks:

- Generating a Congestion Report
- Generating a Congestion Map
- Performing Design Rule Checking Using Zroute
- Performing Signoff Design Rule Checking
- Performing Design Rule Checking in an External Tool
- Performing Layout-Versus-Schematic Checking
- Reporting the Routing Results
- Reporting the Wire Length
- Using the DRC Query Commands

Generating a Congestion Report

To generate a congestion report, run the `report_congestion` command.

```
icc2_shell> report_congestion
```

By default, the `report_congestion` command uses the congestion map stored with the block to report an overflow summary for the entire block. If a congestion map is not stored with the design, the command generates a congestion map by running global routing in congestion-map-only mode.

Note:

By default, only the hard congestion data is saved in the design library. To also save the soft congestion data, set the `route.global.export_soft_congestion_maps` application option to `true` before performing global routing.

The command calculates the overflow as the sum of the overflow for each layer, ignoring any underflow. Example 24 shows the default report, which includes only the hard congestion data.

Example 24 Default Global Route Congestion Report

```
****************************************
Report : congestion
Design :
```

Feedback

```
Version:
Date   :
****************************************

Layer      |    overflow     |              # GRCs has
Name       |  total   |  max   | overflow (%)       | max overflow
-------------------------------------------------------------------
Both Dirs  |      39  |     8  |      14   ( 0.26%) |       1
H routing  |       6  |     2  |       4   ( 0.07%) |       2
V routing  |      33  |     8  |      10   ( 0.18%) |       1
```

In the default congestion report,

- “H routing” refers to results for horizontal routes only and “V routing” refers to results for vertical routes only.
- The total overflow value is the total number of wires in the block that do not have a corresponding track available. The max overflow value is the highest number of overutilized wires in a single global routing cell.
- The GRCs overflow value is the total number of overcongested global routing cells in the design. The GRCs max overflow value is the number of global routing cells that have the maximum overflow.

Note:

The overflow and global routing cell numbers reported by the `report_congestion` command might look slightly more optimistic than those reported by the `route_global` command because the tool rounds down the congestion information before saving it with the design.

Use the following options to modify the default behavior:

- `-rerun_global_router`

  Use this option to rerun the global routing even if the block already has a congestion map.

- `-boundary` *`coordinates`*

  Use this option to restrict the reporting to a specific region of the block.

- `-layers` *`layers`*

  Use this option to restrict the reporting to specific layers.

- `-mode global_route_cell_edge_based`

  Use this option to report overflow information for each global routing cell.

- `-include_soft_congestion_maps`

Feedback

Use this option to output soft congestion reports, if they exist. A soft congestion report includes the demand from the soft nondefault spacing rules, as well as tool-generated soft rules. The command outputs a soft congestion report for each spacing weight level of the soft nondefault spacing rules used in the block. Each soft congestion report contains the sum of the hard and soft congestion with a weight level of at least the current soft level.

See Also

- Global Routing
- Defining Minimum Wire Spacing Rules
- Generating a Congestion Map

Generating a Congestion Map

To display the global route congestion map, choose **View > Map > Global Route Congestion** in the GUI. If the design library contains global route congestion information for the block, the tool generates the congestion map based on this information; otherwise, you must click Reload to generate the congestion map. When you click Reload, the tool opens a dialog box that contains the following command:

```
route_global -congestion_map_only true
```

Note:

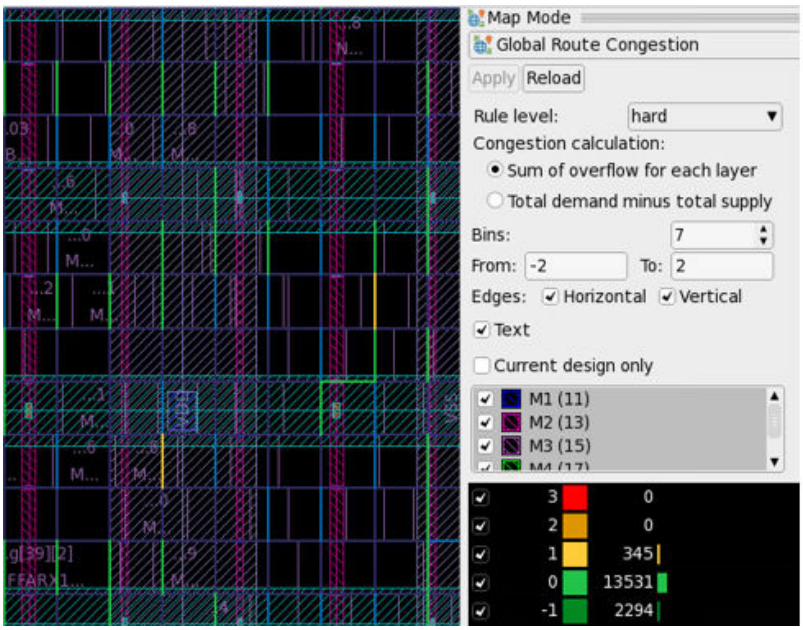
By default, only the hard congestion data is saved in the design library. To also save the soft congestion data, set the `route.global.export_soft_congestion_maps` application option to `true` before performing global routing.

When you click OK in this dialog box, the tool generates a new congestion map. If you want to use different options for the `route_global` command, you can modify this command before clicking OK.

Feedback

Figure 80 shows an example of a congestion map.

Figure 80 Global Route Congestion Map



By default, the congestion map displays only the hard congestion data. To display soft congestion data, select the desired rule level in the Rule level drop-down list. The rule level refers to the spacing weight level of the soft nondefault spacing rules. The congestion map displays the sum of the hard and soft congestion with a weight level of at least the selected rule level.

The congestion map shows the borders between global routing cells highlighted with different colors that represent the congestion values. The congestion map supports two methods for calculating the congestion value:

- Sum of overflow for each layer (the default)

  In this mode, the tool calculates the congestion value as the sum of the overflow for all selected layers. Underflow is not considered; if a layer has underflow, it contributes zero overflow to the total overflow calculation.

- Total demand minus total supply

  In this mode, the tool calculates the congestion value by subtracting the supply for all selected layers from the demand for all selected layers. Note that because this calculation considers the underflow, it produces a more optimistic congestion result in regions that contain both overflow and underflow.

For example, assume that the METAL2 layer is heavily congested with a demand of 13 routing tracks and a supply of only 7 tracks for an overflow of 6. The METAL4 is

Feedback

moderately congested with an overflow of 4, and the METAL6 layer is not congested and contains an underflow of 3. Other layers are not used in the congestion calculation in this example.

- The sum of overflow calculation results in a congestion value of 6+4=10. The underflow of 3 for the METAL4 layer is not used in the calculation.
- The total demand calculation results in a congestion value of 6+4-3=7. In this case, the underflow of 3 for the METAL4 layer is used in the calculation.

To select the mode, select either “Sum of overflow for each layer” or “Total demand minus total supply” in the “Congestion calculation” section of the Map Mode panel.

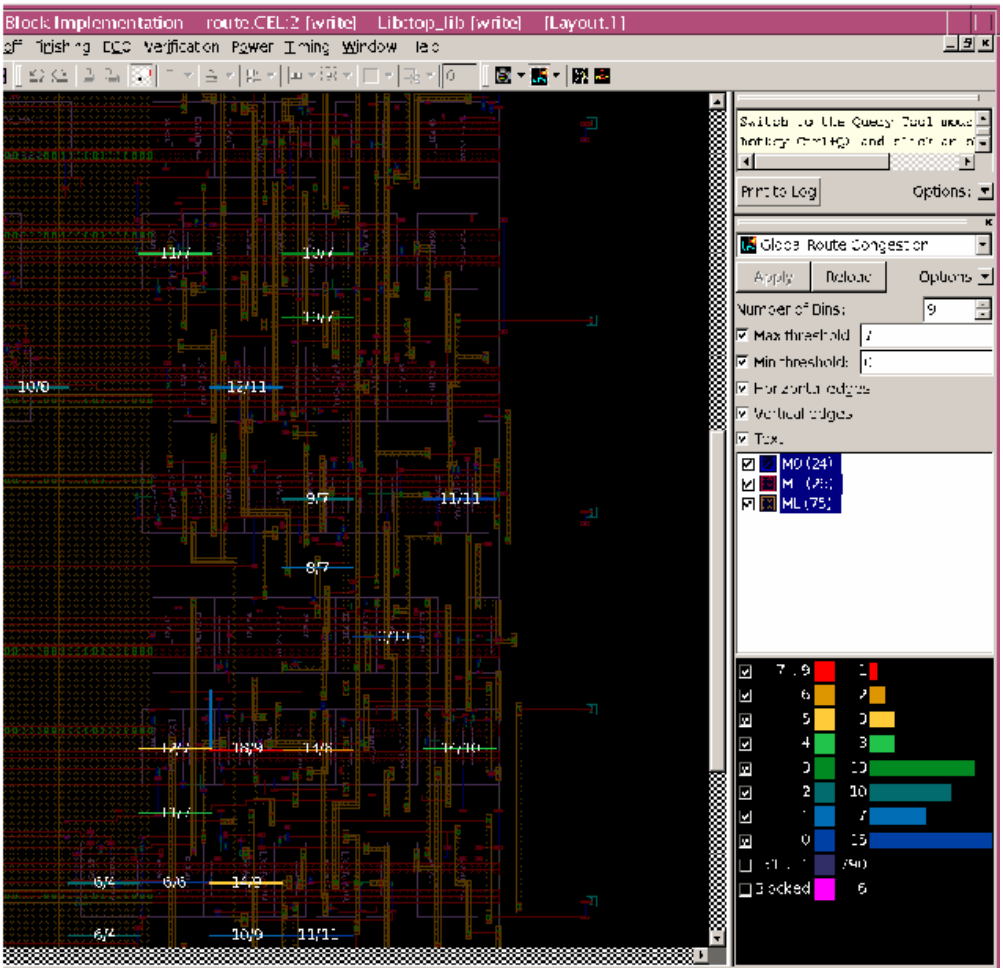
By default, all metal layers are selected in the congestion map, except those specified as ignored layers with the `set_ignored_layers` command. To display the congestion map for a subset of layers, select (or deselect) the layers on the Map Mode panel. For example, if the global routing report shows that the maximum overflow occurs on the METAL2 layer, you can deselect all layers, except for METAL2, to display only the METAL2 congestion.

The Map Mode panel also displays a histogram showing the number of global routing cells in different ranges (bins) of congestion values for the selected layers. If your block contains global routing cells that have no available routing resources, an additional bin named Blocked is displayed that shows the number of global routing cells with no routing resources. You can select which bins to display in the congestion map by selecting or deselecting them on the Map Mode panel.

If the block shows congested areas, zoom into the congested area to see the congestion value on the global routing cell. For example, in Figure 81, the red highlight on the edge of the global routing cell shows 18/9. This means there are 9 wire tracks available, but 18 tracks are needed.

Feedback

Figure 81 Global Route Overflow on Global Routing Cell



Performing Design Rule Checking Using Zroute

To use Zroute to check the routing design rules defined in the technology file, run the `check_routes` command.

By default, the `check_routes` command checks for routing DRC violations, unconnected nets, antenna rule violations, and voltage area violations on all routed signal nets in the block, except those marked as user nets, frozen nets, and PG nets.

- To verify the routing only for specific nets, specify the nets by using the `-nets` option.
- To check user routes, set the `-check_from_user_shapes` option to `true`.
- To check frozen routes, set the `-check_from_frozen_shapes` option to `true`.

  A net is considered frozen when its `physical_status` attribute is set to `locked`.

Feedback

To disable checks for routing DRC violations, set the `-drc` option to `false`. To disable checks for unconnected nets, set the `-open_net` option to `false`. To disable checks for antenna rule violations, set the `-antenna` option to `false`. To disable checks for voltage area violations, set the `-voltage_area` option to `false`.

To save time, you can restrict the routing verification to specific regions of the block by using the `-coordinates` option to specify the lower-left and upper-right coordinates for each rectangular region. When you perform area-based DRC, the `check_routes` command checks only for DRC violations and voltage area violations. It does not check for unconnected nets, antenna violations, or tie-to-rail violations, as these are net-based violations.

Note:

The `-coordinates` option and the `-nets` option are mutually exclusive; you can use only one of these options.

The `check_routes` command reports the following DRC violations:

- Spacing violations
  - Different-net wire spacing
  - Different-net nondefault wire spacing (note that the DRC report refers to nondefault routing rules as variable rules)
  - Different-net via-cut spacing
  - Different-net nondefault via-cut spacing (note that the DRC report refers to nondefault routing rules as variable rules)
  - Different-net fat extension spacing
  - Dog bone spacing
  - End-of-line spacing
  - Enclosed via spacing
  - Same-net spacing
  - Same-net via-cut spacing
  - Same-net fat extension spacing
  - Special notch spacing
  - U-shape spacing
  - Via-cut to metal spacing
  - Soft spacing

Feedback

- Area violations
  - Less than minimum area
  - Less than minimum enclosed area
  - Fat wire via keepout area
  - Jog wire via keepout area
- Length and width violations
  - Less than minimum width
  - Less than minimum length
  - Less than minimum edge length
  - Protrusion length
- Contact violations
  - Needs fat contact
  - Needs poly contact
  - Needs fat contact on extension
  - Over maximum stack level
- Enclosure violations
  - End-of-line wire via enclosure
  - Jog wire via enclosure
  - T-shape wire via enclosure
- Others
  - Open nets, except when doing area-based DRC

    By default, the `check_routes` command reports a maximum of 200 open nets. To report all open nets, use the `-report_all_open_nets true` option (or select “Report all open nets” in the GUI).
  - Antenna violations, except when doing area-based DRC
  - Nets crossing the top-cell boundary
  - Frozen layers
  - Minimum layer

Feedback

- Maximum layer
- Voltage area violations

Note:

These violations are also reported after each detail route iteration.

The `check_routes` command saves the error data to a file named zroute.err. You cannot control the naming of the error data file generated by the `check_routes` command, but you can rename the error data file after running the command. To rename the error data file, use the following commands:

```
icc2_shell> write_drc_error_data -error_data zroute.err \
   -file_name export.err
icc2_shell> open_drc_error_data -file_name export.err
icc2_shell> attach_drc_error_data [get_drc_error_data export.err] \
   -name newname.err
```

After you run the `check_routes` command, you can use the DRC query commands to get more information about the violations or use the error browser to examine the violations in the GUI. For information about analyzing the DRC violations, see Using the DRC Query Commands. For information about using the error browser, see the *IC Compiler II Graphical User Interface User Guide*.

After running `check_routes`, you can use the following command to run incremental detail routing that uses the `check_routes` results as input:

```
icc2_shell> route_detail -incremental true \
   -initial_drc_from_input true
```

Note:

Incremental detail routing does not fix open nets. To fix open nets, you must run ECO routing. For information about ECO routing, see Performing ECO Routing.

Performing Signoff Design Rule Checking

Signoff design rule checking runs the IC Validator tool within the IC Compiler II tool to check the routing design rules defined in the foundry runset. To perform signoff design rule checking, run the `signoff_check_drc` command, as described in Performing Signoff Design Rule Checking. In addition, you can use the `signoff_fix_drc` command to automatically fix the DRC violations detected by the `signoff_check_drc` command. For more information, see Automatically Fixing Signoff DRC Violations.

Note:

An IC Validator license is required to run the `signoff_check_drc` and `signoff_fix_drc` commands.

Feedback

Performing Design Rule Checking in an External Tool

You can perform design rule checking with the Calibre tool, convert the Calibre DRC error file to an IC Compiler II error data file, and then report or view the errors in the IC Compiler II tool. To convert the Calibre DRC error file to an IC Compiler II error data file, use the `read_drc_error_file` command.

For example,

```
icc2_shell> read_drc_error_file -file Calibre_error_file
```

By default, the command generates an error data file named *cell_name*.err, where *cell_name* is derived from the Calibre error report. You can specify a name for the error data file by using the `-error_data` option.

Note:

The `read_drc_error_file` command supports only flat Calibre DRC error files; it does not support Calibre hierarchy DRC error files.

You can load the error data file created by the `read_drc_error_file` command into the error browser to report or display the DRC violations. For information about using the error browser, see the *IC Compiler II Graphical User Interface User Guide*.

Performing Layout-Versus-Schematic Checking

To perform layout-versus-schematic (LVS) checking, which checks for inconsistencies in the physical layout, use the `check_lvs` command.

By default, the `check_lvs` command performs the following checks for all signal, clock, and PG nets:

- Shorted nets

  A shorted net occurs when a net shapes from different nets touch or intersect.

  By default, the command

  - Checks for shorts between net shapes in the top-level design, including shapes in top-level blockages

    To disable checking for shapes in top-level blockages, use the `-check_top_level_blockages false` option.

  - Does not check for shorts between net shapes in the top-level design and net-shapes in child cells

    To enable this checking, use the `-check_child_cells true` option. To exclude certain types of child cells from checking, use the `-exclude_child_cell_types`

Feedback

option to specify one or more of the following cell types: `abstract`, `analog`, `black_box`, `corner`, `cover`, `diode`, `end_cap`, `fill`, `filler`, `flip_chip_driver`, `flip_chip_pad`, `lib_cell`, `macro`, `module`, `pad`, `pad_spacer`, `physical_only`, and `well_tap`.

- Does not check for shorts with zero-spacing blockages

  To enable this checking, use the `-check_zero_spacing_blockages true` option.

Note:

The `check_lvs` command supports only default routing blockages that apply to all net types and have a blockage group ID of 0. If the blockage applies only to specific net types or has a nonzero blockage group ID, the command ignores the blockage. In addition, the command ignores corridor routing blockages (which are created when you use the `-reserve_for_top_level_routing` option with the `create_routing_blockage` command) and boundary routing blockages (which are created when you use the `-boundary_internal` or `-boundary_external` options with the `create_routing_blockage` command).

- Open nets

  An open net occurs when the pins of a net are not connected by its net shapes.

  By default, the command

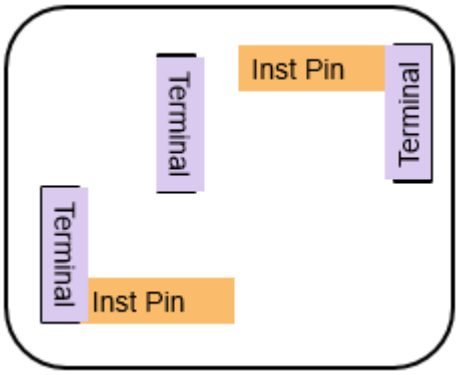
  - Treats power and ground terminals as unconnected voltage sources

    To treat power and ground terminals as connected, use the `-treat_terminal_as_voltage_source true` option.

    For example, assume your block contains the layout shown in Figure 82. By default, the `check_lvs` command reports two opens for this layout. If you use the `-treat_terminal_as_voltage_source true` option, no opens are reported.

Feedback

Figure 82 Layout With Power and Ground Terminals



  - Reports open nets as the bounding box of the open net and does not report floating pins

    To report detailed open locations, use the `-open_reporting detailed` option. Note that using this option might increase the runtime. To report the floating pins, use the `-report_floating_pins true` option.

- Floating net shapes

  A floating net shape occurs when a net shape is not physically connecting to a pin of its net.

To perform a subset of these checks, use the `-checks` option to specify one or more of the following checks: `short` (shorted nets), `open` (open nets), and `floating_routes` (floating net shapes). To check only specific nets, use the `-nets` option to specify the nets of interest.

By default, the `check_lvs` command reports a maximum of 20 violations for each type of error. To change this limit, use the `-max_errors` option. To report all violations, specify the maximum number of violations as zero (`-max_errors 0`). You can view the violations reported by the `check_lvs` command in the GUI by using the error browser. For information about using the error browser, see the *IC Compiler II Graphical User Interface User Guide*.

To reduce the runtime required by the `check_lvs` command, enable multithreading by using the `set_host_options` command, as described in Enabling Multicore Processing.

Feedback

Reporting the Routing Results

To report statistics about the routing results, use the `report_design -routing` command. This command reports the following information:

- Final wiring statistics, including the
  - Number of signal net shapes for each metal layer
  - Signal wire length for each metal layer
  - Number of PG net shapes for each metal layer
  - PG wire length for each metal layer
  - Horizontal and vertical wire distribution for each metal layer
  - Total number of signal net shapes for the block
  - Total signal wire length for the block
- Final via statistics, including the
  - Simple vias used in each via layer, including the number of instances of each via
  - Double via conversion rate for each via layer
  - Double via conversion rate for the block
  - Custom vias used in the block, including the number of instances of each user-defined via

By default, this command reports the information only for the top-level block. To report the information for the entire physical hierarchy, use the `-hierarchical` option.

See Also

- Defining Vias

Reporting the Wire Length

To report the total wire length at any stage of the routing flow, use the consolidated `report_wirelength` command. This command supports all net types, except power ground nets, by default such as signal nets, clock nets. It also supports non-stripe power ground nets by option control. This command generates a report, which includes the following information in the form of two tables, as shown in the figure:

- Total wire length per layer with separate global and detailed routing lengths
- Total wire length by both horizontal and vertical direction

Feedback

```
****************************************
Report : Nets wirelength summary
Design : r4000
Version: S-2021.06-DEV
Date   : Mon Apr 19 17:21:33 2021
****************************************
PER LAYER WIRELENGTH
Layer             Global       Detailed
---------------------------------------
M1                 0.000       1465.440
M2                 0.000      13621.805
M3                 0.000      20729.190
M4                 0.000      13226.870
M5                 0.000       4675.820
M6                 0.000       1161.515
M7                 0.000        108.640
---------------------------------------

TOTAL WIRELENGTH
                 Virtual        Global      Detailed  BestAvailable
-------------------------------------------------------------------
TotalH       26421.001          0.000     27840.660     27840.660
TotalV       26069.774          0.000     27148.620     27148.620
Total        52490.775          0.000     54989.280     54989.280
-------------------------------------------------------------------

Number of nets = 2741
```

In the generated report,

- Virtual value means the estimated routing wire length for each net. For each net, the engine provides a virtual routing length before real routing. It remains even after `route_auto`.
- BestAvailable means the sum of wire length of all nets irrespective of the stage they are in the current design.

This command also supports slanting wires and counts the real length of an angle wire in total and counts the projection length of the x-axis and y-axis.

Using the DRC Query Commands

You can get information about DRC violations by using the `get_drc_errors` command to create a collection of DRC violations and then using the `get_attribute` command to query the attributes of the errors. Some attributes that provide information about DRC errors are `type_name`, `bbox`, `objects`, and `shape`. Note that the availability of attribute values depends on the error type and the verification method used. For a list of all attributes associated with DRC errors, use the `list_attributes -application -class drc_error` command.

Before you run the `get_drc_errors` command, you must load the error data files that you want to query. To determine the error data files attached to the current block, use the `get_drc_error_data` command. You must use the `-all` option to include both the opened and unopened error data files in the result.

```
icc2_shell> get_drc_error_data -all
```

Feedback

To load an error data file, use the open_drc_error_data command.

```
icc2_shell> open_drc_error_data zroute.err
```

To determine the error data types included in the error data file, use the get_drc_error_types command.

```
icc2_shell> get_drc_error_types -error_data zroute.err
```

By default, the get_drc_errors command creates a collection that contains all DRC violations contained in the specified error data file. Use the -filter option to restrict the returned errors.

For example, to return only “Diff net spacing” errors, use the following command:

```
icc2_shell> get_drc_errors -error_data zroute.err \
   -filter {type_name == "Diff net spacing"}
```

To get the nets associated with an error, use the get_attribute command. For example,

```
icc2_shell> get_attribute [get_drc_errors -error_data zroute.err 859] \
   objects
{u0_1/n237}
```

Saving Route Information

To save the route information, use the write_routes command. This command generates a Tcl script that reproduces the metal shapes and vias for a block, including their attribute settings.

Note that the write_routes command reproduces routes, but not routing blockages. To generate a Tcl script that reproduces routing blockages, use the write_floorplan command.

Deriving Mask Colors

At advanced technology nodes, mask colors must be assigned to wires and vias to ensure correct processing of the net shapes during mask synthesis. The route_detail command and other routing commands in the tool usually add the necessary mask colors during routing. For net shapes that do not have a color assignment, use the derive_mask_constraint command to derive the color mask for the specified wires and vias from the nearest underlying tracks. In case of wide wires that occupy multiple tracks, the command takes the mask color opposite to the next unoccupied track.

The following example derives the mask color for the net shapes of the net1 net.

```
icc2_shell> derive_mask_constraint \
   [get_shapes -of_objects [get_nets net1]]
```

Feedback

Use options to the `derive_mask_constraint` command to control how the mask colors are derived.

- Derive cut mask constraints for the specified nets or vias.

```
icc2_shell> derive_mask_constraint -derive_cut_mask \
   [get_vias -within {{600 600} {700 700}}]
```

- Derive mask constraints from overlapping or touching pins or ports.

```
icc2_shell> derive_mask_constraint -follow_pin_mask \
   [get_shapes -of_objects [get_nets net1]]
```

- Derive the mask constraint from the wider overlapping or touching object rather than the wire track.

```
icc2_shell> derive_mask_constraint -follow_wider_object_mask \
   [get_shapes -of_objects [get_nets net1]]
```

See Also

- Multiple-Patterning Concepts

Inserting and Removing Cut Metal Shapes

Cut metal shapes are used in double-patterning designs to reduce the line-end minimum spacing. A cut metal layer and the dimensions of the cut metal shapes inserted on that layer are defined in a `Layer` section in the technology file. The width of the cut metal shapes is defined by the `cutMetalWidth` attribute. The height of the cut metal shapes is defined by either the `cutMetalHeight` or `cutMetalExtension` attribute. A cut metal layer corresponds to the metal layer with the same mask number. For example, cutMetal1 corresponds to metal1. For information about the technology file, see the *Synopsys Technology File and Routing Rules Reference Manual*.

After the routing is finalized, you add cut metal shapes by using the `create_cut_metals` command. This command inserts a cut metal shape between metal shapes when all of the following conditions are met:

- The spacing between the metal shapes in the preferred direction is `cutMetalWidth`.
- There is no existing cut metal shape in the location.
- The metal shapes are on a metal layer that corresponds to a cut metal layer defined in the technology file.

The command inserts cut metal shapes between net shapes, a net shape and a PG shape, a PG shape and an obstruction, and metal shapes within a child cell. After inserting

Feedback

the cut metal shapes, the command propagates the color of the metal shapes to the cut metal shapes.

When you write a DEF file for a block with cut metal shapes, you must use the `-version 5.8` option with the `write_def` command. When you write a GDSII or OASIS file for a block with cut metal shapes, you must use the `-connect_below_cut_metal` option with the `write_gds` or `write_oasis` command.

If the routing changes after inserting cut metal shapes, you must remove the existing cut metal shapes and reinsert them. To remove all cut metal shapes from a block, use the `remove_cut_metals` command.

6

Chip Finishing and Design for Manufacturing

The IC Compiler II tool provides chip finishing and design for manufacturing and yield capabilities that you can apply throughout the various stages of the design flow to address process design issues encountered during chip manufacturing.

For information about the chip finishing and design for manufacturing features, see the following topics:

- Inserting Tap Cells
- Performing Boundary Cell Insertion
- Finding and Fixing Antenna Violations
- Inserting Redundant Vias
- Optimizing Wire Length and Via Count
- Reducing Critical Areas
- Inserting Metal-Insulator-Metal Capacitors
- Inserting Filler Cells
- Inserting Metal Fill

Inserting Tap Cells

A *tap cell* is a special nonlogic cell with a well tie, substrate tie, or both. These cells are typically used when most or all of the standard cells in the library contain no substrate or well taps. Generally, the design rules specify the maximum distance allowed between every transistor in a standard cell and a well or substrate tap.

Before global placement (during the floorplanning stage), you can insert tap cells in the block to form a two-dimensional array structure to ensure that all standard cells placed subsequently comply with the maximum diffusion-to-tap distance limit. After you insert the tap cells, visually check to ensure that all standard-cell placeable areas are properly protected by tap cells.

Feedback

The IC Compiler II tool provides two methods to insert a tap cell array:

- Insert the tap cell array based on the site rows and the maximum distance between tap cells

  To use this method, use the `create_tap_cells` command, as described in Using the create_tap_cells Command.

- Insert the tap cell array using a specified offset and pitch, and optionally insert additional tap cells near the boundary

  To use this method, use the `create_cell_array` command, as described in Using the create_cell_array Command.

Advanced nodes often require the insertion of additional tap cells to manage the substrate and well noise. You can use the following capabilities to insert additional tap cells after standard tap cell insertion:

- Insertion of tap walls

  A *tap wall* is a row or column of tap cells placed linearly without any gaps between cells. You can insert a tap wall outside or inside of the boundary of a block, macro, or hard placement blockage. A tap wall that is outside a boundary is called an *exterior tap wall*. A tap wall that is inside a boundary is called an *interior tap wall*.

  - To insert an exterior tap wall, use the `create_exterior_tap_walls` command, as described in Inserting Exterior Tap Walls.
  - To insert an interior tap wall, use the `create_interior_tap_walls` command, as described in Inserting Interior Tap Walls.

- Insertion of tap meshes

  A *tap mesh* is a two-dimensional array of tap cells with a tap cell in each mesh window. To insert a tap mesh, use the `create_tap_meshes` command, as described in Inserting Tap Meshes.

- Insertion of dense tap arrays

  A *dense tap array* is a tap cell array whose tap distance is smaller than the tap distance used by the `create_tap_cells` command. To insert a dense tap array, use the `create_dense_tap_cells` command, as described in Inserting Dense Tap Arrays.

Using the create_tap_cells Command

To add a tap cell array based on the site rows, use the `create_tap_cells` command. You must specify the name of the library cell to use for tap cell insertion (`-lib_cell` option) and the maximum distance, in microns, between tap cells (`-distance` option).

Feedback

For example,

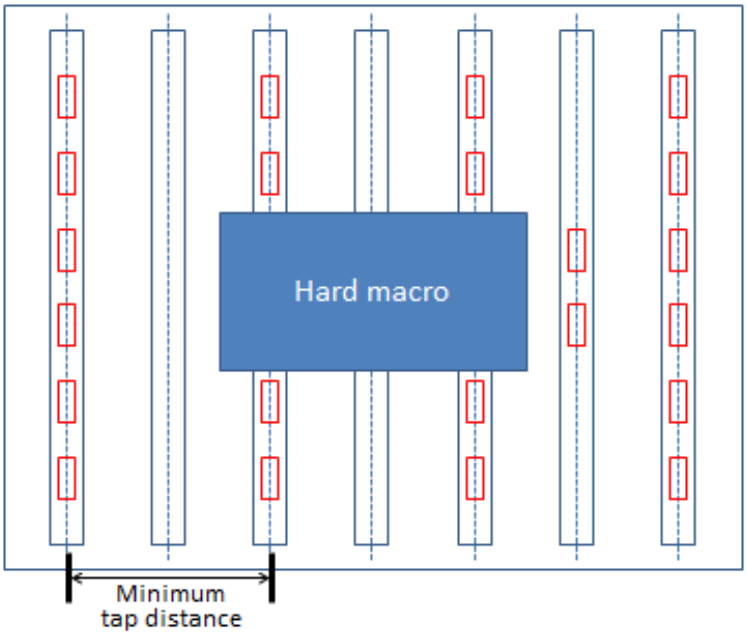
```
icc2_shell> create_tap_cells -lib_cell myreflib/mytapcell -distance 30
```

Note:

If the `place.legalize.enable_pin_color_alignment_check` application option is `true` (its default is `false`), the command ensures that internal cell pins and metal of the tap cells align with a routing track of the appropriate color when inserting the tap cells and shifts the tap cells to avoid violating color alignment. To avoid signoff DRC errors due to this shifting, reduce the tap distance slightly when this application option is `true`.

By default, the `create_tap_cells` command inserts tap cells in every row for the entire block. The tool starts inserting the tap cells at the left edge of the row and uses the specified tap distance to determine the location of the subsequent tap cells. In addition, if a tap cell does not exist within the minimum tap distance (half the specified tap distance) from each row edge adjacent to the block's boundary, a hard macro, or a hard placement blockage, the tool inserts an additional tap cell. Figure 83 shows the default tap cell placement for a block that uses the every-row insertion pattern. Note that extra tap cells are added to the right of the hard macro to ensure that a tap cell exists within the minimum tap distance from the edge of the hard macro.

Figure 83 Default Tap Cell Placement



Feedback

You can modify the following aspects of the default behavior:

- The library cell used for tap cell insertion on rows with a mirrored (MX) orientation

  Use the `-mirrored_row_lib_cell` option to specify the library cell to use for mirrored rows.

- The pattern used to insert the tap cells

  Use the `-pattern` option to specify one of the following tap cell insertion patterns:

  - `every_row` (the default)

    This pattern inserts tap cells in every row. For this pattern, the tap distance specified with the `-distance` option should be approximately twice the maximum diffusion-to-tap value specified in the technology design rules.

  - `every_other_row`

    This pattern inserts tap cells only in the odd-numbered rows. For this pattern, the tap distance specified with the `-distance` option should be approximately twice the maximum diffusion-to-tap value specified in the technology design rules.

  - `stagger`

    This pattern inserts tap cells in every row with the tap cells in even rows offset by half the offset value (`-offset` option) relative to the odd rows, which produces a checkerboard-like pattern. For this pattern, the tap distance specified with the `-distance` option should be approximately four times the maximum diffusion-to-tap value specified in the technology design rules.

- The offset from the left edge of the row

  Use the `-offset` option to shift the pattern startpoint to the right by the specified distance in microns.

- The handling of fixed cells

  By default, tap cells are inserted at the computed tap locations, regardless of whether a fixed cell occupies that location. Use the `-skip_fixed_cells` option to prevent the insertion of tap cells in locations occupied by fixed cells. By default, when you use this option, the command considers the fixed cell as a blockage and breaks the row at the fixed cell, which can result in a tap cell being inserted on each side of the fixed cell. To prevent tap cells from overlapping fixed cells without breaking the row, use the `-preserve_distance_continuity` option with the `-skip_fixed_cells` option. When you use both options, the command shifts the tap cell to avoid overlap with the fixed cell. The `-preserve_distance_continuity` option shifts only those tap cells whose computed tap location is occupied by an instance of one of the specified library cells.

Feedback

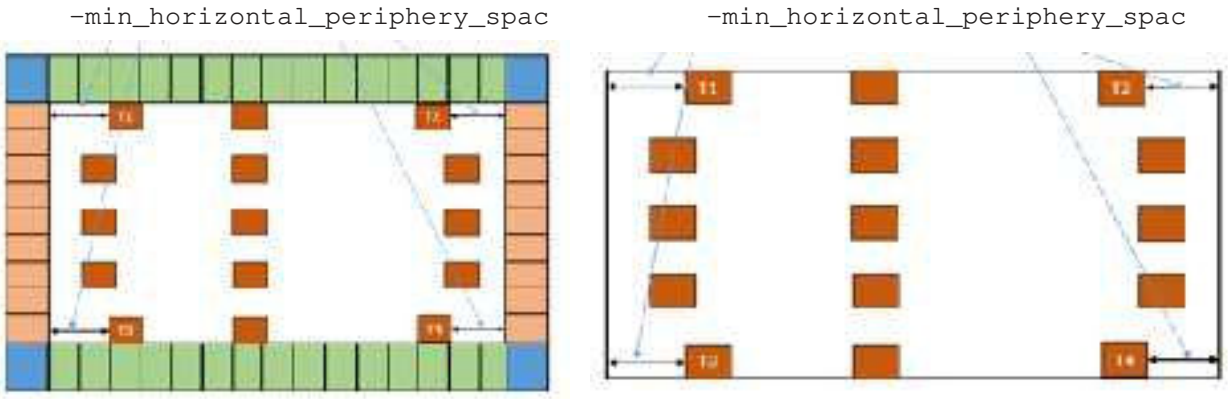
For example, the following command shifts tap cells whose computed tap location is occupied by a fixed instance of the mylib/cell1 library cell:

```
icc2_shell> create_tap_cells -lib_cell mylib/tap -distance 30 \
   -skip_fixed_cells -preserve_distance_continuity mylib/cell1
```

When you use the `-skip_fixed_cells` option, you can also specify the minimum horizontal spacing between the boundary and a corner tap cell that is above or below a blocked area by using the `-min_horizontal_periphery_spacing` option. This option specifies the minimum number of unit tiles between the boundary and an affected corner tap cell.

Figure 84 shows how using this option affects the tap cell placement both with and without boundary cells.

Figure 84 Tap Cell Boundary Row Spacing



- The addition of extra tap cells

  You can use one or both of the following methods to prevent the insertion of extra tap cells:

  - Use the `-row_end_tap_bypass` option to specify that the boundary cells contain taps, so additional tap cells are not inserted at the boundary. Note that the command does not verify that the block has boundary cells or that the boundary cells actually contain taps.
  - Use the `-at_distance_only` option to allow the insertion of tap cells only at the specified tap distance, half of the tap distance, or one fourth of the tap distance (for the stagger pattern only). Note that using this option can cause DRC violations.

- The regions in which to insert tap cells

  Use the `-voltage area` option to restrict the tap cell insertion to the specified voltage areas.

Feedback

- The naming convention used to identify the inserted tap cell instances

  By default, the tool uses the following naming convention for inserted tap cells:

  tapfiller!*library_cell_name*!*number*

  Use the `-separator` option to change the separator character from its default of "!."

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  tapfiller!*prefix*!*library_cell_name*!*number*

Using the create_cell_array Command

To add a tap cell array with specific placement, use the `create_cell_array` command. You must specify the name of the library cell to use for tap cell insertion (`-lib_cell` option), and the x- and y-pitches.

For example,

```
icc2_shell> create_cell_array -lib_cell myreflib/mytapcell \
   -x_pitch 10 -y_pitch 10
```

By default, the `create_cell_array` command inserts a tap cell in each location in the specified grid for the entire block. The tool starts inserting the tap cells at the bottom-left of the core area and uses the specified x- and y-pitches to determine the location of the subsequent tap cells. The inserted tap cells are snapped to the site rows and placed in the default orientation for the site row. If a location is occupied by a fixed cell, hard macro, soft macro, or power-switch cell, the command does not insert a tap cell at that location.

You can modify the following aspects of the default behavior:

- The region in which to insert tap cells

  Use the `-voltage area` option to restrict the tap cell insertion to the specified voltage areas.

  Use the `-boundary` option to restrict the tap cell insertion to the specified region.

  If you use both options, the command inserts tap cells only in the overlapping region.

- The offset from the insertion region boundary

  Use the `-x_offset` option to shift the pattern startpoint to the right by the specified distance in microns.

  Use the `-y_offset` option to shift the pattern startpoint up by the specified distance in microns.

Feedback

- The insertion pattern

  Use the `-checkerboard` option to insert a tap cell in every other location in the specified grid.

  - To place a tap cell in the lower-left corner, use the `-checkerboard even` option.
  - To keep the lower-left corner empty, use the `-checkerboard odd` option.

  In some design methodologies, you must insert a tap cell in empty spaces in the checkerboard pattern that are above or below specific cells, such as critical area breaker cells or boundary cells. To enable the insertion of these extra tap cells, use the `-preserve_boundary_row_lib_cells` option to specify the affected library cells.

- The snapping behavior

  To disable snapping of the inserted tap cells to the site rows, set the `-snap_to_site_row false` option. When you set this option to `false`, the tool uses `R0` as the default orientation for the inserted cells.

- The orientation of the inserted tap cell instances

  To specify the orientation of the inserted tap cells, use the `-orient` option to specify one of the following orientation values: `R0`, `R90`, `R180`, `R270`, `MX`, `MY`, `MXR90`, or `MYR90`.

- The naming convention used to identify the inserted tap cell instances

  By default, the tool uses the following naming convention for inserted tap cells:

  headerfooter__*library_cell_name*_R#_C#_*number*

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  *prefix*__*library_cell_name*_R#_C#_*number*

Feedback

Inserting Exterior Tap Walls

An *exterior tap wall* is a row or column of tap cells that is placed outside the boundary of a block, macro, or hard placement blockage. The tap cells in the tap wall are placed linearly without any gaps between cells.

To insert an exterior tap wall, use the `create_exterior_tap_walls` command. You must specify the following information:

- The tap cell used in the tap wall (the `-lib_cell` option)

  By default, the command inserts the tap cells with R0 orientation. To place the cells with a different orientation, use the `-orientation` option.

  If the design has a FinFET grid, follow these guidelines when selecting the tap cell to ensure proper insertion of the tap wall:

  - For horizontal tap walls, ensure that the cell's width in the specified orientation is an integer multiple of the x-pitch of the FinFET grid
  - For vertical tap walls, ensure that the cell's height in the specified orientation is an integer multiple of the y-pitch of the FinFET grid

  By default, the tool uses the following naming convention for inserted tap cells:

  extTapWall__*library_cell_name*_R#_C#_*number*

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  *prefix*__*library_cell_name*_R#_C#_*number*

- The side along which to insert the tap wall (the `-side` option)

  You can specify only a single side, `top`, `bottom`, `right`, or `left`. To insert a tap wall on more than one side, run the command multiple times.

- The tap insertion region (the `-bbox` option)

  You specify the tap insertion region by specifying the lower-left and upper-right corners of its bounding box:

  `-bbox {{llx lly} {urx ury}}`

  The bounding box must span a portion of the object's edge on the specified side along which to insert the tap wall. If the bounding box spans multiple edges along the specified side, the command inserts the tap wall along the longest edge. If the bounding box does not span any edges of the specified side, the command issues an error message.

Feedback

By default, a horizontal tap wall starts at the left edge of the specified bounding box and abuts the top or bottom boundary edge; a vertical tap wall starts at the bottom edge of the specified bounding box and abuts the left or right boundary edge. To specify margins for the tap wall insertion, use the `-x_margin` and `-y_margin` options. If the specified region, including the margins, is too small to fit a tap wall, the command does not insert tap cells.

You should ensure that the insertion region does not contain any placement blockages, fixed cells, or macros. These obstructions can prevent the insertion of the tap cells and result in an incomplete or missing tap wall.

When inserting the tap wall, the command does not honor the standard cell rows or sites and does not cross the object boundary. After insertion, the command fixes the placement of the inserted tap cells.

Inserting Interior Tap Walls

An *interior tap wall* is a row or column of tap cells that is placed inside the boundary of the standard cell placement area of a block, a macro, hard placement blockage, or nondefault voltage area.

Note:

You can insert interior tap walls only for designs with a single site definition.

To insert an interior tap wall, use the `create_interior_tap_walls` command. You must specify the following information:

- The tap cell used in the tap wall (the `-lib_cell` option)

  The specified tap cell must be a single-height cell that matches the row height and site width.

  By default, the command inserts the tap cells with the row orientation. To place the cells with a different orientation, use the `-orientation` option.

  By default, the tool uses the following naming convention for inserted tap cells:

  tapfiller__*library_cell_name*_R#_C#_*number*

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  *prefix*__*library_cell_name*_R#_C#_*number*

- The side along which to insert the tap wall (the `-side` option)

  You can specify only a single side, `top`, `bottom`, `right`, or `left`. To insert a tap wall on more than one side, run the command multiple times.

Feedback

By default, the tap cells are placed along all edges on the specified side of the standard cell placement area with no gaps between the tap cells.

- To specify the tap insertion region, use the `-bbox` option to specify the lower-left and upper-right corners of its bounding box:

  `-bbox {{llx lly} {urx ury}}`

  When you use this option, the command inserts a tap wall along the longest edge on the specified side of the placeable area encompassed by the bounding box. The alignment of the tap wall depends on the number of corners of the edge segment encompassed by the bounding box.

  - If the edge segment does not contain a corner, the tap wall starts at the left edge for a horizontal tap wall or the bottom edge for a vertical tap wall.
  - If the edge segment contains one corner, the tap wall starts at that corner.
  - If the edge segment contains two corners along the specified side, the tap wall starts at the left edge for a horizontal tap wall or the bottom edge for a vertical tap wall.

    For a horizontal tap wall with two corners, the spacing between the last two cells might be shortened to align with the right corner. If there is not enough space, the last cell might not abut the right corner.

- For horizontal tap walls, you can specify a gap distance between tap cells by using the `-x_spacing` option.

  If you specify a value smaller than the tap cell width, the command uses a spacing of zero. If you specify a value that is not a multiple of the site width, the command rounds it down to a multiple of the site width.

When inserting the tap wall, the command honors the standard cell rows and sites; if the rows and sites are not defined, the command does not insert tap cells. Obstructions such as placement blockages, fixed cells, and macros can prevent the insertion of the tap cells and result in an incomplete or missing tap wall.

After insertion, the command fixes the placement of the inserted tap cells.

Feedback

Inserting Tap Meshes

A *tap mesh* is a two-dimensional array of tap cells with a tap cell in each mesh window. A tap mesh is typically inserted outside of a place and route block, but you can insert one in any area that is either completely inside or completely outside the core area.

To insert a tap mesh, use the `create_tap_meshes` command. You must specify the following information:

- The tap cell used in the tap mesh (the `-lib_cell` option)

  By default, the command inserts the tap cells with R0 orientation. To place the cells with a different orientation, use the `-orientation` option.

  By default, the tool uses the following naming convention for inserted tap cells:

  headerfooter__*library_cell_name*_R#_C#_*number*

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  *prefix*__*library_cell_name*_R#_C#_*number*

- The tap insertion region (the `-bbox` option)

  You specify the tap insertion region by specifying the lower-left and upper-right corners of its bounding box:

  `-bbox {{llx lly} {urx ury}}`

- The mesh window (the `-mesh_window` option)

  You specify the size of the mesh window by specifying an integer value for the pitch in the x- and y-directions.

  `-mesh_window {x_pitch y_pitch}`

  The mesh window is aligned with the lower-left corner of the tap insertion region.

When inserting the tap mesh, the command does not honor the standard cell rows or sites. It places a tap cell in the lower-left corner of each mesh window. If that location is blocked, the command uses the closest available location, which is measured as the Manhattan distance to the lower-left corner of the mesh window. If there is no available location, the command issues a warning and does not place a tap cell in that mesh window.

After insertion, the command fixes the placement of the inserted tap cells.

Feedback

Inserting Dense Tap Arrays

A *dense tap array* is a tap cell array whose tap distance is smaller than the tap distance used by the `create_tap_cells` command.

To insert a dense tap array, use the `create_dense_tap_cells` command. You must specify the following information:

- The tap cell used in the tap wall (the `-lib_cell` option)

  You must specify the same tap cell as when you ran the `create_tap_cells` command.

  By default, the tool uses the following naming convention for inserted tap cells:

  tapfiller!*library_cell_name*!*number*

  Use the `-separator` option to change the separator character from its default of "!."

  To identify the tap cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  tapfiller!*prefix*!*library_cell_name*!*number*

- The tap distance (the `-distance` option)

  The specified distance must be smaller than the tap distance used by the `create_tap_cells` command.

- The tap insertion region (the `-bbox` option)

  You specify the tap insertion region by specifying the lower-left and upper-right corners of its bounding box:

  `-bbox {{llx lly} {urx ury}}`

The `create_dense_tap_cells` command first removes the existing tap cells that are completely within the tap insertion region, and then reinserts the tap cells with the specified tap distance to create a denser array in the specified region. To avoid tap rule violations, specify the same tap cell insertion pattern as was used by the `create_tap_cells` command. The `create_dense_tap_cells` command supports the following options to control the tap cell insertion: `-offset`, `-pattern`, `-skip_fixed_cells`, and `-at_distance_only`. For details about these options, see the man page.

After insertion, the command fixes the placement of the inserted tap cells.

Feedback

Performing Boundary Cell Insertion

Before placing the standard cells, you can add boundary cells to the block. Boundary cells consist of end-cap cells, which are added to the ends of the cell rows and around the boundaries of objects such as the core area, hard macros, blockages, and voltage areas, and corner cells, which fill the empty space between horizontal and vertical end-cap cells. End-cap cells are typically nonlogic cells such as a decoupling capacitor for the power rail. Because the tool accepts any standard cell as an end-cap cell, ensure that you specify suitable end-cap cells.

To insert boundary cells,

1. Specify the boundary cell insertion requirements by using the `set_boundary_cell_rules` command, as described in Specifying the Boundary Cell Insertion Requirements.
2. Insert the boundary cells based on the specified rules by using the `compile_boundary_cells` or `compile_targeted_boundary_cells` command, as described in Inserting Boundary Cells.
3. Verify the boundary cell placement by using the `check_boundary_cells` or `check_targeted_boundary_cells` command, as described in Verifying the Boundary Cell Placement.

This process supports a single site definition per run. The `compile_boundary_cells` and `compile_targeted_boundary_cells` commands determine the site definition based on the library cells specified with the `set_boundary_cell_rules` command, and insert boundary cells only in the regions that use this site definition. If your design has multiple site definitions, you must run this process one time for each site definition.

Specifying the Boundary Cell Insertion Requirements

To specify the boundary cell insertion requirements, use the `set_boundary_cell_rules` command. You use this command to specify the following requirements:

- The library cells to use for the boundary cells, as described in Specifying the Library Cells for Boundary Cell Insertion
- The rules used to place the boundary cells, as described in Specifying Boundary Cell Placement Rules
- The naming convention used for the inserted cells, as described in Specifying the Naming Convention for Boundary Cells
- The creation of routing guides to honor the metal cut allowed and forbidden preferred grid extension rules, as described in Creating Routing Guides During Boundary Cell Insertion

Feedback

Note:

This feature is supported only by the `compile_boundary_cells` command; it is not supported by the `compile_targeted_boundary_cells` command.

- The creation of placement blockages to honor the minimum jog and minimum separation rules, as described in Creating Placement Blockages During Boundary Cell Insertion

  Note:

  This feature is supported only by the `compile_boundary_cells` command; it is not supported by the `compile_targeted_boundary_cells` command.

Note:

The settings specified by the `set_boundary_cell_rules` command are saved in the design library.

See Also

- Reporting the Boundary Cell Insertion Requirements

Specifying the Library Cells for Boundary Cell Insertion

Boundary cells include both end-cap cells placed on the left, right, top, and bottom boundaries, and inside and outside corner cells. You can specify different library cells for each boundary cell type. To specify the library cells, use the following options with the `set_boundary_cell_rules` command:

- `-left_boundary_cell` and `-right_boundary_cell`

  These options specify a single library cell that is used for the end-cap cells for the left and right boundaries, respectively.

- `-top_boundary_cells` and `-bottom_boundary_cells`

  These options specify a list of library cells that are used for the end-cap cells for the top and bottom boundaries, respectively. The command inserts the cells in the specified order. If the remaining space is smaller than the current cell, the command inserts the next cell in order that fits in the remaining space.

  For a vertical-row block, rows start at the bottom and end at the top, so the top boundary is along the left side of the block and the bottom boundary is along the right side of the block.

  For the flipped rows in a double-back block, the top boundary cells are used on the bottom boundaries and the bottom boundary cells are used on the top boundaries.

- `-top_tap_cell` and `-bottom_tap_cell`

Feedback

These options specify a single library cell that is used for the tap cells on the top and bottom boundary rows, respectively. The tool inserts the tap cells to ensure that the end-cap cells inserted on the top and bottom boundary rows comply with the maximum diffusion-to-tap distance limit.

- `-top_left_outside_corner_cell`, `-top_right_outside_corner_cell`, `-bottom_left_outside_corner_cell`, and `-bottom_right_outside_corner_cell`

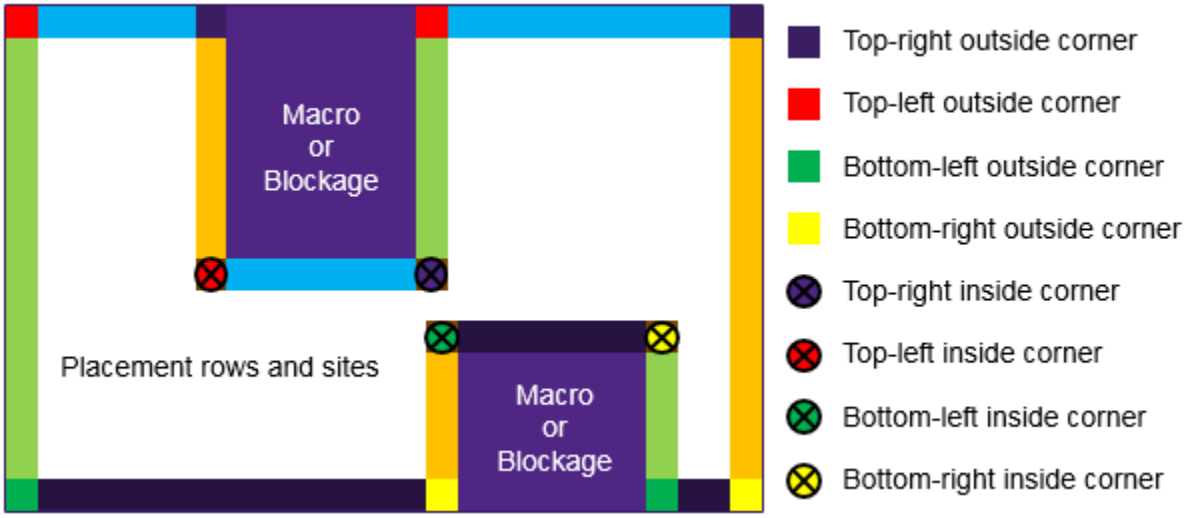
  These options specify a single library cell that is used for each outside corner location.

- `-top_left_inside_corner_cells`, `-top_right_inside_corner_cells`, `-bottom_left_inside_corner_cells`, and `-bottom_right_inside_corner_cells`

  These options specify a list of library cells for each inside corner location. The tool inserts the first corner cell that matches the size of the inside corner. If none matches exactly, it inserts the first cell that can be placed without violating any rules.

Figure 85 shows the boundary and corner cell locations for a horizontal-row block with two hard macros or blockages. Note that the cell locations are flipped when the row is flipped.

Figure 85 Boundary Cell Locations



Feedback

Specifying Boundary Cell Placement Rules

By default,

- When you run the `compile_boundary_cells` command, the tool places the boundary cells in their default orientation around the core area, hard macros, and hard placement blockages.
- When you run the `compile_targeted_boundary_cells` command, the tool places the boundary cells in their default orientation around the specified objects.

To flip the boundary cell orientations, use one or more of the following options with the `set_boundary_cell_rules` command: `-mirror_left_boundary_cell`, `-mirror_right_boundary_cell`, `-mirror_left_outside_corner_cell`, `-mirror_right_outside_corner_cell`, `-mirror_left_inside_corner_cell`, `-mirror_right_inside_corner_cell`, `-mirror_left_inside_horizontal_abutment_cell`, and `-mirror_right_inside_horizontal_abutment_cell`. You cannot flip the orientation of the top and bottom boundary cells.

In addition, when you use the `compile_boundary_cells` command, you can modify the following aspects of the default placement behavior:

- Swapping of the top and bottom inside corner cells on flipped rows

  Use the `-do_not_swap_top_and_bottom_inside_corner_cell` option with the `set_boundary_cell_rules` command to prevent the command from using the bottom inside corner cell on the top inside corner of flipped rows and the top inside corner cell on the bottom inside corner of flipped rows.

- Consideration of voltage areas

  Use the `-at_va_boundary` option with the `set_boundary_cell_rules` command to insert horizontal boundary cells on both sides of the voltage area boundaries.

- Existence of one-unit-tile gaps

  Use the `-no_1x` option with the `set_boundary_cell_rules` command to prevent boundary cell insertion from creating one-unit-tile gaps. When you use this option, the command does not insert boundary cells on a row when the row length equals two times the corner cell width plus one unit tile width. Note that if the row length equals two times the corner cell width, the command does insert boundary cells.

- Distance between tap cells

  Use the `-tap_distance` option with the `set_boundary_cell_rules` command to specify the distance in microns between the tap cells inserted on the top and bottom boundary rows.

Feedback

- Insertion of boundary cells on short rows

  Use the `-min_row_width` option with the `set_boundary_cell_rules` command to prevent insertion of boundary cells on short rows. This option defines the width threshold for inserting boundary cells on a row. If the row width is less than the specified value, the command does not insert boundary cells on the row.

- Insertion of boundary cells in child blocks

  Use the `-insert_into_blocks` option with the `set_boundary_cell_rules` command to recursively insert boundary cells in the child blocks. By default, the tool does not insert boundary cells in the child blocks.

Specifying the Naming Convention for Boundary Cells

By default, the tool uses the following naming convention for inserted boundary cells:

boundarycell!*library_cell_name*!*number*

To change the separator character from its default of "!," use the `-separator` option with the `set_boundary_cell_rules` command..

To identify the boundary cells inserted in a specific run, use the `-prefix` option with the `set_boundary_cell_rules` command to specify a prefix string. When you use this option, the tool uses the following naming convention:

boundarycell!*prefix*!*library_cell_name*!*number*

Creating Routing Guides During Boundary Cell Insertion

If your technology file defines metal cut allowed and forbidden preferred grid extension rules, you can create routing guides for these rules during boundary cell insertion by using the `-add_metal_cut_allowed` option with the `set_boundary_cell_rules` command. When you use this option, the `compile_boundary_cells` command creates the following routing guides:

- Metal cut allowed routing guides, which cover the area taken up by all the placeable site rows reduced by the vertical shrink factor, which is 50 percent of the smallest site row height
- Forbidden preferred grid extension routing guides, which cover the remaining area up to the block boundary

Note:

This feature is supported only by the `compile_boundary_cells` command; it is not supported by the `compile_targeted_boundary_cells` command.

Feedback

Creating Placement Blockages During Boundary Cell Insertion

If your technology has minimum jog or minimum separation requirements, you can create placement blockages for these requirements during boundary cell insertion with the `compile_boundary_cells` command by using the following options with the `set_boundary_cell_rules` command:

- `-min_horizontal_jog`

  If a horizontal edge of a macro, including its hard keepout margin, hard placement blockage, or voltage area, including its guard band, is less than the specified value, the `compile_boundary_cells -add_placement_blockage` command creates a placement blockage to prevent a violation.

- `-min_vertical_jog`

  If a vertical edge of a macro, including its hard keepout margin, hard placement blockage, or voltage area, including its guard band, is less than the specified value, the `compile_boundary_cells -add_placement_blockage` command creates a placement blockage to prevent a violation.

- `-min_horizontal_separation`

  If the continuous horizontal placement area is less than the specified value, the `compile_boundary_cells -add_placement_blockage` command creates a placement blockage to prevent a violation.

- `-min_vertical_separation`

  If the continuous vertical placement area is less than the specified value, the `compile_boundary_cells -add_placement_blockage` command creates a placement blockage to prevent a violation.

Note:

By default, the `compile_boundary_cells` command checks the specified rules, but does not create the placement blockages. To create the placement blockages, you must use the `-add_placement_blockage` option with the `compile_boundary_cells` command.

This feature is supported only by the `compile_boundary_cells` command; it is not supported by the `compile_targeted_boundary_cells` command.

Reporting the Boundary Cell Insertion Requirements

To report the boundary cell insertion requirements specified by the `set_boundary_cell_rules` command, use the `report_boundary_cell_rules` command. This command reports only the user-specified settings; it does not report any default settings.

Feedback

Removing Boundary Cell Insertion Requirements

To remove one or more of the boundary cell insertion requirements specified by the `set_boundary_cell_rules` command, use the `remove_boundary_cell_rules` command.

Inserting Boundary Cells

The IC Compiler II tool provides several methods for inserting boundary cells, depending on where you want to insert the boundary cells.

- To insert boundary cells around the core area, hard macros, and placement blockages, use the `compile_boundary_cells` command.
- To insert boundary cells only around one or more voltage areas, use the `compile_boundary_cells` command with the `-voltage_area` option.
- To insert boundary cells only around specific objects, use the `compile_targeted_boundary_cells` command with the `-target_objects` option. You can use this command to insert boundary cells around specific macros, voltage areas, voltage area shapes, placement blockages, routing blockages, and the core area.

The `set_boundary_cell_rules` command configures boundary cell insertion for both the `compile_boundary_cells` and `compile_targeted_boundary_cells` commands. However, some rules are honored only by the `compile_boundary_cells` command. For details, see Specifying the Boundary Cell Insertion Requirements.

Verifying the Boundary Cell Placement

After inserting the boundary cells with the `compile_boundary_cells` command, verify the placement by using the `check_boundary_cells` command. To create an error data file to view the errors in the error browser, use the `-error_view` option. For information about using the error browser, see the *IC Compiler II Graphical User Interface User Guide*.

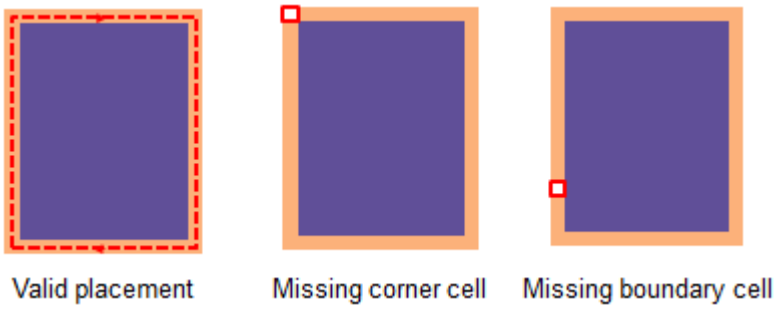
This command checks the boundary cell placement for the following issues:

- Missing boundary or corner cells

  This check verifies that there are boundary and corner cells around the entire boundary, with no gaps. Figure 86 shows a valid placement, as well as errors caused by missing cells.

Feedback

Figure 86 Check for Missing Boundary or Corner Cells



Note:

For established nodes, boundary cells are inserted only on the left and right sides. In this case, the command verifies only that there are no gaps on these sides.

- Incorrect boundary or corner cells

  This check verifies that the library cells used for the boundary and corner cells match the cells specified by the `set_boundary_cell_rules` command.

- Incorrect orientation of boundary cells

  This check verifies that the orientation of each boundary cell matches the allowed orientations specified by the `set_boundary_cell_rules` command.

- Short rows and edges

  This check verifies that each row of boundary cells is wider than the value specified by the `set_boundary_cell_rules -min_row_width` option and that the horizontal edges of each blockage are greater than the value specified by the `set_boundary_cell_rules -min_horizontal_jog` option.

Finding and Fixing Antenna Violations

In chip manufacturing, gate oxide can be easily damaged by electrostatic discharge. The static charge that is collected on wires during the multilevel metalization process can damage the device or lead to a total chip failure. The phenomenon of an electrostatic charge being discharged into the device is referred to as either antenna or charge-collecting antenna problems.

Feedback

To prevent antenna problems, the tool verifies that for each input pin the metal antenna area divided by the gate area is less than the maximum antenna ratio given by the foundry:

(antenna-area)/(gate-area) < (max-antenna-ratio)

This check is based on the global and layer-specific antenna rules defined for the block and the antenna properties of the cells in the block.

The antenna flow consists of the following steps:

1. Define the antenna rules.
2. Specify the antenna properties of the pins and ports.
3. Analyze and fix the antenna violations.

Defining Antenna Rules

Antenna rules define how to calculate the maximum antenna ratio for the nets in a block, as well as the antenna ratio for a pin. You must define a global antenna rule by using the `define_antenna_rule` command. You can also specify layer-specific antenna rules by using the `define_antenna_layer_rule` command. The tool uses the global antenna rule whenever a layer-specific antenna rule does not exist.

The following topics describe how to define the antenna rules:

- Calculating the Maximum Antenna Ratio
- Calculating the Antenna Ratio for a Pin

Calculating the Maximum Antenna Ratio

To control the method used to calculate the maximum allowable antenna ratio, you must specify the following information:

- How much protection the diodes provide (the diode protection mode)

  To specify the diode protection mode, use the `-diode_mode` option with the `define_antenna_rule` command, as described in Setting the Diode Protection Mode. This is a required option of the `define_antenna_rule` command.

- How to perform antenna ratio calculation with diode protection (the diode ratio vector)

  To specify the diode ratio vector, use the `-diode_ratio` option with the `define_antenna_layer_rule` command, as described in Specifying the Diode Ratio Vector. This is a required option of the `define_antenna_layer_rule` command.

Feedback

Setting the Diode Protection Mode

The diode protection mode specifies how much protection the diodes provide.

Advanced technology designs often use cells and macros with different gate oxide thicknesses, which affect the antenna rule calculations. The IC Compiler II tool uses gate classes to represent the various gate oxide thicknesses. The tool supports up to four gate oxide thicknesses: gate class 0, gate class 1, gate class 2, and gate class 3. If a cell pin does not have gate class data, it is treated as gate class 0.

The IC Compiler II tool supports 18 diode protection modes, most of which support only a single gate oxide thickness. For advanced technology designs that use cells and macros with different gate oxide thicknesses, you must use one of the diode protection modes that support multiple gate oxide thicknesses: 14, 16, and 18.

To specify the diode protection mode, use the `-diode_mode` option with the `define_antenna_rule` command. This is a required option of the `define_antenna_rule` command.

Table 29 defines how each of the diode protection mode settings affect the maximum antenna ratio for a net. The diode protection mode also affects the computation of the maximum antenna ratio for individual diodes. The antenna ratio calculation formulas for diodes are shown in Table 31; these formulas use the values in the diode ratio vector specified in the `-diode_ratio` option of the `define_antenna_layer_rule` command, as described in Specifying the Diode Ratio Vector.

Table 29 Diode Protection Mode Settings

Diode protection mode	Definition
0	The diodes do not provide any protection.
1	The diodes provide unlimited protection. In this mode, the highest metal layer is not checked for antenna violations because the input-to-output connection is completed when the highest metal layer is formed.
2	Diode protection is limited; if more than one diode is connected, the largest value of the maximum antenna ratio for all diodes is used.
3	Diode protection is limited; if more than one diode is connected, the sum of the maximum antenna ratios for all diodes is used.
4	Diode protection is limited; if more than one diode is connected, the sum of the diode-protection values for all diodes is used to compute the maximum antenna ratio.
5	Diode protection is limited; the maximum diode-protection value for all diodes is used to calculate the equivalent gate area.

Feedback

Table 29 Diode Protection Mode Settings (Continued)

Diode protection mode	Definition
6	Diode protection is limited; the sum of the diode-protection values for all diodes is used to calculate the equivalent gate area.
7	Diode protection is limited; the maximum diode-protection value for all diodes is used to calculate the equivalent metal area.
8	Diode protection is limited; the sum of the diode-protection values for all diodes is used to calculate the equivalent metal area.
9	Diode protection is limited; scaling is based on maximum diode protection.
10	Diode protection is limited; scaling is based on total diode protection.
11	Diode protection is limited; scaling is based on maximum diode protection.
12	Diode protection is limited; scaling is based on total diode protection.
13	Obsolete.
14	Diode protection is limited; two separate calculation formulas. Supports gate classes.
15	Internal use only.
16	Diode protection is limited; if more than one diode is connected, the sum of the diode-protection values for all diodes is used to compute the maximum antenna ratio. Supports gate classes.
17	Diode protection is limited; the sum of the diode-protection values for all diodes is used to calculate the equivalent gate area. The gate area calculation is done separately for the input and output pins.
18	Diode protection is limited; if more than one diode is connected, the sum of the diode-protection values for all diodes is used to compute the maximum antenna ratio. Supports gate classes and calculation depends on whether the pin is connected to a diode.
25	Diode protection is limited; if more than one diode is connected, the sum of the diode-protection values for all diodes is used to compute the maximum antenna ratio. This diode mode support exponential formula for allowable max-antenna-ratio.

Specifying the Diode Ratio Vector

The diode ratio vector specifies the values used to calculate the maximum antenna ratio. To specify the diode ratio vector, use the `-diode_ratio` option with the

Feedback

`define_antenna_layer_rule` command. If you do not specify a diode ratio, the tool uses {0 0 1 0 0}. The actual usage of the diode ratio vector depends on the diode mode that was specified in the `define_antenna_rule -diode_mode` option.

Table 30 shows the format of the diode ratio vector for the various diode modes. Table 31 shows how the vector values are used to calculate the maximum antenna ratio for each diode mode. In this table,

- *dp* represents the diode protection value specified for an output pin. For information about specifying this value, see Specifying Antenna Properties.
- *layerMaxRatio* represents the maximum antenna ratio for the layer, as specified by the `-ratio` option of the `define_antenna_layer_rule` command or one the `-metal_ratio` or `-cut_ratio` option of the `define_antenna_rule` command if the layer-specific value is not defined.

For examples of the maximum antenna ratio calculations, see

- Example for Diode Modes 2, 3, and 4
- Example for Diode Modes 5 and 6
- Example for Diode Modes 7 and 8
- Example for Diode Mode 14 With Multiple Gate Oxide Thicknesses

Table 30 Diode Ratio Vector Formats

Diode modes	Format
2-10, 17	`{v0 v1 v2 v3 [v4]}` The *v4* value represents the upper limit of the diode protection and is optional. If you do not specify the *v4* value, it is assumed to be 0, which means there is no upper limit.
11, 12	`{{v0 v1 v2 v3 [v4]}{s0 s1 s2 s2 s4 s5}}` The second vector, `{s0 s1 s2 s3 s4 s5}`, specifies scaling values.
14	$\{v0_1\ v1_1\ v2_1\ v3_1\ v4_1\ v5_1\ v6_1\ v7_1\ v8_1\ v9_1\}$ $\{v0_2\ v1_2\ v2_2\ v3_2\ v4_2\ v5_2\ v6_2\ v7_2\ v8_2\ v9_2\}$ $\{v0_3\ v1_3\ v2_3\ v3_3\ v4_3\ v5_3\ v6_3\ v7_3\ v8_3\ v9_3\}$ $\{v0_4\ v1_4\ v2_4\ v3_4\ v4_4\ v5_4\ v6_4\ v7_4\ v8_4\ v9_4\}$ You specify one 10-value vector for each of the four gate classes.
16	$\{v0_1\ v1_1\ v2_1\ v3_1\ v4_1\ v5_1\ v6_1\ v7_1\}$ $\{v0_2\ v1_2\ v2_2\ v3_2\ v4_2\ v5_2\ v6_2\ v7_2\}$ $\{v0_3\ v1_3\ v2_3\ v3_3\ v4_3\ v5_3\ v6_3\ v7_3\}$ $\{v0_4\ v1_4\ v2_4\ v3_4\ v4_4\ v5_4\ v6_4\ v7_4\}$ You specify one 8-value vector for each of the four gate classes.

Feedback

Table 30 Diode Ratio Vector Formats (Continued)

Diode modes	Format
18	{$v0_1$ $v1_1$ $v2_1$ $v3_1$ $v4_1$ $v5_1$ $v6_1$ $v7_1$ $v8_1$ $v9_1$ $v10_1$ $v11_1$} {$v0_2$ $v1_2$ $v2_2$ $v3_2$ $v4_2$ $v5_2$ $v6_2$ $v7_2$ $v8_2$ $v9_2$ $v10_2$ $v11_2$} {$v0_3$ $v1_3$ $v2_3$ $v3_3$ $v4_3$ $v5_3$ $v6_3$ $v7_3$ $v8_3$ $v9_3$ $v10_3$ $v11_3$} {$v0_4$ $v1_4$ $v2_4$ $v3_4$ $v4_4$ $v5_4$ $v6_4$ $v7_4$ $v8_4$ $v9_4$ $v10_4$ $v11_4$} You specify one 12-value vector for each of the four gate classes.
25	{$v0_1$ $v1_1$ $v2_1$ $v3_1$ $v4_1$ $v5_1$ $v6_1$ $v7_1$ $v8_1$ $v9_1$ $v10_1$ $v11_1$} {$v0_2$ $v1_2$ $v2_2$ $v3_2$ $v4_2$ $v5_2$ $v6_2$ $v7_2$ $v8_2$ $v9_2$ $v10_2$ $v11_2$} {$v0_3$ $v1_3$ $v2_3$ $v3_3$ $v4_3$ $v5_3$ $v6_3$ $v7_3$ $v8_3$ $v9_3$ $v10_3$ $v11_3$} {$v0_4$ $v1_4$ $v2_4$ $v3_4$ $v4_4$ $v5_4$ $v6_4$ $v7_4$ $v8_4$ $v9_4$ $v10_4$ $v11_4$} You specify one 12-value vector for each of the four gate classes.

Table 31 Antenna Ratio Calculation Based on Diode Mode

Diode mode	Calculation
0, 1	Not used
2, 3, 4	• If *dp*>*v0* and *v4*<>0, antenna_ratio = min (((*dp* + *v1*) * *v2* + *v3*), *v4*) • If *dp*>*v0* and *v4*=0, antenna_ratio = (*dp* + *v1*) * *v2* + *v3* • If *dp*<=*v0*, antenna_ratio = *layerMaxRatio*
5	antenna_ratio = metal_area / (gate_area + equi_gate_area) • If max_diode_protection>*v0* and *v4*<>0, equi_gate_area = min (((max_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If max_diode_protection>*v0* and *v4*=0, equi_gate_area = (max_diode_protection + *v1*) * *v2* + *v3* • If max_diode_protection<=*v0*, equi_gate_area = 0
6	antenna_ratio = metal_area / (gate_area + equi_gate_area) • If total_diode_protection>*v0* and *v4*<>0, equi_gate_area = min (((total_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If total_diode_protection>*v0* and *v4*=0, equi_gate_area = (total_diode_protection + *v1*) * *v2* + *v3* • If total_diode_protection<=*v0*, equi_gate_area = 0

Feedback

Table 31 Antenna Ratio Calculation Based on Diode Mode (Continued)

Diode mode	Calculation
7	antenna_ratio = (metal_area - equi_metal_area) / gate_area • If max_diode_protection>*v0* and *v4*<>0, equi_metal_area = min (((max_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If max_diode_protection>*v0* and *v4*=0, equi_metal_area = (max_diode_protection + *v1*) * *v2* + *v3* • If max_diode_protection<=*v0*, equi_metal_area = 0 • If equi_metal_area>metal_area, equi_metal_area = metal_area
8	antenna_ratio = (metal_area - equi_metal_area) / gate_area • If total_diode_protection>*v0* and *v4*<>0, equi_metal_area = min (((total_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If total_diode_protection>*v0* and *v4*=0, equi_metal_area = (total_diode_protection + *v1*) * *v2* + *v3* • If total_diode_protection<=*v0*, equi_metal_area = 0 • If equi_metal_area > metal_area, equi_metal_area = metal_area
9	antenna_ratio = scale * metal_area / gate_area • If max_diode_protection>*v0*, scale = max (1 / ((max_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If max_diode_protection<=*v0*, scale = 1.0
10	antenna_ratio = scale * metal_area / gate_area • If total_diode_protection>*v0*, scale = max (1 / ((total_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If total_diode_protection<= *v0*, scale = 1.0
11	antenna_ratio = scale * metal_area / gate_area • If max_diode_protection<*v0*, scale = *s0* • If max_diode_protection<*v1*, scale = *s1* • If max_diode_protection<*v2*, scale = *s2* • If max_diode_protection<*v3*, scale = s3 • If max_diode_protection<*v4*, scale = *s4* • If max_diode_protection>=*v4*, scale = *s5*

Feedback

Table 31 Antenna Ratio Calculation Based on Diode Mode (Continued)

Diode mode	Calculation
12	antenna_ratio = scale * metal_area / gate_area • If total_diode_protection<*v0*, scale = *s0* • If total_diode_protection<*v1*, scale = *s1* • If total_diode_protection<*v2*, scale = *s2* • If total_diode_protection<*v3*, scale = *s3* • If total_diode_protection<*v4*, scale = *s4* • If total_diode_protection>=*v4*, scale = *s5*
13	N/A
14	If connected to a diode, • (metal or via area) / (gate-area) <= (*dp***v1* + *v2*) // formula1 and (metal or via area) / (*v5**gate-area + *v6** *dp*) <= *v7*// formula2 If not connected to a diode, • (metal or via area) / (gate-area) <= *v3* // formula1 and (metal or via area) / (*v8* * gate-area) <= *v9* // formula2
15	N/A
16	• If *dp*>*v0* and *v4*<>0, allowable max-antenna-ratio = min (((*dp* + *v1*) * *v2* + *v3*), *v4*) • If *dp*>*v0* and *v4*=0, allowable max-antenna-ratio = (*dp* + *v1*) * *v2* + *v3* • If *dp*<=*v0*, allowable max-antenna-ratio = *layerMaxRatio* • If (gate-area <= *v5*), antenna-ratio = antenna-area/(gate-area + *v6***dp*) else antenna-ratio = antenna-area/ (gate-area + *v6***dp* + *v7*)
17	antenna_ratio = metal_area / (gate_area + equi_gate_area) • If total_diode_protection>*v0* and *v4*<>0, equi_gate_area = min (((total_diode_protection + *v1*) * *v2* + *v3*), *v4*) • If total_diode_protection>*v0* and *v4*=0, equi_gate_area = (total_diode_protection + *v1*) * *v2* + *v3* • If total_diode_protection<=*v0*, equi_gate_area = 0

Feedback

Table 31 Antenna Ratio Calculation Based on Diode Mode (Continued)

Diode mode	Calculation
18	If connected to a diode, • If *dp*>*v0* and *v4*<>0, allowable max-antenna-ratio = min (((*dp* + *v1*) * *v2* + *v3*), *v4*) • If *dp*>*v0* and *v4*=0, allowable max-antenna-ratio = (*dp* + *v1*) * *v2* + *v3* • If *dp*<=*v0*, allowable max-antenna-ratio = *layerMaxRatio* • If (gate-area <= *v5*), antenna-ratio = antenna-area/(gate-area + *v6***dp*) else antenna-ratio = antenna-area/(gate-area + *v6***dp* + *v7*) If not connected to a diode, • If (gate-area < *v8*), allowable max-antenna-ratio = *v9* else allowable-max-antenna-ratio = *v10**(gate-area)^{v11} • antenna-ratio = (antenna-area)/(gate-area)
25	Allowable max-antenna-ratio calculation • If *dp*>*v0* and *v4*<>*0*, allowable max-antenna-ratio = min (((*dp* + *v1*) * *v2* + *v3*), *v4*) • If *dp*>*v0* and *v4=0*, allowable max-antenna-ratio = (*dp* + *v1*) * *v2* + *v3* • If *dp*<=*v0*, 1.If (gate_area < *v8*), allowable max-antenna-ratio = *LayerMaxRatio* 2.If (gate_area >= *v8*), allowable max-antenna-ratio = *v9* * (gate-area)^{v10} + *v11* Antenna-ratio calculation • If *dp*>*v0* 1.If (gate-area <= *v5*), antenna-ratio = antenna-area/(gate-area + *v6***dp*) 2.If (gate-area > *v5*), antenna-ratio = antenna-area/(gate-area + *v6***dp* + *v7*) • If *dp*<=*v0*, antenna-ratio = antenna-area/gate-area

Example for Diode Modes 2, 3, and 4

Assume that the antenna area is calculated using surface area in single-layer mode (antenna mode 1), the diode ratio vector for the M1 layer is {0.7 0.0 200 2000}, the *layerMaxRatio* value for the M1 layer is 400, and the following diodes are connected to a single M1 net: diode A with a diode-protection value of 0.5, diode B with a diode-protection value of 1.0, and diode C with a diode-protection value of 1.5. In this example, the *v4* value is not specified so a value of 0 is used.

Use the following commands to define the antenna rules for this example:

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
   -metal_ratio 400 -cut_ratio 20
```

Feedback

```
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
   -ratio 400 -diode_ratio {0.7 0.0 200 2000}
```

The maximum antenna ratio for each diode is computed by using the formula for the diode mode from Table 31. In this case,

- If *dp*>*v0* and *v4*<>0,

  allowable max-antenna-ratio = min (((*dp* + *v1*) * *v2* + *v3*), *v4*)

- If *dp*>*v0* and *v4*=0,

  allowable max-antenna-ratio = (*dp* + *v1*) * *v2* + *v3*

- If *dp*<=*v0*,

  allowable max-antenna-ratio = *layerMaxRatio*

Table 32 shows the maximum antenna ratio for each diode calculated using this formula.

Table 32 Calculation of Maximum Antenna Ratio for Each Diode

Diode	Protection value	Maximum Antenna Ratio
A	0.5	400 Protection value is less than the *v0* value of 0.7, so use maximum antenna ratio of 400.
B	1.0	2200 Protection value is greater than the *v0* value of 0.7, so maximum antenna ratio is (1.0+0.0) * 200 + 2000 = 2200.
C	1.5	2300 Protection value is greater than the *v0* value of 0.7, so maximum antenna ratio is (1.5+0.0) * 200 + 2000 = 2300.

The maximum antenna ratio for the net is computed by using the formula for the diode mode from Table 30:

- For diode mode 2, the maximum antenna ratio for the net is the largest of the maximum antenna ratio values for the diodes, 2300.
- For diode mode 3, the maximum antenna ratio for the net is the sum of the maximum antenna ratios for the diodes, 400 + 2200 + 2300 = 4900.
- For diode mode 4, the maximum antenna ratio for the net is computed by using the formula from Table 31 using the sum of the diode-protection values of the diodes, (0.5+1.0+1.5) * 200 + 2000 = 2600.

Feedback

Example for Diode Modes 5 and 6

Assume that the antenna area is calculated using surface area in single-layer mode (antenna mode 1), the diode ratio vector for the M1 layer is {0 0 1 0} (the default diode ratio vector), the *layerMaxRatio* value for the M1 layer is 400, and the following diodes are connected to a single M1 net: diode A with a diode-protection value of 0.5, diode B with a diode-protection value of 1.0, and diode C with a diode-protection value of 1.5. In this example, the *v4* value is not specified so a value of 0 is used.

Use the following command to define the antenna rules for this example:

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
   -metal_ratio 400 -cut_ratio 20
```

Because this example uses the global *layerMaxRatio* value and the default diode ratio vector, you do not need to define a layer-specific antenna rule.

As shown in Table 31, for diode modes 5 and 6, the maximum antenna ratio is computed as

metal_area / (*gate_area* + *equi_gate_area*)

where the equivalent gate area, *equi_gate_area*, is computed by using the diode ratio vector: (*diode_protection* + *v1*) * (*v2* + *v3*).

- For diode mode 5, the equivalent gate area is computed using the maximum diode protection, which is 1.5, so the maximum antenna ratio for the net is

  metal_area / (*gate_area* + ((1.5 + 0) * (1 + 0))) = *metal_area*/(gate_area + 1.5)

- For diode mode 6, the equivalent gate area is computed using the total diode protection, which is 0.5+1.0+1.5=3.0, so the maximum antenna ratio for the net is

  metal_area / (gate_area + ((3.0 + 0) * (1 + 0))) = *metal_area*/(gate_area + 3.0)

Example for Diode Modes 7 and 8

Assume that the antenna area is calculated using surface area in single-layer mode (antenna mode 1), the diode ratio vector for the M1 layer is {0.7 0.0 150 800}, the *layerMaxRatio* value for the M1 layer is 400, and the following diodes are connected to a single M1 net: diode A with a diode-protection value of 0.5, diode B with a diode-protection value of 1.0, and diode C with a diode-protection value of 1.5. In this example, the *v4* value is not specified so a value of 0 is used.

Use the following commands to define the antenna rules for this example:

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
   -metal_ratio 400 -cut_ratio 20
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
   -ratio 400 -diode_ratio {0.7 0.0 150 800}
```

Feedback

As shown in Table 31, for diode modes 7 and 8, the maximum antenna ratio is computed as

(*metal_area* - *equi_gate_area*) / *gate_area*

where the equivalent gate area, *equi_gate_area*, is computed by using the diode ratio vector: (*diode_protection* + *v1*) * (*v2* + *v3*).

- For diode mode 7, the equivalent metal area is computed using the maximum diode protection, which is 1.5, so the maximum antenna ratio for the net is

  (*metal_area* - ((1.5 + 0) * (150 + 800))) / *gate_area* = (*metal_area* - 1025) / *gate_area*

- For diode mode 8, the equivalent metal area is computed using the total diode protection, which is 0.5+1.0+1.5=3.0, so the maximum antenna ratio for the net is

  (*metal_area* - ((3.0 + 0) * 150 + 800)) / *gate_area* = (*metal_area* - 1250) / *gate_area*

Example for Diode Mode 14 With Multiple Gate Oxide Thicknesses

Assume that the antenna area is calculated using surface area in single-layer mode (antenna mode 1), the design contains cells with two different gate oxide thicknesses, the diode ratio vector for the first oxide thickness (gate class 0) for the M1 layer is {1 0 1e9 1e9 0 1 2 285 1 285}, the diode ratio vector for the second oxide thickness (gate class 1) for the M1 layer is {1 0 1e9 1e9 1 0 2 165 1 28}, and the *layerMaxRatio* value for the M1 layer is 285.

For diode mode 14, the diode ratio vector contains 10 values for each gate class. Because the design uses only two gate oxide thicknesses, only the vectors for gate class 0 and gate class 1 are used. However, you must specify all 40 values when defining the antenna rule. For unused gate classes (gate class 2 and gate class 3 in this example), specify very large ratios to disable the checks.

Use the following commands to define the antenna rules for this example:

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode 14 \
   -metal_ratio 285 -cut_ratio 10
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
   -ratio 285 -diode_ratio {1 0 1e9 1e9 0 1 2 285 1 285 \
                            1 0 1e9 1e9 1 0 2 165 1 28 \
                            1 0 1e9 1e9 0 1 2 1e9 1 1e9 \
                            1 0 1e9 1e9 0 1 2 1e9 1 1e9 }
```

The following table shows the vector usage:

	Formula 1	Formula 2

Feedback

	Gate sum switch	With Diode		No Diode		With Diode			No Diode	
		Diode protection multiplier	Offset	Ratio		Gate area multiplier	Diode protection multiplier	Ratio	Gate area multiplier	Ratio
Gate Class 0	$v0_1$	$v1_1$	$v2_1$	$v3_1$	$v4_1$	$v5_1$	$v6_1$	$v7_1$	$v8_1$	$v9_1$
Gate Class 1	$v0_2$	$v1_2$	$v2_2$	$v3_2$	$v4_2$	$v5_2$	$v6_2$	$v7_2$	$v8_2$	$v9_2$
Gate Class 2	$v0_3$	$v1_3$	$v2_3$	$v3_3$	$v4_3$	$v5_3$	$v6_3$	$v7_3$	$v8_3$	$v9_3$
Gate Class 3	$v0_4$	$v1_4$	$v2_4$	$v3_4$	$v4_4$	$v5_4$	$v6_4$	$v7_4$	$v8_4$	$v9_4$

Here:

- V0: Defines which gate areas are used for the check for the first formula:
  - 0: Gate area of all gates that belong to the current gate class
  - 1: Total gate area regardless of gate class
- V1: Defines the diode protection multiplier for the connected to diode case for the first formula
- V2: Defines the offset value for the connected to diode case for the first formula
- V3: Defines the ratio requirement for the not connected to diode case for the first formula
- V4: Defines which gate areas are used for the check for the second formula:
  - 0: Gate area of all gates that belong to the current gate class
  - 1: Total gate area regardless of gate class
- V5: Defines the gate area multiplier for the connected to diode case for the second formula
- V6: Defines the diode protection multiplier for the connected to diode case for the second formula
- V7: Defines the ratio requirement for the connected to diode case for the second formula

Feedback

- V8: Defines the gate area multiplier for the not connected to diode case for the second formula
- V9: Defines the antenna ratio requirement for the not connected to diode case for the second formula

Calculating the Antenna Ratio for a Pin

The tool supports several ways to calculate the antenna ratio (antenna-area/gate-area). You control this calculation by specifying the following information:

- How the antenna area is calculated (the antenna area mode)
- Which metal segments are considered for the calculation (the antenna recognition mode)

To specify these modes, use the `-mode` option with the `define_antenna_rule` and `define_antenna_layer_rule` commands, as described in Setting the Antenna Mode. This is a required option for both of these commands.

Setting the Antenna Mode

The antenna mode controls the antenna ratio calculation by determining

- How the antenna area is calculated (the antenna area mode)

  The tool supports the following area calculation modes:

  - Surface area, which is calculated as W x L
  - Sidewall area, which is calculated as (W + L) x 2 x thickness

    Note:

    If you use sidewall area calculation, you must define the metal thickness by specifying the `unitMinThickness`, `unitNomThickness`, and `unitMaxThickness` attributes in each `Layer` section of the technology file.

- Which metal segments are considered for the calculation (the antenna recognition mode)

  To tool supports the following antenna recognition modes:

  - Single-layer mode

    In single-layer mode, the tool considers only the metal segments on the current layer; the metal segments on all lower layers are ignored. This mode allows the best routability. In this mode, the antenna ratio is calculated as

    antenna_ratio = connected metal area of the layer / total gate area

  - Accumulated-ratio mode

    In accumulated-ratio mode, the tool considers the metal segments on the current layer and the lower-layer segments connected to the input pins. In this mode, the antenna ratio is calculated as

    antenna_ratio = accumulation of the single-layer mode ratios of the current layer and layers below

  - Accumulated-area mode

    In accumulated-area mode, the tool considers the metal segments on the current and lower layers. In this mode, the antenna ratio is calculated as

    antenna_ratio = connected metal area of the current and lower layers/ total gate area

  For a detailed example of the antenna recognition modes, see Antenna Recognition Mode Example.

The IC Compiler II tool supports six antenna modes, which are described in Table 33. To specify the antenna mode, use the `-mode` option with the `define_antenna_rule` and `define_antenna_layer_rule` commands. This is a required option for both commands.

Table 33 Antenna Mode Settings

Antenna mode value	Area calculation mode	Antenna recognition mode
1	Surface	Single-layer
2	Surface	Accumulated-ratio
3	Surface	Accumulated-area
4	Sidewall	Single-layer
5	Sidewall	Accumulated-ratio

Feedback

Table 33 Antenna Mode Settings (Continued)

Antenna mode value	Area calculation mode	Antenna recognition mode
6	Sidewall	Accumulated-area

Antenna Recognition Mode Example

Figure 87 shows a layout example with a lateral view, which is used to explain the antenna recognition modes. Table 34 shows the antenna ratios for each antenna recognition mode for this layout example.

Figure 87 Layout Example for Antenna Recognition Modes

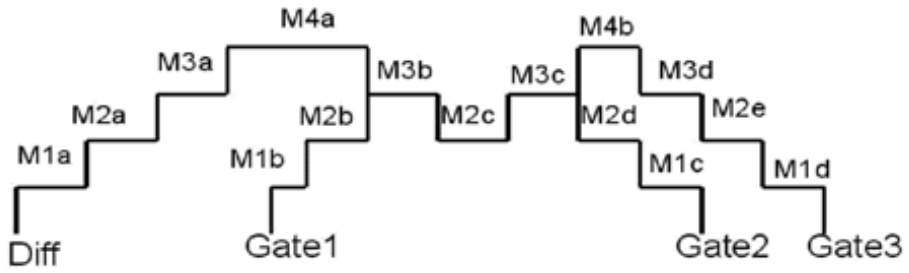


Table 34 Antenna Recognition Modes and Ratios

Considered segments	Antenna ratio
Single-layer mode	M1 ratios • Gate1: M1b / Gate1 • Gate2: M1c / Gate2 • Gate3: M1d / Gate3 M2 ratios • Gate1: M2b / Gate1 • Gate2: M2d / Gate2 • Gate3: M2e / Gate3 M3 ratios • Gate1, 2: (M3b + M3c) / (Gate1 + Gate2) • Gate3: M3d / Gate3 M4 ratios • Gate1, 2, 3: (M4a + M4b) / (Gate1 + Gate2 + Gate3)

Feedback

Table 34 Antenna Recognition Modes and Ratios (Continued)

Considered segments	Antenna ratio
Accumulated-ratio mode	M1 ratios • Gate1: M1b / Gate1 • Gate2: M1c / Gate2 • Gate3: M1d / Gate3 M2 ratios • Gate1: M1b / Gate1 + M2b / Gate1 • Gate2: M1c / Gate2 + M2d / Gate2 • Gate3: M1d / Gate3 + M2e / Gate3 M3 ratios • Gate1: M1b / Gate1 + M2b / Gate1 + (M3b + M3c) / (Gate1 + Gate2) • Gate2: M1c / Gate2 + M2d / Gate2 + (M3b + M3c) / (Gate1 + Gate2) • Gate3: M1d / Gate3 + M2e / Gate3 + M3d / Gate3 M4 ratios • Gate1: M1b / Gate1 + M2b / Gate1 + (M3b + M3c) / (Gate1 + Gate2) + (M4a + M4b) / (Gate1 + Gate2 + Gate3) • Gate2: M1c / Gate2 + M2d / Gate2 + (M3b + M3c) / (Gate1 + Gate2) + (M4a + M4b) / (Gate1 + Gate2 + Gate3) • Gate3: M1d / Gate3 + M2e / Gate3 + M3d / Gate3 + (M4a + M4b) / (Gate1 + Gate2 + Gate3)
Accumulated-area mode	M1 ratios • Gate1: M1b / Gate1 • Gate2: M1c / Gate2 • Gate3: M1d / Gate3 M2 ratios • Gate1: (M1b + M2b) / Gate1 • Gate2: (M1c + M2d) / Gate2 • Gate3: (M1d + M2e) / Gate3 M3 ratios • Gate1, 2: (M1b + M1c + M2b + M2c + M2d + M3b + M3c) / (Gate1 + Gate2) • Gate3: (M1d + M2e + M3d) / Gate3 M4 ratios • Gate1, 2, 3: all metal areas / (Gate1 + Gate2 + Gate3)

Specifying Antenna Properties

In general, the antenna properties for standard cells and hard macros are defined in their frame views in the reference libraries. You can set default values for the antenna

Feedback

properties, which apply to cells that do not have antenna properties defined in the reference libraries.

- `route.detail.default_diode_protection`

  Specifies the diode protection value used for standard cell output pins during antenna analysis if the diode protection value is not specified in the e view of the cell.

- `route.detail.default_gate_size`

  Specifies the gate size used for standard cell input pins during antenna analysis if the gate size is not specified in the e view of the cell.

- `route.detail.default_port_external_antenna_area`

  Specifies the antenna area used for ports (top-level pins) during antenna analysis if the antenna area is not specified in the e view of the cell.

- `route.detail.default_port_external_gate_size`

  Specifies the gate size used for ports (top-level pins) during antenna analysis if the gate size is not specified in the e view of the cell.

- `route.detail.macro_pin_antenna_mode`

  Specifies how macro cell pins are treated for antenna considerations.

- `route.detail.port_antenna_mode`

  Specifies how the ports (top-level pins) are treated for antenna considerations.

If you are using a hierarchical flow and create a block abstraction for a block, you must use the `derive_hier_antenna_property` command to extract the antenna information from the block to use at the next level of hierarchy.

Note:

If the hierarchical antenna properties are not defined for all layers for a macro, Zroute treats the data as incomplete, skips antenna analysis, and issues a ZRT-311 warning message. If you get this error message, see SolvNet article 027178, “Debugging the ZRT-311 Message.”

See Also

- Annotating Antenna Properties on Hard Macro Cells

Feedback

Analyzing and Fixing Antenna Violations

If the design library contains antenna rules, Zroute automatically analyzes and fixes antenna violations. To disable the analysis and correction of antenna rules during detail routing, set the `route.detail.antenna` application option to `false`.

Just like other design rules, antenna rules are checked and corrected during detail routing. This concurrent antenna rule correction architecture reduces total runtime by minimizing the iterations.

By default, Zroute

- Checks antenna rules and corrects violations for all clock and signal nets

  To disable fixing of antenna violations on specific nets, set the `route.detail.skip_antenna_fixing_for_nets` application option. Note that Zroute analyzes the antenna rules and reports the antenna violations on these nets, but does not fix the violations.

- Does not check or correct antenna rules for power and ground nets

  To check and correct antenna rules for power and ground nets, set the `route.detail.check_antenna_on_pg` application option to `true`.

- Starts fixing antenna violations in the second iteration, after initial routing is complete and the basic DRC violations have been fixed

  To change the iteration in which Zroute starts fixing antenna violations, set the `route.detail.antenna_on_iteration` application option.

- Performs layer hopping to fix antenna violations

  Layer hopping decreases the antenna ratio by splitting a large metal polygon into several upper-level polygons. Zroute performs the following types of layer hopping:

  - Breaking the antenna with a higher-level metal segment

    Zroute uses this technique to fix most antenna violations. For antenna violations that happen at metal-N, inserting a small segment of metal-(N+1) close to the gate reduces the ratio between the remaining metal-N, making the ratio much lower. This approach is not suitable for fixing top-metal layer antenna violations when the output pin can provide only limited protection because there is no way for the router to break antenna violations at the topmost metal layer.

  - Moving down to a lower-level metal

    Zroute uses this technique to fix only topmost layer antenna violations when output pins provide only limited protection. For antenna violations that happen at metal-N,

Feedback

replace part of the metal-N with metal-(N-1 or lower) to reduce the ratio. However, splitting the metal layer into many pieces might have a negative impact on RC and timing delay.

Zroute can also insert diodes to fix antenna violations. To enable the insertion of diodes to fix antenna violations, set the `route.detail.insert_diodes_during_routing` application option to `true`. To force Zroute to fix antenna violations by inserting diodes, disable layer hopping by setting the `route.detail.hop_layers_to_fix_antenna` application option to `false`. For information about inserting diodes to fix antenna violations, see Inserting Diodes During Detail Routing.

If both layer hopping and diode insertion are enabled, by default, Zroute first tries to use layer hopping to fix the antenna violation. To change the preference to diode insertion, set the `route.detail.antenna_fixing_preference` application option to `use_diodes`.

To delete the redundant diodes that are not needed to fix antenna violations, set the `route.detail.delete_redundant_diodes_during_routing` application option to `true`. Spare diodes are not deleted as they do not connect to any nets. Port protection diodes that are inserted by detail route are not deleted.

As with other design rule violations, antenna violations are reported at the end of each detail routing iteration. For example,

```
DRC-SUMMARY:
   @@@@@@@ TOTAL VIOLATIONS =      506
    @@@@ Total number of instance ports with antenna violations =   1107
```

To check for antenna violations, use the `check_routes -antenna true` command.

Inserting Diodes During Detail Routing

One way to protect gates from antenna effects is to provide a discharge path for the accumulated charge to leave the net. However, the discharge path should not allow current to flow during normal chip operation. Discharging can be accomplished by inserting a reverse-biased diode on the net close to the gate that is being protected.

To enable diode insertion during detail routing, set the `route.detail.insert_diodes_during_routing` application option to `true`.

To control diode insertion, set the following application options:

- To specify a preference for fixing antenna violations by using diode insertion rather than layer hopping, set the `route.detail.antenna_fixing_preference` application option to `use_diodes`.
- To require fixing of antenna violations by using diode insertion, disable layer hopping by setting the `route.detail.hop_layers_to_fix_antenna` application option to `false`.

Feedback

- By default, when you enable diode insertion, Zroute can fix an antenna violation either by adding a new diode or by using an existing spare diode. Zroute determines which method to use, based on which is closest to the required location: an empty location for a new diode or an existing spare diode.
  - To specify a preference for using a new diode or using an existing spare diode, set the `route.detail.diode_preference` application option to `new` or `spare`, respectively. To reset the diode preference to the default behavior, set the `route.detail.diode_preference` application option to `none`.
  - If you want Zroute to use only one of these methods, set the `route.detail.diode_insertion_mode` application option to `new` to force the insertion of new diodes or to `spare` to force the use of existing spare diodes.

    To reset the diode insertion method to the default behavior, set the `route.detail.diode_insertion_mode` application option to `new_and_spare`.

  Note:

  To take advantage of spare diodes for antenna violation fixing, you need to add the spare diodes either before or after standard-cell placement and before routing the areas where antenna violations might occur.
- When inserting new diodes, Zroute selects the diodes from the reference libraries and inserts them into existing open spaces. To control which diodes are used, set the `route.detail.diode_libcell_names` application option.

  When you specify the diode library cells, use only the cell names. If you include the library name, the tool does not recognize the diode cells.
- By default, Zroute reuses existing filler cell locations for diode insertion.

  To prevent Zroute from reusing these locations, set the `route.detail.reuse_filler_locations_for_diodes` application option to `false`.

Zroute considers voltage areas when inserting diode cells and also observes the logic hierarchy assignments for diode cells.

- If a pin has an antenna violation, the diode cells are inserted at the same level of logic hierarchy as the violating pin.
- If a top-level port has an antenna violation, by default, the diode cells are inserted at the top level. However, if the port belongs to a voltage area, you can insert the diode cells in the logic hierarchy associated with the voltage area by setting the `route.detail.use_lower_hierarchy_for_port_diodes` application option to `true`.

To remove the redundant diodes during routing, set the `route.detail.delete_redundant_diodes_during_routing` application option to `true`.

Feedback

Inserting Diodes After Detail Routing

You can fix antenna violations after detail routing by explicitly specifying which violations to fix and providing constraints for fixing them. Based on the specified constraints and the setting of the `route.detail.diode_insertion_mode` application option, Zroute either inserts new diodes or reuses existing spare diodes to fix the specified violations.

To insert diodes after detail routing, use the `create_diodes` command. When you use this command, you must use the `-options` option to specify the location of each antenna violation to fix by specifying the port and cell instance, the reference cell for the diode, the number of diodes to insert, the highest allowed routing layer used for connecting the diode, and the maximum distance from the specified pin that the diode can be inserted. If a diode cannot be inserted or reused within the specified distance, the tool does not insert a diode for that violation.

Use the following format to specify these values:

```
{port_name instance_name diode_reference number_of_diodes
   max_routing_layer max_routing_distance}
```

Note:

You can use this command to insert diodes for top-level ports by specifying the name of the top-level block for the cell instance.

The `create_diodes` command uses the following application options to control diode insertion:

- To control whether Zroute inserts new diodes or reuses spare diodes, set the `route.detail.diode_insertion_mode` application option.
- To control whether Zroute can reuse filler cell locations, set the `route.detail.reuse_filler_locations_for_diodes` application option.
- To specify the logic hierarchy in which to insert the diodes for top-level ports, set the `route.detail.use_lower_hierarchy_for_port_diodes` application option.

For example, to insert 2 Adiode diode cells to fix an antenna violation on the A port of the CI cell instance, where the diodes must be inserted using no higher layer than M5 and within 2.5 microns of the port, enter the following command:

```
icc2_shell> create_diodes {{A CI Adiode 2 M5 2.5}}
```

Inserting Redundant Vias

Redundant via insertion is an important design-for-manufacturing (DFM) feature that is supported by Zroute throughout the routing flow. In each routing stage, Zroute concurrently optimizes via count as well as wire length. The redundant via result is measured by the redundant via conversion rate, which is defined as the percentage of single vias

Feedback

converted into redundant vias. You should also pay attention to the number of unoptimized single vias. If a block has fewer unoptimized single vias, it is usually better for DFM. The following topics describe how to insert redundant vias:

- Inserting Redundant Vias on Clock Nets
- Inserting Redundant Vias on Signal Nets

Inserting Redundant Vias on Clock Nets

Zroute can insert redundant vias on clock nets either during or after routing. This topic describes how to insert redundant vias during clock routing. For information about postroute redundant via insertion, see Postroute Redundant Via Insertion.

To insert redundant vias on clock nets during clock routing,

1. Specify the redundant vias in a nondefault routing rule by using the `create_routing_rule` command, as described in Specifying Nondefault Vias.

   Be sure to consider both DFM and routing when you select the redundant vias; otherwise, if you select the redundant vias based only on DFM considerations, you could negatively impact the routability.

   In addition to using nondefault routing rules to define the redundant vias for clock nets, you can also use them to define stricter wire width and spacing rules and to define the tapering distance. For more information about nondefault routing rules, see Using Nondefault Routing Rules.

2. Assign the nondefault routing rule to the clock nets by using the `set_clock_routing_rules` command.

   ```
   icc2_shell> set_clock_routing_rules -rules clock_via_rule
   ```

3. Run clock tree synthesis.

   ```
   icc2_shell> synthesize_clock_trees
   ```

4. Route the clock nets.

   ```
   icc2_shell> route_group -all_clock_nets \
      -reuse_existing_global_route true
   ```

   Zroute reserves space for the redundant vias during global routing and inserts the redundant vias during detail routing.

Feedback

Inserting Redundant Vias on Signal Nets

You can perform redundant via insertion in the following ways:

- Postroute redundant via insertion
- Concurrent soft-rule-based redundant via insertion
- Near 100 percent redundant via insertion

In general, you should start with postroute redundant via insertion. If postroute redundant via insertion results in a redundant via rate of at least 80 percent, you can try to improve the redundant via rate by using concurrent soft-rule-based redundant via insertion. If postroute redundant via insertion results in a redundant via rate of at least 90 percent, you can try to improve the redundant via rate by using near 100 percent redundant via insertion.

Note:

As the redundant via rate increases, it becomes more difficult to converge on the routing design rules and you might see a reduction in signal integrity; therefore, you should use near 100 percent redundant via insertion only for those blocks that truly require such a high redundant via rate. In addition, achieving very high redundant via rates might require you to modify the floorplan utilization to allow enough space for the redundant vias.

The following topics describe the default via mapping table, how to define a customized via mapping table, how to insert redundant vias by using various methods, and how to report the redundant via rate.

- Viewing the Default Via Mapping Table
- Defining a Customized Via Mapping Table
- Postroute Redundant Via Insertion
- Concurrent Soft-Rule-Based Redundant Via Insertion
- Near 100 Percent Redundant Via Insertion
- Preserving Timing During Redundant Via Insertion
- Reporting Redundant Via Rates

Feedback

Viewing the Default Via Mapping Table

By default, Zroute reads the default contact codes from the technology file and generates an optimized via mapping table. To see the default mapping table, use the `add_redundant_vias -list_only true` command.

In most cases you achieve better results if you use a customized mapping table rather than the default mapping table. For information about defining a customized mapping table, see Defining a Customized Via Mapping Table.

If you have not previously defined a customized mapping table for the block, you can see the default mapping table by using the `add_redundant_vias -list_only true` command.

Note:

After you have created a customized mapping table by using the method described in Defining a Customized Via Mapping Table, this command shows the customized mapping table.

Example 25 shows an example of a default via mapping table.

Example 25 Default Via Mapping Table

```
icc2_shell> add_redundant_vias -list_only
...
Redundant via optimization will attempt to replace the following vias:

   VIA12SQ_C     -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2    VIA12SQ_
C_1x2(r)
   VIA12SQ_C(r) -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2    VIA12SQ_
C_1x2(r)
...
   VIA89(r)      -> VIA89_C_1x2(r)  VIA89_1x2(r)  VIA89_1x2    VIA89_C_1x2
                    VIA89_C_2x1(r)  VIA89_2x1(r)  VIA89_2x1    VIA89_C_2x1
   VIA9RDL       -> VIA9RDL_2x1    VIA9RDL_1x2
```

Feedback

Defining a Customized Via Mapping Table

To define a customized via mapping, use the `add_via_mapping` command. At a minimum, you must specify the source via and its replacement vias by using the `-from` and `-to` options. The vias listed in these options must be either vias defined in the technology file or design-specific vias created by the `create_via_def` command. The vias listed in the `-from` option must be simple vias or via arrays. The vias listed in the `-to` option can be simple vias, simple via arrays, or custom vias. For information about creating design-specific vias, see Defining Vias.

Use the following options to refine the via mapping:

- `-weight`

  By default, all mappings have the same priority, and Zroute selects the redundant vias to use based on routability. To set the priority for a mapping, use the `-weight` option to assign an integer weight value between 1 and 10 to the mapping. If you do not assign a weight, the tool assigns a weight of 1. During redundant via insertion, Zroute uses the higher weighted redundant vias first.

- `-transform`

  By default, Zroute can rotate or flip the via arrays during redundant via insertion (the `-transform all` option).

  - To allow only rotation of the via arrays during redundant via insertion, use the `-transform rotate` option.
  - To allow only flipping of the via arrays during redundant via insertion, use the `-transform flip` option.
  - To allow only the specified orientation of the via array during redundant via insertion, use the `-transform none` option.

The tool saves the mappings defined by the `add_via_mapping` command in a via mapping table in the design library. By default, if you try to add a via mapping that already exists in the table, the command fails. To overwrite an existing mapping definition, use the `-force` option.

Example 26 shows an example of using the `add_via_mapping` command to define a customized via mapping table.

Example 26 Customized Via Mapping Table

```
icc2_shell> add_via_mapping \
   -from {VIA12 1x1} -to {VIA12T 2x1} -weight 5
icc2_shell> add_via_mapping \
   -from {VIA12 1x1} -to {VIA12 2x1} -weight 1
```

Feedback

To see the customized via mappings associated with a block, use the `report_via_mapping` command.

- By default, this command shows all user-defined via mappings.
- To show the via mappings for specific source vias, use the `-from` option.
- To show the via mappings for specific replacement vias, use the `-to` option.

To remove mappings from the via mapping table, use the `remove_via_mappings` command.

- To remove all via mappings, use the `-all` option.
- To remove the via mappings for specific source vias, use the `-from` option.
- To remove the via mappings for specific replacement vias, use the `-to` option.

Using a Subset of the Via Mapping Table for Redundant Via Insertion

To use a subset of the via mapping table for redundant via insertion, specify which weight groups from the via mapping table to use by setting one or both of the `route.common.redundant_via_include_weight_group_by_layer_name` and `route.common.redundant_via_exclude_weight_group_by_layer_name` application options.

- If you set only the `route.common.redundant_via_include_weight_group_by_layer_name` application option, Zroute selects redundant vias only from the specified weight groups. If you do not specify an entry for a layer, redundant via insertion is not performed on that layer.
- If you set only the `route.common.redundant_via_exclude_weight_group_by_layer_name` application option, Zroute selects redundant vias from all weight groups in the original via mapping table, except the specified weight groups. If you do not specify an entry for a layer, all weight groups are used for that layer.
- If you set both application options, Zroute selects redundant vias from the weight groups specified in the `route.common.redundant_via_include_weight_group_by_layer_name` application, excluding the weight groups specified in the `route.common.redundant_via_exclude_weight_group_by_layer_name` application option.

Postroute Redundant Via Insertion

To perform postroute redundant via insertion, use the `add_redundant_vias` command.

This command can replace single-cut vias with multiple-cut via arrays, single-cut vias with other single-cut vias that have a different contact code, and multiple-cut via arrays

Feedback

with different multiple-cut via arrays. During redundant via insertion, the detail router also checks the design rules within the neighboring partition to minimize DRC violations.

By default, the `add_redundant_vias` command inserts redundant vias on all nets. To insert redundant vias only on specific nets, use the `-nets` option to specify the nets. Using net-specific redundant via insertion allows you to further improve the optimized via rate without causing large-scale routing and timing changes.

After the vias are checked and replaced, the detail router rechecks for DRC violations and corrects any violations.

If the percentage of redundant vias is not high enough, you can increase the effort level by using the `-effort` option to get a better redundant via rate. Increasing the effort level to high can increase the redundant via rate by about 3 to 5 percent by shifting the vias to make room for additional vias. However, because high-effort redundant via insertion moves the vias more, it can result in a less lithography-friendly pattern at the 45-nm technology node and below. In this case, you should use concurrent soft-rule-based redundant via insertion to improve the redundant via rate.

You can also try to increase the postroute redundant via rate by setting the `route.detail.optimize_wire_via_effort_level` application option to `high`, which reduces the number of single vias and makes more room for redundant vias by reducing wire length.

After you perform the initial postroute redundant via insertion, set the `route.common.post_detail_route_redundant_via_insertion` application option to enable automatic insertion of redundant vias after subsequent detail routing or ECO routing. This helps to maintain the redundant via rate in your block.

Concurrent Soft-Rule-Based Redundant Via Insertion

Soft-rule-based redundant via insertion can improve the redundant via rate by reserving space for the redundant vias during routing. You can use concurrent soft-rule-based redundant via insertion during both initial routing and ECO routing. The actual via insertion is not done during routing; you must still perform postroute redundant via insertion by using the `add_redundant_vias` command.

Note:

Reserving space during routing increases the routing runtime. You should use this method only when needed to improve the redundant via rate beyond that provided by postroute redundant via insertion and the postroute approach resulted in a redundant via rate of at least 80 percent.

Feedback

To perform concurrent soft-rule-based redundant via insertion,

1. (Optional) Define the via mapping table as described in Defining a Customized Via Mapping Table.
2. Enable concurrent soft-rule-based redundant via insertion.

   By default, concurrent redundant via insertion is disabled.

   - To enable concurrent soft-rule-based redundant via insertion during initial routing, set the `route.common.concurrent_redundant_via_mode` application option to `reserve_space`.

     ```
     icc2_shell> set_app_options \
        -name route.common.concurrent_redundant_via_mode \
        -value reserve_space
     ```

     To control the effort used to reserve space for the redundant vias during initial routing, set the `route.common.concurrent_redundant_via_effort_level` application option. By default, Zroute uses low effort. The higher effort levels result in a better redundant via conversion rate at the expense of runtime. The low and medium efforts affect only global routing and track assignment, while high effort also affects detail routing, which can impact design rule convergence.

     Note:

     If you enable the `route.common.concurrent_redundant_via_mode` option before running the `place_opt` command, the redundant vias are considered during congestion estimation.

   - To enable concurrent soft-rule-based redundant via insertion during ECO routing, set the `route.common.eco_route_concurrent_redundant_via_mode` application option to `reserve_space`.

     ```
     icc2_shell> set_app_options \
        -name route.common.eco_route_concurrent_redundant_via_mode \
        -value reserve_space
     ```

     To control the effort used to reserve space for the redundant vias during ECO routing, set the `route.common.eco_route_concurrent_redundant_via_effort_level` application option.

     Note:

     Using concurrent soft-rule-based redundant via insertion during ECO routing can impact timing and design rule convergence. In general, you should use this method only when you used near 100 percent redundant via insertion during initial routing.

Feedback

3. Route the block.

   During routing, Zroute reserves space for the redundant vias and fixes hard design rule violations.

4. Perform postroute redundant via insertion.

   During postroute redundant via insertion, Zroute inserts the redundant vias in the reserved locations.

Near 100 Percent Redundant Via Insertion

You can achieve a redundant via rate near 100 percent by using hard-rule-based redundant via insertion. Hard-rule-based redundant via insertion can improve the redundant via rate by treating redundant vias as hard design rules during routing. You can use nearly 100 percent redundant via insertion only during initial routing; this method is not supported during ECO routing. When you use near 100 percent redundant via insertion during initial routing, you should use soft-rule-based redundant via insertion during ECO routing to preserve the redundant via rate achieved during initial routing.

Note:

This method can result in a very large runtime increase for congested blocks. You should use this method only when needed to improve the redundant via rate beyond that provided by concurrent soft-rule-based redundant via insertion and the soft-rule-based approach resulted in a redundant via rate of at least 90 percent.

To perform concurrent hard-rule-based redundant via insertion,

1. Enable nearly 100 percent via insertion by setting the `route.common.concurrent_redundant_via_mode` application option to `insert_at_high_cost`. (By default, concurrent redundant via insertion is disabled.)

   ```
   icc2_shell> set_app_options \
      -name route.common.concurrent_redundant_via_mode \
      -value insert_at_high_cost
   ```

   To control the effort used to reserve space for the redundant vias, set the `route.common.concurrent_redundant_via_effort_level` application option.

   Note:

   If you enable the `route.common.concurrent_redundant_via_mode` option before running the `place_opt` command, the redundant vias are considered during congestion estimation.

Feedback

2. Route the block.

   During routing, Zroute inserts the redundant vias and fixes hard design rule violations. In general, redundant via insertion has the same priority as other hard design rules; however, if design rule checking does not converge during detail routing, Zroute automatically relaxes the redundant via constraints to improve DRC convergence.

Preserving Timing During Redundant Via Insertion

When you insert redundant vias, it changes the timing of your block. Short nets tend to slow down due to increased capacitance, whereas long nets tend to speed up due to decreased resistance. Zroute redundant via insertion has a timing-preservation mode that allows you to perform redundant via insertion without affecting the block timing by preventing insertion of redundant vias on critical nets.

To enable timing-preservation mode for redundant via insertion, define the timing preservation constraints by using the following options of the `add_redundant_vias` command:

- `-timing_preserve_setup_slack_threshold`
- `-timing_preserve_hold_slack_threshold`
- `-timing_preserve_nets`

You should timing-preservation mode only at the end of the flow, after using the normal redundant via insertion flows, which converge both the redundant via rate and the timing QoR. Timing-preservation mode can slightly increase the redundant via rate while maintaining timing. However, if you use timing-preservation mode earlier in the flow, before timing is met, it might severely reduce the redundant via rate due to critical nets.

Reporting Redundant Via Rates

After redundant via insertion, whether concurrent or postroute, Zroute generates a redundant via report that provides the following information:

- The via conversion rate for nondefault vias

  The via conversion rate for nondefault vias is listed at the top of the report as the total optimized via conversion rate.

Feedback

- The optimized via conversion rate for each layer

  The optimized via conversion rate includes both double vias and DFM-friendly bar vias, which have a single cut but a larger metal enclosure. The tool reports two values for the via conversion rate:

  - The total optimized via conversion rate

    This value is computed based on the total via count, which includes both fixed vias and routed vias. Fixed vias are vias that cannot be optimized by the router, such as unrouted vias and user-defined vias.

  - The optimized via conversion rate based on the total routed via count

    This value is computed based only on the routed via count, which includes only those vias that can be optimized by the router.

  Note:

  The optimized via conversion rate is not useful if you are using bar vias.

- The distribution of optimized vias by weight for each layer

  To determine the via conversion rate for conversions above a certain weight, you must add the reported conversion rates for those weights. For example, in Example 27, the via conversion rate for weight 5 and above for layer V03 is 10.75+64.50=75.25%.

  Note:

  The conversion rate for unweighted vias is reported as “Un-optimized.”

- The total double via conversion rate for the block

Note:

The redundant via rate reported by the `report_design` command differs from the redundant via rate reported by Zroute during redundant via insertion or the `check_routes` command. This difference occurs because the `report_design` command reports the double via rate for both PG vias and signal vias, while Zroute reports the double via rate only for signal vias. In addition, Zroute bases the conversion rate only on the redundant via mapping.

Example 27 shows an example of the redundant via report.

Example 27 Redundant Via Report

```
Total optimized via conversion rate = 96.94% (1401030 / 1445268 vias)
Layer V01         = 41.89% (490617 / 1171301 vias)
    Weight 10     =  9.64% (112869  vias)
    Weight 5      = 32.25% (377689  vias)
    Weight 1      =  0.01% (59      vias)
    Un-optimized = 58.11% (680684  vias)
```

Feedback

```
Layer V02         = 76.20% (1567822/ 2057614 vias)
    Weight 10     = 43.51% (895270  vias)
    Weight 5      = 28.62% (588805  vias)
    Weight 1      =  4.07% (83747   vias)
    Un-optimized = 23.80% (489792  vias)
Layer V03         = 81.87% (687115 / 839297  vias)
    Weight 10     = 64.50% (541369  vias)
    Weight 5      = 10.75% (90224   vias)
    Weight 1      =  6.62% (55522   vias)
    Un-optimized = 18.13% (152182  vias)
Layer V04         = 81.60% (226833 / 277977  vias)
    Weight 10     = 81.45% (226418  vias)
    Weight 1      =  0.15% (415     vias)
    Un-optimized = 18.40% (51144   vias)
...
Layer V09         = 85.47% (1329   / 1555    vias)
    Weight 10     = 85.47% (1329    vias)
    Un-optimized = 14.53% (226     vias)

Total double via conversion rate    = 46.69% (2158006 / 4622189 vias)
```

Optimizing Wire Length and Via Count

During detail routing, Zroute optimizes wire length and via count in the areas where DRC violations occur; however, it does not optimize the layout in areas where no DRC violations occur.

To improve the manufacturing yield, use the `optimize_routes` command to perform standalone optimization of wire length and via count after performing detail routing and redundant via insertion.

By default, Zroute selects the nets to reroute based on the overall cost. For each selected net, Zroute determines whether to reroute all the shapes in the net or just a portion of them. To select the nets to reroute, use the `-nets` option to specify the nets. When you specify the nets to optimize, you can also use the `-reroute_all_shapes_in_nets` option to control whether Zroute must reroute all the associated net shapes.

By default, Zroute performs a maximum of 40 detail routing iterations to fix DRC violations that exist after the optimization. You can use the `-max_detail_route_iterations` option to control the maximum number of detail routing iterations.

Reducing Critical Areas

A critical area is a region of the block where, if the center of a random particle defect falls there, the defect causes circuit failure, thereby reducing yield. A conductive defect causes a short fault, and a nonconductive defect causes an open fault.

Feedback

The following topics describe how to

- Reduce critical area short faults by performing wire spreading
- Reduce critical area open faults by performing wire widening

Performing Wire Spreading

After you have performed detail routing and redundant via insertion, you can perform wire spreading to increase the average spacing between wires, which reduces the critical area short faults and therefore improves yield.

To perform wire spreading, use the `spread_wires` command. By default, the `spread_wires` command uses the following settings to spread the signal wires on the same layer:

To perform wire spreading, use the `spread_wires` command. By default, the `spread_wires` command uses the following settings to spread the signal wires on the same layer:

- Spreads the wires by half a pitch in the preferred direction

  To modify the spread distance, use the `-pitch` option. You specify the spread distance as a multiplier for the layer pitch. For example, to specify a spread distance of 1.5 times the layer pitch, use the following command:

  ```
  icc2_shell> spread_wires -pitch 1.5
  ```

- Uses twice the layer pitch as the minimum jog length

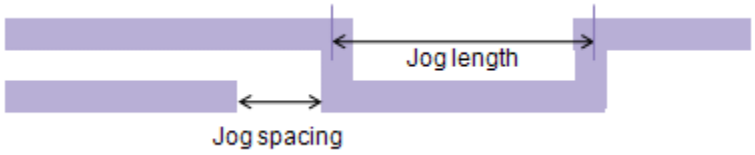
  To modify the minimum jog length, use the `-min_jog_length` option. You specify the minimum jog length as an integer multiple of the layer pitch.

- Uses the minimum layer spacing plus one half the layer pitch as the minimum jog length

  To modify the minimum jog spacing, use the `-min_jog_spacing_by_layer_name` option. You specify the minimum jog spacing in microns for each layer. The tool uses the default jog spacing for any unspecified layers.

Figure 88 shows how the jog length and jog spacing values are used in wire spreading.

Figure 88 Wire Spreading Results



Feedback

In the following example, the minimum jog length is set to three times the layer pitch, the minimum jog spacing for the M1 layer is set to 0.07 microns, and the minimum jog spacing for the M2 layer is 0.08 microns. All other metal layers use the default minimum jog spacing.

```
icc2_shell> spread_wires -min_jog_length 3 \
   -min_jog_spacing_by_layer_name {{M1 0.07} {M2 0.08}}
```

After spreading, the `spread_wires` command performs detail routing iterations to fix any DRC violations caused as a result of spreading.

When you change the layout, it can change the timing of your block. Wire spreading has a timing-preservation mode that allows you to perform wire spreading without affecting the block timing.

To enable timing-preservation mode for wire spreading, define the timing preservation constraints by using the following options with the `spread_wires` command:

- `-timing_preserve_setup_slack_threshold` *threshold*
- `-timing_preserve_hold_slack_threshold` *threshold*
- `-timing_preserve_nets` *nets*

The threshold values are floating-point numbers in library units. Wire spreading is performed only on nets with slack greater than or equal to the specified values or the nets specified in the `-timing_preserve_nets` option, as well as adjacent nets on the same layer within two routing pitches.

Performing Wire Widening

After you have performed detail routing, redundant via insertion, and wire spreading, you can perform wire widening to increase the average width of the wires, which reduces the critical area open faults and therefore improves yield.

To perform wire widening, use the `widen_wires` command. By default, the `widen_wires` command widens all wires in the block to 1.5 times their original width. For more flexibility, you can use the `-widen_widths_by_layer_name` option to define up to five possible wire widths to use for each layer. For example, to define possible wire widths of 0.07 and 0.06 microns for the M1 layer; wire widths of 0.08 and 0.07 microns for the M2 layer; and 1.5 times the existing wire width for all other layers, enter the following command:

```
icc2_shell> widen_wires \
   -widen_widths_by_layer_name {{M1 0.07 0.06} {M2 0.08 0.07}}
```

When you perform wire widening, the spacing between neighboring wires is decreased, which can reduce the improvement in critical area shorts gained from wire spreading. You can control the tradeoff between wire spreading and wire widening by using the

Feedback

`-spreading_widening_relative_weight` option. By default, wire spreading and wire widening are given equal priority. To weight the priority toward wire widening and reduced critical area open faults, set this option to a value between 0.0 and 0.5. To weight the priority toward wire spreading and reduced critical area short faults, set this option to a value between 0.5 and 1.0.

After widening, the `widen_wires` command performs detail routing iterations to fix any DRC violations caused as a result of widening. Note that the widened wires do not trigger fat wire spacing rules.

When you widen the wires, it changes the timing of your block. Wire widening has a timing-preservation mode that allows you to perform wire widening without affecting the block timing.

To enable timing-preservation mode for wire widening, define the timing preservation constraints by using the following options with the `widen_wires` command:

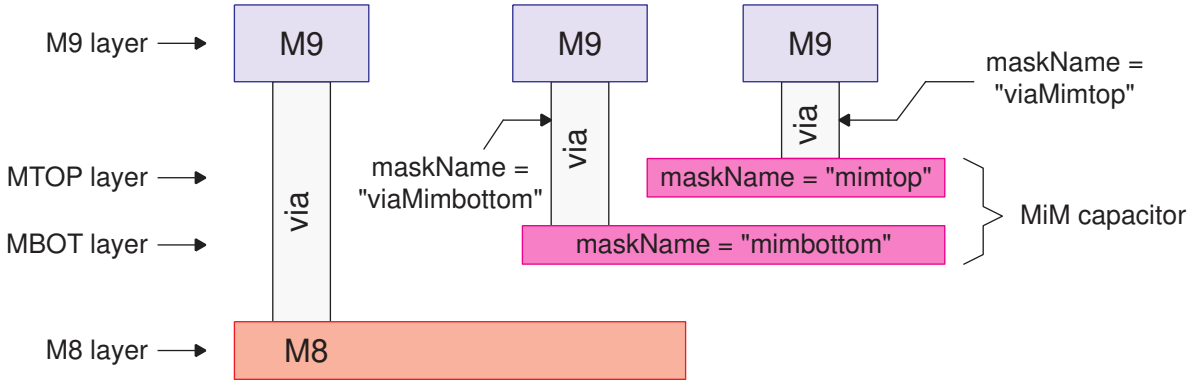
- `-timing_preserve_setup_slack_threshold` *`threshold`*
- `-timing_preserve_hold_slack_threshold` *`threshold`*
- `-timing_preserve_nets` *`nets`*

The threshold values are floating-point numbers in library units. Wire widening is not performed on nets with slack less than the specified values or the nets specified in the `-timing_preserve_nets` option.

Inserting Metal-Insulator-Metal Capacitors

A metal-insulator-metal (MiM) capacitor is made of two special-purpose conducting layers separated by an insulator. This capacitor is inserted between two regular metal layers, as shown in Figure 89.

Figure 89 Cross-Section View of MiM Capacitor Layers



Feedback

MiM capacitors are typically connected between power and ground to help maintain a constant supply voltage in the presence of electrical noise. The dimensions of a MiM capacitor are usually customized to fit the power strap geometry of a specific power plan.

To insert an array of MiM capacitors into a block and connect them to the power rails, use the `create_mim_capacitor_array` command. At a minimum, you must specify the MiM capacitor library cell, as well as the x- and y-increments of the array.

For example,

```
icc2_shell> create_mim_capacitor_array \
            -lib_cell my_lib/mim_ref_cell
            -x_increment 20 -y_increment 20
```

By default, the `create_mim_capacitor_array` command

- Inserts MiM capacitors for the entire block

  To restrict the insertion to a specific region, use the `-boundary` option to specify the rectangular or rectilinear region. You can specify only a single region.

  The command does not consider standard cells, macros, placement blockages, or voltage areas when inserting the MiM capacitors.

- Inserts the MiM capacitors in the R0 orientation

  To change the orientation, use the `-orientation` option. When you change the orientation, it affects all MiM capacitor cells in the inserted array.

- Uses the following naming convention for the inserted MiM capacitor cells:

  mimcap!*library_cell_name*!*number*

  To identify the MiM capacitor cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the tool uses the following naming convention:

  mimcap!*prefix*!*library_cell_name*!*number*

Inserting Filler Cells

To ensure that all power nets are connected, you can fill empty space in the standard-cell rows with filler cells. The IC Compiler II tool supports filler cells with and without metal and

Feedback

supports both single-height and multiheight filler cells. Filler cell insertion is often used to add decoupling capacitors to improve the stability of the power supply.

Note:

Before inserting filler cells, ensure that the block is legalized by using the `check_legality` command.

The IC Compiler II tool can select the filler cells to use based on

- An ordered list of filler cell references

  In the standard filler cell insertion flow, you specify an ordered list of filler library cells and the tool inserts the first cell that fits in each gap. If your technology requires specific filler cells abutting the standard cells, you can insert these required filler cells first, and then use standard filler cell insertion to fill the remaining space.

  For details about this method, see Standard Filler Cell Insertion.

- Threshold-voltage rules

  In the threshold-voltage-based flow, the tool selects the filler cells by using user-defined insertion rules, which are based the threshold-voltage types of the cells that border the gap. This flow is typically used only for established foundry nodes.

  For details about this method, see Threshold-Voltage-Based Filler Cell Insertion.

Standard Filler Cell Insertion

The standard filler cell insertion flow uses the `create_stdcell_fillers` command to insert filler cells in the block and the `remove_stdcell_fillers_with_violation` command to perform design rule checking on the filler cells and remove filler cells with violations.

During filler cell insertion, the `create_stdcell_fillers` command uses the placement legalizer to ensure that the inserted filler cells honor advanced node placement rules and physical constraints such as placement blockages and keepout margins. When the `place.legalize.enable_advanced_legalizer` application option is set to `true`, the command uses the advanced legalization algorithms for 2D rule checking and cell interaction, which can reduce filler cell insertion runtime.

To insert filler cells in your block,

1. Ensure that the block is legalized by using the `check_legality` command.

   When verifying that a block is fillable, the `check_legality` command assumes that all standard cells that have a `design_type` attribute of `filler` can be used as filler cells. However, the `create_stdcell_fillers` command inserts only the cells specified in

Feedback

the -lib_cells option. This difference might result in unfillable gaps after filler cell insertion even though the check_legality command did not report any issues.

2. (Optional) Insert required filler cells to abut specific standard cells by using the create_left_right_filler_cells command, as described in Abutting Standard Cells With Specific Filler Cells.

3. (Optional) Enable multithreaded filler cell insertion by using the set_host_options command to specify the multicore configuration.

```
icc2_shell> set_host_options -max_cores n
```

The create_stdcell_fillers command uses the multicore configuration only when the technology file contains minimum threshold voltage, oxide-diffusion (OD), or trimpoly (TPO) rules.

Note:

The multicore configuration specified with the set_host_options command applies to all IC Compiler II commands that support multicore processing.

4. Insert metal filler cells by using the create_stdcell_fillers command.

Use the -lib_cells option to specify the metal filler library cells to insert. The command tries to insert the filler cells in the order that you specify; for the best results, specify them from the largest to the smallest.

To determine the filler cells available in a cell library, use the following command:

```
icc2_shell> get_lib_cells ref_lib/* -filter "design_type==filler"
```

To sort the filler cells by decreasing size, use the sort_collection command, as shown in the following example:

```
icc2_shell> set FILLER_CELLS \
   [get_object_name [sort_collection -descending \
      [get_lib_cells ref_lib/* -filter "design_type==filler"] area]]
```

For more information, see Controlling Standard Filler Cell Insertion.

5. Connect the inserted filler cells to the power and ground (PG) network by using the connect_pg_net -automatic command.

6. Remove the metal filler cells with DRC violations by using the remove_stdcell_fillers_with_violation command.

Removing the filler cells can expose new violations; therefore, you must sometimes run this command multiple times to remove all violating filler cells.

For more information, see Checking for Filler Cell DRC Violations.

Feedback

7. Insert nonmetal filler cells by using the `create_stdcell_fillers` command.

   Use the `-lib_cells` option to specify the nonmetal filler library cells to insert. The tool tries to insert the filler cells in the order that you specify; for the best results, specify them from the largest to the smallest.

   For more information, see Controlling Standard Filler Cell Insertion.

8. Connect the inserted filler cells to the PG network by using the `connect_pg_net -automatic` command.

See Also

- Generic ECO Flow for Timing or Functional Changes
- Removing Filler Cells

Controlling Standard Filler Cell Insertion

By default, the `create_stdcell_fillers` command fills all empty space in the horizontal standard-cell rows of the entire block by inserting instances of the filler library cells specified by the `-lib_cells` option.

You can control the following aspects of the filler cell insertion:

- The selection of filler cells based on lowest leakage current

  By default, the command does not consider leakage current when inserting filler cells. To reduce the leakage current, use the `-leakage_vt_order` option to specify the threshold voltage layers in order of decreasing leakage current. The command uses this information to select the filler cells with the lowest leakage current that meet the legalization requirements.

  To check whether the filler cells inserted in the current block result in the lowest possible leakage power, use the `-leakage_vt_check` option with the `create_stdcell_fillers` command. When you use this option, the command only checks the existing filler cells; it does not insert filler cells. You must also use the `-lib_cells` and `-leakage_vt_order` options to specify the available filler cells. The check reports the filler cells that could be replaced with a specified filler cell with lower leakage current without causing legalization errors. By default, the command reports a maximum of 100 violations. To change the maximum number of reported violations, set the `chf.create_stdcell_fillers.max_leakage_vt_order_violations` application option.

Feedback

Note:

This feature has the following requirements:

- You must enable the advanced legalizer by setting the `place.legalize.enable_advanced_legalizer` application option to `true`. For more information about the advanced legalizer, see Enabling Advanced Legalization Algorithms.
- The threshold voltage layers must be defined in the technology file. A threshold voltage layer is identified by a `maskType` attribute of `implant` in the `Layer` section of the technology file.

- The percentage of empty space to fill

  By default, the command fills all the empty space. To leave some empty space, use the `-utilization` option to specify the percentage of empty space to fill.

  To control the relative amount of various types of filler cells, such as ULVT and LVT decoupling capacitor cells, use the `-type_utilization` option to specify the insertion percentage for each type of cell. The sum of all the percentages specified in this option must be less than or equal to 100. When you use this option, the `create_stdcell_fillers` command might leave some empty spaces. To fill these empty spaces, use the `-fill_remaining` option. When you use this option, the command uses the cells specified in the `-lib_cells` option but not the `-type_utilization` option to fill the empty spaces. For example,

```
# insert 70 percent ULVT cells, 30 percent LVT cells
icc2_shell> create_stdcell_fillers -lib_cells $FILLER_CELLS \
   -type_utilization { {*/DCAP16*ULVT */DCAP8*ULVT */DCAP4*ULVT
      */DCAP2*ULVT */DCAP1*ULVT} 70
                        {*/DCAP16*LVT */DCAP8*LVT */DCAP4*LVT
      */DCAP2*LVT */DCAP1*LVT} 30 } \
   -fill_remaining
```

  By default, the tool randomly selects the filler cells from the specified library cells. To control the cell selection priority, use the `-prefer_type_ordering` option with the `-type_utilization` option. When you use the `-prefer_type_ordering` option, the tool selects the library cells in each group specified in the `-type_utilization` option in priority order, from left to right. If some of the cells are difficult to insert due to legalization rules or library cell size, specify them before cells that are easier to insert to

Feedback

ensure that they are selected. For example, when the `-prefer_type_ordering` option is used with the previous example, the tool

- First tries to insert the ULVT decoupling capacitors to a utilization of 70 percent, first trying to insert */DCAP16*ULVT cells, then */DCAP8*ULVT cells, and so on
- Then tries to insert the LVT decoupling capacitors to a utilization of 30 percent, first trying to insert */DCAP16*LVT cells, then */DCAP8*LVT cells, and so on
- Then filling the remaining gaps using the remaining cells specified in the -lib_cells option

```
# insert 70 percent ULVT cells, 30 percent LVT cells
icc2_shell> create_stdcell_fillers -lib_cells $FILLER_CELLS \
   -type_utilization { {*/DCAP16*ULVT */DCAP8*ULVT */DCAP4*ULVT
      */DCAP2*ULVT */DCAP1*ULVT} 70
                     {*/DCAP16*LVT */DCAP8*LVT */DCAP4*LVT
      */DCAP2*LVT */DCAP1*LVT} 30 } \
   -prefer_type_ordering \
   -fill_remaining
```

- The region in which to insert the filler cells
  - Use the `-bboxes` option to restrict filler cell insertion to the specified bounding boxes.
  - Use the `-voltage_area` option to restrict filler cell insertion to the specified voltage areas.
- Whether to check for power net violations

  By default, the command does not check for power net violations, as this check increases runtime and is required only for metal filler cell insertion. However, this might result in the `remove_stdcell_fillers_with_violation` command removing many decoupling capacitors.

  To prevent power net violations caused by filler cell insertion, use the `-rules check_pnet` option.
- Gap prevention

  By default, the command assumes that the smallest cell size is one unit site. If the smallest cell in your design is larger than one unit site, this could cause gaps during filler cell insertion. To prevent the command from inserting filler cells that leave a gap, specify the smallest cell size as a multiple of the unit site by using the `-smallest_cell_size` option. You can specify a value of 1, 2, or 3.

Feedback

Note:

The advanced legalizer automatically prevents gaps; therefore, this option is ignored when `place.legalize.enable_advanced_legalizer` application option is set to `true`.

- The handling of hard placement blockages

  By default, the command honors hard placement blockages and does not insert filler cells in regions affected by these blockages. To ignore the hard placement blockages and insert filler cells in those regions, use the `-ignore_hard_blockages` option.

- The naming convention used to identify the inserted filler cell instances

  By default, the command uses the following naming convention for inserted filler cells:

  xofiller!*filler_library_cell_name*!*number*

  To identify the filler cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the command uses the following naming convention:

  xofiller!*prefix*!*filler_library_cell_name*!*number*

- The behavior when the legalizer detects errors

  By default, the command fails if the legalizer detects an error. To force the command to complete filler cell insertion even when errors occur, use the `-continue_on_error` option. When you use this option, the resulting block might have legalization errors that must be fixed manually.

Checking for Filler Cell DRC Violations

By default, the `remove_stdcell_fillers_with_violation` command checks for DRC violations between all cell instances with *xofiller* in their name and top-level signal routing objects and removes the violating filler cells.

You can control the following aspects of the filler cell checking:

- The string used to identify the filler cells

  To specify a different string to identify the filler cells, use the `-name` option.

- The region in which to check for violations

  To restrict filler cell check to specific regions, use the `-boundary` option.

Feedback

- The checked objects

  To check for violations between filler cells and all neighboring objects, such as signal net shapes, other filler cells, standard cells, PG rails, and terminals, use the `-check_between_fixed_objects true` option.

- Restrict DRC to shorts-checking only

  By default, the command removes all filler cells that cause a routing DRC violation. To remove only those filler cells that cause a short by overlapping a routing shape, use the `-shorts_only true` option.

- Whether double-patterning rules are checked

  The command checks for double-patterning violations only when the following criteria are met:

  - The `route.common.color_based_dpt_flow` application option is `true`.
  - The technology file defines double-patterning rules

To check for DRC violations on filler cells before removing them, use the `-check_only true` option. When you use this option, this command writes a report to a file named *block*_fillers_with_violation.rpt in the current working directory. The report lists the filler cells with violations and reports the first violation for each filler cell.

Fixing Remaining Mask Design Rule Violations

Some mask design rules are not handled during filler cell insertion because they have a low probability of occurrence and they require extensive computation time if handled during insertion.

After filler cell insertion, use the `replace_fillers_by_rules` command to fix these design rule violations. You must specify the rule to fix by using the `-replacement_rule` option. Table 35 lists the design rules supported by this command and the keywords used to specify these rules.

Table 35 Design Rules Supported by the replace_fillers_by_rules Command

Design rule	Description	-replacement_rule keyword
End orientation	This rule allows the leftmost or rightmost filler cell to be flipped to a different orientation. For more information, see End Orientation Rule.	`end_orientation`
Half row adjacency for half-height filler cells	This rule restricts the placement of half-height filler cells. For more information, see Half Row Adjacency Rule.	`half_row_adjacency`

Feedback

Table 35 Design Rules Supported by the replace_fillers_by_rules Command (Continued)

Design rule	Description	-replacement_rule keyword
Illegal abutment	This rule prevents a filler cell from abutting certain other filler cells. For more information, see Illegal Abutment Rule.	`illegal_abutment`
Maximum horizontal length	This rule restricts the horizontal length of a continuous row of filler cells. For more information, see Maximum Horizontal Length Rule.	`od_horiztonal_distance`
Maximum horizontal edge length	This rule restricts the horizontal length of a sequence of abutted objects on a specific layer. For more information, see Maximum Horizontal Edge Length Rule.	`horizontal_edges_distance`
Maximum stacking for small filler cells	This rule restricts the horizontal length of a sequence of abutted objects on a specific layer. For more information, see Maximum Stacking for Small Filler Cells Rule.	`small_filler_stacking`
Maximum vertical edge length	This rule restricts the vertical length of a column of stacked filler cells. For more information, see Maximum Vertical Edge Length Rule.	`max_vertical_constraint`
Tap cell spacing	This rule restricts the distance between a tap cell's oxide diffusion (OD) layer and the neighboring OD layers. For more information, see Tap Cell Spacing Rule.	`od_tap_distance`
N/A	Randomly replace filler cells with custom filler cells. For more information, see Random Filler Cell Replacement.	`random`

End Orientation Rule

The end orientation rule searches for a continuous horizontal sequence of cells and modifies the orientation of the leftmost or rightmost filler cells if they are in the specified list of target filler cells.

To select this rule, use the `-replacement_rule end_orientation` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The target filler cells

  To specify the target filler cells, use the `-target_fillers` option.

Feedback

- The orientation modification
  - To change the orientation for the leftmost filler cells, use the `-left_end` option.
  - To change the orientation for the rightmost filler cells, use the `-right_end` option.
  - To change the orientation for both the leftmost and rightmost filler cells, use both options.

  The valid values for these options are
  - `inverse`, which flips the cell from its existing orientation
  - `R0_or_MX`, which changes an MY orientation to R0 or an R180 orientation to MX
  - `MY_or_R180`, which changes an R0 orientation to MY or an MX orientation to R180

To report the cells whose orientation would be changed, without actually changing the orientations, use the `-check_only` option. This option is valid only for the end orientation rule.

Half Row Adjacency Rule

The half row adjacency rule searches for half-height filler cells and replaces them based on the types of their neighboring standard cells. The standard cells are classified as one of the following types:

- Inbound

  The top or bottom boundary of an inbound standard cell is at the half-height location of the cell rows. When a half-height filler cell abuts an inbound cell, the abutting side of the half-height filler cell must have inbound-cell interaction polygons (ICIP).

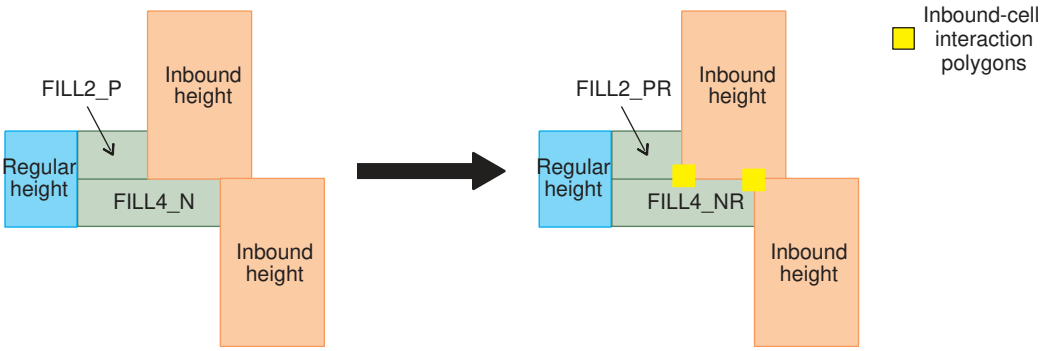
- Regular

  The top and bottom boundaries of a regular standard cell are at the full-height location of the cell rows. When a half-height filler cell abuts a regular cell, the abutting side of the half-height filler cell must not have inbound-cell interaction polygons (ICIP).

The replacement filler cells have the same height, width, and threshold voltage as the original filler cells. The cell libraries include both p-type and n-type replacement filler cells with inbound-cell interaction polygons (ICIP). For p-type half-height filler cells, the inbound-cell interaction polygons (ICIP) are at the bottom corners. For n-type half-height filler cells, the inbound-cell interaction polygons (ICIP) are at the top corners. Figure 90 shows the replacement of the half-height filler cells using the half row adjacency rule.

Feedback

Figure 90 Half-Height Filler Cell Replacement With Half Row Adjacency Rule



To select this rule, use the `-replacement_rule half_row_adjacency` option with the `replace_fillers_by_rules` command. The following table lists the options used to specify the replacement half-height filler cells. Each list can contain only one cell of each size.

Table 36 Half-Height Replacement Filler Cells

Abutting inbound standard cells	Option to specify replacement filler cells
None	`-p_none`, `-n_none`
Left-only	`-p_left`, `-n_left`
Right-only	`-p_right`, `-n_right`
Left and right	`-p_left_right`, `-n_left_right`
Top or bottom (applies only to 2X filler cells)	`-p_left_center_right`, `-n_left_center_right`

Feedback

Illegal Abutment Rule

The illegal abutment rule restricts the filler cells that can abut certain other filler cells.

To select this rule, use the `-replacement_rule illegal_abutment` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The filler cells considered for the check and their replacements

  The filler cells considered for the check are called *target cells*. To specify the target filler cells and their replacements, use the following syntax with the `-replace_abutment` option:

  `{ {`*target_cell*$_1$ *refill_cell*$_1$`} ... }`

  To ignore filler cells that have a `physical_status` attribute of `fixed` or `locked`, use the `-skip_fixed_cells` option with the `replace_fillers_by_rules` command.

- The filler cells that cannot abut the target filler cells

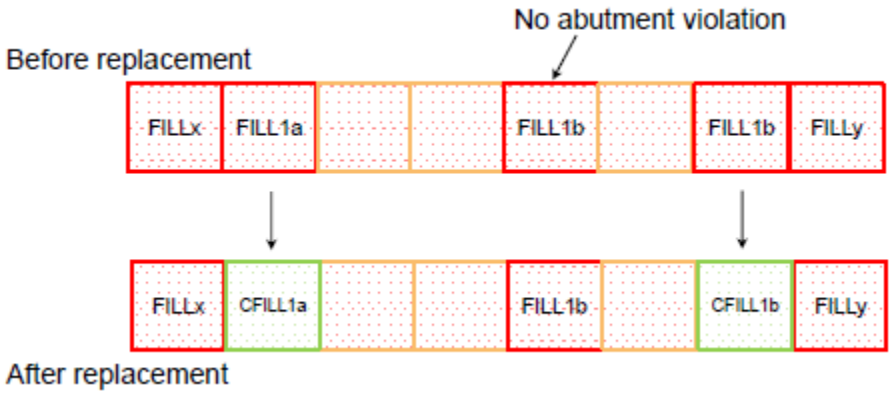
  These filler cells are called *illegal cells*. To specify the illegal cells, use the `-illegal_abutment` option. Note that the target filler cells specified in the `-replace_abutment` option are also considered illegal cells.

For example, the following command fixes illegal abutment violations where the FILL1a and FILL1b filler cells cannot abut the FILLx and FILLy cells:

```
icc2_shell> replace_fillers_by_rules -replacement_rule illegal_abutment \
   -replace_abutment { {FILL1a CFILL1a} {FILL1b CFILL1b} } \
   -illegal_abutment {FILLx FILLy}
```

Figure 91 shows the original row, which violates this rule, and the resulting row, which uses the replacement filler cells to fix the violations. The target filler cells are shown in red, while the replacement filler cells are shown in green.

Figure 91 Fixing an Illegal Abutment Violation



Feedback

Maximum Horizontal Edge Length Rule

The maximum horizontal edge length rule restricts the horizontal length of a sequence of abutted objects on a specific layer.

To select this rule, use the `-replacement_rule horizontal_edges_distance` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The maximum horizontal length

  To specify the maximum length, use the `-max_constraint_length` option.

- The layer to check

  To specify the layer, use the `-layer` option.

- The cells considered for the check and the filler cells that can be used to replace the violating filler cells

  The filler cells considered for the check are called *constrained cells*. The filler cells considered to replace violating cells are called *unconstrained cells*.

  - To specify multiple constrained cell groups and their replacement cells, use the following syntax with the `-refill_table` option:

    `{ {{`*refill_cell*$_1$ `{`*constrained_cells*$_1$`}} ... }`

    You can use this syntax to fix violations for threshold-voltage-based filler cells.

  - To specify a single constrained cell group, use the `-constraint_fillers` option. To specify the replacement cells, use the `-non_constraint_fillers` option.

    Note:

    If you use both methods, the command uses only the information specified in the `-refill_table` option.

For example, the following command identifies and fixes maximum horizontal edge length violations where the horizontal length on the M2 layer exceeds 30 um:

```
icc2_shell> replace_fillers_by_rules \
   -replacement_rule horizontal_edges_distance \
   -max_constraint_length 30 -layer M2 \
   -refill_table { {{VT1_FILL1} {VT1_DECAP4 VT1_DECAP2}}
                   {{VT2_FILL1} {VT2_DECAP4 VT2_DECAP2}} }
```

Feedback

Maximum Horizontal Length Rule

The maximum horizontal length rule restricts the horizontal length of a sequence of abutted filler cells.

To select this rule, use the `-replacement_rule od_horiztonal_distance` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The maximum horizontal length

  To specify the maximum length, use the `-max_constraint_length` option.

- The cells considered for the check and the filler cells that can be used to replace the violating filler cells

  The filler cells considered for the check are called *constrained cells*. The filler cells considered to replace violating cells are called *unconstrained cells*.

  - To specify multiple constrained cell groups and their replacement cells, use the following syntax with the `-refill_table` option:

    `{ {{`*refill_cell*$_1$ `{`*constrained_cells*$_1$`}} ... }`

    You can use this syntax to fix violations for threshold-voltage-based filler cells.

  - To specify a single constrained cell group, use the `-constraint_fillers` option. To specify the replacement cells, use the `-non_constraint_fillers` option.

    Note:

    If you use both methods, the command uses only the information specified in the `-refill_table` option.

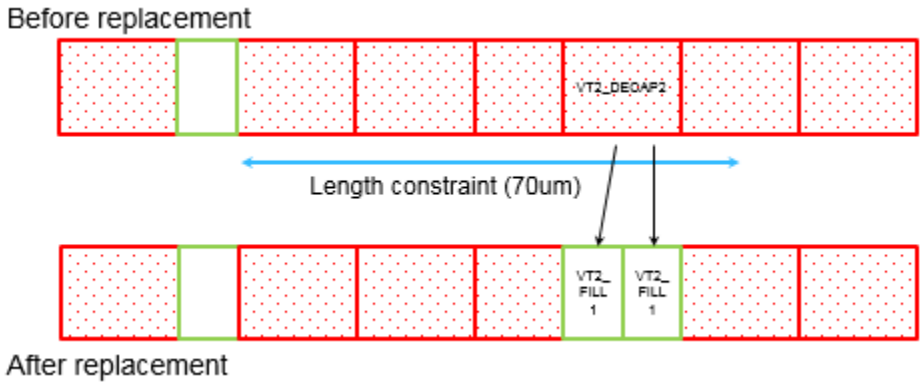
For example, the following command identifies and fixes maximum horizontal length violations where the horizontal length exceeds 70 um:

```
icc2_shell> replace_fillers_by_rules \
   -replacement_rule od_horiztonal_distance -max_constraint_length 70 \
   -refill_table { {{VT1_FILL1} {VT1_DECAP4 VT1_DECAP2}}
                   {{VT2_FILL1} {VT2_DECAP4 VT2_DECAP2}} }
```

Figure 92 shows the original row, which violates this rule, and the resulting row, which uses unconstrained filler cells to fix the violation. The constrained cells are shown in red, while the unconstrained cells are shown in green.

Feedback

Figure 92 Fixing a Maximum Vertical Edge Length Violation



Maximum Stacking for Small Filler Cells Rule

The maximum stacking for small filler cells rule restricts the stacking height of small filler cells.

To select this rule, use the `-replacement_rule small_filler_stacking` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The maximum stacking height

  To specify the maximum stacking height, use the `-max_constraint_length` option.

- The cells considered for the check

  To specify the small filler cells to consider for the check, use the `-small_fillers` option.

- The filler cells that can be used to replace the violating filler cells

  To specify the filler cells that can be used to replace the violating filler cells, use the `-replacement_fillers` option.

If the vertical stacking of the small filler cells specified in the `-small_fillers` option exceeds the height specified in the `-max_constraint_length` option, the command replaces two or three consecutive filler cells adjacent to the violating cell with one of the large filler cells specified in the `-replacement_fillers` option. If it is not possible to replace the filler cells, the command reorders the filler cells to fix the violation. The command maintains the threshold voltage at the location of the violating filler cell; however, the threshold voltage at the location of non-violating filler cells might change. The command does not check for threshold voltage violations caused by these changes.

Feedback

For example, the following command identifies and fixes maximum stacking violations where the stacking height exceeds 10 um:

```
icc2_shell> replace_fillers_by_rules \
   -replacement_rule small_filler_stacking -max_constraint_length 10 \
   -small_fillers {FILL1 FILL2} -replacement_fillers {FILL3 FILL4} \
```

The following figures illustrate the replacement and reordering techniques used to fix maximum stacking violations. In these figures, the small filler cells are shown in red, while the large filler cells are shown in green.

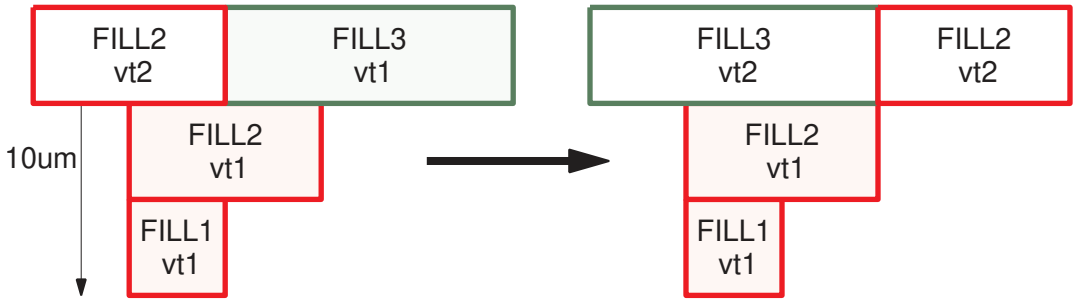
Figure 93 shows the original stack, which violates this rule, and the resulting stack, which uses a replacement filler cell to fix the violation.

Figure 93 Fixing Maximum Stacking Violation by Replacement



Figure 94 shows the original stack, which violates this rule, and the resulting stack, which uses reordering to fix the violation. Note that the threshold voltage of the FILL3 cell is changed so that the vt2 threshold voltage is maintained in the original location.

Figure 94 Fixing Maximum Stacking Violation by Reordering



Feedback

Maximum Vertical Edge Length Rule

The maximum vertical edge length rule restricts the vertical length of a stack of certain filler cells and standard cells.

To select this rule, use the `-replacement_rule max_vertical_constraint` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The maximum vertical length

  To specify the maximum vertical length, use the `-max_constraint_length` option.

- The cells considered for the check

  These cells are called *constrained cells*. To specify the constrained filler cells, use the `-constraint_fillers` option. By default, all standard cells are constrained. To exclude specific standard cells from consideration, use the `-exception_cells` option. To ignore the length violation when a constrained cell abuts an exception cell, use the `-no_violation_along_exception_cells` option.

- The filler cells that can be used to replace the violating filler cells

  These filler cells are called *unconstrained cells*; they are not considered when measuring the stack height.

  - To specify the filler cells that can be used to break a continuous edge of constrained cells, use the `-non_constraint_fillers` option.
  - To specify the filler cells whose left side can be used to break a continuous edge of constrained cells, use the `-non_constraint_left_fillers` option.
  - To specify the filler cells whose right side can be used to break a continuous edge of constrained cells, use the `-non_constraint_right_fillers` option.

Note:

Any filler cells that are not identified as constrained, unconstrained, or exception cells are considered constrained cells.

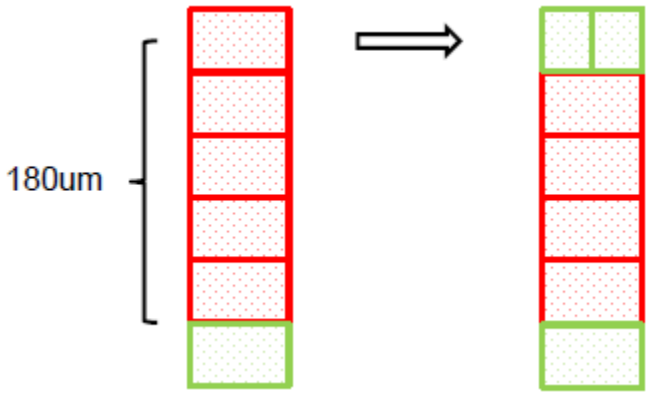
For example, the following command identifies and fixes maximum vertical edge length violations where the vertical length exceeds 180 um:

```
icc2_shell> replace_fillers_by_rules \
   -replacement_rule max_vertical_constraint -max_constraint_length 180 \
   -constraint_fillers {DECAP8 DECAP4} -non_constraint_fillers {FILL1} \
   -exception_cells {BUF1 TAP1}
```

Figure 95 shows the original stack, which violates this rule, and the resulting stack, which uses unconstrained filler cells to fix the violation. The constrained cells are shown in red, while the unconstrained cells are shown in green.

Feedback

Figure 95 Fixing a Maximum Vertical Edge Length Violation



Tap Cell Spacing Rule

The tap cell spacing rule restricts the distance between a target tap cell's oxide diffusion (OD) layer and the OD layers of its neighboring cells on the left and right sides.

To select this rule, use the `-replacement_rule od_tap_distance` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The disallowed spacing range

  The spacing distance is specified as the number of sites. To specify the disallowed spacing range, use the `-tap_distance_range` option.

- The tap cells considered for the check

  These cells are called *target cells*. To specify the target tap cells, use the `-tap_cells` option.

- The tap cells used to replace the violating tap cells

  These cells are called *replacement cells*.

  - To specify the replacement tap cell for left-side violations, use the `-left_violation_tap` option.
  - To specify the replacement tap cell for right-side violations, use the `-right_violation_tap` option.
  - To specify the replacement tap cell for both-side violations, use the `-both_violation_tap` option.

Note:

The target and replacement tap cells must all have the same size.

Feedback

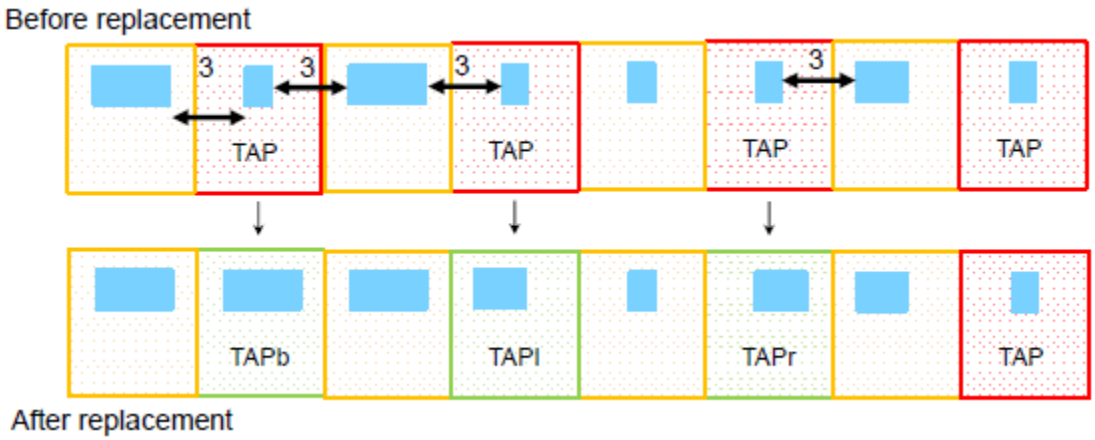
By default, the command measures the distance to the cells that abut the target tap cell. If the tap cells abut library cells that do not have an OD layer, use the `-adjacent_non_od_cells` option to specify these library cells. The command skips the specified non-OD cells and measures to distance from the target tap cell to the first neighbor with an OD layer.

For example, the following command replaces tap cells named TAP that have a separation distance of three sites:

```
icc2_shell> replace_fillers_by_rules -replacement_rule od_tap_distance \
   -tap_cells {TAP} -tap_distance_range {3 3} \
   -left_violation_tap {TAPl} -right_violation_tap {TAPr} \
   -both_violation_tap {TAPb}
```

Figure 96 shows the original row, which violates this rule, and the resulting row, which uses the replacement filler cells to fix the violations. The target filler cells are shown in red, while the replacement filler cells are shown in green.

Figure 96 Fixing a Tap Cell Spacing Violation



Random Filler Cell Replacement

To randomly replace existing filler cells, use the `-replacement_rule random` option with the `replace_fillers_by_rules` command. When you select this rule, you must specify

- The filler cells to be replaced and their replacements

  The filler cells to be replaced are called *target cells*. To specify the target filler cells and their replacements, use the following syntax with the `-random_replace` option:

  { {*target_cell*$_1$ {*refill_cell*$_{1a}$ *refill_cell*$_{1b}$ ...}} ... }

  To ignore filler cells that have a `physical_status` attribute of `fixed` or `locked`, use the `-skip_fixed_cells` option with the `replace_fillers_by_rules` command.

Feedback

When the command finds one of the target cells, it replaces it by randomly selecting one of the replacement cells specified for that target cell. For example, the following command replaces FILL1 cells with one of FILL1a, FILL1b, or FILL1c and replaces FILL2 cells with one of FILL2a or FILL2b:

```
icc2_shell> replace_fillers_by_rules -replacement_rule random \
   -random_replace { {FILL1 {FILL1a FILL1b FILL1c}}
                     {FILL2 {FILL2a FILL2b}} }
```

Abutting Standard Cells With Specific Filler Cells

If your technology requires specific filler cells on the left or right sides of certain standard cells, use the `create_left_right_filler_cells` command to insert these filler cells. After you run the `create_left_right_filler_cells` command, perform standard filler cell insertion to fill the remaining gaps in the design.

You must use the `-lib_cells` option to specify the rules for filler cell insertion. Each rule uses the following format:

{*standard_cells*} {*left_filler_cells*} {*right_filler_cells*}

- For the *standard_cells* argument,
  - You must specify a list of one or more standard cells
  - You can use wildcards to specify the cell names
  - If you specify multiple rules for the same standard cell, the command uses only the last rule specified for that cell.
- For the *left_filler_cells* and *right_filler_cells* arguments,
  - At least one of these arguments must contain one or more filler library cells

    If only one of the arguments contains filler library cells, the tool inserts filler cells only on that side of the standard cells.
  - The specified filler cells must meet the following requirements:
    - They must have a `design_type` attribute of `lib_cell` or `filler`.
    - They must use the same site definition as the standard cells.
    - The height of the standard cells must be an integer multiple of the filler cell heights.
  - The command tries to insert the filler cells in the order that you specify; for the best results, specify them from the largest to the smallest.

Feedback

- You can use wildcards for the filler cell names.

  When you use wildcards, the order of the cells is the same as that returned by the `get_lib_cells` command.

The following example inserts filler cells on the left and right sides of the SC1 standard cells and only on the left side of the SC2 standard cells. The left and right filler cells are specified in decreasing size order, so that the command inserts the largest possible filler cell.

```
icc2_shell> create_left_right_filler_cells \
   -lib_cells {
      { {mylib/SC1} {mylib/LF4X mylib/LF2X} {mylib/RF2X mylib/RF1X} }
      { {mylib/SC2} {mylib/LF2X mylib/LF1X} {} } }
```

When the `create_left_right_filler_cells` command inserts the filler cells,

- It checks for legal locations for the filler cells, but does not check advanced rules such as minimum threshold voltage, oxide-diffusion (OD), or trimpoly (TPO) rules.
- It inserts the filler cells immediately next to the standard cells; it does not consider the intercell spacing rules.

See Also

- Standard Filler Cell Insertion

Controlling Cell-Based Filler Cell Insertion

You can control the following aspects of the filler cell insertion with the `create_left_right_filler_cells` command:

- The region in which to insert the filler cells

  By default, the command inserts filler cells next to the specified standard cells in the entire block.

  - Use the `-boundaries` option to restrict filler cell insertion to the specified regions. You can specify one or more rectangular or rectilinear regions.
  - Use the `-voltage_areas` option to restrict filler cell insertion to the specified voltage areas.

- The orientation of the filler cells

  By default, the command uses the allowable orientations of the row to place the filler cells. To use the same orientation as the standard cell, use the `-follow_stdcell_orientation` option. When you use this option, the tool swaps the right and left filler cells when the cell is flipped.

Feedback

- The handling of one-unit-tile gaps

  By default, the command does not consider the one-unit-tile rule. To prevent the tool from inserting filler cells that would leave a one-unit-tile gap, use the `-rules no_1x` option. When you enable this rule, do not include filler library cells with a width of one-unit tile in the `-lib_cells` option; otherwise, the tool ignores this rule.

- The naming convention used to identify the inserted filler cell instances

  By default, the command uses the following naming convention for inserted filler cells:

  xofiller!*filler_library_cell_name*!*number*

  To identify the filler cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the command uses the following naming convention:

  xofiller!*prefix*!*filler_library_cell_name*!*number*

Threshold-Voltage-Based Filler Cell Insertion

In the threshold-voltage-based flow, the tool selects the filler cells by using user-defined insertion rules, which are based the threshold-voltage types of the cells that border the gap. This flow is typically used only for established foundry nodes.

To perform threshold-voltage-based filler cell insertion,

1. Ensure that the block is legalized by using the `check_legality` command.
2. Annotate the threshold-voltage types on the reference cells by using the `set_cell_vt_type` command.

   For example, to specify that all cells whose names end with *_vtA* have a threshold voltage type of *vtA* and the invx1 cell has a threshold voltage type of *default*, use the following commands:

   ```
   icc2_shell> set_cell_vt_type -lib_cells "*/*_vtA" -vt_type vtA
   icc2_shell> set_cell_vt_type -lib_cells "mylib/invx1" -vt_type default
   ```

3. Define the rules for inserting the filler cells by using the `set_vt_filler_rule` command.

   You define the rules by specifying the filler cells to insert based on the threshold-voltage types of the cells on the left and right of the gap being filled.

   For example, to specify that filler2x or filler1x cells can be inserted in a gap that has default threshold-voltage cells on the left and right sides and fillerA2x and fillerA1x cells

Feedback

can be inserted in a gap that has vtA threshold-voltage cells on the left and right sides, use the following commands:

```
icc2_shell> set_vt_filler_rule -vt_type {default default} \
   -filler_cells {myLib/filler2x  myLib/filler1x}
icc2_shell> set_vt_filler_rule -vt_type {vtA vtA} \
   -filler_cells {myLib/fillerA2x  myLib/fillerA1x}
```

4. Insert the filler cells by using the `create_vtcell_fillers` command.

Controlling Threshold-Voltage-Based Filler Cell Insertion

By default, the `create_vtcell_fillers` command fills all empty space in the horizontal standard-cell rows of the entire block.

You can control the following aspects of the filler cell insertion:

- The region in which to insert the filler cells
  - Use the `-region` option to restrict filler cell insertion to the specified regions.
  - Use the `-voltage_area` option to restrict filler cell insertion to the specified voltage areas.
- The naming convention used to identify the inserted filler cell instances

  By default, the command uses the following naming convention for inserted filler cells:

  xofiller!*filler_library_cell_name*!*number*

  To identify the filler cells inserted in a specific run, use the `-prefix` option to specify a prefix string. When you use this option, the command uses the following naming convention:

  xofiller!*prefix*!*filler_library_cell_name*!*number*

  To change the separator character from the default exclamation mark (!), use the `-separator` option.

Feedback

Removing the Threshold-Voltage Filler Cell Information

To remove the filler cell insertion rules and the threshold-voltage type annotations, use the `-clear_vt_information` option with the `create_vtcell_fillers` command.

Removing Filler Cells

Filler cells inserted by the IC Compiler II tool have instance names that start with the string *xofiller*. To remove all the filler cells from a block, use the `remove_cells` command, as shown in the following example:

```
icc2_shell> remove_cells [get_cells xofiller*]
```

If you used the standard filler cell insertion flow and want to remove only those filler cells with DRC violations, use the `remove_stdcell_fillers_with_violation` command, as described in Checking for Filler Cell DRC Violations.

Inserting Metal Fill

After routing, you can fill the empty spaces in the block with metal wires to meet the metal density rules required by most fabrication processes. Before inserting metal fill, the block should be close to meeting timing and have very few or no DRC violations.

To insert metal fill, run the `signoff_create_metal_fill` command, as described in Inserting Metal Fill With IC Validator In-Design.

Note:

An IC Validator license is required to run the `signoff_create_metal_fill` command.

7

IC Validator In-Design

The IC Validator In-Design feature provides the ability to use the IC Validator tool to perform physical implementation and verification tasks within the IC Compiler II tool. You can use IC Validator In-Design to perform the following tasks:

- Signoff design rule checking (the `signoff_check_drc` command)
- Fixing the violations detected during signoff design rule checking (the `signoff_fix_drc` command)
- Augmenting the power grid based on voltage drop information (the `signoff_create_pg_augmentation` command)
- Inserting metal fill (the `signoff_create_metal_fill` command)
- Fixing isolated vias (the `signoff_fix_isolated_via` command)

Note:

An IC Validator license is required to run the IC Validator In-Design functions.

To learn about using these IC Validator In-Design functions, see the following topics:

- Preparing to Run IC Validator In-Design Commands
- Performing Signoff Design Rule Checking
- Automatically Fixing Signoff DRC Violations
- Improving Instance Voltage Drop by Augmenting the Power Grid
- Inserting Metal Fill With IC Validator In-Design
- Automatically Fixing Isolated Vias

Feedback

Preparing to Run IC Validator In-Design Commands

The following topics describe the tasks you must perform before you run the IC Validator In-Design commands:

- Setting Up the IC Validator Environment
- Enabling IC Validator Multicore Processing
- Defining the Layer Mapping for IC Validator In-Design Commands

Setting Up the IC Validator Environment

To run an IC Validator In-Design command, you must specify the location of the IC Validator executable by setting the `ICV_HOME_DIR` environment variable. You can set this variable in your .cshrc file. To specify the location of the IC Validator executable, use commands similar to those shown in the following example:

```
setenv ICV_HOME_DIR /root_dir/icv
set path = ($path $ICV_HOME_DIR/bin/LINUX.64)
```

You must ensure that the version of the IC Validator executable that you specify is compatible with the IC Compiler II version that you are using. To report the version compatibility, use the `report_versions` command.

For more information about the IC Validator tool, see the IC Validator documentation, which is available on SolvNet.

Enabling IC Validator Multicore Processing

By default, the IC Validator In-Design commands use a single process to perform design rule checking. To reduce the turnaround time for design rule checking, use multicore processing, which includes multithreading, distributing processing, and parallel command execution. After you specify how many cores to use on different hosts, the IC Validator tool determines exactly how to take advantage of multithreading and parallel command execution to achieve the fastest runtimes.

To enable multicore processing, you must define the configuration by using the `set_host_options` command. By default, the configuration applies to all IC Compiler II commands that support multicore processing. To define a configuration that applies only to the IC Validator In-Design commands, use the `-target ICV` option.

Feedback

You can enable multicore processing on specific hosts, by using a job scheduler such as the Load Sharing Facility (LSF), or by using a combination of these methods. The following topics describe these methods:

- Running IC Validator on Specific Hosts
- Running IC Validator Using a Job Scheduler
- Running IC Validator Using Hybrid Multicore Processing

For general information about the `set_host_options` command, see Enabling Multicore Processing.

Running IC Validator on Specific Hosts

To enable IC Validator multicore processing on one or more specific hosts, use the `set_host_options` command to specify the following information:

- The host names

  If you do not specify a host name or submit protocol, the IC Validator tool runs on the host on which the IC Compiler II tool is currently running.

- The number of cores

  Use the `-num_processes` or `-max_cores` options to specify the number of cores. If you use both options, the IC Validator tool uses the product of the settings as the number of cores.

  The specified number of cores applies to each specified host.

For example, to enable the IC Validator tool to use 4 cores on the current host, use the following command:

```
icc2_shell> set_host_options -target ICV -num_processes 4
```

To enable the IC Validator tool to use 16 cores (8 cores each on machineA and machineB), use the following command:

```
icc2_shell> set_host_options -target ICV \
   -num_processes 2 -max_cores 4 {machineA machineB}
```

Feedback

Running IC Validator Using a Job Scheduler

To enable IC Validator multicore processing using a job scheduler, use the `set_host_options` command to specify the following information:

- The submit protocol
  - To use LSF, use the `-submit_protocol lsf` option.

    The LSF `bsub` command must be in your Linux path.
  - To use the Univa Grid Engine (UGE) job scheduler, which was previously known as SGE (Sun Grid Engine) or GRD (Global Resource Directory), use the `-submit_protocol grid` option.

    The Grid `qsub` command must be in your Linux path.

  If you do not specify a host name or submit protocol, the IC Validator tool runs on the host on which the IC Compiler II tool is currently running.
- The number of hosts to use

  Use the `-num_processes` option to specify the number of hosts.
- The number of cores to use on each host

  Use the `-max_cores` option to specify the number of cores per host.
- (Optional) Extra arguments needed for the submit command

  Use the `-submit_command` option to specify the submit command with the extra arguments.

For example, to enable 4 hosts to be acquired by LSF, each of which can use 8 cores (for a total of 32 cores), use the following command:

```
icc2_shell> set_host_options -submit_protocol lsf -target ICV \
   -num_processes 4 -max_cores 8
```

To set the memory requirement of the jobs in the previous example to 1000M, use the following command:

```
icc2_shell> set_host_options -submit_protocol lsf -target ICV \
   -num_processes 4 -max_cores 8 \
   -submit_command {bsub -R rusage[mem=1000]}
```

To enable 4 hosts to be acquired by the Grid Engine, each of which can use 8 cores (for a total of 32 cores), use the following command:

```
icc2_shell> set_host_options -submit_protocol grid -target ICV \
   -num_processes 4 -max_cores 8
```

Feedback

To set the process name of the jobs in the previous example to bnormal, use the following command:

```
icc2_shell> set_host_options -submit_protocol grid -target ICV \
   -num_processes 4 -max_cores 8 -submit_command "qsub -P bnormal"
```

Running IC Validator Using Hybrid Multicore Processing

If you are using LSF, you can specify both specific hosts and an LSF configuration for adding additional hosts as they become available. To use this method,

- Specify the configuration for the specific hosts as described in Running IC Validator on Specific Hosts.
- Specify the LSF configuration as described in Running IC Validator Using a Job Scheduler, but use the `-add_hosts` option in addition to the other command options.

  This configuration represents both optional resources and maximum resources – the resources can be given to the IC Validator tool at the beginning, middle, or end of the run, or not given at all.

For example, to use four cores on the current host and potentially acquire up to four more hosts from LSF, use the following commands:

```
icc2_shell> set_host_options -target ICV -max_cores 4
icc2_shell> set_host_options -target ICV -add_hosts \
   -submit_protocol lsf -num_processes 4
```

Defining the Layer Mapping for IC Validator In-Design Commands

A layer mapping file maps the technology layers to the layers used in the runset file. You specify the location of the layer mapping file by setting the `signoff.physical.layer_map_file` application option.

Each line in the layer mapping file specifies a layer in the technology file and the corresponding layer in the runset file using the following syntax:

```
tf_layer[:tf_purpose][:use_type][:mask_type]
      runset_layer[:runset_data_type]
```

Each line can contain the following items:

- *`tf_layer`* – Layer number in the technology file, a required item
- *`tf_purpose`* – Purpose number in the technology file
- *`use_type`* – IC Compiler II usage of the geometries in the design

Feedback

Valid values are `power`, `ground`, `signal`, `clock`, `boundary`, `hard_placement_blockage`, `soft_placement_blockage`, `routing_blockage`, `area_fill`, and `track`.

- *`mask_type`* – Multiple-patterning mask constraint of the geometries

  Valid values for metal layers are `mask_one`, `mask_two`, `mask_three`, and `mask_same`. Via layers support the additional values `MASK_FOUR` through `MASK_FIFTEEN`.

- *`runset_layer`* – Layer number in the runset file, a required item
- *`runset_data_type`* – Layer data type number in the runset file

Any text in a line that comes after a semicolon character (`;`) is considered a comment and has no effect on layer mapping.

Performing Signoff Design Rule Checking

IC Validator In-Design signoff design rule checking (DRC) runs the IC Validator tool within the IC Compiler II tool to check the routing design rules defined in the foundry signoff runset.

You can perform signoff design rule checking by

- Running the `signoff_check_drc` command
- Using the Live DRC feature in the IC Compiler II GUI

After place and route, use the `signoff_check_drc` command to perform signoff design rule checking on the entire block. After engineering change orders (ECO), use either method to perform incremental signoff design rule checking on the modified portions of the block.

See Also

- Automatically Fixing Signoff DRC Violations

Running the signoff_check_drc Command

To perform signoff design rule checking by running the `signoff_check_drc` command,

1. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
2. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.

Feedback

3. Set the application options for signoff design rule checking.

   At a minimum, you must specify the foundry runset to use for design rule checking by setting the `signoff.check_drc.runset` application option.

   For information about the options for command-based signoff design rule checking, see Setting Options for Signoff Design Rule Checking.

4. Save the block to disk.

   When you run signoff design rule checking, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running signoff design rule checking.

5. Run signoff design rule checking by using the `signoff_check_drc` command.

By default, the `signoff_check_drc` command performs the following tasks:

1. Loads the block into the IC Validator tool, as described in Reading Blocks for Signoff Design Rule Checking
2. Performs signoff design rule checking, as described in Signoff Design Rule Checking
3. Generates an error data file and IC Validator results files, as described in Signoff DRC Results Files

   You can use the results files to view the violations in the error browser, as described in Using the Error Browser, or to perform automatic design rule fixing, as described in Automatically Fixing Signoff DRC Violations.

You can use the following methods to analyze the DRC violations reported by the `signoff_check_drc` command:

- View the violations in the error browser, as described in Using the Error Browser.
- View the density of violations in a heat map, as described in Viewing the Violations in an ICV Heat Map.

See Also

- Checking Signoff Design Rules Interactively in the GUI

Feedback

Setting Options for Signoff Design Rule Checking

Before you run the `signoff_check_drc` command, configure the run by setting the application options shown in Table 37. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Table 37 Application Options for Signoff Design Rule Checking

Application option	Default	Description
`signoff.check_drc.runset` (required)	N/A	Specifies the foundry runset to use for design rule checking.
`signoff.check_drc.auto_eco_threshold_value`	20	Specifies the maximum percentage of change to the block to perform incremental design rule checking using the `-auto_eco true` option.
`signoff.check_drc.excluded_cell_types`	{}	Specifies the cell types to exclude from checking. You can specify one or more of the following values: `lib_cell` (standard cells), `macro` (macro cells), `pad` (I/O pad cells), and `filler` (filler cells).
`signoff.check_drc.fill_view_data`	`read_if_uptodate`	Controls whether the IC Validator tool reads the fill data. By default, it reads the fill data only if its timestamp is newer than the timestamp of the design view. To read the fill data regardless of its timestamp, set this application option to (`read`). To never read the fill data, set this application option to `discard`.
`signoff.check_drc.ignore_blockages_in_cells`	`true`	Controls whether the IC Validator tool reads only the pin information from the frame view (`true`) or both the blockages and the pin information (`false`).
`signoff.check_drc.ignore_child_cell_errors`	`false`	Controls whether the IC Validator tool reports both top-level and child-level errors to the error data file (`false`) or only top-level errors (`true`).
`signoff.check_drc.max_errors_per_rule`	1000	Specifies the maximum number of errors to report per rule.
`signoff.check_drc.read_design_views`	{}	Specifies the reference cells for which the IC Validator tool reads the design view instead of the frame view. Using the design view can expose problems that are masked by the frame view abstraction.

Feedback

Table 37 Application Options for Signoff Design Rule Checking (Continued)

Application option	Default	Description
`signoff.check_drc. read_layout_views`	{}	Specifies the reference cells for which the IC Validator tool reads the layout view instead of the frame view. Using the layout view can expose problems that are masked by the frame view abstraction.
`signoff.check_drc. run_dir`	signoff_check_drc_run	Specifies the run directory, which contains the files generated by the `signoff_check_drc` command. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.
`signoff.check_drc. user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.
`signoff.physical. layer_map_file`	(none)	Specifies the name of the layer mapping file. If the technology file and the foundry runset file used by the IC Validator tool do not use the same layer numbers, you must supply a layer mapping file to map the technology layers to the layers used in the runset file. For details about the format of the layer mapping file, see Defining the Layer Mapping for IC Validator In-Design Commands.
`signoff.physical. merge_exclude_libraries`	(none)	Specifies the cell libraries whose cells are excluded from replacement with the cell data in the stream files specified in the `signoff.physical. merge_stream_files` application option.
`signoff.physical. merge_stream_files`	(none)	Specifies the stream (GDSII or OASIS) files to merge into the current block for signoff design rule checking. When you use this option, the GDSII or OASIS data replaces the cell library data for the cells defined in the specified stream files.

Feedback

Reading Blocks for Signoff Design Rule Checking

By default, the IC Validator tool reads the design view for the top-level block and library cell instances, and the pin information from the frame view for the macro cell and I/O pad cell instances.

- To read both the pin information and the routing blockages from the frame view for macro cells and I/O pad cells, set the `signoff.check_drc.ignore_blockages_in_cells` application option to `false`.
- To read the GDSII or OASIS data for specific reference cells, specify the stream files by setting the `signoff.physical.merge_stream_files` application option.

  When you use this option, the GDSII or OASIS data replaces the cell library view for the cells defined in the specified stream files, except for cells in the cell libraries specified in the `signoff.physical.merge_exclude_libraries` application option.
- To read the layout view for specific reference cells, specify the cells by setting the `signoff.check_drc.read_layout_views` application option.

  The layout view is identical to the GDSII or OASIS data that was used to create it, but reduces the runtime for signoff design rule checking.

  Note:

  By default, layout views are not included in the cell libraries. To save the layout views, you must set the `lib.workspace.save_layout_views` application option to `true` during library preparation, as described in the *Library Manager User Guide*.
- To read the design view for specific reference cells, specify the cells by setting the `signoff.check_drc.read_design_views` application option.

  Note:

  By default, design views are not included in the cell libraries. To save the design views, you must set the `lib.workspace.save_design_views` application option to `true` during library preparation, as described in the *Library Manager User Guide*.

The order of precedence for child cell data is

1. GDSII or OASIS data specified with the `signoff.physical.merge_stream_files` application option
2. Layout views specified with the `signoff.check_drc.read_layout_views` application option

   If the IC Validator tool cannot find a layout view and the cell is specified in the `signoff.check_drc.read_design_views` application option, it reads the design view.

Feedback

Otherwise, it reads the frame view. If it cannot find the frame view, the cell is ignored during design rule checking.

3. Design views specified with the `signoff.check_drc.read_design_views` application option

   If the IC Validator tool cannot find a design view, it reads the frame view instead. If it cannot find the frame view, the cell is ignored during design rule checking.

4. Frame views

For example, to read the design view for all blocks, use the following commands:

```
icc2_shell> set_app_options \
   -name signoff.check_drc.read_design_views -value {*}
icc2_shell> signoff_check_drc
```

To read the design view for the top-level block and all instances of reference cells whose name start with mc, and the frame view for all other child blocks, use the following commands:

```
icc2_shell> set_app_options \
   -name signoff.check_drc.read_design_views -value {mc*}
icc2_shell> signoff_check_drc
```

Signoff Design Rule Checking

By default, signoff design rule checking has the following default behavior:

- Performs checking on all cell types (standard cells, filler cells, macro cells, and I/O pad cells)

  To exclude certain cell types from design rule checking, set the `signoff.check_drc.excluded_cell_types` application option. Specify one or more of the following values: `lib_cell` (standard cells), `filler` (filler cells), `macro` (macro cells), and `pad` (I/O pad cells).

- Performs design rule checking on the entire block
  - To perform design rule checking only on those areas of the block that have been modified since the last run, use the `-auto_eco` option.

    Note:

    You can use this option only if you have previously run the `signoff_check_drc` command and the percentage of change to the block since that run is less than the change threshold. The default change threshold is 20 percent; to modify the change threshold, set the `signoff.check_drc.auto_eco_threshold_value` application option.

Feedback

By default, the tool compares the current block to the version used for the previous `signoff_check_drc` run. To compare the current block to a different block, use the `-pre_eco_design` option to specify the comparison block. The tool gets the information required for incremental change detection from the error data file named *block*_sdrc.err, where *block* is either the current block name or the name specified by the `-pre_eco_design` option.

By default, the DRC violations reported after performing incremental signoff design rule checking represent only the DRC violations detected in the changed areas of the block. To merge the initial DRC violations with the DRC violations detected during the incremental run, set the `signoff.check_drc.merge_incremental_error_data` application option to `true` before running the `signoff_check_drc -auto_eco` command. By default, the command reads the initial DRC violations from the signoff_check_drc.err file. To read the initial DRC violations from a different error data file, specify the file name with the `signoff.check_drc.merge_base_error_name` application option.

- To perform design rule checking only in specific regions, use the `-coordinates` option, the `-excluded_coordinates` option, or both. You can specify one or more regions for both options. If you specify both options, the command does not check design rules in the overlapping regions.

  To specify the coordinates, use the following format:

  $\{\{x_1\ y_1\}\ \{x_2\ y_2\}\ \ldots\ \{x_n\ y_n\}\}$

  Note that there must be a space between each set of coordinates.

- Performs design rule checking for all rules specified in the foundry runset
  - To check only specific rules, use the `-select_rules` option. Specify the rules by specifying a matching pattern for the rule names. The rule names are specified in the COMMENT section in the runset file.
  - To prevent checking of specific rules, use the `-unselect_rules` option. Specify the rules by specifying a matching pattern for the rule names. The rule names are specified in the COMMENT section in the runset file.

    Note:

    You can use this option with the `-select_rules` option to customize the set of rules checked by the `signoff_check_drc` command. For example, to restrict the `signoff_check_drc` command to route validation, select the metal layer rules and exclude the metal density rules.

Feedback

- Performs design rule checking for all routing layers
  - To perform design rule checking only on specific routing layers, use the `-select_layers` option. Specify the routing layers by using the layer names from the technology file.
  - To perform design rule checking on all runset layers, including nonrouting layers, use the `-check_all_runset_layers true` option.

    You would typically use this setting to perform a quick design rule check on the entire block after you complete design rule checking on the routing layers. When you use this option, the block information comes from the design view and the validation tool checks all layers, including the nonrouting layers.
- Reports a maximum of 1000 errors for each rule, including both top-level and child-level violations
  - To override the maximum error count, set the `signoff.check_drc.max_errors_per_rule` application option.
  - To ignore child-level violations, set the `signoff.check_drc.ignore_child_cell_errors` application option to `true`.

The `signoff_check_drc` command uses the options that you specify to generate the command used to invoke signoff design rule checking in the IC Validator tool. You can specify additional options for the IC Validator command line by setting the `signoff.check_drc.user_defined_options` application option. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.

Generating Input for the Automatic Fixing Flow

If you plan to use the signoff DRC results as input to the automatic fixing flow, set the `signoff.check_drc.ignore_child_cell_errors` application option to `true` before you run the `signoff_check_drc` command. Zroute can fix only top-level violations; using this option ensures that signoff design rule checking reports only fixable violations.

Signoff DRC Results Files

The `signoff_check_drc` command writes its output files to the run directory, which is specified by the `signoff.check_drc.run_dir` application option (or the signoff_check_drc_run directory if you do not use this option). The following files are written to the run directory:

- The error data file

  By default, the error data generated by the `signoff_check_drc` command is saved in a file named signoff_check_drc.err, which is stored in the design library. To specify the name for the error data file, use the `-error_data` option. The error data shows child-

Feedback

level errors on only one of the cell instances, which makes it easier to identify lower-level errors.

You can use the error data file to report or display the DRC violations in the error browser. For information about using the error browser, see Using the Error Browser.

- The IC Validator results files

  You can use the following IC Validator results files for debugging the signoff DRC results:

  - *block*.LAYOUT_ERRORS

    This file contains details about the errors detected during the `signoff_check_drc` run.

  - *block*.RESULTS

    This file contains a summary of the `signoff_check_drc` run results.

  - signoff_check_drc.log

    This file contains a summary of the `signoff_check_drc` run environment.

  - icv_config_out

    This file contains the paths to the top-level block and the cell libraries.

  - icv_sdrc.conclude

    This file contains an error summary report.

  - layer.map

    This file contains the layer mapping file generated by the `signoff_check_drc` command.

  - signoff_check_drc.rc

    This file contains the IC Validator runset environment variables.

  - ./run_details directory

    This directory contains all the data generated by the IC Validator tool for the signoff DRC run.

  For more information about these files, see the IC Validator documentation.

Viewing the Violations in an ICV Heat Map

An ICV heat map displays the density of signoff DRC violations in a block. You can use the heat map to locate problematic areas in your design.

Feedback

Note:

This feature requires version P-2019.06 or later of the IC Validator tool and an IC Validator NXT license.

To view the ICV heat map,

1. Enable the ICV heat map by setting the `signoff.check_drc.enable_icv_explorer_mode` application option to `true`.

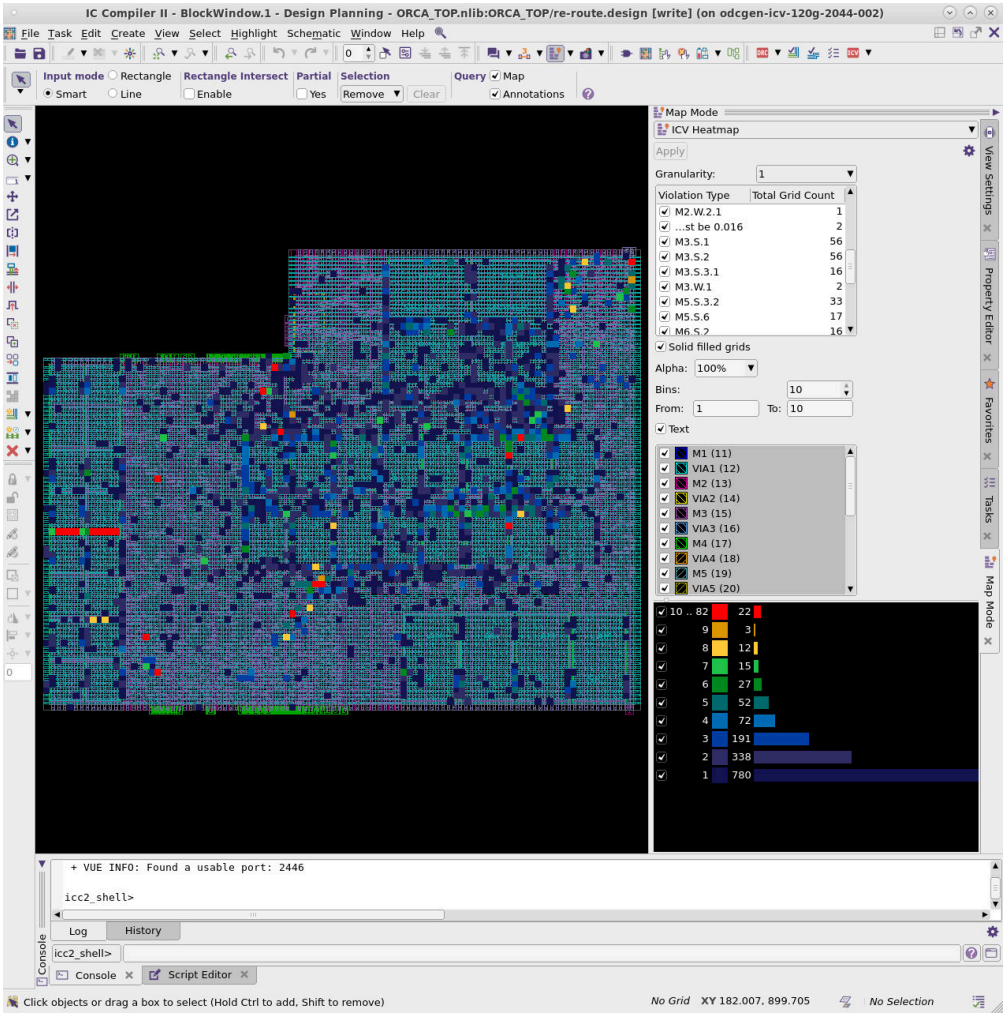
```
icc2_shell> set_app_options -name
 signoff.check_drc.enable_icv_explorer_mode \
   -value true
```

2. Perform signoff design rule checking as described in Running the signoff_check_drc Command.
3. In the GUI, choose View > Map > ICV Heatmap.

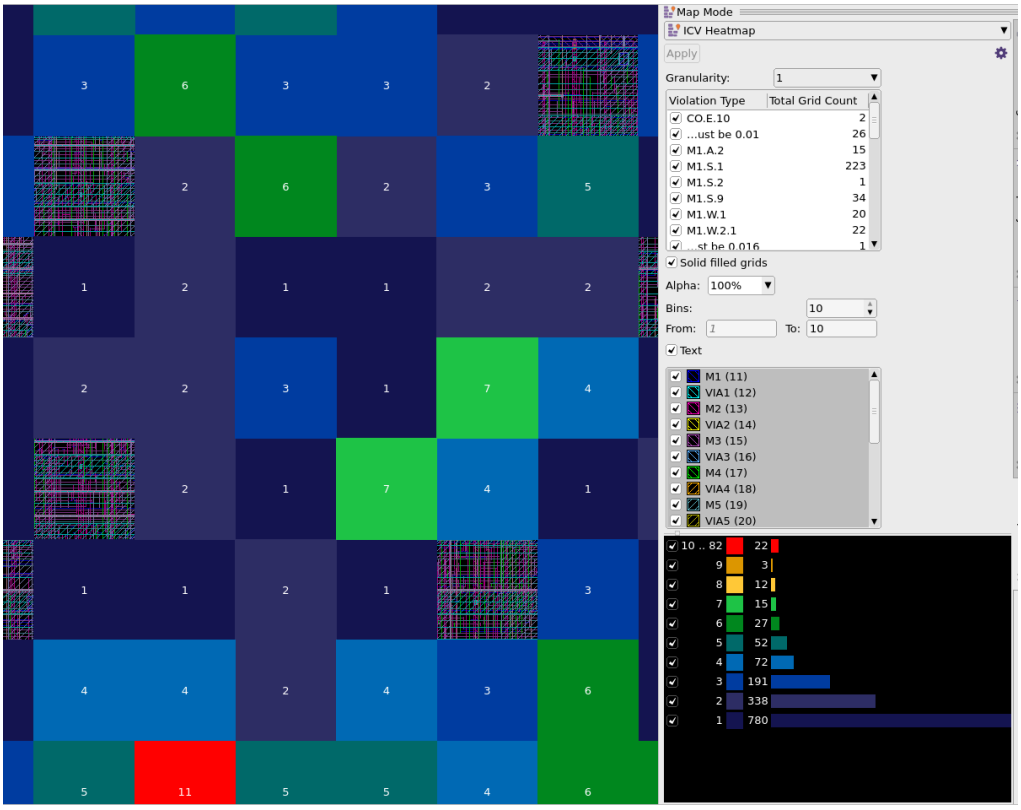
Figure 97 shows an ICV heat map.

Feedback

Figure 97 ICV Heat Map



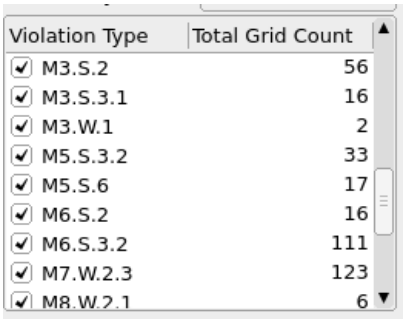
Feedback



Configuring an ICV Heat Map

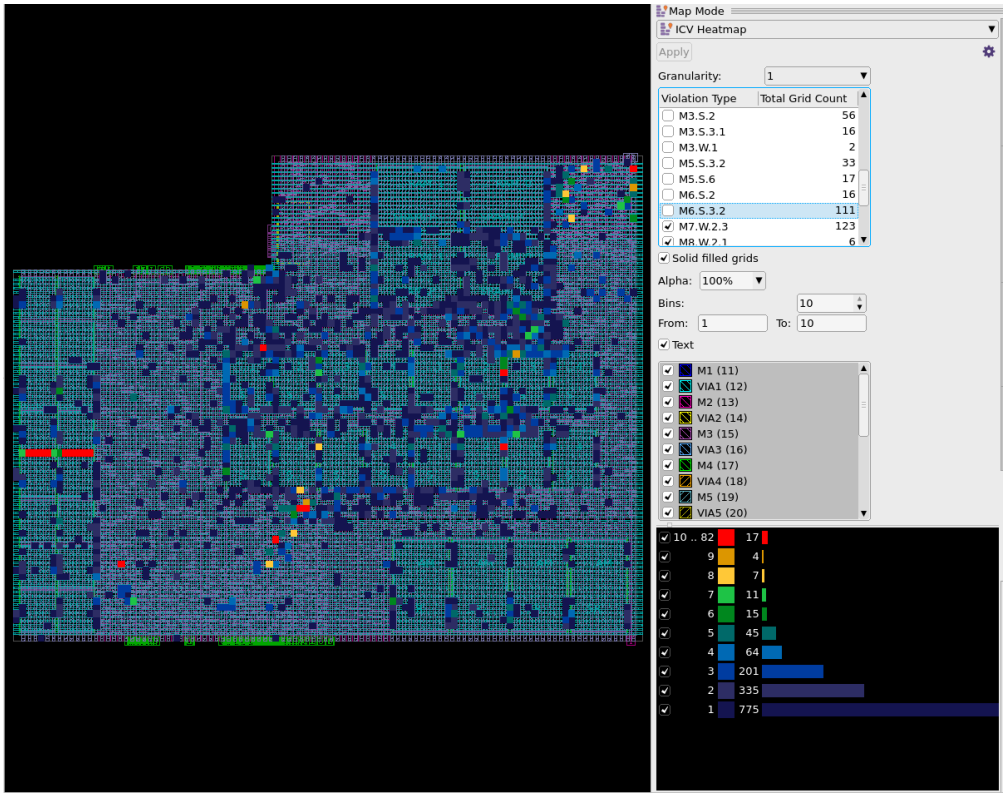
You can select or unselect violation types from the heat map, as shown in figure Figure 98 and Figure 99.

Figure 98 Violation Type



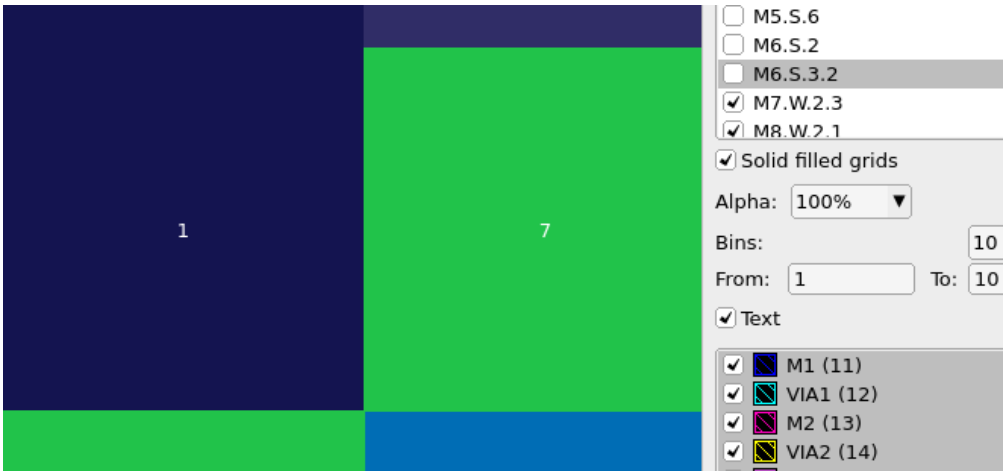
Feedback

Figure 99 Violation Type in ICV Heat Map



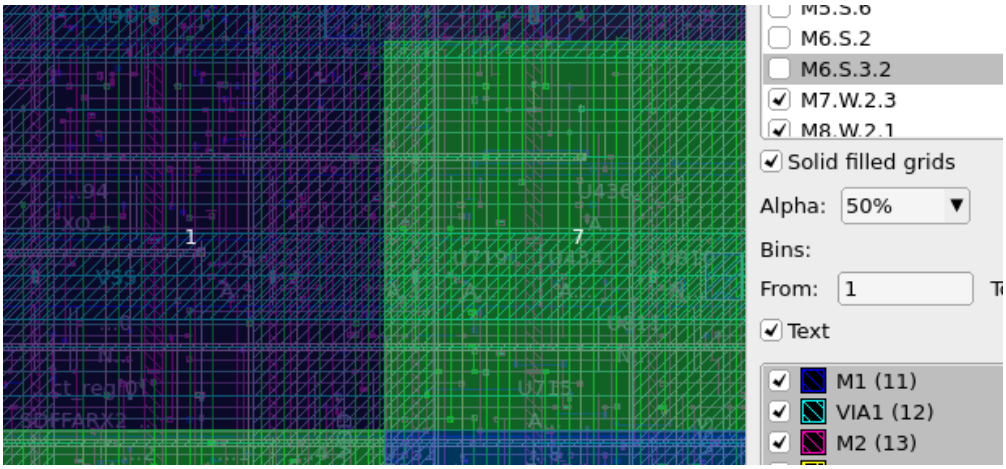
To control the opacity of the heat map squares, adjust the Alpha percentage value. Set this percentage to 100% to create a completely filled block. The lower you set the percentage, the more transparent the block becomes, as shown in Figure 100 and Figure 101.

Figure 100 Completely Filled Block



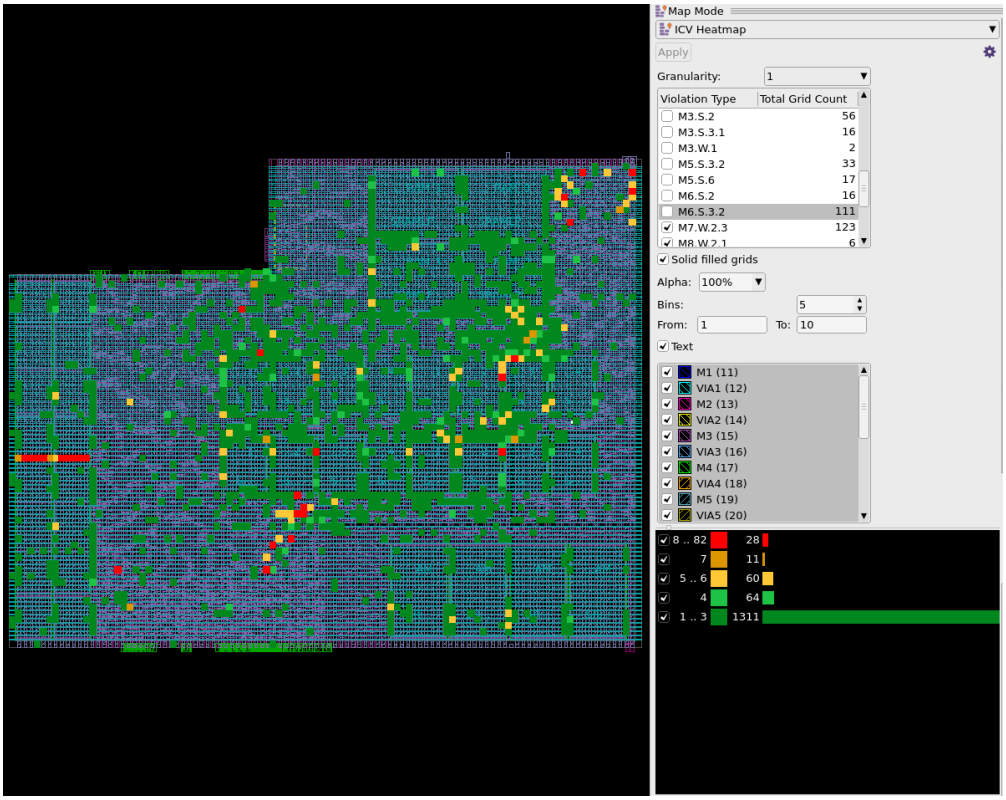
Feedback

Figure 101



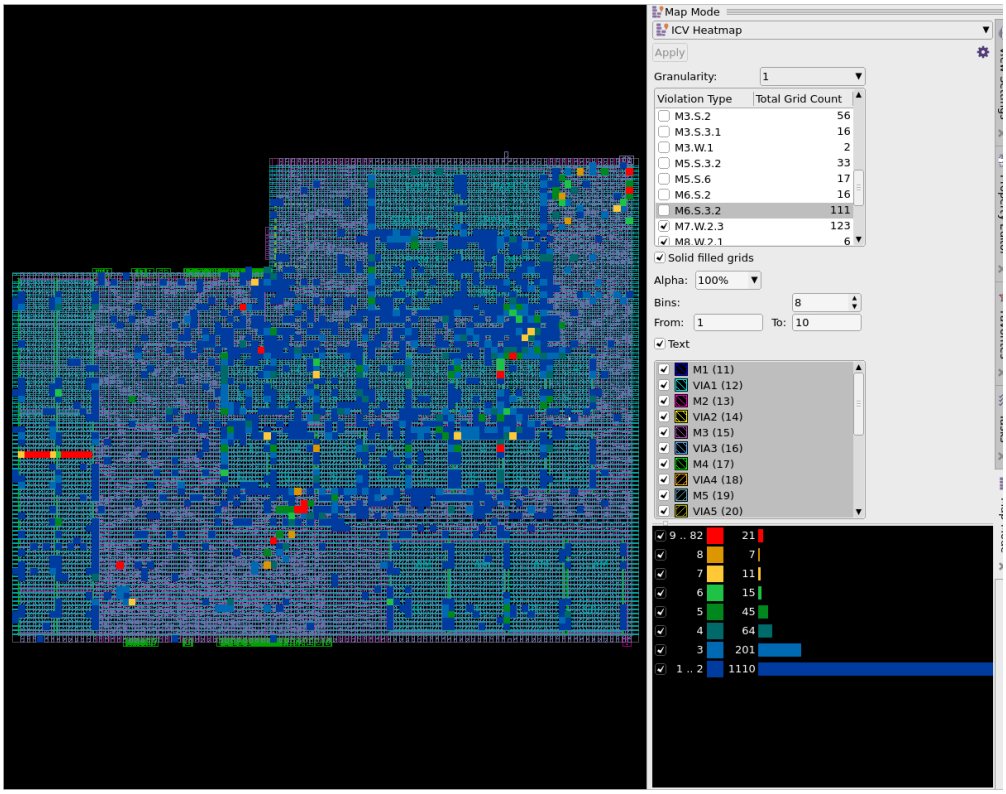
You can adjust the range of colored squares by modifying the Bins value, as shown in Figure 102 and Figure 103.

Figure 102 Bins Value Set to 5



Feedback

Figure 103 Bins Value Set to 8



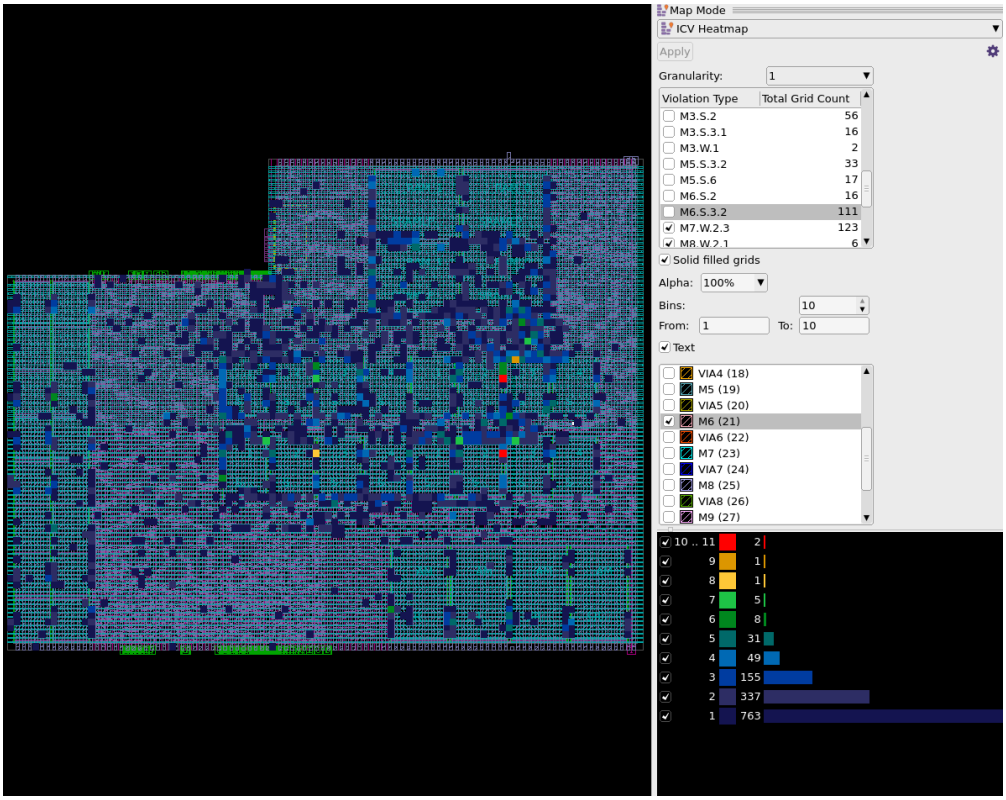
You can select or unselect layers from the heat map, as shown in Figure 104 and Figure 105.

Figure 104 Selecting Layers



Feedback

Figure 105 Selecting Layers in the ICV Heatmap



You can modify the squares of the heat map by right-clicking the colored squares and selecting Set Style. See Figure 106 and Figure 107.

Feedback

Figure 106 Set Style

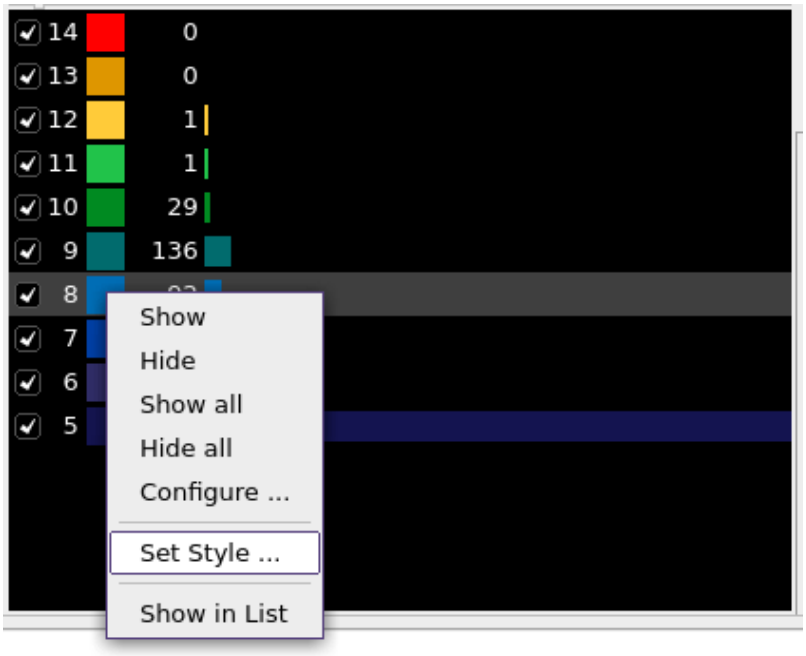
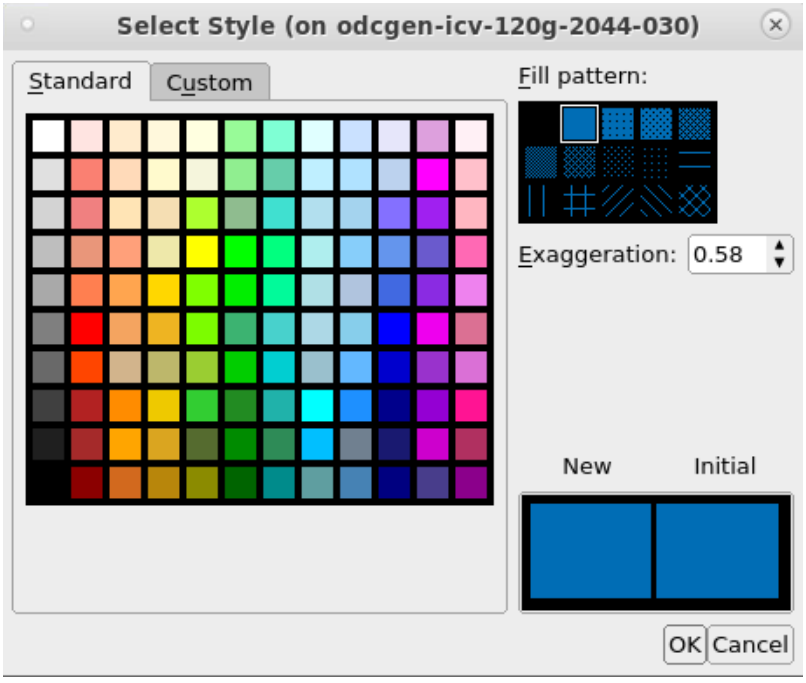


Figure 107 Select Style

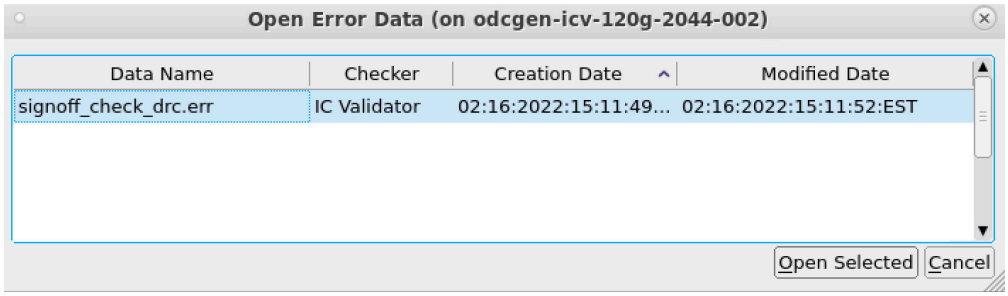


Feedback

Highlighting Violations From the Error Browser Onto an ICV Heat Map

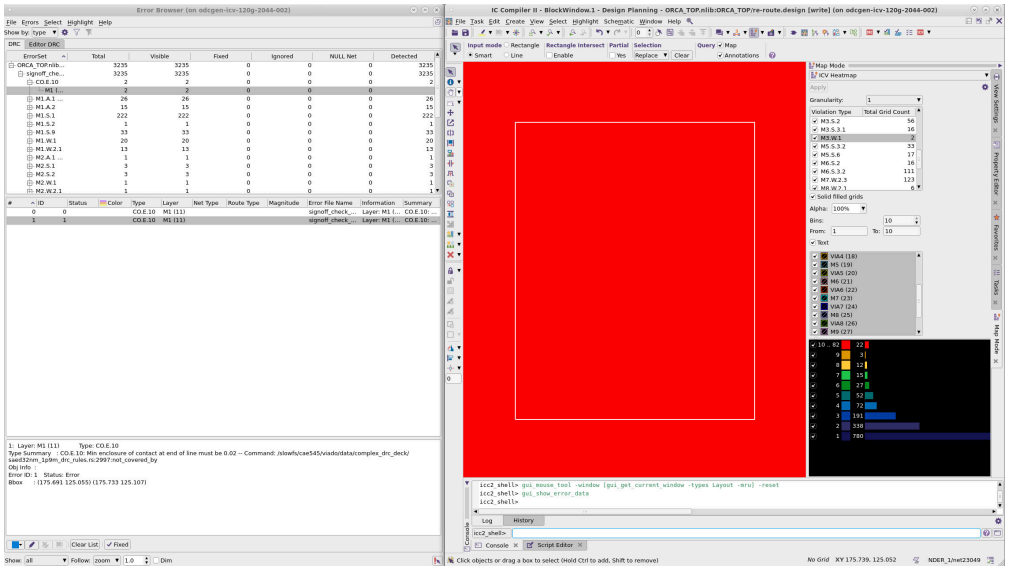
To highlight violations from the Error Browser:

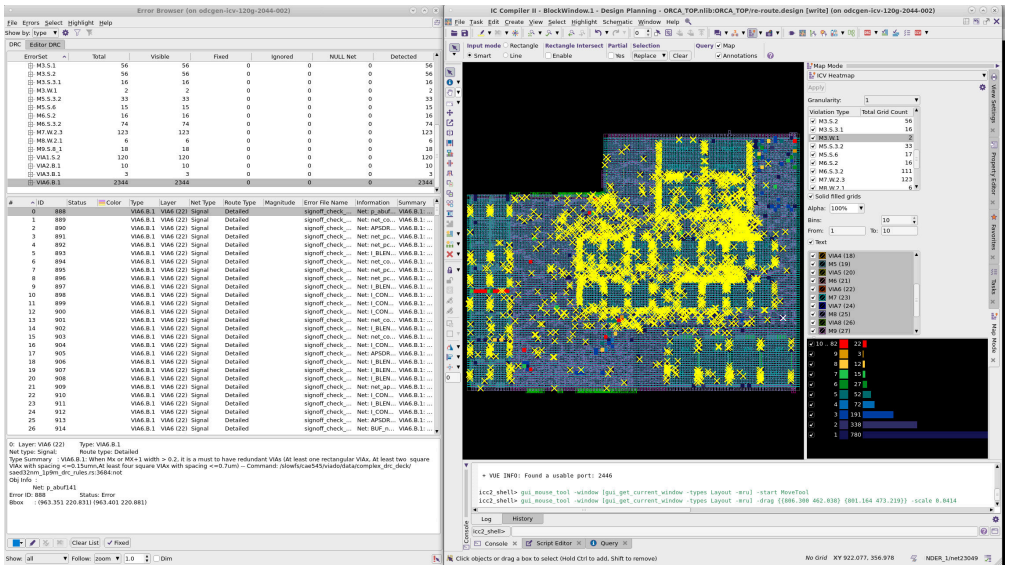
1. In the GUI, choose View > Error Browser.
2. Select the error data generated from the DRC run.



3. Select violations from the Error Browser to automatically highlight them on the heat map.

Feedback





Feedback

Automatically Fixing Signoff DRC Violations

Zroute can use the signoff DRC results generated by the `signoff_check_drc` command to automatically fix the detected design rule violations.

To perform automatic fixing of the signoff DRC violations,

1. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
2. Set up the physical signoff options as described in Setting Options for Signoff Design Rule Checking.
3. (Optional) Generate a configuration file that aids in the fixing process.

   The configuration file defines the layer-to-DRC-rule mapping. The `signoff_fix_drc` command processes only those rules that are specified in the configuration file.

   By default, the `signoff_fix_drc` command automatically generates the configuration. You can manually generate a configuration file, as described in Creating an Autofix Configuration File.
4. Set the application options for signoff DRC fixing.

   For information about the options for signoff DRC fixing, see Setting Options for Signoff Design Rule Checking.
5. Save the block to disk.

   When you run the `signoff_fix_drc` command, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running the `signoff_fix_drc` command.
6. Run signoff DRC fixing by using the `signoff_fix_drc` command as described in Running the signoff_fix_drc Command.

If your block uses double-patterning technology, first perform signoff DRC fixing for all other routing design rules, and then perform signoff DRC fixing for only the double-patterning rules, as described in Automatically Fixing Double-Patterning Odd-Cycle Violations.

Creating an Autofix Configuration File

The autofix configuration file has the following format:

```
"mask_layer_name" "full_IC_Validator_DRC_rule_comment"
```

You can put comments in the configuration file by starting the line with the pound sign (#).

Feedback

If you have already run the `signoff_check_drc` command, you can generate a configuration file by running the $ICV_HOME_DIR/contrib/generate_layer_rule_map.pl script. This script has the following syntax:

```
generate_layer_rule_map.pl
   -dplog IC_Validator_log_file
   -tech_file technology_file
   -o config_file_name
```

The IC Validator log file that you specify in the `-dplog` option is the log file that was generated by the previous `signoff_check_drc` run. This file is called *runset*.dp.log and is located in the run_details subdirectory of the signoff_drc_run directory (or the directory specified by the `signoff.check_drc.run_dir` application option.

To specify the location of the configuration file you created, use the `signoff.fix_drc.config_file` application option.

Setting Options for Signoff DRC Fixing

Before you run the `signoff_fix_drc` command, configure the run by setting the application options shown in Table 38. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Table 38 Application Options for Signoff DRC Fixing

Application option	Default	Description
`signoff.fix_drc.advanced_guidance_for_rules`	`all`	Controls whether signoff DRC fixing uses advanced routing guidance. By default, the `signoff_fix_drc` command uses advanced routing guidance (`all`). To disable this feature, set the application option to `off`.
`signoff.fix_drc.check_drc`	`local`	Controls whether signoff DRC checking is performed only on those portions of the block affected by signoff DRC fixing (`local`, which is the default) or on the entire block (`global`).
`signoff.fix_drc.custom_guidance`	(none)	Enables a signoff DRC fixing run that targets double-patterning odd-cycle violations when it is set to `dpt`. For information about fixing double-patterning odd-cycle violations, see Automatically Fixing Double-Patterning Odd-Cycle Violations.

Feedback

Table 38 Application Options for Signoff DRC Fixing (Continued)

Application option	Default	Description
`signoff.fix_drc.fix_detail_route_drc`	`global`	Controls whether the `signoff_fix_drc` command fixes routing DRC violations for the entire block (`global`) or only in the areas near the signoff DRC violations (`local`).
`signoff.fix_drc.init_drc_error_db`	(none)	Specifies the location of the signoff DRC results from the previous run. By default, the `signoff_fix_drc` command runs the `signoff_check_drc` command to generate the initial signoff DRC results.
`signoff.fix_drc.last_run_full_chip`	`false`	Controls whether the signoff DRC checking in the final repair loop runs only on those portions of the block affected by signoff DRC fixing (`false`) or on the whole block (`true`).
`signoff.fix_drc.max_detail_route_iterations`	5	Specifies the maximum number of search and repair loops to run after signoff DRC fixing.
`signoff.fix_drc.max_errors_per_rule`	1000	Specifies the maximum number of DRC violations to fix per rule.
`signoff.fix_drc.run_dir`	signoff_fix_drc_run	Specifies the run directory, which contains the files generated by the `signoff_fix_drc` command. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.
`signoff.fix_drc.target_clock_nets`	`false`	Controls whether signoff DRC fixing targets data nets (`false`) or clock nets (`true`).
`signoff.fix_drc.user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.

Feedback

Running the signoff_fix_drc Command

By default, the `signoff_fix_drc` command performs the following tasks:

1. Runs the `signoff_check_drc` command to generate the initial signoff DRC results, as described in Checking for DRC Violations

   If you have already run the `signoff_check_drc` command, you can use the existing signoff DRC results instead of rerunning signoff design rule checking. To reuse the existing results, specify the directory that contains the results by setting the `signoff.fix_drc.init_drc_error_db` application option. You can specify either a relative path or an absolute path. If you specify a relative path, it is relative to the current working directory.

   For example, to use the results from the default `signoff_check_drc` run directory, use the following commands:

   ```
   icc2_shell> set_app_options \
      -name signoff.fix_drc.init_drc_error_db \
      -value signoff_check_drc_run
   icc2_shell> signoff_fix_drc
   ```

   Note:

   To use existing signoff DRC results as input to the `signoff_fix_drc` command, you must use the signoff DRC application options described in Generating Input for the Automatic Fixing Flow.

2. Runs two repair loops to fix the DRC violations

   In each repair loop, the `signoff_fix_drc` command performs the following tasks;

   a. Tries to fix the DRC violations detected in the previous IC Validator signoff DRC run, as described in Fixing DRC Violations

   b. Runs IC Validator signoff design rule checking on the modified block, as described in Checking for DRC Violations

   To change the number of repair loops, use the `-max_number_repair_loop` option; you can specify an integer between 1 and 10. If no signoff DRC violations remain after a loop, the tool does not perform additional repair loops.

   As the repair loop number increases, so does the physical scope of the rerouting performed by that repair loop. To change the scope, increase the number of the initial repair loop by using the `-start_repair_loop` option. Note that increasing the initial loop number might cause a greater disturbance to the block, which would require additional design rule checking and fixing by Zroute.

Feedback

3. Writes a summary report of the results

   For information about the summary report, see Summary Report for Automatic Design Rule Fixing.

Fixing DRC Violations

In each repair loop, the `signoff_fix_drc` command tries to fix the DRC violations detected in the previous signoff DRC run.

- For the first loop, the tool uses the IC Validator data in the directory specified by the `signoff.fix_drc.init_drc_error_db` application option (or the signoff_drc_run_init directory if you do not use this option).
- For successive loops, the tool uses the IC Validator DRC data from the previous loop, which is stored in the directory specified by the `signoff.fix_drc.run_dir` application option (or the signoff_fix_drc_run directory if you do not use this option).

When fixing DRC violations, the `signoff_fix_drc` command has the following default behavior:

- Tries to fix all design rule violations detected in the previous signoff DRC run except those design rules that have more than 1000 violations
- Performs DRC fixing on data nets for the whole block

  To modify the regions for design rule fixing, use one or both of the following options:

  - `-coordinates`

    This option restricts design rule fixing to the specified regions.

  - `-excluded_coordinates`

    This option prevents design rule fixing in the specified regions. If you specify this option with the `-coordinates` option, the command does not fix design rules in the overlapping regions.

  To specify the coordinates, use the following format:

  $\{\{x_1\ y_1\}\ \{x_2\ y_2\}\ \ldots\ \{x_n\ y_n\}\}$

  Note that there must be a space between each set of coordinates.

Feedback

To prevent fixes that might impact timing-critical nets, use one or both of the following options:

  - `-nets`

    This option explicitly specifies the critical nets.

  - `-timing_preserve_setup_slack_threshold`

    This option enables the tool to automatically determine the critical nets based on a setup slack threshold. The unit for the slack threshold setting is the library time unit.

- Performs five iterations of detail routing after DRC fixing to fix any DRC violations introduced during DRC fixing
- Saves the modified block in a design view named *block*_ADR_#

To change the default behavior, use the following options:

- `signoff.fix_drc.max_errors_per_rule`

  This application option specifies the error threshold for ignoring a design rule.

- `signoff.fix_drc.fix_detail_route_drc`

  This application option controls the scope of DRC fixing. To reduce runtime by performing DRC fixing only in areas near the signoff DRC violations, set this application option to `local`.

- `signoff.fix_drc.target_clock_nets`

  This application option controls whether DRC fixing targets data nets or clock nets. To target clock nets, set this application option to `true`.

- `signoff.fix_drc.max_detail_route_iterations`

  This application option specifies the number of detail routing iterations; you can specify an integer between 0 and 1000.

Checking for DRC Violations

When performing signoff design rule checking, the `signoff_fix_drc` command has the following default behavior:

- Reads the design view for the top-level block and library cell instances, and the pin information from the frame view for the macro cell and I/O pad cell instances

  For information about changing the view used for specific cells, see Reading Blocks for Signoff Design Rule Checking.

- After the initial run, performs design rule checking only on those portions of the design affected by design rule fixing

Feedback

To perform signoff design rule checking on the entire block for the last repair loop, set the `signoff.fix_drc.last_run_full_chip` application option to `true`. Setting this option to `true` increases accuracy but also increases runtime.

To perform signoff design rule checking on the entire block for all repair loops, set the `signoff.fix_drc.check_drc` application option to `global`. Setting this option to `global` increases accuracy but can greatly increase runtime.

- Does not check for violations in child blocks
- Excludes rules that are not suitable for automatic design rule fixing

  To explicitly specify the design rules checked by the IC Validator tool, set the `-select_rules` *`rules`* and `-unselect_rules` *`rules`* application options.

- Stores the IC Validator results in the signoff_fix_drc_run directory

  To use a different run directory, set the `signoff.fix_drc.run_dir` application option.

  Note:

  The IC Validator results for the initial signoff DRC run are stored in the signoff_drc_run_init directory.

In addition, you can specify options to add to the IC Validator command line by setting the `signoff.fix_drc.user_defined_options` application option.

See Also

- Performing Signoff Design Rule Checking

Feedback

Automatically Fixing Double-Patterning Odd-Cycle Violations

If your block uses double-patterning technology, you must perform separate signoff DRC fixing runs for the non-double-patterning routing rules and the double-patterning routing rules.

To perform automatic fixing of the signoff DRC violations for a block that uses double-patterning technology,

1. Perform automatic fixing for the non-double-patterning signoff DRC violations by using the process described in Automatically Fixing Signoff DRC Violations.

   When you run the `signoff_check_drc` and `signoff_fix_drc` commands, use the `-unselect_rules` option to ignore the double-patterning rules during signoff design rule checking.

   ```
   icc2_shell> signoff_check_drc -unselect_rules {list_of_dpt_rules}
   ```

   Note:

   To determine the double-patterning rules for your technology, see the design rule manual (DRM) provided by your vendor.

2. Perform automatic fixing for the double-patterning odd-cycle violations by using the process described in Automatically Fixing Signoff DRC Violations.

   Before you run the `signoff_fix_drc` command, set the `signoff.fix_drc.custom_guidance` application option to `dpt`.

   ```
   icc2_shell> set_app_options \
      -name signoff.fix_drc.custom_guidance -value dpt
   ```

   When you run the `signoff_check_drc` and `signoff_fix_drc` commands, use the `-select_rules` option to consider only the double-patterning rules during signoff design rule checking.

   ```
   icc2_shell> signoff_check_drc -select_rules {list_of_dpt_rules}
   ```

3. Perform signoff design rule checking to verify the results by using the process described in Performing Signoff Design Rule Checking.

Summary Report for Automatic Design Rule Fixing

The `signoff_fix_drc` command writes a summary report file, result_summary.rpt, to the current working directory. Example 28 shows an example of this report.

Example 28 result_summary.rpt Example

```
Input DRC Error Database..... ./inputs/signoff_check_drc_run
Limit to Cell..... my_design
Maximum Errors/Command..... 1000
```

Feedback

```
SIGNOFF AUTOFIX DRC SUMMARY:
                :        :           :           : REMAINING :         :
                : TOTAL  : PROCESSED : TARGETED  : TARGETED  :  FIX    :
DRC NAME        :   DRC  :       DRC :       DRC : DRC       :  RATE   : COMMENT
M0.W.3 :
Maximum width   :       1 :         1 :         0 :         0 :   --    : 1
 ignored: <#1>
M4.L.3          :
Edge length wi  :       4 :         4 :         0 :         0 :   --    : 4 ignor
ed: <#2>
M4.L.5:2 :
Edge length     :       2 :         2 :         0 :         0 :   --    : 2
 ignored: <#2>
M4.W.3 :
Maximum width   :       1 :         1 :         0 :         0 :   --    : 1
 ignored: <#1>
VIA1.S.10.3 :
Space of        :      10 :        10 :         2 :         0 : 100.0% : 8
 ignored: <#2>
TOTAL           :        :           :         2 :         0 : 100.0% :

NOTE:
Reasons : Detail Info
   #1 : size of error shapes greater than threshold
   #2 : shapes associated with error are not Zroute modifiable
```

The following table describes each of the columns in the summary report:

Table 39 Definitions of Columns in result_summary.rpt

Column heading	Description
DRC NAME	Name of the design rule from the runset
TOTAL DRC	Total number of DRC violations detected by the `signoff_check_drc` run
PROCESSED DRC	Total number of the detected DRC violations that went through the filtering process to determine whether they can be targeted
TARGETED DRC	Number of DRC violations that automatic DRC fixing will attempt to fix after the filtering process
REMAINING TARGETED DRC	Number of DRC violations that remain after automatic DRC fixing
FIX RATE	The ratio of remaining targeted DRC violations to targeted DRC violations
COMMENT	Explanation for ignored DRC violations; the detailed reason codes are provided in the NOTES section of the report

Feedback

Checking Signoff Design Rules Interactively in the GUI

The Live DRC feature performs interactive signoff design rule checking in the IC Compiler II GUI.

Note:

This feature requires version P-2019.06 or later of the IC Validator tool.

To perform interactive signoff design rule checking using Live DRC,

1. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
2. Set the application options for interactive signoff design rule checking.

   At a minimum, you must specify the following information:

   - The foundry runset to use for design rule checking by setting the `signoff.check_drc_live.runset` application option
   - The layer mapping file that maps the technology file layers to the runset layers by setting the `signoff.physical.layer_map_file` application option

   For information about the options for interactive signoff design rule checking, see Setting Options for Interactive Design Rule Checking.
3. Ensure that the layout window displays the objects on which you want to perform design rule checking.

   By default, interactive design rule checking uses the frame view of each cell. If you want more detail than what is provided in the frame view, use the GDSII or OASIS view for specific cells by specifying the files containing the cells in the `signoff.physical.merge_stream_files` application option. For example,

   ```
   icc2_shell> set_app_options \
      -name signoff.physical.merge_stream_files \
      -value {stdcell.gds macro.oas}
   ```

   For information about controlling the objects displayed for design rule checking, see Displaying Objects for Design Rule Checking.
4. Display the DRC toolbar by right-clicking in the GUI menu bar and selecting DRC Tools.

   For details about the icons in the DRC toolbar, see DRC Toolbar.
5. Configure the rules to use for design rule checking by choosing Edit > Options > ICV-Live or by clicking the gear icon () in the DRC toolbar.

Feedback

To exclude all density or connectivity rules from design rule checking, set the`signoff.check_drc_live.exclude_command_class` application option.

6. Run design rule checking by choosing Edit > ICV > Run ICV-Live on Current View or by clicking the ICV Run button () in the DRC toolbar.

The first time that you run Live DRC, the tool performs the following initialization steps, which can take a couple of minutes:

- Processes the IC Validator runset to get the list of available rules and caches the runset
- Processes the GDSII or OASIS files specified in the `signoff.physical.merge_stream_files` application option

Unless the layers, rules, or stream files change, subsequent runs do not perform this caching and therefore are much quicker. To reduce the initial runtime, you can perform this initialization before running Live DRC by using the `signoff_init_live_drc` command.

The tool displays the design rule checking results in the error browser. You can select errors in the error browser and then fix them interactively. For information about using the error browser, see Using the Error Browser

See Also

- Running the signoff_check_drc Command

Displaying Objects for Design Rule Checking

When you run interactive design rule checking, the IC Validator tool performs design rule checking on what is displayed in the layout window, plus an extension of 1 micron. You can modify the size of the extension by setting the `signoff.check_drc_live.halo` application option.

Use the View Settings and Hierarchy Settings panels to control the layers, levels, and cell types displayed in the layout window.

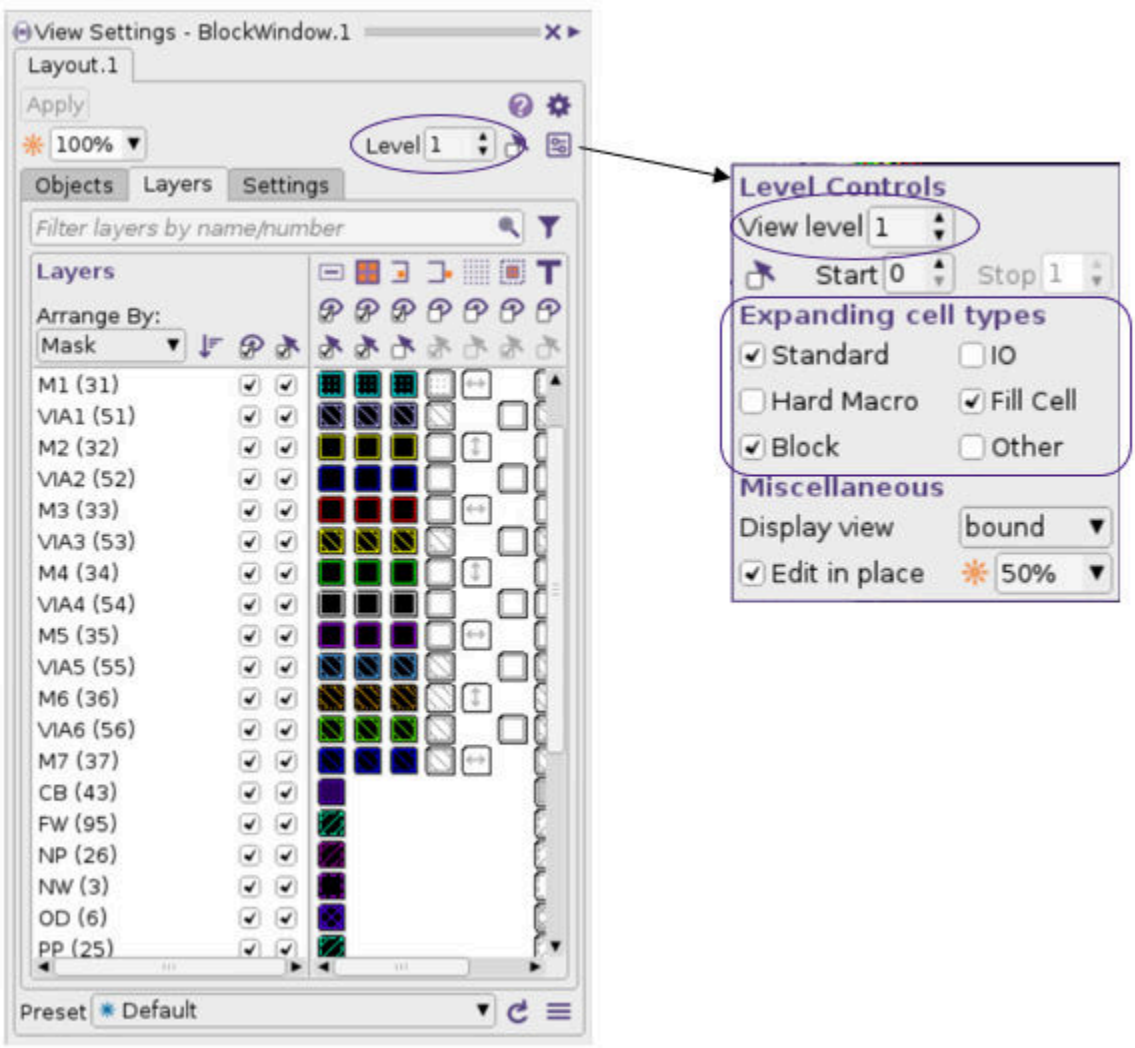
- Display the View Settings panel by clicking the View Settings icon () in the panel area.
  - Specify the number of levels to display in the Level field.
  - Select the Layers tab and enable visibility for the desired layers.
- Display the Hierarchy Settings panel by clicking the Hierarchy Settings icon () in the View Settings panel or the menu bar.

Feedback

- Specify the number of view levels in the “View level” field.
- Select Standard, “Fill Cell,” and Block in the “Expanding cell types” section.

Figure 108shows the View Settings and Hierarchy Settings panels used to control the layout window display for design rule checking.

Figure 108 Settings Used to Display Objects for Design Rule Checking



DRC Toolbar

The DRC toolbar provides buttons that you can use to configure and run interactive design rule checking with Live DRC, and to view the reported DRC violations in the layout view.

Feedback

Figure 109 shows the DRC toolbar for ICV mode, and Figure 110 shows the DRC toolbar for ICVP mode.

Figure 109 ICV DRC Toolbar



Figure 110 ICVP DRC Toolbar



Table 40 DRC Toolbar Buttons

Button	Description
	Runs design rule checking using Live DRC.
	Runs design rule checking using Live DRC in parallel.
	Stops the execution of ICVP. This button is enabled only when the ICVP is running.
	Opens the dialog box to view and sets the Live DRC application options.
	Opens the dialog box to configure the rules to use for design rule checking .
	Selects the region in which to perform design rule checking.
	Resets the design rule checking region to that used in the previous Live DRC run.
	Enables or disables the error environment overlay of GDSII or OASIS shapes from lower level cells that contribute to DRC errors. The error environment overlay is available only when the `signoff.physical.merge_stream_files` application option is set before doing DRC checking.
	Clears or highlights all DRC violations detected by Live DRC.
	Iterates through the DRC violations for the current rule from Live DRC.

Feedback

Table 40 DRC Toolbar Buttons (Continued)

Button	Description
	Controls zoom or pan when iterating through the DRC violations.
	Shows the design rule being highlighted.

Feedback

Setting Options for Interactive Design Rule Checking

Before you run interactive design rule checking, configure the run by setting the application options shown in Table 41. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Table 41 Application Options for Interactive Design Rule Checking

Application option	Default	Description
`signoff.check_drc_live.check_only_visible_layers`	`all_layers`	Controls whether Live DRC includes the rules defined on non-displayed layers. By default, Live DRC includes the rules for all layers. Valid values are `all_layers`, `visible_involved`, and `visible_only`.
`signoff.check_drc_live.default_layers`	(none)	Specifies the layers with color masks to write into the default layers for Live DRC.
`signoff.check_drc_live.exclude_command_class`	`{{density true} {connectivity true}}`	Specifies the types of rules to exclude from DRC checking by Live DRC. By default, the density and connectivity rules are excluded.
`signoff.check_drc_live.halo`		Specifies the distance in microns by which to expand the region in which Live DRC performs checking.
`signoff.check_drc_live.hierarchy_level`	`visible`	Specifies the maximum hierarchical depth processed by Live DRC..
`signoff.check_drc_live.keep_enclosing_results`	`false`	Specifies whether to keep DRC violations that are outside of the checking region. By default, Live DRC keeps only those violations that are fully within the checking region. If you set this application option to `true`, Live DRC keeps the DRC violations that are fully inside the trim region. The trim region is the checking region plus half of the distance specified by the `signoff.check_drc_live.halo`application option. Violations that are fully outside the trim region are always discarded.

Feedback

Table 41 Application Options for Interactive Design Rule Checking (Continued)

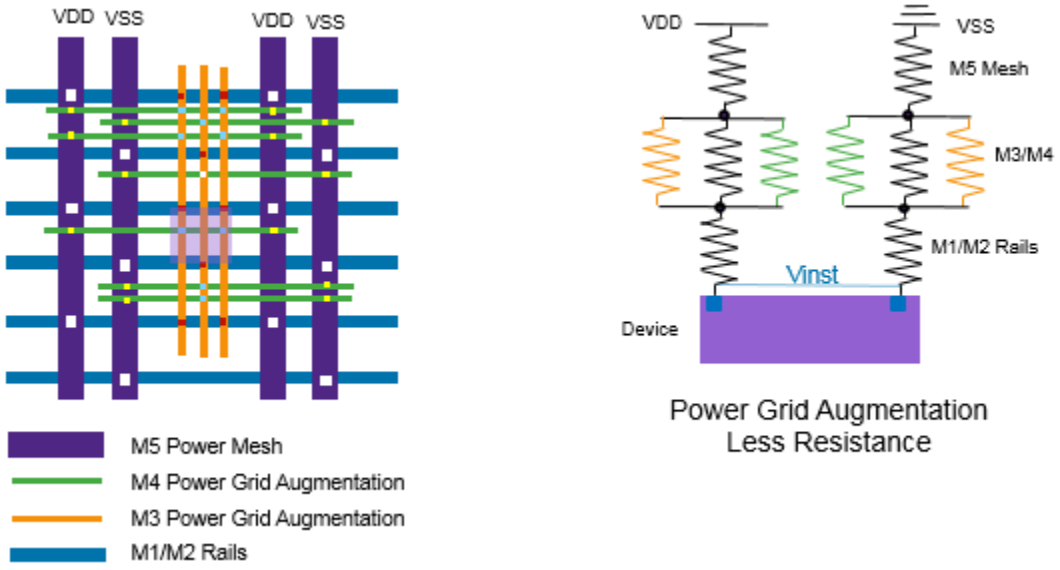
Application option	Default	Description
`signoff.check_drc_live.run_dir`	signoff_check_drc_live_run	Specifies the run directory, which contains the files generated by the interactive signoff design rule checking. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.
`signoff.check_drc_live.runset` (required)	(none)	Specifies the foundry runset to use for design rule checking.
`signoff.check_drc_live.user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.
`signoff.physical.layer_map_file` (required)	(none)	Specifies the name of the layer mapping file that maps the layer names in the technology file to those in the foundry runset. For details about the format of the layer mapping file, see Defining the Layer Mapping for IC Validator In-Design Commands.
`signoff.physical.merge_stream_files`	(none)	Specifies the stream (GDSII or OASIS) files to merge into the current block for interactive design rule checking. When you use this option, the GDSII or OASIS data replaces the cell library data for the cells defined in the specified stream files.

Feedback

Improving Instance Voltage Drop by Augmenting the Power Grid

Power grid augmentation is a technique used to improve the instance voltage drop. When you augment the power grid, the added metal shapes act as parallel resistors, which reduces the resistance of the power grid as seen by the standard cells. Figure 111 shows an example of power grid augmentation and how it lowers the power grid resistance.

Figure 111 Power Grid Augmentation



The IC Compiler II tool supports the following methods of power grid augmentation:

- Standard power grid augmentation, which inserts as many PG augmentation shapes as possible to lower the instance voltage drop
- Timing-driven power grid augmentation, which inserts PG augmentation shapes in the specified regions of the block, except around timing-critical nets
- Guided power grid augmentation, which inserts PG augmentation shapes only from cells with IR violations to their tap with the minimum path resistance

Each method uses RedHawk Fusion analysis to drive the power grid augmentation and uses the IC Validator tool to insert the PG augmentation shapes. For detailed information about performing power grid augmentation, see the topic associated with the method you want to use.

See Also

- RedHawk and RedHawk-SC Fusion

Feedback

Standard Power Grid Augmentation

Standard power grid augmentation inserts as many PG augmentation shapes as possible to lower the instance voltage drop.

To perform standard power grid augmentation,

1. Ensure that the block is fully routed with very few or no DRC violations.
2. Run voltage drop analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis.

   The IC Validator tool uses the results of this voltage drop analysis to drive the power grid augmentation.
3. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
4. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.
5. Set the application options for performing power grid augmentation, as described in Setting Options for Power Grid Augmentation.

   At a minimum, you must define the power and ground nets on which to perform PG augmentation.

   - To specify the power net, set the `signoff.create_pg_augmentation.power_net_name` application option.
   - To specify the ground net, set the `signoff.create_pg_augmentation.ground_net_name` application option.

   If your design contains more than one power net, you must perform PG augmentation once for each power net.
6. Save the block to disk.

   When you run the `signoff_create_pg_augmentation` command, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running the `signoff_create_pg_augmentation` command.
7. Perform power grid augmentation by using the `signoff_create_pg_augmentation` command.

   When you run this command, you must specify the technology node for which to perform the augmentation by using the `-node` option.

Feedback

If you must perform incremental PG augmentation because your design contains more than one power net, use the `-mode add` command on subsequent runs to append the PG augmentation shapes to the existing shapes.

8. Run voltage drop analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis.

   This second voltage drop analysis run measures the voltage drop after power grid augmentation.

During standard PG augmentation, the `signoff_create_pg_augmentation` command performs the following tasks:

1. Inserts PG augmentation shapes for the specified regions

   When the IC Validator tool inserts the PG augmentation shapes, it considers the routing design rules to ensure that it does not create design rule violations.

   By default, the command performs PG augmentation for the entire block. To specify the regions for PG augmentation, use one or more of the following options:

   - `-coordinates_ground`

     This option restricts PG augmentation for the ground net to the specified regions.

   - `-excluded_coordinates_ground`

     This option prevents PG augmentation for the ground net in the specified regions. If you specify this option with the `-coordinates_ground` option, the command does not perform PG augmentation in the overlapping regions.

   - `-coordinates_power`

     This option restricts PG augmentation for the power net to the specified regions.

   - `-excluded_coordinates_power`

     This option prevents PG augmentation for the power net in the specified regions. If you specify this option with the `-coordinates_power` option, the command does not perform PG augmentation in the overlapping regions.

   The shapes added during PG augmentation have a `shape_use` attribute of `pg_augmentation`. You can use this attribute to query the shapes added during PG augmentation. For example,

   ```
   icc2_shell> get_shapes -filter {shape_use=~pg_augmentation}
   ```

Feedback

2. Generates a summary report that shows the following information for each PG net:
   - The distribution of power grid augmentation shapes added on each layer
   - The total number of metal shapes added, the total number of via shapes added, and the total number of all shapes added
3. Saves the updated block to disk

See Also

- Timing-Driven Power Grid Augmentation
- Guided Power Grid Augmentation
- Removing PG Augmentation Shapes

Setting Options for Power Grid Augmentation

Before you run the `signoff_create_pg_augmentation` command, configure the run by setting the application options shown in Table 42. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Table 42 Application Options for Power Grid Augmentation

Application option	Default	Description
`signoff.create_pg_augmentation.ground_net_name` (required)	(none)	Specifies the name of the ground net.
`signoff.create_pg_augmentation.power_net_name` (required)	(none)	Specifies the name of the power net.
`signoff.create_pg_augmentation.run_dir`	signoff_create_pg_augmentation_run	Specifies the run directory, which contains the files generated by the `signoff_create_pg_augmentation` command. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.

Feedback

Table 42 Application Options for Power Grid Augmentation (Continued)

Application option	Default	Description
`signoff.create_pg_augmentation.user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.

Timing-Driven Power Grid Augmentation

Timing-driven power grid augmentation inserts PG augmentation shapes in the specified regions of the block, except around timing-critical nets. By default, the tool does not perform timing-driven PG augmentation.

To perform timing-driven power grid augmentation,

1. Ensure that the block is fully routed with very few or no DRC violations.
2. Run voltage drop analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis.

   The IC Validator tool uses the results of this voltage drop analysis to drive the power grid augmentation.
3. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
4. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.
5. Set the application options for performing power grid augmentation, as described in Setting Options for Power Grid Augmentation.

   At a minimum, you must define the power and ground nets on which to perform PG augmentation.

   - To specify the power net, set the `signoff.create_pg_augmentation.power_net_name` application option.
   - To specify the ground net, set the `signoff.create_pg_augmentation.ground_net_name` application option.

   If your design contains more than one power net, you must perform PG augmentation once for each power net.

Feedback

6. Save the block to disk.

   When you run the `signoff_create_pg_augmentation` command, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running the `signoff_create_pg_augmentation` command.

7. Perform timing-driven power grid augmentation by using the `signoff_create_pg_augmentation` command with one or both of the following options:

   - `-nets`

     This option explicitly specifies the critical nets.

   - `-timing_preserve_setup_slack_threshold`

     This option enables the tool to automatically determine the critical nets based on a setup slack threshold. The unit for the slack threshold setting is the library time unit.

   You must also specify the technology node for which to perform the augmentation by using the `-node` option.

   If you must perform incremental PG augmentation because your design contains more than one power net, use the `-mode add` command on subsequent runs to append the PG augmentation shapes to the existing shapes.

8. Run voltage drop analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis.

   This second voltage drop analysis run measures the voltage drop after power grid augmentation.

During timing-driven PG augmentation, the `signoff_create_pg_augmentation` command

1. Performs timing analysis, including multicorner-multimode analysis, to minimize timing impact
2. Identifies timing-critical nets based on the options you specify
3. Removes existing PG augmentation shapes

   By default, the command removes the PG augmentation shapes from the entire block. To keep the existing PG augmentation shapes, use the `-mode add` option.

4. Inserts PG augmentation shapes for the specified regions, except in the areas around the critical nets

5. Saves the updated block to disk
6. Generates a summary report

See Also

- Standard Power Grid Augmentation
- Guided Power Grid Augmentation
- Removing PG Augmentation Shapes

Guided Power Grid Augmentation

Guided power grid augmentation inserts PG augmentation shapes only near cells with IR violations. For each candidate cell, guided PG augmentation inserts PG augmentation shapes in the region from the cell to its tap with the minimum path resistance. Because the shapes are added only in specific regions, guided PG augmentation inserts fewer metal shapes and uses less routing area, which decreases the coupling capacitance impacts.

To perform guided power grid augmentation,

1. Ensure that the block is fully routed with very few or no DRC violations.
2. Run voltage drop and minimum path resistance analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis and Performing Minimum Path Resistance Analysis.

   The IC Validator tool uses the results of these analyses to drive the power grid augmentation.
3. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
4. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.
5. Save the block to disk.

   When you run the `fix_pg_wire` command, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running the `fix_pg_wire` command.

Feedback

6. Perform guided power grid augmentation by using the `fix_pg_wire` command with one or both of the following options:

   - `-number_pins`

     This option specifies the number of worst IR-drop violators for which to perform PG augmentation.

   - `-voltage_drop_threshold`

     This option enables the tool to automatically determine the cells to process based on a voltage drop threshold.

   In addition, you must specify

   - The power and ground nets on which to perform PG augmentation by using the `-supply_nets` option
   - The technology node for which to perform the augmentation by using the `-node` option

   For information about the `fix_pg_wire` command, see the man page.

7. Run voltage drop analysis using RedHawk Fusion, as described in Performing Voltage Drop Analysis.

   This second voltage drop analysis run measures the voltage drop after power grid augmentation.

See Also

- Standard Power Grid Augmentation
- Timing-Driven Power Grid Augmentation
- Removing PG Augmentation Shapes

Removing PG Augmentation Shapes

To remove PG augmentation shapes, use the `-mode remove` option with the `signoff_create_pg_augmentation` command. When you use this command, the IC Validator tool removes all PG augmentation shapes from the current block.

ECO-Aware Removal of PG Augmentation Shapes

To automatically remove PG augmentation shapes that are causing DRC violations after ECO, use the `-mode remove_by_drc_auto` option with the `signoff_create_pg_augmentation` command. This action removes DRC violating

Feedback

PG augmentation metal shapes or vias from design, including any floating connections created due to removal of DRC violating shape. For example,

```
icc2_shell> signoff_create_pg_augmentation -node <nodename> -mode
 remove_by_drc_auto
```

Performing Timing-Aware Trim PG Augmentation

Timing-aware trim power grid augmentation (PG augmentation) inserts PG augmentation with maximum density without impacting timing QOR. After inserting the PG augmentation, the tool accesses the timing impact and trims it back to maintain its original timing performance.

The `signoff.create_pg_augmentation.timing_driven` and `signoff.create_pg_augmentation.timing_iterations` application option support the timing-aware trim PG augmentation solution.

To perform timing-aware PG augmentation,

1. Run PG augmentation without timing protection and generate maximum PG augmentation.

   Note:
   In this step, clock net protection is turned on.

   The maximum PG augmentation inserted might affect timing QoR due to the PG augmentation added close to the critical nets. To improve the timing QoR, you need PG augmentation trimming. Set the `signoff.create_pg_augmentation.timing_driven` application option to `true` .

   ```
   set_app_options -name signoff.create_pg_augmentation.timing_driven
   -value true;
   ```

2. Run the timer and get critical nets after PG augmentation. Timing data gets updated with the RC impact of the PG augmentation shapes. Protecting nets below slack `0` is recommended to maintain timing close to what it was before PG augmentation. Picking nets or paths below slack `0` is sufficient since the impact of PG augmentation is already considered. Set the `signoff.create_pg_augmentation.timing_iterations` application option to `1` and `2`.

   ```
   1=> PGA with max insertion + PGA trim

   2=> PGA with max insertion + PGA trim + PGA shape removal
   ```

   The `signoff.create_pg_augmentation.timing_iterations` application option shows the number of timer runs and post PG augmentation removal stages. The default for timer runs is `1` and the maximum recommended value is `2` .

Feedback

Note:
The number of iterations takes precedence.

3. Define blockage around nets.
4. Perform smart removal.
   - Removal on the same layer

     Remove PG augmentation around the critical net blocking region by trimming that PG augmentation shape on the same layer over the blocking region. For most cases, part of the shape and vias that are outside are preserved.
   - Removal on the adjacent layer

     Remove PG augmentation around the critical net blocking region by trimming that PG augmentation shape on the adjacent layer over the blocking region. The blockage is created by sizing the critical net shapes. It leaves at least one adjacent track empty next to these critical net shapes.

See Also

- Standard Power Grid Augmentation
- Guided Power Grid Augmentation
- Removing PG Augmentation Shapes

Trim Approach for Optimal QoR and Timing Recovery

The IC Compiler II tool evaluates timing impact after inserting maximum density Power Grid Augmentation (PGA) shapes. After assessing the timing impact, it trims PGA around clock nets and signal nets to recover lost timing performance. This approach allows the insertion of the most optimal amount of PGA shapes into the design while balancing timing QOR.

To maximize PGA QOR preservation with minimal timing impact, follow these steps:

1. Insert PGA.
2. Trim PGA.

Use the following commands:

```
setenv pga_enable_user_spacing_overrides 1
set_app_options -name
 signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode
 -value false
set_app_options -name signoff.create_metal_fill.space_to_nets -value
 { {m1 4x} {m2 4x} {m3 4x} {m4 4x} {m5 4x} {m6 4x} {m7 4x} {m8 4x} {m9
 4x} {m10 4x} {m11 4x} {m12 4x} {m13 4x} {m14 4x} {m15 4x} {m16 4x} }
```

```
signoff_create_pg_augmentation -node tsmc3 -allow_lower_via true
 -output_colored_shapes true -parameter_file ./override_param.rh -mode
 trim_pga_by_timing -max_timing_path 1000000

icc2_shell> setenv pga_enable_user_spacing_overrides 1
set_app_options -name
 signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode
 -value false
set_app_options -name signoff.create_metal_fill.space_to_nets -value
 { {m1 4x} {m2 4x} {m3 4x} {m4 4x} \
{m5 4x} {m6 4x} {m7 4x} {m8 4x} {m9 4x} {m10 4x} {m11 4x} {m12 4x} {m13
 4x} {m14 4x} {m15 4x} {m16 4x} }
signoff_create_pg_augmentation -node tsmc3 -allow_lower_via true
 -output_colored_shapes true -parameter_file \
./override_param.rh -mode trim_pga_by_timing -max_timing_path 1000000
```

For aggressive timing recovery, follow these steps:

1. Insert PGA.
2. Trim PGA.

Use the following commands:

```
setenv pga_enable_user_spacing_overrides 1
set_app_options -name
 signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode
 -value false
signoff_create_pg_augmentation -node tsmc3 -allow_lower_via true
 -output_colored_shapes true -parameter_file ./override_param.rh
 -mode trim_pga_by_timing -timing_preserve_setup_slack_threshold 0.0
 -max_timing_path 0

icc2_shell> setenv pga_enable_user_spacing_overrides 1
set_app_options -name
 signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode
 -value false
signoff_create_pg_augmentation -node tsmc3 -allow_lower_via true
 -output_colored_shapes true -parameter_file \
./override_param.rh -mode trim_pga_by_timing
 -timing_preserve_setup_slack_threshold 0.0 -max_timing_path 0
```

Feedback

When you use the `trim_pga_by_timing` mode, the following two command-line options have specific differences:

1. `-max_paths`
   a. **Focus**: Path-based analysis.
   b. **Behavior**:
      i. Considers the top N worst timing paths. Default: 100.
      ii. Analyzes and optimizes only the net shapes between the source pin and sink pin of a timing-critical path.
      iii. Trims PGA shapes and vias around these selected net shapes within a certain spacing to recover timing.
   c. **Use Case**: Useful for focusing on the most critical timing paths in the design. For example, you can optimize the top 10000 worst paths to improve overall timing after the default PGA flow.
2. `-timing_preserve_setup_slack_threshold`
   a. **Focus**: Net-based analysis.
   b. **Behavior**:
      i. Considers all nets with slack less than the specified threshold.
      ii. Preserves timing for these nets by preventing PGA shapes within a certain spacing of the entire net shapes associated with these critical nets.
      iii. Unlike `-max_timing_paths`, it does not limit the analysis to specific paths but instead considers all shapes of a net.
   c. **Use Case**: Useful for protecting all timing-critical nets below a specific slack threshold. For example, you can ensure that no net with slack below 0.1 is impacted by PGA shapes.

The following table summarizes the key differences between `-max_timing_paths` and `-timing_preserve_setup_slack_threshold`:

Aspect	`-max_timing_paths`	`-timing_preserve_setup_slack_threshold`
Scope	Top N worst timing paths	All nets with slack below the threshold
Granularity	Path-level (source pin → sink pin)	Net-level (entire net shapes)

Optimization	Trims PGA shapes around selected path segments	Trims PGA shapes near all critical net shapes
Use Case	Focus on worst paths to improve timing	Protect all timing-critical nets

Consider the following examples:

- `-max_timing_paths 1000`: Optimizes the top 1000 worst timing paths by trimming PGA shapes around the net segments between source and sink pins.
- `-timing_preserve_setup_slack_threshold 0.0`: Protects all nets with slack below 0.0 by preventing PGA shapes near their entire net shapes.

In summary, `-max_paths` is path-centric and focuses on specific paths, while `-timing_preserve_setup_slack_threshold` is net-centric and focuses on all critical nets.

PG Augmentation Advanced User Mode for Improved Ease of Use

To enhance the usability of power grid augmentation (PG Augmentation), including validation checks and automatic setting of PG augmentation values, the IC Compiler II tool improves robustness in advanced or custom modes. You can modify the PG augmentation width and spacing recipe using a parameter file.

The PG augmentation parameter file checker verifies whether the syntax of the parameter file is defined correctly. If the syntax is incorrect, the checker issues either a warning or an error message during PG augmentation. You can enable the parameter file checker by setting the `signoff.create_pg_augmentation.parameter_file_check_mode` application option.

```
icc2_shell> set_app_options -name
 signoff.create_pg_augmentation.parameter_file_check_mode  -value
 [off|warn|error]
```

- By default, the checker is set to `off`.
- If you set the value to `warn` or `error`, the tool issues a warning or an error message, respectively.

Note:
When the checker is enabled, it adds description for each parameter file syntax in the `track_fill_params.rh` file located in the PG augmentation directory.

The tool implements validation checks to detect invalid parameter values early. If you provide incorrect values, the tool immediately generates an error message instead of

proceeding to IC Validator (ICV) and producing undesired PG augmentation results. These checks ensure that all possible parameters used in PG augmentation are validated.

The following table outlines the parameters that require validation:

Table 43 PG Augmentation Parameters

PG Augmentation Parameters	Types of Validation Check
`mx_pga2route_side_spacing = double_val;`	Advanced mode to specify side spacing to keep from route shapes by PG augmentation shapes.
`mx_pga2power_side_spacing = double_val;`	Advanced mode to specify PGA-to-PGA side spacing for power net.
`mx_pga2gound_side_spacing = double_val;`	Advanced mode to specify PGA-to-PGA side spacing for ground net.
`mx_pga2pga_side_spacing = double_val;`	Advanced mode to specify PGA-to-PGA side spacing for both power and ground nets.
`mx_pg_augmentation_disable_layer = 1 | 0; // default 0`	Option to disable PGA on a particular layer. Generally used to block PGA on lower most or upper most metal layers only

Note:

Substitute `mx_` by `m1_`, `m2_`, and so on, for the respective layer to which the parameter applies.

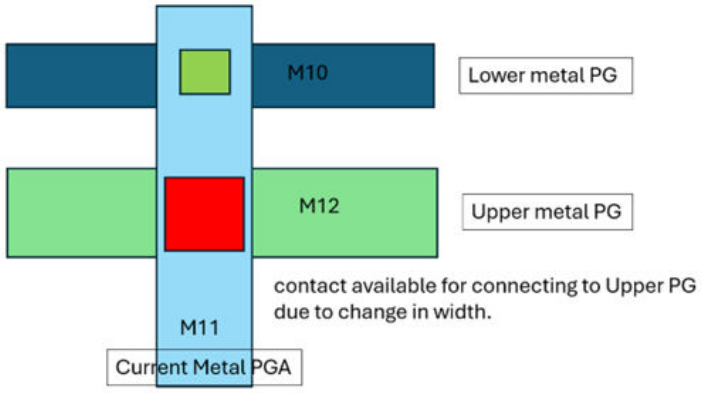
Automating Transition Metal and Via Width Using Power Grid Augmentation

Power grid augmentation (PGA) creates the metal shapes of default width. A valid via is selected from the technology file based on the lower and upper metal width. In some cases, the default metal width specified in the technology file might prevent valid via connections between lower and upper layers, which impacts the PGA connectivity and affect PGA-QoR. This can be resolved by calculating the suitable combination of lower and upper metal width to get a valid via from technology file.

For non-transitional layers, a valid via exists when the lower and upper metal layers have the same width. For transitional metals layers, the technology file may not have a valid default via contact because the lower and upper metal layers have different default widths.

Feedback

Figure 112 Default Via Contact in Transitional Metal Layers



In such a case, to improve PGA-QoR, it is required to increase metal width of the lower metal layer by calculating metal width automatically based on technology file rules and contact information from technology file.

The width of PGA shapes is automatically adjusted when the default width from the technology file does not support the formation of via connections on both sides. This adjustment ensures a valid via connection for both lower and upper layers, enhancing the QoR of PGA.

Inserting Metal Fill With IC Validator In-Design

After routing, you can fill the empty spaces in the block with fill shapes to meet the metal density rules required by most fabrication processes. Before inserting metal and via fill, the block should be close to meeting timing and have only a very few or no DRC violations.

To insert metal fill,

1. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
2. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.
3. Set the application options for metal fill insertion.

   For information about the metal fill insertion options, see Setting Options for Signoff Metal Fill Insertion.

Feedback

4. Save the block to disk.

   When you run the `signoff_create_metal_fill` command, the IC Validator tool uses the on-disk information for the block, not the information in memory. To ensure accurate information, use the `save_block` command to save the current state of the block before running the `signoff_create_metal_fill` command.

5. Perform metal fill insertion by using the `signoff_create_metal_fill` command as described in Performing Metal Fill Insertion.

   The `signoff_create_metal_fill` command respects routing blockages defined by the `create_routing_blockage` command and does not insert metal fill in the blockages; however, it does not respect routing guides created by the `create_routing_guide` command.

   Note:

   If you are performing pattern-based metal fill insertion and the blockage layer numbers differ between the technology file and the foundry runset file, you must provide a layer mapping file, as described in Defining the Layer Mapping for IC Validator In-Design Commands.

When the IC Validator tool performs metal fill insertion, it creates an internal subdesign named *block*.FILL in the design view of the block and inserts the fill cells in this internal subdesign. The fill cells are named FILL_INST_#. To query the fill cells, use the `get_fill_cells` command.

For information about the result files generated by the `signoff_create_metal_fill` command, see Signoff Metal Fill Result Files.

After you insert metal fill, you can

- Display the added metal fill in the layout view in the GUI, as described in Viewing Metal Fill in the GUI
- Modify the metal fill, as described in Modifying Metal Fill
- Perform extraction for timing analysis using the real metal fill, as described in Performing Real Metal Fill Extraction

Setting Options for Signoff Metal Fill Insertion

Before you run the `signoff_create_metal_fill` command, configure the run by setting the application options shown in Table 44. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Feedback

Table 44 Application Options for Signoff Metal Fill Insertion

Application option	Default	Description
Options that apply to all flows		
`signoff.create_metal_fill.apply_nondefault_rules`	`false`	Controls whether metal fill insertion honors nondefault routing rules.
`signoff.create_metal_fill.auto_eco_threshold_value`	20	Specifies the maximum percentage of change to the block to perform incremental metal fill insertion using the `-auto_eco true` option.
`signoff.create_metal_fill.flat`	`false`	Controls whether the IC Validator tool uses the metal fill mode specified in the runset file (`false`) or uses the flat metal fill mode (`true`).
`signoff.create_metal_fill.read_design_views`	{}	Specifies the reference cells for which the IC Validator tool reads the design view instead of the frame view. Using the design view can expose problems that are masked by the frame view abstraction.
`signoff.create_metal_fill.read_layout_views`	{}	Specifies the reference cells for which the IC Validator tool reads the layout view instead of the frame view. Using the layout view can expose problems that are masked by the frame view abstraction.
`signoff.create_metal_fill.run_dir`	signoff_fill_run	Specifies the run directory, which contains the files generated by the `signoff_create_metal_fill` command. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.
`signoff.create_metal_fill.user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.
`signoff.physical.merge_exclude_libraries`	(none)	Specifies the cell libraries whose cells are excluded from replacement with the cell data in the stream files specified in the `signoff.physical. merge_stream_files` application option.

Feedback

Table 44 Application Options for Signoff Metal Fill Insertion (Continued)

Application option	Default	Description
`signoff.physical.merge_stream_files`	(none)	Specifies the stream (GDSII or OASIS) files to merge into the current block for metal fill insertion. When you use this option, the GDSII or OASIS data replaces the cell library data for the cells defined in the specified stream files.
Options that apply only to pattern-based metal fill insertion		
`signoff.create_metal_fill.runset`	N/A	Specifies the foundry runset to use for pattern-based metal fill insertion. This application option is not used for track-based metal fill insertion
`signoff.physical.layer_map_file`	(none)	Specifies the name of the layer mapping file. In general, the technology file and the foundry runset file used by the IC Validator tool use the same layer numbers. If they do not, you must supply a layer mapping file to map the technology layers to the layers used in the runset file (for details about the format of the layer mapping file, see Defining the Layer Mapping for IC Validator In-Design Commands.
Options that apply only to track-based metal fill insertion		
`signoff.create_metal_fill.max_density_threshold`	(none)	Specifies the maximum density threshold value for a track fill layer. Use the following format to specify this information for each layer: `{layer_name max_density}` This application option applies only to track-based metal fill insertion and does not apply when you use the `-mode add` option.
Options that apply only to timing-driven metal fill insertion (pattern-based or track-based)		
`signoff.create_metal_fill.fill_over_net_on_adjacent_layer`	`false`	Controls whether the IC Validator tool inserts metal fill within the minimum spacing in the vertical extension of the net on the adjacent layers when performing timing-driven metal fill insertion.
`signoff.create_metal_fill.fill_shielded_clock`	`false`	Controls whether the IC Validator tool inserts metal fill for shielded clock nets when performing timing-driven metal fill insertion.

Feedback

Table 44 Application Options for Signoff Metal Fill Insertion (Continued)

Application option	Default	Description
`signoff.create_metal_fill.fix_density_errors`	`false`	Controls whether the IC Validator tool performs density error fixing during timing-driven metal fill insertion.
`signoff.create_metal_fill.space_to_clock_nets`	Two times the minimum spacing specified in the technology file	Specifies the minimum spacing between metal fill and a clock net on the same layer. Note that you must specify clock nets by using the `-nets` option. Any unspecified layers use the default spacing.
`signoff.create_metal_fill.space_to_nets`	Two times the minimum spacing specified in the technology file	Specifies the minimum spacing between metal fill and a timing-critical net on the same layer. Any unspecified layers use the default spacing.
`signoff.create_metal_fill.space_to_nets_on_adjacent_layer`	Two times the minimum spacing specified in the technology file	Specifies the minimum spacing between metal fill and a timing-critical net on an adjacent layer. Any unspecified layers use the default spacing.

Performing Metal Fill Insertion

You can use the `signoff_create_metal_fill` command to perform the following tasks:

- Pattern-Based Metal Fill Insertion
- Track-Based Metal Fill Insertion
- Using an IC Validator Parameter File
- Typical Critical Dimension Metal Fill Insertion
- Timing-Driven Metal Fill Insertion
- Incremental Metal Fill Insertion
- Foundry Fill and Track Fill Combined for Different Flows of Foundry

Pattern-Based Metal Fill Insertion

Pattern-based metal fill insertion is the default mode for the `signoff_create_metal_fill` command. It inserts metal and via fill by using a foundry runset. Before running the

Feedback

`signoff_create_metal_fill` command, you must specify the runset by setting the `signoff.create_metal_fill.runset` application option.

During pattern-based metal fill insertion, the `signoff_create_metal_fill` command performs the following tasks:

1. Loads the block into the IC Validator tool, as described in Reading Blocks for Signoff Metal Fill Insertion
2. Removes existing metal and via fill

   By default, the command removes the metal and via fill from the entire block. For information about performing incremental metal fill insertion, see Incremental Metal Fill Insertion.
3. Inserts metal and via fill in the empty regions

   By default, the command uses the spacing rules defined in the technology file to perform metal fill insertion on all metal and via layers for the entire block. The command uses the metal fill mode specified in the runset file, which is either hierarchical or flat.

   To modify the default behavior,

   - Set one or more of the following application options before running the `signoff_create_metal_fill` command:
     - `signoff.create_metal_fill.apply_nondefault_rules`

       To enable the use of nondefault spacing rules in addition to the spacing rules defined in the technology file, set this application option to `true`.
     - `signoff.create_metal_fill.flat`

       To force the use of the flat metal fill mode, set this application option to `true`.
   - Use one or more of the following command options with the `signoff_create_metal_fill` command:
     - `-select_layers` or `-all_runset_layers`

       To restrict the layers on which to insert metal fill, use these options, as described in Specifying the Layers for Metal Fill Insertion.
     - `-coordinates` and `-excluded_coordinates`

       To restrict the regions on which to insert metal fill, use these options, as described in Specifying the Regions for Metal Fill Insertion.
4. Removes floating via fill from the specified via layers and from the via layers associated with the specified metal layers

Feedback

Floating via fill is via fill that does not have both an upper and lower metal enclosure.

5. Saves the fill data in the design view
6. Writes the result files to the run directory

   For information about the generated result files, see Signoff Metal Fill Result Files.

See Also

- Timing-Driven Metal Fill Insertion

Reading Blocks for Signoff Metal Fill Insertion

By default, the IC Validator tool reads the design view for the top-level block and the library cell instances, and the pin information from the frame view for the macro cell and I/O pad cell instances.

- To read the GDSII or OASIS data for specific reference cells, specify the stream files by setting the `signoff.physical.merge_stream_files` application option.

  When you use this option, the GDSII or OASIS data replaces the cell library view for the cells defined in the specified stream files, except for cells in the cell libraries specified with the `signoff.physical.merge_exclude_libraries` application option.

- To read the layout view for specific reference cells, specify the cells by setting the `signoff.create_metal_fill.read_layout_views` application option.

  The layout view is identical to the GDSII or OASIS data that was used to create them, but reduces the runtime for signoff design rule checking.

  Note:

  By default, layout views are not included in the cell libraries To save the layout views, you must set the `lib.workspace.save_layout_views` application option to `true` during library preparation, as described in the *Library Manager User Guide*.

- To read the design view for specific reference cells, specify the cells by setting the `signoff.create_metal_fill.read_design_views` application option.

  Note:

  By default, design views are not included in the cell libraries To save the design views, you must set the `lib.workspace.save_design_views` application option to `true` during library preparation, as described in the *Library Manager User Guide*.

Feedback

The order of precedence for child cell data is

1. GDSII or OASIS data specified with the `signoff.physical.merge_stream_files` application option
2. Layout views specified with the `signoff.create_metal_fill.read_layout_views` application option

   If the IC Validator tool cannot find a layout view and the cell is specified in the `signoff.check_drc.read_design_views` application option, it reads the design view. Otherwise, it reads the frame view. If it cannot find the frame view, the cell is ignored during design rule checking.
3. Design views specified with the `signoff.create_metal_fill.read_design_views` application option

   If the IC Validator tool cannot find a design view, it reads the frame view instead. If it cannot find the frame view, the cell is ignored during design rule checking.
4. Frame views

Specifying the Layers for Metal Fill Insertion

By default, the `signoff_create_metal_fill` command inserts metal fill on all the metal routing layers and via fill on all the via layers. To modify the layers for metal fill insertion, use one of the following options:

- `-select_layers` *`metal_fill_layers`*

  This option restricts metal fill insertion to the specified set of metal and via layers.
- `-all_runset_layers true`

  This option enables metal fill insertion on all the fill layers specified in the runset. This option applies only to pattern-based metal fill insertion.

When you specify the layers for metal fill insertion, the `signoff_create_metal_fill` command

1. Removes the existing metal and via fill from the block

   By default, the command removes all existing metal and via fill. For incremental metal fill insertion, the command removes metal and via fill only from the specified layers.
2. Inserts metal and via fill only on the specified layers
3. Removes floating via fill from the specified via layers and from the via layers associated with the specified metal layers

   For example, if you specify `-select_layers {M2}`, the tool removes floating via fill from the V1 and V2 layers.

Feedback

Specifying the Regions for Metal Fill Insertion

By default, the `signoff_create_metal_fill` command inserts metal and via fill for the whole chip. To modify the regions for metal fill insertion, use one or both of the following options:

- `-coordinates`

  This option restricts metal fill insertion to the specified regions.

  Note:

  The bounding box coordinates passed to the IC Validator tool in the `METAL_FILL_SELECT_WINDOW` parameter are enlarged by 1 um to avoid DRC violations on the boundary of the specified regions during metal fill insertion. The actual metal fill insertion occurs within the regions specified by the `-coordinates` option.

  In addition, when you use the `-coordinates` option, the `signoff_create_metal_fill` command always uses the flat metal fill mode.

- `-excluded_coordinates`

  This option prevents metal fill insertion in the specified regions. If you specify this option with the `-coordinates` option, the command does not perform metal fill insertion in the overlapping regions.

To specify the coordinates, use the following format:

$\{\{x_1\ y_1\}\ \{x_2\ y_2\}\ \ldots\ \{x_n\ y_n\}\}$

Note that there must be a space between each set of coordinates.

When you specify the regions for metal fill insertion, the `signoff_create_metal_fill` command

1. Removes all existing metal and via fill from the block

   For standard metal fill insertion, the command removes all existing metal and via fill. For incremental metal fill insertion, the command removes metal and via fill only from the specified regions.

2. Inserts metal and via fill only on the specified regions

3. Removes floating via fill from the specified regions

For example, to remove all metal fill from the block and then fill all empty regions outside the bounding box with corners at (100,150) and (300,200), use the following command:

```
icc2_shell> signoff_create_metal_fill \
   -excluded_coordinates {{100 150} {300 200}}
```

Feedback

Track-Based Metal Fill Insertion

Track-based metal fill insertion inserts metal and via fill by using a runset derived from the attributes and rules in the technology file to create fill shapes aligned to tracks. It does not use the runset specified by the `signoff.create_metal_fill.runset` application option. Track-based metal fill insertion offers the following benefits as compared to pattern-based metal fill insertion:

- Higher density
- Better control of density
- Well-balanced mask distribution for double-patterning layers

To perform track-based metal fill insertion, set the `-track_fill` option of the `signoff_create_metal_fill` command to a value other than `off`. Track-based metal fill insertion supports three modes:

- Sparse

  This is the default mode. In this mode, track-based metal fill insertion skips one track between signal shapes and fill shapes.

- Dense

  To enable this mode, use the `-fill_all_tracks true` option. In this mode, track-based metal fill insertion does not skip tracks between signal shapes and fill shapes.

  Note:

  If your block uses double-patterning technology, using this option increases runtime if the block is not precolored.

- Mixed

  To use sparse mode for some layers and dense mode for other layers, you must set the appropriate parameters in a parameter file, as described in Using an IC Validator Parameter File.

During track-based metal fill insertion, the `signoff_create_metal_fill` command performs the following tasks:

1. Loads the block into the IC Validator tool, as described in Reading Blocks for Signoff Metal Fill Insertion
2. Removes existing metal and via fill

   By default, the command removes the metal and via fill from the entire block. For information about performing incremental metal fill insertion, see Incremental Metal Fill Insertion.

Feedback

3. Inserts metal and via fill in the empty regions

   By default, the command uses the design rules defined in the technology file to insert fill shapes on-track on all metal and via layers for the entire block. The fill shapes have a length of 5 microns and a width equal to the `defaultWidth` attribute defined for the layer in the technology file.

   You can modify the default behavior by using

   - Application options set before running the `signoff_create_metal_fill` command:
     - `signoff.create_metal_fill.apply_nondefault_rules`

       To enable the use of nondefault spacing rules in addition to the spacing rules defined in the technology file, set this application option to `true`.
   - Command options used with the `signoff_create_metal_fill` command:
     - `-track_fill` *`foundry_node`*

       To enable foundry-specific design rules, use the appropriate foundry keyword with the `-track_fill` option. To see the list of supported keywords, use the following command:

       ```
       icc2_shell> signoff_create_metal_fill -track_fill list
       ```

     - `-select_layers`

       To restrict the layers on which to insert metal fill, use this option, as described in Specifying the Layers for Metal Fill Insertion.

     - `-coordinates` and `-excluded_coordinates`

       To restrict the regions on which to insert metal fill, use these options, as described in Specifying the Regions for Metal Fill Insertion.
   - Parameters set in the IC Validator parameter file:
     - `mx_fill_width`, `mx_min_fill_length`, and `mx_max_fill_length`

       To change the size of the fill shapes, set these parameters, as described in Using an IC Validator Parameter File.

4. Removes floating via fill from the specified via layers and from the via layers associated with the specified metal layers

   Floating via fill is via fill that does not have both an upper and lower metal enclosure.

Feedback

5. (Optional) Trims the metal fill for each layer to try to meet the maximum density threshold defined by the `signoff.create_metal_fill.max_density_threshold` application option

   The metal fill is trimmed only for those layers specified in the application option. If you do not set this option, the metal fill is not trimmed.

6. Saves the fill data in the design view

   By default,

   - The IC Validator tool assigns a data type of 0 to the fill shapes

     To use different data type values, set the `mx_fill_datatype` and `viax_fill_datatype` parameters in the IC Validator parameter file.

   - The IC Validator tool does not set mask constraints on the fill shapes

     If your block uses double-patterning technology, use the `-output_colored_fill true` option to set mask constraints on the fill shapes.

     If your block uses double-patterning technology, use the `-output_colored_fill` option to set mask constraints on the fill shapes. When you use this option, the IC Validator tool assigns a data type of 235 for fill shapes with a `mask_one` mask constraint and 236 for fill shapes with a `mask_two` mask constraint. To assign different data types for the colored fill, set the following parameters in the IC Validator parameter file: `mx_fill_datatype_color1`, `viax_fill_datatype_color1`, `mx_fill_datatype_color2`, and `viax_fill_datatype_color2`.

   - The IC Validator tool does not write exclude layers

     If your foundry uses exclude layers, define the data types for the exclude layers by setting the `mx_exclude_layer_datatype` parameters in the IC Validator parameter file. The IC Validator tool outputs the exclude layers that have defined data types.

   For information about using an IC Validator parameter file, see Using an IC Validator Parameter File.

7. Writes the result files to the run directory

   In addition to the standard output files generated by signoff metal fill insertion, when you perform track-based metal fill insertion, you can output detailed density and density gradient reports by using the `-report_density` option. When you enable this option, the `signoff_create_metal_fill` command writes the following report files:

   - *prefix*_color_balance_and_density_report.txt
   - *prefix*_fill_density_gradient_report.txt

Feedback

You can specify either `on` (in which case the tool uses “track_fill” as the prefix) or the prefix string. For example, to perform track-based metal fill insertion and use a prefix of my_prefix for the detailed density and density gradient reports, use the following command:

```
icc2_shell> signoff_create_metal_fill -track_fill true \
   -report_density my_prefix
```

For information about the generated result files, see Signoff Metal Fill Result Files.

See Also

- Timing-Driven Metal Fill Insertion

Using an IC Validator Parameter File

You can customize the metal fill insertion by using an IC Validator parameter file.

- For pattern-based metal fill insertion, you pass the parameter file to the IC Validator tool by setting the `signoff.create_metal_fill.user_defined_options` application option.

  ```
  icc2_shell> set_app_options \
     -name signoff.create_metal_fill.user_defined_options \
     -value {-D INDESIGN_USER_DEFINED_PARAM_FILE=file_name}
  ```

- For track-based metal fill insertion, you pass the parameter file to the IC Validator tool by using the`-track_fill_parameter_file` option with the `signoff_create_metal_fill` command.

  ```
  icc2_shell> signoff_create_metal_fill -track_fill true \
     -track_fill_parameter_file file_name
  ```

  When you run track-based metal fill insertion, the IC Validator tool writes a parameter file named `track_fill_params.rh` into the run directory. This file contains the default settings for the supported parameters. You can modify this file as necessary and use it on subsequent `signoff_create_metal_fill` runs.

Note:

In some cases, the IC Validator behavior can be controlled by either an IC Validator parameter or an application option. In these cases, if both are specified, the IC Validator parameter setting overrides the application option setting.

Table 45 shows the commonly used parameters supported in the parameter file. Unless otherwise specified, these parameters apply only to track-based metal fill insertion. The `mx` or `vx` prefix in the parameters refers to the mask name, which can be found in the layer section of the technology file.

Feedback

For example, if layer `M11` has the mask name `metal12` in the technology file, the prefix `m12` would replace `mx` in the parameters below. Therefore, the M11 layer would have the parameters `m12_spare_fill`, `m12_fill_staggering`, and so on.

Table 45 IC Validator Metal Fill Insertion Parameters

Parameter	Default	Description
Metal fill insertion mode parameters		
`mx_sparse_fill`	1	Controls whether metal fill insertion uses sparse mode (1) or dense mode (0) for the specified metal layer. **Note:** If you use the `-fill_all_tracks true` option, the default changes to 0.
`mx_fill_staggering`	0.075 microns	Controls staggering for fill patterns.
`mx_ignore_route_guide`	0	Controls whether the IC Validator tool honors (0) or ignores (1) routing guides on the specified layer during metal fill insertion.
`mx_ignore_system_blockage`	0	Controls whether the IC Validator tool honors (0) or ignores (1) system blockages on the specified layer during metal fill insertion.
`mx_via_enclosure`	Derived from `ContactCode` section	Specifies the minimum metal enclosure values for vias.
Fill size and spacing parameters		
`mx_fill_width`	`defaultWidth`[9]	Specifies the width of the metal fill shapes.
`mx_min_fill_length`	Derived from `minArea`[9]	Specifies the minimum length of the metal fill shapes.
`mx_max_fill_length`	5 microns	Specifies the maximum length of the metal fill shapes. Decreasing this value can increase the via density at the expense of decreasing the metal density.
`mx_fill2fill_side_spacing`		Specifies the minimum spacing between fill shapes.
`mx_fill2route_side_spacing`		Specifies the minimum spacing between metal fill and net shapes.

9. *Attribute setting from the Layer section of the technology file*

Feedback

Table 45 IC Validator Metal Fill Insertion Parameters (Continued)

Parameter	Default	Description
mx_fill2fill_end_ spacing		Specifies the minimum end-to-end spacing between fill shapes.
mx_fill2route_end_ spacing		Specifies the minimum end-to-end spacing between metal fill and net shapes.
mx_fill2Blockage_x mx_fill2Blockage_y		Specifies the minimum spacing between fill shapes and blockages.
mx_fill2routeGuide_x mx_fill2routeeGuide_y		Specifies the minimum spacing between fill shapes and routing guides.
mx_fill2chipBoundary_x mx_fill2chipBoundary_y		Specifies the minimum spacing between fill shapes and the chip boundary.
mx_min_area_with_via		Specifies the minimum fill area on which vias can be created.
mx_EXCLUDED_CELLS_ OVERSIZE_VALUE	0.2 microns	Specifies the minimum spacing between the macros specified in the mx_EXCLUDED_CELLS parameter and fill shapes.
mx_EXCLUDED_CELLS	{}	Specifies the macros that use the spacing defined in the mx_EXCLUDED_CELLS_OVERSIZE_VALUE parameter.
vx_iso_via_distance	3.0 microns	Specifies the maximum distance within which a neighboring via must exist.
vx_min_spacing	minSpacing [9]	Specifies the minimum spacing between vias. Be careful when changing the value of these parameters. A value smaller than the default can cause DRC violations, while a value larger than the default negatively impacts the via density.
vx_per_metal	1	Specifies the maximum number of via fill shapes contained in a metal fill shape. You can use this parameter to increase the via density. However, changing the default can cause many metal fill shapes to be connected to each other, which can increase the overall capacitance for the nearby signal nets.

Exclude layer parameters

Feedback

Table 45 IC Validator Metal Fill Insertion Parameters (Continued)

Parameter	Default	Description
mx_exclude_layer_ datatype	-1	Creates an exclude layer with the specified data type. The default setting of -1 prevents the creation of an exclude layer.
Fill output mode parameters		
mx_compress_fill vx_compress_fill	0	Controls whether the fill is compressed (1) or uncompressed (0) for the specified layer. **Note:** If you use the -mode add option, compression is not performed regardless of the setting of this parameter.
mx_fill_datatype vx_fill_datatype	0	Specifies the data type of the inserted fill shapes.
output_colored_fill	0	Controls whether fill shapes are assigned mask constraints in blocks that use double-patterning technology. **Note:** If you use the -output_colored_fill true option, the default changes to 1.
mx_fill_datatype_ color1 vx_fill_datatype_ color1 mx_fill_datatype_ color2 vx_fill_datatype_ color2	0	Specifies the data type of the inserted colored fill shapes. **Note:** This parameter is used only colored fill is enabled.
Density calculation parameters		
exclude_bounding_box_ blockage_for_density_ computation	1	Controls whether the regions specified by the -excluded_coordinates option are excluded from the density computation.
exclude_route_guides_ for_density_computation	0	Controls whether routing guides are excluded from the density computation.
exclude_system_metal_ blockages_for_density_ computation	0	Controls whether system metal blockages are excluded from the density computation.

Feedback

Table 45 IC Validator Metal Fill Insertion Parameters (Continued)

Parameter	Default	Description
`mx_density_gradient_window` [10]	`windowSize` [9]	Specifies the window size used for density gradient checking. You can also control the window size by setting the `signoff.report_metal_density.gradient_window_size` application option. **Note:** If the `windowSize` attribute is not specified in the technology file and the `signoff.report_metal_density.gradient_window_size` application option is not set, the tool uses a default of 50 microns.
`mx_max_density_window`	50 microns	Specifies the window size used for maximum density checking.
`mx_max_open_area_rule` [10]		Specifies the maximum size of a square area that contains no polygons and does not interact with any polygons.
`mx_min_density` [10]	`minDensity` [9]	Specifies the minimum percentage of metal allowed in the window. You can also control the window size by setting the `signoff.report_metal_density.min_density` application option. **Note:** If the `minDensity` attribute is not specified in the technology file and the `signoff.report_metal_density.min_density` application option is not set, the tool uses a default of 10 percent.
`mx_min_density_window` [10]	`windowSize` [9]	Specifies the window size used for minimum density checking. You can also control the window size by setting the `signoff.report_metal_density.density_window` application option. **Note:** If the `windowSize` attribute is not specified in the technology file and the `signoff.report_metal_density.density_window` application option is not set, the tool uses a default of 50 microns.

10. This parameter applies to timing-driven metal fill insertion, whether pattern-based or track-based.

Feedback

Table 45 IC Validator Metal Fill Insertion Parameters (Continued)

Parameter	Default	Description
`mx_window_step_size`	Half the window size	Specifies the step size in the x- and y-directions used for density checking. The first value is the x-direction step size. The second value is the y-direction step size. These can be the same or different values. You can also control the step size by setting the `signoff.report_metal_density.density_window_step` application option. When you use this application option, it uses the specified value for the step size in both the x- and y-directions.

Typical Critical Dimension Metal Fill Insertion

Typical critical dimension (TCD) structures can improve yield for designs. These structures are made up of marker and fill layers, both of which are added to the block during TCD metal fill insertion.

To enable TCD metal fill insertion, set the `signoff.create_metal_fill.tcd_fill` application option to `true` before running the `signoff_create_metal_fill` command.

By default, the command inserts TCD structures on all metal and via layers for the entire block. To modify the default behavior, use the following options:

- `-select_layers`

  To restrict the layers on which to insert metal fill, use this option, as described in Specifying the Layers for Metal Fill Insertion.

- `-coordinates` and `-excluded_coordinates`

  To restrict the regions on which to insert metal fill, use these options, as described in Specifying the Regions for Metal Fill Insertion.

Note:

You cannot use timing-driven metal fill insertion with TCD metal fill insertion.

See Also

- Removing Metal Fill

Timing-Driven Metal Fill Insertion

Timing-driven metal fill insertion inserts metal and via fill in the specified regions of the block, except around timing-critical nets. Timing-driven metal fill insertion is supported

Feedback

for both pattern-based and track-based metal fill insertion. By default, the tool does not perform timing-driven metal fill insertion.

To perform timing-driven metal fill insertion, use one or both of the following options with the `signoff_create_metal_fill` command:

- `-nets`

  This option explicitly specifies the critical nets.

- `-timing_preserve_setup_slack_threshold`

  This option enables the tool to automatically determine the critical nets based on a setup slack threshold. The unit for the slack threshold setting is the library time unit.

  Be careful when choosing the threshold value; using a large threshold value can result in too many critical nets, which could reduce the metal density and create large empty areas.

During timing-driven metal fill insertion, the `signoff_create_metal_fill` command

1. Performs timing analysis, including multicorner-multimode analysis, to minimize timing impact
2. Identifies timing-critical nets based on the options you specify
3. Removes existing metal and via fill

   By default, the command removes the metal and via fill from the entire block. For information about performing incremental metal fill insertion, see Incremental Metal Fill Insertion.

4. Inserts metal and via fill in the empty regions for the specified regions, except in the areas around the critical nets

   To enable fill insertion for shielded timing-critical clock nets, set the `signoff.create_metal_fill.fill_shielded_clock` application option to `true`.

5. Invokes the IC Validator tool to perform metal fill insertion

   By default, the minimum spacing between the metal fill and the net shapes of the critical nets is two times the minimum spacing specified in the technology file. This spacing requirement also applies to the vertical extension of the critical net on the adjacent layers. To modify these requirements, set the appropriate application options, as described in Specifying the Spacing Requirements for Timing-Driven Metal Fill Insertion.

   Note:

   You can restrict the regions in which to perform timing-driven metal fill insertion, as described in Specifying the Regions for Metal Fill Insertion

Feedback

and Performing Metal Fill Insertion Only in Modified Regions; however, you cannot specify the layers. When performing timing-driven metal fill insertion, the `signoff_create_metal_fill` command inserts metal fill on all routing layers (or all changed layers, if you perform metal fill insertion only in the changed regions).

6. (Optional) Fixes density errors

   In some cases, the spacing requirements for timing-driven metal fill insertion might cause density errors. To fix these errors automatically during timing-driven metal fill insertion, set the `signoff.create_metal_fill.fix_density_errors` application option to `true` (the default is `false`). For information about the density design rules, see Defining the Density Design Rules.

7. Stores the metal fill data as fill cell objects in the design view

8. Writes the result files to the run directory

   For information about the generated result files, see Signoff Metal Fill Result Files.

See Also

- Pattern-Based Metal Fill Insertion
- Track-Based Metal Fill Insertion

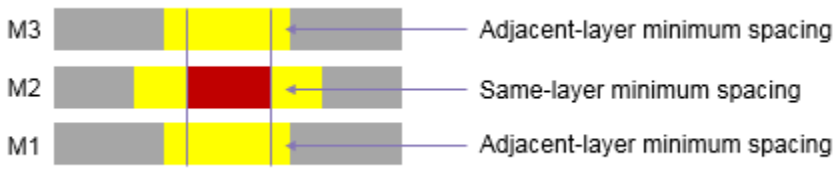
Specifying the Spacing Requirements for Timing-Driven Metal Fill Insertion

By default, the minimum spacing between the metal fill and the net shapes of the critical nets is based on the minimum spacing specified in the technology file.

- For metal fill on the same layer as the net shape, the default minimum spacing is two times the minimum spacing value specified for the layer in the technology file.
- For metal fill on layers adjacent to the net shape, the default minimum spacing is the minimum spacing value specified for the layer in the technology file.

For example, assume that the red rectangle in Figure 113 is a critical net shape. The yellow regions indicate where fill cannot be inserted and the gray regions indicate the inserted fill.

Figure 113 Minimum Spacing From Metal Fill to Critical Net Shape



Feedback

To override the default spacing requirements for timing-driven metal fill insertion, set the following application options:

- `signoff.create_metal_fill.space_to_nets`

  This application option defines the same-layer minimum spacing requirements between metal fill and a net shape of a timing-critical net.

- `signoff.create_metal_fill.space_to_clock_nets`

  This application option defines the same-layer minimum spacing requirements between metal fill and a net shape of a user-defined clock net that is specified in the `-nets` option.

- `signoff.create_metal_fill.space_to_nets_on_adjacent_layer`

  This application option defines the adjacent-layer minimum spacing requirements between metal fill and a net shape of a timing-critical net.

- `signoff.create_metal_fill.space_to_clock_nets_on_adjacent_layer`

  This application option defines the adjacent-layer minimum spacing requirements between metal fill and a net shape of a user-defined clock net that is specified in the `-nets` option.

For each of these application options, use the following syntax to specify the spacing to use for each layer:

`{ {`$layer_1$ $value_1$`} ... {`$layer_n$ $value_n$`} }`

You can specify the spacing value either as a multiple of the minimum spacing (*n*x) or a distance in microns.

For example, to set the minimum spacing between metal fill and a timing-critical net on the same layer to four times the minimum spacing for the M2 and M3 layers, use the following command:

```
icc2_shell> set_app_options \
   -name signoff.create_metal_fill.space_to_nets \
   -value {{M2 4x} {M3 4x}}
```

To set the minimum spacing between metal fill and a timing-critical net on the same layer to 0.125 microns for the M2 layer and 0.133 microns for the M3 layer, use the following command:

```
icc2_shell> set_app_options \
   -name signoff.create_metal_fill.space_to_nets \
   -value {{M2 0.125} {M3 0.133}}
```

Feedback

Defining the Density Design Rules

The IC Validator tool can check and fix the following density design rules:

- Minimum density, which specifies the minimum percentage of metal allowed in the density checking window

  This rule is checked when you enable density error fixing by setting the `signoff.create_metal_fill.fix_density_errors` application option to `true`.

  By default, the IC Validator tool uses the setting defined for the `minDensity` attribute in the `DensityRule` sections of the technology file. If this attribute is not defined for a layer, the IC Validator tool uses a default of 10 percent. To override the default, define the `mx_min_density` parameter in the IC Validator parameter file.

- Density gradient, which specifies the maximum percentage difference between the fill density of adjacent density checking windows

  This rule is checked if it is defined in the technology file and you enable density error fixing by setting the `signoff.create_metal_fill.fix_density_errors` application option to `true`.

  The IC Validator tool uses the setting defined for the `maxGradientDensity` attribute in the `DensityRule` sections of the technology file. If this attribute is not defined for a layer, the IC Validator tool does not check this rule.

- Maximum open area, which specifies the maximum size of a square area that contains no polygons and does not interact with any polygons

  This rule is checked if it is defined.

  The maximum open area rule is defined in an IC Validator parameter file. Use the following syntax to define the maximum open area rule for each metal layer:

  ```
  mn_max_open_area_rule = value;
  ```

  where *n* is the metal layer number and *value* is the side length of the square in microns.

For more information about the IC Validator parameter file, see Using an IC Validator Parameter File.

For more information about the technology file, see the *Synopsys Technology File and Routing Rules Reference Manual*.

Feedback

Incremental Metal Fill Insertion

If you have already used the `signoff_create_metal_fill` command to performed metal fill insertion on a block, you can use incremental metal fill insertion to perform the following tasks:

- Insert additional metal and via fill to specific layers or regions, as described in Adding to Existing Metal and Via Fill.
- Replacing existing metal and via fill in specific locations, as described in Replacing Existing Metal and Via Fill.
- Replacing existing metal and via fill in modified regions, as described in Performing Metal Fill Insertion Only in Modified Regions.

Adding to Existing Metal and Via Fill

To insert metal and via fill without first removing the existing metal fill, use the `-mode add` option with the `signoff_create_metal_fill` command. When you use this mode, you can control where the metal fill insertion occurs by using one or more of the following options:

- `-select_layers`, which restricts the layers on which to perform metal fill insertion, as described in Specifying the Layers for Metal Fill Insertion
- `-coordinates` or `-excluded_coordinates`, which restrict the regions on which to perform metal fill insertion, as described in Specifying the Regions for Metal Fill Insertion.

  When you use these options with the `-mode add` option, the tool does not fix density errors regardless of the setting of the `signoff.create_metal_fill.fix_density_errors` application option.

See Also

- Pattern-Based Metal Fill Insertion
- Track-Based Metal Fill Insertion

Replacing Existing Metal and Via Fill

To remove and insert metal and via fill only for the specified locations, use the `-mode replace` option with the `signoff_create_metal_fill` command. You would typically

Feedback

use this mode after ECO changes. When you use this mode, you must also specify one or more of the following options:

- `-select_layers`, which restricts the layers on which to perform metal fill insertion

  For example, to remove all metal fill on the M1 and M3 layers and refill those two layers without affecting the metal fill on other layers, use the following command:

  ```
  icc2_shell> signoff_create_metal_fill -mode replace \
     -select_layers {M1 M3}
  ```

  For more information about this option, see Specifying the Layers for Metal Fill Insertion.

- `-coordinates` or `-excluded_coordinates`, which restrict the regions on which to perform metal fill insertion

  For more information about these options, see Specifying the Regions for Metal Fill Insertion.

- `-nets` or `-timing_preserve_setup_slack_threshold`, which restricts the nets on which to perform metal fill insertion

  For more information about these options, see Timing-Driven Metal Fill Insertion.

See Also

- Pattern-Based Metal Fill Insertion
- Track-Based Metal Fill Insertion
- Timing-Driven Metal Fill Insertion

Performing Metal Fill Insertion Only in Modified Regions

If you have previously run the `signoff_create_metal_fill` command on a block, you can use the `-auto_eco true` option to restrict the metal fill insertion to those regions of the block that have been modified since the last run.

Note:

You can use this option only if the percentage of change to the block since the previous `signoff_create_metal_fill` run is less than the change threshold. The default change threshold is 20 percent; to modify the change threshold, set the `signoff.create_metal_fill.auto_eco_threshold_value` application option.

When you use the `-auto_eco true` option, the tool automatically determines the modified regions and layers, and then removes the existing metal fill and redoes metal fill insertion only in those locations; all other existing metal fill is retained.

Feedback

By default, the tool compares the current block to the version used for the previous `signoff_create_metal_fill` run. To compare the current block to a different block, use the `-pre_eco_design` option to specify the comparison block.

Note:

A change on a metal layer triggers metal fill removal and insertion on the adjacent via layers to ensure that no floating or hanging vias remain when the metal shapes are removed from the ECO area. If a cell instance changes, the tool takes a conservative approach and considers changes on all layers.

For example, to remove the metal fill from the regions modified since the last time you ran the `signoff_create_metal_fill` command and then fill only those regions using the pattern-based mode, use the following command:

```
icc2_shell> signoff_create_metal_fill -auto_eco true
```

You can use the `-select_layers` option to specify the layers on which to perform metal fill insertion; however, the tool performs metal fill insertion on a specified layer only if it is one of the automatically detected changed layers. You cannot use the `-coordinates` or `-excluded_coordinates` options to restrict the metal fill insertion regions.

When you use the `-auto_eco true` option,

- You cannot use the `-mode`, `-all_runset_layers`, and `-report_density` options
- The tool does not insert dense fill on the double-patterning layers regardless of the setting of the `-fill_all_tracks` option
- The tool does not fix density errors regardless of the setting of the `signoff.create_metal_fill.fix_density_errors` application option

See Also

- Pattern-Based Metal Fill Insertion
- Track-Based Metal Fill Insertion
- Timing-Driven Metal Fill Insertion

Foundry Fill and Track Fill Combined for Different Flows of Foundry

To support designs that require combined foundry fill and track fill in the Fusion Compiler tool, you can use the `-mode add` option with the `signoff_create_metal_fill` command.

```
icc2_shell> signoff_create_metal_fill -mode add
```

For a few advanced nodes, you can add foundry fill on top of track fill, which is the default methodology to create fill on some special layers.

Feedback

You can use the `all_runset_layers` option with the `signoff_create_metal_fill` command. Set the `all_runset_layers` option to `true` to insert fill for all output fill layers defined in the runset settings. By default, this option is set to `false` and inserts fill only for the routing layers (metal and via layers).

Signoff Metal Fill Result Files

The `signoff_create_metal_fill` command stores the output files in the run directory specified by the `signoff.create_metal_fill.run_dir` application option (or the signoff_fill_run directory if you do not use this option). For each fill run, the command creates a subdirectory named icv_run_# that contains the files generated for that run.

When you perform metal fill insertion, the tool writes the following files to the output directory:

- *block*.LAYOUT_ERRORS, which contains details about the detected errors
- *block*.RESULTS, which contains a summary of the run results
- signoff_create_metal_fill.log, which contains a summary of the run environment
- icv_config_out, which contains the paths to the top-level block and the cell libraries
- layer.map, which contains the generated layer mapping file
- metal_fill_params.rh, which contains the metal fill parameters and options that you specified
- metal_fill_compress_params.rh, which contains the storage commands and indicates whether the compression mode was flat (NONE) or hierarchical (AUTO)
- ./run_details directory, which contains all the data generated by the IC Validator tool for the metal fill insertion run

Querying Metal Fill

Metal fill shapes are stored in fill cells, which use the following naming convention: FILL_INST_#.

- To query the fill cells in the block, use the `get_fill_cells` command.

  By default, this command reports only the fill cells in the top-level design. To report all of the fill cells in the hierarchy, use the `-hierarchical` option.

- To query the fill shapes, use the `get_shapes -include_fill` command.

  To return only the fill shapes associated with specific fill cells, use the `-of_objects` option to specify the fill cells.

Feedback

Viewing Metal Fill in the GUI

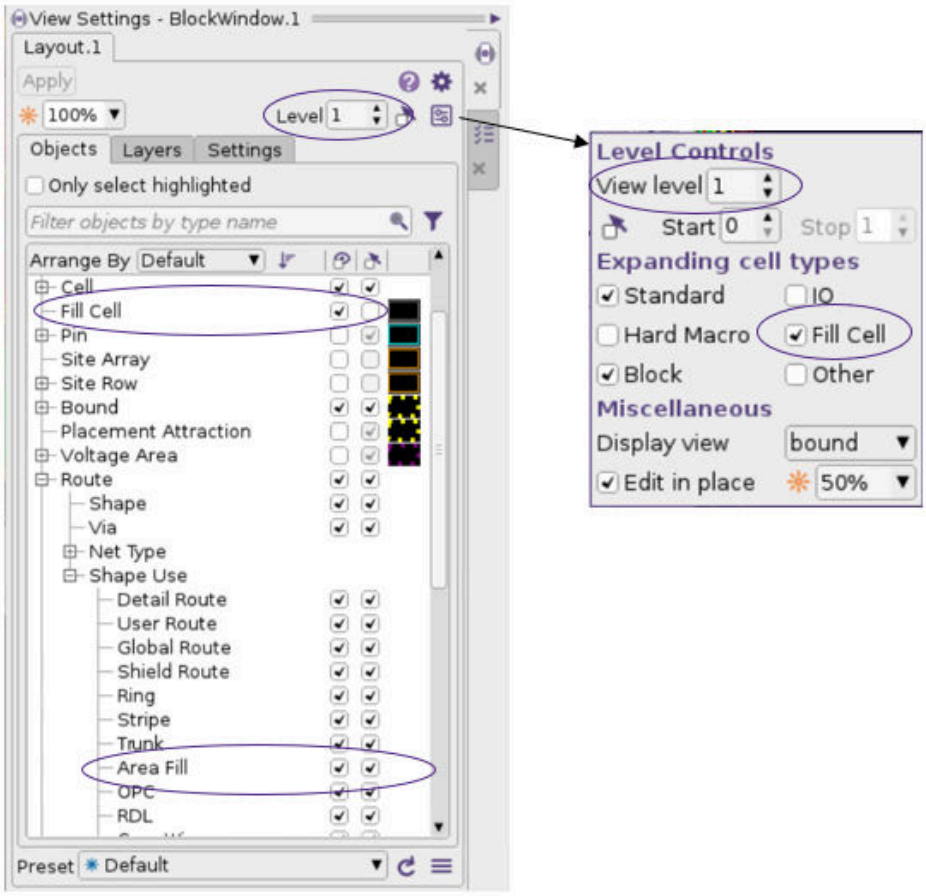
To view the metal fill shapes in the layout view of the GUI,

1. Display the View Settings panel by clicking the View Settings icon ( ) in the panel area.
   - Set the Level to 1.
   - Select the Objects tab and enable visibility for Fill Cell objects and Area Fill metal shapes (if you do not see Area Fill in the object list, expand the Route object and Shape Use attribute).
2. Display the Hierarchy Settings panel by clicking the Hierarchy Settings icon ( ) in the View Settings panel or the menu bar.
   - Set the “View level” to 1.
   - Select “Fill Cell” in the “Expanding cell types” section.

Figure 114 shows the View Settings and Hierarchy Settings panels with the required settings for displaying fill shapes.

Feedback

Figure 114 Settings Used to Display Fill Shapes in the GUI



Reporting the Metal Density

To report the metal density information, run the `signoff_report_metal_density` command.

Feedback

To calculate the metal density information for a metal layer, the tool requires the following information:

- The density rule for the layer

  The tool determines the density rule for each layer by using the following settings, in order of priority:

  1. The setting of the `mx_min_density` parameter in the IC Validator parameter file specified by the `signoff.report_metal_density.user_defined_options` application option
  2. The setting of the `signoff.report_metal_density.min_density` application option
  3. The setting of the `minDensity` attribute in the `DensityRule` section of the technology file
  4. 10 percent

- The grid sizes for the layer

  The tool determines the grid sizes for each layer by using the following settings, in order of priority:

  1. The setting of the `mx_min_density_window` or `mx_density_gradient_window` parameter in the IC Validator parameter file specified by the `signoff.report_metal_density.user_defined_options` application option
  2. The setting of the `signoff.report_metal_density.density_window` or `signoff.report_metal_density.gradient_window_size` application option
  3. The setting of the `windowSize` attribute in the `DensityRule` section of the technology file
  4. 50 microns

For information about defining the metal density and density gradient rules in the technology file, including the `windowSize` attribute, see the *Synopsys Technology File and Routing Rules Reference Manual*.

By default, the `signoff_report_metal_density` command

- Uses the fill data for the metal density calculation
  - To use the design view if its timestamp is newer than the fill data, set the `signoff.report_metal_density.fill_view_data` application option to `read_if_uptodate`.
  - To use the design view regardless of its timestamp, set the `signoff.report_metal_density.fill_view_data` application option to `discard`.

Feedback

- Creates the density and gradient windows starting from the lower-left corner of the chip boundary using a step size of half the window size
  - To change the starting point for creating the windows, use the `-starting_point` option with the `signoff_report_metal_density` command.
  - To change the step size, set the `signoff.report_metal_density.density_window_step` application option or the `mx_window_step_size` parameter in the IC Validator parameter file specified by the `signoff.report_metal_density.user_defined_options` application option. If you specify both, the IC Validator parameter overrides the application option.
- Computes the metal density for the entire block

  To compute the metal density for specific regions, use the `-coordinates` option with the `signoff_report_metal_density` command. When you use this option, the command combines the specified regions when reporting the metal density; it does not report the metal density individually for each specified region.

  To specify the coordinates, use the following format:

  $\{\{x_1\ y_1\}\ \{x_2\ y_2\}\ \ldots\ \{x_n\ y_n\}\}$

  Note that there must be a space between each set of coordinates.
- Computes the metal density for each metal layer from the minimum routing layer to the maximum routing layer

  To compute the metal density for specific layers, use the `-select_layers` option with the `signoff_report_metal_density` command.
- Does not generate heat maps

  To generate heat maps to enable viewing of the metal density information in the GUI, set the `signoff.report_metal_density.create_heat_maps` application option to `true`. For information about viewing the heat maps, see Viewing Density Heat Maps in the GUI.
- Writes the report files to a directory named signoff_report_metal_density_run under the current working directory

  To change the name of the IC Validator working directory, set the `signoff.report_metal_density.run_dir` application option.

  By default, the report files are named report_metal_density.txt and gradient_density_report.txt. To change the file names, use the `-output` option with the `signoff_report_metal_density` command. When you use this option, the density report has the specified file name, while the gradient density report adds a .gradient file extension. For example, if you specify `-output density.rpt`, the density report is named density.rpt and the gradient density report is named density.rpt.gradient.

Feedback

The `signoff_report_metal_density` command uses the options that you specify to generate the command line to invoke the IC Validator tool. You can specify additional options for the IC Validator command line by setting the `signoff.report_metal_density.user_defined_options` application option. The string that you specify in this option is added to the command line used to invoke metal density reporting in the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.

See Also

- Using an IC Validator Parameter File

Viewing Density Heat Maps in the GUI

After running the `signoff_report_metal_density` command, you can view heat maps in the GUI for both the minimum density and gradient density reports.

Note:

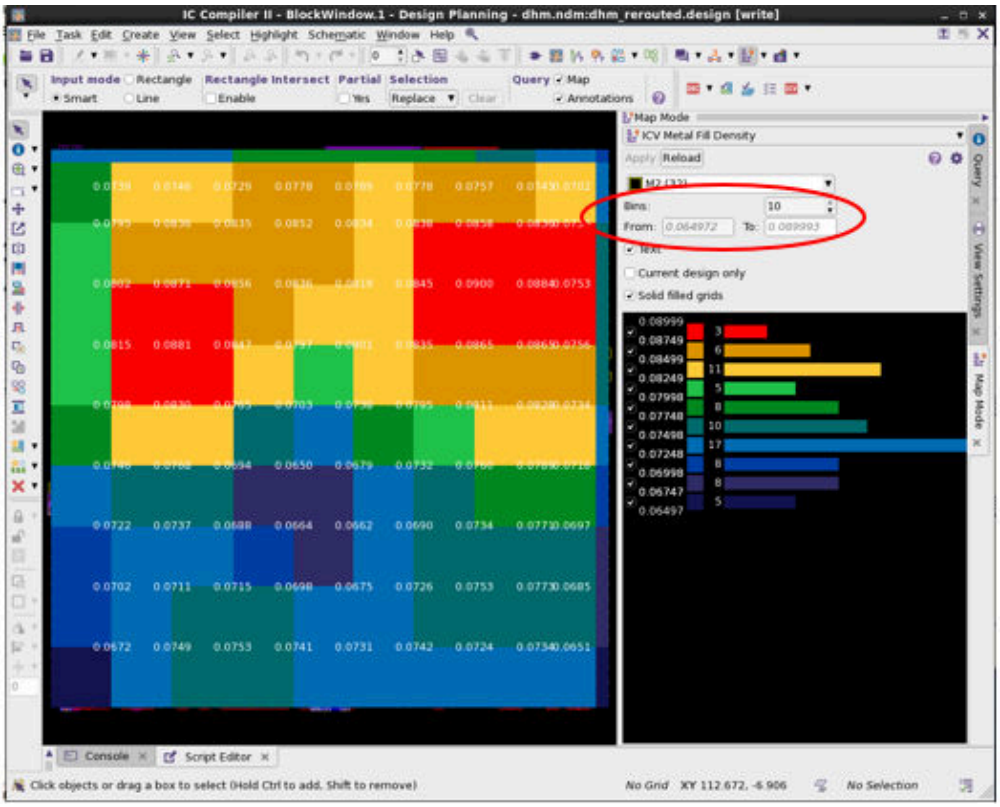
The `signoff_report_metal_density` command generates the heat maps only when the `signoff.report_metal_density.create_heat_maps` application option is set to `true`.

- To display the minimum density heat map, choose **View > Map > ICV Metal Fill Density**.

  Figure 115 shows a minimum density heat map. In this heat map, each colored tile corresponds in size to the density window step. To customize or standardize the density values in the heat map, set the bin range, which is indicated by a red oval in the figure.

Feedback

Figure 115 Minimum Density Heat Map

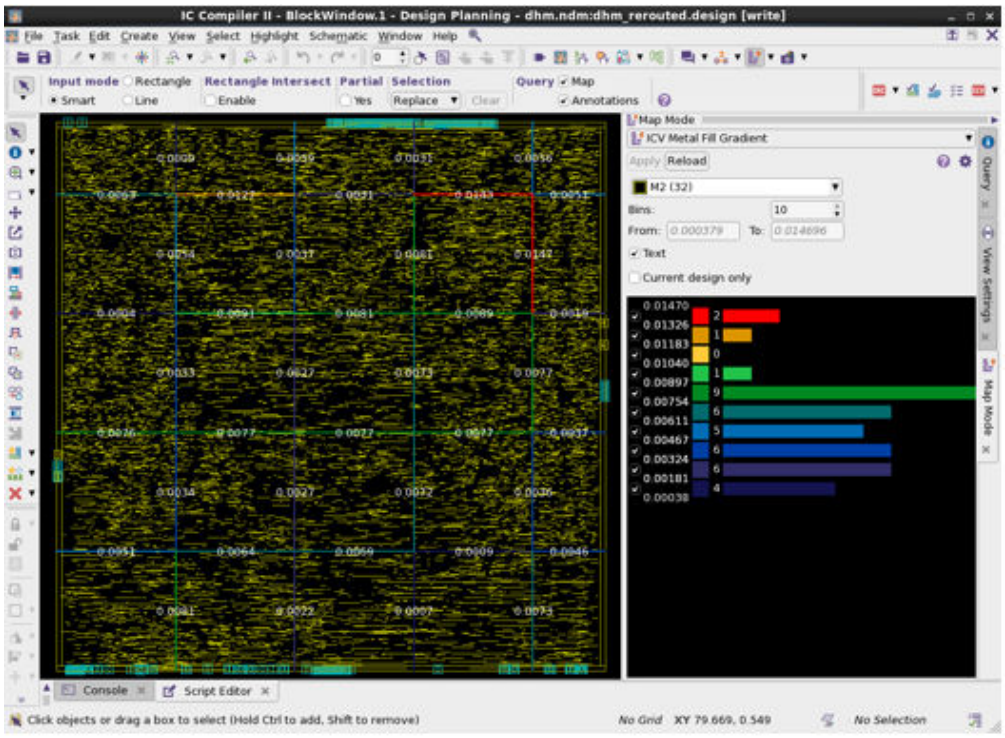


- To display the density gradient heat map, choose **View > Map > ICV Metal Fill Gradient**.

Feedback

Figure 116 shows a density gradient heat map. In this heat map, each colored bar corresponds in size to the gradient window size.

Figure 116 Density Gradient Heat Map



Removing Metal Fill

The method you use to remove metal fill depends on the extent of the metal fill removal:

- To remove all metal fill from a block, specific layers of a block, or specific regions of a block, use the `signoff_create_metal_fill` command with the `-mode remove` option.

  This command uses the IC Validator tool to remove metal fill from the current block, as described in Removing Metal Fill With the IC Validator Tool.

- To remove specific fill shapes, use the `remove_shapes` or `remove_fill_cells` commands.

  You would typically use this method when manually fixing DRC violations related to the metal fill. You might also be able to fix the DRC violation by modifying the metal fill shapes, as described in Modifying Metal Fill.

Feedback

Removing Metal Fill With the IC Validator Tool

By default, when you use the `signoff_create_metal_fill -mode remove` command, the IC Validator tool removes all metal and via fill from the current block, including the typical critical dimension (TCD) structures.

- To remove the metal fill only from specific regions, use the `-coordinates` option, as described in Specifying the Regions for Metal Fill Insertion.
- To remove the metal fill only from specific layers, use the `-select_layers` option, as described in Specifying the Layers for Metal Fill Insertion.

  By default, when you use this option, the IC Validator tool does not remove the TCD structures. To remove the TCD structures in addition to the metal and via fill, set the `signoff.create_metal_fill.tcd_fill` application option to `true` before running the `signoff_create_metal_fill` command.
- To remove the metal fill only over critical nets, use one or both of the `-nets` and `-timing_preserve_setup_slack_threshold` options to identify the critical nets.

  Note:

  The existing fill is not considered when determining the critical nets for pattern-based metal fill, but not for track-based metal fill.
- To honor certain rules when removing the metal fill, use the `-remove_by_rule` option. You can specify one or more of the following rules:
  - Nondefault routing rules (`ndr`)

    This rule applies to both pattern-based and track-based metal fill.
  - Maximum density rules (`max_density_threshold`)

    This rule applies only to track-based metal fill. When you enable this rule, metal fill removal honors the maximum density threshold set by the `signoff.create_metal_fill.max_density_threshold` application option.

  For example, to honor nondefault routing rules during fill removal, use the following command:

```
icc2_shell> signoff_create_metal_fill -mode remove \
   -remove_by_rule {ndr}
```

See Also

- Specifying the Layers for Metal Fill Insertion
- Specifying the Regions for Metal Fill Insertion

Feedback

ECO-Aware Removal of Track Based Metal Fill Shapes

To automatically remove track based metal fill shapes that are causing DRC violations after ECO, use the `-remove_by_rule drc_auto` option with the `signoff_create_metal_fill` command. This action removes DRC violating track based metal fill shapes or vias from design, including any floating connections created due to removal of DRC violating shape. For example,

```
icc2_shell> signoff_create_metal_fill -remove_by_rule drc_auto
```

Modifying Metal Fill

In some cases, you might find that you can manually fix a DRC violation by changing the boundary of metal fill shapes, adding metal fill shapes, or removing metal fill shapes.

- To change the boundary of a metal fill shape, use the `set_attribute` command to modify its `bbox` attribute.
- To add a metal fill shape in the top-level fill cell, use the `create_shape -shape_use area_fill` command.
- To add a metal fill shape in a specific fill cell, use the `create_shape -fill_cell` command.
- To remove a metal fill shape, use the `remove_shapes` command.

You can also modify metal fill shapes in the GUI. To select or modify fill shapes in the GUI,

1. Select “Multiple Levels Active” () in the View Settings panel.
2. In the “Hierarchy Settings” panel, set “View level” to a minimum of 2 and select “Fill Cell” in “Expanding cell types.”

Performing Real Metal Fill Extraction

To enable real metal fill extraction,

1. Associate non-emulation TLUPlus files with the timing corners by using the `set_parasitic_parameters` command, as described in the *IC Compiler II Timing Analysis User Guide*.
2. Enable real metal fill extraction by using the following command:

```
icc2_shell> set_extraction_options \
   -real_metalfill_extraction floating
```

For more information about performing extraction in the IC Compiler II tool, see the *IC Compiler II Timing Analysis User Guide*.

Feedback

Automatically Fixing Isolated Vias

An isolated via is a via that does not have neighboring vias close enough to meet the requirements of the technology.

To check for and fix isolated vias,

1. Set up the IC Validator environment as described in Setting Up the IC Validator Environment.
2. (Optional) Enable distributed processing by using the `set_host_options` command, as described in Enabling IC Validator Multicore Processing.
3. Set the application options for fixing isolated vias.

   At a minimum, you must define the maximum distance within which a neighboring via must exist so that a via is not considered an isolated via. To define this information for each via layer, set the `signoff.fix_isolated_via.isolated_via_max_range` application option, which has the following syntax:

   `{ {`*`via_layer`*$_1$ *`distance`*$_1$`} ... {`*`via_layer`*$_n$ distance$_n$`} }`

   where the *via_layer* argument uses the mask names, such as via1, and the *distance* argument is in microns.

   For information about the options available for fixing isolated vias, see Setting Options for Fixing Isolated Vias.
4. Run isolated via checking and fixing by using the `signoff_fix_isolated_via` command as described in Running the signoff_fix_isolated_via Command.

Setting Options for Fixing Isolated Vias

Before you run the `signoff_fix_isolated_via` command, configure the run by setting the application options shown in Table 46. To set the application options, use the `set_app_options` command. To see the current settings, use the `report_app_options` command.

Feedback

Table 46 Application Options for Signoff Isolated Via Fixing

Application option	Default	Description
`signoff.fix_isolated_via.isolated_via_max_range` (required)	N/A	Specifies the distance in microns within which a neighboring via must exist on each via layer. If a neighboring via is not found within this distance, a via is considered an isolated via and the command tries to add a fixing via within the specified distance.
`signoff.fix_isolated_via.avoid_net_types`	`{clock pg}`	Specifies the types of nets to avoid when fixing isolated vias. Specify a list that contains one or more of the following values: `clock`, `pg`, and `none`.
`signoff.fix_isolated_via.run_dir`	signoff_fix_isolated_via_run	Specifies the run directory, which contains the files generated by the `signoff_fix_isolated_via` command. You can specify either a relative path, in which case the directory is created under the current working directory, or an absolute path.
`signoff.fix_isolated_via.user_defined_options`	(none)	Specifies additional options for the IC Validator command line. The string that you specify in this option is added to the command line used to invoke the IC Validator tool. The IC Compiler II tool does not perform any checking on the specified string.

Running the signoff_fix_isolated_via Command

You can use the `signoff_fix_isolated_via` command either to check for isolated vias only, or to check for and fix the isolated vias.

Checking for Isolated Vias

To check for isolated vias without fixing them, run the `signoff_fix_isolated_via` command with the `-check_only true` option.

```
icc2_shell> signoff_fix_isolated_via -check_only true
```

Feedback

When run you run the `signoff_fix_isolated_via` command in check-only mode, it generates a summary report that specifies the number of isolated vias detected on each via layer.

Checking and Fixing Isolated Vias

By default, the `signoff_fix_isolated_via` command both checks for and fixes isolated vias.

The command performs the following tasks:

1. Checks for isolated vias by using the ranges defined in the `signoff.fix_isolated_via.isolated_via_max_range` application option
2. (Optional) Performs track-based metal fill insertion to insert dummy fill shapes within the specified range around each detected isolated via
   - By default, the `signoff_fix_isolated_via` command ignores the existing fill data and performs track-based metal fill insertion to insert the fill shapes to use for fixing the isolated vias.

     The fill shapes are inserted on-track following the design rules defined in the technology file. Some foundries have additional design rules, which you enable by setting the appropriate `TRACK_FILL_FOUNDARY_`*`name`* IC Validator variable. To set this variable, set the `signoff.create_metal_fill.user_defined_options` application option, as shown in the following example:

     ```
     icc2_shell> set_app_options \
        -name signoff.create_metal_fill.user_defined_options \
        -value {-D TRACK_FILL_FOUNDARY_name}
     ```

   - If you have already performed track-based metal fill insertion, you can use the existing fill shapes to fix isolated vias by using the `-update_track_fill true` option.

     When you use this option, you must use the `-track_fill_runset_include_file` option to specify the parameter file used for the initial track-based metal fill insertion. This parameter file is named track_fill_params.rh and is located in the run directory for the initial run. For example, if the run directory for the initial track-based metal fill insertion run is named init_fill, use the following command to fix isolated vias using the existing fill shapes:

     ```
     icc2_shell> signoff_fix_isolated_via -update_track_fill true \
        -track_fill_runset_include_file init_fill/track_fill_params.rh
     ```

For information about performing track-based metal fill insertion, see Track-Based Metal Fill Insertion.

Feedback

3. Inserts fixing vias within the specified range by using one of the following methods, in order of priority:

   a. Inserting a via between an existing non-wide net shape and a fill shape

   b. Extending the line end of an existing non-wide net shape and inserting a via between the extension and a fill shape

   c. Inserting a via between a wide metal shape and a fill shape

   When inserting the fixing vias, the tool considers the routing rules, including double-patterning rules and nondefault spacing rules, to prevent the introduction of DRC violations.

   Note:

   If the block is either very congested or very sparse, the `signoff_fix_isolated_via` command might not be able to fix all isolated vias.

4. Removes the unused dummy fill shapes

5. (Optional) Saves the updated block to disk

   - By default, the `signoff_fix_isolated_via` command does not save the updated block to disk. After running the command, you must use the `save_block` command to save the updated block to disk.
   - To save the block at the end of the `signoff_fix_isolated_via` run, use the `-save_design true` option with the `signoff_fix_isolated_via` command.

6. Saves the results to disk

   The `signoff_fix_isolated_via` command generates the following results files:

   - A summary report

     This report specifies the number of isolated vias for each via layer before and after fixing.

   - An error data file

     By default, the error data generated by the `signoff_fix_isolated_via` command is saved in a file named signoff_fix_isolated_via.err. To specify the name for the error data file, use the `-error_data` option. To report or display the information in the error data file, use the error browser, as described in the *IC Compiler II Graphical User Interface User Guide*.

8

Routing Using Custom Router

Custom Router is a shape-based router that supports gridded or gridless routing for custom digital, mixed signal, and analog routing needs. In the IC Compiler II environment, you can use Custom Router to create interconnects (or routes) for critical signals between blocks and continue with the IC Compiler II tool to complete the physical implementation. You can also preroute critical signals or clock nets by using a set of custom routing constraints.

In the IC Compiler II environment, Custom Router provides the following features:

- Prerouting in the batch mode
- Tcl-based routing constraint management
- Automatic routing with custom routing constraints
- Hybrid flow for prerouting
- Double Data Rate (DDR) net routing flow

Custom Router is fully integrated with the IC Compiler II tool, and supports advanced design rules for 20 nm and below technologies.

To learn about using Custom Router in the IC Compiler II environment, see the following topics:

- Using Custom Router in the IC Compiler II Tool
- Before Using Custom Router
- Defining Routing Constraints
- Managing Constraint Groups
- Using Custom Routing Application Options
- Routing With the Custom Router
- Shielding the Nets
- Checking the Routing Results

Feedback

- Using a Hybrid Routing Flow
- Using a DDR Routing Flow

These topics describe how to use Custom Router to create custom routes in the IC Compiler II environment. For detailed information about Custom Router, see the *Custom Compiler Custom Router User Guide*.

Using Custom Router in the IC Compiler II Tool

The IC Compiler II tool provides a set of Tcl commands to run Custom Router in the IC Compiler II environment. These commands define routing constraints for adding wires on one or more nets at the block level or adding differential pair routing, bus routing, shielding, and other routing features.

A set of application options is also available for specifying the parameters for performing custom routing. For more information about how to define routing constraints and application options, see Defining Routing Constraints and Using Custom Routing Application Options.

The route_custom command uses the specified routing constraints, as well as information from the technology file and the design, to perform automatic routing using the Custom Router. For more information about using the `route_custom` command, see Routing With the Custom Router.

Custom Router supports the following constraints:

- Net shielding and differential pairs
- Matched length
- Variable width and space
- Pin width matching and tapering

Custom Router supports the following IC Compiler II nondefault routing rules:

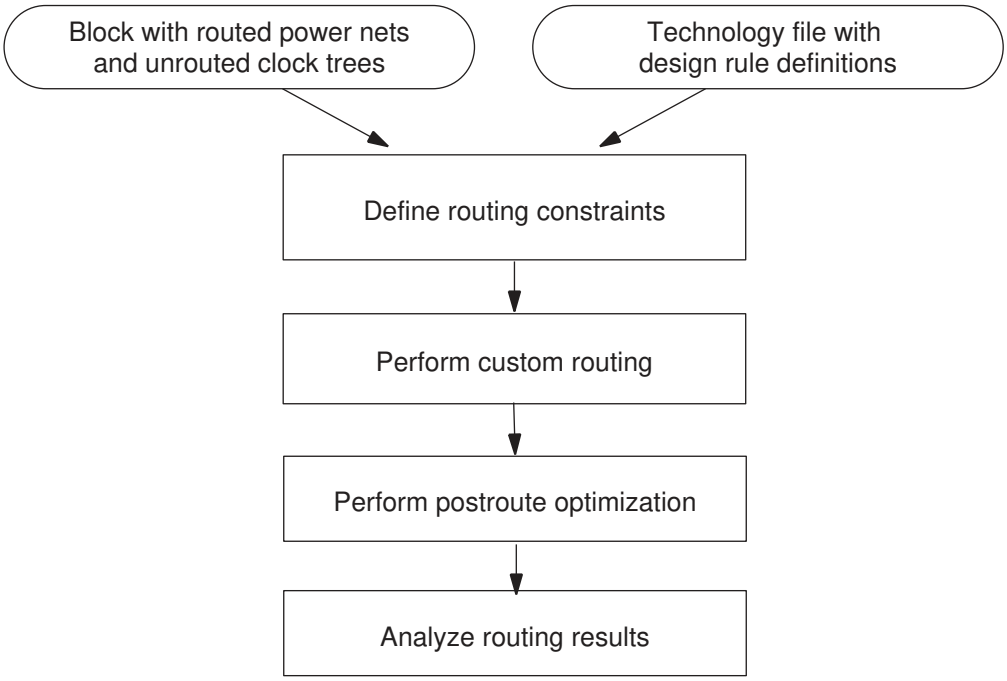
- Routing layers and vias
- Routing width and spacing
- Routing grids
- Inter- and intra-group spacing
- Taper halo
- Routing blockage

Feedback

- Routing corridors
- Shielding

Figure 117 shows the basic Custom Router flow.

Figure 117 Basic Custom Router Flow



Before Using Custom Router

Before you run Custom Router, you must

- Ensure the necessary design rules are correctly defined in the technology file, such as layer and via definitions.

  For more information, see the *Synopsys Technology File and Routing Rules Reference Manual*.

- Check the connectivity information and look for pin blockages in the design.

  For more information, see the "Reviewing the Design" topic in the *Custom Compiler Custom Router User Guide*.

- Ensure sure that the power and ground nets in the block have been routed.

Feedback

For more information, see the *IC Compiler II Design Planning User Guide*.

- Check the routability of the placement as explained in Checking Routability.

Defining Routing Constraints

The IC Compiler II tool provides a set of application options to control custom routing results. The application options are applied globally. See Using Custom Routing Application Options.

You can also define net-specific routing constraints that override the routing application options. Routing constraints provide guidance during routing. In the IC Compiler II environment, Custom Router honors both the Custom Router constraints and the IC Compiler II nondefault routing rules.

Note:

You must apply net-specific routing constraints to physical nets. When specifying nets that will be constrained, use the `get_nets -physical_context` command to ensure the physical net is returned. For example:

```
icc2_shell> create_net_shielding \
    -for [get_nets -physical_context pr*]\
    -disabled_layers {M5 M6} -sharing true \
    -layer_gaps {M2 3 M3 5}
 {shielding_4}
```

Table 47 lists the commands to define the Custom Router constraints by creating constraint groups. For information about the supported IC Compiler II nondefault routing rules, see Using Nondefault Routing Rules.

To check or remove the created constraint groups, use the `report_constraint_groups` or `remove_constraint_groups` command. For more information about checking or removing constraint groups, see Managing Constraint Groups.

Table 47 Commands to Define Custom Router Routing Constraints

Command	Description
`create_bus_routing_style`	Specifies the routing style for group of nets that are routed as a single bus structure (or trunk). For more information, see Defining the Bus Routing Style.
`create_differential_group`	Defines a differential group for routing. For more information, see Creating Differential Groups.

Feedback

Table 47 Commands to Define Custom Router Routing Constraints (Continued)

Command	Description
`create_wire_matching`	Defines matched wire length for a group of nets or pin-to-pin connections. For more information, see Defining Matching Wire Lengths.
`create_length_limit`	Defines the minimum wire length for a group of nets or pin-to-pin connections. For more information, see Defining Minimum Wire Lengths.
`create_net_shielding`	Defines the shielding style for a group of nets or bundles. For more information, see Inserting Shields on the Nets
`create_net_priority`	Determines the order in which the nets are routed. Nets with a higher priority are routed first. For more information, see Defining the Net Priority.

Defining the Bus Routing Style

To route a collection of nets or bundles as a single bus structure (trunk), create a constraint group that defines the bus routing style by using the `create_bus_routing_style` command.

You need to specify the following information:

- The name of the constraint group

  If you do not specify a name, the command names the constraint group bus_style_*n*, where *n* is a unique integer.

  By default, the command returns an error if the assigned name already exists. To delete the existing constraint group, use the `-force` option.

- The nets, bundles, and topology edges to which the constraint group applies

  To specify the objects that constitute the constraint group, use the `-for` option.

- The constraints

  To specify the constraints that are applied per layer to the trunk and override any settings on the individual bits, use the options listed in Table 48.

Feedback

Table 48 Bus Routing Style Constraints

Option	Description
`-valid_layers`	Specifies a list of the valid layers for trunk routing. When specified, the tool derives the minimum and maximum layer information from the list.
`-layer_spacings`	Specifies the minimum spacing for each routing layer. For example, `{M1 5.0 M2 6.0}`
`-layer_widths`	Specifies the minimum width for each routing layer. For example, `{M1 5.0 M2 6.0}`
`-shield_placement`	Specifies the trunk shielding placement. Valid values are: ◦ `default` (default): Uses the value from `custom.route.bus_intra_shield_placement`. ◦ `double_interleave`: Shields each wire individually. ◦ `half_interleave`: Inserts shared shielding every two wires. ◦ `interleave`: Inserts shared shielding between adjacent wires. ◦ `outside`: Places shields on the outside of the trunk or group. See also Intra-shield Placement.
`-corner_type`	Specifies how the tool routes the bus trunks at corners. Valid values are: ◦ `auto` (default): Automatically determines the optimal corner type for the bit or pin alignment to reduce routing congestion. ◦ `cross`: Routes the bit so that they overlap at the corner. ◦ `river`: Routes the bit so that they do not overlap at the corner. See also Bus Routing Options.
`-gap`	Specifies the spacing between the outermost bus bits and other shapes in the block. The default is 0.

The following example creates a bus routing constraint group for the nets that match the pattern pr*. The command names the constraint group bus_style_1, because no name is assigned to the constraint group. The shield placement is interleave. The minimum spacing is 3 microns for the M2 layer and 5 microns for the M3 layer.

```
icc2_shell> create_bus_routing_style \
    -for [get_nets -physical_context pr*] \
    -shield_placement interleave \
    -layer_spacings {M2 3 M3 5}
{bus_style_1}
```

Feedback

Creating Differential Groups

To create a differential group or a differential pair for the nets, bundles, or topology edges, use the `create_differential_group` command. Custom Router pairs the closest pins, one from each net, and routes from a common gather point between the pins. The tool routes the net pair the closest to each other with the most similar routing patterns.

You need to specify the following information:

- The name of the constraint group

  If you do not specify a name, the command names the constraint group differential_pair_*n* or differential_group_*n*, where *n* is a unique integer.

  By default, the command returns an error if the assigned name already exists. To delete the existing constraint group, use the `-force` option.

- The nets in the differential group

  To specify the nets included in the created group, use the `-for` option. If you specify two nets, the nets form a differential pair. If you specify three nets, the nets form a differential group.

- The constraints for the differential group

  To define the constraints that are applied per layer to the trunks and override any settings on the individual bits, use the options listed in Table 49.

Table 49 Differential Group Constraints

Option	Constraint
`-twist_style`	Specifies the twist style. Valid values are ◦ `diagonal`: Applies 45-degree twists to the wires. ◦ `none` (default): Does not twist the wires. ◦ `orthogonal`: Applies 90-degree twists to the wires.
`-twist_interval`	Specifies the distance between each twist.
`-twist_offset`	Specifies the distance between the first twist and the connected pins. The default is 0.
`-valid_layers`	Specifies the layers on which to route the differential group.
`-layer_spacings`	Specifies the minimum spacing for each routing layer.
`-layer_widths`	Specifies the minimum width for each routing layer.

Feedback

Option	Constraint
`-shield_placement`	Specifies how to shield the main bus trunk if one or more nets of the constraint group are shielded. Valid values are ◦ `double_interleave`: Shields each wire individually. ◦ `half_interleave`: Inserts shared shielding every two wires. ◦ `interleave`: Inserts shared shielding between adjacent wires. ◦ `outside (default)`: Places shields on the outside of the trunk or group. See also Intra-shield Placement.
`-gap`	Specifies the spacing in microns between the outermost bus bits and other shapes in the block. The default is 0.

The following example creates a differential pair for the prn and prp nets. The command names the differential pair differential_pair_0, because there is no name assigned to the different pair. No twist style is specified and the shield placement is set to `default`.

```
icc2_shell> create_differential_group \
    -for [get_nets -physical_context {prn prp}] \
    -shield_placement default
{differential_pair_0}
```

Inserting Shields on the Nets

To add shielding on the selected nets to reduce crosstalk or parasitic effects, run the `create_net_shielding` command.

You need to specify the following information:

- The name of the constraint group

  If you do not specify a name, the command names the constraint group shielding_*n*, where *n* is a unique integer.

  By default, the command returns an error if the assigned name already exists. To delete the existing constraint group, use the `-force` option.

- The nets, bundles, and topology edges to which the constraint group applies

  Use the `-for` option.

- The constraints for shielding styles and layer-specific requirements

  To define the net shielding constraints that are applied per layer to the trunk and override any settings on the individual bits, use the options listed in Table 50.

Feedback

Table 50 Net Shielding Constraints

Option	Description
`-shield_net`	Specifies the net you want to use as the shield. When routing a single signal net, the tool uses this net for the left and bottom shields.
`-shield_net_2`	Specifies the secondary net you want to use as the shield. When routing a single signal net, the tool uses this net for the right and top shields.
`-via_defs`	Specifies the vias that can be used to connect to the shield. By default, all via definitions can be used.
`-gap`	Specifies the global minimum spacing between route objects and shield objects. The default is 0.
`-max_gap`	Specifies the global maximum spacing between route objects and shield objects. The default is 0.
`-width`	Specifies the global shield width. The default is 0.
`-layer_gaps`	Specifies the layer-specific minimum spacing between route objects and shield objects. The default is 0.
`-layer_max_gaps`	Specifies the layer-specific maximum spacing between route objects and shield objects. The default is 0.
`-layer_widths`	Specifies the layer-specific shield width. The default is 0.
`-min_segment`	Specifies the minimum length of wiring to shield. The default is 0.
`-sharing`	Specifies whether the shielding can be shared among the nets in the constraint group. Valid values are `unset, true and false`.
`-group_shield`	Uses a single set of shield shapes to shield all nets in the constraint group. This option is mutually exclusive with the `-enclose_pins` option.
`-enclose_pins`	Specifies whether pins are enclosed by the shield. This option is mutually exclusive with the `-group_shield` option. Valid values are `unset, true, and false`.
`-enclose_vias`	Specifies whether vias are enclosed by the shield.Valid values are `unset, true, and false`.
`-disabled_layers`	Specifies the layers that cannot be used for shielding.

Feedback

The following example creates a constraint group and specifies constraints for adding shields on the nets whose names start with pr. The minimum spacing is 3 microns for the M2 layer and 5 microns for the M3 layer. All constituent objects can share the shield; the M5 and M6 layers cannot be used for shielding.

```
icc2_shell> create_net_shielding \
    -for [get_nets -physical_context pr*] \
    -disabled_layers {M5 M6} -sharing true -layer_gaps {M2 3 M3 5}
{shielding_4}
```

Defining the Net Priority

Use the net priority constraint to define the routing order for specific nets. Nets with a higher priority are routed first. To define the priority for a collection of nets or bundles, create a constraint group by using the `create_net_priority` command. You need to specify the following information:

- The name of the constraint group

  If you do not specify a name, the command assigns a name.

  By default, the command returns an error if the assigned name already exists. To delete the existing constraint group, use the `-force` option.

- The nets, bundles, and topology edges to which the constraint group applies

  Use the `-for` option.

- The net priority

  To specify the priority, use the `-priority` option. You can specify an integer between -128 and 128, inclusive.

The following example creates a constraint group named abc for the nets and bundles that match the pattern pr*. The net priority is set to 15.

```
icc2_shell> create_net_priority abc \
    -for [get_nets -physical_context pr*] \
    -priority 15
{abc}
```

Defining Minimum Wire Lengths

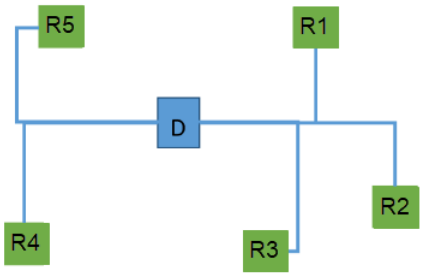
To define different minimum wire lengths for each object in a group of nets or pin-to-pin connections, use the `create_length_limit` command. Use this feature to perform length-limit routing on the nets with a single driver and multiple receivers, and run point-to-point routing between the drivers, steiners, and receivers. The Custom Router generates the steiner nodes in the background when the `create_length_limit` command is

Feedback

executed. Each of the receivers has either a matched length constraint or an independent length constraint from the driver.

Figure 118 shows an example of a single driver with multiple receivers, where each receiver has an individual length constraint.

Figure 118 Receivers With Individual Wire Length Constraints From Driver D



Syntax:

```
icc2_shell> create_length_limit
- for objects
[-min_valuefloat]
[-excludepins or ports]
[-driverpin or port]
[-force]
[intent_name]
```

Command or Options	Description
`create_length_limit`	Creates a length-limit constraint.
`-for`	Name of the driver pin or port, or net group that must comply with the constraint.
`-min_value`	(Optional) Minimum wire length. Default: 0
`-exclude`	(Optional) Excludes the specified pins or ports from the length matching constraint.
`-driver`	(Optional) Name of the driver pin or port.
`-force`	(Optional) Overwrites the existing constraint. Use the option to delete existing constraint groups.
`intent_name`	(Optional) Name of the constraint. If you do not specify a name, the command assigns a name. By default, the command returns an error if the assigned name already exists. To delete an existing constraint, use the `-force` option.

Feedback

In Example 29, the commands create five wire-length constraints, one for each of the receivers shown in Figure 118.

Example 29 Five Wire-Length Constraints for the Five Receivers Connected to Driver D

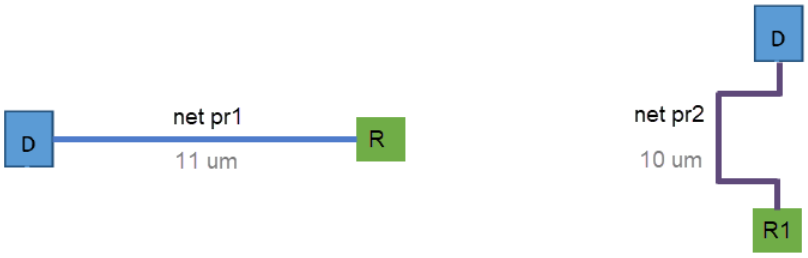
```
icc2_shell> create_length_limit -for [get_nets -physical_context $net] \
    -driver [get_pins -physical_context D/o1]
icc2_shell> create_length_limit -for [get_pins -physical_context R1/a] \
    -min_value 38.0
icc2_shell> create_length_limit -for [get_pins -physical_context R2/a] \
    -min_value 40.0
icc2_shell> create_length_limit -for [get_pins -physical_context R3/a] \
    -min_value 45.0
icc2_shell> create_length_limit -for [get_pins -physical_context R4/a] \
    -min_value 42.0
icc2_shell> create_length_limit -for [get_pins -physical_context R5/a] \
    -min_value 41.0
```

In Example 30, the command creates a constraint group named abc for the nets and bundles that match the pattern pr*. The minimum wire length is set to 10, so the length for each of the wires in the group is at least 10 microns. See Figure 119.

Example 30 Minimum Wire Length Constraint for pr Nets*

```
icc2_shell> create_length_limit abc -for [get_nets -physical_context pr*]
 \
    -min_value 10
{abc}
```

Figure 119 Minimum Wire Length Constraint for pr Nets*



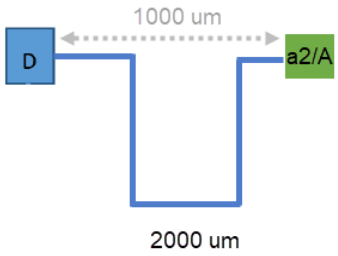
In Example 31, the command creates a constraint group named gcr_length_limit_a2 for the a2/A pin. The minimum wire length is set to 2000. See Figure 120.

Example 31 Minimum Wire Length Constraint for the a2/A Pin

```
icc2_shell> create_length_limit -for [get_pins -physical_context a2/A] \
   -min_value 2000.0 -force gcr_length_limit_a2
```

Feedback

Figure 120 Minimum Wire Length Constraint for the a2/A Pin



Defining Matching Wire Lengths

To define a group of nets or pin-to-pin connections that must have the same wire length, use the `create_wire_matching` command. By matching wire lengths, you can minimize clock skews and timing violations.

Two types of length-matching methods are available:

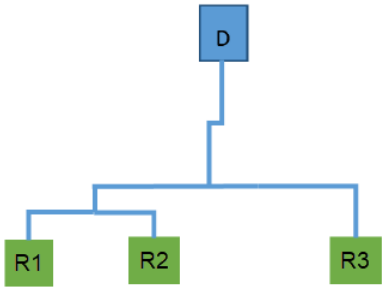
- Pin-based: Wire lengths between pins must match.

  Figure 121 and Figure 122 show examples of pin-based wire length matching. In both examples, all the receivers have a matched length constraint from driver D.

- Net-based: Total length of wires of specified nets must match.

  Figure 123 shows an example of net-based wire length matching. In the example, the total wire length of nets pr1 and pr2 are within the specified 10-um tolerance.

Figure 121 Receivers With Matched Wire Length From Driver D



Feedback

Figure 122 Receivers With Matched Wire Length From Driver D'

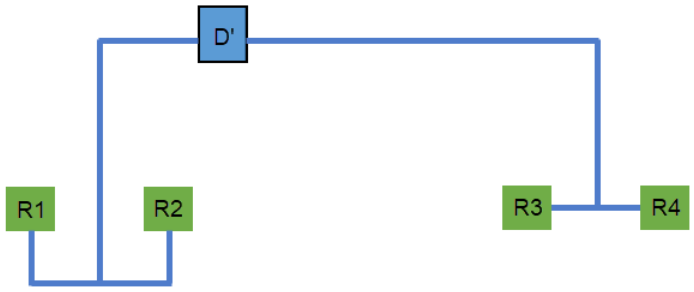
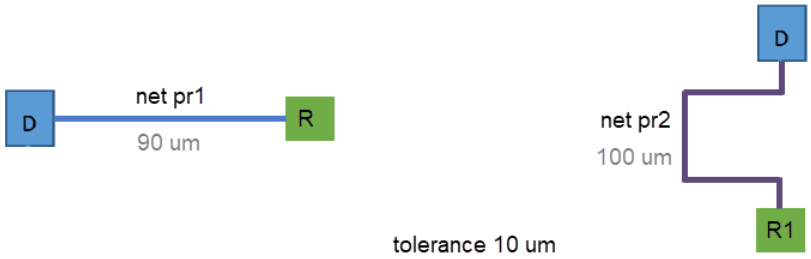


Figure 123 Net-Based Wire Length Matching



Syntax:

```
icc2_shell> create_wire_matching
   -for objects
   -tolerance float
   [-match_type type]
   [-exclude objects]
   [-relative]
   [-driver pin or port]
   [-force]
   [intent_name]
```

Command or Options	Description
`create_wire_matching`	Creates a length-matching constraint.
`-for`	Name of the driver pin or port (Figure 121 and Figure 122), or net group (Figure 123) that must comply with the constraint.
`-tolerance`	Length tolerance. The float value must be >= 0. To specify the tolerance as a percentage, use the `-relative` option.
`-relative`	(Optional) Interprets a `-tolerance` value <=1 as a percentage. The option must always be used with `the -tolerance` option.

Feedback

Command or Options	Description
`-match_type`	(Optional) Type of length matching. Valid values are: • `length`: (Default) Compares the length of each object in the constraint group to make sure they have the same total length. • `length_per_layer`: Compares the per-layer length.
`-exclude`	(Optional) Excludes the specified pins or ports from the length matching constraint.
`-driver`	(Optional) Name of the driver pin or port.
`-force`	(Optional) Overwrites the existing constraint. Use the option to delete existing constraint groups.
`intent_name`	(Optional) Name of the constraint. If you do not specify a name, the command assigns a name. By default, the command returns an error if the assigned name already exists. To delete an existing constraint, use the `-force` option.

After routing, a matched-length summary report is generated. You can use the report to troubleshoot and resolve the lengths that do not meet the set tolerance.

The content of the report depends on the `-for` option that you specified:

- If the `-for` option specifies pin-based length matching, a Length Constraint Report is generated. See Figure 124.
- If the `-for` option specifies net-based length matching, a Routed Length report is generated. See Figure 125.

Figure 124 Pin-Based Length Matching Report

```
*****************************************************************************
Length Constraint Report
Net : net1
Driver : net1-net1
  Target     Required Length      Actual Length       Difference
  ------     ---------------      -------------       ----------
     i0a          20.204                20.009            -0.195(-0.965%)
     i1a          20.204                19.985            -0.219(-1.084%)
     i2a          20.204                19.052            -1.152(-5.702%)
     i3a          20.204                19.052            -1.152(-5.702%)

Total expected length : 80.816002 Total Routed length : 78.098007 (= 96.64%)
*****************************************************************************
```

Feedback

Figure 125 Net-Based Length Matching Report

```
************************************************************
 Routed Length(Min:54.760 Max:55.144 Target Tolerance:1.00%)
  ----------------------------------
        Net Name            Length
           net2p            55.144
           net1p            54.760
  ----------------------------------
       Target Net             net2p
    Matched Range   [54.760,55.144]
Match Percentage            100.00%
************************************************************
```

In Example 32, the commands create wire matching constraints for the scenario shown in Figure 121. All receivers (R) have a matched length constraint from the balanced driver (D).

Example 32 Wire Matching Constraints for All Receivers to Driver D

```
icc2_shell> create_wire_matching -for [get_nets -physical_context $net] \
    -relative -tolerance 0.01 -driver [get_pins -physical_context D/o1]

icc2_shell> create_wire_matching -for [get_pins \
    -physical_context {R1/a R2/a R3/a}] -tolerance 0.01 -relative
```

In Example 33, the commands create wire matching constraints for the example in Figure 122.

Example 33 Wire Matching Constraints for Receivers Connected to Driver D'

```
icc2_shell> create_wire_matching -for [get_nets -physical_context $net] \
    -tolerance 0.01 -relative -driver $driver_full_name

icc2_shell> create_wire_matching \
    -for [get_pins -physical_context -of_objects $net] \
    -exclude $driver_full_name -tolerance 0.01 -relative
```

In Example 34, the command creates a constraint group named abc for the nets and bundles that match the pattern pr*. An absolute tolerance of 10 um is set to match the length of each object in the constraint group. See Figure 123.

Example 34 Wire Matching Constraints for pr Nets*

```
icc2_shell> create_wire_matching abc \
    -for [get_nets -physical_context pr*] \
    -tolerance 10
{abc}
```

In Example 35, the command creates a constraint group named def for the nets and bundles that match the pattern pr*. A tolerance of 10 percent is set to match the length of each object in the constraint group.

Feedback

Example 35 Wire Matching Constraints for pr Nets With Percent Tolerance*

```
icc2_shell> create_wire_matching def \
    -for [get_nets -physical_context pr*] \
    -tolerance 0.10 -relative
{def}
```

In Example 36, the command creates a wire matching constraint named clk_match for the clk net. Each of the endpoints that connect to the clk net has an individual matching constraint.

Example 36 Wire Matching Constraints for the clk Net

```
icc2_shell> create_wire_matching -for [get_pins -physical_context \
    -of_objects [get_nets -physical_context {clk}]] \
    -tolerance 0.01 -match_type length_per_layer -relative \
    -force clk_match
```

In Example 37, the command creates a wire matching constraint named clk_match2 for all the nets in the block, except the pins whose names match the keyword u1/a.

Example 37 Wire Matching Constraints With Exclusions

```
icc2_shell> create_wire_matching -for $nets -tolerance 0.05 \
   -exclude [get_pins -physical_context u1/a] -force clk_match2
```

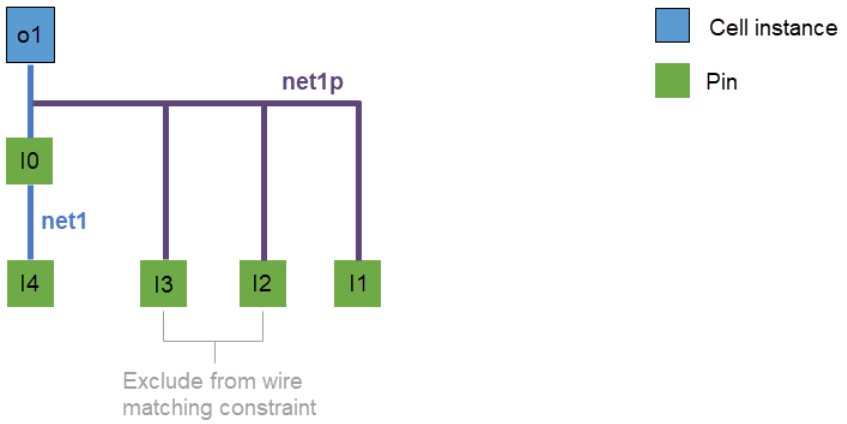
In Example 38, the script creates a net-based wire matching constraint for the scenario shown in Figure 126. The cell instance has an o1 terminal that servers as an inverter pin, driving signals on net1 and net1p to the pins. The I2 and I3 pins are excluded from the length matching constraint.

Example 38 Net-Based Wire Matching Constraints With Pin Exclusions

```
# set nets to match length
set nets [get_nets -physical_context {net1 net1p}]
# set allowed layers
set_routing_rule -min_routing_layer m7 -max_routing_layer m8 \
    -min_layer_mode allow_pin_connection -max_layer_mode hard $nets
# setup Custom Router wire match constraint
set exclude_pin_names [get_pins -physical_context {i2/a i3/a}]
create_wire_matching -for $nets -tolerance 0.01 -relative \
    -exclude $exclude_pin_names
# run Custom Router
route_custom -nets $nets
```

Feedback

Figure 126 Net-Based Wire Matching Constraints With Pin Exclusions



Managing Constraint Groups

You can report or remove the constraint groups that you create when specifying the routing constraints.

To report the created constraint groups, use the `report_constraint_groups` command and specify the constraint types to report by using the `-type` option. By default, all constraint types are reported. To specify the number of objects to report, use the `-count` option.

The supported constraint types are:

- `bus_style`, which is specified by the `create_bus_routing_style` command
- `differential_group`, which is specified by the `create_differential_group` command
- `differential_pair`, which is specified by the `create_differential_group` command
- `matched_wire`, which is specified by the `create_wire_matching` command
- `net_priority`, which is specified by the `create_net_priority` command
- `shielding`, which is specified by the `create_net_shielding` command
- `wire_length_limit`, which is specified by the `create_length_limit` command

Feedback

The following example shows the default output of the `report_constraint_groups` command:

```
icc2_shell> report_constraint_groups
****************************************
Report : report_constraint_groups
Design : top_new
Version:
Date   :
****************************************

--------------------------------------------------------------------------------
Name                           Type                           Objects
--------------------------------------------------------------------------------
NDR_bus                        bus_style                      bus1
NDR_diff                       differential_pair              clk0 clk1
shield_diff                    shielding                      clk0 clk1
matchL_con                     matched_wire                   match1
```

To remove the created constraint groups, use the `remove_constraint_groups` command and specify the constraint groups to remove. To remove all the constraint groups, use the `-all` option. The command disassociates the constituent objects from the removed constraint groups, and reports the number of the groups that are removed.

For example,

```
icc2_shell> report_constraint_groups
{shield_1} {shield_2}
icc2_shell> remove_constraint_groups $a
```

Using Custom Routing Application Options

The IC Compiler II tool provides a set of application options to control custom routing results. The application options are applied globally. To define net-specific routing constraints to override the routing applications options, see Defining Routing Constraints.

To list all application options that are available for custom routing, use the following command:

```
icc2_shell> report_app_options custom.route.*
```

To use the hybrid flow for custom routing, set the `custom.route.hybrid_flow` application option to `on`. This sets the values for each of hybrid flow related application options automatically.

Table 51 lists the application options for custom routing.

Feedback

Table 51 Application Options for Running Custom Routing

Application option	Description

`custom.route.bus_corner_type`
`custom.route.bus_intra_shield_placement`
`custom.route.bus_pin_trunk_offset`
`custom.route.bus_split_even_bits`
`custom.route.bus_split_ignore_width`
`custom.route.bus_tap_off_enable`
`custom.route.bus_tap_off_shielding`
	Specifies constraints for bus routing. See Bus Routing Options.

`custom.route.match_box`	Specifies the area within which the Custom Router performs length matching.
`custom.route.layer_grid_mode`	Specifies whether the route and corresponding parallel shields, jumpers for parallel shields, and bus shields snap to the wire tracks or the routing grid. See Track Adherence Options.

`custom.route.diffpair_twist_jumper_enable`
`custom.route.diffpair_twist_jumper_interval`
`custom.route.diffpair_twist_jumper_offset`
`custom.route.diffpair_twist_jumper_style`
	Specifies constraints for differential-pair routing. See Differential-Pair Options.

`custom.route.net_min_layer_mode`
`custom.route.net_min_layer_mode_soft_cost`
`custom.route.net_max_layer_mode`
`custom.route.net_max_layer_mode_soft_cost`
	Specifies the minimum and maximum layer mode and the layer cost. See Specifying Net-Specific Layer Constraints.

`custom.route.routing_area`	Defines an area within which the Custom Router creates routes. Use this option if your block is large and you only need to create or modify routes within a specific area.

`custom.route.shield_connect_mesh_overlap`
`custom.route.shield_connect_route`
`custom.route.shield_connect_to_supply`
`custom.route.shield_min_open_loop`
`custom.route.shield_min_shield_seg`
`custom.route.shield_min_signal_seg`
`custom.route.shield_net`
`custom.route.shield_second_net`
	Specifies shielding constraints. See Shielding Options.

Feedback

Table 51 Application Options for Running Custom Routing (Continued)

Application option	Description

`custom.route.single_loop_match`
`custom.route.single_loop_match_max_spacing`
`custom.route.single_loop_match_min_spacing`
`custom.route.single_loop_match_offset_layer`
	Specifies how to extend the wire. See Single-Loop Matching and Using a DDR Routing Flow.

`custom.route.skip_connect_pin_type`	Enables the hybrid flow.
`custom.route.distance_to_net_pin`	(Hybrid flow only) When using the `skip_connect_pin_type` option, skips the connection if the distance between the pin and the route is <= `distance_to_net_pin`. Specify the `distance_to_net_pin` option as a list of pairs. The syntax for each pair of values is as follows: {{*`netName distance`*}}
`custom.route.balance_mode`	Reduces the skews caused by wire length and via cut count differences in interposer designs. See Balance Mode Option.

Bus Routing Options

The Custom Router bus routing application options allow you to specify:

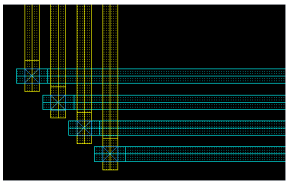
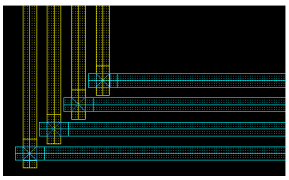
- Corner Type
- Intra-shield Placement
- Pin-Trunk Offset
- Trunk Splitting
- Tapoffs

Corner Type

The `custom.route.bus_corner_type` application option specifies how the tool routes the bus trunks at corners. Valid values are:

Valid Value	Description
`auto`	(Default) Automatically determines the optimal corner type for the bit or pin alignment to reduce routing congestion.

Feedback

Valid Value	Description
`cross`	Routes the bits so that they overlap at the corner. 
`river`	Routes the bits so that they do not overlap at the corner. 

Intra-shield Placement

The `custom.route.bus_intra_shield_placement` application option specifies how the bus trunk will be shielded. Valid values are:

Valid Value	Description
`double_interleave`	Shields each wire individually. 
`half_interleave`	Inserts shared shielding every two wires. 
`interleave`	Inserts shared shielding between adjacent wires. 
`outside`	(Default) Places shields on the outside of the trunk or group. 

Feedback

Pin-Trunk Offset

The `custom.route.bus_pin_trunk_offset` application option changes the default spacing between the bus trunk and the bus bit pins.

Trunk Splitting

The following Custom Router application options determine whether bus trunks will be split when routing through obstacles and how the bits will be split.

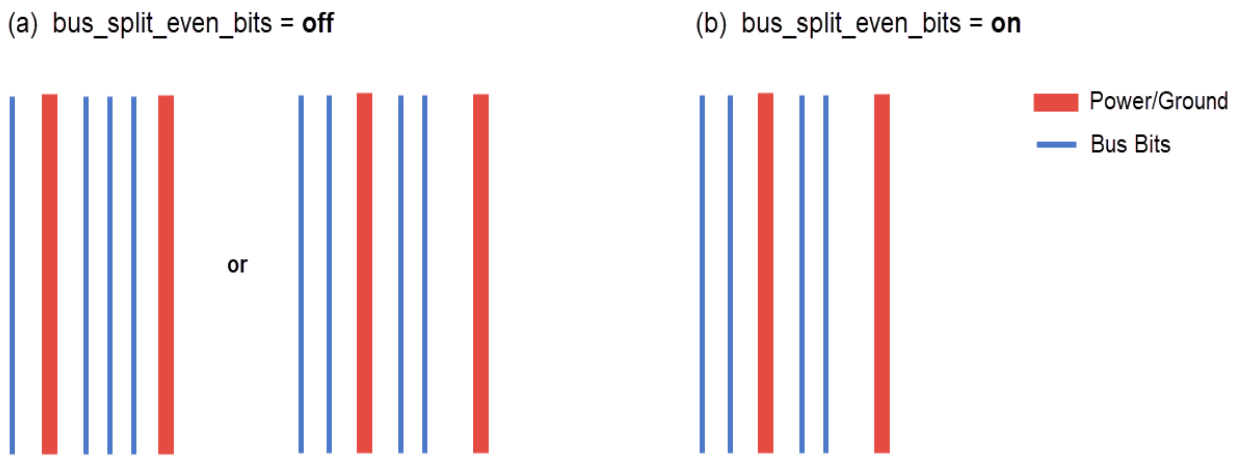
- `custom.route.bus_split_ignore_width`
- `custom.route.bus_split_even_bits`

The `custom.route.bus_split_ignore_width` application option specifies the wire width threshold of any obstructing power grid or route that triggers the bus trunk to be split into more than one section.

The `custom.route.bus_split_even_bits` application option requires that you set the `custom.route.bus_split_ignore_width` application option. If the `custom.route.bus_split_ignore_width` requirement is met, you can use the `custom.route.bus_split_even_bits` application option to control whether automatic bus splitting routes an even number of nets between pre-routes (power stripes).

In the following example, the bus trunk consists of four bits and the `custom.route.bus_split_ignore_width` application option is set. The spacing between the power and ground rails can accommodate three bits only. Figure 127a shows the two possible results when the `custom.route.bus_split_even_bits` application option is turned off. Figure 127b shows the result when the `custom.route.bus_split_even_bits` application option is turned on.

Figure 127 Splitting Bus Trunk to Route Through Power and Ground Rails



Feedback

Tapoffs

Two Custom Router application options allow you to control tapoff operations:

- `custom.route.bus_tap_off_enable`
- `custom.route.bus_tap_off_shielding`

The `custom.route.bus_tap_off_enable` application option connects pins to the bus trunk.

If the `custom.route.bus_tap_off_enable` application option is turned on, you can use the `custom.route.bus_tap_off_shielding` application option to add shielding to the bus tapoffs.

Track Adherence Options

To specify whether the route and corresponding parallel shields, jumpers for parallel shields, and bus shields will snap to the wire tracks or the routing grid, use the `custom.route.layer_grid_mode` application option. Specify the application option with a list of paired values. The syntax for each pair of values is as follows:

```
{{layerName mode}}
```

where

`layerName` is the name of the layer. To include all layers, specify `All`.

`mode` is one of the following:

- `on`: Snaps routes and shields to tracks. If the layer does not have tracks, the routes and shields will snap to the routing grid.
- `off`: Does not snap routes and shields to tracks or grids. Choose this option for gridless routing.
- `off_cost <costValue>`: Snaps routes and shields to tracks when possible. The value you specify determines the adherence of routes to tracks. The higher the value, the more likely the route will remain on the track or grid.

If tracks and routing grids are not defined and you specify `on`, the Custom Router uses an internally determined grid. The grid pitch is route width + minSpacing between shapes. The grid offset is 0 or pitch/2, whichever permits more pins on the grid.

Example 39 Layer and Cost Specification for Track Adherence

```
{{M2 off} {M3 off_cost 4}}
```

Feedback

Differential-Pair Options

The following Custom Router application options allow you to control differential-pair routing:

Application Option	Description
`custom.route.diffpair_twist_jumper_enable`	Twists the nets in the group.
`custom.route.diffpair_twist_jumper_interval`	Specifies the distance between each twist.
`custom.route.diffpair_twist_jumper_offset`	Specifies the distance between the first twist and the connected pins. The default is 0.
`custom.route.diffpair_twist_jumper_style`	Specifies the twist style. Valid values are: • `diagonal`: Applies 45-degree twists to the wires. • `none` (default): Does not twist the wires. • `orthogonal`: Applies 90-degree twists to the wires.

Shielding Options

The following Custom Router shielding application options allow you to control shielding:

Application Option	Description
`custom.route.shield_connect_mesh_overlap`	Creates multiple shield connections to the power mesh for a shield. Creates a shield via at each intersection of a shield path segment and a power strap. If a shield does not intersect with any power strap, the default four-pass connecting process is followed.
`custom.route.shield_connect_route`	Connects shields to ground. This option creates long ties that might block future routing. Set this option only if you must connect shields to ground. Note that these shields are marked as dynamic shields.
`custom.route.shield_connect_to_supply`	Connects shields to the nearest power mesh.

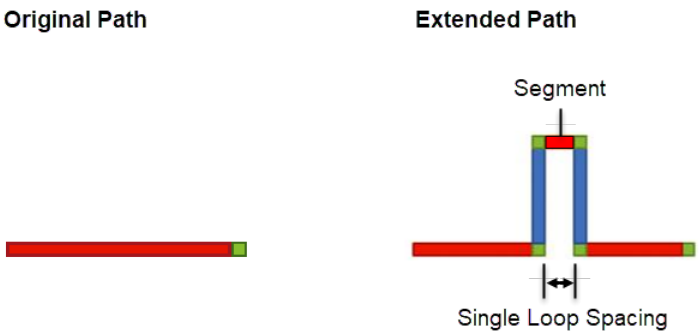
Feedback

Application Option	Description
`custom.route.shield_min_open_loop`	Specifies the minimum total length of an open shield-loop placed on the same layer. The shield shapes must be longer than shield_min_open_loop, or no shielding will be added.
`custom.route.shield_min_shield_seg`	Specifies the minimum length of shield segments. The default value is 0 um.
`custom.route.shield_min_signal_seg`	Specifies the minimum length of wiring to shield. The default value is 0 um.
`custom.route.shield_net`	Specifies the net you want to use as the shield. When routing a single-signal net, the tool uses this net for the left and bottom shields.
`custom.route.shield_second_net`	Specifies the secondary net you want to use as the shield. When routing a single-signal net, the tool uses this net for the right and top shields.

Single-Loop Matching

To create single-loops to extend wires for length matching, use the Custom Router single-loop matching application options. Single-loop matching is typically used in DDR routing flows.

Figure 128 Adding Single Loops to Extend Wires



Application Option	Description
`custom.route.single_loop_match`	Adds single loops to extend wire lengths.

Feedback

Application Option	Description
custom.route.single_loop_match_max_spacing	Specifies the maximum spacing in the loop.
custom.route.single_loop_match_min_spacing	Specifies the minimum spacing in the loop.
custom.route.single_loop_match_offset_layer	Routes the segment at the top of the figure on a layer different from other segments in the loop.

Balance Mode Option

You use the custom.route.balance_mode application option in interposer designs to reduce the skews caused by wire length and via cut count differences. This topic describes how to set the custom.route.balance_mode application option and provides examples to demonstrate its use.

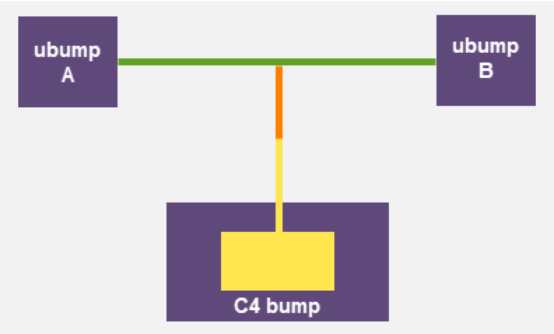
To route using the balance mode:

► Set the custom.route.balance_mode application option to true.

```
set_app_options -name custom.route.balance_mode -value true
```

When custom.route.balance_mode is true:

- The Custom Router creates feedthrough routes between two µbumps and a C4 bump on the same net.
- The wire lengths between the µbumps and the C4 bump are equal.
- The number of via cuts is the same in all related µbump-C4 bump connections.



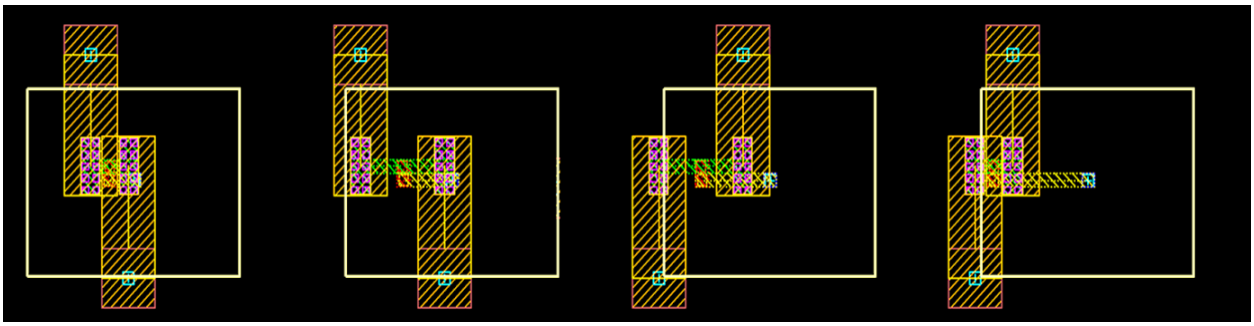
Example 1 shows how the custom.route.balance_mode option is used when routing µbumps to a C4 bump. The designated routing layers are M1 through M5. The M2 and M4 layers are used for horizontal connections; and the M1, M3, and M5 layers are used for vertical connections. The number of via cuts and the array dimensions are also defined.

Feedback

Example 40 Routing with custom.route.balance_mode turned on

```
create_routing_rule skew -widths {M1 6 M2 6 M3 6 M4 6 M5 25} -vias
 {{VIA12_1cut 3x3 R} {VIA23_1cut 3x3 R} {VIA34_1cut 3x3 R} {VIA45_1cut
 2x5 R}}
set_routing_rule $nets_to_route -rule skew -max_routing_layer M5
 -min_routing_layer M1
set_attribute [get_layers "M2 M4"] routing_direction horizontal
set_attribute [get_layers "M1 M3 M5"] routing_direction vertical
set_ignored_layers -min_routing_layer M1 -max_routing_layer M5
set_app_options -list {custom.route.balance_mode true}
route_custom -nets $nets_to_route
```

The results from the example show that the wire lengths and the number of vias are the same in each of the four µbump-to-C4 bump connections.



Voltage Area Aware Routing in Custom Router

The IC Compiler II tool supports PG aware routing and buffer distance aware routing.

PG aware routing enables galaxy custom router (GCR) to read the supply net voltage area regions (VAR) information and generate some internal keepouts to route through the bufferable voltage area regions.

Buffer distance aware routing enables GCR to read the supply net VAR information and the buffer distance value. Buffer distance aware routing supports adding buffers to the routing results at a later stage.

Note:

If GCR is unable to find the solution within the specified buffer distance, it gradually increases the buffer distance value and issues a warning message (GCR-435).

Feedback

Routing With the Custom Router

Before performing custom routing on your block, you need to define routing constraints by using the commands described in Defining Routing Constraints, and setting the appropriate application options.

To perform custom routing with Custom Router in the IC Compiler II environment, run the `route_custom` command. By default, the tool routes all nets in the block. To route specific nets, use the `-nets` option.

By default, the tool removes the configuration data when the command run is complete. To keep the data in the cache for running other Custom Router commands in the current session, set the `-keep_session` option to `true`. The default is `false`.

The following example specifies the bus routing style and then performs bus routing.

```
icc2_shell> create_bundle -name Bus1 {net0 net1}
icc2_shell> create_bus_routing_style -for {Bus1} \
   -valid_layers {M5 M6} \
   -layer_widths {M5 0.4 M6 0.44} -force bus1
icc2_shell> create_net_shielding -for {Bus1} -shield_net VSS \
   -layer_gaps 0.21 -layer_widths 0.2 -sharing true -force sh1
icc2_shell> set_app_options -name custom.route.bus_corner_type \
   -value river
icc2_shell> set_app_options \
   -name custom.route.bus_intra_shield_placement -value interleave
icc2_shell> route_custom -nets {net0 net1}
```

The following example creates a constraint group and then performs routing on the nets with the same length.

```
icc2_shell> set matchNets {net1 net2 net3 net4}
icc2_shell> create_bundle -name MatchL1 $matchNets
icc2_shell> create_wire_matching -for MatchL1s \
   -match_type length -tolerance 1 \
   -force matchL1_con
icc2_shell> route_custom -nets {net1 net2 net3 net4}
```

The following example creates a differential group and then performs custom routing.

```
icc2_shell> create_differential_group -for {net1 net2} \
   -valid_layers {M3 M4} -layer_widths {M3 0.4 M4 0.4} \
   -layer_spacings {M3 0.4 M4 0.4} \
   -twist_style diagonal -twist_interval 80.0 -force Diff
icc2_shell> set_app_options\
   -name custom.route.diffpair_twist_jumper_offset -value 10
icc2_shell> route_custom -nets {net1 net2}
```

Feedback

Shielding the Nets

Shielding is needed around sensitive nets to reduce noise and crosstalk. When you perform custom routing by using the route_custom command, the tool automatically creates shields for the nets based on the constraints you specify by using the `create_net_shielding` command.

To add shielding to the selected nets in a separate run, use the `create_custom_shields` command.The following example creates shields for net1 and net2.

```
icc2_shell> create_custom_shields -nets {net1 net2}
```

To remove the shielding created by the `create_custom_shields` command, use the `remove_custom_shields` command. Doing this also removes the shielding created by the `route_custom` command.

Checking the Routing Results

After running Custom Router to perform custom routing on the nets, you can check the routing results by reporting the routing results, generating a congestion map, running DRC checks, and so on.

For more information about the tasks to check the routing results, see Analyzing the Routing Results.

Using a Hybrid Routing Flow

In the hybrid flow, Custom Router is used to perform trunk routing without completing some pin connections, and then Zroute is used to complete the pin connections. This flow enables the use of Custom Router for prerouting while avoiding or reducing inconsistency and DRC violations associated with differences in the pin connection behavior between the two tools.

To enable the hybrid routing flow, set the `custom.route.skip_connect_pin_type` application option to one or more of the following values, depending on your routing requirements. By default, this application option is set to `none`, which disables the hybrid routing flow.

- `auto`

  Custom Router identifies the routability of the pins in the block, and then completes the pin connections for the pins that can be connected.

- `all`

  Custom Router does not complete the pin connections for all pins in the block.

Feedback

- `stdcell`

  Custom Router completes the pin connections for all pins in the block, except for standard cell pins.

- `io`

  Custom Router completes the pin connections for all pins in the block, except for I/O cell pins.

- `macro`

  Custom Router completes the pin connections for all pins in the block, except for macro pins.

By default, when the hybrid flow is enabled, Custom Router determines how much space to leave between a route and an unconnected pin. To specify the maximum distance, use the `custom.route.distance_to_net_pin` application option. Set the value in the following format: `{net_name distance}`.

The following script uses Custom Router for trunk routing and Zroute for all pin connections.

```
set_app_options -name custom.route.skip_connect_pin_type -value all
route_custom -nets {net1 net2}
route_eco -nets {net1 net2}
remove_redundant_shapes -nets {net1 net2}
route_detail -incremental true
```

The following script uses Custom Router for trunk routing and pin connections except standard cell pins, and then uses Zroute for standard cell pin connections.

```
set_routing_rule -min_routing_layer m3 \
   -max_routing_layer m9 \
   -min_layer_mode allow_pin_connection \
   -max_layer_mode hard {net1 net2}
set_app_options -name custom.route.skip_connect_pin_type \
   -value stdcell
set_app_options -name custom.route.distance_to_net_pin \
   -value {{{net1 5.0} {net2 3.0}}}
route_custom -nets {net1 net2}
route_eco -nets {net1 net2}
remove_redundant_shapes -nets {net1 net2}
route_detail -incremental true
```

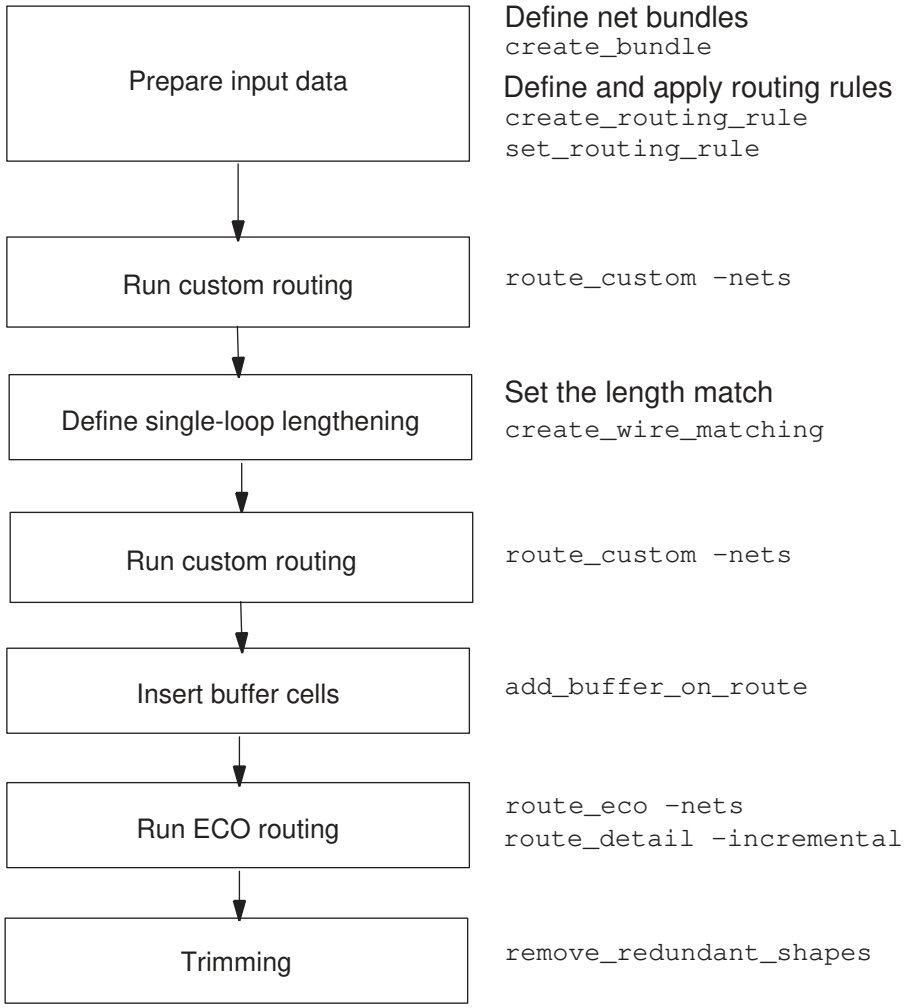
Using a DDR Routing Flow

This topic describes how to use Custom Router to perform routing on a double data rate (DDR) design.

Feedback

Figure 129 illustrates the basic steps to create interconnects (routes) between a DDR net and the core.

Figure 129 A Basic DDR Flow



To perform routing on a DDR design with Custom Router,

1. Before you run Custom Router to perform routing on a DDR design,
   - Group the nets as a bundle for length matching.

     ```
     icc2_shell> create_bundle -name GRP1 [get_nets $nets]
     ```

   - Create and apply the routing rules for the DDR nets.

Feedback

```
icc2_shell> create_routing_rule DDR -widths {layer value …} \
   -spacings {layer value … }
icc2_shell> set_routing_rule -rule DDR -min_routing_layer M4 \
   -max_routing_layer M5 $all_ddr_nets
```

Set two or more layers for initial routing and length matching. For postroute length matching, you should include two additional layers. For example, if you use the M4 and M5 layers for initial routing, set the M3, M4, M5, and M6 layers for postroute length matching.

- Add routing blockages as needed.

  Add routing blockages to prevent Custom Router from routing in the area that is covered by placement blockages. This helps avoid possible unplaceable buffers during buffer insertion.

2. Create routes for the DDR nets by running the `route_custom` command.

```
icc2_shell> route_custom -nets $all_ddr_nets
```

3. Define the routing constraints.

   - Use the following application options to set the single-loop constraints.

```
icc2_shell> set_app_options \
   -name custom.route.single_loop_match -value true
icc2_shell> set_app_options \
   -name custom.route.single_loop_match_min_spacing -value 0.75
icc2_shell> set_app_options \
   -name custom.route.single_loop_match_max_spacing -value 10.0
icc2_shell> set_app_options \
   -name custom.route.single_loop_match_offset_layer -value true
```

   - Set the wire matching constraint for the group.

```
icc2_shell> create_wire_matching -for [get_bundles GRP1] \
   -match_type length -tolerance 20 -force match1
icc2_shell> create_wire_matching -for [get_bundles GRP2] \
   -match_type length -tolerance 20 -force match2
```

   - Set the bounding box for the wire matching routing constraint.

```
icc2_shell> set_app_options -name custom.route.match_box \
    -value {{1000 1000} {1500 1500}}
```

     By default, the bounding box coordinates are set to {{0 0} {0 0}}. When set to other values, the router limits matching routes to the area inside the box, which might result in connections that do not meet the matching constraint. The matching box should cover the entire available channel space.

Feedback

4. Perform custom routing with length matching.

```
icc2_shell> route_custom -nets [get_bundles {GRP1}]
```

   If you define multiple matching boxes at step 3, you need to run the `route_custom` command on each matching box.

5. Report the list of mismatching nets.

```
% grep "Mismatch" log_file
```

6. Reduce the number of the mismatched nets (if any) by using one of the following methods:

   - Rebalance the net routing of one or more groups in the same channel among the available routing layers by

     a. Resetting the routing layers. For example,

        - Set the routing layers for route group A to the M3 and M4 layers, and for group B to the M5 and M6 layers, or

        - Set the routing layers for route groups A and B to the M3 and M4 layers, and route all mismatched nets on the M5 and M6 layers.

     b. Rerunning initial routing with the rebalanced routing layer settings.

   - Set a larger value for the `custom.route.single_loop_match_max_spacing` application option.

   - Allow different layers for the single loop.

```
icc2_shell> set_app_options \
   -name custom.route.single_loop_match_offset_layer -value true
```

7. Insert buffer cells on the routed DDR net.

   In the following example, ddr_BUF is used as the prefix for the names of the added ECO nets and buffers for easy identification. The buffer cells are added at an interval that is 20 percent of the total net length.

```
icc2_shell> set ddr_buf "BUF001"
icc2_shell> add_buffer_on_route -net_prefix ddr_BUF \
   -cell_prefix ddr_BUF \
   -repeater_distance_length_ratio 0.2 \
   -respect_blockages [get_nets $ddr_nets] $ddr_BUF
```

8. Perform legalization and verify that the placement is legal.

```
icc2_shell> legalize_placement -cells $ddr_BUF
icc2_shell> check_legality -cells $ddr_BUF
```

Feedback

Note:

Using the `-cells` option with the `legalize_placement` and `check_legality` commands is supported only in the standard legalizer.

9. Preserve the route shapes of the clock nets by setting the `shape_use` attribute of the clock nets to `user_route`.

```
icc2_shell> set_attribute [get_shapes -of_object $all_nets] \
   shape_use "user_route"
icc2_shell> set_attribute [get_vias -of_object $all_nets] \
   shape_use "user_route"
```

10. Limit rerouting to minor changes by setting the `physical_status` attribute on the nets to `minor_change`.

```
icc2_shell> set_attribute [get_shapes -of_object $all_nets] \
   physical_status "minor_change"
icc2_shell> set_attribute [get_vias -of_object $all_nets] \
   physical_status "minor_change"
```

11. Run ECO routing to reconnect the nets by using the `route_eco` command.

```
icc2_shell> route_eco -nets $all_nets -reroute modified_nets_only
```

12. Run incremental detail routing to fix the DRC violations by running the `route_detail -incremental true` command. After routing is complete, clean up the routed nets by running the `remove_redundant_shapes` command.

```
icc2_shell> set_attribute \
   [get_shapes -of_object $all_nets] shape_use "detail_route"
icc2_shell> set_attribute  \
   [get_vias -of_object $all_nets] shape_use "detail_route"
icc2_shell> route_detail -incremental true
icc2_shell> remove_redundant_shapes -nets $all_nets
```

13. Check the routing results by using one of the following methods:

    - Check for DRC violations.

      ```
      icc2_shell> check_routes
      ```

      By default, the tool checks for DRC violations between signal shapes. To check for DRC violations between signal shapes and user shapes, use the following command:

      ```
      icc2_shell> check_routes -check_from_user_shapes true
      ```

    - View the ECO nets and buffers in the GUI.
    - Report the lengths of the DDR nets and the ECO nets.

9

Physical Datapath With Relative Placement

The physical datapath with relative placement capability provides a way for you to create structures in which you specify the relative column and row positions of instances. During placement and legalization, these structures, which are placement constraints called relative placement structures, are preserved and the cells in each structure are placed as a single entity. Relative placement is also called physical datapath and structured placement.

The concepts and tasks necessary for doing relative placement are described in these sections:

- Introduction to Physical Datapath With Relative Placement
- Relative Placement Flow
- Creating Relative Placement Groups
- Adding Objects to a Group
- Specifying Options for Relative Placement Groups
- Changing the Structures of Relative Placement Groups
- Generating Relative Placement Groups for Clock Sinks
- Performing Placement and Legalization of Relative Placement Groups
- Analyzing Relative Placement Groups
- Saving Relative Placement Information
- Summary of Relative Placement Commands

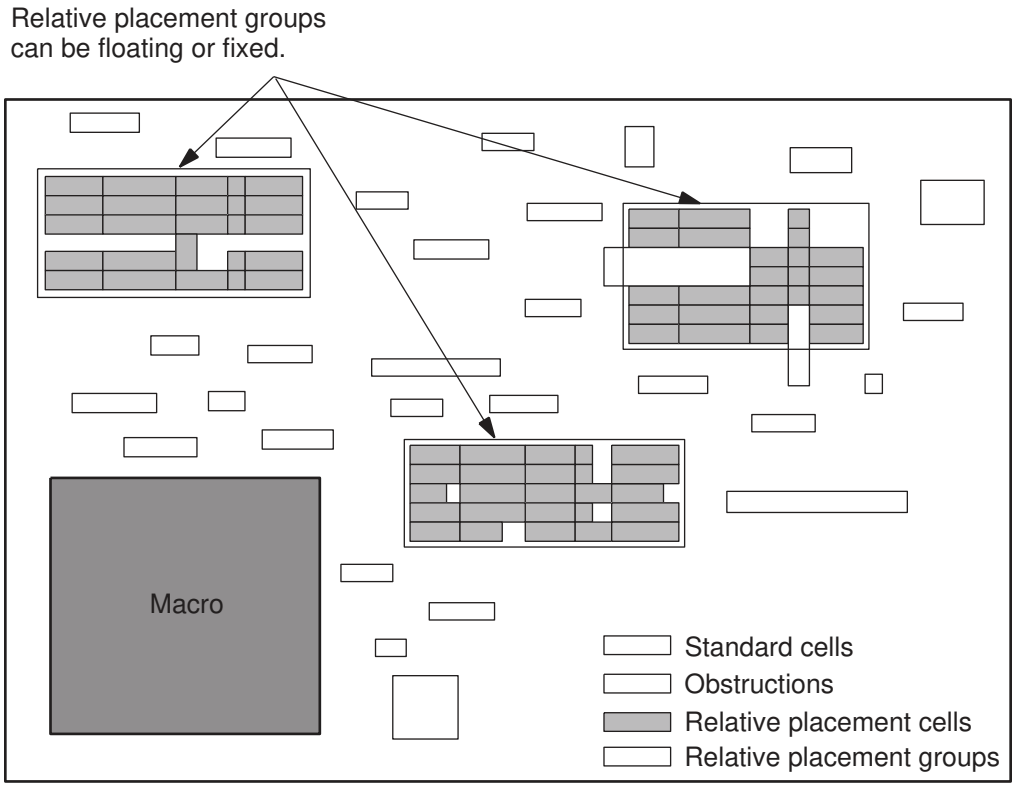
Introduction to Physical Datapath With Relative Placement

Relative placement is usually applied to datapaths and registers, but you can apply it to any cell in your design, controlling the exact relative placement topology of gate-level logic groups and defining the circuit layout. You can use relative placement to explore QoR benefits, such as shorter wire lengths, reduced congestion, better timing, skew control, fewer vias, better yield, and lower dynamic and leakage power.

Feedback

The relative placement constraints that you create and annotate implicitly generate a matrix structure of the instances and control the placement of the instances. You use the resulting annotated netlist for physical optimization, during which the tool preserves the structure and places it as a single entity or group, as shown in Figure 130.

Figure 130 Relative Placement in a Floorplan



Benefits of Relative Placement

Along with being technology-independent and having the ability to improve routability, relative placement provides the following benefits:

- Reduces the placement search space in critical areas of the design, which improves the predictability of QoR (wire length, timing, power, area) and congestion.
- Maintains relative placement during placement, optimization, clock tree synthesis, and routing.
- Provides a method for maintaining structured placement for legacy or intellectual property (IP) designs.

- Handles flat and hierarchical designs.
- Allows sizing of relative placement cells while maintaining relative placement.

Relative Placement Flow

The relative placement flow consists of the following steps:

1. Prepare the design as described in Preparing the Design.
2. Specify placement constraints as described in Setting Up Multivoltage Designs.
3. Define the relative placement constraints and settings.
   a. Create the relative placement groups by using the `create_rp_group` command, as described in Creating Relative Placement Groups.
   b. Add relative placement objects to the groups by using the `add_to_rp_group` command.

      See Adding Objects to a Group.
   c. Specify options for the relative placement groups by using the `set_rp_group_options` command, as described in Specifying Options for Relative Placement Groups.
4. Perform placement and optimization as described in Performing Placement and Optimization.
5. Analyze the relative placement results as described in Analyzing Relative Placement Groups.

   If the relative placement is not what you want, modify the relative placement group or the constraints and settings, and rerun placement and optimization.

Creating Relative Placement Groups

A relative placement group is an association of cells, other relative placement groups, and blockages. A group is defined by the number of rows and columns it uses.

Use the `create_rp_group` command to create a relative placement group. When you do so, you must specify a name for the relative placement group by using the `-name` option.

You can specify the number of columns and rows for the relative placement group by using the `-columns` and `-rows` options. If you do not do so, the tool create a relative placement group with one row and column.

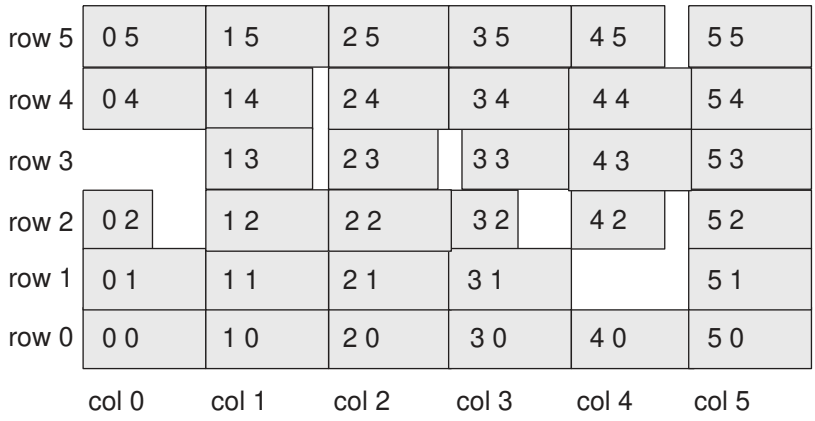
Feedback

For example, to create a relative placement group named RP1 that has six columns and six rows, use the following command:

```
icc2_shell> create_rp_group -name RP1 -columns 6 -rows 6
```

Figure 131 shows the positions of columns and rows in a relative placement group.

Figure 131 Relative Placement Column and Row Positions



For the relative placement group in Figure 131,

- The column count begins from column 0 (the leftmost column).
- The row count begins from row 0 (the bottom row).
- The width of a column is the width of the widest cell in that column.
- The height of a row is the height of the tallest cell in that row.
- All positions in the structure are not used. For example, positions 0 3 (column 0, row 3) and 4 1 (column 4, row 1) are not used.

By default, the tool creates the relative placement group in the current design. You can create it in a different design by specifying its name using the `-design` option. If you create a relative placement group in a design that has multiple instantiations in a top-level design, the changes you make to the relative placement group in one instance is reflected in all its instances.

To remove relative placement groups, use the `remove_rp_groups` command.

Adding Objects to a Group

After you create a relative placement group by using the `create_rp_group` command, you can add the following types of objects to it by using the `add_to_rp_group` command:

- Leaf cells, as described in Adding Leaf Cells
- Leaf cells, as described in Adding Hard Macro Cells
- Relative placement groups, as described in Adding Relative Placement Groups
- Blockages, as described in Adding Blockages

When you add an object to a relative placement group,

- The relative placement group to which you are adding the object must exist.
- The object must be added to an empty location in the relative placement group.
- Only one object can be added in one location of relative placement group.

To remove objects from a relative placement group, use the `remove_from_rp_group` command. You can remove leaf cells (`-cells`), relative placement groups (`-rp_group`), and blockages (`-blockage`).

When you remove objects from a group, the space previously occupied by the removed objects is left unoccupied.

Adding Leaf Cells

To add leaf cells, including physical only cells, to a relative placement group, use the `-cells` option with the `add_to_rp_group` command. Specify the column and row position within the relative placement group at which to add the cell by using the `-column` and `-row` options.

In a relative placement group, a leaf cell can occupy multiple column positions or multiple row positions, which is known as leaf cell straddling. To specify the number of columns and rows the cells occupies, use the `-num_columns` and `-num_rows` options respectively. If you do not set these options, the default is 1 for both the number of columns and rows.

For example, to add a leaf cell that occupies two columns and one row at position (0,0), use

```
icc2_shell> add_to_rp_group rp1 -cells U23 \
   -column 0 -row 0 -num_columns 2 -num_rows 1
```

Straddling is for leaf cells only, and not for hierarchical groups or blockages.

Feedback

Specifying Orientations for Leaf Cells

You can specify orientations for leaf cells when you add them to a relative placement group. If you do not specify a leaf cell orientation, the tool automatically assigns a legal orientation for the leaf cells.

To specify the orientation for leaf cells, use one of the following two methods:

- Use the `-orientation` option with a list of possible orientations when you add the cells to the group with the `add_to_rp_group` command.
- Set the `rp_orientation` attribute on leaf cells by using the `set_attribute` command.

The tool chooses one legal orientation from the list of orientations that you provide.

Adding Hard Macro Cells

The `add_to_rp_group` command supports hard macro cells. When you add macro cells to a relative placement group, you cannot specify

- A pin name with which to align the macro cell by using the `-pin` option
- An orientation for the macro cells by using the `-orientation` option

When you add hard macro cells, use the following steps:

1. Create the relative placement groups, add the cells, including the hard macro cells, and specify the relative placement options and settings.
2. Place the design by using the `create_placement -floorplan` command.
3. Fix the placement of the hard macro cells in the relative placement groups by using the `set_placement_status fixed` command.

   Before you can perform further placement and optimization,

   - The hard macro cells in the relative placement groups must be placed and fixed
   - The other cells in the relative placement groups that contain hard macro cells must be placed
4. Perform further placement and optimization.

Adding Relative Placement Groups

Hierarchical relative placement allows relative placement groups to be embedded within other relative placement groups. The embedded groups then are handled similarly to leaf cells.

Feedback

You can use hierarchical relative placement to simplify the expression of relative placement constraints. With hierarchical relative placement, you do not need to provide relative placement information multiple times for a recurring pattern.

Using hierarchical relative placement provides these benefits:

- Allows you to organize your relative placement in a manner that is easier to maintain and understand. For example, you can create the relative placement group to parallel your Verilog or VHDL organization.
- Allows reuse of a repeating placement pattern, such as an adder.
- Can reduce the number of lines of relative placement information you need to write.
- Allows integrating blocks.
- Provides flexibility for the configuration you want.

Creating Hierarchical Relative Placement Groups

To create a hierarchical relative placement group by adding a group to another group, use the `-rp_group` option with the `add_to_rp_group` command. Specify the column and row position within the relative placement group by using the `-column` and `-row` options.

The group you specify with the `-rp_group` option must be in the same design as the hierarchical group in which you are including it.

When you include a relative placement group in a hierarchical group, it is as if the included group is directly embedded within its parent group. An included group can be used only in a group of the same design and only one time. However, a group that contains an included group can be further included in another group in the same design or can be instantiated in a group of a different design.

The script in Example 41 creates a hierarchical group (rp4) that contains three included groups (rp1, rp2, and rp3). Groups rp1, rp2, rp3, and rp4 are all in the design top. The contents of groups rp1, rp2, and rp3 are treated as leaf cells when they are included in group rp4. You can further include group rp4 in another group in the design top, or you can instantiate group rp4 in a group of a different design.

The resulting hierarchical relative placement group is shown in Figure 132.

Example 41 Including Groups in a Hierarchical Group

```
create_rp_group -name rp1 -columns 2 -rows 1
add_to_rp_group rp1 -cells U1 -column 0 -row 0
add_to_rp_group rp1 -cells U4 -column 1 -row 0

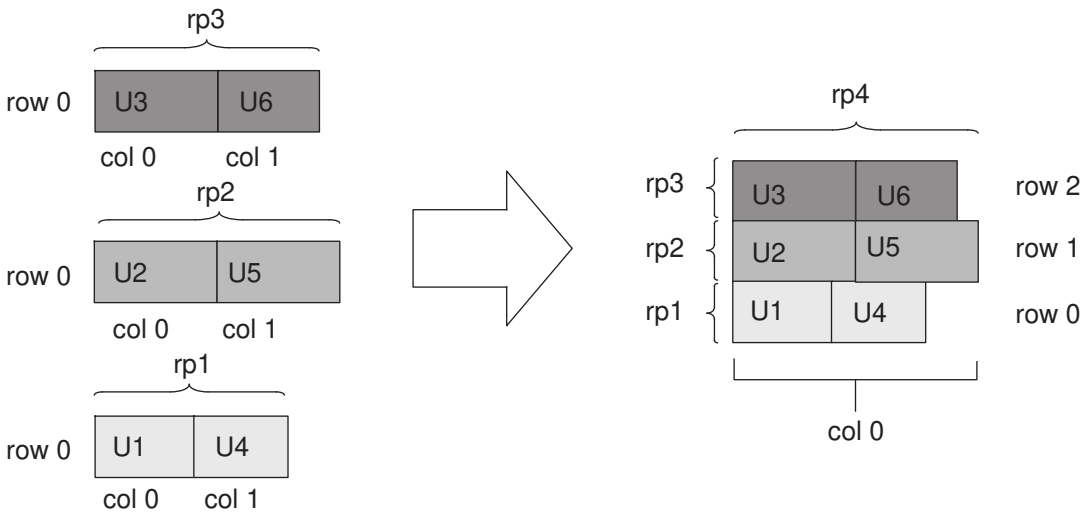
create_rp_group -name rp2 -columns 2 -rows 1
add_to_rp_group rp2 -cells U2 -column 0 -row 0
add_to_rp_group rp2 -cells U5 -column 1 -row 0
```

Feedback

```
create_rp_group -name rp3 -columns 2 -rows 1
add_to_rp_group rp3 -cells U3 -column 0 -row 0
add_to_rp_group rp3 -cells U6 -column 1 -row 0

create_rp_group -name rp4 -columns 1 -rows 3
add_to_rp_group rp4 -rp_group rp1 \
   -column 0 -row 0
add_to_rp_group rp4 -rp_group rp2 \
   -column 0 -row 1
add_to_rp_group rp4 -rp_group rp3 \
   -column 0 -row 2
```

Figure 132 Including Groups in a Hierarchical Group

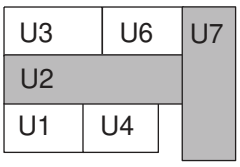


Using Hierarchical Relative Placement for Straddling

A cell can occupy multiple column positions or multiple row positions, which is known as straddling. For more information about leaf cell straddling, see Adding Leaf Cells.

Figure 133 shows a relative placement group in which cells straddle columns (instance U2) and rows (instance U7).

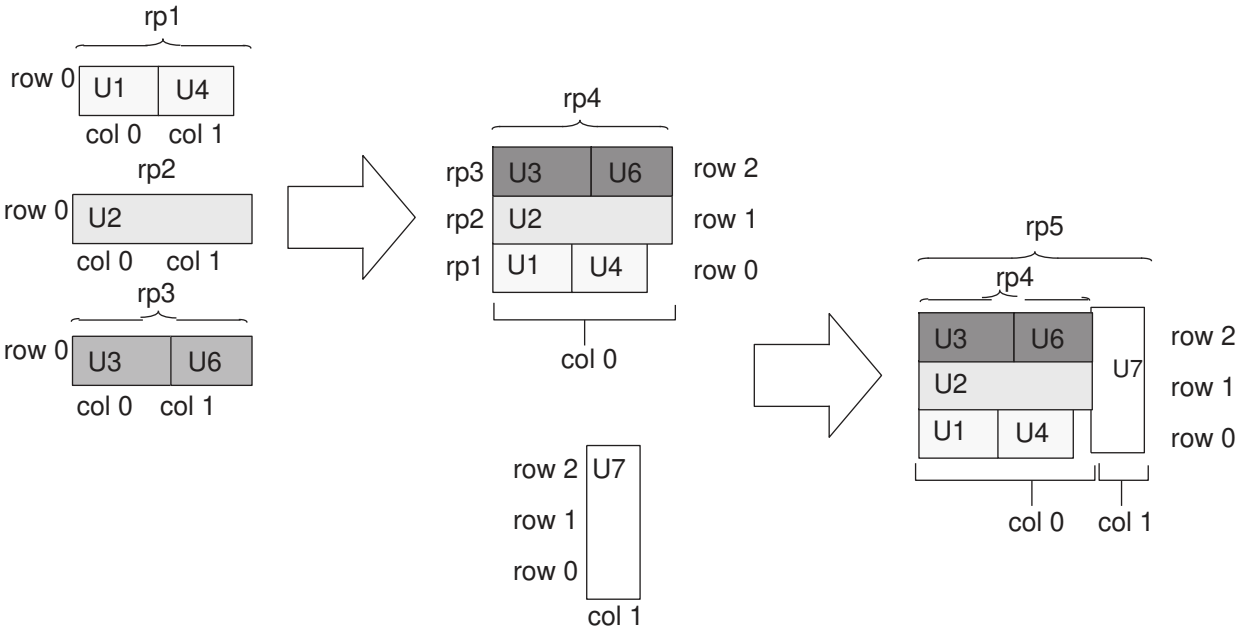
Figure 133 Hierarchical Relative Placement Group With Straddling



Feedback

Figure 134 shows the process of using hierarchical relative placement to build this structure. First, define relative placement groups that contain the leaf cells: rp1 contains U1 and U4, rp2 contains U2, and rp3 contains U3 and U6. Then define a group (rp4) that contains these groups. Finally, define a group (rp5) that contains the hierarchical group rp4 and the leaf cell U7. The resulting group includes both the column and the row straddle. Example 42 shows the commands used in this process.

Figure 134 Straddling With Hierarchical Relative Placement



Example 42 Straddling With Hierarchical Relative Placement

```
create_rp_group -name rp1 -columns 2 -rows 1
add_to_rp_group rp1 -cells U1 -column 0 -row 0
add_to_rp_group rp1 -cells U4 -column 1 -row 0

create_rp_group -name rp2 -columns 1 -rows 1
add_to_rp_group rp2 -cells U2 -column 0 -row 0

create_rp_group -name rp3 -columns 2 -rows 1
add_to_rp_group rp3 -cells U3 -column 0 -row 0
add_to_rp_group rp3 -cells U6 -column 1 -row 0

create_rp_group -name rp4 -columns 1 -rows 3
add_to_rp_group rp4 -rp_group rp1 -column 0 -row 0
add_to_rp_group rp4 -rp_group rp2 -column 0 -row 1
add_to_rp_group rp4 -rp_group rp3 -column 0 -row 2

create_rp_group -name rp5 -columns 2 -rows 1
```

Feedback

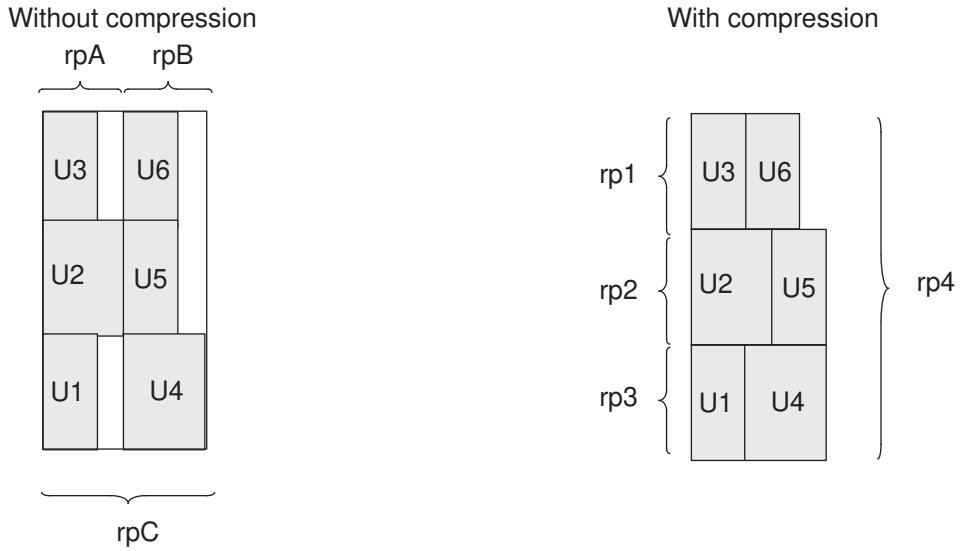
```
add_to_rp_group rp5 -rp_group rp4 -column 0 -row 0
add_to_rp_group rp5 -cells U7 -column 1 -row 0
```

Using Hierarchical Relative Placement for Compression

By default, construction for relative placement aligns cells from their bottom-left corner. Compression removes empty space in rows to create a more compact structure. The columns are no longer aligned, and utilization is higher in the area of the compressed cells.

Figure 135 shows the same cells aligned with and without compression. To create the compressed structure shown in this example, first create three relative placement groups, rp1, rp2, and rp3, that contains a row of leaf cells. Then create a group, rp4, that contains all these groups. Example 43 shows the commands used to build the compressed structure.

Figure 135 Bottom-Left Alignment Construction and Compression



Example 43 Compression With Hierarchical Relative Placement

```
create_rp_group -name rp1 -columns 2 -rows 1
add_to_rp_group rp1 -cells U1 -column 0 -row 0
add_to_rp_group rp1 -cells U4 -column 1 -row 0

create_rp_group -name rp2  -columns 2 -rows 1
add_to_rp_group rp2 -cells U2 -column 0 -row 0
add_to_rp_group rp2 -cells U5 -column 1 -row 0

create_rp_group -name rp3 -columns 2 -rows 1
add_to_rp_group rp3 -cells U3 -column 0 -row 0
add_to_rp_group rp3 -cells U6 -column 1 -row 0
```

Feedback

```
create_rp_group -name rp4 -columns 1 -rows 3
add_to_rp_group rp4 -rp_group rp1 -column 0 -row 0
add_to_rp_group rp4 -rp_group rp2 -column 0 -row 1
add_to_rp_group rp4 -rp_group rp3 -column 0 -row 2
```

Alternatively, you can apply compression in the horizontal direction by using the `-tiling_type` option with the `set_rp_group_options` command, as described in Controlling the Tiling Within Relative Placement Groups.

Adding Blockages

To add a blockage within relative placement groups, use the `-blockage` option with the `add_to_rp_group` command.

When you add a blockage using this command, you can specify

- The column and row position within the relative placement group by using the `-column` and `-row` options.

  If you do not specify a position, the tool adds the blockage to position (0,0).

- The size of the blockage by using the `-height` and `-width` options.

  If you do not specify the `-height` or `-width` option, the tool determines the size based on the tiling type of the relative placement group as follows:

  - For a tiling type setting of `bit_slice`, the height default to the height of site row and the width to width of the column.
  - For a tiling type setting of `compression`, the height default to the height of site row and the width to the width of one site.

- That the blockages can overlap with other relative placement blockages or other objects that are not relative placement cells by using the `-allow_overlap` option.

The following example adds a blockage named gap1 to the rp1 relative placement group at position (0,2) that is one site row high and five site rows wide:

```
icc2_shell> add_to_rp_group rp1-blockage gap1 \
   -column 0 -row 2 -width 5 -height 1
```

Feedback

Adding Cells Within a Predefined Relative Placement Area

To add cells anywhere within a predefined area of a relative placement group, use the `-free_placement` option with the `add_to_rp_group` command. With this option, you must specify

- The origin of the placement area by using the `-column` and `-row` options
- The height and width of the placement area by using the `-height` and `-width` options
- The number of columns and rows the cells occupy by using the `-num_columns` and `-num_rows` options

For example, to add a leaf cell that occupies two columns and two rows to a placement area with a height of 4 and a width of 5 and is at position (0,0), use

```
icc2_shell> add_to_rp_group rp2 -cells U41 -free_placement \
   -column 0 -row 0 -height 4 -width 5 -num_columns 2 -num_rows 2
```

Specifying Options for Relative Placement Groups

To specify properties of relative placement groups, use the `set_rp_group_options` command as described in Table 52.

Table 52 Specifying Relative Placement Group Properties

To do this	Use this option
Specify the anchor location. See Anchoring Relative Placement Groups.	`-x_offset` `-y_offset`
Specify a corner for the anchor point set by the `-x_offset` and `-y_offset` options. See Anchoring Relative Placement Groups.	`-anchor_corner`
Specify the type of alignment used by the cells in the group. See Aligning Leaf Cells Within a Column.	`-alignment`
Specify the group alignment pin name, when using pin alignment. See Aligning by Pin Location.	`-pin_name`
Specify a tiling type. See Controlling the Tiling Within Relative Placement Groups.	`-tiling_type`
Specify the orientation. See Specifying the Orientation of Relative Placement Groups.	`-group_orientation`
Specify the utilization (default is 100 percent).	`-utilization`

Feedback

Table 52 Specifying Relative Placement Group Properties (Continued)

To do this	Use this option
Specify how to treat fixed cells in the floorplan during legalization. See Handling Fixed Cells During Relative Placement.	`-place_around_fixed_cells`
Control the type of optimization that is allowed for relative placement cells See Controlling the Optimization of Relative Placement Cells.	`-optimization_restriction`
Control the movement of the group during legalization. See Controlling Movement When Legalizing Relative Placement Groups.	`-move_effort`

To remove relative placement group option settings, use the `remove_rp_group_options` command. You must specify the group name and at least one option; otherwise, this command has no effect.

Anchoring Relative Placement Groups

By default, the IC Compiler II tool can place a relative placement group anywhere within the core area. You can control the placement of a top-level relative placement group by anchoring it.

To anchor a relative placement group, use the `set_rp_group_options` command with the `-x_offset` and `-y_offset` options. The offset values are float values, in microns, relative to the lower-left corner in the core area.

If you specify both the x- and y-coordinates, the group is anchored at that location. If you specify only one coordinate, the IC Compiler II tool determines the placement by maintaining the specified coordinate and sliding the group along the line passing through the unspecified coordinate.

To specify a corner of relative placement group to anchor it by, use the `-anchor_corner` option. The tool places the relative placement group such that the corner specified by this option is placed on the anchor point specified by the `-x_offset` and `-y_offset` options.

The settings for the `-anchor_corner` option are as follows, and are shown in Figure 136:

- `bottom_left`

  The anchor point of the relative placement group is set to its bottom-left corner. The default is the bottom-left corner.

- `bottom_right`

  The anchor point of the relative placement group is set to its bottom-right corner.

Feedback

- `top_left`

  The anchor point of the relative placement group is set to its top-left corner.

- `top_right`

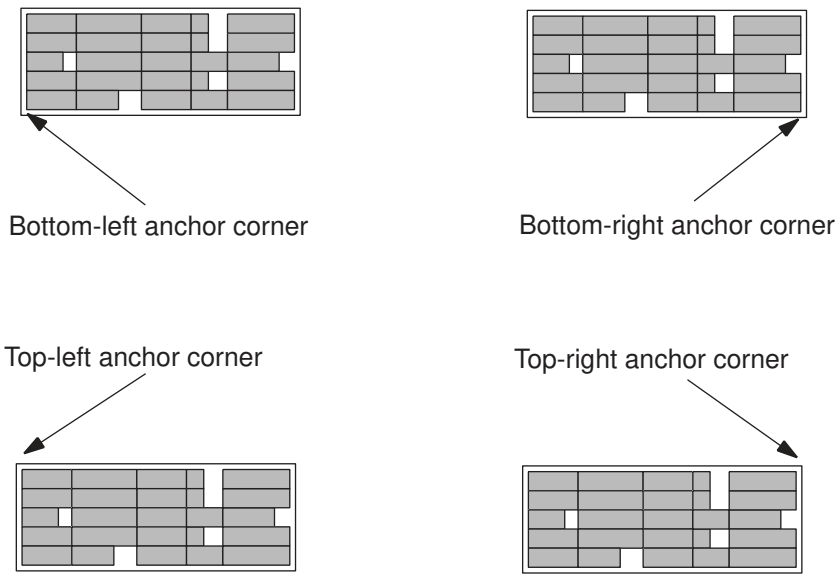
  The anchor point of the relative placement group is set to its top-right corner.

- `rp_location`

  The anchor point of the relative placement group is the element in the relative placement group at the position specified by the `-anchor_row` and `-anchor_column` options.

  When you use the `-anchor_corner rp_location` setting, the position specified with the `-anchor_row` and `-anchor_column` options must contain a cell, keepout, or a relative placement hierarchy.

Figure 136 Bottom-Left, Bottom-Right, Top-Left and Top-Right Anchor Corners

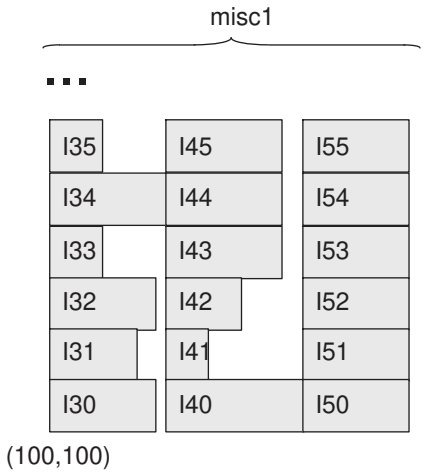


For example, to anchor a relative placement by its bottom let corner at location (100, 100), as shown Figure 137, use the following command:

```
icc2_shell> set_rp_group_options misc1 -anchor_corner bottom_left \
    -x_offset 100 -y_offset 100
```

Feedback

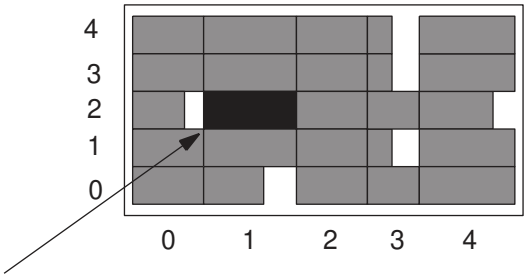
Figure 137 Anchored Relative Placement Group



The following example specifies that relative placement cell at column 1, row 2 of the RP1 relative placement group should be anchored at location (100, 100).

```
icc2_shell> set_rp_group_options RP1 \
   -anchor_corner rp_location -anchor_column 1 -anchor_row 2 \
   -x_offset 100 -y_offset 100
```

Figure 138 Using an Object Within the Relative Placement Group for Anchoring



The relative placement group is anchored by placing the cell at column 1 row 2 at location (100, 100)

Feedback

Aligning Leaf Cells Within a Column

You can align the leaf cells in a column of a relative placement group by using the following alignment methods:

- Left alignment (default)
- Right alignment
- Pin alignment

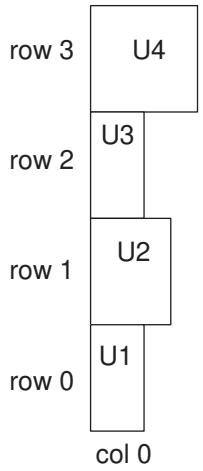
Controlling the cell alignment can improve the timing and routability of your design.

Aligning by the Left Edges

By default, the IC Compiler II tool aligns the leaf cells by aligning the left edges. To explicitly specify this alignment method, use the `-alignment left` option of the `set_rp_group_options` command.

Figure 139 shows cells that are left aligned.

Figure 139 Bottom-Left-Aligned Relative Placement Group



Aligning by the Right Edges

To align a group by aligning the right edges, use the `-alignment right` option of the `set_rp_group_options` command.

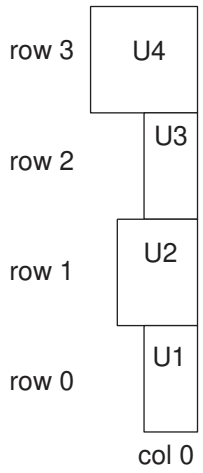
Note:

For hierarchical relative placement groups, the bottom-right alignment does not propagate through the hierarchy.

Figure 139 shows cells that are right aligned.

Feedback

Figure 140 Bottom-Right-Aligned Relative Placement Group



Aligning by Pin Location

To align a group by pin location, use the `-alignment pin` and `-pin_name` options with the `set_rp_group_options` command.

The tool looks for the specified alignment pin in each cell in the column. If the alignment pin exists in a cell, the cell is aligned by using the pin location. If the specified alignment pin does not exist in a cell, the cell is aligned by the left edge, and the tool issues an information message. If the specified alignment pin does not exist in any cell in the column, the IC Compiler II tool issues a warning message.

The script in Example 44 creates a relative placement group rp1, adds cells to it, and specifies that the cells are aligned by pin A.

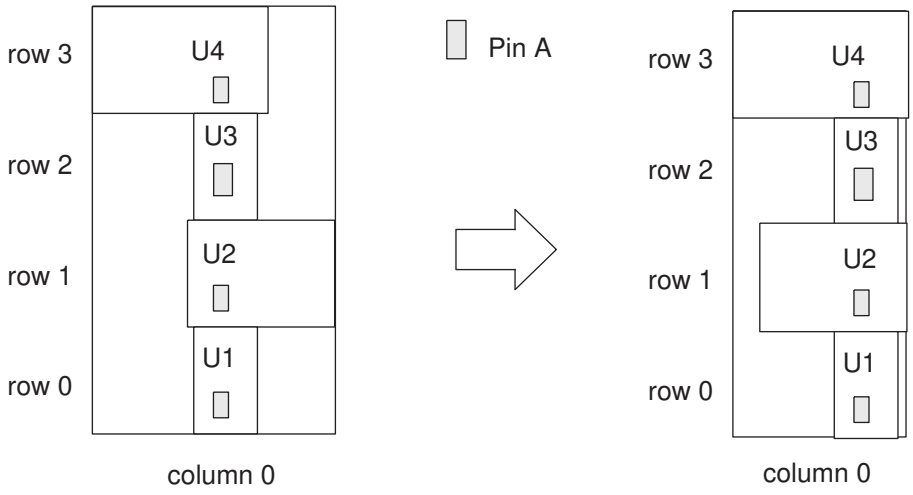
Example 44 Definition for Relative Placement Group Aligned by Pins

```
create_rp_group -name rp1 -columns 1 -rows 4
set_rp_group_options -alignment pin -pin_name A
add_to_rp_group rp1 -cells U1 -column 0 -row 0
add_to_rp_group rp1 -cells U2 -column 0 -row 1
add_to_rp_group rp1 -cells U3 -column 0 -row 2
add_to_rp_group rp1 -cells U4 -column 0 -row 3
```

When aligning by pins, the tool tries different orientations for the cells and selects the orientation for each cell that gives the minimum column width. For example, changing the orientation of cell U2, as shown in Figure 141, reduces the width of column 0. However, if you specify an orientation when adding a cell to a relative placement group by using the `-cells` and `-orientation` options with the `add_to_rp_group` command, the tool honors the orientation you specify.

Feedback

Figure 141 Minimizing the Column Width of a Relative Placement Group Aligned by Pins



When you specify an alignment pin for a group, the pin applies to all cells in the group. You can override the group alignment pin for specific cells in the group by specifying the `-pin_name` option when you use the `add_to_rp_group` command to add the cells to the group.

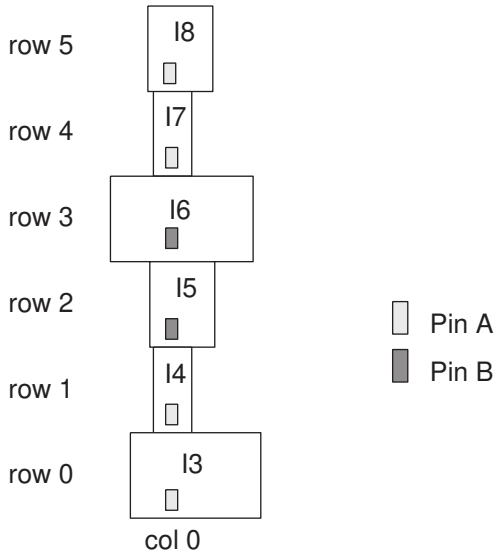
The script in Example 45 defines relative placement group rp2, and specified pin A as the group alignment pin. However, instances I5 and I6 use pin B as their alignment pin, rather than the group alignment pin. The resulting structure is shown in Figure 142.

Example 45 Definition for Aligning a Group and Leaf Cells by Pins

```
create_rp_group -name rp2 -columns 1 -rows 6
set_rp_group_options rp2 -alignment pin -pin_name A
add_to_rp_group rp2 -cells I3 -column 0 -row 0
add_to_rp_group rp2 -cells I4 -column 0 -row 1
add_to_rp_group rp2 -cells I5 -column 0 -row 2 -pin_name B
add_to_rp_group rp2 -cells I6 -column 0 -row 3 -pin_name B
add_to_rp_group rp2 -cells I7 -column 0 -row 4
add_to_rp_group rp2 -cells I8 -column 0 -row 5
```

Feedback

Figure 142 Relative Placement Group Aligned by Different Pins



Overriding the Alignment When Adding Objects

When you add an object to a relative placement group by using the `add_to_rp_group` command, you can override its alignment and specify a different alignment for the object you are adding by using the `-override_alignment` option. However, if the relative placement group is pin aligned, you cannot override the alignment with the `-override_alignment` option.

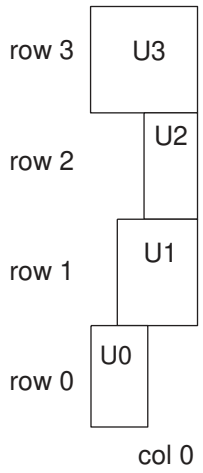
The following example creates a relative placement group named rp1 that is right aligned. It then adds a cell named U0, which overrides the alignment of the group and cells named U1, U2, and U3, which honor the alignment of the relative placement group:

```
icc2_shell> create_rp_group rp1 -name rp1 -columns 1 -rows 4
icc2_shell> set_rp_group_options rp1 -alignment right
icc2_shell> add_to_rp_group rp1 -cells U0 -column 0 -row 0 \
   -override_alignment left
icc2_shell> add_to_rp_group rp1 -cells U1 -column 0 -row 1
icc2_shell> add_to_rp_group rp1 -cells U2 -column 0 -row 2
icc2_shell> add_to_rp_group rp1 -cells U3 -column 0 -row 3
```

The following figure shows the relative placement group after placement.

Feedback

Figure 143 Right-Aligned Relative Placement Group With One Cell That is Left Aligned



Controlling the Tiling Within Relative Placement Groups

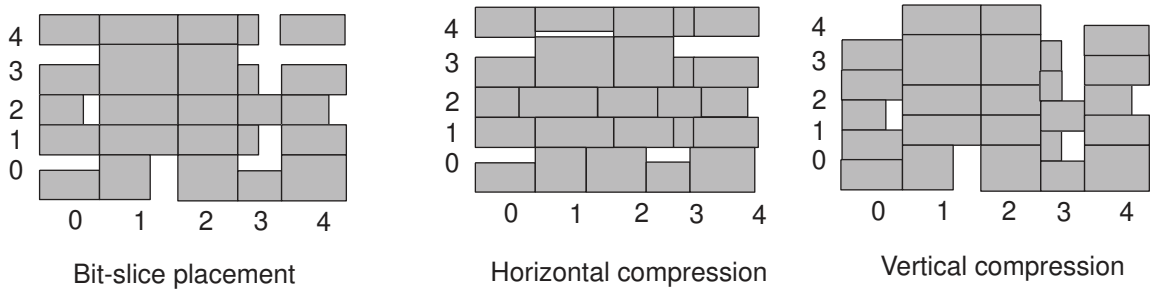
To control the tiling of objects within a relative placement group, use the `set_rp_group_options -tiling_type` command with the settings shown in the following table.

Table 53 Controlling Placement With the set_rp_group_options -tiling_type Command

To do this	Use this setting
Tile objects in a bit-slice pattern and preserve both the row and column alignment. • The height of a row is determined by the tallest object in the row • The width of a column is determined by the widest object in the column	`bit_slice` (Default)
Horizontally compress the objects in each row and maintain row alignment.	`horizontal_compression`
Vertically compress the objects in each column and preserve the column alignment	`vertical_compression`

Feedback

Figure 144 Bit-Slice Placement Versus Horizontal or Vertical Compression



The setting of the `-tiling_type` option is not propagated from a parent group to child groups.

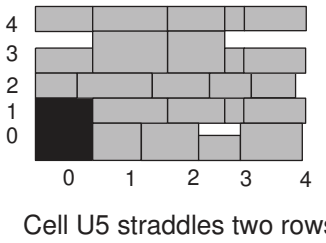
Applying Compression to Groups With Straddling Leaf Cells

You can apply compression to a relative placement group with cells that straddle multiple rows or columns, as shown in the following example:

```
icc2_shell> add_to_rp_group rp -cells U5 \
    -column 0 -row 0 -num_columns 1 -num_rows 2
icc2_shell> set_rp_group_options rp -tiling_type horizontal_compression
```

Figure 145 shows the placement of the relative placement group in the previous example.

Figure 145 Compression of a Relative Placement Group With a Cell That Straddles Multiple Rows



For information about adding cells that straddle multiple rows or columns to a relative placement group, see Adding Leaf Cells.

Feedback

Specifying the Orientation of Relative Placement Groups

The IC Compiler II tool supports the following orientations for relative placement groups:

- R0

  The column position of the relative placement group is from left to right, and the row position is from bottom to top.

- R180

  The column position of the relative placement group is from right to left, and the row position is from top to bottom.

- MY

  The column position of the relative placement group is from right to left, and the row position is from bottom to top; that is, the orientation of the group is flipped with respect to the R0 orientation.

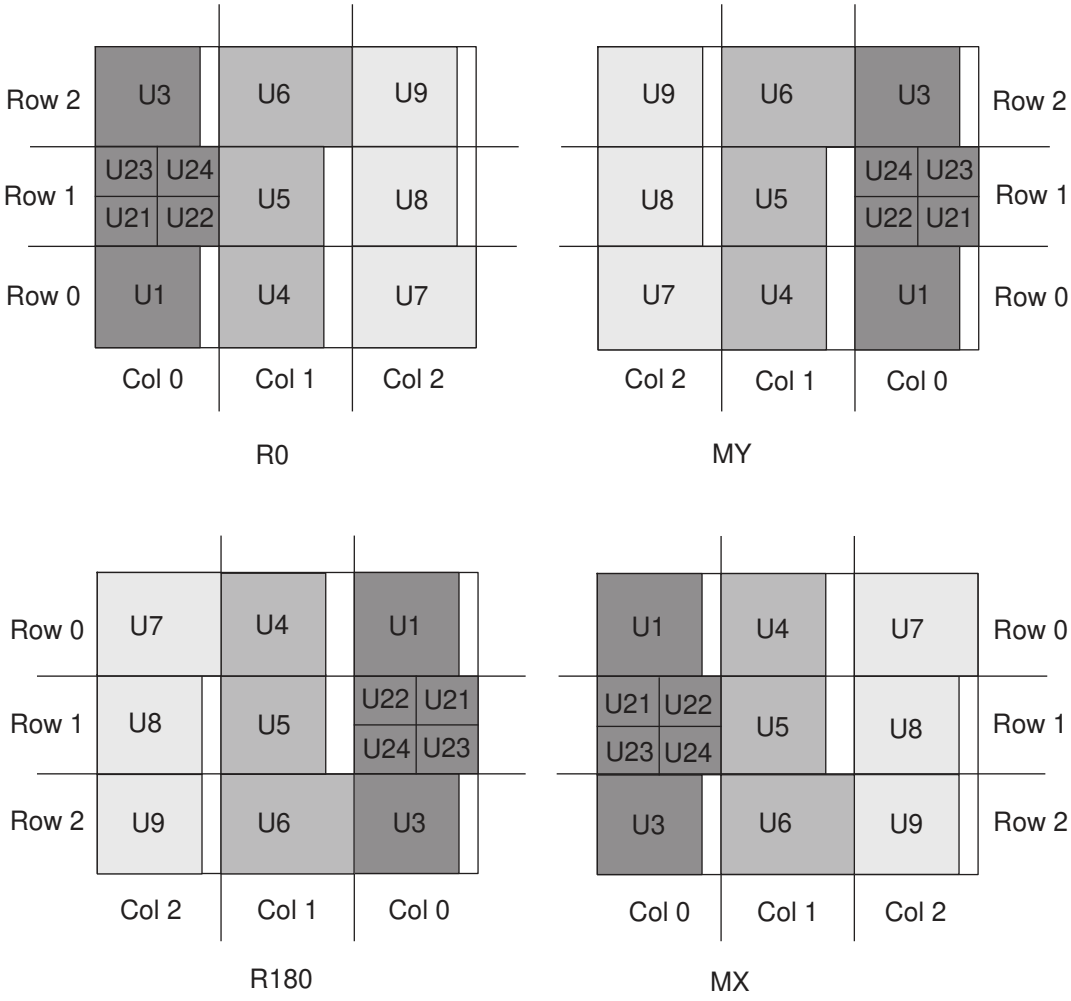
- MX

  The column position of the relative placement group is from left to right, and the row position is from top to bottom; that is, the group is flipped with respect to the R180 orientation.

Figure 146 shows how the column and row positions in a relative placement group are placed for the four orientations.

Feedback

Figure 146 Orientation of Relative Placement Groups



The orientation of relative placement groups is automatically set by the tool to minimize wire length. You can also choose to set the orientation of relative placement groups by using the `set_rp_group_options` command.

For example, the following command sets the relative placement group orientation to MY.

```
icc2_shell> set_rp_group_options [get_rp_groups design::rp] \
   -group_orientation MY
```

For designs with hierarchical relative placement groups, the orientation settings are propagated down to the lowest level in hierarchy.

Feedback

Note:

When the orientation of a relative placement group is changed, the constraints on the relative placement group, such as alignment and utilization, are preserved according to the specifications that you provide.

Specifying a Keepout Margin

To prevent other relative placement groups from being placed close to a specific relative placement group, you can specify a keepout margin that applies only to other relative placement groups. To do so, use the `-rp_only_keepout_margin` option with the `set_rp_group_options` command. You can specify a different margin for the left, bottom, right, and top sides of the group.

The following command applies a margin of 15 on the left and right and a margin of 10 on the top and bottom of the relative placement group named rp1:

```
icc2_shell> set_rp_group_options rp1 \
    -rp_only_keepout_margin {15 10 15 10}
```

Handling Fixed Cells During Relative Placement

To specify how to handle fixed cells in the floorplan during legalization of relative placement groups, use the `-place_around_fixed_cells` option with the `set_rp_group_options` command. Table 54 shows the different settings you can specify for the `-place_around_fixed_cells` option.

Table 54 Settings for the -place_around_fixed_cells Option

To do this	Use this setting
Legalize relative placement groups around fixed standard cells and avoid fixed physical-only cells.	`standard`
Legalize relative placement groups around fixed physical-only cells and avoid fixed standard cells.	`physical_only`
Legalize relative placement groups around both fixed standard cells and fixed physical-only cells. This is the default.	`all`
Avoid both fixed standard cells and fixed physical-only cells.	`none`

For hierarchical relative placement groups, you can use the `set_rp_group_options -place_around_fixed_cells` command and specify different settings for the top-level and the lower-level relative placement groups. If you do not specify a setting for the lower-level placement groups, the value of the top-level relative placement group is used for the lower-level relative placement groups.

Feedback

Assume a hierarchical relative placement group named top-rp contains three lower-level relative placement groups named rp1, rp2, and rp3. The following example specifies a less restrictive setting for the top level and a more restrictive setting for the lower-level group named rp1. It also specifies that the setting for the rp1 lower-level group overrides the top-level setting when legalizing the cells in the rp1 group.

```
icc2_shell> set_rp_group_options top-rp \
-place_around_fixed_cells physical_only
icc2_shell> set_rp_group_options rp1 \
-place_around_fixed_cells none
icc2_shell> set_app_options -name set_rp_group_options -value true
```

Allowing Nonrelative Placement Cells

By default, nonrelative placement cells are

- Not allowed within relative placement groups during coarse placement
- Allowed within relative placement groups during optimization and legalization

To reduce congestion and improve QoR, you can allow the tool to place nonrelative placement cells within the unused areas of a specific relative placement group by using the `-allow_non_rp_cells` option with the `set_rp_group_options` command.

You can add blockages to relative placement group by using the `-blockage` option with the `add_to_rp_group` command. By default, no cells are allowed within these blockages during placement, optimization, and legalization. To allow nonrelative placement cells within relative placement blockages and unused areas of a specific relative placement group, use the `-allow_non_rp_cells_on_blockages` option with the `set_rp_group_options` command.

The following example allows nonrelative placement cells within the unused areas of the RP1 relative placement group:

```
icc2_shell> set_rp_group_options rp1 \
-allow_non_rp_cells
```

The following example allows nonrelative placement cells within the blockages and unused areas of the RP2 relative placement group:

```
icc2_shell> set_rp_group_options rp2 \
-allow_non_rp_cells_on_blockages
```

Controlling the Optimization of Relative Placement Cells

When a relative placement cell is modified or moved, the relative placement structure can be disturbed. When a relative placement cell is removed during optimization, the relative

Feedback

placement information of the instance is also removed, disrupting the relative placement structure.

To preserve the relative placement structures during various postplacement optimization processes, use the `-optimization_restriction` option with the `set_rp_group_options` command and specify the appropriate setting as shown in Table 55.

Table 55 Settings for the -optimization_restriction Option

To do this	Use this setting
Allow unrestricted optimization of the relative placement cells	`all_opt`
Allow only sizing of the relative placement cells	`size_only`
Allow only in-place sizing of the relative placement cells	`size_in_place`
Prevent any optimization of the relative placement cells	`no_opt`

For a hierarchical relative placement group, the `-optimization_restriction` option setting applied to the top level is propagated to the lower-level groups and any settings applied to the lower-level groups are ignored.

Controlling Movement When Legalizing Relative Placement Groups

You can control the movement of a relative placement group during legalization by using the `-move_effort` option of the `set_rp_group_options` command. During coarse placement, the tool estimates an initial location for every top-level relative placement group. The `-move_effort` option controls the extent to which a relative placement group can be moved from its initial location to preserve the relative placement without violating relative placement constraints. When you change the option setting from a higher effort level to a lower effort level, you reduce the size of the region searched for placement of a relative placement group.

Handling Tap Columns and Relative Placement Group Overlaps

During coarse placement, to avoid the overlap of relative placement groups with the tap cell columns, use the `place.coarse.no_split_rp_width_threshold` application option. The overlap can be removed for only those relative placement groups whose width is less than the distance between tap columns. This application option helps the relative placement engine know about the distance between the tap columns.

Feedback

This helps in better placement and structures of relative placement groups in the design. You must use this application option for n5 and older technical nodes. For n3 and beyond designs, the distance between tap columns is calculated automatically.

```
place.coarse.no_split_rp_width_threshold <value>
For example,
place.coarse.no_split_rp_width_threshold 46
```

Changing the Structures of Relative Placement Groups

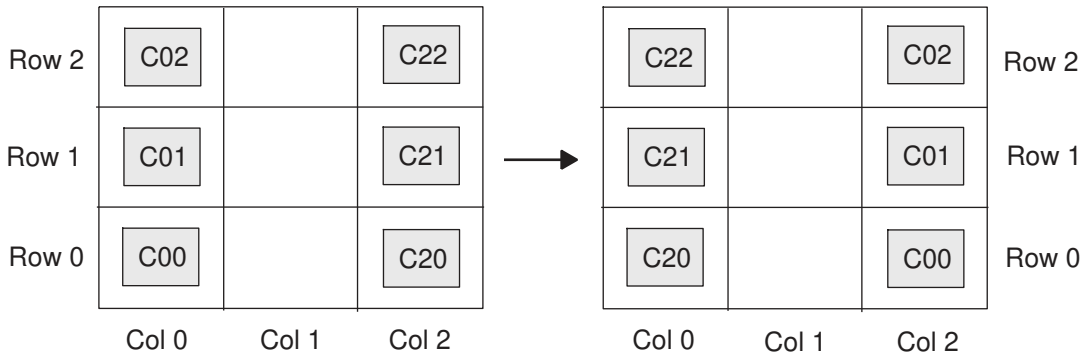
To modify the structures of existing relative placement groups, use the `modify_rp_groups` command as follows:

- To add a row or column to a relative placement group, use the `-add_rows` or `-add_columns` option respectively.
- To remove a row or column, use the `-remove_rows` or `-remove_columns` option respectively.
- To flip a row or column, use the `-flip_row` or `-flip_column` option respectively.
- To swap two rows or columns, use the `-swap_rows` or `-swap_columns` option respectively.

For example, to swap the first and third columns of the my_rp_group relative placement group, as shown in Figure 147, enter

```
icc2_shell> modify_rp_groups [get_rp_groups my_rp_group] \
   -swap_columns {0 2}
```

Figure 147 Swapping Columns of a Relative Placement Group

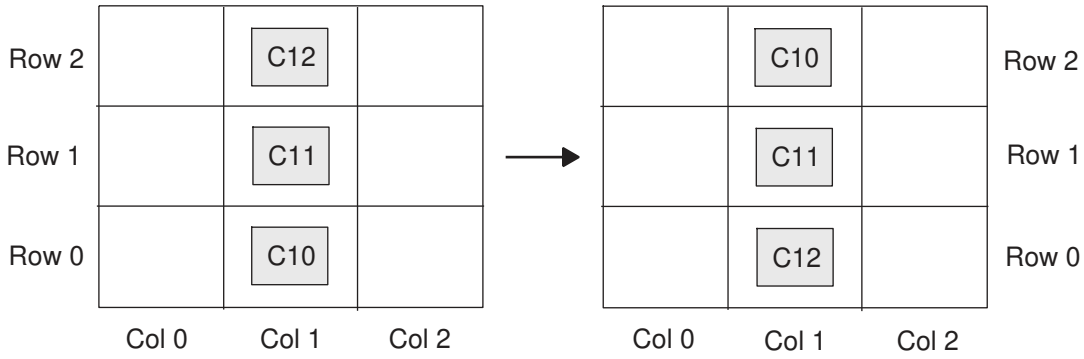


Feedback

To flip the second column of the my_rp_group relative placement group, as shown in Figure 148, enter

```
icc2_shell> modify_rp_groups [get_rp_groups my_rp_group] \
   -flip_column 1
```

Figure 148 Flipping a Column of a Relative Placement Group



Generating Relative Placement Groups for Clock Sinks

You can generate a relative placement group for clock sinks and their drivers in a placed design by using the `create_clock_rp_groups` command. When you run placement and optimization, the tool places the clocks sinks and their drivers based on their relative placement constraints it generated. This can improve routability and reduce dynamic power.

You can control the sinks being considered for relative placement groups as follows:

- Specify the minimum and maximum of sinks that should be driven by a single driver to be considered by using the `-min_sinks` and `-max_sinks` options.

  The default minimum is 2 sinks and the default maximum is 128 sinks.

- Exclude timing critical sinks by using the `-timing_driven` option.

  By default, the tool excludes all sinks with a negative slack.

- Specify the cells to consider by using the `-cells` option.

  B default, the tool considers all the sinks.

Feedback

You can control the size and shape of the relative placement group by using one of the following methods:

- Specify the maximum allowed Manhattan distance between the sinks in one relative placement group by using the `-distance` option.

  If the Manhattan distance between sinks is more than the specified distance, they are put into separate relative placement groups. The default distance is 100 microns.

- Specify a maximum number of rows by using the `-max_rp_rows` option.

  The default is 32.

- Allow the tool to decide the number of rows and columns based on the distribution of the cells by using the `-auto_shape` option.

Before you run the `create_clock_rp_groups` command, the block must be placed. After you create the clock relative placement groups, reset the placement of the block by using the `reset_placement` command, and rerun placement and optimization by using the `place_opt` command.

Performing Placement and Legalization of Relative Placement Groups

The following topics provide information related to the placement and legalization of relative placement groups:

- Relative Placement in a Design Containing Obstructions
- Legalizing Relative Placement Groups in a Placed Design
- Creating New Relative Placement Groups in a Placed Design

Relative Placement in a Design Containing Obstructions

During placement, relative placement groups avoid placement blockages (obstructions) that are defined in the DEF file or created by the `create_placement_blockage` command. A relative placement group can be broken into pieces that straddle obstructions, yet maintain the relative placement structure.

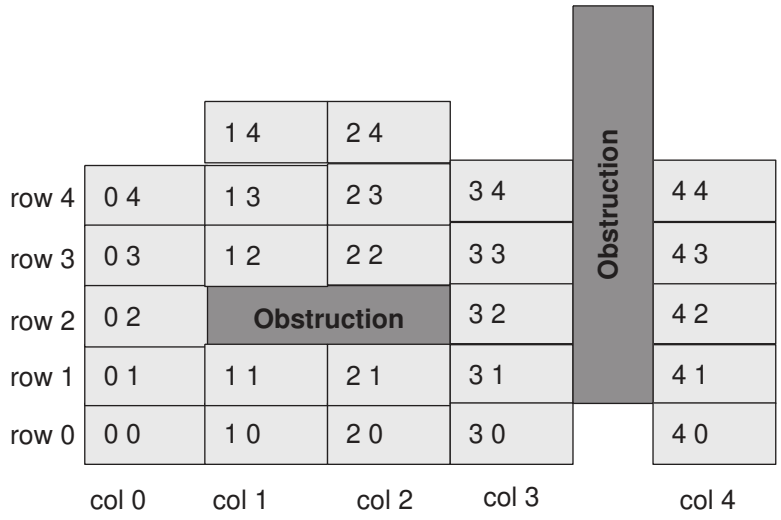
If the height of the obstruction is below a certain threshold, the relative placement cells are shifted vertically; otherwise, the relative placement column is shifted horizontally.

Figure 149 shows the placement of relative placement cells in a design containing obstructions that are either defined in the DEF file or created by `create_placement_blockage`. The obstruction in columns one and two is below the

Feedback

threshold, so the tool shifts the cells vertically. The obstruction in column four is greater than the threshold, so the tool shifts all the cells of the column horizontally.

Figure 149 Relative Placement in a Design Containing Obstructions



Legalizing Relative Placement Groups in a Placed Design

You can improve the placement of relative placement groups in a placed design by legalizing only the relative placement groups. To legalize the placement of only the relative placement groups, but not nonrelative placement cells, use the `legalize_rp_groups` command. You can also specify a list of relative placement groups to be legalized.

To perform a fast legalization of the relative placement groups, use the `-prototype` option. When you use this option, the tool does not ensure that all legalization constraints are met. Therefore, use it during the prototyping stages of the design flow when you are developing relative placement groups.

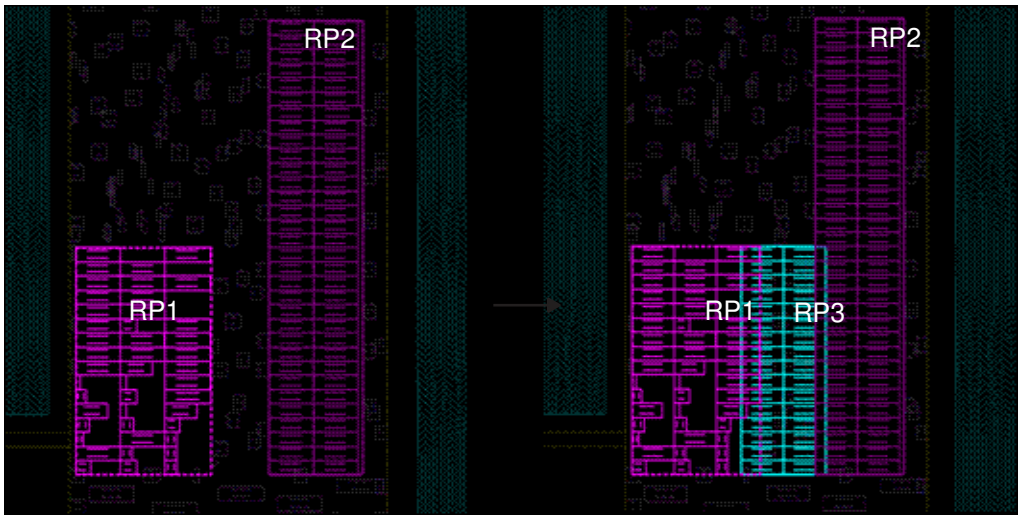
To specify that relative placement groups can overlap with each other, use the `-legalize_over_rp` option. For example, the following command legalizes the RP3 relative placement group over the RP1 and RP2 relative placement groups.

```
icc2_shell> legalize_rp_groups -legalize_over_rp RP3
```

Figure 150 shows the placement before and after running the command.

Feedback

Figure 150 Legalizing the RP3 Relative Placement Group Over Other Groups



After you legalize one or more relative placement groups by using the `legalize_rp_groups` command, there might be overlaps with cells in other relative placement groups or cells that are not in relative placement groups. Use the `check_legality` command to identify any cell overlaps and use the `legalize_placement` command to resolve any remaining cell overlaps.

Creating New Relative Placement Groups in a Placed Design

You can create a new relative placement group of cells in a placed design and place the relative placement groups incrementally by using the `place_opt -from final_place` command. If you select cells that are placed far apart in the initial placement for the same relative placement group, performing incremental relative placement might degrade the QoR.

The following example shows how to add a new relative placement group to a design that is already placed and optimized, and performs incremental placement and optimization:

```
icc2_shell> create_rp_group -name new_rp -columns 1 -rows 2
icc2_shell> add_to_rp_group new_rp -cells U1 -column 0 -row 0
icc2_shell> add_to_rp_group new_rp -cells U2 -column 0 -row 1
...
icc2_shell> place_opt -from final_place
```

For a placed design, if you create a new relative placement group and specify an anchor location by using the `-x_offset` and `-y_offset` options with the `set_rp_group_options` command, you can use the `legalize_rp_groups` command to legalizes the newly created group anchored by the specified x- and y-coordinates. Using the

Feedback

`legalize_rp_groups` command for incremental relative placement groups reduces the turnaround time.

Analyzing Relative Placement Groups

The following sections explain methods for analyzing your relative placement groups:

- Checking Relative Placement Groups Before Placement
- Analyzing the Placeability of a Relative Placement Group
- Reporting Relative Placement Constraint Violations
- Querying Relative Placement Groups
- Analyzing Relative Placement in the GUI

Checking Relative Placement Groups Before Placement

Before you run placement and optimization, you can check the relative placement constraints for issues that might lead to critical or noncritical failures after placement by using the `check_rp_constraints` command.

The following example checks for possible relative placement constraint violation in the group named rp_volt1:

```
icc2_shell> check_rp_constraints rp_volt1

 ************************************************************************
***
Report : Relative Placement Summary
Total number of specified top level relative placement groups: 1
Total number of relative placement groups which may not honor its
 constraints: 1
************************************************************************
RP Group: rp_volt1
------------------------------------------------------------------------
Warning: The height of relative placement group 'rp_volt1' is more than
 the height of voltage area or exclusive move bound. (RPGP-018)
```

Feedback

Analyzing the Placeability of a Relative Placement Group

Before you run placement and optimization, you can check if a specific relative placement group can be placed by using the `check_rp_constraints -analyze_placement` command.

- To limit the analysis to a specific region or a specific number of random sites of the core area, use the `-region` or `-trials` option, respectively.
- To ignore physical design constraints, advanced design rules, or relative placement constraints during placement analysis, use the `-no_pdc`, `-no_adv`, or `-no_rp_constraints` option, respectively.
- To report only the relative placement groups that do not meet a specific threshold, use the `-threshold` option. The tool reports a group only if the percentage of sites the group can be placed, relative to the total number of sites analyzed, is less than the specified threshold.

Reporting Relative Placement Constraint Violations

After you run placement and optimization, use the `report_rp_groups` command to identify placement issues and relative placement violations. You must either specify which relative placement groups to analyze or specify the `-all` option to analyze all relative placement groups.

By default, the command reports the following types of relative placement groups:

- Placed groups that do not have constraint violations
- Placed groups that have constraint violations that are not critical
- Failed groups that have constraint violations that are critical
- Groups that have not yet been placed

You can modify the default behavior by using the options described in Table 56.

Table 56 The report_rp_groups Command Options

To do this	Use this option
Report information only about the groups that are not placed due to critical relative placement violations	`-critical`
Report information only about the groups that are placed but do not meet their relative placement constraints	`-non_critical`
Report information only about the groups that have not yet been placed	`-unplaced`

Feedback

Table 56 The report_rp_groups Command Options (Continued)

To do this	Use this option
Report detailed information	`-verbose`

You can also run this command before you run placement and optimization and identify the unplaced groups in the design.

Querying Relative Placement Groups

To query relative placement groups that contain specific objects or attribute values, use the `get_rp_groups` command.

For example, the following command returns a collection consisting of all the relative placement groups:

```
icc2_shell> get_rp_groups
{RP_TA RP_TCO RP_HO_1 RP_HO_2 RP_HO_3 RP_HO_4 RP_THO}
```

The following command returns only the top-level relative placement groups:

```
icc2_shell> get_rp_groups -top
{RP_TA RP_TCO RP_THO}
```

The following command returns the relative placement group that contains the leaf cell named U129:

```
icc2_shell> get_rp_groups -of_objects U129
{RP_HO_4}
```

Analyzing Relative Placement in the GUI

The IC Compiler II GUI provides tools to help you visualize and analyze the relative placement groups in your design:

- Relative Placement (RP) Groups Visual Mode
- Relative Placement (RP) Net Connection Visual Mode

For more information about analyzing relative placement groups in the GUI, see the *Using Map and Visual Modes* topic in the *IC Compiler II Graphical User Interface User Guide.*

Feedback

Saving Relative Placement Information

The relative placement information is automatically saved in the design library database when you save the design by using the `save_lib` command.

You can also save the relative placement information to a file that contains Tcl commands that re-creates the relative placement groups, their objects, and their settings. To do so, use the `write_rp_groups -file_name` command. You must either specify which relative placement groups to write commands for or specify the `-all` option to write commands for all relative placement groups.

By default, the `write_rp_groups` command writes out commands for creating the specified relative placement groups and to add leaf cells, hierarchical groups, and blockages to these groups. The commands for generating subgroups within hierarchical groups are not written. You can modify the default behavior by using the options described in Table 57.

Table 57 The write_rp_groups Command Options

To do this	Use this option
Write all the relative placement groups within the hierarchy of the relative placement groups. If you omit this option, only the top-level group is written and subgroups are not.	`-hierarchical`
Write only `create_rp_group` commands to the script.	`-create`
Write only `add_to_rp_group -cells` commands to the script.	`-cell`
Write only `add_to_rp_group -rp_group` commands to the script.	`-rp_group`
Write only `add_to_rp_group -blockage` commands to the script.	`-blockage`

Summary of Relative Placement Commands

Table 58 shows some of the key commands used to perform relative placement.

Table 58 Relative Placement Commands

Command	Described in section
`create_rp_group` and `remove_rp_groups`	Creating Relative Placement Groups
`add_to_rp_group` and `remove_from_rp_group`	Adding Objects to a Group

Feedback

Table 58 Relative Placement Commands (Continued)

Command	Described in section
`set_rp_group_options` and `remove_rp_group_options`	Specifying Options for Relative Placement Groups
`modify_rp_groups`	Changing the Structures of Relative Placement Groups
`legalize_rp_groups`	Legalizing Relative Placement Groups in a Placed Design
`check_rp_constraints`	Checking Relative Placement Groups Before Placement
`report_rp_groups`	Reporting Relative Placement Constraint Violations
`get_rp_groups`	Querying Relative Placement Groups
`write_rp_groups`	Saving Relative Placement Information

10

Hierarchical Implementation

The following topics describe how to perform placement, optimization, clock tree synthesis, routing, and postroute optimization on hierarchical designs.

- Overview of Abstract Views
- Creating Abstract Views
- Reporting Abstract Inclusion Reasons
- Making Changes to a Block After Creating an Abstract
- Shrinking Abstract Views While Maintaining Timing
- Creating a Frame View
- Linking to Abstract Views at the Top-Level
- Changing the Block From Abstract to Design View
- Linking to Subblocks With Multiple Labels
- Specifying the Editability of Blocks From the Top-Level
- Preparing for Top-Level Closure With Abstracts
- Checking Designs With Abstracts for Top-Level-Closure Issues
- Performing Top-Level Closure With Abstract Views
- Creating ETMs and ETM Cell Libraries
- Linking to ETMs at the Top Level
- Performing Top-Level Closure With ETMs
- Transparent Hierarchy Optimization

Feedback

Overview of Abstract Views

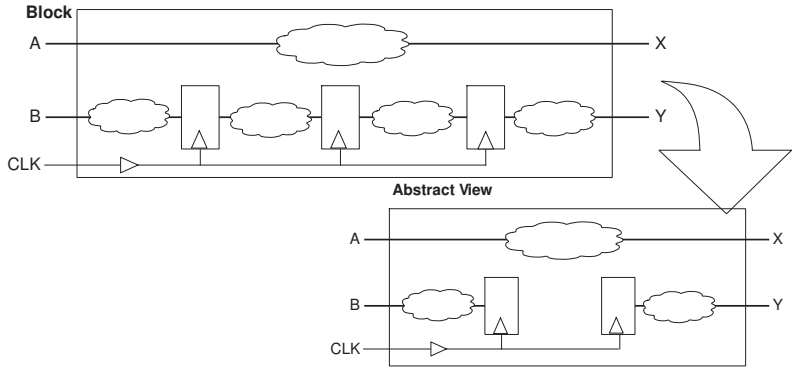
In an abstract view, the gate-level netlist for the block is modeled by a partial gate-level netlist that contains only the required interface logic of the block. All other logic is removed.

Figure 151 shows a block and its abstract view, where the logic is preserved between

- The input port and the first register of each timing path
- The last register of each timing path and the output port

Logic associated with pure combinational input-port-to-output-port timing paths (A to X) is also preserved. Clock connections to the preserved registers are kept as well. The register-to-register logic is discarded.

Figure 151 A Block and Its Abstract View



The interface logic of an abstract view consists of the following:

- All cells, pins, and nets in timing paths from input ports to registers or output ports
- All cells, pins, and nets in timing paths to output ports from registers or input ports
- Any logic in the connection from a master clock to generated clocks
- The clock trees that drive interface registers, including any logic in the clock tree
- The longest and shortest clock paths from the clock ports
- All pins with timing constraints that are part of the interface logic

Feedback

In addition to the interface logic, an abstract view contains the following information associated with the interface logic:

- Placement information
- Timing constraints
- Clock tree exceptions
- Parasitic information
- Transition and case values of dangling pins
- Power management cells and UPF constraints
- Nondefault routing rule association and minimum and maximum layer constraints, if any, for the nets that are retained in the abstract view

Creating Abstract Views

In the IC Compiler II tool, you can create an abstract view for a block at various stages of the design flow, such as, after , placement, optimization, clock tree synthesis, routing, and so on. Before you create the abstract view, ensure that the scenarios needed at the top level have been created and are active.

To create an abstract view for top-level closure, use the `create_abstract` command.

- To control the amount of timing information in the abstract view, use the `-timing_level` option with the following settings:
  - To create an abstract that contains only the boundary cells (one level of logic) connected to each boundary port, use the `boundary` setting.

    Such an abstract also contains feedthrough data paths, feedthrough combinational clock paths, and internal clock logic driving output ports. You can use this type of abstract to fix DRC violations at the top level.
  - To create a compact abstract that contains the timing information for only the critical setup and hold timings paths of the interface logic, use the `compact` setting. This is the default setting, and is preferred for top-level design closure.
  - To create an abstract with timing information for all the interface logic, use the `full_interface` setting. A `full_interface` abstract might be required for multiply-instantiated blocks (MIBs) when the constraints for MIB instances are different at the top-level.

Feedback

- To create the abstracts for lower-level blocks of the current block, use one of the following methods:
  - Create abstracts for specific lower-level blocks by using the `-blocks` option and specify the names of the blocks.
  - Create abstracts for all lower-level blocks by using the `-all_blocks` option.

  When you use the `-blocks` or `-all_blocks` option, if a specified block has an abstract view, by default, the tool does not re-create it. To force the tool to re-create the abstract, use the `-force_recreate` option.
- To include specific nets, ports, hierarchical pins, or leaf pins in a placement or timing abstract, use the `-include_objects` option.
- To create a read-only abstract, use the `-read_only` option.

  By default, the tool creates an editable abstract that you can modify in the context of a parent block. However, the tool makes the design view of the block read-only, preventing any changes from being made to the block.

  When you create the abstract view as read-only, the design view of the block is editable, and you can make changes to it.
- To flatten the physical hierarchy and retain only the relevant logic of the lower-level blocks, use the `-preserve_block_instances false` option.
- To specify if the abstract is going to be used for design planning or top-level implementation, use the `-target_use planning` or `-target_use implementation` option.

  Based on the intended usage, the tool applies appropriate internal settings to the abstract.
- To specify host options for distributed processing, use `-host_options` option and specify the appropriate settings. You can use this option to reduce the runtime when you create more than one abstract view by using the `-blocks` or `-all_blocks` option.

Feedback

Creating Abstracts With Power Information

To create an abstract that contains power information of the corresponding block, use the following steps at the block level:

1. Activate the scenarios for which you want to perform power analysis by using the `-active true` option of the `set_scenario_status` command and enable them for power analysis by using the following options:
   - `-dynamic_power true` for dynamic power analysis
   - `-leakage_power true` for leakage power analysis

   Power information is stored only for active scenarios that are enabled for dynamic or leakage power.
2. (Optional) Apply switching activity by either reading in a SAIF file with the `read_saif` command or annotating the switching activity information on the nets with the `set_switching_activity` command.

   For more information about applying switching activity, see Annotating the Switching Activity.
3. Store the power information in the abstract by setting the `abstract.annotate_power` application option to `true`.
4. Create the abstract for the block by using the `create_abstract` command.

When you create an abstract with power information, the tool stores the power information only for the logic that is removed.

When you instantiate such an abstract and perform power analysis at the top level, the tool

- Recomputes the power for the interface logic in the abstract, in context, based on the switching activity and loading seen at the top level
- Uses the stored power in the abstract for the logic that was removed when the abstract was created

To report the power information in an abstract instantiated at the top level, use the `report_power -blocks` command. If there are multiple levels of physical hierarchy, to specify the number of level for which you want to report, use the `-levels` option.

Creating Abstracts for Signal Electromigration Analysis

To create an abstract that can be used for signal electromigration analysis at the top-level, set the `abstract.enable_signal_em_analysis` application option to `true` before you run the `create_abstract` command.

Feedback

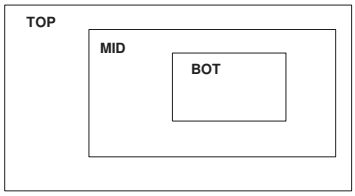
Handling Multiple Levels of Physical Hierarchy

For designs with multiple levels of physical hierarchy, before you create an abstract for a block with an abstract instantiated in it,

- Bind the lower-level blocks that you want represented by abstracts to the specific abstracts

For example, assume you have a design with multiple levels of physical hierarchy as shown in the following figure.

Figure 152 A Design With Multiple Levels of Physical Hierarchy



To create an abstract for the block named BOT, use the following commands:

```
icc2_shell> open_block BOT
icc2_shell> create_abstract
```

To create an abstract for the block named MID, use the following commands:

```
icc2_shell> open_block MID
icc2_shell> change_abstract -view abstract -references BOT
icc2_shell> create_abstract
```

By default, the `create_abstract` command preserves all levels of the physical hierarchy. To flatten the physical hierarchy and retain only the relevant logic of the lower-level blocks, use the `-preserve_block_instances false` option, as shown in the following example:

```
icc2_shell> open_block MID
icc2_shell> create_abstract -preserve_block_instances false
```

Use flattened abstracts in top-level implementation flows where the lower-level blocks are treated as read-only. Using flattened abstracts reduces the netlist size at the top level. It also simplifies data management because you do not need to include the lower-level blocks in the reference library list of the top level.

Creating Abstracts for Minimum Pulse Width Analysis

In the top-level timing abstracts, the Fusion Compiler tool enables you to check minimum pulse width (MPW) violations. The tool retains the clock sinks with MPW violations including their fan-in cells and nets at the time of abstract creation.

Feedback

You can use the following application options to:

- Check and retain registers with MPW violations including their full clock path, set the `abstract.keep_min_pulse_width_violation_pins` application option to `true`. The data paths to these registers are not retained. By default, this application is set to `false`.
- Limit the MPW violations within a given slack limit, set the `abstract.min_pulse_width_violation_slack_limit` application option to the required slack. This helps to preserve MPW pins that might not be violating but have a slightly positive slack.

Reporting Abstract Inclusion Reasons

In the IC Compiler II tool, the `create_abstract` command includes netlist objects such as the cells, nets, and pins in the abstract view at both the top-level and block-level. This command reports the reasons for the inclusion of netlist objects.

To report the reasons for including specific netlist objects in the abstract view, you can use the `report_abstract_inclusion_reason` command for objects across all hierarchies in the abstract.

For each reason reported, a reason code is displayed. These reason codes correspond to the valid reasons for inclusion in the abstract. There can be multiple reasons for the same netlist object.

Note:

The `report_abstract_inclusion_reason` command does not report reasons for the hierarchical cells or pins on hierarchical cells.

You can perform reason reporting from the top-level design, where the abstract is instantiated, without having the abstract as the current block.

The following example reports the reasons for abstract inclusion for the pins in abstract view:

```
> report_abstract_inclusion_reason [get_pins -of_object [get_cells
 -filter "is_hierarchical==false" -hierarchical *]]

Legend
mv - MV Logic
comp - Compact interface logic
cell - Cell inclusion
------------------------------------------------------------------------
Pin name                              Reason(s) for inclusion
------------------------------------------------------------------------
AINV_P_260/I                          comp, cell
AINV_P_260/VDD                        cell
```

Feedback

```
AINV_P_260/VSS                    cell
AINV_P_260/ZN                     comp, cell
AINV_P_262/I                      comp, cell
AINV_P_262/VDD                    cell
AINV_P_262/VSS                    cell
AINV_P_262/ZN                     comp, cell
AINV_P_263/I                      mv, comp, cell
```

A legend is displayed at the beginning of the report for the queried objects, specifying the detailed reason for each reason code.

Making Changes to a Block After Creating an Abstract

When you create an abstract view for a block with the `create_abstract` command, by default, the tool saves both the abstract and design views of the block and changes the design view to read-only.

You can open a read-only block and make changes to it in-memory, but you cannot save these changes by using the `save_block` or `save_lib` command. To save the change you make to a read-only block, use one of the following methods:

- Save it as a different block by using the `save_block -as` command.
- Re-create an abstract for the block by using the `create_abstract` command. The tool then automatically saves the design view.
- Remove the abstract by using the `remove_abstract` command, if you no longer need the abstract. This changes the design view from being read-only to editable, and you can save the block by using the `save_block` or `save_lib` command.

If you create a read-only abstract by using the `create_abstract -read_only` command, the tool does not make the design view of the block read-only and you can save any subsequent changes to the block by using the `save_block` or `save_lib` command, as shown in the following example:

```
icc2_shell> create_abstract -read_only
icc2_shell> change_link [get_cells U25] AND2
icc2_shell> save_block
```

Shrinking Abstract Views While Maintaining Timing

You can incrementally update the physical information of the abstract from the top level, without changing the associated timing information inside the abstract. With this feature, the Top Engineer can continue the place and route flow after shrinking the block abstract, while the block owner reimplements the subblock with a modified floorplan.

Feedback

After shrinking an abstract view, for example, using the GUI, use the `update_block_views` command to update the physical information of the abstract view as well as the frame view.

Note:

Use the `-verbose` option to output to the log file detailed information about the changes.

If there is an existing frame view, the frame view is re-created using the same value for `-block_all` that was used to create the existing frame view. If there is no existing frame view, `-block_all true` is used for the frame view.

After block shrinking, because of pin-snapping (of blocks), the length of the top segment of the boundary nets changes, which impacts parasitics and therefore the timing of these net segments.

The procedure for using the `update_block_views` command is as follows:

1. To allow updated views of the subblock to be saved, open the reference library in edit mode:

   ```
   icc2_shell> open_lib top.nlib -ref_libs_for_edit
   ```

   Note:

   If using `-ref_libs_for_edit` is not allowed, copy the subblock to the top design library.

2. To prevent the changes made to the abstract from being merged to the design view, create abstracts using `create_abstract -read_only`, open `top.design` linked to read-only abstracts, then enable editing of the subblock:

   ```
   icc2_shell> set_editability -blocks <> -value true
   ```

3. After shrinking the subblock (for example, using the GUI), use the `update_block_views` command to update the physical information of the abstract and frame views, then disable editing of the subblock:

   ```
   icc2_shell> update_block_views -verbose
   icc2_shell> set_editability -blocks <> -value false
   ```

The `update_block_views` commands handles different design objects as follows:

- Standard cells: Moved inside the resized block boundary.
- Macros and child subblock instances: Moved inside the resized block boundary and may overlap with existing objects in the block. If the block has been reshaped such that the aspect ratio of an object cannot fit, then the tool issues an error.
- Pins: Snapped to the new boundary.

- Global Route (GR) congestion map: Removed from the abstract view.
- Voltage area and routing blockages: Trimmed and adjusted inside the resized block boundary.
- Placement blockages, bounds, edit groups, routes, floating shapes, g-links, and routing corridors: Removed if they are partially or completely outside the resized block boundary.
- Site rows: Any portion present outside the resized block boundary is cut.

Recreating the Power and Ground (PG) Structure

After shrinking an abstract view and using the `update_block_views` command, the top-level power and ground (PG) structure becomes obsolete, so you must re-create it as follows:

1. Open `top.design` linked to read-only abstracts, then remove the top PG structure and block PG pins:

```
icc2_shell> remove_shapes [get_shapes -filter "net_type==power"]
icc2_shell> remove_shapes [get_shapes -filter "net_type==ground"]
icc2_shell> remove_vias [get_vias -filter  "net_type==power"]
icc2_shell> remove_vias [get_vias -filter  "net_type==ground"]
icc2_shell> remove_terminals [get_terminals -of_objects [get_pins
 block_name/VSS ]]

icc2_shell> remove_terminals [get_terminals -of_objects [get_pins
 block_name/VDD]]
```

2. Shrink the subblocks, then use the `update_block_views` command:

```
icc2_shell> update_block_views -verbose
```

3. Ensure that the top level PG strategies are defined:

```
icc2_shell> compile_pg {< Top PG strategies >}
```

Creating a Frame View

To perform top-level routing, including virtual routing, every abstract view must have a corresponding frame view. To create a frame view, use the `create_frame` command, which extracts the blockage, pin, and via information from the design view.

After you create the frame view, save the design library by using the `save_lib` command.

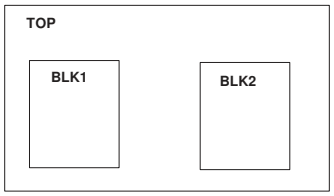
Feedback

Linking to Abstract Views at the Top-Level

For a top-level design to link to an abstract view of a lower-level block, the design library containing the abstract view of the lower-level block must be one of the reference libraries of the top level.

For example, assume you have a top-level design named TOP with two lower-level blocks named BLK1 and BLK2, as shown in the following figure.

Figure 153 Top-Level Design With Instantiated Blocks



To link to the abstracts of the BLK1 and BLK2 blocks, the design libraries containing the abstract views of these block must be reference libraries of the TOP design. If not, you can add it to the reference libraries of the TOP design by using the `set_ref_libs -add` command as shown in the following example:

```
current_block TOP
set_ref_libs -add ../BLK1/BLK1.nib
set_ref_libs -add ../BLK2/BLK2.nib
```

To report the reference libraries for the current design, use the `report_ref_libs` command.

When you link a top-level design, if an abstract view for a lower-level block is available, the tool links to that abstract view by default. The default precedence of the different views used when linking lower-level blocks is as follows:

1. Abstract view
2. Design view
3. Frame view
4. Outline view

Changing the Block From Abstract to Design View

To change a block from its abstract to its design view, use the `change_abstract` command and specify the view you are changing to by using the `-view` option. When you

Feedback

change a block from its abstract view to its design view, remove the existing constraints and apply full-chip timing constraints.

The following example changes the BLK1 block from its abstract view to its design view and applies full-chip timing constraints.

1. Remove any existing timing constraints:

```
icc2_shell> remove_scenarios -all
icc2_shell> remove_modes -all
icc2_shell> remove_corners -all
```

2. Use the `change_abstract` command to change from the abstract view to the design view:

```
icc2_shell> change_abstract -view design -references BLK1
```

3. Apply the full-chip scenario creation script:

```
icc2_shell> source full_chip_scenario_creation.tcl
```

If the abstract view for a block has changed in its design library, you can reload the new abstract view by using the `change_abstract -reload` command.

To report the abstract views the design is linking to, use the `report_abstracts` command.

More information about specifying reference libraries and linking to abstract views at the top level, see SolvNet article 2436119, Setting Up Designs for Hierarchical Place and Route.

Linking to Subblocks With Multiple Labels

When saving a block at different stages of an implementation flow, you can use the `-as` *`blockName/labelName`* or `-label` *`labelName`* option with the `save_block` command to specify a unique label for each version of the block you save.

By default, during linking, the top-level block links to lower-level blocks with the same label. If a subblock has been saved with multiple labels, you can specify which label the top level should link to by using the `set_label_switch_list` command. The command specifies a precedence-ordered list of labels to use during linking. By default, the command applies to all subblocks; to specify a different block or blocks, use the `-reference` option.

The following example specifies that the top level should link to the preroute label of the BLOCK1 and BLOCK2 subblocks:

```
icc2_shell> set_label_switch_list \
    -reference {BLOCK1 BLOCK2} preroute
```

Feedback

The following example specifies that the postroute label has a higher priority than the preroute label when linking any subblock at the top level.

```
icc2_shell> set_label_switch_list {postroute preroute}
```

You can use the `set_label_switch_list` command again to update the block's label switch list and then relink the block using the `link_block -rebind` command. Note the following when relinking a block:

- If none of the labels in the switch list are available in a subblock, or if you specify an empty switch list, the tool links the subblock to the label it was previously linked to
- If you have removed all the labels for a subblock, the tool links the subblock to the label of its parent block

Specifying the Editability of Blocks From the Top-Level

For designs with physical hierarchy, you can specify if changes can be made to lower-level blocks by using the `set_editability` command at the top level.

You can change the editability of

- Specific blocks by using the `-blocks` option
- All blocks starting from a specific level of the physical hierarchy by using the `-from_level` option
- All blocks up to a specific level of the physical hierarchy by using the `-to_level` option

For top-level implementation flows with read-only abstracts, after you link the design, explicitly set the lower-level blocks as read-only by using the following command:

```
icc2_shell> set_editability -blocks [get_blocks -hierarchical] \
   -value false
```

By default, if you change a library or block name with one of the following commands, the tool does not propagate the editability settings from the old reference to the new reference:

- `change_abstract -lib`
- `link_block -rebind -force`
- `set_reference`

To retain the editability settings, set the `design.preserve_reference_editability` application option to `true` before you change a library or block name.

When you save the hierarchical design with the `save_block` command, the editability settings are saved.

Feedback

Preparing for Top-Level Closure With Abstracts

Before you can perform , placement, optimization, clock tree synthesis, and routing at the top level, you must perform the following tasks:

- Apply the top-level-timing constraints and settings.

  You can split the chip-level constraints in to separate top- and block-level constraints by using the `split_constraints` command.

- Review and specify the relationship between the top- and block-level modes, corners, and clocks by using the `set_block_to_top_map` command.

- If the pre-clock tree implemented blocks, which have undergone CCD optimization, are used for top-level placement and optimization, then the ideal clock latencies adjusted dynamically by different engines such as CCD and clock-gate estimation on the clock pins of registers or integrated clock gating (ICG) cells inside the blocks might not be seen at the top-level leading to timing inaccuracies at the top-level. Promote these clock latency constraints from block-level to the top-level by using the `promote_clock_data -latency_offset -auto_clock connected` command.

- Apply the top-level clock tree synthesis settings and exceptions.

  - If the top-level exceptions do not include the balance points on the pins within lower-level blocks, promote the balance points from the lower levels by using the `promote_clock_data -auto_clock connected -balance_points` command.

  - If concurrent clock and data (CCD) optimization is performed at the block level, the tool derives median clock latencies for the block-level clock ports, which accounts for the latency adjustments that were derived for the block-level registers during concurrent clock and data optimization. Promote these block-level clock latencies as clock balance point delays for the block-level clock pins by using the `promote_clock_data -auto_clock connected -port_latency` command.

  - If the lower-level blocks contain clock-meshes, promote the mesh annotations (annotated transitions and delays) by using the `promote_clock_data -mesh_annotations` command.

- If you need to promote clock data for a specific top-level clock, use the `promote_clock_data -clocks {top_clock} -port_latency` command. The command also promotes clock-independent exceptions. If any of the given top-level clocks has no mapping to any block-level clock in any mode, the tool issues a warning.

- If there are any active modes, corners, or clocks at the top-level, which cannot be mapped to corresponding modes, corners, or clocks in the block-level, you can still promote the clock data for other mapped modes, corners, or clocks using the `promote_clock_data -balance_points -force` command. This command issues errors for the unmapped modes, or corners, or clocks.

Feedback

- Apply the top-level UPF constraints, which you can create from the full-chip UPF constraints by using the `split_constraints` command.

  The tool promotes the required UPF constraints from the lower-level blocks to the top level.

Checking Designs With Abstracts for Top-Level-Closure Issues

You can check a hierarchical design that contains abstracts for possible issues such as the application option consistency issues and the design issues related to top-level closure by using the `check_hier_design` command. Identifying issues and fixing them before you perform top-level closure can help reduce turnaround time.

When you use the `check_hier_design` command, you can specify

- The references to check by using the `-reference` option.

  If you do not specify this option, the command checks all the references in the physical hierarchy.

- The following type of checks to perform by using the `-stage` option:
  - Use the `-stage timing` option to perform timing related checks.

    If you do not specify the `-stage` option, by default, the tool performs timing related checks.
  - Use the `-stage pre_placement` option to perform both timing and preplacement related checks.
  - Use the `-stage pre_compile_tho` option to perform both preplacement and transparent hierarchy optimization (THO) related checks and reports the feasibility of compile THO (`final_opto` stage) implementation in the design.
  - Use the `-stage pre_route_tho` option to perform both preroute and THO related checks and reports the feasibility of preroute THO implementation in the design.

The `check_hier_design` command can check the consistency between the top-level constraints and the constraints of every lower-level instance linked to an abstract. To enable this feature, set the `abstract.check_constraints_consistency` application option to `true` before you run the `check_hier_design` command.

The tool saves the settings of the predetermined list of timer application options during the `create_abstract` command. At the top-level design, the `check_hier_design` command compares these timer application option settings with those of the block-level design.

Feedback

The following application options are automatically saved during the `create_abstract` command and verified by the `check_hier_design` command:

- `time.case_analysis_propagate_through_icg`
- `time.case_analysis_sequential_propagation`
- `time.clock_gating_propagate_enable`
- `time.clock_gating_user_setting_only`
- `time.clock_marking`
- `time.clock_reconvergence_pessimism`
- `time.create_clock_no_input_delay`
- `time.crpr_remove_clock_to_data_crp`
- `time.delay_calc_waveform_analysis_mode`
- `time.delay_calculation_style`
- `time.disable_case_analysis_ti_hi_lo`
- `time.disable_clock_gating_checks`
- `time.disable_cond_default_arcs`
- `time.disable_internal_inout_net_arcs`
- `time.disable_recovery_removal_checks`
- `time.edge_specific_source_latency`
- `time.enable_auto_mux_clock_exclusivity`
- `time.enable_ccs_rcv_cap`
- `time.enable_clock_propagation_through_preset_clear`
- `time.enable_clock_propagation_through_three_state_enable_pins`
- `time.enable_clock_to_data_analysis`
- `time.enable_non_sequential_checks`
- `time.enable_preset_clear_arcs`
- `time.enable_si_timing_windows`
- `time.gclock_source_network_num_master_registers`
- `time.special_path_group_precedence`

Feedback

- `time.use_lib_cell_generated_clock_name`
- `time.use_special_default_path_groups`

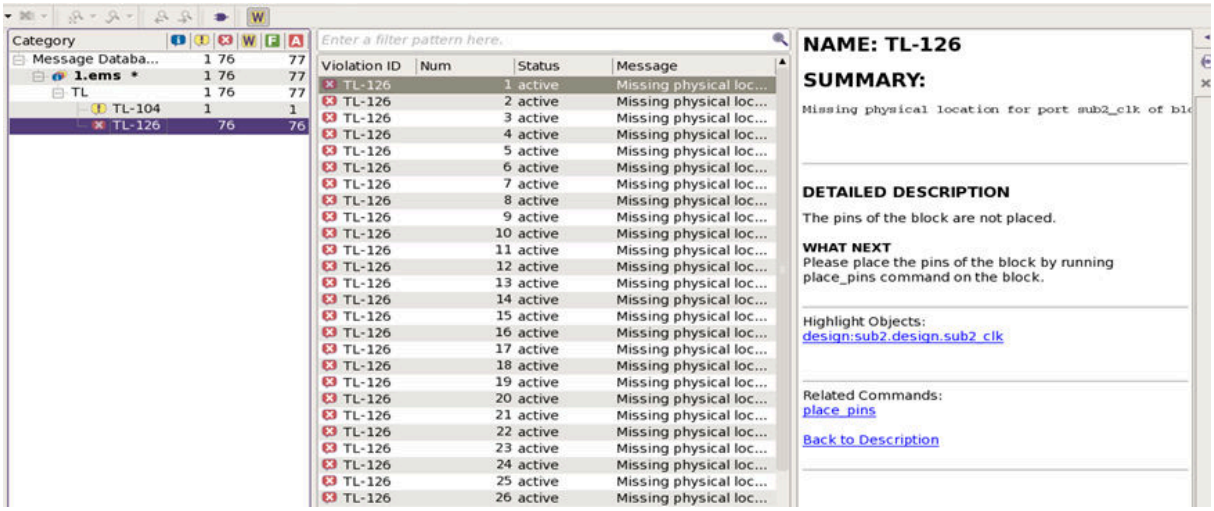
For more information about the issues the `check_hier_design` command identifies and how to fix them, see the man page for the corresponding message ID.

In addition to generating a report, the `check_hier_design` command generates an enhanced messaging system (EMS) database that you can view by using the message browser in the IC Compiler II GUI. Create the EMS database before you run the `check_hier_design` command, as shown in the following example:

```
icc2_shell> create_ems_database check_hier.ems
icc2_shell> check_hier_design -stage timing
icc2_shell> save_ems_database
```

In the IC Compiler II GUI message browser, you can sort, filter, and link the messages to the corresponding man page, as shown in the following figure.

Figure 154 Viewing the EMS Database in the Message Browser



You can also output the information in the EMS database in ASCII format by using the `report_ems_database` command.

The checks performed by the `check_hier_design` command are also available in the `check_design` command.

Feedback

You can perform

- Timing checks specific to top-level closure by using the `check_design -checks hier_timing` command
- All timing checks, including those specific to top-level closure, by using the `check_design -checks timing` command
- Preplacement checks specific to top-level closure by using the `check_design -checks hier_preplacement` command
- All preplacement checks, including those specific to top-level closure, by using the `check_design -checks preplacement` command

You can generate an EMS database for the `check_design` command and view it in the GUI, similar to the `check_hier_design` command.

Handling Design Data Using the Early Data Check Manager

While you identify the issues related to top-level closure and fix them, the tool enables you to explore the top-level flow in the presence of design data violations. The hierarchical checks can detect different types of mismatched, incomplete, and inconsistent data in designs. You can configure the following policies for the hierarchical checks to manage the design data violations:

- Error: When you apply this policy, the tool does not mitigate the violation, but issues an error message.
- Tolerate: When you apply this policy, the tool makes predictable and explainable assumptions to continue with the flow. No changes are made to the design or setup.
- Repair: When you apply this policy, the tool mitigates the violation using one or more repair strategies and records it. Repair can include changes in design and setup.
- Strict: When you apply this policy, depending on the policy supported by the check, the tool applies the policies in this order – error, tolerate, repair.
- Lenient: When you apply this policy, depending on the policy supported by the check, the tool applies the policies in this order – repair, tolerate, error.

Note:

- Quality of results of the top-level flow might get impacted due to the toleration and repair of errors. Error toleration and repair is mainly provided for managing data violations during the early design exploration phases. In the implementation flow, you should use the strict policy for all checks.

Feedback

Prerequisites for Handling Early Design Data

The tool performs the repairs and saves the records in the subblocks within the `check_hier_design` command itself. For the repairs, which need frame recreation, you need to run the `save_lib` command to save the frame on disk.

- To save the repair and its record to the subblock successfully, the following setup is required:

  1. Enable the reference libraries, containing the subblocks, for edit

     ```
     open_lib top.nlib -ref_libs_for_edit
     ```

  2. Enable editability on the subblocks, on which repair has to be performed

     ```
     set_editability -blocks <> -value true
     ```

The `check_hier_design` command checks for the required editability settings, if repair has to be performed. If the required editability settings are not performed,then the `check_hier_design` command issues `TL-170` error message.

Note:

The tool supports the repairs on subblocks, which are linked to the `read_only` abstracts rather than editable abstracts because managing the mismatched design data mechanism only aims to continue the top-level flow for exploration and as such no repair work performed on the subblock abstracts should get merged with the actual design view. If the top-level uses editable design views, the repair work still continues.

Early Data Checks, Policies, and Strategies

The tool can identify issues and violations in data during the early stages of design. You can set policies and strategy configurations for the predefined checks to allow, tolerate, or repair data. Table 59 describes the predefined checks for top-level closure.

Table 59 Top-Level Closure Checks, Policies, and Strategies

Check	Description	Strategy	Supported Policies	Supported References
`hier.block.missing_frame_view`	Checks whether the frame view is missing for a hierarchical block	Re-create frame views from the bound view	`error`, `tolerate`, `repair`	`MID`, `BOT`

Feedback

Table 59 Top-Level Closure Checks, Policies, and Strategies (Continued)

Check	Description	Strategy	Supported Policies	Supported References
hier.block.reference_missing_port_location	Checks whether the location of a physical hierarchy boundary pin is missing	Assign a location for the block pin on the block boundary based on connectivity	error, tolerate, repair	MID, BOT
hier.block.reference_port_outside_boundary	Checks whether the location of a physical hierarchy boundary pin is outside the physical hierarchy boundary	Reassign a location for the block pin on the block boundary based on connectivity	error, tolerate, repair	MID, BOT
hier.block.reference_missing_port	Checks whether a port is missing in the physical hierarchy reference	Create missing ports in the reference block and update its location	error, tolerate, repair	MID, BOT
hier.block.instance_bound_to_frame	Checks whether a physical hierarchy instance is bound to a frame view		error, tolerate	TOP
hier.block.instance_with_design_type_macro	Checks whether a physical hierarchy instance is bound to a macro		error, tolerate	TOP
hier.top.estimated_corner	Checks whether the estimated corner is at top level		error, tolerate	TOP
hier.block.missing_leaf_cell_location	Checks whether the location of a leaf cell of a physical hierarchy is missing	Assign an approximate location inside the block boundary for the leaf cell (legality not considered)	error, tolerate, repair	MID, BOT

Feedback

Table 59 Top-Level Closure Checks, Policies, and Strategies (Continued)

Check	Description	Strategy	Supported Policies	Supported References
`hier.block.leaf_cell_outside_boundary`	Checks whether the location of a leaf cell of a physical hierarchy is outside the physical hierarchy boundary	Reassign the leaf cell location to the nearest location inside the block boundary (legality not considered)	`error`, `tolerate`, `repair`	`MID`, `BOT`
`hier.block.missing_child_block_instance_location`	Checks whether the location of a child physical hierarchy instance is missing in the specified physical hierarchy	Assign an approximate location for the lower-level subblock based on connectivity inside the block boundary (overlap removal not considered)	`error`, `tolerate`, `repair`	`MID`
`hier.block.child_block_instance_outside_boundary`	Checks whether the location of a child physical hierarchy instance is outside the specified physical hierarchy boundary	Move the subblock to the nearest location inside the block boundary (overlap removal not considered).	`error`, `tolerate`, `repair`	`MID`
`hier.block.port_mismatch_between_views`	Checks whether there is a mismatch of ports between the views of a physical hierarchy	Bound view is treated as golden and the frame view is recreated to match the bound view	`error`, `tolerate`, `repair`	`MID`, `BOT`
`hier.block.missing_design_view_for_abs`	Checks whether the design view for an abstract is missing		`error, tolerate`	`MID`, `BOT`

Feedback

Table 59 Top-Level Closure Checks, Policies, and Strategies (Continued)

Check	Description	Strategy	Supported Policies	Supported References
hier.block.instance_unlinked	Checks whether the physical hierarchy reference is not linked properly to the top-level design due to a view change from frame or ETM to abstract or design		error	TOP
hier.block.abstract_type_non_timing	Checks whether the abstract type is not timing		error	MID, BOT
hier.block.abstract_target_use_non_implementation	Checks whether the target use of an abstract is not implementation		error	MID, BOT
hier.block.missing_core_area	Checks whether the core area for a physical hierarchy is missing		error	MID, BOT
hier.block.unmapped_logic	Checks whether unmapped logic is present in the physical hierarchy		error	MID, BOT

Where

- BOT signifies bottom level design.
- MID signifies intermediate level design, which instantiates BOT.
- TOP signifies the top-level design, which instantiates MID.

Feedback

Setting the Policy for Early Data Checks

To set or modify the policy for all the checks, or modify the policy settings for data checks, use the `set_early_data_check_policy` command. Use the following options to configure the policy:

- `-checks`: Specifies a predefined list of data checks. It supports an asterisk wildcard character (*) as the name of a check.
- `-policy`: Specifies the type of policy depending on what is supported by the check. This can be
  - `error`: The tool issues an error message if the check does not support `error` as a policy.
  - `tolerate`: The tool issues an error message if the check does not support `tolerate` as a policy.
  - `repair`: The tool issues an error message if the check does not support `repair` as a policy.
  - `strict`: The tool sets the first matching policy in the following order:

    1. `error`
    2. `tolerate`
    3. `repair`

    For example, when a check supports the `error` and `tolerate` policies, and you select `strict` as the policy type, the `error` policy type is set.
  - `lenient`: The tool sets the first matching policy in the following order:

    1. `repair`
    2. `tolerate`
    3. `error`

    For example, when a check supports the `error` and `tolerate` policies, and you select `lenient` as the policy type, the `tolerate` policy type is set.

Note:

If you specify both the checks and policies, the configuration supports all the policies described in this section, depending on what the application-specific check supports. However, if you specify only the policies, but not the checks, the configuration supports only the `strict` and `lenient` policies.

Feedback

- You can also use the `if_not_exists` keyword with the `-policy` option to set a policy on the current design only if the design does not already have a policy set.
- `-strategy`: Specifies the strategy to apply for a check.

  Note:

  `-strategy` is not applicable for the hierarchical checks.
- `-references`: Specifies the policy and strategy for reference subdesigns.

Reporting Early Data Check Records

You can report the policy set for all checks or for a specific set of checks by using the `report_early_data_checks` command. Use the following options to configure the report:

- `-policy`: Reports configuration details for all checks. It shows the selected policy and strategy for each check. Using the `-hierarchical` option with the `-policy` option reports the configuration of the subblocks.
- `-checks`: Reports the current policy, supported policies, strategies, and help text for each check. It supports an asterisk wildcard character (*) as the name of a check. Using the `-hierarchical` option with the `-checks` option reports the checks of the subblocks.
- `-verbose`: Reports a summary of all failed checks, policies, strategies applied, and checked objects with comments. Using the `-hierarchical` option with the `-verbose` option reports the records of all failed checks, policies, strategies applied, and checked objects from the subblocks.
- `-hierarchical`: Traverses the hierarchy and reports the check, policy, strategy, and fail count for each design in the hierarchy.

In the following example, the `report_early_data_checks -hierarchical -policy` command reports the policy settings for the hierarchical checks set on the top-level design and the block-level designs:

```
icc2_shell> report_early_data_checks -policy
****************************************
Report : report_early_data_checks
Design : top
Version: R-2020.09
Date   : Thu Aug 20 23:53:54 2020
****************************************
Design                    Check                                                Policy
 Strategy
--------------------------------------------------------------------------------
-------
unit_des.nlib:top.design hier.block.instance_bound_to_frame                     error

unit_des.nlib:top.design hier.block.instance_unlinked                           error
```

```
     unit_des.nlib:top.design hier.block.instance_with_design_type_macro        error

     unit_des.nlib:top.design hier.top.estimated_corner                         error
     ------------------------------------------------------------------------------
     ------
     blk                      hier.block.abstract_missing_design_view           error

     blk                      hier.block.abstract_target_use_non_implementation error

     blk                      hier.block.abstract_type_non_timing               error

     blk                      hier.block.leaf_cell_outside_boundary             error
     ------------------------------------------------------------------------------
     -----
```

In the following example, the `report_early_data_checks -hierarchical -verbose` command reports the repair performed on the checked object and also provides the details of policies, strategies applied, and checked objects on the subblocks.

```
icc2_shell> report_early_data_checks -hierarchical -verbose
**************************************************
Design Check                                 Policy Strategy Checked  Comment
                                                             Object
------------------------------------------------------------------------------------
blk  hier.block.missing_leaf_cell_location repair          eco_cell Instance location
                                                                    updated to
                                                                    X:150145 Y:69620
------------------------------------------------------------------------------------
```

While running the top-level early flow, you might run into top-level errors or warnings because of the design data violations in the subblocks or top-level design. To use the data checks and policies capability for exploring early flows, you need to know the check name corresponding to the top-level errors seen on the design. The `report_hier_check_description` command provides the required information about the checks associated with each top-level error. The following example shows the report generated by the `report_hier_check_description` command:

```
icc2_shell> report_hier_check_description
****************************************
Report : Hier check description
Design : top
Version: R-2020.09
Date   : Fri Aug 21 00:08:32 2020
****************************************

Legend
    E - error
    R - repair
    T - tolerate
    TOP - Top-Design
    BLK - Respective Block-Ref

------------------------------------------------------------------------------------
---------------------------
Check                                    Error  Tolerate Repair Allowed  Allowed
 Description
                                           ID      ID      ID    Policies References
------------------------------------------------------------------------------------
---------------------------
```

Feedback

```
hier.block.missing_frame_view              TL-101  TL-401  TL-501  E|R|T     BLK
 Missing frame view
hier.block.abstract_missing_design_view    TL-101  TL-401  N/A     E|T       BLK
 Missing design view for abstract
hier.block.reference_missing_port_location TL-126  TL-426  TL-526  E|R|T     BLK
 Missing location of physical

 hierarchy boundary pin
hier.block.reference_port_outside_boundary TL-127  TL-427  TL-527  E|R|T     BLK
 Location of physical hierarchy boundary

 pin outside physical hierarchy boundary
```

Generating a Report of Early Data Check Records

You can generate a report of check records from a design by using the `get_early_data_check_records` command. Using the `-hierarchical` option with this command enables to you get check records for all subblocks.

The report generated is in the following format:

check_name@object

For example,

```
icc2_shell> get_early_data_check_records -hierarchical
{hier.block.missing_leaf_cell_location@eco_cell}
```

You can use the `-filter` option to filter the results of this command by various parameters such as object class and check name.

For example,

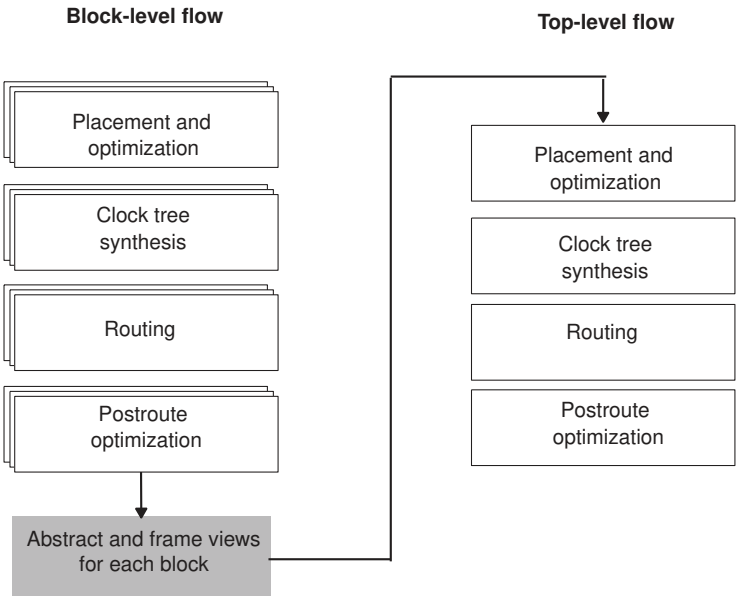
```
icc2_shell> get_early_data_check_records \
-filter "checked_object_class==port"
{place.port_type_mismatch@clk route.missing_layer_direction@feed_in
 route.layer_name_mismatch@out}
```

Feedback

Performing Top-Level Closure With Abstract Views

You can perform top-level closure by implementing the blocks first, and then implementing the top level, as shown in the following flow diagram:

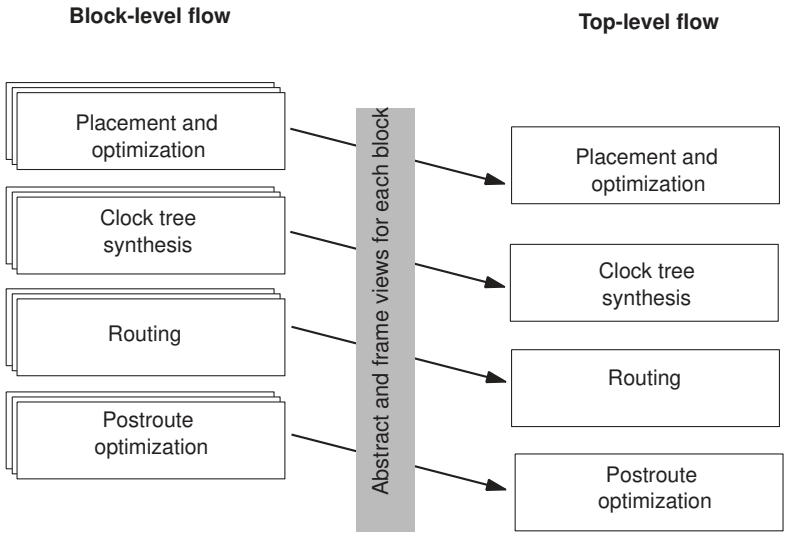
Figure 155 Top-Level Implemented After Blocks are Completed



Feedback

Alternatively, you can implement the blocks and top-level in parallel, as shown in the following flow diagram:

Figure 156 Top and Block Levels Implemented in Parallel



When using abstracts at the top-level, you can perform top-level synthesis, placement, optimization, clock tree synthesis, routing, and postroute optimization using the commands supported at the block level. Currently the tool does not make changes within the abstracts during top-level closure. Therefore, you can create and use read-only abstracts.

Creating ETMs and ETM Cell Libraries

To create ETMs and ETM cell libraries in the IC Compiler II tool, see the Creating ETMs and ETM Cell Libraries in the IC Compiler II Tool topic.

To create ETMs and ETM cell libraries in the PrimeTime and the Library Manager tools respectively, see the following topics:

- Creating ETMs in the PrimeTime Tool
- Creating ETM Cell Libraries in the Library Manager Tool

Creating ETMs and ETM Cell Libraries in the IC Compiler II Tool

Rather than creating an ETM for each mode and corner combination using the `extract_model` command in the PrimeTime tool and then using the Library Manager tool

Feedback

to combine the ETMs with the corresponding physical information to create a cell library, you can perform both steps together by using the `extract_model` command directly in the IC Compiler II tool.

The following IC Compiler II script creates an ETM for every mode and corner of the design and then combines them with the corresponding physical information and creates the corresponding cell library:

```
# Open the design, create a frame view, and set the PrimeTime options
open block.nlib:block.design
create_frame <options>        ;# extract_model needs Frame for library
 preparation
set_pt_options -pt_exec_path <> -work_dir ETM_work_dir \
               -post_link_script <tcl script with extract_model*
 variables> \

# To use StarRC for Parasitic extraction
set_app_options -name extract.starrc_mode -value true
set_starrc_options -config <starrc_config_file>

# Create the ETM and generate the ETM cell library
extract_model
```

By default, the IC Compiler II tool writes out the generated ETM library into

- `[pwd]/ETM_Lib_work_dir/block_name/label_name/` for labelled blocks
- `[pwd]/ETM_Lib_work_dir/block_name/` for non-labelled blocks

Creating ETMs in the PrimeTime Tool

You can create an ETM for design by using the `extract_model` command in the PrimeTime tool. For multicorner-multimode designs, you must create an ETM for each scenario by applying the appropriate corner and mode constraints for each scenario.

The following PrimeTime script creates an ETM for the S3 scenario, which consists of the m1 mode and c3 corner, of the AMS_BLK design:

```
#Read in the design
read_verilog ./AMS_BLK.v
link

# Apply parasitics
read_parasitics ./AMS_BLK.spef

#For multivoltage designs, apply UPF data and settings
load_upf ./AMD_BLK.upf
set extract_model_include_upf_data true

# Apply the mode (m1) and corner (c3) constraints for the scenario (S3)
source m1_constraints.tcl
```

Feedback

```
source c3_constraints.tcl

# Enable clock latencies for designs with synthesized clock trees
set extract_model_with_clock_latency_arcs true
set extract_model_clock_latency_arcs_include_all_registers false

# Create the ETM
extract_model -library_cell -format db -output AMS_BLK_m1_c3
```

For more information about creating ETMs in the PrimeTime tool, see the Extracted Timing Models chapter in the *PrimeTime User Guide*.

Feedback

Creating ETM Cell Libraries in the Library Manager Tool

After you create an ETM for each mode and corner combination, use the Library Manager tool to combine the ETMs with the corresponding physical information and create a cell library. For more information, see the *Library Manager User Guide*.

The following Library Manager scripts combine the ETMs created for every mode and corner of the AMS_BLK design with the corresponding physical information and creates the corresponding cell library:

```
# Create a library work space
create_workspace -flow etm_moded AMS_BLK

# Read the physical data (frame view)
read_ndm -views frame AMS_BLK.ndm

# Read the ETMs for every scenario
read_db -mode_label m1 AMS_BLK_m1_c1.db
read_db -mode_label m1 AMS_BLK_m1_c2.db
read_db -mode_label m1 AMS_BLK_m1_c3.db
read_db -mode_label m2 AMS_BLK_m2_c1.db
read_db -mode_label m2 AMS_BLK_m2_c2.db
read_db -mode_label m2 AMS_BLK_m2_c3.db


# Check the compatibility of the libraries you read in
check_workspace

# Generate cell library
commit_workspace -output AMS_BLK_ETM.ndm
```

When you create a cell library for an ETM, you can use one of the following methods to obtain the physical data:

- Read the frame view of the corresponding block by using the `read_ndm -views frame` command.
- Read a LEF file for the block by using the `read_lef` command.
- Read a GDSII file for the block by using the `read_gds` command
- Read an OASIS file for the block by using the `read_oasis` command.

Feedback

Linking to ETMs at the Top Level

To link to ETMs at the top level, add the ETM libraries to the reference library list before you create the top-level design library, as shown in the following example:

```
icc2_shell> lappend nt_ref_lib "AMS_BLK_ETM.ndm"
icc2_shell> create_lib -technology tech.tf -ref_libs $nt_ref_lib TOP
```

An ETM that contains multiple modes is called a moded ETM. For a cell instance that links to a moded ETM, you must specify the required mode by using the `set_cell_mode` command, as shown in the following example:

```
icc2_shell> current_mode m1
icc2_shell> set_cell_mode m1 U1
```

For a cell instance that links to a moded ETM, if you do not specify a mode, the tool does not activate the timing arcs, timing checks, generated clocks, and case values.

To report the cell modes for specific cell instances, use the `report_cell_modes` command.

To switch a cell instance from its abstract view to its ETM or vice versa, use the `set_reference` command. The following example switches the UI cell instance from its abstract view to its ETM:

```
icc2_shell> set_reference -block AMS_BLK_ETM.ndm:AMS_BLK.timing U1
```

The following example switches the UI cell instance from its ETM to its abstract view:

```
icc2_shell> set_reference -block AMS_BLK.ndm:AMS_BLK.abstract U1
```

When you switch between ETMs and abstract views, reapply the top-level timing constraints.

Feedback

Performing Top-Level Closure With ETMs

Before you can use the ETMs at the top level, you must perform the following tasks:

- Apply the top-level-only timing constraints and settings.

  If the cell instances represented by ETMs have internal clocks, you must reapply these clock definition from the top level. Avoid cross-boundary timing exceptions that refer to objects inside the blocks.

- To apply the UPF constraints and settings, you must:

  1. Generate the block-level UPF from each of the block-level designs using the `save_upf` command.
  2. At the top level, load the block UPF for each of the block instances that are represented by an ETM and the top only UPF:

     ```
     load_upf -scope <instance_name> block.upf
     load_upf top.upf
     ```

- Apply top-level placement constraints and settings such as hard keepout margins.

When using ETMs at the top-level, you can perform , top-level placement, optimization, clock tree synthesis, routing, and postroute optimization using the commands supported at the block level.

Transparent Hierarchy Optimization

Transparent hierarchy optimization (THO) enables the tool to transparently optimize hierarchical designs.

THO enables you to optimize the top and subblocks concurrently while honoring the physical block boundaries, and at the same time using all advanced optimization techniques such as CCD, AWP, PrimeTime delay calculation, and so on.

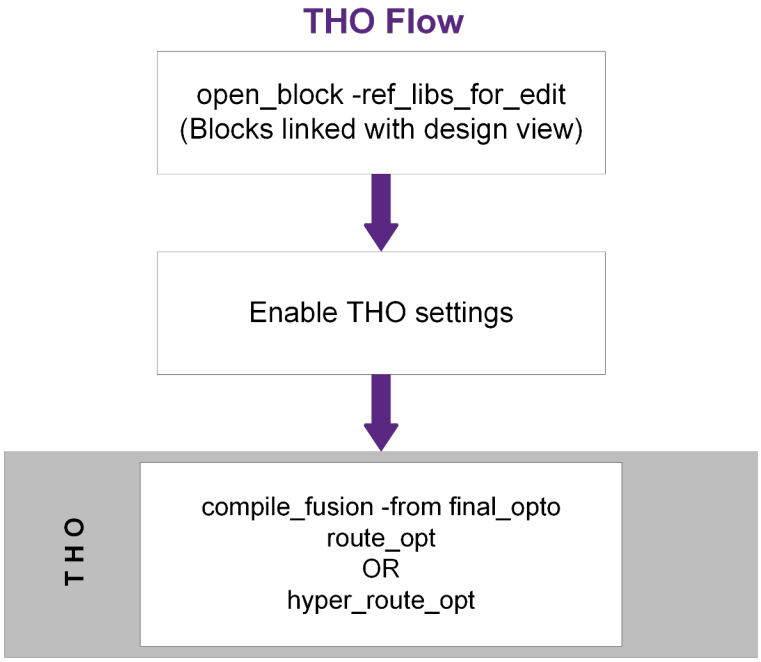
During THO, the tool optimizes the top-level and specific subblocks (design views) in the context of top-level timing. THO does not change the subblock shape and physical hierarchy. By default, the tool honors the pin locations, but provides an option to reassign pin locations to improve QoR.

You can continue your signoff flow with hierarchical blocks because you do not have to flatten the design. Signoff extraction and PrimeTime STA can be run hierarchically. In addition, any late ECOs can be easier to implement as well.

THO is available during `place_opt` in the `final_opto` stages, as well as during `route_opt` and `hyper_route_opt`.

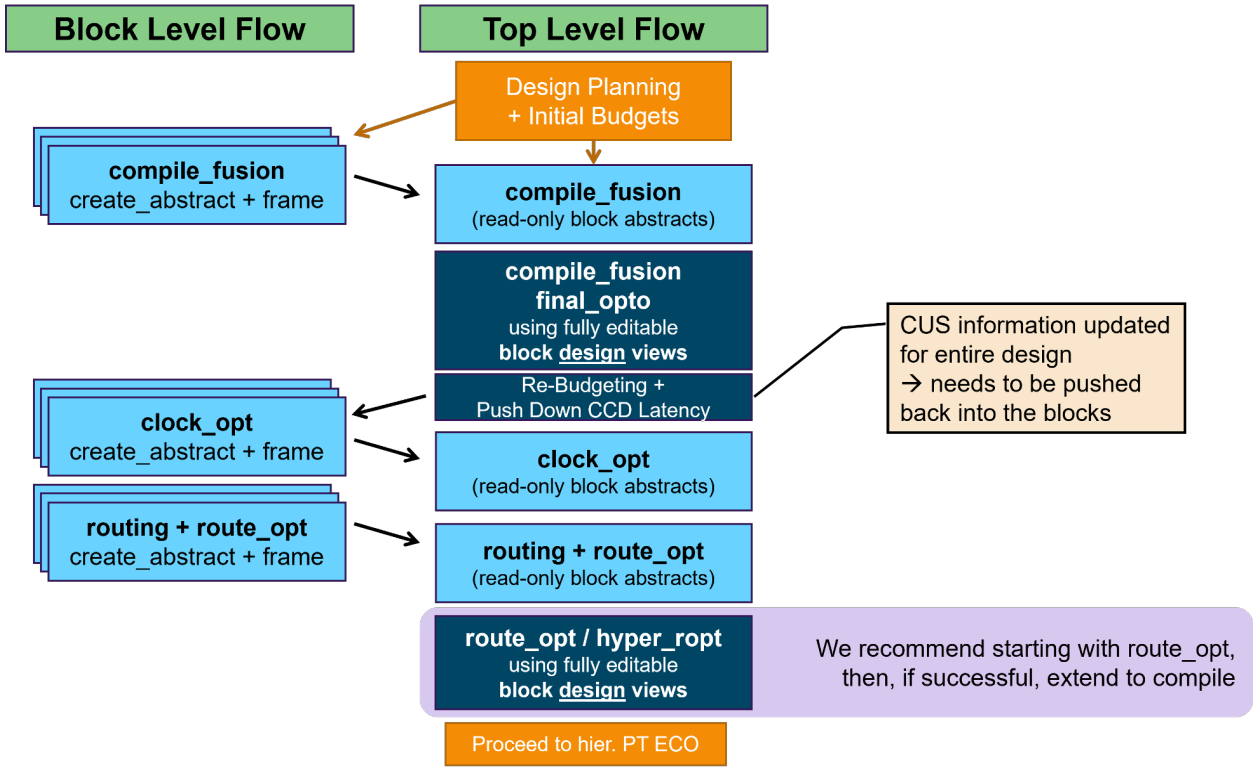
Feedback

Figure 157 Transparent Hierarchy Optimization Flow



The following example flow shows the usage of THO in the overall implementation flow. In this example, the THO operations are run as an additional incremental optimization step on top of the typical hierarchical optimization flow.

Feedback



The THO flow includes the following steps:

- Accessing Libraries Before Enabling Transparent Hierarchy Optimization
- Setting up Transparent Hierarchy Optimization
- Performing THO During place_opt final_opto Stage
- Rebudgeting After place_opt
- Performing THO During route_opt or hyper_route_opt

Accessing Libraries Before Enabling Transparent Hierarchy Optimization

Before enabling the THO settings, you must ensure that:

- All the subblock libraries are opened in edit mode. You must open the parent library using the `-ref_libs_for_edit` option with the `open_lib` command.
- The design view of the blocks is editable.

Feedback

Setting up Transparent Hierarchy Optimization

To perform THO, you need to have design views. You might be linking to abstract views at the top. Use the instructions in the Changing the Block From Abstract to Design View section to switch the blocks you want to optimize to design views. Perform the following steps to set up THO:

1. To perform THO after block-level `place_opt`, you must propagate the CCD useful skew balancing offsets and latency offsets from block-level clock pins to the top-level timing context. The CCD clock data promotion occurs automatically when you run the `init_hier_optimization` command.

   By default, only connected clock data is promoted to the top-level, which is preferred for most designs. You can customize the clock mapping rule for promoting clock data during the `init_hier_optimization` command using the `top_level.tho_block_to_top_map_auto_clock` application option.

   If you have already mapped clocks using the `set_block_to_top_map` command, the tool honors those mappings and only apply the rule to unmapped clocks. The default is `connected`, but you can override this using the new `top_level.tho_block_to_top_map_auto_clock` application option.

2. To enable and perform THO on the blocks, ensure that the blocks you want to optimize with THO are enabled for editing:

   ```
   icc2_shell> set_editability -blocks {list of blocks} -value true
   ```

3. To verify block editability, use the following command:

   ```
   icc2_shell> report_editability -blocks [get_blocks -hierarchical]
   ```

4. During THO, operations such as congestion updates and route-driven estimation (RDE) calculations inside subblocks as well as ECO-routing can be run using distributed processing. Use the following commands to perform the setup. You must specify your own queue parameters such as the `qsub` parameters as shown in the following example:

   Note:

   If host options are not specified, the tool runs the operations for the subblocks serially in the main process.

   ```
   icc2_shell> set_host_options -name for_tho -num_processes 4 -max_cores
    16
    -submit_command [list qsub -P queue -pe mt 16 -l mem_free=100G -cwd]
   icc2_shell> set_hierarchy_options -host_option for_tho -blocks {list
    of blocks}
   ```

Feedback

5. To specify if the whole sub-block logic or only the interface logic of the sub-block is open for optimization, use the `-optimize_interface_only` option with the `set_hierarchy_options` command.

   If the value is set to `true`, only the interface logic is open for optimization. When set to `false`, the whole sub-block logic is open for optimization.

6. To specify if the shared logic between the interface and register-to-register or only the pure interface logic of the sub-block is open for optimization, use the `-optimize_shared_logic` option with the `set_hierarchy_options` command.

   If the value is set to `unrestricted`, the shared logic is open for optimization. When set to `none`, the shared logic is not open for optimization.

7. To report the settings applied with the `set_hierarchy_options` command, use the `report_hierarchy_options` command.

8. To enable THO, use the `init_hier_optimization` command. Depending on the stage, you must use different options as shown in the following sections.

Performing THO During place_opt final_opto Stage

The IC Compiler II tool performs transparent hierarchical optimization during the `final_opto` stage, which ensures that all interblock and block-top paths are optimized as if the design were flat. In this manner, CCD optimizations can address any violations on these paths completely.

The `final_opto` stage is the last optimization step in the physical synthesis flow. It focuses on:

- Fixing timing violations (setup and hold)
- Reducing power consumption
- Minimizing area while maintaining timing and power constraints
- Resolving design rule check (DRC) violations
- Improving routability

```
icc2_shell> init_hier_optimization -flow pre_route
icc2_shell> place_opt -from final_opto -to final_opto
icc2_shell> commit_hier_optimization
```

Rebudgeting After place_opt

After running the `place_opt` command with THO, the block interface timing changes as compared to when the blocks were implemented individually. Because the blocks are

Feedback

now optimized based on the top-level timing picture, you must create updated timing constraints for the blocks before you continue with the `clock_opt` command for the blocks. The CCD useful skew balance point offsets might have been modified during THO. These offsets need to also be made block-context-specific so that the clock tree synthesis step can take them into account.

This example shows how you can perform budgeting:

```
icc2_shell> set budget_instances "u1/a/i1 u1/b/i2 …"
icc2_shell> set_budget_options -reset -all
icc2_shell> set_budget_options -add_blocks $budget_instances
icc2_shell> set_app_options -list { plan.budget.use_ccd_latency true }
icc2_shell> set_app_options -list { plan.budget.write_hold_budgets
 false }
icc2_shell> set_app_options -list { plan.budget.estimate_timing_mode
 false }
icc2_shell> compute_budget_constraints -setup_delay -latency_targets
 actual -balance false -boundary
icc2_shell> write_budgets -output budgets-post-compile -force -nosplit
 -include_balance_points -include_pin_latency
```

Performing THO During route_opt or hyper_route_opt

By performing THO at the end of the implementation flow, you have the best optimization potential because you optimize the complete design. The `route_opt` and `hyper_route_opt` commands perform postroute optimization on the top and blocks together. ECO routing after optimization is performed on all subblocks that are open for editing.

THO optimization is completely integrated with the `route_opt` and `hyper_route_opt` commands. Follow this example to use the `hyper_route_opt` or `route_opt` command on a single host:

```
icc2_shell> init_hier_optimization -flow post_route
icc2_shell> route_opt or hyper_route_opt
icc2_shell> commit_hier_optimization
```

When using the `route_opt` command, it is possible to perform the ECO routing operations, which have to occur after each `route_opt` command in parallel, reducing overall runtime. You must set up the host options as shown in the Setting up Transparent Hierarchy Optimization section.

11

Thermal Analysis Using Kelvin

Kelvin is a thermal solution that provides flexibility and customization in thermal analysis. It allows you to control the trade-off between accuracy and runtime by specifying the number of grids, making it adaptable to different design requirements and computational resources. Additionally, Kelvin supports multi-scenario thermal analysis, enabling evaluation of thermal behaviors across various operating conditions.

Supporting Flows in Kelvin

The availability of multiple flows ensures that the tool caters to a wide range of design requirements, from simple to highly complex designs. Each flow is optimized for specific scenarios, which helps achieve the best performance, power, and area (PPA) while meeting thermal and timing constraints.

Basic Flow

- Opening the Library and Block:

```
open_lib
open_block
```

- Setting Application Options:

  Note:
  The following application options have defaults and do not need to be explicitly set unless customization is required.

  - Thermal Engine:

```
set_app_options -name thermal.engine -value helios(default) /compact
```

  - Grid Resolution:

```
set_app_options -name thermal.grid_resolution -value {100 100}
```

    The unit of grid-size is Micrometer.

  - Heat Transfer Rate:

```
set_app_options -name thermal.heat_transfer_rate -value {10000.0 0}
```

Feedback

- Thermal Database:

```
set_app_options -name thermal.database -value THERMAL_DATABASE
```

- Thermal Tech File:

  Note:

  The `thermal.tech_file` must be provided. It wont be generated in memory.

```
set_app_options -name thermal.tech_file -value TECH_FILE
```

- If the following files are available, they can be specified. Otherwise, they are generated in memory (Optional):
  - Power Profile:

```
set_app_options -name thermal.power_profile -value POWER_PROFILE
```

  - Metal Profile:

```
set_app_options -name thermal.metal_profile -value METAL_PROFILE
```

- Running Thermal Analysis:

```
analyze_thermal
```

Design Block With Macro Model

This flow builds upon the basic flow and includes additional options for design block with macro model.

- Opening the Library and Block:

```
open_lib
open_block
```

- Setting Application Options:

  Note:
  The following application options have defaults and do not need to be explicitly set unless customization is required.

  - Thermal Engine:

```
set_app_options -name thermal.engine -value helios (default)/Compact
```

  - Grid Resolution:

```
set_app_options -name thermal.grid_resolution -value {100 100}
```

Feedback

The unit of grid-size is Micrometer.

- Heat Transfer Rate:

```
set_app_options -name thermal.heat_transfer_rate -value {10000.0 0}
```

Note:
Providing a customized heat transfer rate is recommended, even though the tool defaults to predefined settings.

- Thermal Database:

```
set_app_options -name thermal.database -value THERMAL_DATABASE
```

- Thermal Tech File:

Note:

The `thermal.tech_file` must be provided. It wont be generated in memory.

```
set_app_options -name thermal.tech_file -value TECH_FILE
```

- If the following files are available, they can be specified. Otherwise, they are generated in memory (Optional):
  - Power Profile:

```
set_app_options -name thermal.power_profile -value POWER_PROFILE
```

  - Metal Profile:

```
set_app_options -name thermal.metal_profile -value METAL_PROFILE
```

- Static Thermal Analysis with Macro Model:

  For the block design with macro model, you can specify additional application options for block-level power and metal profiles:

  - Specifying power values for all macros on design:

Note:
Unit is watt.

```
set_app_options -name thermal.hier_block_power_profile -value "
Instance-Block-A 0.01 \
Instance-Block-B 0.03 \
.
"
```

Feedback

- Specifying metal density for all macros on design:

  Note:
  Full metal density is 1.

```
set_app_options -name thermal.hier_block_metal_profile -value "
MASTER_BLOCK_x 0.1 \ #metal density 10%
MASTER_BLOCK_Y 0.3 \ #metal density 30%
```

- Running Thermal Analysis:

```
analyze_thermal
```

Compact Model Flow

The compact model flow for thermal analysis is designed for scenarios where a simplified thermal model is sufficient. This flow uses a compact thermal model file and provides options for customization, while some settings have defaults.

Note:
Ensure you have the appropriate compact model file to proceed with this flow.

- Opening the Library and Block:

```
open_lib
open_block
```

- Setting Application Options:
  - Thermal Engine:

```
set_app_options -name thermal.engine -value compact
```

  - Thermal Database:

```
set_app_options -name thermal.database -value THERMAL_DATABASE
```

- If the following files are available, they can be specified. Otherwise, they are generated in memory (Optional):
  - Power Profile:

```
set_app_options -name thermal.power_profile -value POWER_PROFILE
```

- Setting Compact Model-Specific Options:
  - Compact Model File (Mandatory):

    ```
    set_app_options -name thermal.compact_model_file -value
    COMPACT_TECH_FILE
    ```

  - Use Layout Information:

    ```
    set_app_options -name thermal.use_layout -value true / false
    ```

- Running Thermal Analysis:

  ```
  analyze_thermal
  ```

Note:

- The `thermal.compact_model_file` application option is mandatory for this flow and must be specified.
- The `thermal.use_layout` application option determines whether layout information is used in the analysis. It is:
  - set to `true` to include layout information.
  - set to `false` to exclude layout information.
- This flow does not require a metal profile, grid resolution, and heat transfer rate application options, thereby simplifying the setup compared to other thermal analysis flows.

Multicorner-Multimode Flow

The Multicorner-Multimode (MCMM) flow for thermal analysis is designed to handle different combinations of modes and corners, providing flexibility and accuracy for complex designs.

- Opening the Library and Block:

  ```
  open_lib
  open_block
  ```

- Setting Application Options:

  Note:
  The following application options have defaults and do not need to be explicitly set unless customization is required.

Feedback

- Thermal Engine:

  ```
  set_app_options -name thermal.engine -value helios (default)/compact
  ```

- Grid Resolution:

  ```
  set_app_options -name thermal.grid_resolution -value {100 100}
  ```

  The unit of grid-size is Micrometer.

- Heat Transfer Rate:

  ```
  set_app_options -name thermal.heat_transfer_rate -value {10000.0 0}
  ```

  Note:
  Providing a customized heat transfer rate is recommended, even though the tool defaults to predefined settings.

- Thermal Database:

  ```
  set_app_options -name thermal.database -value THERMAL_DATABASE
  ```

- Defining Modes, Corners, and Scenarios.

  Create the modes, corners, and scenarios for the MCMM flow:

  - Create Mode M1 and Corner C1, then combine them into Scenario S1:

    ```
    create_mode M1
    create_corner C1
    create_scenario -name S1 -mode M1 -corner C1
    ```

  - Create Mode M2 and Corner C2, then combine them into Scenario S2:

    ```
    create_mode M2
    create_corner C2
    create_scenario -name S2 -mode M2 -corner C2
    ```

- Setting Current Scenarios and Loading Constraints:

  - Set the current scenario to S1 and load its corresponding SDC file:

    ```
    current_scenario S1
    read_sdc S1.sdc
    ```

  - Set the current scenario to S2 and load its corresponding SDC file:

    ```
    current_scenario S2
    read_sdc S2.sdc
    ```

- Setting Thermal Scenario Status

Feedback

Activate the thermal scenarios for analysis:

```
set_scenario_status -thermal true [get_scenarios S1] -active true
set_scenario_status -thermal true [get_scenarios S2] -active true
```

- Setting Thermal Scenario Details:

  Specify additional details for each thermal scenario, such as power profile, metal profile, and ITF file:

  Note:
  Only `power_profile` and `metal_profile` are optional; the `-itf` is required.

  - For Scenario S1:

    ```
    set_thermal_scenario -name S1 -power_profile XXX.pwr -metal_profile
    XXX.metal -itf XXX.itf
    ```

  - For Scenario S2:

    ```
    set_thermal_scenario -name S2 -power_profile XXX.pwr -metal_profile
    XXX.metal -itf XXX.itf
    ```

- Running Thermal Analysis:
  - Analyze specific scenarios:

    ```
    analyze_thermal -scenarios { S1 S2 }
    ```

  - Analyze all scenarios:

    ```
    analyze_thermal
    ```

Displaying Maps in GUI

The `.spr` file is a specific file format used to store thermal profile data, which is used for visualization in the GUI as a thermal profile map. The process involves loading the `.spr` file into the GUI and displaying the thermal map.

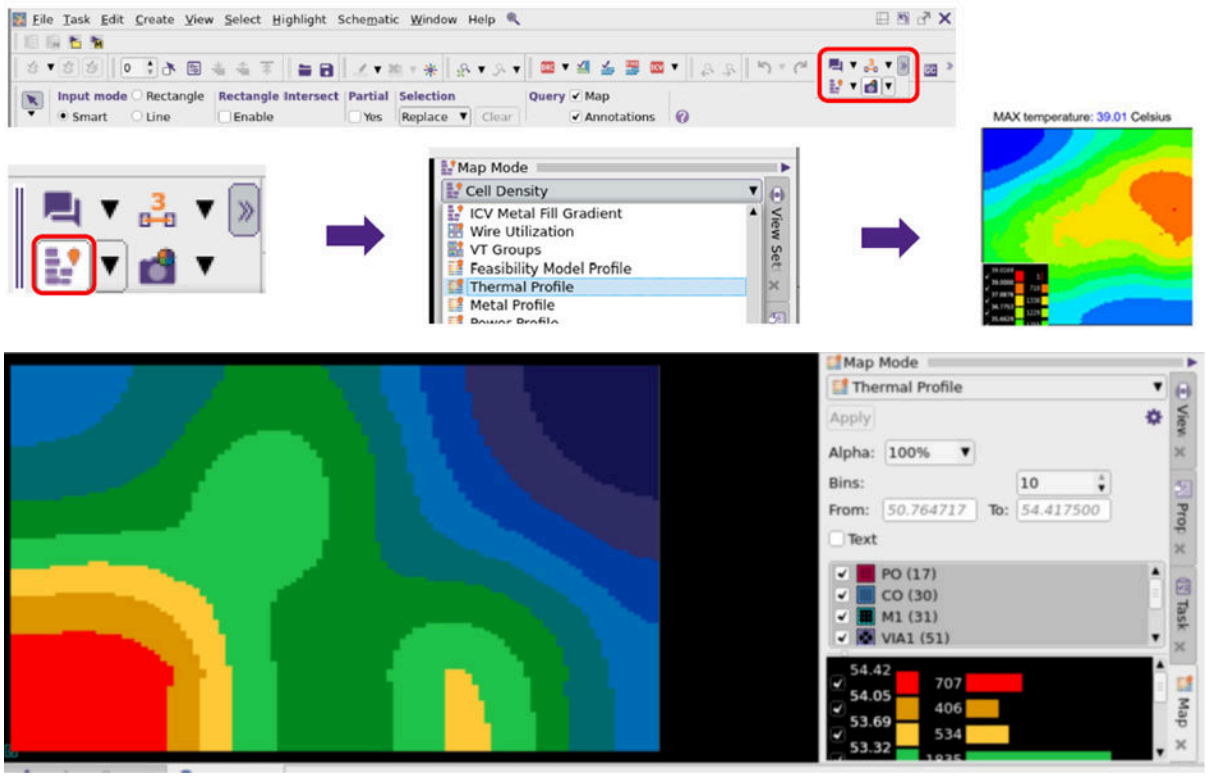
Execute the following commands to perform the respective functionalities:

- Start the GUI using the `start_gui` command.
- Display the Thermal Profile Map by executing the following command:

  ```
  gui_show_map -window [gui_get_current_window -types Layout -mru] -map
  {thermal_profile} -show {true}
  ```

Feedback

or by selecting Thermal Profile from the Map Mode in the GUI as shown:



- You can also use the `analyze_thermal` command to perform thermal analysis and load the resulting thermal profile map into the GUI.

To reopen an existing `THERMAL_DATABASE` and support reopening a single scenario after running the MCMM flow, you can use the `open_fusion_thermal_result` command with the `-result` option to specify the required scenario.

For example:

```
open_fusion_thermal_result -result {S1}
```

Reporting Thermal Results

You can use the `report_thermal_qor` command to generate a report of the thermal analysis results for a design.

Given is the syntax:

```
report_thermal_qor -threshold Temperature_Value -scenario {scenario_name}
```

Feedback

- `-threshold Temperature_Value`: Specifies the temperature threshold in Celsius. Only results with temperatures above this threshold are included in the report.
- `-scenario {scenario_name}`: Specifies the scenario for which the thermal results are to be reported. A scenario typically represents a specific operating condition or mode of the design.

The following is an example:

```
report_thermal_qor -threshold 35.9 -scenario FUNC_0.85V_SETUP
```

The tool now processes the thermal analysis results and generates a report based on the specified threshold and scenario. This helps identify hotspots or areas of concern in the design.

If you want to include detailed information about specific instances in the design, add the `-instance` option. For example:

```
report_thermal_qor -threshold 35.9 -scenario FUNC_0.85V_SETUP -instance
```

This provides a more granular view of the thermal results, showing data for individual instances.

12

RedHawk and RedHawk-SC Fusion

The RedHawk™ and RedHawk-SC™ power integrity solution is integrated with the IC Compiler II implementation flow through the RedHawk Fusion and RedHawk-SC Fusion interface. You can use the RedHawk Fusion and RedHawk-SC Fusion feature for rail analysis at different points in the physical implementation flow after power planning and initial placement are completed. This enables you to detect potential power network issues before you perform detail routing and thus significantly reduce the turnaround time of the design cycle.

When placement is complete and the PG mesh is available, use RedHawk Fusion or RedHawk-SC Fusion to perform voltage drop analysis on the power and ground network to calculate power consumption and to check for voltage drop violations. You can perform voltage drop analysis at other stages in the design flow, such as after detail routing. When chip finishing is complete, use RedHawk Fusion or RedHawk-SC to perform PG electromigration analysis to check for current density violations.

This chapter describes how to use the RedHawk Fusion or RedHawk-SC Fusion feature to perform rail analysis in the IC Compiler II environment. For a detailed description about the RedHawk or RedHawk-SC analysis flows and commands, see the *RedHawk User Manual* or *RedHawk-SC User Manual*.

This chapter includes the following topics:

- Running Rail Analysis Using RedHawk-SC Fusion
- An Overview for RedHawk Fusion and RedHawk-SC Fusion
- Setting Up the Executables
- Specifying RedHawk and RedHawk-SC Working Directories
- Preparing Design and Input Data for Rail Analysis
- Specifying Ideal Voltage Sources as Taps
- Missing Via and Unconnected Pin Checking
- Running Rail Analysis with Multiple Rail Scenarios
- Performing Voltage Drop Analysis
- Performing PG Electromigration Analysis

Feedback

- Performing Minimum Path Resistance Analysis
- Performing Effective Resistance Analysis
- Performing Distributed RedHawk Fusion Rail Analysis
- Voltage Driven Power Switch Cell Sizing
- Working With Macro Models
- Performing Signoff Analysis
- Writing Analysis and Checking Reports
- Performing Effective Voltage Drop (Eff-VD) and Node-Based IR-Drop Flows
- Displaying Maps in the GUI
- Displaying ECO Shapes in the GUI
- Voltage Hotspot Analysis
- Querying Attributes

Running Rail Analysis Using RedHawk-SC Fusion

Similar to the RedHawk Fusion capability, RedHawk-SC Fusion supports gate-level rail analysis and checking capabilities in the IC Compiler II environment. To enable the RedHawk-SC Fusion capability, you need to enable the `rail.enable_redhawk_sc` application option and disable the `rail.enable_redhawk` application option at the same time.

For more information about RedHawk-SC Fusion, see Setting Up the Executables and Specifying RedHawk and RedHawk-SC Working Directories.

RedHawk Fusion and RedHawk-SC share the same set of analysis and checking commands for rail analysis, and the same GUI for examining the analysis results, except for some limitations. Table 60 compares the `analyze_rail` options that are supported in either or both of RedHawk Fusion and RedHawk-SC Fusion.

Table 60 Comparing Analysis Features Between RedHawk Fusion and RedHawk-SC Fusion

	RedHawk Fusion	**RedHawk-SC Fusion**
`-redhawk_script_file`	Supported	Supported
`-voltage_drop`	Supported	Supported
`-switching_activity`	Supported	Supported

Feedback

Table 60 Comparing Analysis Features Between RedHawk Fusion and RedHawk-SC Fusion (Continued)

	RedHawk Fusion	RedHawk-SC Fusion
`-electromigration`	Supported	Supported
`-power_analysis`	Supported	Supported
`-result_name`	Supported	Supported
`-script_only`	Supported	Supported
`-bg`	Supported	Supported
`-min_path_resistance`	Supported	Supported together with the `-voltage_drop` option only
`-effective_resistance`	Supported	Supported together with the `-voltage_drop` -option only
`-check_missing_via`	Supported	Supported together with the `-voltage_drop` option only Displaying missing via check results does not honor the IR threshold setting.
`-extra_gsr_option_file`	Supported	Not supported
`-multiple_script_files`	Supported	Not supported
`-submit_to_other_machines`	Supported	Not supported

Before performing rail analysis using the RedHawk-SC Fusion capability, you must specify the location of the libraries and the required input files as described in Preparing Design and Input Data for Rail Analysis. The following two application options are available only in RedHawk-SC Fusion:

`rail.toggle_rate`:

Specify toggle rates for different cell types for rail analysis.

For example:

```
icc2_shell> set_app_options -name rail.toggle_rate  \
   -value {clock 2.0 data 0.2 combinational 0.15 \
   sequential 0.15}
```

Feedback

`rail.em_only_tech_file`:

Specify the electromigration rule file for performing PG electromigration analysis. Use this option when the electromigration rule information is defined in a separate technology file other than the one specified by the `rail.tech_file` application option.

For example,

```
icc2_shell> set_app_options -name rail.em_only_tech_file \
   -value EM_ONLY.rule
```

An Overview for RedHawk Fusion and RedHawk-SC Fusion

Both RedHawk Fusion and RedHawk-SC Fusion support gate-level rail analysis capabilities, including static and dynamic rail analysis. You can use RedHawk Fusion and RedHawk-SC Fusion to perform the following types of checking and analysis in the IC Compiler II environment:

- Missing via and unconnected pin shape checking

  To check for missing vias or unconnected pin shapes in the design, you first configure the checking-related settings by using the `set_missing_via_check_options` command, and then perform the check by using the `analyze_rail -check_missing_via` command.

  For more information, see Missing Via and Unconnected Pin Checking.

- Voltage drop analysis

  To perform voltage drop analysis, use the `analyze_3d_rail -voltage_drop -electromigration` command.

  To perform voltage drop analysis, use the `analyze_rail -voltage_drop` command or choose Rail > Analyze Rail and select “Voltage drop analysis” in the GUI.

  Set the `-voltage_drop` option to `static`, `dynamic`, `dynamic_vcd`, or `dynamic_vectorless` to choose the type of analysis you want to perform.

  For more information, see Performing Voltage Drop Analysis.

- PG electromigration analysis

  To perform PG electromigration analysis, use the `analyze_rail -electromigration` command.

  For more information, see Performing PG Electromigration Analysis.

Feedback

- Minimum path resistance analysis

  To perform minimum path resistance analysis, use the `analyze_rail -min_path_resistance` command.

  In the RedHawk-SC analysis flow, you must use the `-voltage_drop` option together with the `-min_path_resistance` option for minimum path resistance analysis.

  For more information, see Performing Minimum Path Resistance Analysis.

Depending on the type of analysis you run, the tool generates visual displays (maps) of the results that you can view in the IC Compiler II GUI, as well as error data that you can display in the IC Compiler II error browser. These maps and error data help you discover problem areas and determine corrective action without leaving the IC Compiler II environment.

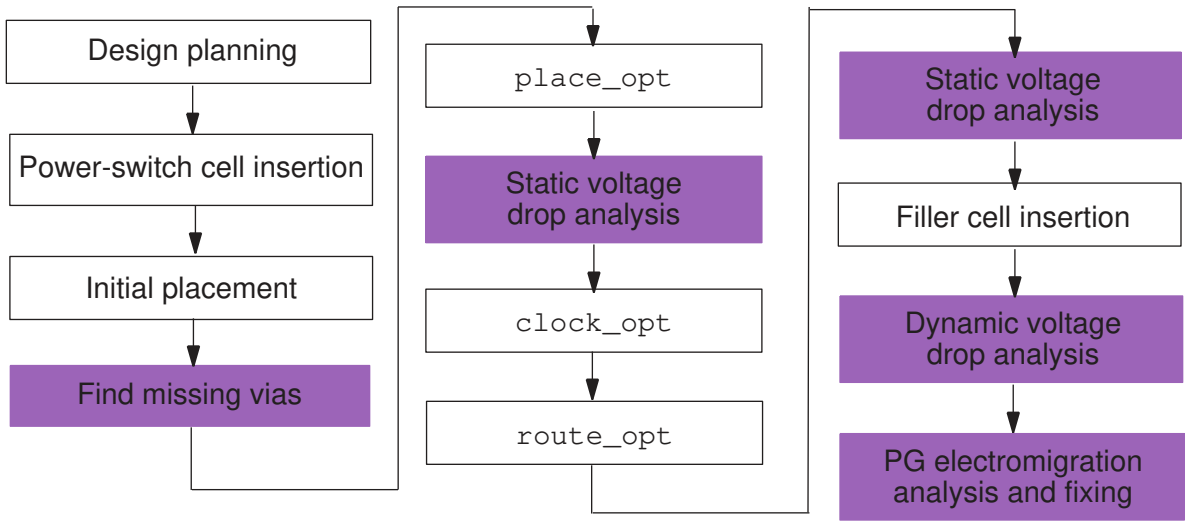
Note:

RedHawk Fusion in the IC Compiler II tool does not support RedHawk signoff analysis features, such as hierarchical analysis or dynamic analysis with lumped or SPICE packages. For more information, see Preparing Design and Input Data for Rail Analysis.

By default, RedHawk Fusion analyzes only the current design scenario when analyzing a multicorner-multimode design. For more information about how to create and specify a rail scenario for rail analysis, see Running Rail Analysis with Multiple Rail Scenarios.

Figure 158 shows where you would use these analysis capabilities in a typical design flow.

Figure 158 Using RedHawk Fusion in the IC Compiler II Design Flow



Feedback

RedHawk Fusion and RedHawk-SC Fusion Data Flow

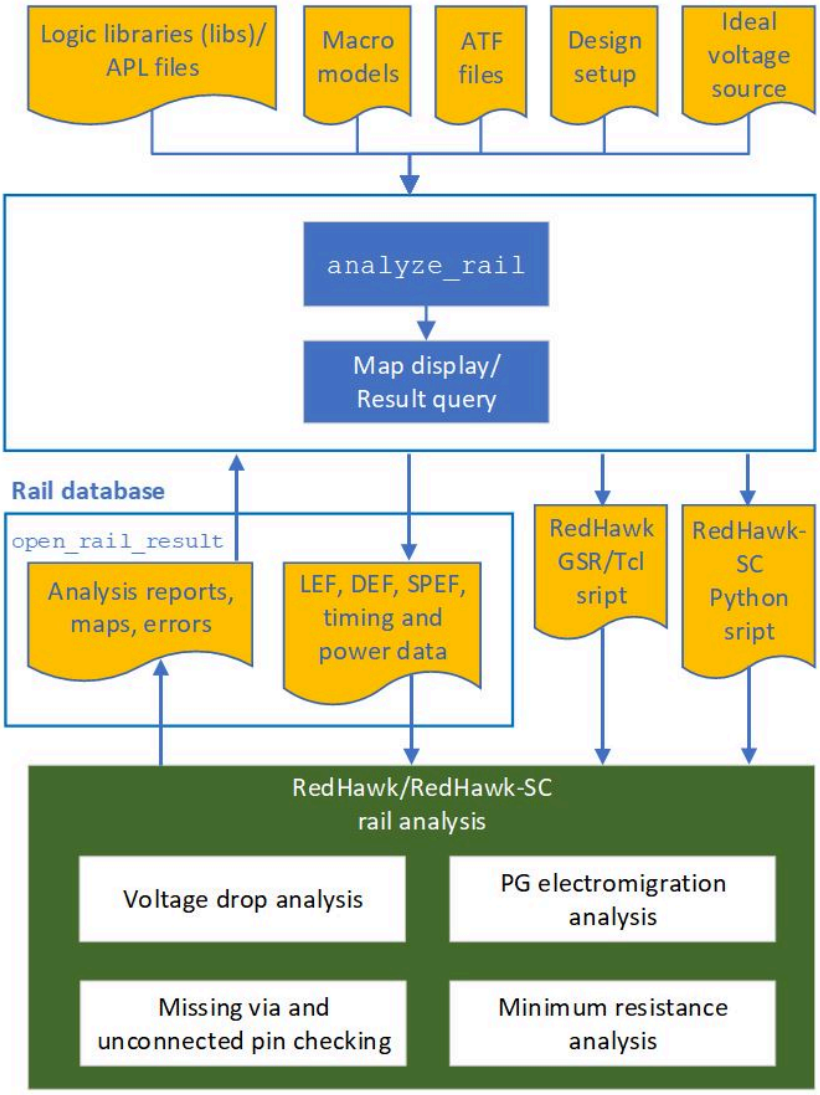
The RedHawk/RedHawk-SC Fusion feature allows you to perform rail analysis during the implementation stage. With the required input files, the IC Compiler II tool creates the RedHawk run script and the Global System Requirements (GSR) configuration file for invoking the RedHawk tool (or the RedHawk-SC Python script for invoking the RedHawk-SC tool) to run PG net extraction, power analysis, voltage drop analysis, and PG electromigration analysis.

When analysis is complete, the IC Compiler II tool generates analysis reports and maps using the results calculated by the RedHawk or RedHawk-SC tool. You can then check for hotspots graphically in the IC Compiler II GUI. Error data and ASCII reports are also available to check for locations where limits are violated.

Figure 159 illustrates the data flow when using RedHawk Fusion or RedHawk-SC Fusion to perform rail analysis in the IC Compiler II environment.

Feedback

Figure 159 RedHawk Fusion and RedHawk-SC Fusion Data Flow



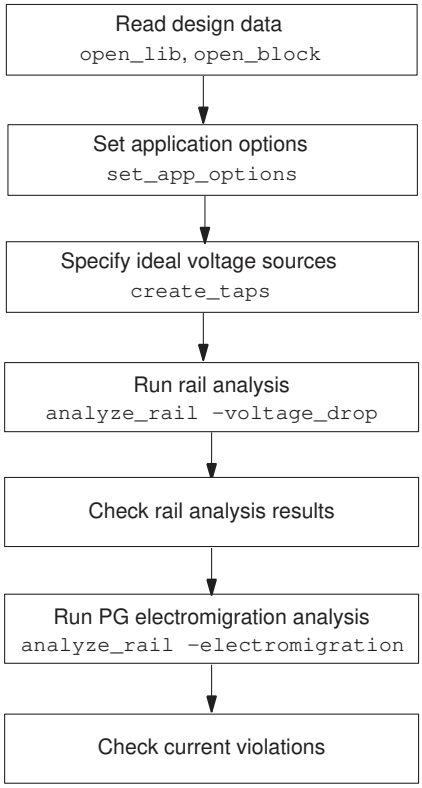
RedHawk/RedHawk-SC Fusion Analysis Flow

When you have specified the necessary input and design data, you can perform voltage drop and PG electromigration analyses on the design.

Figure 160 illustrates the steps in a basic static rail analysis flow.

Feedback

Figure 160 Required Steps for a Basic Rail Analysis Using RedHawk and RedHawk-SC Fusion



Running RedHawk Fusion Commands in the Background

By default, when you execute the `analyze_rail` command to perform rail analysis, the RedHawk or RedHawk-SC tool is invoked to perform the specified analysis types with the input data and then load the analysis results back to the rail database when rail analysis is finished. You cannot run any IC Compiler II commands unless the RedHawk Fusion or RedHawk-SC Fusion process is finished.

To perform layout editing tasks while the tool is running in the background, run the `analyze_rail -bg` command. When analysis is completed, run the `open_rail_result -back_annotate` command to upload the analysis results to the rail database.

By default, RedHawk or RedHawk-SC loads the analysis results back to the rail database when analysis is completed. However, when you specify the `-bg` option with the `analyze_rail` command, you need to run the `open_rail_result -back_annotate` command to reconstruct rail database and create the new RAIL_DATABASE file from the latest analysis results.

Feedback

Note:

When back-annotating the analysis results from the previous run after moving nets in the design, the instance-based map reflects the updated instance location change, but the parasitic map does not.

When opening rail results, the tool does not detect if the result is generated with or without the `-bg` option. The command might not work correctly if you run the `open_rail_result -back_annotate` command to open the rail result that is generated without the `-bg` option, or vice versa.

Reporting and Checking the Status of the Background Process

When the RedHawk or RedHawk-SC Fusion process is finished, the tool issues the following message:

```
Info: Running REDHAWK_BINARY in background with log file: LOG_FILE
```

The LOG_FILE file is saved in the PWD directory.

To check if the background `analyze_rail` process is active, use the `report_background_jobs` command.

Setting Up the Executables

To run RedHawk Fusion or RedHawk-SC Fusion features, enable the features and specify the location of the RedHawk or RedHawk-SC executable by setting the `rail.product` and `rail.redhawk_path` application options.

For example,

```
icc2_shell> set_app_options -name rail.product -value redhawk|redhawk_sc
icc2_shell> set_app_options -name rail.redhawk_path \
    -value /tools/RedHawk_Linux64e5_V19.0.2p2/bin
```

You must ensure that the specified executable is compatible with the IC Compiler II version that you are using.

See Also

- Specifying RedHawk and RedHawk-SC Working Directories

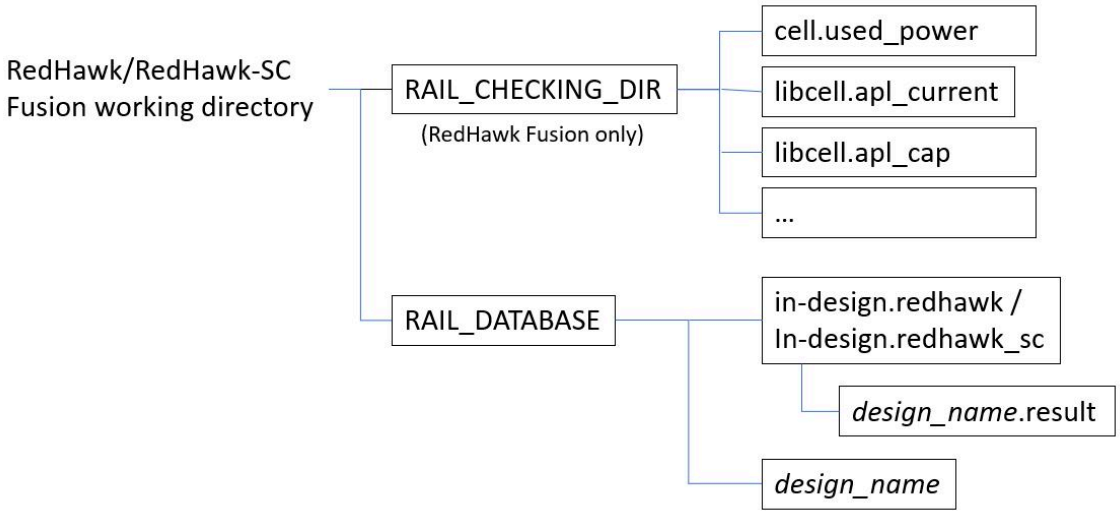
Specifying RedHawk and RedHawk-SC Working Directories

During rail analysis, RedHawk/RedHawk-SC Fusion creates a working directory to store the generated files, including analysis logs, scripts, and various output data. By default,

Feedback

the RedHawk or RedHakw-SC working directory is named RAIL_DATABASE. To use a different name for the working directory, use the `rail.database` application option.

The following figure shows the data structures:



- RAIL_CHECKING_DIR: The directory where RedHawk Fusion saves report files on the missing information during rail analysis.

  For example, the tool generates the following report files in the RAIL_CHECKING_DIR directory when static rail analysis is run:

  - libcell.apl_current
  - libcell.apl_cap
  - libcell.missing_liberty

- RAIL_DATABASE: The directory where the tool saves and retrieves results and log files that are generated during power calculation and rail analysis.

  This directory contains the following sub-directories:

  - in-design.redhawk or in-design.redhawk_sc: The directory that contains the following sub-directory:

    design_name.result: The directory where RedHawk or RedHawk-SC saves the analysis result files.

  - *design_name*: The directory where RedHawk or RedHawk-SC writes analysis results for the IC Compiler II tool to retrieve for map display and data query. For example, running the `open_rail_result` command loads the data in this *design_name* directory for displaying maps in the GUI.

Feedback

If you finish the analysis and want to keep the data for later retrieval, you need to save the existing rail data to another working directory by using the `rail.database` application option before proceeding to another `analyze_rail` run.

For example,

```
icc2_shell> set_app_options -name rail.database \
   -value RAIL_DATABASE_STATIC_RUN
```

Otherwise, the RedHawk/RedHawk-SC tool overwrites the previously generated rail data with the new data, and issues the following warning message:

```
Warning: Rail result still open! Will be overwritten!
```

To avoid the warning message, run the `close_rail_result` command to remove all rail data from memory before proceeding to another `analyze_rail` command run.

Preparing Design and Input Data for Rail Analysis

Before analyzing voltage drop and current density violations in a design using the RedHawk/RedHawk-SC Fusion capability, use the `set_app_options` command to specify the location of the libraries and the required input files.

Table 61 lists the application options for specifying design and input data for running RedHawk/RedHawk-SC rail analysis within the IC Compiler II environment.

Table 61 Application Options for Specifying Design and Input Data

Application Option	Description

`rail.lib_files`	Specifies the library files in .lib format.
	For example,
	`icc2_shell> set_app_options \`
	`  -name rail.lib_files \`
	`  -value {test1.lib test2.lib}`

`rail.tech_file`	Specifies the Apache technology file for performing PG extraction and PG electromigration analysis.
	For example,
	`icc2_shell> set_app_options \`
	`  -name rail.tech_file \`
	`  -value ChipTop.tech`
	
	Specifies the dies that are included in detailed analysis.
	For example,
	`icc2_shell> set_app_options \`
	`  -name rail.3dic_enable_die\`
	`  -value {DIE1 true DIE2 true ...}`

Feedback

Table 61 Application Options for Specifying Design and Input Data (Continued)

Application Option	Description

rail.instance_power_file	Specifies user-defined Instance Power File (IPF) for providing power to pins.
	For example,
	`icc2_shell> set_app_options \`
	`-name rail.instance_power_file\`
	`-value {E1 FILE1 DIE2 FILE2 ...}`

rail.apl_files	Specifies the Apache APL files, which contain current waveforms and intrinsic parasitics for performing dynamic rail analysis.
	For example,
	`icc2_shell> set_app_options \`
	`-name rail.apl_files -value \`
	`{cell.current current \`
	`cell.cdev cap}`

rail.switch_model_files	Specifies the RedHawk switch cell model files for performing dynamic rail analysis.
	For example,
	`icc2_shell> set_app_options \`
	`-name rail.switch_model_files \`
	`-value {switch.model}`

rail.macro_models	Specifies the macro model files.
	For example,
	`icc2_shell> set_app_options \`
	`-name rail.macro_models \`
	`-value {gds_cell1 mm_dir1 \`
	`gds_cell2 mm_dir2}`

rail.lef_files	(Optional) Specifies a list of LEF files. When not specified, the tool generates one using the data in the design library.
rail.def_files	(Optional) Specifies a list of DEF files. When not specified, the tool generates one using the data in the design library.
rail.pad_files	(Optional) Specifies a list of pad location files. When not specified, you need to specify the ideal voltage sources by running the `create_taps` command.
rail.sta_file	(Optional) Specifies the static timing file (STA), which contains the final slew and delay information. When not specified, the tool generates the static timing window information using the data in the design library.
rail.spef_files	(Optional) Specifies a list of SPEF files, which contain the detailed parasitic resistive and capacitive loading data of the signal nets. When not specified, the IC Compiler II tool generates the SPEF file using the data in the design library.

Feedback

Table 61 Application Options for Specifying Design and Input Data (Continued)

Application Option	Description
`rail.effective_resistance_instance_file`	(Optional) Specifies the instance file, which lists the cell instances for effective resistance calculation.

`rail.generate_file_type`	(Optional) Specifies the type of the script file to generate for RedHawk Fusion or RedHawk-SC Fusion rail analysis. The available file types are:
	• `tcl` (default): Generates the file in Tcl format. The default file name is *set_user_input.tcl*.
	• `python`: Generates the file in python format. The default file name is *input_files.py*. The python script file is supported only in RedHawk-SC Fusion rail analysis flow.
	This application option is required when you run the RedHawk-SC rail analysis with a customized python script file. For more information, see Supporting RedHawk-SC Customized Python Script Files.
	Example:
	`icc2_shell> set_app_options \`
	`  -name rail.generate_file_type -value python`

`rail.generate_file_variables`	This application option is supported in RedHawk-SC only. Specifies the preferred user variables when a customized python script file is used for RedHawk-SC rail analysis. This application option is required when the script contains non-standard collateral python variable names. See Supporting RedHawk-SC Customized Python Script Files for details.
	Supported file types are:
	• LEF
	• DEF
	• SPEF
	• TWF
	• PLOC
	Example:
	`icc2_shell> set_app_options \`
	`  -name rail.generate_file_variables -value \`
	`  {PLOC ploc LEF lef_files DEF def_files \`
	`  SPEF spef_files TWF tw_files}`

`rail.custom_script_variables`	Specifies the variable names for all the scenarios in a multiple rail scenario. Example:
	`icc2_shell> set_app_options -name\`
	`rail.custom_script_variables \`
	`-value {create_desgin_view dv DEF test_def_files GPD gpd123}`

`rail.enable_new_rail_scenario`	(Optional) Enables RedHawk Fusion or RedHawk-SC Fusion rail analysis with multiple rail scenario. The default is `false`. See Running Rail Analysis with Multiple Rail Scenarios for details.

Feedback

Table 61 Application Options for Specifying Design and Input Data (Continued)

Application Option	Description
`rail.enable_parallel_run_rail_scenario`	(Optional) Enables distributed RedHawk Fusion or RedHawk SC Fusion analysis within the analysis flow. The default is `true`.
`rail.scenario_name`	(Optional) Specifies the scenario for RedHawk or RedHawk-SC Fusion rail analysis. The tool honors the specified design scenario for the IR-driven features in power integrity flow, such as IR-driven placement, IR-driven concurrent clock and data optimization, and IR-driven optimizationSee Running Rail Analysis with Multiple Rail Scenarios for details..
`rail.dump_icc2_results_style -value 2.0`	Specifies how to load the results with the new Ansys map style.

Generating Rail Analysis Script Files

You can run the `analyze_rail -script_only` command to generate the settings required for running rail analysis based on the information in the design library without actually executing the analysis run.

The `analyze_rail -script_only` command writes data to the working directory. The output files that are saved to the directory are

Table 62 Output data when running analyze_rail -script_only

File Name	Description	RedHawk	RedHawk-SC
RedHawk GSR file	Contains configuration settings for running RedHawk Fusion rail analysis	Yes	No
LEF/DEF, SPEF, and STA files	Generated if you do not specify these input files before running the `analyze_rail` command	Yes	Yes
RedHawk Fusion run script file (analyze_rail.tcl)	Includes the commands that are required for running rail analysis	Yes	No
RedHawk-SC Fusion run script file (analyze_rail.py)	Includes the python commands that are required for running rail analysis	No	Yes

Feedback

Supporting RedHawk-SC Customized Python Script Files

RedHawk-SC Fusion provides application options and commands to read in user-customized python script files. The tool generates all design collateral (including DEF, LEF, SPEF, TWF, PLOC, and GPD) from the design library and adds them to the user-customized python script to run voltage drop analysis in a single iteration of `analyze_rail`. The user-customized python script file is supported in rail analysis with multiple rail scenarios, as well as other IR-driven features, such as IR-driven placement or IR-driven concurrent clock and data optimization.

To use a customized python script file for RedHawk-SC Fusion rail analysis,

1. Set the following application options for tech file, RedHawk path, and library files:

```
set_app_options -name rail.tech_file -value $TECH
set_app_options -name rail.redhawk_path -value $RHSC_PATH
set_app_options -name rail.lib_files -value $LIB
```

2. Set the `rail.custom_script_variables` application option to set variable names for all the rail scenarios in a multi-rail-scenario rail analysis. For single scenario rail analysis, use the `-custom_script_variables` option with the `set_rail_scenario` command. You do not need to set this application option if the customized python script file uses the same values as in RedHawk-SC Fusion.

   Note:
   SPEF and GPD files are mutually exclusive.

   The `set_rail_scenario` command supports electromigration and minimum path resistance analysis as well. For example:

```
set_rail_scenario -electromigration -min_path_resistance
```

   The following table shows the default variable names in IC Compiler II and RedHawk-SC for the python script.

	DEF File	LEF File	SPEF File	TWF File	GPD File	PLOC File
Default name in icc2_shell	`DEF`	`LEF`	`SPEF`	`TWF`	`GPD`	`PLOC`
Default variable name in RedHawk-SC Fusion	`def_files`	`lef_files`	`spef_files`	`tw_files`	`gpd_files`	`ploc`

   For example, when the customized python uses a different variable called `test_def` for `DEF` which is different from default variable (`def_files`) in RHSC,

```
set_app_option -name rail.custom_script_variables -value DEF test_def
```

Feedback

3. Create and specify rail scenarios for generating the necessary input files to add to the customized python script.

   Run the `create_rail_scenario` command to create a rail scenario and associate it with a design scenario in the current design. You can create multiple rail scenarios and associate them with the same scenario in the current design.

   Then, run the `set_rail_scenario -generate_file {LEF | TWF | DEF | SPEF | GPD}` command to specify the rail scenario and the type of input files to generate from the design library.

   Note:

   The tool automatically generates the PLOC (Pad Location) file, you do not have to explicitly define it using the `set_rail_scenario -generate_file` command.

4. Use the `-pad_files` option with the `set_rail_scenario` command to specify the required pad files for the rail analysis. For example:

   ```
   set_rail_scenario -pad_files TAPs.ploc
   ```

   The input file for the pad file needs to be in RedHawk format. For example:

   ```
   Tap_146 100 200 M6 VDD
   Tap_148 120 220 M6 VDD
   ```

   Note:

   The `rail.pad_files` application option provides the same functionality, but the `-pad_files` option in `set_rail_scenario` command takes priority if both are specified.

5. Use the `-custom_script_file` option with `set_rail_scenario` to provide your customized python file.

   Note:

   When running rail analysis using multicorner-multimode technology, you must create rail scenarios using the `create_rail_scenario` command before the `set_rail_scenario` command.

   The following example creates the T_low and T_high rail scenarios and generates input files for them:

   ```
   icc2_shell> create_rail_scenario -name T_low \
      -scenario func_cbest
   icc2_shell> set_rail_scenario -name T_low \
      -generate_file { DEF SPEF|GPD TWF LEF} \
      -custom_script_file ./customized.py
   icc2_shell> create_rail_scenario -name T_high \
      -scenario func_cbest
   ```

Feedback

```
icc2_shell> set_rail_scenario -name T_high \
   -generate_file { DEF SPEF|GPD TWF LEF} \
   -custom_script_file ./customized.py
```

You can then use the customized.py file for multi-rail-scenario rail analysis or other IR-driven features, such as IR-driven placement or IR-driven concurrent clock and data optimization.

Examples

The following example uses a customized python script for multi-rail-scenario rail analysis on a grid system.

```
Specify the below options to enable multi-rail-scenario rail analysis.

#Use user-defined test_def_files for DEF variable to match
#with customized python.
set_app_options -name rail.generate_file_variables \
   -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
   -generate_file {PLOC DEF SPEF TWF LEF} \
   -custom_script_file ./customized.py

set_host_options -target Redhawk -submit_protocol sge \
   -submit_command {qsub -V -notify -b y \
   -cwd -j y -P bnormal -l mfree=16G}

analyze_rail -rail_scenario {func.ss_125c func.ff_125c} \
   -submit_to_other_machine
```

The following example uses a customized python script for multi-rail-scenario rail analysis on a local machine.

```
#Required. Specify the below options to enable multi-rail-scenario rail
 analysis.
set_app_options -list {
            rail.enable_new_rail_scenario true
            rail.enable_parallel_run_rail_scenario true
}

#Required. Set the type of the script file to python.
set_app_options -name rail.generate_file_type -value python

#Use user-defined test_def_files for DEF variable to match
#with customized python.
```

Feedback

```
set_app_options -name rail.generate_file_variables \
   -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
   -generate_file {PLOC DEF SPEF TWF LEF} \
   -custom_script_file ./customized.py

set_host_options -target Redhawk  -submit_command {local}

analyze_rail -rail_scenario {func.ss_125c func.ff_125c}
```

The following example uses a customized python script for single-scenario rail analysis on a grid system.

```
Required. Specify the below options to enable multi-rail-scenario rail
 analysis.
set_app_options -list {
            rail.enable_new_rail_scenario true
            rail.enable_parallel_run_rail_scenario true
}

#Required. Set the type of the script file to python.
set_app_options -name rail.generate_file_type -value python

#Use user-defined test_def_files for DEF variable to match
#with customized python.
set_app_options -name rail.generate_file_variables \
   -value {DEF test_def_files }

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
   -generate_file {PLOC DEF SPEF TWF LEF} \
   -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
   -submit_command {qsub -V -notify -b y \
   -cwd -j y -P bnormal -l mfree=16G}

analyze_rail -rail_scenario {func.ff_125c} \
   -submit_to_other_machine
```

The following example uses a customized python script for IR-driven placement on a grid system.

```
Required. Specify the below options to enable multi-rail-scenario rail
 analysis.
set_app_options -list {
            rail.enable_new_rail_scenario true
            rail.enable_parallel_run_rail_scenario true
```

Feedback

```
}

#Required. Set the type of the script file to python.
set_app_options -name rail.generate_file_type -value python

#Use user-defined test_def_files for DEF variable to match
#with customized python.
set_app_options -name rail.generate_file_variables \
   -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
   -generate_file {PLOC DEF SPEF TWF LEF} \
   -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
   -submit_command {qsub -V -notify -b y \
   -cwd -j y -P bnormal -l mfree=16G}
# Enable IR-driven placement before running clock_opt -from final_opto
set_app_options -name opt.common.power_integrity -value true

clock_opt -from final_opto
```

The following example uses a customized python script for concurrent clock and data optimization on a multicorner and multimode design on a grid system.

```
#Required. Specify the below options to enable MCMM-based optimization.
set_app_options -list {
            rail.enable_new_rail_scenario true
            rail.enable_parallel_run_rail_scenario true
}
set_app_options -name rail.generate_file_type -value python

#Use user-defined test_def_files for DEF variable to match
#with customized python
set_app_options -name rail.generate_file_variables \
   -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
   -generate_file {PLOC DEF SPEF TWF LEF} \
   -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
   -submit_command {qsub -V -notify -b y -cwd -j y \
   -P bnormal -l mfree=16G}
```

Feedback

```
# Enable IR-driven CCD analysis before running route_opt
set_app_options -name opt.common.power_integrity -value true
route_opt
```

See Also

- Specifying RedHawk and RedHawk-SC Working Directories

Specifying Ideal Voltage Sources as Taps

Taps are used to model the external voltage source environment in which the device under analysis operates; they are not part of the design itself, but can be thought of as virtual models of voltage sources. The locations of the ideal voltage sources and the ideal power supplies in the design are required to achieve accurate rail analysis results.

To create a tap, use the `create_taps` command. You can create multiple taps by running multiple `create_taps` commands.

To specify a name for the created tap, use the `-name` option. If the specified name is already present in the design, the tool issues a warning message, and replaces the original tap with the new one. When you create multiple taps by a single invocation of the `create_taps` command, all of the created taps are named by using the *tap_name*_NNN naming convention, where NNN is a unique integer identifier and *tap_name* is the string argument to the `-name` option.

You can create taps in any of the following ways:

- Treating top-level PG pins as taps

  Use the `-top_pg` option when the block does not have physical pad or bump information available.

  Use the `-supply_net` option to explicitly associate a tap to the supply (either power or ground) net connected to the object from which the tap was defined. To implicitly associate a tap with the supply net, run the `create_taps` command without the `-supply_net` option.

  The following example creates taps for all the top-level supply pins.

  ```
  icc2_shell> create_taps -supply_net VDD -top_pg
  ```

- Creating taps at absolute coordinates

  Use the `-layer` and `-point` options to create taps at the specified coordinates. If there is no supply net, supply pin, or via shape at the specified location, which means there is no conductive path to the supply network, the tap has no effect on rail analysis despite being present.

Feedback

If you define a tap in a location where a tap already exists, the tool issues a warning message and replaces the original tap with the new one.

You must specify the supply net with which this tap is associated by using the `-supply_net` option.

Note:

When you use the `-point` option with the `-of_objects` option, the location is relative to the origin of the specified object. Otherwise, it is relative to the origin of the current top-level block.

The following example creates a tap using absolute coordinates. The command checks whether the coordinate location touches any layout shape.

```
icc2_shell> create_taps -supply_net VDD \
   -layer 3 -point {100.0 200.0}
Warning: Tap point (100.0, 200.0) on layer 3 for net VDD does not
touch any polygon (RAIL-305)
```

- Using existing instances as taps

  To create taps on specified objects, use the `create_taps -of_objects` command. Each item in the object list can be a cell or a library cell, or a collection of cells and library cells. By default, the tool creates the taps on the power and ground pins of the objects. If you specify the `-point` option, the tool creates the taps at the specified physical location relative to the origin of each object.

  Use existing instances for creating taps when analyzing a top-level design where pad cells are instantiated physically.

  Use the `-supply_net` option to explicitly associate a tap to the supply (either power or ground) net connected to the object from which the tap was defined. To implicitly associate a tap with the supply net, run the `create_taps` command without the `-supply_net` option.

  The following example creates taps for each instance of the some_pad_cell library cell. The tool creates one tap per instance on the metal1 layer at a location of {30.4 16.3} relative to the origin of the cell instance.

  ```
  icc2_shell> create_taps -of_objects [get_lib_cell */some_pad_cell]\
     -layer metal1 -point {30.4 16.3}
  ```

  The following example creates taps on all power and ground pins of all cell instances that match the *power_pad_cell* pattern.

  ```
  icc2_shell> create_taps -of_objects [get_cells -hier \
     *power_pad_cell*]
  ```

- Importing a tap file

Feedback

If you are working with a block that does not yet have pins defined, or if the location or design of the pad cells is not finalized, you can define tap locations, layer numbers, and supply nets in an ASCII file and then import it into the tool by using the `create_taps -import` command.

The supported tap file format is as follows:

```
net_name layer_number [X-coord Y-coord]
```

Lines starting with semicolon (;) or pound (#) symbols are treated as comments. The x-coordinate and y-coordinate values are specified in microns.

For example,

```
icc2_shell> exec cat tap_file
   VDD 3 500.000 500.000
   VSS 5 500.000 500.000
icc2_shell> create_taps -import tap_file
```

- Creating taps with packages

  A simple package RLC (resistance-inductance-capacitance) model provides a fixed set of electrical characteristics assigned to all power and ground pads. To use simple package RLC models for rail analysis, you need to specify the pad location file which defines the simple RLC model by using `rail.pad_files` application option.

  For a detailed description about simple package RLC models, see the related section in the *RedHawk User Manual*.

  Note:

  Before creating taps with simple package RLC models, you must enable the RedHawk signoff license key using the `rail.allow_redhawk_license_checkout` application option.

Validating the Taps and Finding Invalid Taps

When creating taps, by default the `create_taps` command verifies that the created taps are inserted in valid locations. The tool issues a warning message if the tap does not touch any layout shape. To place taps outside a shape without issuing a warning message, disable the checking by using the `-nocheck` option.

The following command creates a tap with the `-nocheck` option enabled. No warning message is issued.

```
icc2_shell> create_taps -supply_net VDD -layer 3 \
   -point {100.0 200.0} -nocheck
```

Feedback

Tap Attributes

During validation checking, the tool marks each inserted tap object with the `is_valid_location` attribute to indicate its validity. When validation checking is enabled, the attribute value is `true` for the valid taps and `false` for the invalid taps. When the `-nocheck` option is used, the attribute value is `unknown` for all taps.

The tool updates the tap attributes in subsequent voltage drop and minimum path resistance analyses. Each tap belonging to the PG nets is checked against the extracted shape. When the analysis is complete, the tool updates the tap validity status based on the rail analysis results.

The following example first lists the attributes of tap objects that are added to the block in this command run, and then checks if the Tap_1437 tap passes the validation checking.

```
icc2_shell> list_attributes -application -class tap
...
Properties:
    A - Application-defined
    U - User-defined
    I - Importable from design/library (for user-defined)
    S - Settable
    B - Subscripted

Attribute Name             Object     Type       Properties  Constraints
-------------------------------------------------------------------------
-------
context                    tap        string     A           integrity,
 rail,
                                                             session
full_name                  tap        string     A
is_valid_location          tap        boolean    A
layer_name                 tap        string     A
layer_number               tap        int        A
net                        tap        string     A
object_class               tap        string     A
parasitics                 tap        string     A
position                   tap        coord      A
static_current             tap        double     A
type                       tap        string     A
 absolute_coord,
auto_pg, cell_coord, cell_pg, lib_cell_coord, lib_cell_pg, signal,
 terminal,
top_pg
icc2_shell> get_attribute [get_taps Tap_1437] is_valid_location
true
```

Finding Invalid Taps

To find taps that fail the validity checking, use the `get_taps` command to report the taps that have `is_valid_location` attribute set to `false`.

Feedback

For example,

```
icc2_shell> get_attribute [ get_taps Tap_1437 ] is_valid_location
false
```

Finding Substitute Locations for Invalid Taps

If the specified location for a tap is invalid, use the `-snap_distance` option of the `create_taps` command to find a substitute location for the tap. The command searches the nearby layout shapes within the specified snap distance both horizontally and vertically on the same layer as defined by the `-layer` option. The tap is reinserted at the corner of a valid shape which is within the minimum distance to the specified location.

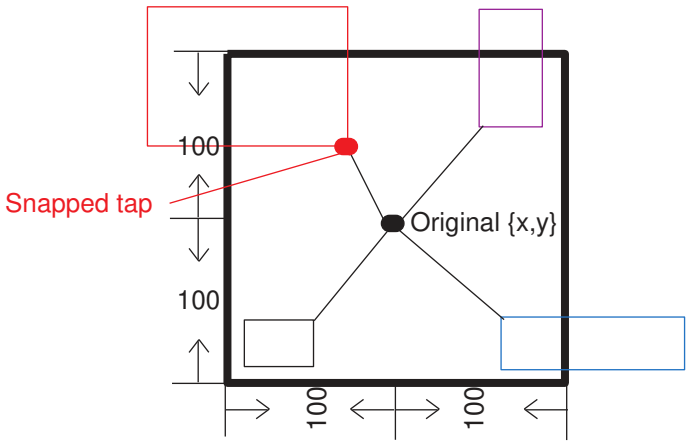
Note:

You can search for a substitute location only for the invalid taps that are specified by the `-point` and `-layer` options. For example,

```
icc2_shell> create_taps -point {50,50} -layer {M1} \
   -snap_distance 100
Snap tap point from original location (50.000, 50.000) to new
 location (90.000, 65.910)
```

As shown in Figure 161, the specified tap location does not touch any layout shape. The red rectangle is the shape closest to the specified location and its lower-right corner has the minimum distance among all the other rectangles. The tool reinserts the tap at the lower-right corner of the red rectangle.

Figure 161 Finding a Substitute Location for an Invalid Tap



Feedback

Removing Taps

To remove taps that were previously created using the `create_taps` command, use the `remove_taps` command. To remove specific taps, use the `-filter` option with the `get_taps` command. The following example removes all the supply pin taps that are of the type top_pg:

```
icc2_shell> remove_taps [get_taps -filter type==top_pg]
```

To remove all the taps in the block, run the following command:

```
icc2_shell> remove_taps
```

Missing Via and Unconnected Pin Checking

Note:

Missing via and unconnected pin checking is supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

Before you check for missing vias and unconnected pins, make sure you have the required input files as described in Preparing Design and Input Data for Rail Analysis.

Check for missing vias or unconnected pins in the block during voltage drop analysis or in a separate run. For example, you can perform the checking after completing the power structure and before running the `place_opt` command. This allows you to find power network errors in the design before rail analysis.

By default, RedHawk/RedHawk-SC Fusion detects the following types of power network errors:

- Unconnected pin shapes: Pin shapes that are not contiguously and physically connected to an ideal voltage source.
- Missing vias: If two overlapping metal shapes do not contain a via within their overlapping area, it is considered a potential missing via error.

To check for and fix missing vias or unconnected pins,

1. Define configuration options by using the `set_missing_via_check_options` command, as described in Setting Checking Options.
2. Save the block by using the `save_block` command.
3. Perform the checking by using the `analyze_rail -check_missing_via` command, as described in Checking Missing Vias and Unconnected Pins. In the RedHawk-SC Fusion flow, you must specify the `-check_missing_via` option together with the `-voltage_drop` option.

Feedback

4. Examine the checking results by opening the generated error files in the error browser or writing the checking results to an output file, as described in Viewing Error Data and Writing Analysis and Checking Reports.
5. Insert PG vias to fix the reported missing vias, as described in Fixing Missing Vias.

Setting Checking Options

Before checking for missing vias or unconnected pins in the design, use the `set_missing_via_check_options` command to specify the necessary options.

To check or remove the option settings, use the `report_missing_via_check_options` or `remove_missing_via_check_options` command, respectively.

Table 63 lists the commonly used options for the `set_missing_via_check_options` command. For a detailed description about each option, see the man page.

Table 63 Commonly Used Options for the set_missing_via_check_options Command

Option	Description
`-redhawk_script_file`	Uses an existing RedHawk script file for the missing via check.
`-exclude_stack_via`	Excludes stacked vias from the missing via check. By default, the tool includes stacked vias during analysis.
`-check_valid_vias`	Reports if the metal overlap can accommodate a valid via model.
`-exclude_cell` / `-exclude_instance`	Excludes the area covered by all instances of the specified cell, or by a list of cell instances from the missing via check. By default, the tool performs checking at the full-chip level.
`-exclude_regions`	Specifies rectangular regions to exclude from the missing via check.
`-sort_by_hot_inst`	Sorts missing vias by voltage drop values. By default, the tool sorts by the average delta voltage between defined layers.
`-threshold`	Sets a voltage threshold that triggers a missing via check. Set the option to -1 to disable voltage check filtering. The default is 1 V. This option is valid only when the checking is run during voltage drop analysis.
`-min_width`	Ignores segments smaller than the specified width.
`-pitch`	Specifies the pitch for missing vias on parallel wires crossing or touching GDSII blocks.

Feedback

Table 63 Commonly Used Options for the set_missing_via_check_options Command (Continued)

Option	Description
`-gds`	Flags missing vias that are inside the GDSII block region and are in routes across the GDSII blocks. By default, the missing via check ignores items that are inside or overlap a block that has been processed by the GDS2DEF or GDSMMX utility.
`-ignore_inter_met`, `-toplayer`, `-bottomlayer`	Ignores wires on all layers between the specified top and bottom layers.

Checking Missing Vias and Unconnected Pins

Use the `analyze_rail -check_missing_via` command to check for missing vias and unconnected pins in the block based on the settings specified by the `set_missing_via_check_options` command.

Use the `-voltage_drop` option if you want to run missing via and unconnected pin checks together with voltage drop analysis. The tool then reports all missing vias found in the geometry with voltage values in the missing via report. To set a voltage threshold across layers for filtering the report, use the `-threshold` option of the `set_missing_via_check_options` command. You can set multiple threshold values in a missing via check.

Note:

In the RedHawk-SC Fusion flow, you must specify the `-check_missing_via` option together with the `-voltage_drop` option.

In cases you want to report missing vias with and without setting a voltage threshold in one `analyze_rail` run, first set the desired threshold value (for example, 0.001) and then reset the value to -1. Setting `-threshold` option to -1 disables voltage checking.

When the missing via check is completed, open the generated error data to examine the errors in the error browser. For details, see Viewing Error Data.

Example 46

```
icc2_shell> set_missing_via_check_options -exclude_stack_via \
   -threshold 0.0001
icc2_shell> analyze_rail -voltage_drop static -check_missing_via \
   -nets {VDD VSS}
```

The above example sets the threshold value to 0.0001 and compares voltages across two ends of a via, excluding stack vias. It then runs the missing via check during static rail analysis.

Feedback

The RedHawk script file includes the following commands:

```
perform extraction -power -ground
perform analysis -static
mesh vias -report_missing -exclude_stack_via -threshold 0.0001 \
  -o apache.missingVias1
```

Example 47

```
icc2_shell> set_missing_via_check_options -exclude_stack_via \
   -threshold -1
icc2_shell> analyze_rail -voltage_drop -check_missing_via \
   -nets {VDD VSS}
```

The above example sets the threshold value to -1 to disable the voltage checking, and runs the missing via check.

The RedHawk script file includes the following commands:

```
perform extraction -power -ground
mesh vias -report_missing -exclude_stack_via -threshold -1 \
 -o apache.missingVias1.nothreshold
```

Example 48

```
icc2_shell> set_missing_via_check_options -exclude_stack_via \
   -threshold 0.001
icc2_shell> set_missing_via_check_options -exclude_stack_via \
   -threshold 0.002
icc2_shell> set_missing_via_check_options -exclude_stack_via \
   -threshold -1
icc2_shell> analyze_rail -voltage_drop static -check_missing_via \
   -nets {VDD VSS}
icc2_shell> report_rail_result -type missing_vias -supply_nets \
   {VDD VSS} rpt.missing_vias
  Information: writing rpt.missing_vias.no_threshold done
  Information: writing rpt.missing_vias done
```

In the above example, three different threshold values are set for a missing via check. The tool generates two missing via reports: rpt.missing_vias.no_threshold includes all missing vias found in the geometry, while rpt.missing_vias includes the vias whose voltage difference across two ends is less than 0.001 and 0.002.

Viewing Error Data

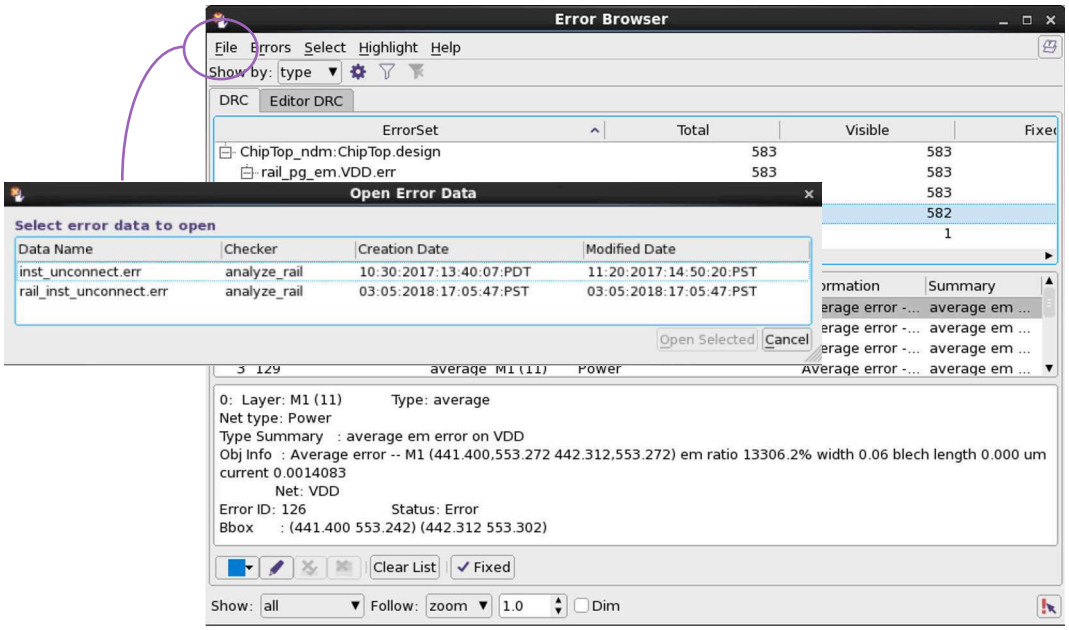
RedHawk or RedHawk-SC Fusion saves the checking error data file, named missing_via.*supplyNetName*.err, in the working directory. The tool generates one error data file for each supply net. Before you quit the current session, be sure to save the block to keep the generated error data in the block.

Feedback

To open an error data file in the error browser, choose File > Open ... from the Error Browser dialog box, and then select the error data file to open in the Open Error Data dialog box (see Figure 162).

For more information about the error browser, see “Using the Error Browser” in the *IC Compiler II Graphical User Interface User Guide*.

Figure 162 Opening Error Data from the Error Browser



See Also

- Specifying RedHawk and RedHawk-SC Working Directories

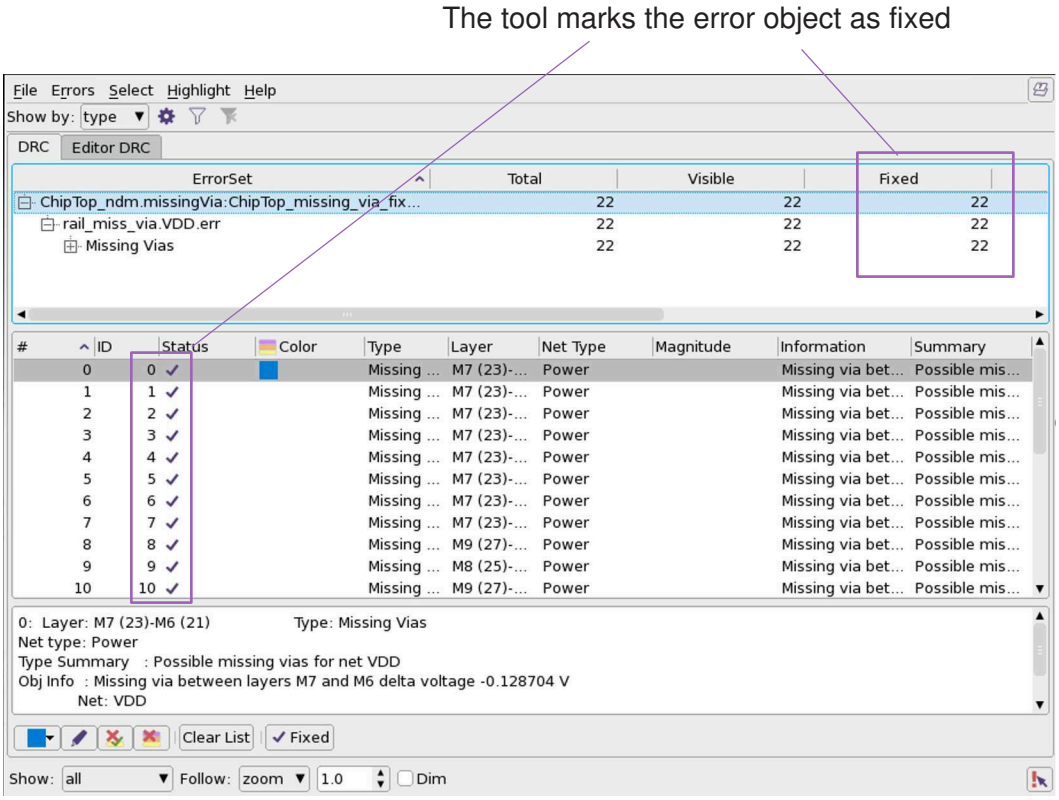
Fixing Missing Vias

To fix the missing vias by inserting PG vias based on the specified DRC error objects written by the `analyze_rail` command, use the `fix_pg_missing_vias` command.

When the tool inserts a PG via for a missing via error object, it sets the status of the error object to `fixed` in the error browser (see Figure 163). This allows you to quickly check if all the missing vias are fixed.

Feedback

Figure 163 Error Status After Inserting PG Vias for Missing Vias



The following example inserts PG vias for all error objects of the VDD nets reported by the `analyze_rail -check_missing_via` command:

```
icc2_shell> set errdm [open_drc_error_data rail_miss_via.VDD.err]
icc2_shell> set errs [get_drc_errors -error_data $errdm]
icc2_shell> fix_pg_missing_vias -error_data $errdm $errs
```

Running Rail Analysis with Multiple Rail Scenarios

To control scenarios for RedHawk Fusion or RedHawk-SC Fusion rail analysis:

- Use the `rail.scenario_name` application option to specify a different design scenario for rail analysis.

  For details, see Specifying a Design Scenario for Rail Analysis.

Feedback

- You can create rail scenarios and associate them with a design scenario in the current design. This allows you to identify possible power integrity issues by running optimization and rail analysis on one or more rail scenarios.

  For details, see Creating and Specifying Rail Scenarios for Rail Analysis.

Specifying a Design Scenario for Rail Analysis

By default, RedHawk or RedHawk-SC Fusion analyzes only the current design scenario for a multicorner-multimode design. To specify a different design scenario for rail analysis, use the `rail.scenario_name` application option. The tool honors the specified design scenario for the IR-driven features in power integrity flow, such as IR-driven placement, IR-driven concurrent clock and data optimization, and IR-driven optimization.

Here is a script example:

```
open_lib
open_block

set_app_option -name rail.technology -value 7
set_app_options -name rail.tech_file -value  ./Apache.tech
set_app_options -name rail.lib_files -value {\
   A.lib \
   B.lib }
set_app_options -name rail.switch_model_files -value
 {./switch_cells.txt }
source ./test_taps.tcl

current_scenario func_max
set_app_options -name rail.scenario_name -value func_typ

 analyze_rail -voltage_drop
```

Creating and Specifying Rail Scenarios for Rail Analysis

You can create multiple rail scenarios and associate them with a design scenario in the current design. The created rail scenarios are saved in the design library.

To run rail analysis on multiple rail scenarios:

1. Specify the following application options to enable multi-rail-scenario rail analysis:

```
icc2_shell> set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
```

Feedback

2. Run the `set_scenario_status -ir_drop` command to enable rail analysis for the specified design scenario.

```
icc2_shell> set_scenario_status func_cbest -ir_drop true
```

3. Create a rail scenario and associate it with an existing design scenario. Specify rail attributes for the created rail scenario with the `set_rail_scenario` command. You can create multiple rail scenarios for different purposes and associate them with an existing design scenario.

```
icc2_shell> create_rail_scenario -name T_low \
   -scenario func_cbest
icc2_shell> set_rail_scenario -name T_low \
   -voltage_drop dynamic -extra_gsr_option_file extra.gsr \
   -nets {VDD VSS}
icc2_shell> create_rail_scenario -name T_high \
   -scenario func_cbest
icc2_shell> set_rail_scenario -name T_high \
   -voltage_drop dynamic -extra_gsr_option_file extra2.gsr \
   -nets {VDD VSS}
```

   To create Timing Window file (TWF) and Instance Power file (IPF), use the `-twf_generation` and `-ipf_generation` options respectively, with the `set_rail_scenario` command. The values for these commands are `icc2` and `pt`. Use `icc2` to create the files using the IC Compiler II tool. Use `pt` as the input value to create the files using `pt_shell` and PrimePower.

   During IPF generation using PrimePower tool, you can use a PrimePower script file to identify the valid commands in the script and write them out in the `post_script_file`. This option can only be used along with the `-ipf_generation <pt>` option.

4. Specify the multicore processing setting with the `set_host_options` command. For example, to run rail analysis on a local host, use the following command:

```
icc2_shell> set_host_options -submit_command {local}
```

   To run analysis on a grid system, use the following command:

```
icc2_shell> set_host_options -submit_protocol sge \
   -submit_command {qsub -V -b y -cwd -P bnormal -l mfree=16G}
```

5. Run rail analysis with the `analyze_rail` command.

```
icc2_shell> analyze_rail -rail_scenario {T_high T_low} \
   -submit_to_other_machines
```

Note:

   To use the multi-rail-scenario rail analysis capability for IR-driven placement or IR-driven concurrent clock and data optimization, you must set the following

Feedback

application option before the `clock_opt -from final_opto` or `route_opt` command:

```
icc2_shell> set_app_options \
   -name opt.common.power_integrity -value true
```

For more information about power integrity analysis, see the Enabling the Power Integrity Features section.

Examples

The following example runs rail analysis on multiple rail scenarios.

```
# Specify options for multi-rail-scenario rail analysis
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y -cwd -j y \
    -P bnormal -l mfree=16G}

# Custom RedHawk-SC Fusion python script to specify user-defined
# toggle rate for scenario T_custom
create_rail_scenario -name T_custom -scenario func_cbest
set_rail_scenario -name T_custom -custom_script_file custom_sc.py

# Default RedHawk-SC Fusion toggle rate for scenario T_def
create_rail_scenario -name T_def -scenario func_cbest
set_rail_scenario -name T_def -voltage_drop dynamic -nets {VDD VSS}

# Run RedHawk-SC Fusion rail analysis on the specified rail scenario
analyze_rail -rail_scenario {T_custom T_def} -submit_to_other_machines

# (Optional) Open and examine the generated rail results
open_rail_result {T_custom T_def}
```

The following example runs multi-rail-scenario rail analysis in the power integrity flow.

```
## Specify RedHawk/RedHawk-SC based options
set_app_options -name rail.enable_redhawk -value true
set_app_options -name rail.enable_redhawk_sc -value false
set_app_options -name rail.redhawk_path -value /test/bin
set_app_options -name rail.tech_file -value design_data/tech/rh.tech
set_app_options -name rail.lib_files -value {
inputs/test/CCSP_test.lib
}
set_host_options -submit_protocol sge -submit_command {qsub -V \
   -notify -b y -cwd -j y -P bnormal -l mfree=16G}

## Specify options for running optimization  on multiple rail scenarios
set_app_options -list {
```

Feedback

```
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
## Enable IR drop flag for a given scenario
set_scenario_status [current_scenario] -ir_drop true

## Create a rail scenario and specify associated options
create_rail_scenario -name T_low -scenario func_cbest
set_rail_scenario -name T_low -voltage_drop dynamic \
   -extra_gsr_option_file extra.gsr -nets {VDD VSS}

create_rail_scenario -name T_high -scenario func_cbest
set_rail_scenario -name T_high -voltage_drop dynamic \
   -extra_gsr_option_file extra2.gsr -nets {VDD VSS}

## Enable power integrity flow
set_app_options -name opt.common.power_integrity -value true
```

Performing Voltage Drop Analysis

Note:

Before running the `analyze_rail` command to calculate voltage drops and current violations, ensure you have provided the input files required by each analysis mode.

For more information about preparing input files, see Preparing Design and Input Data for Rail Analysis.

Voltage drop analysis is supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

To invoke the RedHawk/RedHawk-SC tool for calculating voltage drops of the specified power and ground networks, use the `analyze_rail` command with the following options:

- Use the `-voltage_drop` option to specify the analysis mode.
  - `static`: Performs static voltage drop analysis.
  - `dynamic`: Performs dynamic voltage drop analysis.
  - `dynamic_vectorless`: Performs dynamic voltage drop analysis without Verilog Change Dump (VCD) inputs.
  - `dynamic_vcd`: Performs dynamic voltage drop analysis with VCD inputs. When enabled, use the `-switching_activity` option to specify a switching activity file.
- Use the `-nets` option to specify the power and ground supply nets to analyze. The tool considers all the switched or internal power nets of the specified power nets in the analysis. You do not need to explicitly specify the internal power nets.

Feedback

- (Optional) Use the `-switching_activity` option to specify a switching activity file. The supported file formats are: `VCD`, `SAIF`, and `ICC_ACTIVITY`. When not specified, by default the tool uses the toggle rate for rail analysis.

  Note:

  If you are using the dynamic vector-free mode, you do not need to provide switching activity information.

  To specify a switching activity file, use the `-switching_activity` option by using the following syntax:

  ```
  -switching_activity {type file_name [strip_path] [start_time
     end_time]}
  ```

  - `VCD`: Specifies a VCD file that is generated from a gate-level simulation. By default, rail analysis reads and uses all time values in the VCD file.

    To specify the time window to read from the VCD file, use the optional *start_time* and *end_time* arguments with the `-switching_activity` option. When specified, the tool reads all time values from the VCD file, but uses only the specified time window for rail analysis.

  - `SAIF`: Specifies one or more SAIF files generated from a gate-level simulation.

  - `icc_activity`: Runs the IC Compiler II `write_saif` command to generate a SAIF file that contains the user-annotated and simulated switching activity (including state-dependent and path-dependent information) for the complete design without running the propagation engine.

  The *strip_path* argument removes the specified string from the beginning of the object names. Using this option modifies the object names in the VCD or SAIF file to make them compatible with the object names in the block. The *strip_path* argument is case-sensitive.

- (Optional)Use the `-generate_file` option to generate the Timing Window File (TWF) and the Instance Power File (IPF) files, which are necessary input collateral for the RedHawk fusion flow. The file extensions are `.sta` and `.pwr` respectively. By default, these two files are generated by the IC Compiler II tool.

  You can also opt to generate the TWF and IPF files using either the IC Compiler II tool or the PrimePower tool.

  - To generate the TWF file, use the `-twf_generation <icc2 | pt>` option. If you provide, `icc2` as the value of this option, the TWF file is generated by the IC Compiler II tool. If you provide `pt` as the value, the TWF file is generated by the PrimePower tool via `pt_shell`.

  - To generate the IPF file, use the `-ipf_generation <icc2 | pt>` option. You can provide `icc2` or `pt` as the value to generate the IPF file using the IC Compiler II tool or the PrimePower tool respectively.

    Note:
    You can use the `-twf_generation` and `-ipf_generation` options independent of each other. If you do not specify either of these options, the RedHawk tool honors the default `icc2` option.

- (Optional) Use the `-primepower_script_file` option to pass a script file for PrimePower and the tool identifies valid commands in the script and writes them out in the`post_script_file`.

  Note:
  This option is only for the in-design PrimePower flow, and thus works only with the `-ipf_generation pt option`.

  For example,

  ```
  analyze_rail -voltage_drop {VDD VSS} -ipf_generation pt
   -primepower_script_file <script_file>
  ```

- (Optional) By default, the `analyze_rail` command automatically generates a GSR file for running RedHawk rail analysis. To specify an external GSR file to append to the GSR file generated in the current run, use the `-extra_gsr_option_file` option.
- (Optional) Use the `-redhawk_script_file` option to specify an existing RedHawk script file generated in an earlier `analyze_rail` run.

  When using an existing script file for rail analysis, the tool honors the settings in this file and ignores all other currently enabled options. For example, if you run the `analyze_rail -voltage_drop static -redhawk_script_file myscript.tcl` command, the tool ignores the `-voltage_drop` option setting.

  When the `-redhawk_script_file` option is not specified, you must specify the nets to analyze by using the `-nets` option.

The following example performs static voltage drop analysis on the VDD and VSS nets with the optional settings defined in a GSR configuration file.

```
icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS} \
   -extra_gsr_option_file add_opt.gsr
```

Viewing Voltage Drop Analysis Results

When analysis is complete, the tool saves the analysis results and logs in the working directory. For more information about the working directories, see Specifying RedHawk and RedHawk-SC Working Directories.

Feedback

The `analyze_rail` command generates the following files to invoke the RedHawk tool for running rail analysis:

- The RedHawk run script (analyze_rail.tcl)
- The RedHawk GSR configuration file (*.gsr)

The `analyze_rail` command generates the following file to invoke the RedHawk-SC tool for running rail analysis:

- The RedHawk-SC Python script (analyze_rail.py)

The RedHawk or RedHawk-SC tool generates the following files in the in-design.redhawk/*design_name*.result (or in-design.redhawk_sc/*design_name*.result in RedHawk-SC flow) directory when rail analysis is complete:

- The RedHawk analysis log (analyze_rail.log.*)
- Timing window file (*.sta)
- Signal SPEF files (*.spef)
- DEF/LEF files (*.def, *.lef) for the full-chip analysis

When voltage drop analysis is complete, the tool saves the analysis results (*.result) in the *design_name* directory under the working directory. The rail analysis results are used to display maps and query attributes to determine the problem areas with large voltage drops.

- To reload the rail results for displaying maps in the GUI, use the `open_rail_result` command.

  For example,

  ```
  icc2_shell> open_rail_result
  REDHAWK_RESULT
  ```

  For more information about displaying maps, see Displaying Maps in GUI.

- To write out the rail analysis results, use the `report_rail_result` command. You must specify the result type with the `-type` option. For example, to write out the voltage drop values for the current block, set the `-type` option with the voltage_drop_or_rise argument.

  For more information, see Writing Analysis and Checking Reports.

- To query the results stored in the current rail results cache, use the `get_attribute` commands.

  For more information, see Querying Attributes.

Note:

RedHawk Fusion does not support results that are generated by the RedHawk tool outside of the IC Compiler II environment. The RedHawk `dump icc2_result` command works only within the IC Compiler II session.

Performing Sigma Analysis

Sigma Technology revolutionizes Dynamic Voltage Drop (DVD) Analysis by enabling early detection and resolution of design weaknesses, even with preliminary and noisy data such as non-final placement or incomplete static timing analysis (STA). It minimizes late-stage engineering change orders (ECOs), reduces design effort, and ensures higher-quality, robust, and efficient integrated circuits.

Sigma Technology integrates seamlessly with add-ons like Sigma Placement Directives (SigmaPD) and Sigma Analysis View (SigmaAV), fundamentally transforming PG analysis.

Note:

Cells impacted by IR-drop are termed victims, while the cells causing the power draw are termed aggressors.

- SigmaDvD: Computes local noise from the simultaneous switching of nearby cells, along with the self-drop caused by the victim cell itself. It does not replace transient simulation but identifies design-related issues that leads to a large number of DVD failures.
- SigmaPD: Extends the analysis of SigmaDvD by incorporating spacing rule recommendations to mitigate IR-drop. These rules ensure proper spacing between components, improving the power delivery network and reducing resistance-related voltage drops.
- SigmaAV: Directly correlates with IR-drop by identifying areas in the design where average voltage levels are significantly impacted. It highlights critical regions that require targeted mitigation, enabling to address IR-drop issues more effectively by focusing on the most affected areas.

Note:

Simultaneous analysis of the following are not supported:

- Sigma rail scenarios and non-sigma rail scenarios.
- Different sigma rail scenarios.

Feedback

The following are the necessary steps to perform voltage drop rail analysis:

- To perform voltage drop rail analysis, you can use the `analyze_rail` command with the following options:

  Note:
  Sigma flow cannot support the input files generated by the `-ipf_generation` and `-twf_generation` options.

  - `-voltage_drop {sigma_dvd | sigma_pd | sigma_av}`: specifies the type of voltage drop analysis to be performed.
  - `-nets <net_list>`: Specifies the power and ground nets to analyze (For example, `VDD`, `VSS`).

  Use the following syntax:

  ```
  analyze_rail -voltage_drop {sigma_dvd | sigma_pd | sigma_av} -nets
  <net_list>
  ```

  For example:

  ```
  analyze_rail -voltage_drop sigma_dvd -nets {VDD VSS}
  ```

- The results are saved in the rail database and are inspected using the `open_rail_result -voltage_drop {sigma_dvd | sigma_pd | sigma_av}` command. For example:

  ```
  open_rail_result -voltage_drop sigma_av
  ```

  You can specify the path to the report file using the `rail.result_file` application option before invoking the `open_rail_result` command.

  Note:

  You cannot load multiple text reports as input.

  Use the following syntax:

  ```
  set_app_options -name rail.result_file -value <RESULT_FILE>

  open_rail_result -voltage_drop {sigma_dvd | sigma_av}
  ```

  For example:

  ```
  set_app_options -name rail.result_file -value
   GOLD/dump/GPU_TOP_DG.sav_drop.rpt

  open_rail_result -voltage_drop sigma_av
  ```

Feedback

- You can configure rail scenario for voltage drop analysis using the `set_rail_scenario -voltage_drop {sigma_dvd | sigma_pd | sigma_av}` command.
- You can also configure the following application options to customize the analysis:
  - `rail.sigma_dvd_top_n_victims`: Specifies the maximum number of victim cells to be printed during dynamic voltage drop (SigmaDvD) analysis.

    The default is `1000000`, you can also reduce or increase this value to focus on a smaller or bigger number. For example:

    ```
    set_app_options -name rail.sigma_dvd_top_n_victims -value 10000
    ```

  - `rail.sigma_av_drop_threshold`: Specifies a threshold value for average voltage drop (SigmaAV) analysis to identify and report areas of the design where the average voltage drop exceeds the specified value. The default is `""`, meaning empty string, that is, no threshold is set by default.

    You can specify a threshold value to filter out less significant voltage drops and focus on critical violations. For example:

    ```
    set_app_options -name rail.sigma_av_drop_threshold -value {VDD VSS 0.02}
    ```

Performing PG Electromigration Analysis

Note:
PG electromigration analysis is supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

To analyze the current density on PG nets and identify the segments with potential electromigration issues, run the `analyze_rail -electromigration` command. The analysis results are saved in the working directory.

To perform electromigration analysis,

1. Set the `rail.tech_file` application option to specify the Apache technology file (*.tech) that defines the layer-by-layer current density limits.

   For example,

   ```
   icc2_shell> set_app_options -name rail.tech_file \
      -value design_data/tech/test.tech
   ```

Feedback

2. Run the `analyze_rail` command with the following options:
   - Use the `-voltage_drop` and `-electromigration` options to enable electromigration analysis.
   - Use the `-nets` option to specify the power and ground supply nets to analyze. The tool considers all the switched or internal power nets of the specified power nets in the analysis. You do not need to explicitly specify the internal power nets.
   - (Optional) By default, the `analyze_rail` command automatically generates a GSR file for running RedHawk rail analysis. To specify an external GSR file to append to the GSR file generated in the current run, use the `-extra_gsr_option_file` option.
   - (Optional) To specify an existing RedHawk script file that was generated in an earlier `analyze_rail` run, use the `-redhawk_script_file` option.

     When using an existing script file for rail analysis, the tool honors the settings in this script file and ignores all other currently enabled options. For example, if you run the `analyze_rail -voltage_drop static -redhawk_script_file myscript.tcl` command, the tool ignores the `-voltage_drop` option.

     When the `-redhawk_script_file` option is not specified, you must specify the nets to analyze by using the `-nets` option.
3. Check the analysis results, as described in Viewing PG Electromigration Analysis Results.

Viewing PG Electromigration Analysis Results

When PG electromigration analysis is complete, you can check for current violations by displaying a PG electromigration map or checking the generated errors in the error browser.

- Displaying the PG Electromigration Map
- Checking PG Electromigration Violations

Displaying the PG Electromigration Map

A PG electromigration map is a visual display of the color-coded electromigration values overlaid on the physical supply nets.

For static analysis, the map displays average electromigration values. For dynamic analysis, the map displays average electromigration values, peak electromigration values, or root mean square electromigration values.

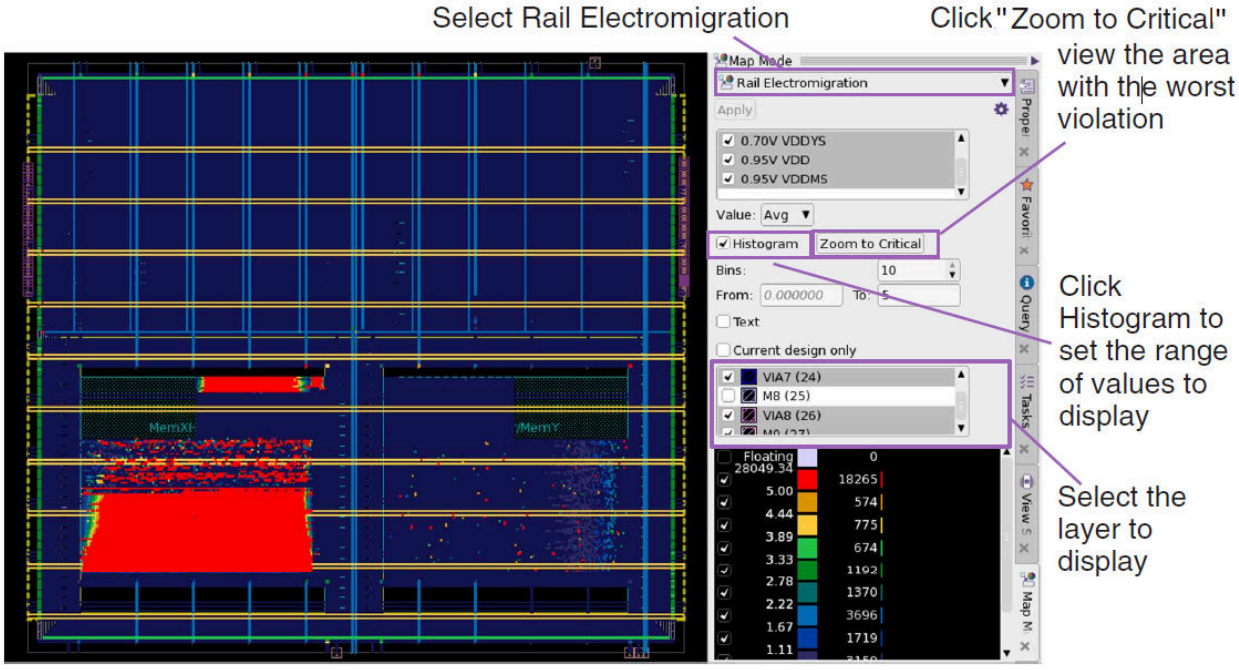
Figure 164 shows an example of the electromigration map in which problem areas are highlighted in different colors. Use the options to investigate the problem areas by

Feedback

selecting layers or nets to display. For example, to examine the current value of one specific net, deselect all the nets and then select the net to display from the list. To analyze only the shapes on a layer, select the layer from the list.

For a detailed procedure about how to display an electromigration map, see Displaying Maps in GUI.

Figure 164 Displaying a PG Electromigration Map



Checking PG Electromigration Violations

When PG electromigration analysis is complete, you can check the generated error files in the error browser or write the generated errors to an output file.

The tool does not write the generated errors to the working directory, meaning the error data is deleted if you exit the tool without saving the block. Ensure that you save the block to keep the generated error data if you want to examine the errors in another session.

Feedback

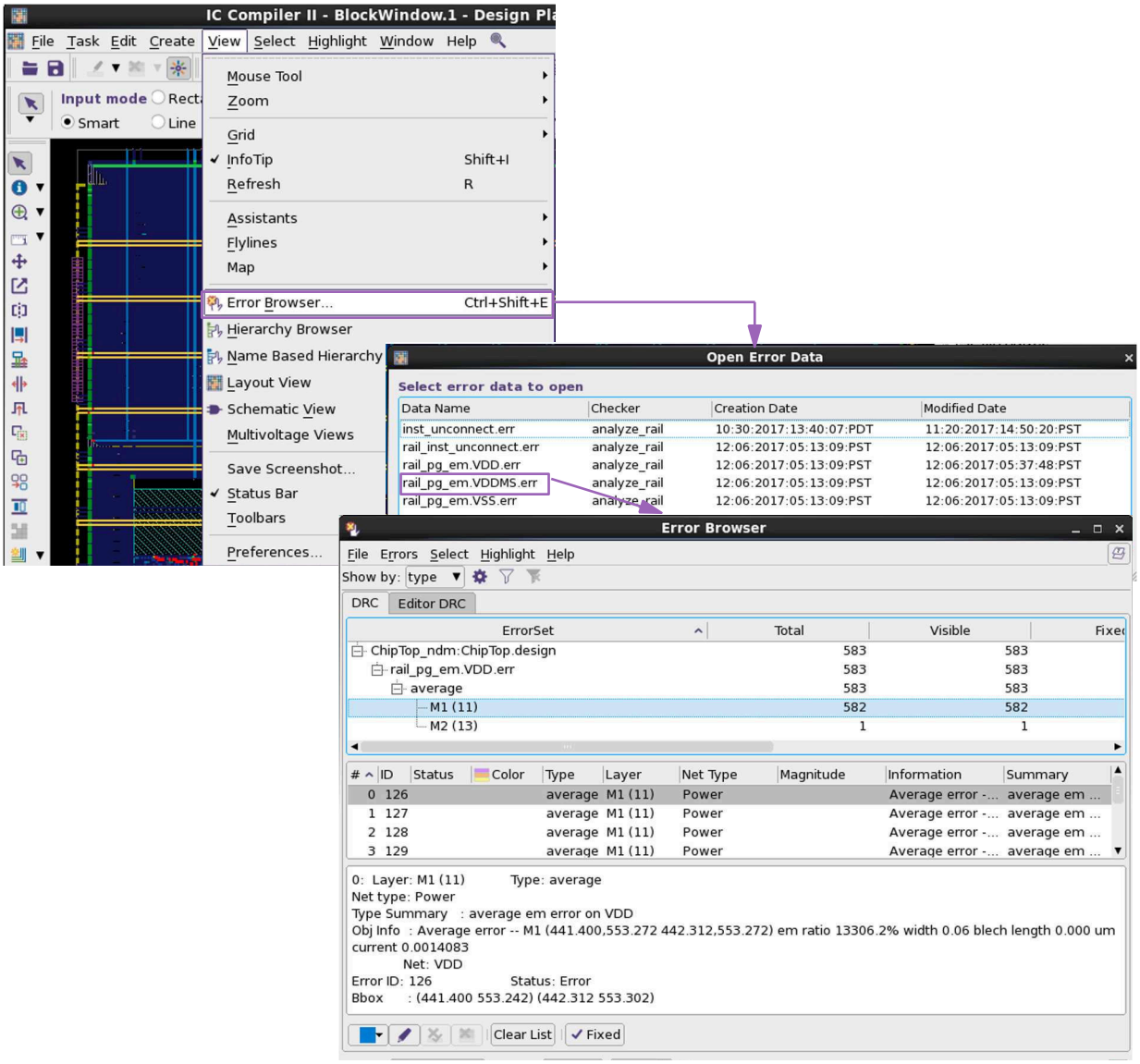
You can examine the generated error files in the error browser or write the generated errors to an output ASCII file.

- To examine the generated errors in the error browser,

  1. Choose View > Error Browser from the GUI.
  2. In the Open Error Data dialog box, select the errors to examine and click Open Selected.
  3. Check the details of the errors in the Error Browser (see Figure 165).

Feedback

Figure 165 Opening Generated Errors in the Error Browser



- To write errors to an ASCII file, use the following script:

```
set fileName "errors.txt"
set fd [open $fileName w]
set dm [open_drc_error_data rail_pg_em.VDD.err]
set all_errs [get_drc_errors -error_data $dm *]
foreach_in_collection err $all_errs {
   set info [ get_attribute $err error_info];
   puts $fd $info }
close $fd
```

Feedback

Here is an example of the output error file:

```
M1 (414.000,546.584 414.344,546.584) em ratio 23585.9% width 0.06
 blech
 length 0.000 um current 0.0024963
M1 (414.344,546.584 416.168,546.584) em ratio 23401.3% width 0.06
 blech
 length 0.000 um current 0.0024768
M1 (416.168,546.584 416.776,546.584) em ratio 23241.7% width 0.06
 blech
 length 0.000 um current 0.0024599
M1 (416.776,546.584 417.992,546.584) em ratio 23082.2% width 0.06
 blech
 length 0.000 um current 0.0024430
M1 (417.992,546.584 419.208,546.584) em ratio 22897.7% width 0.06
 blech
 length 0.000 um current 0.0024235
M1 (419.208,546.584 419.512,546.584) em ratio 22731.1% width 0.06
 blech
 length 0.000 um current 0.0024059
M1 (419.512,546.584 421.184,546.584) em ratio 22564.4% width 0.06
 blech
 length 0.000 um current 0.0023882
M1 (421.184,546.584 421.488,546.584) em ratio 22349% width 0.06 blech
 length 0.000 um current 0.0023654
M1 (421.488,546.584 423.160,546.584) em ratio 22133.5% width 0.06
 blech
 length 0.000 um current 0.0023426
M1 (423.160,546.584 423.616,546.584) em ratio 21967.2% width 0.06
 blech
 length 0.000 um current 0.0023250
```

Performing Minimum Path Resistance Analysis

Note:

Minimum path resistance analysis is supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

Perform minimum path resistance analysis to quickly detect power and ground network weaknesses in the design. The minimum path resistance value of a node is the resistance value on the smallest resistive path to the ideal voltage source locations (taps). The ideal voltage sources can be power or ground pins, user-defined taps, or packages. Because this value represents only the resistance on a single path, it is not necessarily same as the effective resistance value from the node to the supply or ground; instead, it represents an upper bound on the effective resistance of a node to the supply or ground.

Feedback

To perform minimum path resistance analysis, run the `analyze_rail -min_path_resistance` command. You must specify the power and ground nets to analyze by using the `-nets` option.

Note:

In RedHawk-SC rail analysis flow, you must specify both the `-voltage_drop` and `-min_path_resistance` options for minimum path resistance calculation.

You can perform minimum path resistance analysis before or during voltage drop analysis. When the analysis is completed, the tool writes the minimum path resistance values and the voltage drop results into the same rail result.

The following example performs voltage drop analysis and minimum path resistance calculation at the same time.

```
icc2_shell> analyze_rail -nets {VDD VSS} \
   -min_path_resistance -voltage_drop static
```

The following example performs only minimum path resistance analysis.

```
icc2_shell> analyze_rail -nets {VDD VSS} -min_path_resistance
```

Viewing Minimum Path Resistance Analysis Results

When the analysis is complete, run the `report_rail_result -type minimum_path_resistance` command to display a minimum path resistance map, write the results to an output file, or query the attributes on the cell instances or pins.

The following example writes the calculated minimum path resistances for the VDD net to an output file called minres.rpt.

```
icc2_shell> open_rail_result
icc2_shell> report_rail_result -type minimum_path_resistance \
   -supply_nets { vdd } minres.rpt
icc2_shell> sh cat minres.rpt
FEED0_14/vbp 518.668
INV61/vdd 158.268
ARCR0_1/vbp 145.582
FEED0_5/vdd 156.228
INV11/vbp 518.669
```

The output minres.rpt report includes the entire resistance information.

```
icc2_shell> sh cat minres.rpt
FEED0_14/vbp 518.668
INV61/vdd 158.268
ARCR0_1/vbp 145.582
FEED0_5/vdd 156.228
INV11/vbp 518.669
```

Feedback

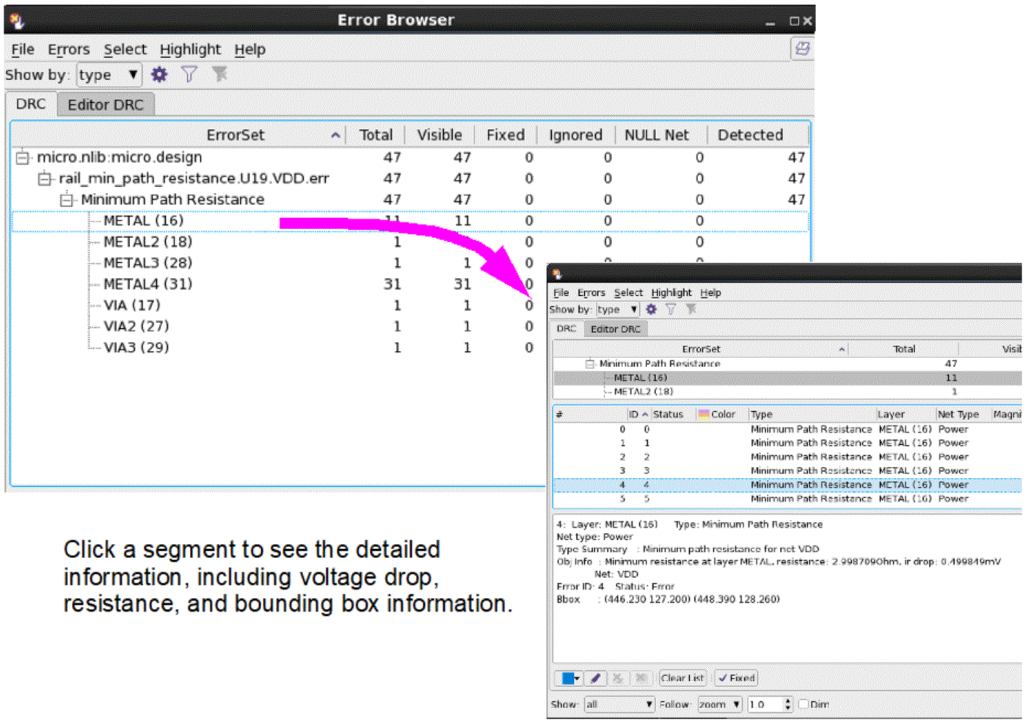
To write out a detailed minimum resistance path report for a cell instance, run the `report_rail_minimum_path` command and specify the net and the cell to report. To see the segment-by-segment resistance of that minimum resistance path shown in the GUI, use the `-error_cell` option of the `report_rail_minimum_path` command to save the geometries on the path of the minimum path resistance to an error cell. You can then open the error cell in an error browser and check the minimum resistance path from the pin of the cell to the nearest tap and trace each go-through geometries.

In the following example, all geometries on the minimum resistance path are traced and written to the error cell named test.err.

```
icc2_shell> open_rail_result
icc2_shell> report_rail_minimum_path -cell U19 -net VDD -error_cell
```

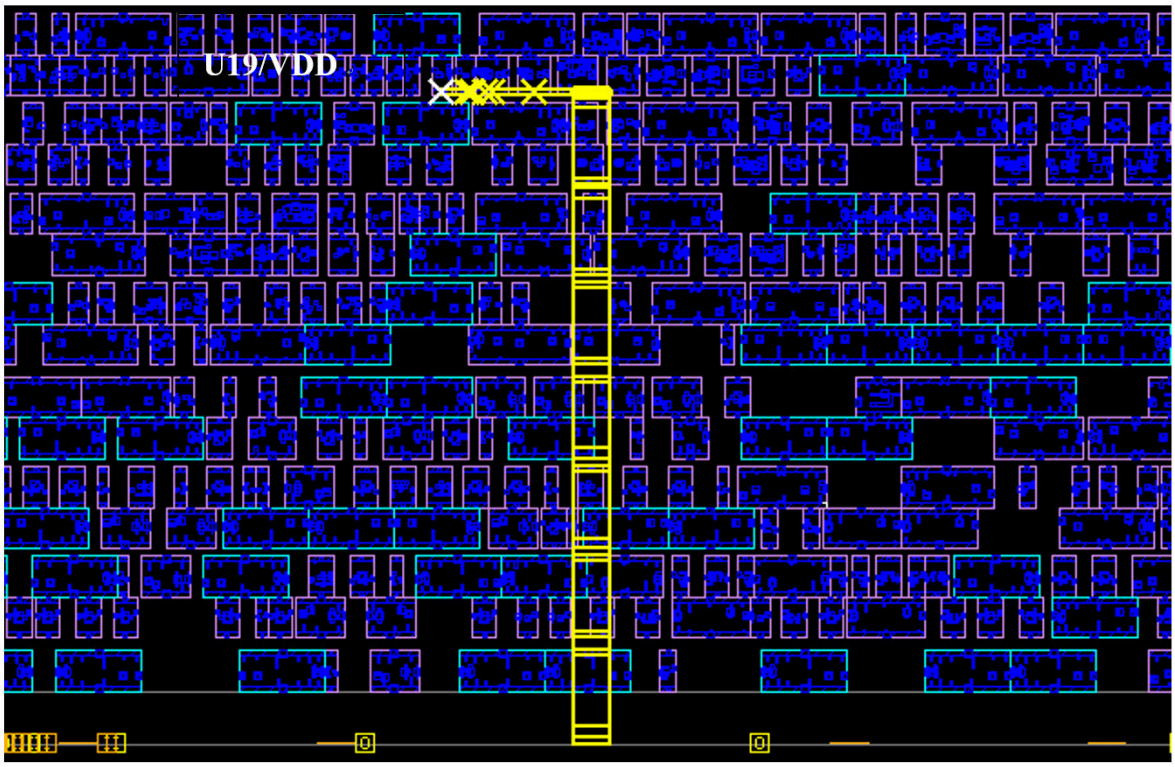
Shown in Figure 166, each geometry is classified by layer in the error browser. Click a geometry to check for detailed information, such as IR drop, resistance, bounding box, and so on. In the GUI, the traced path is highlighted in yellow in the design layout (see Figure 167).

Figure 166 Displaying Error Cells in the Error Browser



Feedback

Figure 167 Traced Segment is Highlighted in the Design Layout



See Also

- Displaying Maps in GUI
- Writing Analysis and Checking Reports
- Querying Attributes

Tracing Minimum Path Resistance Using the Mouse Tool

Use the minimum path resistance (MPR) mouse tool to interactively trace the path with the least resistance from the specified power or ground net to the boundary nodes in the design.

To trace the calculated minimum path resistance with the MPR interactive tracing tool,

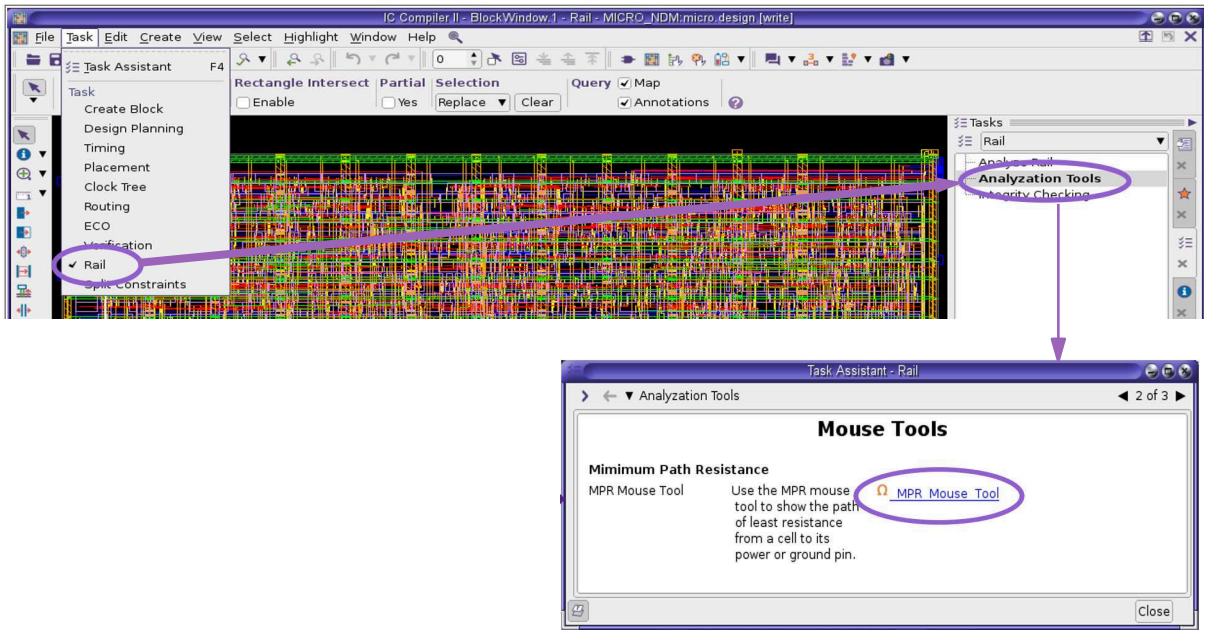
1. In the GUI, choose Task > Rail and then select Analysis Tools from the task list to open the Task Assistant window (see Figure 168).
2. In the Task Assistant - Rail window that opens, click MPR Mouse Tool.
3. Specify the nets to analyze. Click OK.

Feedback

The MPR interactive mouse tool, which is shown in Figure 169, provides the following features:

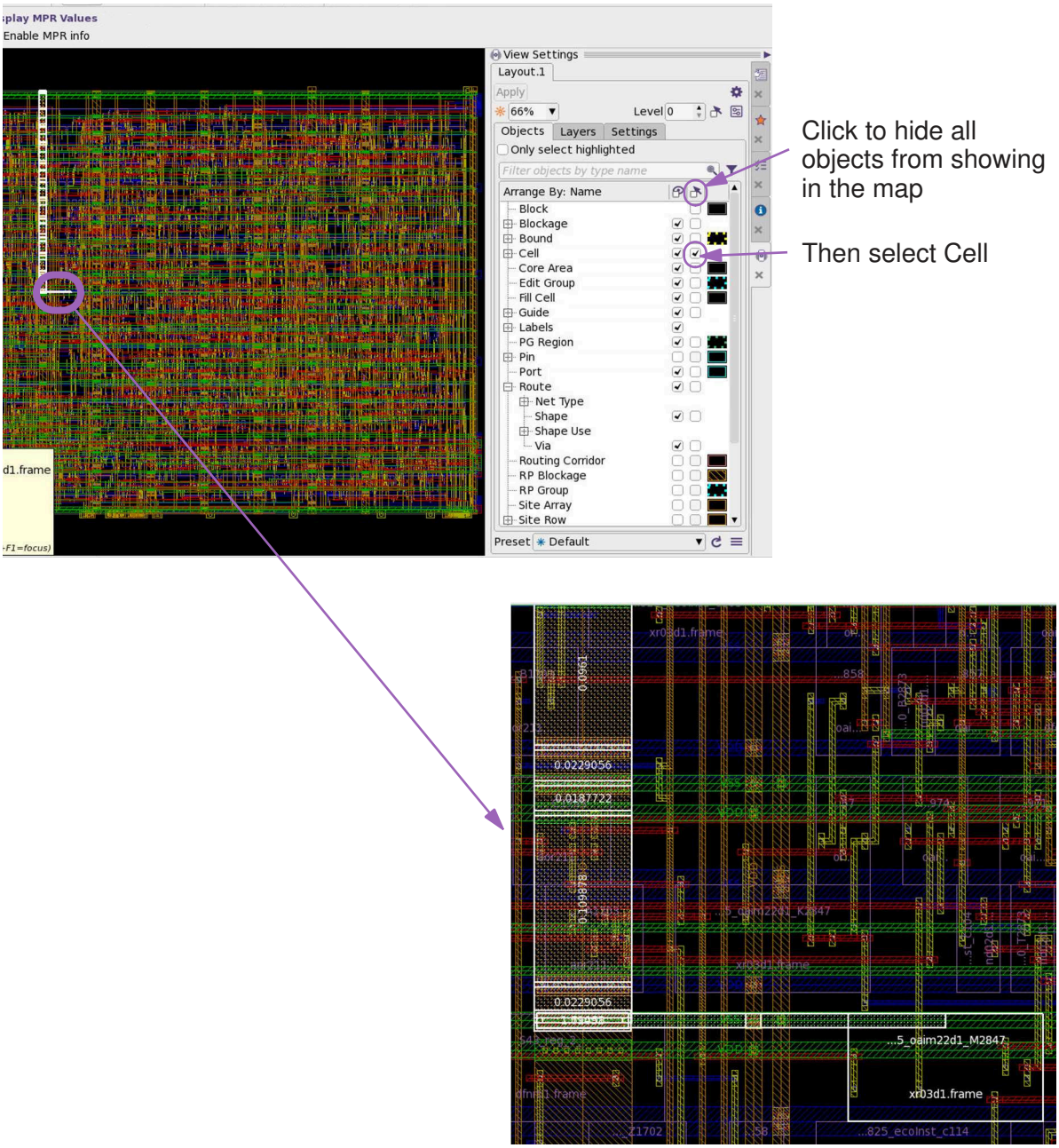
- Zoom in and out to review the traces
- Provide two operation modes: the Add mode to trace multiple paths in the map, and the Replace mode to trace one path at a time in the map
- Use the standard query tool to show tracing information
- Select a region to trace multiple cells at a time
- Document the interactive actions by scripts

Figure 168 Invoking the MPR Mouse Tool



Feedback

Figure 169 Tracing the Path With the Least Resistance



Feedback

Performing Effective Resistance Analysis

Note:

Effective resistance analysis is supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

To calculate effective resistance for the specified power and ground nets, use the `analyze_rail -effective_resistance` command.

To specify a list of cell instances for effective resistance calculation, describe the target instances in a text file. Then specify this text file with the `rail.effective_resistance_instance_file` application option.

For example,

```
icc2_shell> sh cat cell_list.txt
u_GS_PRO_65
u_GS_PRO_64
ChipTop_U_compressor_mode/U3

icc2_shell> set_app_options \
   -name rail.effective_resistance_instance_file \
   -value cell_list.txt
icc2_shell> analyze_rail -nets {VDD VSS} -effective_resistance
```

You can perform effective resistance analysis before or during voltage drop analysis. When the analysis is completed, the tool writes the effective resistance values into the same rail result with the voltage drop results.

Note:

In the RedHawk-SC rail analysis flow, you must specify both the `-effective_resistance` and `-voltage_drop` options for effective resistance calculation.

When the calculation is complete, the tool saves the results in the working directory. To write the effective resistance values to a text file, use the `report_rail_result -type effective_resistance` command. To query the resistance value, use the `get_attribute` command.

For example,

```
icc2_shell> get_attribute [get_pins u_GS_PRO_65/VDD] \
   effective_resistance
```

Feedback

See Also

- Specifying RedHawk and RedHawk-SC Working Directories
- Writing Analysis and Checking Reports

Performing Distributed RedHawk Fusion Rail Analysis

Note:

The distributed rail analysis feature is supported only in the RedHawk Fusion analysis flow.

By default, the `analyze_rail` command uses a single process to perform RedHawk rail analysis. To reduce memory usage and total runtime, use the distributed processing method to run RedHawk Fusion on a machine other than the one on which the IC Compiler II tool is run.

You can submit multiple RedHawk jobs to the target farm machines and run the jobs in parallel in the same IC Compiler II session. This allows you to run multiple RedHawk analyses with only one IC Compiler II license. Each of the RedHawk jobs requires one SNPS_INDESIGN_RH_RAIL license key.

To examine the generated analysis result, first run the `open_rail_result` command to load the result that is saved in the directory you specify. You can then investigate the problematic areas in the map and check for violations in a text file or in an error view.

You can open one result context at a time. To load the rail context generated in another analysis run, first close the current rail result using the `close_rail_result` command, and then open the desired rail result (see Displaying Maps in GUI).

License Requirement

By default, the number of required licenses is the number of script files or configurations specified with the `-multiple_script_files` option during distributed processing mode. If the number of available licenses is less than the number of script files at the time when the `analyze_rail` command is run, the command stops with an error message.

To check for available SNPS_INDESIGN_RH_RAIL licenses continuously during the `analyze_rail` run, set the `rail.license_checkout_mode` application option to `wait`. If the tool detects that a license becomes available, the tool reuses or checks out the license and submits a script to a farm machine immediately. By default, the `rail.license_checkout_mode` application option is set to `nowait`.

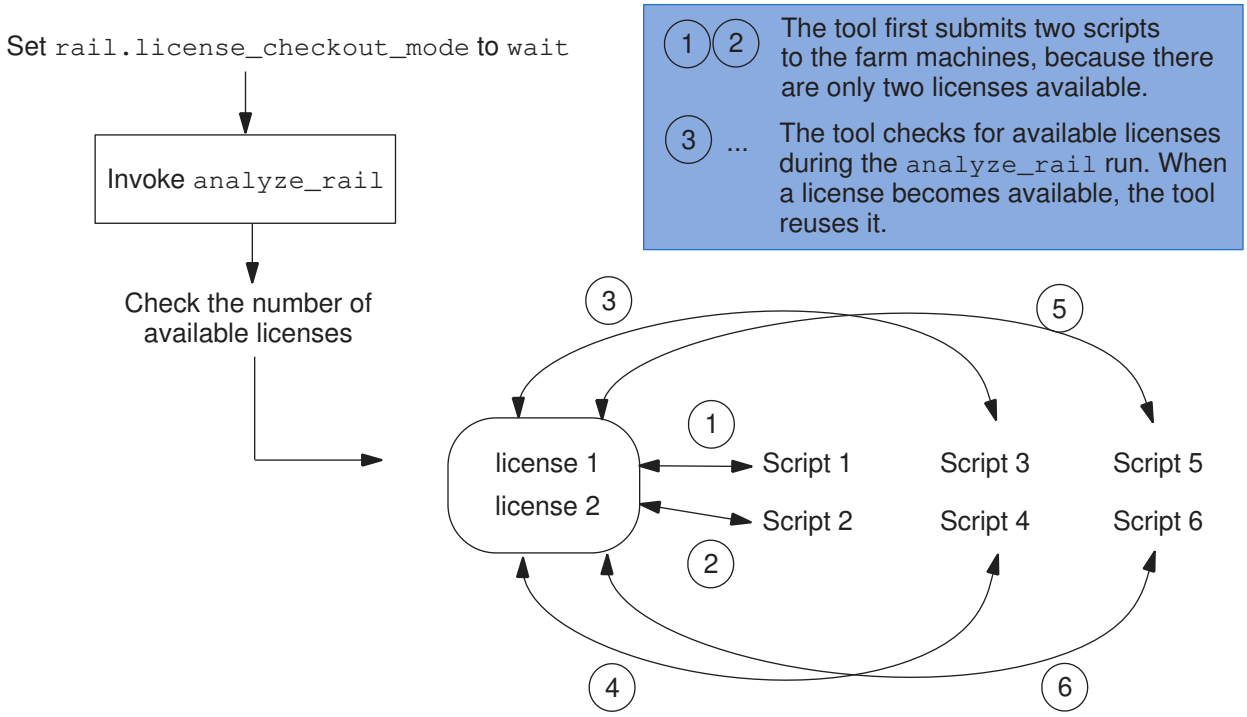
When the analysis is finished, run the `remove_licenses` command to release the SNPS_INDESIGN_RH_RAIL licenses.

```
icc2_shell> remove_licenses SNPS_INDESIGN_RH_RAIL
```

Feedback

In Figure 170, the `rail.license_checkout_mode` application option is set to `wait`. There are six scripts to submit to the farm machines with only two available SNPS_INDESIGN_RH_RAIL licenses. In this case, the tool first checks out two licenses and waits for other licenses to become available. When a script finishes the job, the tool reuses its license immediately to submit other scripts.

Figure 170 Waiting for Available Licenses During Distributed RedHawk Fusion Run



Steps to Enable Distributed Processing

To enable distributed processing for RedHawk Fusion,

1. Define the distributed processing configuration by using the `set_host_options` command. You can specify one or more of the following options:

Option	Description
`-submit_protocol`	Specifies the protocol to submit jobs, such as LSF, GRD, or SGE.Note: Before job submission, run the `rsh` command to check if the target farm machine supports the rsh protocol to accept distributed processing from on a remote machine.
`-target`	Specifies to submit jobs for RedHawk only.
`-timeout`	Specifies the job submission timeout value.

Feedback

Option	Description
`-submit_command`	Specifies the full path name of the LSF, GRD, SGE, or custom job submission program along with other options as needed, such as qsub or bsub.
`-max_cores`	Controls the number of cores each distributed worker job can use. The default is 1.

2. Run the `analyze_rail` command with the following options:

Option	Description
`-submit_to_other_machines`	Enables RedHawk job submission to the target farm machines.
`-multiple_script_files\ directory_name script_name`	Specifies the RedHawk scripts and the directories for saving analysis results.

Example 49

```
%> rsh myMachine ls
test1
CSHRC
DESKTOP
> set_host_options "myMachine" -target RedHawk \
   -max_cores 2 -timeout 100
> analyze_rail -submit_to_other_machines \
   -voltage_drop static -nets {VDD VDDG VDDX VDDY VSS}
```

The above example first checks if the machine myMachine supports the rsh protocol, and then enables job submission to the myMachine machine for RedHawk Fusion only. Timeout value is 100 and the number of cores to use is 2.

Example 50

```
> set_host_options -submit_protocol sge \
   -submit_command { qsub -P normal }
> analyze_rail -submit_to_other_machines -voltage_drop \
   static -nets {VDD VSS}
```

The above example enables job submission to a machine on the SGE farm machine.

Example 51

```
> set_host_options -submit_protocol lsf \
   -submit_command {bsub}
> analyze_rail -submit_to_other_machines \
   -multiple_script_files {{result1 RH1.tcl} {result2 RH2.tcl}}

> open_rail_result result1
```

Feedback

```
> close_rail_result
> open_rail_result result2
```

The above example submits two RedHawk jobs with two RedHawk scripts to the LSF farm machines and specifies the directories for saving the analysis results as result1 and result2.

Example 52

```
> set_app_options -name rail.license_checkout_mode \
   -value wait
> set_host_options -submit_protocol sge \
   -submit_command {qsub}
> analyze_rail -submit_to_other_machines \
   -multiple_script_files {{ result1 RH1.tcl} {result2 RH2.tcl} \
   {result3 RH3.tcl}}
```

The above example submits three RedHawk scripts to the SGE farm machines and waits for an available license during the `analyze_rail` run.

Voltage Driven Power Switch Cell Sizing

The IC Compiler II tool can resize power switch cells to reduce total power and to mitigate the voltage drop in the design. Power switch cells have different voltage thresholds (V_T) and are classified as high, standard, low, and ultra-low. For a power cell with high threshold voltage, the power consumption and leakage current is low, but the timing is also slow with a smaller drive strength. For a power cell with a low threshold voltage, the timing is faster but the power consumption and leakage current is high. With power switch cell sizing, the tool supports different drive strength and voltage threshold to reduce IR (voltage) drop in the design.

Note:

When the tool switches power cells (swapping), it can only swap cells with the same footprint but with different threshold voltages. This means, the cell cannot be swapped if the cell size and pin shapes are different.

To perform power switch cell swapping, you can use the `size_power_switches` command to set the maximum IR drop target. To meet this target when swapping power cells, the tool tries to find another cell with higher V_T, but lower R_{ON} value, so that it can reduce the IR drop. You can set the ON resistance (R_{ON}) value to be considered by the tool for swapping using the `set_power_switch_resistance` command. Based on the IR drop target and the set R_{ON} value of the power cell, the tool tries to replace power cells to meet the targeted voltage drop.

Feedback

Note:

If the tool does not find any suitable power cell that matches the targeted voltage drop and R_{ON} value, it skips cell resizing.

To perform voltage driven cell sizing,

1. Perform a voltage drop analysis using RH Fusion to get the maximum voltage drop of the cells driven by the power switch in question. Refer Performing Voltage Drop Analysis for more information.
2. Set the RON value limit to be considered for cell switching using the `set_power_switch_resistance` command. Provide the following values as input,
   - Lib cell name - specifies the name of the library cell.
   - On resistance value - specifies the RON value limit to be considered for cell sizing. You can get this information from the switch model files.

     For example,

     ```
     icc2_shell> set_power_switch_resistance
      saed32hvt_pg_ss0p_v125c/HEAD2X2_HVT 0.015
     ```
3. Run the `size_power_switches` command to perform cell size switching. Provide the value for`-max_irdrop` option which specifies the allowed maximum effective voltage drop on supply nets for standard cells driven by switches. This is a mandatory input.

   For example,

   ```
   icc2_shell> size_power_switches -max_irdop 0.05
   ```

   For the other non-mandatory inputs to further refine the command action, see the man page for the `size_power_switches` command.

The tool also provides the following commands to further help with power switch resistance:

- `get_power_switch_resistance` - This command fetches the R_{ON} value of a power switch cell.
- `reset_power_switch_resistance` - This command resets the R_{ON} value specified for a power switch cell.
- `report_power_switch_resistance` - This command reports all power switch library cells with a set R_{ON}. Use the -verbose option with the command to display the group of cells with the same footprint, the R_{on} value and the number of cells instantiated in the design. When using the `-verbose` option, you can also use `-same_vt` option to report power switch cells and their R_{ON} value from current library with same VT but different driving strength cells.

Feedback

Working With Macro Models

Memory and macro cells are used in designs with advanced process technologies to reduce memory usage and improve the rail analysis performance. Macro models are supported in both RedHawk Fusion and RedHawk-SC Fusion analysis flows.

To learn about working with macro models, see the following topics:

- Generating Macro Models
- Creating Block Contexts

Generating Macro Models

To consider current and power distribution inside the memory blocks when analyzing hierarchical designs, use the `exec_gds2rh` command to generate RedHawk macro models for memory blocks. The command invokes the RedHawk `gds2rh` utility under the hood and extracts power grids and current sources from the memory blocks with varying levels of abstraction—from fully detailed models for accurate block-level analysis to abstract models for full-chip dynamic analysis.

Note:

To create RedHawk macro models using the RedHawk `gds2rh` utility, you must specify the path to the RedHawk executable with the `rail.redhawk_path` application option before running the `exec_gds2rh` command.

The generated macro models are saved in the current working directory or the one you specify in the configuration file. To run rail analysis with the generated macro models, set the `rail.macro_models -value {`*`cell1 path1`*`}` application option before running the `analyze_rail` command.

For more information about how to run the RedHawk `gds2rh` utility, see the *RedHawk User Manual*.

Required Configuration File

To generate a RedHawk macro model using the `gds2rh` utility, you must provide a configuration file that includes the following information:

- GDS II files
- Top design name
- Layer mapping file
- Power and ground net names

Feedback

Specify one configuration file per memory. Make sure the configuration file contains the correct syntax for macro model generation. Otherwise, the tool issues an error message and terminates the model generation process.

The following is an example of the configuration file:

```
LEF_FILE {
  ./lef/saed32sram.lef
}
GDS_FILE ./gds/SRAMLP2RW128x16.gds
GDS_MAP_FILE ./redhawk.gdslayermap
OUTPUT_DIRECTORY SRAMLP2RW128x16_output

USE_LEF_PINS_FOR_TRACING   1

VDD_NETS {
 VDD
}
GND_NETS {
   VSS
}
TOP_CELL SRAMLP2RW128x16
```

Examples

The following example generates RedHawk macro models for the SRAMLP2RW128x16, SRAMLP2RW32x4, SRAMLP2RW64x32 and SRAMLP2RW64x8 memories:

```
## Specify the path to the RedHawk executable ##
set_app_options -name rail.redhawk_path -value $env(REDHAWK_IMAGE)/bin

## Specify one configuration file per memory ##
exec_gds2rh -config_file SRAMLP2RW128x16.config
exec_gds2rh -config_file SRAMLP2RW32x4.config
exec_gds2rh -config_file SRAMLP2RW64x32.config
exec_gds2rh -config_file SRAMLP2RW64x8.config
```

The following example runs top-level rail analysis with the macro models that are generated in the previous example:

```
## Run rail analysis at top level ##

open_lib
open_block top
link_block

## Specify rail analysis setting ##
create_taps
set_app_options -name rail.tech_file -value test.tech
set_app_options -name rail.lib_files -value test.lib
…
## Set macro model file ##
set_app_options -name rail.macro_models \
```

Feedback

```
  -value { SRAMLP2RW128x16 ./data_ SRAMLP2RW128x16
           SRAMLP2RW32x4 ./data_SRAMLP2RW32x4
           SRAMLP2RW64x32 ./data_SRAMLP2RW64x32
           SRAMLP2RW64x8 ./data_ SRAMLP2RW64x8
         }

## Run rail analysis ##
analyze_rail -voltage_drop static -all_nets

## Check rail results ##
open_rail_result
gui_start
```

Creating Block Contexts

Macro models are used to reduce total turnaround time for designs with advanced process technology. When violations are reported in the macros during full-chip signoff voltage drop analysis, it might be time-consuming to fix the violations at the block level and then rerun full-chip analysis to verify the fix.

To reduce the turnaround time, you can create a context model for the macro block. A context model depicts the relationship between the block and the full-chip design, including both physical and electrical information, and enables you to verify the fixes by running block-level analysis rather than full-chip analysis.

To create a context model, use the `create_context_for_sub_block` command.

To run voltage drop analysis with the generated block contexts, set the `rail.pad_files` and `rail.block_context_model_file` application options before running the `analyze_rail` command.

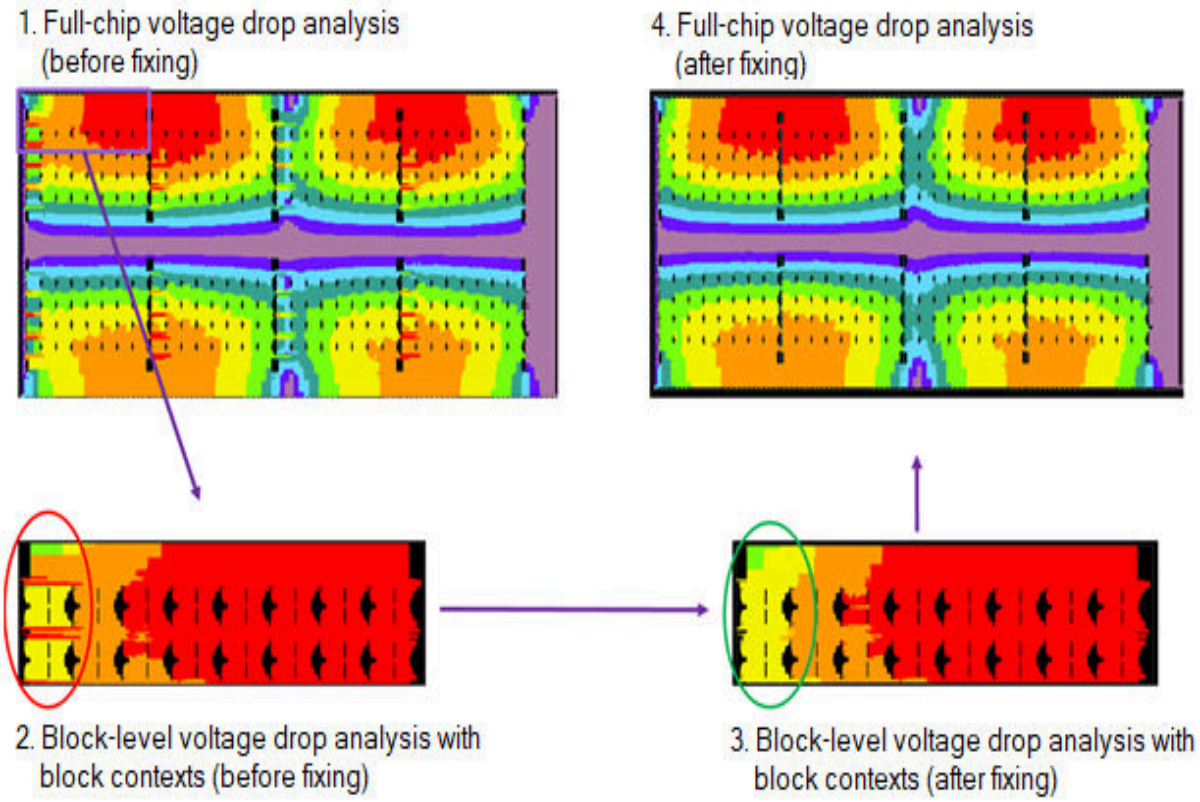
Note:

Creating full-chip contexts for blocks is supported only in RedHawk Fusion.

The following figure shows that a fix in the block can effectively improve the performance of the top design.

Feedback

Figure 171 Fixing Violations in Block Contexts



Content of Block Contexts

The `create_context_for_sub_block` command models the full-chip context of a block, including physical models and electrical models.

- Physical model, which describes the locations of the connections between the block and full-chip design. These tap locations are derived by tracing the RedHawk parasitics data. The physical model is generated when the `create_context_for_sub_block` command process is complete, in the .ploc format.
- Electrical model, which is an effective resistance- and capacitance-based model. The tool uses the native resistance engine to calculate the effective resistance, which is used to calculate equivalent current during peak dynamic voltage drop analysis. By default, the tool models the full-chip context for the specified block. The capacitance data is derived by the RedHawk extraction engine.

  To create an electrical model for the specified block, specify the `-electric_model_file` option with the `create_context_for_sub_block` command. The electrical model is written in the .context format.

Feedback

Examples

The following example creates a block context for the top/cell_inst_1 block. The generated physical and electrical models are block.ploc and block.context, respectively.

```
##Create block context in top block ##
open_block top
create_context_for_sub_block \
    -block_instance top/cell_inst_1 block.ploc \
    -nets {VDD_TOP VSS_TOP} \
    -electric_model_file block.context
close_block

##Run IR analysis with block context information in block-level design ##
open_block block
set_app_options -name rail.pad_files -value block.ploc
set_app_options -name rail.block_context_model_file \
    -value block.context
analyze_rail -nets {VDD_BLOCK VSS_BLOCK} \
     -voltage_drop static -extra_gsr_option_file extra.gsr
...
```

The following example creates only a physical model for the top/cell_inst_1 block.

```
##Create block context in top block ##
open_block top
create_context_for_sub_block \
  -block_instance top/cell_inst_1 block.ploc \
  -nets {VDD_TOP VSS_TOP}
close_block

##Run IR analysis with block context information in block-level design ##
open_block block
set_app_options -name rail.pad_files -value block.ploc
analyze_rail -nets {VDD_BLOCK VSS_BLOCK} \
     -voltage_drop static -extra_gsr_option_file extra.gsr
...
```

Performing Signoff Analysis

RedHawk Fusion or RedHawk-SC Fusion does not support RedHawk signoff analysis capabilities, such as signal electromigration or inrush current analysis. If you have the RedHawk signoff licenses, you can enable the following RedHawk signoff analysis features in the IC Compiler II environment by using the `rail.allow_redhawk_license_checkout` application option. Signoff analysis features not included in the following list can only be run with the RedHawk standalone tool.

- Dynamic analysis with custom macro models
- Dynamic analysis with package models
- Analysis with RTL-level VCD files

To perform RedHawk signoff analysis,

1. Modify your GSR file or RedHawk run script to include the related configuration settings.
2. Set the following application option to enable the RedHawk signoff features:

   ```
   icc2_shell> set_app_options \
    -name rail.allow_redhawk_license_checkout -value true
   ```

   Setting the `rail.allow_redhawk_license_checkout` application option to `true` allows the RedHawk tool to retrieve the related RedHawk licenses for performing signoff features inside the IC Compiler II tool. The default is `off`.
3. Run the `analyze_rail` command with the `-extra_gsr_option_file` or `-redhawk_script_file` option to perform the analysis defined in the GSR or RedHawk script file. Specify other settings as necessary.

   Note:

   You cannot display the RedHawk signoff analysis results in the IC Compiler II GUI.
4. Proceed to other steps in the analysis flow.

See Also

- An Overview for RedHawk Fusion and RedHawk-SC Fusion

Writing Analysis and Checking Reports

When the checking or analysis is complete, run the `report_rail_result` command to write the analysis or checking results to a text file. The power unit in the report file is watts, the current unit is amperes, and the voltage unit is volts.

To display block-level rail results from top-level RAIL_DATABASE for debugging purposes, use the`-top_design` and `-block_instance` options. For more information, see Displaying Block-Level Rail Results.

To write rail results for the cells or geometries in the hotspot area only, use the `get_instance_result` and `get_geometry_result` commands. For more information, see Generating Instance-Based Analysis Reports and Generating Geometry-Based Analysis Reports.

Feedback

To specify the data or error types to report, use the `-type` option. Table 64 lists the supported data and error types.

To limit the number of the elements to report, use the `-limit` option. Set the value to zero to write all elements to the output file in descending order.

To filter the data to write to the report file, use the `-threshold` option. The tool writes the data greater than the specified threshold to the output file. The option is valid only for the following data types:

- `pg_pin_power`
- `voltage_drop_or_rise`
- `effective_voltage_drop`

To restrict the report to the specified supply nets, use the `-supply_nets` option. This option is valid only for the following data types:

- `effective_voltage_drop`
- `instance_minimum_path_resistance`
- `minimum_path_resistance`
- `pg_pin_power`
- `voltage_drop_or_rise`

Table 64 Supported Data and Error Types for report_rail_result

Type	Description
`effective_voltage_drop`	The effective voltage values, in volts.
`effective_resistance`	The effective resistance values.
`instance_minimum_path_resistance`	The minimum path resistance value on each instance.
`instance_power`	The power value on each instance, in watts.
`minimum_path_resistance`	The minimum path resistance value on each power or ground pin.
`missing_vias`	Overlaps of supply net routing on different layers that do not have vias connected.
`pg_pin_power`	The power value on each power or ground pin, in watts.
`unconnected_instances`	Instances that are not connected to any ideal voltage drop sources.

Feedback

Table 64 Supported Data and Error Types for report_rail_result (Continued)

Type	Description
`voltage_drop_or_rise`	The dynamic voltage drop or rise violations, in volts.

In the output report file, the tool lists the data in different formats. For example:

- For the `effective_voltage_drop` type,
  - When reporting the static analysis results, the format is

    ```
    cell_instance_pg_pin_name mapped_pg_pin_name supply_net_name
       effective_voltage_drop
    ```

  - When reporting the dynamic analysis results, the format is

    ```
    cell_instance_pg_pin_name mapped_pg_pin_name supply_net_name
       average_effective_voltage_drop_in_tw
       max_effective_voltage_drop_in_tw
       min_effective_voltage_drop_in_tw
       max_effective_voltage_drop
    ```

- For the `instance_minimum_path_resistance` type, the result is sorted by the `Total_R` value in the following format:

  ```
  Supply_net Total_R(ohm) R_to_power(ohm) R_to_ground(ohm)
     Location Pin_name Instance_name
  ```

- For the `minimum_path_resistance` type, the format is

  ```
  full_path_cell_instance_name/pg_pin_name resistance
  ```

- For the `missing_vias` type, the format is

  ```
  net_name via_location_x_y top_metal_layer bottom_via_layer
     delta_voltage
  ```

- For the `pg_pin_power` type, the format is

  ```
  full_path_cell_instance_name pg_pin_name power
  ```

- For the `voltage_drop_or_rise` type, the format is

  ```
  full_path_cell_instance_name/pg_pin_name voltage
  ```

- For the `unconnected_instances` error type, the format depends on the error condition.
  - When either or both of power and ground nets are physically disconnected from ideal voltage sources, the format is

    ```
    unconnected_type {net_names} instance_name instance bbox
    ```

Feedback

- When the power or ground nets are either floating or logically floating but physically connected to ideal voltage sources, the format is

  unconnected_type {net_names} instance_name:pin_name instance bbox

The following example writes a file containing the top five PG pin power values to an output file called power.rpt.

```
icc2_shell> open_rail_result
icc2_shell> report_rail_result -type pg_pin_power -limit 5 \
   -supply_nets { VSS } power.rpt
icc2_shell> sh cat power.rpt
FI2/vss 4.81004e-06
FI6/vss 4.80959e-06
FI3/vss 4.79702e-06
FI4/vss 4.79684e-06
FI1/vss 4.79594e-06
...
```

The following example writes the calculated effective voltage drop values for the VDD and VSS nets to an output file called inst_effvd.rpt.

```
icc2_shell> open_rail_result
icc2_shell> report_rail_result -type effective_voltage_drop \
   -supply_nets { VDD VSS } inst_effvd.rpt
icc2_shell> sh cat inst_effvd.rpt

#cell_instance_pg_pin_name mapped_pg_pin_name supply_net_name
#average_effective_voltage_drop_in_tw
#max_effective_voltage_drop_in_tw min_effective_voltage_drop_in_tw
#max_effective_voltage_drop

S32_reg_14_/VDD VSS VDD -7.750034332e-03 -1.017999649e-02
-5.850076675e-03 -1.029992104e-02

S32_reg_15_/VDD VSS VDD -7.709980011e-03 -1.013994217e-02
-5.810022354e-03 -1.026010513e-02

S46_reg_8_/VDD VSS VDD -7.179975510e-03 -9.490013123e-03 -5.330085754e-03
-1.021003723e-02
...
```

The following example writes the minimum path resistance values of cell instances to an output file called inst_minres.rpt.

```
icc2_shell> open_rail_result
icc2_shell> report_rail_result -type instance_minimum_path_resistance \
   -supply_nets { VDD VSS } inst_minres.rpt
icc2_shell> sh cat inst_minres.rpt

Rmax(ohm) = 162.738
```

Feedback

```
Rmin(ohm) = 4.418
=========================================================================
Supply_net Total_R(ohm) R_to_power(ohm) R_to_ground(ohm)
Location Pin_name Instance_name
=========================================================================
VDD 162.738 152.686 10.052 586.000,
496.235 VDDL dataout_44_u
VSS 156.266 137.963 18.303 544.000,
613.235 VSS GPRs/U2317
VDD 156.266 137.963 18.303 544.000,
613.235 TVDD GPRs/U2317
VDD 152.690 135.223 17.467 602.400,
467.435 VDDL dataout_33_u
```

Displaying Block-Level Rail Results

After you perform rail analysis on the top-level design, the tool saves the analysis results in the top-level rail database. To locate issues that are reported in a block-level design, run the `open_rail_result` command with the `-top_design` and `-block_instance` options to excerpt block-level instance data from the top-level rail database. The tool then displays maps for the specified block instances based on the extracted block-level rail analysis data.

Note:

The excerpt of the block-level rail results is saved in memory and is deleted when you exit the current IC Compiler II session.

The following example opens the rail result named REDHAWK_RESULT in the ./RAIL_DATABASE directory; the result is the rail analysis result for the top design named bit_coin. The example then creates an excerpt of rail analysis data for the block instance slice_5. Note that in this example, the block design of instance slice_5 has to be opened, not the top-level design.

```
icc2_shell> open_lib block.nlib
icc2_shell> open_block block
icc2_shell> set_app_options -name rail.database -value RAIL_DATABASE
icc2_shell> open_rail_result REDHAWK_RESULT \
   -top_design bit_coin \
   -block_instance slice_5
icc2_shell> report_rail_result
```

Generating Instance-Based Analysis Reports

Use the `get_instance_result` command to write the cell instances with rail data to an output text file, in the units defined with the `report_units` command.

Feedback

To specify which analysis or checking types to report, use the `-type` option. The supported analysis types are: `effective_voltage_drop`, `current`, `effective_resistance`, `min_path_resistance`, and `power`.

Here are commonly used options of the `get_instance_result` command:

Option	Description
`-net`	(Optional) Specifies the net to report. When not specified, the command reports data for all power nets.
`-touching`	(Optional) Reports cell instances that touch the specified rectangle region in the format of `{{x1 y1} {x2 y2}}`.
`-top value`	(Optional) Reports cell instances with the top N voltage drop values.
`-threshold`	(Optional) Reports cell instances with voltage drop values that are greater than the specified threshold. This option is supported by all analysis types.
`-percentage`	(Optional) Reports cell instances with voltage drop values that are greater than specified percentage of the ideal voltage drop value. This option is supported only by the `voltage_drop` type. The default is 0.1, meaning 10% of the ideal supply voltage.
`-histogram`	(Optional) Includes a histogram that shows the average voltage drop value per net in the report.
`-collection`	(Optional) Reports cell instances as a cell collection rather than a column-based text report.

Example

The following example reports instance voltage drop results within the specified area:

```
> get_instance_result -net VDD -type voltage_drop \
   -touching {{1610 1968} {1620 1969}}
Isw2/Isw2_pktpro/U179070 -0.053533
Isw2/Isw2_pktpro/U64076 -0.0535949
Isw2/Isw2_pktpro/U83035 -0.0535949
Isw2/Isw2_pktpro/U181017 -0.0535949
Isw2/Isw2_pktpro/U103335 -0.05366
Isw2/Isw2_pktpro/U181049 -0.05366
Isw2/Isw2_pktpro/U98988 -0.05366
Isw2/Isw2_pktpro/U201986 -0.053713
Isw2/Isw2_pktpro/U181064 -0.053719
Isw2/Isw2_pktpro/U79791 -0.053809
Isw2/Isw2_pktpro/U179052 -0.054007
Isw2/Isw2_pktpro/U4120 -0.054073
Isw2/Isw2_pktpro/U20408 -0.054287
```

Feedback

See Also

- Voltage Hotspot Analysis

Generating Geometry-Based Analysis Reports

Use the `get_geometry_result` command to write the geometry information with rail results to an output file, in the units defined with the `report_units` command.

You can specify one net at a time. When more than one net is specified, an error message is displayed. To specify which analysis or checking types to report, use the `-type` option. The supported analysis types are: `voltage_drop` and `min_path_resistance`.

The report is in the following format:

```
geometry bbox_value
```

Here are commonly used options of the `get_geometry_result` command:

Option	Description
`-layer`	(Optional) Specifies the name or number of the layer to report. When not specified, all layers are reported.
`-touching`	(Optional) Specifies a rectangular region in the format of `{{x1 y1} {x2 y2}}` in which all geometry layers that touch the specified region are reported.
`-top value`	(Optional) Reports the geometry layers with the top N voltage drop values.
`-threshold`	(Optional) Specifies a threshold value. Reports geometry layers with voltage drop values that are greater than specified threshold.
`-percentage`	(Optional) Specifies a percentage value. Reports geometry layers with voltage drop values that are greater than the specified percent of the ideal supply voltage. For example, if the percentage is set to 0.1, geometries with voltage drop values greater than 10% of the ideal supply voltage are reported.
`-histogram`	(Optional) Includes a histogram that shows the average voltage drop value per net in the report.

Example

The following example writes geometry data with voltage drop values that are within the specified area:

```
> get_geometry_result -net VDD -type voltage_drop \
   -touching {{1610 1968} {1620 1969}}
```

```
{ { 1616.475 1967.000 } { 1617.525 1969.800 } } metal5 0.00448
{ { 1608.810 1946.000 } { 1611.190 1974.000 } } metal7 0.00446
{ { 1608.000 1946.000 } { 1612.000 1974.000 } } metal9 0.00446
```

See Also

- Voltage Hotspot Analysis

Performing Effective Voltage Drop (Eff-VD) and Node-Based IR-Drop Flows

When the instance-based or node-level IR-drop data from the results are the only requirement, the text-report-based flows are used. The benefits of these flows are:

- Removes the need to convert and manage thousands of compressed node or edge result files, thereby reducing file fragmentation.
- Avoids failures related to exceeding file descriptor limits.
- Loads only the necessary result data, thereby reducing memory and disk usage.
- Enables parallel processing and saves disk space.

The following are the two text-report-based flows:

- **Instance Eff-VD Results:**

  This flow provides voltage drop data specifically for each instance in the design. It focuses on reporting the effective voltage drop (Eff-VD) at the instance level, which is useful for analyzing how individual cells or instances are affected by IR-drop. After the Eff-VD flow is enabled, the `eff_vd` text report is generated for analysis.

- **Node-Based IR-Drop Results:**

  The node-based IR-drop result complements the `eff_vd` text report. When the Block Power Analysis (BPA) flow is used, the Eff-VD report is not generated, that is, Node-Based IR-Drop results are activated only with BPA flow.

Note:
Eff-VD reports are not generated when BPA is used. Node-based IR-drop results are only available when BPA is enabled. Both the flows are supported in 2-D and 3DIC designs.

Enabling Eff-VD and Node-Based IR-Drop Flows

You can enable the Eff-VD flow by setting the `rail.enable_eff_vd_flow` application option to `true`.

Feedback

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value true
```

You can enable node voltage reporting by setting the `rail.enable_node_voltage` application option to `true`.

```
icc2_shell> set_app_options -name rail.enable_node_voltage -value true
```

The following is an example script:

```
open_lib
open_block
# RHSC settings
create_taps xxx
set_app_options -name rail.product -value redhawk_sc
set_app_options -name rail.enable_redhawk_sc -value true
set_app_options -name rail.technology -value 28
set_app_options -name rail.tech_file
 -value ./design_data/tech/saed32nm_1p9m_Cmax.tech
set_app_options -name rail.redhawk_path -value $RH_RHSC_PATH
set_app_options -name rail.switch_model_files -value {
./design_data/switch_cells.txt
}
remove_host_options -all
set_host_options -target Redhawk -max_cores 16 -submit_command { local }
# New eff-VD flow
set_app_options -name rail.enable_eff_vd_flow -value true ;#(default
 false)
# Node-Based IR-Drop Results: Activated with BPA, while Eff-VD results is
 none
set_app_options -name rail.enable_node_voltage -value true ;#(default
 false)
analyze_rail -nets {VDD VSS} -voltage_drop dynamic
```

You can also use the following application options to:

- Specify the sort fields for node voltage report:

  ```
  icc2_shell> set_app_options -name rail.node_voltage_sort_fields
  ```

- Specify the line limit for instance voltage report:

  ```
  icc2_shell> set_app_options -name rail.instance_voltage_max_lines
  ```

- Specify the line limit for node voltage report:

  ```
  icc2_shell> set_app_options rail.node_voltage_max_lines
  ```

- Specify the line limit for layer voltage report:

  ```
  icc2_shell> set_app_options rail.layer_voltage_max_lines
  ```

The following are the report examples:

- Eff-VD report format:

Figure 172 Eff-VD Report Format

```
#   loc_x       loc_y      min_pg_sim     pg_arc      instance
#    (u)         (u)         (v)         pwr/gnd
   541.5200     153.6000      2.48        VDD/VSS     or23d1_N2524
   543.1200     153.6000      2.48        VDD/VSS     or23d1_O2524
   488.7200     153.6000     2.479        VDD/VSS     or23d1_Z2002
   491.1200     188.8000     2.478        VDD/VSS     or23d1_B2003
```

- Node voltage report format:

Figure 173 Node Voltage Report Format

```
# contents are sorted in descending order by layer, net, node_voltage
#   loc_x       loc_y       layer        net      node_voltage
#                                          (v)
    672.3        0.2        METAL4       VDD         2.5
    672.3       409.74      METAL4        VDD         2.5
    672.3       409.74      METAL4        VDD         2.5
    672.3        0.2        METAL4       VDD         2.5
```

- Eff-R report format:

Figure 174 Eff-R Report Format

```
#Reff Report
# min_res    avg_res    max_res     loc_x      loc_y       net      pin      cell     instance
 2.93872    2.93872    2.93872    663.120000  339.470000   VDD      VDD      nd02d1   ecoInst_c76
 1.86788    2.37917    2.88064    655.350000  199.730000   VDD      VDD      dfnrb1
instruction_pointer_reg_29_
 1.84966    2.36095    2.86241    655.350000  178.670000   VDD      VDD      dfnrb1
instruction_pointer_reg_30_
 1.88089    2.34732    2.85164    654.545000  392.270000   VDD      VDD      dfcrq1   addr_reg_15_
```

Feedback

Displaying Maps in GUI

In the GUI, maps provide a visual representation of voltage drops and node voltages across different layers of a design, helping to identify problem areas and optimize power integrity.

- Loading the 2-D flow Eff-VD map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1

# open 2D die NDM

open_block …

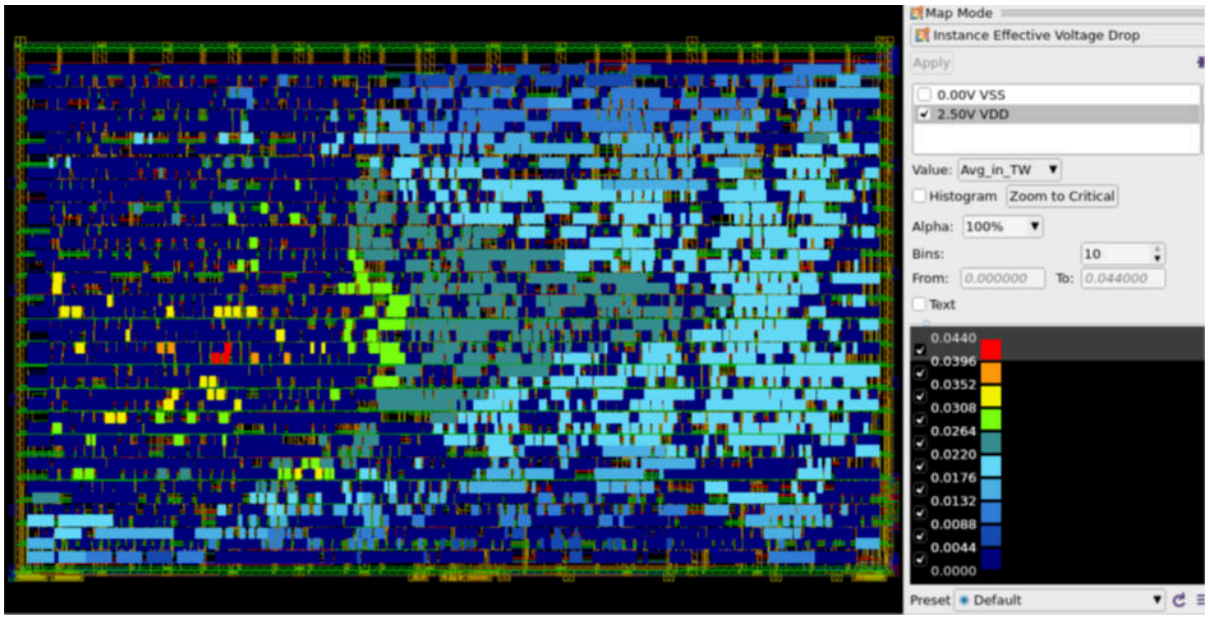
# open 2D result

open_rail_result

# launch GUI

gui_start
```

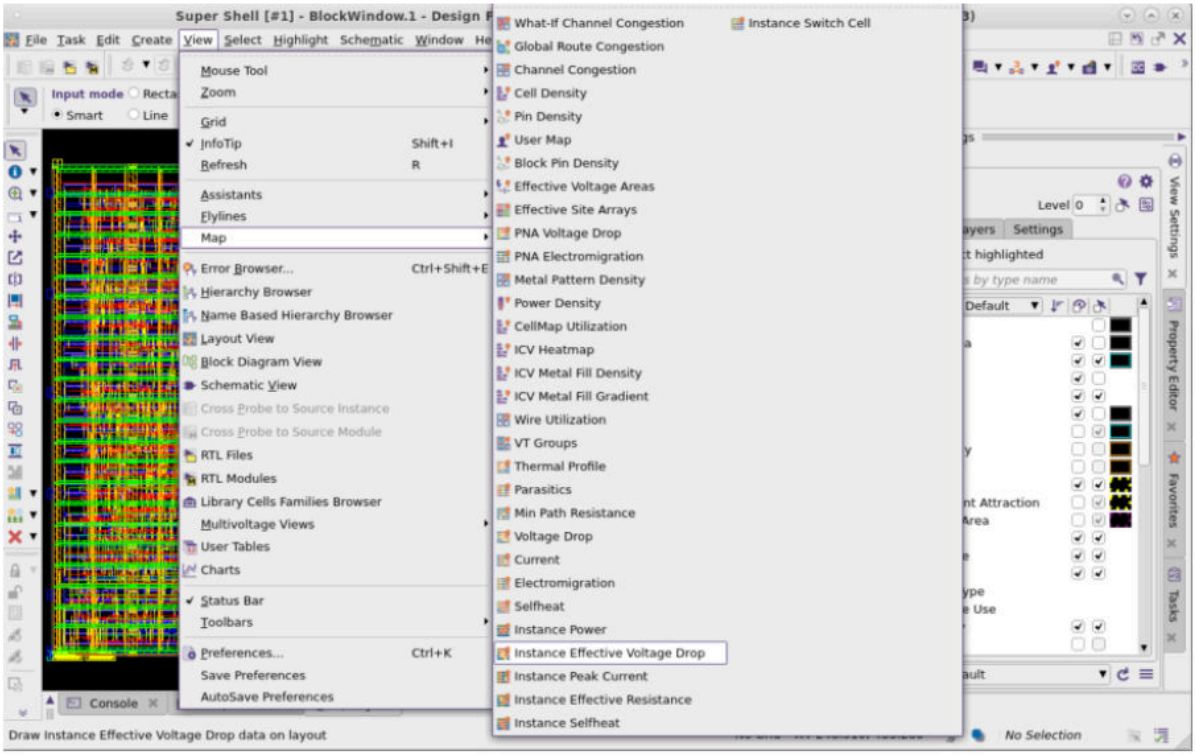
Figure 175 2-D Flow Eff-VD Map



Feedback

You can choose **View > Map > instance Effective Voltage Drop** in the GUI to load the 2-D Flow Eff-VD Map as shown in the following figure:

Figure 176 2-D Flow Eff-VD Map GUI Option



- Loading the 2-D flow node voltage map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1

set_app_options -name rail.enable_node_voltage -value 1

# open 2D die NDM

open_block

# open 2D result

open_rail_result

# launch GUI

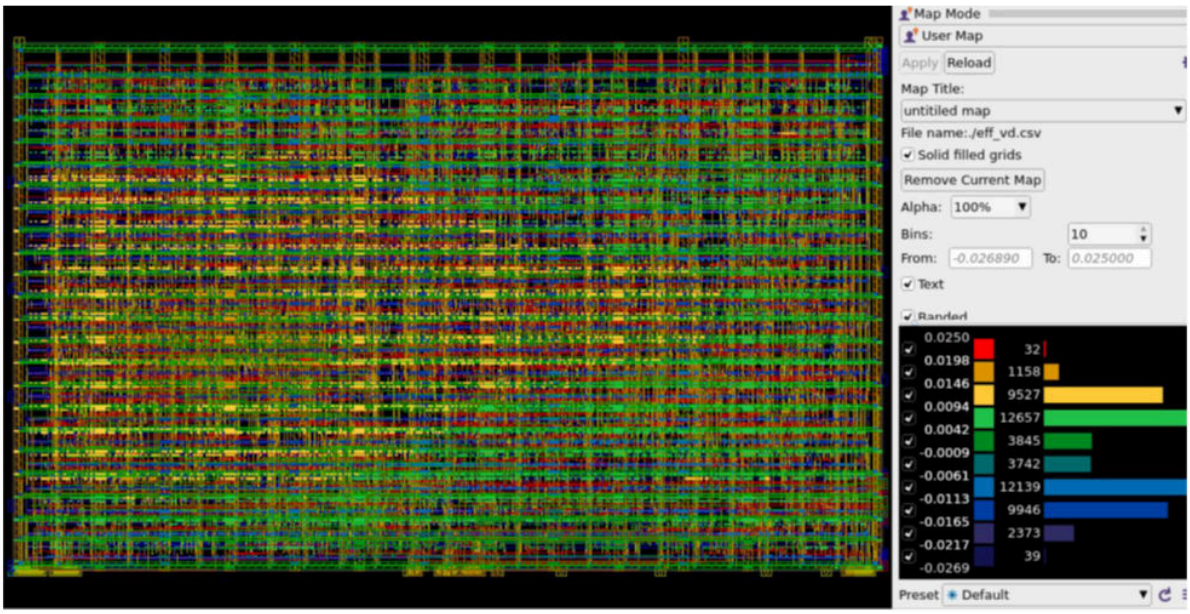
gui_start

# show map
```

Feedback

```
lookup_eff_vd -net {VDD VSS} -layer {METAL METAL2 METAL3 METAL4
 METAL5} -show_map
```

Figure 177 2-D Flow Node Voltage Map



There is no GUI menu option for node voltage.

- Loading the 3-D flow Eff-VD map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1

# open 3D die NDM

open_block

# open 3D result

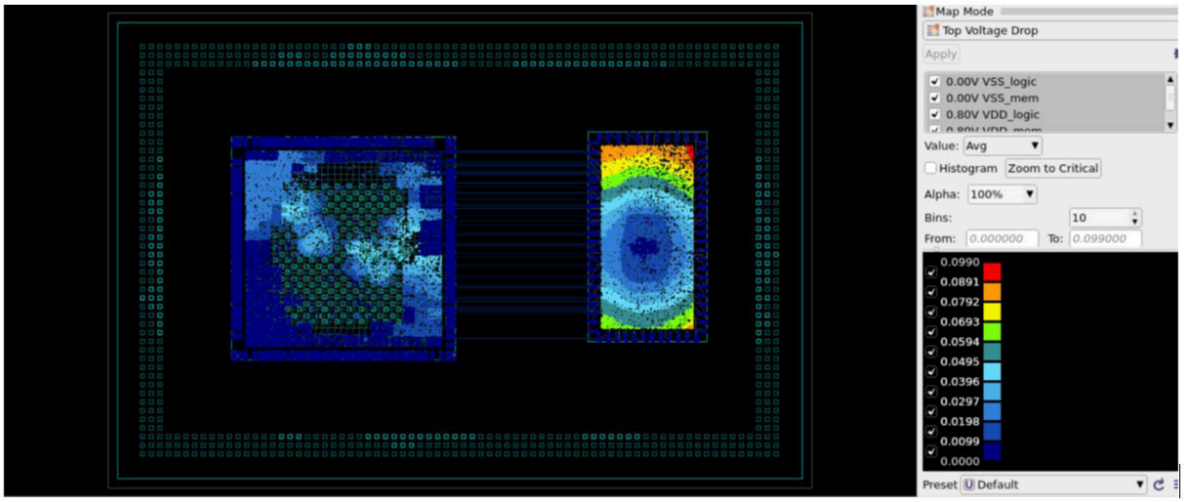
open_rail_result

# launch GUI

gui_start
```

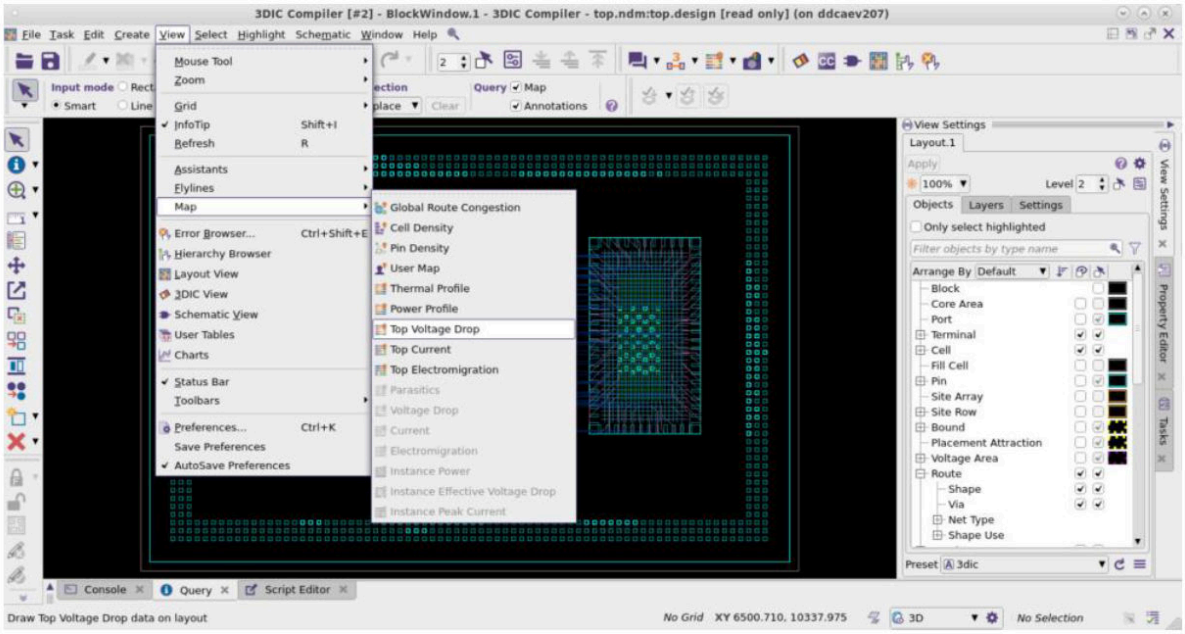
Feedback

Figure 178 3-D Flow Eff-VD Map



You can choose **View > Map > Top Voltage Drop** in the GUI to load the 3-D Flow Eff-VD Map as shown in the following figure:

Figure 179 3-D Flow Eff-VD Map GUI Option



- Loading the 3-D flow node voltage map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1
```

Feedback

```
set_app_options -name rail.enable_node_voltage -value 1
# open 3D die NDM

open_block

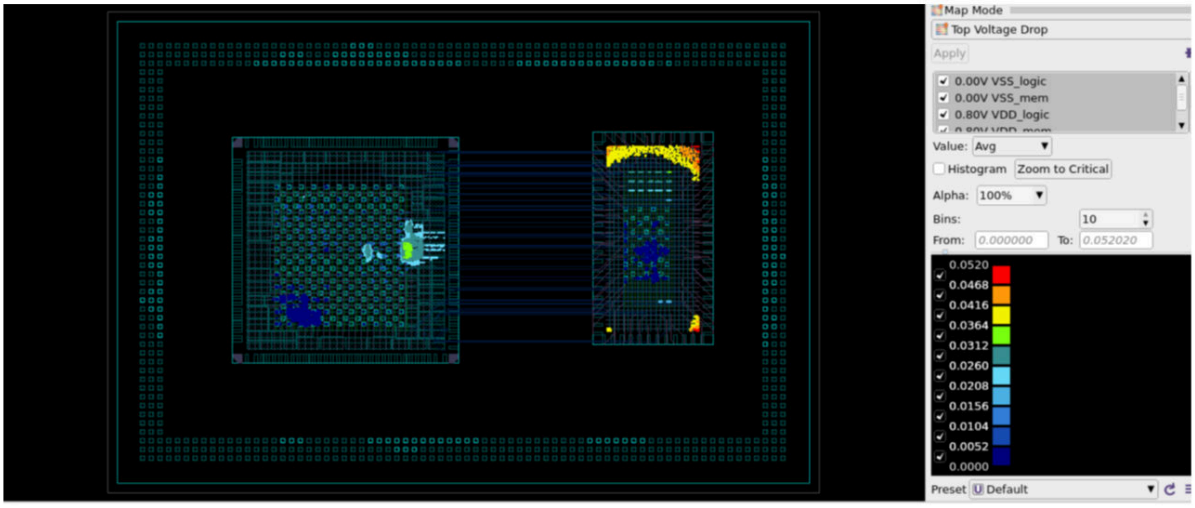
# open 3D result

open_rail_result

# launch GUI

gui_start
```

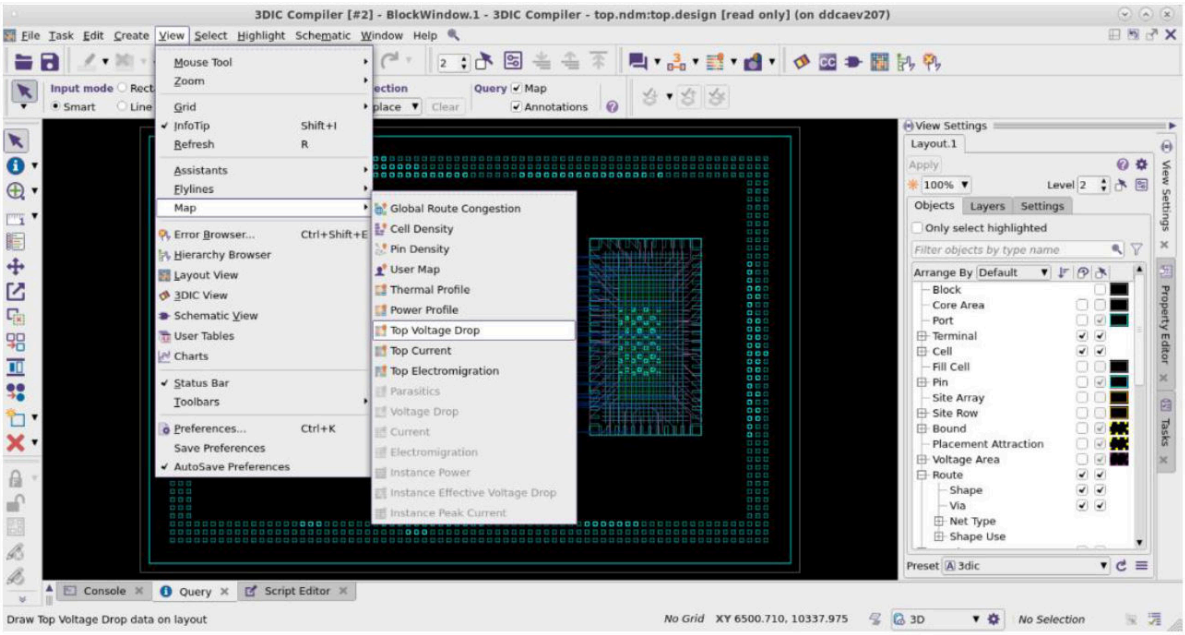
Figure 180 3-D Flow Node Voltage Map



Feedback

You can choose **View > Map > Top Voltage Drop** in the GUI to load the 3-D Flow Node Voltage Map as shown in the following figure:

Figure 181 3-D Flow Node Voltage Map GUI Option



- Loading the 3-D flow Eff-VD 2-D map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1

# open 3D die NDM

open_block

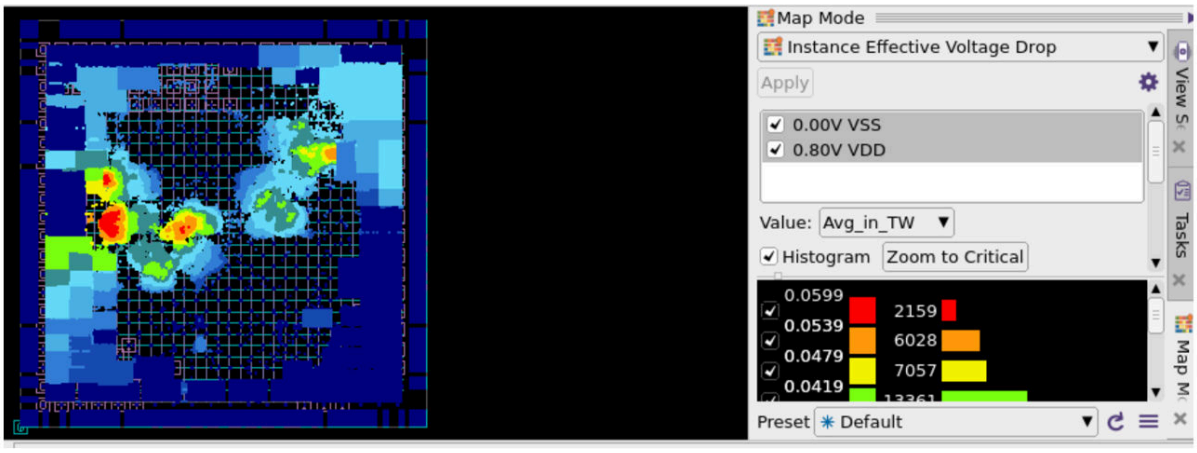
# push into 2D die NDM
set_working_design -push logic_die_inst

# open 2D result
open_rail_result -inst_name logic_die_inst

# launch GUI
gui_start
```

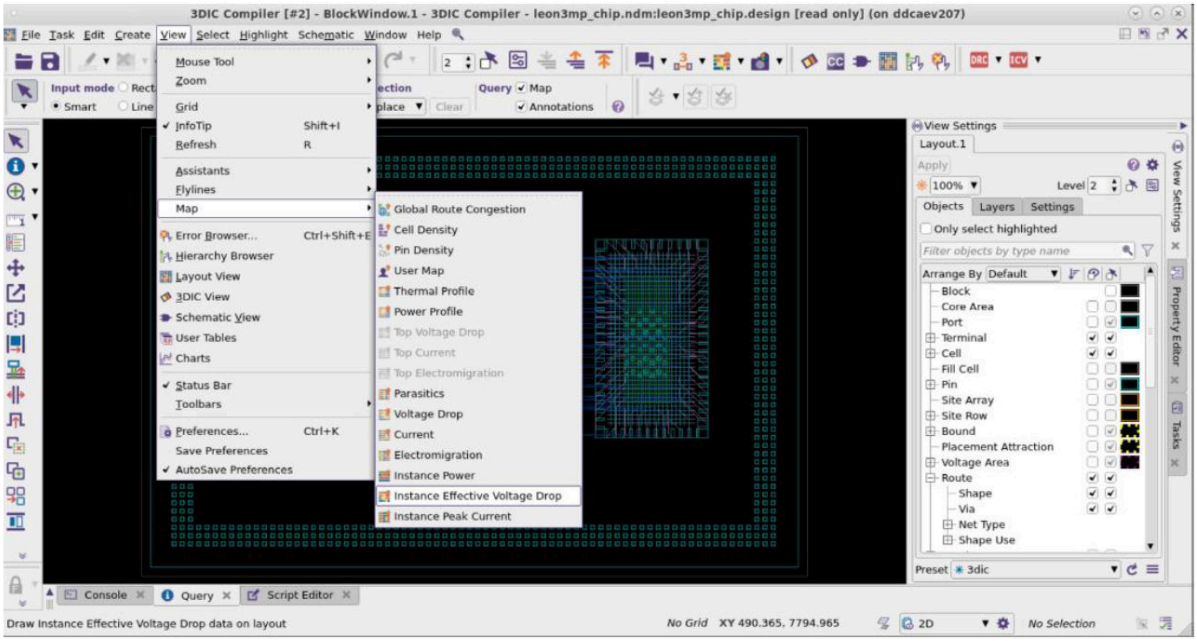
Feedback

Figure 182 3-D Flow Eff-VD 2-D Map



You can choose **View > Map > Instance Effective Voltage Drop** in the GUI to load the 3-D Flow Eff-VD 2-D Map as shown in the following figure:

Figure 183 3-D Flow Eff-VD 2-D Map GUI Option



- Loading the 3-D flow node voltage 2-D map:

```
icc2_shell> set_app_options -name rail.enable_eff_vd_flow -value 1
set_app_options -name rail.enable_node_voltage -value 1
# open 3D die NDM
open_block
```

```
# push into 2D die NDM
set_working_design - push logic_die_inst

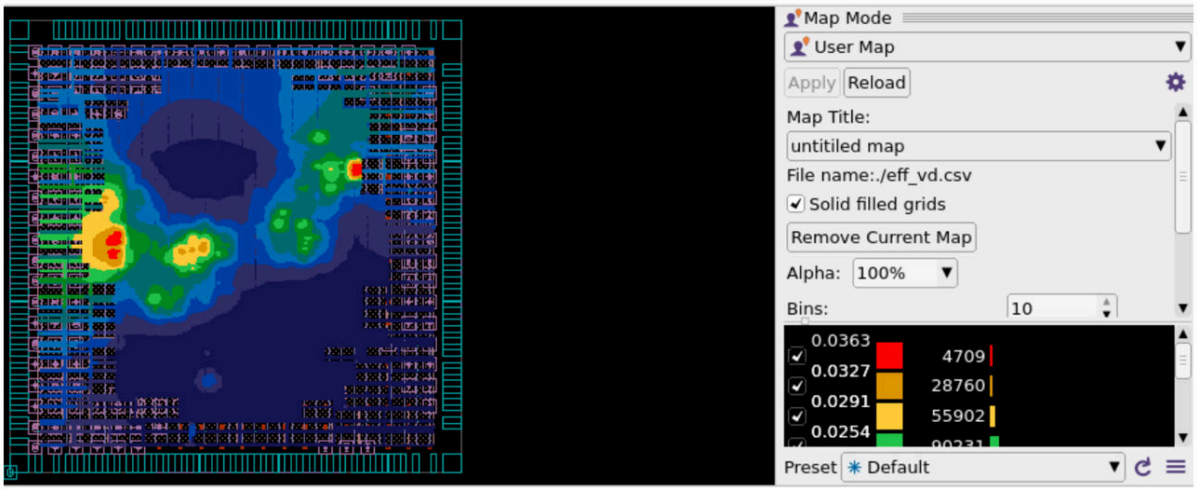
# open 2D result
open_rail_result -inst_name logic_die_inst

# launch GUI
gui_start

# show map
lookup_eff_vd -nets {VDD VSS} -layers {M1 M2 M3 M4 M5 M6 M7 M8 M9}
 -show_map
```

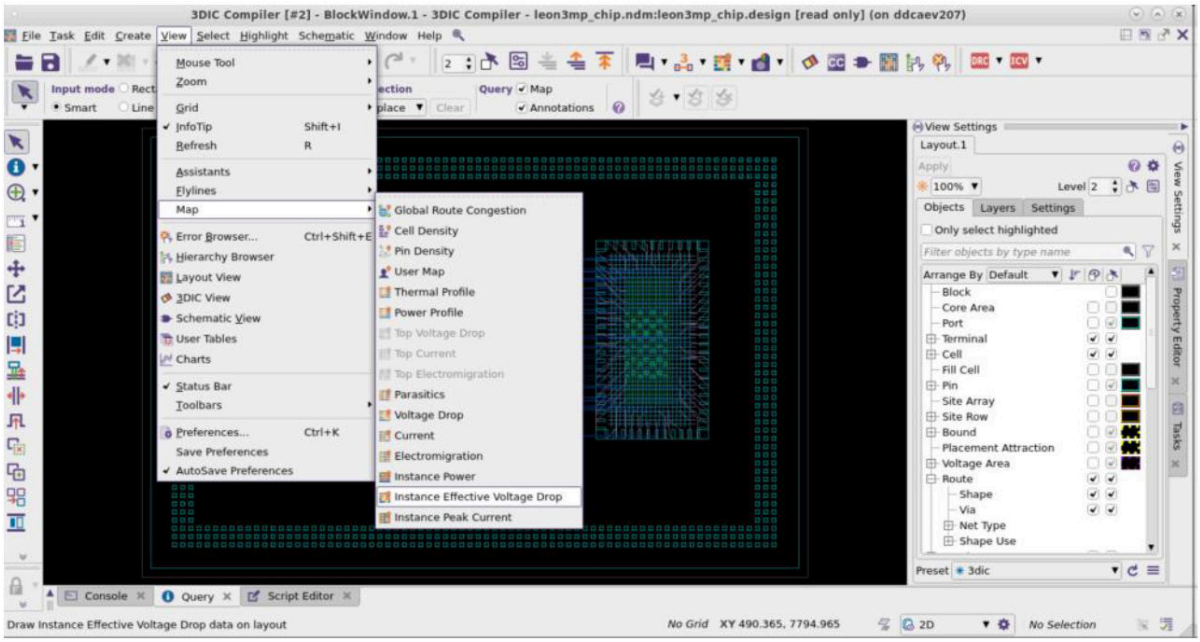
Feedback

Figure 184 3-D Flow Node Voltage 2-D Map



You can choose **View > Map > Instance Effective Voltage Drop** in the GUI to load the 3-D Flow Node Voltage 2-D Map as shown in the following figure:

Figure 185 3-D Flow Node Voltage 2-D Map GUI Option



Feedback

Displaying Maps in the GUI

When analysis is complete, the tool saves the design data and analysis results (*.result) in the in-design.redhawk/*design_name*.result directory under the RedHawk working directory. You must run the `open_rail_result` command to load the analysis results before displaying a map in the IC Compiler II GUI.

Table 65 lists the map types that are supported in the RedHawk Fusion analysis flow.

Table 65 Analysis Maps Supported in RedHawk Fusion Flow

Map Type	Description
Rail voltage drop map	Displays the voltage drop map, which is a color-coded display of voltage drop values overlaid on the physical supply nets. For static analysis, the map displays average voltage drop values. For dynamic analysis, the map displays peak voltage drop values, average voltage drop values, or peak voltage rise values. **Note:** The tool shows all the layers in the LEF or DEF file on the voltage drop map even if the layers are absent in the IC Compiler II library file.
Rail parasitics map	Shows parasitic resistance of a given supply net in the block when voltage drop analysis is complete. It shows the resistance for any given shape of the net.
Rail power map	Displays instance-based power map or power density map for the block when voltage drop analysis is complete, based on the RedHawk power calculation results. Instance-based power map: Shows power values of the cell instances in the block Power density map: Shows power density values for area in the block
Rail current map	Displays current distribution for the block when voltage drop analysis is complete.
Rail minimal path resistance map	Shows minimum path resistance values for a selected net when minimal path resistance analysis is complete.
Rail electromigration map	Shows current violations for the block when static or dynamic electromigration analysis is complete.
Rail instance peak current map	Shows peak current values for different instances or areas in the block.
Rail instance effective voltage drop map	Shows effective voltage values of a given instance in the block when voltage drop analysis is complete.
Rail instance effective resistance map	Shows effective resistance values of a given instance in the block when effective resistance analysis is complete.

Feedback

Table 65 Analysis Maps Supported in RedHawk Fusion Flow (Continued)

Map Type	Description
Rail instance switch cell	Displays current, effective voltage, and voltage difference maps for switch cells in the block when dynamic analysis is complete and switch cell model input files are available. If no switch model input file is available, the tool treats these switch cells as black boxes and no rail map is displayed. Instance-based current map: This map shows how current flows through the selected switch cell.Instance-based effective voltage map: This map shows the effective voltage at the selected switch cell.Instance-based voltage difference map: This map shows the voltage difference across the selected switch cell.

To display maps in the IC Compiler II GUI,

1. To load the rail analysis result using the GUI, choose **Task > Rail Analysis > 2D Rail Analysis**. The Block Level 2D Rail Analysis window appears. In the window, click the **open_rail_result** link.

   Alternatively, run the `open_rail_result` command to load the design data and analysis results (*.result) saved in the *design_name* directory under the RedHawk working directory when analysis is complete.

2. In the GUI layout window, choose **View > Map** and select the map to display.

   In the map, problem areas are highlighted in different colors. Move the pointer over an instance to view more information in the InfoTip. By default, the tool displays information for all instances in the block. To examine information of one specific instance, deselect all the instances and then select the instance to display from the list.

   Use the options in the map panel to change map display:

   - Value: Sets the type of map to display
   - Bins: Changes the number of unique bins used to display the map
   - From: and To: Sets the range of density values displayed in the power density map
   - Text: Displays the calculated values in the layout; if the values are not visible, zoom in to view them

Examples

The following examples show how to display various types of maps in the GUI:

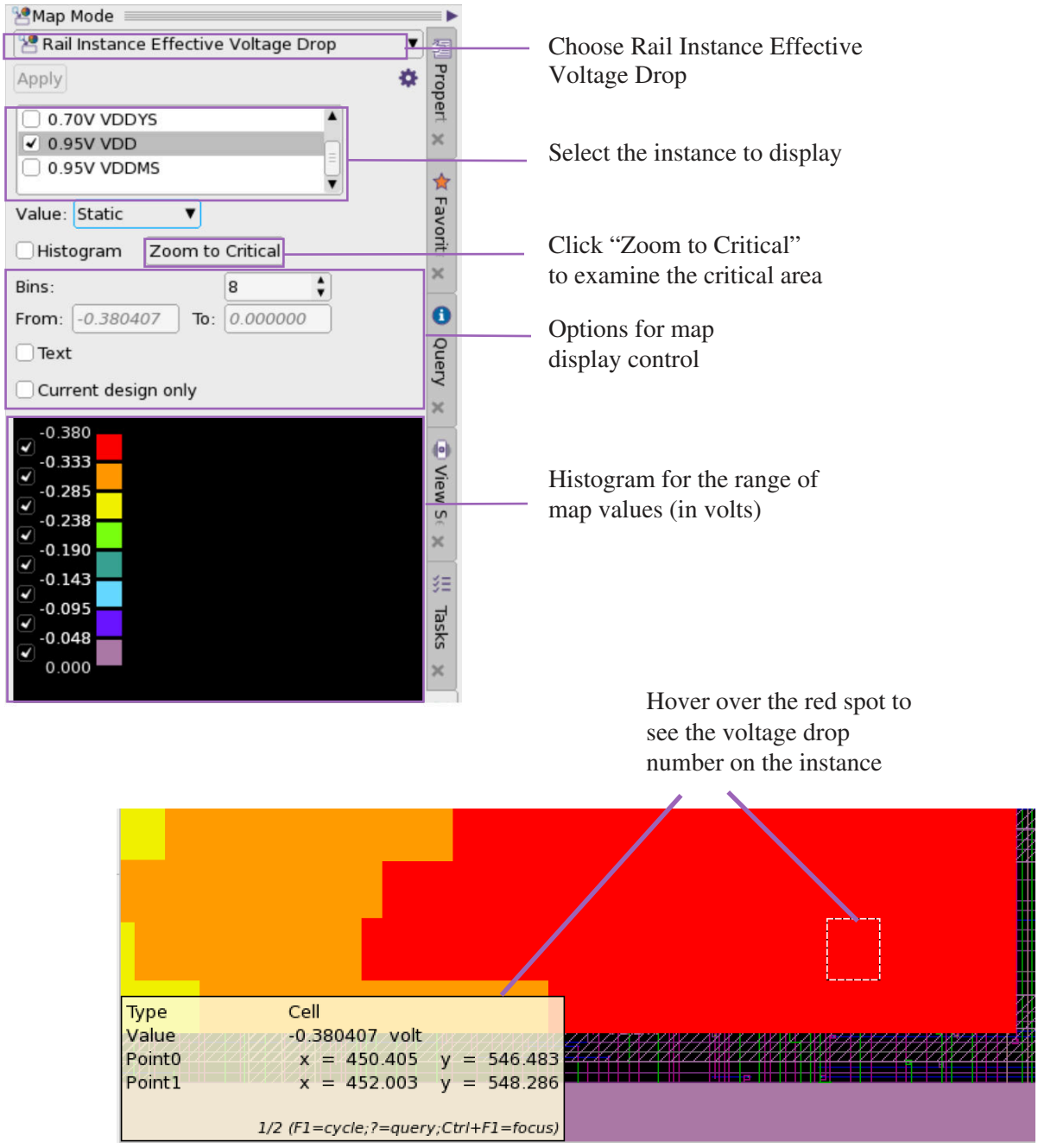
Figure 186 shows how to display a rail instance effective voltage drop map and check for hotspots in the instance effective voltage drop map.

Feedback

Figure 187 shows how to display a rail map for the switch cells in the block and examine the voltage information of the selected switch cell.

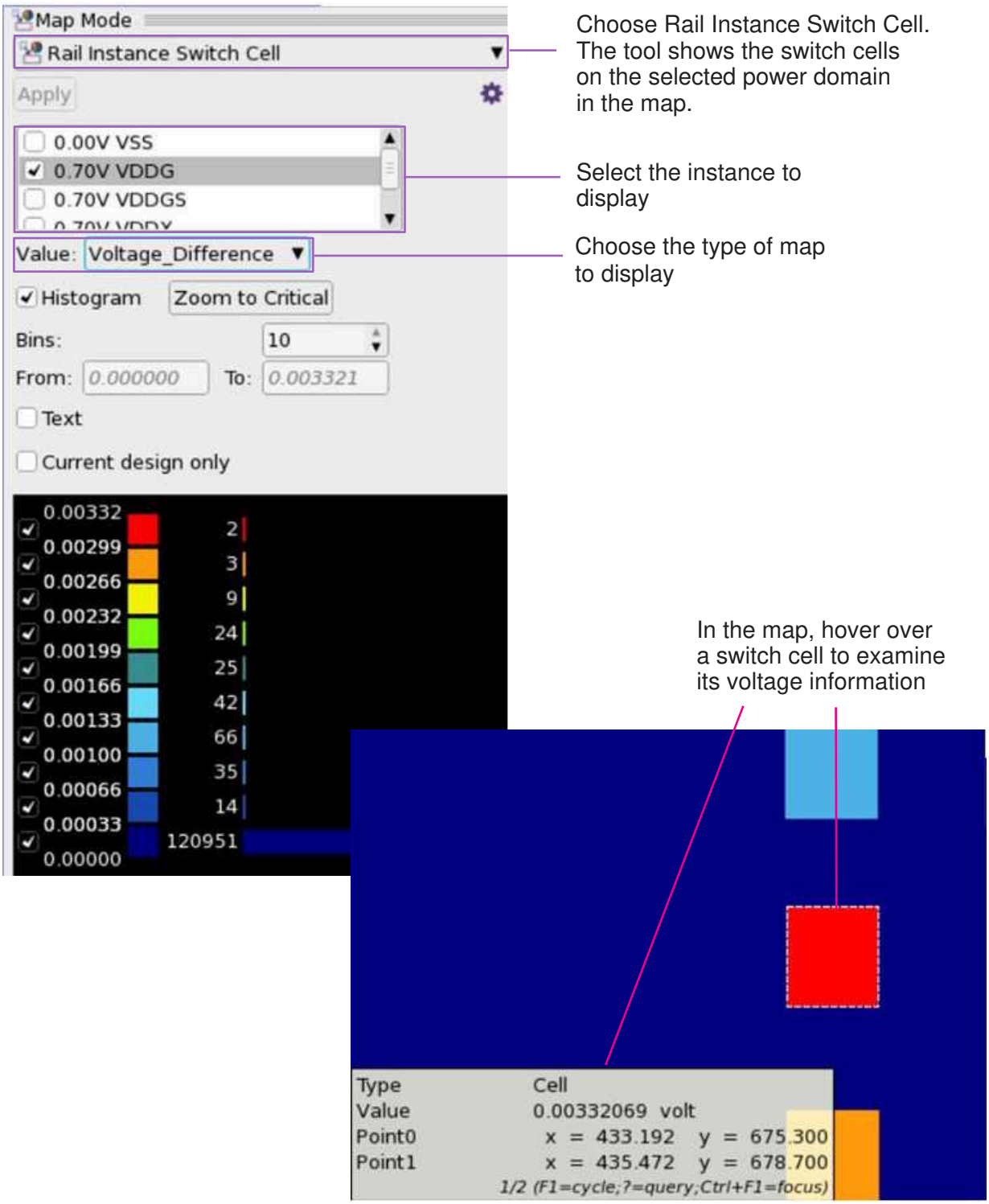
Figure 188 shows how to display a power density map for the block.

Figure 186 Displaying a Rail Instance Effective Voltage Drop Map



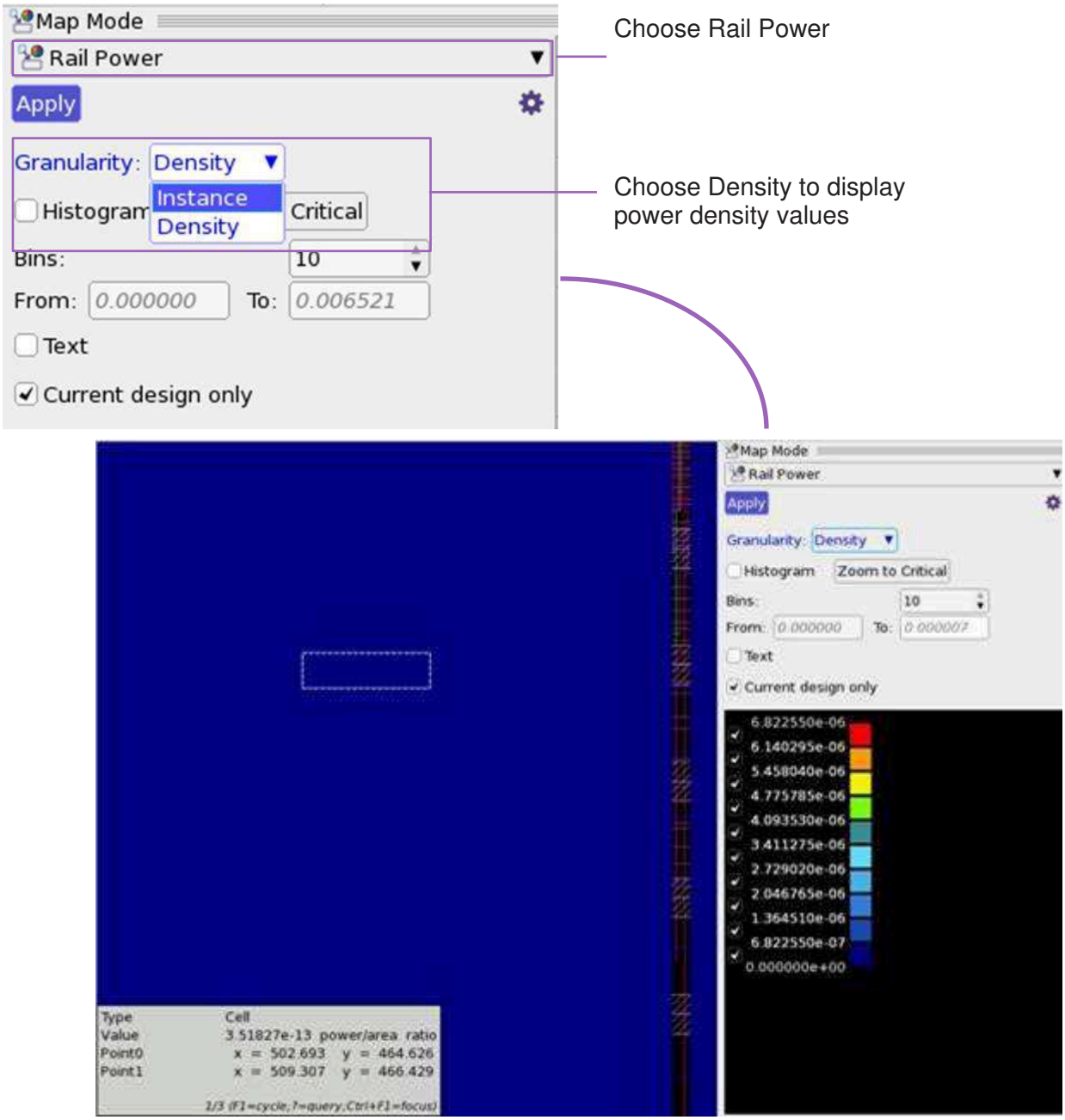
Feedback

Figure 187 Displaying Maps for Switch Cells



Feedback

Figure 188 Displaying a Power Density Map



See Also

- Specifying RedHawk and RedHawk-SC Working Directories

Feedback

Displaying ECO Shapes in the GUI

When you use the RedHawk `mesh add` command to add virtual PG meshes in the top block for the current subblock, by default the virtual PG meshes created by the RedHawk `mesh add` command are not shown as layout objects in the GUI. Therefore, these GUI shapes cannot be queried using layout object collection commands (such as, `get_shapes`), and are not saved to the design library when you save the block.

To save ECO shapes along with rail results and display them in the GUI, enable the `rail.display_eco_shapes` application option, like

```
set_app_options -name rail.display_eco_shapes -value true
```

Note:

The `mesh add` command is available only in RedHawk, not in RedHawk SC. Therefore, the tool issues an error message when you enable both the `rail.enable_redhawk_sc` and `rail.display_eco_shapes` application options.

To display ECO shapes in GUI, you must first enable the RedHawk signoff license key by setting the `rail.allow_redhawk_license_checkout` application option to `true`. Otherwise, an error message is issued.

To add virtual PG meshes and display the created ECO shapes in the GUI,

1. Prepare a script file (such as, mesh.tcl) that contains the `mesh add` command.
2. Run design setup with the mesh.tcl script file.

   ```
   > set_rail_command_options -script_file mesh.tcl \
      -command setup_design -order after_the_command
   ```

   The tool generates the RedHawk script file and sources the mesh.tcl file after design setup.
3. Perform rail analysis using the following command:

   ```
   > analyze_rail -nets -voltage_drop
   ```
4. Run the `open_rail_result` command to load the design data and analysis results.
5. Run the `gui_start` command to open the IC Compiler II GUI. In the GUI layout window, choose View > Map and select the map to display.

   You can now display and query the ECO shapes that are created by the RedHawk `mesh add` command in the GUI.

Script Examples

Feedback

The following script displays ECO shapes in the GUI by using the `set_rail_command_option` command.

```
## Open design ##
open_block
link_block
## Specify taps ##
create_taps
## Specify RedHawk Fusion input files or variables ##
set_app_options -name rail.enable_redhawk -value 1
set_app_options -name rail.redhawk_path -value
set_app_options -name rail.disable_timestamps -value true
set_app_options -name rail.display_eco_shapes -value true
set_rail_command_options -script_file mesh.tcl -command \
   setup_design ...
## Analyze##
analyze_rail -voltage_drop ... -nets {VDD VSS}
## Check GUI ##
open_rail_result
```

The following script displays ECO shapes in the GUI by using a RedHawk script.

```
## Open design ##
open_block
link_block
## Specify taps ##
create_taps
## Specify RedHawk Fusion input files or variables ##
set_app_options -name rail.enable_redhawk -value 1
set_app_options -name rail.redhawk_path -value
set_app_options -name rail.disable_timestamps -value true
set_app_options -name rail.display_eco_shapes -value true
## Analyze ##
analyze_rail -voltage_drop ... -nets {VDD VSS} -script_only
## Modify the script.tcl containing mesh add command ##
analyze_rail -redhawk_script_file
## Check GUI ##
open_rail_result
```

Voltage Hotspot Analysis

When rail analysis is complete, there might be hundreds to thousands of voltage drop violations reported in the analysis report, which might make it difficult to identify the root causes for the violations.

Use the voltage hotspot analysis capability to generate hotspots for a power or ground net by dividing the whole chip into small grid boxes, and report rail-related information for the generated grid boxes. In addition, you can query cell- or geometry-based rail results only for the specific analysis type by using the `get_instance_result` and `get_geometry_result` commands.

Feedback

Hotspot analysis provides the following features:

- Identify the root cause of aggressors with high voltage drop, such as large current from the aggressor itself or from the neighboring cells due to simultaneous switching by the overlapping timing windows.
- Determine which design technique to use for fixing voltage violations based on the report, such as choosing a candidate reference cell for cell sizing replacement, moving or relocating cells to reduce voltage drops, or applying PG augmentation to solve a voltage drop issue.

This section contains the following topics:

- Generating Hotspots
- Reporting Hotspots
- Voltage Hotspot Analysis Examples

To write rail results only for the cell instances or geometries that reside in the hotspot area, use the `get_instance_result` and `get_geometry_result` commands. For more information, see Generating Instance-Based Analysis Reports and Generating Geometry-Based Analysis Reports.

Generating Hotspots

To generate voltage drop hotspots for a power or ground net, use the `generate_hot_spots` command. This command divides the whole chip area into multiple grids in terms of rows and columns, such as 100x100. The grid boxes are sorted by the maximum effective voltage drop value in each grid box and are indexed with an integer number starting from 0.

To create rows based on standard cell site rows, and columns by using vertical PG straps on the lowest metal layers, use the `-site_row` option. Alternatively, you can specify the number of rows and columns for dividing the whole region with the `-row` and `-column` options.

The command creates error cells for the grids with voltage drop values greater than the specified percentage of the ideal voltage drop. The default is 0.1, meaning that error cells are created for the grids with voltage drop values greater than 10% of the ideal supply voltage. The error cell type is effective voltage drop.

To examine the content of the generated error cells, open the error cells in the error browser. To examine the content of the grid box, use the `report_hot_spots` command.

The tool saves the generated hotspot data in memory. If you want to generate hotspots for other nets in another run, you need to remove the previously generated hotspot data

Feedback

from memory with the `remove_hot_spots` command before proceeding to another hotspot analysis run.

For example, if you have run the `generate_hot_spots` command on the VDD net, you must first remove the hotspot data for the VDD net before generating hotspots for other nets.

Option	Description
`-net`	Specifies the net name for generating hotspots. When not specified, all power nets are used.
`-percentage`	Specifies the percentage used to generate hotspot error cells. The default is 0.1.
`-site_row integer`	By default, the command uses the standard site row height to divide the whole region into multiple rows, and the lowest PG net vertical straps to divide the whole region into multiple columns. To use a multiple of the site row height for creating grid rows, specify the `-site_row` option with an integer. For example, when the `-site_row` option is set to 2, the command uses twice the site row height to construct grid rows; that is, a row is constructed for every two site rows.
`-row`	Specifies the number of rows into which to divide the whole region.
`-column`	Specifies the number of columns into which to divide the whole region.

See Also

- Reporting Hotspots
- Voltage Hotspot Analysis Examples

Reporting Hotspots

After you generate hotspots on the block using the `generate_hot_spots` command, run the `report_hot_spots` command to report cells with voltage-related cell attributes for a hotspot grid box. The voltage-related cell attributes are: `current`, `overlap_current`, `effective_resistance`, `timing_ window`, `slack`, `output_load` and `static_power`.

Use these voltage-related cell attributes to identify root causes of voltage violations. For example, you can

- Use the current and output loading capacitance information to determine if a cell's high peak is caused by too much load. If this is the case, splitting the output might be an effective approach to reduce high voltage drops.
- Check and compare the timing windows in the current hotspot to those of the neighboring grids to determine if a cell can be moved to other grids (targets). Doing so reduces the voltage drop of the original grid without increasing the voltage drop of the target grid.
- Compare slew and load capacitance among all cells in the region to determine if high peak current is caused by sharp slew or large load capacitance.
- Find cells that have overlapping timing windows to identify cells that might switch at the same time during dynamic rail analysis.

Here are commonly used options of the `report_hot_spots` command:

Option	Description
`-index`	Lists the hotspot grids by index.
`-summary`	Reports a summary for all grids, such as the number of grids created, the number of instances in the block, and the number of grids whose voltage drop values are higher than the specified percentage of the ideal voltage drop value.
`-object`	Specifies the name of the object to report. Lists the hotspot grid that contains the specified cell instance object.
`-verbose`	Prints a detailed report.

Listing Voltage Grid Boxes by Index

By default, the tool searches three rows up and down and three columns left and right to determine the neighboring grids of the current one. Use the `-index` option to specify the index used to report voltage hotspot grids.

The grid box with the smallest index number has the highest effective voltage drop values. This allows you to determine the size of the hotspot area. For example, if all the surrounding grids have bigger index numbers and only a few central grids have much smaller numbers, this voltage drop hotspot is assumed to be an island.

Accumulating Current

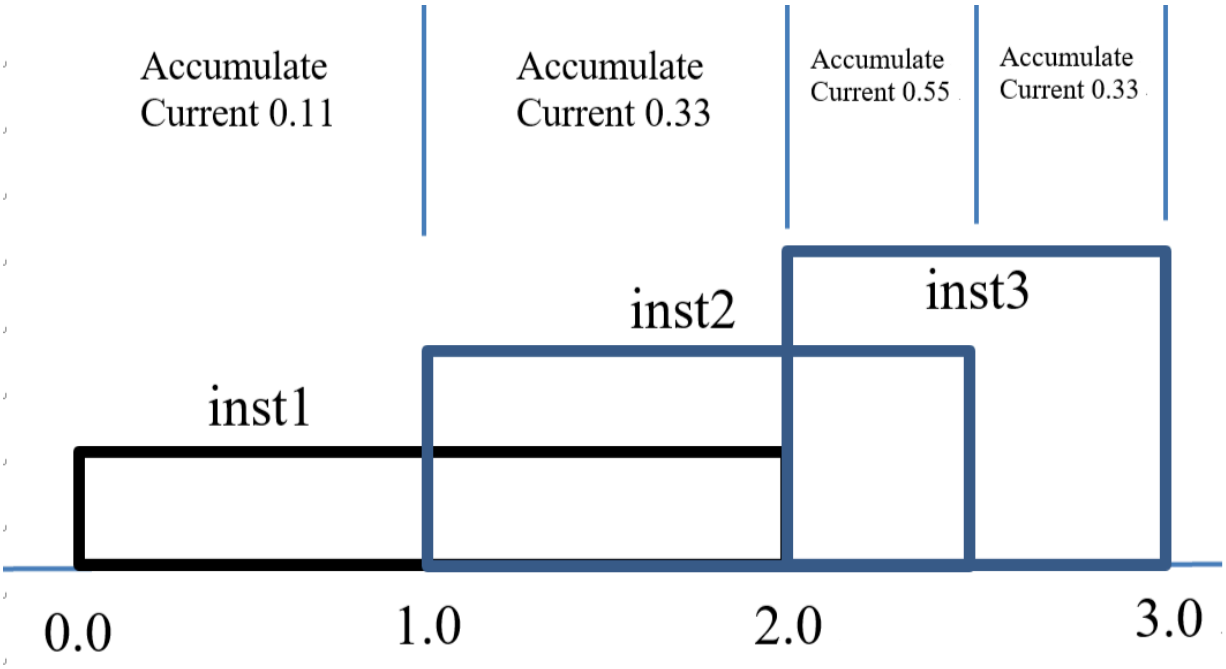
For an instance in a grid box, the overlap current is estimated by accumulating the timing-window-based current across its neighboring cells.

The command scans all timing windows from smallest to biggest, and accumulates current on the way as time passes by. Use the information to determine how many cells are

Feedback

switching simultaneously and how much time shift to apply when shifting timing windows for voltage drop reduction.

Assume the design has three instances with current as follows:



As shown in the following report, the accumulated current on point 2.0 is from inst2 and inst3, and does not include the current from inst1.

```
inst1 [0.0 2.0] current 0.11
inst2 [1.0 2.5] current 0.22
inst3 [2.0 3.0] current 0.33
```

See Also

- Generating Hotspots
- Voltage Hotspot Analysis Examples

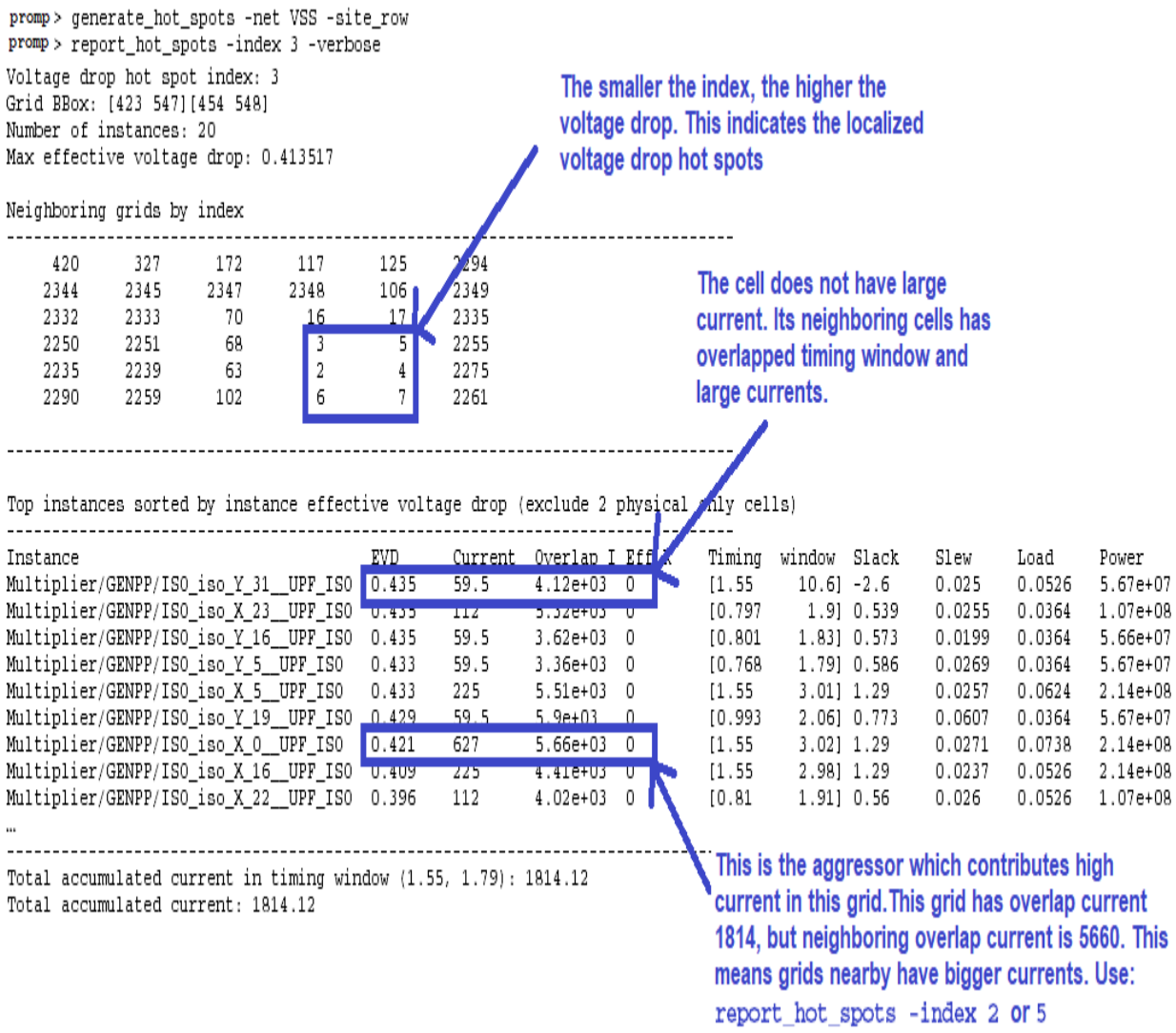
Voltage Hotspot Analysis Examples

This topic provides examples about how to use the hotspot analysis capability to identify the root cause of voltage violations.

Cells with overlapping timing windows might switch at the same time, leading to high voltage drops. Figure 189 shows how to identify if the cell has overlapping timing windows.

Feedback

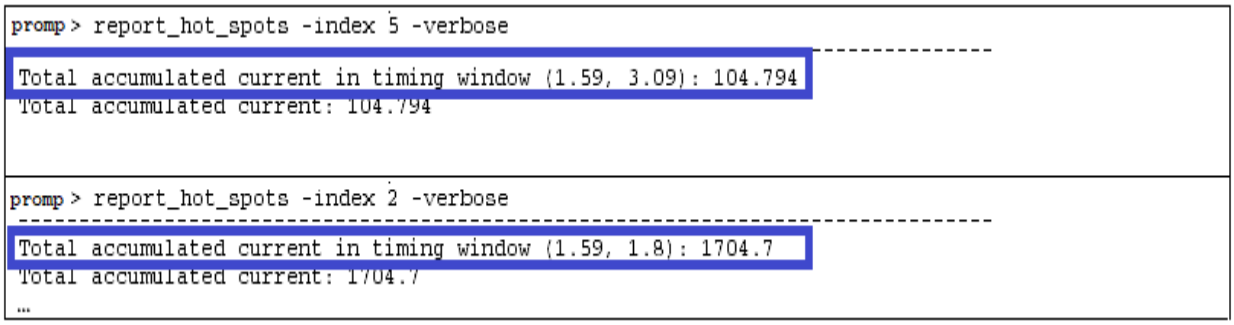
Figure 189 Checking High Voltage Drop Caused by Overlapping Timing Window (I)



Next, run the `report_hot_spots` command with the `-index` option to find which grid has overlapping timing windows that contribute to large current. As shown in Figure 190, when the `-index` option is set to 5, the grid 5 is reported with small total current. When the `-index` option is set to 2, the grid 2 is reported with large total current. In this case, overlapping time windows contribute to the high effective voltage drop in grid 2.

Feedback

Figure 190 Checking High Voltage Drop Caused by Overlapping Timing Window (II)



See Also

- Generating Hotspots
- Reporting Hotspots

Querying Attributes

When rail analysis or missing via checking is complete, use the `get_attribute` attribute to query the results stored in the RedHawk working directory. You can query the attributes shown in Table 66, which are specific to the rail analysis results.

Table 66 Rail Result Object Attributes

Object	Attributes
Cell	`static_power`

Pin	`static_current`
	`peak_current`
	`static_power`
	`switching_power`
	`leakage_power`
	`internal_power`
	`min_path_resistance`
	`voltage_drop`
	`average_effective_voltage_drop_in_tw`
	`max_effective_voltage_drop`
	`(=effective_voltage_drop)`
	`max_effective_voltage_drop_in_tw`
	`min_effective_voltage_drop_in_tw`
	`effective_resistance`

Feedback

The following example retrieves the `effective_voltage_drop` attribute for the cellA/VDD pin.

```
icc2_shell> get_attribute [get_pins cellA/VDD] effective_voltage_drop
-0.02812498807907104
```

The following example retrieves the `static_power` attribute for the cellA cell.

```
icc2_shell> get_attribute [get_cells cellA] static_power
2.014179968645724e-05
```

13

ECO Flow

An engineering change order (ECO) is an incremental change made to a complete or nearly complete design. You can use ECOs to fix functional, timing, noise, and crosstalk violations without synthesizing, placing and routing the entire design. You can also use ECOs to implement late-arriving design changes while maintaining design performance.

The following topics describe the various ECO flows supported in the IC Compiler II tool, and tasks you perform in these flows:

- Generic ECO Flow for Timing or Functional Changes
- Freeze Silicon ECO Flow
- Signoff ECO Flow
- Incremental Signoff ECO Flow
- ECO Fusion Flow
- ECO Fusion Power Integrity Flow
- Resolving RTL Mismatches in Power and Signal Elevation
- Manually Instantiating Spare Cells
- Automatically Adding Spare Cells
- Adding Programmable Spare Cells
- Making ECO Changes Using the eco_netlist Command
- Making ECO Changes Using Netlist Editing Commands
- Resizing Cells
- Adding Buffers on Nets
- Adding Buffers on Routed Nets
- Optimizing the Fanout of a Net
- Reporting Available Sites for Placing ECO Cells
- Identifying and Reverting Nonoptimal ECO Changes

- Placing ECO Cells
- Placing and Mapping ECO Cells to Spare Cells
- Updating Supply Nets for ECO Cells
- Recording the Changes Made to a Layout
- Performing Prerequisite Check for Group Repeater Insertion and Placement
- Adding a Group of Repeaters
- Querying Group Repeater
- Swapping Variant Cell
- Fixing Multivoltage Violations

Generic ECO Flow for Timing or Functional Changes

Use this flow to incorporate timing or functional ECO changes when you have the flexibility to add new cells and move or delete existing cells. This flow is recommended if you have not taped out your design.

The unconstrained ECO flow consists of the following steps:

1. Update the design with the ECO changes by using one of the following methods:
   - Using the `eco_netlist` command, as described in Making ECO Changes Using the eco_netlist Command
   - Using netlist editing Tcl commands, as described in Making ECO Changes Using Netlist Editing Commands.
2. Update the placement by using the `place_eco_cells` command, as described in Placing ECO Cells.
3. Add filler cells to the empty spaces in the site array, as described in Inserting Filler Cells.

Feedback

To reduce runtime, use the `-post_eco` option when you

- Insert metal filler cells with the `create_stdcell_fillers` command, and the tool marks the inserted filler cells as post-ECO cells.
- Remove filler cells with DRC violations with the `remove_stdcell_fillers_with_violation` command, and the tool performs DRC checking only for the post-ECO cells.

4. Update the routing by using the `route_eco` command, as described in Performing ECO Routing, or by manually rerouting the affected nets.

Freeze Silicon ECO Flow

Use this flow if your cell placement is fixed, and you can only change the metal and via mask patterns. This flow is recommended if you have taped out your design and you want to avoid the expense of generating a whole new mask set.

To perform the freeze silicon ECO flow, your block must contain spare cells. You can add spare cells to a block, anytime during the design flow, by using one of the following methods:

- Manually instantiate spare cells, as described in Manually Instantiating Spare Cells.
- Automatically add spare cells after placement by using the `add_spare_cells` command, as described in Automatically Adding Spare Cells.
- Add programmable spare cells during the chip finishing stage by using the `create_stdcell_fillers`, as described in Adding Programmable Spare Cells.

The freeze silicon ECO flow consists of the following steps:

1. Enable ECO changes in the freeze silicon mode by setting the `design.eco_freeze_silicon_mode` application option to `true`.
2. Update the design with the ECO changes by using one of the following methods:
   - Using the `eco_netlist` command, as described in Making ECO Changes Using the eco_netlist Command
   - Using netlist editing Tcl commands, as described in Making ECO Changes Using Netlist Editing Commands.
3. Analyze the mapping of ECO cells to spare cells by using the `check_freeze_silicon` command.
4. Automatically map all the ECO changes to spare cells by using the `place_freeze_silicon` command or manually map each ECO cell to a specific

Feedback

spare cell by using the `map_freeze_silicon` command, as described in Placing and Mapping ECO Cells to Spare Cells.

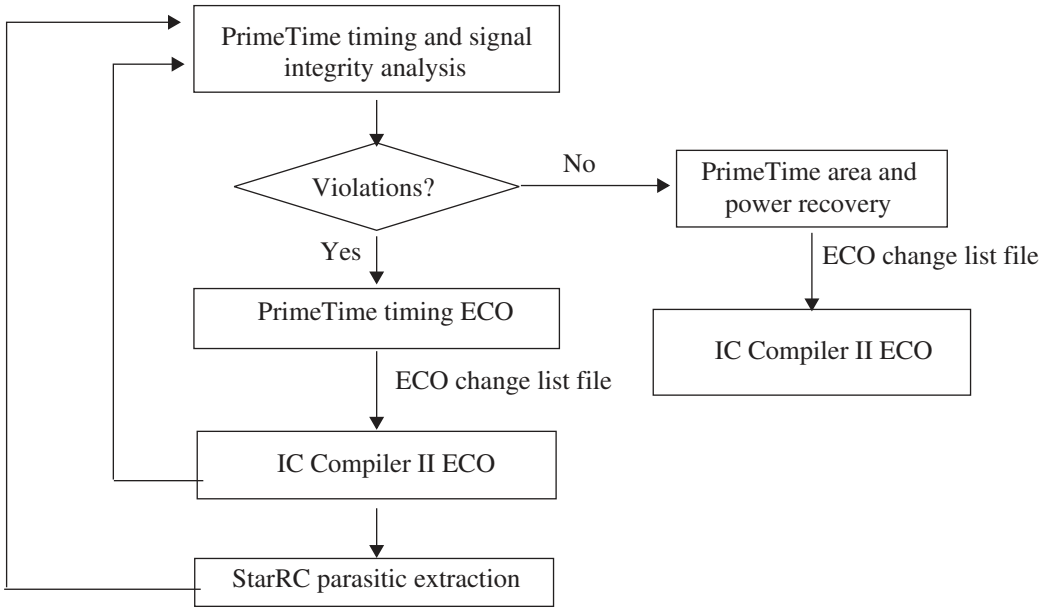
5. Update the routing by using the `route_eco` command, as described in Performing ECO Routing, or by manually rerouting the affected nets.

Signoff ECO Flow

After you perform place and route in the IC Compiler II tool, if your design has timing or design rule violations, you can fix these violations in the PrimeTime tool. You can also perform power or area recovery in the PrimeTime tool.

If you make changes to your design in the PrimeTime tool, you can generate an ECO change list file and incorporate those changes into the design by using the IC Compiler II ECO capabilities, as shown in Figure 191.

Figure 191 Signoff ECO Flow



Feedback

To incorporate the PrimeTime ECO changes, use the following steps:

1. Update the design by sourcing the PrimeTime ECO change list file, which is a Tcl file containing netlist editing commands.
2. Update the placement by using the `place_eco_cells` command as shown in the following example:

   ```
   place_eco_cells -legalize_mode minimum_physical_impact \
      -eco_changed_cells -legalize_only -displacement_threshold 10
   ```

   For more information about the `place_eco_cells` command, see Placing ECO Cells.
3. Add filler cells to the empty spaces in the site array, as described in Inserting Filler Cells.

   To reduce runtime, use the `-post_eco` option when you

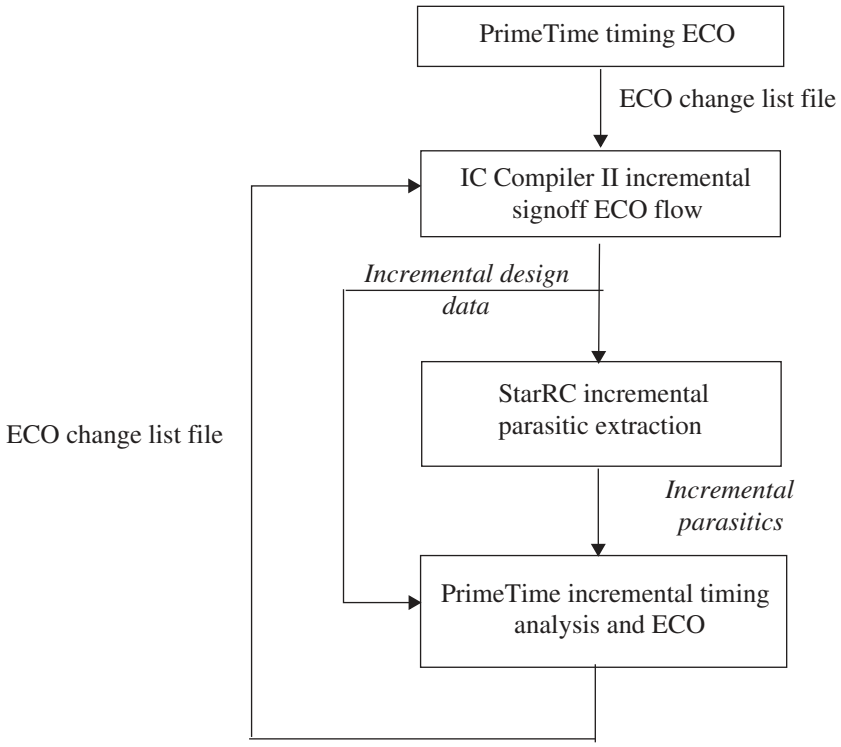
   - Insert metal filler cells with the `create_stdcell_fillers` command, and the tool marks the inserted filler cells as post ECO cells
   - Remove filler cells with DRC violations with the `remove_stdcell_fillers_with_violation` command, and the tool performs DRC checking only for the post ECO cells
4. Update the routing by using the `route_eco` command, as described in Performing ECO Routing, or by manually rerouting the affected nets.

Incremental Signoff ECO Flow

When you make ECO changes in the PrimeTime tool and incorporate these changes by performing the IC Compiler II unconstrained ECO flow, it might be necessary to iterate multiple times between the tools to meet the required QoR goals. To reduce the overall turnaround time for the flow, you can reduce the runtime of each iteration by using the incremental signoff ECO flow. This flow generates incremental design data from the IC Compiler II tool, which enables you to perform incremental extraction in the StarRC tool and incremental timing analysis and ECO in the PrimeTime tool, as shown in the following figure.

Feedback

Figure 192 Incremental Signoff ECO Flow



To perform the incremental signoff ECO flow within the IC Compiler II tool, use the `record_signoff_eco_changes` command. This command incorporates the PrimeTime ECO into the design library, tracks all the changes made to the design, and generates the incremental files that are required to run StarRC incremental extraction and PrimeTime incremental timing analysis and ECO.

The IC Compiler II incremental signoff ECO flow consists of the following steps:

1. Open the design library by using the `open_block` command.
2. Incorporate the PrimeTime ECO changes and begin tracking the ECO changes to the design by using the `record_signoff_eco_changes -start -input` command as shown in the following example:

   ```
   icc2_shell> record_signoff_eco_changes -start -input pt_eco.tcl
   ```

3. (Optional) Perform additional timing ECO changes to the design by using netlist editing commands.

   Do not perform functional ECO changes to the design. If you do so, the tool stops tracking the ECO changes being performed on the design.

Feedback

4. Place the ECO cells by using the `place_eco_cells` command.
5. Update the routing by using the `route_eco` command, as described in Performing ECO Routing, or by manually rerouting the affected nets.
6. Stop tracking the ECO changes and complete the incremental signoff ECO flow by using the `record_signoff_eco_changes -stop` command as shown in the following example:

```
icc2_shell> record_signoff_eco_changes -stop
```

ECO Fusion Flow

The IC Compiler II ECO Fusion flow allows you to use the PrimeTime physically aware ECO capabilities and the StarRC extraction capabilities (In-Design signoff extraction) within the IC Compiler II tool. You can use this flow to fix timing and DRC violations and improve area and power QoR at the very late stages of the implementation flow.

The IC Compiler II ECO Fusion flow consists of the following steps:

1. Specify the settings required to run the PrimeTime ECO capabilities within the IC Compiler II tool by using the `set_pt_options` command.

   The following example specifies the path to the PrimeTime executable and the settings for distributed processing:

```
icc2_shell> set_host_options -name pteco_host_option \
   -submit_command "/lsf/bin/bsub -R \"rusage\[mem=$MEM\]\""
icc2_shell> set_pt_options -pt_exec /snps_tools/PT/pt_shell \
   -host_option pteco_host_option
```

   If there is a .synopsys_pt.setup file in the current working directory, the IC Compiler II tool uses the information in this file. However, you can specify additional PrimeTime settings by using the `-pre_link_script` and `-post_link_script` options of the `set_pt_options` command.

   For more information, see the man page for the `set_pt_options` command.

2. Set up StarRC extraction as follows:

   a. Ensure that StarRC extraction is enabled.

      StarRC extraction is controlled by the `extract.starrc_mode` application option. The default is `fusion_adv`. You can also set the value to `in-design` or `none`.

      To use the native extractor, set the value to `none`.

   b. Specify a configuration file for running StarRC extraction by using the `set_starrc_options -config` command.

Feedback

3. Perform optimization by using the `eco_opt` command.

   By default, the `eco_opt` command fixes timing (setup and hold) and DRC violations, removes redundant buffers, and improves the total power QoR by using PrimeTime ECO capabilities. It then incorporates the ECO changes by using the IC Compiler II ECO place and route capabilities. However, you can control the type of optimization by using the `-type` option.

   The following example uses the `eco_opt` command to fix only timing and DRC violations using exhaustive path-based analysis:

   ```
   icc2_shell> eco_opt -pba_mode exhaustive -types {timing drc}
   ```

   The following example uses the `eco_opt` command to improve the leakage power QoR. However, it does not incorporate the ECO changes. Instead, it generates a file containing the ECO changes.

   ```
   icc2_shell> eco_opt -pba_mode exhaustive -types {timing drc} \
      -write_change_file_only
   ```

   For more information, see the man page for the `eco_opt` command.

4. Analyze the PrimeTime QoR by using the `check_pt_qor` command.

   For more information, see the man page for the `check_pt_qor` command.

Note:

When you run the `eco_opt` command, the IC Compiler II tool specifies the PrimeTime settings based on the IC Compiler II timing analysis settings, such as derating factors, on-chip-variation settings, and so on. Therefore, for optimal convergence, ensure that the IC Compiler II timing analysis settings at the postroute stage are consistent with the PrimeTime timing analysis settings. You can identify the differences in the IC Compiler II and PrimeTime settings by using the `check_consistency_settings` command.

ECO Fusion Power Integrity Flow

The IC Compiler II ECO Fusion power integrity flow allows you to use the RedHawk Fusion rail analysis feature to identify power integrity issues and the PrimeTime physically aware ECO capabilities to fix them.

Feedback

The IC Compiler II ECO Fusion power integrity flow consists of the following steps:

1. Specify the path for saving the RedHawk Fusion rail analysis results by setting the `rail.database` application option.
2. Perform dynamic-vectorless or dynamic-vector-based voltage-drop analysis by using the `analyze_rail -voltage_drop dynamic_vcd` or `analyze_rail -voltage_drop dynamic_vectorless` command.
3. Specify the settings required to run the PrimeTime ECO capabilities within the IC Compiler II tool by using the `set_pt_options` command.
4. Fix the identified power integrity issues by using the `eco_opt -type power_integrity` command.

To fix power integrity (voltage drop) violations, the tool downsizes aggressor cells while considering the timing and DRC QoR. However, for this optimization to be effective,

- The aggressor cells should not be timing- or DRC-critical
- The cell library should have smaller versions of the aggressor cells without `dont_use` or `dont_touch` attribute settings.

The following is an example script for running the ECO Fusion power integrity flow:

```
set_app_options -name rail.database -value RAIL_DATABASE_BASELINE
analyze_rail -voltage_drop dynamic_vcd -all_nets \
   -switching_activity {VCD ./top.vcd top}
set_pt_options -pt_exec /snps_tools/PT/pt_shell
eco_opt -type power_integrity
```

Resolving RTL Mismatches in Power and Signal Elevation

Power Elevation Using elevate_pg_nets

The `elevate_pg_nets` command inserts power elevation tie cells at regular intervals and promotes their outputs to ports. This process ensures proper power elevation by creating a structured and consistent connection between the power elevation tie cells and the ports.

To resolve RTL mismatches in power elevation, use the `-rtl_mode` option with the `elevate_pg_nets` command. The `-rtl_mode` option is mutually exclusive with the `-pitch` and `-bbox` options.

```
icc2_shell> elevate_pg_nets -lib_cells {TIEHI TIELO} -rtl_mode
```

This command creates tie cells in the specified power domains and connects them to the RTL-defined power ports.

Feedback

Signal Elevation Using elevate_signals

There are three types of signal elevation:

- Specified Signals: Signals are elevated with a buffer to the port, and a terminal is created on the specified layer.
- Signal Elevation Pins on Macros (IP): If signal elevation pins on macros exist, signals are automatically elevated to the port, and a terminal is created on the specified layer.
- Existing RTL Ports: If RTL ports for the signals already exist, the signals are automatically elevated with a buffer to the RTL port.

If you use this command to elevate a signal with a buffer, the buffer output is connected to a newly created port, and terminal is created. For the signals elevated without a buffer, a new port and a new terminal are also created.

The `elevate_signals` command elevates specified signals to be accessible through ports. This command can either buffer the signals using a specified library cell or create terminals for the signals at a specified routing layer. If you use this command to elevate a signal with a buffer, the buffer output is connected to a newly created port, and terminal is created. For the signals elevated without a buffer, a new port and a new terminal are also created. You can use this command to make certain signals available at higher levels of the design hierarchy for debugging or other purposes. The `elevate_signals` command automatically connects elevated signals to predefined RTL ports, eliminating the need for manual modifications.

```
icc2_shell> elevate_signals -nets {clk net1 net2} -lib_cell NBUFFX2_LVT
```

This command promotes the specified nets to top-level ports and connects them using SE buffers.

Manually Instantiating Spare Cells

You can manually instantiate spare cells in a block by using one of the following methods:

- Instantiating them in the Verilog netlist
- Adding them by using netlist editing commands such as `create_cell`, `connect_pin`s, and so on

When manually instantiating spare cells, you should

- Evenly distribute the spare cells through the logical hierarchy to better handle ECO changes anywhere in the block
- Tie their inputs to power or ground, as appropriate, to prevent the spare cells from creating noise and consuming power

Feedback

If a cell meets the following criteria, the tool automatically identifies it as a spare cell:

- It is not a physical-only cell
- All inputs except for the clock pin are unconnected or tied to a logic constant

  The clock pin of the spare cell, if any, can be connected to the clock network.
- All outputs are unconnected

If you instantiate the spare cells before you perform physical synthesis on a block, the tool places and legalizes the spare cells during the subsequent physical synthesis steps. However, if you instantiate the spare cells in a block that is optimized, placed and legalized, you must place and legalize the spare cells by using the following steps:

1. Spread the spare cells by using the `spread_spare_cells` command.

   By default, this command distributes and places all the spare cells evenly throughout the core area.

   You can specify the spare cells to place by using one of the following two methods:

   - To place specific spare cells, use the `-cells` option.
   - To place all the spare cells that belong to specific voltage areas, use the `-voltage_areas` option.

   You can control the placement of the spare cells as follows:

   - Specify an area within which to place the spare cells by using the `-boundary` option.
   - Ignore specific types of placement blockages by using the `-ignore_blockage_types` option. By default, the `add_spare_cells` command honors all placement blockage types.
   - Ignore the current cell density and place the spare cells randomly by using the `-random_distribution` option.
   - Specify a percentage of spare cells to be placed based on the cell density distribution by using the `-density_aware_ratio` option. The rest of the spare cells are placed randomly throughout the design. By default, all the spare cells are placed based on the cell density distribution.

     For example, if you specify a setting of `-density_aware_ratio 80`, the tool places 80 percent of the spare cells based on the cell density distribution and 20 percent randomly across the design.
2. Legalize the spare cells by using the `place_eco_cells -legalize_only -cells` command.

Feedback

Automatically Adding Spare Cells

After placement and optimization, you can add spare cells and legalize them by using the following steps:

1. Add spare cells by using the `add_spare_cells` command and specify the following information:
   - A name prefix for the spare cells by using the `-cell_name` option
   - The type and number of spare cells to insert by using one of the following two methods:
     - Specify the library cells to use for the spare cell and the number of instances of each library cell by using the `-lib_cell` and `-num_instances` options

       For example, to insert 250 instances each of the AND2 and OR2 library cells, use the following command:

       ```
       icc2_shell> add_spare_cells -cell_name spare \
          -lib_cell {AND2 OR2} -num_instances 250
       ```

     - Specify the library cells and a different number of instances for each library cell by using the `-num_cells` option

       For example, to insert 200 instances of the NAND2 library cell and 150 instances of the NOR2 library cell, use the following command:

       ```
       icc2_shell> add_spare_cells -cell_name spare \
          -num_cells {NAND2 200 NOR 150}
       ```

   - (Optional) A repetitive placement window in which to add the spare cells by using the `-repetitive_window` option

     For example, to add 15 instances each of the AND2 and OR2 library cells in a 20 by 20 micron window that is repeated throughout the placement area, use the following command:

     ```
     icc2_shell> add_spare_cells -cell_name spare \
       -lib_cell {AND2 OR2} -num_instances 15 \
       -repetitive_window {20 20}
     ```

     To add 20 instances of the NAND2 and 15 instances of the NOR2 library cells in a 25 by 20 micron window that is repeated throughout the placement area, use the following command:

     ```
     icc2_shell> add_spare_cells -cell_name spare \
        -num_cells {NAND2 20 NOR 15} -repetitive_window {25 20}
     ```

Feedback

By default, the `add_spare_cells` command distributes the spare cells evenly throughout the entire design. To distribute the spare cells within a specific

- Hierarchical block, use the `-hier_cell` option.

  When you use this option, the tool distributes the spare cells in a rectangular area that encloses all of the cells that belong to the specified hierarchical block.

- Bounding box, use the `-boundary` option.
- Voltage areas, use the `-voltage_areas` option.

You can further control the placement of the spare cells as follows:

- Ignore specific types of placement blockages by using the `-ignore_blockage_types` option. By default, the `add_spare_cells` command honors all placement blockage types.
- Ignore the current cell density and place the spare cells randomly by using the `-random_distribution` option.
- Specify a percentage of spare cells to be placed based on the cell density distribution by using the `-density_aware_ratio` option. The rest of the spare cells are placed randomly throughout the design. By default, all the spare cells are placed based on the cell density distribution.

  For example, if you specify a setting of `-density_aware_ratio 80`, the tool places 80 percent of the spare cells based on the cell density distribution and 20 percent randomly across the design.

- Specify the type of connection for the input pins of the spare cells by using the `-input_pin_connect_type` option. The valid values are `tie_low`, `tie_high`, or `open`.

2. Legalize the spare cells by using the `place_eco_cells -legalize_only -cells` command.

Feedback

Adding Programmable Spare Cells

The IC Compiler II tool supports programmable spare cells, also known as gate array filler cells, if they are provided by your vendor. These cells can be programmed by metal mask changes for ECO implementation, reducing mask costs and time-to-results.

To insert programmable spare cells, use the following steps:

1. Specify the programmable spare cell that a standard cell can map to by setting the same `psc_type_id` attribute setting on the corresponding library cell for the standard cell and the programmable spare cell.

   The following example specifies that

   - The standard cells named BUF1 and INV1 can map to the programmable spare cell named fill1x by setting the `psc_type_id` attribute to `1` for the corresponding library cells.
   - The standard cells named NAND2 and NOR2 can map to the programmable spare cell named fill2x by setting the `psc_type_id` attribute to `2` for the corresponding library cells.

```
icc2_shell> set_attribute [get_lib_cells fill_lib/fill1x] \
   psc_type_id 1
icc2_shell> set_attribute [get_lib_cells stdcell_lib/BUF1] \
   psc_type_id 1
icc2_shell> set_attribute [get_lib_cells stdcell_lib/INV1] \
   psc_type_id 1
icc2_shell> set_attribute [get_lib_cells fill_lib/fill2x] \
   psc_type_id 2
icc2_shell> set_attribute [get_lib_cells stdcell_lib/NAND2] \
   psc_type_id 2
icc2_shell> set_attribute [get_lib_cells stdcell_lib/NOR2] \
   psc_type_id 2
```

2. Insert the programmable spare cells by using the `create_stdcell_fillers` command.

   The following example inserts programmable spare cells named fill1x and fill2x:

```
icc2_shell> create_stdcell_fillers \
        -lib_cells {fill_lib/fill1x fill_lib/fill2x}
```

During the ECO flow, the tool swaps an ECO cell with a programmable spare cell based on the `psc_type_id` attribute setting, cell width, and voltage area. If the tool removes an ECO cell from the design, it can reuse the corresponding programmable spare cell.

Feedback

Making ECO Changes Using the eco_netlist Command

You can make ECO changes to a block by using the `eco_netlist` command.

If you are using the freeze silicon ECO flow, enable ECO changes in the freeze silicon mode by setting the `design.eco_freeze_silicon_mode` application option to `true`, before you run the `eco_netlist` command.

When you use the `eco_netlist` command, use one of the following two methods to make the ECO changes:

- Specify a golden Verilog netlist that includes the ECO changes by using the `-by_verilog_file` option
- Specify a golden block that includes the ECO changes by using the `-block` option

  When you use the `-block` option, by default,

  - The tool assumes the golden block is in the current design library.

    To specify a different design library, use the `-golden_lib` option.
  - The tool makes the ECO changes to the current design.

    To make the ECO changes to a different design, use the `-working_block` option. To specify the design library that contains the design specified by the `-working_block` option, use the `-working_lib` option.

The tool compares the working design to the input Verilog netlist or the golden design and generates a change file containing netlist editing Tcl commands that implements the functional changes. You must specify the name of the output file by using the `-write_changes` option.

By default, the tool ignores the following:

- Differences in the physical-only cells

  To consider the differences in the physical-only cells, use the `-compare_physical_only_cells` option.
- Timing ECO changes, such as cells that are resized or repeaters that are added or removed.

  To consider the timing ECO changes, in addition to the functional ECO changes, use the `-extract_timing_eco_changes` option.
- Differences in power and ground objects.

Feedback

To make the ECO changes to the working design, source the change file that the `eco_netlist` command generates, as shown in the following example:

```
icc2_shell> eco_netlist -by_verilog_file eco.v \
   -compare_physical_only_cells  -write_changes eco_changes.tcl
icc2_shell> source eco_changes.tcl
```

Making ECO Changes Using Netlist Editing Commands

If the ECO changes are minimal, you can update the design by using netlist editing commands given. If you are using the freeze silicon ECO flow, enable ECO changes in the freeze silicon mode by setting the `design.eco_freeze_silicon_mode` application option to `true`, before you run the netlist editing commands.

For a list of netlist editing commands, see the *Common Design Objects* topic in the *IC Compiler II Data Model User Guide*.

Using ECO Scripts for Netlist Editing

The IC Compiler II tool supports using ECO scripts to make changes in the netlist during design planning. You can use the

- `write_split_net_eco` command to push up the branching of a physical multiple-fanout net to the top level
- `write_push_down_eco` command to push down a tree of standard cells one level
- `write_spare_ports_eco` command to create spare ports and nets on a child block

For more information, see the *Generating ECO Scripts for Netlist Editing* topic in the *IC Compiler II Design Planning User Guide*.

Resizing Cells

You can resize a cell by using the `size_cell` command, as shown in the following example:

```
icc2_shell> size_cell U21 -lib_cell AND2X4
```

If you enable the freeze silicon mode by setting the `design.eco_freeze_silicon_mode` application option to `true`, by default, the `size_cell` command checks if a compatible

Feedback

spare cell is available within a distance of five times the unit site height before sizing the cell. If a compatible spare cell is not available, the tool does not size the cell.

- To control this distance, use the `-max_distance_to_spare_cell` option.
- To disable this feature, use the `-not_spare_cell_aware` option.

Reverting Changes Made During Resizing

To revert the ECO changes made with the `size_cell` command, use the `revert_cell_sizing` command. To include the sized cells that are adjacent to the specified cell in the same row, use the `-include_adjacent_sized_cells` option. When you use this option, by default, the tool also reverts the cells that abut the specified cell on either side, if they are also sized cells. However, if you use the `-adjacent_cell_distance` option, the tool recursively reverts the sized cells that are adjacent to sized cells within the specified distance in the same row.

Adding Buffers on Nets

To add buffers on nets connected to specified pins, use the `add_buffer` command.

Specify the following options when using the `add_buffer` command:

- `-new_net_names`: Specifies the names of the new nets to add. You must specify one net name per buffer when adding buffers, and two net names per inverter pair when adding inverter pairs.
- `-new_cell_names`: Specifies the names of the new cells to be added. You must specify one cell name per buffer when adding buffers, and two cell names per inverter pair when adding inverter pairs.
- `-inverter_pair`: Adds inverter pairs instead of buffer cells. If you specify this option, you must supply a library cell that has an inverting output.
- `-respect_voltage_areas`: Inserts buffers within the hierarchy of the voltage area such that the buffer is physically in the layout. This option is mutually exclusive with the `-snap` option.
- `-no_of_cells`: Specifies the number of buffer cells or inverter pairs to be inserted per net.
- `-object_list`: Specifies a list of nets, pins, or ports that must be buffered. The new buffer cells or inverter pairs are placed close to the specified pins or ports if their cells are placed.

- `-lib_cell`: Specifies the library cell to be used as buffer. In this case, the object is either a named library cell or a library cell collection. This is a required option. This option is mutually exclusive with `buffer_lib_cell`.
- `-snap`: Puts the new buffer cells next to the target cell, then snaps the buffer cells to the closest site, and flips the buffer cells when needed.

Adding Buffers on Routed Nets

To add buffers or pairs of inverters on a fully routed net, use the `add_buffer_on_route` command as described in the following topics:

- Specifying the Net Names, Buffers Types, and Their Locations
- Controlling How Buffers are Added
- Specifying Settings for Multivoltage Designs
- Specifying Settings for the Freeze Silicon ECO Flow

Specifying the Net Names, Buffers Types, and Their Locations

When you use the `add_buffer_on_route` command, you must specify the following:

- The net on which to add buffers
- The type of buffers to add and where to add them on the net by using one of the following methods:
  - Adding Buffers in a Specified Configuration
  - Adding Buffers at Specified Locations
  - Adding Buffers at a Specified Interval
  - Adding Buffers at an Interval That is a Ratio of the Net Length
  - Adding Buffers on a Bus in a Specified Pattern

Feedback

Adding Buffers in a Specified Configuration

To add buffers in an exact configuration, specify the configuration of cells and their locations by using the `-user_specified_buffers` option.

With the `-user_specified_buffers` option, you can add cells only on one net at a time. You can insert different types of buffers or inverter pairs by using this option, but you cannot combine both buffers and inverter pairs.

For each cell you add, use the {instance_name library_cell_name x y layer_name... } format with the `-user_specified_buffers` option to specify the

- Instance name
- Library cell to use
- Coordinates of the exact location
- Layer at that location with an existing routing shape to connect to

  Instead of specifying the routing layer, you can have the tool automatically detect the closest routing shape to that location by using the `-detect_layer` option.

The following example adds two cells named ECO1 and ECO2 on net n22. The ECO1 cell is of type BUF2, at location (100, 70), connecting to a routing shape on the M3 layer. The ECO2 cell is of type BUF4, at location (150, 70), also connecting to a routing shape on the M3 layer.

```
icc2_shell> add_buffer_on_route net22 \
   -user_specified_buffers {ECO1 BUF2 100 70 M3 ECO2 BUF4 150 70 M3}
```

Adding Buffers at Specified Locations

To add buffers at specific locations of a net, specify

- A list of one or more library cells to select from by using the `-lib_cell` option
- The exact locations by using the `-location` option

  With the `-location` option, you must specify the x- and y-coordinates and a layer with an existing routing shape at that location by using the {x1 y1 layer1 ....} format. Instead of specifying the routing layer, you can have the tool automatically detect the closest routing shape to that location by using the `-detect_layer` option.

With this method, you can add cells only on one net at a time.

The following example adds the BUF1 library cell on the net1 net at location (100, 200) and connects it to an existing route shape on the M4 layer.

```
icc2_shell> add_buffer_on_route net1 -lib_cell BUF1 \
   -location {100 200 M4}
```

Feedback

By default, the tool uses eco_cell and eco_net as the name prefix of all new cells and nets. To specify a name prefix for the new cells and nets, use the `-cell_prefix` and `-net_prefix` options, respectively.

Adding Buffers at a Specified Interval

To add buffers to one or more nets at a specified interval, specify

- A list of one or more library cells to select from by using the `-lib_cell` option
- The distance between the driver of the net and the first buffer by using the `-first_distance` option
- The distance between the load pin of the net and the last buffer by using the `-last_distance` option

  You must use the `-last_distance` option with the `-repeater_distance` option or the `-repeater_distance_length_ratio` option. You can also use the `-last_distance` option with the `-first_distance` option. However, you cannot use it with the following options:

  - `-location`
  - `-user_specified_buffers`
  - `-user_specified_bus_buffers`

  Note:

  When you use the `-last_distance` option, the value of the `eco.add_buffer_on_route.min_distance_buffer_to_load` application option is ignored.

- The interval between the buffers by using the `-repeater_distance` option

  Optionally, you can scale the distance between the buffers by

  - Specifying a different scaling factor for each layer using the `-scaled_by_layer` option
  - Using the ratio between the default width for the layer and the actual route width as the scaling factor by specifying the `-scaled_by_width` option

The following example adds the BUF1 library cell on the nets n2 and n5 at an interval of 150 microns. The distance between the driver and the first repeater cell is 100 microns.

```
icc2_shell> add_buffer_on_route {n2 n5} -lib_cell BUF1 \
  -first_distance 100 -repeater_distance 150
```

Feedback

The following example adds the BUF2 library cell on the net n1 at an interval of 100 microns. The distance between the driver and the first repeater cell is 50 microns and the last distance is 20 microns.

```
icc2_shell> add_buffer_on_route -repeater_distance 100 \
  -first_distance 50 -last_distance 20 net1 -lib_cell BUF2
```

By default, the tool uses eco_cell and eco_net as the name prefix of all new cells and nets. To specify a name prefix for the new cells and nets, use the `-cell_prefix` and `-net_prefix` options, respectively.

Adding Buffers at an Interval That is a Ratio of the Net Length

To add buffers at an interval that is a ratio of the total net length, specify

- A list of one or more library cells to select from by using the `-lib_cell` option
- The distance between the driver and the first buffer as a ratio of the total net length by using the `-first_distance_length_ratio` option
- The distance between the buffers as a ratio of the total net length by using the `-repeater_distance_length_ratio` option

The following example adds the BUF1 library cell on the nets n3, n21, and n41 at an interval that is 20 percent of the total net length. The distance between the driver and the first repeater cell is 10 percent of the total net length.

```
icc2_shell> add_buffer_on_route {n3 n21 n41} -lib_cell mylib/BUF1 \
  -first_distance_length_ratio 0.1 \
  -repeater_distance_length_ratio 0.2
```

By default, the tool uses eco_cell and eco_net ad the name prefix of all new cells and nets. To specify a name prefix for the new cells and nets, use the `-cell_prefix` and `-net_prefix` options, respectively.

Adding Buffers on a Bus in a Specified Pattern

To add buffers to the individual nets of a bus in a specific pattern, use the following steps:

1. Define buffer patterns for the nets of a bus by using the `create_eco_bus_buffer_pattern` command.

   This command allows you to define patterns for staggering buffers on bused nets, which prevent clumping and overlapping of buffers.

Table 67 Commands Associated With Bus Patterns

To do this	Use this command
Create bus patterns	`create_eco_bus_buffer_pattern`

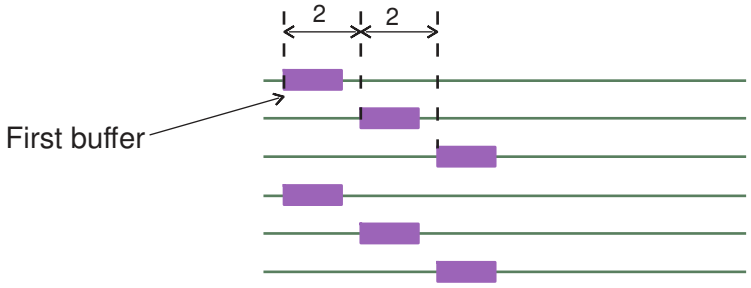
Feedback

To do this	Use this command
Report information about bus patterns	`report_eco_bus_buffer_patterns`
Get a collection of buffer patterns	`get_eco_bus_buffer_patterns`
Remove bus patterns	`remove_eco_bus_buffer_patterns`

The following example creates the buffer pattern shown in Figure 193 for a horizontal bus, where the first buffer is placed on the topmost net. The buffers are staggered by a distance of 2, starting from the left, and the pattern is repeated after every three buffers.

```
icc2_shell> create_eco_bus_buffer_pattern -name top_left \
 -first_buffer top -measure_from left -distance 2 -repeat_after 3
```

Figure 193 Buffer Patterns for a Horizontal Bus

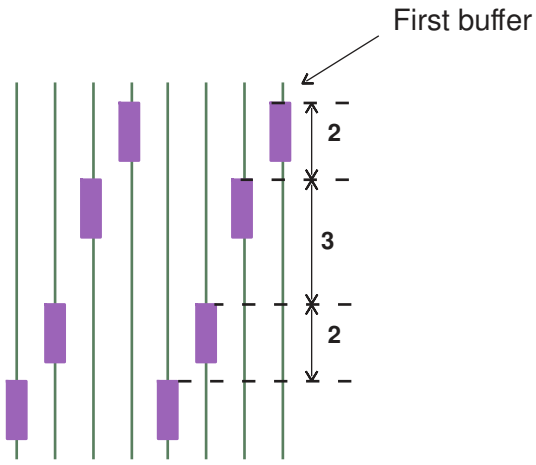


The following example creates the buffer pattern shown in Figure 194 for a vertical bus, where the first buffer is placed on the rightmost net. Starting from the top, the second buffer is staggered by a distance of 2 from the first, the third a distance of 3 from the second, and the fourth a distance 2 from the third. After that, the pattern is repeated.

```
icc2_shell> create_eco_bus_buffer_pattern -name right_top \
   -first_buffer right -measure_from top \
   -user_specified_distance {2 3 2}
```

Feedback

Figure 194 Buffer Patterns for a Vertical Bus



2. Add buffers on the nets of the bus by using the `add_buffer_on_route -user_specified_bus_buffers` command.

   For each group of buffers that you add on the bused nets, use the {buffer_pattern_name library_cell_name x y .... } format with the `-user_specified_bus_buffers` option to specify the

   - Name of the buffer pattern that you defined using the `create_eco_bus_buffer_pattern` command
   - Library cell to use
   - Coordinates of the location of the first buffer in the pattern

   The following example adds a group of buffers of type BUF2 on the bus named data_A, using a buffer pattern named BP1. The first buffer of the pattern is placed at location (100, 70).

```
icc2_shell> add_buffer_on_route [get_net data_A*] \
   -user_specified_bus_buffers {BP1 BUF2 100 70}
```

Feedback

Controlling How Buffers are Added

By default, the `add_buffer_on_route` command

- Adds buffers over all placement blockages, soft macros, and hard macros.

  To prevent buffers from being placed over specific macro cells, use the `-dont_allow_insertion_over_cell` option.

  Alternatively, you can prevent the buffers from being placed over all blockages and macro cells by using the `-respect_blockages` option. When you use this option, you can specify a list of macro cells over which buffers are allowed by using the `-allow_insertion_over_cell` option.

- Adds buffers on both global routed and detail routed nets.

  To add buffers only on nets that are global routed only, without assigned tracks or detail routing, use the `-only_global_routed_nets` option.

- Adds buffers at the lowest common level of hierarchy of the pins being driven by the buffers.

  To add the buffers on the highest possible level of hierarchy, use the `-on_top_hierarchy` option.

- Does not add buffers on a route segment if it is necessary to create new ports because of a difference in the logical and physical topology of the net.

  To add buffers on such route segments by creating new ports, use the `-punch_port` option.

Specifying Settings for Multivoltage Designs

For multivoltage designs, to

- Ensure that primary and secondary voltage areas are honored and cells are added in logical hierarchies that are physically in the corresponding voltage areas, use the `-respect_voltage_areas` option with the `add_buffer_on_route` command.
- Specify different library cells for different voltage areas, use the `-voltage_area_specific_lib_cells` option and specify the list of library cells using the {va1 lib_cell1 va2 lib_cell2 ...} format.

  When you use this option, you must also use the `-lib_cell` option.

- Specify different single-rail and dual-rail library cells for different voltage areas, use the `-select_mv_buffers` option and specify the list of library cells by using the {va1 single_rail_lib_cell1 dual_rail_lib_cell1 va2 single_rail_lib_cell2 dual_rail_lib_cell2 ...} format.

Feedback

When you use this option, you must also use the `-lib_cell` option. For voltage areas that you do not specify with this option, the tool uses library cells specified with the `-lib_cell` option. For the default voltage area, specify DEFAULT_VA as the voltage area name.

- Allow buffers on physical feedthrough nets of a voltage area, specify the voltage areas with the `-allow_physical_feedthrough_buffer` option.

  This option can only be used with the `-respect_voltage_areas` or `-voltage_area_specific_lib_cells` option, and the voltage area you specify must be a primary voltage area.

  After you run the `add_buffer_on_route` command with this option, update the supply net and power domain setting for the added buffers by using the `set_eco_power_intention` command.

- Add cells within gas stations of specified supply nets, use the `-respect_gas_station` option and specify the supply net.

  Gas stations are areas with a constant power supply. These areas used for buffering nets that go through power domains that are powered down.

  To specify the maximum allowed distance from the gas station to the added buffer, use the `-max_distance_route_to_gas_station` option.

Specifying Settings for the Freeze Silicon ECO Flow

When you set the `design.eco_freeze_silicon_mode` application option to `true` and run the `add_buffer_on_route` command in the freeze silicon mode, the tool checks if a spare cell is available within a distance of five times the unit site height before adding a buffer. If a spare cell is not available, the tool does not add the buffer.

- To control this distance, use the `-max_distance_to_spare_cell` option.
- To disable this feature, use the `-not_spare_cell_aware` option.

Optimizing the Fanout of a Net

During the timing ECO flow, you can add buffers to a net and optimize its fanout by using the `split_fanout` command.

With this command, you must specify

- The net to optimize by specifying its name by using the `-net` option or its driver by using the `-driver` option
- The library cell to use by using the `-lib_cell` option

Feedback

- The method in which to optimize the net by specifying one of the following:
  - A maximum fanout constraints for the net by using the `-max_fanout` option
  - The load pins or ports to buffer by using the `-load` option

    When you use this option, you can specify the logical hierarchy to add the buffer by using the `-hierarchy` option.

In addition, you can

- Add the buffers on the existing route topology of the net by using the `-on_route` option

  If you use this option with the `-load` option, you cannot specify the `-hierarchy` option.
- Split the fanout of a routed net that is not fully connected to the terminals of the driver or loads of the net by specifying a maximum distance between terminals of the driver or load pins and the incomplete route using the `-max_distance_for_incomplete_route` option.

  If the tool is unable to find a routed segment or finds more than one routed segment within the specified distance from the unconnected terminal, it issues an error message.
- Avoid placement blockages and macro cells when placing buffers by using the `-respect_blockages` option
- Specify a name prefix for the new cells and nets by using `-cell_prefix` and `-net_prefix` options

  By default, the tool uses eco_cell and eco_net as the name prefix for the new cells and nets.

Reporting Available Sites for Placing ECO Cells

You can report sites available for placing ECO cells by using the `report_cell_feasible_space` command. By default, the tool reports the available sites in the entire block. You can restrict the report to specific voltage areas by using the `-voltage_areas` option or to a specific rectangle or rectilinear area by using the `-boundary` option.

Identifying and Reverting Nonoptimal ECO Changes

You can identify if ECO changes made with the `size_cell`, `add_buffer`, and `add_buffer_on_route` commands can cause large displacements during ECO placement and legalization by using the `report_eco_physical_changes` command.

Feedback

You can revert the nonoptimal ECO changes made by the `size_cell`, `add_buffer`, and `add_buffer_on_route` commands by using the `revert_eco_changes` command. To specify the cells for which to revert the ECO changes, use the `-cells` option.

Placing ECO Cells

For the unconstrained ECO flow, place the ECO cells by using the `place_eco_cells` command. This command derives the placement of each ECO cell based on the connectivity and the delays associated with the cell. The tool then legalizes each ECO cell to the closest unoccupied site. The existing cells are untouched to minimize the impact to their placement.

To place

- All unplaced cells, use the `-unplaced_cells` option.
- Specific cells, use the `-cells` option.
- Only the cells that have changed due to ECO operations, use the `-eco_changed_cells` option.

  A cell is considered changed if the `eco_change_status` attribute of the cell has one of the following values:

  - `create_cell`
  - `change_link`
  - `add_buffer`
  - `size_cell`

  When you run the `eco_netlist` command, the tool automatically sets the `eco_change_status` attribute on the changed cells. If you update the design without using the `eco_netlist` command, you can manually set the `eco_change_status` attribute by using the `set_attribute` command.

Controlling Placement When Using the place_eco_cells Command

The `place_eco_cells` command places and legalizes each ECO cell based on the connectivity and the delays associated with the cell.

You can control the placement of the ECO cells as follows:

- To use the channels areas between macro cells for ECO placement, use the `-channel_aware` option.

  By default, the command avoids channel areas during ECO placement.

Feedback

- To place ECO cells closer to fixed points, such as I/O pads or macro cells, specify a weight for the nets connected to the fixed cells by using the `-fixed_connection_net_weight` option.

  By default, nets connected to fixed cells have a weight of one. To place ECO cells closer to fixed points, specify an integer value greater than one for the corresponding net.

- To prioritize specific nets during ECO placement by specifying net weights,

  1. Specify net weights by using the `set_eco_placement_net_weight` command.
  2. Specify that the tool honors the net weights by using the `-honor_user_net_weight` option with the `place_eco_cells` command.

  To report the net weights you specify, use the `report_eco_placement_net_weight` command.

- To create virtual connections for ECO cells and use them during ECO placement, instead of the actual connections,

  1. Create virtual connections by using the `create_virtual_connection` command
  2. Use these virtual connections during ECO placement by using the `-use_virtual_connection` option with the `place_eco_cells` command.

  The following example creates virtual connections for the inputs and output of the cell named ECO17 and performs ECO placement using these virtual connections:

```
icc2_shell> create_virtual_connection -name VC_0 \
   -pins {U1/Y ECO17/A}
icc2_shell> create_virtual_connection -name VC_1 \
   -pins {U2/Y ECO17/B}
icc2_shell> create_virtual_connection -name VC_2 \
   -pins {ECO17/Y D17_OUT} -weight 3
icc2_shell> place_eco_cells -cells ECO17 -use_virtual_connection
```

  To remove virtual connections you have created, use the `remove_virtual_connections` command. To query virtual connections, use the `get_virtual_connections` command.

- To ignore high-fanout nets connected to ECO cells that exceed a specific threshold, use the `-max_fanout` option with the `place_eco_cells` command.
- To ignore the connections of specific pins on ECO cells, use the `-ignore_pin_connection` option with the `place_eco_cells` command.
- To limit movement to only those cells that are within a specified distance from the ECO cells, use the `-moveable_distance` option with the `place_eco_cells` command.

Feedback

Controlling Legalization When Using the place_eco_cells Command

After placing the ECO cells, the `place_eco_cells` command legalizes each ECO cell to the closest unoccupied site. The existing cells are untouched to minimize the impact to their placement.

You can control the legalization of the ECO cells by using one of the following methods:

- To not legalize the cells after ECO placement, use the `-no_legalize` option.
- To perform legalization only, use the `-legalize_only` option.
- To specify the mode in which to legalize, use the `-legalize_mode` option with one of the following settings:
  - `free_site_only`

    When you specify this setting, the default, the tool legalizes the ECO cells on free sites without moving preexisting cells. An ECO cells can have a large displacement, if a free site is unavailable nearby.
  - `allow_move_other_cells`

    When you specify this setting, the tool legalizes the ECO cells to the nearest legal location by moving preexisting cells.
  - `minimum_physical_impact`

    When you specify this setting, the tool first legalizes the ECO cells the can be legalized to a free site nearby without moving preexisting cells. For the ECO cells that do not have a free site nearby, the tool legalizes them by moving preexisting cells.
- To specify a displacement threshold for legalization, use the `-displacement_threshold` option.

  When you use the `-displacement_threshold` option, you can also use the `-max_displacement_threshold` option to specify a second displacement threshold that is larger than the value specified with the `-displacement_threshold` option.

  How the tool uses these displacement thresholds depends on the setting you use for the `-legalize_mode` option as follows:
  - `free_site_only`

    When you specify this setting, the tool legalizes the ECO cells that have available free sites within the specified displacement threshold.

Feedback

If the displacement of an ECO cell exceeds this threshold, the tool does not legalize the cell. It creates a collection named epl_legalizer_rejected_cells that consists of such ECO cells that are not legalized.

You can subsequently legalize the ECO cells in the epl_legalizer_rejected_cells collection by using the following command:

```
icc2_shell> place_eco_cells
   -legalize_mode allow_move_other_cells \
   -legalize_only -cells $epl_legalizer_rejected_cells
```

This command moves existing cells to find legal locations for the ECO cells in the collection named epl_legalizer_rejected_cells.

If you also specify the `-max_displacement_threshold` option with `-displacement_threshold` option, the tool also creates a collection named epl_max_displacement_cells that consists of ECO cells with a displacement larger than that specified by this option. The collection epl_max_displacement_cells is a subset of the collection epl_legalizer_rejected_cells and its cells are also not legalized.

You can use the `-max_displacement_threshold` option to identify ECO cells with a very large displacement, for which you want to reject the ECO changes.

- `allow_move_other_cells`

  When you specify this setting, you cannot specify the `-displacement_threshold` and `-max_displacement_threshold` options.

- `minimum_physical_impact`

  When you specify this setting, the tool first legalizes the ECO cells that have available free sites within the specified displacement threshold.

  If the displacement of an ECO cell exceeds this threshold, the tool legalizes them by moving preexisting cells.

  If you also specify the `-max_displacement_threshold` option, the tool does not legalize the cells that exceed this maximum displacement threshold specified with this option and the tool creates a collection named epl_max_displacement_cells that consists of ECO cells with a displacement larger than that specified by this option. You can use the `-max_displacement_threshold` option to identify ECO cells with a very large displacement, for which you want to reject the ECO changes.

- To specify the types of filler cells that can be removed during legalization, use the `-remove_filler_references` option.

Feedback

When you use this option, the filler cells are removed if they do not have a fixed placement and they overlap with ECO cells. By default, the tool does not remove any filler cells when legalizing ECO cells.

- To enable Advanced Legalizer and remove the filler cells that have violations, set the `place.legalize.enable_advanced_legalizer` application option to `true`.

  When you specify this setting, the filler cells that have violations are removed after legalizing ECO cells. Using the `-remove_filler_references` option with this application option set to `true` results in the legalizer engine removing only those fillers that are specified by this option.

  The `-min_filler_distance` option is ignored when you use the `place.legalize.enable_advanced_legalizer` application option.

Placing ECO Cells With Minimal Physical Impact (MPI)

To minimize the disturbance to the existing placement during ECO placement, use the following options with the `place_eco_cells` command:

- `-legalize_mode minimum_physical_impact`
- `-max_displacement_threshold`
- `-remove_filler_references`

Optimizing ECO Cell Legalization

You can streamline the process of legalizing ECO cell placement with minimal disturbance to existing cells.

To optimize ECO cell legalization,

1. Enable optimized legalization by setting the `eco.placement.optimized_legalization` application option to `true`. The default for this application option is `false`.

   ```
   icc2_shell> set_app_options -name eco.placement.optimized_legalization
    -value true
   ```

2. Limit the movement of existing cells around ECO cells using the `-moveable_distance` option of the `place_eco_cells` command.

   ```
   icc2_shell> place_eco_cells -cells $ecoCells -legalize_mode
    minimum_physical_impact -moveable_distance 10
   ```

   This restricts the legalizer to move only those cells that are within the specified distance from the ECO cells.

Feedback

Alternatively, you can legalize ECO cells with minimal input by using the `legalize_eco_cells` command. The `-displacement_tolerance_level` option of this command accepts two values - `low` and `high`. The `low` value denotes stricter displacement and less movement allowed. The `high` value denotes more relaxed displacement, allows greater movement.

```
icc2_shell> legalize_eco_cells -cells $ecoCells
 -displacement_tolerance_level low
```

This command is equivalent to:

```
icc2_shell> set_app_options -name eco.placement.optimized_legalization
 -value true
icc2_shell> place_eco_cells -cells $ecoCells -legalize_mode
 minimum_physical_impact -legalize_only -displacement_tolerance_level low
```

Using the `legalize_eco_cells` command:

- Reduces movement of both ECO and existing cells, preserving original placement as much as possible.
- Allows you to specify tolerance levels instead of exact displacement values, thus simplifying setup.
- Reduces manual configuration
- Ensures robust placement of ECO cells

Placing and Mapping ECO Cells to Spare Cells

For the freeze silicon ECO flow, place and map the ECO cells to spare cells by using the `place_freeze_silicon` command. By default, this command places and maps all ECO cells. To place and map specific ECO cells, use the `-cells` option. The following example places and maps the ECO cells named ECO1, ECO2, and ECO3:

```
icc2_shell> place_freeze_silicon -cells {ECO1 ECO2 ECO3}
```

This command places each ECO cell and maps it to the nearest matching spare cell. Cells that are deleted as a result of the ECO changes are not removed from the design. They are converted to spare cells and remain in the design.

When you use the `place_freeze_silicon` command, you can

- Place the ECO cells by the target spare cells, but not map them to the spare cells, by using the `-no_spare_cell_swapping` option

  This feature allows you to place the ECO cells and analyze the QoR before you map them.

Feedback

- Map the spare cells that are already placed by the target spare cells by using the `-map_spare_cells_only` option.
- Generate an ECO cell to spare cell mapping file by using the `-write_map_file` command.

  You can edit the mapping file, if necessary, and map the ECO cells by using the `map_freeze_silicon` command.
- Specify a minimum distance between ECO cells that are mapped to programmable filler cells by using the `-min_filler_distance` option.

Specifying Mapping Rules for Programmable Spare Cells

You can define mapping rules for programmable spare cells to control the

- Overlapping of spare cells and PG nets
- Splitting of multiple-height and merging of single-height spare cells
- Compatibility of the different types of spare cells

To do so, use the `set_programmable_spare_cell_mapping_rule` command before you run the `place_freeze_silicon` command.

To report or remove the mapping rules you specify, use the `report_programmable_spare_cell_mapping_rule` command.

Mapping ECO Cells to Specific Spare Cells

To map an ECO cell to a specific spare cell, use the `map_freeze_silicon` command. Specify the ECO cell name and the corresponding spare cell name by using the `-eco_cell` and `-spare_cell` options. The following example maps the ECO cell named ECO1 to the spare cell named spare_1:

```
icc2_shell> map_freeze_silicon -eco_cell ECO1 -spare_cell spare_1
```

You can specify the ECO cell to spare cell mapping by using a map file and specifying this map file name by using the `-map_file` option with the `map_freeze_silicon` command. The map file has the following format:

```
ECO_cell_1    spare_cell_1
ECO_cell_2    spare_cell_2
...           ...
```

You can specify the location of the ECO cell inside the specified programmable spare cell (PSC) filler by using the `-filler_map_strategy` option. The values of this option are `pack_left`, `pack_right`, `pack_top`, `pack_bottom`, and `closest`.

Feedback

Note:

The `-filler_map_strategy` option is mutually exclusive with the `-map_file` option and can only be used together with the `-eco_cell` and `-spare_cell` options. When you use the `-map_file` option, you can specify the location strategy inside the map file.

To map ECO cells to spare cells for advanced node technologies, use the `-advanced_nodes` option. This is especially useful because automated mapping might not provide the required control or debuggability for advanced nodes.

To skip PG rule check and all legality rule checks, use the `skip_rule_check` option. This option is only available when you use the `-advanced_nodes` option.

Note:

- The `-advanced_nodes` and `-filler_map_strategy` options are mutually exclusive.
- The `-advanced_nodes` and `-map_file` options are mutually exclusive.

Mapping ECO Cells to Logically Equivalent Spare Cells

When the tool maps an ECO cell, it looks for a spare cell with the same library cell name. If it is unable to find such a spare cell, the ECO cells remains unmapped. To identify and map such ECO cells to logically equivalent (LEQ) spare cells, use the following steps:

1. Identify the ECO cells that do not have matching spare cells by using the `check_freeze_silicon` command.
2. Find logically equivalent spare cells for these ECO cells by using the `create_freeze_silicon_leq_change_list` command.

   Specify

   - The names of the unmapped ECO cells by using the `-cells` option
   - A name for the output file by using the `-output` option

     This output file is a Tcl script with netlist editing commands. The commands replace each ECO cell with logically equivalent spare cells, which can be a

     - Single spare cell with the same logical functionality, but a different library cell name
     - Combination of up to two spare cells that give the same logical functionality

3. View the output Tcl file to ensure that the ECO mapping is satisfactory, and edit it if necessary.
4. Source the Tcl file in the tool to replace the ECO cells with their logical equivalents.

Updating Supply Nets for ECO Cells

To update the supply nets for ECO cells, use the `eco_update_supply_net` command. By default, it updates the supply nets for all cell added by using the following ECO commands:

- `size_cell` in freeze-silicon mode
- `add_buffer`
- `add_eco_repeater`
- `split_fanout`
- `add_buffer_on_route`

To update the supply nets for specific cells, use the `eco_update_supply_net -cells` command.

Recording the Changes Made to a Layout

You can record the changes you make to a layout and generate a Tcl file containing the changes by using the `record_layout_editing` command, as shown in the following example:

```
icc2_shell> record_layout_editing -start
icc2_shell> remove_shapes RECT_32_0
icc2_shell> record_layout_editing -stop -output layout_changes1.tcl
```

You can make the following layout changes:

- Create or remove layout objects by using Tcl commands
- Set attributes by using the `set_attribute` command
- Move or resize objects by using the GUI

With this feature, multiple users can make ECO changes to different parts of a layout in parallel. Each user can output a Tcl file that contains the layout changes they make by using the `record_layout_editing` command, and all the Tcl files can be applied to the original layout.

Feedback

Performing Prerequisite Check for Group Repeater Insertion and Placement

Use the `-check_prerequisites` option of the `place_group_repeaters` or `add_group_repeaters` command to perform a prerequisite check for a design. This prerequisite check detects the design errors at the beginning of the command flow.

- For the `place_group_repeaters` command, use the `-check_prerequisites` option with the following options to provide necessary information for the check:

```
place_group_repeaters
   -cells | -repeater_groups
   [-lib_cell_input]
   [-lib_cell_output]
   -check_prerequisites
```

- For the `add_group_repeaters` command, use the `-check_prerequisites` option with the following options to provide necessary information for the check:

```
add_group_repeaters
   -nets | -bundles
   [-lib_cell_input]
   [-lib_cell_output]
   -check_prerequisites
```

- The prerequisite check validates the following in a design and search region:
  - Site rows are present in the design.
  - Each net has only one driver and can have multiple loads.
  - Locations of pins or ports are assigned.
  - For pins on standard cells, the standard cells are placed.
  - Supernets must be routed.
  - Route ends must not be more than 5 µm away from the driver, load pins, or ports in the search region.
  - The routes are defined for a driver or load in the search region.
  - Load is connected to a driver.
  - When the end of a route is open, no other route of the same net is present in the vicinity of the pin or port.
  - Routes must be assigned to the net of the supernet driver pin.

Feedback

- Vias must be created for routes.
- Repeaters have a physical library cell.

The following example shows the usage of `-check_prerequisites` option using the `add_group_repeaters` command:

```
icc2_shell> add_group_repeaters -bundles bundle_wo_load \
   -lib_cell ref_lib1/libCell1 -lib_cell_input A -lib_cell_output X \
   -repeater_distance 10 -check_prerequisite
Error: The net net_wo_load does not have loads. (ECO-329)
Error: The cell driver1 of the pin driver1/X is not placed. (ECO-331)
Error: 0
        Use error_info for more info. (CMD-013)
```

Adding a Group of Repeaters

To add a group of repeaters, use the `add_group_repeaters` command as described in the following sections:

- Defining a group of repeaters
- Grouping a list of repeaters
- Setting Constraints for a Group of Repeaters
- Adding Voltage Area Aware Group Repeaters
- Reporting the Constraints Assigned to a Group of a Repeaters
- Removing Constraints for a Group of Repeaters
- Placing Group Repeaters Before Routing
- Performing On Route Placement of Repeaters
- Placing Group Repeaters For Multibit Registers
- Specifying Locations for Repeater Groups
- Allowing Repeater Groups Over Macros
- Specifying Cut Space and Cut Distance for Repeater Groups
- Specifying Horizontal and Vertical Spacing for Repeater Groups
- Specifying Library Cells as Repeaters
- Avoiding Overlapping Repeaters With Existing Tap Cells
- Avoiding Crosstalk During Group Repeater Insertion

Feedback

- Previewing Repeater Groups
- Unplacing the Repeaters
- Removing Repeater Groups

Defining a group of repeaters

To define a group of repeaters for cutline support, use the `set_repeater_group` or `create_repeater_groups` command.

The following examples show the command usage.

```
set_repeater_group -group_id group_id \
   -check_connection  \
   [-cells cell_list] \
   [-cutline { {{x1 y} {x2 y}} | {{x y1} {x y2}} }] \
   [-driver_group_id group_id] \
   [-path_drivers pin_port_list] \
   [-path_loads pin_port_list] \
   [-override]
   [-lib_cell_input string]
   [-lib_cell_output string]
   [-clear]
```

Grouping a list of repeaters

To automatically group a list of repeaters or repeaters of supernets, use the `create_repeater_groups` command.

This command takes a list of repeaters, a list of supernets, or bundles of supernets and returns a list of repeater groups created by the `set_repeater_group` command.

The command supports both single load and multiple loads supernets.

```
create_repeater_groups -supernets supernet_list | \
   -supernet_bundles supernet_bundle_list | \
   -cells cell_list \
   -lib_cells lib_cell_list \
   [-lib_cell_input lib_pin] \
   [-lib_cell_output lib_pin] \
   [-group_pin_spacing spacing] \
   [-ignore_pin_layers]
```

Setting Constraints for a Group of Repeaters

To define the constraints for a group of repeaters, use the `set_repeater_group_constraints` command.

Feedback

The set_repeater_group_constraints command has a lower priority than the corresponding command options in the add_group_repeaters and place_group_repeaters commands.

```
set_repeater_group_constraints -type type_list \
   [-horizontal_repeater_spacing spacing] \
   [-vertical_repeater_spacing spacing] \
   [-layer_cutting_distance layer_scale_list]
```

Adding Voltage Area Aware Group Repeaters

Voltage area aware group repeaters choose legal library cells for repeaters based on the corresponding voltage area and updates supply nets for repeaters to meet multivoltage constraints. You can use the add_group_repeaters command with specific options to add voltage area aware group repeaters in three different ways, depending on the design methodology:

- When the design is single-rail buffer only, use the -va_aware option

  ```
  icc2_shell> add_group_repeaters -lib_cell <lib_cell> -va_aware
  ```

- When the design is single-rail buffer based on voltage area, use the -voltage_area_specific_lib_cell option

  ```
  icc2_shell> add_group_repeaters -lib_cell <default_lib_cell> \
     -voltage_area_specific_lb_cell {{VA1 libcell1}} {{VA2 libcell2}}
  ```

- When the design is single- and dual-rail buffers, use the -select_mv_buffer option

  ```
  icc2_shell> add_group_repeaters -lib_cell <default_lib_cell> \
     -select_mv_buffer {{VA1 SingleRail1 DualRail1}} \
     {{VA1 SingleRail2 DualRail2}}
  ```

Voltage area aware group repeaters help resolve multivoltage violations by choosing the library cell based on voltage area and repeater location.

Reporting the Constraints Assigned to a Group of a Repeaters

To report the constraints that were assigned to a group of repeaters using the place_group_repeaters, add_group_repeaters, or set_repeater_group_constraints commands, use the report_repeater_group_constraints command. For example:

```
icc2_shell> report_repeater_group_constraints
```

Feedback

Removing Constraints for a Group of Repeaters

To remove the constraints that were assigned to a group of repeaters, use `remove_repeater_group_constraints` command. For example:

```
icc2_shell> remove_repeater_group_constraints -type type_list
```

Placing Group Repeaters Before Routing

Use the `preplace_group_repeaters` command to place the group repeaters before the routing. This command places the group repeaters either in an ascending or descending order, default is ascending. The ascending order means that the command places the cells in a group from the lowest or left-most available track to the highest or right-most track.

Note:

A FAA-Base-Beta license is required to run the `preplace_group_repeaters` command.

You must specify either the cell collections or group IDs of the group repeaters.

The following example places group repeaters when the cell collections are specified.

```
icc2_shell> preplace_group_repeaters      \
   -repeater_group_locations { { $cellGroup1, M2, {720 363.5}, \
    north, descending} {$cellGroup2, M2, {920 363.5}, east, ascending} \
   {$cellGroup3, M2, {1230 328.5}, east, ascending} \
   {$cellGroup4, M2, {1430 328.5}}, east}
```

The following example places group repeaters when the group ID is specified.

```
icc2_shell> preplace_group_repeaters \
   -repeater_group_locations { { 1,M2, {720 363.5}, south, ascending} \
   {2, M2, {920 363.5},west, ascending} {3, M2, {1230 328.5}, \
   south, ascending} {4, M2, {1430 328.5}, west, descending}}
```

Performing On Route Placement of Repeaters

To perform on route placement of interconnect repeaters, use the `place_group_repeaters` command.

```
place_group_repeaters \
   -cells cell_list | -repeater_groups group_id_list \
   [-lib_cell_input pin_name] \
   [-lib_cell_output pin_name] \
   [-max_distance_for_incomplete_route distance] \
   [-first_distance distance] \
   [-last_distance distance] \
   [-blockage_aware] \
```

Feedback

```
[-horizontal_repeater_spacing spacing] \
[-vertical_repeater_spacing spacing] \
[-layer_cutting_distance layer_scale_list]
[-verbose]
```

Placing Group Repeaters For Multibit Registers

To place group repeaters for multibit registers, use the `create_repeater_groups` command with the `-multi_bit` option with the following command options:

```
create_repeater_groups
 -cells | -supernet_bundles | -supernets
 -multi_bit
 -pin_mapping {{lib_cell1 {inp1 outp1} {inp2 outp2}…}
```

The `-multi_bit` option determines the order of a multibit repeater based on the path driver and creates the repeater groups for the multibit repeaters. In a design, when a net has no route defined, this option places the group repeaters based on their location.

To verify the support for multibit register placement, use the following command:

```
icc2_shell> preplace_group_repeater
   -repeater_group_locations repeater_group_location_list
```

The format for the *`repeater_group_location_list`* argument is

```
{ {cell_collection, layer, location, potential routing direction, placing
 order} ...}
```

or

```
 { {group_id, layer, location, potential routing direction, placing
 order} ...}.
```

Specifying Locations for Repeater Groups

Use one of the following mutually exclusive options to control repeater group location:

- `-repeater_distance` specifies the distance between repeater groups. This option supports the following additional options:
  - `-first_distance` specifies the distance between the driver and the center of the first repeater group.
  - `-min_distance_repeater_to_load` specifies the minimum distance between the last repeater group and the load. The default is one half of the value specified for the `-repeater_distance` option.
  - `-tolerance` specifies the maximum delta distance over the value specified for `-repeater_distance`. The default is 5 µm.

Feedback

- `-relative_distance` specifies the distance between the driver and the first repeater group, the distance between the first and second repeater groups, the distance between the second and third repeater group, and so on.
- `-location` specifies the location of the center of each repeater group.
- `-number_of_repeater_groups` specifies the total number of repeater groups to be inserted evenly along the route.
- `-cutlines` specifies the location pairs that define the cutlines. Cutlines must be perpendicular to the routes and the repeater groups are centered around the cutlines.

Allowing Repeater Groups Over Macros

Use the `-allow_insertion_over_block` option to allow repeater groups to be added on top of soft or hard macros.

Note:

If the repeater cell is an inverter, you are responsible for ensuring logical correctness.

Specifying Cut Space and Cut Distance for Repeater Groups

Use one of the following mutually exclusive options to control cutting for repeater groups:

- `-layer_cutting_spacing` specifies the cut space of the routes to the driver pin. The cut starts from the driver side.
- `-layer_cutting_distance` specifies the cut distance from the repeater center to the input and output routes.

Specifying Horizontal and Vertical Spacing for Repeater Groups

Use one of the following mutually exclusive options to control spacing for repeater groups:

- `-horizontal_repeater_spacing` specifies the horizontal spacing between repeaters by cell site width.
- `-vertical_repeater_spacing` specifies the vertical spacing between repeaters by site row height.

Feedback

Specifying Library Cells as Repeaters

Use the `-lib_cell` option to specify a library cell as a repeater. This option supports the following additional options:

- `-lib_cell_input` specifies the input pin of the library cell to connect, if it has multiple inputs.
- `-lib_cell_output` specifies the output pin of the library cell to connect, if it has multiple outputs.

Avoiding Overlapping Repeaters With Existing Tap Cells

Use the `-honor_special_cell` option to avoid overlapping with existing tap cells when inserting repeaters.

Avoiding Crosstalk During Group Repeater Insertion

The crosstalk management is only supported in the `create_group_repeaters_guidance` and `add_group_repeaters` commands flow.

The flow to avoid crosstalk during group repeater insertion is as follows:

1. Use the following command to query interleaving groups:

```
create_group_repeaters_guidance -net
 -number_of_interleaving_groups number
```

   By default, the *number* argument is set to 1.

   For example, if the value of the *number* is 2, one net group is separated into two interleaving net groups.

2. Use the following command to manage crosstalk during group repeater insertion:

```
add_group_releaters
   -nets | -bundles
   -first_distance_of_net_groups
       { {group_id1 $first_dist1} {group_id2
   $first_dist2} …}
     -repeater_distance $dist
```

   or

```
add_group_releaters
    -nets | -bundles
    -cutlines_of_net_groups
       { {group_id1 $cutline_list1} {group_id2 $cutline_list2} …}
```

Feedback

- If the option `-first_distance_of_net_groups` is specified, the command groups the nets based on the existing group IDs on nets and inserts repeaters based on the specified first distance for the various net groups. The repeater distance is same for all net groups.
- If the option `-cutlines_of_net_groups` is specified, the command groups the nets based on the existing group IDs of the nets and inserts repeaters based on the cutlines for the various net groups.

Previewing Repeater Groups

Use the `-preview` option to preview repeater group locations before insertion. The repeaters are represented by annotation objects at the same locations where they are to be inserted, with the same width and height as a repeater lib cell, and with the group ID `eco_preview_id`.

Note:

Before previewing the repeater groups again, you must first remove the annotation objects using either the `gui_remove_annotations` or `gui_remove_all_annotations` command.

Unplacing the Repeaters

To unplace the previous placement of repeaters, use the `unplace_group_repeaters` command.

```
unplace_group_repeaters \
   -cells cell_list | -repeater_groups group_id_list \
   [-lib_cell_input pin_name] \
   [-lib_cell_output pin_name]
```

Removing Repeater Groups

To remove a group of repeaters, use the `remove_eco_repeater` command. The nets are restored to their original names and routes shapes.

Querying Group Repeater

The group repeater query commands provide improved ease of use for interconnect repeater while planning. The group repeater query

- Provides capability to query group repeater path info
- Supports cross-session query operation
- Supports query operation before group repeater placement

To query the group repeater during planning, you must be familiar with the following key information

- Cells of a repeater group
- Cutline of a repeater group
- Group virtual connection

If a group has either driver group or path drivers, then it is categorized as group virtual connection. Cell connection between groups depend on cell order in a group.

The following topics describe the steps in this flow

- Performing Auto Grouping Flow
- Performing Manual Grouping Flow
- Cell Input Mode

Performing Auto Grouping Flow

Auto repeater group creation allows you to derive virtual connections for register paths. For repeater group input mode, `place_group_repeaters` applies cutline based placement. You must specify cutline for each group. Repeaters are placed based on the intersection point between cutline and route.

Build connectivity based on real connections for target cells.

1. To group a list of repeaters or repeaters of supernets, use the `create_repeater_groups` command. Also, specify the constraint option.

   The command automatically performs cell grouping and groups virtual connection with the following attributes:

   - group_repeater_driver
   - group_repeater_loads
   - pin_pair
   - group_id

2. Verify the path with the `get_attribute`, `report_repeater_group`, and `get_repeater_path_info` commands. The `get_attribute` command reports the

Feedback

attributes of the `group_repeater_driver` or `group_repeater_loads` command. The command `report_repeater_groups` reports group information for specified groups or all groups, and reports repeater paths for all groups if `-verbose` is on. The `get_repeater_paths_info` command returns cells on repeater paths that include input cells.

3. Specify the cutline for a group manually with the `set_repeater_group -group_id-cutline` command.
4. Place interconnect repeaters on route with cutline for input repeater groups.

Performing Manual Grouping Flow

Auto grouping is based on the `create_repeater_groups` command. Use manual grouping in case the auto grouping command result does not meet the requirement.

Check your target cell connection before placement. Perform the following steps for manual grouping:

1. Group your cells by -cells, cell order intends the cell connection.
2. Define group virtual connection by -driver_group_id or -path_drivers.
3. Specify the cutline by -cutline.

1. To define register path connections (virtual connection) for each set of repeater groups, use the `set_repeater_group` command.
   - To check cell connection before placement, use the `set_repeater_group -check_connection` command. If the cell connection is not placed as expected, repeat a reset repeater group for desired cell connection. It also adds new options to record pin pair for the `-check_connection` option. The `pin_pair` option is used to find cell real connection based on specified lib cell pin name.
2. Verify the path with the `get_attribute`, `report_repeater_group`, and `get_repeater_path_info` commands. The `get_attribute` command reports the attributes of the `group_repeater_driver` or `group_repeater_loads` commands. The `report_repeater_groups` command reports group information for specified groups or all groups, and reports repeater paths for all groups if `-verbose` is on. The command `get_repeater_paths_info` returns cells on repeater paths that include input cells.
3. Check and report invalid paths based on real connection with the `set_repeater_group -check connection` command.
4. Place interconnect repeaters on route with cutline for input repeater groups.

Feedback

Cell Input Mode

Cell input mode enables to set distance based on route placement. By default, repeaters are placed based on even distance on route for one repeater path. This mode allows you to set the `-first_distance` or `-last_distance` option to control the distance at driver side or load side.

1. Use the `place_group_repeaters -cells -blockage_aware` command, to internally enable auto grouping . To internally enable auto grouping use the `create_repeater_groups -cells` command and derive cutline based on distance. Then, define repeater group by using the `set_repeater_group -group_id -cutline` command,

   After placing group repeaters, you can query repeater groups using the `report_repeater_groups` command and repeater paths info using the `get_repeater_paths_info` command.

2. To establish cell connection based on real connection and apply distance based placement, use the `place_group_repeaters -cells` command. No repeater group is created.

Swapping Variant Cell

The `add_group_repeater` and `place_group_repeater` commands support variant cell in group repeater insertion and placement.

The following topics describe the steps in this flow

- Setting Constraints of Variant Cell
- Setting Application Option
- Grouping Variant Cell
- Running and Placing Group Repeaters
- Troubleshooting

Feedback

Setting Constraints of Variant Cell

To set the constraints of variant cell:

1. Use the `set_repeater_group_constraints -type variant_cell_swapping` command to set the variant for cell swapping.
2. Use the `report_repeater_group_constraints` command to view the report of the repeater group constraint settings, after setting the variant for cell swapping. The report provides the
   - Track pattern of the variant cell group
   - Variant site ids
   - Honor cell group not sharing timing
   - First track alignment
3. Use the `remove_repeater_group_constraints` command to remove the constraints and reset the settings based on the report.

Setting Application Option

To set application option before using the `add group repeaters` and `place_group_repeaters` commands, use the `set_app_option -name eco.placement.variant_cell_swapping -value true` command.

Based on the requirement, you can plan the variant cell flow. The rules of variant reference swapping differ in various variant cell flows. By default, the variant cell support in the `eco_change_legal_reference` command detects the variant mode and settings automatically.

The `eco_change_legal_reference` command checks whether the current reference of the cell is legal or illegal for the placed location. If it is illegal, the command attempts to find a legal reference from its variant cell group defined to swap with the illegal one.

To use the variant cell features directly for other cells, use the `eco_change_legal_reference` command.

Feedback

For complicated scenarios, you can switch the variant mode and tune the settings manually. Following are the new application options for variant cell support:

- To set variant mode for different customers and different design flows, use the `set_app_options -name eco.placement.variant_cell_group_mode -value` command.
- To honor the variant cell groups, which are not sharing timing view, use the `set_app_options -name eco.placement.honor_cell_group_not_sharing_timing -value` command.

Grouping Variant Cell

In the same variant cell group, the library cell variants differ from the master-variant only in mask color (or small geometry), while the timing data is same for these variants.

The variant cell feature supports the following

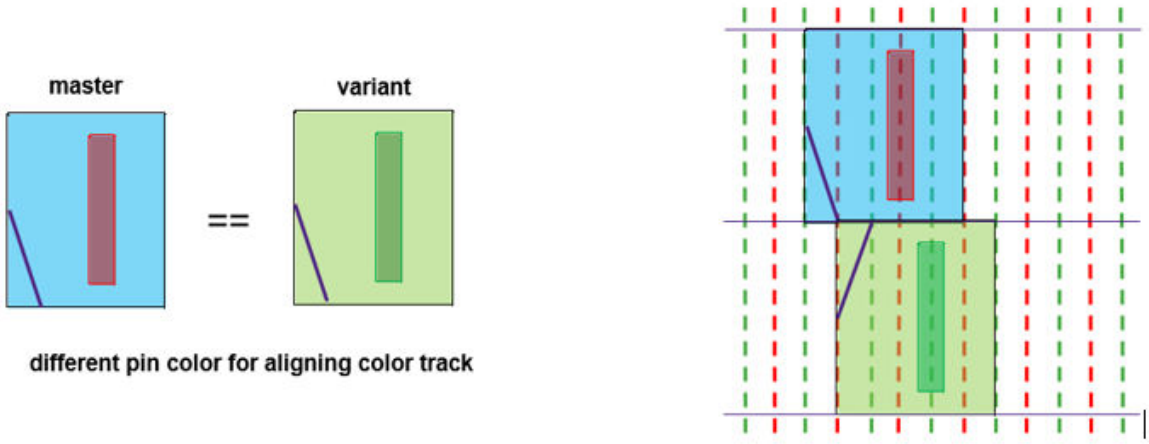
- track_pattern mode
- siteId_based mode

track_pattern mode

To specify the first metal layer for track alignment checking in track_pattern mode, use the `set_app_options -name eco.placement.first_track_alignment_layer -value name` command.

Figure 195 shows the cell group with two variants.

Figure 195 Cell Group With Two Variants

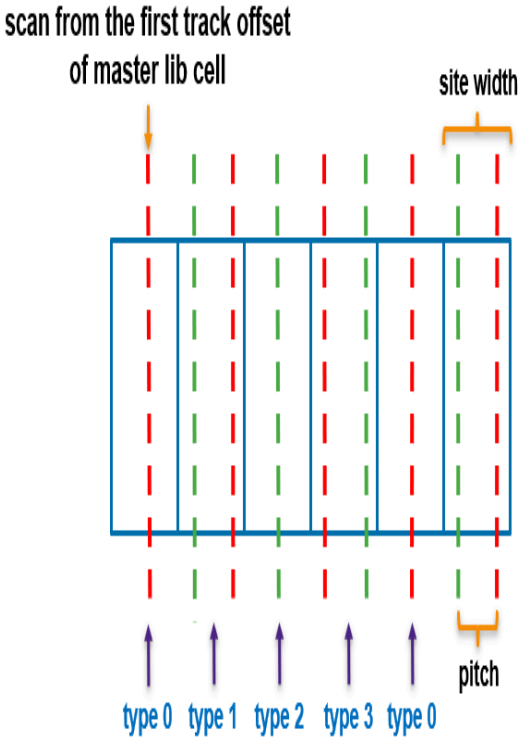


Feedback

Based on the first track offset of library cells and the pitch value, you can define and derive the variant library cell types, which start from 0.

Figure 196 shows variant type id definition.

Figure 196 Variant Type id Definition



Example (N5 design):

Type	Mask Color	Track Offset	Library Cell
0	mask_one	1/2 site width	Master
1	mask_two	1/6 site width	Variant_1
2	mask_two	1/2 site width	Variant_2
3	mask_one	1/6 site width	Variant_3

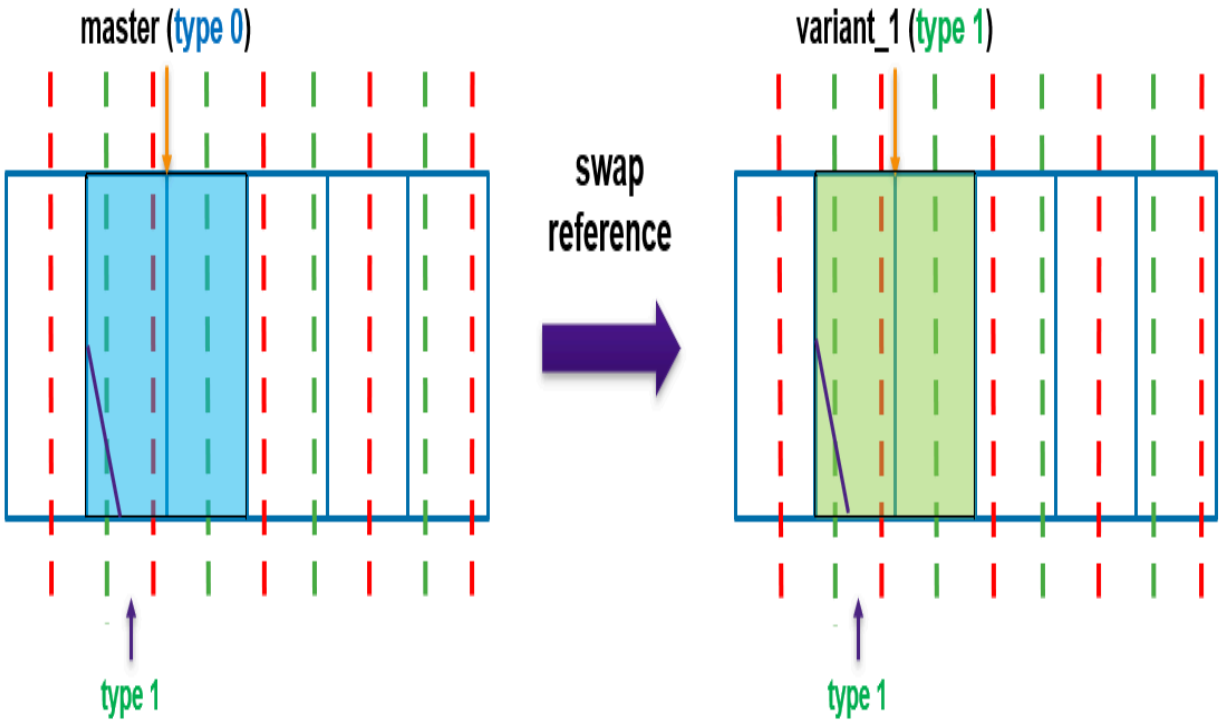
The default pin color alignment layer is M1. User can change it by setting an app option.

You can detect the variant type id for target cell current location. You can also check if the current reference is legal or illegal. If it is not legal, select the legal reference from cell group and swap it.

Figure 197 shows the check and swap with legal reference for target cell.

Feedback

Figure 197 Check and Swap With Legal Reference for Target Cell.



siteId_based mode

The variant mapping and flipped mapping information are defined in the cell group. Following is an example of the `report_cell_groups` command.

Figure 198 shows the cell group definition for mapping id and flipped mapping id.

Feedback

Figure 198 Cell Group Definition for Mapping id and Flipped Mapping id

```
icc2_lm_shell> report_cell_groups
**************************************

Report : report_cell_groups
Lib    : variant
Version: N-2017.09-SP4
Date   : Wed Feb  7 22:25:05 2018
**************************************

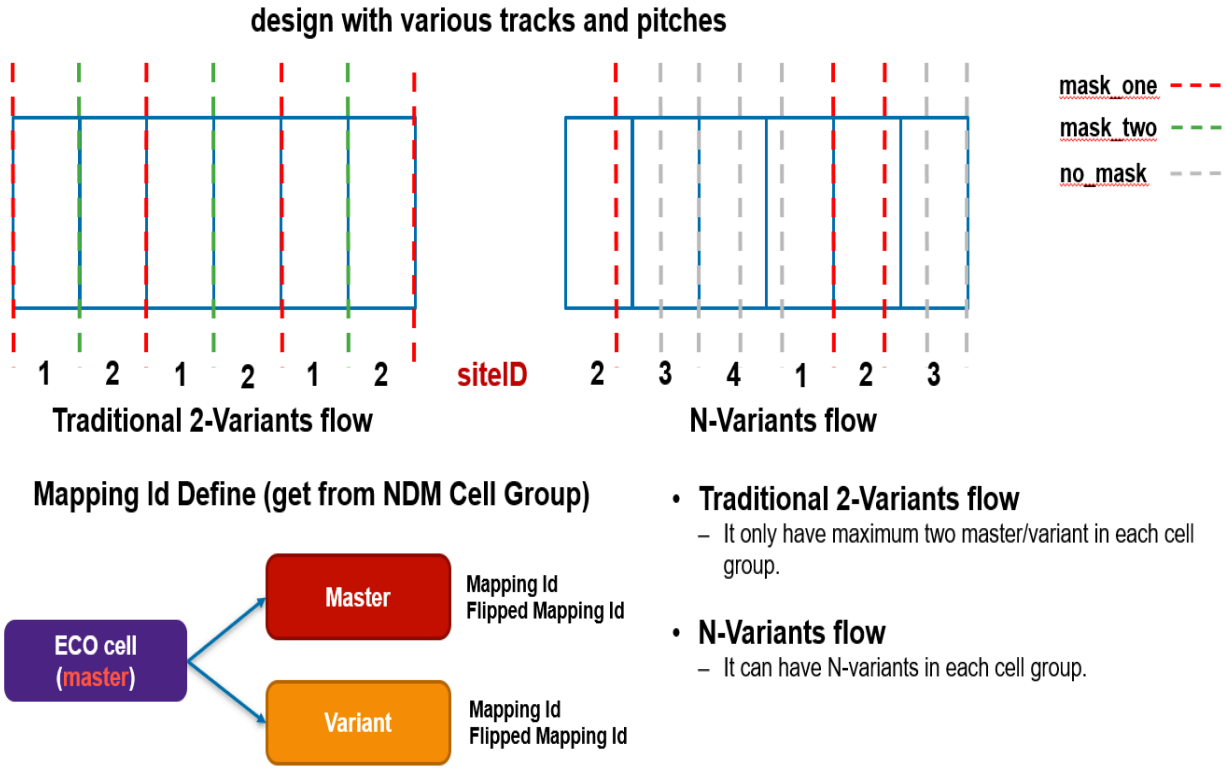
--------------------------------------------------------------------------------------------
Cell Group        Group Contents     Mapping          Flipped Mapping    Share Timing View
--------------------------------------------------------------------------------------------
A (L)         A A_M1_1 A_M1_1_M2_1 A_M2_1      1 2 3 4       1 2 3 4          true
```

There are two sub flows for siteId_based mode: 2-Variants and N-Variant.

Figure 199 shows the 2-Variants and N-Variants sub flows in siteId_based mode

Feedback

Figure 199 2-Variants and N-Variants Sub Flows in siteId_based Mode

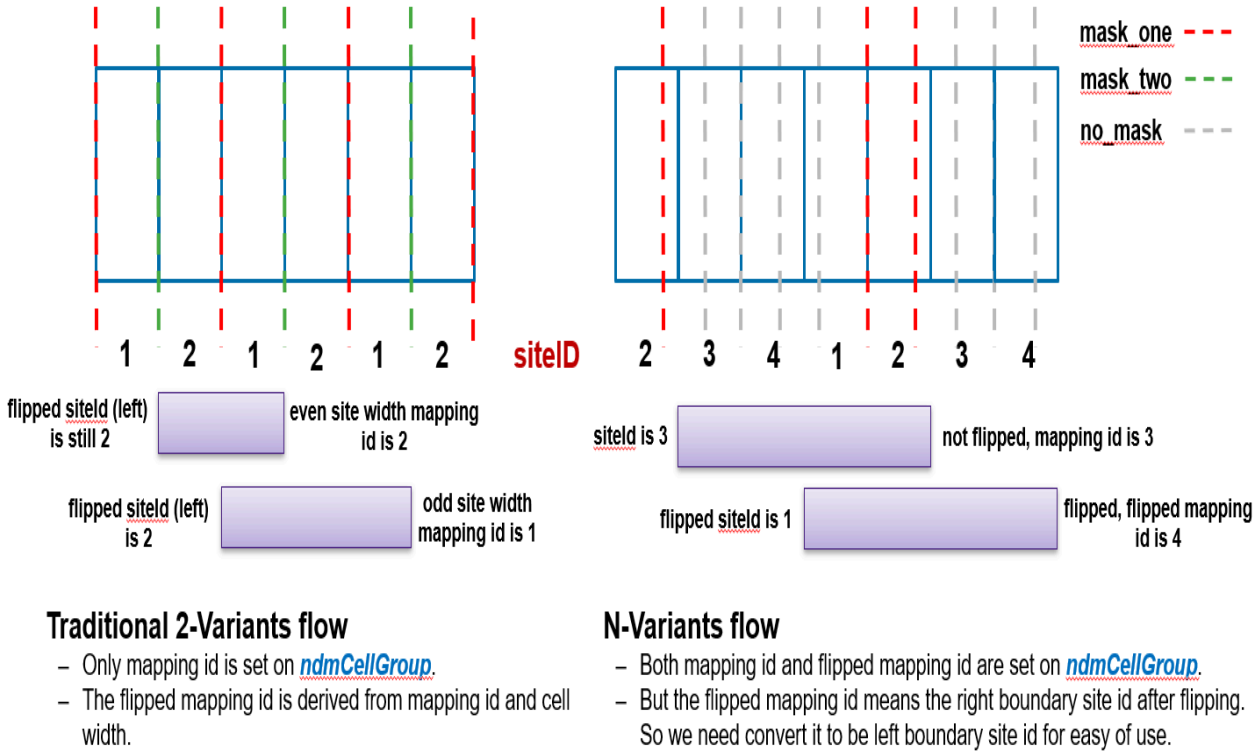


Each variant reference has both legal mapping id and flipped mapping id.

Figure 200 shows the variant mapping and flipped mapping id.

Feedback

Figure 200 Variant Mapping and Flipped Mapping id

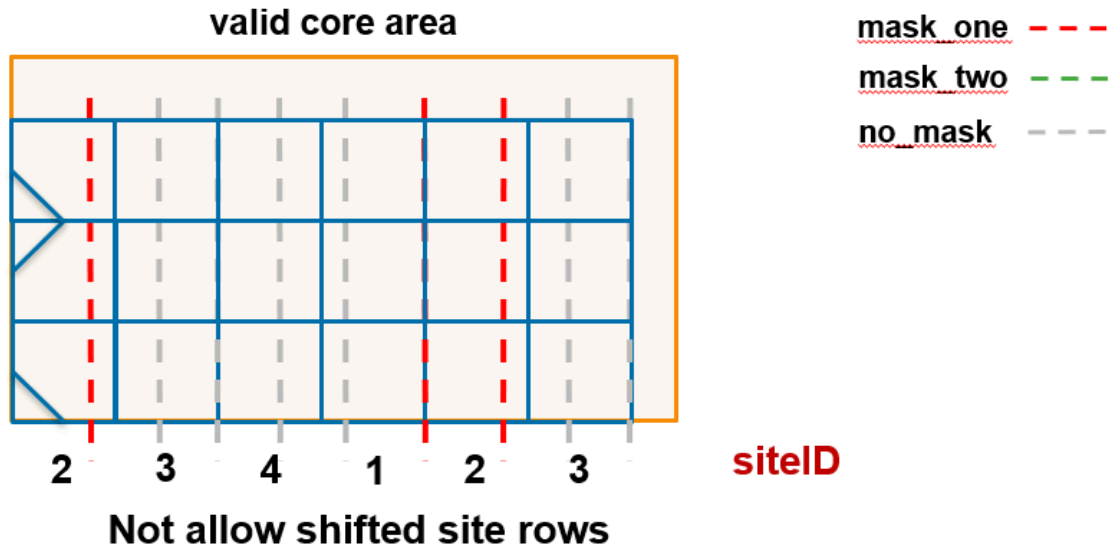


For variant siteId, you can get the *cycle* and *offset* from site rows first, then use the cell placed location to do the calculation.

Figure 201 shows the calculation for variant siteId

Feedback

Figure 201 Calculation for Variant siteId



Running and Placing Group Repeaters

Use the `add_group_repeaters` and `place_group_repeaters` commands to display the summary report for variant cell swapping in command cell collections.

Troubleshooting

Table 68 lists the messages of group repeater placement and insertion for the problems encountered during defining the group repeaters.

Table 68 Group Repeater Placement and Insertion Troubleshooting

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters`	`ECOUI-136`	`Error: The repeater group (%d) is invalid.`	Check if the input group id exists by running the `report_repeater_groups` command.

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters`	ECO-220	`Warning: Failed to connect to route for %d object: {%s }.`	Check the pin shape of object and route shapes. The object as driver or load can be port, macro pin, and placed cell.
`place_group_repeaters`	ECO-224	`Error: The option -repeater_groups cannot be used when the app option eco.placement.eco_enable_pipeline_register_placer is false.`	Check the pin shape of object and route shapes. The object as driver or load can be port, macro pin, and placed cell.
`place_group_repeaters`	ECO-227	`Warning: Failed to place %d repeaters for %d drivers in collection (%s), and failed cells are in collection (%s).`	Check the failed drivers by running the `query_objects`, `get_ports`, `get_pins` and `get_cells` commands. The driver can be port, macro pin, and placed cell.
`place_group_repeaters`	ECO-228	`Warning: Some repeaters from path driver (%s) have same repeater group id. The cells belonging to the path are {%s}.`	Check if the group id of cells are on the same path. The command skips to place some repeaters. The group id of cells on one path should be different.
`place_group_repeaters`	ECO-229	`Warning: Failed to find intersection of cutline %s and route for cell {%s}.`	
`place_group_repeaters`	ECO-231	`Error: The repeater group (%d) has no cutline or invalid cutline.`	Check the right cutline for this group. Set right cutline for this group using the `set_repeater_group` command.

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters`	ECO-232	`Error: Cutline {{%s} {%s}} is invalid.`	Check the format of the cutline. The valid format of cutline is {{x1 y} {x2 y}} or {{x y1} {x y2}}.
`place_group_repeaters` `unplace_group_repeaters`	ECO-233	`Error: The cells in collection (%s) are invalid since they have no target input pin name (%s) and output pin name (%s).`	Check the input and output pin name. The input pin name and output pin name can be specified by using the `-lib_cell_input` and `-lib_cell_output` options if needed, or filter out invalid cells from input. The default value for `-lib_cell_input` is D. The default value for `-lib_cell_output` is Q.
`place_group_repeaters`	ECO-234	`Warning: There is no repeater to be placed for the path driver (%s).`	
`place_group_repeaters`	ECO-235	`Warning: There is no route on driver net (%s).`	
`place_group_repeaters`	ECO-236	`Warning: There is no target load for path driver (%s).`	
`place_group_repeaters`	ECO-237	`Warning: Some cell to be placed has no load for path driver (%s).`	
`place_group_repeaters`	ECO-239	`Info:%s.`	Check the place and connection mode. Show the place mode (cutline/distance) and connection mode (virtual connection/real connection) on this command.

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
place_group_repeaters unplace_group_repeaters	ECO-240	Error: The option -cells cannot be used when the app option eco.placement.eco_enable_virtual_connection is true.	Check the command for virtual connection. Specify the -repeater_groups command if you need to use virtual connection.
place_group_repeaters	ECO-241	Warning: The distance based placement does not support multi-fanout path driver(%s).	Check the cutline based placement for the multi-fanout path driver.
place_group_repeaters unplace_group_repeaters	ECO-242	Warning: The cell (%s) has no driver or load (number of driver: %d, number of load: %d).	
place_group_repeaters	ECO-243	Error: The cells in collection (%s) are invalid since they have no driver or load.	Check for invalid cells. Ensure the cell that is placed has both driver and load. Else, remove such cells from input.
place_group_repeaters	ECO-244	Warning: The %s %s has no shape.	Check if the top port or macro pin has shape.
place_group_repeaters	ECO-246	Warning: Skip the path driver (%s) since the driver or load for the path has no shape.	
place_group_repeaters	ECO-251	Info: Place group repeaters successfully.	Check if all the input cells are placed successfully.
place_group_repeaters	ECO-252	Warning: Failed to place repeaters for repeater path which driver is %s.	
place_group_repeaters	ECO-253	Warning: The cutline information %s for cell %s is not on valid route.	

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters`	ECO-254	`Warning: Failed to cut route for cell %s.`	Check if the command failed to cut route for cell.
`place_group_repeaters`	ECO-255	`Warning: Gap %s between driver and nearest route is larger than placing distance %s`	Check if the route to the nearest pin distance is larger than placing distance for driver or load.
`place_group_repeaters`	ECO-256	`Warning: The first distance (%s) specified by command option is shorter than the gap (%s) between pin and route for driver (%s). (warning) The last distance (%s) specified by command option is shorter than the gap (%s) between pin and route for load (%s).`	Check if the `-first_distance` or `-last_distance` option specified is shorter than the gap between pin and route for driver or load.
`place_group_repeaters`	ECO-257	`Warning: Cannot find cutline information for cell %s.`	
`place_group_repeaters`	ECO-258	`Info: The app option about virtual connection is turned on. Please make sure the cells in repeater group keep in order.`	Check if the `eco.placement.eco_enable_virtual_connection` option is set to true and the `place_group_repeaters -repeater_groups` option is specified.
`place_group_repeaters unplace_group_repeaters`	ECO-259	`Warning: The number of cells from repeater group (%d) is not consistent with the number of drivers from driver group (%s).`	
`place_group_repeaters unplace_group_repeaters`	ECO-260	`Warning: The number of cells from repeater group (%d) is not consistent with the number of loads from load group (%s).`	

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters` `unplace_group_repeaters`	ECO-261	`Warning: The number of path loads is less than the number of cells in group (%d), so skip these path loads.`	
`place_group_repeaters` `unplace_group_repeaters`	ECO-262	`Warning: The number of path loads is not an integral multiple of the number of cells in group (%d), so use path loads in order.`	
`place_group_repeaters`	ECO-265	`Warning: The pin shape {%s} of %s (%s) is projected to route %s {%s}.`	Check if the route is far away from driver or loads. The tool projects the closest pin shape to the route.
`unplace_group_repeaters`	ECO-266	`Warning: The cells in collection %s which are not placed before cannot be unplaced by unplace_group_repeaters.`	Check if the cells are not placed. The cells that are not placed cannot be unplaced. Also, only the cells added by the `add_group_repeaters` command and placed by the `placed_group_repeaters`command can be unplaced by the `unplace_group_repeaters` command.
`unplace_group_repeaters`	ECO-267	`Error: Input and output routes are not merged after unplacing repeater %s since the routes are not aligned or not at the same layer.`	Check if the routes are merged after unplacing repeaters. This error occurs when the routes are not aligned or not at the same layer since they are not merged.

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
`place_group_repeaters`	ECO-268	`Error: The cells in collection (%s) are invalid since they are placed before. Apply the command unplace_group_repeaters to unplace them.`	Check if the cells were placed before and added by the `add_group_repeaters` command or placed the by `place_group_repeaters` command.
`add_group_repeaters` `place_group_repeaters`	ECO-269	`Error: Unable to check the current reference of cell '%s': %s.`	Check the reported reason and other printed messages for more details. Use the appropriate command to check if the current reference of cell is legal or illegal for the reported issue.
`add_group_repeaters` `place_group_repeaters`	ECO-270	`Error: Unable to find swappable variant reference for cell '%s': %s.`	Check if the current reference of cell is illegal for the reported issue.
`add_group_repeaters` `place_group_repeaters`	ECO-271	`Error: Candidate reference '%s' and reference (%s) of cell (%s) are not in the same variant cell group.`	Check errors for compatible issues between the candidate and current reference of cell. Error also occurs if they do not belong to the same variant cell group.
`unplace_group_repeaters`	ECO-272	`Error: The mixed input cells are not supported, which includes the cells added by add_group_repeaters and the cells placed by place_group_repeaters.`	
`unplace_group_repeaters`	ECO-273	`Error: There are no placed cells to be unplaced. Please double check the cell status.`	Check if there are placed cells that are not qualified.

Feedback

Table 68 Group Repeater Placement and Insertion Troubleshooting (Continued)

Command	Error Code	Message	Troubleshooting Tips
	ECO-274	Warning: You are deleting cell %s with eco_repeater_group_id %d.	Check if you are trying to delete a cell belonging to a repeater group.
place_group_repeaters	ECO-275	Error: Failed to traverse cutlines. The nets may have too many blockages.	Check the location to insert cell groups. Remove blockages or find cutlines and use the -repeater_group option.
add_group_repeaters	ECO-276	Warning: Unable to find free area for repeater on net (%s) at location (%s) within range {%s}.	Check if checkerboard snapping service can find free space for a repeater
set_repeater_group	ECO-277	Warning: You are trying to override cells of repeater group %d. Use -override if you have to override cells.	Check if you are trying to override cells without using the -override option.
create_repeater_groups	ECO-278	Error: Get 0 transparent cells from all supernets.	Check if the input supernets have no transparent cells to group.
create_repeater_groups	ECO-279	Error: %s has multiple(%lu) inputs.	Check if a cell or load has multiple inputs.

Fixing Multivoltage Violations

Design changes in the ECO flow sometimes result in isolation or voltage violations. The fix_mv_design command can identify and fix violations for buffer trees and diodes.

You must use either the -buffer or -diode option with the fix_mv_design command. For more information, see the following topics:

- Fixing Buffers
- Fixing Diodes

Feedback

If the UPF has not already been committed, the `fix_mv_design` command automatically commits the UPF before performing any operations.

Netlist changes that the tool makes as a result of the `fix_mv_design` command are not guaranteed to be placement or timing legal. You must check the design after running the `fix_mv_design` command.

Fixing Buffers

The `-buffer` option of the `fix_mv_design` command fixes isolation violations and illegal library cells in buffer trees throughout the design. The command can perform the following changes:

- Minimize the number of dual-rail buffers or inverters
- Fix isolation violations in buffer trees
- Fix buffers and inverters that have illegal settings, as follows:
  - Process, voltage, and temperature settings
  - Library cell purpose or subset settings
  - Target library subset settings
  - Bias voltages
- Fix a mismatch between a buffer's power domain and the voltage area in which the buffer or inverter is physically placed

The command can change the netlist as follows:

- Swap buffer library cells to fix illegal settings
- Swap single-rail buffers for dual-rail buffers
- Swap dual-rail buffers for single-rail buffers
- Change the backup supplies of dual-rail buffers and inverters
- Change the input or output supplies of level shifters
- Fix a mismatch between a buffer's power domain and a voltage area by using a physical or logical feedthrough
- Fix a mismatch between a buffer's power domain and a gas station voltage area by using a physical or logical feedthrough

The `fix_mv_design` command keeps the placement of the original buffers or inverters and honors the repeater strategies associated with existing buffers and inverters. In addition,

Feedback

the command only uses supplies that are available in the voltage area or voltage area region.

The `fix_mv_design` command does not insert isolation cells or level-shifter cells to fix existing isolation or voltage violations. Netlist changes caused by the command do not introduce new isolation or voltage violations.

To specify a buffer tree, use the `-from` option with the name of a net or driver pin. If you specify a net name, its driver pin is used. In both cases, the driver pin defines a full or partial buffer tree. Wildcards are supported.

The `-lib_cells` option specifies a list of single-rail and dual-rail buffer or inverter library cells that the `fix_mv_design` command can use as replacement cells. By default, the command does not swap dual-rail and single-rail cells if the swap is not necessary to fix isolation violations. However, if you also specify the `-minimize_dual_rail` option, the tool tries to minimize the usage of dual-rail buffers to save area.

If the tool inserts a new library cell as a result of the `fix_mv_design` command, the new cell is chosen according to the following rules, in priority order:

- A cell from the same library as the original library cell
- A cell from a library with a name similar to the original library
- A cell with a name similar to the original library cell name

By default, buffer trees stop at level-shifter cells. Specify the `-level_shifter` option to allow the `fix_mv_design` command to modify the input or output supplies of level-shifter cells. However, the command does not change level-shifter library cells.

To report the changes that the tool recommends without making any netlist changes, use the `-report_only` option.

Fixing Diodes

The `-diode` option of the `fix_mv_design` command fixes mismatches between power domains and voltage areas for diodes throughout the design. The `fix_mv_design -diode` command examines all diodes, including diodes inserted by the `create_cell`, `create_diodes`, and `add_port_protection_diodes` commands, as well as several routing commands that insert diodes to fix antenna violations.

If a power domain and voltage area mismatch for a diode is found, the tool uses the voltage area to explicitly assign a corresponding power domain. If multiple power domains exist in a single voltage area, the tool picks one of the available power domains. Any of the available power domains is acceptable because all diode cells are single-rail cells and all power domains in the same voltage area have the same primary supply.

Feedback

To obtain a report of the changes that the tool recommends without making any netlist changes, use the `-report_only` option. To obtain detailed information in the report, use the `-verbose` option. These are the only two options that you can use with the `-diode` option.

14

Glossary

Active edge

The low-to-high or high-to-low transition of a clock signal that causes data to be latched into a flip-flop.

Add Buffer On Route

Typically used as an Engineering Change Order operation where the default buffering results are insufficient. Refers to adding buffers with locations based on existing routing topology to avoid excessive disturbance to routing during postroute optimization. Can also be used to buffer special higher priority paths before the rest are done.

Add Buffer On Route (ABOR)

ABOR

Advanced Legalizer

AL

Advanced on-chip variation

Advanced on-chip variation, a technique to improve timing accuracy by applying derates to cells to model process variation. Less pessimistic than traditional on-chip variation, by using different derates based on cell type and path depth.

AOCV

Aggressor net

A net that causes undesirable crosstalk effects on another net (called the victim net) due to parasitic cross-capacitance between the two nets.

Always-On

The characteristic of a cell, circuit, or power domain that is on while a reference logic is shut down. For example, a logic cell used for isolation or retention is an always-

Feedback

on cell. The cell must have separate power supply pins for operation during power-down.

Always-On (AO)

AO

Arc

Short for timing arc. A representation of the timing relationship between two pins or ports in a circuit.

Associated internal clocks

A set of one or more user-defined internal clocks in a clock network, each of which is the root of a skew subdomain within its parent clock network.

Asynchronous

The absence of a timing correlation between two distinct clocks or signals means their transitions can happen at any moment relative to each other. A signal can be either synchronous or asynchronous within a specific timer mode, but it cannot be both simultaneously.

Attribute

Represents properties of objects in the NDM database. Query the attributes of objects using the `get_attribute` command. Set attributes on objects using the `set_attribute` command. Filter collections of objects based on their attribute values using the `filter_collection` command or the `get_* -filter` command for that object class (for example, `get_cells -filter` for cell objects).

Automatic Density Control

ADC

Automatic Density Control Value Enhanced

ADCe

Automatic Scenario Reduction

A technique used during optimization to improve timer runtime by reducing the number of active scenarios.

Automatic Scenario Reduction (ASR)

Feedback

ASR

Automated Test Equipment

An industrial equipment used to test semiconductor devices by applying input stimuli, observing the device response, and comparing it against the expected response.

Automated Test Equipment (ATE)

ATE

Automated Test Pattern Generation

Automated Test Pattern Generation, the generation of scan data sequences used for scan testing, with the goal of achieving as much test coverage as possible using the smallest possible number of patterns. The test patterns contain nonfunctional data selected to detect faults on nets in the design.

Automated Test Pattern Generation (ATPG)

ATPG

Auto Non-Default Rules

Non-default rules are created automatically by the tool during preroute or global-route based optimization, for the purposes of improving timing on a path or power on that net.

Auto Non-Default Rules (ANDR)

ANDR

Behavioral view

The set of Verilog statements that describe the behavior of a design by using sequential statements. These statements are similar in expressive capability to those found in many other programming languages. See also the data flow view, sequential statement, and structural view definitions.

Bit-width

The width of a variable, signal, or expression in bits. For example, the bit-width of the constant 5 is 3 bits.

Boundary Scan Design

Refers to test logic that implements IEEE Std 1149.1.

Feedback

Boundary Scan Design (BSD)

BSD

Boundary Track Supply Reduction

BTSR

Bubble register

The internal part of a retention register that is always supplied with power and retains data during power-down; is constructed of higher-threshold transistors to minimize leakage current. Also called the shadow register.

Buffering-Aware Placement

BAP

Built-in self-test

Design-for-test logic where both the scan data generation and the scan data comparison logic are included in the design.

Built-in self-test (BIST)

BIST

Burn-in mode

A mode that continuously runs autonomous self-test. The scan and capture activity stresses the tested logic and causes continuous power draw during self-test. Burn-in operation can be configured to stop or continue if self-test fails.

Capacitance

Capacitance is the ability of an object to store electrical charge. It is measured by the change in charge relative to a difference in electric potential, represented as the ratio of these quantities.

cBuf

Post-route buffering engine

Cell

A component within an integrated circuit with known functional and timing characteristics, which can be used as an element in building a larger circuit.

Feedback

Classic Legalizer

CL

Clock

A periodic signal that latches data into sequential elements. You define a signal to be a clock and specify its timing characteristics by using the `create_clock` or `create_generated_clock` command.

Clock chain

A special scan chain segment associated with an on-chip clocking (OCC) controller whose scanned-in values control the pulse sequence of a controlled clock.

Clock gating

A method of reducing power by shutting off clocks to circuits that are not being used. Clock gating is typically done using integrated clock gating cells (ICGs).

Clock latency

Clock latency refers to the time it takes for a clock signal to travel from its source to a specific point within a design. Clock latency is the sum of source latency and network latency.

Source latency is the time taken for the clock signal to propagate from its origin to the clock definition point within the design. Network latency is the time taken for the clock signal to travel from the clock definition point to a downstream physical point in the clock network.

Use the `set_clock_latency` command to specify the clock latency at the ideal clock stage.

Clock Reconvergence Pessimism

An inaccuracy in static timing analysis occurs when two different clock paths partially share a common physical path segment. The analysis tool uses the minimum delay for one path and the maximum delay for the other path on this shared segment. This inaccuracy is corrected through a process known as Clock Reconvergence Pessimism Removal (CRPR).

Clock Reconvergence Pessimism (CRP)

Feedback

CRP

Clock Reconvergence Pessimism Removal

CRPR

Clock skew

Clock skew refers to the maximum variation in the arrival times of clock signals at registers within a single clock domain or across different clock domains. At the pre-CTS stage, clock skew is modeled as part of clock uncertainty. Use the `set_clock_uncertainty` command to specify the skew characteristics for one or more clock networks.

Clock source

The pin or port where the clock waveform is applied to the design. The clock signal reaches the registers in the transitive fanout of all its sources. A clock can have multiple sources. Use the `create_clock` command with the `source_objects` option to specify clock sources.

Clock tree

The combinational logic between a clock source and the clock endpoints within its transitive fanout of that source. These clock trees are synthesized by vendors based on physical placement data and clock-related constraints specified by the user, either within a single clock domain or across multiple clock domains.

Clock Tree Mesh

CTMESH

Clock uncertainty

Clock uncertainty models the clock skew, clock jitter, and margin at pre-CTS stage. For post-CTS stage clock uncertainty models clock jitter, and margin only.

Coarse-grain switching

A power-switching strategy that switches power on and off for a block as a whole, with all the transistors in the block sharing a single power switch.

(coarse-grain switching)

Feedback

Codec

The combination of the decompressor and compressor in a compressed scan flow. A single design can have multiple codecs, but each codec consists of its own decompressor/compressor pair.

Collection

A set of objects taken from the loaded design. For example, the set mylist `[get_clocks "CLK\*"]` command creates a collection of clocks and sets a variable called `mylist` to the collection. You can use commands such as `remove_clock $mylist` that operate on the collection. Additionally, you can also generate and operate on a collection within a single command, such as `remove_clock [get_clocks "CLK\*"]`.

Compressed scan

A scan methodology that uses more scan chains (called compressed scan chains) than scan-in/scan-out pairs. A decompressor decompresses the scan-in data to drive the greater number of scan chains. A compressor compresses the scan chain data to drive the lesser number of scan-outs. The combination of the decompressor and compressor wrapped around the scan chains is called the *codec*.

Compressed scan chains

In a compressed scan flow, the scan chains that are driven by the decompressor and drive the compressor. There are a greater number of compressed scan chains than scan-in/scan-out pairs.

Compressor

In a compressed scan flow, the part of a codec that compresses the scan chain data to drive the lesser number of scan-outs.

Compute Useful skew

Refer to the `compute_useful_skew` command, or any useful skew optimization done by the tool before clock trees are inserted. Contrast with CCD, which uses helpful skew operations on a clock tree that has already been constructed.

Compute Useful skew (CUS)

CUS

Concurrent Clock and Data Optimization

Refers to useful skew techniques applied after clock tree synthesis to improve timing.

Concurrent Clock and Data Optimization (CCD)

CCD

Concurrent Legalization and Optimization

CLO

Conjugate Gradient Placer (coarse placer engine)

CGPL

Core

A design block that is DFT-inserted and has CTL model information about the inserted DFT structures. Cores are used in hierarchical scan synthesis flows.

Core Data Register

An optional register in an IEEE 1500 implementation that can be loaded with a value after loading a corresponding instruction into the wrapper instruction register (WIR).

Core Data Register (CDR)

CDR

Core Test Language model

This describes the characteristics of a DFT-inserted design in STIL (IEEE Std 1450) format. This model includes information such as: DFT signals, scan chains, test clocks and clock control, and test modes.

Core Test Language model (CTL model)

CTL model

Core wrapping

See *wrapped core*.

CPU time

The total CPU processing time for a task. This is in contrast to elapsed time, which represents the actual amount of time that has elapsed during a task. When CPU time exceeds elapsed time, multithreaded commands or engines benefit from speedup that allows the work to be finished faster than if it were single threaded.

Feedback

Create Buffer Tree

This is a command that builds buffer trees for the specified driver pins or nets.

create_buffer_tree (CBT)

CBT

Crowbar current

The current that flows from the power supply to the ground current through a PMOS-NMOS stack during a logic transition, when both the PMOS and NMOS transistors are conducting for a brief period of time.

(crowbar current)

Clock Tree Optimization

CTO

Clock Tree Synthesis

CTS

Clock uncertainty

The amount of variation away from the nominal, ideal-clock times at which clock edges occur.

Current Design

The active design (the design being worked on). Most commands are specific to the current design; they operate within the context of the current design. You specify the current design with the `current_design` command.

Current instance

The instance (hierarchical cell) in a design hierarchy on which instance-specific commands operate by default. Set the current instance with the `current_instance` command, which works like the `cd` command in Linux.

Decompressor

In a compressed scan flow, the part of a codec that decompresses the scan-in data to drive the greater number of scan chains.

Feedback

Design Constraints

A set of restrictions placed on the design to drive optimization goals like timing, power, and area. Timing constraints use SDC format. Many other types of constraints exist to drive engines like clock tree synthesis (max skew), placement (bounds), optimization (low VT limits), and routing (non-default rules). Design constraints can be hard, meaning that they must be met or they are reported as a violation. They can also be soft, meaning that they are a best effort constraint to encourage the engine to do something if possible.

Design-for-Test

This pertains to logic that helps the testability of a design. It involves incorporating specific design features and methodologies to ensure that the design can be effectively tested for faults and defects. DFT techniques are used to improve the ease and thoroughness of testing integrated circuits and systems.

Design-for-Test (DFT)

DFT

Design Mismatch Manager

DMM

Design Rule Check

A check or constraint used to ensure correct operation of the integrated circuit. DRC might refer to several different things, including routing DRCs that indicate violations in the routing topologies like shorts or different net spacing rules, logical DRCs that indicate problems with the timing characteristics of the circuit like maximum transition or capacitance value.

Design Rule Check (DRC)

DRC

DFTMAX scan compression

Scan compression implemented by the TestMAX DFT tool that uses combinational codecs to yield high scan compression ratios.

DFTMAX Ultra scan compression

Scan compression implemented by DFTMAX Ultra compression that uses streaming (sequential) codecs to yield very high scan compression ratios, even down to a single scan-in/scan-out pair.

Feedback

DFT Partition

A DFT specification that allows the sequential cells in a design to be separated into multiple independent partitions for the purpose of DFT insertion.

Direct Congestion Driven Placement

DCDP

Direct Timing Driven Placement

DTDP

Dual Core Lock Step

DCLS

Dynamic Power

Dynamic power is the energy consumed by CMOS circuits during switching (changing of logic states). It consists of two main components: switching power and internal power.

Dynamic Power

Dynamic Power Shaping

DPS

Dynamic Rule Ordering for Legalization

DRO

Dynamic Voltage and Frequency Scaling

Dynamic voltage and frequency scaling is a speed-versus-power adjustment technique based on changing the supply voltage and operating frequency during operation to meet the current workload requirements. DVFS techniques are very effective in reducing power, since lowering the voltage has a squared effect on active power consumption. DVFS optimizes both the frequency and the voltage, it is one of the only techniques that is highly effective on both dynamic and static power.

Dynamic Voltage and Frequency Scaling (DVFS)

Feedback

DVFS

Early Data Check

EDC

Electronic Design Automation

Electronic design automation is the process of using advanced software tools to design integrated circuits.

Electronic Design Automation (EDA)

EDA

Enable Level Shifter

A logic cell that performs both level-shifting and isolation functions, required where power domains can operate at different voltages and can be powered down.

Enable Power Improvement

EPI

Enable Runtime Improvements

Refers to the `enable_runtime_improvements` command, which enables code changes for runtime improvement that will be on-by-default in a future release.

Enable Runtime Improvements (ERI)

ERI

Enable Wireopt Improvements

. Refers to the `enable_wireopt_improvements` command, which enables wire optimization techniques like ANDR (auto-NDR) and layer promotion to improve timing and power, mostly during global-route based optimization in the `clock_opt` command.

Enable Wireopt Improvements (EWI)

Feedback

EWI

Endpoint

The place in a design where a timing path ends. This is where data gets captured by a clock edge or where the data must be available at a specific time. Every endpoint must be either a register data input pin or an output port.

Enhanced Delay Optimization

EDO

Enhanced Low Power Placement

eLPP

Exclusive clocks

Exclusive clocks are clock signals that exist within a design but cannot be active at the same time. Consequently, there are no timing paths between them. These clocks are typically managed by a multiplexer.

Extent

The set of cells in a design that belong to a power domain.

EXTEST mode

Outward-facing test mode used to test logic external to (outside) a core, independent of the logic inside the core. It uses the wrapper chain to drive scan-controllable values at the output wrapper cells and capture the values at the input wrapper cells.

False path

A path that exists in a design that should not be analyzed for timing, such as a path between two multiplexed blocks that are never enabled at the same time. A false path is constrained using the `set_false_path` SDC command.

Fanin

The cone of logic leads to a particular pin, port, net, or cell. Fanin means different things in different contexts. A fanin on a timing path is all of the logic on that path leading to a particular point. A fanin in a general timing context is all of the logic that can be traced in the timing graph upstream of that point. A fanin in the logical context is all of the logic that can be traced through the logical netlist connected to that point. A fanin in the context of a net is all of the pins or ports directly driving that net.

Feedback

Feedthrough Buffering

FTB

Find Legal Location

FLL

Fine-grain switching

A power-switching strategy that switches power on and off for individual library cells, thereby allowing the power supply to each cell to be controlled individually.

Fine-grain switching

Finite State Machine

A logic construct that moves through states (like navigating elements in a flowchart diagram) to implement a certain logic behavior.

FSM

Flip-flop

A memory element controlled by an edge-sensitive clock input. Typically, a flip-flop has an input D, and output Q, a clock input, and possibly asynchronous set and clear inputs. When the active edge occurs on the clock input, the value on the input D is latched and then held constant at the output Q. The active edge can be either rising or falling, depending on the type of flip-flop.

Footer switch

A power supply switch that connects the GND power supply to the GND supply pins of the switchable cells on the chip.

Footer switch

Functional Safety

FuSa

Gate leakage

The current that flows between the gate and the source or drain of a transistor, caused by quantum-effect tunneling of electrons through the thin gate insulator.

Feedback

Gate leakage

Graphic Design System

GDS

Generated clock

A clock signal that is generated internally by the integrated circuit itself; a clock that does not come directly from an external source. An example of a generated clock is a divide-by-2 clock generated from the system clock. You define a generated clock with the `create_generated_clock`command.

Golden UPF

In the golden UPF flow or mode, the same original *golden* UPF script file is used throughout the synthesis, physical implementation, and verification flow. Typically, this one same UPF file is referred to as the *golden* UPF. A more common flow is the UPF' or UPF prime flow, where the UPF file is modified across the flow with all the changes that the implementation front end and back end tools include.

Golden UPF

Global Route Optimization

Refers to global-route based optimization. See also GRE.

GRO

GROPTO

Global Route based Optimization in HFS

Graphical User Interface

A graphical interface for interacting with a software package. In the context of the tool, the GUI is launched with the `gui_start` command and enables many features for interactive editing and seeing the layout, schematic, or other views of the design.

Feedback

GUI

Half Perimeter Wire Length

HPWL

Header switch

A power supply switch that connects the supply VDD of the chip to the VDD pins of the switchable cells on the chip.

Header switch

Hierarchical DFT insertion

Refers to a flow that performs DFT insertion in a lower level block (known as a *core*), then incorporates that block's scan structures into a higher level.

This term pertains to how DFT insertion is performed in the tool flow. Do not confuse this term with *hierarchical testing*, which pertains to how manufacturing test is run on the ATE.

Hierarchical testing

Refers to the process of testing different hierarchy levels of the design independently. Wrapped cores are often used to enable hierarchical testing.

This term pertains to how manufacturing test is run on the ATE. Do not confuse this term with *hierarchical DFT insertion*, which pertains to how DFT insertion is performed.

High Effort Area

HEA

High Effort Congestion

High Effort Congestion for `set_qor_strategy` (SQS) mega switch

Feedback

HEC

High Effort Timing

HET

High Fanout Net Synthesis

HFNS

High Performance Core

HPC

Hold Area Budgeting

HAB

Hold Buffering Engine

hBuf

Hold time

Hold time is the minimum amount of time that a signal on the data pin endpoint must remain stable after the active edge of the clock. The hold time creates a minimum delay requirement for paths leading to the data pin of the cell.

HRO

hyper_route_opt. Also called as HROPT.

Hybrid (site-row) Aware Placement

HAP

IC Compiler II

ICC2

IC Validator

The IC Validator product from Synopsys

Feedback

ICV

Ideal clock

An ideal clock is considered to have zero delay as it propagates through the clock network. The transition time for ideal clocks is considered zero unless explicitly specified using the command such as `set_clock_transition`. You can override this default behavior by using the `set_clock_latency` and `set_propagated_clock` commands to obtain non-zero clock network delay and specify information about the clock network delays. Use the `set_ideal_network` command to mark clock networks as ideal.

Ideal net

Ideal net is a network of cells that are assigned for ideal timing conditions. The timer does not compute arrival and transition times for these nets. Instead, ideal latency, transition, and capacitance constraints are applied. Ideal nets are typically used to model high-fanout nets before they are buffered, including very high-fanout data nets early in the synthesis flow (like scan nets), and clock trees before running clock tree synthesis, that is, latency, transition time, and capacitance are assigned a value of zero. The set_ideal_net command can constraint a net as ideal.

Inout pin

An inout pin is a bidirectional pin of a cell that can function both as an input and an output.

Input delay

A constraint that specifies the minimum or maximum amount of delay from a clock edge to the arrival of a signal at a specified input port. Use the `set_input_delay` command to set the input delay on a pin or input port relative to a specified clock signal.

Instance

An instance refers to a specific occurrence in a circuit of a reference (a library component or design) loaded in memory. Each instance has a unique name. A design can contain multiple instances where each instance points to the same reference but has a unique name to distinguish it from other instances. An instance is also known as a cell.

Integrated Clock Gate

A clockgate with the latch mechanism built-in to the standard cell.

Feedback

ICG

Internal chains

The scan chains inside a block (uncompressed or compressed), as opposed to other kinds of chains, such as boundary scan chains or core-wrapping chains.

Internal clock

A scan clock, defined on an internal pin in the design, that is the root driver of a *skew subdomain*. Internal clocks can be automatically determined by the tool at multi-input gate outputs or specified manually at associated internal clock pins within the clock network.

Internal power

Power consumed as a result of the short-circuit (crowbar) current through a PMOS-NMOS stack during a logic transition; is a type of dynamic power.

Internal power

INTEST mode

Inward-facing test mode used to test logic internal to (inside) a core, independent of the logic outside the core. It uses the wrapper chain to drive scan-controllable values at the input wrapper cells and capture the resulting values at the output wrapper cells.

Insulated well

Usually refers to the N-well inside a cell that has some physical insulation that prevents physical abutment with its surrounding bias, allowing to be powered by a supply which behaves differently than the supply that powers the abutted N-wells in the physical area where the cell is going to be located.

Insulated well

Inward-facing test mode

See INTEST mode.

Isolation cell

A logic cell that can isolate a power-down domain from a domain that is not powered down. The cell operates as a buffer when the input and output sides of the cell are both powered up, but provides a known, constant output signal when the input side is powered down.

Isolation cell

Feedback

Interconnect Technology File

An interconnect technology file defines a cross section profile of the process. The file contains an ordered list of conductor and dielectric layer definition statements. The layers are defined from the topmost dielectric layer to the bottom most dielectric layer, excluding the substrate. The ITF parameters are specified layer by layer in a way that is consistent with the physical process.

ITF

Latency

The time taken for a clock signal to travel from the clock source to a specific endpoint inside the design.

Latency Aware Placement

LAP

Latency Driven Placement

LDP

Leading edge

The first edge of a clock waveform definition. It is a rising edge for a return-to-zero clock, and it is a falling edge for a return-to-one clock.

Leaf Cell

A fundamental unit of logic design. A leaf cell cannot be broken into smaller logic units. Examples are NAND gates and inverters. For example, standard cells and macro cells defined in a library are leaf cells. Contrast to a hierarchical cell.

Level shifter

A circuit or library cell that converts signals from one voltage level to another to make the signal compatible with the supply voltage of the power domain at the output.

Level shifter

Library Compiler

A Synopsys tool used to create library databases for Fusion Compiler, PrimeTime, Design Compiler, and other tools. Library Compiler reads the description of an ASIC library from a text file in Liberty (.lib) format and compiles the description into a database in .db format for synthesis and analysis.

Feedback

Line End Extension

LEE

Linear Feedback Shift Register

A shift register whose next data word is a linear XOR function of its current data word. The PRPG uses an LSFR to generate pseudorandom data.

LFSR

Load Sharing Facility

A software package that handle scheduling and running jobs on a compute cluster. Many such software packages exist, LSF is often used as a generic term to refer to them.

LSF

Logic Equivalency

Refers to checking done by any optimization engine that transforms netlist logic through operations like sizing and remapping. Ensures that the transformed cone of logic is logically equivalent to the same cone of logic before transformation.

LEQ

MLGR

Using Machine Learning technology to perform global route extraction

Multi-Threading Clock Tree Optimization

MTCTO

Multiplexer

An N:1 multiplexer is a combinational circuit that connects any one input line, out of multiple N input lines, to the single output line based on its control input signal or selection lines. Usually, for n selection lines, there are $N = 2^n$ input lines. The nomenclature N:1 denotes it has N input lines and one output line.

Multiplexer (MUX)

Feedback

MUX

Multiple Instantiated Modules

MIM

Multiple-input signature register

A recirculating shift register that XORs scan-captured data values into the loop. After capturing all values, the MISR contains a signature value for that test.

MISR

Multiple-Threshold CMOS

Multiple-Threshold CMOS is a feature of a process technology that supports transistors having different threshold voltages. High-threshold transistors can be used where leakage reduction is important, while low-threshold transistors are used where switching speed is important.

Multiple-Threshold CMOS (MTCMOS)

MTCMOS

Multiply Instantiated Block

MIB

Multisource Clock Tree Synthesis

A technique where a regular structure like an H-tree or clock mesh is used to drive the initial parts of the clock tree. The tool supports two main multisource CTS approaches such as structured multisource CTS (SMSCTS) and regular multisource CTS (RMSCTS)

MSCTS

Multivoltage

Refers to a design technique where multiple power supplies are used as a power reduction technique. It can be different voltage or some power supplies that are fully switched off in certain conditions. Also, this refers to many techniques used in the tool to enable the multivoltage design. Multivoltage design constraints are written in a standard format called UPF.

Feedback

MV

Multivoltage violation

Violations of multivoltage-related electrical rules of the design. This includes violations of rules related to power strategies: isolation, level shifter, retention, and repeater; violations of rules related with voltage thresholds in the design; violations derived from Power State Table (PST) analysis, among others.

Multivoltage violation

Mux Extraction and Optimization

MUXEO

N-type Metal Oxide Semiconductor

This is a type of transistor consisting of an N-type source and N-type drain separated by a P-type channel. Applying a positive voltage on the gate induces an N-type conducting channel between the source and drain.

N-type Metal Oxide Semiconductor (NMOS)

NMOS

Netlist

A file in ASCII or binary format that describes a circuit schematic, the netlist contains a list of circuit elements and interconnections in a design. Netlist transfer is the most common way of moving design information from one design system or tool to another.

New Data Model

NDM

Non-Default Routing Rule

NDR

Non-Linear Delay Model

Is a highly accurate timing model derived from SPICE characterizations. Most cell libraries use look-up table models to specify the delay and timing checks for the various timing arcs of the cell. The table model is referred to as an NLDM and are used for calculating the delay, output slew, or other timing checks. The table gives the delay through the cell for various combinations of input transition time at the cell

Feedback

input pin and total output capacitance at the cell output. An NDLM delay model is represented in two-dimensional form, with two independent variables namely the input transition time and the output load capacitance and the entries in the table denote the delay.

Non-Linear Delay Model (NLDM)

NLDM

On By Default

A tool feature is on-by-default, so that there is no need to adjust an application option or other setting to enable it.

OBD

On-chip clocking controller

A DFT design structure that controls a free-running on-chip clocking source (such as a PLL output clock) in test mode.

OCC controller

Output delay

A constraint that specifies the minimum or maximum amount of delay from an output port to the sequential element that captures data from the output port. This constraint establishes the times at which signals must be available at the output port to meet the setup and hold requirements of the sequential element. Use the `set_output_delay` command to set the output delay on a pin or output port relative to a specified clock signal.

Outward-facing test mode

See EXTEST.

P-type Metal Oxide Semiconductor

Is a type of transistor consisting of a P-type source and P-type drain separated by an N-type channel. Applying a negative voltage on the gate induces a P-type conducting channel between the source and drain.

P-type Metal Oxide Semiconductor (PMOS)

Feedback

PMOS

Pad cell

A special cell at the chip boundaries that allows communication with other integrated circuits outside the chip, as opposed to an internal core cell, which makes up the core of an integrated circuit.

Parasitic Capacitance

Capacitance of a net due to the physical proximity between the net interconnections and adjacent nets or the substrate; or due to charge storage effects of p-n junctions in the net.

Parasitic Resistance

Resistance of an interconnection or net due to the finite conductivity of the interconnect material.

Pattern Based Pin Access Checker

PBPAC

PG pin

A power or ground pin of a cell, as specified in the library definition of the cell.

PG pin

Phase-Locked Loop

An analog design block that creates a stable on-chip latency-adjusted clock from a free-running (and possibly less stable) input clock.

PLL

Pin

A part of a cell that provides input and output connections. Pins can be bidirectional. The ports of a sub-design are pins within the parent design.

Feedback

Pin Access Checker

PAC

Pipelined scan data

A feature that inserts additional scan registers, called *pipeline registers*, at the scan-in and scan-out ports of the design to accommodate long wires between the scan chain input and the first flip-flop and between the last flip-flop and the scan chain output. Pipeline registers inserted at the scan-in and scan-out ports are called *head* and *tail* pipeline registers, respectively.

Pipelined scan enable

A feature that adds a pipeline register to the scan-enable signal. It is used for transition delay testing by making use of launch-on-extra-shift (LOES). This method of transition delay testing requires additional circuitry to manipulate the scan-enable signals.

Port

A signal declared in the interface list of an entity.

Port punching

The automatic creation of a power supply port by a synthesis or implementation tool, which makes a power connection from one hierarchical level to the next.

(port punching)

Port state

A possible state of a power supply port. Each state has a name and an associated condition. The condition can be either a single voltage value, a set of three voltage values (minimum, nominal, and maximum), or the *off* state. The state names are used in power state tables.

(port state)

Post-DFT DRC

DRC checking run after DFT insertion, evaluates the implemented DFT functionality of the design for correct operation.

Power

The rate of energy usage, usually expressed in watts (joules per second) and calculated as current (amps) multiplied by voltage drop (volts).

Feedback

(power)

Power domain

A group of elements in the design that share a common power supply distribution network. The set of cells belonging to the power domain is called the extent of the power domain. All of these cells share the same set of power supply rails. The level of hierarchy where the power domain exists is called the scope of the power domain. A power domain must have one primary supply net and one primary ground net, and might optionally have additional supply nets, supply ports, and power switches.

(power domain)

Power domain boundary

Each design element, typically pins and ports, that is at the interface of two power domains. Power strategies are usually defined and applied at power domain boundaries.

(power domain boundary)

Power down

The process of shutting off the power to a part of the chip by turning off a power switch.

(power down)

Power intent

Equivalent to the functional intent of a design, the power intent is specified in a UPF file. The power intent specifies the power distribution architecture (power domains, supply rails, and shutdown control), the power states of a design (through voltage definitions and possible states of supplies according to the Power State Tables) and the usage of power management cells such as the isolation cells, level shifters, power switches, and retention registers.

(power intent)

Power State Table

A list of the allowed combinations of voltage values and states of the power switches, for all power domains in the design.

Power State Table (PST)

Feedback

PST

Power strategy

A rule specified by the user, that specifies where and how to apply isolation, level-shifting, state retention, and buffering in the implementation of the power intent.

(power strategy)

Power switch

A transistor or transistor array that can switch the power on and off for a portion of the chip, either at the VDD power supply (header switch) or at GND (footer switch), thereby cutting off power to parts of the chip during periods of inactivity. Conceptually, it is a device that turns on and turns off power for a supply net. A switch has an input supply net, an output supply net that can be switched on or off, and at least one input signal to control the switching. The switch can optionally have multiple input control signals and one or more output acknowledge signals.

(power switch)

Pre-DFT DRC

DRC checking run before DFT insertion, evaluates the readiness of the design for DFT insertion. Certain types of pre-DFT DRC violations result in scan cells being excluded from scan chains.

Primary supply

The supply net connections inferred for all instances in the power domain, unless overridden.

(primary supply)

Procedure (Tcl)

A procedure in the Tcl language is defined by the proc command, and is the same as what is called a function in most other programming languages.

Process, Voltage, and Temperature

To make chips to work in all the possible conditions after fabrication, they are simulated at different process, voltage, and temperature conditions. These conditions are called corners. All these three parameters directly affect the delay of the cell.

Process, Voltage, and Temperature (PVT)

Feedback

PVT

Propagated clock

A clock that incurs delay through the clock network. Propagated clocks are used to determine clock latency at register clock pins. Registers clocked by a propagated clock have edge times skewed by the path delay from the clock source to the register clock pin. Use the `set_propagated_clock` command to specify that clock latency be propagated through the clock network.

Pseudo-Random Pattern Generator

This uses an LSFR and an XOR phase shifter to generate a stream of pseudorandom data values that have the appearance of random values, but are actually a function of a seed value.

Pseudo-random pattern generator (PRPG)

PRPG

Real clock

A clock that has a source, meaning its waveform is applied to pins or ports in the design. Create a real clock by using the `create_clock` command and including a source list of ports or pins. Real clocks can be either ideal or propagated.

Receiver supply

For a receiver that is the gate of a transistor, the supply connected to the source or drain of that transistor. For a receiver that is the input to an inverter, the pair of supplies connected to the source or drain of the transistor pair comprising the inverter. For a receiver that is part of an active component, the primary supply of the power domain to which that receiver belongs or, in some cases, the secondary supply of the component if it has a secondary supply.

(receiver supply)

Reduced Standard Parasitic Format

A format used to store circuit parasitic information for back-annotation. This format is defined by Cadence Design Systems, Inc.

RSPF

Reference

A library component or design that can be used as an element in building a larger circuit. The structure of the reference might be a simple logic gate or a more complex

Feedback

design (RAM core or CPU). A design can contain multiple occurrences of a reference; each occurrence is an instance. See also instance.

Register Transfer Level

Is a high-level description of a digital circuit expressed in terms of the data transfers between the registers and the logical operations performed on the data. The RTL description is written in a hardware description language such as Verilog or VHDL.

Register Transfer Level (RTL)

RTL

Related supply net

Supply net that is associated to a signal port or pin. In the case of a lib-pin, the association happens through the relations of signal pins and PG-pins of a lib-cell. In UPF, hierarchical pins can also have a related supply, which happens through the specification of UPF constraints (such as in power domain definitions, `set_port_attributes -driver_supply` command, or `set_port_attributes receiver_supply` command).

(related supply net)

Related supply set

Supply set that is associated to a signal port or pin. In the case of a lib-pin, the association happens through the relations of signal pins and PG-pins of a lib-cell. In UPF, hierarchical pins can also have a related supply, which happens through the specification of UPF constraints (such as in power domain definitions, `set_port_attributes -driver_supply` command, or `set_port_attributes receiver_supply` command).

(related supply set)

Retention register

A memory register that retains data during power-down periods by keeping the data in a shadow register having an always-on power supply.

(retention register)

Return-to-one clock

A clock whose value is normally high, with an active-low pulse during the clock period.

Feedback

Return-to-zero clock

A clock whose value is normally low, with an active-high pulse during the clock period.

Reused supply net

A supply net that spans different domains and passes through a supply port.

(reused supply net)

Root cell

A cell defined as belonging to a power domain by the `-elements` option of the `create_power_domain` command, or specified as the scope with the `-scope` option of the command.

(root cell)

Regular Multisource Clock Tree Synthesis

RMSCTS

Route Driven Estimation

An extraction engine to predict parasitics for a pre-routed design.

RDE

Scan cell

A sequential cell that has both functional and scan-shift modes of operation.

Scan compression

See *compressed scan*.

SCANDEF

A DEF file that uses a set of scan-specific constructs to describe how a design's scan chains can be reordered and repartitioned by a layout tool.

Scan group

A group of scan cells to be kept together in a scan chain.

Scope

The level of a design hierarchy where a power domain exists.

Feedback

(scope)

Seed value

The initial value loaded into a PRPG. Different seed values result in different pseudorandom value sequences.

Shadow register

The internal part of a retention register that is always supplied with power and retains data during power-down. It is constructed using higher-threshold transistors to minimize leakage current. Is also called the bubble register.

(shadow register)

Shadow wrapper

A “wrapper” around an untestable block or macrocell that allows surrounding logic to be tested. Known values are forced at the outputs, and to improve coverage, the values at the inputs are captured. It can be easily inserted using automatic test point insertion.

Shared codec I/O

A feature that allows the scan-in and scan-out connections of DFTMAX cores or codecs to be shared to reduce scan I/O requirements.

Shared well

Usually refers to the N-well inside a cell that is physically abutted with the N-well of other cells in the surrounding region and shares electrical characteristics with them.

(shared well)

Shift register

A sequence of sequential cells in the design whose functional operation is to shift bits through the register like a scan chain. Shift registers only need their first (head) element scan-replaced to be stitched into a scan chain.

Skew subdomain

Part of a parent clock network that is considered to have different clock skew characteristics than the rest of the clock network. Lock-up latches are inserted whenever a scan chain crosses a skew subdomain boundary. A skew subdomain is driven by an *internal clock* pin.

Feedback

Slack

A value that represents the difference between the actual arrival time and the required arrival time of data at the timing path endpoint. Slack values can be positive, negative, or zero. A positive slack value represents the amount by which the delay of a path can be increased without violating any timing constraints. A negative slack value represents the amount by which the delay of a path must be reduced to meet its timing constraints.

Source latency

The amount of time a clock signal takes to propagate from waveform origin point (for example, PLL) to the clock definition point.

SQS

This `set_qor_strategy` command is one of the mega switches in the tool for changing many engine configurations simultaneously. This command allows you to specify their primary design targets, whether that is timing, leakage power, total power, area, or congestion.

Standard Parasitic Exchange Format

A format used to store circuit parasitic information for back-annotation. This format is defined by Open Verilog International (OVI).

Standard Parasitic Exchange Format (SPEF)

SPEF

Standard scan

A scan methodology in which there is a one-to-one relationship between each scan-in/scan-out pair and each scan chain.

Standard Test Interface Language

This is documented in IEEE Std 1450, which is a language used to describe the DFT capabilities of a design. It is a standard for simplifying the number of test vector formats that automated test equipment (ATE) vendors and computer-aided engineering (CAE) tool vendors must support.

Standard Test Interface Language (STIL)

Feedback

STIL

Static power

The power consumed even when the circuit is not switching, consisting of reverse-biased P-N junction leakage, sub-threshold leakage, and gate leakage.

(static power)

STIL Protocol File

A file written out by DFT Compiler to describe DFT aspects of the design, such as: test ports, test clocks, primary input constraints, scan chains, codecs, and test modes. It is used by TestMAX ATPG (or other ATPG tool) for DRC and ATPG.

STIL Protocol File (SPF)

SPF

Structural Congestion Optimization

SCO

Structural Multisource Clock Tree Synthesis

SMSCTS

Sub-threshold leakage

The small amount of current that flows between the source and drain of a transistor when the gate voltage is below what is considered the threshold voltage.

(sub-threshold leakage)

Supplemental UPF

In the golden UPF mode, a supplemental UPF file contains only the UPF changes made by the tool during the implementation session, such as tool-derived power intent and addition of power management cells. In the golden UPF flow, a supplemental UPF is required as the original golden UPF script file that is used throughout the synthesis, physical implementation and verification flow is used without incorporating any changes to it.

(supplemental UPF)

Feedback

Supply net

An abstract representation of a power or ground net in the design, representing a contiguous conductor that carries a supply voltage or ground connection.

(supply net)

Supply port

A power supply connection point between two adjacent levels of a design hierarchy. A supply net that crosses from one level of the design hierarchy to the next passes through a supply port.

(supply port)

Supply set

An abstract representation of a bundle of supply nets that is specific to a power domain and that has a power and a ground function definition.

(supply set)

Supply set handle

An abstract supply set created for a power domain, which lets you synthesize a design even before you create any supply sets, supply nets, or supply ports for the domain.

(supply set handle)

Switch

A power switch; a device that turns on and turns off power for a supply net.

(switch)

Switching activity

The relative presence or absence of logic transitions occurring on nets over a period of time, which affects the amount of dynamic power consumed during that period of time.

(switching activity)

Feedback

Switching power

The power consumed as a result of charging and discharging the external capacitive load on the output of a cell.

(switching power)

Switching Activity Interchange Format

A SAIF file is an ASCII file which captures the toggle rate of the signals in the design and is used for analyzing the power consumption in the design. SAIF is developed by Synopsys to facilitate the interchange of information between EDA simulation tools and Synopsys power tools. SAIF file can be generated from the Value Change Dump (VCD) file which is defined in the IEEE standard 1364-2001 to support the logging of signal and direction. VCD files are generated by the EDA simulation tools.

Switching Activity Interchange Format (SAIF)

SAIF

Table Look Up

This is a parasitic data format used by ICC/ICC2/FC to perform extraction.

TLU

Targeted Endpoint Optimization

.An option with `route_opt` to work only on a subset of the endpoints in the design. You can specify this option by using the `set_route_opt_target_endpoints` command.

Targeted Endpoint Optimization (TEP)

TEP

Test Control Module

A DFT design structure that selects the current test mode. The input is a vector of test-mode selection signals that can have binary, one-hot, or user-defined encodings. The output is a set of decoded one-hot enable signals, one for each test mode encoding.

Test Control Module (TCM)

Feedback

TCM

Test-mode Core Data Register

A special core data register (CDR) in an IEEE 1500 implementation that takes the place of traditional port-driven test-mode signals.

Test-mode Core Data Register (TMCDR)

TMCDR

Threshold Voltage

The gate-to-source voltage at which the transistor begins conducting between the source and drain.

(threshold voltage)

Timing Driven Structuring

Refers to logic restructuring steps in the optimization flow that target improving timing.

Timing Driven Structuring (TDS)

TDS

Timing Path

A chain of logic through which data propagates. Data is launched by a clock edge at a startpoint, propagated through combinational logic elements, and captured at an endpoint by another clock edge. The startpoint of a timing path is an input port or clock pin of a sequential element. The endpoint of a timing path is an output port or a data pin of a sequential element.

Tool Command Language

The tool shell is based on the standard scripting language Tcl. Newer versions (U-2022.12-SP5 or later) of the tool also support Python shell to communicate with the tool.

Tool Command Language (Tcl)

Feedback

Tcl

Total Negative Slack

TNS

Total Power Logic Restructuring

tpLR

Total Power Optimization

TPO

Trailing edge

The last edge of a clock waveform definition. It is a falling edge for a return-to-zero clock, and it is a rising edge for a return-to-one clock.

Transition time

The amount of time it takes for a signal to change from low to high or from high to low.

Transparent

The state of a level sensitive latch while the gate input is asserted. During this time, signals pass from input to output and the device operates like a buffer.

Transparent Hierarchy Optimization

THO

Type

In Verilog, the mechanism by which objects are restricted in the assigned values and the operations that can be applied to them.

Unified Power Format

Refers to a standard set of commands used to specify the low-power design intent for electronic systems. Is an alternative name of the IEEE 1801™ Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems.

Unified Power Format (UPF)

Feedback

UPF

Unified Power Format prime

In the traditional UPF flow used in synthesis tools, the UPF' file refers to the UPF file to which the `save_upf` command writes out a full set of UPF commands reflecting the changes made by the tool to the UPF power intent of the design. The file also includes the UPF commands run during the tool session.

Unified Power Format prime (UPF')

UPF'

Unified Physical Synthesis

This refers to the optimization flow in the `compile_fusion` command. UPS brings advanced physically aware optimization earlier into the synthesis flow, and sophisticated logic restructuring techniques deeper into the place-and-route flow.

Unified Physical Synthesis (UPF)

UPS

User Interface

UI

VDD

A name typically given to a power supply net or pin having a positive voltage.

(VDD)

VHSIC Hardware Description Language

This language is used to model the behavior and structure of digital systems at multiple levels of abstraction, ranging from the system level down to the logic gates.

VHSIC Hardware Description Language (VHDL)

VHDL

Victim Net

A net that receives crosstalk effects due to parasitic cross-capacitance by another net (called the aggressor net).

Feedback

Virtual Clock

A clock that exists on the chip but does not have a physical clock tree in this block. Typically used for I/O constraints, for example, a path might be launched by this virtual clock outside the block and come in one of its input ports. That input port's delay or clock latency might be constrained using this virtual clock.

Virtual Metal Fill

Before the insertion of real metal fill, you can insert virtual metal fill polygons for better estimation of parasitics and timing early in the design process. Virtual metal fill emulates track-fill style metal fill using large aspect ratio polygons.

VMF

Voltage area

A physically contiguous area of the chip belonging to a single power domain and using the same power supply distribution network.

(voltage area)

VSS

A name typically given to a ground-voltage supply net or pin.

(VSS)

Vt

The threshold voltage of a PMOS or NMOS transistor; the gate-to-source voltage at which the transistor begins conducting between the source and drain. Standard cell libraries typically have cells of multiple different Vt classes. Low Vt cells have a lower threshold voltage which makes them faster but have higher leakage power. High Vt cells have a higher threshold voltage which makes them slower but with better (lower) leakage power.

(Vt)

Weighted capture groups

An OCC-based capture method that uses comparator logic to enable one capture group in each pattern, based on user-specified probabilities.

Feedback

Wrapper Instruction Register

A required register in an IEEE 1500 implementation that controls what data register is selected for access.

Wrapper instruction register (WIR)

WIR

Worker Process

In multithreaded jobs, one of a set of processes that performs distributed analysis work. The worker processes are controlled by the manager processes.

Worst Negative Slack

WNS

Wrapper chain

A special scan chain in a core-wrapped design that consists of wrapper cells along the I/O boundary of the design. It can operate in inward-facing or outward-facing modes during testing to isolate the logic inside the core from logic outside the core.

Wrapped core

A core that has a wrapper chain along the I/O boundary of the design. Wrapped cores are used to implement hierarchical testing capability, which allows different hierarchy levels of the design to be tested independently.

ZRT

Zroute, which is the detailed router. Most of the router logfile messages use ZRT in the message tag.